
Keysight InfiniiVision 4000G X-Series Oscilloscopes

Notices

© Keysight Technologies, Inc. 2005-2025

No part of this manual may be reproduced in any form or by any means (including electronic storage and retrieval or translation into a foreign language) without prior agreement and written consent from Keysight Technologies, Inc. as governed by United States and international copyright laws.

Revision

Version 07.66.0000

Edition

October 31, 2025

Available in electronic format only

Published by:

Keysight Technologies, Inc.
1900 Garden of the Gods Road
Colorado Springs, CO 80907 USA

Warranty

The material contained in this document is provided "as is," and is subject to being changed, without notice, in future editions. Further, to the maximum extent permitted by applicable law, Keysight disclaims all warranties, either express or implied, with regard to this manual and any information contained herein, including but not limited to the implied warranties of merchantability and fitness for a particular purpose. Keysight shall not be liable for errors or for incidental or consequential damages in connection with the furnishing, use, or performance of this document or of any information contained herein. Should Keysight and the user have a separate written agreement with warranty terms covering the material in this document that conflict with these terms, the warranty terms in the separate agreement shall control.

Technology License

The hardware and/or software described in this document are furnished under a license and may be used or copied only in accordance with the terms of such license.

U.S. Government Rights

The Software is "commercial computer software," as defined by Federal Acquisition Regulation ("FAR") 2.101. Pursuant to FAR 12.212 and 27.405-3 and Department of Defense FAR Supplement ("DFARS") 227.7202, the U.S. government acquires commercial computer software under the same terms by which the software is customarily provided to the public. Accordingly, Keysight provides the Software to U.S. government customers under its standard commercial license, which is embodied in its End User License Agreement (EULA), a copy of which can be found at www.keysight.com/find/sweula. The license set forth in the EULA represents the exclusive authority by which the U.S. government may use, modify, distribute, or disclose the Software. The EULA and the license set forth therein, does not require or permit, among other things, that Keysight: (1) Furnish technical information related to commercial computer software or commercial computer software documentation that is not customarily provided to the public; or (2) Relinquish to, or otherwise provide, the government rights in excess of these rights customarily provided to the public to use, modify, reproduce, release, perform, display, or disclose commercial computer software or commercial computer software documentation. No additional government requirements beyond those set forth in the EULA shall apply, except to the extent that those terms, rights, or licenses are explicitly required from all providers of commercial computer software pursuant to the FAR and the DFARS and are set forth specifically in writing elsewhere in the EULA. Keysight shall be under no obligation to update, revise or otherwise modify the Software. With respect to any technical data as defined by FAR 2.101, pursuant to FAR 12.211 and 27.404.2 and DFARS 227.7102, the U.S. government acquires no greater than Limited Rights as defined in FAR 27.401 or DFAR 227.7103-5 (c), as applicable in any technical data.

Safety Notices

CAUTION

A **CAUTION** notice denotes a hazard. It calls attention to an operating procedure, practice, or the like that, if not correctly performed or adhered to, could result in damage to the product or loss of important data. Do not proceed beyond a **CAUTION** notice until the indicated conditions are fully understood and met.

WARNING

A **WARNING** notice denotes a hazard. It calls attention to an operating procedure, practice, or the like that, if not correctly performed or adhered to, could result in personal injury or death. Do not proceed beyond a **WARNING** notice until the indicated conditions are fully understood and met.

In This Book

This book is your guide to programming the 4000G X-Series oscilloscopes:

Table 1 InfiniiVision 4000G X-Series Oscilloscope Models, Bandwidths, Sample Rates

Bandwidth	200 MHz	350 MHz	500 MHz	1 GHz	1.5 GHz
Sample Rate (interleaved, non-interleaved)	5 GSa/s, 2.5 GSa/s	5 GSa/s, 2.5 GSa/s	5 GSa/s, 2.5 GSa/s	5 GSa/s, 2.5 GSa/s	5 GSa/s, 2.5 GSa/s
2-Channel + 16 Logic Channels MSO	MSO-X 4022G	MSO-X 4032G	MSO-X 4052G		
4-Channel + 16 Logic Channels MSO	MSO-X 4024G	MSO-X 4034G	MSO-X 4054G	MSO-X 4104G	MSO-X 4154G
2-Channel DSO	DSO-X 4022G	DSO-X 4032G	DSO-X 4052G		
4-Channel DSO	DSO-X 4024G	DSO-X 4034G	DSO-X 4054G	DSO-X 4104G	DSO-X 4154G

The first few chapters describe how to set up and get started:

- **Chapter 1**, “What's New,” starting on page 41, describes programming command changes in the latest version of oscilloscope software.
- **Chapter 2**, “Setting Up,” starting on page 47, describes the steps you must take before you can program the oscilloscope.
- **Chapter 3**, “Getting Started,” starting on page 55, gives a general overview of oscilloscope program structure and shows how to program the oscilloscope using a few simple examples.
- **Chapter 4**, “Sequential (Blocking) vs. Overlapped Commands,” starting on page 69, describes these command types and how they affect the oscilloscope and synchronization.
- **Chapter 5**, “Commands Quick Reference,” starting on page 71, is a brief listing of the 4000G X-Series oscilloscope commands and syntax.

The next chapters provide reference information on common commands, root level commands, other subsystem commands, and error messages:

- **Chapter 6**, “Common (*) Commands,” starting on page 209, describes commands defined by the IEEE 488.2 standard that are common to all instruments.
- **Chapter 7**, “Root (:) Commands,” starting on page 237, describes commands that reside at the root level of the command tree and control many of the basic functions of the oscilloscope.
- **Chapter 8**, “:ACQuire Commands,” starting on page 281, describes commands for setting the parameters used when acquiring and storing data.
- **Chapter 9**, “:BUS<n> Commands,” starting on page 303, describes commands that control all oscilloscope functions associated with the digital channels bus display.

- **Chapter 10**, “:CALibrate Commands,” starting on page 313, describes utility commands for determining the state of the calibration factor protection button.
- **Chapter 11**, “:CHANnel<n> Commands,” starting on page 325, describes commands that control all oscilloscope functions associated with individual analog channels or groups of channels.
- **Chapter 12**, “:COMpliance Commands,” starting on page 357, describes commands that control the USB 2.0 signal quality analysis feature.
- **Chapter 13**, “:COUNter Commands,” starting on page 369, describes commands that control the counter analysis feature.
- **Chapter 14**, “:DEMO Commands,” starting on page 375, describes commands that control the education kit demonstration signals that can be output on the oscilloscope's Demo 1 and Demo 2 terminals.
- **Chapter 15**, “:DIGital<d> Commands,” starting on page 383, describes commands that control all oscilloscope functions associated with individual digital channels.
- **Chapter 16**, “:DISPlay Commands,” starting on page 391, describes commands that control how waveforms, graticule, and text are displayed and written on the screen.
- **Chapter 17**, “:DVM Commands,” starting on page 417, describes commands that control the digital voltmeter analysis feature.
- **Chapter 18**, “:EXTernal Trigger Commands,” starting on page 423, describes commands that control the input characteristics of the external trigger input.
- **Chapter 19**, “:FRANalysis Commands,” starting on page 429, describes commands that control oscilloscope functions associated with the Frequency Response Analysis (FRA) feature, which is available is with the license-enabled built-in waveform generator).
- **Chapter 20**, “:FUNCTion<m> Commands,” starting on page 449, describes commands that control math waveforms.
- **Chapter 21**, “:HARDcopy/:HCOPY Commands,” starting on page 505, describes commands that set and query the selection of hardcopy device and formatting options.
- **Chapter 22**, “:HISTogram Commands,” starting on page 525, describes commands that control the histogram analysis feature.
- **Chapter 23**, “:LISTer Commands,” starting on page 541, describes commands that turn on/off the Lister display for decoded serial data and get the Lister data.
- **Chapter 24**, “:LTEST Commands,” starting on page 545, describes commands that control the Measurement Limit Test feature.
- **Chapter 25**, “:MARKer Commands,” starting on page 559, describes commands that set and query the settings of X-axis markers (X1 and X2 cursors) and the Y-axis markers (Y1 and Y2 cursors).
- **Chapter 26**, “:MEASure Commands,” starting on page 581, describes commands that select automatic measurements (and control markers).

- **Chapter 27**, “:MEASure Power Commands,” starting on page 687, describes measurement commands that are available when the power measurements and analysis application is licensed and enabled.
- **Chapter 28**, “:MTEST Commands,” starting on page 711, describes commands that control the license-enabled mask test feature.
- **Chapter 29**, “:POD Commands,” starting on page 745, describes commands that control all oscilloscope functions associated with groups of digital channels.
- **Chapter 30**, “:POWER Commands,” starting on page 751, describes commands that control the license-enabled power measurement application.
- **Chapter 31**, “:RECall Commands,” starting on page 855, describes commands that recall previously saved oscilloscope setups, reference waveforms, or masks.
- **Chapter 32**, “:SAVE Commands,” starting on page 867, describes commands that save oscilloscope setups, screen images, and data.
- **Chapter 33**, “:SBUS<n> Commands,” starting on page 899, describes commands that control oscilloscope functions associated with the serial decode bus and serial triggering.
- **Chapter 34**, “:SEARCh Commands,” starting on page 1195, describes commands that control oscilloscope functions associated with searching for waveform events.
- **Chapter 35**, “:SYSTem Commands,” starting on page 1305, describes commands that control basic system functions of the oscilloscope.
- **Chapter 36**, “:TIMEbase Commands,” starting on page 1331, describes commands that control all horizontal sweep functions.
- **Chapter 37**, “:TRIGger Commands,” starting on page 1345, describes commands that control the trigger modes and parameters for each trigger type.
- **Chapter 38**, “:WAVEform Commands,” starting on page 1433, describes commands that provide access to waveform data.
- **Chapter 39**, “:WGEN<w> Commands,” starting on page 1471, describes commands that control waveform generator (Option WGN) functions and parameters.
- **Chapter 40**, “:WMEMory<r> Commands,” starting on page 1521, describes commands that control reference waveforms.
- **Chapter 41**, “Obsolete and Discontinued Commands,” starting on page 1531, describes obsolete commands which still work but have been replaced by newer commands and discontinued commands which are no longer supported.
- **Chapter 42**, “Error Messages,” starting on page 1589, lists the instrument error messages that can occur while programming the oscilloscope.

The command descriptions in this reference show upper and lowercase characters. For example, :AUToscale indicates that the entire command name is :AUTOSCALE. The short form, :AUT, is also accepted by the oscilloscope.

Then, there are chapters that describe programming topics and conceptual information in more detail:

- **Chapter 43**, "Status Reporting," starting on page 1597, describes the oscilloscope's status registers and how to check the status of the instrument.
- **Chapter 44**, "Synchronizing Acquisitions," starting on page 1629, describes how to wait for acquisitions to complete before querying measurement results or performing other operations with the captured data.
- **Chapter 45**, "More About Oscilloscope Commands," starting on page 1649, contains additional information about oscilloscope programming commands.

Finally, there is a chapter that contains programming examples:

- **Chapter 46**, "Programming Examples," starting on page 1659.

Mixed-Signal Oscilloscope Channel Differences

Because both the "analog channels only" oscilloscopes (DSO models) and the mixed-signal oscilloscopes (MSO models) have analog channels, topics that describe analog channels refer to all oscilloscope models. Whenever a topic describes digital channels, that information applies only to the mixed-signal oscilloscope models.

See Also

- For more information on using the SICL, VISA, and VISA COM libraries in general, see the documentation that comes with the Keysight IO Libraries Suite.
- For information on controller PC interface configuration, see the documentation for the interface card used (for example, the Keysight 82350B GPIB interface).
- For information on oscilloscope front-panel operation, see the *User's Guide*.
- For detailed connectivity information, refer to the *Keysight Technologies USB/LAN/GPIB Connectivity Guide*. For a printable electronic copy of the *Connectivity Guide*, direct your Web browser to www.keysight.com and search for "Connectivity Guide".
- For the latest versions of this and other manuals, see:
<http://www.keysight.com/find/4000GX-Series-manual>

Contents

In This Book / 3

1 What's New

What's New in Version 7.66 / 42

What's New in Version 7.65 / 43

Version 7.60 at Introduction / 44

Command Differences From 4000 X-Series Oscilloscopes / 45

2 Setting Up

Step 1. Install Keysight IO Libraries Suite software / 48

Step 2. Connect and set up the oscilloscope / 49

Using the USB (Device) Interface / 49

Using the LAN Interface / 49

Step 3. Verify the oscilloscope connection / 51

3 Getting Started

Basic Oscilloscope Program Structure / 56

Initializing / 56

Capturing Data / 56

Analyzing Captured Data / 57

Programming the Oscilloscope / 58

Referencing the IO Library / 58

Opening the Oscilloscope Connection via the IO Library / 59

Initializing the Interface and the Oscilloscope / 59

Using :AUToscale to Automate Oscilloscope Setup / 60

Using Other Oscilloscope Setup Commands / 60

Capturing Data with the :DIGitize Command / 61

Reading Query Responses from the Oscilloscope / 63

Reading Query Results into String Variables / 64

Reading Query Results into Numeric Variables / 64

Reading Definite-Length Block Query Response Data / 64

Sending Multiple Queries and Reading Results / 65

Checking Instrument Status / 66

Other Ways of Sending Commands	/ 67
Telnet Sockets	/ 67
Sending SCPI Commands Using Browser Web Control	/ 67

4 Sequential (Blocking) vs. Overlapped Commands

5 Commands Quick Reference

Command Summary	/ 72
Syntax Elements	/ 206
Number Format	/ 206
<NL> (Line Terminator)	/ 206
[] (Optional Syntax Terms)	/ 206
{ } (Braces)	/ 206
::= (Defined As)	/ 206
< > (Angle Brackets)	/ 207
... (Ellipsis)	/ 207
n,...,p (Value Ranges)	/ 207
d (Digits)	/ 207
Quoted ASCII String	/ 207
Definite-Length Block Response Data	/ 207

6 Common (*) Commands

*CLS (Clear Status)	/ 215
*ESE (Standard Event Status Enable)	/ 216
*ESR (Standard Event Status Register)	/ 218
*IDN (Identification Number)	/ 220
*LRN (Learn Device Setup)	/ 221
*OPC (Operation Complete)	/ 222
*OPT (Option Identification)	/ 223
*RCL (Recall)	/ 225
*RST (Reset)	/ 226
*SAV (Save)	/ 229
*SRE (Service Request Enable)	/ 230
*STB (Read Status Byte)	/ 232
*TRG (Trigger)	/ 234
*TST (Self Test)	/ 235
*WAI (Wait To Continue)	/ 236

7 Root (:) Commands

:ACTivity	/ 241
:AER (Arm Event Register)	/ 242

- :AUToscale / 243
- :AUToscale:AMODE / 245
- :AUToscale:CHANnels / 246
- :AUToscale:FDEBug / 247
- :BLANk / 248
- :DIGitize / 250
- :HWEenable (Hardware Event Enable Register) / 252
- :HWERegister:CONDition (Hardware Event Condition Register) / 254
- :HWERegister[:EVENT] (Hardware Event Event Register) / 255
- :MTEenable (Mask Test Event Enable Register) / 256
- :MTERegister[:EVENT] (Mask Test Event Event Register) / 258
- :OPEE (Operation Status Enable Register) / 260
- :OPERRegister:CONDition (Operation Status Condition Register) / 262
- :OPERRegister[:EVENT] (Operation Status Event Register) / 265
- :OVLenable (Overload Event Enable Register) / 268
- :OVLRegister (Overload Event Register) / 270
- :PRINt / 272
- :RState? / 273
- :RUN / 274
- :SERial / 275
- :SINGle / 276
- :STATus / 277
- :STOP / 278
- :TER (Trigger Event Register) / 279
- :VIEW / 280

8 :ACQuire Commands

- :ACQuire:AALias / 284
- :ACQuire:COMPlEte / 285
- :ACQuire:COUNt / 286
- :ACQuire:DAALias / 287
- :ACQuire:DIGitizer / 288
- :ACQuire:MODE / 289
- :ACQuire:POINts[:ANALog] / 290
- :ACQuire:POINts[:ANALog]:AUTO / 291
- :ACQuire:RSIGnal / 292
- :ACQuire:SEGmented:ANALyze / 293
- :ACQuire:SEGmented:COUNt / 294
- :ACQuire:SEGmented:INDex / 295
- :ACQuire:SRATe[:ANALog] / 298
- :ACQuire:SRATe[:ANALog]:AUTO / 299

:ACQuire:TYPE / 300

9 :BUS<n> Commands

:BUS<n>:BIT<m> / 305

:BUS<n>:BITS / 306

:BUS<n>:CLEar / 308

:BUS<n>:DISPlay / 309

:BUS<n>:LABel / 310

:BUS<n>:MASK / 311

10 :CALibrate Commands

:CALibrate:DATE / 315

:CALibrate:LABel / 316

:CALibrate:OUTPut / 317

:CALibrate:PROTected / 319

:CALibrate:STARt / 320

:CALibrate:STATus / 321

:CALibrate:TEMPerature / 322

:CALibrate:TIME / 323

11 :CHANnel<n> Commands

:CHANnel<n>:BANDwidth / 329

:CHANnel<n>:BWLimit / 330

:CHANnel<n>:COUPling / 331

:CHANnel<n>:DISPlay / 332

:CHANnel<n>:IMPedance / 333

:CHANnel<n>:INVert / 334

:CHANnel<n>:LABel / 335

:CHANnel<n>:OFFSet / 336

:CHANnel<n>:PROBe / 337

:CHANnel<n>:PROBe:BTN / 338

:CHANnel<n>:PROBe:CALibration / 339

:CHANnel<n>:PROBe:EXTernal / 340

:CHANnel<n>:PROBe:EXTernal:GAIN / 341

:CHANnel<n>:PROBe:EXTernal:OFFSet / 342

:CHANnel<n>:PROBe:EXTernal:UNITs / 343

:CHANnel<n>:PROBe:HEAD[:TYPE] / 344

:CHANnel<n>:PROBe:ID / 345

:CHANnel<n>:PROBe:MMODEl / 346

:CHANnel<n>:PROBe:MODE / 347

:CHANnel<n>:PROBe:RSENse / 348

:CHANnel<n>:PROBe:SKEW / 349
:CHANnel<n>:PROBe:STYPe / 350
:CHANnel<n>:PROBe:ZOOM / 351
:CHANnel<n>:PROTection / 352
:CHANnel<n>:RANGe / 353
:CHANnel<n>:SCALE / 354
:CHANnel<n>:UNITs / 355
:CHANnel<n>:VERNier / 356

12 :COMPLIance Commands

:COMPLIance:USB:AUTosetup / 358
:COMPLIance:USB:HUBS / 359
:COMPLIance:USB:RUN / 360
:COMPLIance:USB:SOURce:ADJacent / 361
:COMPLIance:USB:SOURce:DIFFerential / 362
:COMPLIance:USB:SOURce:DMINus / 363
:COMPLIance:USB:SOURce:DPLus / 364
:COMPLIance:USB:TEST / 365
:COMPLIance:USB:TEST:CONNection / 366
:COMPLIance:USB:TEST:TYPE / 367

13 :COUNter Commands

:COUNter:CURRent / 370
:COUNter:ENABLe / 371
:COUNter:MODE / 372
:COUNter:NDIGits / 373
:COUNter:SOURce / 374

14 :DEMO Commands

:DEMO:FUNCTion / 376
:DEMO:FUNCTion:PHASe:PHASe / 381
:DEMO:OUTPut / 382

15 :DIGital<d> Commands

:DIGital<d>:DISPlay / 385
:DIGital<d>:LABel / 386
:DIGital<d>:POSition / 387
:DIGital<d>:SIZE / 388
:DIGital<d>:THReshold / 389

16 :DISPlay Commands

:DISPlay:ANNotation<n> / 394
:DISPlay:ANNotation<n>:BACKground / 395
:DISPlay:ANNotation<n>:COLor / 396
:DISPlay:ANNotation<n>:TEXT / 397
:DISPlay:ANNotation<n>:X1Position / 398
:DISPlay:ANNotation<n>:Y1Position / 399
:DISPlay:BACKlight / 400
:DISPlay:CLEar / 401
:DISPlay:DATA / 402
:DISPlay:GRATicule:ALABels / 404
:DISPlay:GRATicule:INTensity / 405
:DISPlay:GRATicule:TYPE / 406
:DISPlay:INTensity:WAVEform / 407
:DISPlay:LABel / 408
:DISPlay:LABList / 409
:DISPlay:MENU / 410
:DISPlay:MESSage:CLEar / 411
:DISPlay:PERsistence / 412
:DISPlay:PERsistence:CLEar / 413
:DISPlay:SIDebar / 414
:DISPlay:TRANSPARENT / 415
:DISPlay:VECTors / 416

17 :DVM Commands

:DVM:ARANge / 418
:DVM:CURREnt / 419
:DVM:ENABle / 420
:DVM:MODE / 421
:DVM:SOURce / 422

18 :EXTeRnal Trigger Commands

:EXTeRnal:BWLimit / 424
:EXTeRnal:PROBe / 425
:EXTeRnal:RANGe / 426
:EXTeRnal:UNITs / 427

19 :FRANalysis Commands

:FRANalysis:DATA / 432
:FRANalysis:ENABle / 433
:FRANalysis:FREQuency:MODE / 434

:FRANalysis:FREQuency:SINGle / 435
 :FRANalysis:FREQuency:STARt / 436
 :FRANalysis:FREQuency:STOP / 437
 :FRANalysis:PPDecade / 438
 :FRANalysis:RUN / 439
 :FRANalysis:SOURce:INPut / 440
 :FRANalysis:SOURce:OUTPut / 441
 :FRANalysis:SWEep:POINts / 442
 :FRANalysis:TRACe / 443
 :FRANalysis:WGEN:LOAD / 444
 :FRANalysis:WGEN:VOLTage / 445
 :FRANalysis:WGEN:VOLTage:PROFile / 447

20 :FUNCTION<m> Commands

:FUNCTION<m>:AVERage:COUNT / 456
 :FUNCTION<m>:BUS:CLOCK / 457
 :FUNCTION<m>:BUS:SLOPe / 458
 :FUNCTION<m>:BUS:YINCrement / 459
 :FUNCTION<m>:BUS:YORigin / 460
 :FUNCTION<m>:BUS:YUNits / 461
 :FUNCTION<m>:CLEar / 462
 :FUNCTION<m>:DISPlay / 463
 :FUNCTION<m>[:FFT]:BSIZe / 464
 :FUNCTION<m>[:FFT]:CENTer / 465
 :FUNCTION<m>[:FFT]:DETection:POINts / 466
 :FUNCTION<m>[:FFT]:DETection:TYPE / 467
 :FUNCTION<m>[:FFT]:FREQuency:STARt / 468
 :FUNCTION<m>[:FFT]:FREQuency:STOP / 469
 :FUNCTION<m>[:FFT]:GATE / 470
 :FUNCTION<m>[:FFT]:PHASe:REFerence / 471
 :FUNCTION<m>[:FFT]:RBWidth / 472
 :FUNCTION<m>[:FFT]:READOut<n> / 473
 :FUNCTION<m>[:FFT]:SPAN / 474
 :FUNCTION<m>[:FFT]:SRATe / 475
 :FUNCTION<m>[:FFT]:VTYPE / 476
 :FUNCTION<m>[:FFT]:WINDow / 477
 :FUNCTION<m>:FREQuency:BANDpass:CENTer / 478
 :FUNCTION<m>:FREQuency:BANDpass:WIDTh / 479
 :FUNCTION<m>:FREQuency:HIGHPass / 480
 :FUNCTION<m>:FREQuency:LOWPass / 481
 :FUNCTION<m>:INTegrate:IICondition / 482

- :FUNCTION<m>:INTEgrate:IOFFset / 483
- :FUNCTION<m>:LINear:GAIN / 484
- :FUNCTION<m>:LINear:OFFSet / 485
- :FUNCTION<m>:OFFSet / 486
- :FUNCTION<m>:OPERation / 487
- :FUNCTION<m>:RANGe / 492
- :FUNCTION<m>:REFerence / 493
- :FUNCTION<m>:SCALE / 494
- :FUNCTION<m>:SERChart:SOURce / 495
- :FUNCTION<m>:SERChart:TIME:MODE / 496
- :FUNCTION<m>:SERChart:TIME:RANGe / 498
- :FUNCTION<m>:SERChart:TIME:OFFSet / 499
- :FUNCTION<m>:SMOoth:POINts / 500
- :FUNCTION<m>:SOURce1 / 501
- :FUNCTION<m>:SOURce2 / 503
- :FUNCTION<m>:TREND:NMEasurement / 504

21 :HARDcopy/:HCOPY Commands

- :HARDcopy:AREA / 508
- :HARDcopy:APRinter / 509
- :HARDcopy:FACTors / 510
- :HARDcopy:FFEed / 511
- :HARDcopy:INKSaver / 512
- :HARDcopy:LAYout / 513
- :HARDcopy:NETWork:ADDRes / 514
- :HARDcopy:NETWork:APPLy / 515
- :HARDcopy:NETWork:DOMain / 516
- :HARDcopy:NETWork:PASSword / 517
- :HARDcopy:NETWork:SLOT / 518
- :HARDcopy:NETWork:USERname / 519
- :HARDcopy:PALette / 520
- :HARDcopy:PRINter:LIST / 521
- :HARDcopy:STARt / 522
- :HCOPY:SDUMp:DATA / 523
- :HCOPY:SDUMp:FORMat / 524

22 :HISTogram Commands

- :HISTogram:AXIS / 528
- :HISTogram:DISPlay / 529
- :HISTogram:MEASurement / 530
- :HISTogram:MODE / 531

:HISTogram:RESet / 532
:HISTogram:SIZE / 533
:HISTogram:TYPE / 534
:HISTogram:WINDow:BLIMit / 535
:HISTogram:WINDow:LLIMit / 536
:HISTogram:WINDow:RLIMit / 537
:HISTogram:WINDow:SOURce / 538
:HISTogram:WINDow:TLIMit / 539

23 :LISTer Commands

:LISTer:DATA / 542
:LISTer:DISPlay / 543
:LISTer:REference / 544

24 :LTEST Commands

:LTEST:COPY / 547
:LTEST:COPY:ALL / 548
:LTEST:COPY:MARGin / 549
:LTEST:ENABle / 550
:LTEST:FAIL / 551
:LTEST:LLIMit / 552
:LTEST:MEASurement / 553
:LTEST:RESults / 554
:LTEST:RUMode:SOFailure / 555
:LTEST:TEST / 556
:LTEST:ULIMit / 557

25 :MARKer Commands

:MARKer:DYDX / 562
:MARKer:MODE / 563
:MARKer:X1:DISPlay / 564
:MARKer:X1Position / 565
:MARKer:X1Y1source / 566
:MARKer:X2:DISPlay / 567
:MARKer:X2Position / 568
:MARKer:X2Y2source / 569
:MARKer:XDELta / 570
:MARKer:XUNits / 571
:MARKer:XUNits:USE / 572
:MARKer:Y1:DISPlay / 573
:MARKer:Y1Position / 574

:MARKer:Y2:DISPlay / 575
:MARKer:Y2Position / 576
:MARKer:YDELta / 577
:MARKer:YUNits / 578
:MARKer:YUNits:USE / 579

26 :MEASure Commands

:MEASure:ALL / 603
:MEASure:AREa / 604
:MEASure:BRATe / 605
:MEASure:BWIDth / 606
:MEASure:CLEar / 607
:MEASure:COUNter / 608
:MEASure:DEFine / 610
:MEASure:DELay / 613
:MEASure:DELay:DEFine / 615
:MEASure:DUAL:CHARge / 616
:MEASure:DUAL:VAMPLitude / 617
:MEASure:DUAL:VAVerage / 618
:MEASure:DUAL:VBASe / 619
:MEASure:DUAL:VPP / 620
:MEASure:DUAL:VRMS / 621
:MEASure:DUTYcycle / 622
:MEASure:FALLtime / 623
:MEASure:FFT:ACPR / 624
:MEASure:FFT:CPOWer / 625
:MEASure:FFT:OBW / 626
:MEASure:FFT:THD / 627
:MEASure:FREQuency / 628
:MEASure:HISTogram:BWIDth / 629
:MEASure:HISTogram:HITS / 630
:MEASure:HISTogram:M1S / 631
:MEASure:HISTogram:M2S / 632
:MEASure:HISTogram:M3S / 633
:MEASure:HISTogram:MAXimum / 634
:MEASure:HISTogram:MEAN / 635
:MEASure:HISTogram:MEDian / 636
:MEASure:HISTogram:MINimum / 637
:MEASure:HISTogram:MODE / 638
:MEASure:HISTogram:PEAK / 639
:MEASure:HISTogram:PPEak / 640

:MEASure:HISTogram:SDEVIation / 641
:MEASure:NDUTy / 642
:MEASure:NEDGes / 643
:MEASure:NPULses / 644
:MEASure:NWIDth / 645
:MEASure:OVERshoot / 646
:MEASure:PEDGes / 648
:MEASure:PERiod / 649
:MEASure:PHASe / 650
:MEASure:PPULses / 651
:MEASure:PREShoot / 652
:MEASure:PWIDth / 653
:MEASure:RESults / 654
:MEASure:RISetime / 657
:MEASure:SDEVIation / 658
:MEASure:SHOW / 659
:MEASure:SLEWrate / 660
:MEASure:SOURce / 661
:MEASure:STATistics / 663
:MEASure:STATistics:DISPlay / 664
:MEASure:STATistics:INCRement / 665
:MEASure:STATistics:MCOunt / 666
:MEASure:STATistics:RESet / 667
:MEASure:STATistics:RSDeviation / 668
:MEASure:TEDGe / 669
:MEASure:TVALue / 672
:MEASure:VAMPLitude / 674
:MEASure:VAverage / 675
:MEASure:VBASe / 676
:MEASure:VMAX / 677
:MEASure:VMIN / 678
:MEASure:VPP / 679
:MEASure:VRATio / 680
:MEASure:VRMS / 681
:MEASure:VTOP / 682
:MEASure:WINDow / 683
:MEASure:XMAX / 684
:MEASure:XMIN / 685
:MEASure:YATX / 686

27 :MEASure Power Commands

:MEASure:ANGLE / 692
:MEASure:APParent / 693
:MEASure:CPLoss / 694
:MEASure:CRESt / 695
:MEASure:EFFiciency / 696
:MEASure:ELOSs / 697
:MEASure:FACTor / 698
:MEASure:IPOWer / 699
:MEASure:OFFTime / 700
:MEASure:ONTime / 701
:MEASure:OPOWer / 702
:MEASure:PCURrent / 703
:MEASure:PLOSs / 704
:MEASure:RDSon / 705
:MEASure:REACtive / 706
:MEASure:REAL / 707
:MEASure:RIPple / 708
:MEASure:TRESponse / 709
:MEASure:VCESat / 710

28 :MTEST Commands

:MTEST:ALL / 716
:MTEST:AMASk:CREate / 717
:MTEST:AMASk:SOURce / 718
:MTEST:AMASk:UNITs / 719
:MTEST:AMASk:XDELta / 720
:MTEST:AMASk:YDELta / 721
:MTEST:COUNt:FWAVEforms / 722
:MTEST:COUNt:RESet / 723
:MTEST:COUNt:TIME / 724
:MTEST:COUNt:WAVEforms / 725
:MTEST:DATA / 726
:MTEST:DELeTe / 727
:MTEST:ENABLE / 728
:MTEST:LOCK / 729
:MTEST:RMODE / 730
:MTEST:RMODE:FACTION:MEASure / 731
:MTEST:RMODE:FACTION:PRINT / 732
:MTEST:RMODE:FACTION:SAVE / 733
:MTEST:RMODE:FACTION:STOP / 734

:MTEST:RMODE:SIGMa / 735
:MTEST:RMODE:TIME / 736
:MTEST:RMODE:WAVEforms / 737
:MTEST:SCALE:BIND / 738
:MTEST:SCALE:X1 / 739
:MTEST:SCALE:XDELta / 740
:MTEST:SCALE:Y1 / 741
:MTEST:SCALE:Y2 / 742
:MTEST:SOURce / 743
:MTEST:TITLe / 744

29 :POD Commands

:POD<n>:DISPlay / 747
:POD<n>:SIZE / 748
:POD<n>:THReshold / 749

30 :POWer Commands

:POWer:CLResponse / 759
:POWer:CLResponse:APPLy / 760
:POWer:CLResponse:DATA / 761
:POWer:CLResponse:DATA:GMARgin / 762
:POWer:CLResponse:DATA:GMARgin:FREQuency / 763
:POWer:CLResponse:DATA:PMARgin / 764
:POWer:CLResponse:DATA:PMARgin:FREQuency / 765
:POWer:CLResponse:FREQuency:MODE / 766
:POWer:CLResponse:FREQuency:SINGLE / 767
:POWer:CLResponse:FREQuency:START / 768
:POWer:CLResponse:FREQuency:STOP / 769
:POWer:CLResponse:PPDecade / 770
:POWer:CLResponse:SOURce:INPut / 771
:POWer:CLResponse:SOURce:OUTPut / 772
:POWer:CLResponse:TRACe / 773
:POWer:CLResponse:WGEN:LOAD / 774
:POWer:CLResponse:WGEN:VOLTage / 775
:POWer:CLResponse:WGEN:VOLTage:PROFile / 776
:POWer:DESKew / 777
:POWer:EFFiciency:APPLy / 778
:POWer:EFFiciency:TYPE / 779
:POWer:ENABLE / 780
:POWer:HARMonics:APPLy / 781
:POWer:HARMonics:DATA / 782

:POWer:HARMonics:DISPlay / 783
:POWer:HARMonics:FAILcount / 784
:POWer:HARMonics:LINE / 785
:POWer:HARMonics:POWerfactor / 786
:POWer:HARMonics:RPOWer / 787
:POWer:HARMonics:RPOWer:USER / 788
:POWer:HARMonics:RUNCount / 789
:POWer:HARMonics:STANdard / 790
:POWer:HARMonics:STATus / 791
:POWer:HARMonics:THD / 792
:POWer:INRush:APPLy / 793
:POWer:INRush:EXIT / 794
:POWer:INRush:NEXT / 795
:POWer:ITYPE / 796
:POWer:MODulation:APPLy / 797
:POWer:MODulation:SOURce / 798
:POWer:MODulation:TYPE / 799
:POWer:ONOff:APPLy / 800
:POWer:ONOff:EXIT / 801
:POWer:ONOff:NEXT / 802
:POWer:ONOff:TEST / 803
:POWer:ONOff:THResholds / 804
:POWer:PSRR / 806
:POWer:PSRR:APPLy / 807
:POWer:PSRR:DATA / 808
:POWer:PSRR:FREQuency:MAXimum / 809
:POWer:PSRR:FREQuency:MINimum / 810
:POWer:PSRR:FREQuency:MODE / 811
:POWer:PSRR:FREQuency:SINGLE / 812
:POWer:PSRR:PPDecade / 813
:POWer:PSRR:SOURce:INPut / 814
:POWer:PSRR:SOURce:OUTPut / 815
:POWer:PSRR:TRACe / 816
:POWer:PSRR:WGEN:LOAD / 817
:POWer:PSRR:WGEN:VOLTage / 818
:POWer:PSRR:WGEN:VOLTage:PROFile / 819
:POWer:QUALity:APPLy / 820
:POWer:RIPPLE:APPLy / 821
:POWer:SIGNals:AUTosetup / 822
:POWer:SIGNals:CYCLes:HARMonics / 823
:POWer:SIGNals:CYCLes:QUALity / 824
:POWer:SIGNals:DURation:EFFiciency / 825

:POWer:SIGNals:DURation:MODulation / 826
 :POWer:SIGNals:DURation:ONOff:OFF / 827
 :POWer:SIGNals:DURation:ONOff:ON / 828
 :POWer:SIGNals:DURation:RIPple / 829
 :POWer:SIGNals:DURation:TRANsient / 830
 :POWer:SIGNals:IEXpected / 831
 :POWer:SIGNals:OVERshoot / 832
 :POWer:SIGNals:VMAXimum:INRush / 833
 :POWer:SIGNals:VMAXimum:ONOff:OFF / 834
 :POWer:SIGNals:VMAXimum:ONOff:ON / 835
 :POWer:SIGNals:VSTeady:ONOff:OFF / 836
 :POWer:SIGNals:VSTeady:ONOff:ON / 837
 :POWer:SIGNals:VSTeady:TRANsient / 838
 :POWer:SIGNals:SOURce:CURRent<i> / 839
 :POWer:SIGNals:SOURce:VOLTage<i> / 840
 :POWer:SLEW:APPLy / 841
 :POWer:SLEW:SOURce / 842
 :POWer:SWITCh:APPLy / 843
 :POWer:SWITCh:CONDUction / 844
 :POWer:SWITCh:IREFerence / 845
 :POWer:SWITCh:RDS / 846
 :POWer:SWITCh:VCE / 847
 :POWer:SWITCh:VREFerence / 848
 :POWer:TRANsient:APPLy / 849
 :POWer:TRANsient:EXIT / 850
 :POWer:TRANsient:IInitial / 851
 :POWer:TRANsient:INEW / 852
 :POWer:TRANsient:NEXT / 853

31 :RECall Commands

:RECall:ARBitrary[:START] / 858
 :RECall:DBC[:START] / 859
 :RECall:FIleName / 860
 :RECall:LDF[:START] / 861
 :RECall:MASK[:START] / 862
 :RECall:PWD / 863
 :RECall:SETup[:START] / 864
 :RECall:WMEMory<r>[:START] / 865

32 :SAVE Commands

:SAVE:ARBitrary[:START] / 871

:SAVE:COMPLIance:USB[:START] / 872
 :SAVE:FILENAME / 873
 :SAVE:IMAGe[:START] / 874
 :SAVE:IMAGe:FACTors / 875
 :SAVE:IMAGe:FORMat / 876
 :SAVE:IMAGe:INKSaver / 877
 :SAVE:IMAGe:PALette / 878
 :SAVE:LISTer[:START] / 879
 :SAVE:MASK[:START] / 880
 :SAVE:MULTi[:START] / 881
 :SAVE:POWEr[:START] / 882
 :SAVE:PWD / 883
 :SAVE:RESults[:START] / 884
 :SAVE:RESults:FORMat:CURSor / 885
 :SAVE:RESults:FORMat:HISTogram / 886
 :SAVE:RESults:FORMat:MASK / 887
 :SAVE:RESults:FORMat:MEASurement / 888
 :SAVE:RESults:FORMat:SEARch / 889
 :SAVE:RESults:FORMat:SEGmented / 890
 :SAVE[:SETup[:START]] / 891
 :SAVE:WAVEform[:START] / 892
 :SAVE:WAVEform:FORMat / 893
 :SAVE:WAVEform:LENGth / 894
 :SAVE:WAVEform:LENGth:MAX / 895
 :SAVE:WAVEform:SEGmented / 896
 :SAVE:WMEMory:SOURce / 897
 :SAVE:WMEMory[:START] / 898

33 :SBUS<n> Commands

General :SBUS<n> Commands / 901
 :SBUS<n>:DISPlay / 902
 :SBUS<n>:MODE / 903
 :SBUS<n>:A429 Commands / 904
 :SBUS<n>:A429:AUTosetup / 906
 :SBUS<n>:A429:BASE / 907
 :SBUS<n>:A429:BAUDrate / 908
 :SBUS<n>:A429:COUNt:ERRor / 909
 :SBUS<n>:A429:COUNt:RESet / 910
 :SBUS<n>:A429:COUNt:WORD / 911
 :SBUS<n>:A429:FORMat / 912
 :SBUS<n>:A429:SIGNal / 913

- :SBUS<n>:A429:SOURce / 914
- :SBUS<n>:A429:SPEEd / 915
- :SBUS<n>:A429:TRIGger:LABel / 916
- :SBUS<n>:A429:TRIGger:PATtern:DATA / 917
- :SBUS<n>:A429:TRIGger:PATtern:SDI / 918
- :SBUS<n>:A429:TRIGger:PATtern:SSM / 919
- :SBUS<n>:A429:TRIGger:RANGe / 920
- :SBUS<n>:A429:TRIGger:TYPE / 921
- :SBUS<n>:CAN Commands / 923
 - :SBUS<n>:CAN:COUNT:ERRor / 926
 - :SBUS<n>:CAN:COUNT:OVERload / 927
 - :SBUS<n>:CAN:COUNT:RESet / 928
 - :SBUS<n>:CAN:COUNT:SPEC / 929
 - :SBUS<n>:CAN:COUNT:TOTal / 930
 - :SBUS<n>:CAN:COUNT:UTILization / 931
 - :SBUS<n>:CAN:DISPlay / 932
 - :SBUS<n>:CAN:FDSPoint / 933
 - :SBUS<n>:CAN:FDSTandard / 934
 - :SBUS<n>:CAN:SAMPlepoint / 935
 - :SBUS<n>:CAN:SIGNal:BAUDrate / 936
 - :SBUS<n>:CAN:SIGNal:DEFinition / 937
 - :SBUS<n>:CAN:SIGNal:FDBaudrate / 938
 - :SBUS<n>:CAN:SOURce / 939
 - :SBUS<n>:CAN:TRIGger / 940
 - :SBUS<n>:CAN:TRIGger:IDFilter / 943
 - :SBUS<n>:CAN:TRIGger:PATtern:DATA / 944
 - :SBUS<n>:CAN:TRIGger:PATtern:DATA:DLC / 945
 - :SBUS<n>:CAN:TRIGger:PATtern:DATA:LENGth / 946
 - :SBUS<n>:CAN:TRIGger:PATtern:DATA:STARt / 947
 - :SBUS<n>:CAN:TRIGger:PATtern:ID / 948
 - :SBUS<n>:CAN:TRIGger:PATtern:ID:MODE / 949
 - :SBUS<n>:CAN:TRIGger:SYMBolic:MESSage / 950
 - :SBUS<n>:CAN:TRIGger:SYMBolic:SIGNal / 951
 - :SBUS<n>:CAN:TRIGger:SYMBolic:VALue / 952
- :SBUS<n>:CXPI Commands / 953
 - :SBUS<n>:CXPI:BAUDrate / 955
 - :SBUS<n>:CXPI:PARity / 956
 - :SBUS<n>:CXPI:SOURce / 957
 - :SBUS<n>:CXPI:TOLerance / 958
 - :SBUS<n>:CXPI:TRIGger / 959
 - :SBUS<n>:CXPI:TRIGger:IDFilter / 961

- :SBUS<n>:CXPI:TRIGger:PTYPE / 962
- :SBUS<n>:CXPI:TRIGger:PATtern:DATA / 963
- :SBUS<n>:CXPI:TRIGger:PATtern:DATA:LENGth / 964
- :SBUS<n>:CXPI:TRIGger:PATtern:DATA:STARt / 965
- :SBUS<n>:CXPI:TRIGger:PATtern:ID / 966
- :SBUS<n>:CXPI:TRIGger:PATtern:INFO:CT / 967
- :SBUS<n>:CXPI:TRIGger:PATtern:INFO:DLC / 968
- :SBUS<n>:CXPI:TRIGger:PATtern:INFO:NM / 969
- :SBUS<n>:FLEXray Commands / 970
 - :SBUS<n>:FLEXray:AUTosetup / 972
 - :SBUS<n>:FLEXray:BAUDrate / 973
 - :SBUS<n>:FLEXray:CHANnel / 974
 - :SBUS<n>:FLEXray:COUNt:NULL / 975
 - :SBUS<n>:FLEXray:COUNt:RESet / 976
 - :SBUS<n>:FLEXray:COUNt:SYNC / 977
 - :SBUS<n>:FLEXray:COUNt:TOTal / 978
 - :SBUS<n>:FLEXray:SOURce / 979
 - :SBUS<n>:FLEXray:TRIGger / 980
 - :SBUS<n>:FLEXray:TRIGger:ERRor:TYPE / 981
 - :SBUS<n>:FLEXray:TRIGger:EVENT:AUToset / 982
 - :SBUS<n>:FLEXray:TRIGger:EVENT:BSS:ID / 983
 - :SBUS<n>:FLEXray:TRIGger:EVENT:TYPE / 984
 - :SBUS<n>:FLEXray:TRIGger:FRAMe:CCBase / 985
 - :SBUS<n>:FLEXray:TRIGger:FRAMe:CCRepetition / 986
 - :SBUS<n>:FLEXray:TRIGger:FRAMe:ID / 987
 - :SBUS<n>:FLEXray:TRIGger:FRAMe:TYPE / 988
- :SBUS<n>:I2S Commands / 989
 - :SBUS<n>:I2S:ALIGnment / 992
 - :SBUS<n>:I2S:BASE / 993
 - :SBUS<n>:I2S:CLOCK:SLOPe / 994
 - :SBUS<n>:I2S:RWIDth / 995
 - :SBUS<n>:I2S:SOURce:CLOCK / 996
 - :SBUS<n>:I2S:SOURce:DATA / 997
 - :SBUS<n>:I2S:SOURce:WSElect / 998
 - :SBUS<n>:I2S:TRIGger / 999
 - :SBUS<n>:I2S:TRIGger:AUDio / 1001
 - :SBUS<n>:I2S:TRIGger:PATtern:DATA / 1002
 - :SBUS<n>:I2S:TRIGger:PATtern:FORMat / 1004
 - :SBUS<n>:I2S:TRIGger:RANGe / 1005
 - :SBUS<n>:I2S:TWIDth / 1006
 - :SBUS<n>:I2S:WSLow / 1007

- :SBUS<n>:IIC Commands / 1008
 - :SBUS<n>:IIC:ASIZe / 1010
 - :SBUS<n>:IIC[:SOURce]:CLOCK / 1011
 - :SBUS<n>:IIC[:SOURce]:DATA / 1012
 - :SBUS<n>:IIC:TRIGger:PATtern:ADDReSS / 1013
 - :SBUS<n>:IIC:TRIGger:PATtern:DATA / 1014
 - :SBUS<n>:IIC:TRIGger:PATtern:DATA2 / 1015
 - :SBUS<n>:IIC:TRIGger:QUALifier / 1016
 - :SBUS<n>:IIC:TRIGger[:TYPE] / 1017
- :SBUS<n>:LIN Commands / 1019
 - :SBUS<n>:LIN:DISPlay / 1021
 - :SBUS<n>:LIN:PARity / 1022
 - :SBUS<n>:LIN:SAMPlepoint / 1023
 - :SBUS<n>:LIN:SIGNAL:BAUDRate / 1024
 - :SBUS<n>:LIN:SOURce / 1025
 - :SBUS<n>:LIN:STANdard / 1026
 - :SBUS<n>:LIN:SYNCbreak / 1027
 - :SBUS<n>:LIN:TRIGger / 1028
 - :SBUS<n>:LIN:TRIGger:ID / 1030
 - :SBUS<n>:LIN:TRIGger:PATtern:DATA / 1031
 - :SBUS<n>:LIN:TRIGger:PATtern:DATA:LENGth / 1033
 - :SBUS<n>:LIN:TRIGger:PATtern:FORMat / 1034
 - :SBUS<n>:LIN:TRIGger:SYMBOLic:FRAMe / 1035
 - :SBUS<n>:LIN:TRIGger:SYMBOLic:SIGNAL / 1036
 - :SBUS<n>:LIN:TRIGger:SYMBOLic:VALue / 1037
- :SBUS<n>:M1553 Commands / 1038
 - :SBUS<n>:M1553:AUTOsetup / 1039
 - :SBUS<n>:M1553:BASE / 1040
 - :SBUS<n>:M1553:SOURce / 1041
 - :SBUS<n>:M1553:TRIGger:PATtern:DATA / 1042
 - :SBUS<n>:M1553:TRIGger:RTA / 1043
 - :SBUS<n>:M1553:TRIGger:TYPE / 1044
- :SBUS<n>:MANChester Commands / 1045
 - :SBUS<n>:MANChester:BASE / 1047
 - :SBUS<n>:MANChester:BAUDRate / 1048
 - :SBUS<n>:MANChester:BITorder / 1049
 - :SBUS<n>:MANChester:DISPlay / 1050
 - :SBUS<n>:MANChester:DSIZe / 1051
 - :SBUS<n>:MANChester:HSIZe / 1052
 - :SBUS<n>:MANChester:IDLE:BITS / 1053

```

:SBUS<n>:MANChester:LOGic / 1054
:SBUS<n>:MANChester:SOURce / 1055
:SBUS<n>:MANChester:SSIZe / 1056
:SBUS<n>:MANChester:START / 1057
:SBUS<n>:MANChester:TOLerance / 1058
:SBUS<n>:MANChester:TRIGger / 1059
:SBUS<n>:MANChester:TRIGger:PATtern:VALue:DATA / 1060
:SBUS<n>:MANChester:TRIGger:PATtern:VALue:WIDTh / 1061
:SBUS<n>:MANChester:TSIZe / 1062
:SBUS<n>:MANChester:WSIZe / 1063

:SBUS<n>:NRZ Commands / 1064
:SBUS<n>:NRZ:BASE / 1066
:SBUS<n>:NRZ:BAUDrate / 1067
:SBUS<n>:NRZ:BITorder / 1068
:SBUS<n>:NRZ:DISPlay / 1069
:SBUS<n>:NRZ:DSIZe / 1070
:SBUS<n>:NRZ:FSIZe / 1071
:SBUS<n>:NRZ:HSIZe / 1072
:SBUS<n>:NRZ:IDLE:BITS / 1073
:SBUS<n>:NRZ:IDLE:STATe / 1074
:SBUS<n>:NRZ:LOGic / 1075
:SBUS<n>:NRZ:SOURce / 1076
:SBUS<n>:NRZ:START / 1077
:SBUS<n>:NRZ:TRIGger / 1078
:SBUS<n>:NRZ:TRIGger:PATtern:VALue:DATA / 1079
:SBUS<n>:NRZ:TRIGger:PATtern:VALue:WIDTh / 1080
:SBUS<n>:NRZ:TSIZe / 1081
:SBUS<n>:NRZ:WSIZe / 1082

:SBUS<n>:SENT Commands / 1083
:SBUS<n>:SENT:CLOCK / 1086
:SBUS<n>:SENT:CRC / 1087
:SBUS<n>:SENT:DISPlay / 1088
:SBUS<n>:SENT:FORMat / 1090
:SBUS<n>:SENT:IDLE / 1092
:SBUS<n>:SENT:LENGth / 1093
:SBUS<n>:SENT:PPULse / 1094
:SBUS<n>:SENT:SIGNal<s>:DISPlay / 1096
:SBUS<n>:SENT:SIGNal<s>:LENGth / 1097
:SBUS<n>:SENT:SIGNal<s>:MULTiplier / 1099
:SBUS<n>:SENT:SIGNal<s>:OFFSet / 1101
:SBUS<n>:SENT:SIGNal<s>:ORDer / 1103

```

```

:SBUS<n>:SENT:SIGNAl<s>:START / 1105
:SBUS<n>:SENT:SOURce / 1107
:SBUS<n>:SENT:TOLerance / 1109
:SBUS<n>:SENT:TRIGger / 1110
:SBUS<n>:SENT:TRIGger:FAST:DATA / 1112
:SBUS<n>:SENT:TRIGger:SLOW:DATA / 1113
:SBUS<n>:SENT:TRIGger:SLOW:ID / 1115
:SBUS<n>:SENT:TRIGger:SLOW:ILENgtH / 1117
:SBUS<n>:SENT:TRIGger:TOLerance / 1118
:SBUS<n>:SPI Commands / 1119
:SBUS<n>:SPI:BITorder / 1121
:SBUS<n>:SPI:CLOCK:SLOPe / 1122
:SBUS<n>:SPI:CLOCK:TIMEout / 1123
:SBUS<n>:SPI:DELay / 1124
:SBUS<n>:SPI:FRAMing / 1125
:SBUS<n>:SPI:SOURce:CLOCK / 1126
:SBUS<n>:SPI:SOURce:FRAME / 1127
:SBUS<n>:SPI:SOURce:MISO / 1128
:SBUS<n>:SPI:SOURce:MOSI / 1129
:SBUS<n>:SPI:TRIGger:PATtern:MISO:DATA / 1130
:SBUS<n>:SPI:TRIGger:PATtern:MISO:WIDTh / 1131
:SBUS<n>:SPI:TRIGger:PATtern:MOSI:DATA / 1132
:SBUS<n>:SPI:TRIGger:PATtern:MOSI:WIDTh / 1133
:SBUS<n>:SPI:TRIGger:TYPE / 1134
:SBUS<n>:SPI:WIDTh / 1135
:SBUS<n>:UART Commands / 1136
:SBUS<n>:UART:BASE / 1139
:SBUS<n>:UART:BAUDrate / 1140
:SBUS<n>:UART:BITorder / 1141
:SBUS<n>:UART:COUNt:ERRor / 1142
:SBUS<n>:UART:COUNt:RESet / 1143
:SBUS<n>:UART:COUNt:RXFRames / 1144
:SBUS<n>:UART:COUNt:TXFRames / 1145
:SBUS<n>:UART:FRAMing / 1146
:SBUS<n>:UART:PARity / 1147
:SBUS<n>:UART:POLarity / 1148
:SBUS<n>:UART:SOURce:RX / 1149
:SBUS<n>:UART:SOURce:TX / 1150
:SBUS<n>:UART:TRIGger:BASE / 1151
:SBUS<n>:UART:TRIGger:BURSt / 1152
:SBUS<n>:UART:TRIGger:DATA / 1153

```

- :SBUS<n>:UART:TRIGger:IDLE / 1154
- :SBUS<n>:UART:TRIGger:QUALifier / 1155
- :SBUS<n>:UART:TRIGger:TYPE / 1156
- :SBUS<n>:UART:WIDTh / 1157
- :SBUS<n>:USB Commands / 1158
 - :SBUS<n>:USB:BASE / 1161
 - :SBUS<n>:USB:SOURce:DMINus / 1162
 - :SBUS<n>:USB:SOURce:DPLus / 1163
 - :SBUS<n>:USB:SOURce:DIFFerential / 1164
 - :SBUS<n>:USB:SPEEd / 1165
 - :SBUS<n>:USB:TRIGger / 1166
 - :SBUS<n>:USB:TRIGger:ADDResS / 1167
 - :SBUS<n>:USB:TRIGger:CRC / 1168
 - :SBUS<n>:USB:TRIGger:DATA / 1169
 - :SBUS<n>:USB:TRIGger:DATA:LENGth / 1170
 - :SBUS<n>:USB:TRIGger:ENDPoint / 1171
 - :SBUS<n>:USB:TRIGger:ET / 1172
 - :SBUS<n>:USB:TRIGger:FRAME / 1173
 - :SBUS<n>:USB:TRIGger:HADDress / 1174
 - :SBUS<n>:USB:TRIGger:PCHeck / 1175
 - :SBUS<n>:USB:TRIGger:PID:DATA / 1176
 - :SBUS<n>:USB:TRIGger:PID:HANDshake / 1177
 - :SBUS<n>:USB:TRIGger:PID:SPECial / 1178
 - :SBUS<n>:USB:TRIGger:PID:TOKen / 1179
 - :SBUS<n>:USB:TRIGger:PORT / 1180
 - :SBUS<n>:USB:TRIGger:SC / 1181
 - :SBUS<n>:USB:TRIGger:SEU / 1182
- :SBUS<n>:USBPd Commands / 1183
 - :SBUS<n>:USBPd:SOURce / 1184
 - :SBUS<n>:USBPd:TRIGger / 1185
 - :SBUS<n>:USBPd:TRIGger:HEADer / 1186
 - :SBUS<n>:USBPd:TRIGger:HEADer:CMESsage / 1188
 - :SBUS<n>:USBPd:TRIGger:HEADer:DMESsage / 1190
 - :SBUS<n>:USBPd:TRIGger:HEADer:EMESsage / 1191
 - :SBUS<n>:USBPd:TRIGger:HEADer:VALue / 1193
 - :SBUS<n>:USBPd:TRIGger:HEADer:QUALifier / 1194

34 :SEARCh Commands

- General :SEARCh Commands / 1196
 - :SEARCh:COUNt / 1197

- :SEARch:EVENT / 1198
- :SEARch:MODE / 1199
- :SEARch:STATe / 1200
- :SEARch:EDGE Commands / 1201
 - :SEARch:EDGE:SLOPe / 1202
 - :SEARch:EDGE:SOURce / 1203
- :SEARch:GLITCh Commands / 1204
 - :SEARch:GLITCh:GREaterthan / 1205
 - :SEARch:GLITCh:LESSthan / 1206
 - :SEARch:GLITCh:POLarity / 1207
 - :SEARch:GLITCh:QUALifier / 1208
 - :SEARch:GLITCh:RANGe / 1209
 - :SEARch:GLITCh:SOURce / 1210
- :SEARch:PEAK Commands / 1211
 - :SEARch:PEAK:EXCursion / 1212
 - :SEARch:PEAK:NPEaks / 1213
 - :SEARch:PEAK:SOURce / 1214
 - :SEARch:PEAK:THReshold / 1215
- :SEARch:RUNT Commands / 1216
 - :SEARch:RUNT:POLarity / 1217
 - :SEARch:RUNT:QUALifier / 1218
 - :SEARch:RUNT:SOURce / 1219
 - :SEARch:RUNT:TIME / 1220
- :SEARch:TRANSition Commands / 1221
 - :SEARch:TRANSition:QUALifier / 1222
 - :SEARch:TRANSition:SLOPe / 1223
 - :SEARch:TRANSition:SOURce / 1224
 - :SEARch:TRANSition:TIME / 1225
- :SEARch:SERial:A429 Commands / 1226
 - :SEARch:SERial:A429:LABel / 1227
 - :SEARch:SERial:A429:MODE / 1228
 - :SEARch:SERial:A429:PATtern:DATA / 1229
 - :SEARch:SERial:A429:PATtern:SDI / 1230
 - :SEARch:SERial:A429:PATtern:SSM / 1231
- :SEARch:SERial:CAN Commands / 1232
 - :SEARch:SERial:CAN:MODE / 1233
 - :SEARch:SERial:CAN:PATtern:DATA / 1235
 - :SEARch:SERial:CAN:PATtern:DATA:LENGth / 1236
 - :SEARch:SERial:CAN:PATtern:ID / 1237

- :SEARch:SERIal:CAN:PATtern:ID:MODE / 1238
- :SEARch:SERIal:CAN:SYMBolic:MESSAge / 1239
- :SEARch:SERIal:CAN:SYMBolic:SIGNal / 1240
- :SEARch:SERIal:CAN:SYMBolic:VALue / 1241
- :SEARch:SERIal:FLEXray Commands / 1242
 - :SEARch:SERIal:FLEXray:CYCLe / 1243
 - :SEARch:SERIal:FLEXray:DATA / 1244
 - :SEARch:SERIal:FLEXray:DATA:LENGth / 1245
 - :SEARch:SERIal:FLEXray:FRAMe / 1246
 - :SEARch:SERIal:FLEXray:MODE / 1247
- :SEARch:SERIal:I2S Commands / 1248
 - :SEARch:SERIal:I2S:AUDio / 1249
 - :SEARch:SERIal:I2S:MODE / 1250
 - :SEARch:SERIal:I2S:PATtern:DATA / 1251
 - :SEARch:SERIal:I2S:PATtern:FORMat / 1252
 - :SEARch:SERIal:I2S:RANGe / 1253
- :SEARch:SERIal:IIC Commands / 1254
 - :SEARch:SERIal:IIC:MODE / 1255
 - :SEARch:SERIal:IIC:PATtern:ADDReSS / 1257
 - :SEARch:SERIal:IIC:PATtern:DATA / 1258
 - :SEARch:SERIal:IIC:PATtern:DATA2 / 1259
 - :SEARch:SERIal:IIC:QUALifier / 1260
- :SEARch:SERIal:LIN Commands / 1261
 - :SEARch:SERIal:LIN:ID / 1262
 - :SEARch:SERIal:LIN:MODE / 1263
 - :SEARch:SERIal:LIN:PATtern:DATA / 1264
 - :SEARch:SERIal:LIN:PATtern:DATA:LENGth / 1265
 - :SEARch:SERIal:LIN:PATtern:FORMat / 1266
 - :SEARch:SERIal:LIN:SYMBolic:FRAMe / 1267
 - :SEARch:SERIal:LIN:SYMBolic:SIGNal / 1268
 - :SEARch:SERIal:LIN:SYMBolic:VALue / 1269
- :SEARch:SERIal:M1553 Commands / 1270
 - :SEARch:SERIal:M1553:MODE / 1271
 - :SEARch:SERIal:M1553:PATtern:DATA / 1272
 - :SEARch:SERIal:M1553:RTA / 1273
- :SEARch:SERIal:SENT Commands / 1274
 - :SEARch:SERIal:SENT:FAST:DATA / 1275
 - :SEARch:SERIal:SENT:MODE / 1276
 - :SEARch:SERIal:SENT:SLOW:DATA / 1277

- :SEARch:SERIal:SENT:SLOW:ID / 1278
- :SEARch:SERIal:SPI Commands / 1279
 - :SEARch:SERIal:SPI:MODE / 1280
 - :SEARch:SERIal:SPI:PATtern:DATA / 1281
 - :SEARch:SERIal:SPI:PATtern:WIDTh / 1282
- :SEARch:SERIal:UART Commands / 1283
 - :SEARch:SERIal:UART:DATA / 1284
 - :SEARch:SERIal:UART:MODE / 1285
 - :SEARch:SERIal:UART:QUALifier / 1286
- :SEARch:SERIal:USB Commands / 1287
 - :SEARch:SERIal:USB:MODE / 1289
 - :SEARch:SERIal:USB:ADDResS / 1290
 - :SEARch:SERIal:USB:CRC / 1291
 - :SEARch:SERIal:USB:DATA / 1292
 - :SEARch:SERIal:USB:DATA:LENGth / 1293
 - :SEARch:SERIal:USB:ENDPoint / 1294
 - :SEARch:SERIal:USB:ET / 1295
 - :SEARch:SERIal:USB:FRAMe / 1296
 - :SEARch:SERIal:USB:HADDress / 1297
 - :SEARch:SERIal:USB:PID:DATA / 1298
 - :SEARch:SERIal:USB:PID:HANDshake / 1299
 - :SEARch:SERIal:USB:PID:SPECial / 1300
 - :SEARch:SERIal:USB:PID:TOKen / 1301
 - :SEARch:SERIal:USB:PORT / 1302
 - :SEARch:SERIal:USB:SC / 1303
 - :SEARch:SERIal:USB:SEU / 1304

35 :SYSTem Commands

- :SYSTem:DATE / 1308
- :SYSTem:DIDentifier / 1309
- :SYSTem:DSP / 1310
- :SYSTem:ERRor / 1311
- :SYSTem:LOCK / 1312
- :SYSTem:PERSonA[:MANUFACTurer] / 1313
- :SYSTem:PERSonA[:MANUFACTurer]:DEFault / 1314
- :SYSTem:PRESet / 1315
- :SYSTem:PROTection:LOCK / 1318
- :SYSTem:RLOGger / 1319
- :SYSTem:RLOGger:DESTination / 1320
- :SYSTem:RLOGger:DISPlay / 1321

:SYSTem:RLOGger:FNAME / 1322
:SYSTem:RLOGger:STATe / 1323
:SYSTem:RLOGger:TRANSPARENT / 1324
:SYSTem:RLOGger:WMODE / 1325
:SYSTem:SETup / 1326
:SYSTem:TIME / 1328
:SYSTem:TOUCH / 1329

36 :TIMebase Commands

:TIMebase:MODE / 1333
:TIMebase:POSition / 1334
:TIMebase:RANGE / 1335
:TIMebase:REFClock / 1336
:TIMebase:REFerence / 1337
:TIMebase:REFerence:LOCation / 1338
:TIMebase:SCALE / 1339
:TIMebase:VERNier / 1340
:TIMebase:WINDow:POSition / 1341
:TIMebase:WINDow:RANGE / 1342
:TIMebase:WINDow:SCALE / 1343

37 :TRIGger Commands

General :TRIGger Commands / 1347
:TRIGger:FORCE / 1349
:TRIGger:HFReject / 1350
:TRIGger:HOLDoff / 1351
:TRIGger:HOLDoff:MAXimum / 1352
:TRIGger:HOLDoff:MINimum / 1353
:TRIGger:HOLDoff:RANDom / 1354
:TRIGger:LEVel:ASETup / 1355
:TRIGger:LEVel:HIGH / 1356
:TRIGger:LEVel:LOW / 1357
:TRIGger:MODE / 1358
:TRIGger:NREject / 1359
:TRIGger:SWEep / 1360
:TRIGger:DELay Commands / 1361
:TRIGger:DELay:ARM:SLOPe / 1362
:TRIGger:DELay:ARM:SOURce / 1363
:TRIGger:DELay:TDELay:TIME / 1364
:TRIGger:DELay:TRIGger:COUNt / 1365
:TRIGger:DELay:TRIGger:SLOPe / 1366

- :TRIGger:DElay:TRIGger:SOURce / 1367
- :TRIGger:EBURst Commands / 1368
 - :TRIGger:EBURst:COUNt / 1369
 - :TRIGger:EBURst:IDLE / 1370
 - :TRIGger:EBURst:SLOPe / 1371
 - :TRIGger:EBURst:SOURce / 1372
- :TRIGger[:EDGE] Commands / 1373
 - :TRIGger[:EDGE]:COUPling / 1375
 - :TRIGger[:EDGE]:LEVel / 1376
 - :TRIGger[:EDGE]:REJect / 1377
 - :TRIGger[:EDGE]:SLOPe / 1378
 - :TRIGger[:EDGE]:SOURce / 1379
- :TRIGger:GLITch Commands / 1380
 - :TRIGger:GLITch:GREaterthan / 1382
 - :TRIGger:GLITch:LESSthan / 1383
 - :TRIGger:GLITch:LEVel / 1384
 - :TRIGger:GLITch:POLarity / 1385
 - :TRIGger:GLITch:QUALifier / 1386
 - :TRIGger:GLITch:RANGe / 1387
 - :TRIGger:GLITch:SOURce / 1388
- :TRIGger:OR Commands / 1389
 - :TRIGger:OR / 1390
- :TRIGger:PATtern Commands / 1391
 - :TRIGger:PATtern / 1392
 - :TRIGger:PATtern:FORMat / 1394
 - :TRIGger:PATtern:GREaterthan / 1395
 - :TRIGger:PATtern:LESSthan / 1396
 - :TRIGger:PATtern:QUALifier / 1397
 - :TRIGger:PATtern:RANGe / 1398
- :TRIGger:RUNT Commands / 1399
 - :TRIGger:RUNT:POLarity / 1400
 - :TRIGger:RUNT:QUALifier / 1401
 - :TRIGger:RUNT:SOURce / 1402
 - :TRIGger:RUNT:TIME / 1403
- :TRIGger:SHOLd Commands / 1404
 - :TRIGger:SHOLd:SLOPe / 1405
 - :TRIGger:SHOLd:SOURce:CLOCK / 1406
 - :TRIGger:SHOLd:SOURce:DATA / 1407
 - :TRIGger:SHOLd:TIME:HOLD / 1408

- :TRIGger:SHOLd:TIME:SETup / 1409
- :TRIGger:TRANSition Commands / 1410
 - :TRIGger:TRANSition:QUALifier / 1411
 - :TRIGger:TRANSition:SLOPe / 1412
 - :TRIGger:TRANSition:SOURce / 1413
 - :TRIGger:TRANSition:TIME / 1414
- :TRIGger:TV Commands / 1415
 - :TRIGger:TV:LINE / 1416
 - :TRIGger:TV:MODE / 1417
 - :TRIGger:TV:POLarity / 1418
 - :TRIGger:TV:SOURce / 1419
 - :TRIGger:TV:STANdard / 1420
 - :TRIGger:TV:UDTV:ENUMber / 1421
 - :TRIGger:TV:UDTV:HSYNc / 1422
 - :TRIGger:TV:UDTV:HTIME / 1423
 - :TRIGger:TV:UDTV:PGTHan / 1424
- :TRIGger:ZONE Commands / 1425
 - :TRIGger:ZONE:SOURce / 1426
 - :TRIGger:ZONE:STATe / 1427
 - :TRIGger:ZONE<n>:MODE / 1428
 - :TRIGger:ZONE<n>:PLACement / 1429
 - :TRIGger:ZONE<n>:VALidity / 1430
 - :TRIGger:ZONE<n>:STATe / 1431

38 :WAVeform Commands

- :WAVeform:BYTeorder / 1441
- :WAVeform:COUNt / 1442
- :WAVeform:DATA / 1443
- :WAVeform:FORMat / 1445
- :WAVeform:POINts / 1446
- :WAVeform:POINts:MODE / 1448
- :WAVeform:PREamble / 1450
- :WAVeform:SEGMented:ALL / 1453
- :WAVeform:SEGMented:COUNt / 1454
- :WAVeform:SEGMented:TTAG / 1455
- :WAVeform:SEGMented:XLIST / 1456
- :WAVeform:SOURce / 1457
- :WAVeform:SOURce:SUBSource / 1461
- :WAVeform:TYPE / 1462
- :WAVeform:UNSigned / 1463

:WAVeform:VIEW / 1464
 :WAVeform:XINCrement / 1465
 :WAVeform:XORigin / 1466
 :WAVeform:XREFerence / 1467
 :WAVeform:YINCrement / 1468
 :WAVeform:YORigin / 1469
 :WAVeform:YREFerence / 1470

39 :WGEN<w> Commands

:WGEN<w>:ARBitrary:BYTeorder / 1476
 :WGEN<w>:ARBitrary:DATA / 1477
 :WGEN<w>:ARBitrary:DATA:ATTRibute:POINts / 1480
 :WGEN<w>:ARBitrary:DATA:CLEar / 1481
 :WGEN<w>:ARBitrary:DATA:DAC / 1482
 :WGEN<w>:ARBitrary:INTErpolate / 1483
 :WGEN<w>:ARBitrary:STORe / 1484
 :WGEN<w>:FREQuency / 1485
 :WGEN<w>:FUNCTion / 1486
 :WGEN<w>:FUNCTion:PULSe:WIDTh / 1490
 :WGEN<w>:FUNCTion:RAMP:SYMMetry / 1491
 :WGEN<w>:FUNCTion:SQUare:DCYCLE / 1492
 :WGEN<w>:MODulation:AM:DEPTTh / 1493
 :WGEN<w>:MODulation:AM:FREQuency / 1494
 :WGEN<w>:MODulation:FM:DEViation / 1495
 :WGEN<w>:MODulation:FM:FREQuency / 1496
 :WGEN<w>:MODulation:FSKey:FREQuency / 1497
 :WGEN<w>:MODulation:FSKey:RATE / 1498
 :WGEN<w>:MODulation:FUNCTion / 1499
 :WGEN<w>:MODulation:FUNCTion:RAMP:SYMMetry / 1500
 :WGEN<w>:MODulation:NOISe / 1501
 :WGEN<w>:MODulation:STATe / 1502
 :WGEN<w>:MODulation:TYPE / 1503
 :WGEN<w>:OUTPut / 1505
 :WGEN<w>:OUTPut:LOAD / 1506
 :WGEN<w>:OUTPut:MODE / 1507
 :WGEN<w>:OUTPut:POLarity / 1508
 :WGEN<w>:OUTPut:SINGLE / 1509
 :WGEN<w>:PERiod / 1510
 :WGEN<w>:RST / 1511
 :WGEN<w>:TRACk / 1512
 :WGEN<w>:TRACk:AMPLitude / 1513

:WGEN<w>:TRACk:CSIGnal / 1514
 :WGEN<w>:TRACk:FREQuency / 1515
 :WGEN<w>:TRACk:PHASe / 1516
 :WGEN<w>:VOLTage / 1517
 :WGEN<w>:VOLTage:HIGH / 1518
 :WGEN<w>:VOLTage:LOW / 1519
 :WGEN<w>:VOLTage:OFFSet / 1520

40 :WMEMory<r> Commands

:WMEMory<r>:CLEar / 1523
 :WMEMory<r>:DISPlay / 1524
 :WMEMory<r>:LABel / 1525
 :WMEMory<r>:SAVE / 1526
 :WMEMory<r>:SKEW / 1527
 :WMEMory<r>:YOFFset / 1528
 :WMEMory<r>:YRANge / 1529
 :WMEMory<r>:YSCale / 1530

41 Obsolete and Discontinued Commands

:CHANnel:ACTivity / 1538
 :CHANnel:LABel / 1539
 :CHANnel:THReshold / 1540
 :CHANnel2:SKEW / 1541
 :CHANnel<n>:INPut / 1542
 :CHANnel<n>:PMODE / 1543
 :DISPlay:CONNect / 1544
 :DISPlay:ORDer / 1545
 :ERASe / 1546
 :EXTernal:PMODE / 1547
 :FUNCTion:GOFT:OPERation / 1548
 :FUNCTion:GOFT:SOURce1 / 1549
 :FUNCTion:GOFT:SOURce2 / 1550
 :FUNCTion:SOURce / 1551
 :FUNCTion:VIEW / 1552
 :HARDcopy:DESTination / 1553
 :HARDcopy:FILename / 1554
 :HARDcopy:GRAYscale / 1555
 :HARDcopy:IGColors / 1556
 :HARDcopy:PDRiver / 1557
 :MEASure:LOWer / 1558
 :MEASure:SCRatch / 1559

:MEASure:TDElta / 1560
 :MEASure:THReshols / 1561
 :MEASure:TMAX / 1562
 :MEASure:TMIN / 1563
 :MEASure:TSTArt / 1564
 :MEASure:TSTOp / 1565
 :MEASure:TVOLt / 1566
 :MEASure:UPPer / 1567
 :MEASure:VDElta / 1568
 :MEASure:VSTArt / 1569
 :MEASure:VSTOp / 1570
 :MEASure:VTIMe / 1571
 :MTESt:AMASk:{SAVE|STORe} / 1572
 :MTESt:AVERage / 1573
 :MTESt:AVERage:COUNT / 1574
 :MTESt:LOAD / 1575
 :MTESt:RUMode / 1576
 :MTESt:RUMode:SOFailure / 1577
 :MTESt:{STArt|STOp} / 1578
 :MTESt:TRIGger:SOURce / 1579
 :PRINt? / 1580
 :SAVE:IMAGe:AREA / 1582
 :SBUS<n>:LIN:SIGNal:DEFinition / 1583
 :SBUS<n>:SPI:SOURce:DATA / 1584
 :SYSTem:MENUp / 1585
 :TIMebase:DELay / 1586
 :TRIGger:THReshold / 1587
 :TRIGger:TV:TVMODE / 1588

42 Error Messages

43 Status Reporting

Status Reporting Data Structures / 1599
 Status Byte Register (STB) / 1602
 Service Request Enable Register (SRE) / 1604
 Trigger Event Register (TER) / 1605
 Output Queue / 1606
 Message Queue / 1607
 (Standard) Event Status Register (ESR) / 1608

- (Standard) Event Status Enable Register (ESE) / 1609
- Error Queue / 1610
- Operation Status Event Register (:OPERRegister[:EVENT]) / 1611
- Operation Status Condition Register (:OPERRegister:CONDition) / 1613
- Arm Event Register (AER) / 1614
- Overload Event Register (:OVLRegister) / 1615
- Hardware Event Event Register (:HWERegister[:EVENT]) / 1616
- Hardware Event Condition Register (:HWERegister:CONDition) / 1617
- Mask Test Event Event Register (:MTERegister[:EVENT]) / 1618
- Clearing Registers and Queues / 1619
- Status Reporting Decision Chart / 1620
- Example: Checking for Armed Status / 1621
- Example: Waiting for IO Operation Complete / 1626

44 Synchronizing Acquisitions

- Synchronization in the Programming Flow / 1630
 - Set Up the Oscilloscope / 1630
 - Acquire a Waveform / 1630
 - Retrieve Results / 1630
- Blocking Synchronization / 1631
- Polling Synchronization With Timeout / 1632
- Synchronizing with a Single-Shot Device Under Test (DUT) / 1634
- Synchronization with an Averaging Acquisition / 1636
- Example: Blocking and Polling Synchronization / 1638

45 More About Oscilloscope Commands

- Command Classifications / 1650
 - Core Commands / 1650
 - Non-Core Commands / 1650
 - Obsolete Commands / 1650
- Valid Command/Query Strings / 1651
 - Program Message Syntax / 1651
 - Duplicate Mnemonics / 1655
 - Tree Traversal Rules and Multiple Commands / 1655
- Query Return Values / 1657

46 Programming Examples

VISA COM Examples /	1660
VISA COM Example in Visual Basic /	1660
VISA COM Example in C# /	1669
VISA COM Example in Visual Basic .NET /	1678
VISA COM Example in Python /	1686
VISA Examples /	1694
VISA Example in C /	1694
VISA Example in Visual Basic /	1703
VISA Example in C# /	1713
VISA Example in Visual Basic .NET /	1724
VISA Example in Python /	1734
VISA.NET Examples /	1741
VISA.NET Example in C# /	1741
VISA.NET Example in Visual Basic .NET /	1747
VISA.NET Example in Python /	1753
SICL Examples /	1760
SICL Example in C /	1760
SICL Example in Visual Basic /	1769
SCPI.NET Examples /	1780

Index

1 What's New

What's New in Version 7.66 / 42

What's New in Version 7.65 / 43

Version 7.60 at Introduction / 44

Command Differences From 4000 X-Series Oscilloscopes / 45

What's New in Version 7.66

No new features are introduced in Version 7.66 of the InfiniiVision 4000G X-Series oscilloscope software, and only documentation corrections have been made to this programmer's guide.

More detailed descriptions of the new and changed commands appear below.

New Commands

Command	Differences
:CHANnel<n>:PROBe:EXTeRnal:OFFSet (see page 342)	Sets the vertical offset associated with External Scaling.

What's New in Version 7.65

No new features are introduced in Version 7.65 of the InfiniiVision 4000G X-Series oscilloscope software, and only documentation corrections have been made to this programmer's guide.

More detailed descriptions of the new and changed commands appear below.

New Commands

Command	Differences
:FRANalysis:SWEep:POINts (see page 442)	Specifies the number of points in the frequency response analysis. This command was previously available, but it is now documented.

Version 7.60 at Introduction

The Keysight InfiniiVision 4000G X-Series oscilloscopes were introduced with version 7.60 of oscilloscope operating software.

The command set is most closely related to the InfiniiVision 4000 X-Series oscilloscopes (and the 3000T/G X-Series, 6000 X-Series, 3000 X-Series, 7000A/B Series, 6000 Series, and 54620/54640 Series oscilloscopes before them). For more information, see "**Command Differences From 4000 X-Series Oscilloscopes**" on page 45.

Command Differences From 4000 X-Series Oscilloscopes

The Keysight InfiniiVision 4000G X-Series oscilloscopes command set is most closely related to the InfiniiVision 4000 X-Series oscilloscopes (and the 3000T/G X-Series, 6000 X-Series, 3000 X-Series, 7000A/B Series, 6000 Series, and 54620/54640 Series oscilloscopes before them).

The main differences between the version 7.60 programming command set for the InfiniiVision 4000G X-Series oscilloscopes and the 7.35 programming command set for the InfiniiVision 4000 X-Series oscilloscopes are related to:

- The addition of the Histogram feature (previously in the 6000 X-Series oscilloscopes).

More detailed descriptions of the new, changed, obsolete, and discontinued commands appear below.

New Commands

Command	Description
:HISTogram Commands (see page 525)	Commands for controlling the histogram analysis feature.
:MEASure:HISTogram:BWIDth (see page 629)	Returns the histogram bin width.
:MEASure:HISTogram:HITS (see page 630)	Returns the histogram total number of hits.
:MEASure:HISTogram:M1S (see page 631)	Returns the histogram percentage of hits within +/-1 standard deviation.
:MEASure:HISTogram:M2S (see page 632)	Returns the histogram percentage of hits within +/-2 standard deviations.
:MEASure:HISTogram:M3S (see page 633)	Returns the histogram percentage of hits within +/-3 standard deviations.
:MEASure:HISTogram:MAXimu m (see page 634)	Returns the histogram maximum bin value.
:MEASure:HISTogram:MEAN (see page 635)	Returns the histogram mean value.
:MEASure:HISTogram:MEDian (see page 636)	Returns the histogram median value.
:MEASure:HISTogram:MINimu m (see page 637)	Returns the histogram minimum bin value.
:MEASure:HISTogram:MODE (see page 638)	Returns the histogram mode value.
:MEASure:HISTogram:PEAK (see page 639)	Returns the histogram number of hits in max bin.

Command	Description
:MEASure:HISTogram:PPEak (see page 640)	Returns the histogram delta value between max bin and min bin.
:MEASure:HISTogram:SDEVIati on (see page 641)	Returns the histogram standard deviation value.
:SAVE:RESults:FORMat:HISTog ram (see page 886)	Specifies whether histogram statistics will be included when analysis results are saved.
:SYSTem:PERSon[:MANufactu rer] (see page 1313)	Lets you change the manufacturer string portion of the identification string returned by the *IDN? query.
:SYSTem:PERSon[:MANufactu rer]:DEFault (see page 1314)	Sets manufacturer string to "KEYSIGHT TECHNOLOGIES".

Changed Commands

Command	Differences From InfiniiVision 4000 X-Series Oscilloscopes
:DISPlay:SIDebar (see page 414)	The HISTogram option is available.
:MARKer:X1Y1source (see page 566)	The HISTogram source is available in certain modes.
:MARKer:X2Y2source (see page 569)	The HISTogram source is available in certain modes.

2 Setting Up

Step 1. Install Keysight IO Libraries Suite software / 48

Step 2. Connect and set up the oscilloscope / 49

Step 3. Verify the oscilloscope connection / 51

This chapter explains how to install the Keysight IO Libraries Suite software, connect the oscilloscope to the controller PC, set up the oscilloscope, and verify the oscilloscope connection.

Step 1. Install Keysight IO Libraries Suite software

- 1 Download the Keysight IO Libraries Suite software from the Keysight web site at:
 - <http://www.keysight.com/find/iolib>
- 2 Run the setup file, and follow its installation instructions.

Step 2. Connect and set up the oscilloscope

The 4000G X-Series oscilloscope has two different interfaces you can use for programming:

- USB (device port).
- LAN. To configure the LAN interface, press the **[Utility]** key on the front panel, then press the **I/O** softkey, then press the **Configure** softkey.

These interfaces are always active.

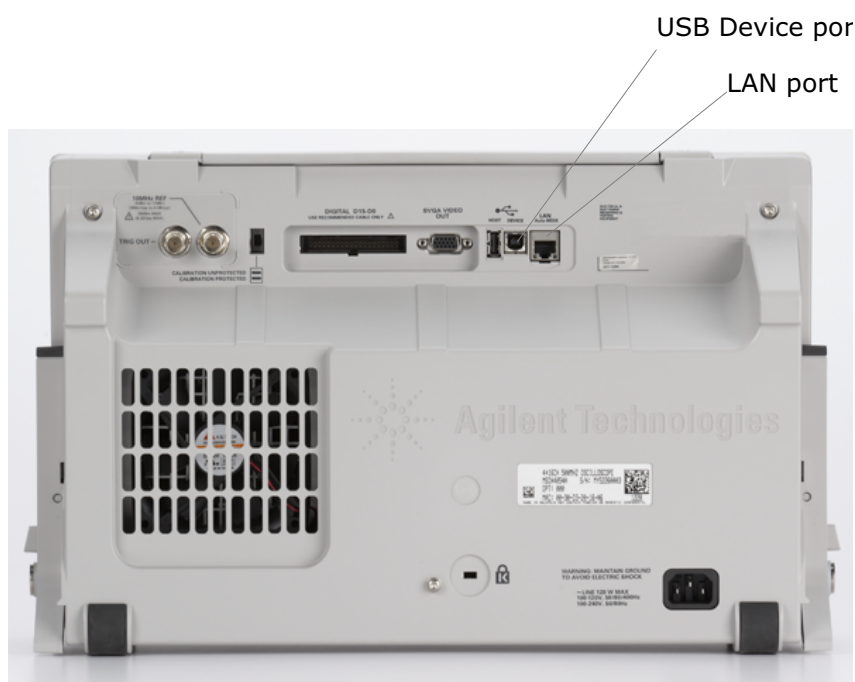


Figure 1 Control Connectors on Rear Panel

Using the USB (Device) Interface

- 1 Connect a USB cable from the controller PC's USB port to the "USB DEVICE" port on the back of the oscilloscope.

This is a USB 2.0 high-speed port.

Using the LAN Interface

- 1 If the controller PC is not already connected to the local area network (LAN), do that first.
- 2 Contact your network administrator about adding the oscilloscope to the network.

Find out if automatic configuration via DHCP or AutoIP can be used. Also, find out whether your network supports Dynamic DNS or Multicast DNS.

If automatic configuration is not supported, get the oscilloscope's network parameters (hostname, domain, IP address, subnet mask, gateway IP, DNS IP, etc.).

- 3 Connect the oscilloscope to the local area network (LAN) by inserting LAN cable into the "LAN" port on the back of the oscilloscope.
- 4 Configure the oscilloscope's LAN interface:
 - a Press the **Configure** softkey until "LAN" is selected.
 - b Press the **LAN Settings** softkey.
 - c Press the **Config** softkey, and enable all the configuration options supported by your network.
 - d If automatic configuration is not supported, press the **Addresses** softkey.

Use the **Modify** softkey (and the other softkeys and the Entry knob) to enter the IP Address, Subnet Mask, Gateway IP, and DNS IP values.

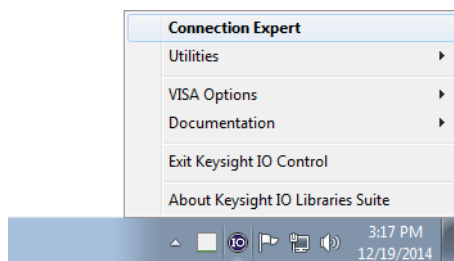
When you are done, press the **[Back up]** key.

- e Press the **Host name** softkey. Use the softkeys and the Entry knob to enter the Host name.

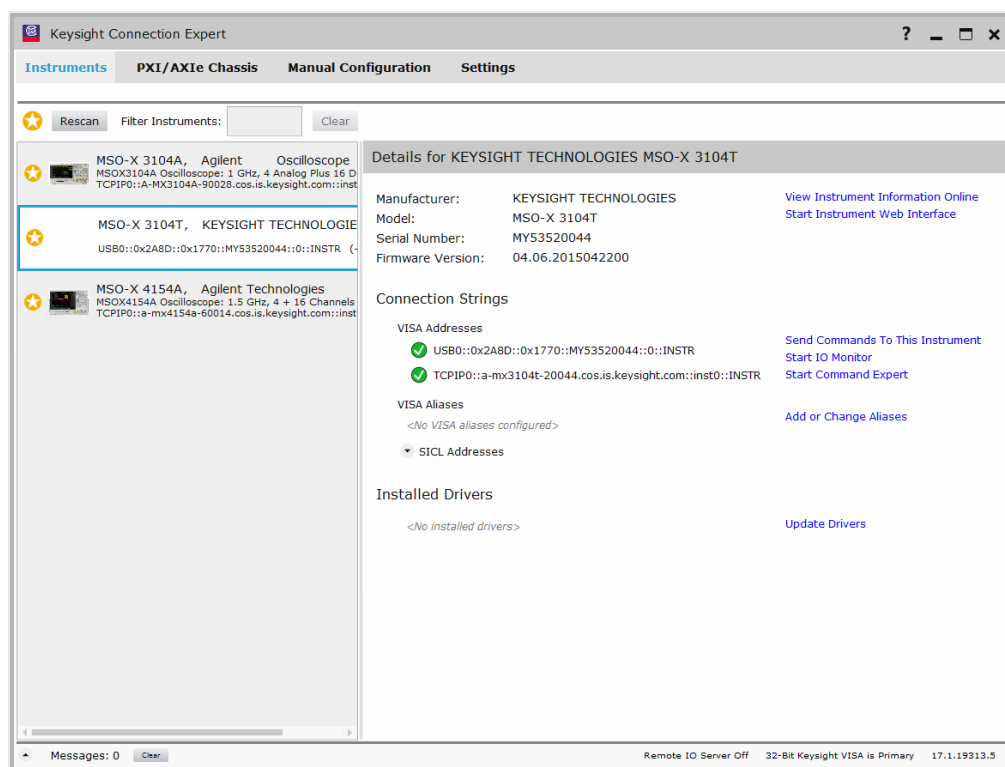
When you are done, press the **[Back up]** key.

Step 3. Verify the oscilloscope connection

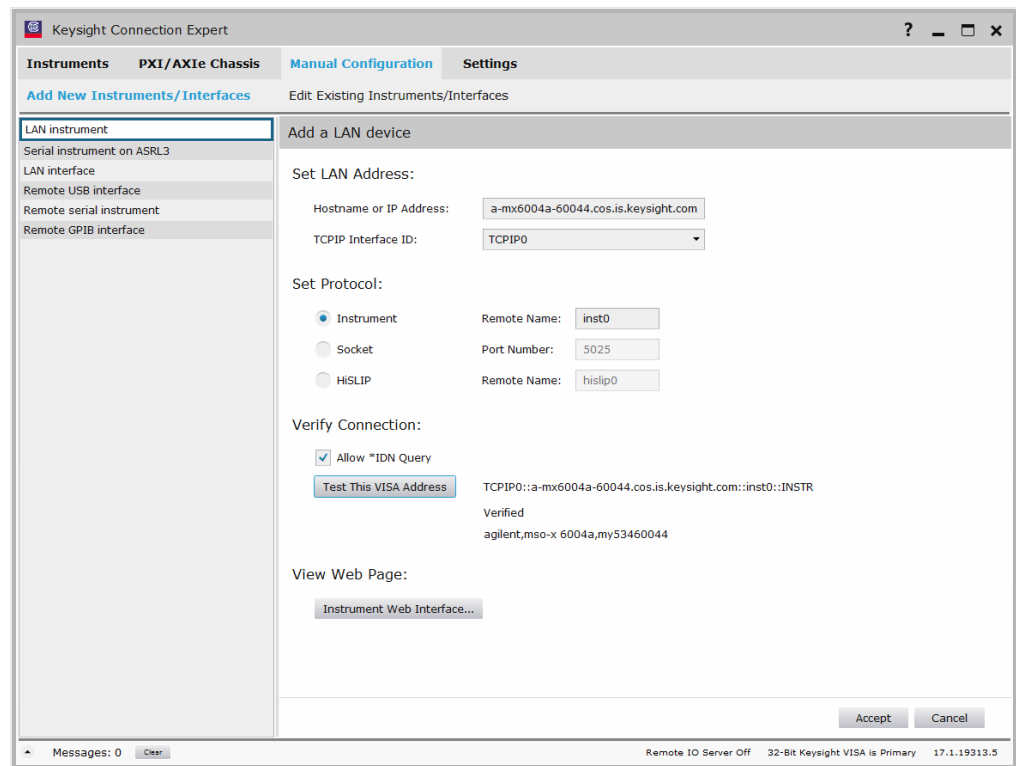
- 1 On the controller PC, click on the Keysight IO Control icon in the taskbar and choose **Connection Expert** from the popup menu.



- 2 In the Keysight Connection Expert application, instruments connected to the controller's USB and GPIB interfaces as well as instruments on the same LAN subnet should automatically appear in the Instruments tab.



- 3 If your instrument does not appear, you can add it using the Manual Configuration tab.

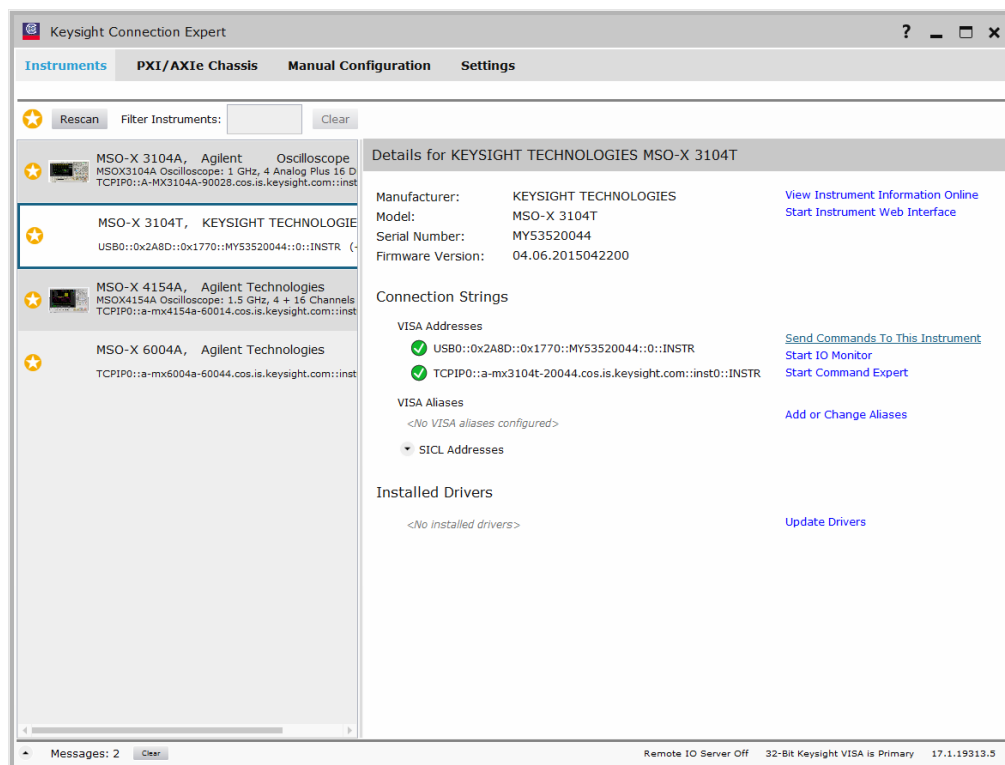


For example, to add a device:

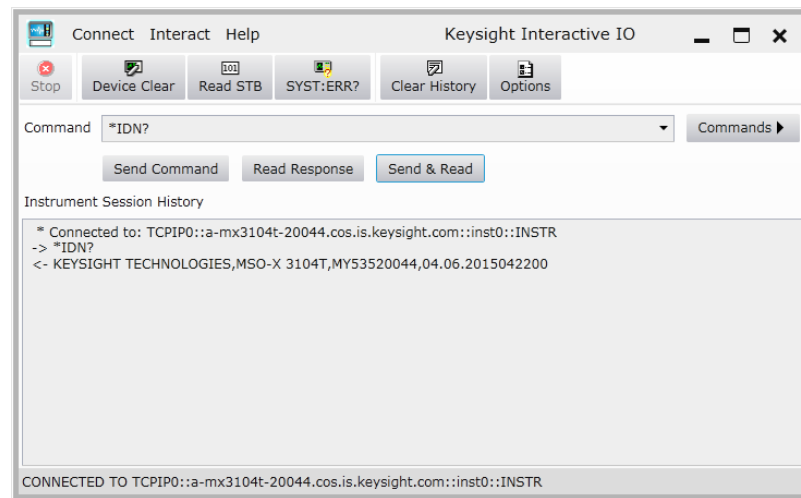
- a** Select **LAN instrument** in the list on the left.
- b** Enter the oscilloscope's **Hostname** or **IP address**.
- c** Select the protocol.
- d** Select **Instrument** under Set Protocol.
- e** Click **Test This VISA Address** to verify the connection.
- f** If the connection test is successful, click **Accept** to add the instrument.

If the connection test is not successful, go back and verify the LAN connections and the oscilloscope setup.

- 4 Test some commands on the instrument:
 - a In the Details for the selected instrument, click **Send Commands To This Instrument**.



- b In the Keysight Interactive IO application, enter commands in the **Command** field and press **Send Command**, **Read Response**, or **Send & Read**.



- c Choose **Connect > Exit** from the menu to exit the Keysight Interactive IO application.
- 5 In the Keysight Connection Expert application, choose **File > Exit** from the menu to exit the application.

3 Getting Started

Basic Oscilloscope Program Structure / 56

Programming the Oscilloscope / 58

Other Ways of Sending Commands / 67

This chapter gives you an overview of programming the 4000G X-Series oscilloscopes. It describes basic oscilloscope program structure and shows how to program the oscilloscope using a few simple examples.

The getting started examples show how to send oscilloscope setup, data capture, and query commands, and they show how to read query results.

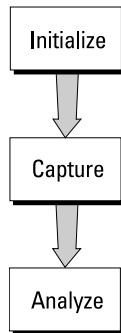
NOTE

Language for Program Examples

The programming examples in this guide are written in Visual Basic using the Keysight VISA COM library.

Basic Oscilloscope Program Structure

The following figure shows the basic structure of every program you will write for the oscilloscope.



Initializing

To ensure consistent, repeatable performance, you need to start the program, controller, and oscilloscope in a known state. Without correct initialization, your program may run correctly in one instance and not in another. This might be due to changes made in configuration by previous program runs or from the front panel of the oscilloscope.

- Program initialization defines and initializes variables, allocates memory, or tests system configuration.
- Controller initialization ensures that the interface to the oscilloscope is properly set up and ready for data transfer.
- Oscilloscope initialization sets the channel configuration, channel labels, threshold voltages, trigger specification, trigger mode, timebase, and acquisition type.

Capturing Data

Once you initialize the oscilloscope, you can begin capturing data for analysis. Remember that while the oscilloscope is responding to commands from the controller, it is not performing acquisitions. Also, when you change the oscilloscope configuration, any data already captured will most likely be rendered.

To collect data, you use the `:DIGitize` command. This command clears the waveform buffers and starts the acquisition process. Acquisition continues until acquisition memory is full, then stops. The acquired data is displayed by the oscilloscope, and the captured data can be measured, stored in acquisition memory in the oscilloscope, or transferred to the controller for further analysis. Any additional commands sent while `:DIGitize` is working are buffered until `:DIGitize` is complete.

You could also put the oscilloscope into run mode, then use a wait loop in your program to ensure that the oscilloscope has completed at least one acquisition before you make a measurement. Keysight does not recommend this because the needed length of the wait loop may vary, causing your program to fail. `:DIGitize`, on the other hand, ensures that data capture is complete. Also, `:DIGitize`, when complete, stops the acquisition process so that all measurements are on displayed data, not on a constantly changing data set.

Analyzing Captured Data

After the oscilloscope has completed an acquisition, you can find out more about the data, either by using the oscilloscope measurements or by transferring the data to the controller for manipulation by your program. Built-in measurements include: frequency, duty cycle, period, positive pulse width, and negative pulse width.

Using the `:WAVEform` commands, you can transfer the data to your controller. You may want to display the data, compare it to a known good measurement, or simply check logic patterns at various time intervals in the acquisition.

Programming the Oscilloscope

- "Referencing the IO Library" on page 58
- "Opening the Oscilloscope Connection via the IO Library" on page 59
- "Using :AUToscale to Automate Oscilloscope Setup" on page 60
- "Using Other Oscilloscope Setup Commands" on page 60
- "Capturing Data with the :DIGitize Command" on page 61
- "Reading Query Responses from the Oscilloscope" on page 63
- "Reading Query Results into String Variables" on page 64
- "Reading Query Results into Numeric Variables" on page 64
- "Reading Definite-Length Block Query Response Data" on page 64
- "Sending Multiple Queries and Reading Results" on page 65
- "Checking Instrument Status" on page 66

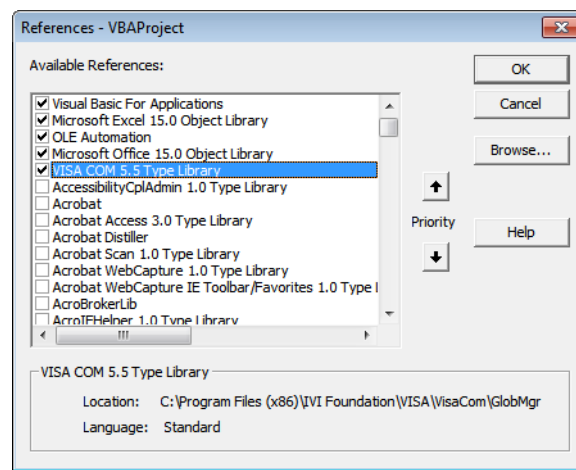
Referencing the IO Library

No matter which instrument programming library you use (SICL, VISA, or VISA COM), you must reference the library from your program.

In C/C++, you must tell the compiler where to find the include and library files (see the Keysight IO Libraries Suite documentation for more information).

To reference the Keysight VISA COM library in Visual Basic for Applications (VBA, which comes with Microsoft Office products like Excel):

- 1 Choose **Tools > References...** from the main menu.
- 2 In the References dialog, check the "VISA COM 5.5 Type Library".



- 3 Click **OK**.

To reference the Keysight VISA COM library in Microsoft Visual Basic 6.0:

- 1 Choose **Project > References...** from the main menu.
- 2 In the References dialog, check the "VISA COM 5.5 Type Library".
- 3 Click **OK**.

Opening the Oscilloscope Connection via the IO Library

PC controllers communicate with the oscilloscope by sending and receiving messages over a remote interface. Once you have opened a connection to the oscilloscope over the remote interface, programming instructions normally appear as ASCII character strings embedded inside write statements of the programming language. Read statements are used to read query responses from the oscilloscope.

For example, when using the Keysight VISA COM library in Visual Basic (after opening the connection to the instrument using the ResourceManager object's Open method), the FormattedIO488 object's WriteString, WriteNumber, WriteList, or WriteIEEEBlock methods are used for sending commands and queries. After a query is sent, the response is read using the ReadString, ReadNumber, ReadList, or ReadIEEEBlock methods.

The following Visual Basic statements open the connection and send a command that turns on the oscilloscope's label display.

```
Dim myMgr As VisaComLib.ResourceManager
Dim myScope As VisaComLib.FormattedIO488

Set myMgr = New VisaComLib.ResourceManager
Set myScope = New VisaComLib.FormattedIO488

' Open the connection to the oscilloscope. Get the VISA Address from the
' Keysight Connection Expert (installed with Keysight IO Libraries Suite
').
Set myScope.IO = myMgr.Open("<VISA Address>")

' Send a command.
myScope.WriteString ":DISPlay:LABel ON"
```

The ":DISPLAY:LABEL ON" in the above example is called a *program message*. Program messages are explained in more detail in **"Program Message Syntax"** on page 1651.

Initializing the Interface and the Oscilloscope

To make sure the bus and all appropriate interfaces are in a known state, begin every program with an initialization statement. When using the Keysight VISA COM library, you can use the resource session object's Clear method to clear the interface buffer:

```

Dim myMgr As VisaComLib.ResourceManager
Dim myScope As VisaComLib.FormattedIO488

Set myMgr = New VisaComLib.ResourceManager
Set myScope = New VisaComLib.FormattedIO488

' Open the connection to the oscilloscope. Get the VISA Address from the
' Keysight Connection Expert (installed with Keysight IO Libraries Suite
').
Set myScope.IO = myMgr.Open("<VISA Address>")

' Clear the interface buffer and set the interface timeout to 10 seconds
.
myScope.IO.Clear
myScope.IO.Timeout = 10000

```

When you are using GPIB, CLEAR also resets the oscilloscope's parser. The parser is the program which reads in the instructions which you send it.

After clearing the interface, initialize the instrument to a preset state:

```
myScope.WriteString "*RST"
```

NOTE

Information for Initializing the Instrument

The actual commands and syntax for initializing the instrument are discussed in [Chapter 6](#), “Common (*) Commands,” starting on page 209.

Refer to the Keysight IO Libraries Suite documentation for information on initializing the interface.

Using :AUToscale to Automate Oscilloscope Setup

The :AUToscale command performs a very useful function for unknown waveforms by setting up the vertical channel, time base, and trigger level of the instrument.

The syntax for the autoscale command is:

```
myScope.WriteString ":AUToscale"
```

Using Other Oscilloscope Setup Commands

A typical oscilloscope setup would set the vertical range and offset voltage, the horizontal range, delay time, delay reference, trigger mode, trigger level, and slope. An example of the commands that might be sent to the oscilloscope are:

```

myScope.WriteString ":CHANnel1:PROBe 10"
myScope.WriteString ":CHANnel1:RANGe 16"
myScope.WriteString ":CHANnel1:OFFSet 1.00"
myScope.WriteString ":TIMEbase:MODE MAIN"
myScope.WriteString ":TIMEbase:RANGe 1E-3"
myScope.WriteString ":TIMEbase:DELay 100E-6"

```


Vertical is set to 16 V full-scale (2 V/div) with center of screen at 1 V and probe attenuation set to 10. This example sets the time base at 1 ms full-scale (100 ms/div) with a delay of 100 μ s.

Example Oscilloscope Setup Code

This program demonstrates the basic command structure used to program the oscilloscope.

```
' Initialize the instrument interface to a known state.
myScope.IO.Clear
myScope.IO.Timeout = 10000    ' Set interface timeout to 10 seconds.

' Initialize the instrument to a preset state.
myScope.WriteString "*RST"

' Set the time base mode to normal with the horizontal time at
' 50 ms/div with 0 s of delay referenced at the center of the
' graticule.
myScope.WriteString ":TIMEbase:RANGe 5E-4"    ' Time base to 50 us/div.
myScope.WriteString ":TIMEbase:DELay 0"        ' Delay to zero.
myScope.WriteString ":TIMEbase:REFerence CENTER" ' Display ref. at
                                                ' center.

' Set the vertical range to 1.6 volts full scale with center screen
' at -0.4 volts with 10:1 probe attenuation and DC coupling.
myScope.WriteString ":CHANnel1:PROBe 10"      ' Probe attenuation
                                                ' to 10:1.
myScope.WriteString ":CHANnel1:RANGe 1.6"     ' Vertical range
                                                ' 1.6 V full scale.
myScope.WriteString ":CHANnel1:OFFSet -0.4"   ' Offset to -0.4.
myScope.WriteString ":CHANnel1:COUPling DC"   ' Coupling to DC.

' Configure the instrument to trigger at -0.4 volts with normal
' triggering.
myScope.WriteString ":TRIGger:SWEep NORMal"   ' Normal triggering.
myScope.WriteString ":TRIGger:LEVel -0.4"     ' Trigger level to -0.4.
myScope.WriteString ":TRIGger:SLOPe POSitive" ' Trigger on pos. slope.

' Configure the instrument for normal acquisition.
myScope.WriteString ":ACQuire:TYPE NORMal"    ' Normal acquisition.
```

Capturing Data with the :DIGitize Command

The :DIGitize command captures data that meets the specifications set up by the :ACQuire subsystem. When the digitize process is complete, the acquisition is stopped. The captured data can then be measured by the instrument or transferred to the controller for further analysis. The captured data consists of two parts: the waveform data record, and the preamble.

NOTE**Ensure New Data is Collected**

When you change the oscilloscope configuration, the waveform buffers are cleared. Before doing a measurement, send the :DIGitize command to the oscilloscope to ensure new data has been collected.

When you send the :DIGitize command to the oscilloscope, the specified channel signal is digitized with the current :ACQuire parameters. To obtain waveform data, you must specify the :WAVEform parameters for the SOURce channel, the FORMat type, and the number of POINTs prior to sending the :WAVEform:DATA? query.

NOTE**Set :TIMebase:MODE to MAIN when using :DIGitize**

:TIMebase:MODE must be set to MAIN to perform a :DIGitize command or to perform any :WAVEform subsystem query. A "Settings conflict" error message will be returned if these commands are executed when MODE is set to ROLL, XY, or WINDow (zoomed). Sending the *RST (reset) command will also set the time base mode to normal.

The number of data points comprising a waveform varies according to the number requested in the :ACQuire subsystem. The :ACQuire subsystem determines the number of data points, type of acquisition, and number of averages used by the :DIGitize command. This allows you to specify exactly what the digitized information contains.

The following program example shows a typical setup:

```
myScope.WriteString ":ACQuire:TYPE AVERage"
myScope.WriteString ":ACQuire:COMPLetE 100"
myScope.WriteString ":ACQuire:COUNt 8"
myScope.WriteString ":DIGitize CHANnel1"
myScope.WriteString ":WAVEform:SOURce CHANnel1"
myScope.WriteString ":WAVEform:FORMat BYTE"
myScope.WriteString ":WAVEform:POINTs 500"
myScope.WriteString ":WAVEform:DATA?"
```

This setup places the instrument into the averaged mode with eight averages. This means that when the :DIGitize command is received, the command will execute until the signal has been averaged at least eight times.

After receiving the :WAVEform:DATA? query, the instrument will start passing the waveform information.

Digitized waveforms are passed from the instrument to the controller by sending a numerical representation of each digitized point. The format of the numerical representation is controlled with the :WAVEform:FORMat command and may be selected as BYTE, WORD, or ASCii.

The easiest method of transferring a digitized waveform depends on data structures, formatting available and I/O capabilities. You must scale the integers to determine the voltage value of each point. These integers are passed starting with the left most point on the instrument's display.

For more information, see the waveform subsystem commands and corresponding program code examples in **Chapter 38**, “:WAVEform Commands,” starting on page 1433.

NOTE

Aborting a Digitize Operation Over the Programming Interface

When using the programming interface, you can abort a digitize operation by sending a Device Clear over the bus (for example, `myScope.IO.Clear`).

Reading Query Responses from the Oscilloscope

After receiving a query (command header followed by a question mark), the instrument interrogates the requested function and places the answer in its output queue. The answer remains in the output queue until it is read or another command is issued. When read, the answer is transmitted across the interface to the designated listener (typically a controller).

The statement for reading a query response message from an instrument's output queue typically has a format specification for handling the response message.

When using the VISA COM library in Visual Basic, you use different read methods (`ReadString`, `ReadNumber`, `ReadList`, or `ReadIEEEBlock`) for the various query response formats. For example, to read the result of the query command `:CHANnel1:COUPling?` you would execute the statements:

```
myScope.WriteString ":CHANnel1:COUPling?"
Dim strQueryResult As String
strQueryResult = myScope.ReadString
```

This reads the current setting for the channel one coupling into the string variable `strQueryResult`.

All results for queries (sent in one program message) must be read before another program message is sent.

Sending another command before reading the result of the query clears the output buffer and the current response. This also causes an error to be placed in the error queue.

Executing a read statement before sending a query causes the controller to wait indefinitely.

The format specification for handling response messages depends on the programming language.

Reading Query Results into String Variables

The output of the instrument may be numeric or character data depending on what is queried. Refer to the specific command descriptions for the formats and types of data returned from queries.

NOTE

Express String Variables Using Exact Syntax

In Visual Basic, string variables are case sensitive and must be expressed exactly the same each time they are used.

The following example shows numeric data being returned to a string variable:

```
myScope.WriteString ":CHANnel1:RANGe?"
Dim strQueryResult As String
strQueryResult = myScope.ReadString
MsgBox "Range (string):" + strQueryResult
```

After running this program, the controller displays:

Range (string): +40.0E+00

Reading Query Results into Numeric Variables

The following example shows numeric data being returned to a numeric variable:

```
myScope.WriteString ":CHANnel1:RANGe?"
Dim varQueryResult As Variant
varQueryResult = myScope.ReadNumber
MsgBox "Range (variant):" + CStr(varQueryResult)
```

After running this program, the controller displays:

Range (variant): 40

Reading Definite-Length Block Query Response Data

Definite-length block query response data allows any type of device-dependent data to be transmitted over the system interface as a series of 8-bit binary data bytes. This is particularly useful for sending large quantities of data or 8-bit extended ASCII codes. The syntax is a pound sign (#) followed by a non-zero digit representing the number of digits in the decimal integer. After the non-zero digit is the decimal integer that states the number of 8-bit data bytes being sent. This is followed by the actual data.

For example, for transmitting 1000 bytes of data, the syntax would be:

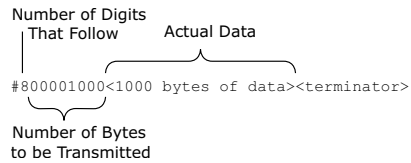


Figure 2 Definite-length block response data

The "8" states the number of digits that follow, and "00001000" states the number of bytes to be transmitted.

The VISA COM library's ReadIEEEBlock and WriteIEEEBlock methods understand the definite-length block syntax, so you can simply use variables that contain the data:

```
' Read oscilloscope setup using ":SYSTem:SETup?" query.
myScope.WriteString ":SYSTem:SETup?"
Dim varQueryResult As Variant
varQueryResult = myScope.ReadIEEEBlock(BinaryType_UI1)

' Write learn string back to oscilloscope using ":SYSTem:SETup" command:
myScope.WriteIEEEBlock ":SYSTem:SETup ", varQueryResult
```

Sending Multiple Queries and Reading Results

You can send multiple queries to the instrument within a single command string, but you must also read them back as a single query result. This can be accomplished by reading them back into a single string variable, multiple string variables, or multiple numeric variables.

For example, to read the :TIMEbase:RANGe?;DELaY? query result into a single string variable, you could use the commands:

```
myScope.WriteString ":TIMEbase:RANGe?;DELaY?"
Dim strQueryResult As String
strQueryResult = myScope.ReadString
MsgBox "Timebase range; delay:" + strQueryResult
```

When you read the result of multiple queries into a single string variable, each response is separated by a semicolon. For example, the output of the previous example would be:

```
Timebase range; delay: <range_value>;<delay_value>
```

To read the :TIMEbase:RANGe?;DELaY? query result into multiple string variables, you could use the ReadList method to read the query results into a string array variable using the commands:

```
myScope.WriteString ":TIMEbase:RANGe?;DELaY?"
Dim strResults() As String
strResults() = myScope.ReadList(ASCIIType_BSTR)
MsgBox "Timebase range: " + strResults(0) + ", delay: " + strResults(1)
```

To read the :TIMEbase:RANGe?;DELaY? query result into multiple numeric variables, you could use the ReadList method to read the query results into a variant array variable using the commands:

```
myScope.WriteString ":TIMEbase:RANGe?;DELaY?"
Dim varResults() As Variant
varResults() = myScope.ReadList
MsgBox "Timebase range: " + FormatNumber(varResults(0) * 1000, 4) + _
      " ms, delay: " + FormatNumber(varResults(1) * 1000000, 4) + " us"
```

Checking Instrument Status

Status registers track the current status of the instrument. By checking the instrument status, you can find out whether an operation has been completed, whether the instrument is receiving triggers, and more.

For more information, see **Chapter 43**, “Status Reporting,” starting on page 1597 which explains how to check the status of the instrument.

Other Ways of Sending Commands

Standard Commands for Programmable Instrumentation (SCPI) can also be sent via a Telnet socket or through the Browser Web Control:

- **"Telnet Sockets"** on page 67
- **"Sending SCPI Commands Using Browser Web Control"** on page 67

Telnet Sockets

The following information is provided for programmers who wish to control the oscilloscope with SCPI commands in a Telnet session.

To connect to the oscilloscope via a telnet socket, issue the following command:

```
telnet <hostname> 5024
```

where <hostname> is the hostname of the oscilloscope. This will give you a command line with prompt.

For a command line without a prompt, use port 5025. For example:

```
telnet <hostname> 5025
```

Sending SCPI Commands Using Browser Web Control

To send SCPI commands using the Browser Web Control feature, establish a connection to the oscilloscope via LAN as described in the *InfiniiVision 4000G X-Series Oscilloscopes User's Guide*. When you make the connection to the oscilloscope via LAN and the instrument's welcome page is displayed, select the **Browser Web Control** tab, then select the **Remote Programming** link.

4 Sequential (Blocking) vs. Overlapped Commands

IEEE 488.2 makes the distinction between sequential and overlapped commands (and queries):

- *Sequential commands* also known as *blocking commands*, finish their task before the execution of the next command starts.

These oscilloscope commands and queries are sequential (blocking):

- :AUToscale
- :DIGitize
- :WMEMemory<r>:SAVE <source>
- :WAVEform:DATA?
- *Overlapped commands* run concurrently. Commands following an overlapped command may be started before the overlapped command is completed.

Some oscilloscope commands are overlapped. For example, the oscilloscope's save and recall commands are overlapped as well as some commands that perform analysis.

With sequential (blocking) commands and queries, the oscilloscope is expected to stop processing inputs, including additional remote commands and queries as well as front panel knobs, until completed.

Pausing Execution Between Overlapped Commands

With overlapped commands, you can use the *OPC? query to prevent any more commands from being executed until the overlapped command is complete. This may be necessary when a command that follows an overlapped command interferes with the overlapped command's processing or analysis. For example:

```
:RECall:SETup "setup.scp";*OPC?;:RECall:ARbitrary "arb_wfm.csv"
```

You can also use the *ESR? query to look at the OPC bit (bit 0) in the Standard Event Status Register to determine when an operation is complete.

Pausing Programs Until Sequential (Blocking) Commands are Complete

Sequential (blocking) commands do not prevent additional commands from being sent to the queue or cause the remote program to wait. For example, if your program does something like:

```
myScope.WriteString ":DIGitize"
Debug.Print "Signal acquired."
```

The "Signal acquired" message will be written immediately after the :DIGitize is sent, not after the acquisition and processing is complete.

To pause the program until a sequential (blocking) command is complete, you must wait for a query result after the sequential (blocking) command. For example, in this case:

```
Public strQueryResult As String
myScope.WriteString ":DIGitize;*OPC?"
strQueryResult = myScope.ReadString
Debug.Print "Signal acquired."
```

The "Signal acquired" message will be written after the acquisition and processing is complete. The *OPC? query is appended to :DIGitize with a semi-colon (;), which essentially ties it to the same thread in the parser. It is immediately dealt with once :DIGitize finishes and gives a "1" back to the program (whether the program uses it or not), allowing the program to move on.

When using a query to wait until a sequential (blocking) command is complete, it is possible for the sequential (blocking) command execution to take longer than the I/O timeout, in which case, there will be a timeout error while waiting for the query results. You can increase the I/O timeout or have your program poll the Status Byte Register using the IO libraries' unblocked ReadSTB function to wait for execution completion.

Using Device Clear to Abort a Sequential (Blocking) Command

When sequential (blocking) commands take too long or fail to complete for some reason, you can send a Device Clear over the bus to clear the input buffer and output queue, reset the parser, and clear any pending commands.

- See Also
- **"*OPC (Operation Complete)"** on page 222
 - **"*ESR (Standard Event Status Register)"** on page 218
 - **Chapter 44**, "Synchronizing Acquisitions," starting on page 1629

5 Commands Quick Reference

Command Summary / 72

Syntax Elements / 206

Command Summary

- Common (*) Commands Summary (see [page 74](#))
- Root (:) Commands Summary (see [page 78](#))
- :ACQuire Commands Summary (see [page 82](#))
- :BUS<n> Commands Summary (see [page 84](#))
- :CALibrate Commands Summary (see [page 85](#))
- :CHANnel<n> Commands Summary (see [page 86](#))
- :COMPliance Commands Summary (see [page 89](#))
- :COUNter Commands Summary (see [page 89](#))
- :DEMO Commands Summary (see [page 90](#))
- :DIGital<n> Commands Summary (see [page 90](#))
- :DISPlay Commands Summary (see [page 91](#))
- :DVM Commands Summary (see [page 93](#))
- :EXternal Trigger Commands Summary (see [page 94](#))
- :FRANalysis Commands Summary (see [page 94](#))
- :FUNCTion Commands Summary (see [page 96](#))
- :HARDcopy Commands Summary (see [page 102](#))
- :HISTogram Commands Summary (see [page 103](#))
- :LISTer Commands Summary (see [page 104](#))
- :LTEST Commands Summary (see [page 105](#))
- :MARKer Commands Summary (see [page 105](#))
- :MEASure Commands Summary (see [page 108](#))
- :MEASure Power Commands Summary (see [page 129](#))
- :MTEST Commands Summary (see [page 133](#))
- :POD<n> Commands Summary (see [page 135](#))
- :POWER Commands Summary (see [page 136](#))
- :RECall Commands Summary (see [page 144](#))
- :SAVE Commands Summary (see [page 145](#))
- General :SBUS<n> Commands Summary (see [page 148](#))
- :SBUS<n>:A429 Commands Summary (see [page 148](#))
- :SBUS<n>:CAN Commands Summary (see [page 150](#))
- :SBUS<n>:CXPI Commands Summary (see [page 152](#))
- :SBUS<n>:FLEXray Commands Summary (see [page 154](#))
- :SBUS<n>:I2S Commands Summary (see [page 155](#))

- :SBUS<n>:IIC Commands Summary (see [page 158](#))
- :SBUS<n>:LIN Commands Summary (see [page 159](#))
- :SBUS<n>:M1553 Commands Summary (see [page 160](#))
- :SBUS<n>:MANChester Commands Summary (see [page 161](#))
- :SBUS<n>:NRZ Commands Summary (see [page 162](#))
- :SBUS<n>:SENT Commands Summary (see [page 164](#))
- :SBUS<n>:SPI Commands Summary (see [page 166](#))
- :SBUS<n>:UART Commands Summary (see [page 168](#))
- :SBUS<n>:USB Commands Summary (see [page 170](#))
- :SBUS<n>:USBPd Commands Summary (see [page 172](#))
- General :SEARCh Commands Summary (see [page 173](#))
- :SEARCh:EDGE Commands Summary (see [page 174](#))
- :SEARCh:GLITCh Commands Summary (see [page 174](#))
- :SEARCh:PEAK Commands Summary (see [page 175](#))
- :SEARCh:RUNT Commands Summary (see [page 175](#))
- :SEARCh:TRANSition Commands Summary (see [page 176](#))
- :SEARCh:SERIal:A429 Commands Summary (see [page 176](#))
- :SEARCh:SERIal:CAN Commands Summary (see [page 177](#))
- :SEARCh:SERIal:FLEXray Commands Summary (see [page 177](#))
- :SEARCh:SERIal:I2S Commands Summary (see [page 178](#))
- :SEARCh:SERIal:IIC Commands Summary (see [page 179](#))
- :SEARCh:SERIal:LIN Commands Summary (see [page 179](#))
- :SEARCh:SERIal:M1553 Commands Summary (see [page 180](#))
- :SEARCh:SERIal:SENT Commands Summary (see [page 181](#))
- :SEARCh:SERIal:SPI Commands Summary (see [page 181](#))
- :SEARCh:SERIal:UART Commands Summary (see [page 182](#))
- :SEARCh:SERIal:USB Commands Summary (see [page 182](#))
- :SYSTem Commands Summary (see [page 184](#))
- :TIMebase Commands Summary (see [page 186](#))
- General :TRIGGer Commands Summary (see [page 187](#))
- :TRIGGer:DELay Commands Summary (see [page 188](#))
- :TRIGGer:EBURst Commands Summary (see [page 189](#))
- :TRIGGer[:EDGE] Commands Summary (see [page 189](#))
- :TRIGGer:GLITCh Commands Summary (see [page 190](#))
- :TRIGGer:OR Commands Summary (see [page 192](#))

- :TRIGger:PATtern Commands Summary (see [page 192](#))
- :TRIGger:RUNT Commands Summary (see [page 193](#))
- :TRIGger:SHOLd Commands Summary (see [page 193](#))
- :TRIGger:TRANsition Commands Summary (see [page 194](#))
- :TRIGger:TV Commands Summary (see [page 195](#))
- :TRIGger:ZONE Commands Summary (see [page 196](#))
- :WAVEform Commands Summary (see [page 196](#))
- :WGEN Commands Summary (see [page 200](#))
- :WMEMory<r> Commands Summary (see [page 204](#))

Table 2 Common (*) Commands Summary

Command	Query	Options and Query Returns
*CLS (see page 215)	n/a	n/a
*ESE <mask> (see page 216)	*ESE? (see page 216)	<mask> ::= 0 to 255; an integer in NR1 format: Bit Weight Name Enables --- 7 128 PON Power On 6 64 URQ User Request 5 32 CME Command Error 4 16 EXE Execution Error 3 8 DDE Dev. Dependent Error 2 4 QYE Query Error 1 2 RQL Request Control 0 1 OPC Operation Complete
n/a	*ESR? (see page 218)	<status> ::= 0 to 255; an integer in NR1 format
n/a	*IDN? (see page 218)	KEYSIGHT TECHNOLOGIES,<model>,<serial number>,X.XX.XX <model> ::= the model number of the instrument <serial number> ::= the serial number of the instrument <X.XX.XX> ::= the software revision of the instrument
n/a	*LRN? (see page 221)	<learn_string> ::= current instrument setup as a block of data in IEEE 488.2 # format
*OPC (see page 222)	*OPC? (see page 222)	ASCII "1" is placed in the output queue when all pending device operations have completed.

Table 2 Common (*) Commands Summary (continued)

Command	Query	Options and Query Returns
n/a	*OPT? (see page 223)	<pre> <return_value> ::= 0,0,<license info> <license info> ::= <All field>, <reserved>, <reserved>, <MSO>, <reserved>, <Memory>, <Low Speed Serial>, <Automotive Serial>, <FlexRay Serial>, <Frequency Response Analysis>, <Power Measurements>, <RS-232/UART Serial>, <Segmented Memory>, <Mask Test>, <reserved>, <Bandwidth>, <reserved>, <reserved>, <reserved>, <reserved>, <reserved>, <reserved>, <I2S Serial>, <reserved>, <Educator's Kit>, <Waveform Generator>, <MIL-1553/ARINC 429 Serial>, <Extended Video>, <Advanced Math>, <reserved>, <reserved>, <reserved>, <Digital Voltmeter>, <reserved>, <Spectrum Visualizer>, <reserved>, <USB 2.0 Low/Full Speed>, <USB 2.0 High Speed>, <USB 2.0 Signal Quality Analysis >, <Remote Command Logging>, <reserved>, <SENT Serial>, <CAN FD Serial>, <CXPI Serial>, <reserved>, <reserved>, <reserved>, <reserved>, <Manchester/NRZ Serial>, <USB PD Serial>, <reserved>, <Automotive Software>, <General Purpose Software>, <Aerospace Software>, <Power Supply Test Software>, <USB Test Software>, <reserved>, <Software Bundle> </pre>

Table 2 Common (*) Commands Summary (continued)

Command	Query	Options and Query Returns
n/a	*OPT? (see page 223) (cont'd)	<All field> ::= {0 All} <reserved> ::= 0 <MSO> ::= {0 MSO} <Memory> ::= {0 MEMUP} <Low Speed Serial> ::= {0 EMBD} <Automotive Serial> ::= {0 AUTO} <FlexRay Serial> ::= {0 FLEX} <Frequency Response Analysis> ::= {0 FRA} <Power Measurements> ::= {0 PWR} <RS-232/UART Serial> ::= {0 COMP} <Mask Test> ::= {0 MASK} <Bandwidth> ::= {0 BW20 BW50} <I2S Serial> ::= {0 AUDIO} <Educator's Kit> ::= {0 EDK} <Waveform Generator> ::= {0 WAVEGEN} <MIL-1553/ARINC 429 Serial> ::= {0 AERO} <Extended Video> ::= {0 VID} <Digital Voltmeter> ::= {0 DVM} <Spectrum Visualizer> ::= {0 ASV} <USB 2.0 Low/Full Speed> ::= {0 USF} <USB 2.0 High Speed> ::= {0 U2H} <USB 2.0 Signal Quality Analysis> ::= {0 USBSQ} <Remote Command Logging> ::= {0 RML}

Table 2 Common (*) Commands Summary (continued)

Command	Query	Options and Query Returns
n/a	*OPT? (see page 223) (cont'd)	<SENT Serial> ::= {0 SENSOR} <CAN FD Serial> ::= {0 CANFD} <CXPI Serial> ::= {0 CXPI} <Manchester/NRZ Serial> ::= {0 NRZ} <USB PD Serial> ::= {0 USBPD} <Automotive Software> ::= {0 D4000AUTA} <General Purpose Software> ::= {0 D4000GENA} <Aerospace Software> ::= {0 D4000AERA} <Power Supply Test Software> ::= {0 D4000PWRA} <USB Test Software> ::= {0 D4000USBA} <Near Field Communications (NFC) Software> ::= {0 D4000NFCA} <Software Bundle> ::= {0 D4000BDLA}
*RCL <value> (see page 225)	n/a	<value> ::= {0 1 4 5 6 7 8 9}
*RST (see page 226)	n/a	See *RST (Reset) (see page 226)
*SAV <value> (see page 229)	n/a	<value> ::= {0 1 4 5 6 7 8 9}
*SRE <mask> (see page 230)	*SRE? (see page 231)	<mask> ::= sum of all bits that are set, 0 to 255; an integer in NR1 format. <mask> ::= following values: Bit Weight Name Enables --- 7 128 OPER Operation Status Reg 6 64 ---- (Not used.) 5 32 ESB Event Status Bit 4 16 MAV Message Available 3 8 ---- (Not used.) 2 4 MSG Message 1 2 USR User 0 1 TRG Trigger

Table 2 Common (*) Commands Summary (continued)

Command	Query	Options and Query Returns
n/a	*STB? (see page 232)	<value> ::= 0 to 255; an integer in NR1 format, as shown in the following: Bit Weight Name "1" Indicates <pre> ----- 7 128 OPER Operation status condition occurred. 6 64 RQS/ Instrument is requesting service. 5 32 ESB Enabled event status condition occurred. 4 16 MAV Message available. 3 8 ---- (Not used.) 2 4 MSG Message displayed. 1 2 USR User event condition occurred. 0 1 TRG A trigger occurred. </pre>
*TRG (see page 234)	n/a	n/a
n/a	*TST? (see page 235)	<result> ::= 0 or non-zero value; an integer in NR1 format
*WAI (see page 236)	n/a	n/a

Table 3 Root (:) Commands Summary

Command	Query	Options and Query Returns
:ACTivity (see page 241)	:ACTivity? (see page 241)	<return value> ::= <edges>,<levels> <edges> ::= presence of edges (32-bit integer in NR1 format) <levels> ::= logical highs or lows (32-bit integer in NR1 format)
n/a	:AER? (see page 242)	{0 1}; an integer in NR1 format

Table 3 Root (:): Commands Summary (continued)

Command	Query	Options and Query Returns
:AUToscale [<source>[,...,<source>]] (see page 243)	n/a	<source> ::= CHANNEL<n> for DSO models <source> ::= {CHANNEL<n> DIGital<d> POD1 POD2} for MSO models <source> can be repeated up to 5 times <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:AUToscale:AMODE <value> (see page 245)	:AUToscale:AMODE? (see page 245)	<value> ::= {NORMAL CURRENT}}
:AUToscale:CHANnels <value> (see page 246)	:AUToscale:CHANnels? (see page 246)	<value> ::= {ALL DISPlayed}}
:AUToscale:FDEBug { {0 OFF} {1 ON} } (see page 247)	:AUToscale:FDEBug? (see page 247)	{0 1}
:BLANk [<source>] (see page 248)	n/a	<source> ::= {CHANNEL<n> FUNCTION<m> MATH<m> SBUS{1 2} WMEMory<r>} for DSO models <source> ::= {CHANNEL<n> DIGital<d> POD{1 2} BUS{1 2} FUNCTION<m> MATH<m> SBUS{1 2} WMEMory<r>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format

Table 3 Root (:) Commands Summary (continued)

Command	Query	Options and Query Returns
:DIGitize [<source>[,...,<source>]] (see page 250)	n/a	<source> ::= {CHANnel<n> FUNCTION<m> MATH<m> SBUS{1 2}} for DSO models <source> ::= {CHANnel<n> DIGital<d> POD{1 2} BUS{1 2} FUNCTION<m> MATH<m> SBUS{1 2}} for MSO models <source> can be repeated up to 5 times <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <m> ::= 1 to (# math functions) in NR1 format
:HWEenable <n> (see page 252)	:HWEenable? (see page 252)	<n> ::= 16-bit integer in NR1 format
n/a	:HWERegister:CONDition? (see page 254)	<n> ::= 16-bit integer in NR1 format
n/a	:HWERegister[:EVENT]? (see page 255)	<n> ::= 16-bit integer in NR1 format
:MTEenable <n> (see page 256)	:MTEenable? (see page 256)	<n> ::= 16-bit integer in NR1 format
n/a	:MTERegister[:EVENT]? (see page 258)	<n> ::= 16-bit integer in NR1 format
:OPEE <n> (see page 260)	:OPEE? (see page 261)	<n> ::= 15-bit integer in NR1 format
n/a	:OPERRegister:CONDition? (see page 262)	<n> ::= 15-bit integer in NR1 format
n/a	:OPERRegister[:EVENT]? (see page 265)	<n> ::= 15-bit integer in NR1 format

Table 3 Root (:): Commands Summary (continued)

Command	Query	Options and Query Returns
:OVLenable <mask> (see page 268)	:OVLenable? (see page 269)	<mask> ::= 16-bit integer in NR1 format as shown: Bit Weight Input <pre> ----- 10 1024 Ext Trigger Fault 9 512 Channel 4 Fault 8 256 Channel 3 Fault 7 128 Channel 2 Fault 6 64 Channel 1 Fault 4 16 Ext Trigger OVL 3 8 Channel 4 OVL 2 4 Channel 3 OVL 1 2 Channel 2 OVL 0 1 Channel 1 OVL </pre>
n/a	:OVLRegister? (see page 270)	<value> ::= integer in NR1 format. See OVLenable for <value>
:PRINT [<options>] (see page 272)	n/a	<options> ::= [<print option>][,...,<print option>] <print option> ::= {COLor GRAYscale PRINter0 BMP8bit BMP PNG NOFactoRs FACToRs} <print option> can be repeated up to 5 times.
n/a	:RSTate? (see page 273)	n/a
:RUN (see page 274)	n/a	n/a
n/a	:SERial (see page 275)	<return value> ::= unquoted string containing serial number
:SINGle (see page 276)	n/a	n/a
n/a	:STATus? <display> (see page 277)	{0 1} <display> ::= {CHANnel<n> DIGital<d> POD{1 2} BUS{1 2} FUNCTION<m> MATH<m> SBUS{1 2} WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format

Table 3 Root (:): Commands Summary (continued)

Command	Query	Options and Query Returns
:STOP (see page 278)	n/a	n/a
n/a	:TER? (see page 279)	{0 1}
:VIEW <source> (see page 280)	n/a	<source> ::= {CHANnel<n> FUNCTION<m> MATH<m> SBUS{1 2} WMEMory<r>} for DSO models <source> ::= {CHANnel<n> DIGital<d> POD{1 2} BUS{1 2} FUNCTION<m> MATH<m> SBUS{1 2} WMEMory<r>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format

Table 4 :ACquire Commands Summary

Command	Query	Options and Query Returns
n/a	:ACquire:AALias? (see page 284)	{1 0}
:ACquire:COMplete <complete> (see page 285)	:ACquire:COMplete? (see page 285)	<complete> ::= 100; an integer in NR1 format
:ACquire:COUNt <count> (see page 286)	:ACquire:COUNt? (see page 286)	<count> ::= an integer from 2 to 65536 in NR1 format
:ACquire:DAALias <mode> (see page 287)	:ACquire:DAALias? (see page 287)	<mode> ::= {DISable AUTO}
:ACquire:DIGitizer {{0 OFF} {1 ON}} (see page 288)	:ACquire:DIGitizer? (see page 288)	{0 1}
:ACquire:MODE <mode> (see page 289)	:ACquire:MODE? (see page 289)	<mode> ::= {RTIME ETIME SEGmented}

Table 4 :ACQUIRE Commands Summary (continued)

Command	Query	Options and Query Returns
:ACQUIRE:POINTS[:ANALog] <points> (see page 290)	:ACQUIRE:POINTS[:ANALog]? (see page 290)	<points> ::= {AUTO <points_value>} <points_value> ::= desired analog memory depth in integer NR1 format
:ACQUIRE:POINTS[:ANALog]:AUTO {{0 OFF} {1 ON}} (see page 291)	:ACQUIRE:POINTS[:ANALog]:AUTO? (see page 291)	{0 1}
:ACQUIRE:RSIGNAL <ref_signal_mode> (see page 292)	:ACQUIRE:RSIGNAL? (see page 292)	<ref_signal_mode> ::= {OFF OUT IN}
:ACQUIRE:SEGMENTed:ANALyze (see page 293)	n/a	n/a
:ACQUIRE:SEGMENTed:COUNT <count> (see page 294)	:ACQUIRE:SEGMENTed:COUNT? (see page 294)	<count> ::= an integer from 2 to 1000 in NR1 format
:ACQUIRE:SEGMENTed:INDEX <index> (see page 295)	:ACQUIRE:SEGMENTed:INDEX? (see page 295)	<index> ::= an integer from 1 to 1000 in NR1 format
:ACQUIRE:SRATE[:ANALog] <rate> (see page 298)	:ACQUIRE:SRATE[:ANALog]? (see page 298)	<rate> ::= {AUTO <sample_rate>} <sample_rate> ::= desired analog sample rate in NR3 format
:ACQUIRE:SRATE[:ANALog]:AUTO {{0 OFF} {1 ON}} (see page 299)	:ACQUIRE:SRATE[:ANALog]:AUTO? (see page 299)	{0 1}
:ACQUIRE:TYPE <type> (see page 300)	:ACQUIRE:TYPE? (see page 300)	<type> ::= {NORMAL AVERAGE HRESolution PEAK}

Table 5 :BUS<n> Commands Summary

Command	Query	Options and Query Returns
:BUS<n>:BIT<m> {{0 OFF} {1 ON}} (see page 305)	:BUS<n>:BIT<m>? (see page 305)	{0 1} <n> ::= 1 or 2; an integer in NR1 format <m> ::= 0-15; an integer in NR1 format
:BUS<n>:BITS <channel_list>, {{0 OFF} {1 ON}} (see page 306)	:BUS<n>:BITS? (see page 306)	<channel_list>, {0 1} <channel_list> ::= (@<m>,<m>:<m> ...) where "," is separator and ":" is range <n> ::= 1 or 2; an integer in NR1 format <m> ::= 0-15; an integer in NR1 format
:BUS<n>:CLear (see page 308)	n/a	<n> ::= 1 or 2; an integer in NR1 format
:BUS<n>:DISPlay {{0 OFF} {1 ON}} (see page 309)	:BUS<n>:DISPlay? (see page 309)	{0 1} <n> ::= 1 or 2; an integer in NR1 format
:BUS<n>:LABel <string> (see page 310)	:BUS<n>:LABel? (see page 310)	<string> ::= quoted ASCII string up to 10 characters <n> ::= 1 or 2; an integer in NR1 format
:BUS<n>:MASK <mask> (see page 311)	:BUS<n>:MASK? (see page 311)	<mask> ::= 32-bit integer in decimal, <nondecimal>, or <string> <nondecimal> ::= #Hnn...n where n ::= {0,...,9 A,...,F} for hexadecimal <nondecimal> ::= #Bnn...n where n ::= {0 1} for binary <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F} for hexadecimal <n> ::= 1 or 2; an integer in NR1 format

Table 6 :CALibrate Commands Summary

Command	Query	Options and Query Returns
n/a	:CALibrate:DATE? (see page 315)	<return value> ::= <year>,<month>,<day>; all in NR1 format
:CALibrate:LABel <string> (see page 316)	:CALibrate:LABel? (see page 316)	<string> ::= quoted ASCII string up to 32 characters
:CALibrate:OUTPut <signal> (see page 317)	:CALibrate:OUTPut? (see page 317)	<signal> ::= {TRIGgers MASK WAVEgen WGEN1 WGEN2 TSource} Note: WAVE and WGEN1 are equivalent. Note: WGEN2 only available on models with 2 WaveGen outputs.
n/a	:CALibrate:PROTected? (see page 319)	{"PROTected" "UNPROtected"}
:CALibrate:STARt (see page 320)	n/a	n/a
n/a	:CALibrate:STATus? (see page 321)	<return value> ::= <status_code>,<status_string> <status_code> ::= an integer status code <status_string> ::= an ASCII status string
n/a	:CALibrate:TEMPeratur e? (see page 322)	<return value> ::= degrees C delta since last cal in NR3 format
n/a	:CALibrate:TIME? (see page 323)	<return value> ::= <hours>,<minutes>,<seconds>; all in NR1 format

Table 7 :CHANnel<n> Commands Summary

Command	Query	Options and Query Returns
:CHANnel<n>:BANDwidth <limit> (see page 329)	:CHANnel<n>:BANDwidth ? [MAXimum] (see page 329)	<limit> ::= {20E6 <full_bw>} in NR3 format <n> ::= 1 to (# analog channels) in NR1 format
:CHANnel<n>:BWLimit { {0 OFF} {1 ON} } (see page 330)	:CHANnel<n>:BWLimit? (see page 330)	{0 1} <n> ::= 1 to (# analog channels) in NR1 format
:CHANnel<n>:COUpling <coupling> (see page 331)	:CHANnel<n>:COUpling? (see page 331)	<coupling> ::= {AC DC} <n> ::= 1 to (# analog channels) in NR1 format
:CHANnel<n>:DISPlay { {0 OFF} {1 ON} } (see page 332)	:CHANnel<n>:DISPlay? (see page 332)	{0 1} <n> ::= 1 to (# analog channels) in NR1 format
:CHANnel<n>:IMPedance <impedance> (see page 333)	:CHANnel<n>:IMPedance ? (see page 333)	<impedance> ::= {ONEMeg FIFTy} <n> ::= 1 to (# analog channels) in NR1 format
:CHANnel<n>:INVert { {0 OFF} {1 ON} } (see page 334)	:CHANnel<n>:INVert? (see page 334)	{0 1} <n> ::= 1 to (# analog channels) in NR1 format
:CHANnel<n>:LABel <string> (see page 335)	:CHANnel<n>:LABel? (see page 335)	<string> ::= any series of 32 or less ASCII characters enclosed in quotation marks <n> ::= 1 to (# analog channels) in NR1 format
:CHANnel<n>:OFFSet <offset>[suffix] (see page 336)	:CHANnel<n>:OFFSet? (see page 336)	<offset> ::= Vertical offset value in NR3 format [suffix] ::= {V mV} <n> ::= 1-2 or 1-4; in NR1 format
:CHANnel<n>:PROBe <attenuation> (see page 337)	:CHANnel<n>:PROBe? (see page 337)	<attenuation> ::= Probe attenuation ratio in NR3 format <n> ::= 1-2 or 1-4; in NR1 format
:CHANnel<n>:PROBe:BTN <setting> (see page 338)	:CHANnel<n>:PROBe:BTN ? (see page 338)	<n> ::= 1 to (# analog channels) in NR1 format <setting> ::= {HEADlight INFinimode RSTop SINGLE CDISplay AUToscale FTRigger QACtion NACtion NONE}

Table 7 :CHANnel<n> Commands Summary (continued)

Command	Query	Options and Query Returns
:CHANnel<n>:PROBe:CALibration (see page 339)	n/a	n/a
:CHANnel<n>:PROBe:EXTernal {{0 OFF} {1 ON}} (see page 340)	:CHANnel<n>:PROBe:EXTernal? (see page 340)	<n> ::= 1 to (# analog channels) in NR1 format <setting> ::= {0 1}
:CHANnel<n>:PROBe:EXTernal:GAIN <gain_factor> (see page 341)	:CHANnel<n>:PROBe:EXTernal:GAIN? (see page 341)	<n> ::= 1 to (# analog channels) in NR1 format <gain_factor> ::= a real number from 0.0001 to 1000 in NR3 format
:CHANnel<n>:PROBe:EXTernal:OFFSet <offset> (see page 342)	:CHANnel<n>:PROBe:EXTernal:OFFSet? (see page 342)	<n> ::= 1 to (# analog channels) in NR1 format <offset> ::= a real number for the offset value at center screen in NR3 format
:CHANnel<n>:PROBe:EXTernal:UNITs <units> (see page 343)	:CHANnel<n>:PROBe:EXTernal:UNITs? (see page 343)	<n> ::= 1 to (# analog channels) in NR1 format <units> ::= {VOLT AMPere}
:CHANnel<n>:PROBe:HEAD[:TYPE] <head_param> (see page 344)	:CHANnel<n>:PROBe:HEAD[:TYPE]? (see page 344)	<head_param> ::= {SEND0 SEND6 SEND12 SEND20 DIFF0 DIFF6 DIFF12 DIFF20 DSMA DSMA6 NONE} <n> ::= 1 to (# analog channels) in NR1 format
n/a	:CHANnel<n>:PROBe:ID? (see page 345)	<probe id> ::= unquoted ASCII string up to 11 characters <n> ::= 1 to (# analog channels) in NR1 format
:CHANnel<n>:PROBe:MMODel <value> (see page 346)	:CHANnel<n>:PROBe:MMODel? (see page 346)	<value> ::= {P5205 P5210 P6205 P6241 P6243 P6245 P6246 P6247 P6248 P6249 P6250 P6251 P670X P671X TCP202} <n> ::= 1 to (# analog channels) in NR1 format
:CHANnel<n>:PROBe:MODE <setting> (see page 347)	:CHANnel<n>:PROBe:MODE? (see page 347)	<n> ::= 1 to (# analog channels) in NR1 format <setting> ::= {DIFFerential DOFFset SEA SEB CM}

Table 7 :CHANnel<n> Commands Summary (continued)

Command	Query	Options and Query Returns
:CHANnel<n>:PROBe:RSE Nse <value> (see page 348)	:CHANnel<n>:PROBe:RSE Nse? (see page 348)	<value> ::= Ohms in NR3 format <n> ::= 1 to (# analog channels) in NR1 format
:CHANnel<n>:PROBe:SKE W <skew_value> (see page 349)	:CHANnel<n>:PROBe:SKE W? (see page 349)	<skew_value> ::= -100 ns to +100 ns in NR3 format <n> ::= 1 to (# analog channels) in NR1 format
:CHANnel<n>:PROBe:STY Pe <signal type> (see page 350)	:CHANnel<n>:PROBe:STY Pe? (see page 350)	<signal type> ::= {DIFFerential SINGle} <n> ::= 1 to (# analog channels) in NR1 format
:CHANnel<n>:PROBe:ZOO M {{0 OFF} {1 ON}} (see page 351)	:CHANnel<n>:PROBe:ZOO M? (see page 351)	<setting> ::= {0 1} <n> ::= 1 to (# analog channels) in NR1 format
:CHANnel<n>:PROTection (see page 352)	:CHANnel<n>:PROTection ? (see page 352)	{NORM TRIP} <n> ::= 1 to (# analog channels) in NR1 format
:CHANnel<n>:RANGe <range>[suffix] (see page 353)	:CHANnel<n>:RANGe? (see page 353)	<range> ::= Vertical full-scale range value in NR3 format [suffix] ::= {V mV} <n> ::= 1 to (# analog channels) in NR1 format
:CHANnel<n>:SCALe <scale>[suffix] (see page 354)	:CHANnel<n>:SCALe? (see page 354)	<scale> ::= Vertical units per division value in NR3 format [suffix] ::= {V mV} <n> ::= 1 to (# analog channels) in NR1 format
:CHANnel<n>:UNITs <units> (see page 355)	:CHANnel<n>:UNITs? (see page 355)	<units> ::= {VOLT AMPere} <n> ::= 1 to (# analog channels) in NR1 format
:CHANnel<n>:VERNier {{0 OFF} {1 ON}} (see page 356)	:CHANnel<n>:VERNier? (see page 356)	{0 1} <n> ::= 1 to (# analog channels) in NR1 format

Table 8 :COMPLIance Commands Summary

Command	Query	Options and Query Returns
:COMPLIance:USB:AUTSetup (see page 358)	n/a	n/a
:COMPLIance:USB:HUBS<number> (see page 359)	:COMPLIance:USB:HUBS? (see page 359)	<number> ::= 0-5 in NR1 format
:COMPLIance:USB:RUN (see page 360)	n/a	n/a
:COMPLIance:USB:SOURCE:ADJacent <source> (see page 361)	:COMPLIance:USB:SOURCE:ADJacent? (see page 361)	<source> ::= {CHANnel<n>}
:COMPLIance:USB:SOURCE:DIFFerential <source> (see page 362)	:COMPLIance:USB:SOURCE:DIFFerential? (see page 362)	<source> ::= {CHANnel<n>}
:COMPLIance:USB:SOURCE:DMINus <source> (see page 363)	:COMPLIance:USB:SOURCE:DMINus? (see page 363)	<source> ::= {CHANnel<n>}
:COMPLIance:USB:SOURCE:DPLUS <source> (see page 364)	:COMPLIance:USB:SOURCE:DPLUS? (see page 364)	<source> ::= {CHANnel<n>}
:COMPLIance:USB:TEST<test> (see page 365)	:COMPLIance:USB:TEST? (see page 365)	<test> ::= {DHSS HHSS DLSS HLSS DFSS HFSS}
:COMPLIance:USB:TEST:CONNECTION <connection> (see page 366)	:COMPLIance:USB:TEST:CONNECTION? (see page 366)	<connection> ::= {SINGLEended DIFFerential}
:COMPLIance:USB:TEST:TYPE <type> (see page 367)	:COMPLIance:USB:TEST:TYPE? (see page 367)	<type> ::= {NEARend FARend}

Table 9 :COUNTER Commands Summary

Command	Query	Options and Query Returns
n/a	:COUNTER:CURRENT? (see page 370)	<value> ::= current counter value in NR3 format
:COUNTER:ENABLE { {0 OFF} {1 ON} } (see page 371)	:COUNTER:ENABLE? (see page 371)	{0 1}
:COUNTER:MODE <mode> (see page 372)	:COUNTER:MODE (see page 372)	<mode> ::= {FREQuency PERiod}

Table 9 :COUNTER Commands Summary (continued)

Command	Query	Options and Query Returns
:COUNTER:NDIGits <value> (see page 373)	:COUNTER:NDIGits (see page 373)	<value> ::= 5 normally, 8 with 10 MHz reference signal, in NR1 format
:COUNTER:SOURce <source> (see page 374)	:COUNTER:SOURce? (see page 374)	<source> ::= {CHANNEL<n> TQEvent} <n> ::= 1 to (# analog channels) in NR1 format

Table 10 :DEMO Commands Summary

Command	Query	Options and Query Returns
:DEMO:FUNCTION <signal> (see page 376)	:DEMO:FUNCTION? (see page 379)	<signal> ::= {SINusoid NOISy PHASe RINGing SINGle AM CLK GLITCh BURSt MSO RUNT TRANSition RFBurst SHOLd LFSine FMBurst ETE CAN LIN UART I2C SPI I2S CANLin CXPI ARINc FLEXray MANChester MIL MIL2 USB NMONotonic DCMotor HARMonics COUPling CFD SENT USBPd KEYSight}
:DEMO:FUNCTION:PHASe: PHASe <angle> (see page 381)	:DEMO:FUNCTION:PHASe: PHASe? (see page 381)	<angle> ::= angle in degrees from 0 to 360 in NR3 format
:DEMO:OUTPut {{0 OFF} {1 ON}} (see page 382)	:DEMO:OUTPut? (see page 382)	{0 1}

Table 11 :DIGital<d> Commands Summary

Command	Query	Options and Query Returns
:DIGital<d>:DISPlay {{0 OFF} {1 ON}} (see page 385)	:DIGital<d>:DISPlay? (see page 385)	<d> ::= 0 to (# digital channels - 1) in NR1 format {0 1}
:DIGital<d>:LABel <string> (see page 386)	:DIGital<d>:LABel? (see page 386)	<d> ::= 0 to (# digital channels - 1) in NR1 format <string> ::= any series of 10 or less ASCII characters enclosed in quotation marks

Table 11 :DIGital<d> Commands Summary (continued)

Command	Query	Options and Query Returns
:DIGital<d>:POSition <position> (see page 387)	:DIGital<d>:POSition? (see page 387)	<d> ::= 0 to (# digital channels - 1) in NR1 format <position> ::= 0-7 if display size = large, 0-15 if size = medium, 0-31 if size = small Returns -1 when there is no space to display the digital waveform.
:DIGital<d>:SIZE <value> (see page 388)	:DIGital<d>:SIZE? (see page 388)	<d> ::= 0 to (# digital channels - 1) in NR1 format <value> ::= {SMALl MEDium LARGE}
:DIGital<d>:THReshold <value>[suffix] (see page 389)	:DIGital<d>:THReshold? (see page 389)	<d> ::= 0 to (# digital channels - 1) in NR1 format <value> ::= {CMOS ECL TTL <user defined value>} <user defined value> ::= value in NR3 format from -8.00 to +8.00 [suffix] ::= {V mV uV}

Table 12 :DISPlay Commands Summary

Command	Query	Options and Query Returns
:DISPlay:ANNotation<n> > {{0 OFF} {1 ON}} (see page 394)	:DISPlay:ANNotation<n>? (see page 394)	{0 1} <n> ::= an integer from 1 to 4 in NR1 format.
:DISPlay:ANNotation<n>: BACKground <mode> (see page 395)	:DISPlay:ANNotation<n>: BACKground? (see page 395)	<mode> ::= {OPAQue INVerted TRANSPARENT} <n> ::= an integer from 1 to 4 in NR1 format.
:DISPlay:ANNotation<n>: COLor <color> (see page 396)	:DISPlay:ANNotation<n>: COLor? (see page 396)	<color> ::= {CH1 CH2 CH3 CH4 DIG MATH REF MARKer WHITE RED} <n> ::= an integer from 1 to 4 in NR1 format.
:DISPlay:ANNotation<n>: TEXT <string> (see page 397)	:DISPlay:ANNotation<n>: TEXT? (see page 397)	<string> ::= quoted ASCII string (up to 254 characters) <n> ::= an integer from 1 to 4 in NR1 format.

Table 12 :DISPlay Commands Summary (continued)

Command	Query	Options and Query Returns
:DISPlay:ANNotation<n>:X1Position <value> (see page 398)	:DISPlay:ANNotation<n>:X1Position? (see page 398)	<value> ::= an integer from 0 to (800 - width of annotation) in NR1 format. <n> ::= an integer from 1 to 4 in NR1 format.
:DISPlay:ANNotation<n>:Y1Position <value> (see page 399)	:DISPlay:ANNotation<n>:Y1Position? (see page 399)	<value> ::= an integer from 0 to (480 - height of annotation) in NR1 format. <n> ::= an integer from 1 to 4 in NR1 format.
:DISPlay:BACKlight { {0 OFF} {1 ON} } (see page 400)	n/a	n/a
:DISPlay:CLear (see page 401)	n/a	n/a
n/a	:DISPlay:DATA? [<format>] [,] [<palette>] (see page 402)	<format> ::= {BMP BMP8bit PNG} <palette> ::= {COLor GRAYscale} <display data> ::= data in IEEE 488.2 # format
:DISPlay:GRATicule:ALABels { {0 OFF} {1 ON} } (see page 404)	:DISPlay:GRATicule:ALABels? (see page 404)	<setting> ::= {0 1}
:DISPlay:GRATicule:INTensity <value> (see page 405)	:DISPlay:GRATicule:INTensity? (see page 405)	<value> ::= an integer from 0 to 100 in NR1 format.
:DISPlay:GRATicule:TYPE <type> (see page 406)	:DISPlay:GRATicule:TYPE? (see page 406)	<type> ::= {FULL MVOLt IRE}
:DISPlay:INTensity:WAVEform <value> (see page 407)	:DISPlay:INTensity:WAVEform? (see page 407)	<value> ::= an integer from 0 to 100 in NR1 format.
:DISPlay:LABel { {0 OFF} {1 ON} } (see page 408)	:DISPlay:LABel? (see page 408)	{0 1}
:DISPlay:LABList <binary block> (see page 409)	:DISPlay:LABList? (see page 409)	<binary block> ::= an ordered list of up to 75 labels, each 10 characters maximum, separated by newline characters

Table 12 :DISPlay Commands Summary (continued)

Command	Query	Options and Query Returns
:DISPlay:MENU <menu> (see page 410)	n/a	<menu> ::= {MASK MEASure SEGmented LISTer POWER}
:DISPlay:MESSAge:CLEAr (see page 411)	n/a	n/a
:DISPlay:PERSiStence <value> (see page 412)	:DISPlay:PERSiStence? (see page 412)	<value> ::= {MINimum INFinite <time>} <time> ::= seconds in in NR3 format from 100E-3 to 60E0
:DISPlay:PERSiStence: CLEAr (see page 413)	n/a	n/a
:DISPlay:SIDeBar <sidebar> (see page 414)	n/a	<sidebar> ::= {SUMMary CURSors MEASurements DVM NAVigate CONTROLS EVENTs COUNTer HISTogram}
:DISPlay:TRANSPARENT {OFF ON} (see page 415)	:DISPlay:TRANSPARENT? (see page 415)	{OFF ON}
:DISPlay:VECTors {{0 OFF} {1 ON}} (see page 416)	:DISPlay:VECTors? (see page 416)	{0 1} OFF or 0 can only be selected with the 1 GHz or 1.5 GHz bandwidth models

Table 13 :DVM Commands Summary

Command	Query	Options and Query Returns
:DVM:ARANge {{0 OFF} {1 ON}} (see page 418)	:DVM:ARANge? (see page 418)	{0 1}
n/a	:DVM:CURRent? (see page 419)	<dvm_value> ::= floating-point number in NR3 format
:DVM:ENABle {{0 OFF} {1 ON}} (see page 420)	:DVM:ENABle? (see page 420)	{0 1}
:DVM:MODE <mode> (see page 421)	:DVM:MODE? (see page 421)	<dvm_mode> ::= {ACRMs DC DCRMs}
:DVM:SOURce <source> (see page 422)	:DVM:SOURce? (see page 422)	<source> ::= {CHANnel<n>} <n> ::= 1-2 or 1-4 in NR1 format

Table 14 :EXternal Trigger Commands Summary

Command	Query	Options and Query Returns
:EXternal:BWLimit <bwlimit> (see page 424)	:EXternal:BWLimit? (see page 424)	<bwlimit> ::= {0 OFF}
:EXternal:PROBe <attenuation> (see page 425)	:EXternal:PROBe? (see page 425)	<attenuation> ::= probe attenuation ratio in NR3 format
:EXternal:RANGe <range>[<suffix> (see page 426)	:EXternal:RANGe? (see page 426)	<range> ::= vertical full-scale range value in NR3 format <suffix> ::= {V mV}
:EXternal:UNITs <units> (see page 427)	:EXternal:UNITs? (see page 427)	<units> ::= {VOLT AMPere}

Table 15 :FRANalysis Commands Summary

Command	Query	Options and Query Returns
n/a	:FRANalysis:DATA? [SWEep SINGle] (see page 432)	<binary_block> ::= comma-separated data with newlines at the end of each row
:FRANalysis:ENABle { {0 OFF} {1 ON} } (see page 433)	:FRANalysis:ENABle? (see page 433)	{0 1}
:FRANalysis:FREQuency :MODE <setting> (see page 434)	:FRANalysis:FREQuency :MODE? (see page 434)	<setting> ::= {SWEep SINGle}
:FRANalysis:FREQuency :SINGle <value>[suffix] (see page 435)	:FRANalysis:FREQuency :SINGle? (see page 435)	<value> ::= {20 100 1000 10000 100000 1000000 10000000 20000000} [suffix] ::= {Hz kHz MHz}
:FRANalysis:FREQuency :STARt <value>[suffix] (see page 436)	:FRANalysis:FREQuency :STARt? (see page 436)	<value> ::= {20 100 1000 10000 100000 1000000 10000000} [suffix] ::= {Hz kHz MHz}
:FRANalysis:FREQuency :STOP <value>[suffix] (see page 437)	:FRANalysis:FREQuency :STOP? (see page 437)	<value> ::= {100 1000 10000 100000 1000000 10000000 20000000} [suffix] ::= {Hz kHz MHz}
:FRANalysis:PPDecade <value> (see page 438)	:FRANalysis:PPDecade? (see page 438)	<value> ::= {10 20 30 40 50 60 70 80 90 100}

Table 15 :FRANalysis Commands Summary (continued)

Command	Query	Options and Query Returns
:FRANalysis:RUN (see page 439)	n/a	n/a
:FRANalysis:SOURce:IN Put <source> (see page 440)	:FRANalysis:SOURce:IN Put? (see page 440)	<source> ::= CHANNEL<n> <n> ::= 1 to (# analog channels) in NR1 format
:FRANalysis:SOURce:OU TPut <source> (see page 441)	:FRANalysis:SOURce:OU TPut? (see page 441)	<source> ::= CHANNEL<n> <n> ::= 1 to (# analog channels) in NR1 format
:FRANalysis:SWEep:POI Nts <points> (see page 442)	:FRANalysis:SWEep:POI Nts? (see page 442)	<points> ::= integer from 1-1000 in NR1 format
:FRANalysis:TRACe <selection> (see page 443)	:FRANalysis:TRACe? (see page 443)	<selection> ::= {NONE ALL GAIN PHASe}[, {GAIN PHASe}]
:FRANalysis:WGEN:LOAD <impedance> (see page 444)	:FRANalysis:WGEN:LOAD ? (see page 444)	<impedance> ::= {ONEMeg FIFTy}
:FRANalysis:WGEN:VOLT age <amplitude>, [<range>] (see page 445)	:FRANalysis:WGEN:VOLT age? [<range>] (see page 445)	<amplitude> ::= amplitude in volts in NR3 format <range> ::= {F20HZ F100HZ F1KHZ F10KHZ F100KHZ F1MHZ F10MHZ F20MHZ}
:FRANalysis:WGEN:VOLT age:PROFile {{0 OFF} {1 ON}} (see page 447)	:FRANalysis:WGEN:VOLT age:PROFile? (see page 447)	{0 1}

Table 16 :FUNCTION<m> Commands Summary

Command	Query	Options and Query Returns
:FUNCTION<m>:AVERAge:COUNT <count> (see page 456)	:FUNCTION<m>:AVERAge:COUNT? (see page 456)	<count> ::= an integer from 2 to 65536 in NR1 format <m> ::= 1 to (# math functions) in NR1 format
:FUNCTION<m>:BUS:CLOCK <source> (see page 457)	:FUNCTION<m>:BUS:CLOCK? (see page 457)	<source> ::= {CHANnel<n> DIGital<d>} <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <m> ::= 1 to (# math functions) in NR1 format
:FUNCTION<m>:BUS:SLOPe <slope> (see page 458)	:FUNCTION<m>:BUS:SLOPe? (see page 458)	<slope> ::= {NEGative POSitive EITHer} <m> ::= 1 to (# math functions) in NR1 format
:FUNCTION<m>:BUS:YINCReMENT <value> (see page 459)	:FUNCTION<m>:BUS:YINCReMENT? (see page 459)	<value> ::= value per bus code, in NR3 format <m> ::= 1 to (# math functions) in NR1 format
:FUNCTION<m>:BUS:YORigin <value> (see page 460)	:FUNCTION<m>:BUS:YORigin? (see page 460)	<value> ::= value at bus code = 0, in NR3 format <m> ::= 1 to (# math functions) in NR1 format
:FUNCTION<m>:BUS:YUNits <units> (see page 461)	:FUNCTION<m>:BUS:YUNits? (see page 461)	<units> ::= {VOLT AMPere NONE} <m> ::= 1 to (# math functions) in NR1 format
:FUNCTION<m>:CLEAr (see page 462)	n/a	n/a
:FUNCTION<m>:DISPlay {{0 OFF} {1 ON}} (see page 463)	:FUNCTION<m>:DISPlay? (see page 463)	{0 1} <m> ::= 1 to (# math functions) in NR1 format
n/a	:FUNCTION<m>[:FFT]:BSIZE? (see page 464)	<bin_size> ::= Hz in NR3 format <m> ::= 1 to (# math functions) in NR1 format

Table 16 :FUNction<m> Commands Summary (continued)

Command	Query	Options and Query Returns
:FUNction<m>[:FFT]:CENTer <frequency> (see page 465)	:FUNction<m>[:FFT]:CENTer? (see page 465)	<frequency> ::= the current center frequency in NR3 format. The range of legal values is from -25 GHz to 25 GHz. <m> ::= 1 to (# math functions) in NR1 format
:FUNction<m>[:FFT]:DETEction:POINTs <number_of_buckets> (see page 466)	:FUNction<m>[:FFT]:DETEction:POINTs? (see page 466)	<number_of_buckets> ::= an integer from 640 to 64K in NR1 format <m> ::= 1 to (# math functions) in NR1 format
:FUNction<m>[:FFT]:DETEction:TYPE <type> (see page 467)	:FUNction<m>[:FFT]:DETEction:TYPE? (see page 467)	<type> ::= {OFF SAMPlE PPOSitive PNEGative NORMAl AVERAge} <m> ::= 1 to (# math functions) in NR1 format
:FUNction<m>[:FFT]:FREQuency:START <frequency> (see page 468)	:FUNction<m>[:FFT]:FREQuency:START? (see page 468)	<frequency> ::= the start frequency in NR3 format. <m> ::= 1 to (# math functions) in NR1 format
:FUNction<m>[:FFT]:FREQuency:STOP <frequency> (see page 469)	:FUNction<m>[:FFT]:FREQuency:STOP? (see page 469)	<frequency> ::= the stop frequency in NR3 format. <m> ::= 1 to (# math functions) in NR1 format
:FUNction<m>[:FFT]:GATE <gating> (see page 470)	:FUNction<m>[:FFT]:GATE? (see page 470)	<gating> ::= {NONE ZOOM} <m> ::= 1-4 in NR1 format
:FUNction<m>[:FFT]:PHASe:REFeRence <ref_point> (see page 471)	:FUNction<m>[:FFT]:PHASe:REFeRence? (see page 471)	<ref_point> ::= {TRIGger DISPlay} <m> ::= 1-4 in NR1 format
n/a	:FUNction<m>[:FFT]:RBWidth? (see page 472)	<resolution_bw> ::= Hz in NR3 format <m> ::= 1 to (# math functions) in NR1 format

Table 16 :FUNctioN<m> Commands Summary (continued)

Command	Query	Options and Query Returns
:FUNctioN<m>[:FFT]:RE ADout<n> <readout_type> (see page 473)	:FUNctioN<m>[:FFT]:RE ADout<n>? (see page 473)	<readout_type> ::= {SRATe BSize RBWidth} <m> ::= 1 to (# math functions) in NR1 format <n> ::= 1-2 in NR1 format, 2 is for dedicated FFT function
:FUNctioN<m>[:FFT]:SP AN (see page 474)	:FUNctioN<m>[:FFT]:SP AN? (see page 474)	 ::= the current frequency span in NR3 format. Legal values are 1 Hz to 100 GHz. <m> ::= 1 to (# math functions) in NR1 format
n/a	:FUNctioN<m>[:FFT]:SR ATe? (see page 475)	<sample_rate> ::= Hz in NR3 format <m> ::= 1 to (# math functions) in NR1 format
:FUNctioN<m>[:FFT]:VT YPe <units> (see page 476)	:FUNctioN<m>[:FFT]:VT YPe? (see page 476)	<units> ::= {DECibel VRMS} for the FFT (magnitude) operation <units> ::= {DEGRees RADians} for the FFTPhase operation <m> ::= 1 to (# math functions) in NR1 format
:FUNctioN<m>[:FFT]:WI NDow <window> (see page 477)	:FUNctioN<m>[:FFT]:WI NDow? (see page 477)	<window> ::= {RECTangular HANNing FLATtop BHARris BARTlett} <m> ::= 1 to (# math functions) in NR1 format
:FUNctioN<m>:FREQuenc y:BANDpass:CENTer <center_freq> (see page 478)	:FUNctioN<m>:FREQuenc y:BANDpass:CENTer? (see page 478)	<center_freq> ::= center frequency of band-pass filter in NR3 format
:FUNctioN<m>:FREQuenc y:BANDpass:WIDTh <freq_width> (see page 479)	:FUNctioN<m>:FREQuenc y:BANDpass:WIDTh? (see page 479)	<freq_width> ::= frequency width of band-pass filter in NR3 format
:FUNctioN<m>:FREQuenc y:HIGHPass <3dB_freq> (see page 480)	:FUNctioN<m>:FREQuenc y:HIGHPass? (see page 480)	<3dB_freq> ::= 3dB cutoff frequency value in NR3 format <m> ::= 1 to (# math functions) in NR1 format

Table 16 :FUNction<m> Commands Summary (continued)

Command	Query	Options and Query Returns
:FUNction<m>:FREQuency:LOWPass <3dB_freq> (see page 481)	:FUNction<m>:FREQuency:LOWPass? (see page 481)	<3dB_freq> ::= 3dB cutoff frequency value in NR3 format <m> ::= 1 to (# math functions) in NR1 format
:FUNction<m>:INTEgrate:IICondition {{0 OFF} {1 ON}} (see page 482)	:FUNction<m>:INTEgrate:IICondition? (see page 482)	{0 1} <m> ::= 1 to (# math functions) in NR1 format
:FUNction<m>:INTEgrate:IOFFset <input_offset> (see page 483)	:FUNction<m>:INTEgrate:IOFFset? (see page 483)	<input_offset> ::= DC offset correction in NR3 format. <m> ::= 1 to (# math functions) in NR1 format
:FUNction<m>:LINear:GAIn <value> (see page 484)	:FUNction<m>:LINear:GAIn? (see page 484)	<value> ::= 'A' in Ax + B, value in NR3 format <m> ::= 1 to (# math functions) in NR1 format
:FUNction<m>:LINear:OFFSet <value> (see page 485)	:FUNction<m>:LINear:OFFSet? (see page 485)	<value> ::= 'B' in Ax + B, value in NR3 format <m> ::= 1 to (# math functions) in NR1 format
:FUNction<m>:OFFSet <offset> (see page 486)	:FUNction<m>:OFFSet? (see page 486)	<offset> ::= the value at center screen in NR3 format. The range of legal values is +/-10 times the current sensitivity of the selected function. <m> ::= 1 to (# math functions) in NR1 format
:FUNction<m>:OPERation <operation> (see page 487)	:FUNction<m>:OPERation? (see page 490)	<operation> ::= {ADD SUBTract MULTiply DIVide INTEgrate DIFF FFT FFTPhase SQRT MAGNify ABSolute SQUare LN LOG EXP TEN LOWPass HIGHpass BANDpass AVERage LINear MAXimum MINimum PEAK MAXHold MINHold TREND BTiming BSTate SERChart} <m> ::= 1 to (# math functions) in NR1 format

Table 16 :FUNction<m> Commands Summary (continued)

Command	Query	Options and Query Returns
:FUNction<m>:RANGe <range> (see page 492)	:FUNction<m>:RANGe? (see page 492)	<range> ::= the full-scale vertical axis value in NR3 format. The range for ADD, SUBT, MULT is 8E-6 to 800E+3. The range for the INTegrate function is 8E-9 to 400E+3. The range for the DIFF function is 80E-3 to 8.0E12 (depends on current sweep speed). The range for the FFT function is 8 to 800 dBV. <m> ::= 1 to (# math functions) in NR1 format
:FUNction<m>:REferenc e <level> (see page 493)	:FUNction<m>:REferenc e? (see page 493)	<level> ::= the value at center screen in NR3 format. The range of legal values is +/-10 times the current sensitivity of the selected function. <m> ::= 1 to (# math functions) in NR1 format
:FUNction<m>:SCALe <scale value>[<suffix>] (see page 494)	:FUNction<m>:SCALe? (see page 494)	<scale value> ::= integer in NR1 format <suffix> ::= {V dB} <m> ::= 1 to (# math functions) in NR1 format
:FUNction<m>:SERChart :SOURce <serialbus> (see page 495)	:FUNction<m>:SERChart :SOURce? (see page 495)	<m> ::= 1 to (# math functions) in NR1 format <serialbus> ::= {SBUS<n>} <n> :: 1 to (# of serial bus) in NR1 format
:FUNction<m>:SERChart :TIME:MODE <value> (see page 496)	:FUNction<m>:SERChart :TIME:MODE? (see page 496)	<m> ::= 1 to (# math functions) in NR1 format <value> ::= {MAIN ROLL}
:FUNction<m>:SERChart :TIME:RANGe <range_value> (see page 498)	:FUNction<m>:SERChart :TIME:RANGe? (see page 498)	<m> ::= 1 to (# math functions) in NR1 format <range_value> ::= time value in NR3 format

Table 16 :FUNction<m> Commands Summary (continued)

Command	Query	Options and Query Returns
:FUNction<m>:SERChart :TIME:OFFSet <offset> (see page 499)	:FUNction<m>:SERChart :TIME:OFFSet? (see page 499)	<m> ::= 1 to (# math functions) in NR1 format <offset> ::= vertical offset value in NR3 format
:FUNction<m>:SMOoth:POINts <points> (see page 500)	:FUNction<m>:SMOoth:POINts? (see page 500)	<points> ::= odd integer in NR1 format
:FUNction<m>:SOURce1 <source> (see page 501)	:FUNction<m>:SOURce1? (see page 501)	<source> ::= {CHANnel<n> FUNction<c> MATH<c> WMEMory<r> BUS} <n> ::= 1 to (# analog channels) in NR1 format <c> ::= {1 2 3}, must be lower than <m> <r> ::= 1 to (# ref waveforms) in NR1 format ::= {1 2} <m> ::= 1 to (# math functions) in NR1 format
:FUNction<m>:SOURce2 <source> (see page 503)	:FUNction<m>:SOURce2? (see page 503)	<source> ::= {CHANnel<n> WMEMory<r> NONE} <n> ::= 1 to (# analog channels) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <m> ::= 1 to (# math functions) in NR1 format
:FUNction<m>:TREND:NMEasurement MEAS<n> (see page 504)	:FUNction<m>:TREND:NMEasurement? (see page 504)	<n> ::= # of installed measurement, from 1 to 10 <m> ::= 1 to (# math functions) in NR1 format

Table 17 :HARDcopy/:HCOPIY Commands Summary

Command	Query	Options and Query Returns
:HARDcopy:AREA <area> (see page 508)	:HARDcopy:AREA? (see page 508)	<area> ::= SCreen
:HARDcopy:APRinter <active_printer> (see page 509)	:HARDcopy:APRinter? (see page 509)	<active_printer> ::= {<index> <name>} <index> ::= integer index of printer in list <name> ::= name of printer in list
:HARDcopy:FACTors {{0 OFF} {1 ON}} (see page 510)	:HARDcopy:FACTors? (see page 510)	{0 1}
:HARDcopy:FFEed {{0 OFF} {1 ON}} (see page 511)	:HARDcopy:FFEed? (see page 511)	{0 1}
:HARDcopy:INKSaver {{0 OFF} {1 ON}} (see page 512)	:HARDcopy:INKSaver? (see page 512)	{0 1}
:HARDcopy:LAYout <layout> (see page 513)	:HARDcopy:LAYout? (see page 513)	<layout> ::= {LANDscape PORTrait}
:HARDcopy:NETWork:ADD Ress <address> (see page 514)	:HARDcopy:NETWork:ADD Ress? (see page 514)	<address> ::= quoted ASCII string
:HARDcopy:NETWork:APP Ly (see page 515)	n/a	n/a
:HARDcopy:NETWork:DOM ain <domain> (see page 516)	:HARDcopy:NETWork:DOM ain? (see page 516)	<domain> ::= quoted ASCII string
:HARDcopy:NETWork:PAS Sword <password> (see page 517)	n/a	<password> ::= quoted ASCII string
:HARDcopy:NETWork:SLO T <slot> (see page 518)	:HARDcopy:NETWork:SLO T? (see page 518)	<slot> ::= {NET0 NET1}
:HARDcopy:NETWork:USE Rname <username> (see page 519)	:HARDcopy:NETWork:USE Rname? (see page 519)	<username> ::= quoted ASCII string
:HARDcopy:PALETTE <palette> (see page 520)	:HARDcopy:PALETTE? (see page 520)	<palette> ::= {COLor GRAYscale NONE}

Table 17 :HARDcopy/:HCOPY Commands Summary (continued)

Command	Query	Options and Query Returns
n/a	:HARDcopy:PRINter:LIS T? (see page 521)	<code><list> ::= [<printer_spec>] ...</code> <code>[printer_spec]</code> <code><printer_spec> ::=</code> <code>"<index>,<active>,<name>;"</code> <code><index> ::= integer index of</code> <code>printer</code> <code><active> ::= {Y N}</code> <code><name> ::= name of printer</code>
:HARDcopy:START (see page 522)	n/a	n/a
n/a	:HCOPY:SDUMp:DATA? (see page 523)	<code><display_data> ::= binary block</code> <code>data in IEEE-488.2 # format.</code>
:HCOPY:SDUMp:FORMat <format> (see page 524)	:HCOPY:SDUMp:FORMat? (see page 524)	<code><format> ::= {BMP BMP8bit </code> <code>PNG}</code>

Table 18 :HISTogram Commands Summary

Command	Query	Options and Query Returns
:HISTogram:AXIS <axis> (see page 528)	:HISTogram:AXIS? (see page 528)	<code><axis> ::= {VERTical </code> <code>HORizontal}</code>
:HISTogram:DISPlay { {0 OFF} {1 ON} } (see page 529)	:HISTogram:DISPlay? (see page 529)	<code>{0 1}</code>
:HISTogram:MEASuremen t MEAS<n> (see page 530)	:HISTogram:MEASuremen t? (see page 530)	<code><n> ::= # of installed</code> <code>measurement, from 1 to 10</code>
:HISTogram:MODE <mode> (see page 531)	:HISTogram:MODE? (see page 531)	<code><mode> ::= {OFF WAVEform </code> <code>MEASurement}</code>
:HISTogram:RESet (see page 532)	n/a	n/a
:HISTogram:SIZE <size> (see page 533)	:HISTogram:SIZE? (see page 533)	<code><size> ::= 1.0 to 5.0 for</code> <code>vertical histograms, 1.0 to 4.0</code> <code>for horizontal histograms, in NR3</code> <code>format</code>
:HISTogram:TYPE <type> (see page 534)	:HISTogram:TYPE? (see page 534)	<code><type> ::= {VERTical HORizontal</code> <code> MEASurement}</code>

Table 18 :HISTogram Commands Summary (continued)

Command	Query	Options and Query Returns
:HISTogram:WINDow:BLIMit <limit> (see page 535)	:HISTogram:WINDow:BLIMit? (see page 535)	<limit> ::= bottom value in NR3 format
:HISTogram:WINDow:LLIMit <limit> (see page 536)	:HISTogram:WINDow:LLIMit? (see page 536)	<limit> ::= left value in NR3 format
:HISTogram:WINDow:RLIMit <limit> (see page 537)	:HISTogram:WINDow:RLIMit? (see page 537)	<limit> ::= right value in NR3 format
:HISTogram:WINDow:SOURce <source> (see page 538)	:HISTogram:WINDow:SOURce? (see page 538)	<source> ::= {CHANnel<n> FUNCTION<m> MATH<m> WMEMory<r> CGRade} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format
:HISTogram:WINDow:TLMit <limit> (see page 539)	:HISTogram:WINDow:TLMit? (see page 539)	<limit> ::= top value in NR3 format

Table 19 :LISTer Commands Summary

Command	Query	Options and Query Returns
n/a	:LISTer:DATA? (see page 542)	<binary_block> ::= comma-separated data with newlines at the end of each row
:LISTer:DISPlay { {OFF 0} {SBUS1 ON 1} {SBUS2 2} ALL} (see page 543)	:LISTer:DISPlay? (see page 543)	{OFF SBUS1 SBUS2 ALL}
:LISTer:REFerence <time_ref> (see page 544)	:LISTer:REFerence? (see page 544)	<time_ref> ::= {TRIGger PREVIOUS}

Table 20 :LTEST Commands Summary

Command	Query	Options and Query Returns
:LTEST:COPY (see page 547)	n/a	n/a
:LTEST:COPY:ALL (see page 548)	n/a	n/a
:LTEST:COPY:MARGin <percent> (see page 549)	:LTEST:COPY:MARGin? (see page 549)	<percent> ::= copy margin percent in NR1 format
:LTEST:ENABle {{0 OFF} {1 ON}} (see page 550)	:LTEST:ENABle? (see page 550)	<setting> ::= {0 1}
:LTEST:FAIL <condition> (see page 551)	:LTEST:FAIL? (see page 551)	<condition> ::= {INSide OUTSide}
:LTEST:LLIMit <lower_value> (see page 552)	:LTEST:LLIMit? (see page 552)	<lower_value> ::= a real number in NR3 format
:LTEST:MEASurement {MEAS<n>} (see page 553)	:LTEST:MEASurement? (see page 553)	<n> ::= # of installed measurement, from 1 to 10
n/a	:LTEST:RESults? [{MEAS<n>}] (see page 554)	<n> ::= # of installed measurement, from 1 to 10
:LTEST:RUMode:SOFailu re {{0 OFF} {1 ON}} (see page 555)	:LTEST:RUMode:SOFailu re? (see page 555)	<setting> ::= {0 1}
:LTEST:TEST {{0 OFF} {1 ON}} (see page 556)	:LTEST:TEST? (see page 556)	<setting> ::= {0 1}
:LTEST:ULIMit <upper_value> (see page 557)	:LTEST:ULIMit? (see page 557)	<upper_value> ::= a real number in NR3 format

Table 21 :MARKer Commands Summary

Command	Query	Options and Query Returns
n/a	:MARKer:DYDX? (see page 562)	<return_value> ::= $\Delta Y / \Delta X$ value in NR3 format
:MARKer:MODE <mode> (see page 563)	:MARKer:MODE? (see page 563)	<mode> ::= {OFF MEASurement MANual WAVeform BINary HEX}

Table 21 :MARKer Commands Summary (continued)

Command	Query	Options and Query Returns
:MARKer:X1:DISPlay { {0 OFF} {1 ON} } (see page 564)	:MARKer:X1:DISPlay? (see page 564)	<setting> ::= {0 1}
:MARKer:X1Position <position>[suffix] (see page 565)	:MARKer:X1Position? (see page 565)	<position> ::= X1 cursor position value in NR3 format [suffix] ::= {s ms us ns ps Hz kHz MHz} <return_value> ::= X1 cursor position value in NR3 format
:MARKer:X1Y1source <source> (see page 566)	:MARKer:X1Y1source? (see page 566)	<source> ::= {CHANnel<n> FUNCTION<m> MATH<m> WMemory<r> HISTogram} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= <source>
:MARKer:X2:DISPlay { {0 OFF} {1 ON} } (see page 567)	:MARKer:X2:DISPlay? (see page 567)	<setting> ::= {0 1}
:MARKer:X2Position <position>[suffix] (see page 568)	:MARKer:X2Position? (see page 568)	<position> ::= X2 cursor position value in NR3 format [suffix] ::= {s ms us ns ps Hz kHz MHz} <return_value> ::= X2 cursor position value in NR3 format
:MARKer:X2Y2source <source> (see page 569)	:MARKer:X2Y2source? (see page 569)	<source> ::= {CHANnel<n> FUNCTION<m> MATH<m> WMemory<r> HISTogram} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= <source>
n/a	:MARKer:XDELta? (see page 570)	<return_value> ::= X cursors delta value in NR3 format

Table 21 :MARKer Commands Summary (continued)

Command	Query	Options and Query Returns
:MARKer:XUNits <mode> (see page 571)	:MARKer:XUNits? (see page 571)	<units> ::= {SECONDS HERTz DEGREes PERCent}
:MARKer:XUNits:USE (see page 572)	n/a	n/a
:MARKer:Y1:DISPlay { {0 OFF} {1 ON} } (see page 573)	:MARKer:Y1:DISPlay? (see page 573)	<setting> ::= {0 1}
:MARKer:Y1Position <position>[suffix] (see page 574)	:MARKer:Y1Position? (see page 574)	<position> ::= Y1 cursor position value in NR3 format [suffix] ::= {V mV dB} <return_value> ::= Y1 cursor position value in NR3 format
:MARKer:Y2:DISPlay { {0 OFF} {1 ON} } (see page 575)	:MARKer:Y2:DISPlay? (see page 575)	<setting> ::= {0 1}
:MARKer:Y2Position <position>[suffix] (see page 576)	:MARKer:Y2Position? (see page 576)	<position> ::= Y2 cursor position value in NR3 format [suffix] ::= {V mV dB} <return_value> ::= Y2 cursor position value in NR3 format
n/a	:MARKer:YDELta? (see page 577)	<return_value> ::= Y cursors delta value in NR3 format
:MARKer:YUNits <mode> (see page 578)	:MARKer:YUNits? (see page 578)	<units> ::= {BASE PERCent}
:MARKer:YUNits:USE (see page 579)	n/a	n/a

Table 22 :MEASure Commands Summary

Command	Query	Options and Query Returns
:MEASure:ALL (see page 603)	n/a	n/a
:MEASure:AREa [<interval>] [, <source>] (see page 604)	:MEASure:AREa? [<interval>] [, <source>] (see page 604)	<interval> ::= {CYCLe DISPlay} <source> ::= {CHANnel<n> FUNctIon<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= area in volt-seconds, NR3 format
:MEASure:BRATe [<source>] (see page 605)	:MEASure:BRATe? [<source>] (see page 605)	<source> ::= {<digital channels> CHANnel<n> FUNctIon<m> MATH<m> WMEMory<r>} <digital channels> ::= DIGital<d> for the MSO models <d> ::= 0 to (# digital channels - 1) in NR1 format <n> ::= 1 to (# of analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= bit rate in Hz, NR3 format
:MEASure:BWIDth [<source>] (see page 606)	:MEASure:BWIDth? [<source>] (see page 606)	<source> ::= {CHANnel<n> FUNctIon<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= burst width in seconds, NR3 format

Table 22 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:CLEar (see page 607)	n/a	n/a
:MEASure:COUNter [<source>] (see page 608)	:MEASure:COUNter? [<source>] (see page 608)	<source> ::= {CHANnel<n> EXTernal} for DSO models <source> ::= {CHANnel<n> DIGital<d> EXTernal} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <return_value> ::= counter frequency in Hertz in NR3 format
:MEASure:DEFine DELay, <delay spec> (see page 610)	:MEASure:DEFine? DELay (see page 612)	<delay spec> ::= <edge_spec1>, <edge_spec2> edge_spec1 ::= [<slope>]<occurrence> edge_spec2 ::= [<slope>]<occurrence> <slope> ::= {+ -} <occurrence> ::= integer
:MEASure:DEFine THResholds, <threshold spec> (see page 610)	:MEASure:DEFine? THResholds (see page 612)	<threshold spec> ::= {STANdard} {<threshold mode>, <upper>, <middle>, <lower>} <threshold mode> ::= {PERCent ABSolute}

Table 22 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:DElay [<source1>] [,<source2>] (see page 613)	:MEASure:DElay? [<source1>] [,<source2>] (see page 613)	<source1,2> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r> <digital channels>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <digital channels> ::= DIGital<d> for the MSO models <d> ::= 0 to (# digital channels - 1) in NR1 format <return_value> ::= floating-point number delay time in seconds in NR3 format
:MEASure:DElay:DEFine <source1_edge_slope>, <source1_edge_number> , <source1_edge_thresho ld>, <source2_edge_slope>, <source2_edge_number> , <source2_edge_thresho ld> (see page 615)	:MEASure:DElay:DEFine ? (see page 615)	<source1_edge_slope>, <source2_edge_slope> ::= {RISing FALLing} <source1_edge_number>, <source2_edge_number> ::= 0 to 1000 in NR1 format <source1_edge_threshold>, <source2_edge_threshold> ::= {LOWer MIDDle UPPer}
:MEASure:DUAL:CHARge [<interval>] [,<source1>][,<source 2>] (see page 616)	:MEASure:DUAL:CHARge? [<interval>] [,<source1>][,<source 2>] (see page 616)	<interval> ::= {CYCLe DISPlay} <source1>,<source2> ::= CHANnel<n> with N2820A probe connected <n> ::= 1 to (# analog channels) in NR1 format <return_value> ::= area in Amp-hours, NR3 format

Table 22 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:DUAL:VAMplitude [<source1>] [,<source2>] >] (see page 617)	:MEASure:DUAL:VAMplitude? [<source1>] [,<source2>] >] (see page 617)	<source1>,<source2> ::= CHANNEL<n> with N2820A probe connected <n> ::= 1 to (# analog channels) in NR1 format <return_value> ::= the amplitude of the selected waveform in volts in NR3 format
:MEASure:DUAL:VAVerage [<interval>] [,<source1>] [,<source2>] >] (see page 618)	:MEASure:DUAL:VAVerage? [<interval>] [,<source1>] [,<source2>] >] (see page 618)	<interval> ::= {CYCLE DISPLAY} <source1>,<source2> ::= CHANNEL<n> with N2820A probe connected <n> ::= 1 to (# analog channels) in NR1 format <return_value> ::= calculated average voltage in NR3 format
:MEASure:DUAL:VBase [<source1>] [,<source2>] >] (see page 619)	:MEASure:DUAL:VBase? [<source1>] [,<source2>] >] (see page 619)	<source1>,<source2> ::= CHANNEL<n> with N2820A probe connected <n> ::= 1 to (# analog channels) in NR1 format <base_voltage> ::= voltage at the base of the selected waveform in NR3 format
:MEASure:DUAL:VPP [<source1>] [,<source2>] >] (see page 620)	:MEASure:DUAL:VPP? [<source1>] [,<source2>] >] (see page 620)	<source1>,<source2> ::= CHANNEL<n> with N2820A probe connected <n> ::= 1 to (# analog channels) in NR1 format <return_value> ::= voltage peak-to-peak of the selected waveform in NR3 format
:MEASure:DUAL:VRMS [<interval>] [,<type>] [,<source1>] [,<source2>] >] (see page 621)	:MEASure:DUAL:VRMS? [<interval>] [,<type>] [,<source1>] [,<source2>] >] (see page 621)	<interval> ::= {CYCLE DISPLAY} <type> ::= {AC DC} <source1>,<source2> ::= CHANNEL<n> with N2820A probe connected <n> ::= 1 to (# analog channels) in NR1 format <return_value> ::= calculated RMS voltage in NR3 format

Table 22 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:DUTYcycle [<source>] (see page 622)	:MEASure:DUTYcycle? [<source>] (see page 622)	<source> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r>} for DSO models <source> ::= {CHANnel<n> DIGital<d> FUNction<m> MATH<m> WMEMory<r>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <return_value> ::= ratio of positive pulse width to period in NR3 format
:MEASure:FALLtime [<source>] (see page 623)	:MEASure:FALLtime? [<source>] (see page 623)	<source> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r>} for DSO models <source> ::= {CHANnel<n> DIGital<d> FUNction<m> MATH<m> WMEMory<r>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <return_value> ::= time in seconds between the lower and upper thresholds in NR3 format

Table 22 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:FFT:ACPR <chan_width>, <chan_spacing>, <chan>[,<source>] (see page 624)	:MEASure:FFT:ACPR? <chan_width>, <chan_spacing>, <chan>[,<source>] (see page 624)	<chan_width> ::= width of main range and sideband channels, Hz in NR3 format <chan_spacing> ::= spacing between main range and sideband channels, Hz in NR3 format <chan> ::= {CENTer HIGH<sb> LOW<sb>} <sb> ::= sideband 1 to 5 <source> ::= {FUNCTION<m> MATH<m> FFT} (source must be an FFT waveform) <m> ::= 1 to (# math functions) in NR1 format <return_value> ::= adjacent channel power ratio, dBV in NR3 format
:MEASure:FFT:CPOWer [<source>] (see page 625)	:MEASure:FFT:CPOWer? [<source>] (see page 625)	<source> ::= {FUNCTION<m> MATH<m> FFT} (source must be an FFT waveform) <m> ::= 1 to (# math functions) in NR1 format <return_value> ::= spectral channel power, dBV in NR3 format
:MEASure:FFT:OBW <percentage>[,<source>] >] (see page 626)	:MEASure:FFT:OBW? <percentage>[,<source>] >] (see page 626)	<percentage> ::= percent of spectral power occupied bandwidth is measured for in NR3 format <source> ::= {FUNCTION<m> MATH<m> FFT} (source must be an FFT waveform) <m> ::= 1 to (# math functions) in NR1 format <return_value> ::= occupied bandwidth, Hz in NR3 format

Table 22 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:FFT:THD [<source>] (see page 627)	:MEASure:FFT:THD? [<source>] (see page 627)	<source> ::= {FUNCTION<m> MATH<m> FFT} (source must be an FFT waveform) <m> ::= 1 to (# math functions) in NR1 format <return_value> ::= total harmonic distortion ratio percent in NR3 format
:MEASure:FREQuency [<source>] (see page 628)	:MEASure:FREQuency? [<source>] (see page 628)	<source> ::= {CHANnel<n> FUNCTION<m> MATH<m> WMemory<r>} for DSO models <source> ::= {CHANnel<n> DIGital<d> FUNCTION<m> MATH<m> WMemory<r>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <return_value> ::= frequency in Hertz in NR3 format
n/a	:MEASure:HISTogram:BWIDth? (see page 629)	<value> ::= bin width in NR3 format
n/a	:MEASure:HISTogram:HITS? (see page 630)	<value> ::= # of total hits in NR3 format
n/a	:MEASure:HISTogram:M1S? (see page 631)	<value> ::= % of hits within +/-1 std dev in NR3 format
n/a	:MEASure:HISTogram:M2S? (see page 632)	<value> ::= % of hits within +/-2 std devs in NR3 format
n/a	:MEASure:HISTogram:M3S? (see page 633)	<value> ::= % of hits within +/-3 std devs in NR3 format
n/a	:MEASure:HISTogram:MAXimum? (see page 634)	<value> ::= value of max bin in NR3 format
n/a	:MEASure:HISTogram:MEAN? (see page 635)	<value> ::= mean of histogram in NR3 format

Table 22 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
n/a	:MEASure:HISTogram:ME Dian? (see page 636)	<value> ::= median of histogram in NR3 format
n/a	:MEASure:HISTogram:MI Nimum? (see page 637)	<value> ::= value of min bin in NR3 format
n/a	:MEASure:HISTogram:MO DE? (see page 638)	<value> ::= mode of histogram in NR3 format
n/a	:MEASure:HISTogram:PE AK? (see page 639)	<value> ::= # of hits in max bin in NR3 format
n/a	:MEASure:HISTogram:PP Eak? (see page 640)	<value> ::= delta value between max bin and min bin in NR3 format
n/a	:MEASure:HISTogram:SD EViation? (see page 641)	<value> ::= standard deviation of histogram in NR3 format
:MEASure:NDUTy [<source>] (see page 642)	:MEASure:NDUTy? [<source>] (see page 642)	<source> ::= {CHANnel<n> FUNctIon<m> MATH<m> WMEMory<r>} for DSO models <source> ::= {CHANnel<n> DIGital<d> FUNctIon<m> MATH<m> WMEMory<r>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <return_value> ::= ratio of negative pulse width to period in NR3 format

Table 22 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:NEDGes [<source>] (see page 643)	:MEASure:NEDGes? [<source>] (see page 643)	<source> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= the falling edge count in NR3 format
:MEASure:NPULses [<source>] (see page 644)	:MEASure:NPULses? [<source>] (see page 644)	<source> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r> <digital channels>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <digital channels> ::= DIGital<d> for the MSO models <d> ::= 0 to (# digital channels - 1) in NR1 format <return_value> ::= the falling pulse count in NR3 format

Table 22 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:NWIDth [<source>] (see page 645)	:MEASure:NWIDth? [<source>] (see page 645)	<source> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r>} for DSO models <source> ::= {CHANnel<n> DIGital<d> FUNction<m> MATH<m> WMEMory<r>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <return_value> ::= negative pulse width in seconds-NR3 format
:MEASure:OVERshoot [<source>] (see page 646)	:MEASure:OVERshoot? [<source>] (see page 646)	<source> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= the percent of the overshoot of the selected waveform in NR3 format
:MEASure:PEDGes [<source>] (see page 648)	:MEASure:PEDGes? [<source>] (see page 648)	<source> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= the rising edge count in NR3 format

Table 22 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:PERiod [<source>] (see page 649)	:MEASure:PERiod? [<source>] (see page 649)	<source> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r>} for DSO models <source> ::= {CHANnel<n> DIGital<d> FUNction<m> MATH<m> WMEMory<r>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <return_value> ::= waveform period in seconds in NR3 format
:MEASure:PHASe [<source1>] [,<source2>] (see page 650)	:MEASure:PHASe? [<source1>] [,<source2>] (see page 650)	<source1,2> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= the phase angle value in degrees in NR3 format

Table 22 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:PPULses [<source>] (see page 651)	:MEASure:PPULses? [<source>] (see page 651)	<source> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r> <digital channels>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <digital channels> ::= DIGital<d> for the MSO models <d> ::= 0 to (# digital channels - 1) in NR1 format <return_value> ::= the rising pulse count in NR3 format
:MEASure:PREShoot [<source>] (see page 652)	:MEASure:PREShoot? [<source>] (see page 652)	<source> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= the percent of preshoot of the selected waveform in NR3 format

Table 22 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:PWIDth [<source>] (see page 653)	:MEASure:PWIDth? [<source>] (see page 653)	<source> ::= {CHANnel<n> FUNCTION<m> MATH<m> WMEMory<r>} for DSO models <source> ::= {CHANnel<n> DIGital<d> FUNCTION<m> MATH<m> WMEMory<r>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <return_value> ::= width of positive pulse in seconds in NR3 format
n/a	:MEASure:RESults? <result_list> (see page 654)	<result_list> ::= comma-separated list of measurement results
:MEASure:RISetime [<source>] (see page 657)	:MEASure:RISetime? [<source>] (see page 657)	<source> ::= {CHANnel<n> FUNCTION<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= rise time in seconds in NR3 format

Table 22 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:SDEviation [<source>] (see page 658)	:MEASure:SDEviation? [<source>] (see page 658)	<source> ::= {CHANnel<n> FUNctIon<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= calculated std deviation in NR3 format
:MEASure:SHOW {{0 OFF} {1 ON}} (see page 659)	:MEASure:SHOW? (see page 659)	{0 1}
:MEASure:SLEWrate [<source>[,<slope>]] (see page 660)	:MEASure:SLEWrate? [<source>[,<slope>]] (see page 660)	<source> ::= {CHANnel<n> FUNctIon<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# of analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format
:MEASure:SOURce <source1> [,<source2>] (see page 661)	:MEASure:SOURce? (see page 661)	<source1,2> ::= {CHANnel<n> FUNctIon<m> MATH<m> WMEMory<r> EXTErnal} for DSO models <source1,2> ::= {CHANnel<n> DIGital<d> FUNctIon<m> MATH<m> WMEMory<r> EXTErnal} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <return_value> ::= {<source> NONE}

Table 22 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:STATistics <type> (see page 663)	:MEASure:STATistics? (see page 663)	<type> ::= {{ON 1} CURRENT MEAN MINimum MAXimum STDDev COUNT} ON ::= all statistics returned
:MEASure:STATistics:DISPlay {{0 OFF} {1 ON}} (see page 664)	:MEASure:STATistics:DISPlay? (see page 664)	{0 1}
:MEASure:STATistics:INCRement (see page 665)	n/a	n/a
:MEASure:STATistics:MCOUNT <setting> (see page 666)	:MEASure:STATistics:MCOUNT? (see page 666)	<setting> ::= {INFinite <count>} <count> ::= 2 to 2000 in NR1 format
:MEASure:STATistics:RESET (see page 667)	n/a	n/a
:MEASure:STATistics:RSSDeviation {{0 OFF} {1 ON}} (see page 668)	:MEASure:STATistics:RSSDeviation? (see page 668)	{0 1}

Table 22 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:TEDGE [<slope>,<occurrence>] >[,<source>] (see page 669)	:MEASure:TEDGE? [<slope>,<occurrence>] >[,<source>] (see page 670)	<slope> ::= {RISing FALLing EITHER} <occurrence> ::= [+ -]<number> <number> ::= the edge number in NR1 format <source> ::= {<digital channels> CHANNEL<n> FUNCTION<m> MATH<m> WMemory<r>} <digital channels> ::= DIGital<d> for the MSO models <n> ::= 1 to (# of analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <return_value> ::= time in seconds of the specified transition

Table 22 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
n/a	:MEASure:TVALue? <value>, [<slope>]<occurrence> [,<source>] (see page 672)	<value> ::= voltage level that the waveform must cross. <slope> ::= direction of the waveform when <value> is crossed. <occurrence> ::= transitions reported. <source> ::= {CHANnel<n> FUNctIon<m> MATH<m> WMEMory<r>} for DSO models <source> ::= {CHANnel<n> DIGital<d> FUNctIon<m> MATH<m> WMEMory<r>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <return_value> ::= time in seconds of specified voltage crossing in NR3 format
:MEASure:VAMPlitude [<source>] (see page 674)	:MEASure:VAMPlitude? [<source>] (see page 674)	<source> ::= {CHANnel<n> FUNctIon<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= the amplitude of the selected waveform in volts in NR3 format

Table 22 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:VAVerage [<interval>] [, <source>] (see page 675)	:MEASure:VAVerage? [<interval>] [, <source>] (see page 675)	<interval> ::= {CYCLe DISPlay} <source> ::= {CHANnel<n> FUNCTION<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= calculated average voltage in NR3 format
:MEASure:VBASe [<source>] (see page 676)	:MEASure:VBASe? [<source>] (see page 676)	<source> ::= {CHANnel<n> FUNCTION<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <base_voltage> ::= voltage at the base of the selected waveform in NR3 format
:MEASure:VMAX [<source>] (see page 677)	:MEASure:VMAX? [<source>] (see page 677)	<source> ::= {CHANnel<n> FUNCTION<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= maximum voltage of the selected waveform in NR3 format

Table 22 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:VMIN [<source>] (see page 678)	:MEASure:VMIN? [<source>] (see page 678)	<source> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= minimum voltage of the selected waveform in NR3 format
:MEASure:VPP [<source>] (see page 679)	:MEASure:VPP? [<source>] (see page 679)	<source> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= voltage peak-to-peak of the selected waveform in NR3 format
:MEASure:VRATio [<interval>] [, <source 1>] [, <source2>] (see page 680)	:MEASure:VRATio? [<interval>] [, <source 1>] [, <source2>] (see page 680)	<interval> ::= {CYCLe DISPlay} <source1,2> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= the ratio value in dB in NR3 format

Table 22 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:VRMS [<interval>] [, <type>] [, <source>] (see page 681)	:MEASure:VRMS? [<interval>] [, <type>] [, <source>] (see page 681)	<interval> ::= {CYCLe DISPlay} <type> ::= {AC DC} <source> ::= {CHANnel<n> FUNCTION<m> MATH<m> WMEMemory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= calculated dc RMS voltage in NR3 format
:MEASure:VTOP [<source>] (see page 682)	:MEASure:VTOP? [<source>] (see page 682)	<source> ::= {CHANnel<n> FUNCTION<m> MATH<m> WMEMemory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= voltage at the top of the waveform in NR3 format
:MEASure:WINDow <type> (see page 683)	:MEASure:WINDow? (see page 683)	<type> ::= {MAIN ZOOM AUTO GATE}
:MEASure:XMAX [<source>] (see page 684)	:MEASure:XMAX? [<source>] (see page 684)	<source> ::= {CHANnel<n> FUNCTION<m> MATH<m> WMEMemory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= horizontal value of the maximum in NR3 format

Table 22 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:XMIN [<source>] (see page 685)	:MEASure:XMIN? [<source>] (see page 685)	<source> ::= {CHANnel<n> FUNCTION<m> MATH<m> WMEMemory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= horizontal value of the minimum in NR3 format
:MEASure:YATX <horiz_location>[,<source>] (see page 686)	:MEASure:YATX? <horiz_location>[,<source>] (see page 686)	<horiz_location> ::= displayed time from trigger in seconds in NR3 format <source> ::= {CHANnel<n> FUNCTION<m> MATH<m> WMEMemory<r>} for DSO models <source> ::= {CHANnel<n> DIGital<d> FUNCTION<m> MATH<m> WMEMemory<r>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <return_value> ::= voltage at the specified time in NR3 format

Table 23 :MEASure Power Commands Summary

Command	Query	Options and Query Returns
:MEASure:ANGLE [<source1>] [, <source2>] >] (see page 692)	:MEASure:ANGLE? [<source1>] [, <source2>] >] (see page 692)	<source1>, <source2> ::= {CHANnel<n>} <n> ::= 1 to (# analog channels) in NR1 format <return_value> ::= the power phase angle in degrees in NR3 format
:MEASure:APParent [<source1>] [, <source2>] >] (see page 693)	:MEASure:APParent? [<source1>] [, <source2>] >] (see page 693)	<source1>, <source2> ::= {CHANnel<n>} <n> ::= 1 to (# analog channels) in NR1 format <return_value> ::= the apparent power value in NR3 format
:MEASure:CPLoss [<source1>] [, <source2>] >] (see page 694)	:MEASure:CPLoss? [<source1>] [, <source2>] >] (see page 694)	<source1>, <source2> <source1> ::= {FUNCTION<m> MATH<m>} <source2> ::= {CHANnel<n>} <m> ::= 1 to (# math functions) in NR1 format <n> ::= 1 to (# analog channels) in NR1 format <return_value> ::= the switching loss per cycle watts value in NR3 format
:MEASure:CRESt [<source>] (see page 695)	:MEASure:CRESt? [<source>] (see page 695)	<source> ::= {CHANnel<n> FUNCTION<m> MATH<m>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <return_value> ::= the crest factor value in NR3 format
:MEASure:EFFiciency (see page 696)	:MEASure:EFFiciency? (see page 696)	<return_value> ::= percent value in NR3 format

Table 23 :MEASure Power Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:ELOSS [<source>] (see page 697)	:MEASure:ELOSS? [<source>] (see page 697)	<source> ::= {CHANnel<n> FUNCTION<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= the energy loss value in NR3 format
:MEASure:FACTOR [<source1>] [, <source2>] >] (see page 698)	:MEASure:FACTOR? [<source1>] [, <source2>] >] (see page 698)	<source1>, <source2> ::= {CHANnel<n>} <n> ::= 1 to (# analog channels) in NR1 format <return_value> ::= the power factor value in NR3 format
:MEASure:IPower (see page 699)	:MEASure:IPower? (see page 699)	<return_value> ::= the input power value in NR3 format
:MEASure:OFFTime [<source1>] [, <source2>] >] (see page 700)	:MEASure:OFFTime? [<source1>] [, <source2>] >] (see page 700)	<source1>, <source2> ::= {CHANnel<n>} <n> ::= 1 to (# analog channels) in NR1 format <return_value> ::= the time in seconds in NR3 format
:MEASure:ONTime [<source1>] [, <source2>] >] (see page 701)	:MEASure:ONTime? [<source1>] [, <source2>] >] (see page 701)	<source1>, <source2> ::= {CHANnel<n>} <n> ::= 1 to (# analog channels) in NR1 format <return_value> ::= the time in seconds in NR3 format
:MEASure:OPower (see page 702)	:MEASure:OPower? (see page 702)	<return_value> ::= the output power value in NR3 format

Table 23 :MEASure Power Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:PCURrent [<source>] (see page 703)	:MEASure:PCURrent? [<source>] (see page 703)	<source> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= the peak current value in NR3 format
:MEASure:PLOss [<source>] (see page 704)	:MEASure:PLOss? [<source>] (see page 704)	<source> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= the power loss value in NR3 format
:MEASure:RDson [<source1>] [, <source2 >] (see page 705)	:MEASure:RDson? [<source1>] [, <source2 >] (see page 705)	<source1>, <source2> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= the Rds(on) value in NR3 format
:MEASure:REACTive [<source1>] [, <source2 >] (see page 706)	:MEASure:REACTive? [<source1>] [, <source2 >] (see page 706)	<source1>, <source2> ::= {CHANnel<n>} <n> ::= 1 to (# analog channels) in NR1 format <return_value> ::= the reactive power value in NR3 format

Table 23 :MEASure Power Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:REAL [<source>] (see page 707)	:MEASure:REAL? [<source>] (see page 707)	<source> ::= {CHANnel<n> FUNctIon<m> MATH<m>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <return_value> ::= the real power value in NR3 format
:MEASure:RIPPlE [<source>] (see page 708)	:MEASure:RIPPlE? [<source>] (see page 708)	<source> ::= {CHANnel<n> FUNctIon<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= the output ripple value in NR3 format

Table 23 :MEASure Power Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:TRESponse [<source>] (see page 709)	:MEASure:TRESponse? [<source>] (see page 709)	<source> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= time in seconds for the overshoot to settle back into the band in NR3 format
:MEASure:VCEsat [<source>] (see page 710)	:MEASure:VCEsat? [<source>] (see page 710)	<source> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= the Vce(sat) value in NR3 format

Table 24 :MTESt Commands Summary

Command	Query	Options and Query Returns
:MTESt:ALL { {0 OFF} {1 ON} } (see page 716)	:MTESt:ALL? (see page 716)	{0 1}
:MTESt:AMASk:CREate (see page 717)	n/a	n/a
:MTESt:AMASk:SOURce <source> (see page 718)	:MTESt:AMASk:SOURce? (see page 718)	<source> ::= CHANnel<n> <n> ::= {1 2 3 4} for 4ch models <n> ::= {1 2} for 2ch models
:MTESt:AMASk:UNITs <units> (see page 719)	:MTESt:AMASk:UNITs? (see page 719)	<units> ::= {CURRent DIVisions}

Table 24 :MTESt Commands Summary (continued)

Command	Query	Options and Query Returns
:MTESt:AMASk:XDELta <value> (see page 720)	:MTESt:AMASk:XDELta? (see page 720)	<value> ::= X delta value in NR3 format
:MTESt:AMASk:YDELta <value> (see page 721)	:MTESt:AMASk:YDELta? (see page 721)	<value> ::= Y delta value in NR3 format
n/a	:MTESt:COUNT:FWAVEforms? [CHANnel<n>] (see page 722)	<failed> ::= number of failed waveforms in NR1 format
:MTESt:COUNT:RESet (see page 723)	n/a	n/a
n/a	:MTESt:COUNT:TIME? (see page 724)	<time> ::= elapsed seconds in NR3 format
n/a	:MTESt:COUNT:WAVEforms? (see page 725)	<count> ::= number of waveforms in NR1 format
:MTESt:DATA <mask> (see page 726)	:MTESt:DATA? (see page 726)	<mask> ::= data in IEEE 488.2 # format.
:MTESt:DELeTe (see page 727)	n/a	n/a
:MTESt:ENABLe {{0 OFF} {1 ON}} (see page 728)	:MTESt:ENABLe? (see page 728)	{0 1}
:MTESt:LOCK {{0 OFF} {1 ON}} (see page 729)	:MTESt:LOCK? (see page 729)	{0 1}
:MTESt:RMODE <rmode> (see page 730)	:MTESt:RMODE? (see page 730)	<rmode> ::= {FORever TIME SIGMa WAVEforms}
:MTESt:RMODE:FACTION:MEASure {{0 OFF} {1 ON}} (see page 731)	:MTESt:RMODE:FACTION:MEASure? (see page 731)	{0 1}
:MTESt:RMODE:FACTION:PRINT {{0 OFF} {1 ON}} (see page 732)	:MTESt:RMODE:FACTION:PRINT? (see page 732)	{0 1}
:MTESt:RMODE:FACTION:SAVE {{0 OFF} {1 ON}} (see page 733)	:MTESt:RMODE:FACTION:SAVE? (see page 733)	{0 1}
:MTESt:RMODE:FACTION:STOP {{0 OFF} {1 ON}} (see page 734)	:MTESt:RMODE:FACTION:STOP? (see page 734)	{0 1}

Table 24 :MTESt Commands Summary (continued)

Command	Query	Options and Query Returns
:MTESt:RMODE:SIGMa <level> (see page 735)	:MTESt:RMODE:SIGMa? (see page 735)	<level> ::= from 0.1 to 9.3 in NR3 format
:MTESt:RMODE:TIME <seconds> (see page 736)	:MTESt:RMODE:TIME? (see page 736)	<seconds> ::= from 1 to 86400 in NR3 format
:MTESt:RMODE:WAVEform s <count> (see page 737)	:MTESt:RMODE:WAVEform s? (see page 737)	<count> ::= number of waveforms in NR1 format
:MTESt:SCALE:BIND { {0 OFF} {1 ON} } (see page 738)	:MTESt:SCALE:BIND? (see page 738)	{0 1}
:MTESt:SCALE:X1 <x1_value> (see page 739)	:MTESt:SCALE:X1? (see page 739)	<x1_value> ::= X1 value in NR3 format
:MTESt:SCALE:XDELta <xdelta_value> (see page 740)	:MTESt:SCALE:XDELta? (see page 740)	<xdelta_value> ::= X delta value in NR3 format
:MTESt:SCALE:Y1 <y1_value> (see page 741)	:MTESt:SCALE:Y1? (see page 741)	<y1_value> ::= Y1 value in NR3 format
:MTESt:SCALE:Y2 <y2_value> (see page 742)	:MTESt:SCALE:Y2? (see page 742)	<y2_value> ::= Y2 value in NR3 format
:MTESt:SOURce <source> (see page 743)	:MTESt:SOURce? (see page 743)	<source> ::= {CHANnel<n> NONE} <n> ::= {1 2 3 4} for 4ch models <n> ::= {1 2} for 2ch models
n/a	:MTESt:TITLe? (see page 744)	<title> ::= a string of up to 128 ASCII characters

Table 25 :POD<n> Commands Summary

Command	Query	Options and Query Returns
:POD<n>:DISPlay { {0 OFF} {1 ON} } (see page 747)	:POD<n>:DISPlay? (see page 747)	{0 1} <n> ::= 1-2 in NR1 format

Table 25 :POD<n> Commands Summary (continued)

Command	Query	Options and Query Returns
:POD<n>:SIZE <value> (see page 748)	:POD<n>:SIZE? (see page 748)	<value> ::= {SMALL MEDIUM LARGE}
:POD<n>:THReshold <type>[suffix] (see page 749)	:POD<n>:THReshold? (see page 749)	<n> ::= 1-2 in NR1 format <type> ::= {CMOS ECL TTL <user defined value>} <user defined value> ::= value in NR3 format [suffix] ::= {V mV uV }

Table 26 :POWer Commands Summary

Command	Query	Options and Query Returns
n/a	:POWer:CLResponse? (see page 759)	n/a
:POWer:CLResponse:APP Ly (see page 760)	n/a	n/a
n/a	:POWer:CLResponse:DATA? [SWEep SINGLE] (see page 761)	<binary_block> ::= comma-separated data with newlines at the end of each row
n/a	:POWer:CLResponse:DATA:GMARgin? (see page 762)	<gain_margin> ::= gain margin in dB in NR3 format.
n/a	:POWer:CLResponse:DATA:GMARgin:FREQuency? (see page 763)	<frequency> ::= 0 degrees phase crossover frequency in Hz in NR3 format
n/a	:POWer:CLResponse:DATA:PMARgin? (see page 764)	<phase_margin> ::= phase margin in degrees in NR3 format.
n/a	:POWer:CLResponse:DATA:PMARgin:FREQuency? (see page 765)	<frequency> ::= 0dB gain crossover frequency in Hz in NR3 format.
:POWer:CLResponse:FREQuency:MODE <mode> (see page 766)	:POWer:CLResponse:FREQuency:MODE? (see page 766)	<mode> ::= {SWEep SINGLE}
:POWer:CLResponse:FREQuency:SINGLE <value>[suffix] (see page 767)	:POWer:CLResponse:FREQuency:SINGLE? (see page 767)	<value> ::= {20 100 1000 10000 100000 1000000 10000000 20000000} [suffix] ::= {Hz kHz MHz}

Table 26 :POWer Commands Summary (continued)

Command	Query	Options and Query Returns
:POWer:CLResponse:FREQuency:STARt <value>[suffix] (see page 768)	:POWer:CLResponse:FREQuency:STARt? (see page 768)	<value> ::= {20 100 1000 10000 100000 1000000 10000000} [suffix] ::= {Hz kHz MHz}
:POWer:CLResponse:FREQuency:STOP <value>[suffix] (see page 769)	:POWer:CLResponse:FREQuency:STOP? (see page 769)	<value> ::= {100 1000 10000 100000 1000000 10000000 20000000} [suffix] ::= {Hz kHz MHz}
:POWer:CLResponse:PPDeade <pts> (see page 770)	:POWer:CLResponse:PPDeade? (see page 770)	<pts> ::= {10 20 30 40 50 60 70 80 90 100}
:POWer:CLResponse:SOURce:INPut <source> (see page 771)	:POWer:CLResponse:SOURce:INPut? (see page 771)	<source> ::= CHANnel<n> <n> ::= 1 to (# analog channels) in NR1 format
:POWer:CLResponse:SOURce:OUTPut <source> (see page 772)	:POWer:CLResponse:SOURce:OUTPut? (see page 772)	<source> ::= CHANnel<n> <n> ::= 1 to (# analog channels) in NR1 format
:POWer:CLResponse:TRACe <selection> (see page 773)	:POWer:CLResponse:TRACe? (see page 773)	<selection> ::= {NONE ALL GAIN PHASe}[, {GAIN PHASe}]
:POWer:CLResponse:WGEN:LOAD <impedance> (see page 774)	:POWer:CLResponse:WGEN:LOAD? (see page 774)	<impedance> ::= {ONEMeg FIFTy}
:POWer:CLResponse:WGEN:VOLTagE <amplitude>[, <range>] (see page 775)	:POWer:CLResponse:WGEN:VOLTagE? [<range>] (see page 775)	<amplitude> ::= amplitude in volts in NR3 format <range> ::= {F20HZ F100HZ F1KHZ F10KHZ F100KHZ F1MHZ F10MHZ F20MHZ}
:POWer:CLResponse:WGEN:VOLTagE:PROFile { {0 OFF} {1 ON} } (see page 776)	:POWer:CLResponse:WGEN:VOLTagE:PROFile? (see page 776)	{0 1}
:POWer:DESKew (see page 777)	n/a	n/a
:POWer:EFFiciency:APPLy (see page 778)	n/a	n/a
:POWer:EFFiciency:TYPE <type> (see page 779)	:POWer:EFFiciency:TYPE? (see page 779)	<type> ::= {DCDC DCAC ACDC ACAC}

Table 26 :POWer Commands Summary (continued)

Command	Query	Options and Query Returns
:POWer:ENABle {{0 OFF} {1 ON}} (see page 780)	:POWer:ENABle? (see page 780)	{0 1}
:POWer:HARMonics:APPLy (see page 781)	n/a	n/a
n/a	:POWer:HARMonics:DATA? (see page 782)	<binary_block> ::= comma-separated data with newlines at the end of each row
:POWer:HARMonics:DISP lay <display> (see page 783)	:POWer:HARMonics:DISP lay? (see page 783)	<display> ::= {TABLE BAR OFF}
n/a	:POWer:HARMonics:FAIL count? (see page 784)	<count> ::= integer in NR1 format
:POWer:HARMonics:LINE <frequency> (see page 785)	:POWer:HARMonics:LINE ? (see page 785)	<frequency> ::= {F50 F60 F400 AUTO}
n/a	:POWer:HARMonics:POWe rfactor? (see page 786)	<value> ::= Class C power factor in NR3 format
:POWer:HARMonics:RPOW er <source> (see page 787)	:POWer:HARMonics:RPOW er? (see page 787)	<source> ::= {MEASured USER}
:POWer:HARMonics:RPOW er:USER <value> (see page 788)	:POWer:HARMonics:RPOW er:USER? (see page 788)	<value> ::= Watts from 1.0 to 600.0 in NR3 format
n/a	:POWer:HARMonics:RUNC out? (see page 789)	<count> ::= integer in NR1 format
:POWer:HARMonics:STAN dard <class> (see page 790)	:POWer:HARMonics:STAN dard? (see page 790)	<class> ::= {A B C D}
n/a	:POWer:HARMonics:STAT us? (see page 791)	<status> ::= {PASS FAIL UNTested}
n/a	:POWer:HARMonics:THD? (see page 792)	<value> ::= Total Harmonics Distortion in NR3 format
:POWer:INRush:APPLy (see page 793)	n/a	n/a
:POWer:INRush:EXIT (see page 794)	n/a	n/a

Table 26 :POWer Commands Summary (continued)

Command	Query	Options and Query Returns
:POWer:INRush:NEXT (see page 795)	n/a	n/a
:POWer:ITYPE <type> (see page 796)	:POWer:ITYPE? (see page 796)	<type> ::= {DC AC}
:POWer:MODulation:APPLy (see page 797)	n/a	n/a
:POWer:MODulation:SOURce <source> (see page 798)	:POWer:MODulation:SOURce? (see page 798)	<source> ::= {V I}
:POWer:MODulation:TYPE <modulation> (see page 799)	:POWer:MODulation:TYPE? (see page 799)	<modulation> ::= {VAverage ACRMs VRATio PERiod FREQuency PWIDith NWIDth DUTYcycle RISetime FALLtime}
:POWer:ONOff:APPLy (see page 800)	n/a	n/a
:POWer:ONOff:EXIT (see page 801)	n/a	n/a
:POWer:ONOff:NEXT (see page 802)	n/a	n/a
:POWer:ONOff:TEST { {0 OFF} {1 ON} } (see page 803)	:POWer:ONOff:TEST? (see page 803)	{0 1}
:POWer:ONOff:THReshols <type>, <input_thr>, <output_thr> (see page 804)	:POWer:ONOff:THReshols? <type> (see page 804)	<type> ::= {0 1} <input_thr> ::= percent from 0-100 in NR1 format <output_thr> ::= percent from 0-100 in NR1 format
n/a	:POWer:PSRR? (see page 806)	n/a
:POWer:PSRR:APPLy (see page 807)	n/a	n/a
n/a	:POWer:PSRR:DATA? [SWEep SINGLE] (see page 808)	<binary_block> ::= comma-separated data with newlines at the end of each row
:POWer:PSRR:FREQuency:MAXimum <value>[suffix] (see page 809)	:POWer:PSRR:FREQuency:MAXimum? (see page 809)	<value> ::= {10 100 1000 10000 100000 1000000 10000000 20000000} [suffix] ::= {Hz kHz MHz}

Table 26 :POWer Commands Summary (continued)

Command	Query	Options and Query Returns
:POWer:PSRR:FREQuency:MINimum <value>[suffix] (see page 810)	:POWer:PSRR:FREQuency:MINimum? (see page 810)	<value> ::= {1 10 100 1000 10000 100000 1000000 10000000} [suffix] ::= {Hz kHz MHz}
:POWer:PSRR:FREQuency:MODE <mode> (see page 811)	:POWer:PSRR:FREQuency:MODE? (see page 811)	<mode> ::= {SWEep SINGLE}
:POWer:PSRR:FREQuency:SINGLE <value>[suffix] (see page 812)	:POWer:PSRR:FREQuency:SINGLE? (see page 812)	<value> ::= {1 10 100 1000 10000 100000 1000000 10000000 20000000} [suffix] ::= {Hz kHz MHz}
:POWer:PSRR:PPDecade <pts> (see page 813)	:POWer:PSRR:PPDecade? (see page 813)	<pts> ::= {10 20 30 40 50 60 70 80 90 100}
:POWer:PSRR:SOURce:INPut <source> (see page 814)	:POWer:PSRR:SOURce:INPut? (see page 814)	<source> ::= CHANNEL<n> <n> ::= 1 to (# analog channels) in NR1 format
:POWer:PSRR:SOURce:OUTPut <source> (see page 815)	:POWer:PSRR:SOURce:OUTPut? (see page 815)	<source> ::= CHANNEL<n> <n> ::= 1 to (# analog channels) in NR1 format
:POWer:PSRR:TRACe <selection> (see page 816)	:POWer:PSRR:TRACe? (see page 816)	<selection> ::= {NONE GAIN}
:POWer:PSRR:WGEN:LOAD <impedance> (see page 817)	:POWer:PSRR:WGEN:LOAD? (see page 817)	<impedance> ::= {ONEMeg FIFTY}
:POWer:PSRR:WGEN:VOLTage <amplitude>[,<range>] (see page 818)	:POWer:PSRR:WGEN:VOLTage? [<range>] (see page 818)	<amplitude> ::= amplitude in volts in NR3 format <range> ::= {F20HZ F100HZ F1KHZ F10KHZ F100KHZ F1MHZ F10MHZ F20MHZ}
:POWer:PSRR:WGEN:VOLTage:PROFile {{0 OFF} {1 ON}} (see page 819)	:POWer:PSRR:WGEN:VOLTage:PROFile? (see page 819)	{0 1}
:POWer:QUALity:APPLY (see page 820)	n/a	n/a
:POWer:RIPPlE:APPLY (see page 821)	n/a	n/a

Table 26 :POWER Commands Summary (continued)

Command	Query	Options and Query Returns
:POWER:SIGNals:AUTose tup <analysis> (see page 822)	n/a	<analysis> ::= {HARMonics EFFiciency RIPple MODulation QUALity SLEW SWITCH RDSVce}
:POWER:SIGNals:CYCLes :HARMonics <count> (see page 823)	:POWER:SIGNals:CYCLes :HARMonics? (see page 823)	<count> ::= integer in NR1 format Legal values are 1 to 100.
:POWER:SIGNals:CYCLes :QUALity <count> (see page 824)	:POWER:SIGNals:CYCLes :QUALity? (see page 824)	<count> ::= integer in NR1 format Legal values are 1 to 100.
:POWER:SIGNals:DURati on:EFFiciency <value>[suffix] (see page 825)	:POWER:SIGNals:DURati on:EFFiciency? (see page 825)	<value> ::= value in NR3 format [suffix] ::= {s ms us ns}
:POWER:SIGNals:DURati on:MODulation <value>[suffix] (see page 826)	:POWER:SIGNals:DURati on:MODulation? (see page 826)	<value> ::= value in NR3 format [suffix] ::= {s ms us ns}
:POWER:SIGNals:DURati on:ONOff:OFF <value>[suffix] (see page 827)	:POWER:SIGNals:DURati on:ONOff:OFF? (see page 827)	<value> ::= value in NR3 format [suffix] ::= {s ms us ns}
:POWER:SIGNals:DURati on:ONOff:ON <value>[suffix] (see page 828)	:POWER:SIGNals:DURati on:ONOff:ON? (see page 828)	<value> ::= value in NR3 format [suffix] ::= {s ms us ns}
:POWER:SIGNals:DURati on:RIPple <value>[suffix] (see page 829)	:POWER:SIGNals:DURati on:RIPple? (see page 829)	<value> ::= value in NR3 format [suffix] ::= {s ms us ns}
:POWER:SIGNals:DURati on:TRANsient <value>[suffix] (see page 830)	:POWER:SIGNals:DURati on:TRANsient? (see page 830)	<value> ::= value in NR3 format [suffix] ::= {s ms us ns}
:POWER:SIGNals:IEXpec ted <value>[suffix] (see page 831)	:POWER:SIGNals:IEXpec ted? (see page 831)	<value> ::= Expected current value in NR3 format [suffix] ::= {A mA}
:POWER:SIGNals:OVERsh oot <percent> (see page 832)	:POWER:SIGNals:OVERsh oot? (see page 832)	<percent> ::= percent of overshoot value in NR1 format [suffix] ::= {V mV}}

Table 26 :POWER Commands Summary (continued)

Command	Query	Options and Query Returns
:POWER:SIGNals:VMAXimum:INRush <value>[suffix] (see page 833)	:POWER:SIGNals:VMAXimum:INRush? (see page 833)	<value> ::= Maximum expected input Voltage in NR3 format [suffix] ::= {V mV}
:POWER:SIGNals:VMAXimum:ONOff:OFF <value>[suffix] (see page 834)	:POWER:SIGNals:VMAXimum:ONOff:OFF? (see page 834)	<value> ::= Maximum expected input Voltage in NR3 format [suffix] ::= {V mV}
:POWER:SIGNals:VMAXimum:ONOff:ON <value>[suffix] (see page 835)	:POWER:SIGNals:VMAXimum:ONOff:ON? (see page 835)	<value> ::= Maximum expected input Voltage in NR3 format [suffix] ::= {V mV}
:POWER:SIGNals:VSTeady:ONOff:OFF <value>[suffix] (see page 836)	:POWER:SIGNals:VSTeady:ONOff:OFF? (see page 836)	<value> ::= Expected steady stage output Voltage value in NR3 format [suffix] ::= {V mV}
:POWER:SIGNals:VSTeady:ONOff:ON <value>[suffix] (see page 837)	:POWER:SIGNals:VSTeady:ONOff:ON? (see page 837)	<value> ::= Expected steady stage output Voltage value in NR3 format [suffix] ::= {V mV}
:POWER:SIGNals:VSTeady:TRANSient <value>[suffix] (see page 838)	:POWER:SIGNals:VSTeady:TRANSient? (see page 838)	<value> ::= Expected steady stage output Voltage value in NR3 format [suffix] ::= {V mV}
:POWER:SIGNals:SOURce:CURRENT<i> <source> (see page 839)	:POWER:SIGNals:SOURce:CURRENT<i>? (see page 839)	<i> ::= 1, 2 in NR1 format <source> ::= CHANnel<n> <n> ::= 1 to (# analog channels) in NR1 format
:POWER:SIGNals:SOURce:VOLTage<i> <source> (see page 840)	:POWER:SIGNals:SOURce:VOLTage<i>? (see page 840)	<i> ::= 1, 2 in NR1 format <source> ::= CHANnel<n> <n> ::= 1 to (# analog channels) in NR1 format
:POWER:SLEW:APPLy (see page 841)	n/a	n/a
:POWER:SLEW:SOURce <source> (see page 842)	:POWER:SLEW:SOURce? (see page 842)	<source> ::= {V I}
:POWER:SWITCh:APPLy (see page 843)	n/a	n/a

Table 26 :POWer Commands Summary (continued)

Command	Query	Options and Query Returns
:POWer:SWITCh:CONduct ion <conduction> (see page 844)	:POWer:SWITCh:CONduct ion? (see page 844)	<conduction> ::= {WAVEform RDS VCE}
:POWer:SWITCh:IREFere nce <percent> (see page 845)	:POWer:SWITCh:IREFere nce? (see page 845)	<percent> ::= percent in NR1 format
:POWer:SWITCh:RDS <value>[suffix] (see page 846)	:POWer:SWITCh:RDS? (see page 846)	<value> ::= Rds(on) value in NR3 format [suffix] ::= {OHM mOHM}
:POWer:SWITCh:VCE <value>[suffix] (see page 847)	:POWer:SWITCh:VCE? (see page 847)	<value> ::= Vce(sat) value in NR3 format [suffix] ::= {V mV}
:POWer:SWITCh:VREFere nce <percent> (see page 848)	:POWer:SWITCh:VREFere nce? (see page 848)	<percent> ::= percent in NR1 format
:POWer:TRANsient:APPL y (see page 849)	n/a	n/a
:POWer:TRANsient:EXIT (see page 850)	n/a	n/a
:POWer:TRANsient:IINi tial <value>[suffix] (see page 851)	:POWer:TRANsient:IINi tial? (see page 851)	<value> ::= Initial current value in NR3 format [suffix] ::= {A mA}
:POWer:TRANsient:INEW <value>[suffix] (see page 852)	:POWer:TRANsient:INEW ? (see page 852)	<value> ::= New current value in NR3 format [suffix] ::= {A mA}
:POWer:TRANsient:NEXT (see page 853)	n/a	n/a

Table 27 :RECall Commands Summary

Command	Query	Options and Query Returns
:RECall:ARBitrary:[ST ARt] [<file_spec>] [, <column>] [, <wavegen_id>] (see page 858)	n/a	<file_spec> ::= {<internal_loc> <file_name>} <column> ::= Column in CSV file to load. Column number starts from 1. <internal_loc> ::= 0-3; an integer in NR1 format <file_name> ::= quoted ASCII string <wavegen_id> ::= {WGEN1 WGEN2}
:RECall:DBC[:START] [<file_name>] [, <serialbus>] (see page 859)	n/a	<file_name> ::= quoted ASCII string If extension included in file name, it must be ".dbc". <serialbus> ::= {SBUS<n>} <n> ::= 1 to (# of serial bus) in NR1 format
:RECall:FIleName <base_name> (see page 860)	:RECall:FIleName? (see page 860)	<base_name> ::= quoted ASCII string
:RECall:LDF[:START] [<file_name>] [, <serialbus>] (see page 861)	n/a	<file_name> ::= quoted ASCII string If extension included in file name, it must be ".ldf". <serialbus> ::= {SBUS<n>} <n> ::= 1 to (# of serial bus) in NR1 format
:RECall:MASK[:START] [<file_spec>] (see page 862)	n/a	<file_spec> ::= {<internal_loc> <file_name>} <internal_loc> ::= 0-3; an integer in NR1 format <file_name> ::= quoted ASCII string
:RECall:PWD <path_name> (see page 863)	:RECall:PWD? (see page 863)	<path_name> ::= quoted ASCII string

Table 27 :RECall Commands Summary (continued)

Command	Query	Options and Query Returns
:RECall:SETup[:START] [<file_spec>] (see page 864)	n/a	<file_spec> ::= {<internal_loc> <file_name>} <internal_loc> ::= 0-9; an integer in NR1 format <file_name> ::= quoted ASCII string
:RECall:WMEMemory<r>[:START] [<file_name> <data>] (see page 865)	n/a	<r> ::= 1 to (# ref waveforms) in NR1 format <file_name> ::= quoted ASCII string If extension included in file name, it must be ".h5". <data> ::= binary block data in IEEE 488.2 # format

Table 28 :SAVE Commands Summary

Command	Query	Options and Query Returns
:SAVE:ARBitrary:[START] [<file_spec>] [, <wavegen_id>] (see page 871)	n/a	<file_spec> ::= {<internal_loc> <file_name>} <internal_loc> ::= 0-3; an integer in NR1 format <file_name> ::= quoted ASCII string <wavegen_id> ::= {WGEN1 WGEN2}
:SAVE:COMpliance:USB[:START] [<file_name>] (see page 872)	n/a	<file_name> ::= quoted ASCII string
:SAVE:FILEname <base_name> (see page 873)	:SAVE:FILEname? (see page 873)	<base_name> ::= quoted ASCII string
:SAVE:IMAGe[:START] [<file_name>] (see page 874)	n/a	<file_name> ::= quoted ASCII string
:SAVE:IMAGe:FACTors {{0 OFF} {1 ON}} (see page 875)	:SAVE:IMAGe:FACTors? (see page 875)	{0 1}
:SAVE:IMAGe:FORMat <format> (see page 876)	:SAVE:IMAGe:FORMat? (see page 876)	<format> ::= {{BMP BMP24bit} BMP8bit PNG NONE}

Table 28 :SAVE Commands Summary (continued)

Command	Query	Options and Query Returns
:SAVE:IMAGe:INKSaver {0 OFF} {1 ON} (see page 877)	:SAVE:IMAGe:INKSaver? (see page 877)	{0 1}
:SAVE:IMAGe:PALette <palette> (see page 878)	:SAVE:IMAGe:PALette? (see page 878)	<palette> ::= {COLor GRAYscale}
:SAVE:LISTer[:START] [<file_name>] (see page 879)	n/a	<file_name> ::= quoted ASCII string
:SAVE:MASK[:START] [<file_spec>] (see page 880)	n/a	<file_spec> ::= {<internal_loc> <file_name>} <internal_loc> ::= 0-3; an integer in NR1 format <file_name> ::= quoted ASCII string
:SAVE:MULTi[:START] [<file_name>] (see page 881)	n/a	<file_name> ::= quoted ASCII string
:SAVE:POWER[:START] [<file_name>] (see page 882)	n/a	<file_name> ::= quoted ASCII string
:SAVE:PWD <path_name> (see page 883)	:SAVE:PWD? (see page 883)	<path_name> ::= quoted ASCII string
:SAVE:RESults:[START] [<file_spec>] (see page 884)	n/a	<file_name> ::= quoted ASCII string
:SAVE:RESults:FORMat: CURSor {{0 OFF} {1 ON}} (see page 885)	:SAVE:RESults:FORMat: CURSor? (see page 885)	{0 1}
:SAVE:RESults:FORMat: HISTogram {{0 OFF} {1 ON}} (see page 886)	:SAVE:RESults:FORMat: HISTogram? (see page 886)	{0 1}
:SAVE:RESults:FORMat: MASK {{0 OFF} {1 ON}} (see page 887)	:SAVE:RESults:FORMat: MASK? (see page 887)	{0 1}
:SAVE:RESults:FORMat: MEASurement {{0 OFF} {1 ON}} (see page 888)	:SAVE:RESults:FORMat: MEASurement? (see page 888)	{0 1}

Table 28 :SAVE Commands Summary (continued)

Command	Query	Options and Query Returns
:SAVE:RESults:FORMat:SEARCh {{0 OFF} {1 ON}} (see page 889)	:SAVE:RESults:FORMat:SEARCh? (see page 889)	{0 1}
:SAVE:RESults:FORMat:SEGMENTed {{0 OFF} {1 ON}} (see page 890)	:SAVE:RESults:FORMat:SEGMENTed? (see page 890)	{0 1}
:SAVE:SETup[:START] [<file_spec>] (see page 891)	n/a	<file_spec> ::= {<internal_loc> <file_name>} <internal_loc> ::= 0-9; an integer in NR1 format <file_name> ::= quoted ASCII string
:SAVE:WAVEform[:START] [<file_name>] (see page 892)	n/a	<file_name> ::= quoted ASCII string
:SAVE:WAVEform:FORMat<format> (see page 893)	:SAVE:WAVEform:FORMat? (see page 893)	<format> ::= {ASCIixy CSV BINary NONE}
:SAVE:WAVEform:LENGth<length> (see page 894)	:SAVE:WAVEform:LENGth? (see page 894)	<length> ::= 100 to max. length; an integer in NR1 format
:SAVE:WAVEform:LENGth:MAX {{0 OFF} {1 ON}} (see page 895)	:SAVE:WAVEform:LENGth:MAX? (see page 895)	{0 1}
:SAVE:WAVEform:SEGMENTed <option> (see page 896)	:SAVE:WAVEform:SEGMENTed? (see page 896)	<option> ::= {ALL CURRENT}

Table 28 :SAVE Commands Summary (continued)

Command	Query	Options and Query Returns
:SAVE:WMEemory:SOURce <source> (see page 897)	:SAVE:WMEemory:SOURce? (see page 897)	<source> ::= {CHANnel<n> FUNction<m> MATH<m> WMEemory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format NOTE: Only ADD or SUBtract math operations can be saved as reference waveforms. <return_value> ::= <source>
:SAVE:WMEemory[:START] [<file_name>] (see page 898)	n/a	<file_name> ::= quoted ASCII string If extension included in file name, it must be ".h5".

Table 29 General :SBUS<n> Commands Summary

Command	Query	Options and Query Returns
:SBUS<n>:DISPlay {{0 OFF} {1 ON}} (see page 902)	:SBUS<n>:DISPlay? (see page 902)	{0 1}
:SBUS<n>:MODE <mode> (see page 903)	:SBUS<n>:MODE? (see page 903)	<mode> ::= {A429 CAN CXPI FLEXray I2S IIC LIN M1553 MANchester NRZ SENT SPI UART USB USBPd}

Table 30 :SBUS<n>:A429 Commands Summary

Command	Query	Options and Query Returns
:SBUS<n>:A429:AUToset up (see page 906)	n/a	n/a
:SBUS<n>:A429:BASE <base> (see page 907)	:SBUS<n>:A429:BASE? (see page 907)	<base> ::= {BINary HEX}
:SBUS<n>:A429:BAUDrat e <baudrate> (see page 908)	:SBUS<n>:A429:BAUDrat e? (see page 908)	<baudrate> ::= integer from 10000 to 1000000

Table 30 :SBUS<n>:A429 Commands Summary (continued)

Command	Query	Options and Query Returns
n/a	:SBUS<n>:A429:COUNT:ERROR? (see page 909)	<error_count> ::= integer in NR1 format
:SBUS<n>:A429:COUNT:RESET (see page 910)	n/a	n/a
n/a	:SBUS<n>:A429:COUNT:WORD? (see page 911)	<word_count> ::= integer in NR1 format
:SBUS<n>:A429:FORMAT<format> (see page 912)	:SBUS<n>:A429:FORMAT? (see page 912)	<format> ::= {LDSDi LDSSm LDATa}
:SBUS<n>:A429:SIGNAL<signal> (see page 913)	:SBUS<n>:A429:SIGNAL? (see page 913)	<signal> ::= {A B DIFFerential}
:SBUS<n>:A429:SOURCE<source> (see page 914)	:SBUS<n>:A429:SOURCE? (see page 914)	<source> ::= {CHANnel<n>} <n> ::= 1 to (# analog channels) in NR1 format
:SBUS<n>:A429:SPEED<speed> (see page 915)	:SBUS<n>:A429:SPEED? (see page 915)	<speed> ::= {LOW HIGH USER}
:SBUS<n>:A429:TRIGGER:LABEL <value> (see page 916)	:SBUS<n>:A429:TRIGGER:LABEL? (see page 916)	<value> ::= 8-bit integer in decimal, <hex>, <octal>, or <string> from 0-255 or "0xxx" (don't care) <hex> ::= #Hnn where n ::= {0,...,9 A,...,F} <octal> ::= #Qnnn where n ::= {0,...,7} <string> ::= "0xnn" where n ::= {0,...,9 A,...,F}
:SBUS<n>:A429:TRIGGER:PATTERN:DATA<string> (see page 917)	:SBUS<n>:A429:TRIGGER:PATTERN:DATA? (see page 917)	<string> ::= "nn...n" where n ::= {0 1 X}, length depends on FORMAT
:SBUS<n>:A429:TRIGGER:PATTERN:SDI<string> (see page 918)	:SBUS<n>:A429:TRIGGER:PATTERN:SDI? (see page 918)	<string> ::= "nn" where n ::= {0 1 X}, length always 2 bits
:SBUS<n>:A429:TRIGGER:PATTERN:SSM<string> (see page 919)	:SBUS<n>:A429:TRIGGER:PATTERN:SSM? (see page 919)	<string> ::= "nn" where n ::= {0 1 X}, length always 2 bits

Table 30 :SBUS<n>:A429 Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:A429:TRIGger :RANGe <min>,<max> (see page 920)	:SBUS<n>:A429:TRIGger :RANGe? (see page 920)	<min> ::= 8-bit integer in decimal, <hex>, <octal>, or <string> from 0-255 <max> ::= 8-bit integer in decimal, <hex>, <octal>, or <string> from 0-255 <hex> ::= #Hnn where n ::= {0,...,9 A,...,F} <octal> ::= #Qnnn where n ::= {0,...,7} <string> ::= "0xnn" where n ::= {0,...,9 A,...,F}
:SBUS<n>:A429:TRIGger :TYPE <condition> (see page 921)	:SBUS<n>:A429:TRIGger :TYPE? (see page 921)	<condition> ::= {WSTArt WSTOp LABEL LBITs PERRor WERRor GERRor WGERrors ALLerrors LRANGe ABITs AOBits AZBits}

Table 31 :SBUS<n>:CAN Commands Summary

Command	Query	Options and Query Returns
n/a	:SBUS<n>:CAN:COUNT:ERRor? (see page 926)	<frame_count> ::= integer in NR1 format
n/a	:SBUS<n>:CAN:COUNT:OVERload? (see page 927)	<frame_count> ::= 0 in NR1 format
:SBUS<n>:CAN:COUNT:RESet (see page 928)	n/a	n/a
n/a	:SBUS<n>:CAN:COUNT:SPEC? (see page 929)	<spec_error_count> ::= integer in NR1 format
n/a	:SBUS<n>:CAN:COUNT:TOTAL? (see page 930)	<frame_count> ::= integer in NR1 format
n/a	:SBUS<n>:CAN:COUNT:UTILization? (see page 931)	<percent> ::= floating-point in NR3 format
:SBUS<n>:CAN:DISPlay<type> (see page 932)	:SBUS<n>:CAN:DISPlay? (see page 932)	<type> ::= {HEXadecimal SYMBOLic}
:SBUS<n>:CAN:FDSPoint<value> (see page 933)	:SBUS<n>:CAN:FDSPoint? (see page 933)	<value> ::= even numbered percentages from 30 to 90 in NR3 format.

Table 31 :SBUS<n>:CAN Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:CAN:FDSTandard <std> (see page 934)	:SBUS<n>:CAN:FDSTandard? (see page 934)	<std> ::= {ISO NISO}
:SBUS<n>:CAN:SAMPLEpoint <percent> (see page 935)	:SBUS<n>:CAN:SAMPLEpoint? (see page 935)	<percent> ::= 30.0 to 90.0 in NR3 format
:SBUS<n>:CAN:SIGNAL:BAUDrate <baudrate> (see page 936)	:SBUS<n>:CAN:SIGNAL:BAUDrate? (see page 936)	<baudrate> ::= integer from 10000 to 4000000 in 100 b/s increments, or 5000000
:SBUS<n>:CAN:SIGNAL:DEFinition <value> (see page 937)	:SBUS<n>:CAN:SIGNAL:DEFinition? (see page 937)	<value> ::= {CANH CANL RX TX DIFFerential DIFL DIFH}
:SBUS<n>:CAN:SIGNAL:FDBaudrate <baudrate> (see page 938)	:SBUS<n>:CAN:SIGNAL:FDBaudrate? (see page 938)	<baudrate> ::= integer from 10000 to 10000000 in 100 b/s increments.
:SBUS<n>:CAN:SOURCE <source> (see page 939)	:SBUS<n>:CAN:SOURCE? (see page 939)	<source> ::= {CHANNEL<n> EXTERNAL} for DSO models <source> ::= {CHANNEL<n> DIGital<d> } for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:SBUS<n>:CAN:TRIGGER <condition> (see page 940)	:SBUS<n>:CAN:TRIGGER? (see page 941)	<condition> ::= {SOF EOF IDData DATA FDData IDRemote IDEither ERROR ACKerror FORMerror STUFFerror CRCerror SPECerror ALLerrors BRSBit CRCDbit EBActive EBPActive OVERload MESSAGE MSIGNAL FDMSignal}
:SBUS<n>:CAN:TRIGGER:IDFilter {{0 OFF} {1 ON}} (see page 943)	:SBUS<n>:CAN:TRIGGER:IDFilter? (see page 943)	{0 1}
:SBUS<n>:CAN:TRIGGER:PATTERN:DATA <string> (see page 944)	:SBUS<n>:CAN:TRIGGER:PATTERN:DATA? (see page 944)	<string> ::= "nn...n" where n ::= {0 1 X \$} <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X \$}

Table 31 :SBUS<n>:CAN Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:CAN:TRIGger: PATtern:DATA:DLC <dlc> (see page 945)	:SBUS<n>:CAN:TRIGger: PATtern:DATA:DLC? (see page 945)	<dlc> ::= integer between -1 (don't care) and 64, in NR1 format.
:SBUS<n>:CAN:TRIGger: PATtern:DATA:LENGth <length> (see page 946)	:SBUS<n>:CAN:TRIGger: PATtern:DATA:LENGth? (see page 946)	<length> ::= integer from 1 to 8 in NR1 format
:SBUS<n>:CAN:TRIGger: PATtern:DATA:STARt <start> (see page 947)	:SBUS<n>:CAN:TRIGger: PATtern:DATA:STARt? (see page 947)	<start> ::= integer between 0 and 63, in NR1 format.
:SBUS<n>:CAN:TRIGger: PATtern:ID <string> (see page 948)	:SBUS<n>:CAN:TRIGger: PATtern:ID? (see page 948)	<string> ::= "nn...n" where n ::= {0 1 X \$} <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X \$}
:SBUS<n>:CAN:TRIGger: PATtern:ID:MODE <value> (see page 949)	:SBUS<n>:CAN:TRIGger: PATtern:ID:MODE? (see page 949)	<value> ::= {STANdard EXTended}
:SBUS<n>:CAN:TRIGger: SYMBolic:MESSAge <name> (see page 950)	:SBUS<n>:CAN:TRIGger: SYMBolic:MESSAge? (see page 950)	<name> ::= quoted ASCII string
:SBUS<n>:CAN:TRIGger: SYMBolic:SIGNal <name> (see page 951)	:SBUS<n>:CAN:TRIGger: SYMBolic:SIGNal? (see page 951)	<name> ::= quoted ASCII string
:SBUS<n>:CAN:TRIGger: SYMBolic:VALue <data> (see page 952)	:SBUS<n>:CAN:TRIGger: SYMBolic:VALue? (see page 952)	<data> ::= value in NR3 format

Table 32 :SBUS<n>:CXPI Commands Summary

Command	Query	Options and Query Returns
:SBUS<n>:CXPI:BAUDrat e <baudrate> (see page 955)	:SBUS<n>:CXPI:BAUDrat e? (see page 955)	<baudrate> ::= integer from 9600 to 40000 in 100 b/s increments.
:SBUS<n>:CXPI:PARity { {0 OFF} {1 ON} } (see page 956)	:SBUS<n>:CXPI:PARity? (see page 956)	{0 1}
:SBUS<n>:CXPI:SOURce <source> (see page 957)	:SBUS<n>:CXPI:SOURce? (see page 957)	<source> ::= {CHANnel<n>} <n> ::= 1 to (# analog channels) in NR1 format

Table 32 :SBUS<n>:CXPI Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:CXPI:TOLerance <percent> (see page 958)	:SBUS<n>:CXPI:TOLerance? (see page 958)	<percent> ::= from 1-30, in NR1 format.
:SBUS<n>:CXPI:TRIGger <mode> (see page 959)	:SBUS<n>:CXPI:TRIGger? (see page 960)	<mode> ::= {SOF EOF PType ID DATA LDATA CRCerror PARityerror IBSerror IFSerror FRAMingerror DLENgtherror SAMPlerror ALLeerrors SLEepframe WAKEuppulse}
:SBUS<n>:CXPI:TRIGger:IDFilter {{0 OFF} {1 ON}} (see page 961)	:SBUS<n>:CXPI:TRIGger:IDFilter? (see page 961)	{0 1}
:SBUS<n>:CXPI:TRIGger:PTYPE {{0 OFF} {1 ON}} (see page 962)	:SBUS<n>:CXPI:TRIGger:PTYPE? (see page 962)	{0 1}
:SBUS<n>:CXPI:TRIGger:PATtern:DATA <string> (see page 963)	:SBUS<n>:CXPI:TRIGger:PATtern:DATA? (see page 963)	<string> ::= "nn...n" where n ::= {0 1 X} <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X}
:SBUS<n>:CXPI:TRIGger:PATtern:DATA:LENGth <length> (see page 964)	:SBUS<n>:CXPI:TRIGger:PATtern:DATA:LENGth? (see page 964)	<start> ::= integer between 0 and 12, in NR1 format.
:SBUS<n>:CXPI:TRIGger:PATtern:DATA:START <start> (see page 965)	:SBUS<n>:CXPI:TRIGger:PATtern:DATA:START? (see page 965)	<start> ::= integer between 0 and 124, in NR1 format.
:SBUS<n>:CXPI:TRIGger:PATtern:ID <string> (see page 966)	:SBUS<n>:CXPI:TRIGger:PATtern:ID? (see page 966)	<string> ::= "nn...n" where n ::= {0 1 X} <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X}
:SBUS<n>:CXPI:TRIGger:PATtern:INFO:CT <string> (see page 967)	:SBUS<n>:CXPI:TRIGger:PATtern:INFO:CT? (see page 967)	<string> ::= "nn" where n ::= {0 1 X}

Table 32 :SBUS<n>:CXPI Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:CXPI:TRIGger :PATtern:INFO:DLC <dlc> (see page 968)	:SBUS<n>:CXPI:TRIGger :PATtern:INFO:DLC? (see page 968)	<dlc> ::= integer between -1 (don't care) and 15, in NR1 format, when trigger is in DATA mode. <dlc> ::= integer between -1 (don't care) and 255, in NR1 format, when trigger is in LDATa mode.
:SBUS<n>:CXPI:TRIGger :PATtern:INFO:Nm <string> (see page 969)	:SBUS<n>:CXPI:TRIGger :PATtern:INFO:Nm? (see page 969)	<string> ::= "nn" where n ::= {0 1 X}

Table 33 :SBUS<n>:FLEXray Commands Summary

Command	Query	Options and Query Returns
:SBUS<n>:FLEXray:AUTo setup (see page 972)	n/a	n/a
:SBUS<n>:FLEXray:BAUD rate <baudrate> (see page 973)	:SBUS<n>:FLEXray:BAUD rate? (see page 973)	<baudrate> ::= {2500000 5000000 10000000}
:SBUS<n>:FLEXray:CHAN nel <channel> (see page 974)	:SBUS<n>:FLEXray:CHAN nel? (see page 974)	<channel> ::= {A B}
n/a	:SBUS<n>:FLEXray:COUN t:NULL? (see page 975)	<frame_count> ::= integer in NR1 format
:SBUS<n>:FLEXray:COUN t:RESet (see page 976)	n/a	n/a
n/a	:SBUS<n>:FLEXray:COUN t:SYNC? (see page 977)	<frame_count> ::= integer in NR1 format
n/a	:SBUS<n>:FLEXray:COUN t:TOTal? (see page 978)	<frame_count> ::= integer in NR1 format
:SBUS<n>:FLEXray:SOUR ce <source> (see page 979)	:SBUS<n>:FLEXray:SOUR ce? (see page 979)	<source> ::= {CHANnel<n>} <n> ::= 1-2 or 1-4 in NR1 format
:SBUS<n>:FLEXray:TRIG ger <condition> (see page 980)	:SBUS<n>:FLEXray:TRIG ger? (see page 980)	<condition> ::= {FRAME ERRor EVENT}

Table 33 :SBUS<n>:FLEXray Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:FLEXray:TRIGGER:ERROR:TYPE <error_type> (see page 981)	:SBUS<n>:FLEXray:TRIGGER:ERROR:TYPE? (see page 981)	<error_type> ::= {ALL HCRC FCRC}
:SBUS<n>:FLEXray:TRIGGER:EVENT:AUTOset (see page 982)	n/a	n/a
:SBUS<n>:FLEXray:TRIGGER:EVENT:BSS:ID <frame_id> (see page 983)	:SBUS<n>:FLEXray:TRIGGER:EVENT:BSS:ID? (see page 983)	<frame_id> ::= {ALL <frame #>} <frame #> ::= integer from 1-2047
:SBUS<n>:FLEXray:TRIGGER:EVENT:TYPE <event> (see page 984)	:SBUS<n>:FLEXray:TRIGGER:EVENT:TYPE? (see page 984)	<event> ::= {WAKEup TSS {FES DTS} BSS}
:SBUS<n>:FLEXray:TRIGGER:FRAME:CCBase <cycle_count_base> (see page 985)	:SBUS<n>:FLEXray:TRIGGER:FRAME:CCBase? (see page 985)	<cycle_count_base> ::= integer from 0-63
:SBUS<n>:FLEXray:TRIGGER:FRAME:CCRepetition <cycle_count_repetition> (see page 986)	:SBUS<n>:FLEXray:TRIGGER:FRAME:CCRepetition? (see page 986)	<cycle_count_repetition> ::= {ALL <rep #>} <rep #> ::= integer values 2, 4, 8, 16, 32, or 64
:SBUS<n>:FLEXray:TRIGGER:FRAME:ID <frame_id> (see page 987)	:SBUS<n>:FLEXray:TRIGGER:FRAME:ID? (see page 987)	<frame_id> ::= {ALL <frame #>} <frame #> ::= integer from 1-2047
:SBUS<n>:FLEXray:TRIGGER:FRAME:TYPE <frame_type> (see page 988)	:SBUS<n>:FLEXray:TRIGGER:FRAME:TYPE? (see page 988)	<frame_type> ::= {NORMAL STARTup NULL SYNC NSTArtup NNULl NSYNc ALL}

Table 34 :SBUS<n>:I2S Commands Summary

Command	Query	Options and Query Returns
:SBUS<n>:I2S:ALIGNment <setting> (see page 992)	:SBUS<n>:I2S:ALIGNment? (see page 992)	<setting> ::= {I2S LJ RJ}
:SBUS<n>:I2S:BASE <base> (see page 993)	:SBUS<n>:I2S:BASE? (see page 993)	<base> ::= {DECimal HEX}

Table 34 :SBUS<n>:I2S Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:I2S:CLOCK:SL OPe <slope> (see page 994)	:SBUS<n>:I2S:CLOCK:SL OPe? (see page 994)	<slope> ::= {NEGative POSitive}
:SBUS<n>:I2S:RWIDTH <receiver> (see page 995)	:SBUS<n>:I2S:RWIDTH? (see page 995)	<receiver> ::= 4-32 in NR1 format
:SBUS<n>:I2S:SOURce:C LOCK <source> (see page 996)	:SBUS<n>:I2S:SOURce:C LOCK? (see page 996)	<source> ::= {CHANnel<n> EXTernal} for DSO models <source> ::= {CHANnel<n> DIGital<d> } for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:SBUS<n>:I2S:SOURce:D ATA <source> (see page 997)	:SBUS<n>:I2S:SOURce:D ATA? (see page 997)	<source> ::= {CHANnel<n> EXTernal} for DSO models <source> ::= {CHANnel<n> DIGital<d> } for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:SBUS<n>:I2S:SOURce:W SElect <source> (see page 998)	:SBUS<n>:I2S:SOURce:W SElect? (see page 998)	<source> ::= {CHANnel<n> EXTernal} for DSO models <source> ::= {CHANnel<n> DIGital<d> } for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:SBUS<n>:I2S:TRIGger <operator> (see page 999)	:SBUS<n>:I2S:TRIGger? (see page 999)	<operator> ::= {EQUal NOTequal LESSthan GREATERthan INRange OUTRange INCReasing DECReasing}
:SBUS<n>:I2S:TRIGger: AUDio <audio_ch> (see page 1001)	:SBUS<n>:I2S:TRIGger: AUDio? (see page 1001)	<audio_ch> ::= {RIGHT LEFT EITHer}

Table 34 :SBUS<n>:I2S Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:I2S:TRIGger:PATtern:DATA <string> (see page 1002)	:SBUS<n>:I2S:TRIGger:PATtern:DATA? (see page 1003)	<string> ::= "n" where n ::= 32-bit integer in signed decimal when <base> = DECimal <string> ::= "nn...n" where n ::= {0 1 X \$} when <base> = BINary <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X \$} when <base> = HEX
:SBUS<n>:I2S:TRIGger:PATtern:FORMat <base> (see page 1004)	:SBUS<n>:I2S:TRIGger:PATtern:FORMat? (see page 1004)	<base> ::= {BINary HEX DECimal}
:SBUS<n>:I2S:TRIGger:RANGe <lower>,<upper> (see page 1005)	:SBUS<n>:I2S:TRIGger:RANGe? (see page 1005)	<lower> ::= 32-bit integer in signed decimal, <nondecimal>, or <string> <upper> ::= 32-bit integer in signed decimal, <nondecimal>, or <string> <nondecimal> ::= #Hnn...n where n ::= {0,...,9 A,...,F} for hexadecimal <nondecimal> ::= #Bnn...n where n ::= {0 1} for binary <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F} for hexadecimal
:SBUS<n>:I2S:TWIDth <word_size> (see page 1006)	:SBUS<n>:I2S:TWIDth? (see page 1006)	<word_size> ::= 4-32 in NR1 format
:SBUS<n>:I2S:WSLow <low_def> (see page 1007)	:SBUS<n>:I2S:WSLow? (see page 1007)	<low_def> ::= {LEFT RIGHT}

Table 35 :SBUS<n>:IIC Commands Summary

Command	Query	Options and Query Returns
:SBUS<n>:IIC:ASize <size> (see page 1010)	:SBUS<n>:IIC:ASize? (see page 1010)	<size> ::= {BIT7 BIT8}
:SBUS<n>:IIC[:SOURce] :CLOCK <source> (see page 1011)	:SBUS<n>:IIC[:SOURce] :CLOCK? (see page 1011)	<source> ::= {CHANnel<n> EXTeRnal} for DSO models <source> ::= {CHANnel<n> DIGital<d> } for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:SBUS<n>:IIC[:SOURce] :DATA <source> (see page 1012)	:SBUS<n>:IIC[:SOURce] :DATA? (see page 1012)	<source> ::= {CHANnel<n> EXTeRnal} for DSO models <source> ::= {CHANnel<n> DIGital<d> } for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:SBUS<n>:IIC:TRIGger: PATTeRn:ADDReSS <value> (see page 1013)	:SBUS<n>:IIC:TRIGger: PATTeRn:ADDReSS? (see page 1013)	<value> ::= integer or <string> <string> ::= "0xnn" n ::= {0,...,9 A,...,F}
:SBUS<n>:IIC:TRIGger: PATTeRn:DATA <value> (see page 1014)	:SBUS<n>:IIC:TRIGger: PATTeRn:DATA? (see page 1014)	<value> ::= integer or <string> <string> ::= "0xnn" n ::= {0,...,9 A,...,F}
:SBUS<n>:IIC:TRIGger: PATTeRn:DATA2 <value> (see page 1015)	:SBUS<n>:IIC:TRIGger: PATTeRn:DATA2? (see page 1015)	<value> ::= integer or <string> <string> ::= "0xnn" n ::= {0,...,9 A,...,F}
:SBUS<n>:IIC:TRIGger: QUALifier <value> (see page 1016)	:SBUS<n>:IIC:TRIGger: QUALifier? (see page 1016)	<value> ::= {EQUal NOTequal LESSthan GREATERthan}
:SBUS<n>:IIC:TRIGger[:TYPE] <type> (see page 1017)	:SBUS<n>:IIC:TRIGger[:TYPE]? (see page 1017)	<type> ::= {START STOP REStart ADDReSS ANACK DNACK NACKnowledge READEprom READ7 WRITe7 R7Data2 W7Data2 WRITe10}

Table 36 :SBUS<n>:LIN Commands Summary

Command	Query	Options and Query Returns
:SBUS<n>:LIN:DISPlay <type> (see page 1021)	:SBUS<n>:LIN:DISPlay? (see page 1021)	<type> ::= {HEXadecimal SYMBolic}
:SBUS<n>:LIN:PARity { {0 OFF} {1 ON} } (see page 1022)	:SBUS<n>:LIN:PARity? (see page 1022)	{0 1}
:SBUS<n>:LIN:SAMPlEpo int <value> (see page 1023)	:SBUS<n>:LIN:SAMPlEpo int? (see page 1023)	<value> ::= {60 62.5 68 70 75 80 87.5} in NR3 format
:SBUS<n>:LIN:SIGNAL:B AUDrate <baudrate> (see page 1024)	:SBUS<n>:LIN:SIGNAL:B AUDrate? (see page 1024)	<baudrate> ::= integer from 2400 to 625000 in 100 b/s increments
:SBUS<n>:LIN:SOURce <source> (see page 1025)	:SBUS<n>:LIN:SOURce? (see page 1025)	<source> ::= {CHANnel<n> EXTernal} for DSO models <source> ::= {CHANnel<n> DIGital<d>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:SBUS<n>:LIN:STANDard <std> (see page 1026)	:SBUS<n>:LIN:STANDard ? (see page 1026)	<std> ::= {LIN13 LIN13NLC LIN20}
:SBUS<n>:LIN:SYNCbrea k <value> (see page 1027)	:SBUS<n>:LIN:SYNCbrea k? (see page 1027)	<value> ::= integer = {11 12 13}
:SBUS<n>:LIN:TRIGger <condition> (see page 1028)	:SBUS<n>:LIN:TRIGger? (see page 1028)	<condition> ::= {SYNCbreak ID DATA}
:SBUS<n>:LIN:TRIGger: ID <value> (see page 1030)	:SBUS<n>:LIN:TRIGger: ID? (see page 1030)	<value> ::= 7-bit integer in decimal, <nondecimal>, or <string> from 0-63 or 0x00-0x3f <nondecimal> ::= #Hnn where n ::= {0,...,9 A,...,F} for hexadecimal <nondecimal> ::= #Bnn...n where n ::= {0 1} for binary <string> ::= "0xnn" where n ::= {0,...,9 A,...,F} for hexadecimal

Table 36 :SBUS<n>:LIN Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:LIN:TRIGger:PATtern:DATA <string> (see page 1031)	:SBUS<n>:LIN:TRIGger:PATtern:DATA? (see page 1031)	<string> ::= "n" where n ::= 32-bit integer in unsigned decimal when <base> = DECimal <string> ::= "nn...n" where n ::= {0 1 X \$} when <base> = BINary <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X \$} when <base> = HEX
:SBUS<n>:LIN:TRIGger:PATtern:DATA:LENGth <length> (see page 1033)	:SBUS<n>:LIN:TRIGger:PATtern:DATA:LENGth? (see page 1033)	<length> ::= integer from 1 to 8 in NR1 format
:SBUS<n>:LIN:TRIGger:PATtern:FORMat <base> (see page 1034)	:SBUS<n>:LIN:TRIGger:PATtern:FORMat? (see page 1034)	<base> ::= {BINary HEX DECimal}
:SBUS<n>:LIN:TRIGger:SYMBolic:FRAME <name> (see page 1035)	:SBUS<n>:LIN:TRIGger:SYMBolic:FRAME? (see page 1035)	<name> ::= quoted ASCII string
:SBUS<n>:LIN:TRIGger:SYMBolic:SIGNAL <name> (see page 1036)	:SBUS<n>:LIN:TRIGger:SYMBolic:SIGNAL? (see page 1036)	<name> ::= quoted ASCII string
:SBUS<n>:LIN:TRIGger:SYMBolic:VALue <data> (see page 1037)	:SBUS<n>:LIN:TRIGger:SYMBolic:VALue? (see page 1037)	<data> ::= value in NR3 format

Table 37 :SBUS<n>:M1553 Commands Summary

Command	Query	Options and Query Returns
:SBUS<n>:M1553:AUTOsetup (see page 1039)	n/a	n/a
:SBUS<n>:M1553:BASE <base> (see page 1040)	:SBUS<n>:M1553:BASE? (see page 1040)	<base> ::= {BINary HEX}
:SBUS<n>:M1553:SOURce <source> (see page 1041)	:SBUS<n>:M1553:SOURce? (see page 1041)	<source> ::= {CHANnel<n>} <n> ::= 1 to (# analog channels) in NR1 format
:SBUS<n>:M1553:TRIGger:PATtern:DATA <string> (see page 1042)	:SBUS<n>:M1553:TRIGger:PATtern:DATA? (see page 1042)	<string> ::= "nn...n" where n ::= {0 1 X}

Table 37 :SBUS<n>:M1553 Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:M1553:TRIGger:RTA <value> (see page 1043)	:SBUS<n>:M1553:TRIGger:RTA? (see page 1043)	<value> ::= 5-bit integer in decimal, <nondecimal>, or <string> from 0-31 <nondecimal> ::= #Hnn where n ::= {0,...,9 A,...,F} <string> ::= "0xnn" where n ::= {0,...,9 A,...,F}
:SBUS<n>:M1553:TRIGger:TYPE <type> (see page 1044)	:SBUS<n>:M1553:TRIGger:TYPE? (see page 1044)	<type> ::= {DSTArt DSTOp CSTArt CSTOp RTA PERRor SERRor MERRor RTA11}

Table 38 :SBUS<n>:MANChesTer Commands Summary

Command	Query	Options and Query Returns
:SBUS<n>:MANChesTer:BASE <base> (see page 1047)	:SBUS<n>:MANChesTer:BASE? (see page 1047)	<base> ::= {HEX DECimal ASCii}
:SBUS<n>:MANChesTer:BAUDrate <baudrate> (see page 1048)	:SBUS<n>:MANChesTer:BAUDrate? (see page 1048)	<baudrate> ::= integer from 500 to 5000000 in 100 b/s increments
:SBUS<n>:MANChesTer:BITorder <bitorder> (see page 1049)	:SBUS<n>:MANChesTer:BITorder? (see page 1049)	<bitorder> ::= {MSBFirst LSBFirst}
:SBUS<n>:MANChesTer:DISPlay <format> (see page 1050)	:SBUS<n>:MANChesTer:DISPlay? (see page 1050)	<format> ::= {BIT WORD}
:SBUS<n>:MANChesTer:DISSize {AUTO <#words>} (see page 1051)	:SBUS<n>:MANChesTer:DISSize? (see page 1051)	<#words> ::= from 1-255, in NR1 format
:SBUS<n>:MANChesTer:DISSize <#bits> (see page 1052)	:SBUS<n>:MANChesTer:DISSize? (see page 1052)	<#bits> ::= from 0-32, in NR1 format
:SBUS<n>:MANChesTer:IDLE:BITS <#bits> (see page 1053)	:SBUS<n>:MANChesTer:IDLE:BITS? (see page 1053)	<#bits> ::= minimum idle time in terms of bit width, from 1.50 to 32.0, in NR3 format.
:SBUS<n>:MANChesTer:LOGic <logic> (see page 1054)	:SBUS<n>:MANChesTer:LOGic? (see page 1054)	<logic> ::= {FALLing RISing}

Table 38 :SBUS<n>:MANChesTer Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:MANChesTer:SOURce <source> (see page 1055)	:SBUS<n>:MANChesTer:SOURce? (see page 1055)	<source> ::= {CHANnel<n> DIGital<d>}
:SBUS<n>:MANChesTer:SIZE <#bits> (see page 1056)	:SBUS<n>:MANChesTer:SIZE? (see page 1056)	<#bits> ::= from 0-255, in NR1 format
:SBUS<n>:MANChesTer:TART <edge#> (see page 1057)	:SBUS<n>:MANChesTer:TART? (see page 1057)	<edge#> ::= from 1-256, in NR1 format
:SBUS<n>:MANChesTer:TOLerance <percent> (see page 1058)	:SBUS<n>:MANChesTer:TOLerance? (see page 1058)	<percent> ::= from 1-30, in NR1 format
:SBUS<n>:MANChesTer:TRIGger <mode> (see page 1059)	:SBUS<n>:MANChesTer:TRIGger? (see page 1059)	<mode> ::= {SOF VALue MERRor}
:SBUS<n>:MANChesTer:TRIGger:PATtern:VALue:DATA <string> (see page 1060)	:SBUS<n>:MANChesTer:TRIGger:PATtern:VALue:DATA? (see page 1060)	<string> ::= "nn...n" where n ::= {0 1 X \$} <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X \$}
:SBUS<n>:MANChesTer:TRIGger:PATtern:VALue:WIDTh <width> (see page 1061)	:SBUS<n>:MANChesTer:TRIGger:PATtern:VALue:WIDTh? (see page 1061)	<width> ::= integer from 4 to 128 in NR1 format
:SBUS<n>:MANChesTer:TRIGger:SIZE <#bits> (see page 1062)	:SBUS<n>:MANChesTer:TRIGger:SIZE? (see page 1062)	<#bits> ::= from 0-32, in NR1 format
:SBUS<n>:MANChesTer:WIDTh <#bits> (see page 1063)	:SBUS<n>:MANChesTer:WIDTh? (see page 1063)	<#bits> ::= from 2-32, in NR1 format

Table 39 :SBUS<n>:NRZ Commands Summary

Command	Query	Options and Query Returns
:SBUS<n>:NRZ:BASE <base> (see page 1066)	:SBUS<n>:NRZ:BASE? (see page 1066)	<base> ::= {HEX DECimal ASCii}
:SBUS<n>:NRZ:BAUDrate <baudrate> (see page 1067)	:SBUS<n>:NRZ:BAUDrate? (see page 1067)	<baudrate> ::= integer from 5000 to 5000000 in 100 b/s increments

Table 39 :SBUS<n>:NRZ Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:NRZ:BITOrder <bitorder> (see page 1068)	:SBUS<n>:NRZ:BITOrder? (see page 1068)	<bitorder> ::= {MSBFirst LSBFirst}
:SBUS<n>:NRZ:DISPlay <format> (see page 1069)	:SBUS<n>:NRZ:DISPlay? (see page 1069)	<format> ::= {BIT WORD}
:SBUS<n>:NRZ:DSIZE <#words> (see page 1070)	:SBUS<n>:NRZ:DSIZE? (see page 1070)	<#words> ::= from 1-255, in NR1 format
:SBUS<n>:NRZ:FSIZE <#bits> (see page 1071)	:SBUS<n>:NRZ:FSIZE? (see page 1071)	<#bits> ::= from 2-255, in NR1 format
:SBUS<n>:NRZ:HSIZE <#bits> (see page 1072)	:SBUS<n>:NRZ:HSIZE? (see page 1072)	<#bits> ::= from 0-32, in NR1 format
:SBUS<n>:NRZ:IDLE:BIT S <#bits> (see page 1073)	:SBUS<n>:NRZ:IDLE:BIT S? (see page 1073)	<#bits> ::= minimum idle time in terms of bit width, from 1.50 to 32.0, in NR3 format.
:SBUS<n>:NRZ:IDLE:STA Te <state> (see page 1074)	:SBUS<n>:NRZ:IDLE:STA Te? (see page 1074)	<state> ::= {LOW HIGH}
:SBUS<n>:NRZ:LOGic <logic> (see page 1075)	:SBUS<n>:NRZ:LOGic? (see page 1075)	<logic> ::= {HIGH LOW}
:SBUS<n>:NRZ:SOURce <source> (see page 1076)	:SBUS<n>:NRZ:SOURce? (see page 1076)	<source> ::= {CHANnel<n> DIGital<d>}
:SBUS<n>:NRZ:START <#bits> (see page 1077)	:SBUS<n>:NRZ:START? (see page 1077)	<#bits> ::= from 0-255, in NR1 format
:SBUS<n>:NRZ:TRIGger <mode> (see page 1078)	:SBUS<n>:NRZ:TRIGger? (see page 1078)	<mode> ::= {SOF VALue}
:SBUS<n>:NRZ:TRIGger: PATtern:VALue:DATA <string> (see page 1079)	:SBUS<n>:NRZ:TRIGger: PATtern:VALue:DATA? (see page 1079)	<string> ::= "nn...n" where n ::= {0 1 X \$} <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X \$}
:SBUS<n>:NRZ:TRIGger: PATtern:VALue:WIDTh <width> (see page 1080)	:SBUS<n>:NRZ:TRIGger: PATtern:VALue:WIDTh? (see page 1080)	<width> ::= integer from 4 to 128 in NR1 format

Table 39 :SBUS<n>:NRZ Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:NRZ:TSIZE <#bits> (see page 1081)	:SBUS<n>:NRZ:TSIZE? (see page 1081)	<#bits> ::= from 0-32, in NR1 format
:SBUS<n>:NRZ:WSIZE <#bits> (see page 1082)	:SBUS<n>:NRZ:WSIZE? (see page 1082)	<#bits> ::= from 2-32, in NR1 format

Table 40 :SBUS<n>:SENT Commands Summary

Command	Query	Options and Query Returns
:SBUS<n>:SENT:CLOCK <period> (see page 1086)	:SBUS<n>:SENT:CLOCK? (see page 1086)	<period> ::= the nominal clock period (tick), from 500 ns to 300 us, in NR3 format.
:SBUS<n>:SENT:CRC <format> (see page 1087)	:SBUS<n>:SENT:CRC? (see page 1087)	<format> ::= {LEGacy RECommended}
:SBUS<n>:SENT:DISPlay <base> (see page 1088)	:SBUS<n>:SENT:DISPlay ? (see page 1088)	<base> ::= {HEX DECimal SYMBolic}
:SBUS<n>:SENT:FORMat <decode> (see page 1090)	:SBUS<n>:SENT:FORMat? (see page 1090)	<decode> ::= {NIBBles FSIGnal FSSerial FESerial SSERial ESERial}
:SBUS<n>:SENT:IDLE <state> (see page 1092)	:SBUS<n>:SENT:IDLE? (see page 1092)	<state> ::= {LOW HIGH}
:SBUS<n>:SENT:LENGth <#_nibbles> (see page 1093)	:SBUS<n>:SENT:LENGth? (see page 1093)	<#_nibbles> ::= from 1-6, in NR1 format.
:SBUS<n>:SENT:PPULse { {0 OFF} {1 ON} SPC } (see page 1094)	:SBUS<n>:SENT:PPULse? (see page 1094)	{ 0 1 SPC }
:SBUS<n>:SENT:SIGNaL<s>:DISPlay { {0 OFF} {1 ON} } (see page 1096)	:SBUS<n>:SENT:SIGNaL<s>:DISPlay? (see page 1096)	<s> ::= 1-6, in NR1 format. { 0 1 }
:SBUS<n>:SENT:SIGNaL<s>:LENGth <length> (see page 1097)	:SBUS<n>:SENT:SIGNaL<s>:LENGth? (see page 1097)	<s> ::= 1-6, in NR1 format. <length> ::= from 1-24, in NR1 format.

Table 40 :SBUS<n>:SENT Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:SENT:SIGNAL<s>:MULTIPLIER<multiplier> (see page 1099)	:SBUS<n>:SENT:SIGNAL<s>:MULTIPLIER? (see page 1099)	<s> ::= 1-6, in NR1 format. <multiplier> ::= from 1-24, in NR3 format.
:SBUS<n>:SENT:SIGNAL<s>:OFFSET<offset> (see page 1101)	:SBUS<n>:SENT:SIGNAL<s>:OFFSET? (see page 1101)	<s> ::= 1-6, in NR1 format. <offset> ::= from 1-24, in NR3 format.
:SBUS<n>:SENT:SIGNAL<s>:ORDER<order> (see page 1103)	:SBUS<n>:SENT:SIGNAL<s>:ORDER? (see page 1103)	<s> ::= 1-6, in NR1 format. <order> ::= {MSNFirst LSNFirst}
:SBUS<n>:SENT:SIGNAL<s>:START<position> (see page 1105)	:SBUS<n>:SENT:SIGNAL<s>:START? (see page 1105)	<s> ::= 1-6, in NR1 format. <position> ::= from 0-23, in NR1 format.
:SBUS<n>:SENT:SOURCE<source> (see page 1107)	:SBUS<n>:SENT:SOURCE? (see page 1107)	<source> ::= {CHANNEL<n> DIGITAL<d>} <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:SBUS<n>:SENT:TOLERANCE<percent> (see page 1109)	:SBUS<n>:SENT:TOLERANCE? (see page 1109)	<percent> ::= from 3-30, in NR1 format.
:SBUS<n>:SENT:TRIGGER<mode> (see page 1110)	:SBUS<n>:SENT:TRIGGER? (see page 1110)	<mode> ::= {SFCMessage SSCMessage FCData SCMid SCData FCCError SCCError CRCError TOLerror PPERror SSPerror}
:SBUS<n>:SENT:TRIGGER:FAST:DATA<string> (see page 1112)	:SBUS<n>:SENT:TRIGGER:FAST:DATA? (see page 1112)	<string> ::= "nnnn..." where n ::= {0 1 X} <string> ::= "0xn..." where n ::= {0,...,9 A,...,F X \$}
:SBUS<n>:SENT:TRIGGER:SLOW:DATA<data> (see page 1113)	:SBUS<n>:SENT:TRIGGER:SLOW:DATA? (see page 1113)	<data> ::= when ILENGTH = SHORT, from -1 (don't care) to 65535, in NR1 format. <data> ::= when ILENGTH = LONG, from -1 (don't care) to 4095, in NR1 format.

Table 40 :SBUS<n>:SENT Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:SENT:TRIGger :SLOW:ID <id> (see page 1115)	:SBUS<n>:SENT:TRIGger :SLOW:ID? (see page 1115)	<id> ::= when ILENgth = SHORT, from -1 (don't care) to 15, in NR1 format. <id> ::= when ILENgth = LONG, from -1 (don't care) to 255, in NR1 format.
:SBUS<n>:SENT:TRIGger :SLOW:ILENgth <length> (see page 1117)	:SBUS<n>:SENT:TRIGger :SLOW:ILENgth? (see page 1117)	<length> ::= {SHORT LONG}
:SBUS<n>:SENT:TRIGger :TOLerance <percent> (see page 1118)	:SBUS<n>:SENT:TRIGger :TOLerance? (see page 1118)	<percent> ::= from 1-18, in NR1 format.

Table 41 :SBUS<n>:SPI Commands Summary

Command	Query	Options and Query Returns
:SBUS<n>:SPI:BITOrder <order> (see page 1121)	:SBUS<n>:SPI:BITOrder ? (see page 1121)	<order> ::= {LSBFirst MSBFirst}
:SBUS<n>:SPI:CLOCK:SL OPe <slope> (see page 1122)	:SBUS<n>:SPI:CLOCK:SL OPe? (see page 1122)	<slope> ::= {NEGative POSitive}
:SBUS<n>:SPI:CLOCK:TI Meout <time_value> (see page 1123)	:SBUS<n>:SPI:CLOCK:TI Meout? (see page 1123)	<time_value> ::= time in seconds in NR3 format
:SBUS<n>:SPI:DELAy <value> (see page 1124)	:SBUS<n>:SPI:DELAy? (see page 1124)	<value> ::= {OFF 2-63}
:SBUS<n>:SPI:FRAMing <value> (see page 1125)	:SBUS<n>:SPI:FRAMing? (see page 1125)	<value> ::= {CHIPselect {NCHIPselect NOTC} TIMEout}
:SBUS<n>:SPI:SOURce:C LOCK <source> (see page 1126)	:SBUS<n>:SPI:SOURce:C LOCK? (see page 1126)	<value> ::= {CHANnel<n> EXTernal} for the DSO models <value> ::= {CHANnel<n> DIGital<d>} for the MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format

Table 41 :SBUS<n>:SPI Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:SPI:SOURce:FRAMe <source> (see page 1127)	:SBUS<n>:SPI:SOURce:FRAMe? (see page 1127)	<value> ::= {CHANnel<n> EXTeRnal} for the DSO models <value> ::= {CHANnel<n> DIGital<d>} for the MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:SBUS<n>:SPI:SOURce:MISO <source> (see page 1128)	:SBUS<n>:SPI:SOURce:MISO? (see page 1128)	<value> ::= {CHANnel<n> EXTeRnal} for the DSO models <value> ::= {CHANnel<n> DIGital<d>} for the MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:SBUS<n>:SPI:SOURce:MOSI <source> (see page 1129)	:SBUS<n>:SPI:SOURce:MOSI? (see page 1129)	<value> ::= {CHANnel<n> EXTeRnal} for the DSO models <value> ::= {CHANnel<n> DIGital<d>} for the MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:SBUS<n>:SPI:TRIGger:PATtern:MISO:DATA <string> (see page 1130)	:SBUS<n>:SPI:TRIGger:PATtern:MISO:DATA? (see page 1130)	<string> ::= "nn...n" where n ::= {0 1 X \$} <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X \$}
:SBUS<n>:SPI:TRIGger:PATtern:MISO:WIDTh <width> (see page 1131)	:SBUS<n>:SPI:TRIGger:PATtern:MISO:WIDTh? (see page 1131)	<width> ::= integer from 4 to 64 in NR1 format
:SBUS<n>:SPI:TRIGger:PATtern:MOSI:DATA <string> (see page 1132)	:SBUS<n>:SPI:TRIGger:PATtern:MOSI:DATA? (see page 1132)	<string> ::= "nn...n" where n ::= {0 1 X \$} <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X \$}
:SBUS<n>:SPI:TRIGger:PATtern:MOSI:WIDTh <width> (see page 1133)	:SBUS<n>:SPI:TRIGger:PATtern:MOSI:WIDTh? (see page 1133)	<width> ::= integer from 4 to 64 in NR1 format

Table 41 :SBUS<n>:SPI Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:SPI:TRIGger:TYPE <value> (see page 1134)	:SBUS<n>:SPI:TRIGger:TYPE? (see page 1134)	<value> ::= {MOSI MISO}
:SBUS<n>:SPI:WIDTh <word_width> (see page 1135)	:SBUS<n>:SPI:WIDTh? (see page 1135)	<word_width> ::= integer 4-16 in NR1 format

Table 42 :SBUS<n>:UART Commands Summary

Command	Query	Options and Query Returns
:SBUS<n>:UART:BASE <base> (see page 1139)	:SBUS<n>:UART:BASE? (see page 1139)	<base> ::= {ASCIi BINary HEX}
:SBUS<n>:UART:BAUDrate <baudrate> (see page 1140)	:SBUS<n>:UART:BAUDrate? (see page 1140)	<baudrate> ::= integer from 100 to 8000000, 10000000, or 12000000
:SBUS<n>:UART:BITorder <bitorder> (see page 1141)	:SBUS<n>:UART:BITorder? (see page 1141)	<bitorder> ::= {LSBFirst MSBFirst}
n/a	:SBUS<n>:UART:COUNT:ERRor? (see page 1142)	<frame_count> ::= integer in NR1 format
:SBUS<n>:UART:COUNT:RESet (see page 1143)	n/a	n/a
n/a	:SBUS<n>:UART:COUNT:RXFrames? (see page 1144)	<frame_count> ::= integer in NR1 format
n/a	:SBUS<n>:UART:COUNT:TXFrames? (see page 1145)	<frame_count> ::= integer in NR1 format
:SBUS<n>:UART:FRAMing <value> (see page 1146)	:SBUS<n>:UART:FRAMing? (see page 1146)	<value> ::= {OFF <decimal> <nondecimal>} <decimal> ::= 8-bit integer from 0-255 (0x00-0xff) <nondecimal> ::= #Hnn where n ::= {0,...,9 A,...,F} for hexadecimal <nondecimal> ::= #Bnn...n where n ::= {0 1} for binary
:SBUS<n>:UART:PARity <parity> (see page 1147)	:SBUS<n>:UART:PARity? (see page 1147)	<parity> ::= {EVEN ODD NONE}

Table 42 :SBUS<n>:UART Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:UART:POLarity <polarity> (see page 1148)	:SBUS<n>:UART:POLarity? (see page 1148)	<polarity> ::= {HIGH LOW}
:SBUS<n>:UART:SOURce:RX <source> (see page 1149)	:SBUS<n>:UART:SOURce:RX? (see page 1149)	<source> ::= {CHANnel<n> EXTernal} for DSO models <source> ::= {CHANnel<n> DIGital<d>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:SBUS<n>:UART:SOURce:TX <source> (see page 1150)	:SBUS<n>:UART:SOURce:TX? (see page 1150)	<source> ::= {CHANnel<n> EXTernal} for DSO models <source> ::= {CHANnel<n> DIGital<d>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:SBUS<n>:UART:TRIGger:BASE <base> (see page 1151)	:SBUS<n>:UART:TRIGger:BASE? (see page 1151)	<base> ::= {ASCii HEX}
:SBUS<n>:UART:TRIGger:BURSt <value> (see page 1152)	:SBUS<n>:UART:TRIGger:BURSt? (see page 1152)	<value> ::= {OFF 1 to 4096 in NR1 format}
:SBUS<n>:UART:TRIGger:DATA <value> (see page 1153)	:SBUS<n>:UART:TRIGger:DATA? (see page 1153)	<value> ::= 8-bit integer from 0-255 (0x00-0xff) in decimal, <hexadecimal>, <binary>, or <quoted_string> format <hexadecimal> ::= #Hnn where n ::= {0,...,9 A,...,F} for hexadecimal <binary> ::= #Bnn...n where n ::= {0 1} for binary <quoted_string> ::= any of the 128 valid 7-bit ASCII characters (or standard abbreviations)
:SBUS<n>:UART:TRIGger:IDLE <time_value> (see page 1154)	:SBUS<n>:UART:TRIGger:IDLE? (see page 1154)	<time_value> ::= time from 1 us to 10 s in NR3 format

Table 42 :SBUS<n>:UART Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:UART:TRIGger:QUALifier <value> (see page 1155)	:SBUS<n>:UART:TRIGger:QUALifier? (see page 1155)	<value> ::= {EQUAL NOTequal GREATERthan LESSthan}
:SBUS<n>:UART:TRIGger:TYPE <value> (see page 1156)	:SBUS<n>:UART:TRIGger:TYPE? (see page 1156)	<value> ::= {RSTart RSTop RDATa RD1 RD0 RDX PARityerror TSTart TSTop TDATa TD1 TD0 TDX}
:SBUS<n>:UART:WIDTh <width> (see page 1157)	:SBUS<n>:UART:WIDTh? (see page 1157)	<width> ::= {5 6 7 8 9}

Table 43 :SBUS<n>:USB Commands Summary

Command	Query	Options and Query Returns
:SBUS<n>:USB:BASE <base> (see page 1161)	:SBUS<n>:USB:BASE? (see page 1161)	<base> ::= {ASCIi BINary DECimal HEX}
:SBUS<n>:USB:SOURce:D MINus <source> (see page 1162)	:SBUS<n>:USB:SOURce:D MINus? (see page 1162)	<source> ::= {CHANnel<n>} for DSO models <source> ::= {CHANnel<n> DIGital<d>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:SBUS<n>:USB:SOURce:D Plus <source> (see page 1163)	:SBUS<n>:USB:SOURce:D Plus? (see page 1163)	<source> ::= {CHANnel<n>} for DSO models <source> ::= {CHANnel<n> DIGital<d>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:SBUS<n>:USB:SOURce:D IFFerential <source> (see page 1164)	:SBUS<n>:USB:SOURce:D IFFerential? (see page 1164)	<source> ::= {CHANnel<n>} <n> ::= 1 to (# analog channels) in NR1 format
:SBUS<n>:USB:SPEEd <speed> (see page 1165)	:SBUS<n>:USB:SPEEd? (see page 1165)	<speed> ::= {LOW FULL HIGH}

Table 43 :SBUS<n>:USB Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:USB:TRIGger<condition> (see page 1166)	:SBUS<n>:USB:TRIGger? (see page 1166)	<condition> ::= {SOP EOP ENTersuspend EXITsuspend RESet TOken DATA HANDshake SPECial ALlerrors PIDerror CRC5error CRC16error GLITCherror STUFFerror SElerror}
:SBUS<n>:USB:TRIGger:ADDRESS <string> (see page 1167)	:SBUS<n>:USB:TRIGger:ADDRESS? (see page 1167)	<string> ::= "nnnnnnn" where n ::= {0 1 X} <string> ::= "0xnn" where n ::= {0,...,9 A,...,F X \$}
:SBUS<n>:USB:TRIGger:CRC <string> (see page 1168)	:SBUS<n>:USB:TRIGger:CRC? (see page 1168)	<string> ::= "nnnnn" where n ::= {0 1 X} <string> ::= "0xnn" where n ::= {0,...,9 A,...,F X \$}
:SBUS<n>:USB:TRIGger:DATA <string> (see page 1169)	:SBUS<n>:USB:TRIGger:DATA? (see page 1169)	<string> ::= "nnnn..." where n ::= {0 1 X} <string> ::= "0xn..." where n ::= {0,...,9 A,...,F X \$}
:SBUS<n>:USB:TRIGger:DATA:LENGth <value> (see page 1170)	:SBUS<n>:USB:TRIGger:DATA:LENGth? (see page 1170)	<length> ::= data length between 1-20
:SBUS<n>:USB:TRIGger:ENDPoint <string> (see page 1171)	:SBUS<n>:USB:TRIGger:ENDPoint? (see page 1171)	<string> ::= "nnnn" where n ::= {0 1 X} <string> ::= "0xn" where n ::= {0,...,9 A,...,F X \$}
:SBUS<n>:USB:TRIGger:ET <string> (see page 1172)	:SBUS<n>:USB:TRIGger:ET? (see page 1172)	<string> ::= "nn" where n ::= {0 1 X} <string> ::= "0xn" where n ::= {0 1 2 3 X \$}
:SBUS<n>:USB:TRIGger:FRAMe <string> (see page 1173)	:SBUS<n>:USB:TRIGger:FRAMe? (see page 1173)	<string> ::= "nnnnnnnnnnnn" where n ::= {0 1 X} <string> ::= "0xnnnn" where n ::= {0,...,9 A,...,F X \$}
:SBUS<n>:USB:TRIGger:HADDRESS <string> (see page 1174)	:SBUS<n>:USB:TRIGger:HADDRESS? (see page 1174)	<string> ::= "nnnnnnnn" where n ::= {0 1 X} <string> ::= "0xnn" where n ::= {0,...,9 A,...,F X \$}

Table 43 :SBUS<n>:USB Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:USB:TRIGger:PCHeck <string> (see page 1175)	:SBUS<n>:USB:TRIGger:PCHeck? (see page 1175)	<string> ::= "nnnn" where n ::= {0 1 X} <string> ::= "0xn" where n ::= {0,...,9 A,...,F X \$}
:SBUS<n>:USB:TRIGger:PID:DATA <pid> (see page 1176)	:SBUS<n>:USB:TRIGger:PID:DATA? (see page 1176)	<pid> ::= {DATA0 DATA1 DATA2 MDATa}
:SBUS<n>:USB:TRIGger:PID:HANDshake <pid> (see page 1177)	:SBUS<n>:USB:TRIGger:PID:HANDshake? (see page 1177)	<pid> ::= {ACK NAK STAl1 NYET}
:SBUS<n>:USB:TRIGger:PID:SPECIAL <pid> (see page 1178)	:SBUS<n>:USB:TRIGger:PID:SPECIAL? (see page 1178)	<pid> ::= {PING PRE ERR SPLit}
:SBUS<n>:USB:TRIGger:PID:TOKEN <pid> (see page 1179)	:SBUS<n>:USB:TRIGger:PID:TOKEN? (see page 1179)	<pid> ::= {OUT IN SETUp SOF}
:SBUS<n>:USB:TRIGger:PORT <string> (see page 1180)	:SBUS<n>:USB:TRIGger:PORT? (see page 1180)	<string> ::= "nnnnnnnn" where n ::= {0 1 X} <string> ::= "0xnn" where n ::= {0,...,9 A,...,F X \$}
:SBUS<n>:USB:TRIGger:SC <string> (see page 1181)	:SBUS<n>:USB:TRIGger:SC? (see page 1181)	<string> ::= "n" where n ::= {0 1 X} <string> ::= "0xn" where n ::= {0 1 X \$}
:SBUS<n>:USB:TRIGger:SEU <string> (see page 1182)	:SBUS<n>:USB:TRIGger:SEU? (see page 1182)	<string> ::= "nn" where n ::= {0 1 X} <string> ::= "0xn" where n ::= {0 1 2 3 X \$}

Table 44 :SBUS<n>:USBPd Commands Summary

Command	Query	Options and Query Returns
:SBUS<n>:USBPd:SOURce <source> (see page 1184)	:SBUS<n>:USBPd:SOURce ? (see page 1184)	<source> ::= {CHANnel<n>} <n> ::= 1 to (# analog channels) in NR1 format
:SBUS<n>:USBPd:TRIGger <mode> (see page 1185)	:SBUS<n>:USBPd:TRIGger ? (see page 1185)	<mode> ::= {PSTart EOP SOP SPRime SDPRime SPDebug SDPDebug HRST CRST CRCerror PERRor HEADer}

Table 44 :SBUS<n>:USBPd Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:USBPd:TRIGger:HEADer <type> (see page 1186)	:SBUS<n>:USBPd:TRIGger:HEADer? (see page 1186)	<type> ::= {CMESsage DMESsage EMESsage VALue}
:SBUS<n>:USBPd:TRIGger:HEADer:CMESsage <type> (see page 1188)	:SBUS<n>:USBPd:TRIGger:HEADer:CMESsage? (see page 1188)	<type> ::= {GOODcrc GOTOmin ACcept REject PING PSRDy GSRCap GSNCap DRSSwap PRSSwap VCSwap WAIT SRST GSCX GSTatus FRSSwap GPSTatus GCCodes}
:SBUS<n>:USBPd:TRIGger:HEADer:DMESsage <type> (see page 1190)	:SBUS<n>:USBPd:TRIGger:HEADer:DMESsage? (see page 1190)	<type> ::= {SRCap REQuest BIST SNCap BSTatus ALERT GCINfo VDEFined}
:SBUS<n>:USBPd:TRIGger:HEADer:EMESsage <type> (see page 1191)	:SBUS<n>:USBPd:TRIGger:HEADer:EMESsage? (see page 1191)	<type> ::= {SCX STATus GBCap GBSTatus BCAP GMINfo MINfo SREQuest SRESponse FREQuest FRESponse PSTatus CINfo CCODEs}
:SBUS<n>:USBPd:TRIGger:HEADer:VALue <string> (see page 1193)	:SBUS<n>:USBPd:TRIGger:HEADer:VALue? (see page 1193)	<string> ::= "nn...n" where n ::= {0 1 X} <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X}
:SBUS<n>:USBPd:TRIGger:HEADer:QUALifier <type> (see page 1194)	:SBUS<n>:USBPd:TRIGger:HEADer:QUALifier? (see page 1194)	<type> ::= {NONE SOP SPRime SDPRime}

Table 45 General :SEARCh Commands Summary

Command	Query	Options and Query Returns
n/a	:SEARCh:COUNT? (see page 1197)	<count> ::= an integer count value
:SEARCh:EVENT <event_number> (see page 1198)	:SEARCh:EVENT? (see page 1198)	<event_number> ::= the integer number of a found search event
:SEARCh:MODE <value> (see page 1199)	:SEARCh:MODE? (see page 1199)	<value> ::= {EDGE GLITCh RUNT TRANSition SERIAL{1 2} PEAK}
:SEARCh:STATe <value> (see page 1200)	:SEARCh:STATe? (see page 1200)	<value> ::= {{0 OFF} {1 ON}}

Table 46 :SEARCH:EDGE Commands Summary

Command	Query	Options and Query Returns
:SEARCH:EDGE:SLOPe <slope> (see page 1202)	:SEARCH:EDGE:SLOPe? (see page 1202)	<slope> ::= {POSitive NEGative EITHer}
:SEARCH:EDGE:SOURce <source> (see page 1203)	:SEARCH:EDGE:SOURce? (see page 1203)	<source> ::= CHANnel<n> <n> ::= 1 to (# analog channels) in NR1 format

Table 47 :SEARCH:GLITCh Commands Summary

Command	Query	Options and Query Returns
:SEARCH:GLITCh:GREate rthan <greater_than_time>[s uffix] (see page 1205)	:SEARCH:GLITCh:GREate rthan? (see page 1205)	<greater_than_time> ::= floating-point number in NR3 format [suffix] ::= {s ms us ns ps}
:SEARCH:GLITCh:LESSth an <less_than_time>[suff ix] (see page 1206)	:SEARCH:GLITCh:LESSth an? (see page 1206)	<less_than_time> ::= floating-point number in NR3 format [suffix] ::= {s ms us ns ps}
:SEARCH:GLITCh:POLari ty <polarity> (see page 1207)	:SEARCH:GLITCh:POLari ty? (see page 1207)	<polarity> ::= {POSitive NEGative}
:SEARCH:GLITCh:QUALif ier <qualifier> (see page 1208)	:SEARCH:GLITCh:QUALif ier? (see page 1208)	<qualifier> ::= {GREaterthan LESSthan RANGE}
:SEARCH:GLITCh:RANGe <less_than_time>[suff ix], <greater_than_time>[s uffix] (see page 1209)	:SEARCH:GLITCh:RANGe? (see page 1209)	<less_than_time> ::= 15 ns to 10 seconds in NR3 format <greater_than_time> ::= 10 ns to 9.99 seconds in NR3 format [suffix] ::= {s ms us ns ps}
:SEARCH:GLITCh:SOURce <source> (see page 1210)	:SEARCH:GLITCh:SOURce ? (see page 1210)	<source> ::= CHANnel<n> <n> ::= 1 to (# analog channels) in NR1 format

Table 48 :SEARCH:PEAK Commands Summary

Command	Query	Options and Query Returns
:SEARCH:PEAK:EXCURSION <delta_level> (see page 1212)	:SEARCH:PEAK:EXCURSION? (see page 1212)	<delta_level> ::= required change in level to be recognized as a peak, in NR3 format.
:SEARCH:PEAK:NPEAKS <number> (see page 1213)	:SEARCH:PEAK:NPEAKS? (see page 1213)	<number> ::= max number of peaks to find, 1-11 in NR1 format.
:SEARCH:PEAK:SOURCE <source> (see page 1214)	:SEARCH:PEAK:SOURCE? (see page 1214)	<source> ::= {FUNCTION<m> MATH<m>} (must be an FFT waveform) <m> ::= 1 to 4 in NR1 format
:SEARCH:PEAK:THRESHOLD <level> (see page 1215)	:SEARCH:PEAK:THRESHOLD? (see page 1215)	<level> ::= necessary level to be considered a peak, in NR3 format.

Table 49 :SEARCH:RUNT Commands Summary

Command	Query	Options and Query Returns
:SEARCH:RUNT:POLARITY <polarity> (see page 1217)	:SEARCH:RUNT:POLARITY? (see page 1217)	<polarity> ::= {POSITIVE NEGATIVE EITHER}
:SEARCH:RUNT:QUALIFIER <qualifier> (see page 1218)	:SEARCH:RUNT:QUALIFIER? (see page 1218)	<qualifier> ::= {GREATERthan LESSthan NONE}
:SEARCH:RUNT:SOURCE <source> (see page 1219)	:SEARCH:RUNT:SOURCE? (see page 1219)	<source> ::= CHANNEL<n> <n> ::= 1 to (# analog channels) in NR1 format
:SEARCH:RUNT:TIME <time>[suffix] (see page 1220)	:SEARCH:RUNT:TIME? (see page 1220)	<time> ::= floating-point number in NR3 format [suffix] ::= {s ms us ns ps}

Table 50 :SEARCh:TRANSition Commands Summary

Command	Query	Options and Query Returns
:SEARCh:TRANSition:QUAlifier <qualifier> (see page 1222)	:SEARCh:TRANSition:QUAlifier? (see page 1222)	<qualifier> ::= {GREATERthan LESSthan}
:SEARCh:TRANSition:SLOPe <slope> (see page 1223)	:SEARCh:TRANSition:SLOPe? (see page 1223)	<slope> ::= {NEGative POSitive}
:SEARCh:TRANSition:SOURce <source> (see page 1224)	:SEARCh:TRANSition:SOURce? (see page 1224)	<source> ::= CHANnel<n> <n> ::= 1 to (# analog channels) in NR1 format
:SEARCh:TRANSition:TIME <time>[suffix] (see page 1225)	:SEARCh:TRANSition:TIME? (see page 1225)	<time> ::= floating-point number in NR3 format [suffix] ::= {s ms us ns ps}

Table 51 :SEARCh:SERial:A429 Commands Summary

Command	Query	Options and Query Returns
:SEARCh:SERial:A429:LABel <value> (see page 1227)	:SEARCh:SERial:A429:LABel? (see page 1227)	<value> ::= 8-bit integer in decimal, <hex>, <octal>, or <string> from 0-255 <hex> ::= #Hnn where n ::= {0,...,9 A,...,F} <octal> ::= #Qnnn where n ::= {0,...,7} <string> ::= "0xnn" where n ::= {0,...,9 A,...,F}
:SEARCh:SERial:A429:MODE <condition> (see page 1228)	:SEARCh:SERial:A429:MODE? (see page 1228)	<condition> ::= {LABel LBITs PERRor WERRor GERRor WGERrors ALLerrors}
:SEARCh:SERial:A429:PATtern:DATA <string> (see page 1229)	:SEARCh:SERial:A429:PATtern:DATA? (see page 1229)	<string> ::= "nn...n" where n ::= {0 1}, length depends on FORMat
:SEARCh:SERial:A429:PATtern:SDI <string> (see page 1230)	:SEARCh:SERial:A429:PATtern:SDI? (see page 1230)	<string> ::= "nn" where n ::= {0 1}, length always 2 bits
:SEARCh:SERial:A429:PATtern:SSM <string> (see page 1231)	:SEARCh:SERial:A429:PATtern:SSM? (see page 1231)	<string> ::= "nn" where n ::= {0 1}, length always 2 bits

Table 52 :SEARCH:SERIAL:CAN Commands Summary

Command	Query	Options and Query Returns
:SEARCH:SERIAL:CAN:MODE <value> (see page 1233)	:SEARCH:SERIAL:CAN:MODE? (see page 1233)	<value> ::= {IDEither IDData DATA IDRemote ERROR ACKError FORMError STUFFerror CRCError ALLerrors OVERload MESSage MSIGnal}
:SEARCH:SERIAL:CAN:PA Ttern:DATA <string> (see page 1235)	:SEARCH:SERIAL:CAN:PA Ttern:DATA? (see page 1235)	<string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X} for hexadecimal
:SEARCH:SERIAL:CAN:PA Ttern:DATA:LENGth <length> (see page 1236)	:SEARCH:SERIAL:CAN:PA Ttern:DATA:LENGth? (see page 1236)	<length> ::= integer from 1 to 8 in NR1 format
:SEARCH:SERIAL:CAN:PA Ttern:ID <string> (see page 1237)	:SEARCH:SERIAL:CAN:PA Ttern:ID? (see page 1237)	<string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X} for hexadecimal
:SEARCH:SERIAL:CAN:PA Ttern:ID:MODE <value> (see page 1238)	:SEARCH:SERIAL:CAN:PA Ttern:ID:MODE? (see page 1238)	<value> ::= {STANdard EXTended}
:SEARCH:SERIAL:CAN:SY MBolic:MESSage <name> (see page 1239)	:SEARCH:SERIAL:CAN:SY MBolic:MESSage? (see page 1239)	<name> ::= quoted ASCII string
:SEARCH:SERIAL:CAN:SY MBolic:SIGNal <name> (see page 1240)	:SEARCH:SERIAL:CAN:SY MBolic:SIGNal? (see page 1240)	<name> ::= quoted ASCII string
:SEARCH:SERIAL:CAN:SY MBolic:VALue <data> (see page 1241)	:SEARCH:SERIAL:CAN:SY MBolic:VALue? (see page 1241)	<data> ::= value in NR3 format

Table 53 :SEARCH:SERIAL:FLEXray Commands Summary

Command	Query	Options and Query Returns
:SEARCH:SERIAL:FLEXray:CYCLE <cycle> (see page 1243)	:SEARCH:SERIAL:FLEXray:CYCLE? (see page 1243)	<cycle> ::= {ALL <cycle #>} <cycle #> ::= integer from 0-63
:SEARCH:SERIAL:FLEXray:DATA <string> (see page 1244)	:SEARCH:SERIAL:FLEXray:DATA? (see page 1244)	<string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X }
:SEARCH:SERIAL:FLEXray:DATA:LENGth <length> (see page 1245)	:SEARCH:SERIAL:FLEXray:DATA:LENGth? (see page 1245)	<length> ::= integer from 1 to 12 in NR1 format

Table 53 :SEARCh:SERIal:FLEXray Commands Summary (continued)

Command	Query	Options and Query Returns
:SEARCh:SERIal:FLEXray:FRAME <frame id> (see page 1246)	:SEARCh:SERIal:FLEXray:FRAME? (see page 1246)	<frame_id> ::= {ALL <frame #>} <frame #> ::= integer from 1-2047
:SEARCh:SERIal:FLEXray:MODE <value> (see page 1247)	:SEARCh:SERIal:FLEXray:MODE? (see page 1247)	<value> ::= {FRAME CYCLe DATA HERRor FERRor AERRor}

Table 54 :SEARCh:SERIal:I2S Commands Summary

Command	Query	Options and Query Returns
:SEARCh:SERIal:I2S:AUDio <audio_ch> (see page 1249)	:SEARCh:SERIal:I2S:AUDio? (see page 1249)	<audio_ch> ::= {RIGHT LEFT EITHER}
:SEARCh:SERIal:I2S:MODE <value> (see page 1250)	:SEARCh:SERIal:I2S:MODE? (see page 1250)	<value> ::= {EQUAL NOTequal LESSthan GREATERthan INRange OUTRange}
:SEARCh:SERIal:I2S:PAITtern:DATA <string> (see page 1251)	:SEARCh:SERIal:I2S:PAITtern:DATA? (see page 1251)	<string> ::= "n" where n ::= 32-bit integer in signed decimal when <base> = DECimal <string> ::= "nn...n" where n ::= {0 1 X} when <base> = BINary <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X} when <base> = HEX
:SEARCh:SERIal:I2S:PAITtern:FORMat <base> (see page 1252)	:SEARCh:SERIal:I2S:PAITtern:FORMat? (see page 1252)	<base> ::= {BINary HEX DECimal}
:SEARCh:SERIal:I2S:RANGe <lower>, <upper> (see page 1253)	:SEARCh:SERIal:I2S:RANGe? (see page 1253)	<lower> ::= 32-bit integer in signed decimal, <nondecimal>, or <string> <upper> ::= 32-bit integer in signed decimal, <nondecimal>, or <string> <nondecimal> ::= #Hnn...n where n ::= {0,...,9 A,...,F} for hexadecimal <nondecimal> ::= #Bnn...n where n ::= {0 1} for binary <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F} for hexadecimal

Table 55 :SEARCH:SERIAL:IIC Commands Summary

Command	Query	Options and Query Returns
:SEARCH:SERIAL:IIC:MODE <value> (see page 1255)	:SEARCH:SERIAL:IIC:MODE? (see page 1255)	<value> ::= {REStart ADDRESS ANACK NACKnowledge READEprom READ7 WRITE7 R7Data2 W7Data2}
:SEARCH:SERIAL:IIC:PA TTern:ADDRESS <value> (see page 1257)	:SEARCH:SERIAL:IIC:PA TTern:ADDRESS? (see page 1257)	<value> ::= integer or <string> <string> ::= "0xnn" n ::= {0,...,9 A,...,F}
:SEARCH:SERIAL:IIC:PA TTern:DATA <value> (see page 1258)	:SEARCH:SERIAL:IIC:PA TTern:DATA? (see page 1258)	<value> ::= integer or <string> <string> ::= "0xnn" n ::= {0,...,9 A,...,F}
:SEARCH:SERIAL:IIC:PA TTern:DATA2 <value> (see page 1259)	:SEARCH:SERIAL:IIC:PA TTern:DATA2? (see page 1259)	<value> ::= integer or <string> <string> ::= "0xnn" n ::= {0,...,9 A,...,F}
:SEARCH:SERIAL:IIC:QUALifier <value> (see page 1260)	:SEARCH:SERIAL:IIC:QUALifier? (see page 1260)	<value> ::= {EQUAL NOTequal LESSthan GREATERthan}

Table 56 :SEARCH:SERIAL:LIN Commands Summary

Command	Query	Options and Query Returns
:SEARCH:SERIAL:LIN:ID <value> (see page 1262)	:SEARCH:SERIAL:LIN:ID ? (see page 1262)	<value> ::= 7-bit integer in decimal, <nondecimal>, or <string> from 0-63 or 0x00-0x3f <nondecimal> ::= #Hnn where n ::= {0,...,9 A,...,F} for hexadecimal <nondecimal> ::= #Bnn...n where n ::= {0 1} for binary <string> ::= "0xnn" where n ::= {0,...,9 A,...,F} for hexadecimal
:SEARCH:SERIAL:LIN:MODE <value> (see page 1263)	:SEARCH:SERIAL:LIN:MODE? (see page 1263)	<value> ::= {ID DATA ERROR}

Table 56 :SEARCh:SErIAL:LIN Commands Summary (continued)

Command	Query	Options and Query Returns
:SEARCh:SErIAL:LIN:PA TTern:DATA <string> (see page 1264)	:SEARCh:SErIAL:LIN:PA TTern:DATA? (see page 1264)	When :SEARCh:SErIAL:LIN:PA TTern:FORMa t DECimal, <string> ::= "n" where n ::= 32-bit integer in unsigned decimal, returns "\$" if data has any don't cares When :SEARCh:SErIAL:LIN:PA TTern:FORMa t HEX, <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X }
:SEARCh:SErIAL:LIN:PA TTern:DATA:LENGth <length> (see page 1265)	:SEARCh:SErIAL:LIN:PA TTern:DATA:LENGth? (see page 1265)	<length> ::= integer from 1 to 8 in NR1 format
:SEARCh:SErIAL:LIN:PA TTern:FORMat <base> (see page 1266)	:SEARCh:SErIAL:LIN:PA TTern:FORMat? (see page 1266)	<base> ::= {HEX DECimal}
:SEARCh:SErIAL:LIN:SY MBolic:FRAMe <name> (see page 1267)	:SEARCh:SErIAL:LIN:SY MBolic:FRAMe? (see page 1267)	<name> ::= quoted ASCII string
:SEARCh:SErIAL:LIN:SY MBolic:SIGNal <name> (see page 1268)	:SEARCh:SErIAL:LIN:SY MBolic:SIGNal? (see page 1268)	<name> ::= quoted ASCII string
:SEARCh:SErIAL:LIN:SY MBolic:VALue <data> (see page 1269)	:SEARCh:SErIAL:LIN:SY MBolic:VALue? (see page 1269)	<data> ::= value in NR3 format

Table 57 :SEARCh:SErIAL:M1553 Commands Summary

Command	Query	Options and Query Returns
:SEARCh:SErIAL:M1553: MODE <value> (see page 1271)	:SEARCh:SErIAL:M1553: MODE? (see page 1271)	<value> ::= {DSTArt CSTArt RTA RTA11 PERRor SERRor MERRor}

Table 57 :SEARCh:SERIal:M1553 Commands Summary (continued)

Command	Query	Options and Query Returns
:SEARCh:SERIal:M1553:PATtern:DATA <string> (see page 1272)	:SEARCh:SERIal:M1553:PATtern:DATA? (see page 1272)	<string> ::= "nn...n" where n ::= {0 1}
:SEARCh:SERIal:M1553:RTA <value> (see page 1273)	:SEARCh:SERIal:M1553:RTA? (see page 1273)	<value> ::= 5-bit integer in decimal, <hexadecimal>, <binary>, or <string> from 0-31 < hexadecimal > ::= #Hnn where n ::= {0,...,9 A,...,F} <binary> ::= #Bnn...n where n ::= {0 1} for binary <string> ::= "0xnn" where n ::= {0,...,9 A,...,F}

Table 58 :SEARCh:SERIal:SENT Commands Summary

Command	Query	Options and Query Returns
:SEARCh:SERIal:SENT:FAST:DATA <string> (see page 1275)	:SEARCh:SERIal:SENT:FAST:DATA? (see page 1275)	<string> ::= "0xn..." where n ::= {0,...,9 A,...,F X \$}
:SEARCh:SERIal:SENT:MODE <mode> (see page 1276)	:SEARCh:SERIal:SENT:MODE? (see page 1276)	<mode> ::= {FCData SCMid SCData CRCerror PPErrror}
:SEARCh:SERIal:SENT:SLow:DATA <data> (see page 1277)	:SEARCh:SERIal:SENT:SLow:DATA? (see page 1277)	<data> ::= from -1 (don't care) to 65535, in NR1 format.
:SEARCh:SERIal:SENT:SLow:ID <id> (see page 1278)	:SEARCh:SERIal:SENT:SLow:ID? (see page 1278)	<id> ::= from -1 (don't care) to 255, in NR1 format.

Table 59 :SEARCh:SERIal:SPI Commands Summary

Command	Query	Options and Query Returns
:SEARCh:SERIal:SPI:MODE <value> (see page 1280)	:SEARCh:SERIal:SPI:MODE? (see page 1280)	<value> ::= {MOSI MISO}

Table 59 :SEARCh:SErIal:SPI Commands Summary (continued)

Command	Query	Options and Query Returns
:SEARCh:SErIal:SPI:PA TTern:DATA <string> (see page 1281)	:SEARCh:SErIal:SPI:PA TTern:DATA? (see page 1281)	<string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X}
:SEARCh:SErIal:SPI:PA TTern:WIDTh <width> (see page 1282)	:SEARCh:SErIal:SPI:PA TTern:WIDTh? (see page 1282)	<width> ::= integer from 1 to 10

Table 60 :SEARCh:SErIal:UART Commands Summary

Command	Query	Options and Query Returns
:SEARCh:SErIal:UART:D ATA <value> (see page 1284)	:SEARCh:SErIal:UART:D ATA? (see page 1284)	<value> ::= 8-bit integer from 0-255 (0x00-0xff) in decimal, <hexadecimal>, <binary>, or <quoted_string> format <hexadecimal> ::= #Hnn where n ::= {0,...,9 A,...,F} for hexadecimal <binary> ::= #Bnn...n where n ::= {0 1} for binary <quoted_string> ::= any of the 128 valid 7-bit ASCII characters (or standard abbreviations)
:SEARCh:SErIal:UART:M ODE <value> (see page 1285)	:SEARCh:SErIal:UART:M ODE? (see page 1285)	<value> ::= {RDATa RD1 RD0 RDX TDATa TD1 TD0 TDX PARityerror AERRor}
:SEARCh:SErIal:UART:Q UALifier <value> (see page 1286)	:SEARCh:SErIal:UART:Q UALifier? (see page 1286)	<value> ::= {EQUal NOTequal GREaterthan LESSthan}

Table 61 :SEARCh:SErIal:USB Commands Summary

Command	Query	Options and Query Returns
:SEARCh:SErIal:USB:MO DE <condition> (see page 1289)	:SEARCh:SErIal:USB:MO DE? (see page 1289)	<event> ::= {TOKen DATA HANDshake SPECial ALLerrors PIDerror CRC5error CRC16error GLITCherror STUFFerror SE1error}
:SEARCh:SErIal:USB:AD DRess <string> (see page 1290)	:SEARCh:SErIal:USB:AD DRess? (see page 1290)	<string> ::= "0xnn" where n ::= {0,...,9 A,...,F X \$}

Table 61 :SEARCH:SERIAL:USB Commands Summary (continued)

Command	Query	Options and Query Returns
:SEARCH:SERIAL:USB:CR C <string> (see page 1291)	:SEARCH:SERIAL:USB:CR C? (see page 1291)	<string> ::= "0xnn" where n ::= {0,...,9 A,...,F X \$}
:SEARCH:SERIAL:USB:DA TA <string> (see page 1292)	:SEARCH:SERIAL:USB:DA TA? (see page 1292)	<string> ::= "0xnn..." where n ::= {0,...,9 A,...,F X \$}
:SEARCH:SERIAL:USB:DA TA:LENGth <value> (see page 1293)	:SEARCH:SERIAL:USB:DA TA:LENGth? (see page 1293)	<length> ::= data length between 1-20
:SEARCH:SERIAL:USB:EN DPoint <string> (see page 1294)	:SEARCH:SERIAL:USB:EN DPoint? (see page 1294)	<string> ::= "0xnn" where n ::= {0,...,9 A,...,F X \$}
:SEARCH:SERIAL:USB:ET <string> (see page 1295)	:SEARCH:SERIAL:USB:ET ? (see page 1295)	<string> ::= "0xnn" where n ::= {0 1 2 3 X \$}
:SEARCH:SERIAL:USB:FR AMe <string> (see page 1296)	:SEARCH:SERIAL:USB:FR AMe? (see page 1296)	<string> ::= "0xnnnn" where n ::= {0,...,9 A,...,F X \$}
:SEARCH:SERIAL:USB:HA DDress <string> (see page 1297)	:SEARCH:SERIAL:USB:HA DDress? (see page 1297)	<string> ::= "0xnn" where n ::= {0,...,9 A,...,F X \$}
:SEARCH:SERIAL:USB:PI D:DATA <pid> (see page 1298)	:SEARCH:SERIAL:USB:PI D:DATA? (see page 1298)	<pid> ::= {DATA0 DATA1 DATA2 MDATa}
:SEARCH:SERIAL:USB:PI D:HANDshake <pid> (see page 1299)	:SEARCH:SERIAL:USB:PI D:HANDshake? (see page 1299)	<pid> ::= {ACK NAK STAl1 NYET}
:SEARCH:SERIAL:USB:PI D:SPECial <pid> (see page 1300)	:SEARCH:SERIAL:USB:PI D:SPECial? (see page 1300)	<pid> ::= {PING PRE ERR SPLit}
:SEARCH:SERIAL:USB:PI D:TOKen <pid> (see page 1301)	:SEARCH:SERIAL:USB:PI D:TOKen? (see page 1301)	<pid> ::= {OUT IN SETUp SOF}
:SEARCH:SERIAL:USB:PO RT <string> (see page 1302)	:SEARCH:SERIAL:USB:PO RT? (see page 1302)	<string> ::= "0xnn" where n ::= {0,...,9 A,...,F X \$}

Table 61 :SEARCh:SERIal:USB Commands Summary (continued)

Command	Query	Options and Query Returns
:SEARCh:SERIal:USB:SC <string> (see page 1303)	:SEARCh:SERIal:USB:SC ? (see page 1303)	<string> ::= "0xn" where n ::= {0 1 X \$}
:SEARCh:SERIal:USB:SE U <string> (see page 1304)	:SEARCh:SERIal:USB:SE U? (see page 1304)	<string> ::= "0xn" where n ::= {0 1 2 3 X \$}

Table 62 :SYSTem Commands Summary

Command	Query	Options and Query Returns
:SYSTem:DATE <date> (see page 1308)	:SYSTem:DATE? (see page 1308)	<date> ::= <year>, <month>, <day> <year> ::= 4-digit year in NR1 format <month> ::= {1,...,12 JANuary FEBruary MARCH APRil MAY JUNE JULy AUGust SEPTember OCTober NOVember DECember} <day> ::= {1,...,31}
n/a	:SYSTem:DIIdentifier? (see page 1309)	n/a
:SYSTem:DSP <string> (see page 1310)	n/a	<string> ::= up to 75 characters as a quoted ASCII string
n/a	:SYSTem:ERRor? (see page 1311)	<error> ::= an integer error code <error string> ::= quoted ASCII string. See Error Messages (see page 1589).
:SYSTem:LOCK <value> (see page 1312)	:SYSTem:LOCK? (see page 1312)	<value> ::= {{1 ON} {0 OFF}}
:SYSTem:PERSonal:MANu facturer] <manufacturer_string> (see page 1313)	:SYSTem:PERSonal:MANu facturer]? (see page 1313)	<manufacturer_string> ::= quoted ASCII string, up to 63 characters
:SYSTem:PERSonal:MANu facturer]:DEFault (see page 1314)	n/a	Sets manufacturer string to "KEYSIGHT TECHNOLOGIES"
:SYSTem:PRESet (see page 1315)	n/a	See :SYSTem:PRESet (see page 1315)

Table 62 :SYSTem Commands Summary (continued)

Command	Query	Options and Query Returns
:SYSTem:PROTection:LOCK <value> (see page 1318)	:SYSTem:PROTection:LOCK? (see page 1318)	<value> ::= {{1 ON} {0 OFF}}
:SYSTem:RLOGger <setting>[,<file_name>[,<write_mode>]] (see page 1319)	n/a	<setting> ::= {{0 OFF} {1 ON}} <file_name> ::= quoted ASCII string <write_mode> ::= {CREate APPend}
:SYSTem:RLOGger:DESTination <dest> (see page 1320)	:SYSTem:RLOGger:DESTination? (see page 1320)	<dest> ::= {FILE SCReen BOTH}
:SYSTem:RLOGger:DISPlay {{0 OFF} {1 ON}} (see page 1321)	:SYSTem:RLOGger:DISPlay? (see page 1321)	<setting> ::= {0 1}
:SYSTem:RLOGger:FNAME <file_name> (see page 1322)	:SYSTem:RLOGger:FNAME? (see page 1322)	<file_name> ::= quoted ASCII string
:SYSTem:RLOGger:STATe {{0 OFF} {1 ON}} (see page 1323)	:SYSTem:RLOGger:STATe? (see page 1323)	<setting> ::= {0 1}
:SYSTem:RLOGger:TRANSPARENT {{0 OFF} {1 ON}} (see page 1324)	:SYSTem:RLOGger:TRANSPARENT? (see page 1324)	<setting> ::= {0 1}
:SYSTem:RLOGger:WMODe <write_mode> (see page 1325)	:SYSTem:RLOGger:WMODe? (see page 1325)	<write_mode> ::= {CREate APPend}
:SYSTem:SETup <setup_data> (see page 1326)	:SYSTem:SETup? (see page 1326)	<setup_data> ::= data in IEEE 488.2 # format.
:SYSTem:TIME <time> (see page 1328)	:SYSTem:TIME? (see page 1328)	<time> ::= hours,minutes,seconds in NR1 format
:SYSTem:TOUCh {{1 ON} {0 OFF}} (see page 1329)	:SYSTem:TOUCh? (see page 1329)	{1 0}

Table 63 :TImebase Commands Summary

Command	Query	Options and Query Returns
:TImebase:MODE <value> (see page 1333)	:TImebase:MODE? (see page 1333)	<value> ::= {MAIN WINDow XY ROLL}
:TImebase:POSition <pos> (see page 1334)	:TImebase:POSition? (see page 1334)	<pos> ::= time from the trigger event to the display reference point in NR3 format
:TImebase:RANGe <range_value> (see page 1335)	:TImebase:RANGe? (see page 1335)	<range_value> ::= time for 10 div in seconds in NR3 format
:TImebase:REFClock { {0 OFF} {1 ON} } (see page 1336)	:TImebase:REFClock? (see page 1336)	{0 1}
:TImebase:REFeRence {LEFT CENTer RIGHT CUSTom} (see page 1337)	:TImebase:REFeRence? (see page 1337)	<return_value> ::= {LEFT CENTer RIGHT CUSTom}
:TImebase:REFeRence:L OCation <loc> (see page 1338)	:TImebase:REFeRence:L OCation? (see page 1338)	<loc> ::= 0.0 to 1.0 in NR3 format
:TImebase:SCALe <scale_value> (see page 1339)	:TImebase:SCALe? (see page 1339)	<scale_value> ::= time/div in seconds in NR3 format
:TImebase:VERNier { {0 OFF} {1 ON} } (see page 1340)	:TImebase:VERNier? (see page 1340)	{0 1}
:TImebase:WINDow:POSi tion <pos> (see page 1341)	:TImebase:WINDow:POSi tion? (see page 1341)	<pos> ::= time from the trigger event to the zoomed view reference point in NR3 format
:TImebase:WINDow:RANG e <range_value> (see page 1342)	:TImebase:WINDow:RANG e? (see page 1342)	<range value> ::= range value in seconds in NR3 format for the zoomed window
:TImebase:WINDow:SCAL e <scale_value> (see page 1343)	:TImebase:WINDow:SCAL e? (see page 1343)	<scale_value> ::= scale value in seconds in NR3 format for the zoomed window

Table 64 General :TRIGger Commands Summary

Command	Query	Options and Query Returns
:TRIGger:FORCE (see page 1349)	n/a	n/a
:TRIGger:HFReject {{0 OFF} {1 ON}} (see page 1350)	:TRIGger:HFReject? (see page 1350)	{0 1}
:TRIGger:HOLDoff <holdoff_time> (see page 1351)	:TRIGger:HOLDoff? (see page 1351)	<holdoff_time> ::= 60 ns to 10 s in NR3 format
:TRIGger:HOLDoff:MAXimum <max_holdoff> (see page 1352)	:TRIGger:HOLDoff:MAXimum? (see page 1352)	<max_holdoff> ::= maximum holdoff time in seconds in NR3 format
:TRIGger:HOLDoff:MINimum <min_holdoff> (see page 1353)	:TRIGger:HOLDoff:MINimum? (see page 1353)	<min_holdoff> ::= minimum holdoff time in seconds in NR3 format
:TRIGger:HOLDoff:RANDOM {{0 OFF} {1 ON}} (see page 1354)	:TRIGger:HOLDoff:RANDOM? (see page 1354)	<setting> ::= {0 1}
:TRIGger:LEVel:ASETup (see page 1355)	n/a	n/a
:TRIGger:LEVel:HIGH <level>, <source> (see page 1356)	:TRIGger:LEVel:HIGH? <source> (see page 1356)	<level> ::= .75 x full-scale voltage from center screen in NR3 format. <source> ::= CHANNEL<n> <n> ::= 1 to (# analog channels) in NR1 format
:TRIGger:LEVel:LOW <level>, <source> (see page 1357)	:TRIGger:LEVel:LOW? <source> (see page 1357)	<level> ::= .75 x full-scale voltage from center screen in NR3 format. <source> ::= CHANNEL<n> <n> ::= 1 to (# analog channels) in NR1 format
:TRIGger:MODE <mode> (see page 1358)	:TRIGger:MODE? (see page 1358)	<mode> ::= {EDGE GLITCh PATtern TV DELay EBURst OR RUNT SHOLd TRANSition SBUS{1 2}} <return_value> ::= {<mode> <none>} <none> ::= query returns "NONE" if the :TIMEbase:MODE is ROLL or XY

Table 64 General :TRIGger Commands Summary (continued)

Command	Query	Options and Query Returns
:TRIGger:NREJect {{0 OFF} {1 ON}} (see page 1359)	:TRIGger:NREJect? (see page 1359)	{0 1}
:TRIGger:SWEep <sweep> (see page 1360)	:TRIGger:SWEep? (see page 1360)	<sweep> ::= {AUTO NORMAl}

Table 65 :TRIGger:DElay Commands Summary

Command	Query	Options and Query Returns
:TRIGger:DElay:ARM:SL OPe <slope> (see page 1362)	:TRIGger:DElay:ARM:SL OPe? (see page 1362)	<slope> ::= {NEGative POSitive}
:TRIGger:DElay:ARM:SO URce <source> (see page 1363)	:TRIGger:DElay:ARM:SO URce? (see page 1363)	<source> ::= {CHANnel<n> DIGital<d>} <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:TRIGger:DElay:TDElay :TIME <time_value> (see page 1364)	:TRIGger:DElay:TDElay :TIME? (see page 1364)	<time_value> ::= time in seconds in NR3 format
:TRIGger:DElay:TRIGge r:COUNT <count> (see page 1365)	:TRIGger:DElay:TRIGge r:COUNT? (see page 1365)	<count> ::= integer in NR1 format
:TRIGger:DElay:TRIGge r:SLOPe <slope> (see page 1366)	:TRIGger:DElay:TRIGge r:SLOPe? (see page 1366)	<slope> ::= {NEGative POSitive}
:TRIGger:DElay:TRIGge r:SOURce <source> (see page 1367)	:TRIGger:DElay:TRIGge r:SOURce? (see page 1367)	<source> ::= {CHANnel<n> DIGital<d>} <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format

Table 66 :TRIGger:EBURst Commands Summary

Command	Query	Options and Query Returns
:TRIGger:EBURst:COUNT <count> (see page 1369)	:TRIGger:EBURst:COUNT ? (see page 1369)	<count> ::= integer in NR1 format
:TRIGger:EBURst:IDLE <time_value> (see page 1370)	:TRIGger:EBURst:IDLE? (see page 1370)	<time_value> ::= time in seconds in NR3 format
:TRIGger:EBURst:SLOPe <slope> (see page 1371)	:TRIGger:EBURst:SLOPe ? (see page 1371)	<slope> ::= {NEGative POSitive}
:TRIGger:EBURst:SOURc e <source> (see page 1372)	:TRIGger:EBURst:SOURc e? (see page 1372)	<source> ::= {CHANnel<n> DIGital<d>} <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format

Table 67 :TRIGger[:EDGE] Commands Summary

Command	Query	Options and Query Returns
:TRIGger[:EDGE]:COUPl ing {AC DC LFReject} (see page 1375)	:TRIGger[:EDGE]:COUPl ing? (see page 1375)	{AC DC LFReject}
:TRIGger[:EDGE]:LEVel <level> [,<source>] (see page 1376)	:TRIGger[:EDGE]:LEVel ? [<source>] (see page 1376)	For internal triggers, <level> ::= .75 x full-scale voltage from center screen in NR3 format. For external triggers, <level> ::= ±(external range setting) in NR3 format. For digital channels (MSO models), <level> ::= ±8 V. <source> ::= {CHANnel<n> EXTErnal} for DSO models <source> ::= {CHANnel<n> DIGital<d> EXTErnal } for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format

Table 67 :TRIGger[:EDGE] Commands Summary (continued)

Command	Query	Options and Query Returns
:TRIGger[:EDGE]:REJect {OFF LFReject HFReject} (see page 1377)	:TRIGger[:EDGE]:REJect? (see page 1377)	{OFF LFReject HFReject}
:TRIGger[:EDGE]:SLOPe <polarity> (see page 1378)	:TRIGger[:EDGE]:SLOPe? (see page 1378)	<polarity> ::= {POSitive NEGative EITHER ALTernate}
:TRIGger[:EDGE]:SOURCe <source> (see page 1379)	:TRIGger[:EDGE]:SOURCe? (see page 1379)	<source> ::= {CHANnel<n> EXTErnal LINE WGEN} for the DSO models <source> ::= {CHANnel<n> DIGital<d> EXTErnal LINE WGEN} for the MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format

Table 68 :TRIGger:GLITCh Commands Summary

Command	Query	Options and Query Returns
:TRIGger:GLITCh:GREAt erthan <greater_than_time>[suffix] (see page 1382)	:TRIGger:GLITCh:GREAt erthan? (see page 1382)	<greater_than_time> ::= floating-point number in NR3 format [suffix] ::= {s ms us ns ps}
:TRIGger:GLITCh:LESSt han <less_than_time>[suffix] (see page 1383)	:TRIGger:GLITCh:LESSt han? (see page 1383)	<less_than_time> ::= floating-point number in NR3 format [suffix] ::= {s ms us ns ps}

Table 68 :TRIGger:GLITch Commands Summary (continued)

Command	Query	Options and Query Returns
:TRIGger:GLITch:LEVel <level> [<source>] (see page 1384)	:TRIGger:GLITch:LEVel ? (see page 1384)	For internal triggers, <level> ::= .75 x full-scale voltage from center screen in NR3 format. For external triggers (DSO models), <level> ::= ±(external range setting) in NR3 format. For digital channels (MSO models), <level> ::= ±8 V. <source> ::= {CHANnel<n> EXTernal} for DSO models <source> ::= {CHANnel<n> DIGital<d>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:TRIGger:GLITch:POLarity <polarity> (see page 1385)	:TRIGger:GLITch:POLarity? (see page 1385)	<polarity> ::= {POSitive NEGative}
:TRIGger:GLITch:QUALifier <qualifier> (see page 1386)	:TRIGger:GLITch:QUALifier? (see page 1386)	<qualifier> ::= {GREaterthan LESSthan RANGE}
:TRIGger:GLITch:RANGe <less_than_time>[suffix], <greater_than_time>[suffix] (see page 1387)	:TRIGger:GLITch:RANGe ? (see page 1387)	<less_than_time> ::= 15 ns to 10 seconds in NR3 format <greater_than_time> ::= 10 ns to 9.99 seconds in NR3 format [suffix] ::= {s ms us ns ps}
:TRIGger:GLITch:SOURce <source> (see page 1388)	:TRIGger:GLITch:SOURce? (see page 1388)	<source> ::= {CHANnel<n> DIGital<d>} <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format

Table 69 :TRIGger:OR Commands Summary

Command	Query	Options and Query Returns
:TRIGger:OR <string> (see page 1390)	:TRIGger:OR? (see page 1390)	<string> ::= "nn...n" where n ::= {R F E X} R = rising edge, F = falling edge, E = either edge, X = don't care. Each character in the string is for an analog or digital channel as shown on the front panel display.

Table 70 :TRIGger:PATtern Commands Summary

Command	Query	Options and Query Returns
:TRIGger:PATtern <string>[,<edge_source>,<edge>] (see page 1392)	:TRIGger:PATtern? (see page 1393)	<string> ::= "nn...n" where n ::= {0 1 X R F} when <base> = ASCii <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X \$} when <base> = HEX <edge_source> ::= {CHANnel<n> NONE} for DSO models <edge_source> ::= {CHANnel<n> DIGital<d> NONE} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <edge> ::= {POSitive NEGative}
:TRIGger:PATtern:FORM at <base> (see page 1394)	:TRIGger:PATtern:FORM at? (see page 1394)	<base> ::= {ASCii HEX}
:TRIGger:PATtern:GREa terthan <greater_than_time>[suffix] (see page 1395)	:TRIGger:PATtern:GREa terthan? (see page 1395)	<greater_than_time> ::= floating-point number in NR3 format [suffix] ::= {s ms us ns ps}
:TRIGger:PATtern:LESS than <less_than_time>[suffix] (see page 1396)	:TRIGger:PATtern:LESS than? (see page 1396)	<less_than_time> ::= floating-point number in NR3 format [suffix] ::= {s ms us ns ps}

Table 70 :TRIGger:PATtern Commands Summary (continued)

Command	Query	Options and Query Returns
:TRIGger:PATtern:QUALifier <qualifier> (see page 1397)	:TRIGger:PATtern:QUALifier? (see page 1397)	<qualifier> ::= {ENTERed GREATERthan LESSthan INRange OUTRange TIMEout}
:TRIGger:PATtern:RANGe <less_than_time>[suffix], <greater_than_time>[suffix] (see page 1398)	:TRIGger:PATtern:RANGe? (see page 1398)	<less_than_time> ::= 15 ns to 10 seconds in NR3 format <greater_than_time> ::= 10 ns to 9.99 seconds in NR3 format [suffix] ::= {s ms us ns ps}

Table 71 :TRIGger:RUNT Commands Summary

Command	Query	Options and Query Returns
:TRIGger:RUNT:POLarity <polarity> (see page 1400)	:TRIGger:RUNT:POLarity? (see page 1400)	<polarity> ::= {POSitive NEGative EITHER}
:TRIGger:RUNT:QUALifier <qualifier> (see page 1401)	:TRIGger:RUNT:QUALifier? (see page 1401)	<qualifier> ::= {GREATERthan LESSthan NONE}
:TRIGger:RUNT:SOURce <source> (see page 1402)	:TRIGger:RUNT:SOURce? (see page 1402)	<source> ::= CHANNEL<n> <n> ::= 1 to (# analog channels) in NR1 format
:TRIGger:RUNT:TIME <time>[suffix] (see page 1403)	:TRIGger:RUNT:TIME? (see page 1403)	<time> ::= floating-point number in NR3 format [suffix] ::= {s ms us ns ps}

Table 72 :TRIGger:SHOLd Commands Summary

Command	Query	Options and Query Returns
:TRIGger:SHOLd:SLOPe <slope> (see page 1405)	:TRIGger:SHOLd:SLOPe? (see page 1405)	<slope> ::= {NEGative POSitive}
:TRIGger:SHOLd:SOURce:CLOCK <source> (see page 1406)	:TRIGger:SHOLd:SOURce:CLOCK? (see page 1406)	<source> ::= {CHANNEL<n> DIGital<d>} <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format

Table 72 :TRIGger:SHOLd Commands Summary (continued)

Command	Query	Options and Query Returns
:TRIGger:SHOLd:SOURce :DATA <source> (see page 1407)	:TRIGger:SHOLd:SOURce :DATA? (see page 1407)	<source> ::= {CHANnel<n> DIGital<d>} <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:TRIGger:SHOLd:TIME:H OLD <time>[suffix] (see page 1408)	:TRIGger:SHOLd:TIME:H OLD? (see page 1408)	<time> ::= floating-point number in NR3 format [suffix] ::= {s ms us ns ps}
:TRIGger:SHOLd:TIME:S ETup <time>[suffix] (see page 1409)	:TRIGger:SHOLd:TIME:S ETup? (see page 1409)	<time> ::= floating-point number in NR3 format [suffix] ::= {s ms us ns ps}

Table 73 :TRIGger:TRANSition Commands Summary

Command	Query	Options and Query Returns
:TRIGger:TRANSition:Q UALifier <qualifier> (see page 1411)	:TRIGger:TRANSition:Q UALifier? (see page 1411)	<qualifier> ::= {GREaterthan LESSthan}
:TRIGger:TRANSition:S LOPe <slope> (see page 1412)	:TRIGger:TRANSition:S LOPe? (see page 1412)	<slope> ::= {NEGative POSitive}
:TRIGger:TRANSition:S OURce <source> (see page 1413)	:TRIGger:TRANSition:S OURce? (see page 1413)	<source> ::= CHANnel<n> <n> ::= 1 to (# analog channels) in NR1 format
:TRIGger:TRANSition:T IME <time>[suffix] (see page 1414)	:TRIGger:TRANSition:T IME? (see page 1414)	<time> ::= floating-point number in NR3 format [suffix] ::= {s ms us ns ps}

Table 74 :TRIGger:TV Commands Summary

Command	Query	Options and Query Returns
:TRIGger:TV:LINE <line number> (see page 1416)	:TRIGger:TV:LINE? (see page 1416)	<line number> ::= integer in NR1 format
:TRIGger:TV:MODE <tv mode> (see page 1417)	:TRIGger:TV:MODE? (see page 1417)	<tv mode> ::= {FIELD1 FIELD2 AFIELDS ALINES LINE LFIELD1 LFIELD2 LALTERnate}
:TRIGger:TV:POLarity <polarity> (see page 1418)	:TRIGger:TV:POLarity? (see page 1418)	<polarity> ::= {POSitive NEGative}
:TRIGger:TV:SOURce <source> (see page 1419)	:TRIGger:TV:SOURce? (see page 1419)	<source> ::= {CHANnel<n>} <n> ::= 1 to (# analog channels) in NR1 format
:TRIGger:TV:STANdard <standard> (see page 1420)	:TRIGger:TV:STANdard? (see page 1420)	<standard> ::= {NTSC PAL PALM SECam} <standard> ::= {GENeric {P480L60HZ P480} {P720L60HZ P720} {P1080L24HZ P1080} P1080L25HZ P1080L50HZ P1080L60HZ {I1080L50HZ I1080} I1080L60HZ} with extended video triggering license
:TRIGger:TV:UDTV:ENUM ber <count> (see page 1421)	:TRIGger:TV:UDTV:ENUM ber? (see page 1421)	<count> ::= edge number in NR1 format
:TRIGger:TV:UDTV:HSYN c {{0 OFF} {1 ON}} (see page 1422)	:TRIGger:TV:UDTV:HSYN c? (see page 1422)	{0 1}
:TRIGger:TV:UDTV:HTIM e <time> (see page 1423)	:TRIGger:TV:UDTV:HTIM e? (see page 1423)	<time> ::= seconds in NR3 format
:TRIGger:TV:UDTV:PGTH an <min_time> (see page 1424)	:TRIGger:TV:UDTV:PGTH an? (see page 1424)	<min_time> ::= seconds in NR3 format

Table 75 :TRIGger:ZONE Commands Summary

Command	Query	Options and Query Returns
:TRIGger:ZONE:SOURce <source> (see page 1426)	:TRIGger:ZONE:SOURce? (see page 1426)	<source> ::= {CHANnel<n>} <n> ::= 1 to (# analog channels) in NR1 format
:TRIGger:ZONE:STATe { {0 OFF} {1 ON} } (see page 1427)	:TRIGger:ZONE:STATe? (see page 1427)	{0 1}
:TRIGger:ZONE<n>:MODE <mode> (see page 1428)	:TRIGger:ZONE<n>:MODE ? (see page 1428)	<mode> ::= {INTERsect NOTintersect} <n> ::= 1-2 in NR1 format
:TRIGger:ZONE<n>:PLAC ement <width>, <height>, <x_center>, <y_center> (see page 1429)	:TRIGger:ZONE<n>:PLAC ement? (see page 1429)	<width> ::= width of zone in seconds <height> ::= height of zone in volts <x_center> ::= center of zone in seconds <y_center> ::= center of zone in volts <n> ::= 1-2 in NR1 format
n/a	:TRIGger:ZONE<n>:VALi dity? (see page 1430)	<value> ::= {VALid INValid OSCRreen} <n> ::= 1-2 in NR1 format
:TRIGger:ZONE<n>:STAT e { {0 OFF} {1 ON} } (see page 1431)	:TRIGger:ZONE<n>:STAT e? (see page 1431)	{0 1} <n> ::= 1-2 in NR1 format

Table 76 :WAVEform Commands Summary

Command	Query	Options and Query Returns
:WAVEform:BYTeorder <value> (see page 1441)	:WAVEform:BYTeorder? (see page 1441)	<value> ::= {LSBFirst MSBFirst}
n/a	:WAVEform:COUNt? (see page 1442)	<count> ::= an integer from 1 to 65536 in NR1 format

Table 76 :WAVeform Commands Summary (continued)

Command	Query	Options and Query Returns
n/a	:WAVeform:DATA? (see page 1443)	<binary block length bytes>, <binary data> For example, to transmit 1000 bytes of data, the syntax would be: #800001000<1000 bytes of data><NL> 8 is the number of digits that follow 00001000 is the number of bytes to be transmitted <1000 bytes of data> is the actual data
:WAVeform:FORMat <value> (see page 1445)	:WAVeform:FORMat? (see page 1445)	<value> ::= {WORD BYTE ASCII}
:WAVeform:POINTs <wfm_points> (see page 1446)	:WAVeform:POINTs? (see page 1446)	<wfm_points> ::= {<#_points> <points_mode>} <#_points> ::= an integer between 100 and 10000000 <points_mode> ::= {NORMal MAXimum RAW}
:WAVeform:POINTs:MODE <points_mode> (see page 1448)	:WAVeform:POINTs:MODE ? (see page 1449)	<points_mode> ::= {NORMal MAXimum RAW}

Table 76 :WAVEform Commands Summary (continued)

Command	Query	Options and Query Returns
n/a	:WAVEform:PREamble? (see page 1450)	<pre><preamble_block> ::= <format NR1>, <type NR1>,<points NR1>,<count NR1>, <xincrement NR3>, <xorigin NR3>, <xreference NR1>,<yincrement NR3>, <yorigin NR3>, <yreference NR1></pre> <pre><format> ::= an integer in NR1 format:</pre> • 0 for BYTE format • 1 for WORD format • 2 for ASCII format <pre><type> ::= an integer in NR1 format:</pre> • 0 for NORMAl type • 1 for PEAK detect type • 3 for AVERAge type • 4 for HRESolution type <pre><count> ::= Average count, or 1 if PEAK detect type or NORMAl; an integer in NR1 format</pre>
:WAVEform:SEGmented:ALL {0 OFF} {1 ON} (see page 1453)	:WAVEform:SEGmented:ALL? (see page 1453)	<pre><setting> ::= {0 1}</pre>
n/a	:WAVEform:SEGmented:COUNT? (see page 1454)	<pre><count> ::= an integer from 2 to 1000 in NR1 format</pre>
n/a	:WAVEform:SEGmented:TAG? (see page 1455)	<pre><time_tag> ::= in NR3 format</pre>
n/a	:WAVEform:SEGmented:XLIST? <xlist_type> (see page 1456)	<pre><xlist_type> ::= {RELXorigin ABSXorigin TTAG}</pre> <pre><return_value> ::= X-info for all segments</pre>
:WAVEform:SOURce <source> (see page 1457)	:WAVEform:SOURce? (see page 1457)	<pre><source> ::= {CHANnel<n> FUNCTION<m> MATH<m> SBUS} for DSO models</pre> <pre><source> ::= {CHANnel<n> POD{1 2} BUS{1 2} FUNCTION<m> MATH<m> SBUS} for MSO models</pre> <pre><n> ::= 1 to (# analog channels) in NR1 format</pre> <pre><m> ::= 1 to (# math functions) in NR1 format</pre>

Table 76 :WAVeform Commands Summary (continued)

Command	Query	Options and Query Returns
:WAVeform:SOURce:SUBS ource <subsource> (see page 1461)	:WAVeform:SOURce:SUBS ource? (see page 1461)	<subsource> ::= {{SUB0 RX MOSI} {SUB1 TX MISO}}
n/a	:WAVeform:TYPE? (see page 1462)	<return_mode> ::= {NORM PEAK AVER HRES}
:WAVeform:UNSigned {{0 OFF} {1 ON}} (see page 1463)	:WAVeform:UNSigned? (see page 1463)	{0 1}
:WAVeform:VIEW <view> (see page 1464)	:WAVeform:VIEW? (see page 1464)	<view> ::= {MAIN ALL}
n/a	:WAVeform:XINCrement? (see page 1465)	<return_value> ::= x-increment in the current preamble in NR3 format
n/a	:WAVeform:XORigin? (see page 1466)	<return_value> ::= x-origin value in the current preamble in NR3 format
n/a	:WAVeform:XREFerence? (see page 1467)	<return_value> ::= 0 (x-reference value in the current preamble in NR1 format)
n/a	:WAVeform:YINCrement? (see page 1468)	<return_value> ::= y-increment value in the current preamble in NR3 format
n/a	:WAVeform:YORigin? (see page 1469)	<return_value> ::= y-origin in the current preamble in NR3 format
n/a	:WAVeform:YREFerence? (see page 1470)	<return_value> ::= y-reference value in the current preamble in NR1 format

Table 77 :WGEN<w> Commands Summary

Command	Query	Options and Query Returns
:WGEN<w>:ARbitrary:BYTeorder <order> (see page 1476)	:WGEN<w>:ARbitrary:BYTeorder? (see page 1476)	<order> ::= {MSBFirst LSBFirst} <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:ARbitrary:DATA {<binary> <value>, <value> ...} (see page 1477)	n/a	<binary> ::= floating point values between -1.0 to +1.0 in IEEE 488.2 binary block format <value> ::= floating point values between -1.0 to +1.0 in comma-separated format <w> ::= 1 to (# WaveGen outputs) in NR1 format
n/a	:WGEN<w>:ARbitrary:DATA:ATTRibute:POINTs? (see page 1480)	<points> ::= number of points in NR1 format <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:ARbitrary:DATA:CLear (see page 1481)	n/a	<w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:ARbitrary:DATA:DAC {<binary> <value>, <value> ...} (see page 1482)	n/a	<binary> ::= decimal 16-bit integer values between -512 to +511 in IEEE 488.2 binary block format <value> ::= decimal integer values between -512 to +511 in comma-separated NR1 format <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:ARbitrary:INTERpolate {{0 OFF} {1 ON}} (see page 1483)	:WGEN<w>:ARbitrary:INTERpolate? (see page 1483)	{0 1} <w> ::= 1 to (# WaveGen outputs) in NR1 format

Table 77 :WGEN<w> Commands Summary (continued)

Command	Query	Options and Query Returns
:WGEN<w>:ARBitrary:STORe <source> (see page 1484)	n/a	<source> ::= {CHANnel<n> WMEMory<r> FUNctioN<m> MATH<m>} <n> ::= 1 to (# analog channels) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:FREQuency <frequency> (see page 1485)	:WGEN<w>:FREQuency? (see page 1485)	<frequency> ::= frequency in Hz in NR3 format <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:FUNctioN <signal> (see page 1486)	:WGEN<w>:FUNctioN? (see page 1489)	<signal> ::= {SINusoid SQUare RAMP PULSe NOISe DC SINC EXPRise EXPFall CARDiac GAUSSian ARBitrary} <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:FUNctioN:PULSe:WIDTh <width> (see page 1490)	:WGEN<w>:FUNctioN:PULSe:WIDTh? (see page 1490)	<width> ::= pulse width in seconds in NR3 format <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:FUNctioN:RAMP:SYMMetry <percent> (see page 1491)	:WGEN<w>:FUNctioN:RAMP:SYMMetry? (see page 1491)	<percent> ::= symmetry percentage from 0% to 100% in NR1 format <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:FUNctioN:SQUare:DCYClE <percent> (see page 1492)	:WGEN<w>:FUNctioN:SQUare:DCYClE? (see page 1492)	<percent> ::= duty cycle percentage from 20% to 80% in NR1 format <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:MODulation:AM:DEPTh <percent> (see page 1493)	:WGEN<w>:MODulation:AM:DEPTh? (see page 1493)	<percent> ::= AM depth percentage from 0% to 100% in NR1 format <w> ::= 1 in NR1 format

Table 77 :WGEN<w> Commands Summary (continued)

Command	Query	Options and Query Returns
:WGEN<w>:MODulation:AM:FREQuency <frequency> (see page 1494)	:WGEN<w>:MODulation:AM:FREQuency? (see page 1494)	<frequency> ::= modulating waveform frequency in Hz in NR3 format <w> ::= 1 in NR1 format
:WGEN<w>:MODulation:FM:DEVIation <frequency> (see page 1495)	:WGEN<w>:MODulation:FM:DEVIation? (see page 1495)	<frequency> ::= frequency deviation in Hz in NR3 format <w> ::= 1 in NR1 format
:WGEN<w>:MODulation:FM:FREQuency <frequency> (see page 1496)	:WGEN<w>:MODulation:FM:FREQuency? (see page 1496)	<frequency> ::= modulating waveform frequency in Hz in NR3 format <w> ::= 1 in NR1 format
:WGEN<w>:MODulation:FSKey:FREQuency <percent> (see page 1497)	:WGEN<w>:MODulation:FSKey:FREQuency? (see page 1497)	<frequency> ::= hop frequency in Hz in NR3 format <w> ::= 1 in NR1 format
:WGEN<w>:MODulation:FSKey:RATE <rate> (see page 1498)	:WGEN<w>:MODulation:FSKey:RATE? (see page 1498)	<rate> ::= FSK modulation rate in Hz in NR3 format <w> ::= 1 in NR1 format
:WGEN<w>:MODulation:FUNction <shape> (see page 1499)	:WGEN<w>:MODulation:FUNction? (see page 1499)	<shape> ::= {SINusoid SQUare RAMP} <w> ::= 1 in NR1 format
:WGEN<w>:MODulation:FUNction:RAMP:SYMMetry <percent> (see page 1500)	:WGEN<w>:MODulation:FUNction:RAMP:SYMMetry? (see page 1500)	<percent> ::= symmetry percentage from 0% to 100% in NR1 format <w> ::= 1 in NR1 format
:WGEN<w>:MODulation:NOISE <percent> (see page 1501)	:WGEN<w>:MODulation:NOISE? (see page 1501)	<percent> ::= 0 to 100 <w> ::= 1 in NR1 format
:WGEN<w>:MODulation:STATe {{0 OFF} {1 ON}} (see page 1502)	:WGEN<w>:MODulation:STATe? (see page 1502)	{0 1} <w> ::= 1 in NR1 format
:WGEN<w>:MODulation:TYPE <type> (see page 1503)	:WGEN<w>:MODulation:TYPE? (see page 1503)	<type> ::= {AM FM FSK} <w> ::= 1 in NR1 format
:WGEN<w>:OUTPut {{0 OFF} {1 ON}} (see page 1505)	:WGEN<w>:OUTPut? (see page 1505)	{0 1} <w> ::= 1 to (# WaveGen outputs) in NR1 format

Table 77 :WGEN<w> Commands Summary (continued)

Command	Query	Options and Query Returns
:WGEN<w>:OUTPut:LOAD <impedance> (see page 1506)	:WGEN<w>:OUTPut:LOAD? (see page 1506)	<impedance> ::= {ONEMeg FIFTy} <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:OUTPut:MODE <mode> (see page 1507)	:WGEN<w>:OUTPut:MODE? (see page 1507)	<mode> ::= {NORMal SINGLE}
:WGEN<w>:OUTPut:POLar ity <polarity> (see page 1508)	:WGEN<w>:OUTPut:POLar ity? (see page 1508)	<polarity> ::= {NORMal INVERTed} <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:OUTPut:SINGl e (see page 1509)	n/a	n/a
:WGEN<w>:PERiod <period> (see page 1510)	:WGEN<w>:PERiod? (see page 1510)	<period> ::= period in seconds in NR3 format <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:RST (see page 1511)	n/a	<w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:TRACk {{0 OFF} {1 ON}} (see page 1512)	:WGEN<w>:TRACk? (see page 1512)	{0 1} <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:TRACk:AMPLit ude {{0 OFF} {1 ON}} (see page 1513)	:WGEN<w>:TRACk:AMPLit ude? (see page 1513)	{0 1} <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:TRACk:CSIGna l (see page 1514)	n/a	<w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:TRACk:FREQue ncy {{0 OFF} {1 ON}} (see page 1515)	:WGEN<w>:TRACk:FREQue ncy? (see page 1515)	{0 1} <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:TRACk:PHASe <phase> (see page 1516)	:WGEN<w>:TRACk:PHASe? (see page 1516)	<w> ::= 1 to (# WaveGen outputs) in NR1 format <phase> ::= -360 to +360 in NR1 format
:WGEN<w>:VOLtage <amplitude> (see page 1517)	:WGEN<w>:VOLtage? (see page 1517)	<amplitude> ::= amplitude in volts in NR3 format <w> ::= 1 to (# WaveGen outputs) in NR1 format

Table 77 :WGEN<w> Commands Summary (continued)

Command	Query	Options and Query Returns
:WGEN<w>:VOLTage:HIGH <high> (see page 1518)	:WGEN<w>:VOLTage:HIGH? (see page 1518)	<high> ::= high-level voltage in volts, in NR3 format <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:VOLTage:LOW <low> (see page 1519)	:WGEN<w>:VOLTage:LOW? (see page 1519)	<low> ::= low-level voltage in volts, in NR3 format <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:VOLTage:OFFS et <offset> (see page 1520)	:WGEN<w>:VOLTage:OFFS et? (see page 1520)	<offset> ::= offset in volts in NR3 format <w> ::= 1 to (# WaveGen outputs) in NR1 format

Table 78 :WMEMemory<r> Commands Summary

Command	Query	Options and Query Returns
:WMEMemory<r>:CLear (see page 1523)	n/a	<r> ::= 1 to (# ref waveforms) in NR1 format
:WMEMemory<r>:DISPlay { {0 OFF} {1 ON} } (see page 1524)	:WMEMemory<r>:DISPlay? (see page 1524)	<r> ::= 1 to (# ref waveforms) in NR1 format { 0 1 }
:WMEMemory<r>:LABel <string> (see page 1525)	:WMEMemory<r>:LABel? (see page 1525)	<r> ::= 1 to (# ref waveforms) in NR1 format <string> ::= any series of 10 or less ASCII characters enclosed in quotation marks
:WMEMemory<r>:SAVE <source> (see page 1526)	n/a	<r> ::= 1 to (# ref waveforms) in NR1 format <source> ::= {CHANnel<n> FUNCTION<m> MATH<m>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format NOTE: Math functions whose x-axis is not frequency can be saved as reference waveforms.

Table 78 :WMEemory<r> Commands Summary (continued)

Command	Query	Options and Query Returns
:WMEemory<r>:SKEW <skew> (see page 1527)	:WMEemory<r>:SKEW? (see page 1527)	<r> ::= 1 to (# ref waveforms) in NR1 format <skew> ::= time in seconds in NR3 format
:WMEemory<r>:YOFFset <offset>[suffix] (see page 1528)	:WMEemory<r>:YOFFset? (see page 1528)	<r> ::= 1 to (# ref waveforms) in NR1 format <offset> ::= vertical offset value in NR3 format [suffix] ::= {V mV}
:WMEemory<r>:YRANge <range> (see page 1529)	:WMEemory<r>:YRANge? (see page 1529)	<r> ::= 1 to (# ref waveforms) in NR1 format <range> ::= vertical full-scale range value in NR3 format
:WMEemory<r>:YSCale <scale> (see page 1530)	:WMEemory<r>:YSCale? (see page 1530)	<r> ::= 1 to (# ref waveforms) in NR1 format <scale> ::= vertical units per division value in NR3 format

Syntax Elements

- **"Number Format"** on page 206
- **"<NL> (Line Terminator)"** on page 206
- **"[] (Optional Syntax Terms)"** on page 206
- **"{ } (Braces)"** on page 206
- **"::= (Defined As)"** on page 206
- **"< > (Angle Brackets)"** on page 207
- **"... (Ellipsis)"** on page 207
- **"n,...,p (Value Ranges)"** on page 207
- **"d (Digits)"** on page 207
- **"Quoted ASCII String"** on page 207
- **"Definite-Length Block Response Data"** on page 207

Number Format

NR1 specifies integer data.

NR3 specifies exponential data in floating point format (for example, -1.0E-3).

<NL> (Line Terminator)

<NL> = new line or linefeed (ASCII decimal 10).

The line terminator, or a leading colon, will send the parser to the "root" of the command tree.

[] (Optional Syntax Terms)

Items enclosed in square brackets, [], are optional.

{ } (Braces)

When several items are enclosed by braces, { }, only one of these elements may be selected. Vertical line (|) indicates "or". For example, {ON | OFF} indicates that only ON or OFF may be selected, not both.

::= (Defined As)

::= means "defined as".

For example, <A> ::= indicates that <A> can be replaced by in any statement containing <A>.

< > (Angle Brackets)

< > Angle brackets enclose words or characters that symbolize a program code parameter or an interface command.

... (Ellipsis)

... An ellipsis (trailing dots) indicates that the preceding element may be repeated one or more times.

n,...,p (Value Ranges)

n,...,p ::= all integers between n and p inclusive.

d (Digits)

d ::= A single ASCII numeric character 0 – 9.

Quoted ASCII String

A quoted ASCII string is a string delimited by either double quotes (") or single quotes ('). Some command parameters require a quoted ASCII string. For example, when using the Keysight VISA COM library in Visual Basic, the command:

```
myScope.WriteString ":CHANNEL1:LABEL 'One' "
```

has a quoted ASCII string of:

```
'One'
```

In order to read quoted ASCII strings from query return values, some programming languages require special handling or syntax.

Definite-Length Block Response Data

Definite-length block response data allows any type of device-dependent data to be transmitted over the system interface as a series of 8-bit binary data bytes. This is particularly useful for sending large quantities of data or 8-bit extended ASCII codes. This syntax is a pound sign (#) followed by a non-zero digit representing the number of digits in the decimal integer. After the non-zero digit is the decimal integer that states the number of 8-bit data bytes being sent. This is followed by the actual data.

For example, for transmitting 1000 bytes of data, the syntax would be

```
#800001000<1000 bytes of data> <NL>
```

8 is the number of digits that follow

00001000 is the number of bytes to be transmitted

<1000 bytes of data> is the actual data

6 Common (*) Commands

Commands defined by IEEE 488.2 standard that are common to all instruments.
See "[Introduction to Common \(*\) Commands](#)" on page 213.

Table 79 Common (*) Commands Summary

Command	Query	Options and Query Returns
*CLS (see page 215)	n/a	n/a
*ESE <mask> (see page 216)	*ESE? (see page 216)	<mask> ::= 0 to 255; an integer in NR1 format: Bit Weight Name Enables --- ----- 7 128 PON Power On 6 64 URQ User Request 5 32 CME Command Error 4 16 EXE Execution Error 3 8 DDE Dev. Dependent Error 2 4 QYE Query Error 1 2 RQL Request Control 0 1 OPC Operation Complete
n/a	*ESR? (see page 218)	<status> ::= 0 to 255; an integer in NR1 format
n/a	*IDN? (see page 218)	KEYSIGHT TECHNOLOGIES,<model>,<serial number>,X.XX.XX <model> ::= the model number of the instrument <serial number> ::= the serial number of the instrument <X.XX.XX> ::= the software revision of the instrument
n/a	*LRN? (see page 221)	<learn_string> ::= current instrument setup as a block of data in IEEE 488.2 # format

Table 79 Common (*) Commands Summary (continued)

Command	Query	Options and Query Returns
*OPC (see page 222)	*OPC? (see page 222)	ASCII "1" is placed in the output queue when all pending device operations have completed.
n/a	*OPT? (see page 223)	<pre> <return_value> ::= 0,0,<license info> <license info> ::= <All field>, <reserved>, <reserved>, <MSO>, <reserved>, <Memory>, <Low Speed Serial>, <Automotive Serial>, <FlexRay Serial>, <Frequency Response Analysis>, <Power Measurements>, <RS-232/UART Serial>, <Segmented Memory>, <Mask Test>, <reserved>, <Bandwidth>, <reserved>, <reserved>, <reserved>, <reserved>, <reserved>, <reserved>, <I2S Serial>, <reserved>, <Educator's Kit>, <Waveform Generator>, <MIL-1553/ARINC 429 Serial>, <Extended Video>, <Advanced Math>, <reserved>, <reserved>, <reserved>, <Digital Voltmeter>, <reserved>, <Spectrum Visualizer>, <reserved>, <USB 2.0 Low/Full Speed>, <USB 2.0 High Speed>, <USB 2.0 Signal Quality Analysis >, <Remote Command Logging>, <reserved>, <SENT Serial>, <CAN FD Serial>, <CXPI Serial>, <reserved>, <reserved>, <reserved>, <reserved>, <Manchester/NRZ Serial>, <USB PD Serial>, <reserved>, <Automotive Software>, <General Purpose Software>, <Aerospace Software>, <Power Supply Test Software>, <USB Test Software>, <reserved>, <Software Bundle> </pre>

Table 79 Common (*) Commands Summary (continued)

Command	Query	Options and Query Returns
n/a	*OPT? (see page 223) (cont'd)	<All field> ::= {0 All} <reserved> ::= 0 <MSO> ::= {0 MSO} <Memory> ::= {0 MEMUP} <Low Speed Serial> ::= {0 EMBD} <Automotive Serial> ::= {0 AUTO} <FlexRay Serial> ::= {0 FLEX} <Frequency Response Analysis> ::= {0 FRA} <Power Measurements> ::= {0 PWR} <RS-232/UART Serial> ::= {0 COMP} <Mask Test> ::= {0 MASK} <Bandwidth> ::= {0 BW20 BW50} <I2S Serial> ::= {0 AUDIO} <Educator's Kit> ::= {0 EDK} <Waveform Generator> ::= {0 WAVEGEN} <MIL-1553/ARINC 429 Serial> ::= {0 AERO} <Extended Video> ::= {0 VID} <Digital Voltmeter> ::= {0 DVM} <Spectrum Visualizer> ::= {0 ASV} <USB 2.0 Low/Full Speed> ::= {0 USF} <USB 2.0 High Speed> ::= {0 U2H} <USB 2.0 Signal Quality Analysis> ::= {0 USBSQ} <Remote Command Logging> ::= {0 RML}

Table 79 Common (*) Commands Summary (continued)

Command	Query	Options and Query Returns
n/a	*OPT? (see page 223) (cont'd)	<SENT Serial> ::= {0 SENSOR} <CAN FD Serial> ::= {0 CANFD} <CXPI Serial> ::= {0 CXPI} <Manchester/NRZ Serial> ::= {0 NRZ} <USB PD Serial> ::= {0 USBPD} <Automotive Software> ::= {0 D4000AUTA} <General Purpose Software> ::= {0 D4000GENA} <Aerospace Software> ::= {0 D4000AERA} <Power Supply Test Software> ::= {0 D4000PWRA} <USB Test Software> ::= {0 D4000USBA} <Near Field Communications (NFC) Software> ::= {0 D4000NFCA} <Software Bundle> ::= {0 D4000BDLA}
*RCL <value> (see page 225)	n/a	<value> ::= {0 1 4 5 6 7 8 9}
*RST (see page 226)	n/a	See *RST (Reset) (see page 226)
*SAV <value> (see page 229)	n/a	<value> ::= {0 1 4 5 6 7 8 9}
*SRE <mask> (see page 230)	*SRE? (see page 231)	<mask> ::= sum of all bits that are set, 0 to 255; an integer in NR1 format. <mask> ::= following values: Bit Weight Name Enables --- 7 128 OPER Operation Status Reg 6 64 ---- (Not used.) 5 32 ESB Event Status Bit 4 16 MAV Message Available 3 8 ---- (Not used.) 2 4 MSG Message 1 2 USR User 0 1 TRG Trigger

Table 79 Common (*) Commands Summary (continued)

Command	Query	Options and Query Returns
n/a	*STB? (see page 232)	<value> ::= 0 to 255; an integer in NR1 format, as shown in the following: Bit Weight Name "1" Indicates --- 7 128 OPER Operation status condition occurred. 6 64 RQS/ Instrument is MSS requesting service. 5 32 ESB Enabled event status condition occurred. 4 16 MAV Message available. 3 8 ---- (Not used.) 2 4 MSG Message displayed. 1 2 USR User event condition occurred. 0 1 TRG A trigger occurred.
*TRG (see page 234)	n/a	n/a
n/a	*TST? (see page 235)	<result> ::= 0 or non-zero value; an integer in NR1 format
*WAI (see page 236)	n/a	n/a

Introduction to Common (*) Commands

The common commands are defined by the IEEE 488.2 standard. They are implemented by all instruments that comply with the IEEE 488.2 standard. They provide some of the basic instrument functions, such as instrument identification and reset, reading the instrument setup, and determining how status is read and cleared.

Common commands can be received and processed by the instrument whether they are sent over the interface as separate program messages or within other program messages. If an instrument subsystem has been selected and a common command is received by the instrument, the instrument remains in the selected subsystem. For example, if the program message ":ACQuire:TYPE AVERAge; *CLS; COUNT 256" is received by the instrument, the instrument sets the acquire type, then clears the status information and sets the average count.

In contrast, if a root level command or some other subsystem command is within the program message, you must re-enter the original subsystem after the command. For example, the program message ":ACQuire:TYPE AVERAge; :AUToscale; :ACQuire:COUNT 256" sets the acquire type, completes the autoscale, then sets the acquire count. In this example, :ACQuire must be sent again after the :AUToscale command in order to re-enter the ACQuire subsystem and set the count.

NOTE

Each of the status registers has an enable (mask) register. By setting the bits in the enable register, you can select the status information you want to use.

*CLS (Clear Status)

C (see [page 1650](#))

Command Syntax

*CLS

The *CLS common command clears the status data structures, the device-defined error queue, and the Request-for-OPC flag.

NOTE

If the *CLS command immediately follows a program message terminator, the output queue and the MAV (message available) bit are cleared.

-
- See Also
- ["Introduction to Common \(*\) Commands"](#) on page 213
 - ["*STB \(Read Status Byte\)"](#) on page 232
 - ["*ESE \(Standard Event Status Enable\)"](#) on page 216
 - ["*ESR \(Standard Event Status Register\)"](#) on page 218
 - ["*SRE \(Service Request Enable\)"](#) on page 230
 - [":SYSTem:ERRor"](#) on page 1311

*ESE (Standard Event Status Enable)

C (see [page 1650](#))

Command Syntax *ESE <mask_argument>

<mask_argument> ::= integer from 0 to 255

The *ESE common command sets the bits in the Standard Event Status Enable Register. The Standard Event Status Enable Register contains a mask value for the bits to be enabled in the Standard Event Status Register. A "1" in the Standard Event Status Enable Register enables the corresponding bit in the Standard Event Status Register. A zero disables the bit.

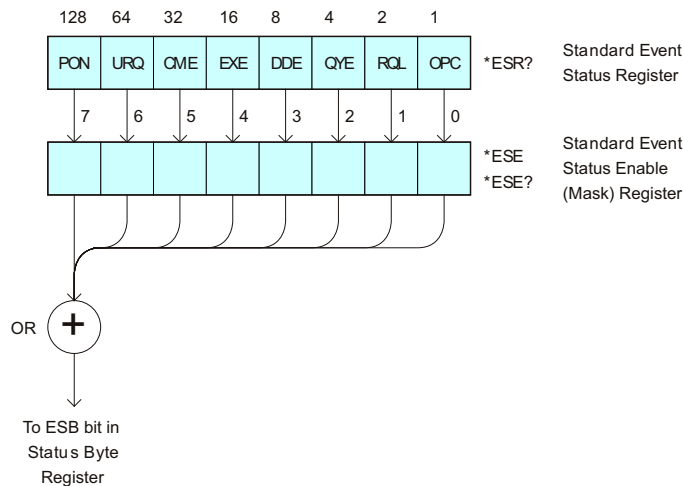


Table 80 Standard Event Status Enable (ESE)

Bit	Name	Description	When Set (1 = High = True), Enables:
7	PON	Power On	Event when an OFF to ON transition occurs.
6	URQ	User Request	Event when a front-panel key is pressed.
5	CME	Command Error	Event when a command error is detected.
4	EXE	Execution Error	Event when an execution error is detected.
3	DDE	Device Dependent Error	Event when a device-dependent error is detected.
2	QYE	Query Error	Event when a query error is detected.
1	RQL	Request Control	Event when the device is requesting control. (Not used.)
0	OPC	Operation Complete	Event when an operation is complete.

Query Syntax *ESE?

The *ESE? query returns the current contents of the Standard Event Status Enable Register.

Return Format <mask_argument><NL>
 <mask_argument> ::= 0,...,255; an integer in NR1 format.

- See Also**
- ["Introduction to Common \(*\) Commands"](#) on page 213
 - ["*ESR \(Standard Event Status Register\)"](#) on page 218
 - ["*OPC \(Operation Complete\)"](#) on page 222
 - ["*CLS \(Clear Status\)"](#) on page 215

*ESR (Standard Event Status Register)

C (see [page 1650](#))

Query Syntax *ESR?

The *ESR? query returns the contents of the Standard Event Status Register. When you read the Event Status Register, the value returned is the total bit weights of all of the bits that are high at the time you read the byte. Reading the register clears the Event Status Register.

The following table shows bit weight, name, and condition for each bit.

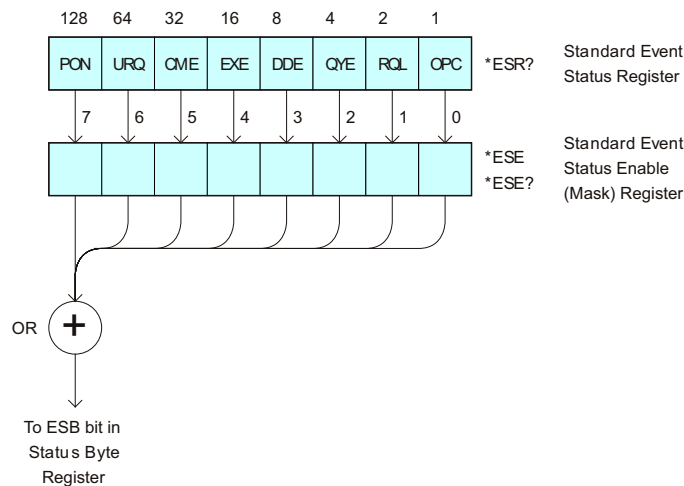


Table 81 Standard Event Status Register (ESR)

Bit	Name	Description	When Set (1 = High = True), Indicates:
7	PON	Power On	An OFF to ON transition has occurred.
6	URQ	User Request	A front-panel key has been pressed.
5	CME	Command Error	A command error has been detected.
4	EXE	Execution Error	An execution error has been detected.
3	DDE	Device Dependent Error	A device-dependent error has been detected.
2	QYE	Query Error	A query error has been detected.
1	RQL	Request Control	The device is requesting control. (Not used.)
0	OPC	Operation Complete	Operation is complete.

Return Format <status><NL>

<status> ::= 0,...,255; an integer in NR1 format.

NOTE

Reading the Standard Event Status Register clears it. High or 1 indicates the bit is true.

- See Also
- ["*Introduction to Common \(*\) Commands"](#) on page 213
 - ["*ESE \(Standard Event Status Enable\)"](#) on page 216
 - ["*OPC \(Operation Complete\)"](#) on page 222
 - ["*CLS \(Clear Status\)"](#) on page 215
 - [":SYSTem:ERRor"](#) on page 1311

*IDN (Identification Number)

C (see [page 1650](#))

Query Syntax

*IDN?

The *IDN? query identifies the instrument type and software version.

Return Format

<manufacturer_string>,<model>,<serial_number>,X.XX.XX <NL>

<manufacturer_string> ::= AGILENT TECHNOLOGIES

<model> ::= the model number of the instrument

<serial_number> ::= the serial number of the instrument

X.XX.XX ::= the software revision of the instrument

See Also

- ["Introduction to Common \(*\) Commands"](#) on page 213
- ["*OPT \(Option Identification\)"](#) on page 223
- [":SYSTem:PERSonA\[:MANUFACTURER\]"](#) on page 1313
- [":SYSTem:PERSonA\[:MANUFACTURER\]:DEFAULT"](#) on page 1314

*LRN (Learn Device Setup)

C (see [page 1650](#))

Query Syntax

*LRN?

The *LRN? query result contains the current state of the instrument. This query is similar to the :SYSTem:SETup? (see [page 1326](#)) query, except that it contains ":SYST:SET " before the binary block data. The query result is a valid command that can be used to restore instrument settings at a later time.

Return Format

<learn_string><NL>

<learn_string> ::= :SYST:SET <setup_data>

<setup_data> ::= binary block data in IEEE 488.2 # format

<learn string> specifies the current instrument setup. The block size is subject to change with different firmware revisions.

NOTE

The *LRN? query return format has changed from previous Keysight oscilloscopes to match the IEEE 488.2 specification which says that the query result must contain ":SYST:SET " before the binary block data.

- See Also
- ["Introduction to Common \(*\) Commands"](#) on page 213
 - ["*RCL \(Recall\)"](#) on page 225
 - ["*SAV \(Save\)"](#) on page 229
 - [":SYSTem:SETup"](#) on page 1326

*OPC (Operation Complete)

C (see [page 1650](#))

Command Syntax

*OPC

The *OPC command sets the operation complete bit in the Standard Event Status Register when all pending device operations have finished.

You can use the *ESR? query to look at the OPC bit (bit 0) in the Standard Event Status Register to determine when an operation is complete.

NOTE

The front-panel graphical user interface can disable most of the remote interface, including the *OPC syntax, in certain situations. Bit 4 in the Operation Status Condition Register shows whether the remote user interface is enabled or disabled. For more information, see [":OPERRegister:CONDition \(Operation Status Condition Register\)"](#) on page 262.

Query Syntax

*OPC?

The *OPC? query places an ASCII "1" in the output queue when all pending device operations have completed. The interface hangs until this query returns.

The *OPC? query can be used between overlapped commands to make sure one is complete before the next one begins. The *OPC? query can also be used to pause a controller program until an operation is complete.

Return Format

<complete><NL>

<complete> ::= 1

See Also

- ["Introduction to Common \(*\) Commands"](#) on page 213
- ["*ESE \(Standard Event Status Enable\)"](#) on page 216
- ["*ESR \(Standard Event Status Register\)"](#) on page 218
- ["*CLS \(Clear Status\)"](#) on page 215
- [":OPERRegister:CONDition \(Operation Status Condition Register\)"](#) on page 262
- [":OPERRegister\[:EVENT\] \(Operation Status Event Register\)"](#) on page 265
- [Chapter 4](#), "Sequential (Blocking) vs. Overlapped Commands," starting on page 69
- ["Synchronization with an Averaging Acquisition"](#) on page 1636
- ["Set Up the Oscilloscope"](#) on page 1630
- ["Example: Blocking and Polling Synchronization"](#) on page 1638
- ["Example: Waiting for IO Operation Complete"](#) on page 1626

*OPT (Option Identification)

C (see [page 1650](#))

Query Syntax *OPT?

The *OPT? query reports the options installed in the instrument. This query returns a string that identifies the module and its software revision level.

Return Format 0,0,<license info>

```

<license info> ::= <All field>, <reserved>, <reserved>, <MSO>,
<reserved>, <Memory>, <Low Speed Serial>,
<Automotive Serial>, <FlexRay Serial>, <Frequency Response Analysis>,
<Power Measurements>, <RS-232/UART Serial>, <Segmented Memory>,
<Mask Test>, <reserved>, <Bandwidth>, <reserved>, <reserved>,
<reserved>, <reserved>, <reserved>, <reserved>, <I2S Serial>,
<reserved>, <Educator's Kit>, <Waveform Generator>,
<MIL-1553/ARINC 429 Serial>, <Extended Video>, <Advanced Math>,
<reserved>, <reserved>, <reserved>, <Digital Voltmeter>, <reserved>,
<Spectrum Visualizer>, <reserved>, <USB 2.0 Low/Full Speed>,
<USB 2.0 High Speed>, <USB 2.0 Signal Quality Analysis>,
<Remote Command Logging>, <reserved>, <SENT Serial>, <CAN FD Serial>,
<CXPI Serial>, <NFC Trigger>, <reserved>, <reserved>, <reserved>,
<Manchester/NRZ Serial>, <USB PD Serial>, <reserved>,
<Automotive Software>, <General Purpose Software>,
<Aerospace Software>, <Power Supply Test Software>,
<USB Test Software>, <Near Field Communications (NFC) Software>,
<Software Bundle>

```

<All field> ::= {0 | All}

<reserved> ::= 0

<MSO> ::= {0 | MSO}

<Memory> ::= {0 | MEMUP}

<Low Speed Serial> ::= {0 | EMBD}

<Automotive Serial> ::= {0 | AUTO}

<FlexRay Serial> ::= {0 | FLEX}

<Frequency Response Analysis> ::= {0 | FRA}

<Power Measurements> ::= {0 | PWR}

<RS-232/UART Serial> ::= {0 | COMP}

<Segmented Memory> ::= {0 | SGM}

<Mask Test> ::= {0 | MASK}

<Bandwidth> ::= {0 | BW50}

<I2S Serial> ::= {0 | AUDIO}

<Educator's Kit> ::= {0 | EDK}

*RCL (Recall)

C (see [page 1650](#))

Command Syntax *RCL <value>

<value> ::= { 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 }

The *RCL command restores the state of the instrument from the specified save/recall register.

- See Also**
- ["Introduction to Common \(*\) Commands"](#) on page 213
 - ["*SAV \(Save\)"](#) on page 229

*RST (Reset)

C (see [page 1650](#))

Command Syntax *RST

The *RST command places the instrument in a known state. This is the same as pressing **[Save/Recall] > Default/Erase > Factory Default** on the front panel.

When you perform a factory default setup, there are no user settings that remain unchanged. To perform the equivalent of the front panel's **[Default Setup]** key, where some user settings (like preferences) remain unchanged, use the :SYSTem:PRESet command.

Reset conditions are:

Acquire Menu	
Mode	Normal
Averaging	Off
# Averages	8

Analog Channel Menu	
Channel 1	On
Channel 2	Off
Volts/division	5.00 V
Offset	0.00
Coupling	DC
Probe attenuation	AutoProbe (if AutoProbe is connected), otherwise 1.0:1
Vernier	Off
Invert	Off
BW limit	Off
Impedance	1 M Ohm
Units	Volts
Skew	0

Cursor Menu	
Source	Channel 1

Digital Channel Menu (MSO models only)	
Channel 0 - 15	Off
Labels	Off
Threshold	TTL (1.4 V)

Display Menu	
Persistence	Off
Grid	20%

Quick Meas Menu	
Source	Channel 1

Run Control	
	Scope is running

Time Base Menu	
Main time/division	100 us
Main time base delay	0.00 s
Delay time/division	500 ns
Delay time base delay	0.00 s
Reference	center
Mode	main
Vernier	Off

Trigger Menu	
Type	Edge
Mode	Auto
Coupling	dc
Source	Channel 1
Level	0.0 V
Slope	Positive

Trigger Menu	
HF Reject and noise reject	Off
Holdoff	40 ns
External probe attenuation	10:1
External Units	Volts
External Impedance	1 M Ohm (cannot be changed)

- See Also
- ["Introduction to Common \(*\) Commands"](#) on page 213
 - [":SYSTem:PRESet"](#) on page 1315

Example Code

```
' RESET - This command puts the oscilloscope into a known state.
' This statement is very important for programs to work as expected.
' Most of the following initialization commands are initialized by
' *RST. It is not necessary to reinitialize them unless the default
' setting is not suitable for your application.
myScope.WriteString "*RST" ' Reset the oscilloscope to the defaults.
```

See complete example programs at: [Chapter 46](#), "Programming Examples," starting on page 1659

*SAV (Save)

C (see [page 1650](#))

Command Syntax *SAV <value>

<value> ::= {0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9}

The *SAV command stores the current state of the instrument in a save register. The data parameter specifies the register where the data will be saved.

- See Also**
- ["Introduction to Common \(*\) Commands"](#) on page 213
 - ["*RCL \(Recall\)"](#) on page 225

*SRE (Service Request Enable)

C (see [page 1650](#))

Command Syntax `*SRE <mask>`

`<mask>` ::= integer with values defined in the following table.

The *SRE command sets the bits in the Service Request Enable Register. The Service Request Enable Register contains a mask value for the bits to be enabled in the Status Byte Register. A one in the Service Request Enable Register enables the corresponding bit in the Status Byte Register. A zero disables the bit.

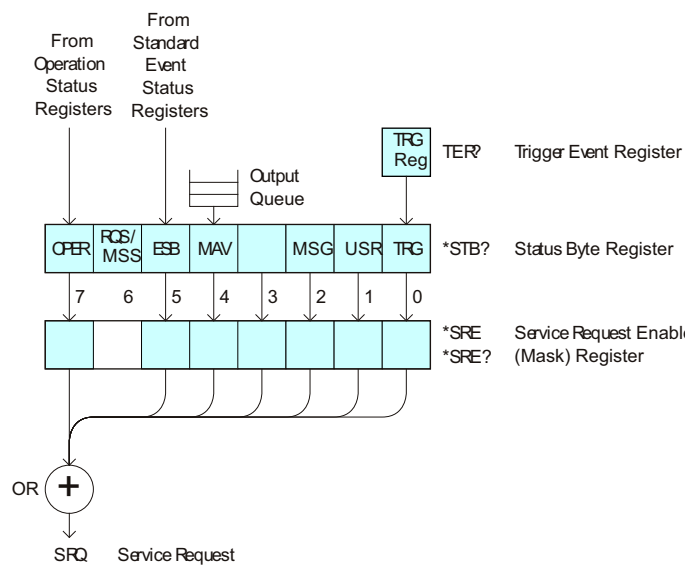


Table 82 Service Request Enable Register (SRE)

Bit	Name	Description	When Set (1 = High = True), Enables:
7	OPER	Operation Status Register	Interrupts when enabled conditions in the Operation Status Register (OPER) occur.
6	---	---	(Not used.)
5	ESB	Event Status Bit	Interrupts when enabled conditions in the Standard Event Status Register (ESR) occur.
4	MAV	Message Available	Interrupts when messages are in the Output Queue.
3	---	---	(Not used.)
2	MSG	Message	Interrupts when an advisory has been displayed on the oscilloscope.
1	USR	User Event	Interrupts when enabled user event conditions occur.
0	TRG	Trigger	Interrupts when a trigger occurs.

Query Syntax *SRE?

The *SRE? query returns the current value of the Service Request Enable Register.

Return Format <mask><NL>

<mask> ::= sum of all bits that are set, 0,...,255;
an integer in NR1 format

- See Also**
- ["Introduction to Common \(*\) Commands"](#) on page 213
 - ["*STB \(Read Status Byte\)"](#) on page 232
 - ["*CLS \(Clear Status\)"](#) on page 215

*STB (Read Status Byte)

C (see [page 1650](#))

Query Syntax `*STB?`

The `*STB?` query returns the current value of the instrument's status byte. The MSS (Master Summary Status) bit is reported on bit 6 instead of the RQS (request service) bit. The MSS indicates whether or not the device has at least one reason for requesting service.

Return Format `<value><NL>`

`<value>` ::= 0,...,255; an integer in NR1 format

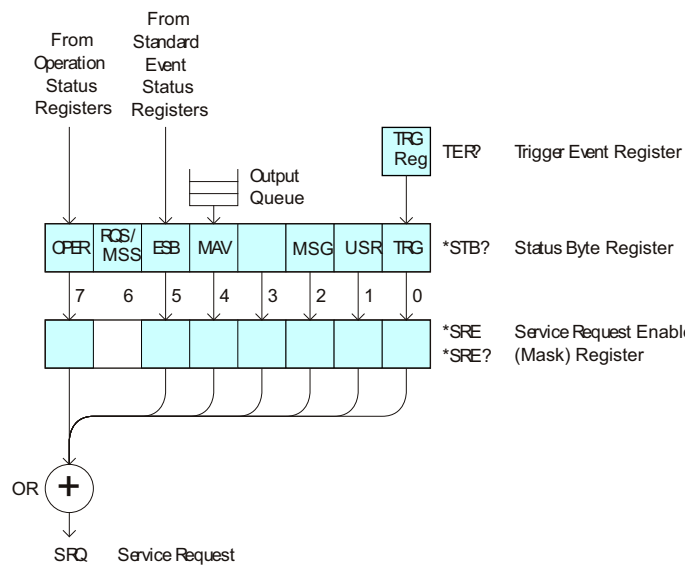


Table 83 Status Byte Register (STB)

Bit	Name	Description	When Set (1 = High = True), Indicates:
7	OPER	Operation Status Register	An enabled condition in the Operation Status Register (OPER) has occurred.
6	RQS	Request Service	When polled, that the device is requesting service.
	MSS	Master Summary Status	When read (by *STB?), whether the device has a reason for requesting service.
5	ESB	Event Status Bit	An enabled condition in the Standard Event Status Register (ESR) has occurred.
4	MAV	Message Available	There are messages in the Output Queue.
3	---	---	(Not used, always 0.)
2	MSG	Message	An advisory has been displayed on the oscilloscope.
1	USR	User Event	An enabled user event condition has occurred.
0	TRG	Trigger	A trigger has occurred.

NOTE

To read the instrument's status byte with RQS reported on bit 6, use the interface Serial Poll.

- See Also
- ["Introduction to Common \(*\) Commands"](#) on page 213
 - ["*SRE \(Service Request Enable\)"](#) on page 230

*TRG (Trigger)

C (see [page 1650](#))

Command Syntax

*TRG

The *TRG command has the same effect as the :DIGitize command with no parameters.

- See Also
- ["Introduction to Common \(*\) Commands"](#) on page 213
 - [":DIGitize"](#) on page 250
 - [":RUN"](#) on page 274
 - [":STOP"](#) on page 278

*TST (Self Test)

C (see [page 1650](#))

Query Syntax *TST?

The *TST? query performs a self-test on the instrument. The result of the test is placed in the output queue. A zero indicates the test passed and a non-zero indicates the test failed. If the test fails, refer to the troubleshooting section of the *Service Guide*.

Return Format <result><NL>

<result> ::= 0 or non-zero value; an integer in NR1 format

See Also • ["Introduction to Common \(*\) Commands"](#) on page 213

*WAI (Wait To Continue)

C (see [page 1650](#))

Command Syntax

*WAI

The *WAI command has no function in the oscilloscope, but is parsed for compatibility with other instruments.

See Also • ["Introduction to Common \(*\) Commands"](#) on page 213

7 Root (:) Commands

Control many of the basic functions of the oscilloscope and reside at the root level of the command tree. See ["Introduction to Root \(:\) Commands"](#) on page 240.

Table 84 Root (:) Commands Summary

Command	Query	Options and Query Returns
:ACTivity (see page 241)	:ACTivity? (see page 241)	<return value> ::= <edges>,<levels> <edges> ::= presence of edges (32-bit integer in NR1 format) <levels> ::= logical highs or lows (32-bit integer in NR1 format)
n/a	:AER? (see page 242)	{0 1}; an integer in NR1 format
:AUToscale [<source>[,...,<source>]] (see page 243)	n/a	<source> ::= CHANNEL<n> for DSO models <source> ::= {CHANNEL<n> DIGital<d> POD1 POD2} for MSO models <source> can be repeated up to 5 times <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:AUToscale:AMODE <value> (see page 245)	:AUToscale:AMODE? (see page 245)	<value> ::= {NORMAL CURRENT}}
:AUToscale:CHANnels <value> (see page 246)	:AUToscale:CHANnels? (see page 246)	<value> ::= {ALL DISPLAYed}}
:AUToscale:FDEBug {{0 OFF} {1 ON}} (see page 247)	:AUToscale:FDEBug? (see page 247)	{0 1}

Table 84 Root (:) Commands Summary (continued)

Command	Query	Options and Query Returns
:BLANK [<source>] (see page 248)	n/a	<source> ::= {CHANnel<n> FUNction<m> MATH<m> SBUS{1 2} WMEMory<r>} for DSO models <source> ::= {CHANnel<n> DIGital<d> POD{1 2} BUS{1 2} FUNction<m> MATH<m> SBUS{1 2} WMEMory<r>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format
:DIGitize [<source>[,...,<source >]] (see page 250)	n/a	<source> ::= {CHANnel<n> FUNction<m> MATH<m> SBUS{1 2}} for DSO models <source> ::= {CHANnel<n> DIGital<d> POD{1 2} BUS{1 2} FUNction<m> MATH<m> SBUS{1 2}} for MSO models <source> can be repeated up to 5 times <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <m> ::= 1 to (# math functions) in NR1 format
:HWEenable <n> (see page 252)	:HWEenable? (see page 252)	<n> ::= 16-bit integer in NR1 format
n/a	:HWErregister:CONDiti on? (see page 254)	<n> ::= 16-bit integer in NR1 format
n/a	:HWErregister[:EVENT]? (see page 255)	<n> ::= 16-bit integer in NR1 format
:MTEenable <n> (see page 256)	:MTEenable? (see page 256)	<n> ::= 16-bit integer in NR1 format
n/a	:MTErregister[:EVENT]? (see page 258)	<n> ::= 16-bit integer in NR1 format

Table 84 Root (:) Commands Summary (continued)

Command	Query	Options and Query Returns																																	
:OPEE <n> (see page 260)	:OPEE? (see page 261)	<n> ::= 15-bit integer in NR1 format																																	
n/a	:OPERRegister:CONDition? (see page 262)	<n> ::= 15-bit integer in NR1 format																																	
n/a	:OPERRegister[:EVENT]? (see page 265)	<n> ::= 15-bit integer in NR1 format																																	
:OVLenable <mask> (see page 268)	:OVLenable? (see page 269)	<mask> ::= 16-bit integer in NR1 format as shown: <table> <tr> <th>Bit</th><th>Weight</th><th>Input</th></tr> <tr> <td>10</td><td>1024</td><td>Ext Trigger Fault</td></tr> <tr> <td>9</td><td>512</td><td>Channel 4 Fault</td></tr> <tr> <td>8</td><td>256</td><td>Channel 3 Fault</td></tr> <tr> <td>7</td><td>128</td><td>Channel 2 Fault</td></tr> <tr> <td>6</td><td>64</td><td>Channel 1 Fault</td></tr> <tr> <td>4</td><td>16</td><td>Ext Trigger OVL</td></tr> <tr> <td>3</td><td>8</td><td>Channel 4 OVL</td></tr> <tr> <td>2</td><td>4</td><td>Channel 3 OVL</td></tr> <tr> <td>1</td><td>2</td><td>Channel 2 OVL</td></tr> <tr> <td>0</td><td>1</td><td>Channel 1 OVL</td></tr> </table>	Bit	Weight	Input	10	1024	Ext Trigger Fault	9	512	Channel 4 Fault	8	256	Channel 3 Fault	7	128	Channel 2 Fault	6	64	Channel 1 Fault	4	16	Ext Trigger OVL	3	8	Channel 4 OVL	2	4	Channel 3 OVL	1	2	Channel 2 OVL	0	1	Channel 1 OVL
Bit	Weight	Input																																	
10	1024	Ext Trigger Fault																																	
9	512	Channel 4 Fault																																	
8	256	Channel 3 Fault																																	
7	128	Channel 2 Fault																																	
6	64	Channel 1 Fault																																	
4	16	Ext Trigger OVL																																	
3	8	Channel 4 OVL																																	
2	4	Channel 3 OVL																																	
1	2	Channel 2 OVL																																	
0	1	Channel 1 OVL																																	
n/a	:OVLRegister? (see page 270)	<value> ::= integer in NR1 format. See OVLenable for <value>																																	
:PRINT [<options>] (see page 272)	n/a	<options> ::= [<print option>][,...,<print option>] <print option> ::= {COLor GRAYscale PRINter0 BMP8bit BMP PNG NOFactoRs FACToRs} <print option> can be repeated up to 5 times.																																	
n/a	:RSTate? (see page 273)	n/a																																	
:RUN (see page 274)	n/a	n/a																																	
n/a	:SERial (see page 275)	<return value> ::= unquoted string containing serial number																																	
:SINGle (see page 276)	n/a	n/a																																	

Table 84 Root (:) Commands Summary (continued)

Command	Query	Options and Query Returns
n/a	:STATus? <display> (see page 277)	{0 1} <display> ::= {CHANnel<n> DIGital<d> POD{1 2} BUS{1 2} FUNCTION<m> MATH<m> SBUS{1 2} WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format
:STOP (see page 278)	n/a	n/a
n/a	:TER? (see page 279)	{0 1}
:VIEW <source> (see page 280)	n/a	<source> ::= {CHANnel<n> FUNCTION<m> MATH<m> SBUS{1 2} WMEMory<r>} for DSO models <source> ::= {CHANnel<n> DIGital<d> POD{1 2} BUS{1 2} FUNCTION<m> MATH<m> SBUS{1 2} WMEMory<r>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format

**Introduction to
Root (:)
Commands**

Root level commands control many of the basic operations of the instrument. These commands are always recognized by the parser if they are prefixed with a colon, regardless of current command tree position. After executing a root-level command, the parser is positioned at the root of the command tree.

:ACTivity

N (see [page 1650](#))

Command Syntax :ACTivity

The :ACTivity command clears the cumulative edge variables for the next activity query.

Query Syntax :ACTivity?

The :ACTivity? query returns whether there has been activity (edges) on the digital channels since the last query, and returns the current logic levels.

NOTE

Because the :ACTivity? query returns edge activity since the last :ACTivity? query, you must send this query twice before the edge activity result is valid.

Return Format <edges>,<levels><NL>

<edges> ::= presence of edges (16-bit integer in NR1 format).

<levels> ::= logical highs or lows (16-bit integer in NR1 format).

bit 0 ::= DIGital 0

bit 15 ::= DIGital 15

NOTE

A bit = 0 (zero) in the <edges> result indicates that no edges were detected on that channel (across the specified threshold voltage) since the last query.

A bit = 1 (one) in the <edges> result indicates that edges have been detected on that channel (across the specified threshold voltage) since the last query.

(The threshold voltage must be set appropriately for the logic levels of the signals being probed.)

- See Also**
- ["Introduction to Root \(:\) Commands"](#) on page 240
 - [":POD<n>:THReshold"](#) on page 749
 - [":DIGital<d>:THReshold"](#) on page 389

:AER (Arm Event Register)

C (see [page 1650](#))

Query Syntax :AER?

The AER query reads the Arm Event Register. After the Arm Event Register is read, it is cleared. A "1" indicates the trigger system is in the armed state, ready to accept a trigger.

The Armed Event Register is summarized in the Wait Trig bit of the Operation Status Event Register. A Service Request can be generated when the Wait Trig bit transitions and the appropriate enable bits have been set in the Operation Status Enable Register (OPEE) and the Service Request Enable Register (SRE).

Return Format <value><NL>

<value> ::= {0 | 1}; an integer in NR1 format.

- See Also**
- ["Introduction to Root \(:\) Commands"](#) on page 240
 - [":OPEE \(Operation Status Enable Register\)"](#) on page 260
 - [":OPERRegister:CONDition \(Operation Status Condition Register\)"](#) on page 262
 - [":OPERRegister\[:EVENT\] \(Operation Status Event Register\)"](#) on page 265
 - ["*STB \(Read Status Byte\)"](#) on page 232
 - ["*SRE \(Service Request Enable\)"](#) on page 230

:AUToscale

C (see [page 1650](#))

Command Syntax :AUToscale

```
:AUToscale [<source>[,...,<source>]]
```

```
<source> ::= CHANnel<n> for the DSO models
```

```
<source> ::= {DIGital<d> | POD1 | POD2 | CHANnel<n>} for the  
MSO models
```

```
<n> ::= 1 to (# analog channels) in NR1 format
```

```
<d> ::= 0 to (# digital channels - 1) in NR1 format
```

The <source> parameter may be repeated up to 5 times.

The :AUToscale command evaluates all input signals and sets the correct conditions to display the signals. This is the same as pressing the **[Auto Scale]** key on the front panel.

If one or more sources are specified, those specified sources will be enabled and all others blanked. The autoscale channels mode (see **"[:AUToscale:CHANnels](#)"** on page 246) is set to DISPlayed channels. Then, the autoscale is performed.

When the :AUToscale command is sent, the following conditions are affected and actions are taken:

- Thresholds.
- Channels with activity around the trigger point are turned on, others are turned off.
- Channels are reordered on screen; analog channel 1 first, followed by the remaining analog channels, then the digital channels 0-15.
- Delay is set to 0 seconds.
- Time/Div.

The :AUToscale command does not affect the following conditions:

- Label names.
- Trigger conditioning.

The :AUToscale command turns off the following items:

- Cursors.
- Measurements.
- Math waveforms.
- Reference waveforms.
- Zoomed (delayed) time base mode.

For further information on :AUToscale, see the *User's Guide*.

- See Also**
- ["Introduction to Root \(:\) Commands"](#) on page 240
 - [":AUToscale:CHANnels"](#) on page 246
 - [":AUToscale:AMODE"](#) on page 245

Example Code

```
' AUTOSCALE - This command evaluates all the input signals and sets
' the correct conditions to display all of the active signals.
myScope.WriteString ":AUToscale"    ' Same as pressing Auto Scale key.
```

See complete example programs at: [Chapter 46](#), "Programming Examples," starting on page 1659

:AUToscale:AMODE

N (see [page 1650](#))

Command Syntax :AUToscale:AMODE <value>
 <value> ::= {NORMal | CURRent}

The :AUToscale:AMODE command specifies the acquisition mode that is set by subsequent :AUToscales.

- When NORMal is selected, an :AUToscale command sets the NORMal acquisition type and the RTIME (real-time) acquisition mode.
- When CURRent is selected, the current acquisition type and mode are kept on subsequent :AUToscales.

Use the :ACquire:TYPE and :ACquire:MODE commands to set the acquisition type and mode.

Query Syntax :AUToscale:AMODE?

The :AUToscale:AMODE? query returns the autoscale acquire mode setting.

Return Format <value><NL>
 <value> ::= {NORM | CURR}

- See Also**
- ["Introduction to Root \(:\) Commands"](#) on page 240
 - [":AUToscale"](#) on page 243
 - [":AUToscale:CHANnels"](#) on page 246
 - [":ACquire:TYPE"](#) on page 300
 - [":ACquire:MODE"](#) on page 289

:AUToscale:CHANnels

N (see [page 1650](#))

Command Syntax :AUToscale:CHANnels <value>
<value> ::= {ALL | DISPlayed}

The :AUToscale:CHANnels command specifies which channels will be displayed on subsequent :AUToscales.

- When ALL is selected, all channels that meet the requirements of :AUToscale will be displayed.
- When DISPlayed is selected, only the channels that are turned on are autoscaled.

Use the :VIEW or :BLANK root commands to turn channels on or off.

Query Syntax :AUToscale:CHANnels?

The :AUToscale:CHANnels? query returns the autoscale channels setting.

Return Format <value><NL>
<value> ::= {ALL | DISP}

- See Also**
- ["Introduction to Root \(:\) Commands"](#) on page 240
 - [":AUToscale"](#) on page 243
 - [":AUToscale:AMODE"](#) on page 245
 - [":VIEW"](#) on page 280
 - [":BLANK"](#) on page 248

:AUToscale:FDEBug

N (see [page 1650](#))

Command Syntax :AUToscale:FDEBug <on_off>
 <on_off> ::= {{1 | ON} | {0 | OFF}}

The :AUToscale:FDEBug command turns fast debug auto scaling on or off.

The Fast Debug option changes the behavior of :AUToscale to let you make quick visual comparisons to determine whether the signal being probed is a DC voltage, ground, or an active AC signal.

Channel coupling is maintained for easy viewing of oscillating signals.

Query Syntax :AUToscale:FDEBug?

The :AUToscale:FDEBug? query returns the current autoscale fast debug setting.

Return Format <on_off><NL>
 <on_off> ::= {1 | 0}

See Also

- ["Introduction to Root \(:\) Commands"](#) on page 240
- [":AUToscale"](#) on page 243

:BLANK

N (see [page 1650](#))

Command Syntax

:BLANK [<source>]

<source> ::= {CHANnel<n> | FUNCTION<m> | MATH<m> | SBUS{1 | 2}
| WMEMory<r>}
for the DSO models

<source> ::= {CHANnel<n> | DIGital<d> | POD{1 | 2}
| BUS{1 | 2} | FUNCTION<m> | MATH<m> | SBUS{1 | 2}
| WMEMory<r>}
for the MSO models

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :BLANK command turns off (stops displaying) the specified channel, digital pod, math function, serial decode bus, or reference waveform location. The :BLANK command with no parameter turns off all sources.

NOTE

To turn on (start displaying) a channel, etc., use the :VIEW command. The DISPlay commands, :CHANnel<n>:DISPlay, :FUNCTION:DISPlay, :POD<n>:DISPlay, :DIGital<n>:DISPlay, :SBUS<n>:DISPlay, or :WMEMory<r>:DISPlay, are the preferred method to turn on/off a channel, etc.

NOTE

MATH<m> is an alias for FUNCTION<m>.

- See Also
- ["Introduction to Root \(:\) Commands"](#) on page 240
 - [":DISPlay:CLEAr"](#) on page 401
 - [":CHANnel<n>:DISPlay"](#) on page 332
 - [":DIGital<d>:DISPlay"](#) on page 385
 - [":FUNCTION<m>:DISPlay"](#) on page 463
 - [":POD<n>:DISPlay"](#) on page 747
 - [":SBUS<n>:DISPlay"](#) on page 902
 - [":WMEMory<r>:DISPlay"](#) on page 1524
 - [":STATus"](#) on page 277
 - [":VIEW"](#) on page 280

Example Code · "Example Code" on page 280

:DIGitize

C (see [page 1650](#))

Command Syntax :DIGitize [<source>[,...,<source>]]

<source> ::= {CHANnel<n> | FUNCTION<m> | MATH<m> | SBUS{1 | 2}}
for the DSO models

<source> ::= {CHANnel<n> | DIGital<d> | POD{1 | 2}
| BUS{1 | 2} | FUNCTION<m> | MATH<m> | SBUS{1 | 2}}
for the MSO models

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

The <source> parameter may be repeated up to 5 times.

The :DIGitize command is a specialized RUN command. It causes the instrument to acquire waveforms according to the settings of the :ACquire commands subsystem. When the acquisition is complete, the instrument is stopped.

If no argument is given, :DIGitize acquires the channels currently displayed. If no channels are displayed, all channels are acquired.

NOTE

The :DIGitize command is only executed when the :TIMEbase:MODE is MAIN or WINDow.

NOTE

To halt a :DIGitize in progress, use the device clear command.

NOTE

MATH<m> is an alias for FUNCTION<m>.

- See Also**
- ["Introduction to Root \(:\) Commands"](#) on page 240
 - [":RUN"](#) on page 274
 - [":SINGle"](#) on page 276
 - [":STOP"](#) on page 278
 - [":TIMEbase:MODE"](#) on page 1333
 - [Chapter 8](#), “:ACquire Commands,” starting on page 281
 - [Chapter 38](#), “:WAVEform Commands,” starting on page 1433

- **Chapter 4**, “Sequential (Blocking) vs. Overlapped Commands,” starting on page 69
- **"Example: Checking for Armed Status"** on page 1621

Example Code

```
' Capture an acquisition using :DIGitize.  
' -----  
myScope.WriteString ":DIGitize CHANnel1"
```

See complete example programs at: **Chapter 46**, “Programming Examples,” starting on page 1659

:HWEenable (Hardware Event Enable Register)

N (see [page 1650](#))

Command Syntax :HWEenable <mask>
 <mask> ::= 16-bit integer

The :HWEenable command sets a mask in the Hardware Event Enable register. Set any of the following bits to "1" to enable bit 12 in the Operation Status Condition Register and potentially cause an SRQ (Service Request interrupt) to be generated.

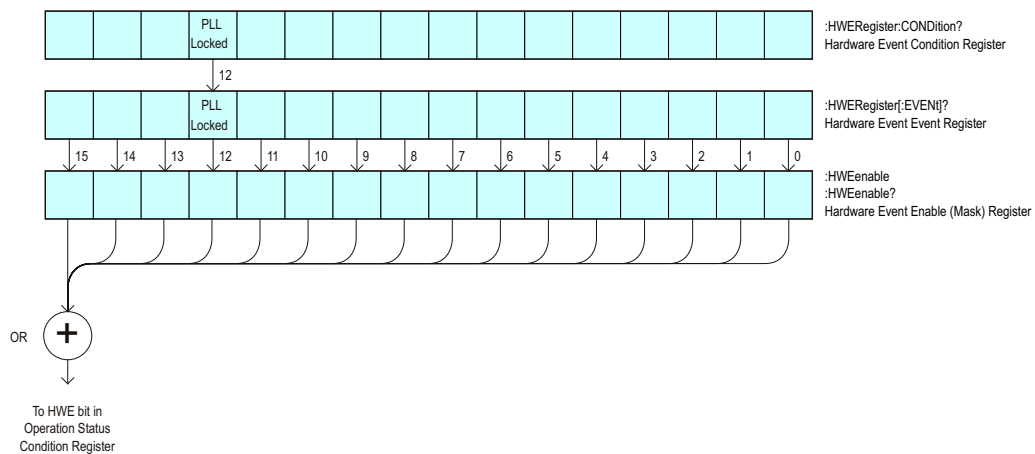


Table 85 Hardware Event Enable Register (HWEenable)

Bit	Name	Description	When Set (1 = High = True), Enables:
15-13	---	---	(Not used.)
12	PLL Locked	PLL Locked	This bit is for internal use and is not intended for general use.
11-0	---	---	(Not used.)

Query Syntax :HWEenable?

The :HWEenable? query returns the current value contained in the Hardware Event Enable register as an integer number.

Return Format <value><NL>
 <value> ::= integer in NR1 format.

- See Also**
- ["Introduction to Root \(:\) Commands"](#) on page 240
 - [":AER \(Arm Event Register\)"](#) on page 242
 - [":CHANnel<n>:PROTection"](#) on page 352

- **":OPERRegister[:EVENT] (Operation Status Event Register)"** on page 265
- **":OVLenable (Overload Event Enable Register)"** on page 268
- **":OVLRegister (Overload Event Register)"** on page 270
- **"*STB (Read Status Byte)"** on page 232
- **"*SRE (Service Request Enable)"** on page 230

:HWERegister:CONDition (Hardware Event Condition Register)

N (see [page 1650](#))

Query Syntax :HWERegister:CONDition?

The :HWERegister:CONDition? query returns the integer value contained in the Hardware Event Condition Register.

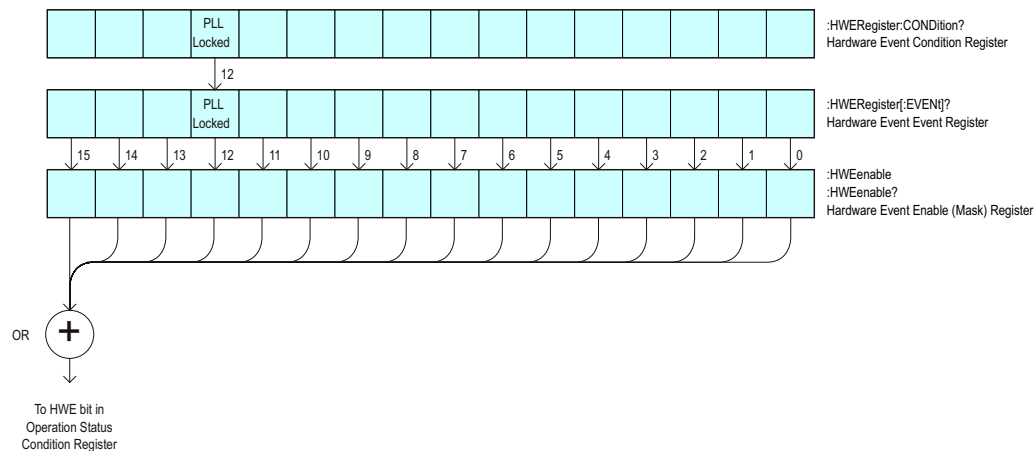


Table 86 Hardware Event Condition Register

Bit	Name	Description	When Set (1 = High = True), Indicates:
15-13	---	---	(Not used.)
12	PLL Locked	PLL Locked	This bit is for internal use and is not intended for general use.
11-0	---	---	(Not used.)

Return Format <value><NL>

<value> ::= integer in NR1 format.

- See Also**
- ["Introduction to Root \(:\) Commands"](#) on page 240
 - [":CHANnel<n>:PROTection"](#) on page 352
 - [":OPEE \(Operation Status Enable Register\)"](#) on page 260
 - [":OPERegister\[:EVENT\] \(Operation Status Event Register\)"](#) on page 265
 - [":OVLenable \(Overload Event Enable Register\)"](#) on page 268
 - [":OVLRegister \(Overload Event Register\)"](#) on page 270
 - ["*STB \(Read Status Byte\)"](#) on page 232
 - ["*SRE \(Service Request Enable\)"](#) on page 230

:HWERegister[:EVENT] (Hardware Event Event Register)

N (see [page 1650](#))

Query Syntax :HWERegister[:EVENT]?

The :HWERegister[:EVENT]? query returns the integer value contained in the Hardware Event Event Register.

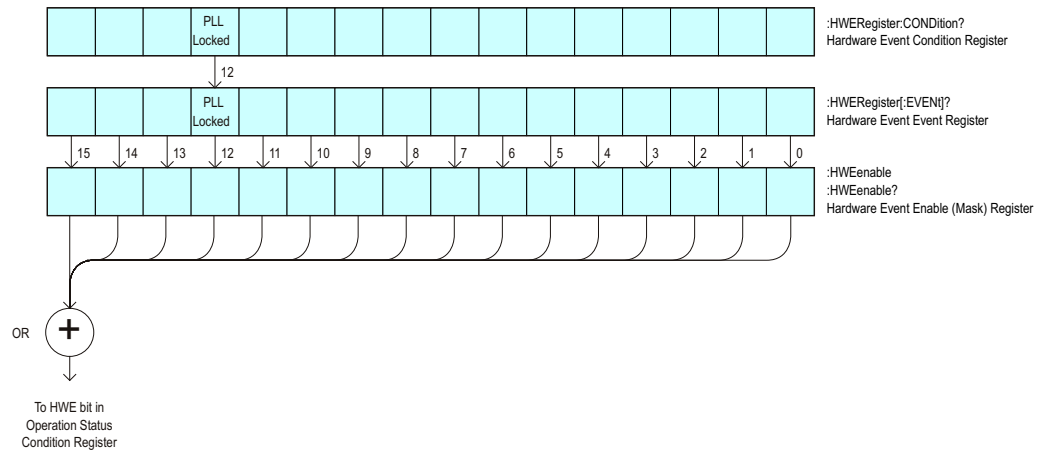


Table 87 Hardware Event Event Register

Bit	Name	Description	When Set (1 = High = True), Indicates:
15-13	---	---	(Not used.)
12	PLL Locked	PLL Locked	This bit is for internal use and is not intended for general use.
11-0	---	---	(Not used.)

Return Format <value><NL>

<value> ::= integer in NR1 format.

- See Also**
- ["Introduction to Root \(:\) Commands"](#) on page 240
 - [":CHANnel<n>:PROTection"](#) on page 352
 - [":OPEE \(Operation Status Enable Register\)"](#) on page 260
 - [":OPERegister:CONDition \(Operation Status Condition Register\)"](#) on page 262
 - [":OVLenable \(Overload Event Enable Register\)"](#) on page 268
 - [":OVLRegister \(Overload Event Register\)"](#) on page 270
 - ["*STB \(Read Status Byte\)"](#) on page 232
 - ["*SRE \(Service Request Enable\)"](#) on page 230

:MTEenable (Mask Test Event Enable Register)

N (see [page 1650](#))

Command Syntax :MTEenable <mask>

<mask> ::= 16-bit integer

The :MTEenable command sets a mask in the Mask Test Event Enable register. Set any of the following bits to "1" to enable bit 9 in the Operation Status Condition Register and potentially cause an SRQ (Service Request) interrupt to be generated.

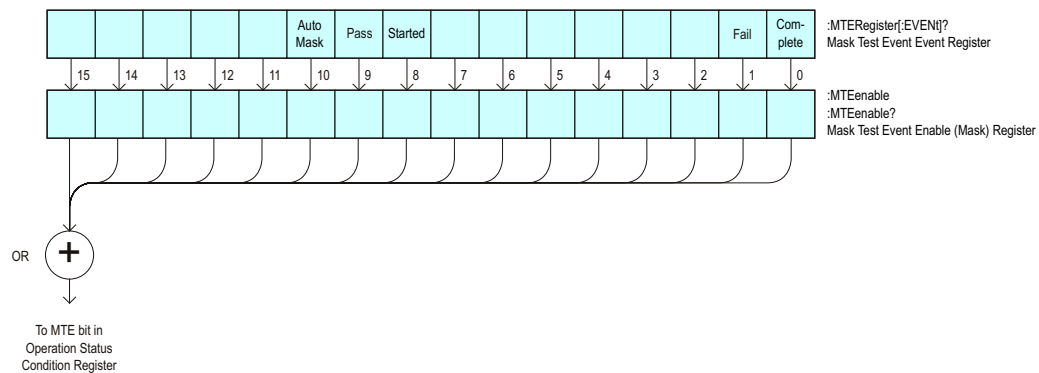


Table 88 Mask Test Event Enable Register (MTEenable)

Bit	Name	Description	When Set (1 = High = True), Enables:
15-11	---	---	(Not used.)
10	Auto Mask	Auto Mask Created	Auto mask creation completed.
9	Pass	Mask Test Pass	Mass test passed.
8	Started	Mask Testing Started	Mask testing started.
7-2	---	---	(Not used.)
1	Fail	Mask Test Fail	Mask test failed.
0	Complete	Mask Test Complete	Mask test is complete.

Query Syntax :MTEenable?

The :MTEenable? query returns the current value contained in the Mask Test Event Enable register as an integer number.

Return Format <value><NL>

<value> ::= integer in NR1 format.

- See Also
- [":Introduction to Root \(:\) Commands"](#) on page 240
 - [":AER \(Arm Event Register\)"](#) on page 242
 - [":CHANnel<n>:PROTection"](#) on page 352
 - [":OPERRegister\[:EVENT\] \(Operation Status Event Register\)"](#) on page 265
 - [":OVLenable \(Overload Event Enable Register\)"](#) on page 268
 - [":OVLRegister \(Overload Event Register\)"](#) on page 270
 - ["*STB \(Read Status Byte\)"](#) on page 232
 - ["*SRE \(Service Request Enable\)"](#) on page 230

:MTERegister[:EVENT] (Mask Test Event Event Register)

N (see [page 1650](#))

Query Syntax :MTERegister[:EVENT]?

The :MTERegister[:EVENT]? query returns the integer value contained in the Mask Test Event Event Register and clears the register.

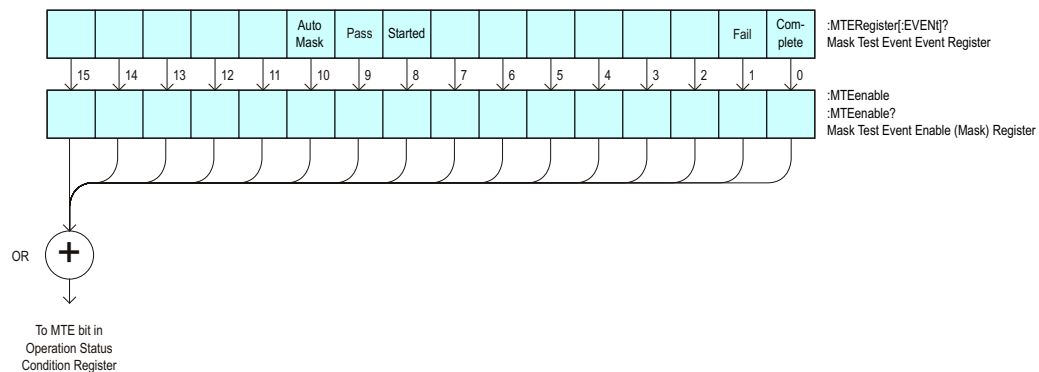


Table 89 Mask Test Event Event Register

Bit	Name	Description	When Set (1 = High = True), Indicates:
15-11	---	---	(Not used.)
10	Auto Mask	Auto Mask Created	Auto mask creation completed.
9	Pass	Mask Test Pass	The mask test passed.
8	Started	Mask Testing Started	Mask testing started.
7-2	---	---	(Not used.)
1	Fail	Mask Test Fail	The mask test failed.
0	Complete	Mask Test Complete	The mask test is complete.

Return Format <value><NL>

<value> ::= integer in NR1 format.

- See Also**
- ["Introduction to Root \(:\) Commands"](#) on page 240
 - [":CHANnel<n>:PROtection"](#) on page 352
 - [":OPEE \(Operation Status Enable Register\)"](#) on page 260
 - [":OPERegister:CONDition \(Operation Status Condition Register\)"](#) on page 262

- **":OVLenable (Overload Event Enable Register)"** on page 268
- **":OVLRegister (Overload Event Register)"** on page 270
- **"*STB (Read Status Byte)"** on page 232
- **"*SRE (Service Request Enable)"** on page 230

:OPEE (Operation Status Enable Register)

C (see [page 1650](#))

Command Syntax :OPEE <mask>

<mask> ::= 15-bit integer

The :OPEE command sets a mask in the Operation Status Enable register. Set any of the following bits to "1" to enable bit 7 in the Status Byte Register and potentially cause an SRQ (Service Request) interrupt to be generated.

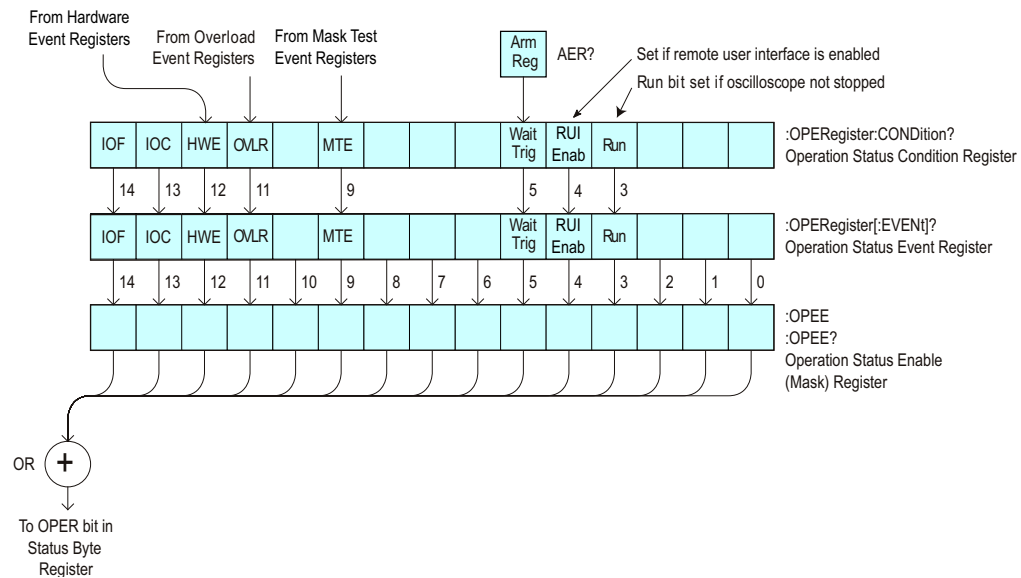


Table 90 Operation Status Enable Register (OPEE)

Bit	Name	Description	When Set (1 = High = True), Enables:
14	IOF	IO Operation Failed	Event only when something causes the IO operation to fail, like disconnecting a USB device or interrupting the operation from the oscilloscope's front panel.
13	IOC	IO Operation Complete	Event when any IO operation completes. IO operations are any remote data request using any interface (USB, LAN, or GPIB). For example, if you connect to an oscilloscope using the USB interface and then request waveform data, the IOC bit will be set when the IO operation completes.
12	HWE	Hardware Event	Event when hardware event occurs.
11	OVL	Overload	Event when 50Ω input overload occurs.

Table 90 Operation Status Enable Register (OPEE) (continued)

Bit	Name	Description	When Set (1 = High = True), Enables:
10	---	---	(Not used.)
9	MTE	Mask Test Event	Event when mask test event occurs.
8-6	---	---	(Not used.)
5	Wait Trig	Wait Trig	Event when the trigger is armed.
4	---	---	(Not used.)
3	Run	Running	Event when the oscilloscope is running (not stopped).
2-0	---	---	(Not used.)

Query Syntax :OPEE?

The :OPEE? query returns the current value contained in the Operation Status Enable register as an integer number.

Return Format <value><NL>

<value> ::= integer in NR1 format.

- See Also**
- ["Introduction to Root \(:\) Commands"](#) on page 240
 - [":AER \(Arm Event Register\)"](#) on page 242
 - [":CHANnel<n>:PROTection"](#) on page 352
 - [":OPERegister\[:EVENT\] \(Operation Status Event Register\)"](#) on page 265
 - [":OVLenable \(Overload Event Enable Register\)"](#) on page 268
 - [":OVLRegister \(Overload Event Register\)"](#) on page 270
 - ["*STB \(Read Status Byte\)"](#) on page 232
 - ["*SRE \(Service Request Enable\)"](#) on page 230
 - ["Operation Status Event Register \(:OPERegister\[:EVENT\]\)"](#) on page 1611
 - ["Example: Checking for Armed Status"](#) on page 1621
 - ["Example: Waiting for IO Operation Complete"](#) on page 1626

:OPERRegister:CONDition (Operation Status Condition Register)

C (see [page 1650](#))

Query Syntax :OPERRegister:CONDition?

The :OPERRegister:CONDition? query returns the integer value contained in the Operation Status Condition Register.

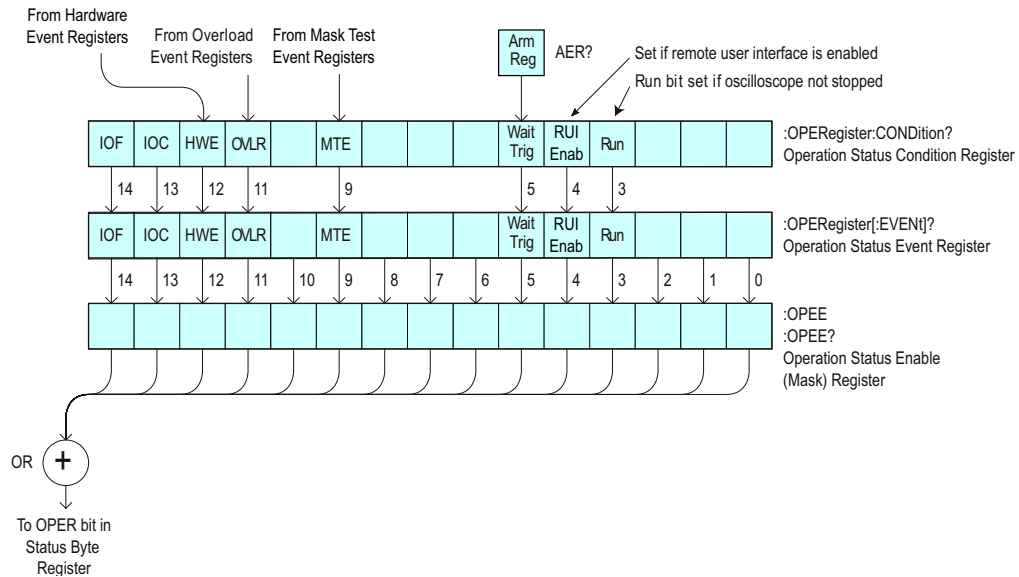


Table 91 Operation Status Condition Register

Bit	Name	Description	When Set (1 = High = True), Indicates:
14	IOF	IO Operation Failed	Event only when something causes the IO operation to fail, like disconnecting a USB device or interrupting the operation from the oscilloscope's front panel.
13	IOC	IO Operation Complete	Event when any IO operation completes. IO operations are any remote data request using any interface (USB, LAN, or GPIB). For example, if you connect to an oscilloscope using the USB interface and then request waveform data, the IOC bit will be set when the IO operation completes.
12	HWE	Hardware Event	Event when hardware event occurs.
11	OVL	Overload	A 50Ω input overload has occurred.
10	---	---	(Not used.)

Table 91 Operation Status Condition Register (continued)

Bit	Name	Description	When Set (1 = High = True), Indicates:
9	MTE	Mask Test Event	A mask test event has occurred.
8-6	---	---	(Not used.)
5	Wait Trig	Wait Trig	The trigger is armed (set by the Trigger Armed Event Register (TER)).
4	RUI Enab	Remote Enabled	Shows whether the remote user interface is enabled. The front-panel graphical user interface can disable most of the remote interface, including the *OPC syntax, for example when: ▪ a modal dialog box is open ▪ the user is being prompted ▪ all segments are being analyzed ▪ there is a channel overload ▪ certain compliance applications are running When disabled, commands or queries are accepted, but "settings conflict" errors are returned. The status model is the only part of the remote user interface that is enabled. To determine when the remote interface is re-enabled, you can read this bit or wait for the event that gets generated when its status goes from disabled to enabled.
3	Run	Running	The oscilloscope is running (not stopped).
2-0	---	---	(Not used.)

Return Format <value><NL>

<value> ::= integer in NR1 format.

- See Also
- ["Introduction to Root \(:\) Commands"](#) on page 240
 - [":CHANnel<n>:PROTection"](#) on page 352
 - [":OPEE \(Operation Status Enable Register\)"](#) on page 260
 - [":OPERRegister\[:EVENT\] \(Operation Status Event Register\)"](#) on page 265
 - [":OVLenable \(Overload Event Enable Register\)"](#) on page 268
 - [":OVLRegister \(Overload Event Register\)"](#) on page 270
 - ["*STB \(Read Status Byte\)"](#) on page 232
 - ["*SRE \(Service Request Enable\)"](#) on page 230
 - [":MTERRegister\[:EVENT\] \(Mask Test Event Event Register\)"](#) on page 258

- **":MTEenable (Mask Test Event Enable Register)"** on page 256
- **"*OPC (Operation Complete)"** on page 222
- **"Operation Status Condition Register (:OPERRegister:CONDition)"** on page 1613
- **"Example: Checking for Armed Status"** on page 1621
- **"Example: Waiting for IO Operation Complete"** on page 1626

:OPERRegister[:EVENT] (Operation Status Event Register)

C (see [page 1650](#))

Query Syntax :OPERRegister[:EVENT]?

The :OPERRegister[:EVENT]? query reads and clears the integer value contained in the Operation Status Event Register.

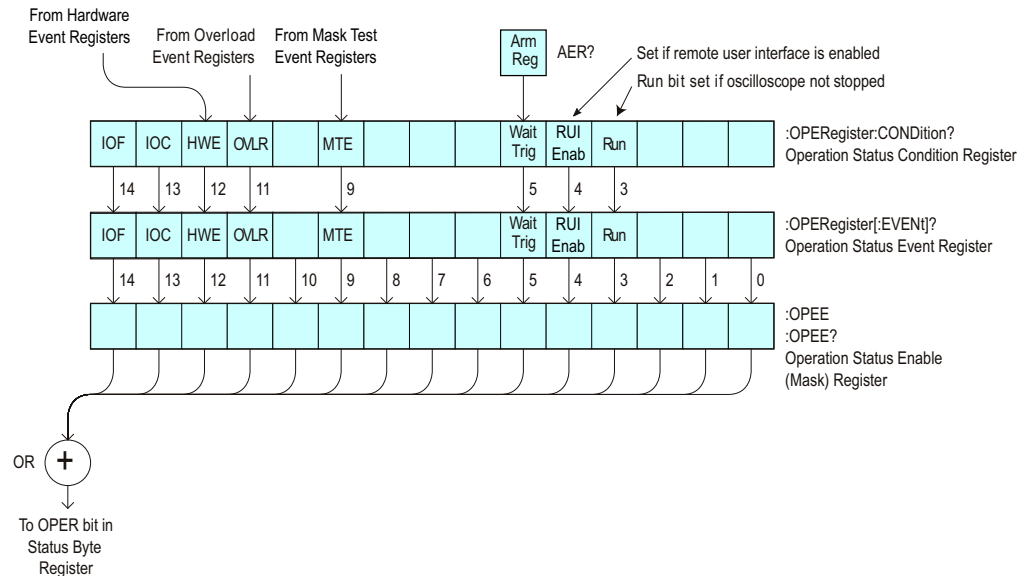


Table 92 Operation Status Event Register

Bit	Name	Description	When Set (1 = High = True), Indicates:
14	IOF	IO Operation Failed	Event only when something causes the IO operation to fail, like disconnecting a USB device or interrupting the operation from the oscilloscope's front panel.
13	IOC	IO Operation Complete	Event when any IO operation completes. IO operations are any remote data request using any interface (USB, LAN, or GPIB). For example, if you connect to an oscilloscope using the USB interface and then request waveform data, the IOC bit will be set when the IO operation completes.
12	HWE	Hardware Event	Event when hardware event occurs.
11	OVL	Overload	A 50Ω input overload has occurred.
10	---	---	(Not used.)

Table 92 Operation Status Event Register (continued)

Bit	Name	Description	When Set (1 = High = True), Indicates:
9	MTE	Mask Test Event	A mask test event has occurred.
8-6	---	---	(Not used.)
5	Wait Trig	Wait Trig	The trigger is armed (set by the Trigger Armed Event Register (TER)).
4	RUI Enab	Remote Enabled	The remote user interface has gone from a disabled state to an enabled state. The front-panel graphical user interface can disable most of the remote interface, including the *OPC syntax, for example when: ▪ a modal dialog box is open ▪ the user is being prompted ▪ all segments are being analyzed ▪ there is a channel overload ▪ certain compliance applications are running When disabled, commands or queries are accepted, but "settings conflict" errors are returned. The status model is the only part of the remote user interface that is enabled. To determine when the remote interface is re-enabled, you can read this bit or wait for the event that gets generated when its status goes from disabled to enabled.
3	Run	Running	The oscilloscope has gone from a stop state to a single or running state.
2-0	---	---	(Not used.)

Return Format <value><NL>

<value> ::= integer in NR1 format.

- See Also**
- ["Introduction to Root \(:\) Commands"](#) on page 240
 - [":CHANnel<n>:PROTection"](#) on page 352
 - [":OPEE \(Operation Status Enable Register\)"](#) on page 260
 - [":OPERegister:CONDition \(Operation Status Condition Register\)"](#) on page 262
 - [":OVLenable \(Overload Event Enable Register\)"](#) on page 268
 - [":OVLRegister \(Overload Event Register\)"](#) on page 270
 - ["*STB \(Read Status Byte\)"](#) on page 232
 - ["*SRE \(Service Request Enable\)"](#) on page 230

- **":MTERegister[:EVENT] (Mask Test Event Event Register)"** on page 258
- **":MTEenable (Mask Test Event Enable Register)"** on page 256
- **"*OPC (Operation Complete)"** on page 222
- **"Operation Status Event Register (:OPERRegister[:EVENT])"** on page 1611
- **"Example: Checking for Armed Status"** on page 1621
- **"Example: Waiting for IO Operation Complete"** on page 1626

:OVLenable (Overload Event Enable Register)

C (see [page 1650](#))

Command Syntax :OVLenable <enable_mask>
 <enable_mask> ::= 16-bit integer

The overload enable mask is an integer representing an input as described in the following table.

The :OVLenable command sets the mask in the Overload Event Enable Register and enables the reporting of the Overload Event Register. If an overvoltage is sensed on a 50Ω input, the input will automatically switch to 1 MΩ input impedance. If enabled, such an event will set bit 11 in the Operation Status Register.

NOTE

You can set analog channel input impedance to 50Ω on the 300 MHz, 500 MHz, and 1 GHz bandwidth oscilloscope models. On these same bandwidth models, if there are only two analog channels, you can also set external trigger input impedance to 50Ω.

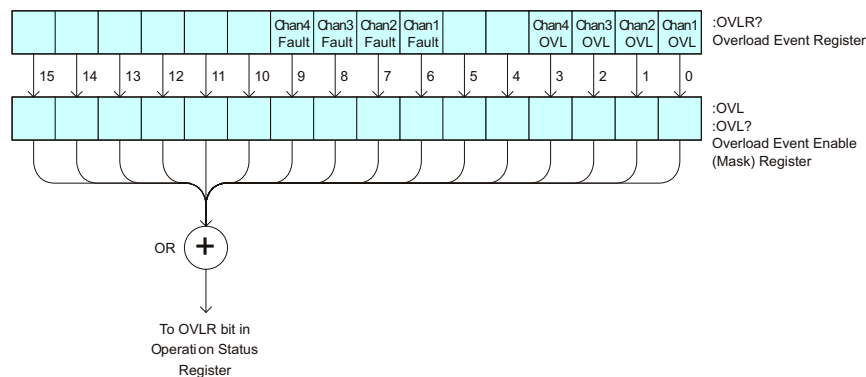


Table 93 Overload Event Enable Register (OVL)

Bit	Description	When Set (1 = High = True), Enables:
15-10	---	(Not used.)
9	Channel 4 Fault	Event when fault occurs on Channel 4 input.
8	Channel 3 Fault	Event when fault occurs on Channel 3 input.
7	Channel 2 Fault	Event when fault occurs on Channel 2 input.
6	Channel 1 Fault	Event when fault occurs on Channel 1 input.
5-4	---	(Not used.)
3	Channel 4 OVL	Event when overload occurs on Channel 4 input.

Table 93 Overload Event Enable Register (OVL) (continued)

Bit	Description	When Set (1 = High = True), Enables:
2	Channel 3 OVL	Event when overload occurs on Channel 3 input.
1	Channel 2 OVL	Event when overload occurs on Channel 2 input.
0	Channel 1 OVL	Event when overload occurs on Channel 1 input.

Query Syntax :OVLenable?

The :OVLenable query returns the current enable mask value contained in the Overload Event Enable Register.

Return Format <enable_mask><NL>

<enable_mask> ::= integer in NR1 format.

- See Also**
- ["Introduction to Root \(:\) Commands"](#) on page 240
 - [":CHANnel<n>:PROTection"](#) on page 352
 - [":OPEE \(Operation Status Enable Register\)"](#) on page 260
 - [":OPERRegister:CONDition \(Operation Status Condition Register\)"](#) on page 262
 - [":OPERRegister\[:EVENT\] \(Operation Status Event Register\)"](#) on page 265
 - [":OVLRegister \(Overload Event Register\)"](#) on page 270
 - ["*STB \(Read Status Byte\)"](#) on page 232
 - ["*SRE \(Service Request Enable\)"](#) on page 230

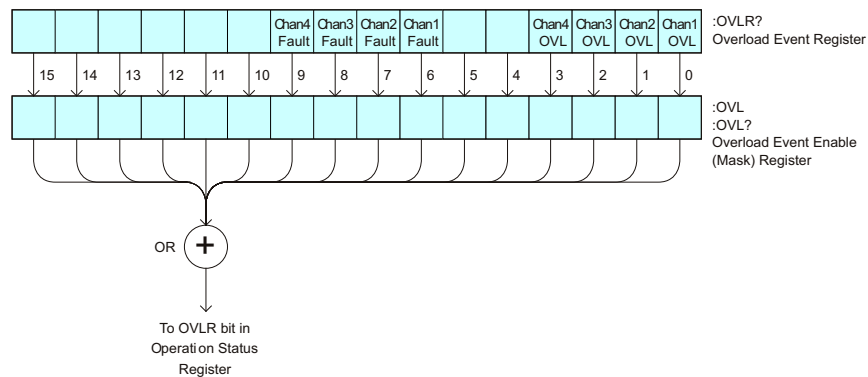
:OVLRegister (Overload Event Register)

C (see [page 1650](#))**Query Syntax** :OVLRegister?

The :OVLRegister query returns the overload protection value stored in the Overload Event Register (OVL). If an overvoltage is sensed on a 50Ω input, the input will automatically switch to 1 MΩ input impedance. A "1" indicates an overload has occurred.

NOTE

You can set analog channel input impedance to 50Ω on the 300 MHz, 500 MHz, and 1 GHz bandwidth oscilloscope models. On these same bandwidth models, if there are only two analog channels, you can also set external trigger input impedance to 50Ω.

**Table 94** Overload Event Register (OVL)

Bit	Description	When Set (1 = High = True), Indicates:
15-10	---	(Not used.)
9	Channel 4 Fault	Fault has occurred on Channel 4 input.
8	Channel 3 Fault	Fault has occurred on Channel 3 input.
7	Channel 2 Fault	Fault has occurred on Channel 2 input.
6	Channel 1 Fault	Fault has occurred on Channel 1 input.
5-4	---	(Not used.)
3	Channel 4 OVL	Overload has occurred on Channel 4 input.
2	Channel 3 OVL	Overload has occurred on Channel 3 input.
1	Channel 2 OVL	Overload has occurred on Channel 2 input.
0	Channel 1 OVL	Overload has occurred on Channel 1 input.

Return Format <value><NL>

<value> ::= integer in NR1 format.

- See Also**
- ["Introduction to Root \(:\) Commands"](#) on page 240
 - [":CHANnel<n>:PROTection"](#) on page 352
 - [":OPEE \(Operation Status Enable Register\)"](#) on page 260
 - [":OVLenable \(Overload Event Enable Register\)"](#) on page 268
 - ["*STB \(Read Status Byte\)"](#) on page 232
 - ["*SRE \(Service Request Enable\)"](#) on page 230

:PRINT

C (see [page 1650](#))

Command Syntax :PRINT [<options>]

<options> ::= [<print option>][,...,<print option>]

<print option> ::= {COLor | GRAYscale | PRINter0 | PRINter1 | BMP8bit
| BMP | PNG | NOFactors | FACTors}

The <print option> parameter may be repeated up to 5 times.

The PRINT command formats the output according to the currently selected format (device). If an option is not specified, the value selected in the Print Config menu is used.

- See Also**
- ["Introduction to Root \(:\) Commands"](#) on page 240
 - ["Introduction to :HARDcopy Commands"](#) on page 506
 - [":HARDcopy:FACTors"](#) on page 510
 - [":HARDcopy:GRAYscale"](#) on page 1555
 - [":DISPlay:DATA"](#) on page 402

:RState?

N (see [page 1650](#))

Query Syntax :RState?

The :RState? query returns the run state:

- RUN – The oscilloscope is acquiring and displaying new waveforms.
- STOP – The oscilloscope is no longer acquiring new waveforms.
- SING – A single acquisition has been started and the oscilloscope is waiting for the trigger condition to be met.

These are the same run states displayed on the front panel and in the user interface.

Return Format {RUN | STOP | SING}<NL>

- See Also
- [":RUN"](#) on page 274
 - [":SINGle"](#) on page 276
 - [":STOP"](#) on page 278
 - [":OPERRegister\[:EVENT\] \(Operation Status Event Register\)"](#) on page 265

:RUN

C (see [page 1650](#))

Command Syntax

:RUN

The :RUN command starts repetitive acquisitions. This is the same as pressing the Run key on the front panel.

- See Also**
- ["Introduction to Root \(:\) Commands"](#) on page 240
 - [":SINGle"](#) on page 276
 - [":STOP"](#) on page 278

Example Code

```
' RUN_STOP - (not executed in this example)
' - RUN starts the data acquisition for the active waveform display.
' - STOP stops the data acquisition and turns off AUTOSTORE.
' myScope.WriteString ":RUN"      ' Start data acquisition.
' myScope.WriteString ":STOP"    ' Stop the data acquisition.
```

See complete example programs at: [Chapter 46](#), "Programming Examples," starting on page 1659

:SERial

N (see [page 1650](#))

Query Syntax :SERial?

The :SERial? query returns the serial number of the instrument.

Return Format: Unquoted string<NL>

See Also • ["Introduction to Root \(:\) Commands"](#) on page 240

:SINGle

C (see [page 1650](#))

Command Syntax

:SINGle

The :SINGle command causes the instrument to acquire a single trigger of data. This is the same as pressing the Single key on the front panel.

- See Also
- ["Introduction to Root \(:\) Commands"](#) on page 240
 - [":RUN"](#) on page 274
 - [":STOP"](#) on page 278

:STATus

N (see [page 1650](#))

Query Syntax

:STATus? <source>

```
<source> ::= {CHANnel<n> | FUNCTION<m> | MATH<m> | SBUS{1 | 2}
              | WMEMory<r>}
              for the DSO models
```

```
<source> ::= {CHANnel<n> | DIGital<d> | POD{1 | 2}
              | BUS{1 | 2} | FUNCTION<m> | MATH<m> | SBUS{1 | 2}
              | WMEMory<r>}
              for the MSO models
```

```
<n> ::= 1 to (# analog channels) in NR1 format
```

```
<d> ::= 0 to (# digital channels - 1) in NR1 format
```

```
<m> ::= 1 to (# math functions) in NR1 format
```

```
<r> ::= 1 to (# ref waveforms) in NR1 format
```

The :STATus? query reports whether the channel, function, serial decode bus, or reference waveform location specified by <source> is displayed.

NOTE

MATH<m> is an alias for FUNCTION<m>.

Return Format

<value><NL>

```
<value> ::= {1 | 0}
```

- See Also
- ["Introduction to Root \(:\) Commands"](#) on page 240
 - [":BLANK"](#) on page 248
 - [":CHANnel<n>:DISPlay"](#) on page 332
 - [":DIGital<d>:DISPlay"](#) on page 385
 - [":FUNCTION<m>:DISPlay"](#) on page 463
 - [":POD<n>:DISPlay"](#) on page 747
 - [":SBUS<n>:DISPlay"](#) on page 902
 - [":WMEMory<r>:DISPlay"](#) on page 1524
 - [":VIEW"](#) on page 280

:STOP

C (see [page 1650](#))

Command Syntax

:STOP

The :STOP command stops the acquisition. This is the same as pressing the Stop key on the front panel.

- See Also**
- ["Introduction to Root \(:\) Commands"](#) on page 240
 - [":RUN"](#) on page 274
 - [":SINGle"](#) on page 276

- Example Code**
- ["Example Code"](#) on page 274

:TER (Trigger Event Register)

C (see [page 1650](#))

Query Syntax :TER?

The :TER? query reads the Trigger Event Register. After the Trigger Event Register is read, it is cleared. A one indicates a trigger has occurred. A zero indicates a trigger has not occurred.

The Trigger Event Register is summarized in the TRG bit of the Status Byte Register (STB). A Service Request (SRQ) can be generated when the TRG bit of the Status Byte transitions, and the TRG bit is set in the Service Request Enable register. The Trigger Event Register must be cleared each time you want a new service request to be generated.

Return Format <value><NL>

<value> ::= {1 | 0}; a 16-bit integer in NR1 format.

- See Also
- ["Introduction to Root \(:\) Commands"](#) on page 240
 - ["*SRE \(Service Request Enable\)"](#) on page 230
 - ["*STB \(Read Status Byte\)"](#) on page 232

:VIEW

N (see [page 1650](#))

Command Syntax

:VIEW <source>

<source> ::= {CHANnel<n> | FUNctIon<m> | MATH<m> | SBUS{1 | 2}
| WMEMory<r>}
for DSO models

<source> ::= {CHANnel<n> | DIGital<d> | POD{1 | 2}
| BUS{1 | 2} | FUNctIon<m> | MATH<m> | SBUS{1 | 2}
| WMEMory<r>}
for MSO models

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :VIEW command turns on the specified channel, function, serial decode bus, or reference waveform location.

NOTE

MATH<m> is an alias for FUNctIon<m>.

- See Also
- ["Introduction to Root \(:\) Commands"](#) on page 240
 - [":BLANK"](#) on page 248
 - [":CHANnel<n>:DISPlay"](#) on page 332
 - [":DIGital<d>:DISPlay"](#) on page 385
 - [":FUNctIon<m>:DISPlay"](#) on page 463
 - [":POD<n>:DISPlay"](#) on page 747
 - [":SBUS<n>:DISPlay"](#) on page 902
 - [":WMEMory<r>:DISPlay"](#) on page 1524
 - [":STATus"](#) on page 277

Example Code

```
' VIEW_BLANK - (not executed in this example)
' - VIEW turns on (starts displaying) a channel.
' - BLANK turns off (stops displaying) a channel.
' myScope.WriteString ":BLANK CHANnel1"      ' Turn channel 1 off.
' myScope.WriteString ":VIEW CHANnel1"      ' Turn channel 1 on.
```

See complete example programs at: [Chapter 46](#), "Programming Examples," starting on page 1659

8 :ACQUIRE Commands

Set the parameters for acquiring and storing data. See ["Introduction to :ACQUIRE Commands"](#) on page 282.

Table 95 :ACQUIRE Commands Summary

Command	Query	Options and Query Returns
n/a	:ACQUIRE:AALias? (see page 284)	{1 0}
:ACQUIRE:COMPLete <complete> (see page 285)	:ACQUIRE:COMPLete? (see page 285)	<complete> ::= 100; an integer in NR1 format
:ACQUIRE:COUNt <count> (see page 286)	:ACQUIRE:COUNt? (see page 286)	<count> ::= an integer from 2 to 65536 in NR1 format
:ACQUIRE:DAALias <mode> (see page 287)	:ACQUIRE:DAALias? (see page 287)	<mode> ::= {DISable AUTO}
:ACQUIRE:DIGitizer {0 OFF} {1 ON} (see page 288)	:ACQUIRE:DIGitizer? (see page 288)	{0 1}
:ACQUIRE:MODE <mode> (see page 289)	:ACQUIRE:MODE? (see page 289)	<mode> ::= {RTIME ETIME SEGMENTed}
:ACQUIRE:POINTs[:ANALog] <points> (see page 290)	:ACQUIRE:POINTs[:ANALog]? (see page 290)	<points> ::= {AUTO <points_value>} <points_value> ::= desired analog memory depth in integer NR1 format
:ACQUIRE:POINTs[:ANALog]:AUTO {0 OFF} {1 ON} (see page 291)	:ACQUIRE:POINTs[:ANALog]:AUTO? (see page 291)	{0 1}
:ACQUIRE:RSIGnal <ref_signal_mode> (see page 292)	:ACQUIRE:RSIGnal? (see page 292)	<ref_signal_mode> ::= {OFF OUT IN}

Table 95 :ACquire Commands Summary (continued)

Command	Query	Options and Query Returns
:ACquire:SEGmented:ANALyze (see page 293)	n/a	n/a
:ACquire:SEGmented:COUnt <count> (see page 294)	:ACquire:SEGmented:COUnt? (see page 294)	<count> ::= an integer from 2 to 1000 in NR1 format
:ACquire:SEGmented:INdex <index> (see page 295)	:ACquire:SEGmented:INdex? (see page 295)	<index> ::= an integer from 1 to 1000 in NR1 format
:ACquire:SRATe[:ANALog] <rate> (see page 298)	:ACquire:SRATe[:ANALog]? (see page 298)	<rate> ::= {AUTO <sample_rate>} <sample_rate> ::= desired analog sample rate in NR3 format
:ACquire:SRATe[:ANALog]:AUTO {{0 OFF} {1 ON}} (see page 299)	:ACquire:SRATe[:ANALog]:AUTO? (see page 299)	{0 1}
:ACquire:TYPE <type> (see page 300)	:ACquire:TYPE? (see page 300)	<type> ::= {NORMAL AVERage HRESolution PEAK}

Introduction to :ACquire Commands The ACquire subsystem controls the way in which waveforms are acquired. These acquisition types are available: normal, averaging, peak detect, and high resolution.

Normal

The :ACquire:TYPE NORMAL command sets the oscilloscope in the normal acquisition mode. For the majority of user models and signals, NORMAL mode yields the best oscilloscope picture of the waveform.

Averaging

The :ACquire:TYPE AVERage command sets the oscilloscope in the averaging mode. You can set the count by sending the :ACquire:COUNT command followed by the number of averages. In this mode, the value for averages is an integer from 2 to 65536. The COUNT value determines the number of averages that must be acquired.

High-Resolution

The :ACquire:TYPE HRESolution command sets the oscilloscope in the high-resolution mode (also known as *smoothing*). This mode is used to reduce noise at slower sweep speeds where the digitizer samples faster than needed to fill memory for the displayed time range. Instead of decimating samples, they are

averaged together to provide the value for one display point. The slower the sweep speed, the greater the number of samples that are averaged together for each display point.

Peak Detect

The :ACQuire:TYPE PEAK command sets the oscilloscope in the peak detect mode. In this mode, :ACQuire:COUNt has no meaning.

Reporting the Setup

Use :ACQuire? to query setup information for the ACQuire subsystem.

Return Format

The following is a sample response from the :ACQuire? query. In this case, the query was issued following a *RST command.

```
:ACQ:MODE RTIM;TYPE NORM;COMP 100;COUNT 8;SEGM:COUN 2
```

:ACquire:AALias

N (see [page 1650](#))

Query Syntax :ACquire:AALias?

The :ACquire:AALias? query returns the current state of the oscilloscope acquisition anti-alias control. This control can be directly disabled or disabled automatically.

Return Format <value><NL>

<value> ::= {1 | 0}

- See Also**
- ["Introduction to :ACquire Commands"](#) on page 282
 - [":ACquire:DAALias"](#) on page 287

:ACQuire:COMPLete

C (see [page 1650](#))

Command Syntax :ACQuire:COMPLete <complete>

<complete> ::= 100; an integer in NR1 format

The :ACQuire:COMPLete command affects the operation of the :DIGitize command. It specifies the minimum completion criteria for an acquisition. The parameter determines the percentage of the time buckets that must be "full" before an acquisition is considered complete. If :ACQuire:TYPE is NORMAl, it needs only one sample per time bucket for that time bucket to be considered full.

The only legal value for the :COMPLete command is 100. All time buckets must contain data for the acquisition to be considered complete.

Query Syntax :ACQuire:COMPLete?

The :ACQuire:COMPLete? query returns the completion criteria (100) for the currently selected mode.

Return Format <completion_criteria><NL>

<completion_criteria> ::= 100; an integer in NR1 format

- See Also**
- ["Introduction to :ACQuire Commands"](#) on page 282
 - [":ACQuire:TYPE"](#) on page 300
 - [":DIGitize"](#) on page 250
 - [":WAVEform:POINts"](#) on page 1446

Example Code

```
' ACQUIRE_COMPLETE - Specifies the minimum completion criteria for
' an acquisition. The parameter determines the percentage of time
' buckets needed to be "full" before an acquisition is considered
' to be complete.
myScope.WriteString ":ACQuire:COMPLete 100"
```

See complete example programs at: [Chapter 46](#), "Programming Examples," starting on page 1659

:ACquire:COUNT

C (see [page 1650](#))

Command Syntax :ACquire:COUNT <count>
 <count> ::= integer in NR1 format

In averaging mode, the :ACquire:COUNT command specifies the number of values to be averaged for each time bucket before the acquisition is considered to be complete for that time bucket. When :ACquire:TYPE is set to AVERage, the count can be set to any value from 2 to 65536.

NOTE

The :ACquire:COUNT 1 command has been deprecated. The AVERage acquisition type with a count of 1 is functionally equivalent to the HRESolution acquisition type; however, you should select the high-resolution acquisition mode with the :ACquire:TYPE HRESolution command instead.

Query Syntax :ACquire:COUNT?

The :ACquire:COUNT? query returns the currently selected count value for averaging mode.

Return Format <count_argument><NL>
 <count_argument> ::= an integer from 2 to 65536 in NR1 format

- See Also**
- ["Introduction to :ACquire Commands"](#) on page 282
 - [":ACquire:TYPE"](#) on page 300
 - [":DIGitize"](#) on page 250
 - [":WAVEform:COUNT"](#) on page 1442

:ACQUIRE:DAALias

N (see [page 1650](#))

Command Syntax :ACQUIRE:DAALias <mode>

<mode> ::= {DISable | AUTO}

The :ACQUIRE:DAALias command sets the disable anti-alias mode of the oscilloscope.

When set to DISable, anti-alias is always disabled. This is good for cases where dithered data is not desired.

When set to AUTO, the oscilloscope turns off anti-alias control as needed. Such cases are when the FFT or differentiate math functions are silent. The :DIGitize command always turns off the anti-alias control as well.

Query Syntax :ACQUIRE:DAALias?

The :ACQUIRE:DAALias? query returns the oscilloscope's current disable anti-alias mode setting.

Return Format <mode><NL>

<mode> ::= {DIS | AUTO}

- See Also**
- ["Introduction to :ACQUIRE Commands"](#) on page 282
 - [":ACQUIRE:AALias"](#) on page 284

:ACquire:DIgitizer

N (see [page 1650](#))

Command Syntax :ACquire:DIgitizer {{0 | OFF} | {1 | ON}}

The :ACquire:DIgitizer command turns Digitizer mode on or off.

Normally, when Digitizer mode is disabled (Automatic mode), the oscilloscope's time per division setting determines the sample rate and memory depth so as to fill the waveform display with data while the oscilloscope is running (continuously making acquisitions). For single acquisitions, the time/division setting still determines the sample rate, but the maximum amount of acquisition memory is used.

In Digitizer mode, you choose the acquisition sample rate and memory depth, and those settings are used even though the captured data may extend way beyond the edges of, or take up just a small portion of, the waveform display, depending on the oscilloscope's time/div setting.

Digitizer mode cannot be used along with these other oscilloscope features: XY and Roll time modes, horizontal Zoom display, time references other than Center, segmented memory, serial decode, digital channels, frequency response analysis, mask test, and the power application. In most cases, enabling one of these features when Digitizer mode is enabled will automatically disable Digitizer mode, and then disabling the feature will automatically reenable Digitizer mode.

Digitizer mode primarily aids external software that controls and combines data from multiple instruments.

Query Syntax :ACquire:DIgitizer?

The :ACquire:DIgitizer? query returns whether Digitizer mode is off or on.

Return Format <setting><NL>
<setting> ::= {0 | 1}

- See Also**
- [":ACquire:SRATe\[:ANALog\]"](#) on page 298
 - [":ACquire:SRATe\[:ANALog\]:AUTO"](#) on page 299
 - [":ACquire:POINts\[:ANALog\]"](#) on page 290
 - [":ACquire:POINts\[:ANALog\]:AUTO"](#) on page 291

:ACquire:MODE

C (see [page 1650](#))

Command Syntax :ACquire:MODE <mode>

<mode> ::= {RTIME | ETIME | SEGmented}

The :ACquire:MODE command sets the acquisition mode of the oscilloscope.

- The :ACquire:MODE RTIME command sets the oscilloscope in real time mode.

NOTE

The obsolete command ACquire:TYPE:REALtime is functionally equivalent to sending ACquire:MODE RTIME; TYPE NORMAl.

- The :ACquire:MODE ETIME command sets the oscilloscope in equivalent time mode. Available with 4000 X-Series 1 GHz and 1.5 GHz bandwidth models only.
- The :ACquire:MODE SEGmented command sets the oscilloscope in segmented memory mode.

Query Syntax :ACquire:MODE?

The :ACquire:MODE? query returns the acquisition mode of the oscilloscope.

Return Format <mode_argument><NL>

<mode_argument> ::= {RTIM | ETIM | SEGM}

- See Also**
- ["Introduction to :ACquire Commands"](#) on page 282
 - [":ACquire:TYPE"](#) on page 300

:ACquire:POINts[:ANALog]

C (see [page 1650](#))

Command Syntax :ACquire:POINts[:ANALog] <points>

<points> ::= {AUTO | <points_value>}

<points_value> ::= desired analog memory depth in integer NR1 format

Sets the desired acquisition memory depth.

- AUTO – puts the oscilloscope into Automatic (default) mode where the optimal memory depth is selected automatically by the oscilloscope.
- <points_value> – If a numerical memory depth is entered, this puts the oscilloscope into Digitizer (manual) Mode with the desired number of points.

The Automatic or Digitizer mode setting also affects the sample rate (see the :ACquire:SRATe[:ANALog] command).

The actual memory depth used is returned by the :ACquire:POINts[:ANALog]? query.

For :SINGle acquisitions, the requested memory depth is used.

When the oscilloscope is running (:RUN command, continuously making acquisitions) or when the :DIGitize command is used, the requested amount of memory is halved.

The maximum amount of memory available is halved when both channels in a pair are turned on. Channels 1 and 2 are one pair, channels 3 and 4 are the other.

Query Syntax :ACquire:POINts[:ANALog] ?

The :ACquire:POINts[:ANALog]? query returns the number of data points that the hardware will acquire from the input signal. The number of points acquired is not directly controllable. To set the number of points to be transferred from the oscilloscope, use the command :WAVEform:POINts. The :WAVEform:POINts? query will return the number of points available to be transferred from the oscilloscope.

Return Format <points_argument><NL>

<points_argument> ::= an integer in NR1 format

- See Also**
- ["Introduction to :ACquire Commands"](#) on page 282
 - [":ACquire:SRATe\[:ANALog\]"](#) on page 298
 - [":ACquire:POINts\[:ANALog\]:AUTO"](#) on page 291
 - [":ACquire:DIGitizer"](#) on page 288
 - [":DIGitize"](#) on page 250
 - [":WAVEform:POINts"](#) on page 1446

:ACQUIRE:POINTS[:ANALOG]:AUTO

N (see [page 1650](#))

Command Syntax :ACQUIRE:POINTS[:ANALOG]:AUTO {{0 | OFF} | {1 | ON}}

The :ACQUIRE:POINTS[:ANALOG]:AUTO command enables or disables Automatic determination of the analog channel memory depth:

- ON – the analog channel memory depth is automatically determined by the oscilloscope based on the horizontal time/div setting (Automatic mode, the oscilloscope's default).

This is equivalent to turning Digitizer mode off using the :ACQUIRE:DIGITIZER command.

- OFF – the analog channel memory depth is set using the :ACQUIRE:POINTS[:ANALOG] command.

This is equivalent to turning Digitizer mode on using the :ACQUIRE:DIGITIZER command.

The Automatic or Digitizer mode setting also affects the sample rate (see the :ACQUIRE:SRATE[:ANALOG] command).

Query Syntax :ACQUIRE:POINTS[:ANALOG]:AUTO?

The :ACQUIRE:POINTS[:ANALOG]:AUTO? query returns the automatic analog memory depth setting.

Return Format <setting><NL>
<setting> ::= {0 | 1}

- See Also**
- [":ACQUIRE:DIGITIZER"](#) on page 288
 - [":ACQUIRE:POINTS\[:ANALOG\]"](#) on page 290
 - [":ACQUIRE:SRATE\[:ANALOG\]"](#) on page 298

:ACquire:RSIGnal

N (see [page 1650](#))

Command Syntax :ACquire:RSIGnal <ref_signal_mode>

<ref_signal_mode> ::= {OFF | OUT | IN}

The :ACquire:RSIGnal command selects the 10 MHz reference signal mode.

- The OFF mode disables the oscilloscope's 10 MHz REF BNC connector.
- The OUT mode is used to synchronize the timebase of two or more instruments.
- The IN mode is used to supply a sample clock to the oscilloscope. A 10 MHz square or sine wave signal is input to the BNC connector labeled 10 MHz REF. The amplitude must be from - 5 dBm to 17 dBm (356 mVpp to 4.48 Vpp).

CAUTION

Do not apply more than 20 dBm Max (6.32 Vpp Max) at the 10 MHz REF BNC connector on the rear panel or damage to the instrument may occur.

Query Syntax :ACquire:RSIGnal?

The :ACquire:RSIGnal? query returns the current 10 MHz reference signal mode.

Return Format <ref_signal_mode><NL>

<ref_signal_mode> ::= {OFF | OUT | IN}

- See Also**
- [":TIMebase:REFClock"](#) on page 1336
 - The *Keysight InfiniiVision 4000 X Series Oscilloscopes User's Guide* for information on using the 10 MHz reference clock.

:ACQUIRE:SEGMENTED:ANALYZE

N (see [page 1650](#))

Command Syntax :ACQUIRE:SEGMENTED:ANALYZE

This command calculates measurement statistics and/or infinite persistence over all segments that have been acquired. It corresponds to the front panel **Analyze Segments** softkey which appears in both the Measurement Statistics and Segmented Memory Menus.

In order to use this command, the oscilloscope must be stopped and in segmented acquisition mode, with either quick measurements or infinite persistence on.

- See Also**
- [":ACQUIRE:MODE"](#) on page 289
 - [":ACQUIRE:SEGMENTED:COUNT"](#) on page 294
 - ["Introduction to :ACQUIRE Commands"](#) on page 282

:ACQuire:SEGMented:COUNT

N (see [page 1650](#))

Command Syntax :ACQuire:SEGMented:COUNT <count>

<count> ::= an integer from 2 to 1000 (w/4M memory) in NR1 format

The :ACQuire:SEGMented:COUNT command sets the number of memory segments to acquire.

The segmented memory acquisition mode is enabled with the :ACQuire:MODE command, and data is acquired using the :DIGitize, :SINGle, or :RUN commands. The number of memory segments in the current acquisition is returned by the :WAVEform:SEGMented:COUNT? query.

The maximum number of segments may be limited by the memory depth of your oscilloscope. For example, an oscilloscope with 1M memory allows a maximum of 250 segments.

Query Syntax :ACQuire:SEGMented:COUNT?

The :ACQuire:SEGMented:COUNT? query returns the current count setting.

Return Format <count><NL>

<count> ::= an integer from 2 to 1000 (w/4M memory) in NR1 format

- See Also**
- [":ACQuire:MODE"](#) on page 289
 - [":DIGitize"](#) on page 250
 - [":SINGle"](#) on page 276
 - [":RUN"](#) on page 274
 - [":WAVEform:SEGMented:COUNT"](#) on page 1454
 - [":ACQuire:SEGMented:ANALyze"](#) on page 293
 - ["Introduction to :ACQuire Commands"](#) on page 282

Example Code • ["Example Code"](#) on page 295

:ACquire:SEGMented:INdex

N (see [page 1650](#))

Command Syntax :ACquire:SEGMented:INdex <index>

<index> ::= an integer from 1 to 1000 (w/4M memory) in NR1 format

The :ACquire:SEGMented:INdex command sets the index into the memory segments that have been acquired.

The segmented memory acquisition mode is enabled with the :ACquire:MODE command. The number of segments to acquire is set using the :ACquire:SEGMented:COUNT command, and data is acquired using the :DIGitize, :SINGle, or :RUN commands. The number of memory segments that have been acquired is returned by the :WAVEform:SEGMented:COUNT? query. The time tag of the currently indexed memory segment is returned by the :WAVEform:SEGMented:TTAG? query.

The maximum number of segments may be limited by the memory depth of your oscilloscope. For example, an oscilloscope with 1M memory allows a maximum of 250 segments.

Query Syntax :ACquire:SEGMented:INdex?

The :ACquire:SEGMented:INdex? query returns the current segmented memory index setting.

Return Format <index><NL>

<index> ::= an integer from 1 to 1000 (w/4M memory) in NR1 format

- See Also**
- [":ACquire:MODE"](#) on page 289
 - [":ACquire:SEGMented:COUNT"](#) on page 294
 - [":DIGitize"](#) on page 250
 - [":SINGle"](#) on page 276
 - [":RUN"](#) on page 274
 - [":WAVEform:SEGMented:COUNT"](#) on page 1454
 - [":WAVEform:SEGMented:TTAG"](#) on page 1455
 - [":ACquire:SEGMented:ANALyze"](#) on page 293
 - ["Introduction to :ACquire Commands"](#) on page 282

Example Code ' Segmented memory commands example.

' -----

Option Explicit

```
Public myMgr As VisaComLib.ResourceManager
Public myScope As VisaComLib.FormattedIO488
Public varQueryResult As Variant
```

```

Public strQueryResult As String

Private Declare Sub Sleep Lib "kernel32" (ByVal dwMilliseconds As Long)

Sub Main()

    On Error GoTo VisaComError

    ' Create the VISA COM I/O resource.
    Set myMgr = New VisaComLib.ResourceManager
    Set myScope = New VisaComLib.FormattedIO488
    Set myScope.IO = _
        myMgr.Open("USB0::0x0957::0x17A6::US50210029::0::INSTR")
    myScope.IO.Clear ' Clear the interface.

    ' Turn on segmented memory acquisition mode.
    myScope.WriteString ":ACquire:MODE SEGmented"
    myScope.WriteString ":ACquire:MODE?"
    strQueryResult = myScope.ReadString
    Debug.Print "Acquisition mode: " + strQueryResult

    ' Set the number of segments to 25.
    myScope.WriteString ":ACquire:SEGmented:COUNT 25"
    myScope.WriteString ":ACquire:SEGmented:COUNT?"
    strQueryResult = myScope.ReadString
    Debug.Print "Acquisition memory segments: " + strQueryResult

    ' If data will be acquired within the IO timeout:
    'myScope.IO.Timeout = 10000
    'myScope.WriteString ":DIGitize"
    'Debug.Print ":DIGitize blocks until all segments acquired."
    'myScope.WriteString ":WAVEform:SEGmented:COUNT?"
    'varQueryResult = myScope.ReadNumber

    ' Or, to poll until the desired number of segments acquired:
    myScope.WriteString ":SINGLE"
    Debug.Print ":SINGLE does not block until all segments acquired."
    Do
        Sleep 100 ' Small wait to prevent excessive queries.
        myScope.WriteString ":WAVEform:SEGmented:COUNT?"
        varQueryResult = myScope.ReadNumber
    Loop Until varQueryResult = 25

    Debug.Print "Number of segments in acquired data: " _
        + FormatNumber(varQueryResult)

    Dim lngSegments As Long
    lngSegments = varQueryResult

    ' For each segment:
    Dim dblTimeTag As Double
    Dim lngI As Long

    For lngI = lngSegments To 1 Step -1

        ' Set the segmented memory index.
        myScope.WriteString ":ACquire:SEGmented:INDEX " + CStr(lngI)

```

```

myScope.WriteString ":ACquire:SEGmented:INDEX?"
strQueryResult = myScope.ReadString
Debug.Print "Acquisition memory segment index: " + strQueryResult

' Display the segment time tag.
myScope.WriteString ":WAVEform:SEGmented:TTAG?"
dblTimeTag = myScope.ReadNumber
Debug.Print "Segment " + CStr(lngI) + " time tag: " _
    + FormatNumber(dblTimeTag, 12)

Next lngI

Exit Sub

VisaComError:
MsgBox "VISA COM Error:" + vbCrLf + Err.Description

End Sub

```

:ACquire:SRATe[:ANALog]

N (see [page 1650](#))

Command Syntax :ACquire:SRATe[:ANALog] <rate>

<rate> ::= {AUTO | <sample_rate>}

<sample_rate> ::= desired analog sample rate in NR3 format

Sets the desired acquisition sample rate:

- AUTO – puts the oscilloscope into Automatic (default) mode where the optimal sample rate is selected automatically by the oscilloscope.
- <sample_rate> – If a numerical sample rate value is entered, this puts the oscilloscope into Digitizer (manual) mode with the desired sample rate.

The Automatic or Digitizer mode setting also affects the memory depth (see the :ACquire:POINts[:ANALog] command).

The actual sample rate used is returned by the :ACquire:SRATe[:ANALog]? query.

Query Syntax :ACquire:SRATe[:ANALog]? [MAXimum]

The :ACquire:SRATe[:ANALog]? query returns the current oscilloscope acquisition sample rate. The sample rate is not directly controllable.

When the MAXimum parameter is used, the oscilloscope's maximum possible sample rate is returned.

Return Format <sample_rate><NL>

<sample_rate> ::= sample rate in NR3 format

- See Also**
- ["Introduction to :ACquire Commands"](#) on page 282
 - [":ACquire:POINts\[:ANALog\]"](#) on page 290
 - [":ACquire:SRATe\[:ANALog\]:AUTO"](#) on page 299
 - [":ACquire:DIGitizer"](#) on page 288

:ACQUIRE:SRATE[:ANALOG]:AUTO

N (see [page 1650](#))

Command Syntax :ACQUIRE:SRATE[:ANALOG]:AUTO {{0 | OFF} | {1 | ON}}

The :ACQUIRE:SRATE[:ANALOG]:AUTO command enables or disables Automatic determination of the analog channel sample rate:

- ON – the analog channel sample rate is automatically determined by the oscilloscope based on the horizontal time/div setting (Automatic mode, the oscilloscope's default).

This is equivalent to turning Digitizer mode off using the :ACQUIRE:DIGITIZER command.

- OFF – the analog channel sample rate is set using the :ACQUIRE:SRATE[:ANALOG] command.

This is equivalent to turning Digitizer mode on using the :ACQUIRE:DIGITIZER command.

The Automatic or Digitizer mode setting also affects the memory depth (see the :ACQUIRE:POINTS[:ANALOG] command).

Query Syntax :ACQUIRE:SRATE[:ANALOG]:AUTO?

The :ACQUIRE:SRATE[:ANALOG]:AUTO? query returns the automatic analog sample rate setting.

Return Format <setting><NL>
<setting> ::= {0 | 1}

- See Also**
- [":ACQUIRE:DIGITIZER"](#) on page 288
 - [":ACQUIRE:SRATE\[:ANALOG\]"](#) on page 298
 - [":ACQUIRE:POINTS\[:ANALOG\]"](#) on page 290

:ACquire:TYPE

C (see [page 1650](#))

Command Syntax :ACquire:TYPE <type>

<type> ::= {NORMal | AVERage | HRESolution | PEAK}

The :ACquire:TYPE command selects the type of data acquisition that is to take place. The acquisition types are:

- **NORMal** – sets the oscilloscope in the normal mode.
- **AVERage** – sets the oscilloscope in the averaging mode. You can set the count by sending the :ACquire:COUNT command followed by the number of averages. In this mode, the value for averages is an integer from 1 to 65536. The COUNT value determines the number of averages that must be acquired.

The AVERage type is not available when in segmented memory mode (:ACquire:MODE SEGmented).

- **HRESolution** – sets the oscilloscope in the high-resolution mode (also known as *smoothing*). This mode is used to reduce noise at slower sweep speeds where the digitizer samples faster than needed to fill memory for the displayed time range.

For example, if the digitizer samples at 200 MSa/s, but the effective sample rate is 1 MSa/s (because of a slower sweep speed), only 1 out of every 200 samples needs to be stored. Instead of storing one sample (and throwing others away), the 200 samples are averaged together to provide the value for one display point. The slower the sweep speed, the greater the number of samples that are averaged together for each display point.

- **PEAK** – sets the oscilloscope in the peak detect mode. In this mode, :ACquire:COUNT has no meaning.

The AVERage and HRESolution types can give you extra bits of vertical resolution. See the *User's Guide* for an explanation. When getting waveform data acquired using the AVERage and HRESolution types, be sure to use the WORD or ASCII waveform data formats to get the extra bits of vertical resolution.

NOTE

The obsolete command ACquire:TYPE:REALtime is functionally equivalent to sending ACquire:MODE RTIME; TYPE NORMal.

Query Syntax :ACquire:TYPE?

The :ACquire:TYPE? query returns the current acquisition type.

Return Format <acq_type><NL>

<acq_type> ::= {NORM | AVER | HRES | PEAK}

- See Also
- ["Introduction to :ACQUIRE Commands"](#) on page 282
 - [":ACQUIRE:COUNT"](#) on page 286
 - [":ACQUIRE:MODE"](#) on page 289
 - [":DIGITIZE"](#) on page 250
 - [":WAVEFORM:FORMAT"](#) on page 1445
 - [":WAVEFORM:TYPE"](#) on page 1462
 - [":WAVEFORM:PREAMBLE"](#) on page 1450

Example Code

```
' ACQUIRE_TYPE - Sets the acquisition mode, which can be NORMAL,
' PEAK, or AVERAGE.
myScope.WriteString ":ACQUIRE:TYPE NORMAL"
```

See complete example programs at: [Chapter 46](#), "Programming Examples," starting on page 1659

9 :BUS<n> Commands

Control all oscilloscope functions associated with buses made up of digital channels. See ["Introduction to :BUS<n> Commands"](#) on page 304.

Table 96 :BUS<n> Commands Summary

Command	Query	Options and Query Returns
:BUS<n>:BIT<m> {{0 OFF} {1 ON}} (see page 305)	:BUS<n>:BIT<m>? (see page 305)	{0 1} <n> ::= 1 or 2; an integer in NR1 format <m> ::= 0-15; an integer in NR1 format
:BUS<n>:BITS <channel_list>, {{0 OFF} {1 ON}} (see page 306)	:BUS<n>:BITS? (see page 306)	<channel_list>, {0 1} <channel_list> ::= (@<m>, <m>:<m> ...) where "," is separator and ":" is range <n> ::= 1 or 2; an integer in NR1 format <m> ::= 0-15; an integer in NR1 format
:BUS<n>:CLear (see page 308)	n/a	<n> ::= 1 or 2; an integer in NR1 format
:BUS<n>:DISPlay {{0 OFF} {1 ON}} (see page 309)	:BUS<n>:DISPlay? (see page 309)	{0 1} <n> ::= 1 or 2; an integer in NR1 format

Table 96 :BUS<n> Commands Summary (continued)

Command	Query	Options and Query Returns
:BUS<n>:LABel <string> (see page 310)	:BUS<n>:LABel? (see page 310)	<string> ::= quoted ASCII string up to 10 characters <n> ::= 1 or 2; an integer in NR1 format
:BUS<n>:MASK <mask> (see page 311)	:BUS<n>:MASK? (see page 311)	<mask> ::= 32-bit integer in decimal, <nondecimal>, or <string> <nondecimal> ::= #Hnn...n where n ::= {0,...,9 A,...,F} for hexadecimal <nondecimal> ::= #Bnn...n where n ::= {0 1} for binary <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F} for hexadecimal <n> ::= 1 or 2; an integer in NR1 format

Introduction to
:BUS<n>
Commands

<n> ::= {1 | 2}

The BUS subsystem commands control the viewing, labeling, and digital channel makeup of two possible buses.

NOTE

These commands are only valid for the MSO models.

Reporting the Setup

Use :BUS<n>? to query setup information for the BUS subsystem.

Return Format

The following is a sample response from the :BUS1? query. In this case, the query was issued following a *RST command.

```
:BUS1:DISP 0;LAB "BUS1";MASK +255
```

:BUS<n>:BIT<m>

N (see [page 1650](#))

Command Syntax :BUS<n>:BIT<m> <display>

<display> ::= {{1 | ON} | {0 | OFF}}

<n> ::= An integer, 1 or 2, is attached as a suffix to BUS and defines the bus that is affected by the command.

<m> ::= An integer, 0,...,15, is attached as a suffix to BIT and defines the digital channel that is affected by the command.

The :BUS<n>:BIT<m> command includes or excludes the selected bit as part of the definition for the selected bus. If the parameter is a 1 (ON), the bit is included in the definition. If the parameter is a 0 (OFF), the bit is excluded from the definition. *Note:* BIT0-15 correspond to DIGital0-15.

NOTE

This command is only valid for the MSO models.

Query Syntax :BUS<n>:BIT<m>?

The :BUS<n>:BIT<m>? query returns the value indicating whether the specified bit is included or excluded from the specified bus definition.

Return Format <display><NL>

<display> ::= {0 | 1}

- See Also**
- ["Introduction to :BUS<n> Commands"](#) on page 304
 - [":BUS<n>:BITS"](#) on page 306
 - [":BUS<n>:CLEar"](#) on page 308
 - [":BUS<n>:DISPlay"](#) on page 309
 - [":BUS<n>:LABel"](#) on page 310
 - [":BUS<n>:MASK"](#) on page 311

Example Code

```
' Include digital channel 1 in bus 1:
myScope.WriteString ":BUS1:BIT1 ON"
```

:BUS<n>:BITS

N (see [page 1650](#))

Command Syntax :BUS<n>:BITS <channel_list>, <display>

<channel_list> ::= (@<m>,<m>:<m>, ...) where commas separate bits and colons define bit ranges.

<m> ::= An integer, 0,...,15, defines a digital channel affected by the command.

<display> ::= {{1 | ON} | {0 | OFF}}

<n> ::= An integer, 1 or 2, is attached as a suffix to BUS and defines the bus that is affected by the command.

The :BUS<n>:BITS command includes or excludes the selected bits in the channel list in the definition of the selected bus. If the parameter is a 1 (ON) then the bits in the channel list are included as part of the selected bus definition. If the parameter is a 0 (OFF) then the bits in the channel list are excluded from the definition of the selected bus.

NOTE

This command is only valid for the MSO models.

Query Syntax :BUS<n>:BITS?

The :BUS<n>:BITS? query returns the definition for the specified bus.

Return Format <channel_list>, <display><NL>

<channel_list> ::= (@<m>,<m>:<m>, ...) where commas separate bits and colons define bit ranges.

<display> ::= {0 | 1}

- See Also**
- ["Introduction to :BUS<n> Commands"](#) on page 304
 - [":BUS<n>:BIT<m>"](#) on page 305
 - [":BUS<n>:CLEAr"](#) on page 308
 - [":BUS<n>:DISPlay"](#) on page 309
 - [":BUS<n>:LABel"](#) on page 310
 - [":BUS<n>:MASK"](#) on page 311

Example Code

```
' Include digital channels 1, 2, 4, 5, 6, 7, 8, and 9 in bus 1:
myScope.WriteString ":BUS1:BITS (@1,2,4:9), ON"

' Include digital channels 1, 5, 7, and 9 in bus 1:
myScope.WriteString ":BUS1:BITS (@1,5,7,9), ON"

' Include digital channels 1 through 15 in bus 1:
```

```
myScope.WriteString ":BUS1:BITS (@1:15), ON"  
  
' Include digital channels 1 through 5, 8, and 14 in bus 1:  
myScope.WriteString ":BUS1:BITS (@1:5,8,14), ON"
```

:BUS<n>:CLEar

N (see [page 1650](#))

Command Syntax :BUS<n>:CLEar

<n> ::= An integer, 1 or 2, is attached as a suffix to BUS and defines the bus that is affected by the command.

The :BUS<n>:CLEar command excludes all of the digital channels from the selected bus definition.

NOTE

This command is only valid for the MSO models.

-
- See Also
- ["Introduction to :BUS<n> Commands"](#) on page 304
 - [":BUS<n>:BIT<m>"](#) on page 305
 - [":BUS<n>:BITS"](#) on page 306
 - [":BUS<n>:DISPlay"](#) on page 309
 - [":BUS<n>:LABel"](#) on page 310
 - [":BUS<n>:MASK"](#) on page 311

:BUS<n>:DISPlay

N (see [page 1650](#))

Command Syntax :BUS<n>:DISPlay <value>

<value> ::= {{1 | ON} | {0 | OFF}}

<n> ::= An integer, 1 or 2, is attached as a suffix to BUS and defines the bus that is affected by the command.

The :BUS<n>:DISPlay command enables or disables the view of the selected bus.

NOTE

This command is only valid for the MSO models.

Query Syntax :BUS<n>:DISPlay?

The :BUS<n>:DISPlay? query returns the display value of the selected bus.

Return Format <value><NL>

<value> ::= {0 | 1}

- See Also**
- ["Introduction to :BUS<n> Commands"](#) on page 304
 - [":BUS<n>:BIT<m>"](#) on page 305
 - [":BUS<n>:BITS"](#) on page 306
 - [":BUS<n>:CLEAr"](#) on page 308
 - [":BUS<n>:LABel"](#) on page 310
 - [":BUS<n>:MASK"](#) on page 311

:BUS<n>:LABel

N (see [page 1650](#))

Command Syntax :BUS<n>:LABel <quoted_string>

<quoted_string> ::= any series of 10 or less characters as a quoted ASCII string.

<n> ::= An integer, 1 or 2, is attached as a suffix to BUS and defines the bus that is affected by the command.

The :BUS<n>:LABel command sets the bus label to the quoted string. Setting a label for a bus will also result in the name being added to the label list.

NOTE

This command is only valid for the MSO models.

NOTE

Label strings are 10 characters or less, and may contain any commonly used ASCII characters. Labels with more than 10 characters are truncated to 10 characters.

Query Syntax :BUS<n>:LABel?

The :BUS<n>:LABel? query returns the name of the specified bus.

Return Format <quoted_string><NL>

<quoted_string> ::= any series of 10 or less characters as a quoted ASCII string.

- See Also**
- ["Introduction to :BUS<n> Commands"](#) on page 304
 - [":BUS<n>:BIT<m>"](#) on page 305
 - [":BUS<n>:BITS"](#) on page 306
 - [":BUS<n>:CLEar"](#) on page 308
 - [":BUS<n>:DISPlay"](#) on page 309
 - [":BUS<n>:MASK"](#) on page 311
 - [":CHANnel<n>:LABel"](#) on page 335
 - [":DISPlay:LABList"](#) on page 409
 - [":DIGital<d>:LABel"](#) on page 386

Example Code

```
' Set the bus 1 label to "Data":
myScope.WriteString ":BUS1:LABel 'Data''
```

:BUS<n>:MASK

N (see [page 1650](#))

Command Syntax :BUS<n>:MASK <mask>

<mask> ::= 32-bit integer in decimal, <nondecimal>, or <string>

<nondecimal> ::= #Hnn...n where n ::= {0,...,9 | A,...,F} for hexadecimal

<nondecimal> ::= #Bnn...n where n ::= {0 | 1} for binary

<string> ::= "0xnn...n" where n ::= {0,...,9 | A,...,F} for hexadecimal

<n> ::= An integer, 1 or 2, is attached as a suffix to BUS and defines the bus that is affected by the command.

The :BUS<n>:MASK command defines the bits included and excluded in the selected bus according to the mask. Set a mask bit to a "1" to include that bit in the selected bus, and set a mask bit to a "0" to exclude it.

NOTE

This command is only valid for the MSO models.

Query Syntax :BUS<n>:MASK?

The :BUS<n>:MASK? query returns the mask value for the specified bus.

Return Format <mask><NL> in decimal format

- See Also**
- ["Introduction to :BUS<n> Commands"](#) on page 304
 - [":BUS<n>:BIT<m>"](#) on page 305
 - [":BUS<n>:BITS"](#) on page 306
 - [":BUS<n>:CLEar"](#) on page 308
 - [":BUS<n>:DISPlay"](#) on page 309
 - [":BUS<n>:LABel"](#) on page 310

10 :CALibrate Commands

Utility commands for viewing calibration status and for starting the user calibration procedure. See ["Introduction to :CALibrate Commands"](#) on page 314.

Table 97 :CALibrate Commands Summary

Command	Query	Options and Query Returns
n/a	:CALibrate:DATE? (see page 315)	<return value> ::= <year>,<month>,<day>; all in NR1 format
:CALibrate:LABel <string> (see page 316)	:CALibrate:LABel? (see page 316)	<string> ::= quoted ASCII string up to 32 characters
:CALibrate:OUTPut <signal> (see page 317)	:CALibrate:OUTPut? (see page 317)	<signal> ::= {TRIGgers MASK WAVEgen WGEN1 WGEN2 TSource} Note: WAVE and WGEN1 are equivalent. Note: WGEN2 only available on models with 2 WaveGen outputs.
n/a	:CALibrate:PROTected? (see page 319)	{"PROTected" "UNPROtected"}
:CALibrate:START (see page 320)	n/a	n/a
n/a	:CALibrate:STATus? (see page 321)	<return value> ::= <status_code>,<status_string> <status_code> ::= an integer status code <status_string> ::= an ASCII status string

Table 97 :CALibrate Commands Summary (continued)

Command	Query	Options and Query Returns
n/a	:CALibrate:TEMPerature? (see page 322)	<return value> ::= degrees C delta since last cal in NR3 format
n/a	:CALibrate:TIME? (see page 323)	<return value> ::= <hours>,<minutes>,<seconds>; all in NR1 format

**Introduction to
:CALibrate
Commands**

The CALibrate subsystem provides utility commands for:

- Determining the state of the calibration factor protection switch (CAL PROTECT).
- Saving and querying the calibration label string.
- Reporting the calibration time and date.
- Reporting changes in the temperature since the last calibration.
- Starting the user calibration procedure.

:CALibrate:DATE

N (see [page 1650](#))

Query Syntax :CALibrate:DATE?

The :CALibrate:DATE? query returns the date of the last calibration.

Return Format <date><NL>

<date> ::= year,month,day in NR1 format<NL>

See Also • ["Introduction to :CALibrate Commands"](#) on page 314

:CALibrate:LABel

N (see [page 1650](#))

Command Syntax :CALibrate:LABel <string>

<string> ::= quoted ASCII string of up to 32 characters in length,
not including the quotes

The CALibrate:LABel command saves a string that is up to 32 characters in length into the instrument's non-volatile memory. The string may be used to record calibration dates or other information as needed.

Query Syntax :CALibrate:LABel?

The :CALibrate:LABel? query returns the contents of the calibration label string.

Return Format <string><NL>

<string> ::= unquoted ASCII string of up to 32 characters in length

See Also • ["Introduction to :CALibrate Commands"](#) on page 314

:CALibrate:OUTPut

N (see [page 1650](#))

Command Syntax :CALibrate:OUTPut <signal>

<signal> ::= {TRIGgers | MASK | WAVEgen | WGEN1 | WGEN2 | TSource}

Note: WAVE and WGEN1 are equivalent.

Note: WGEN2 only available on models with 2 WaveGen outputs.

The CALibrate:OUTPut command sets the signal that is available on the rear panel TRIG OUT BNC:

- TRIGgers – pulse when a trigger event occurs. The output level is 0–5 V into an open circuit, and 0–2.5 V into 50 Ω .
- MASK – signal from mask test indicating a failure.
- WAVEgen, WGEN1, WGEN2 – waveform generator sync output signal. This signal depends on the :WGEN<w>:FUNCTION setting:

Waveform Type	Sync Signal Characteristics
SINusoid, SQUare, RAMP, PULSe, SINC, EXPRise, EXPFall, GAUSSian	The Sync signal is a TTL positive pulse that occurs when the waveform rises above zero volts (or the DC offset value).
DC, NOISe, CARDiac	N/A

- TSource – The raw trigger signal from the oscilloscope's trigger circuit is output to Trig Out. It produces a rising edge whenever the input source would cause a trigger, even though that might occur multiple times within the time of a single acquisition. The trigger source can be a front panel analog input channel or an external trigger input. The output level is 0–5 V into an open circuit, and 0–2.5 V into 50 Ω . This option is not available in all trigger modes.

Query Syntax :CALibrate:OUTPut?

The :CALibrate:OUTPut query returns the current source of the TRIG OUT BNC signal.

Return Format <signal><NL>

<signal> ::= {TRIG | MASK | WAVE | WGEN2 | TSO}

See Also • ["Introduction to :CALibrate Commands"](#) on page 314

- **":WGEN<w>:FUNction"** on page 1486

:CALibrate:PROTected

N (see [page 1650](#))

Query Syntax :CALibrate:PROTected?

The :CALibrate:PROTected? query returns the rear-panel calibration protect (CAL PROTECT) button state. The value PROTected indicates calibration is disabled, and UNPRotected indicates calibration is enabled.

Return Format <switch><NL>

<switch> ::= { "PROTected" | "UNPRotected" }

See Also • ["Introduction to :CALibrate Commands"](#) on page 314

:CALibrate:START

N (see [page 1650](#))

Command Syntax :CALibrate:START

The CALibrate:START command starts the user calibration procedure.

NOTE

Before starting the user calibration procedure, you must set the rear panel CALIBRATION switch to UNPROTECTED, and you must connect BNC cables from the TRIG OUT connector to the analog channel inputs. See the *User's Guide* for details.

-
- See Also**
- ["Introduction to :CALibrate Commands"](#) on page 314
 - [":CALibrate:PROTECTED"](#) on page 319

:CALibrate:STATus

N (see [page 1650](#))

Query Syntax :CALibrate:STATus?

The :CALibrate:STATus? query returns the summary results of the last user calibration procedure.

Return Format <return value><NL>
<return value> ::= <status_code>,<status_string>
<status_code> ::= an integer status code
<status_string> ::= an ASCII status string

The status codes and strings can be:

Status Code	Status String
+0	Calibrated
-1	Not Calibrated

See Also • ["Introduction to :CALibrate Commands"](#) on page 314

:CALibrate:TEMPerature

N (see [page 1650](#))

Query Syntax :CALibrate:TEMPerature?

The :CALibrate:TEMPerature? query returns the change in temperature since the last user calibration procedure.

Return Format <return value><NL>

<return value> ::= degrees C delta since last cal in NR3 format

See Also • ["Introduction to :CALibrate Commands"](#) on page 314

:CALibrate:TIME

N (see [page 1650](#))

Query Syntax :CALibrate:TIME?

The :CALibrate:TIME? query returns the time of the last calibration.

Return Format <date><NL>

<date> ::= hour,minutes,seconds in NR1 format

See Also • ["Introduction to :CALibrate Commands"](#) on page 314

11 :CHANnel<n> Commands

Control all oscilloscope functions associated with individual analog channels or groups of channels. See ["Introduction to :CHANnel<n> Commands"](#) on page 328.

Table 98 :CHANnel<n> Commands Summary

Command	Query	Options and Query Returns
:CHANnel<n>:BANDwidth <limit> (see page 329)	:CHANnel<n>:BANDwidth? ? [MAXimum] (see page 329)	<limit> ::= {20E6 <full_bw>} in NR3 format <n> ::= 1 to (# analog channels) in NR1 format
:CHANnel<n>:BWLimit { {0 OFF} {1 ON} } (see page 330)	:CHANnel<n>:BWLimit? (see page 330)	{0 1} <n> ::= 1 to (# analog channels) in NR1 format
:CHANnel<n>:COUpling <coupling> (see page 331)	:CHANnel<n>:COUpling? (see page 331)	<coupling> ::= {AC DC} <n> ::= 1 to (# analog channels) in NR1 format
:CHANnel<n>:DISPlay { {0 OFF} {1 ON} } (see page 332)	:CHANnel<n>:DISPlay? (see page 332)	{0 1} <n> ::= 1 to (# analog channels) in NR1 format
:CHANnel<n>:IMPedance <impedance> (see page 333)	:CHANnel<n>:IMPedance? ? (see page 333)	<impedance> ::= {ONEMeg FIFTy} <n> ::= 1 to (# analog channels) in NR1 format
:CHANnel<n>:INVert { {0 OFF} {1 ON} } (see page 334)	:CHANnel<n>:INVert? (see page 334)	{0 1} <n> ::= 1 to (# analog channels) in NR1 format
:CHANnel<n>:LABel <string> (see page 335)	:CHANnel<n>:LABel? (see page 335)	<string> ::= any series of 32 or less ASCII characters enclosed in quotation marks <n> ::= 1 to (# analog channels) in NR1 format

Table 98 :CHANnel<n> Commands Summary (continued)

Command	Query	Options and Query Returns
:CHANnel<n>:OFFSet <offset>[suffix] (see page 336)	:CHANnel<n>:OFFSet? (see page 336)	<offset> ::= Vertical offset value in NR3 format [suffix] ::= {V mV} <n> ::= 1-2 or 1-4; in NR1 format
:CHANnel<n>:PROBe <attenuation> (see page 337)	:CHANnel<n>:PROBe? (see page 337)	<attenuation> ::= Probe attenuation ratio in NR3 format <n> ::= 1-2 or 1-4; in NR1 format
:CHANnel<n>:PROBe:BTN <setting> (see page 338)	:CHANnel<n>:PROBe:BTN? (see page 338)	<n> ::= 1 to (# analog channels) in NR1 format <setting> ::= {HEADlight INFinimode RSTop SINGLE CDISplay AUToscale FTRigger QACtion NACtion NONE}
:CHANnel<n>:PROBe:CAL ibration (see page 339)	n/a	n/a
:CHANnel<n>:PROBe:EXT ernal {0 OFF} {1 ON} (see page 340)	:CHANnel<n>:PROBe:EXT ernal? (see page 340)	<n> ::= 1 to (# analog channels) in NR1 format <setting> ::= {0 1}
:CHANnel<n>:PROBe:EXT ernal:GAIN <gain_factor> (see page 341)	:CHANnel<n>:PROBe:EXT ernal:GAIN? (see page 341)	<n> ::= 1 to (# analog channels) in NR1 format <gain_factor> ::= a real number from 0.0001 to 1000 in NR3 format
:CHANnel<n>:PROBe:EXT ernal:OFFSet <offset> (see page 342)	:CHANnel<n>:PROBe:EXT ernal:OFFSet? (see page 342)	<n> ::= 1 to (# analog channels) in NR1 format <offset> ::= a real number for the offset value at center screen in NR3 format
:CHANnel<n>:PROBe:EXT ernal:UNITs <units> (see page 343)	:CHANnel<n>:PROBe:EXT ernal:UNITs? (see page 343)	<n> ::= 1 to (# analog channels) in NR1 format <units> ::= {VOLT AMPere}
:CHANnel<n>:PROBe:HEA D[:TYPE] <head_param> (see page 344)	:CHANnel<n>:PROBe:HEA D[:TYPE]? (see page 344)	<head_param> ::= {SEND0 SEND6 SEND12 SEND20 DIFF0 DIFF6 DIFF12 DIFF20 DSMA DSMA6 NONE} <n> ::= 1 to (# analog channels) in NR1 format

Table 98 :CHANnel<n> Commands Summary (continued)

Command	Query	Options and Query Returns
n/a	:CHANnel<n>:PROBe:ID? (see page 345)	<probe id> ::= unquoted ASCII string up to 11 characters <n> ::= 1 to (# analog channels) in NR1 format
:CHANnel<n>:PROBe:MMODel <value> (see page 346)	:CHANnel<n>:PROBe:MMODel? (see page 346)	<value> ::= {P5205 P5210 P6205 P6241 P6243 P6245 P6246 P6247 P6248 P6249 P6250 P6251 P670X P671X TCP202} <n> ::= 1 to (# analog channels) in NR1 format
:CHANnel<n>:PROBe:MODE <setting> (see page 347)	:CHANnel<n>:PROBe:MODE? (see page 347)	<n> ::= 1 to (# analog channels) in NR1 format <setting> ::= {DIFFerential DOFFset SEA SEB CM}
:CHANnel<n>:PROBe:RSENse <value> (see page 348)	:CHANnel<n>:PROBe:RSENse? (see page 348)	<value> ::= Ohms in NR3 format <n> ::= 1 to (# analog channels) in NR1 format
:CHANnel<n>:PROBe:SKEW <skew_value> (see page 349)	:CHANnel<n>:PROBe:SKEW? (see page 349)	<skew_value> ::= -100 ns to +100 ns in NR3 format <n> ::= 1 to (# analog channels) in NR1 format
:CHANnel<n>:PROBe:STYPe <signal type> (see page 350)	:CHANnel<n>:PROBe:STYPe? (see page 350)	<signal type> ::= {DIFFerential SINGLE} <n> ::= 1 to (# analog channels) in NR1 format
:CHANnel<n>:PROBe:ZOOM {{0 OFF} {1 ON}} (see page 351)	:CHANnel<n>:PROBe:ZOOM? (see page 351)	<setting> ::= {0 1} <n> ::= 1 to (# analog channels) in NR1 format
:CHANnel<n>:PROTection (see page 352)	:CHANnel<n>:PROTection? (see page 352)	{NORM TRIP} <n> ::= 1 to (# analog channels) in NR1 format
:CHANnel<n>:RANGe <range>[suffix] (see page 353)	:CHANnel<n>:RANGe? (see page 353)	<range> ::= Vertical full-scale range value in NR3 format [suffix] ::= {V mV} <n> ::= 1 to (# analog channels) in NR1 format

Table 98 :CHANnel<n> Commands Summary (continued)

Command	Query	Options and Query Returns
:CHANnel<n>:SCALe <scale>[suffix] (see page 354)	:CHANnel<n>:SCALe? (see page 354)	<scale> ::= Vertical units per division value in NR3 format [suffix] ::= {V mV} <n> ::= 1 to (# analog channels) in NR1 format
:CHANnel<n>:UNITs <units> (see page 355)	:CHANnel<n>:UNITs? (see page 355)	<units> ::= {VOLT AMPere} <n> ::= 1 to (# analog channels) in NR1 format
:CHANnel<n>:VERNier { {0 OFF} {1 ON} } (see page 356)	:CHANnel<n>:VERNier? (see page 356)	{0 1} <n> ::= 1 to (# analog channels) in NR1 format

Introduction to :CHANnel<n> Commands

<n> ::= 1 to (# analog channels) in NR1 format

The CHANnel<n> subsystem commands control an analog channel (vertical or Y-axis of the oscilloscope). Channels are independently programmable for all offset, probe, coupling, bandwidth limit, inversion, vernier, and range (scale) functions. The channel number (1, 2, 3, or 4) specified in the command selects the analog channel that is affected by the command.

A label command provides identifying annotations of up to 10 characters.

You can toggle the channel displays on and off with the :CHANnel<n>:DISPlay command as well as with the root level commands :VIEW and :BLANK.

NOTE

The obsolete CHANnel subsystem is supported.

Reporting the Setup

Use :CHANnel1?, :CHANnel2?, :CHANnel3? or :CHANnel4? to query setup information for the CHANnel<n> subsystem.

Return Format

The following are sample responses from the :CHANnel<n>? query. In this case, the query was issued following a *RST command.

```
:CHAN1:RANG +40.0E+00;OFFS +0.00000E+00;COUP DC;IMP ONEM;DISP 1;BWL 0;  
INV 0;LAB "1";UNIT VOLT;PROB +10E+00;PROB:SKEW +0.00E+00;STYP SING
```

:CHANnel<n>:BANDwidth

N (see [page 1650](#))

Command Syntax :CHANnel<n>:BANDwidth <limit>

<limit> ::= value in NR3 format

<n> ::= 1 to (# analog channels) in NR1 format

The :CHANnel<n>:BANDwidth command has the effect of enabling or disabling bandwidth limiting just like the :CHANnel<n>:BWLimit command. If you specify a limit of 20E6 or lower, 20 MHz bandwidth limiting is enabled. If you specify a value higher than 20E6, bandwidth limiting is disabled, and you get the full bandwidth of the oscilloscope.

Turning the bandwidth limit on removes unwanted high frequency noise from the waveform.

Bandwidth limit also limits the trigger signal path of the channel.

Query Syntax :CHANnel<n>:BANDwidth? [MAXimum]

The :CHANnel<n>:BANDwidth? query returns the current setting of the low-pass filter. The value will either be 20E6 or the full bandwidth of the oscilloscope.

When the MAXimum parameter is used, the oscilloscope's maximum possible (full) bandwidth is returned.

Return Format <limit><NL>

<limit> ::= {20E6 | <full_bw>} in NR3 format)

See Also • [":CHANnel<n>:BWLimit"](#) on page 330

:CHANnel<n>:BWLimit

C (see [page 1650](#))

Command Syntax :CHANnel<n>:BWLimit <bwlimit>

<bwlimit> ::= {{1 | ON} | {0 | OFF}}

<n> ::= 1 to (# analog channels) in NR1 format

The :CHANnel<n>:BWLimit command controls an internal low-pass filter. When the filter is on, the bandwidth of the specified channel is limited to approximately 25 MHz.

Query Syntax :CHANnel<n>:BWLimit?

The :CHANnel<n>:BWLimit? query returns the current setting of the low-pass filter.

Return Format <bwlimit><NL>

<bwlimit> ::= {1 | 0}

See Also • ["Introduction to :CHANnel<n> Commands"](#) on page 328

:CHANnel<n>:COUpling

C (see [page 1650](#))

Command Syntax :CHANnel<n>:COUpling <coupling>

<coupling> ::= {AC | DC}

<n> ::= 1 to (# analog channels) in NR1 format

The :CHANnel<n>:COUpling command selects the input coupling for the specified channel. The coupling for each analog channel can be set to AC or DC.

Query Syntax :CHANnel<n>:COUpling?

The :CHANnel<n>:COUpling? query returns the current coupling for the specified channel.

Return Format <coupling value><NL>

<coupling value> ::= {AC | DC}

See Also • ["Introduction to :CHANnel<n> Commands"](#) on page 328

:CHANnel<n>:DISPlay

C (see [page 1650](#))

Command Syntax :CHANnel<n>:DISPlay <display value>

<display value> ::= {{1 | ON} | {0 | OFF}}

<n> ::= 1 to (# analog channels) in NR1 format

The :CHANnel<n>:DISPlay command turns the display of the specified channel on or off.

Query Syntax :CHANnel<n>:DISPlay?

The :CHANnel<n>:DISPlay? query returns the current display setting for the specified channel.

Return Format <display value><NL>

<display value> ::= {1 | 0}

- See Also**
- ["Introduction to :CHANnel<n> Commands"](#) on page 328
 - [":VIEW"](#) on page 280
 - [":BLANK"](#) on page 248
 - [":STATus"](#) on page 277
 - [":POD<n>:DISPlay"](#) on page 747
 - [":DIGital<d>:DISPlay"](#) on page 385

:CHANnel<n>:IMPedance

C (see [page 1650](#))

Command Syntax :CHANnel<n>:IMPedance <impedance>

<impedance> ::= {ONEMeg | FIFTy}

<n> ::= 1 to (# analog channels) in NR1 format

The :CHANnel<n>:IMPedance command selects the input impedance setting for the specified analog channel. The legal values for this command are ONEMeg (1 M Ω) and FIFTy (50 Ω).

Query Syntax :CHANnel<n>:IMPedance?

The :CHANnel<n>:IMPedance? query returns the current input impedance setting for the specified channel.

Return Format <impedance value><NL>

<impedance value> ::= {ONEM | FIFT}

See Also • ["Introduction to :CHANnel<n> Commands"](#) on page 328

:CHANnel<n>:INVert

N (see [page 1650](#))

Command Syntax :CHANnel<n>:INVert <invert value>

<invert value> ::= {{1 | ON} | {0 | OFF}}

<n> ::= 1 to (# analog channels) in NR1 format

The :CHANnel<n>:INVert command selects whether or not to invert the input signal for the specified channel. The inversion may be 1 (ON/inverted) or 0 (OFF/not inverted).

Query Syntax :CHANnel<n>:INVert?

The :CHANnel<n>:INVert? query returns the current state of the channel inversion.

Return Format <invert value><NL>

<invert value> ::= {0 | 1}

See Also • ["Introduction to :CHANnel<n> Commands"](#) on page 328

:CHANnel<n>:LABel

N (see [page 1650](#))

Command Syntax :CHANnel<n>:LABel <string>
 <string> ::= quoted ASCII string
 <n> ::= 1 to (# analog channels) in NR1 format

NOTE

Label strings are 32 characters or less, and may contain any commonly used ASCII characters. Labels with more than 32 characters are truncated to 32 characters.

The :CHANnel<n>:LABel command sets the analog channel label to the string that follows. Setting a label for a channel also adds the name to the label list in non-volatile memory (replacing the oldest label in the list).

Query Syntax :CHANnel<n>:LABel?

The :CHANnel<n>:LABel? query returns the label associated with a particular analog channel.

Return Format <string><NL>
 <string> ::= quoted ASCII string

- See Also**
- ["Introduction to :CHANnel<n> Commands"](#) on page 328
 - [":DISPlay:LABel"](#) on page 408
 - [":DIGital<d>:LABel"](#) on page 386
 - [":DISPlay:LABList"](#) on page 409
 - [":BUS<n>:LABel"](#) on page 310

Example Code

```
' LABEL - This command allows you to write a name (32 characters
' maximum) next to the channel number. It is not necessary, but
' can be useful for organizing the display.
myScope.WriteString ":CHANnel1:LABel " "CAL 1" " " ' Label ch1 "CAL 1".
myScope.WriteString ":CHANnel2:LABel " "CAL2" " " ' Label ch1 "CAL2".
```

See complete example programs at: [Chapter 46](#), "Programming Examples," starting on page 1659

:CHANnel<n>:OFFSet

C (see [page 1650](#))

Command Syntax :CHANnel<n>:OFFSet <offset> [<suffix>]

<offset> ::= Vertical offset value in NR3 format

<suffix> ::= {V | mV}

<n> ::= 1 to (# analog channels) in NR1 format

The :CHANnel<n>:OFFSet command sets the value that is represented at center screen for the selected channel. The range of legal values varies with the value set by the :CHANnel<n>:RANGe and :CHANnel<n>:SCALE commands. If you set the offset to a value outside of the legal range, the offset value is automatically set to the nearest legal value. Legal values are affected by the probe attenuation setting.

Query Syntax :CHANnel<n>:OFFSet?

The :CHANnel<n>:OFFSet? query returns the current offset value for the selected channel.

Return Format <offset><NL>

<offset> ::= Vertical offset value in NR3 format

- See Also**
- ["Introduction to :CHANnel<n> Commands"](#) on page 328
 - [":CHANnel<n>:RANGe"](#) on page 353
 - [":CHANnel<n>:SCALE"](#) on page 354
 - [":CHANnel<n>:PROBe"](#) on page 337

:CHANnel<n>:PROBe

C (see [page 1650](#))

Command Syntax :CHANnel<n>:PROBe <attenuation>

<attenuation> ::= probe attenuation ratio in NR3 format

<n> ::= 1 to (# analog channels) in NR1 format

The obsolete attenuation values X1, X10, X20, X100 are also supported.

The :CHANnel<n>:PROBe command specifies the probe attenuation factor for the selected channel.

The probe attenuation factor may be from 0.001 to 10000.

This command does not change the actual input sensitivity of the oscilloscope. It changes the reference constants for scaling the display factors, for making automatic measurements, and for setting trigger levels.

If an AutoProbe probe is connected to the oscilloscope, the attenuation value cannot be changed from the sensed value. Attempting to set the oscilloscope to an attenuation value other than the sensed value produces an error.

Query Syntax :CHANnel<n>:PROBe?

The :CHANnel<n>:PROBe? query returns the current probe attenuation factor for the selected channel.

Return Format <attenuation><NL>

<attenuation> ::= probe attenuation ratio in NR3 format

- See Also**
- ["Introduction to :CHANnel<n> Commands"](#) on page 328
 - [":CHANnel<n>:RANGe"](#) on page 353
 - [":CHANnel<n>:SCALE"](#) on page 354
 - [":CHANnel<n>:OFFSet"](#) on page 336

Example Code

```
' CHANNEL_PROBE - Sets the probe attenuation factor for the
' selected channel. The probe attenuation factor may be set
' from 0.001 to 10000.
myScope.WriteString ":CHANnel1:PROBe 10" ' Set Probe to 10:1.
```

See complete example programs at: [Chapter 46](#), "Programming Examples," starting on page 1659

:CHANnel<n>:PROBe:BTN

N (see [page 1650](#))

Command Syntax :CHANnel<n>:PROBe:BTN <setting>

<setting> ::= {HEADlight | INFiniiMode | RSTop | SINGLE | CDISplay
| AUToscale | FTRigger | QACTion | NACTion | NONE}

<n> ::= 1 to (# analog channels) in NR1 format

For N275xA InfiniiMode probes, the :CHANnel<n>:PROBe:BTN command sets the probe's quick-action button mode:

- HEADlight – Pressing the probe's quick-action button toggles the probe headlight on or off. Holding the quick-action button adjusts the intensity of the probe's headlight.
- INFiniiMode – Cycles through the InfiniiMode settings (see :CHANnel<n>:PROBe:MODE).
- RSTop (Run/Stop) – Toggles between continuous data acquisition (running) and stopping data acquisitions (same as the oscilloscope's **[Run/Stop]** key).
- SINGLE – Performs a single acquisition on the oscilloscope (same as the oscilloscope's **[Single]** key).
- CDISplay – Clears the oscilloscope display.
- AUToscale – Performs an oscilloscope Autoscale operation.
- FTRigger (Force Trigger) – Performs an oscilloscope "force trigger".
- QACTion (Quick Action) – Performs the quick action configured on the oscilloscope (see the oscilloscope's *user's guide* for more information).
- NACTion (No Action) – Disables the probe's quick-action button.
- NONE – When a probe does not have a quick-action button, this is the only valid setting.

Query Syntax :CHANnel<n>:PROBe:BTN?

The :CHANnel<n>:PROBe:BTN? query returns the N275xA InfiniiMode probe's quick-action button setting.

Return Format <setting><NL>

<setting> ::= {HEAD | INF | RST | SING | CDIS | AUT | FTR | QACT
| NACT | NONE}

See Also • [":CHANnel<n>:PROBe:MODE"](#) on page 347

:CHANnel<n>:PROBe:CALibration

N (see [page 1650](#))

Command Syntax :CHANnel<n>:PROBe:CALibration

The :CHANnel<n>:PROBe:CALibration command begins the probe degauss operation. For the N7026A and N2893A probes only.

:CHANnel<n>:PROBe:EXTeRnal

N (see [page 1650](#))

Command Syntax :CHANnel<n>:PROBe:EXTeRnal {{0 | OFF} | {1 | ON}}

<n> ::= 1 to (# analog channels) in NR1 format

The :CHANnel<n>:PROBe:EXTeRnal command enables or disables External Scaling on the analog input channel. External Scaling can apply additional gain to the input channel to account for additional attenuators, adapters, etc., in the probing system.

Query Syntax :CHANnel<n>:PROBe:EXTeRnal?

The :CHANnel<n>:PROBe:EXTeRnal? query returns whether External Scaling is enabled (1) or disabled (0).

Return Format <setting><NL>

<setting> ::= {0 | 1}

- See Also**
- [":CHANnel<n>:PROBe:EXTeRnal:GAIN"](#) on page 341
 - [":CHANnel<n>:PROBe:EXTeRnal:UNITs"](#) on page 343

:CHANnel<n>:PROBe:EXTeRnal:GAIN

N (see [page 1650](#))

Command Syntax :CHANnel<n>:PROBe:EXTeRnal:GAIN <gain_factor>

<gain_factor> ::= a real number from 0.0001 to 1000 in NR3 format

<n> ::= 1 to (# analog channels) in NR1 format

The :CHANnel<n>:PROBe:EXTeRnal:GAIN command sets the gain associated with External Scaling.

Query Syntax :CHANnel<n>:PROBe:EXTeRnal:GAIN?

The :CHANnel<n>:PROBe:EXTeRnal:GAIN? query returns the External Scaling gain setting.

Return Format <gain_factor><NL>

- See Also**
- [":CHANnel<n>:PROBe:EXTeRnal"](#) on page 340
 - [":CHANnel<n>:PROBe:EXTeRnal:UNITs"](#) on page 343

:CHANnel<n>:PROBe:EXternal:OFFSet

N (see [page 1650](#))

Command Syntax :CHANnel<n>:PROBe:EXternal:OFFSet <offset>

<offset> ::= a real number for the offset value at center screen
in NR3 format

<n> ::= 1 to (# analog channels) in NR1 format

The :CHANnel<n>:PROBe:EXternal:OFFSet command sets the vertical offset associated with External Scaling.

Query Syntax :CHANnel<n>:PROBe:EXternal:OFFSet?

The :CHANnel<n>:PROBe:EXternal:OFFSet? query returns the External Scaling vertical offset setting.

Return Format <offset><NL>

- See Also**
- [":CHANnel<n>:PROBe:EXternal"](#) on page 340
 - [":CHANnel<n>:PROBe:EXternal:GAIN"](#) on page 341
 - [":CHANnel<n>:PROBe:EXternal:UNITs"](#) on page 343

:CHANnel<n>:PROBe:EXTeRnal:UNITs

N (see [page 1650](#))

Command Syntax :CHANnel<n>:PROBe:EXTeRnal:UNITs <units>

<units> ::= {VOLT | AMPere}

<n> ::= 1 to (# analog channels) in NR1 format

The :CHANnel<n>:PROBe:EXTeRnal:UNITs command sets the measurement units for the connected probe. Select VOLT for a voltage probe and select AMPere for a current probe. Measurement results, channel sensitivity, and trigger level will reflect the measurement units you select.

Query Syntax :CHANnel<n>:PROBe:EXTeRnal:UNITs?

The :CHANnel<n>:PROBe:EXTeRnal:UNITs? query returns the current units setting for the specified channel.

Return Format <units><NL>

<units> ::= {VOLT | AMP}

- See Also**
- [":CHANnel<n>:PROBe:EXTeRnal"](#) on page 340
 - [":CHANnel<n>:PROBe:EXTeRnal:GAIN"](#) on page 341

:CHANnel<n>:PROBe:HEAD[:TYPE]

C (see [page 1650](#))

Command Syntax

NOTE

This command is valid only for the 113xA Series probes.

```
:CHANnel<n>:PROBe:HEAD[:TYPE] <head_param>

<head_param> ::= {SEND0 | SEND6 | SEND12 | SEND20 | DIFF0 | DIFF6
                  | DIFF12 | DIFF20 | DSMA | DSMA6 | NONE}

<n> ::= {1 | 2 | 3 | 4}
```

The :CHANnel<n>:PROBe:HEAD[:TYPE] command sets an analog channel probe head type and dB value. You can choose from:

- SEND0 – Single-ended, 0 dB.
- SEND6 – Single-ended, 6 dB.
- SEND12 – Single-ended, 12 dB.
- SEND20 – Single-ended, 20 dB.
- DIFF0 – Differential, 0 dB.
- DIFF6 – Differential, 6 dB.
- DIFF12 – Differential, 12 dB.
- DIFF20 – Differential, 20 dB.
- DSMA – Differential SMA probe head, 0 dB.
- DSMA6 – Differential SMA probe head, 6 dB.

Query Syntax :CHANnel<n>:PROBe:HEAD[:TYPE] ?

The :CHANnel<n>:PROBe:HEAD[:TYPE]? query returns the current probe head type setting for the selected channel.

Return Format <head_param><NL>

```
<head_param> ::= {SEND0 | SEND6 | SEND12 | SEND20 | DIFF0 | DIFF6
                  | DIFF12 | DIFF20 | DSMA | DSMA6 | NONE}
```

- See Also**
- ["Introduction to :CHANnel<n> Commands"](#) on page 328
 - [":CHANnel<n>:PROBe"](#) on page 337
 - [":CHANnel<n>:PROBe:ID"](#) on page 345
 - [":CHANnel<n>:PROBe:SKEW"](#) on page 349
 - [":CHANnel<n>:PROBe:SType"](#) on page 350

:CHANnel<n>:PROBe:ID

C (see [page 1650](#))

Query Syntax :CHANnel<n>:PROBe:ID?

<n> ::= 1 to (# analog channels) in NR1 format

The :CHANnel<n>:PROBe:ID? query returns the type of probe attached to the specified oscilloscope channel.

Return Format <probe id><NL>

<probe id> ::= unquoted ASCII string up to 11 characters

Some of the possible returned values are:

- 1131A
- 1132A
- 1134A
- 1147A
- 1153A
- 1154A
- 1156A
- 1157A
- 1158A
- 1159A
- AutoProbe
- E2621A
- E2622A
- E2695A
- E2697A
- HP1152A
- HP1153A
- NONE
- Probe
- Unknown
- Unsupported

See Also • ["Introduction to :CHANnel<n> Commands"](#) on page 328

:CHANnel<n>:PROBe:MMODEl

N (see [page 1650](#))

Command Syntax :CHANnel<n>:PROBe:MMODEl <value>

<value> ::= {P5205 | P5210 | P6205 | P6241 | P6243 | P6245 | P6246
| P6247 | P6248 | P6249 | P6250 | P6251 | P670X | P671X | TCP202}

<n> ::= 1 to (# analog channels) in NR1 format

The :CHANnel<n>:PROBe:MMODEl command sets the model number of the supported Tektronix probe.

Query Syntax :CHANnel<n>:PROBe:MMODEl?

The :CHANnel<n>:PROBe:MMODEl? query returns the model number setting.

Return Format <value><NL>

<value> ::= {P5205 | P5210 | P6205 | P6241 | P6243 | P6245 | P6246
| P6247 | P6248 | P6249 | P6250 | P6251 | P670X | P671X | TCP202}

See Also • [":CHANnel<n>:PROBe:ID"](#) on page 345

:CHANnel<n>:PROBe:MODE

N (see [page 1650](#))

Command Syntax :CHANnel<n>:PROBe:MODE <setting>

<setting> ::= {DIFFerential | DOFFset | SEA | SEB | CM}

<n> ::= 1 to (# analog channels) in NR1 format

For N275xA InfiniiMode probes, the :CHANnel<n>:PROBe:MODE command sets the probe's mode:

- DIFFerential – Differential.
- DOFFset – Differential with Offset.
- SEA – Single-Ended, side A.
- SEB – Single-Ended, side B.
- CM – Common Mode.

Query Syntax :CHANnel<n>:PROBe:MODE?

The :CHANnel<n>:PROBe:MODE? query returns the N275xA InfiniiMode probe's mode setting.

Return Format <setting><NL>

<setting> ::= {DIFF | DOFF | SEA | SEB | CM}

See Also • [":CHANnel<n>:PROBe:BTN"](#) on page 338

:CHANnel<n>:PROBe:RSENse

N (see [page 1650](#))

Command Syntax :CHANnel<n>:PROBe:RSENse <value>

<value> ::= Ohms in NR3 format

<n> ::= 1 to (# analog channels) in NR1 format

When the N2820A high-sensitivity current probe is used with the N2825A user-defined R-sense head, the :CHANnel<n>:PROBe:RSENse command specifies the value of the R-sense resistor that is being probed in the device under test (DUT). Supported R-sense resistor values are from 1 m Ω to 1M Ω .

Query Syntax :CHANnel<n>:PROBe:RSENse?

The :CHANnel<n>:PROBe:RSENse? query returns the R-sense resistor value setting.

When the N2820A high-sensitivity current probe is not attached, the query returns "INF".

When the N2820A high-sensitivity current probe is attached and a special head is attached to the probe, the query returns the value of the special head. For example, if the head is a "20 m Ω " head, the query returns 0.02.

Return Format {<value> | INF}<NL>

<value> ::= Ohms in NR3 format

- See Also**
- [":CHANnel<n>:PROBe:ZOOM"](#) on page 351
 - [":MEASure:DUAL:CHARge"](#) on page 616
 - [":MEASure:DUAL:VAMPlitude"](#) on page 617
 - [":MEASure:DUAL:VAverage"](#) on page 618
 - [":MEASure:DUAL:VBASe"](#) on page 619
 - [":MEASure:DUAL:VPP"](#) on page 620
 - [":MEASure:DUAL:VRMS"](#) on page 621

:CHANnel<n>:PROBe:SKEW

C (see [page 1650](#))

Command Syntax :CHANnel<n>:PROBe:SKEW <skew value>

<skew value> ::= skew time in NR3 format

<skew value> ::= -100 ns to +100 ns

<n> ::= 1 to (# analog channels) in NR1 format

The :CHANnel<n>:PROBe:SKEW command sets the channel-to-channel skew factor for the specified channel. Each analog channel can be adjusted + or -100 ns for a total of 200 ns difference between channels. You can use the oscilloscope's probe skew control to remove cable-delay errors between channels.

Query Syntax :CHANnel<n>:PROBe:SKEW?

The :CHANnel<n>:PROBe:SKEW? query returns the current probe skew setting for the selected channel.

Return Format <skew value><NL>

<skew value> ::= skew value in NR3 format

See Also • ["Introduction to :CHANnel<n> Commands"](#) on page 328

:CHANnel<n>:PROBe:STYPe

C (see [page 1650](#))

Command Syntax

NOTE

This command is valid only for the 113xA Series probes.

```
:CHANnel<n>:PROBe:STYPe <signal type>
<signal type> ::= {DIFFerential | SINGle}
<n> ::= 1 to (# analog channels) in NR1 format
```

The :CHANnel<n>:PROBe:STYPe command sets the channel probe signal type (STYPe) to differential or single-ended when using the 113xA Series probes and determines how offset is applied.

When single-ended is selected, the :CHANnel<n>:OFFSet command changes the offset value of the probe amplifier. When differential is selected, the :CHANnel<n>:OFFSet command changes the offset value of the channel amplifier.

Query Syntax :CHANnel<n>:PROBe:STYPe?

The :CHANnel<n>:PROBe:STYPe? query returns the current probe signal type setting for the selected channel.

Return Format <signal type><NL>
 <signal type> ::= {DIFF | SING}

- See Also**
- ["Introduction to :CHANnel<n> Commands"](#) on page 328
 - [":CHANnel<n>:OFFSet"](#) on page 336

:CHANnel<n>:PROBe:ZOOM

N (see [page 1650](#))

Command Syntax :CHANnel<n>:PROBe:ZOOM {{0 | OFF} | {1 | ON}}

<n> ::= 1 to (# analog channels) in NR1 format

When the N2820A high-sensitivity current probe is used with both the Primary and Secondary cables, the :CHANnel<n>:PROBe:ZOOM command specifies whether this cable will have the Zoom In waveform (ON) or the Zoom Out waveform (OFF). The other cable will have the opposite waveform.

Query Syntax :CHANnel<n>:PROBe:ZOOM?

The :CHANnel<n>:PROBe:ZOOM? query returns the zoom setting.

Return Format <setting><NL>

<setting> ::= {0 | 1}

- See Also**
- [":CHANnel<n>:PROBe:RSENse"](#) on page 348
 - [":MEASure:DUAL:CHARge"](#) on page 616
 - [":MEASure:DUAL:VAMPlitude"](#) on page 617
 - [":MEASure:DUAL:VAverage"](#) on page 618
 - [":MEASure:DUAL:VBASe"](#) on page 619
 - [":MEASure:DUAL:VPP"](#) on page 620
 - [":MEASure:DUAL:VRMS"](#) on page 621

:CHANnel<n>:PROTection

N (see [page 1650](#))

Command Syntax :CHANnel<n>:PROTection[:CLEar]

<n> ::= 1 to (# analog channels) in NR1 format | 4}

When the analog channel input impedance is set to 50 Ω , the input channels are protected against overvoltage. When an overvoltage condition is sensed, the input impedance for the channel is automatically changed to 1 M Ω .

The :CHANnel<n>:PROTection[:CLEar] command is used to clear (reset) the overload protection. It allows the channel to be used again in 50 Ω mode after the signal that caused the overload has been removed from the channel input.

Reset the analog channel input impedance to 50 Ω (see [":CHANnel<n>:IMPedance"](#) on page 333) after clearing the overvoltage protection.

Query Syntax :CHANnel<n>:PROTection?

The :CHANnel<n>:PROTection query returns the state of the input protection for CHANnel<n>. If a channel input has experienced an overload, TRIP (tripped) will be returned; otherwise NORM (normal) is returned.

Return Format {NORM | TRIP}<NL>

- See Also**
- ["Introduction to :CHANnel<n> Commands"](#) on page 328
 - [":CHANnel<n>:COUPling"](#) on page 331
 - [":CHANnel<n>:IMPedance"](#) on page 333
 - [":CHANnel<n>:PROBe"](#) on page 337

:CHANnel<n>:RANGe

C (see [page 1650](#))

Command Syntax :CHANnel<n>:RANGe <range>[<suffix>]

<range> ::= vertical full-scale range value in NR3 format

<suffix> ::= {V | mV}

<n> ::= 1 to (# analog channels) in NR1 format

The :CHANnel<n>:RANGe command defines the full-scale vertical axis of the selected channel. When using 1:1 probe attenuation, legal values for the range are from 8 mV to 40 V.

If the probe attenuation is changed, the range value is multiplied by the probe attenuation factor.

Query Syntax :CHANnel<n>:RANGe?

The :CHANnel<n>:RANGe? query returns the current full-scale range setting for the specified channel.

Return Format <range_argument><NL>

<range_argument> ::= vertical full-scale range value in NR3 format

- See Also**
- ["Introduction to :CHANnel<n> Commands"](#) on page 328
 - [":CHANnel<n>:SCALE"](#) on page 354
 - [":CHANnel<n>:PROBe"](#) on page 337

Example Code

```
' CHANNEL_RANGE - Sets the full scale vertical range in volts. The
' range value is 8 times the volts per division.
myScope.WriteString ":CHANnel1:RANGe 8" ' Set the vertical range to
8 volts.
```

See complete example programs at: [Chapter 46](#), "Programming Examples," starting on page 1659

:CHANnel<n>:SCALE

N (see [page 1650](#))

Command Syntax :CHANnel<n>:SCALE <scale>[<suffix>]

<scale> ::= vertical units per division in NR3 format

<suffix> ::= {V | mV}

<n> ::= 1 to (# analog channels) in NR1 format

The :CHANnel<n>:SCALE command sets the vertical scale, or units per division, of the selected channel.

If the probe attenuation is changed, the scale value is multiplied by the probe's attenuation factor.

Query Syntax :CHANnel<n>:SCALE?

The :CHANnel<n>:SCALE? query returns the current scale setting for the specified channel.

Return Format <scale value><NL>

<scale value> ::= vertical units per division in NR3 format

- See Also**
- ["Introduction to :CHANnel<n> Commands"](#) on page 328
 - [":CHANnel<n>:RANGe"](#) on page 353
 - [":CHANnel<n>:PROBe"](#) on page 337

:CHANnel<n>:UNITs

N (see [page 1650](#))

Command Syntax :CHANnel<n>:UNITs <units>

<units> ::= {VOLT | AMPere}

<n> ::= 1 to (# analog channels) in NR1 format

The :CHANnel<n>:UNITs command sets the measurement units for the connected probe. Select VOLT for a voltage probe and select AMPere for a current probe. Measurement results, channel sensitivity, and trigger level will reflect the measurement units you select.

Query Syntax :CHANnel<n>:UNITs?

The :CHANnel<n>:UNITs? query returns the current units setting for the specified channel.

Return Format <units><NL>

<units> ::= {VOLT | AMP}

- See Also**
- ["Introduction to :CHANnel<n> Commands"](#) on page 328
 - [":CHANnel<n>:RANGe"](#) on page 353
 - [":CHANnel<n>:PROBe"](#) on page 337
 - [":EXternal:UNITs"](#) on page 427

:CHANnel<n>:VERNier

N (see [page 1650](#))

Command Syntax :CHANnel<n>:VERNier <vernier value>

<vernier value> ::= {{1 | ON} | {0 | OFF}}

<n> ::= 1 to (# analog channels) in NR1 format

The :CHANnel<n>:VERNier command specifies whether the channel's vernier (fine vertical adjustment) setting is ON (1) or OFF (0).

Query Syntax :CHANnel<n>:VERNier?

The :CHANnel<n>:VERNier? query returns the current state of the channel's vernier setting.

Return Format <vernier value><NL>

<vernier value> ::= {0 | 1}

See Also • ["Introduction to :CHANnel<n> Commands"](#) on page 328

12 :COMPLIance Commands

Control the license-enabled USB 2.0 signal quality analysis feature.

Table 99 :COMPLIance Commands Summary

Command	Query	Options and Query Returns
:COMPLIance:USB:AUTOSetup (see page 358)	n/a	n/a
:COMPLIance:USB:HUBS<number> (see page 359)	:COMPLIance:USB:HUBS? (see page 359)	<number> ::= 0-5 in NR1 format
:COMPLIance:USB:RUN (see page 360)	n/a	n/a
:COMPLIance:USB:SOURCE:ADJacent <source> (see page 361)	:COMPLIance:USB:SOURCE:ADJacent? (see page 361)	<source> ::= {CHANnel<n>}
:COMPLIance:USB:SOURCE:DIFFerential <source> (see page 362)	:COMPLIance:USB:SOURCE:DIFFerential? (see page 362)	<source> ::= {CHANnel<n>}
:COMPLIance:USB:SOURCE:DMINus <source> (see page 363)	:COMPLIance:USB:SOURCE:DMINus? (see page 363)	<source> ::= {CHANnel<n>}
:COMPLIance:USB:SOURCE:DPLUS <source> (see page 364)	:COMPLIance:USB:SOURCE:DPLUS? (see page 364)	<source> ::= {CHANnel<n>}
:COMPLIance:USB:TEST<test> (see page 365)	:COMPLIance:USB:TEST? (see page 365)	<test> ::= {DHSS HHSS DLSS HLSS DFSS HFSS}
:COMPLIance:USB:TEST:CONNECTION <connection> (see page 366)	:COMPLIance:USB:TEST:CONNECTION? (see page 366)	<connection> ::= {SINGLEended DIFFerential}
:COMPLIance:USB:TEST:TYPE <type> (see page 367)	:COMPLIance:USB:TEST:TYPE? (see page 367)	<type> ::= {NEARend FARend}

:COMPLIance:USB:AUTOsetup

N (see [page 1650](#))

Command Syntax :COMPLIance:USB:AUTOsetup

The :COMPLIance:USB:AUTOsetup command automatically sets up the oscilloscope for the selected signal quality test and USB compliance test packets. Automatically set are:

- Horizontal scale and delay.
- Analog input channel(s) scale and vertical offset.
- Trigger mode and trigger level.

The :COMPLIance:USB:AUTOsetup command causes the oscilloscope to run (that is, capture acquisitions that meet the trigger condition).

If you are testing the signal quality of real-world packets (instead of USB compliance test packets), you may need to adjust the auto setup.

Note that gating of measurements can only be done via the oscilloscope's horizontal controls. This can be done after the :COMPLIance:USB:AUTOsetup command and before the :COMPLIance:USB:RUN command. The horizontal controls should be adjusted so that only the data to be tested is on screen when the :COMPLIance:USB:RUN command is executed. This is normally one packet of data.

- See Also**
- [":COMPLIance:USB:TEST"](#) on page 365
 - [":COMPLIance:USB:TEST:TYPE"](#) on page 367
 - [":COMPLIance:USB:TEST:CONNECTION"](#) on page 366
 - [":COMPLIance:USB:RUN"](#) on page 360

:COMpliance:USB:HUBS

N (see [page 1650](#))

Command Syntax :COMpliance:USB:HUBS <number>

<number> ::= 0-5 in NR1 format

The :COMpliance:USB:HUBS command specifies the number of internal hubs between the host and the test point.

When the Near-end test type is selected, you can specify 0 to 5 hubs.

When the Far-end test type is selected, the number of hubs is set to 0.

The specified number of hubs affects the test limits for the Sync results and the EOP Width results.

Query Syntax :COMpliance:USB:HUBS?

The :COMpliance:USB:HUBS? query returns the specified the number of internal hubs.

Return Format <number><NL>

<number> ::= 0-5 in NR1 format

- See Also**
- [":COMpliance:USB:TEST"](#) on page 365
 - [":COMpliance:USB:TEST:TYPE"](#) on page 367
 - [":COMpliance:USB:SOURce:DPLus"](#) on page 364
 - [":COMpliance:USB:SOURce:DMINus"](#) on page 363
 - [":COMpliance:USB:SOURce:ADJacent"](#) on page 361
 - [":COMpliance:USB:SOURce:DIFFerential"](#) on page 362

:COMpliance:USB:RUN

N (see [page 1650](#))

Command Syntax :COMpliance:USB:RUN

The :COMpliance:USB:RUN command runs the selected signal quality test.

Please be patient as tests can take several minutes to complete.

When tests are run, the oscilloscope stops acquisitions if they are running, analyzes the data on screen, and then displays the results. The analyzed acquisition remains on screen and can be viewed by moving or dismissing the results dialog.

You can run tests when the oscilloscope is already stopped. This is useful in the case of embedded hosts (and other cases) where the device under test (DUT) may not easily be placed in a test mode, and you want to analyze already acquired data.

- See Also
- [":COMpliance:USB:TEST"](#) on page 365
 - [":COMpliance:USB:AUTosetup"](#) on page 358
 - ["*OPC \(Operation Complete\)"](#) on page 222
 - [":SAVE:COMpliance:USB\[:START\]"](#) on page 872

:COMPLIance:USB:SOURce:ADJacent

N (see [page 1650](#))

Command Syntax :COMPLIance:USB:SOURce:ADJacent <source>

<source> ::= {CHANnel<n>}

<n> ::= 1 to (# analog channels) in NR1 format

The :COMPLIance:USB:SOURce:ADJacent command specifies the analog input channel that is probing the adjacent D+ or D- signal.

When the Device Full Speed Signal Quality test is selected, the specified channel probes the adjacent D+ signal.

When the Device Low Speed Signal Quality test is selected, this specified channel probes the adjacent D- signal.

Query Syntax :COMPLIance:USB:SOURce:ADJacent?

The :COMPLIance:USB:SOURce:ADJacent? query returns the specified analog input channel.

Return Format <source><NL>

<source> ::= {CHAN<n>}

- See Also**
- [":COMPLIance:USB:SOURce:DPLus"](#) on page 364
 - [":COMPLIance:USB:SOURce:DMINus"](#) on page 363

:COMPLIance:USB:SOURce:DIFFerential

N (see [page 1650](#))

Command Syntax :COMPLIance:USB:SOURce:DIFFerential <source>

<source> ::= {CHANnel<n>}

<n> ::= 1 to (# analog channels) in NR1 format

The :COMPLIance:USB:SOURce:DIFFerential command specifies the analog input channel whose differential probe is connected to the Hi-Speed signal to be tested.

Query Syntax :COMPLIance:USB:SOURce:DIFFerential?

The :COMPLIance:USB:SOURce:DIFFerential? query returns the specified analog input channel.

Return Format <source><NL>

<source> ::= {CHAN<n>}

- See Also**
- [":COMPLIance:USB:TEST"](#) on page 365
 - [":COMPLIance:USB:TEST:CONNection"](#) on page 366

:COMpliance:USB:SOURce:DMINus

N (see [page 1650](#))

Command Syntax :COMpliance:USB:SOURce:DMINus <source>

<source> ::= {CHANnel<n>}

<n> ::= 1 to (# analog channels) in NR1 format

The :COMpliance:USB:SOURce:DMINus command specifies the analog input channel that is probing the D- signal.

On 4-channel oscilloscopes, you are forced to use different channel pairs for the D+ and D- signals. This provides the maximum sample rate. (Channels 1 and 2 are one pair and channels 3 and 4 are the other pair.)

Query Syntax :COMpliance:USB:SOURce:DMINus?

The :COMpliance:USB:SOURce:DMINus? query returns the specified analog input channel.

Return Format <source><NL>

<source> ::= {CHAN<n>}

- See Also**
- [":COMpliance:USB:SOURce:DPLus"](#) on page 364
 - [":COMpliance:USB:SOURce:ADJacent"](#) on page 361

:COMPLIance:USB:SOURce:DPLus

N (see [page 1650](#))

Command Syntax :COMPLIance:USB:SOURce:DPLus <source>

<source> ::= {CHANnel<n>}

<n> ::= 1 to (# analog channels) in NR1 format

The :COMPLIance:USB:SOURce:DPLus command specifies the analog input channel that is probing the D+ signal.

On 4-channel oscilloscopes, you are forced to use different channel pairs for the D+ and D- signals. This provides the maximum sample rate. (Channels 1 and 2 are one pair and channels 3 and 4 are the other pair.)

Query Syntax :COMPLIance:USB:SOURce:DPLus?

The :COMPLIance:USB:SOURce:DPLus? query returns the specified analog input channel.

Return Format <source><NL>

<source> ::= {CHAN<n>}

- See Also**
- [":COMPLIance:USB:SOURce:DMINus"](#) on page 363
 - [":COMPLIance:USB:SOURce:ADJacent"](#) on page 361

:COMpliance:USB:TEST

N (see [page 1650](#))

Command Syntax :COMpliance:USB:TEST <test>

<test> ::= {DHSS | HHSS | DLSS | HLSS | DFSS | HFSS}

The :COMpliance:USB:TEST command selects the type of signal quality test to perform:

- DHSS – Device Hi-Speed Signal Quality.
- HHSS – Host Hi-Speed Signal Quality.
- DLSS – Device Low Speed Signal Quality.
- HLSS – Host Low Speed Signal Quality.
- DFSS – Device Full Speed Signal Quality.
- HFSS – Host Full Speed Signal Quality.

For Hub upstream testing, select the Device tests.

For Hub downstream testing, select the Host tests.

Low and Full Speed tests require two or three single-ended probes.

Hi-Speed tests are available only on 1.5 GHz bandwidth oscilloscopes, and a differential probe or SMA cable connections are required.

Note that the N2750A Series InfiniiMode differential probes cannot be used with the Hi-Speed Signal Quality test fixtures because they have damping resistors and the USB test fixture also has damping resistors specifically designed for the E2678A socketed probe heads.

Query Syntax :COMpliance:USB:TEST?

The :COMpliance:USB:TEST? query returns the selected signal quality test type.

Return Format <test><NL>

<test> ::= {DHSS | HHSS | DLSS | HLSS | DFSS | HFSS}

- See Also**
- [":COMpliance:USB:TEST:TYPE"](#) on page 367
 - [":COMpliance:USB:SOURce:DPLus"](#) on page 364
 - [":COMpliance:USB:SOURce:DMINus"](#) on page 363
 - [":COMpliance:USB:SOURce:ADJacent"](#) on page 361
 - [":COMpliance:USB:SOURce:DIFFerential"](#) on page 362

:COMPLIance:USB:TEST:CONNection

N (see [page 1650](#))

Command Syntax :COMPLIance:USB:TEST:CONNection <connection>

<connection> ::= {SINGleended | DIFFerential}

When a Hi-Speed test has been selected, the :COMPLIance:USB:TEST:CONNection command specifies the test fixture connection type:

- DIFFerential – specifies that a differential probe is used to probe the signal under test.

In this case, use the :COMPLIance:USB:SOURce:DIFFerential command to specify the analog input channel connected to the differential probe.

- SINGleended – specifies that two single-ended SMA cables are used to probe the signal under test.

In this case, use the :COMPLIance:USB:SOURce:DPLus command to specify the analog input channel that is probing the D+ signal, and use the :COMPLIance:USB:SOURce:DMINus command to specify the analog input channel that is probing the D- signal.

Query Syntax :COMPLIance:USB:TEST:CONNection?

The :COMPLIance:USB:TEST:CONNection? query returns the specified the test fixture connection type.

Return Format <connection><NL>

<connection> ::= {SING | DIFF}

- See Also**
- [":COMPLIance:USB:SOURce:DIFFerential"](#) on page 362
 - [":COMPLIance:USB:SOURce:DPLus"](#) on page 364
 - [":COMPLIance:USB:SOURce:DMINus"](#) on page 363

:COMPLIance:USB:TEST:TYPE

N (see [page 1650](#))

Command Syntax :COMPLIance:USB:TEST:TYPE <type>

<type> ::= {NEARend | FARend}

When a Hi-Speed test has been selected, the :COMPLIance:USB:TEST:TYPE command selects whether the test type is near-end or far-end.

Query Syntax :COMPLIance:USB:TEST:TYPE?

The :COMPLIance:USB:TEST:TYPE? query returns the selected test type.

Return Format <type><NL>

<type> ::= {NEAR | FAR}

See Also • [":COMPLIance:USB:TEST"](#) on page 365

13 :COUNter Commands

These commands control the counter feature. See ["Introduction to :COUNTER Commands"](#) on page 369.

Table 100 :COUNter Commands Summary

Command	Query	Options and Query Returns
n/a	:COUNter:CURRent? (see page 370)	<value> ::= current counter value in NR3 format
:COUNter:ENABle {{0 OFF} {1 ON}} (see page 371)	:COUNter:ENABle? (see page 371)	{0 1}
:COUNter:MODE <mode> (see page 372)	:COUNter:MODE (see page 372)	<mode> ::= {FREQuency PERiod}
:COUNter:NDIGits <value> (see page 373)	:COUNter:NDIGits (see page 373)	<value> ::= 5 normally, 8 with 10 MHz reference signal, in NR1 format
:COUNter:SOURce <source> (see page 374)	:COUNter:SOURce? (see page 374)	<source> ::= {CHANnel<n> TQEvent} <n> ::= 1 to (# analog channels) in NR1 format

Introduction to :COUNter Commands The :COUNter subsystem provides commands to control the counter feature.
Reporting the Setup

Use :COUNter? to query setup information for the COUNter subsystem.

Return Format

The following is a sample response from the :COUNter? query. In this case, the query was issued following the *RST command.

```
:COUN:ENAB 0;SOUR CHAN1;MODE FREQ;NDIG 5
```

:COUNter:CURRent

N (see [page 1650](#))

Query Syntax :COUNter:CURRent?

The :COUNter:CURRent? query returns the current counter value.

Return Format <value><NL>

<value> ::= current counter value in NR3 format

- See Also**
- [":COUNter:ENABle"](#) on page 371
 - [":COUNter:MODE"](#) on page 372
 - [":COUNter:NDIGits"](#) on page 373
 - [":COUNter:SOURce"](#) on page 374

:COUNter:ENABle

N (see [page 1650](#))

Command Syntax :COUNter:ENABle {{0 | OFF} | {1 | ON}}

The :COUNter:ENABle command enables or disables the counter feature.

Query Syntax :COUNter:ENABle?

The :COUNter:ENABle? query returns whether the counter is enabled or disabled.

Return Format <off_on><NL>
{0 | 1}

- See Also**
- [":COUNter:CURRent"](#) on page 370
 - [":COUNter:MODE"](#) on page 372
 - [":COUNter:NDIGits"](#) on page 373
 - [":COUNter:SOURce"](#) on page 374

:COUNTER:MODE

N (see [page 1650](#))

Command Syntax :COUNTER:MODE <mode>

<mode> ::= {FREQUENCY | PERIOD}

The :COUNTER:MODE command sets the counter mode:

- FREQUENCY – the cycles per second (Hz) of the signal.
- PERIOD – the time periods of the signal's cycles.

Query Syntax :COUNTER:MODE?

The :COUNTER:MODE? query returns the counter mode setting.

Return Format <mode><NL>

<mode> ::= {FREQ | PER}

- See Also**
- [":COUNTER:CURRENt"](#) on page 370
 - [":COUNTER:ENABLE"](#) on page 371
 - [":COUNTER:NDIGits"](#) on page 373
 - [":COUNTER:SOURce"](#) on page 374

:COUNter:NDIGits

N (see [page 1650](#))

Command Syntax :COUNter:NDIGits <value>

<value> ::= 5 normally, 8 with 10 MHz reference signal, in NR1 format

The :COUNter:NDIGits command sets the number of digits of resolution used for the frequency or period counter.

The number of digits is not adjustable and is normally 5 digits. However, the resolution can be increased to 8 digits by inputting an external 10 MHz reference signal to the oscilloscope (on the rear panel 10MHz REF connector).

Higher resolutions require longer gate times, which cause the measurement times to be longer as well.

Query Syntax :COUNter:NDIGits?

The :COUNter:NDIGits? query returns the currently set number of digits of resolution.

Return Format <value><NL>

<value> ::= 5 or 8 in NR1 format

- See Also**
- [":COUNter:CURRent"](#) on page 370
 - [":COUNter:ENABle"](#) on page 371
 - [":COUNter:MODE"](#) on page 372
 - [":COUNter:SOURce"](#) on page 374

:COUNTER:SOURce

N (see [page 1650](#))

Command Syntax :COUNTER:SOURce <source>

<source> ::= {CHANnel<n> | TQEvent}

<n> ::= 1 to (# analog channels) in NR1 format

The :COUNTER:SOURce command selects the waveform source that the counter measures. You can select one of the analog input channels or the trigger qualified event signal.

With the TQEvent (trigger qualified event) source (available when the trigger mode is not EDGE), you can see how often trigger events are detected. This can be more often than when triggers actually occur, due to the oscilloscope's acquisition time or update rate capabilities. The TRIG OUT signal shows when triggers actually occur. Remember that the oscilloscope's trigger circuitry does not re-arm until the holdoff time occurs (:TRIGger:HOLDoff) and that the minimum holdoff time is 40 ns; therefore, the maximum trigger qualified event frequency that can be counted is 25 MHz.

Query Syntax :COUNTER:SOURce?

The :COUNTER:SOURce? query returns the currently set counter source channel.

Return Format <source><NL>

- See Also**
- [":COUNTER:CURRENT"](#) on page 370
 - [":COUNTER:ENABLE"](#) on page 371
 - [":COUNTER:MODE"](#) on page 372
 - [":COUNTER:NDIGits"](#) on page 373

14 :DEMO Commands

You can output demonstration signals on the oscilloscope's Demo 1 and Demo 2 terminals. See "[Introduction to :DEMO Commands](#)" on page 375.

Table 101 :DEMO Commands Summary

Command	Query	Options and Query Returns
:DEMO:FUNCTION <signal> (see page 376)	:DEMO:FUNCTION? (see page 379)	<signal> ::= {SINusoid NOISy PHASe RINGing SINGLE AM CLK GLITCh BURSt MSO RUNT TRANSition RFBurst SHOLd LFSine FMBurst ETE CAN LIN UART I2C SPI I2S CANLin CXPI ARINc FLEXray MANChester MIL MIL2 USB NMONotonic DCMotor HARMonics COUPling CFD SENT USBPd KEYSight}
:DEMO:FUNCTION:PHASe: PHASe <angle> (see page 381)	:DEMO:FUNCTION:PHASe: PHASe? (see page 381)	<angle> ::= angle in degrees from 0 to 360 in NR3 format
:DEMO:OUTPut {{0 OFF} {1 ON}} (see page 382)	:DEMO:OUTPut? (see page 382)	{0 1}

Introduction to :DEMO Commands The :DEMO subsystem provides commands to output demonstration signals on the oscilloscope's Demo 1 and Demo 2 terminals.

Reporting the Setup

Use :DEMO? to query setup information for the DEMO subsystem.

Return Format

The following is a sample response from the :DEMO? query. In this case, the query was issued following the *RST command.

```
:DEMO:FUNC SIN;OUTP 0
```

:DEMO:FUNCTION

N (see [page 1650](#))

Command Syntax :DEMO:FUNCTION <signal>

```
<signal> ::= {SINusoid | NOISy | PHASe | RINGing | SINGLE | AM | CLK
| GLITCh | BURSt | MSO | RUNT | TRANSition | RFBurst | SHOLd
| LFSine | FMBurst | ETE | CAN | LIN | UART | I2C | SPI | I2S
| CANLin | CXPI | ARINc | FLEXray | MANChester | MIL | MIL2 | USB
| NMONotonic | DCMotor | HARMonics | COUPling | CFD | SENT
| USBPd | KEYSight}
```

The :DEMO:FUNCTION command selects the type of demo signal:

Demo Signal Function	Demo 1 Terminal	Demo 2 Terminal
SINusoid	5 MHz sine wave @ ~ 6 Vpp, 0 V offset	Off
NOISy	1 kHz sine wave @ ~ 2.4 Vpp, 0.0 V offset, with ~ 0.5 Vpp of random noise added	Off
PHASe	1 kHz sine wave @ 2.4 Vpp, 0.0 V offset	1 kHz sine wave @ 2.4 Vpp, 0.0 V offset , phase shifted by the amount entered using the ":DEMO:FUNCTION:PHASe:PHASe" on page 381 command
RINGing	500 kHz digital pulse @ ~ 3 Vpp, 1.5 V offset, and ~500 ns pulse width with ringing	Off
SINGLE	~500 ns wide digital pulse with ringing @ ~ 3 Vpp, 1.5 V offset Press the front panel Set Off Single-Shot softkey to cause the selected single-shot signal to be output.	Off
AM	26 kHz sine wave, ~ 7 Vpp, 0 V offset	Amplitude modulated signal, ~ 3 Vpp, 0 V offset, with ~13 MHz carrier and sine envelope
CLK	3.6 MHz clock @ ~2 Vpp, 1 V offset, with infrequent glitch (1 glitch per 1,000,000 clocks)	Off
GLITCh	Burst of 6 digital pulses (plus infrequent glitch) that occurs once every 80 μ s @ ~3.6 Vpp, ~1.8 V offset	Off

Demo Signal Function	Demo 1 Terminal	Demo 2 Terminal
BURSt	Burst of digital pulses that occur every 50 μ s @ ~ 3.6 Vpp, ~1.5 V offset	Off
MSO	3.1 kHz stair-step sine wave output of DAC @ ~1.5 Vpp, 0.75 V offset DAC input signals are internally routed to digital channels D0 through D7	~3.1 kHz sine wave filtered from DAC output @ ~ 600 mVpp, 300 mV offset
RUNT	Digital pulse train with positive and negative runt pulses @ ~ 2.9 Vpp, 1.5 V offset	Off
TRANSition	Digital pulse train with two different edge speeds @ ~ 3.5 Vpp, 1.75 V offset	Off
RFBurst	5-cycle burst of a 10 MHz amplitude modulated sine wave @ ~ 2.6 Vpp, 0 V offset occurring once every 4 ms	Off
SHOLd	6.25 MHz digital clock @ ~ 3.5 Vpp, 1.75 V offset	Data signal @ ~3.5 Vpp, 1.75 V offset
LFSine	30 Hz sine wave @ ~2.7 Vpp, 0 V offset, with very narrow glitch near each positive peak	Off
FMBurst	FM burst, modulated from ~100 kHz to ~1 MHz, ~5.0 Vpp, ~600 mV offset.	Off
ETE	100 kHz pulse, 400 ns wide @ ~3.3 Vpp, 1.65 V offset	600 ns analog burst (@ ~3.3 Vpp, 0.7 V offset) followed by 3.6 μ s digital burst @ ~3.3 Vpp, 1.65 V offset) at a 100 kHz repetitive rate
CAN	CAN_L, 125 kbps dominant-low, ~2.8 Vpp, ~1.4 V offset	Off
LIN	LIN, 19.2 kbs, ~2.8 Vpp, ~1.4 V offset	Off
UART	Receive data (RX) with odd parity, 19.2 kbps, 8-bit words, LSB out 1st, low idle @ ~2.8 Vpp, 1.4 V offset	Transmit data (TX) with odd parity, 19.2 kbps, 8-bit words, LSB out 1st, low idle @ ~ 2.8 Vpp, 1.4 V offset
I2C	I2C serial clock signal (SCL) @ ~2.8 Vpp, 1.4 V offset	I2C serial data signal (SDA) @ ~ 2.8 Vpp, 1.4 V offset

Demo Signal Function	Demo 1 Terminal	Demo 2 Terminal
SPI	Off Signals are internally routed to digital channels D6 through D9: ▪ D9 – MOSI, TTL level, with MSB out 1st (internally routed to digital input). ▪ D8 – MISO, TTL level, with MSB out 1st (internally routed to digital input). ▪ D7 – CLK, TTL level (internally routed to digital input). ▪ D6 – ~CS, low-enable, TTL level (internally routed to digital input).	Off
I2S	Off Signals are internally routed to digital channels D7 through D9: ▪ D9 – SDATA, TTL level, with "standard" alignment (internally routed to digital input). ▪ D8 – SCLK, TTL level, (internally routed to digital input). ▪ D7 – WS, TTL level, low for left channel, high for right channel (internally routed to digital input)	Off
CANLin	CAN_L, 250 kbps dominant-low, ~2.8 Vpp, ~1.4 V offset	LIN, 19.2 kbps, ~2.8 Vpp, ~1.4 V offset
CXPI	CXPI, ~2.5 Vpp, ~1.25 V offset, frame IDs: 20, 21, and 29 (with a frame error).	Off
ARINC	ARINC 429, 100 kbps, ~5 Vpp, ~0 V offset.	Off
FLEXray	FlexRay @ 10 Mbps, ~2.8 Vpp, ~0 V offset	Off
MANchester	Manchester/NRZ @ 125 kbps, ~2.5 Vpp, 1.25 V offset, a 125 kb/s 10-bit PSII-like signal	Off
MIL	MIL-STD-1553 RT to RT transfer, received ~1.3 Vpp, transmitted ~4.8 Vpp, 0 V offset	Off

Demo Signal Function	Demo 1 Terminal	Demo 2 Terminal
MIL2	MIL-STD-1553 RT to RT transfer, received ~1.3 Vpp, transmitted ~4.8 Vpp, 0 V offset	MIL-STD-1553 RT to BC transfer, received ~1.3 Vpp, transmitted ~4.8 Vpp, 0 V offset
USB	USB Low Speed D+ signal, ~2.8 Vpp, 1.4 V offset	USB Low Speed D- signal, ~2.8 Vpp, 1.4 V offset
NMONotonic	Digital pulse train with infrequent non-monotonic rising edges @ ~2.85 Vpp, 1.42 V offset	Off
DCMotor	Output of DAC controlling a DC motor: 800 mV pulse, 1 μ s wide, every 10 μ s, runt pulse every 100 ms.	Off
HARMonics	1 kHz sine wave @ ~3.5 Vpp, 0.0 V offset, with a ~2 kHz sine wave coupled in	Off
COUPling	1 kHz square wave @ ~1 Vpp, 0.0 V offset, with a ~90 kHz sine wave with ~180 mVpp riding on top	Off
CFD	CAN FD, ~2.4 Vpp, ~1.2 V offset, 500 kb/s standard baud rate (sample point at 80%), 10 Mb/s FD baud rate (sample point at 50%)	Off
SENT	SENT, ~2.7 Vpp, ~1.35 V offset, 3 μ s clock period, 6 nibbles in a Fast Channel Message, Slow Channel Messages in enhanced format, idle state high, with pause pulse	Off
USBPd	USB PD, ~2.5 Vpp, ~1.25 V offset, with vendor-defined data message packet and Good CRC control message packet	Off
KEYSight	Series of positive and negative pulses, 125 μ s wide, amplitude modulated, ~2.6 Vpp, ~0 V offset	Off

Query Syntax :DEMO:FUNCTION?

The :DEMO:FUNCTION? query returns the currently selected demo signal type.

Return Format <signal><NL>

```
<signal> ::= {SIN | NOIS | PHAS | RING | SING | AM | CLK | GLIT | BURS
             | MSO | RUNT | TRAN | RFB | SHOL | LFS | FMB | ETE | CAN
             | LIN | UART | I2C | SPI | I2S | CANL | CXPI | ARIN | FLEX | MANC}
```

```
| MIL | MIL2 | USB | NMON | DCM | HARM | COUP | CFD | SENT | USBP  
| KEYS }
```

See Also · ["Introduction to :DEMO Commands"](#) on page 375

:DEMO:FUNCTION:PHASe:PHASe

N (see [page 1650](#))

Command Syntax :DEMO:FUNCTION:PHASe:PHASe <angle>

<angle> ::= angle in degrees from 0 to 360 in NR3 format

For the phase shifted sine demo signals, the :DEMO:FUNCTION:PHASe:PHASe command specifies the phase shift in the second sine waveform.

Query Syntax :DEMO:FUNCTION:PHASe:PHASe?

The :DEMO:FUNCTION:PHASe:PHASe? query returns the currently set phase shift.

Return Format <angle><NL>

<angle> ::= angle in degrees from 0 to 360 in NR3 format

- See Also**
- ["Introduction to :DEMO Commands"](#) on page 375
 - [":DEMO:FUNCTION"](#) on page 376

:DEMO:OUTPut

N (see [page 1650](#))

Command Syntax :DEMO:OUTPut <on_off>

<on_off> ::= {{1 | ON} | {0 | OFF}}

The :DEMO:OUTPut command specifies whether the demo signal output is ON (1) or OFF (0).

Query Syntax :DEMO:OUTPut?

The :DEMO:OUTPut? query returns the current state of the demo signal output setting.

Return Format <on_off><NL>

<on_off> ::= {1 | 0}

- See Also**
- ["Introduction to :DEMO Commands"](#) on page 375
 - [":DEMO:FUNCTION"](#) on page 376

15 :DIGital<d> Commands

Control all oscilloscope functions associated with individual digital channels. See ["Introduction to :DIGital<d> Commands"](#) on page 384.

Table 102 :DIGital<d> Commands Summary

Command	Query	Options and Query Returns
:DIGital<d>:DISPlay { {0 OFF} {1 ON} } (see page 385)	:DIGital<d>:DISPlay? (see page 385)	<d> ::= 0 to (# digital channels - 1) in NR1 format {0 1}
:DIGital<d>:LABel <string> (see page 386)	:DIGital<d>:LABel? (see page 386)	<d> ::= 0 to (# digital channels - 1) in NR1 format <string> ::= any series of 10 or less ASCII characters enclosed in quotation marks
:DIGital<d>:POSition <position> (see page 387)	:DIGital<d>:POSition? (see page 387)	<d> ::= 0 to (# digital channels - 1) in NR1 format <position> ::= 0-7 if display size = large, 0-15 if size = medium, 0-31 if size = small Returns -1 when there is no space to display the digital waveform.
:DIGital<d>:SIZE <value> (see page 388)	:DIGital<d>:SIZE? (see page 388)	<d> ::= 0 to (# digital channels - 1) in NR1 format <value> ::= {SMALL MEDIUM LARGE}
:DIGital<d>:THReshold <value>[suffix] (see page 389)	:DIGital<d>:THReshold ? (see page 389)	<d> ::= 0 to (# digital channels - 1) in NR1 format <value> ::= {CMOS ECL TTL <user defined value>} <user defined value> ::= value in NR3 format from -8.00 to +8.00 [suffix] ::= {V mV uV}

**Introduction to
:DIGital<d>
Commands**

<d> ::= 0 to (# digital channels - 1) in NR1 format

The DIGital subsystem commands control the viewing, labeling, and positioning of digital channels. They also control threshold settings for groups of digital channels, or *pods*.

NOTE

These commands are only valid for the MSO models.

Reporting the Setup

Use :DIGital<d>? to query setup information for the DIGital subsystem.

Return Format

The following is a sample response from the :DIGital0? query. In this case, the query was issued following a *RST command.

```
:DIG0:DISP 0;THR +1.40E+00;LAB 'D0';POS +0
```

:DIGital<d>:DISPlay

N (see [page 1650](#))

Command Syntax :DIGital<d>:DISPlay <display>

<d> ::= 0 to (# digital channels - 1) in NR1 format

<display> ::= {{1 | ON} | {0 | OFF}}

The :DIGital<d>:DISPlay command turns digital display on or off for the specified channel.

NOTE

This command is only valid for the MSO models.

Query Syntax :DIGital<d>:DISPlay?

The :DIGital<d>:DISPlay? query returns the current digital display setting for the specified channel.

Return Format <display><NL>

<display> ::= {0 | 1}

- See Also**
- ["Introduction to :DIGital<d> Commands"](#) on page 384
 - [":POD<n>:DISPlay"](#) on page 747
 - [":CHANnel<n>:DISPlay"](#) on page 332
 - [":VIEW"](#) on page 280
 - [":BLANK"](#) on page 248
 - [":STATus"](#) on page 277

:DIGital<d>:LABel

N (see [page 1650](#))

Command Syntax :DIGital<d>:LABel <string>

<d> ::= 0 to (# digital channels - 1) in NR1 format

<string> ::= any series of 10 or less characters as quoted ASCII string.

The :DIGital<d>:LABel command sets the channel label to the string that follows. Setting a label for a channel also adds the name to the label list in non-volatile memory (replacing the oldest label in the list).

NOTE

This command is only valid for the MSO models.

NOTE

Label strings are 10 characters or less, and may contain any commonly used ASCII characters. Labels with more than 10 characters are truncated to 10 characters.

Query Syntax :DIGital<d>:LABel?

The :DIGital<d>:LABel? query returns the name of the specified channel.

Return Format <label string><NL>

<label string> ::= any series of 10 or less characters as a quoted ASCII string.

- See Also**
- ["Introduction to :DIGital<d> Commands"](#) on page 384
 - [":CHANnel<n>:LABel"](#) on page 335
 - [":DISPlay:LABList"](#) on page 409
 - [":BUS<n>:LABel"](#) on page 310

:DIGital<d>:POSition

N (see [page 1650](#))

Command Syntax :DIGital<d>:POSition <position>
 <d> ::= 0 to (# digital channels - 1) in NR1 format
 <position> ::= integer in NR1 format.

Channel Size	Position	Top	Bottom
Large	0-7	7	0
Medium	0-15	15	0
Small	0-31	31	0

The :DIGital<d>:POSition command sets the position of the specified channel. Note that bottom positions might not be valid depending on whether digital buses, serial decode waveforms, or the zoomed time base are displayed.

NOTE

This command is only valid for the MSO models.

Query Syntax :DIGital<d>:POSition?

The :DIGital<d>:POSition? query returns the position of the specified channel.

If the returned value is "-1", this indicates there is no space to display the digital waveform (for example, when all serial lanes, digital buses, and the zoomed time base are displayed).

Return Format <position><NL>
 <position> ::= integer in NR1 format.

See Also • ["Introduction to :DIGital<d> Commands"](#) on page 384

:DIGital<d>:SIZE

N (see [page 1650](#))

Command Syntax :DIGital<d>:SIZE <value>

<d> ::= 0 to (# digital channels - 1) in NR1 format

<value> ::= {SMALl | MEDium | LARGe}

The :DIGital<d>:SIZE command specifies the size of digital channels on the display. Sizes are set for all digital channels. Therefore, if you set the size on digital channel 0 (for example), the same size is set on all other as well.

NOTE

This command is only valid for the MSO models.

Query Syntax :DIGital<d>:SIZE?

The :DIGital<d>:SIZE? query returns the size setting for the specified digital channels.

Return Format <size_value><NL>

<size_value> ::= {SMAL | MED | LARG}

- See Also**
- ["Introduction to :DIGital<d> Commands"](#) on page 384
 - [":POD<n>:SIZE"](#) on page 748
 - [":DIGital<d>:POSition"](#) on page 387

:DIGital<d>:THReshold

N (see [page 1650](#))

Command Syntax :DIGital<d>:THReshold <value>

<d> ::= 0 to (# digital channels - 1) in NR1 format

<value> ::= {CMOS | ECL | TTL | <user defined value>[<suffix>]}

<user defined value> ::= -8.00 to +8.00 in NR3 format

<suffix> ::= {V | mV | uV}

- TTL = 1.4V
- CMOS = 2.5V
- ECL = -1.3V

The :DIGital<d>:THReshold command sets the logic threshold value for all channels in the same *pod* as the specified channel. The threshold is used for triggering purposes and for displaying the digital data as high (above the threshold) or low (below the threshold).

NOTE

This command is only valid for the MSO models.

Query Syntax :DIGital<d>:THReshold?

The :DIGital<d>:THReshold? query returns the threshold value for the specified channel.

Return Format <value><NL>

<value> ::= threshold value in NR3 format

- See Also**
- ["Introduction to :DIGital<d> Commands"](#) on page 384
 - [":POD<n>:THReshold"](#) on page 749
 - [":TRIGger\[:EDGE\]:LEVel"](#) on page 1376

16 :DISPlay Commands

Control how waveforms, graticule, and text are displayed and written on the screen. See ["Introduction to :DISPlay Commands"](#) on page 393.

Table 103 :DISPlay Commands Summary

Command	Query	Options and Query Returns
:DISPlay:ANNotation<n> { {0 OFF} {1 ON} } (see page 394)	:DISPlay:ANNotation<n>? (see page 394)	{0 1} <n> ::= an integer from 1 to 4 in NR1 format.
:DISPlay:ANNotation<n>:BACKground <mode> (see page 395)	:DISPlay:ANNotation<n>:BACKground? (see page 395)	<mode> ::= {OPAQue INVerted TRANSPARENT} <n> ::= an integer from 1 to 4 in NR1 format.
:DISPlay:ANNotation<n>:COLor <color> (see page 396)	:DISPlay:ANNotation<n>:COLor? (see page 396)	<color> ::= {CH1 CH2 CH3 CH4 DIG MATH REF MARKer WHITE RED} <n> ::= an integer from 1 to 4 in NR1 format.
:DISPlay:ANNotation<n>:TEXT <string> (see page 397)	:DISPlay:ANNotation<n>:TEXT? (see page 397)	<string> ::= quoted ASCII string (up to 254 characters) <n> ::= an integer from 1 to 4 in NR1 format.
:DISPlay:ANNotation<n>:X1Position <value> (see page 398)	:DISPlay:ANNotation<n>:X1Position? (see page 398)	<value> ::= an integer from 0 to (800 - width of annotation) in NR1 format. <n> ::= an integer from 1 to 4 in NR1 format.
:DISPlay:ANNotation<n>:Y1Position <value> (see page 399)	:DISPlay:ANNotation<n>:Y1Position? (see page 399)	<value> ::= an integer from 0 to (480 - height of annotation) in NR1 format. <n> ::= an integer from 1 to 4 in NR1 format.

Table 103 :DISPlay Commands Summary (continued)

Command	Query	Options and Query Returns
:DISPlay:BACKlight { {0 OFF} {1 ON} } (see page 400)	n/a	n/a
:DISPlay:CLear (see page 401)	n/a	n/a
n/a	:DISPlay:DATA? [<format>] [,] [<palette>] e] (see page 402)	<format> ::= {BMP BMP8bit PNG} <palette> ::= {COLor GRAYscale} <display data> ::= data in IEEE 488.2 # format
:DISPlay:GRATicule:ALABels { {0 OFF} {1 ON} } (see page 404)	:DISPlay:GRATicule:ALABels? (see page 404)	<setting> ::= {0 1}
:DISPlay:GRATicule:INTensity <value> (see page 405)	:DISPlay:GRATicule:INTensity? (see page 405)	<value> ::= an integer from 0 to 100 in NR1 format.
:DISPlay:GRATicule:TYPE <type> (see page 406)	:DISPlay:GRATicule:TYPE? (see page 406)	<type> ::= {FULL MVOLT IRE}
:DISPlay:INTensity:WAVEform <value> (see page 407)	:DISPlay:INTensity:WAVEform? (see page 407)	<value> ::= an integer from 0 to 100 in NR1 format.
:DISPlay:LAbel { {0 OFF} {1 ON} } (see page 408)	:DISPlay:LAbel? (see page 408)	{0 1}
:DISPlay:LABList <binary block> (see page 409)	:DISPlay:LABList? (see page 409)	<binary block> ::= an ordered list of up to 75 labels, each 10 characters maximum, separated by newline characters
:DISPlay:MENU <menu> (see page 410)	n/a	<menu> ::= {MASK MEASure SEGmented LISTer POWER}
:DISPlay:MESSAge:CLear (see page 411)	n/a	n/a
:DISPlay:PERsistence <value> (see page 412)	:DISPlay:PERsistence? (see page 412)	<value> ::= {MINimum INFinite <time>} <time> ::= seconds in in NR3 format from 100E-3 to 60E0
:DISPlay:PERsistence:CLear (see page 413)	n/a	n/a

Table 103 :DISPlay Commands Summary (continued)

Command	Query	Options and Query Returns
:DISPlay:SIDebar <sidebar> (see page 414)	n/a	<sidebar> ::= {SUMMary CURSors MEASurements DVM NAVigate CONTrols EVENTs COUNTER HISTogram}
:DISPlay:TRANSPARENT {OFF ON} (see page 415)	:DISPlay:TRANSPARENT? (see page 415)	{OFF ON}
:DISPlay:VECTors {{0 OFF} {1 ON}} (see page 416)	:DISPlay:VECTors? (see page 416)	{0 1} OFF or 0 can only be selected with the 1 GHz or 1.5 GHz bandwidth models

Introduction to :DISPlay Commands

The DISPlay subsystem is used to control the display storage and retrieval of waveform data, labels, and text. This subsystem allows the following actions:

- Clear the waveform area on the display.
- Turn vectors on or off.
- Set waveform persistence.
- Specify labels.
- Save and Recall display data.

Reporting the Setup

Use :DISPlay? to query the setup information for the DISPlay subsystem.

Return Format

The following is a sample response from the :DISPlay? query. In this case, the query was issued following a *RST command.

```
:DISP:LAB 0;VECT 1;PERS MIN
```

:DISPlay:ANNotation<n>

N (see [page 1650](#))

Command Syntax :DISPlay:ANNotation<n> <setting>

<setting> ::= {{1 | ON} | {0 | OFF}}

<n> ::= an integer from 1 to 10 in NR1 format.

The :DISPlay:ANNotation<n> command turns the annotation on and off. When on, the annotation appears in the upper left corner of the oscilloscope's display.

The annotation is useful for documentation purposes, to add notes before capturing screens.

Query Syntax :DISPlay:ANNotation<n>?

The :DISPlay:ANNotation<n>? query returns the annotation setting.

Return Format <value><NL>

<value> ::= {0 | 1}

- See Also**
- [":DISPlay:ANNotation<n>:TEXT"](#) on page 397
 - [":DISPlay:ANNotation<n>:COLor"](#) on page 396
 - [":DISPlay:ANNotation<n>:BACKground"](#) on page 395
 - [":DISPlay:ANNotation<n>:X1Position"](#) on page 398
 - [":DISPlay:ANNotation<n>:Y1Position"](#) on page 399
 - ["Introduction to :DISPlay Commands"](#) on page 393

:DISPlay:ANNotation<n>:BACKground

N (see [page 1650](#))

Command Syntax :DISPlay:ANNotation<n>:BACKground <mode>

<mode> ::= {OPAQue | INVerted | TRANSPARENT}

<n> ::= an integer from 1 to 10 in NR1 format.

The :DISPlay:ANNotation<n>:BACKground command specifies the background of the annotation:

- OPAQue – the annotation has a solid background.
- INVerted – the annotation's foreground and background colors are switched.
- TRANSPARENT – the annotation has a transparent background.

Query Syntax :DISPlay:ANNotation<n>:BACKground?

The :DISPlay:ANNotation<n>:BACKground? query returns the specified annotation background mode.

Return Format <mode><NL>

<mode> ::= {OPAQ | INV | TRAN}

- See Also**
- [":DISPlay:ANNotation<n>"](#) on page 394
 - [":DISPlay:ANNotation<n>:TEXT"](#) on page 397
 - [":DISPlay:ANNotation<n>:COLor"](#) on page 396
 - [":DISPlay:ANNotation<n>:X1Position"](#) on page 398
 - [":DISPlay:ANNotation<n>:Y1Position"](#) on page 399
 - ["Introduction to :DISPlay Commands"](#) on page 393

:DISPlay:ANNotation<n>:COLor

N (see [page 1650](#))

Command Syntax :DISPlay:ANNotation<n>:COLor <color>

<color> ::= {CH1 | CH2 | CH3 | CH4 | DIG | MATH | REF | MARKer | WHITE
| RED}

<n> ::= an integer from 1 to 10 in NR1 format.

The :DISPlay:ANNotation<n>:COLor command specifies the annotation color. You can choose white, red, or colors that match analog channels, digital channels, math waveforms, reference waveforms, or markers.

Query Syntax :DISPlay:ANNotation<n>:COLor?

The :DISPlay:ANNotation<n>:COLor? query returns the specified annotation color.

Return Format <color><NL>

<color> ::= {CH1 | CH2 | CH3 | CH4 | DIG | MATH | REF | MARK | WHIT
| RED}

- See Also**
- [":DISPlay:ANNotation<n>"](#) on page 394
 - [":DISPlay:ANNotation<n>:TEXT"](#) on page 397
 - [":DISPlay:ANNotation<n>:BACKground"](#) on page 395
 - [":DISPlay:ANNotation<n>:X1Position"](#) on page 398
 - [":DISPlay:ANNotation<n>:Y1Position"](#) on page 399
 - ["Introduction to :DISPlay Commands"](#) on page 393

:DISPlay:ANNotation<n>:TEXT

N (see [page 1650](#))

Command Syntax :DISPlay:ANNotation<n>:TEXT <string>

<string> ::= quoted ASCII string (up to 254 characters)

<n> ::= an integer from 1 to 10 in NR1 format.

The :DISPlay:ANNotation<n>:TEXT command specifies the annotation string. The annotation string can contain as many characters as will fit in the Edit Annotation box on the oscilloscope's screen, up to 254 characters.

You can include a carriage return in the annotation string using the characters "\n". Note that this is not a new line character but the actual "\" (backslash) and "n" characters in the string. Carriage returns lessen the number of characters available for the annotation string.

Use :DISPlay:ANNotation<n>:TEXT "" to remotely clear the annotation text. (Two sets of quote marks without a space between them creates a NULL string.)

Query Syntax :DISPlay:ANNotation<n>:TEXT?

The :DISPlay:ANNotation<n>:TEXT? query returns the specified annotation text.

When carriage returns are present in the annotation text, they are returned as the actual carriage return character (ASCII 0x0D).

Return Format <string><NL>

<string> ::= quoted ASCII string

- See Also**
- [":DISPlay:ANNotation<n>"](#) on page 394
 - [":DISPlay:ANNotation<n>:COLor"](#) on page 396
 - [":DISPlay:ANNotation<n>:BACKground"](#) on page 395
 - [":DISPlay:ANNotation<n>:X1Position"](#) on page 398
 - [":DISPlay:ANNotation<n>:Y1Position"](#) on page 399
 - ["Introduction to :DISPlay Commands"](#) on page 393

:DISPlay:ANNotation<n>:X1Position

N (see [page 1650](#))

Command Syntax :DISPlay:ANNotation<n>:X1Position <value>

<value> ::= an integer from 0 to (800 - width of annotation) in NR1 format.

<n> ::= an integer from 1 to 10 in NR1 format.

The :DISPlay:ANNotation<n>:X1Position command sets the annotation's horizontal X1 position.

Query Syntax :DISPlay:ANNotation<n>:X1Position?

The :DISPlay:ANNotation<n>:X1Position? query returns the annotation's horizontal X1 position.

Return Format <value><NL>

<value> ::= an integer from 0 to (800 - width of annotation) in NR1 format.

- See Also**
- [":DISPlay:ANNotation<n>:Y1Position"](#) on page 399
 - [":DISPlay:ANNotation<n>"](#) on page 394
 - [":DISPlay:ANNotation<n>:COLor"](#) on page 396
 - [":DISPlay:ANNotation<n>:BACKground"](#) on page 395
 - [":DISPlay:ANNotation<n>:TEXT"](#) on page 397

:DISPlay:ANNotation<n>:Y1Position

N (see [page 1650](#))

Command Syntax :DISPlay:ANNotation<n>:Y1Position <value>

<value> ::= an integer from 0 to (480 - height of annotation) in NR1 format.

<n> ::= an integer from 1 to 10 in NR1 format.

The :DISPlay:ANNotation<n>:Y1Position command sets the annotation's vertical Y1 position.

Query Syntax :DISPlay:ANNotation<n>:Y1Position?

The :DISPlay:ANNotation<n>:Y1Position? query returns the annotation's vertical Y1 position.

Return Format <value><NL>

<value> ::= an integer from 0 to (480 - height of annotation) in NR1 format.

- See Also**
- [":DISPlay:ANNotation<n>:X1Position"](#) on page 398
 - [":DISPlay:ANNotation<n>"](#) on page 394
 - [":DISPlay:ANNotation<n>:COLor"](#) on page 396
 - [":DISPlay:ANNotation<n>:BACKground"](#) on page 395
 - [":DISPlay:ANNotation<n>:TEXT"](#) on page 397

:DISPlay:BACKlight

N (see [page 1650](#))

Command Syntax :DISPlay:BACKlight {{0 | OFF} | {1 | ON}}

The :DISPlay:BACKlight command turns the display's backlight off or on.

:DISPlay:CLEar

N (see [page 1650](#))

Command Syntax :DISPlay:CLEar

The :DISPlay:CLEar command clears the display and resets all associated measurements. If the oscilloscope is stopped, all currently displayed data is erased. If the oscilloscope is running, all of the data for active channels and functions is erased; however, new data is displayed on the next acquisition.

See Also • ["Introduction to :DISPlay Commands"](#) on page 393

:DISPlay:DATA

N (see [page 1650](#))

Query Syntax :DISPlay:DATA? [<format>] [, <palette>]

<format> ::= {BMP | BMP8bit | PNG}

<palette> ::= {COLor | GRAYscale}

The :DISPlay:DATA? query reads screen image data. You can choose 24-bit BMP, 8-bit BMP8bit, or 24-bit PNG formats in color or grayscale.

Note that the returned image is also affected by the :HARDcopy:INKSaver command, which is ON by default and returns an inverted image. To get a non-inverted image, send the ":HARDcopy:INKSaver OFF" command before the DATA? query.

If no format or palette option is specified, the screen image is returned in whatever image format is selected by the front panel's **[Save/Recall] > Save > Format** softkey. If the **Format** softkey does not select an image format (in other words, it selects a setup or data format), the BMP, COLor format is used.

Screen image data is returned in the IEEE-488.2 # binary block data format.

Return Format <display data><NL>

<display data> ::= binary block data in IEEE-488.2 # format.

- See Also**
- ["Introduction to :DISPlay Commands"](#) on page 393
 - [":HARDcopy:INKSaver"](#) on page 512
 - [":HCOPY:SDUMp:DATA"](#) on page 523
 - [":HCOPY:SDUMp:FORMat"](#) on page 524
 - [":PRINT"](#) on page 272
 - ["*RCL \(Recall\)"](#) on page 225
 - ["*SAV \(Save\)"](#) on page 229
 - [":VIEW"](#) on page 280

Example Code

```
' IMAGE_TRANSFER - In this example, we will query for the image data
' with ":DISPlay:DATA?", read the data, and then save it to a file.
Dim byteData() As Byte
myScope.IO.Timeout = 15000
myScope.WriteString ":DISPlay:DATA? BMP, COLOR"
byteData = myScope.ReadIEEEBlock(BinaryType_UI1)
' Output display data to a file:
strPath = "c:\scope\data\screen.bmp"
' Remove file if it exists.
If Len(Dir(strPath)) Then
    Kill strPath
End If
Close #1 ' If #1 is open, close it.
```

```

    Open strPath For Binary Access Write Lock Write As #1      ' Open file f
or output.
    Put #1, , byteData      ' Write data.
    Close #1      ' Close file.
    myScope.IO.Timeout = 5000

```

See complete example programs at: [Chapter 46](#), “Programming Examples,” starting on page 1659

:DISPlay:GRATicule:ALABels

N (see [page 1650](#))

Command Syntax :DISPlay:GRATicule:ALABels {{0 | OFF} | {1 | ON}}

The :DISPlay:GRATicule:ALABels command turns graticule (grid) axis labels on or off.

Query Syntax :DISPlay:GRATicule:ALABels?

The :DISPlay:GRATicule:ALABels? query returns the graticule (grid) axis labels setting

Return Format <setting><NL>
 <setting> ::= {0 | 1}

- See Also**
- [":DISPlay:GRATicule:INTensity"](#) on page 405
 - [":DISPlay:GRATicule:TYPE"](#) on page 406

:DISPlay:GRATicule:INTensity

N (see [page 1650](#))

Command Syntax :DISPlay:GRATicule:INTensity <value>

<value> ::= an integer from 0 to 100 in NR1 format.

The :DISPlay:GRATicule:INTensity command sets the graticule (grid) intensity.

Query Syntax :DISPlay:GRATicule:INTensity?

The :DISPlay:GRATicule:INTensity? query returns the graticule (grid) intensity setting.

Return Format <value><NL>

- See Also**
- [":DISPlay:GRATicule:ALABels"](#) on page 404
 - [":DISPlay:GRATicule:TYPE"](#) on page 406

:DISPlay:GRATicule:TYPE

N (see [page 1650](#))

Command Syntax :DISPlay:GRATicule:TYPE <type>

<type> ::= {FULL | MVOLt | IRE}

The :DISPlay:GRATicule:TYPE command sets the graticule (grid) type.

When the TV trigger type is selected (see [":TRIGger:MODE"](#) on page 1358), and the vertical scaling of at least one displayed channel is 140 mV/div, the :DISPlay:GRATicule:TYPE command lets you select from these grid types:

- FULL – the normal oscilloscope grid.
- MVOLt – shows vertical grids, labeled on the left, from -0.3 V to 0.8 V.
- IRE – (Institute of Radio Engineers) shows vertical grids in IRE units, labeled on the left, from -40 to 100 IRE. The 0.35 V and 0.7 V levels from the MVOLt grid are also shown and labeled at the right. When the IRE grid is selected, cursor values on the display are also shown in IRE units. However, cursor values via the remote interface are not in IRE units.

The MVOLt and IRE grid values are accurate when the vertical scaling is 140 mV/division and the vertical offset is 245 mV.

Query Syntax :DISPlay:GRATicule:TYPE?

The :DISPlay:GRATicule:TYPE? query returns the graticule (grid) type setting.

Return Format <type><NL>

<type> ::= {FULL | MVOL | IRE}

- See Also**
- [":TRIGger:MODE"](#) on page 1358
 - [":CHANnel<n>:SCALE"](#) on page 354
 - [":CHANnel<n>:OFFSet"](#) on page 336
 - [":DISPlay:GRATicule:ALABels"](#) on page 404
 - [":DISPlay:GRATicule:INTensity"](#) on page 405

:DISPlay:INTensity:WAVeform

N (see [page 1650](#))

Command Syntax :DISPlay:INTensity:WAVeform <value>

<value> ::= an integer from 0 to 100 in NR1 format.

The :DISPlay:INTensity:WAVeform command sets the waveform intensity.

This is the same as adjusting the front panel **[Intensity]** knob.

Query Syntax :DISPlay:INTensity:WAVeform?

The :DISPlay:INTensity:WAVeform? query returns the waveform intensity setting.

Return Format <value><NL>

<value> ::= an integer from 0 to 100 in NR1 format.

See Also • ["Introduction to :DISPlay Commands"](#) on page 393

:DISPlay:LABel

N (see [page 1650](#))

Command Syntax :DISPlay:LABel <value>

<value> ::= {{1 | ON} | {0 | OFF}}

The :DISPlay:LABel command turns the analog and digital channel labels on and off.

Query Syntax :DISPlay:LABel?

The :DISPlay:LABel? query returns the display mode of the analog and digital labels.

Return Format <value><NL>

<value> ::= {0 | 1}

- See Also**
- ["Introduction to :DISPlay Commands"](#) on page 393
 - [":CHANnel<n>:LABel"](#) on page 335

Example Code

```
' DISP_LABEL
' - Turns label names ON or OFF on the analyzer display.
myScope.WriteString ":DISPlay:LABel ON" ' Turn on labels.
```

See complete example programs at: [Chapter 46](#), "Programming Examples," starting on page 1659

:DISPlay:LABList

N (see [page 1650](#))

Command Syntax :DISPlay:LABList <binary block data>

<binary block> ::= an ordered list of up to 75 labels, a maximum of 32 characters each, separated by newline characters.

The :DISPlay:LABList command adds labels to the label list. Labels are added in alphabetical order.

NOTE

Labels that begin with the same alphabetic base string followed by decimal digits are considered duplicate labels. Duplicate labels are not added to the label list. For example, if label "A0" is in the list and you try to add a new label called "A1234567890123456789012345678901", the new label is not added.

Query Syntax :DISPlay:LABList?

The :DISPlay:LABList? query returns the label list.

Return Format <binary block><NL>

<binary block> ::= an ordered list of up to 75 labels, a maximum of 32 characters each, separated by newline characters.

- See Also**
- ["Introduction to :DISPlay Commands"](#) on page 393
 - [":DISPlay:LABel"](#) on page 408
 - [":CHANnel<n>:LABel"](#) on page 335
 - [":DIGital<d>:LABel"](#) on page 386
 - [":BUS<n>:LABel"](#) on page 310

:DISPlay:MENU

N (see [page 1650](#))

Command Syntax :DISPlay:MENU <menu>

<menu> ::= {MASK | MEASure | SEGmented | LISTer | POWer | OFF}

The :DISPlay:MENU command changes the front panel softkey menu or turns it off. When off, channel setup information is displayed instead.

:DISPlay:MESSage:CLEar

N (see [page 1650](#))

Command Syntax :DISPlay:MESSage:CLEar

The :DISPlay:MESSage:CLEar command removes all user messages that are currently on screen.

See Also • [":SYSTem:DSP"](#) on page 1310

:DISPlay:PERSistence

N (see [page 1650](#))

Command Syntax :DISPlay:PERSistence <value>

<value> ::= {MINimum | INFinite | <time>}

<time> ::= seconds in in NR3 format from 100E-3 to 60E0

The :DISPlay:PERSistence command specifies the persistence setting:

- MINimum – indicates zero persistence.
- INFinite – indicates infinite persistence.
- <time> – for variable persistence, that is, you can specify how long acquisitions remain on the screen.

Use the :DISPlay:PERSistence:CLEar command to erase points stored by persistence.

Query Syntax :DISPlay:PERSistence?

The :DISPlay:PERSistence? query returns the specified persistence setting.

Return Format <value><NL>

<value> ::= {MIN | INF | <time>}

- See Also**
- ["Introduction to :DISPlay Commands"](#) on page 393
 - [":DISPlay:PERSistence:CLEar"](#) on page 413

:DISPlay:PERSistence:CLEar

N (see [page 1650](#))

Command Syntax :DISPlay:PERSistence:CLEar

The :DISPlay:PERSistence:CLEar command erases all persistence data from the display, leaving the data from the last acquisition.

If the oscilloscope is running, the display will begin to accumulate waveform and persistence data again.

The :DISPlay:CLEar command clears all waveform data from the display, including the data from the last acquisition.

- See Also**
- [":DISPlay:PERSistence"](#) on page 412
 - [":DISPlay:CLEar"](#) on page 401

:DISPlay:SIDebar

N (see [page 1650](#))

Command Syntax :DISPlay:SIDebar <sidebar>

```
<sidebar> ::= {SUMMary | CURSors | MEASurements | DVM | NAVigate  
              | CONTrols | EVENTs | COUNter | HISTogram}
```

The :DISPlay:SIDebar command specifies the sidebar dialog to display on the screen.

:DISPlay:TRANSPARENT

N (see [page 1650](#))

Command Syntax :DISPlay:TRANSPARENT <setting>

<setting> ::= {OFF | ON}

The :DISPlay:TRANSPARENT command enables or disables transparent mode for dialog box backgrounds in the front panel graphical user interface.

This command maps to the **Transparent** softkey that appears in the front panel user interface under **[Utility] > Options > Preferences**.

Query Syntax :DISPlay:TRANSPARENT?

The :DISPlay:TRANSPARENT? query returns the transparent setting.

Return Format <setting><NL>

<setting> ::= {OFF | ON}

See Also • ["Introduction to :DISPlay Commands"](#) on page 393

:DISPlay:VECTors

N (see [page 1650](#))

Command Syntax `:DISPlay:VECTors <vectors>`
 `<vectors> ::= {{1 | ON} | {0 | OFF}}`

The :DISPlay:VECTors command turns vector display on or off.

When vectors are turned on, the oscilloscope displays lines connecting sampled data points.

On the 1 GHz and 1.5 GHz bandwidth models, you can turn off vectors to view just waveform data points.

Query Syntax `:DISPlay:VECTors?`

The :DISPlay:VECTors? query returns the vectors setting.

Return Format `<vectors><NL>`
 `<vectors> ::= {1 | 0}`

See Also • ["Introduction to :DISPlay Commands"](#) on page 393

17 :DVM Commands

These commands control the digital voltmeter (DVM) feature.

Table 104 :DVM Commands Summary

Command	Query	Options and Query Returns
:DVM:ARANge {{0 OFF} {1 ON}} (see page 418)	:DVM:ARANge? (see page 418)	{0 1}
n/a	:DVM:CURREnt? (see page 419)	<dvm_value> ::= floating-point number in NR3 format
:DVM:ENABle {{0 OFF} {1 ON}} (see page 420)	:DVM:ENABle? (see page 420)	{0 1}
:DVM:MODE <mode> (see page 421)	:DVM:MODE? (see page 421)	<dvm_mode> ::= {ACRMs DC DCRMs}
:DVM:SOURce <source> (see page 422)	:DVM:SOURce? (see page 422)	<source> ::= {CHANnel<n>} <n> ::= 1-2 or 1-4 in NR1 format

:DVM:ARANge

N (see [page 1650](#))

Command Syntax :DVM:ARANge <setting>

<setting> ::= {{OFF | 0} | {ON | 1}}

If the selected digital voltmeter (DVM) source channel is not used in oscilloscope triggering, the :DVM:ARANge command turns the digital voltmeter's Auto Range capability on or off.

- When on, the DVM channel's vertical scale, vertical (ground level) position, and trigger (threshold voltage) level (used for the counter frequency measurement) are automatically adjusted.

The Auto Range capability overrides attempted adjustments of the channel's vertical scale and position.

- When off, you can adjust the channel's vertical scale and position normally.

Query Syntax :DVM:ARANge?

The :DVM:ARANge? query returns a flag indicating whether the digital voltmeter's Auto Range capability is on or off.

Return Format <setting><NL>

<setting> ::= {0 | 1}

- See Also**
- [":DVM:SOURce"](#) on page 422
 - [":DVM:ENABLE"](#) on page 420
 - [":DVM:MODE"](#) on page 421

:DVM:CURRent

N (see [page 1650](#))

Query Syntax :DVM:CURRent?

The :DVM:CURRent? query returns the displayed 3-digit DVM value based on the current mode.

NOTE

It can take up to a few seconds after DVM analysis is enabled before this query starts to produce good results, that is, results other than +9.9E+37. To wait for good values after DVM analysis is enabled, programs should loop until a value less than +9.9E+37 is returned.

Return Format <dvm_value><NL>

<dvm_value> ::= floating-point number in NR3 format

- See Also**
- [":DVM:SOURce"](#) on page 422
 - [":DVM:ENABLE"](#) on page 420
 - [":DVM:MODE"](#) on page 421

:DVM:ENABLE

N (see [page 1650](#))

Command Syntax :DVM:ENABLE <setting>

<setting> ::= {{OFF | 0} | {ON | 1}}

The :DVM:ENABLE command turns the digital voltmeter (DVM) analysis feature on or off.

Query Syntax :DVM:ENABLE?

The :DVM:ENABLE? query returns a flag indicating whether the digital voltmeter (DVM) analysis feature is on or off.

Return Format <setting><NL>

<setting> ::= {0 | 1}

- See Also**
- [":DVM:SOURce"](#) on page 422
 - [":DVM:MODE"](#) on page 421
 - [":DVM:ARANge"](#) on page 418

:DVM:MODE

N (see [page 1650](#))

Command Syntax :DVM:MODE <dvm_mode>
 <dvm_mode> ::= {ACRMs | DC | DCRMs}

The :DVM:MODE command sets the digital voltmeter (DVM) mode:

- ACRMs – displays the root-mean-square value of the acquired data, with the DC component removed.
- DC – displays the DC value of the acquired data.
- DCRMs – displays the root-mean-square value of the acquired data.

Query Syntax :DVM:MODE?

The :DVM:MODE? query returns the selected DVM mode.

Return Format <dvm_mode><NL>
 <dvm_mode> ::= {ACRM | DC | DCRM}

- See Also**
- [":DVM:ENABLE"](#) on page 420
 - [":DVM:SOURce"](#) on page 422
 - [":DVM:ARANge"](#) on page 418
 - [":DVM:CURREnt"](#) on page 419

:DVM:SOURce

N (see [page 1650](#))

Command Syntax :DVM:SOURce <source>

<source> ::= {CHANnel<n>}

<n> ::= 1-2 or 1-4 in NR1 format

The :DVM:SOURce command sets the select the analog channel on which digital voltmeter (DVM) measurements are made.

The selected channel does not have to be on (displaying a waveform) in order for DVM measurements to be made.

Query Syntax :DVM:SOURce?

The :DVM:SOURce? query returns the selected DVM input source.

Return Format <source><NL>

<source> ::= {CHAN<n>}

<n> ::= 1-2 or 1-4 in NR1 format

- See Also**
- [":DVM:ENABle"](#) on page 420
 - [":DVM:MODE"](#) on page 421
 - [":DVM:ARANge"](#) on page 418
 - [":DVM:CURREnt"](#) on page 419

18 :EXternal Trigger Commands

Control the input characteristics of the external trigger input. See ["Introduction to :EXternal Trigger Commands"](#) on page 423.

Table 105 :EXternal Trigger Commands Summary

Command	Query	Options and Query Returns
:EXternal:BWLimit <bwlimit> (see page 424)	:EXternal:BWLimit? (see page 424)	<bwlimit> ::= {0 OFF}
:EXternal:PROBe <attenuation> (see page 425)	:EXternal:PROBe? (see page 425)	<attenuation> ::= probe attenuation ratio in NR3 format
:EXternal:RANGe <range>[<suffix>] (see page 426)	:EXternal:RANGe? (see page 426)	<range> ::= vertical full-scale range value in NR3 format <suffix> ::= {V mV}
:EXternal:UNITs <units> (see page 427)	:EXternal:UNITs? (see page 427)	<units> ::= {VOLT AMPere}

Introduction to :EXternal Trigger Commands

The EXternal trigger subsystem commands control the input characteristics of the external trigger input. The probe factor, impedance, input range, input protection state, units, and bandwidth limit settings may all be queried. Depending on the instrument type, some settings may be changeable.

Reporting the Setup

Use :EXternal? to query setup information for the EXternal subsystem.

Return Format

The following is a sample response from the :EXternal query. In this case, the query was issued following a *RST command.

```
:EXT:BWL 0;RANG +8.0E+00;UNIT VOLT;PROB +1.000E+00
```

:EXternal:BWLimit

C (see [page 1650](#))

Command Syntax :EXternal:BWLimit <bwlimit>

<bwlimit> ::= {0 | OFF}

The :EXternal:BWLimit command is provided for product compatibility. The only legal value is 0 or OFF. Use the :TRIGger:HFReject command to limit bandwidth on the external trigger input.

Query Syntax :EXternal:BWLimit?

The :EXternal:BWLimit? query returns the current setting of the low-pass filter (always 0).

Return Format <bwlimit><NL>

<bwlimit> ::= 0

- See Also**
- ["Introduction to :EXternal Trigger Commands"](#) on page 423
 - ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:HFReject"](#) on page 1350

:EXternal:PROBe

C (see [page 1650](#))

Command Syntax :EXternal:PROBe <attenuation>

<attenuation> ::= probe attenuation ratio in NR3 format

The :EXternal:PROBe command specifies the probe attenuation factor for the external trigger. The probe attenuation factor may be 0.1 to 1000. This command does not change the actual input sensitivity of the oscilloscope. It changes the reference constants for scaling the display factors and for setting trigger levels.

If an AutoProbe probe is connected to the oscilloscope, the attenuation value cannot be changed from the sensed value. Attempting to set the oscilloscope to an attenuation value other than the sensed value produces an error.

Query Syntax :EXternal:PROBe?

The :EXternal:PROBe? query returns the current probe attenuation factor for the external trigger.

Return Format <attenuation><NL>

<attenuation> ::= probe attenuation ratio in NR3 format

- See Also**
- ["Introduction to :EXternal Trigger Commands"](#) on page 423
 - [":EXternal:RANGe"](#) on page 426
 - ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":CHANnel<n>:PROBe"](#) on page 337

:EXternal:RANGe

C (see [page 1650](#))

Command Syntax :EXternal:RANGe <range>[<suffix>]

<range> ::= vertical full-scale range value in NR3 format

<suffix> ::= {V | mV}

The :EXternal:RANGe command is provided for product compatibility. When using 1:1 probe attenuation, the range is either 1.6 V or 8 V.

The range is automatically recalculated when the external trigger probe attenuation factor is changed.

Query Syntax :EXternal:RANGe?

The :EXternal:RANGe? query returns the current full-scale range setting for the external trigger.

Return Format <range_argument><NL>

<range_argument> ::= external trigger range value in NR3 format

- See Also**
- ["Introduction to :EXternal Trigger Commands"](#) on page 423
 - [":EXternal:PROBe"](#) on page 425
 - ["Introduction to :TRIGger Commands"](#) on page 1345

:EXternal:UNITs

N (see [page 1650](#))

Command Syntax :EXternal:UNITs <units>

<units> ::= {VOLT | AMPere}

The :EXternal:UNITs command sets the measurement units for the probe connected to the external trigger input. Select VOLT for a voltage probe and select AMPere for a current probe. Measurement results, channel sensitivity, and trigger level will reflect the measurement units you select.

Query Syntax :EXternal:UNITs?

The :CHANnel<n>:UNITs? query returns the current units setting for the external trigger.

Return Format <units><NL>

<units> ::= {VOLT | AMP}

- See Also**
- ["Introduction to :EXternal Trigger Commands"](#) on page 423
 - ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":EXternal:RANGe"](#) on page 426
 - [":EXternal:PROBe"](#) on page 425
 - [":CHANnel<n>:UNITs"](#) on page 355

19 :FRANalysis Commands

Control oscilloscope functions associated with the Frequency Response Analysis (FRA) feature, which is available in oscilloscope models that have a license-enabled built-in waveform generator. See **"Introduction to :FRANalysis Commands"** on page 430.

Table 106 :FRANalysis Commands Summary

Command	Query	Options and Query Returns
n/a	:FRANalysis:DATA? [SWEep SINGle] (see page 432)	<binary_block> ::= comma-separated data with newlines at the end of each row
:FRANalysis:ENABle {{0 OFF} {1 ON}} (see page 433)	:FRANalysis:ENABle? (see page 433)	{0 1}
:FRANalysis:FREQuency :MODE <setting> (see page 434)	:FRANalysis:FREQuency :MODE? (see page 434)	<setting> ::= {SWEep SINGle}
:FRANalysis:FREQuency :SINGle <value>[suffix] (see page 435)	:FRANalysis:FREQuency :SINGle? (see page 435)	<value> ::= {20 100 1000 10000 100000 1000000 10000000 20000000} [suffix] ::= {Hz kHz MHz}
:FRANalysis:FREQuency :STARt <value>[suffix] (see page 436)	:FRANalysis:FREQuency :STARt? (see page 436)	<value> ::= {20 100 1000 10000 100000 1000000 10000000} [suffix] ::= {Hz kHz MHz}
:FRANalysis:FREQuency :STOP <value>[suffix] (see page 437)	:FRANalysis:FREQuency :STOP? (see page 437)	<value> ::= {100 1000 10000 100000 1000000 10000000 20000000} [suffix] ::= {Hz kHz MHz}
:FRANalysis:PPDecade <value> (see page 438)	:FRANalysis:PPDecade? (see page 438)	<value> ::= {10 20 30 40 50 60 70 80 90 100}
:FRANalysis:RUN (see page 439)	n/a	n/a

Table 106 :FRANalysis Commands Summary (continued)

Command	Query	Options and Query Returns
:FRANalysis:SOURce:IN Put <source> (see page 440)	:FRANalysis:SOURce:IN Put? (see page 440)	<source> ::= CHANnel<n> <n> ::= 1 to (# analog channels) in NR1 format
:FRANalysis:SOURce:OU TPut <source> (see page 441)	:FRANalysis:SOURce:OU TPut? (see page 441)	<source> ::= CHANnel<n> <n> ::= 1 to (# analog channels) in NR1 format
:FRANalysis:SWEep:POI Nts <points> (see page 442)	:FRANalysis:SWEep:POI Nts? (see page 442)	<points> ::= integer from 1-1000 in NR1 format
:FRANalysis:TRACe <selection> (see page 443)	:FRANalysis:TRACe? (see page 443)	<selection> ::= {NONE ALL GAIN PHASe}[, {GAIN PHASe}]
:FRANalysis:WGEN:LOAD <impedance> (see page 444)	:FRANalysis:WGEN:LOAD ? (see page 444)	<impedance> ::= {ONEMeg FIFTy}
:FRANalysis:WGEN:VOLT age <amplitude>, [<range>] (see page 445)	:FRANalysis:WGEN:VOLT age? [<range>] (see page 445)	<amplitude> ::= amplitude in volts in NR3 format <range> ::= {F20HZ F100HZ F1KHZ F10KHZ F100KHZ F1MHZ F10MHZ F20MHZ}
:FRANalysis:WGEN:VOLT age:PROFile {{0 OFF} {1 ON}} (see page 447)	:FRANalysis:WGEN:VOLT age:PROFile? (see page 447)	{0 1}

Introduction to :FRANalysis Commands

The FRANalysis subsystem controls the Frequency Response Analysis feature in the oscilloscope.

The Frequency Response Analysis (FRA) feature controls the built-in waveform generator to sweep a sine wave across a range of frequencies while measuring the input to and output from a device under test (DUT). At each frequency, gain (A) and phase are measured and plotted on a frequency response chart.

Reporting the Setup

Use :FRANalysis? to query setup information for the FRANalysis subsystem.

Return Format

The following is a sample response from the :FRANalysis? query. In this case, the query was issued following a *RST command.

```
:FRAN:SOUR:INP CHAN1;OUTP CHAN2;:FRAN:FREQ:STAR +100E+00;  
STOP +20.000000E+06;:FRAN:WGEN:VOLT +200.0E-03;LOAD FIFT
```

:FRANalysis:DATA

N (see [page 1650](#))

Query Syntax :FRANalysis:DATA? [SWEep | SINGLE]

The :FRANalysis:DATA? query returns the frequency response analysis data.

The data is returned in four comma-separated columns of data for each step in the sweep: Frequency (Hz), Amplitude (Vpp), Gain (dB), and Phase (°).

You can use the :FRANalysis:TRACe command to specify whether to include gain, phase, both gain and phase, or neither in the frequency response analysis results.

The SWEep or SINGLE option specifies whether to get the data from a sweep or single-frequency analysis (see :FRANalysis:FREQuency:MODE). If this option is not specified, the data from the sweep analysis is returned by default.

Return Format <binary_block><NL>

<binary_block> ::= comma-separated data with newlines at the end of each row

- See Also**
- [":FRANalysis:ENABle"](#) on page 433
 - [":FRANalysis:FREQuency:MODE"](#) on page 434
 - [":FRANalysis:FREQuency:SINGLE"](#) on page 435
 - [":FRANalysis:FREQuency:STARt"](#) on page 436
 - [":FRANalysis:FREQuency:STOP"](#) on page 437
 - [":FRANalysis:RUN"](#) on page 439
 - [":FRANalysis:SOURce:INPut"](#) on page 440
 - [":FRANalysis:SOURce:OUTPut"](#) on page 441
 - [":FRANalysis:TRACe"](#) on page 443
 - [":FRANalysis:WGEN:LOAD"](#) on page 444
 - [":FRANalysis:WGEN:VOLTage"](#) on page 445

:FRANalysis:ENABLE

N (see [page 1650](#))

Command Syntax :FRANalysis:ENABLE <setting>

<setting> ::= {{0 | OFF} | {1 | ON}}

The :FRANalysis:ENABLE command turns the Frequency Response Analysis (FRA) feature on or off.

Query Syntax :FRANalysis:ENABLE?

The :FRANalysis:ENABLE? query returns a flag indicating whether the Frequency Response Analysis (FRA) feature is on or off.

Return Format <setting><NL>

<setting> ::= {0 | 1}

- See Also**
- [":FRANalysis:DATA"](#) on page 432
 - [":FRANalysis:FREQuency:STARt"](#) on page 436
 - [":FRANalysis:FREQuency:STOP"](#) on page 437
 - [":FRANalysis:RUN"](#) on page 439
 - [":FRANalysis:SOURce:INPut"](#) on page 440
 - [":FRANalysis:SOURce:OUTPut"](#) on page 441
 - [":FRANalysis:WGEN:LOAD"](#) on page 444
 - [":FRANalysis:WGEN:VOLTage"](#) on page 445

:FRANalysis:FREQuency:MODE

N (see [page 1650](#))

Command Syntax :FRANalysis:FREQuency:MODE <setting>

<setting> ::= {SWEep | SINGle}

The :FRANalysis:FREQuency:MODE command lets you select between the normal swept frequency response analysis or analysis at a single frequency, which can be useful when debugging.

Query Syntax :FRANalysis:FREQuency:MODE?

The :FRANalysis:FREQuency:MODE? query returns the frequency mode setting.

Return Format <setting><NL>

<setting> ::= {SWEep | SINGle}

- See Also**
- [":FRANalysis:RUN"](#) on page 439
 - [":FRANalysis:FREQuency:SINGle"](#) on page 435
 - [":FRANalysis:FREQuency:STARt"](#) on page 436
 - [":FRANalysis:FREQuency:STOP"](#) on page 437
 - [":FRANalysis:SWEep:POINts"](#) on page 442
 - [":FRANalysis:PPDecade"](#) on page 438
 - [":FRANalysis:WGEN:VOLTage:PROFile"](#) on page 447

:FRANalysis:FREQuency:SINGle

N (see [page 1650](#))

Command Syntax :FRANalysis:FREQuency:SINGle <value>[suffix]

<value> ::= {20 | 100 | 1000 | 10000 | 100000 | 1000000 | 10000000
| 2000000}

[suffix] ::= {Hz | kHz | MHz}

The :FRANalysis:FREQuency:SINGle command sets the single frequency value. The frequency response analysis is displayed on a log scale Bode plot, so you can select from decade values in addition to the minimum frequency of 20 Hz.

Query Syntax :FRANalysis:FREQuency:SINGle?

The :FRANalysis:FREQuency:SINGle? query returns the single frequency setting.

Return Format <value><NL>

<value> ::= {20 | 100 | 1000 | 10000 | 100000 | 1000000 | 10000000
| 2000000}

- See Also**
- [":FRANalysis:DATA"](#) on page 432
 - [":FRANalysis:ENABLE"](#) on page 433
 - [":FRANalysis:SWEep:POINts"](#) on page 442
 - [":FRANalysis:PPDecade"](#) on page 438
 - [":FRANalysis:FREQuency:MODE"](#) on page 434
 - [":FRANalysis:RUN"](#) on page 439
 - [":FRANalysis:SOURce:INPut"](#) on page 440
 - [":FRANalysis:SOURce:OUTPut"](#) on page 441
 - [":FRANalysis:WGEN:LOAD"](#) on page 444
 - [":FRANalysis:WGEN:VOLTage"](#) on page 445
 - [":FRANalysis:WGEN:VOLTage:PROFile"](#) on page 447

:FRANalysis:FREQuency:START

N (see [page 1650](#))

Command Syntax :FRANalysis:FREQuency:START <value>[suffix]

<value> ::= {20 | 100 | 1000 | 10000 | 100000 | 1000000 | 10000000}

[suffix] ::= {Hz | kHz | MHz}

The :FRANalysis:FREQuency:START command sets the frequency sweep start value. The frequency response analysis is displayed on a log scale Bode plot, so you can select from decade values in addition to the minimum frequency of 20 Hz.

Query Syntax :FRANalysis:FREQuency:START?

The :FRANalysis:FREQuency:START? query returns the frequency sweep start setting.

Return Format <value><NL>

<value> ::= {20 | 100 | 1000 | 10000 | 100000 | 1000000 | 10000000}

- See Also**
- [":FRANalysis:DATA"](#) on page 432
 - [":FRANalysis:ENABle"](#) on page 433
 - [":FRANalysis:FREQuency:STOP"](#) on page 437
 - [":FRANalysis:SWEep:POINts"](#) on page 442
 - [":FRANalysis:PPDecade"](#) on page 438
 - [":FRANalysis:FREQuency:MODE"](#) on page 434
 - [":FRANalysis:RUN"](#) on page 439
 - [":FRANalysis:SOURce:INPut"](#) on page 440
 - [":FRANalysis:SOURce:OUTPut"](#) on page 441
 - [":FRANalysis:WGEN:LOAD"](#) on page 444
 - [":FRANalysis:WGEN:VOLTage"](#) on page 445
 - [":FRANalysis:WGEN:VOLTage:PROFile"](#) on page 447

:FRANalysis:FREQuency:STOP

N (see [page 1650](#))

Command Syntax :FRANalysis:FREQuency:STOP <value>[suffix]

<value> ::= {100 | 1000 | 10000 | 100000 | 1000000 | 10000000 | 20000000}

[suffix] ::= {Hz | kHz | MHz}

The :FRANalysis:FREQuency:STOP command sets the frequency sweep stop value. The frequency response analysis is displayed on a log scale Bode plot, so you can select from decade values in addition to the maximum frequency of 20 MHz.

Query Syntax :FRANalysis:FREQuency:STOP?

The :FRANalysis:FREQuency:STOP? query returns the frequency sweep stop setting.

Return Format <value><NL>

<value> ::= {100 | 1000 | 10000 | 100000 | 1000000 | 10000000 | 20000000}

- See Also**
- [":FRANalysis:DATA"](#) on page 432
 - [":FRANalysis:ENABle"](#) on page 433
 - [":FRANalysis:FREQuency:START"](#) on page 436
 - [":FRANalysis:SWEep:POINts"](#) on page 442
 - [":FRANalysis:PPDecade"](#) on page 438
 - [":FRANalysis:FREQuency:MODE"](#) on page 434
 - [":FRANalysis:RUN"](#) on page 439
 - [":FRANalysis:SOURce:INPut"](#) on page 440
 - [":FRANalysis:SOURce:OUTPut"](#) on page 441
 - [":FRANalysis:WGEN:LOAD"](#) on page 444
 - [":FRANalysis:WGEN:VOLTage"](#) on page 445
 - [":FRANalysis:WGEN:VOLTage:PROFile"](#) on page 447

:FRANalysis:PPDecade

N (see [page 1650](#))

Command Syntax :FRANalysis:PPDecade <value>

<value> ::= {10 | 20 | 30 | 40 | 50 | 60 | 70 | 80 | 90 | 100}

The :FRANalysis:PPDecade command specifies the number of points per decade in the frequency response analysis.

Query Syntax :FRANalysis:PPDecade?

The :FRANalysis:PPDecade? query returns the points per decade setting.

Return Format <value><NL>

- See Also**
- [":FRANalysis:SWEep:POINts"](#) on page 442
 - [":FRANalysis:FREQuency:STARt"](#) on page 436
 - [":FRANalysis:FREQuency:STOP"](#) on page 437
 - [":FRANalysis:WGEN:VOLTage:PROFile"](#) on page 447

:FRANalysis:RUN

N (see [page 1650](#))

Command Syntax :FRANalysis:RUN

The :FRANalysis:RUN command performs the Frequency Response Analysis. This analysis controls the built-in waveform generator to sweep a sine wave across a range of frequencies while measuring the input to and output from a device under test (DUT). At each frequency, gain (A) and phase are measured and plotted on a Bode frequency response chart.

The :FRANalysis:APPLY command is a valid compatible alias for the :FRANalysis:RUN command.

When the frequency response analysis completes, you can use the :FRANalysis:DATA? query to get four comma-separated columns of data for each step in the sweep: Frequency (Hz), Amplitude (Vpp), Gain (dB), and Phase (°).

You can use the :FRANalysis:TRACe command to specify whether to include gain, phase, both gain and phase, or neither in the frequency response analysis results.

It takes some time for the frequency sweep analysis to complete. You can query bit 0 of the Standard Event Status Register (*ESR?) to find out when the analysis is complete.

- See Also**
- [":FRANalysis:DATA"](#) on page 432
 - [":FRANalysis:ENABle"](#) on page 433
 - [":FRANalysis:FREQuency:MODE"](#) on page 434
 - [":FRANalysis:FREQuency:SINGle"](#) on page 435
 - [":FRANalysis:FREQuency:START"](#) on page 436
 - [":FRANalysis:FREQuency:STOP"](#) on page 437
 - [":FRANalysis:SOURce:INPut"](#) on page 440
 - [":FRANalysis:SOURce:OUTPut"](#) on page 441
 - [":FRANalysis:TRACe"](#) on page 443
 - [":FRANalysis:WGEN:LOAD"](#) on page 444
 - [":FRANalysis:WGEN:VOLTage"](#) on page 445
 - ["*ESR \(Standard Event Status Register\)"](#) on page 218

:FRANalysis:SOURce:INPut

N (see [page 1650](#))

Command Syntax :FRANalysis:SOURce:INPut <source>

<source> ::= CHANnel<n>

<n> ::= 1 to (# analog channels) in NR1 format

The :FRANalysis:SOURce:INPut command specifies the analog input channel that is probing the input voltage to the device under test (DUT) in the frequency response analysis.

Query Syntax :FRANalysis:SOURce:INPut?

The :FRANalysis:SOURce:INPut? query returns the currently selected channel probing the input voltage.

Return Format <source><NL>

<source> ::= CHAN<n>

- See Also**
- [":FRANalysis:DATA"](#) on page 432
 - [":FRANalysis:ENABLE"](#) on page 433
 - [":FRANalysis:FREQuency:START"](#) on page 436
 - [":FRANalysis:FREQuency:STOP"](#) on page 437
 - [":FRANalysis:RUN"](#) on page 439
 - [":FRANalysis:SOURce:OUTPut"](#) on page 441
 - [":FRANalysis:WGEN:LOAD"](#) on page 444
 - [":FRANalysis:WGEN:VOLTage"](#) on page 445

:FRANalysis:SOURce:OUTPut

N (see [page 1650](#))

Command Syntax :FRANalysis:SOURce:OUTPut <source>

<source> ::= CHANnel<n>

<n> ::= 1 to (# analog channels) in NR1 format

The :FRANalysis:SOURce:OUTPut command specifies the analog input channel that is probing the output voltage from the device under test (DUT) in the frequency response analysis.

Query Syntax :FRANalysis:SOURce:OUTPut?

The :FRANalysis:SOURce:OUTPut? query returns the currently selected channel probing the output voltage.

Return Format <source><NL>

<source> ::= CHAN<n>

- See Also**
- [":FRANalysis:DATA"](#) on page 432
 - [":FRANalysis:ENABLE"](#) on page 433
 - [":FRANalysis:FREQuency:START"](#) on page 436
 - [":FRANalysis:FREQuency:STOP"](#) on page 437
 - [":FRANalysis:RUN"](#) on page 439
 - [":FRANalysis:SOURce:INPut"](#) on page 440
 - [":FRANalysis:WGEN:LOAD"](#) on page 444
 - [":FRANalysis:WGEN:VOLTage"](#) on page 445

:FRANalysis:SWEep:POINts

N (see [page 1650](#))

Command Syntax :FRANalysis:SWEep:POINts <points>

<points> ::= integer from 1-1000 in NR1 format

The :FRANalysis:SWEep:POINts command specifies the number of points in the frequency response analysis.

Depending on the number of points specified and the number of decades in the frequency range, this command can also change the :FRANalysis:PPDecade setting.

Query Syntax :FRANalysis:SWEep:POINts?

The :FRANalysis:SWEep:POINts? query returns the points setting.

Return Format <points><NL>

<points> ::= integer from 1-1000 in NR1 format

- See Also**
- [":FRANalysis:PPDecade"](#) on page 438
 - [":FRANalysis:FREQuency:START"](#) on page 436
 - [":FRANalysis:FREQuency:STOP"](#) on page 437
 - [":FRANalysis:WGEN:VOLTage:PROFile"](#) on page 447

:FRANalysis:TRACe

N (see [page 1650](#))

Command Syntax :FRANalysis:TRACe <selection>

<selection> ::= {NONE | ALL | GAIN | PHASe}[, {GAIN | PHASe}]

The :FRANalysis:TRACe command specifies whether to include gain, phase, both gain and phase, or neither in the frequency response analysis results.

NOTE

This command affects the oscilloscope's front panel graphical user interface (plot and table) as well as when saving analysis data.

Query Syntax :FRANalysis:TRACe?

The :FRANalysis:TRACe? query returns a comma-separated list of the types of data that are currently included in the frequency response analysis results, or "NONE" if neither gain nor phase data is included.

Return Format <selection_list><NL>

<selection_list> ::= {"NONE" | "GAIN" | "PHASe" | "GAIN, PHASe"}

- See Also**
- [":FRANalysis:RUN"](#) on page 439
 - [":FRANalysis:DATA"](#) on page 432

:FRANalysis:WGEN:LOAD

N (see [page 1650](#))

Command Syntax :FRANalysis:WGEN:LOAD <impedance>

<impedance> ::= {ONEMeg | FIFTy}

The :FRANalysis:WGEN:LOAD command selects the expected output load impedance.

The output impedance of the Gen Out BNC is fixed at 50 ohms. However, the output load selection lets the waveform generator display the correct amplitude and offset levels for the expected output load.

If the actual load impedance is different than the selected value, the displayed amplitude and offset levels will be incorrect.

Query Syntax :FRANalysis:WGEN:LOAD?

The :FRANalysis:WGEN:LOAD? query returns the current expected output load impedance.

Return Format <impedance><NL>

<impedance> ::= {ONEM | FIFT}

- See Also**
- [":FRANalysis:DATA"](#) on page 432
 - [":FRANalysis:ENABLE"](#) on page 433
 - [":FRANalysis:FREQuency:START"](#) on page 436
 - [":FRANalysis:FREQuency:STOP"](#) on page 437
 - [":FRANalysis:RUN"](#) on page 439
 - [":FRANalysis:SOURce:INPut"](#) on page 440
 - [":FRANalysis:SOURce:OUTPut"](#) on page 441
 - [":FRANalysis:WGEN:VOLTage"](#) on page 445

:FRANalysis:WGEN:VOLTage

N (see [page 1650](#))

Command Syntax :FRANalysis:WGEN:VOLTage <amplitude>,[<range>]

<amplitude> ::= amplitude in volts in NR3 format

<range> ::= {F20HZ | F100HZ | F1KHZ | F10KHZ | F100KHZ | F1MHZ
| F10MHZ | F20MHZ}

The :FRANalysis:WGEN:VOLTage command specifies the waveform generator's output sine wave amplitude.

Use the :WGEN:VOLTage:OFFSet command to specify the offset voltage or DC level.

You can also specify the amplitude and offset indirectly using the :WGEN:VOLTage:HIGH and :WGEN:VOLTage:LOW commands. For example, an amplitude of 5 V and an offset of 1 V is the same as a high-level voltage of 4 V and a low-level voltage of -1 V.

When the amplitude profile setting is on (:FRANalysis:WGEN:VOLTage:PROFile ON), the <range> option specifies the initial ramp amplitude at the frequency setting. Amplitudes ramp between the settings specified for individual frequencies.

Query Syntax :FRANalysis:WGEN:VOLTage? [<range>]

The :FRANalysis:WGEN:VOLTage? query returns the currently specified waveform generator amplitude.

When the amplitude profile setting is on (:FRANalysis:WGEN:VOLTage:PROFile ON), the <range> option specifies the frequency whose initial ramp amplitude is returned.

Return Format <amplitude><NL>

<amplitude> ::= amplitude in volts in NR3 format

- See Also**
- [":FRANalysis:WGEN:VOLTage:PROFile"](#) on page 447
 - [":FRANalysis:DATA"](#) on page 432
 - [":FRANalysis:ENABle"](#) on page 433
 - [":FRANalysis:FREQuency:STARt"](#) on page 436
 - [":FRANalysis:FREQuency:STOP"](#) on page 437
 - [":FRANalysis:SWEep:POINts"](#) on page 442
 - [":FRANalysis:PPDecade"](#) on page 438
 - [":FRANalysis:RUN"](#) on page 439
 - [":FRANalysis:SOURce:INPut"](#) on page 440
 - [":FRANalysis:SOURce:OUTPut"](#) on page 441

- **":FRANalysis:WGEN:LOAD"** on page 444

:FRANalysis:WGEN:VOLTage:PROFile

N (see [page 1650](#))

Command Syntax :FRANalysis:WGEN:VOLTage:PROFile {{0 | OFF} | {1 | ON}}

The :FRANalysis:WGEN:VOLTage:PROFile command enables or disables the ability to specify amplitude ramping within different decades.

Query Syntax :FRANalysis:WGEN:VOLTage:PROFile?

The :FRANalysis:WGEN:VOLTage:PROFile? query returns the voltage profile setting.

Return Format <setting><NL>
<setting> ::= {0 | 1}

- See Also**
- [":FRANalysis:WGEN:VOLTage"](#) on page 445
 - [":FRANalysis:SWEep:POINts"](#) on page 442
 - [":FRANalysis:PPDecade"](#) on page 438
 - [":FRANalysis:FREQuency:START"](#) on page 436
 - [":FRANalysis:FREQuency:STOP"](#) on page 437

20 :FUNction<m> Commands

Control math functions in the oscilloscope. See "[Introduction to :FUNction<m> Commands](#)" on page 455.

Table 107 :FUNction<m> Commands Summary

Command	Query	Options and Query Returns
:FUNction<m>:AVERage:COUNT <count> (see page 456)	:FUNction<m>:AVERage:COUNT? (see page 456)	<count> ::= an integer from 2 to 65536 in NR1 format <m> ::= 1 to (# math functions) in NR1 format
:FUNction<m>:BUS:CLOCK <source> (see page 457)	:FUNction<m>:BUS:CLOCK? (see page 457)	<source> ::= {CHANnel<n> DIGital<d>} <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <m> ::= 1 to (# math functions) in NR1 format
:FUNction<m>:BUS:SLOPe <slope> (see page 458)	:FUNction<m>:BUS:SLOPe? (see page 458)	<slope> ::= {NEGative POSitive EITHer} <m> ::= 1 to (# math functions) in NR1 format
:FUNction<m>:BUS:YINCrement <value> (see page 459)	:FUNction<m>:BUS:YINCrement? (see page 459)	<value> ::= value per bus code, in NR3 format <m> ::= 1 to (# math functions) in NR1 format
:FUNction<m>:BUS:YORigin <value> (see page 460)	:FUNction<m>:BUS:YORigin? (see page 460)	<value> ::= value at bus code = 0, in NR3 format <m> ::= 1 to (# math functions) in NR1 format

Table 107 :FUNCTION<m> Commands Summary (continued)

Command	Query	Options and Query Returns
:FUNCTION<m>:BUS:YUNITS <units> (see page 461)	:FUNCTION<m>:BUS:YUNITS? (see page 461)	<units> ::= {VOLT AMPere NONE} <m> ::= 1 to (# math functions) in NR1 format
:FUNCTION<m>:CLEar (see page 462)	n/a	n/a
:FUNCTION<m>:DISPlay {{0 OFF} {1 ON}} (see page 463)	:FUNCTION<m>:DISPlay? (see page 463)	{0 1} <m> ::= 1 to (# math functions) in NR1 format
n/a	:FUNCTION<m>[:FFT]:BSIZE? (see page 464)	<bin_size> ::= Hz in NR3 format <m> ::= 1 to (# math functions) in NR1 format
:FUNCTION<m>[:FFT]:CENTER <frequency> (see page 465)	:FUNCTION<m>[:FFT]:CENTER? (see page 465)	<frequency> ::= the current center frequency in NR3 format. The range of legal values is from -25 GHz to 25 GHz. <m> ::= 1 to (# math functions) in NR1 format
:FUNCTION<m>[:FFT]:DETECTION:POINTS <number_of_buckets> (see page 466)	:FUNCTION<m>[:FFT]:DETECTION:POINTS? (see page 466)	<number_of_buckets> ::= an integer from 640 to 64K in NR1 format <m> ::= 1 to (# math functions) in NR1 format
:FUNCTION<m>[:FFT]:DETECTION:TYPE <type> (see page 467)	:FUNCTION<m>[:FFT]:DETECTION:TYPE? (see page 467)	<type> ::= {OFF SAMPLE PPositive PNEGative NORMal AVERage} <m> ::= 1 to (# math functions) in NR1 format
:FUNCTION<m>[:FFT]:FREQUENCY:START <frequency> (see page 468)	:FUNCTION<m>[:FFT]:FREQUENCY:START? (see page 468)	<frequency> ::= the start frequency in NR3 format. <m> ::= 1 to (# math functions) in NR1 format
:FUNCTION<m>[:FFT]:FREQUENCY:STOP <frequency> (see page 469)	:FUNCTION<m>[:FFT]:FREQUENCY:STOP? (see page 469)	<frequency> ::= the stop frequency in NR3 format. <m> ::= 1 to (# math functions) in NR1 format
:FUNCTION<m>[:FFT]:GATE <gating> (see page 470)	:FUNCTION<m>[:FFT]:GATE? (see page 470)	<gating> ::= {NONE ZOOM} <m> ::= 1-4 in NR1 format

Table 107 :FUNCTION<m> Commands Summary (continued)

Command	Query	Options and Query Returns
:FUNCTION<m>[:FFT]:PHASE:REFERENCE<ref_point> (see page 471)	:FUNCTION<m>[:FFT]:PHASE:REFERENCE? (see page 471)	<ref_point> ::= {TRIGger DISPlay} <m> ::= 1-4 in NR1 format
n/a	:FUNCTION<m>[:FFT]:RBWidth? (see page 472)	<resolution_bw> ::= Hz in NR3 format <m> ::= 1 to (# math functions) in NR1 format
:FUNCTION<m>[:FFT]:READout<n><readout_type> (see page 473)	:FUNCTION<m>[:FFT]:READout<n>? (see page 473)	<readout_type> ::= {SRATE BSIZE RBWidth} <m> ::= 1 to (# math functions) in NR1 format <n> ::= 1-2 in NR1 format, 2 is for dedicated FFT function
:FUNCTION<m>[:FFT]:SPAN (see page 474)	:FUNCTION<m>[:FFT]:SPAN? (see page 474)	 ::= the current frequency span in NR3 format. Legal values are 1 Hz to 100 GHz. <m> ::= 1 to (# math functions) in NR1 format
n/a	:FUNCTION<m>[:FFT]:SRATE? (see page 475)	<sample_rate> ::= Hz in NR3 format <m> ::= 1 to (# math functions) in NR1 format
:FUNCTION<m>[:FFT]:VTYPE<units> (see page 476)	:FUNCTION<m>[:FFT]:VTYPE? (see page 476)	<units> ::= {DECibel VRMS} for the FFT (magnitude) operation <units> ::= {DEGREes RADians} for the FFTPhase operation <m> ::= 1 to (# math functions) in NR1 format
:FUNCTION<m>[:FFT]:WINDOW<window> (see page 477)	:FUNCTION<m>[:FFT]:WINDOW? (see page 477)	<window> ::= {RECTangular HANNing FLATtop BHARRis BARTlett} <m> ::= 1 to (# math functions) in NR1 format
:FUNCTION<m>:FREQuency:BANDpass:CENTer<center_freq> (see page 478)	:FUNCTION<m>:FREQuency:BANDpass:CENTer? (see page 478)	<center_freq> ::= center frequency of band-pass filter in NR3 format

Table 107 :FUNCTION<m> Commands Summary (continued)

Command	Query	Options and Query Returns
:FUNCTION<m>:FREQuency:BANDpass:WIDTh <freq_width> (see page 479)	:FUNCTION<m>:FREQuency:BANDpass:WIDTh? (see page 479)	<freq_width> ::= frequency width of band-pass filter in NR3 format
:FUNCTION<m>:FREQuency:HIGHPass <3dB_freq> (see page 480)	:FUNCTION<m>:FREQuency:HIGHPass? (see page 480)	<3dB_freq> ::= 3dB cutoff frequency value in NR3 format <m> ::= 1 to (# math functions) in NR1 format
:FUNCTION<m>:FREQuency:LOWPass <3dB_freq> (see page 481)	:FUNCTION<m>:FREQuency:LOWPass? (see page 481)	<3dB_freq> ::= 3dB cutoff frequency value in NR3 format <m> ::= 1 to (# math functions) in NR1 format
:FUNCTION<m>:INTEgrate:IICondition {{0 OFF} {1 ON}} (see page 482)	:FUNCTION<m>:INTEgrate:IICondition? (see page 482)	{0 1} <m> ::= 1 to (# math functions) in NR1 format
:FUNCTION<m>:INTEgrate:IOFFset <input_offset> (see page 483)	:FUNCTION<m>:INTEgrate:IOFFset? (see page 483)	<input_offset> ::= DC offset correction in NR3 format. <m> ::= 1 to (# math functions) in NR1 format
:FUNCTION<m>:LINear:GAIn <value> (see page 484)	:FUNCTION<m>:LINear:GAIn? (see page 484)	<value> ::= 'A' in $Ax + B$, value in NR3 format <m> ::= 1 to (# math functions) in NR1 format
:FUNCTION<m>:LINear:OFFSet <value> (see page 485)	:FUNCTION<m>:LINear:OFFSet? (see page 485)	<value> ::= 'B' in $Ax + B$, value in NR3 format <m> ::= 1 to (# math functions) in NR1 format
:FUNCTION<m>:OFFSet <offset> (see page 486)	:FUNCTION<m>:OFFSet? (see page 486)	<offset> ::= the value at center screen in NR3 format. The range of legal values is +/-10 times the current sensitivity of the selected function. <m> ::= 1 to (# math functions) in NR1 format

Table 107 :FUNCTION<m> Commands Summary (continued)

Command	Query	Options and Query Returns
:FUNCTION<m>:OPERatio n <operation> (see page 487)	:FUNCTION<m>:OPERatio n? (see page 490)	<operation> ::= {ADD SUBtract MULTiply DIVide INTegrate DIFF FFT FFTPhase SQRT MAGNify ABSolute SQUare LN LOG EXP TEN LOWPass HIGHpass BANDpass AVERage LINear MAXimum MINimum PEAK MAXHold MINHold TREnd BTIMing BSTate SERChart} <m> ::= 1 to (# math functions) in NR1 format
:FUNCTION<m>:RANGE <range> (see page 492)	:FUNCTION<m>:RANGE? (see page 492)	<range> ::= the full-scale vertical axis value in NR3 format. The range for ADD, SUBT, MULT is 8E-6 to 800E+3. The range for the INTegrate function is 8E-9 to 400E+3. The range for the DIFF function is 80E-3 to 8.0E12 (depends on current sweep speed). The range for the FFT function is 8 to 800 dBV. <m> ::= 1 to (# math functions) in NR1 format
:FUNCTION<m>:REferenc e <level> (see page 493)	:FUNCTION<m>:REferenc e? (see page 493)	<level> ::= the value at center screen in NR3 format. The range of legal values is +/-10 times the current sensitivity of the selected function. <m> ::= 1 to (# math functions) in NR1 format
:FUNCTION<m>:SCALE <scale value>[<suffix>] (see page 494)	:FUNCTION<m>:SCALE? (see page 494)	<scale value> ::= integer in NR1 format <suffix> ::= {V dB} <m> ::= 1 to (# math functions) in NR1 format

Table 107 :FUNCTION<m> Commands Summary (continued)

Command	Query	Options and Query Returns
:FUNCTION<m>:SERChart :SOURCE <serialbus> (see page 495)	:FUNCTION<m>:SERChart :SOURCE? (see page 495)	<m> ::= 1 to (# math functions) in NR1 format <serialbus> ::= {SBUS<n>} <n> ::= 1 to (# of serial bus) in NR1 format
:FUNCTION<m>:SERChart :TIME:MODE <value> (see page 496)	:FUNCTION<m>:SERChart :TIME:MODE? (see page 496)	<m> ::= 1 to (# math functions) in NR1 format <value> ::= {MAIN ROLL}
:FUNCTION<m>:SERChart :TIME:RANGE <range_value> (see page 498)	:FUNCTION<m>:SERChart :TIME:RANGE? (see page 498)	<m> ::= 1 to (# math functions) in NR1 format <range_value> ::= time value in NR3 format
:FUNCTION<m>:SERChart :TIME:OFFSet <offset> (see page 499)	:FUNCTION<m>:SERChart :TIME:OFFSet? (see page 499)	<m> ::= 1 to (# math functions) in NR1 format <offset> ::= vertical offset value in NR3 format
:FUNCTION<m>:SMOoth:POINTs <points> (see page 500)	:FUNCTION<m>:SMOoth:POINTs? (see page 500)	<points> ::= odd integer in NR1 format
:FUNCTION<m>:SOURCE1 <source> (see page 501)	:FUNCTION<m>:SOURCE1? (see page 501)	<source> ::= {CHANnel<n> FUNCTION<c> MATH<c> WMEMory<r> BUS} <n> ::= 1 to (# analog channels) in NR1 format <c> ::= {1 2 3}, must be lower than <m> <r> ::= 1 to (# ref waveforms) in NR1 format ::= {1 2} <m> ::= 1 to (# math functions) in NR1 format

Table 107 :FUNCTION<m> Commands Summary (continued)

Command	Query	Options and Query Returns
:FUNCTION<m>:SOURCE2 <source> (see page 503)	:FUNCTION<m>:SOURCE2? (see page 503)	<source> ::= {CHANNEL<n> WMEMORY<r> NONE} <n> ::= 1 to (# analog channels) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <m> ::= 1 to (# math functions) in NR1 format
:FUNCTION<m>:TEND:NMEASUREMENT MEAS<n> (see page 504)	:FUNCTION<m>:TEND:NMEASUREMENT? (see page 504)	<n> ::= # of installed measurement, from 1 to 10 <m> ::= 1 to (# math functions) in NR1 format

Introduction to :FUNCTION<m> Commands

The FUNCTION subsystem controls the math functions in the oscilloscope. Four math functions are available – the <m> in :FUNCTION<m> can be from 1 to 4.

The math function operator, transform, filter, or visualization is selected using the :FUNCTION<m>:OPERATION command. Depending on the selected operation, there may be other commands for specifying options for that operation. See [":FUNCTION<m>:OPERATION"](#) on page 487.

The SOURCE1, DISPLAY, RANGE, and OFFSET (or REFERENCE) commands apply to any function.

Reporting the Setup

Use :FUNCTION<m>? to query setup information for the FUNCTION subsystem.

Return Format

The following is a sample response from the :FUNCTION1? query. In this case, the query was issued following a *RST command.

```
:FUNC1:OPER ADD;DISP 0;SOUR1 CHAN1;SOUR2 CHAN2;RANG +8.00E+00;OFFS  
+0.0E+00
```

:FUNCTION<m>:AVERage:COUNT

N (see [page 1650](#))

Command Syntax :FUNCTION<m>:AVERage:COUNT <count>

<count> ::= an integer from 2 to 65536 in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

The :FUNCTION<m>:AVERage:COUNT command sets the number of waveforms to be averaged together.

The number of averages can be set from 2 to 65536 in increments of powers of 2.

Increasing the number of averages will increase resolution and reduce noise.

Query Syntax :FUNCTION<m>:AVERage:COUNT?

The :FUNCTION<m>:AVERage:COUNT? query returns the number of waveforms to be averaged together.

Return Format <count><NL>

<count> ::= an integer from 2 to 65536 in NR1 format

See Also • [":FUNCTION<m>:OPERation"](#) on page 487

:FUNCTION<m>:BUS:CLOCK

N (see [page 1650](#))

Command Syntax :FUNCTION<m>:BUS:CLOCK <source>

<m> ::= 1 to (# math functions) in NR1 format

<source> ::= {DIGital<d>}

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :FUNCTION<m>:BUS:CLOCK command selects the clock signal source for the Chart Logic Bus State operation.

Query Syntax :FUNCTION<m>:BUS:CLOCK?

The :FUNCTION<m>:BUS:CLOCK query returns the source selected for the clock signal.

Return Format <source><NL>

<source> ::= {DIGital<d>}

<d> ::= 0 to (# digital channels - 1) in NR1 format

See Also • [":FUNCTION<m>:OPERation"](#) on page 487

:FUNCTION<m>:BUS:SLOPe

N (see [page 1650](#))

Command Syntax :FUNCTION<m>:BUS:SLOPe <slope>

<m> ::= 1 to (# math functions) in NR1 format

<slope> ::= {NEGative | POSitive | EITHER}

The :FUNCTION<m>:BUS:SLOPe command specifies the clock signal edge for the Chart Logic Bus State operation.

Query Syntax :FUNCTION<m>:BUS:SLOPe?

The :FUNCTION<m>:BUS:SLOPe query returns the clock edge setting.

Return Format <slope><NL>

<slope> ::= {NEGative | POSitive | EITHER}

See Also • [":FUNCTION<m>:OPERation"](#) on page 487

:FUNCTION<m>:BUS:YINCrement

N (see [page 1650](#))

Command Syntax :FUNCTION<m>:BUS:YINCrement <value>

<m> ::= 1 to (# math functions) in NR1 format

<value> ::= value per bus code, in NR3 format

The :FUNCTION<m>:BUS:YINCrement command specifies the value associated with each increment in Chart Logic Bus data.

Query Syntax :FUNCTION<m>:BUS:YINCrement?

The :FUNCTION<m>:BUS:YINCrement query returns the value associated with each increment in Chart Logic Bus data.

Return Format <value><NL>

<value> ::= value per bus code, in NR3 format

See Also • [":FUNCTION<m>:OPERation"](#) on page 487

:FUNCTION<m>:BUS:YORigin

N (see [page 1650](#))

Command Syntax :FUNCTION<m>:BUS:YORigin <value>

<m> ::= 1 to (# math functions) in NR1 format

<value> ::= value at bus code = 0, in NR3 format

The :FUNCTION<m>:BUS:YORigin command specifies the value associated with Chart Logic Bus data equal to zero.

Query Syntax :FUNCTION<m>:BUS:YORigin?

The :FUNCTION<m>:BUS:YORigin query returns the value for associated with data equal to zero.

Return Format <value><NL>

<value> ::= value at bus code = 0, in NR3 format

See Also • [":FUNCTION<m>:OPERation"](#) on page 487

:FUNCTION<m>:BUS:YUNits

N (see [page 1650](#))

Command Syntax :FUNCTION<m>:BUS:YUNits <units>

<m> ::= 1 to (# math functions) in NR1 format

<units> ::= {VOLT | AMPere | NONE}

The :FUNCTION<m>:BUS:YUNits command specifies the vertical units for the Chart Logic Bus operations.

Query Syntax :FUNCTION<m>:BUS:YUNits?

The :FUNCTION<m>:BUS:YUNits query returns the Chart Logic Bus vertical units.

Return Format <units><NL>

<units> ::= {VOLT | AMP | NONE}

See Also • [":FUNCTION<m>:OPERation"](#) on page 487

:FUNCTION<m>:CLEar

N (see [page 1650](#))

Command Syntax :FUNCTION<m>:CLEar

When the :FUNCTION<m>:OPERation is AVERage, MAXHold, or MINHold, the :FUNCTION<m>:CLEar command clears the number of evaluated waveforms.

See Also • [":FUNCTION<m>:AVERage:COUNt"](#) on page 456

:FUNCTION<m>:DISPlay

N (see [page 1650](#))

Command Syntax :FUNCTION<m>:DISPlay <display>

<m> ::= 1 to (# math functions) in NR1 format

<display> ::= {{1 | ON} | {0 | OFF}}

The :FUNCTION<m>:DISPlay command turns the display of the function on or off. When ON is selected, the function operates as specified by the other :FUNCTION<m> commands.

NOTE

Automatic vertical scaling of the math function waveform occurs when the function display is turned from off to on. If you wait for the operation to complete (with an *OPC? query for example), then query the math function's scale, you can see the vertical scaling that was automatically determined.

One math function can be displayed at a time. If one math function is on and then another math function is turned on, the first math function will be turned off.

When math functions are off, they are still calculated so that they can be cascaded, that is, used as a source for a higher math function.

Query Syntax :FUNCTION<m>:DISPlay?

The :FUNCTION<m>:DISPlay? query returns whether the function display is on or off.

Return Format <display><NL>

<display> ::= {1 | 0}

- See Also**
- ["Introduction to :FUNCTION<m> Commands"](#) on page 455
 - ["*OPC \(Operation Complete\)"](#) on page 222
 - [":FUNCTION<m>:SCALE"](#) on page 494
 - [":VIEW"](#) on page 280
 - [":BLANK"](#) on page 248
 - [":STATus"](#) on page 277

:FUNCTION<m>[:FFT]:BSIZE

N (see [page 1650](#))

Query Syntax :FUNCTION<m>[:FFT]:BSIZE?

<m> ::= 1-4 in NR1 format

The :FUNCTION<m>[:FFT]:BSIZE? query returns the Bin Size setting for the FFT.

Return Format <bin_size><NL>

<bin_size> ::= Hz in NR3 format

See Also • [":FUNCTION<m>\[:FFT\]:READout<n>"](#) on page 473

:FUNCTION<m>[:FFT]:CENTER

N (see [page 1650](#))

- Command Syntax** :FUNCTION<m>[:FFT]:CENTER <frequency>
- <m> ::= 1 to (# math functions) in NR1 format
- <frequency> ::= the current center frequency in NR3 format. The range of legal values is from -25 GHz to 25 GHz.
- The :FUNCTION<m>[:FFT]:CENTER command sets the center frequency when FFT (Fast Fourier Transform) is selected.
- Query Syntax** :FUNCTION<m>[:FFT]:CENTER?
- The :FUNCTION<m>[:FFT]:CENTER? query returns the current center frequency in Hertz.
- Return Format** <frequency><NL>
- <frequency> ::= the current center frequency in NR3 format. The range of legal values is from -25 GHz to 25 GHz.

NOTE

After a *RST (Reset) or :AUToscale command, the values returned by the :FUNCTION<m>[:FFT]:CENTER? and :FUNCTION<m>:SPAN? queries depend on the current :TIMEbase:RANGE value. Once you change either the :FUNCTION<m>[:FFT]:CENTER or :FUNCTION<m>:SPAN value, they no longer track the :TIMEbase:RANGE value.

- See Also**
- ["Introduction to :FUNCTION<m> Commands"](#) on page 455
 - [":FUNCTION<m>\[:FFT\]:SPAN"](#) on page 474
 - [":TIMEbase:RANGE"](#) on page 1335
 - [":TIMEbase:SCALE"](#) on page 1339

:FUNCTION<m>[:FFT]:DETection:POINts

N (see [page 1650](#))

Command Syntax :FUNCTION<m>[:FFT]:DETection:POINts <number_of_buckets>
 <number_of_buckets> ::= an integer from 640 to 64K in NR1 format
 <m> ::= 1-4 in NR1 format

The :FUNCTION<m>[:FFT]:DETection:POINts command specifies the maximum number of points that the FFT detector should decimate to. This is also the number of buckets that sampled FFT points are grouped into before the selected detection type reduction (decimation) is applied.

The minimum number of points is 640.

The maximum number of points, 64K, is the measurement record limit.

Query Syntax :FUNCTION<m>[:FFT]:DETection:POINts?

The :FUNCTION<m>[:FFT]:DETection:POINts? query returns the FFT detector points setting.

Return Format <number_of_buckets><NL>
 <number_of_buckets> ::= an integer from 640 to 64K in NR1 format

See Also • [":FUNCTION<m>\[:FFT\]:DETection:TYPE"](#) on page 467

:FUNCTION<m>[:FFT]:DETECTION:TYPE

N (see [page 1650](#))

Command Syntax :FUNCTION<m>[:FFT]:DETECTION:TYPE <type>

<type> ::= {OFF | SAMPLE | PPOSitive | PNEGative | NORMal | AVERage}

<m> ::= 1-4 in NR1 format

The :FUNCTION<m>[:FFT]:DETECTION:TYPE command sets the FFT detector decimation type.

Detectors give you a way of manipulating the acquired data to emphasize different features of the data. Detectors reduce (decimate) the number of FFT points to at most the number of specified detector points. In this reduction, sampled FFT points are bucketized, that is, split into a number of groups that equals the specified number of detector points. Then, the points in each bucket are reduced to a single point according to the selected detection type. The detector types are:

- OFF – No detector is used.
- SAMPLE – Takes the point nearest to the center of every bucket.
- PPOSitive (+ Peak) – Takes the most positive point in every bucket.
- PNEGative (- Peak) – Takes the most negative point in every bucket.
- AVERage – Takes the average of all points in every bucket.
- NORMal – Implements a rosenfell algorithm. This method to picks either the minimum or maximum sample in every bucket depending on whether the data is monotonically increasing, decreasing, or varying. For details, see the [Spectrum Analysis Basics application note](#) at www.keysight.com.

When detectors are used, the FFT's output is decimated, and any analysis is performed on the reduced or detected data set.

Query Syntax :FUNCTION<m>[:FFT]:DETECTION:TYPE?

The :FUNCTION<m>[:FFT]:DETECTION:TYPE? query returns the FFT detector type setting

Return Format <type><NL>

<type> ::= {OFF | SAMPLE | PPOSitive | PNEGative | NORMal | AVERage}

See Also • [":FUNCTION<m>\[:FFT\]:DETECTION:POINTS"](#) on page 466

:FUNCTION<m>[:FFT]:FREQUENCY:START

N (see [page 1650](#))

Command Syntax :FUNCTION<m>[:FFT]:FREQUENCY:START <frequency>

<m> ::= 1 to (# math functions) in NR1 format

<frequency> ::= the start frequency in NR3 format.

The :FUNCTION<m>[:FFT]:FREQUENCY:START command sets the start frequency in the FFT (Fast Fourier Transform) math function's displayed range.

The FFT (Fast Fourier Transform) math function's displayed range can also be set with the :FUNCTION<m>[:FFT]:CENTER and :FUNCTION<m>[:FFT]:SPAN commands.

Query Syntax :FUNCTION<m>[:FFT]:FREQUENCY:START?

The :FUNCTION<m>[:FFT]:FREQUENCY:START? query returns the current start frequency in Hertz.

Return Format <frequency><NL>

<frequency> ::= the start frequency in NR3 format.

- See Also**
- [":FUNCTION<m>\[:FFT\]:FREQUENCY:STOP"](#) on page 469
 - [":FUNCTION<m>\[:FFT\]:CENTER"](#) on page 465
 - [":FUNCTION<m>\[:FFT\]:SPAN"](#) on page 474

:FUNCTION<m>[:FFT]:FREQUENCY:STOP

N (see [page 1650](#))

Command Syntax :FUNCTION<m>[:FFT]:FREQUENCY:STOP <frequency>

<m> ::= 1 to (# math functions) in NR1 format

<frequency> ::= the stop frequency in NR3 format.

The :FUNCTION<m>[:FFT]:FREQUENCY:STOP command sets the stop frequency in the FFT (Fast Fourier Transform) math function's displayed range.

The FFT (Fast Fourier Transform) math function's displayed range can also be set with the :FUNCTION<m>[:FFT]:CENTER and :FUNCTION<m>[:FFT]:SPAN commands.

Query Syntax :FUNCTION<m>[:FFT]:FREQUENCY:STOP?

The :FUNCTION<m>[:FFT]:FREQUENCY:STOP? query returns the current stop frequency in Hertz.

Return Format <frequency><NL>

<frequency> ::= the stop frequency in NR3 format.

- See Also**
- [":FUNCTION<m>\[:FFT\]:FREQUENCY:START"](#) on page 468
 - [":FUNCTION<m>\[:FFT\]:CENTER"](#) on page 465
 - [":FUNCTION<m>\[:FFT\]:SPAN"](#) on page 474

:FUNCTION<m>[:FFT]:GATE

N (see [page 1650](#))

Command Syntax :FUNCTION<m>[:FFT]:GATE <gating>

<m> ::= 1-4 in NR1 format

<gating> ::= {NONE | ZOOM}

The :FUNCTION<m>[:FFT]:GATE command specifies whether the FFT is performed on the Main time base window (NONE) or the ZOOM window when the zoomed time base is displayed.

Query Syntax :FUNCTION<m>[:FFT]:GATE?

The :FUNCTION<m>[:FFT]:GATE? query returns the gate setting.

Return Format <gating><NL>

<gating> ::= {NONE | ZOOM}

- See Also**
- [":FUNCTION<m>\[:FFT\]:VTYPE"](#) on page 476
 - [":FUNCTION<m>\[:FFT\]:WINDOW"](#) on page 477
 - ["Introduction to :FUNCTION<m> Commands"](#) on page 455
 - [":FUNCTION<m>:OPERation"](#) on page 487

:FUNCTION<m>[:FFT]:PHASe:REFeRence

N (see [page 1650](#))

Command Syntax :FUNCTION<m>[:FFT]:PHASe:REFeRence <ref_point>

<ref_point> ::= {TRIGger | DISPlay}

<m> ::= 1-4 in NR1 format

The :FUNCTION<m>[:FFT]:PHASe:REFeRence command sets the reference point for calculating the FFT Phase function to either the trigger point or beginning of the displayed waveform.

Query Syntax :FUNCTION<m>[:FFT]:PHASe:REFeRence?

The :FUNCTION<m>[:FFT]:PHASe:REFeRence? query returns the selected reference point.

Return Format <ref_point><NL>

<ref_point> ::= {TRIGger | DISPlay}

- See Also**
- [":FUNCTION<m>:OPERation"](#) on page 487
 - ["Introduction to :FUNCTION<m> Commands"](#) on page 455

:FUNCTION<m>[:FFT]:RBWidth

N (see [page 1650](#))

Query Syntax :FUNCTION<m>[:FFT]:RBWidth?

<m> ::= 1-4 in NR1 format

The :FUNCTION<m>[:FFT]:RBWidth? query returns the Resolution Bandwidth setting for the FFT.

Return Format <resolution_bw><NL>

<resolution_bw> ::= Hz in NR3 format

See Also • [":FUNCTION<m>\[:FFT\]:READout<n>"](#) on page 473

:FUNCTION<m>[:FFT]:READout<n>

N (see [page 1650](#))

Command Syntax :FUNCTION<m>[:FFT]:READout<n> <readout_type>

<readout_type> ::= {SRATe | BSIZe | RBWidth}

<m> ::= 1-4 in NR1 format

<n> ::= 1-2 in NR1 format, 2 is valid only on oscilloscopes that have the dedicated FFT function

The :FUNCTION<m>[:FFT]:READout<n> command selects from these types of readouts for the FFT:

- SRATe – Sample Rate
- BSIZe – Bin Size
- RBWidth – Resolution Bandwidth

Note that READout1 is used for both the dedicated FFT and the general-purpose math FFT function and READout2 is used only for oscilloscopes that have a dedicated FFT function (like the 3000T X-Series).

Query Syntax :FUNCTION<m>[:FFT]:READout<n>?

The :FUNCTION<m>[:FFT]:READout<n>? query returns the readout selection.

Return Format <readout_type><NL>

<readout_type> ::= {SRAT | BSIZ | RBW}

- See Also**
- [":FUNCTION<m>\[:FFT\]:BSIZe"](#) on page 464
 - [":FUNCTION<m>\[:FFT\]:RBWidth"](#) on page 472
 - [":FUNCTION<m>\[:FFT\]:SRATe"](#) on page 475

:FUNCTION<m>[:FFT]:SPAN

N (see [page 1650](#))

Command Syntax :FUNCTION<m>[:FFT]:SPAN

<m> ::= 1 to (# math functions) in NR1 format

 ::= the current frequency span in NR3 format. Legal values are 1 Hz to 100 GHz.

If you set the frequency span to a value outside of the legal range, the step size is automatically set to the nearest legal value.

The :FUNCTION<m>[:FFT]:SPAN command sets the frequency span of the display (left graticule to right graticule) when FFT (Fast Fourier Transform) is selected.

Query Syntax :FUNCTION<m>[:FFT]:SPAN?

The :FUNCTION<m>[:FFT]:SPAN? query returns the current frequency span in Hertz.

NOTE

After a *RST (Reset) or :AUTOScale command, the values returned by the :FUNCTION<m>[:FFT]:CENTER? and :FUNCTION<m>:SPAN? queries depend on the current :TIMEbase:RANGE value. Once you change either the :FUNCTION<m>[:FFT]:CENTER or :FUNCTION<m>:SPAN value, they no longer track the :TIMEbase:RANGE value.

Return Format <NL>

 ::= the current frequency span in NR3 format. Legal values are 1 Hz to 100 GHz.

- See Also**
- ["Introduction to :FUNCTION<m> Commands"](#) on page 455
 - [":FUNCTION<m>\[:FFT\]:CENTER"](#) on page 465
 - [":TIMEbase:RANGE"](#) on page 1335
 - [":TIMEbase:SCALE"](#) on page 1339

:FUNCTION<m>[:FFT]:SRATe

N (see [page 1650](#))

Query Syntax :FUNCTION<m>[:FFT]:SRATe?

<m> ::= 1-4 in NR1 format

The :FUNCTION<m>[:FFT]:SRATe? query returns the Sample Rate setting for the FFT.

Return Format <sample_rate><NL>

<sample_rate> ::= Hz in NR3 format

See Also • [":FUNCTION<m>\[:FFT\]:READOut<n>"](#) on page 473

:FUNCTION<m>[:FFT]:VTYPE

N (see [page 1650](#))

Command Syntax :FUNCTION<m>[:FFT]:VTYPE <units>

<m> ::= 1 to (# math functions) in NR1 format

<units> ::= {DECibel | VRMS} for the FFT (magnitude) operation

<units> ::= {DEGREes | RADians} for the FFTPhase operation

The :FUNCTION<m>[:FFT]:VTYPE command specifies FFT vertical units.

For the FFT (Magnitude) operation units, DECibel equates to the user interface's Logarithmic selection, and VRMS equates to the user interface's Linear selection.

Query Syntax :FUNCTION<m>[:FFT]:VTYPE?

The :FUNCTION<m>[:FFT]:VTYPE? query returns the current FFT vertical units.

Return Format <units><NL>

<units> ::= {DEC | VRMS} for the FFT (magnitude) operation

<units> ::= {DEGR | RAD} for the FFTPhase operation

- See Also**
- [":FUNCTION<m>\[:FFT\]:GATE"](#) on page 470
 - ["Introduction to :FUNCTION<m> Commands"](#) on page 455
 - [":FUNCTION<m>:OPERation"](#) on page 487

:FUNCTION<m>[:FFT]:WINDow

N (see [page 1650](#))

Command Syntax :FUNCTION<m>[:FFT]:WINDow <window>

<m> ::= 1 to (# math functions) in NR1 format

<window> ::= {RECTangular | HANNing | FLATtop | BHARris | BARTlett}

The :FUNCTION<m>[:FFT]:WINDow command allows the selection of different windowing transforms or operations for the FFT (Fast Fourier Transform) function.

The FFT operation assumes that the time record repeats. Unless an integral number of sampled waveform cycles exist in the record, a discontinuity is created between the end of one record and the beginning of the next. This discontinuity introduces additional frequency components about the peaks into the spectrum. This is referred to as leakage. To minimize leakage, windows that approach zero smoothly at the start and end of the record are employed as filters to the FFTs. Each window is useful for certain classes of input signals.

- RECTangular – useful for transient signals, and signals where there are an integral number of cycles in the time record.
- HANNing – useful for frequency resolution and general purpose use. It is good for resolving two frequencies that are close together, or for making frequency measurements. This is the default window.
- FLATtop – best for making accurate amplitude measurements of frequency peaks.
- BHARris (Blackman-Harris) – reduces time resolution compared to the rectangular window, but it improves the capacity to detect smaller impulses due to lower secondary lobes (provides minimal spectral leakage).
- BARTlett – (triangular, with end points at zero) window is similar to the Hanning window in that it is good for making accurate frequency measurements, but its higher and wider secondary lobes make it not quite as good for resolving frequencies that are close together.

Query Syntax :FUNCTION<m>[:FFT]:WINDow?

The :FUNCTION<m>[:FFT]:WINDow? query returns the value of the window selected for the FFT function.

Return Format <window><NL>

<window> ::= {RECT | HANN | FLAT | BHAR | BART}

- See Also**
- [":FUNCTION<m>\[:FFT\]:GATE"](#) on page 470
 - ["Introduction to :FUNCTION<m> Commands"](#) on page 455

:FUNCTION<m>:FREQUENCY:BANDpass:CENTer

N (see [page 1650](#))

Command Syntax :FUNCTION<m>:FREQUENCY:BANDpass:CENTer <center_freq>
 <center_freq> ::= center frequency of band-pass filter in NR3 format

The :FUNCTION<m>:FREQUENCY:BANDpass:CENTer command sets the center frequency of the band-pass filter.

Query Syntax :FUNCTION<m>:FREQUENCY:BANDpass:CENTer?

The :FUNCTION<m>:FREQUENCY:BANDpass:CENTer? query returns the band-pass filter's center frequency setting.

Return Format <center_freq><NL>

- See Also**
- [":FUNCTION<m>:FREQUENCY:BANDpass:WIDTH"](#) on page 479
 - [":FUNCTION<m>:OPERation"](#) on page 487

:FUNCTION<m>:FREQUENCY:BANDpass:WIDTH

N (see [page 1650](#))

Command Syntax :FUNCTION<m>:FREQUENCY:BANDpass:WIDTH <freq_width>

<freq_width> ::= frequency width of band-pass filter in NR3 format

The :FUNCTION<m>:FREQUENCY:BANDpass:WIDTH command sets the frequency width of the band-pass filter. The width specifies the filter's -3 dB cutoff frequencies (center frequency minus half the width and center frequency plus half the width).

Query Syntax :FUNCTION<m>:FREQUENCY:BANDpass:WIDTH?

The :FUNCTION<m>:FREQUENCY:BANDpass:WIDTH? query returns the band-pass filter's frequency width setting.

Return Format <freq_width><NL>

- See Also**
- [":FUNCTION<m>:FREQUENCY:BANDpass:CENTer"](#) on page 478
 - [":FUNCTION<m>:OPERation"](#) on page 487

:FUNCTION<m>:FREQUENCY:HIGHPASS

N (see [page 1650](#))

Command Syntax :FUNCTION<m>:FREQUENCY:HIGHPASS <3dB_freq>

<m> ::= 1 to (# math functions) in NR1 format

<3dB_freq> ::= -3dB cutoff frequency value in NR3 format

The :FUNCTION<m>:FREQUENCY:HIGHPASS command sets the high-pass filter's -3 dB cutoff frequency.

The high-pass filter is a single-pole high pass filter.

Query Syntax :FUNCTION<m>:FREQUENCY:HIGHPASS?

The :FUNCTION<m>:FREQUENCY:HIGHPASS query returns the high-pass filter's cutoff frequency.

Return Format <3dB_freq><NL>

<3dB_freq> ::= -3dB cutoff frequency value in NR3 format

See Also • [":FUNCTION<m>:OPERATION"](#) on page 487

:FUNCTION<m>:FREQUENCY:LOWPass

N (see [page 1650](#))

Command Syntax :FUNCTION<m>:FREQUENCY:LOWPass <3dB_freq>

<m> ::= 1 to (# math functions) in NR1 format

<3dB_freq> ::= -3dB cutoff frequency value in NR3 format

The :FUNCTION<m>:FREQUENCY:LOWPass command sets the low-pass filter's -3 dB cutoff frequency.

The low-pass filter is a 4th order Bessel-Thompson filter.

Query Syntax :FUNCTION<m>:FREQUENCY:LOWPass?

The :FUNCTION<m>:FREQUENCY:LOWPass query returns the low-pass filter's cutoff frequency.

Return Format <3dB_freq><NL>

<3dB_freq> ::= -3dB cutoff frequency value in NR3 format

See Also • [":FUNCTION<m>:OPERation"](#) on page 487

:FUNCTION<m>:INTEgrate:IICondition

N (see [page 1650](#))

Command Syntax :FUNCTION<m>:INTEgrate:IICondition {{0 | OFF} | {1 | ON}}

<m> ::= 1 to (# math functions) in NR1 format

The :FUNCTION<m>:INTEgrate:IICondition command sets the Initial Condition of the Integrate math function waveform:

- ON – the integrate math waveform is vertically centered in the screen. In other words, the top and bottom of the math waveform are equal distances from the top and bottom of the screen.
- OFF – the integrate math waveform starts at the zero-level reference on the left side of the screen.

Query Syntax :FUNCTION<m>:INTEgrate:IICondition?

The :FUNCTION<m>:INTEgrate:IICondition? query returns Initial Condition setting for the Integrate math function.

Return Format <setting><NL>

<setting> ::= {0 | 1}

- See Also**
- [":FUNCTION<m>:OPERation"](#) on page 487
 - [":FUNCTION<m>:INTEgrate:IOFFset"](#) on page 483

:FUNCTION<m>:INTEgrate:IOfFset

N (see [page 1650](#))

Command Syntax :FUNCTION<m>:INTEgrate:IOfFset <input_offset>

<m> ::= 1 to (# math functions) in NR1 format

<input_offset> ::= DC offset correction in NR3 format.

The :FUNCTION<m>:INTEgrate:IOfFset command lets you enter a DC offset correction factor for the integrate math waveform input signal. This DC offset correction lets you level a "ramp"ed waveform.

Query Syntax :FUNCTION<m>:INTEgrate:IOfFset?

The :FUNCTION<m>:INTEgrate:IOfFset? query returns the current input offset value.

Return Format <input_offset><NL>

<input_offset> ::= DC offset correction in NR3 format.

- See Also**
- ["Introduction to :FUNCTION<m> Commands"](#) on page 455
 - [":FUNCTION<m>:OPERation"](#) on page 487
 - [":FUNCTION<m>:INTEgrate:IICondition"](#) on page 482

:FUNCTION<m>:LINEar:GAIN

N (see [page 1650](#))

Command Syntax :FUNCTION<m>:LINEar:GAIN <value>

<m> ::= 1 to (# math functions) in NR1 format

<value> ::= 'A' in Ax + B, value in NR3 format

The :FUNCTION<m>:LINEar:GAIN command specifies the 'A' value in the Ax + B operation.

Query Syntax :FUNCTION<m>:LINEar:GAIN?

The :FUNCTION<m>:LINEar:GAIN query returns the gain value.

Return Format <value><NL>

<value> ::= 'A' in Ax + B, value in NR3 format

See Also • [":FUNCTION<m>:OPERation"](#) on page 487

:FUNCTION<m>:LINear:OFFSet

N (see [page 1650](#))

Command Syntax :FUNCTION<m>:LINear:OFFSet <value>

<m> ::= 1 to (# math functions) in NR1 format

<value> ::= 'B' in Ax + B, value in NR3 format

The :FUNCTION<m>:LINear:OFFSet command specifies the 'B' value in the Ax + B operation.

Query Syntax :FUNCTION<m>:LINear:OFFSet?

The :FUNCTION<m>:LINear:OFFSet query returns the offset value.

Return Format <value><NL>

<value> ::= 'B' in Ax + B, value in NR3 format

See Also • [":FUNCTION<m>:OPERation"](#) on page 487

:FUNCTION<m>:OFFSet

N (see [page 1650](#))

Command Syntax :FUNCTION<m>:OFFSet <offset>

<m> ::= 1 to (# math functions) in NR1 format

<offset> ::= the value at center screen in NR3 format.

The :FUNCTION<m>:OFFSet command sets the voltage or vertical value represented at center screen for the selected function. The range of legal values is generally +/-10 times the current scale of the selected function, but will vary by function. If you set the offset to a value outside of the legal range, the offset value is automatically set to the nearest legal value.

NOTE

The :FUNCTION<m>:OFFSet command is equivalent to the :FUNCTION<m>:REFerence command.

Query Syntax :FUNCTION<m>:OFFSet?

The :FUNCTION<m>:OFFSet? query outputs the current offset value for the selected function.

Return Format <offset><NL>

<offset> ::= the value at center screen in NR3 format.

- See Also**
- ["Introduction to :FUNCTION<m> Commands"](#) on page 455
 - [":FUNCTION<m>:RANGe"](#) on page 492
 - [":FUNCTION<m>:REFerence"](#) on page 493
 - [":FUNCTION<m>:SCALE"](#) on page 494

:FUNCTION<m>:OPERation

N (see [page 1650](#))

Command Syntax :FUNCTION<m>:OPERation <operation>

<m> ::= 1 to (# math functions) in NR1 format

<operation> ::= {ADD | SUBTract | MULTiPLY | DIVide | DIFF | INTegrate
| FFT | FFTPhase | SQRT | MAGNify | ABSolute | SQUare | LN | LOG
| EXP | TEN | LOWPass | HIGHpass | BANDpass | AVERage | SMOoth
| ENVELOpe | LINear | MAXimum | MINimum | PEAK | MAXHold | MINHold
| TREND | BTIMing | BState | SERChart}

The :FUNCTION<m>:OPERation command sets the desired waveform math operator, transform, filter or visualization:

- Operators:

- ADD – Source1 + source2.
- SUBTract – Source1 - source2.
- MULTiPLY – Source1 * source2.
- DIVide – Source1 / source2.

Operators perform their function on two analog channel sources.

- Transforms:

- DIFF – Differentiate
- INTegrate – The :FUNCTION<m>:INTegrate:IOFFset command lets you specify a DC offset correction factor. The :FUNCTION<m>:INTegrate:IICondition command lets you sets the Initial Condition of the Integrate math function waveform.
- FFT (magnitude) – Using the Fast Fourier Transform (FFT), this operation displays the magnitudes of the frequency content that makes up the source waveform. The FFT takes the digitized time record of the specified source and transforms it to the frequency domain.

The SPAN, CENTer, VTYPe, and WINDow commands are used for FFT functions. When FFT is selected, the horizontal cursors change from time to frequency (Hz), and the vertical cursors change from volts to decibels or V RMS.

- FFTPhase – Using the Fast Fourier Transform (FFT), this operation shows the phase relationships of the frequency content that makes up the source waveform. The FFT takes the digitized time record of the specified source and transforms it to the frequency domain.

The SPAN, CENTer, VTYPe, and WINDow commands are used for FFT functions. When FFTPhase is selected, the horizontal cursors change from time to frequency (Hz), and the vertical cursors change from volts to degrees or radians.

- LINear – $Ax + B$ – The LINear commands set the gain (A) and offset (B) values for this function.
- SQUare
- SQRT – Square root
- ABSolute – Absolute Value
- LOG – Common Logarithm
- LN – Natural Logarithm
- EXP – Exponential (e^x)
- TEN – Base 10 exponential (10^x)

Transforms operate on a single analog channel source or on lower math functions.

- Filters:
 - LOWPass – Low pass filter – The FREQUENCY:LOWPass command sets the -3 dB cutoff frequency.
 - HIGHpass – High pass filter – The FREQUENCY:HIGHPass command sets the -3 dB cutoff frequency.
 - BANDpass – Band pass filter – The FREQUENCY:BANDpass:CENTer and FREQUENCY:BANDpass:WIDTH commands set the center frequency and width of the filter.
 - AVERage – Averaged value – The AVERage:COUNt command specifies the number of averages.

Unlike acquisition averaging, the math averaging operator can be used to average the data on a single analog input channel or math function.

If acquisition averaging is also used, the analog input channel data is averaged and the math function averages it again. You can use both types of averaging to get a certain number of averages on all waveforms and an increased number of averages on a particular waveform.

Averages are calculated using a "decaying average" approximation, where:

$$\text{next_average} = \text{current_average} + (\text{new_data} - \text{current_average})/N$$

Where N starts at 1 for the first acquisition and increments for each following acquisition until it reaches the selected number of averages, where it holds.

- SMOoth – Smoothing – The resulting math waveform is the selected source with a normalized rectangular (boxcar) FIR filter applied.

The boxcar filter is a moving average of adjacent waveform points, where the number of adjacent points is specified by the SMOoth:POINts command. You can choose an odd number of points, from 3 to 999.

The smoothing operator limits the bandwidth of the source waveform. The smoothing operator can be used, for example, to smooth measurement trend waveforms.

- ENvelope – Envelope – The resulting math waveform is the amplitude envelope for an amplitude modulated (AM) input signal.

This function uses a Hilbert transform to get the real (in-phase, I) and imaginary (quadrature, Q) parts of the input signal and then performs a square root of the sum of the real and imaginary parts to get the demodulated amplitude envelope waveform.

Filters operate on a single analog channel source or on a lower math function.

- Visualizations:
 - MAGNify – Operates on a single analog channel source or on a lower math function.
 - MAXimum – This operator is like the MAXHold operator without the hold. The maximum vertical values found at each horizontal bucket are used to build a waveform.
 - MINimum – This operator is like the MINHold operator without the hold. The minimum vertical values found at each horizontal bucket are used to build a waveform.
 - PEAK – The PEAK operator is like the MAXimum operator minus the MINimum operator. At each horizontal bucket, the minimum vertical values found are subtracted from the maximum vertical values found to build a waveform.
 - MAXHold – Operates on a single analog channel source or on a lower math function. The Max Hold (or Max Envelope) operator records the maximum vertical values found at each horizontal bucket across multiple analysis cycles and uses those values to build a waveform.
 - MINHold – Operates on a single analog channel source or on a lower math function. The Min Hold (or Min Envelope) operator records the minimum vertical values found at each horizontal bucket across multiple analysis cycles and uses those values to build a waveform.
 - TREND – Measurement trend – Operates on a single analog channel source. The TREND:NMEasurement command selects the measurement whose trend you want to measure.
 - BTIMing – Chart logic bus timing – Operates on a bus made up of digital channels. The BUS:YINcrement, BUS:YORigin, and BUS:YUNits commands specify function values.
 - BState – Chart logic bus state – Operates on a bus made up of digital channels. The BUS:YINcrement, BUS:YORigin, and BUS:YUNit commands specify function values. The BUS:CLOCK and BUS:SLOPe commands specify the clock source and edge.

- SERChart – Chart serial signal – Operates on a serial decode that is displaying symbolic data values. For example, CAN or LIN symbolic display or the SENT transfer function display allow charting of serial data values.

NOTE

If a math function is on (see :FUNCTION<m>:DISPlay), changing the operator will cause an automatic vertical scaling of the new math function waveform. If you wait for the operation to complete (with an *OPC? query for example), then query the math function's scale, you can see the vertical scaling that was automatically determined.

Query Syntax :FUNCTION<m>:OPERation?

The :FUNCTION<m>:OPERation? query returns the current operation for the selected function.

Return Format <operation><NL>

```
<operation> ::= {ADD | SUBT | MULT | DIV | INT | DIFF | FFT | FFTP
| Sqrt | MAGN | ABS | SQU | LN | LOG | EXP | TEN | LOWP | HIGH
| BAND | AVER | SMO | ENV | LIN | MAX | MIN | PEAK | MAXH | MINH
| TREN | BTIM | BST | SERC}
```

- See Also**
- "Introduction to :FUNCTION<m> Commands" on page 455
 - ":FUNCTION<m>:DISPlay" on page 463
 - "*OPC (Operation Complete)" on page 222
 - ":FUNCTION<m>:SOURce1" on page 501
 - ":FUNCTION<m>:SOURce2" on page 503
 - ":FUNCTION<m>:INTEgrate:IICondition" on page 482
 - ":FUNCTION<m>:INTEgrate:IOFFset" on page 483
 - ":FUNCTION<m>[:FFT]:SPAN" on page 474
 - ":FUNCTION<m>[:FFT]:CENTer" on page 465
 - ":FUNCTION<m>[:FFT]:PHASe:REFeRence" on page 471
 - ":FUNCTION<m>[:FFT]:VType" on page 476
 - ":FUNCTION<m>[:FFT]:WINDow" on page 477
 - ":FUNCTION<m>:LINear:GAIN" on page 484
 - ":FUNCTION<m>:LINear:OFFSet" on page 485
 - ":FUNCTION<m>:FREQuency:BANDpass:CENTer" on page 478
 - ":FUNCTION<m>:FREQuency:BANDpass:WIDTh" on page 479
 - ":FUNCTION<m>:FREQuency:LOWPass" on page 481
 - ":FUNCTION<m>:FREQuency:HIGHPass" on page 480
 - ":FUNCTION<m>:AVERage:COUNt" on page 456
 - ":FUNCTION<m>:SMOoth:POINts" on page 500

- **":FUNction<m>:TRENd:NMEasurement"** on page 504
- **":FUNction<m>:SERChart:SOURce"** on page 495
- **":FUNction<m>:SERChart:TIME:MODE"** on page 496
- **":FUNction<m>:SERChart:TIME:RANGe"** on page 498
- **":FUNction<m>:SERChart:TIME:OFFSet"** on page 499
- **":FUNction<m>:BUS:YINCrement"** on page 459
- **":FUNction<m>:BUS:YORigin"** on page 460
- **":FUNction<m>:BUS:YUNits"** on page 461
- **":FUNction<m>:BUS:CLOCK"** on page 457
- **":FUNction<m>:BUS:SLOPe"** on page 458

:FUNCTION<m>:RANGe

N (see [page 1650](#))

Command Syntax :FUNCTION<m>:RANGe <range>

<m> ::= 1 to (# math functions) in NR1 format

<range> ::= the full-scale vertical axis value in NR3 format.

The :FUNCTION<m>:RANGe command defines the full-scale vertical axis for the selected function.

Query Syntax :FUNCTION<m>:RANGe?

The :FUNCTION<m>:RANGe? query returns the current full-scale range value for the selected function.

Return Format <range><NL>

<range> ::= the full-scale vertical axis value in NR3 format.

- See Also**
- ["Introduction to :FUNCTION<m> Commands"](#) on page 455
 - [":FUNCTION<m>:SCALE"](#) on page 494

:FUNCTION<m>:REFerence

N (see [page 1650](#))

Command Syntax :FUNCTION<m>:REFerence <level>

<m> ::= 1 to (# math functions) in NR1 format

<level> ::= the current reference level in NR3 format.

The :FUNCTION<m>:REFerence command sets the voltage or vertical value represented at center screen for the selected function. The range of legal values is generally +/-10 times the current scale of the selected function, but will vary by function. If you set the reference level to a value outside of the legal range, the level is automatically set to the nearest legal value.

NOTE

The FUNCTION:REFerence command is equivalent to the :FUNCTION<m>:OFFSet command.

Query Syntax :FUNCTION<m>:REFerence?

The :FUNCTION<m>:REFerence? query outputs the current reference level value for the selected function.

Return Format <level><NL>

<level> ::= the current reference level in NR3 format.

- See Also**
- ["Introduction to :FUNCTION<m> Commands"](#) on page 455
 - [":FUNCTION<m>:OFFSet"](#) on page 486
 - [":FUNCTION<m>:RANGe"](#) on page 492
 - [":FUNCTION<m>:SCALE"](#) on page 494

:FUNCTION<m>:SCALE

N (see [page 1650](#))

Command Syntax :FUNCTION<m>:SCALE <scale value>[<suffix>]

 <m> ::= 1 to (# math functions) in NR1 format

 <scale value> ::= vertical units/div value in NR3 format

 <suffix> ::= {V | dB}

The :FUNCTION<m>:SCALE command sets the vertical scale, or units per division, of the selected function. Legal values for the scale depend on the selected function.

NOTE

Automatic vertical scaling of the math function waveform occurs when the function display is turned from off to on (see :FUNCTION<m>:DISPlay) or, if the function is already on, when the operation is changed (see :FUNCTION<m>:OPERation). If you want to change the math function's vertical scaling, you should do it after the math function display is turned on or after the operation is changed.

Query Syntax :FUNCTION<m>:SCALE?

The :FUNCTION<m>:SCALE? query returns the current scale value for the selected function.

Return Format <scale value><NL>

 <scale value> ::= vertical units/div value in NR3 format

- See Also**
- ["Introduction to :FUNCTION<m> Commands"](#) on page 455
 - [":FUNCTION<m>:DISPlay"](#) on page 463
 - [":FUNCTION<m>:OPERation"](#) on page 487
 - [":FUNCTION<m>:RANGe"](#) on page 492

:FUNCTION<m>:SERChart:SOURce

N (see [page 1650](#))

Command Syntax :FUNCTION<m>:SERChart:SOURce <serialbus>

<m> ::= 1 to (# math functions) in NR1 format

<serialbus> ::= {SBUS<n>}

<n> ::= 1 to (# of serial bus) in NR1 format

The :FUNCTION<m>:SERChart:SOURce command sets the serial bus source for the Chart Serial Signal math function.

Query Syntax :FUNCTION<m>:SERChart:SOURce?

The :FUNCTION<m>:SERChart:SOURce? query returns the serial bus source.

Return Format <serialbus><NL>

<serialbus> ::= {SBUS<n>}

- See Also**
- [":FUNCTION<m>:OPERation"](#) on page 487
 - [":FUNCTION<m>:SERChart:TIME:MODE"](#) on page 496
 - [":FUNCTION<m>:SERChart:TIME:RANGe"](#) on page 498
 - [":FUNCTION<m>:SERChart:TIME:OFFSet"](#) on page 499

:FUNCTION<m>:SERChart:TIME:MODE

N (see [page 1650](#))

Command Syntax :FUNCTION<m>:SERChart:TIME:MODE <value>

<m> ::= 1 to (# math functions) in NR1 format

<value> ::= {MAIN | ROLL}

The :FUNCTION<m>:SERChart:TIME:MODE command sets the time mode for the Chart Serial Signal math function:

- **MAIN** — The signal data values are plotted per acquisition, in the same horizontal time scale as the other waveforms, and all waveform data is time-correlated.

In this mode, the oscilloscope's timebase (time/div) is typically set for a long acquisition time to capture multiple occurrences of the message/frame/packet that contains the signal to be charted.

If the timebase is set to longer than 200 ms/div for high-speed CAN or SENT, undersampling may occur due to limited acquisition memory and reduced sample rate. Timebase settings for LIN can usually be much longer.

- **ROLL** — The signal data values are plotted at the right side of the display and roll across the display for a specified amount of time. This resulting math function waveform is not time-correlated with other waveforms on the display.

Roll mode establishes a second and typically much slower timebase and is useful for charting very slow changing signals (up to 1 hour), such as temperature or pressure.

Signal values are typically under-sampled (not every occurrence of a signal will be plotted). Signal values roll across the display as new signal values are received and plotted.

If charting a CAN or LIN signal in this mode, you should set up the oscilloscope to trigger on the message/frame that contains the signal.

If charting SENT, the oscilloscope should be set up to trigger on the start of any Fast Channel Message (":SBUS<n>:SENT:TRIGger SFCMessage").

In Roll mode, the oscilloscope's main timebase setting is typically set to capture just a few messages.

Query Syntax :FUNCTION<m>:SERChart:TIME:MODE?

The :FUNCTION<m>:SERChart:TIME:MODE? query returns the time mode setting.

Return Format <value><NL>

<value> ::= {MAIN | ROLL}

See Also • [":FUNCTION<m>:OPERation"](#) on page 487

- **":FUNction<m>:SERChart:SOURce"** on page 495
- **":FUNction<m>:SERChart:TIME:RANGe"** on page 498
- **":FUNction<m>:SERChart:TIME:OFFSet"** on page 499
- **":SBUS<n>:SENT:TRIGger"** on page 1110

:FUNCTION<m>:SERChart:TIME:RANGe

N (see [page 1650](#))

Command Syntax :FUNCTION<m>:SERChart:TIME:RANGe <range_value>

<m> ::= 1 to (# math functions) in NR1 format

<range_value> ::= time value in NR3 format

When the ROLL serial chart time mode is selected (see :FUNCTION<m>:SERChart:TIME:MODE), the :FUNCTION<m>:SERChart:TIME:RANGe command sets the full-scale time range value for the Chart Serial Signal math function.

The time range defines the width across the display for the charted serial signal waveform.

When oscilloscope acquisitions are stopped, you can adjust the time offset and time range to scroll and zoom the plotted signal values.

Query Syntax :FUNCTION<m>:SERChart:TIME:RANGe?

The :FUNCTION<m>:SERChart:TIME:RANGe? query returns the time range setting.

Return Format <range_value><NL>

<range_value> ::= time value in NR3 format

- See Also**
- [":FUNCTION<m>:OPERation"](#) on page 487
 - [":FUNCTION<m>:SERChart:SOURce"](#) on page 495
 - [":FUNCTION<m>:SERChart:TIME:MODE"](#) on page 496
 - [":FUNCTION<m>:SERChart:TIME:OFFSet"](#) on page 499

:FUNCTION<m>:SERChart:TIME:OFFSet

N (see [page 1650](#))

Command Syntax :FUNCTION<m>:SERChart:TIME:OFFSet <offset>

<m> ::= 1 to (# math functions) in NR1 format

<offset> ::= time offset value in NR3 format

When the ROLL serial chart time mode is selected (see :FUNCTION<m>:SERChart:TIME:MODE), the :FUNCTION<m>:SERChart:TIME:OFFSet command sets the time offset for the Chart Serial Signal math function.

The time offset shows the time value of the center of the display. When oscilloscope acquisitions are running, the time offset is automatically set to the middle of the time range.

When oscilloscope acquisitions are stopped, you can adjust the time offset and time range to scroll and zoom the plotted signal values.

Query Syntax :FUNCTION<m>:SERChart:TIME:OFFSet?

The :FUNCTION<m>:SERChart:TIME:OFFSet? query returns the time offset setting.

Return Format <offset><NL>

<offset> ::= time offset value in NR3 format

- See Also**
- [":FUNCTION<m>:OPERation"](#) on page 487
 - [":FUNCTION<m>:SERChart:SOURce"](#) on page 495
 - [":FUNCTION<m>:SERChart:TIME:MODE"](#) on page 496
 - [":FUNCTION<m>:SERChart:TIME:RANGe"](#) on page 498

:FUNCTION<m>:SMOoth:POINts

N (see [page 1650](#))

Command Syntax :FUNCTION<m>:SMOoth:POINts <points>

<points> ::= odd integer in NR1 format

When the :FUNCTION<m>:OPERation is SMOoth, the :FUNCTION<m>:SMOoth:POINts command sets the number of smoothing points to use.

You can choose an odd number of points, from 3 up to half of the measurement record or precision analysis record.

Query Syntax :FUNCTION<m>:SMOoth:POINts?

The :FUNCTION<m>:SMOoth:POINts? query returns the number of smoothing points specified.

Return Format <points><NL>

See Also • [":FUNCTION<m>:OPERation"](#) on page 487

:FUNCTION<m>:SOURCE1

N (see [page 1650](#))

Command Syntax :FUNCTION<m>:SOURCE1 <value>

<m> ::= 1 to (# math functions) in NR1 format

<value> ::= {CHANnel<n> | FUNCTION<c> | MATH<c> | WMemory<r> | BUS}

<n> ::= 1 to (# analog channels) in NR1 format

<c> ::= {1 | 2 | 3}, must be lower than <m>

<r> ::= 1 to (# ref waveforms) in NR1 format

 ::= {1 | 2}

The :FUNCTION<m>:SOURCE1 command is used for any :FUNCTION<m>:OPERation selection. This command selects the first source for the operator math functions or the single source for the transform functions, filter functions, or visualization functions.

The FUNCTION<c> or MATH<c> parameters are available for the transform functions, filter functions, and the magnify visualization function (see ["Introduction to :FUNCTION<m> Commands"](#) on page 455) when <c> is lower than <m>.

In other words, higher math functions can operate on lower math functions when using operators other than the simple arithmetic operations (+, -, *, /). For example, if :FUNCTION1:OPERation is a SUBtract of CHANnel1 and CHANnel2, the :FUNCTION2:OPERation could be set up as a FFT operation on the FUNCTION1 source. These are called cascaded math functions.

To cascade math functions, select the lower math function using the :FUNCTION<m>:SOURCE1 command.

When cascading math functions, to get the most accurate results, be sure to vertically scale lower math functions so that their waveforms take up the full screen without being clipped.

The BUS<m> parameter is available for the bus charting visualization functions.

NOTE

Another shorthand notation for SOURCE1 in this command/query (besides SOUR1) is SOUR.

When :FUNCTION<m>:OPERation is TRENd, the :FUNCTION<m>:SOURCE1 command reports error -221, "Settings conflict" because the TRENd function operates on a measurement and not a source waveform.

Query Syntax :FUNCTION<m>:SOURCE1?

The :FUNCTION<m>:SOURce1? query returns the current source1 for function operations.

When :FUNCTION<m>:OPERation is TRENd, the :FUNCTION<m>:SOURce1? query returns the source of the measurement.

Return Format <value><NL>

<value> ::= {CHAN<n> | FUNC<c> | MATH<c> | WMEM<r> | BUS}

- See Also**
- ["Introduction to :FUNCTION<m> Commands"](#) on page 455
 - [":FUNCTION<m>:OPERation"](#) on page 487

:FUNCTION<m>:SOURce2

N (see [page 1650](#))

Command Syntax :FUNCTION<m>:SOURce2 <value>

<m> ::= 1 to (# math functions) in NR1 format

<value> ::= {CHANnel<n> | WMEMory<r> | NONE}

<n> ::= 1 to (# analog channels) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :FUNCTION<m>:SOURce2 command specifies the second source for math operator functions that have two sources. (The :FUNCTION<m>:SOURce1 command specifies the first source.)

The :FUNCTION<m>:SOURce2 setting is not used for the transform functions, filter functions, or visualization functions (except when the measurement trend visualization's measurement requires two sources).

When :FUNCTION<m>:OPERation is TREND, the :FUNCTION<m>:SOURce2 command reports error -221, "Settings conflict" because the TREND function operates on a measurement and not a source waveform.

Query Syntax :FUNCTION<m>:SOURce2?

The :FUNCTION<m>:SOURce2? query returns the currently specified second source for math operations.

When :FUNCTION<m>:OPERation is TREND, the :FUNCTION<m>:SOURce2? query returns the source of the measurement.

Return Format <value><NL>

<value> ::= {CHAN<n> | WMEM<r> | NONE}

- See Also**
- ["Introduction to :FUNCTION<m> Commands"](#) on page 455
 - [":FUNCTION<m>:OPERation"](#) on page 487
 - [":FUNCTION<m>:SOURce1"](#) on page 501

:FUNCTION<m>:TREND:NMEasurement

N (see [page 1650](#))

Command Syntax :FUNCTION<m>:TREND:NMEasurement MEAS<n>

<n> ::= # of installed measurement, from 1 to 10

<m> ::= 1 to (# math functions) in NR1 format

The :FUNCTION<m>:TREND:NMEasurement command selects the measurement whose trend is shown in the math waveform.

There are 10 locations (or slots) that installed measurements can occupy. The MEAS<n> parameter specifies the location of the measurement whose trend you want to analyze.

You can view the trend math function waveform for these installed measurements:

- :MEASure:VAverage, <interval> ::= CYCLE
- :MEASure:VRMS, <type> ::= AC (AC RMS)
- :MEASure:VRATio, <interval> ::= CYCLE
- :MEASure:PERiod
- :MEASure:FREQuency
- :MEASure:PWIDth
- :MEASure:NWIDth
- :MEASure:DUTYcycle
- :MEASure:NDUTy
- :MEASure:RISetime
- :MEASure:FALLtime

Query Syntax :FUNCTION<m>:TREND:NMEasurement?

The :FUNCTION<m>:TREND:NMEasurement? query returns the selected measurement.

If no measurements are installed, the :FUNCTION<m>:TREND:NMEasurement? query will return NONE.

Return Format MEAS<n><NL>

<n> ::= # of installed measurement, from 1 to 10

See Also • [":FUNCTION<m>:OPERation"](#) on page 487

21 :HARDcopy/:HCOPY Commands

Set and query the selection of hardcopy device and formatting options. See ["Introduction to :HARDcopy Commands"](#) on page 506.

Table 108 :HARDcopy/:HCOPY Commands Summary

Command	Query	Options and Query Returns
:HARDcopy:AREA <area> (see page 508)	:HARDcopy:AREA? (see page 508)	<area> ::= SCReen
:HARDcopy:APRinter <active_printer> (see page 509)	:HARDcopy:APRinter? (see page 509)	<active_printer> ::= {<index> <name>} <index> ::= integer index of printer in list <name> ::= name of printer in list
:HARDcopy:FACTors { {0 OFF} {1 ON} } (see page 510)	:HARDcopy:FACTors? (see page 510)	{0 1}
:HARDcopy:FFEEd { {0 OFF} {1 ON} } (see page 511)	:HARDcopy:FFEEd? (see page 511)	{0 1}
:HARDcopy:INKSaver { {0 OFF} {1 ON} } (see page 512)	:HARDcopy:INKSaver? (see page 512)	{0 1}
:HARDcopy:LAYout <layout> (see page 513)	:HARDcopy:LAYout? (see page 513)	<layout> ::= {LANDscape PORTrait}
:HARDcopy:NETWork:ADD Ress <address> (see page 514)	:HARDcopy:NETWork:ADD Ress? (see page 514)	<address> ::= quoted ASCII string
:HARDcopy:NETWork:APP Ly (see page 515)	n/a	n/a

Table 108 :HARDcopy/:HCOPY Commands Summary (continued)

Command	Query	Options and Query Returns
:HARDcopy:NETWork:DOM ain <domain> (see page 516)	:HARDcopy:NETWork:DOM ain? (see page 516)	<domain> ::= quoted ASCII string
:HARDcopy:NETWork:PAS Sword <password> (see page 517)	n/a	<password> ::= quoted ASCII string
:HARDcopy:NETWork:SLO T <slot> (see page 518)	:HARDcopy:NETWork:SLO T? (see page 518)	<slot> ::= {NET0 NET1}
:HARDcopy:NETWork:USE Rname <username> (see page 519)	:HARDcopy:NETWork:USE Rname? (see page 519)	<username> ::= quoted ASCII string
:HARDcopy:PALETTE <palette> (see page 520)	:HARDcopy:PALETTE? (see page 520)	<palette> ::= {COLor GRAYscale NONE}
n/a	:HARDcopy:PRINter:LI ST? (see page 521)	<list> ::= [<printer_spec>] ... [printer_spec] <printer_spec> ::= "<index>,<active>,<name>;" <index> ::= integer index of printer <active> ::= {Y N} <name> ::= name of printer
:HARDcopy:START (see page 522)	n/a	n/a
n/a	:HCOPY:SDUMp:DATA? (see page 523)	<display_data> ::= binary block data in IEEE-488.2 # format.
:HCOPY:SDUMp:FORMat <format> (see page 524)	:HCOPY:SDUMp:FORMat? (see page 524)	<format> ::= {BMP BMP8bit PNG}

Introduction to :HARDcopy Commands

The HARDcopy subsystem provides commands to set and query the selection of hardcopy device and formatting options such as inclusion of instrument settings (FACTors) and generation of formfeed (FFeEd).

:HARDC or :HCOPY are acceptable short forms for :HARDcopy.

Reporting the Setup

Use :HARDcopy? to query setup information for the HARDcopy subsystem.

Return Format

The following is a sample response from the :HARDcopy? query. In this case, the query was issued following the *RST command.

```
:HARD:APR " ";AREA SCR;FACT 0;FFE 0;INKS 1;PAL NONE;LAY PORT
```

:HARDcopy:AREA

N (see [page 1650](#))

Command Syntax :HARDcopy:AREA <area>

<area> ::= SCReen

The :HARDcopy:AREA command controls what part of the display area is printed. Currently, the only legal choice is SCReen.

Query Syntax :HARDcopy:AREA?

The :HARDcopy:AREA? query returns the selected display area.

Return Format <area><NL>

<area> ::= SCR

- See Also**
- ["Introduction to :HARDcopy Commands"](#) on page 506
 - [":HARDcopy:START"](#) on page 522
 - [":HARDcopy:APRinter"](#) on page 509
 - [":HARDcopy:PRINter:LIST"](#) on page 521
 - [":HARDcopy:FACTors"](#) on page 510
 - [":HARDcopy:FFEed"](#) on page 511
 - [":HARDcopy:INKSaver"](#) on page 512
 - [":HARDcopy:LAYout"](#) on page 513
 - [":HARDcopy:PALette"](#) on page 520

:HARDcopy:APRinter

N (see [page 1650](#))

Command Syntax :HARDcopy:APRinter <active_printer>
 <active_printer> ::= {<index> | <name>}
 <index> ::= integer index of printer in list
 <name> ::= name of printer in list

The :HARDcopy:APRinter command sets the active printer.

Query Syntax :HARDcopy:APRinter?

The :HARDcopy:APRinter? query returns the name of the active printer.

Return Format <name><NL>
 <name> ::= name of printer in list

- See Also**
- ["Introduction to :HARDcopy Commands"](#) on page 506
 - [":HARDcopy:PRINTER:LIST"](#) on page 521
 - [":HARDcopy:START"](#) on page 522

:HARDcopy:FACTors

N (see [page 1650](#))

Command Syntax :HARDcopy:FACTors <factors>
 <factors> ::= {{OFF | 0} | {ON | 1}}

The HARDcopy:FACTors command controls whether the scale factors are output on the hardcopy dump.

Query Syntax :HARDcopy:FACTors?

The :HARDcopy:FACTors? query returns a flag indicating whether oscilloscope instrument settings are output on the hardcopy.

Return Format <factors><NL>
 <factors> ::= {0 | 1}

- See Also**
- ["Introduction to :HARDcopy Commands"](#) on page 506
 - [":HARDcopy:START"](#) on page 522
 - [":HARDcopy:FFEd"](#) on page 511
 - [":HARDcopy:INKSaver"](#) on page 512
 - [":HARDcopy:LAYout"](#) on page 513
 - [":HARDcopy:PALette"](#) on page 520

:HARDcopy:FFeEd

N (see [page 1650](#))

Command Syntax :HARDcopy:FFeEd <ffeed>

<ffeed> ::= {{OFF | 0} | {ON | 1}}

The :HARDcopy:FFeEd command controls whether a formfeed is output between the screen image and factors of a hardcopy dump.

Query Syntax :HARDcopy:FFeEd?

The :HARDcopy:FFeEd? query returns a flag indicating whether a formfeed is output at the end of the hardcopy dump.

Return Format <ffeed><NL>

<ffeed> ::= {0 | 1}

- See Also**
- ["Introduction to :HARDcopy Commands"](#) on page 506
 - [":HARDcopy:START"](#) on page 522
 - [":HARDcopy:FACTors"](#) on page 510
 - [":HARDcopy:INKSaver"](#) on page 512
 - [":HARDcopy:LAYout"](#) on page 513
 - [":HARDcopy:PALette"](#) on page 520

:HARDcopy:INKSaver

N (see [page 1650](#))

Command Syntax :HARDcopy:INKSaver <value>
<value> ::= {{OFF | 0} | {ON | 1}}

The :HARDcopy:INKSaver command controls whether the graticule colors are inverted or not.

Query Syntax :HARDcopy:INKSaver?

The :HARDcopy:INKSaver? query returns a flag indicating whether graticule colors are inverted or not.

Return Format <value><NL>
<value> ::= {0 | 1}

- See Also**
- ["Introduction to :HARDcopy Commands"](#) on page 506
 - [":HARDcopy:START"](#) on page 522
 - [":HARDcopy:FACTors"](#) on page 510
 - [":HARDcopy:FFEed"](#) on page 511
 - [":HARDcopy:LAYout"](#) on page 513
 - [":HARDcopy:PALette"](#) on page 520

:HARDcopy:LAYout

N (see [page 1650](#))

Command Syntax :HARDcopy:LAYout <layout>

<layout> ::= {LANDscape | PORTrait}

The :HARDcopy:LAYout command sets the hardcopy layout mode.

Query Syntax :HARDcopy:LAYout?

The :HARDcopy:LAYout? query returns the selected hardcopy layout mode.

Return Format <layout><NL>

<layout> ::= {LAND | PORT}

- See Also**
- ["Introduction to :HARDcopy Commands"](#) on page 506
 - [":HARDcopy:START"](#) on page 522
 - [":HARDcopy:FACTors"](#) on page 510
 - [":HARDcopy:PALette"](#) on page 520
 - [":HARDcopy:FFEed"](#) on page 511
 - [":HARDcopy:INKSaver"](#) on page 512

:HARDcopy:NETWork:ADDResS

N (see [page 1650](#))

Command Syntax :HARDcopy:NETWork:ADDResS <address>

<address> ::= quoted ASCII string

The :HARDcopy:NETWork:ADDResS command sets the address for a network printer slot. The address is the server/computer name and the printer's share name in the \\server\share format.

The network printer slot is selected by the :HARDcopy:NETWork:SLOT command.

To apply the entered address, use the :HARDcopy:NETWork:APPLY command.

Query Syntax :HARDcopy:NETWork:ADDResS?

The :HARDcopy:NETWork:ADDResS? query returns the specified address for the currently selected network printer slot.

Return Format <address><NL>

<address> ::= quoted ASCII string

- See Also**
- ["Introduction to :HARDcopy Commands"](#) on page 506
 - [":HARDcopy:NETWork:SLOT"](#) on page 518
 - [":HARDcopy:NETWork:APPLY"](#) on page 515
 - [":HARDcopy:NETWork:DOMain"](#) on page 516
 - [":HARDcopy:NETWork:USERname"](#) on page 519
 - [":HARDcopy:NETWork:PASSword"](#) on page 517

:HARDcopy:NETWork:APPLy

N (see [page 1650](#))

Command Syntax :HARDcopy:NETWork:APPLy

The :HARDcopy:NETWork:APPLy command applies the network printer settings and makes the printer connection.

- See Also**
- ["Introduction to :HARDcopy Commands"](#) on page 506
 - [":HARDcopy:NETWork:SLOT"](#) on page 518
 - [":HARDcopy:NETWork:ADDRes"](#) on page 514
 - [":HARDcopy:NETWork:DOMain"](#) on page 516
 - [":HARDcopy:NETWork:USERname"](#) on page 519
 - [":HARDcopy:NETWork:PASSword"](#) on page 517

:HARDcopy:NETWork:DOMain

N (see [page 1650](#))

Command Syntax :HARDcopy:NETWork:DOMain <domain>

<domain> ::= quoted ASCII string

The :HARDcopy:NETWork:DOMain command sets the Windows network domain name.

The domain name setting is a common setting for both network printer slots.

Query Syntax :HARDcopy:NETWork:DOMain?

The :HARDcopy:NETWork:DOMain? query returns the current Windows network domain name.

Return Format <domain><NL>

<domain> ::= quoted ASCII string

- See Also**
- ["Introduction to :HARDcopy Commands"](#) on page 506
 - [":HARDcopy:NETWork:SLOT"](#) on page 518
 - [":HARDcopy:NETWork:APPLY"](#) on page 515
 - [":HARDcopy:NETWork:ADDRESS"](#) on page 514
 - [":HARDcopy:NETWork:USERNAME"](#) on page 519
 - [":HARDcopy:NETWork:PASSWORD"](#) on page 517

:HARDcopy:NETWork:PASSword

N (see [page 1650](#))

Command Syntax :HARDcopy:NETWork:PASSword <password>

<password> ::= quoted ASCII string

The :HARDcopy:NETWork:PASSword command sets the password for the specified Windows network domain and user name.

The password setting is a common setting for both network printer slots.

- See Also**
- ["Introduction to :HARDcopy Commands"](#) on page 506
 - [":HARDcopy:NETWork:USERname"](#) on page 519
 - [":HARDcopy:NETWork:DOMain"](#) on page 516
 - [":HARDcopy:NETWork:SLOT"](#) on page 518
 - [":HARDcopy:NETWork:APPLY"](#) on page 515
 - [":HARDcopy:NETWork:ADDRESS"](#) on page 514

:HARDcopy:NETWork:SLOT

N (see [page 1650](#))

Command Syntax :HARDcopy:NETWork:SLOT <slot>

<slot> ::= {NET0 | NET1}

The :HARDcopy:NETWork:SLOT command selects the network printer slot used for the address and apply commands. There are two network printer slots to choose from.

Query Syntax :HARDcopy:NETWork:SLOT?

The :HARDcopy:NETWork:SLOT? query returns the currently selected network printer slot.

Return Format <slot><NL>

<slot> ::= {NET0 | NET1}

- See Also**
- ["Introduction to :HARDcopy Commands"](#) on page 506
 - [":HARDcopy:NETWork:APPLY"](#) on page 515
 - [":HARDcopy:NETWork:ADDRESS"](#) on page 514
 - [":HARDcopy:NETWork:DOMAIN"](#) on page 516
 - [":HARDcopy:NETWork:USERNAME"](#) on page 519
 - [":HARDcopy:NETWork:PASSWORD"](#) on page 517

:HARDcopy:NETWork:USERname

N (see [page 1650](#))

Command Syntax :HARDcopy:NETWork:USERname <username>

<username> ::= quoted ASCII string

The :HARDcopy:NETWork:USERname command sets the user name to use when connecting to the Windows network domain.

The user name setting is a common setting for both network printer slots.

Query Syntax :HARDcopy:NETWork:USERname?

The :HARDcopy:NETWork:USERname? query returns the currently set user name.

Return Format <username><NL>

<username> ::= quoted ASCII string

- See Also**
- ["Introduction to :HARDcopy Commands"](#) on page 506
 - [":HARDcopy:NETWork:DOMain"](#) on page 516
 - [":HARDcopy:NETWork:PASSword"](#) on page 517
 - [":HARDcopy:NETWork:SLOT"](#) on page 518
 - [":HARDcopy:NETWork:APPLY"](#) on page 515
 - [":HARDcopy:NETWork:ADDRESS"](#) on page 514

:HARDcopy:PALETTE

N (see [page 1650](#))

Command Syntax :HARDcopy:PALETTE <palette>
<palette> ::= {COLOR | GRAYscale | NONE}

The :HARDcopy:PALETTE command sets the hardcopy palette color.

The oscilloscope's print driver cannot print color images to color laser printers, so the COLOR option is not available when connected to laser printers.

Query Syntax :HARDcopy:PALETTE?

The :HARDcopy:PALETTE? query returns the selected hardcopy palette color.

Return Format <palette><NL>
<palette> ::= {COL | GRAY | NONE}

- See Also**
- ["Introduction to :HARDcopy Commands"](#) on page 506
 - [":HARDcopy:START"](#) on page 522
 - [":HARDcopy:FACTors"](#) on page 510
 - [":HARDcopy:LAYout"](#) on page 513
 - [":HARDcopy:FFeed"](#) on page 511
 - [":HARDcopy:INKSaver"](#) on page 512

:HARDcopy:PRINter:LIST

N (see [page 1650](#))

Query Syntax :HARDcopy:PRINter:LIST?

The :HARDcopy:PRINter:LIST? query returns a list of available printers. The list can be empty.

Return Format <list><NL>

<list> ::= [<printer_spec>] ... [<printer_spec>]

<printer_spec> ::= "<index>,<active>,<name>;"

<index> ::= integer index of printer

<active> ::= {Y | N}

<name> ::= name of printer (for example "DESKJET 950C")

- See Also**
- ["Introduction to :HARDcopy Commands"](#) on page 506
 - [":HARDcopy:APRinter"](#) on page 509
 - [":HARDcopy:START"](#) on page 522

:HARDcopy:START

N (see [page 1650](#))

Command Syntax :HARDcopy:START

The :HARDcopy:START command starts a print job.

- See Also**
- ["Introduction to :HARDcopy Commands"](#) on page 506
 - [":HARDcopy:APRinter"](#) on page 509
 - [":HARDcopy:PRINter:LIST"](#) on page 521
 - [":HARDcopy:FACTors"](#) on page 510
 - [":HARDcopy:FFEed"](#) on page 511
 - [":HARDcopy:INKSaver"](#) on page 512
 - [":HARDcopy:LAYout"](#) on page 513
 - [":HARDcopy:PALette"](#) on page 520

:HCOPY:SDUMp:DATA

N (see [page 1650](#))

Query Syntax :HCOPY:SDUMp:DATA? [<format>]

<format> ::= {PNG | BMP | BMP8bit}

The :HCOPY:SDUMp:DATA? query reads and returns screen image data. You can choose 24-bit BMP, 8-bit BMP8bit, or 24-bit PNG formats.

Note that the returned image is also affected by the :HARDcopy:INKSaver command, which is ON by default and returns an inverted image. To get a non-inverted image, send the ":HARDcopy:INKSaver OFF" command before the DATA? query.

In addition to the <format> option of this query, the screen image data format can also be set by the :HCOPY:SDUMp:FORMat command or the front panel's **[Save/Recall] > Save > Format** softkey, when an image format (instead of a setup or data format) is selected.

If no <format> option is specified with this query, the screen image data is returned in the currently selected format. After a *RST (factory default) command, the PNG format is selected by default.

If the <format> option is specified with this query, the format setting will affect the front panel's **[Save/Recall] > Save > Format** softkey setting if the **Format** softkey currently selects an image format (instead of a setup or data format).

Screen image data is returned in the IEEE-488.2 # binary block data format.

Return Format <display_data><NL>

<display_data> ::= binary block data in IEEE-488.2 # format.

- See Also**
- [":HCOPY:SDUMp:FORMat"](#) on page 524
 - [":DISPlay:DATA"](#) on page 402
 - [":HARDcopy:INKSaver"](#) on page 512

:HCOPY:SDUMp:FORMat

N (see [page 1650](#))

Command Syntax :HCOPY:SDUMp:FORMat <format>

<format> ::= {PNG | BMP | BMP8bit}

The :HCOPY:SDUMp:FORMat command specifies the format for screen image data: 24-bit PNG, 24-bit BMP, or 8-bit BMP8bit.

Note that the screen image data is also affected by the :HARDcopy:INKSaver command, which is ON by default and results in an inverted image. To get a non-inverted image, send the ":HARDcopy:INKSaver OFF" command before querying the screen image data.

The :HCOPY:SDUMp:FORMat setting will persist when cycling power but will be reset to PNG after a *RST (factory default) command.

The :HCOPY:SDUMp:FORMat setting will affect the front panel's **[Save/Recall] > Save > Format** softkey setting if the **Format** softkey currently selects an image format (instead of a setup or data format).

Query Syntax :HCOPY:SDUMp:FORMat?

The :HCOPY:SDUMp:FORMat? query returns the specified screen image data format.

The screen image data format can be set by the :HCOPY:SDUMp:FORMat command or the front panel's **[Save/Recall] > Save > Format** softkey, when it selects an image format (instead of a setup or data format).

Return Format <format><NL>

<format> ::= {PNG | BMP | BMP8}

- See Also**
- [":HCOPY:SDUMp:DATA"](#) on page 523
 - [":DISPlay:DATA"](#) on page 402
 - [":HARDcopy:INKSaver"](#) on page 512
 - ["*RST \(Reset\)"](#) on page 226

22 :HISTogram Commands

These commands control the histogram analysis feature.

The histogram analysis feature provides a statistical view of a waveform or a measurement's results.

Waveform histograms show the number of times a waveform crosses into (or hits) a row or column of a user-defined window. For vertical histograms, the window is divided into rows. For horizontal histograms, the window is divided into columns.

Measurement histograms show the distribution of a measurement's results similar to ordinary measurement statistics.

As waveforms are acquired and displayed, or as measurements are made, a counter database accumulates the total number of hits (or measurements), and the histogram data is displayed as a bar graph on the graticule.

When the oscilloscope is stopped, that counter database remains intact until the acquired data display is modified, usually horizontally or vertically. At that time, the database is reset and the last displayed acquisition is used to start a new database.

Table 109 :HISTogram Commands Summary

Command	Query	Options and Query Returns
:HISTogram:AXIS <axis> (see page 528)	:HISTogram:AXIS? (see page 528)	<axis> ::= {VERTical HORizontal}
:HISTogram:DISPlay { {0 OFF} {1 ON} } (see page 529)	:HISTogram:DISPlay? (see page 529)	{0 1}
:HISTogram:MEASurement MEAS<n> (see page 530)	:HISTogram:MEASurement? (see page 530)	<n> ::= # of installed measurement, from 1 to 10
:HISTogram:MODE <mode> (see page 531)	:HISTogram:MODE? (see page 531)	<mode> ::= {OFF WAVEform MEASurement}
:HISTogram:RESet (see page 532)	n/a	n/a

Table 109 :HISTogram Commands Summary (continued)

Command	Query	Options and Query Returns
:HISTogram:SIZE <size> (see page 533)	:HISTogram:SIZE? (see page 533)	<size> ::= 1.0 to 5.0 for vertical histograms, 1.0 to 4.0 for horizontal histograms, in NR3 format
:HISTogram:TYPE <type> (see page 534)	:HISTogram:TYPE? (see page 534)	<type> ::= {VERTical HORizontal MEASurement}
:HISTogram:WINDow:BLIMit <limit> (see page 535)	:HISTogram:WINDow:BLIMit? (see page 535)	<limit> ::= bottom value in NR3 format
:HISTogram:WINDow:LLIMit <limit> (see page 536)	:HISTogram:WINDow:LLIMit? (see page 536)	<limit> ::= left value in NR3 format
:HISTogram:WINDow:RLIMit <limit> (see page 537)	:HISTogram:WINDow:RLIMit? (see page 537)	<limit> ::= right value in NR3 format
:HISTogram:WINDow:SOURce <source> (see page 538)	:HISTogram:WINDow:SOURce? (see page 538)	<source> ::= {CHANnel<n> FUNCTION<m> MATH<m> WMEMory<r> CGRade} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format
:HISTogram:WINDow:TLMit <limit> (see page 539)	:HISTogram:WINDow:TLMit? (see page 539)	<limit> ::= top value in NR3 format

Histogram results statistics can be queried with these commands:

- **":MEASure:HISTogram:HITS"** on page 630 – The sum of all hits in all bins (buckets) in the histogram.
- **":MEASure:HISTogram:PEAK"** on page 639 – Maximum number of hits in any single bin.
- **":MEASure:HISTogram:MAXimum"** on page 634 – The value corresponding to the maximum bin that has any hits.
- **":MEASure:HISTogram:MINimum"** on page 637 – The value corresponding to the minimum bin that has any hits.
- **":MEASure:HISTogram:PPEak"** on page 640 – Delta between the Max and Min values.

- **":MEASure:HISTogram:MEAN"** on page 635 – Mean value of the histogram.
- **":MEASure:HISTogram:MEDian"** on page 636 – Median value of the histogram.
- **":MEASure:HISTogram:MODE"** on page 638 – Mode value of the histogram.
- **":MEASure:HISTogram:BWIDth"** on page 629 – The width of each bin (bucket) in the histogram.
- **":MEASure:HISTogram:SDEViation"** on page 641 – Standard deviation of the histogram values.
- **":MEASure:HISTogram:M1S"** on page 631 – Percentage of histogram hits that lie within ± 1 Std Dev of the mean.
- **":MEASure:HISTogram:M2S"** on page 632 – Percentage of histogram hits that lie within ± 2 Std Dev of the mean.
- **":MEASure:HISTogram:M3S"** on page 633 – Percentage of histogram hits that lie within ± 3 Std Dev of the mean.

Histogram Interaction With Other Oscilloscope Features

When you enable a histogram, horizontal zoom, XY time mode, or roll time mode are automatically disabled.

When using segmented memory acquisitions, histograms perform similarly to math waveforms and measurements. The histogram is reset at the start of a segmented acquisition and all segments are histogrammed at the end of the acquisition.

Because it is not possible to histogram every waveform, be aware that persistence can show a peak or glitch that does not show up in the histogram data.

Histograms do not wait for the specified number of averaged acquisitions to be acquired before starting analysis, so intermediate results could be erratic.

:HISTogram:AXIS

N (see [page 1650](#))

Command Syntax :HISTogram:AXIS <axis>
 <axis> ::= {VERTical | HORizontal}

When the :HISTogram:MODE is WAVEform, the :HISTogram:AXIS command specifies the orientation of the histogram:

- HORizontal – the limits window is divided into columns, and the number of hits in each column is displayed in a histogram bar graph at the bottom of the graticule.
- VERTical – the limits window is divided into rows, and the number of hits in each row is displayed in a histogram bar graph at the left of the graticule.

(When the :HISTogram:MODE is MEASurement, histograms are always displayed horizontally at the bottom of the graticule.)

Query Syntax :HISTogram:AXIS?

The :HISTogram:AXIS? query returns the currently specified type of waveform histogram.

Return Format <axis><NL>
 <axis> ::= {VERT | HOR}

- See Also**
- [":HISTogram:DISPlay"](#) on page 529
 - [":HISTogram:TYPE"](#) on page 534
 - [":HISTogram:MODE"](#) on page 531
 - [":HISTogram:RESet"](#) on page 532
 - [":HISTogram:SIZE"](#) on page 533
 - [":HISTogram:WINDow:BLIMit"](#) on page 535
 - [":HISTogram:WINDow:LLIMit"](#) on page 536
 - [":HISTogram:WINDow:RLIMit"](#) on page 537
 - [":HISTogram:WINDow:SOURce"](#) on page 538
 - [":HISTogram:WINDow:TLIMit"](#) on page 539

:HISTogram:DISPlay

N (see [page 1650](#))

Command Syntax :HISTogram:DISPlay {{0 | OFF} | {1 | ON}}

The :HISTogram:DISPlay command turns the histogram display on or off.

Query Syntax :HISTogram:DISPlay?

The :HISTogram:DISPlay? query returns the current display setting for the histogram.

Return Format <off_on><NL>

{0 | 1}

- See Also**
- [":HISTogram:TYPE"](#) on page 534
 - [":HISTogram:RESet"](#) on page 532
 - [":HISTogram:AXIS"](#) on page 528
 - [":HISTogram:MEASurement"](#) on page 530
 - [":HISTogram:MODE"](#) on page 531
 - [":HISTogram:SIZE"](#) on page 533
 - [":HISTogram:WINDow:BLIMit"](#) on page 535
 - [":HISTogram:WINDow:LLIMit"](#) on page 536
 - [":HISTogram:WINDow:RLIMit"](#) on page 537
 - [":HISTogram:WINDow:SOURce"](#) on page 538
 - [":HISTogram:WINDow:TLIMit"](#) on page 539

:HISTogram:MEASurement

N (see [page 1650](#))

Command Syntax :HISTogram:MEASurement MEAS<n>

<n> ::= # of installed measurement, from 1 to 10

When the :HISTogram:MODE is MEASurement, the :HISTogram:MEASurement command selects the measurement to analyze.

There are 10 locations (or slots) that installed measurements can occupy. The MEAS<n> parameter specifies the location of the measurement you want to analyze.

Query Syntax :HISTogram:MEASurement?

The :HISTogram:MEASurement? query returns the location of the measurement currently selected for histogram analysis.

Return Format MEAS<n><NL>

<n> ::= # of installed measurement, from 1 to 10

- See Also**
- [":HISTogram:DISPlay"](#) on page 529
 - [":HISTogram:TYPE"](#) on page 534
 - [":HISTogram:MODE"](#) on page 531
 - [":HISTogram:RESet"](#) on page 532
 - [":HISTogram:SIZE"](#) on page 533

:HISTogram:MODE

N (see [page 1650](#))

Command Syntax :HISTogram:MODE <mode>
 <mode> ::= {OFF | WAVEform | MEASurement}

The :HISTogram:MODE command selects the type of histogram or disables the histogram analysis feature:

- OFF – disables the histogram analysis feature.
- WAVEform – shows the number of times a waveform crosses into (or hits) a row or column of a user-defined window. For vertical histograms, the window is divided into rows. For horizontal histograms, the window is divided into columns. Use the :HISTogram:AXIS command to specify a vertical or horizontal histogram.
- MEASurement – shows the distribution of a measurement's results just like ordinary measurement statistics.

Query Syntax :HISTogram:MODE?

The :HISTogram:MODE? query returns the currently set histogram mode.

Return Format <mode><NL>
 <mode> ::= {OFF | WAV | MEAS}

- See Also**
- [":HISTogram:DISPlay"](#) on page 529
 - [":HISTogram:TYPE"](#) on page 534
 - [":HISTogram:AXIS"](#) on page 528
 - [":HISTogram:MEASurement"](#) on page 530
 - [":HISTogram:RESet"](#) on page 532
 - [":HISTogram:SIZE"](#) on page 533
 - [":HISTogram:WINDow:BLIMit"](#) on page 535
 - [":HISTogram:WINDow:LLIMit"](#) on page 536
 - [":HISTogram:WINDow:RLIMit"](#) on page 537
 - [":HISTogram:WINDow:SOURce"](#) on page 538
 - [":HISTogram:WINDow:TLIMit"](#) on page 539

:HISTogram:RESet

N (see [page 1650](#))

Command Syntax :HISTogram:RESet

The :HISTogram:RESet command zeros the histogram counters.

- See Also**
- [":HISTogram:DISPlay"](#) on page 529
 - [":HISTogram:TYPE"](#) on page 534
 - [":HISTogram:AXIS"](#) on page 528
 - [":HISTogram:MEASurement"](#) on page 530
 - [":HISTogram:MODE"](#) on page 531
 - [":HISTogram:SIZE"](#) on page 533
 - [":HISTogram:WINDow:BLIMit"](#) on page 535
 - [":HISTogram:WINDow:LLIMit"](#) on page 536
 - [":HISTogram:WINDow:RLIMit"](#) on page 537
 - [":HISTogram:WINDow:SOURce"](#) on page 538
 - [":HISTogram:WINDow:TLIMit"](#) on page 539

:HISTogram:SIZE

N (see [page 1650](#))

Command Syntax :HISTogram:SIZE <size>

<size> ::= 1.0 to 5.0 for vertical histograms, 1.0 to 4.0 for horizontal histograms, in NR3 format

The :HISTogram:SIZE command specifies the number of divisions the histogram bar graph should use:

- For horizontal histograms, you can choose whole and half divisions from 1 to 4.
- For vertical histograms, you can choose whole and half divisions from 1 to 5.

Query Syntax :HISTogram:SIZE?

The :HISTogram:SIZE? query returns the number of divisions currently used for the histogram bar graph.

Return Format <size><NL>

<size> ::= 1.0 to 5.0 for vertical histograms, 1.0 to 4.0 for horizontal histograms, in NR3 format

- See Also**
- [":HISTogram:DISPlay"](#) on page 529
 - [":HISTogram:TYPE"](#) on page 534
 - [":HISTogram:AXIS"](#) on page 528
 - [":HISTogram:MEASurement"](#) on page 530
 - [":HISTogram:MODE"](#) on page 531
 - [":HISTogram:RESet"](#) on page 532
 - [":HISTogram:WINDow:BLIMit"](#) on page 535
 - [":HISTogram:WINDow:LLIMit"](#) on page 536
 - [":HISTogram:WINDow:RLIMit"](#) on page 537
 - [":HISTogram:WINDow:SOURce"](#) on page 538
 - [":HISTogram:WINDow:TLIMit"](#) on page 539

:HISTogram:TYPE

 (see [page 1650](#))

Command Syntax :HISTogram:TYPE <type>
 <type> ::= {VERTical | HORizontal | MEASurement}

The :HISTogram:TYPE command selects either a waveform histogram (horizontal or vertical) or a measurement histogram:

- **HORizontal** – the limits window is divided into columns, and the number of hits in each column is displayed in a histogram bar graph at the bottom of the graticule.
- **VERTical** – the limits window is divided into rows, and the number of hits in each row is displayed in a histogram bar graph at the left of the graticule.
- **MEASurement** – shows the distribution of a measurement's results similar to ordinary measurement statistics.

Query Syntax :HISTogram:TYPE?

The :HISTogram:TYPE? query returns the currently selected histogram type.

Return Format <type><NL>
 <type> ::= {VERT | HOR | MEAS}

- See Also**
- [":HISTogram:DISPlay"](#) on page 529
 - [":HISTogram:RESet"](#) on page 532
 - [":HISTogram:AXIS"](#) on page 528
 - [":HISTogram:MEASurement"](#) on page 530
 - [":HISTogram:MODE"](#) on page 531
 - [":HISTogram:SIZE"](#) on page 533
 - [":HISTogram:WINDow:BLIMit"](#) on page 535
 - [":HISTogram:WINDow:LLIMit"](#) on page 536
 - [":HISTogram:WINDow:RLIMit"](#) on page 537
 - [":HISTogram:WINDow:SOURce"](#) on page 538
 - [":HISTogram:WINDow:TLIMit"](#) on page 539

:HISTogram:WINDow:BLIMit

N (see [page 1650](#))

Command Syntax :HISTogram:WINDow:BLIMit <limit>

<limit> ::= bottom value in NR3 format

When the :HISTogram:MODE is WAVEform, the :HISTogram:WINDow:BLIMit command specifies the histogram limits window's bottom boundary.

The units are the same as the input waveform's vertical units. The input waveform is specified using the :HISTogram:WINDow:SOURce command.

Query Syntax :HISTogram:WINDow:BLIMit?

The :HISTogram:WINDow:BLIMit? query returns the histogram limits window's bottom boundary.

Return Format <limit><NL>

<limit> ::= bottom value in NR3 format

- See Also**
- [":HISTogram:DISPlay"](#) on page 529
 - [":HISTogram:TYPE"](#) on page 534
 - [":HISTogram:AXIS"](#) on page 528
 - [":HISTogram:MODE"](#) on page 531
 - [":HISTogram:RESet"](#) on page 532
 - [":HISTogram:SIZE"](#) on page 533
 - [":HISTogram:WINDow:LLIMit"](#) on page 536
 - [":HISTogram:WINDow:RLIMit"](#) on page 537
 - [":HISTogram:WINDow:SOURce"](#) on page 538
 - [":HISTogram:WINDow:TLIMit"](#) on page 539

:HISTogram:WINDow:LLIMit

N (see [page 1650](#))

Command Syntax :HISTogram:WINDow:LLIMit <limit>

<limit> ::= left value in NR3 format

When the :HISTogram:MODE is WAVEform, the :HISTogram:WINDow:LLIMit command specifies the histogram limits window's left boundary.

The units are the same as the input waveform's horizontal units. The input waveform is specified using the :HISTogram:WINDow:SOURce command.

Query Syntax :HISTogram:WINDow:LLIMit?

The :HISTogram:WINDow:LLIMit? query returns the histogram limits window's left boundary.

Return Format <limit><NL>

<limit> ::= left value in NR3 format

- See Also**
- [":HISTogram:DISPlay"](#) on page 529
 - [":HISTogram:TYPE"](#) on page 534
 - [":HISTogram:AXIS"](#) on page 528
 - [":HISTogram:MODE"](#) on page 531
 - [":HISTogram:RESet"](#) on page 532
 - [":HISTogram:SIZE"](#) on page 533
 - [":HISTogram:WINDow:BLIMit"](#) on page 535
 - [":HISTogram:WINDow:RLIMit"](#) on page 537
 - [":HISTogram:WINDow:SOURce"](#) on page 538
 - [":HISTogram:WINDow:TLIMit"](#) on page 539

:HISTogram:WINDow:RLIMit

N (see [page 1650](#))

Command Syntax :HISTogram:WINDow:RLIMit <limit>

<limit> ::= right value in NR3 format

When the :HISTogram:MODE is WAVEform, the :HISTogram:WINDow:RLIMit command specifies the histogram limits window's right boundary.

The units are the same as the input waveform's horizontal units. The input waveform is specified using the :HISTogram:WINDow:SOURce command.

Query Syntax :HISTogram:WINDow:RLIMit?

The :HISTogram:WINDow:RLIMit? query returns the histogram limits window's right boundary.

Return Format <limit><NL>

<limit> ::= right value in NR3 format

- See Also**
- [":HISTogram:DISPlay"](#) on page 529
 - [":HISTogram:TYPE"](#) on page 534
 - [":HISTogram:AXIS"](#) on page 528
 - [":HISTogram:MODE"](#) on page 531
 - [":HISTogram:RESet"](#) on page 532
 - [":HISTogram:SIZE"](#) on page 533
 - [":HISTogram:WINDow:BLIMit"](#) on page 535
 - [":HISTogram:WINDow:LLIMit"](#) on page 536
 - [":HISTogram:WINDow:SOURce"](#) on page 538
 - [":HISTogram:WINDow:TLIMit"](#) on page 539

:HISTogram:WINDow:SOURce

N (see [page 1650](#))

Command Syntax :HISTogram:WINDow:SOURce <source>

<source> ::= {CHANnel<n> | FUNCTION<m> | MATH<m> | WMEMory<r> | CGRade}

<n> ::= 1 to (# analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

When the :HISTogram:MODE is WAVEform, the :HISTogram:WINDow:SOURce command specifies the waveform that is used as the histogram source.

Query Syntax :HISTogram:WINDow:SOURce?

The :HISTogram:WINDow:SOURce? query returns the currently specified waveform histogram source.

Return Format <source><NL>

- See Also**
- [":HISTogram:DISPlay"](#) on page 529
 - [":HISTogram:TYPE"](#) on page 534
 - [":HISTogram:AXIS"](#) on page 528
 - [":HISTogram:MODE"](#) on page 531
 - [":HISTogram:RESet"](#) on page 532
 - [":HISTogram:SIZE"](#) on page 533
 - [":HISTogram:WINDow:BLIMit"](#) on page 535
 - [":HISTogram:WINDow:LLIMit"](#) on page 536
 - [":HISTogram:WINDow:RLIMit"](#) on page 537
 - [":HISTogram:WINDow:TLIMit"](#) on page 539

:HISTogram:WINDow:TLIMit

N (see [page 1650](#))

Command Syntax :HISTogram:WINDow:TLIMit <limit>

<limit> ::= top value in NR3 format

When the :HISTogram:MODE is WAVEform, the :HISTogram:WINDow:TLIMit command specifies the histogram limits window's top boundary.

The units are the same as the input waveform's vertical units. The input waveform is specified using the :HISTogram:WINDow:SOURce command.

Query Syntax :HISTogram:WINDow:TLIMit?

The :HISTogram:WINDow:TLIMit? query returns the histogram limits window's top boundary.

Return Format <limit><NL>

<limit> ::= top value in NR3 format

- See Also**
- [":HISTogram:DISPlay"](#) on page 529
 - [":HISTogram:TYPE"](#) on page 534
 - [":HISTogram:AXIS"](#) on page 528
 - [":HISTogram:MODE"](#) on page 531
 - [":HISTogram:RESet"](#) on page 532
 - [":HISTogram:SIZE"](#) on page 533
 - [":HISTogram:WINDow:BLIMit"](#) on page 535
 - [":HISTogram:WINDow:LLIMit"](#) on page 536
 - [":HISTogram:WINDow:RLIMit"](#) on page 537
 - [":HISTogram:WINDow:SOURce"](#) on page 538

23 :LISTer Commands

Table 110 :LISTer Commands Summary

Command	Query	Options and Query Returns
n/a	:LISTer:DATA? (see page 542)	<binary_block> ::= comma-separated data with newlines at the end of each row
:LISTer:DISPlay { {OFF 0} {SBUS1 ON 1} {SBUS2 2} ALL} (see page 543)	:LISTer:DISPlay? (see page 543)	{OFF SBUS1 SBUS2 ALL}
:LISTer:REFeRence <time_ref> (see page 544)	:LISTer:REFeRence? (see page 544)	<time_ref> ::= {TRIGger PREVIOUS}

Introduction to :LISTer Commands The LISTer subsystem is used to turn on/off the serial decode Lister display and return data from the Lister display.

:LISTer:DATA

N (see [page 1650](#))

Query Syntax :LISTer:DATA?

The :LISTer:DATA? query returns the lister data.

Return Format <binary_block><NL>

<binary_block> ::= comma-separated data with newlines at the
end of each row

- See Also**
- ["Introduction to :LISTer Commands"](#) on page 541
 - [":LISTer:DISPlay"](#) on page 543
 - ["Definite-Length Block Response Data"](#) on page 207

:LISTer:DISPlay

N (see [page 1650](#))

Command Syntax :LISTer:DISPlay <value>

<value> ::= {{OFF | 0} | {SBUS1 | ON | 1} | {SBUS2 | 2} | ALL}

The :LISTer:DISPlay command configures which of the serial buses to display in the Lister, or whether the Lister is off. "ON" or "1" is the same as "SBUS1".

When set to "ALL", the decode information for different buses is interleaved in time.

Serial bus decode must be on before it can be displayed in the Lister.

Query Syntax :LISTer:DISPlay?

The :LISTer:DISPlay? query returns the Lister display setting.

Return Format <value><NL>

<value> ::= {OFF | SBUS1 | SBUS2 | ALL}

- See Also**
- ["Introduction to :LISTer Commands"](#) on page 541
 - [":SBUS<n>:DISPlay"](#) on page 902
 - [":LISTer:DATA"](#) on page 542

:LISTer:REfERENCE

N (see [page 1650](#))

Command Syntax :LISTer:REfERENCE <time_ref>

<time_ref> ::= {TRIGger | PREvious}

The :LISTer:REfERENCE command selects whether the time value for a Lister row is relative to the trigger or the previous Lister row.

Query Syntax :LISTer:REfERENCE?

The :LISTer:REfERENCE? query returns the Lister time reference setting.

Return Format <time_ref><NL>

<time_ref> ::= {TRIGger | PREvious}

- See Also**
- ["Introduction to :LISTer Commands"](#) on page 541
 - [":SBUS<n>:DISPlay"](#) on page 902
 - [":LISTer:DATA"](#) on page 542
 - [":LISTer:DISPlay"](#) on page 543

24 :LTEST Commands

Table 111 :LTEST Commands Summary

Command	Query	Options and Query Returns
:LTEST:COPY (see page 547)	n/a	n/a
:LTEST:COPY:ALL (see page 548)	n/a	n/a
:LTEST:COPY:MARGin <percent> (see page 549)	:LTEST:COPY:MARGin? (see page 549)	<percent> ::= copy margin percent in NR1 format
:LTEST:ENABLE {{0 OFF} {1 ON}} (see page 550)	:LTEST:ENABLE? (see page 550)	<setting> ::= {0 1}
:LTEST:FAIL <condition> (see page 551)	:LTEST:FAIL? (see page 551)	<condition> ::= {INSide OUTSide}
:LTEST:LLIMit <lower_value> (see page 552)	:LTEST:LLIMit? (see page 552)	<lower_value> ::= a real number in NR3 format
:LTEST:MEASurement {MEAS<n>} (see page 553)	:LTEST:MEASurement? (see page 553)	<n> ::= # of installed measurement, from 1 to 10
n/a	:LTEST:RESults? [{MEAS<n>}] (see page 554)	<n> ::= # of installed measurement, from 1 to 10
:LTEST:RUMode:SOFailu re {{0 OFF} {1 ON}} (see page 555)	:LTEST:RUMode:SOFailu re? (see page 555)	<setting> ::= {0 1}

Table 111 :LTEST Commands Summary (continued)

Command	Query	Options and Query Returns
:LTEST:TEST {{0 OFF} {1 ON}} (see page 556)	:LTEST:TEST? (see page 556)	<setting> ::= {0 1}
:LTEST:ULIMit <upper_value> (see page 557)	:LTEST:ULIMit? (see page 557)	<upper_value> ::= a real number in NR3 format

Introduction to :LTEST Commands The LTEST subsystem is used for the Measurement Limit Test feature.

:LTEST:COPY

N (see [page 1650](#))

Command Syntax :LTEST:COPY

For the measurement selected by :LTEST:MEASurement, the :LTEST:COPY command sets upper and lower limits to the last measurement result plus or minus a percent margin (see :LTEST:COPY:MARGIN).

- See Also**
- [":LTEST:COPY:ALL"](#) on page 548
 - [":LTEST:COPY:MARGIN"](#) on page 549
 - [":LTEST:ENABLE"](#) on page 550
 - [":LTEST:FAIL"](#) on page 551
 - [":LTEST:LLIMit"](#) on page 552
 - [":LTEST:MEASurement"](#) on page 553
 - [":LTEST:RESults"](#) on page 554
 - [":LTEST:RUMode:SOFailure"](#) on page 555
 - [":LTEST:TEST"](#) on page 556
 - [":LTEST:ULIMit"](#) on page 557

:LTEST:COPY:ALL

N (see [page 1650](#))

Command Syntax :LTEST:COPY:ALL

For all measurements whose limits are being tested, the :LTEST:COPY:ALL command sets upper and lower limits to the last measurement result plus or minus a percent margin (see :LTEST:COPY:MARGIN).

In essence, this command performs the :LTEST:COPY operation on all measurements whose limits are being tested.

- See Also
- [":LTEST:COPY"](#) on page 547
 - [":LTEST:COPY:MARGIN"](#) on page 549
 - [":LTEST:ENABLE"](#) on page 550
 - [":LTEST:FAIL"](#) on page 551
 - [":LTEST:LLIMit"](#) on page 552
 - [":LTEST:MEASurement"](#) on page 553
 - [":LTEST:RESults"](#) on page 554
 - [":LTEST:RUMode:SOFailure"](#) on page 555
 - [":LTEST:TEST"](#) on page 556
 - [":LTEST:ULIMit"](#) on page 557

:LTEST:COPY:MARGIN

N (see [page 1650](#))

Command Syntax :LTEST:COPY:MARGIN <percent>

<percent> ::= copy margin percent in NR1 format

For the measurement selected by :LTEST:MEASUREMENT, the :LTEST:COPY:MARGIN command specifies the percent margin used when setting upper and lower limits by copying from measurement results.

Query Syntax :LTEST:COPY:MARGIN?

The :LTEST:COPY:MARGIN? query returns the percent margin setting for the measurement selected by :LTEST:MEASUREMENT.

Return Format <percent><NL>

<percent> ::= copy margin percent in NR1 format

- See Also**
- [":LTEST:COPY"](#) on page 547
 - [":LTEST:COPY:ALL"](#) on page 548
 - [":LTEST:ENABLE"](#) on page 550
 - [":LTEST:FAIL"](#) on page 551
 - [":LTEST:LLIMIT"](#) on page 552
 - [":LTEST:MEASUREMENT"](#) on page 553
 - [":LTEST:RESULTS"](#) on page 554
 - [":LTEST:RUMODE:SOFailure"](#) on page 555
 - [":LTEST:TEST"](#) on page 556
 - [":LTEST:ULIMIT"](#) on page 557

:LTEST:ENABLE

N (see [page 1650](#))

Command Syntax :LTEST:ENABLE {{0 | OFF} | {1 | ON}}

The :LTEST:ENABLE command turns the measurement limit test feature on or off.

Query Syntax :LTEST:ENABLE?

The :LTEST:ENABLE? query returns whether the measurement limit test feature is on or off.

Return Format <setting><NL>

<setting> ::= {0 | 1}

- See Also**
- [":LTEST:COPY"](#) on page 547
 - [":LTEST:COPY:ALL"](#) on page 548
 - [":LTEST:COPY:MARGIN"](#) on page 549
 - [":LTEST:FAIL"](#) on page 551
 - [":LTEST:LLIMIT"](#) on page 552
 - [":LTEST:MEASUREMENT"](#) on page 553
 - [":LTEST:RESULTS"](#) on page 554
 - [":LTEST:RUMODE:SOFailure"](#) on page 555
 - [":LTEST:TEST"](#) on page 556
 - [":LTEST:ULIMIT"](#) on page 557

:LTEST:FAIL

N (see [page 1650](#))

Command Syntax :LTEST:FAIL <condition>
 <condition> ::= {INSide | OUTSide}

For the active measurement currently selected by the :LTEST:MEASurement command, the :LTEST:FAIL command sets the fail condition for the measurement.

When a measurement failure is detected by the limit test, the fail action conditions are executed, and there is the potential to generate an SRQ.

- INSide – causes the oscilloscope to fail a test when the measurement results are within the parameters set by the :LTEST:LIMit and :LTEST:ULIMit commands.
- OUTSide – causes the oscilloscope to fail a test when the measurement results exceed the parameters set by :LTEST:LLIMit and :LTEST:ULIMit commands.

Query Syntax :LTEST:FAIL?

The :LTEST:FAIL? query returns the currently set fail condition.

Return Format <condition><NL>
 <condition> ::= {INS | OUTS}

- See Also**
- [":LTEST:COPY"](#) on page 547
 - [":LTEST:COPY:ALL"](#) on page 548
 - [":LTEST:COPY:MARGin"](#) on page 549
 - [":LTEST:ENABLE"](#) on page 550
 - [":LTEST:LLIMit"](#) on page 552
 - [":LTEST:MEASurement"](#) on page 553
 - [":LTEST:RESults"](#) on page 554
 - [":LTEST:RUMode:SOFailure"](#) on page 555
 - [":LTEST:TEST"](#) on page 556
 - [":LTEST:ULIMit"](#) on page 557

:LTEST:LLIMit

N (see [page 1650](#))

Command Syntax :LTEST:LLIMit <lower_value>
<lower_value> ::= a real number in NR3 format

For the active measurement currently selected by the :LTEST:MEASurement command, the :LTEST:LLIMit (Lower LIMit) command sets the lower test limit.

Query Syntax :LTEST:LLIMit?

The :LTEST:LLIMit? query returns the current lower limit setting.

Return Format <lower_value><NL>

- See Also**
- [":LTEST:COPY"](#) on page 547
 - [":LTEST:COPY:ALL"](#) on page 548
 - [":LTEST:COPY:MARGin"](#) on page 549
 - [":LTEST:ENABle"](#) on page 550
 - [":LTEST:FAIL"](#) on page 551
 - [":LTEST:MEASurement"](#) on page 553
 - [":LTEST:RESults"](#) on page 554
 - [":LTEST:RUMode:SOFailure"](#) on page 555
 - [":LTEST:TEST"](#) on page 556
 - [":LTEST:ULIMit"](#) on page 557

:LTEST:MEASurement

N (see [page 1650](#))

Command Syntax :LTEST:MEASurement {MEAS<n>}

<n> ::= # of installed measurement, from 1 to 10

The :LTEST:MEASurement command selects the measurement source for the :LTEST:FAIL, :LTEST:LLIMit, :LTEST:ULIMit, and :LTEST:TEST commands. It selects one of the active measurements by its number, where MEAS1 is the first added measurement. When you add more than 10 measurements, measurement numbering starts again from MEAS1.

Query Syntax :LTEST:MEASurement?

The :LTEST:MEASurement? query returns the currently selected measurement source.

Return Format MEAS<n><NL>

- See Also**
- [":LTEST:COPY"](#) on page 547
 - [":LTEST:COPY:ALL"](#) on page 548
 - [":LTEST:COPY:MARGin"](#) on page 549
 - [":LTEST:ENABle"](#) on page 550
 - [":LTEST:FAIL"](#) on page 551
 - [":LTEST:LLIMit"](#) on page 552
 - [":LTEST:RESults"](#) on page 554
 - [":LTEST:RUMode:SOFailure"](#) on page 555
 - [":LTEST:TEST"](#) on page 556
 - [":LTEST:ULIMit"](#) on page 557

:LTEST:RESults

N (see [page 1650](#))

Query Syntax :LTEST:RESults? [{MEAS<n>}]

<n> ::= # of installed measurement, from 1 to 10

The :LTEST:RESults? query returns the measurement results for selected measurement.

When :LTEST:TEST is ON, the :LTEST:RESults? query returns the failed minimum value (Fail Min), the failed maximum value (Fail Max), and the total number of measurements made (# of Meas).

When :LTEST:TEST is OFF, the :LTEST:RESults? query returns nothing.

Return Format <fail_min>,<fail_max>,<num_meas><NL>

- <fail_min> – A real number representing the total number of measurements that have failed the minimum limit.
- <fail_max> – A real number representing the total number of measurements that have failed the maximum limit.
- <num_meas> – A real number representing the total number of measurements that have been made.

- See Also**
- [":LTEST:COPY"](#) on page 547
 - [":LTEST:COPY:ALL"](#) on page 548
 - [":LTEST:COPY:MARGin"](#) on page 549
 - [":LTEST:ENABle"](#) on page 550
 - [":LTEST:FAIL"](#) on page 551
 - [":LTEST:LLIMit"](#) on page 552
 - [":LTEST:MEASurement"](#) on page 553
 - [":LTEST:RUMode:SOFailure"](#) on page 555
 - [":LTEST:TEST"](#) on page 556
 - [":LTEST:ULIMit"](#) on page 557

:LTEST:RUMode:SOFailure

N (see [page 1650](#))

Command Syntax :LTEST:RUMode:SOFailure {{0 | OFF} | {1 | ON}}

The :LTEST:RUMode:SOFailure command enables or disables the limit test "stop on failure" option.

When ON, the oscilloscope acquisition system stops once a limit failure is detected. If more than one measurement limit test is enabled, a failure of any of the measurements stops the oscilloscope from acquiring new waveforms.

Query Syntax :LTEST:RUMode:SOFailure?

The :LTEST:RUMode:SOFailure? query returns the "stop on failure" setting.

Return Format <setting><NL>

<setting> ::= {0 | 1}

- See Also**
- [":LTEST:COPY"](#) on page 547
 - [":LTEST:COPY:ALL"](#) on page 548
 - [":LTEST:COPY:MARGIN"](#) on page 549
 - [":LTEST:ENABLE"](#) on page 550
 - [":LTEST:FAIL"](#) on page 551
 - [":LTEST:LLIMIT"](#) on page 552
 - [":LTEST:MEASUREMENT"](#) on page 553
 - [":LTEST:RESULTS"](#) on page 554
 - [":LTEST:TEST"](#) on page 556
 - [":LTEST:ULIMIT"](#) on page 557

:LTEST:TEST

N (see [page 1650](#))

Command Syntax :LTEST:TEST {{0 | OFF} | {1 | ON}}

For the active measurement currently selected by the :LTEST:MEASUREMENT command, the :LTEST:TEST command enables or disables the limit test function on that measurement.

When any measurement has its limit test function enabled, the overall Limit Test feature is enabled.

The :LTEST:RESULTS? query returns nothing when :LTEST:TEST is OFF.

Query Syntax :LTEST:TEST?

The :LTEST:TEST? query returns the state of the TEST control for the active measurement currently selected by the :LTEST:MEASUREMENT command.

Return Format <setting><NL>

<setting> ::= {0 | 1}

- See Also**
- [":LTEST:COPY"](#) on page 547
 - [":LTEST:COPY:ALL"](#) on page 548
 - [":LTEST:COPY:MARGIN"](#) on page 549
 - [":LTEST:ENABLE"](#) on page 550
 - [":LTEST:FAIL"](#) on page 551
 - [":LTEST:LLIMIT"](#) on page 552
 - [":LTEST:MEASUREMENT"](#) on page 553
 - [":LTEST:RESULTS"](#) on page 554
 - [":LTEST:RUMODE:SOFAILURE"](#) on page 555
 - [":LTEST:ULIMIT"](#) on page 557

:LTEST:ULIMit

N (see [page 1650](#))

Command Syntax :LTEST:ULIMit <upper_value>
 <upper_value> ::= a real number in NR3 format

For the active measurement currently selected by the :LTEST:MEASurement command, the :LTEST:ULIMit (Upper LIMit) command sets the upper test limit.

Query Syntax :LTEST:ULIMit?

The :LTEST:ULIMit? query returns the current upper limit setting.

Return Format <upper_value><NL>

- See Also**
- [":LTEST:COPY"](#) on page 547
 - [":LTEST:COPY:ALL"](#) on page 548
 - [":LTEST:COPY:MARGin"](#) on page 549
 - [":LTEST:ENABLE"](#) on page 550
 - [":LTEST:FAIL"](#) on page 551
 - [":LTEST:LLIMit"](#) on page 552
 - [":LTEST:MEASurement"](#) on page 553
 - [":LTEST:RESults"](#) on page 554
 - [":LTEST:RUMode:SOFailure"](#) on page 555
 - [":LTEST:TEST"](#) on page 556

25 :MARKer Commands

Set and query the settings of X-axis markers (X1 and X2 cursors) and the Y-axis markers (Y1 and Y2 cursors). See ["Introduction to :MARKer Commands"](#) on page 561.

Table 112 :MARKer Commands Summary

Command	Query	Options and Query Returns
n/a	:MARKer:DYDX? (see page 562)	<return_value> ::= $\Delta Y / \Delta X$ value in NR3 format
:MARKer:MODE <mode> (see page 563)	:MARKer:MODE? (see page 563)	<mode> ::= {OFF MEASurement MANual WAVEform BINary HEX}
:MARKer:X1:DISPlay {{0 OFF} {1 ON}} (see page 564)	:MARKer:X1:DISPlay? (see page 564)	<setting> ::= {0 1}
:MARKer:X1Position <position>[suffix] (see page 565)	:MARKer:X1Position? (see page 565)	<position> ::= X1 cursor position value in NR3 format [suffix] ::= {s ms us ns ps Hz kHz MHz} <return_value> ::= X1 cursor position value in NR3 format
:MARKer:X1Y1source <source> (see page 566)	:MARKer:X1Y1source? (see page 566)	<source> ::= {CHANnel<n> FUNCTION<m> MATH<m> WMEMory<r> HISTogram} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= <source>
:MARKer:X2:DISPlay {{0 OFF} {1 ON}} (see page 567)	:MARKer:X2:DISPlay? (see page 567)	<setting> ::= {0 1}

Table 112 :MARKer Commands Summary (continued)

Command	Query	Options and Query Returns
:MARKer:X2Position <position>[suffix] (see page 568)	:MARKer:X2Position? (see page 568)	<position> ::= X2 cursor position value in NR3 format [suffix] ::= {s ms us ns ps Hz kHz MHz} <return_value> ::= X2 cursor position value in NR3 format
:MARKer:X2Y2source <source> (see page 569)	:MARKer:X2Y2source? (see page 569)	<source> ::= {CHANnel<n> FUNCTION<m> MATH<m> WMemory<r> HISTogram} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= <source>
n/a	:MARKer:XDELta? (see page 570)	<return_value> ::= X cursors delta value in NR3 format
:MARKer:XUNits <mode> (see page 571)	:MARKer:XUNits? (see page 571)	<units> ::= {SEConds HERTz DEGREes PERCent}
:MARKer:XUNits:USE (see page 572)	n/a	n/a
:MARKer:Y1:DISPlay {{0 OFF} {1 ON}} (see page 573)	:MARKer:Y1:DISPlay? (see page 573)	<setting> ::= {0 1}
:MARKer:Y1Position <position>[suffix] (see page 574)	:MARKer:Y1Position? (see page 574)	<position> ::= Y1 cursor position value in NR3 format [suffix] ::= {V mV dB} <return_value> ::= Y1 cursor position value in NR3 format
:MARKer:Y2:DISPlay {{0 OFF} {1 ON}} (see page 575)	:MARKer:Y2:DISPlay? (see page 575)	<setting> ::= {0 1}
:MARKer:Y2Position <position>[suffix] (see page 576)	:MARKer:Y2Position? (see page 576)	<position> ::= Y2 cursor position value in NR3 format [suffix] ::= {V mV dB} <return_value> ::= Y2 cursor position value in NR3 format

Table 112 :MARKer Commands Summary (continued)

Command	Query	Options and Query Returns
n/a	:MARKer:YDELta? (see page 577)	<return_value> ::= Y cursors delta value in NR3 format
:MARKer:YUNits <mode> (see page 578)	:MARKer:YUNits? (see page 578)	<units> ::= {BASE PERCent}
:MARKer:YUNits:USE (see page 579)	n/a	n/a

Introduction to :MARKer Commands The MARKer subsystem commands set and query the settings of X-axis markers (X1 and X2 cursors) and the Y-axis markers (Y1 and Y2 cursors). You can set and query the marker mode and source, the position of the X and Y cursors, and query delta X and delta Y cursor values.

Reporting the Setup

Use :MARKer? to query setup information for the MARKer subsystem.

Return Format

The following is a sample response from the :MARKer? query. In this case, the query was issued following a *RST and ":MARKer:MODE MANual" command.

```
:MARK:X1Y1 CHAN1;X2Y2 CHAN1;MODE MAN
```

:MARKer:DYDX

N (see [page 1650](#))

Query Syntax :MARKer:DYDX?

The MARKer:DYDX? query returns the cursor $\Delta Y/\Delta X$ value.

X cursor units are set by the :MARKer:XUNits command.

NOTE

If the front-panel cursors are off, the marker position values are not defined. Make sure to set :MARKer:MODE to MANual or WAVEform to put the cursors in the front-panel Normal mode.

Return Format <value><NL>

<value> ::= $\Delta Y/\Delta X$ value in NR3 format.

- See Also
- ["Introduction to :MARKer Commands"](#) on page 561
 - [":MARKer:MODE"](#) on page 563
 - [":MARKer:X1Position"](#) on page 565
 - [":MARKer:X2Position"](#) on page 568
 - [":MARKer:X1Y1source"](#) on page 566
 - [":MARKer:X2Y2source"](#) on page 569
 - [":MARKer:XUNits"](#) on page 571

:MARKer:MODE

N (see [page 1650](#))

Command Syntax :MARKer:MODE <mode>

<mode> ::= {OFF | MEASurement | MANual | WAVEform | BINary | HEX}

The :MARKer:MODE command sets the cursors mode:

- OFF – removes the cursor information from the display.
- MANual – enables manual placement of the X and Y cursors.

If the front-panel cursors are off, or are set to the front-panel Hex or Binary mode, setting :MARKer:MODE MANual will put the cursors in the front-panel Normal mode.

- MEASurement – cursors track the most recent measurement.

Setting the mode to MEASurement sets the marker sources (:MARKer:X1Y1source and :MARKer:X2Y2source) to the measurement source (:MEASure:SOURce). Setting the measurement source remotely always sets the marker sources.

- WAVEform – the Y1 cursor tracks the voltage value at the X1 cursor of the waveform specified by the X1Y1source, and the Y2 cursor does the same for the X2 cursor and its X2Y2source.
- BINary – logic levels of displayed waveforms at the current X1 and X2 cursor positions are displayed in the Cursor sidebar dialog in binary.
- HEX – logic levels of displayed waveforms at the current X1 and X2 cursor positions are displayed in the Cursor sidebar dialog in hexadecimal.

Query Syntax :MARKer:MODE?

The :MARKer:MODE? query returns the current cursors mode.

Return Format <mode><NL>

<mode> ::= {OFF | MEAS | MAN | WAV | BIN | HEX}

- See Also**
- ["Introduction to :MARKer Commands"](#) on page 561
 - [":MARKer:X1Y1source"](#) on page 566
 - [":MARKer:X2Y2source"](#) on page 569
 - [":MEASure:SOURce"](#) on page 661
 - [":MARKer:X1Position"](#) on page 565
 - [":MARKer:X2Position"](#) on page 568
 - [":MARKer:Y1Position"](#) on page 574
 - [":MARKer:Y2Position"](#) on page 576

:MARKer:X1:DISPlay

N (see [page 1650](#))

Command Syntax :MARKer:X1:DISPlay {{0 | OFF} | {1 | ON}}

The :MARKer:X1:DISPlay command specifies whether the X1 cursor is displayed.

Query Syntax :MARKer:X1:DISPlay?

The :MARKer:X1:DISPlay? query returns the X1 cursor display setting.

Return Format <setting><NL>
<setting> ::= {0 | 1}

See Also • [":MARKer:X1:DISPlay"](#) on page 564

:MARKer:X1Position

N (see [page 1650](#))

Command Syntax :MARKer:X1Position <position> [suffix]
 <position> ::= X1 cursor position in NR3 format
 <suffix> ::= {s | ms | us | ns | ps | Hz | kHz | MHz}

The :MARKer:X1Position command:

- Sets :MARKer:MODE to MANual if it is not currently set to WAVEform (see [":MARKer:MODE"](#) on page 563).
- Sets the X1 cursor position to the specified value.

X cursor units are set by the :MARKer:XUNits command.

Query Syntax :MARKer:X1Position?

The :MARKer:X1Position? query returns the current X1 cursor position. This is functionally equivalent to the obsolete :MEASure:TSTart command/query.

NOTE

If the front-panel cursors are off, the marker position values are not defined and an error is generated. Make sure to set :MARKer:MODE to MANual or WAVEform to put the cursors in the front-panel Normal mode.

Return Format <position><NL>
 <position> ::= X1 cursor position in NR3 format

- See Also**
- ["Introduction to :MARKer Commands"](#) on page 561
 - [":MARKer:MODE"](#) on page 563
 - [":MARKer:X2Position"](#) on page 568
 - [":MARKer:X1Y1source"](#) on page 566
 - [":MARKer:X2Y2source"](#) on page 569
 - [":MARKer:XUNits"](#) on page 571
 - [":MEASure:TSTart"](#) on page 1564

:MARKer:X1Y1source

N (see [page 1650](#))

Command Syntax :MARKer:X1Y1source <source>

<source> ::= {CHANnel<n> | FUNction<m> | MATH<m> | WMEMory<r>
| HISTogram}

<n> ::= 1 to (# analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MARKer:X1Y1source command sets the source for the cursors. The channel you specify must be enabled for cursors to be displayed. If the channel or function is not on, an error message is issued.

If the marker mode is not currently WAVEform (see [":MARKer:MODE"](#) on page 563):

- Sending a :MARKer:X1Y1source command will put the cursors in the MANual mode.
- Setting the source for one pair of markers (for example, X1Y1) sets the source for the other (for example, X2Y2).

If the marker mode is currently WAVEform, the X1Y1 source can be set separate from the X2Y2 source.

If :MARKer:MODE is set to OFF or MANual, setting :MEASure:SOURce to CHANnel<n>, FUNction, MATH, or WMEMory<r> will also set :MARKer:X1Y1source and :MARKer:X2Y2source to this value.

NOTE

MATH is an alias for FUNction. The query will return FUNC if the source is FUNction or MATH.

Query Syntax :MARKer:X1Y1source?

The :MARKer:X1Y1source? query returns the current source for the cursors. If all channels are off or if :MARKer:MODE is set to OFF, the query returns NONE.

Return Format <source><NL>

<source> ::= {CHAN<n> | FUNC | WMEM<r> | HIST | NONE}

- See Also**
- ["Introduction to :MARKer Commands"](#) on page 561
 - [":MARKer:MODE"](#) on page 563
 - [":MARKer:X2Y2source"](#) on page 569
 - [":MEASure:SOURce"](#) on page 661

:MARKer:X2:DISPlay

N (see [page 1650](#))

Command Syntax :MARKer:X2:DISPlay {{0 | OFF} | {1 | ON}}

The :MARKer:X2:DISPlay command specifies whether the X2 cursor is displayed.

Query Syntax :MARKer:X2:DISPlay?

The :MARKer:X2:DISPlay? query returns the X2 cursor display setting.

Return Format <setting><NL>
<setting> ::= {0 | 1}

See Also • [":MARKer:X2:DISPlay"](#) on page 567

:MARKer:X2Position

N (see [page 1650](#))

Command Syntax :MARKer:X2Position <position> [suffix]

<position> ::= X2 cursor position in NR3 format

<suffix> ::= {s | ms | us | ns | ps | Hz | kHz | MHz}

The :MARKer:X2Position command:

- Sets :MARKer:MODE to MANual if it is not currently set to WAVEform (see [":MARKer:MODE"](#) on page 563).
- Sets the X2 cursor position to the specified value.

X cursor units are set by the :MARKer:XUNits command.

Query Syntax :MARKer:X2Position?

The :MARKer:X2Position? query returns current X2 cursor position. This is functionally equivalent to the obsolete :MEASure:TSTOp command/query.

NOTE

If the front-panel cursors are off, the marker position values are not defined and an error is generated. Make sure to set :MARKer:MODE to MANual or WAVEform to put the cursors in the front-panel Normal mode.

Return Format <position><NL>

<position> ::= X2 cursor position in NR3 format

- See Also**
- ["Introduction to :MARKer Commands"](#) on page 561
 - [":MARKer:MODE"](#) on page 563
 - [":MARKer:X1Position"](#) on page 565
 - [":MARKer:X2Y2source"](#) on page 569
 - [":MARKer:XUNits"](#) on page 571
 - [":MEASure:TSTOp"](#) on page 1565

:MARKer:X2Y2source

N (see [page 1650](#))

Command Syntax :MARKer:X2Y2source <source>

<source> ::= {CHANnel<n> | FUNCTION<m> | MATH<m> | WMEMory<r>
| HISTogram}

<n> ::= 1 to (# analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MARKer:X2Y2source command sets the source for the cursors. The channel you specify must be enabled for cursors to be displayed. If the channel or function is not on, an error message is issued.

If the marker mode is not currently WAVEform (see [":MARKer:MODE"](#) on page 563):

- Sending a :MARKer:X2Y2source command will put the cursors in the MANual mode.
- Setting the source for one pair of markers (for example, X2Y2) sets the source for the other (for example, X1Y1).

If the marker mode is currently WAVEform, the X2Y2 source can be set separate from the X1Y1 source.

If :MARKer:MODE is set to OFF or MANual, setting :MEASure:SOURce to CHANnel<n>, FUNCTION, MATH, or WMEMory<r> will also set :MARKer:X1Y1source and :MARKer:X2Y2source to this value.

NOTE

MATH is an alias for FUNCTION. The query will return FUNC if the source is FUNCTION or MATH.

Query Syntax :MARKer:X2Y2source?

The :MARKer:X2Y2source? query returns the current source for the cursors. If all channels are off or if :MARKer:MODE is set to OFF, the query returns NONE.

Return Format <source><NL>

<source> ::= {CHAN<n> | FUNC | WMEM<r> | HIST | NONE}

- See Also**
- ["Introduction to :MARKer Commands"](#) on page 561
 - [":MARKer:MODE"](#) on page 563
 - [":MARKer:X1Y1source"](#) on page 566
 - [":MEASure:SOURce"](#) on page 661

:MARKer:XDELta

N (see [page 1650](#))

Query Syntax :MARKer:XDELta?

The MARKer:XDELta? query returns the value difference between the current X1 and X2 cursor positions.

$Xdelta = (\text{Value at X2 cursor}) - (\text{Value at X1 cursor})$

X cursor units are set by the :MARKer:XUNits command.

NOTE

If the front-panel cursors are off, the marker position values are not defined. Make sure to set :MARKer:MODE to MANual or WAVEform to put the cursors in the front-panel Normal mode.

Return Format <value><NL>

<value> ::= difference value in NR3 format.

- See Also
- ["Introduction to :MARKer Commands"](#) on page 561
 - [":MARKer:MODE"](#) on page 563
 - [":MARKer:X1Position"](#) on page 565
 - [":MARKer:X2Position"](#) on page 568
 - [":MARKer:X1Y1source"](#) on page 566
 - [":MARKer:X2Y2source"](#) on page 569
 - [":MARKer:XUNits"](#) on page 571

:MARKer:XUNits

N (see [page 1650](#))

Command Syntax :MARKer:XUNits <units>

<units> ::= {SEConds | HERTz | DEGRees | PERCent}

The :MARKer:XUNits command sets the X cursors units:

- SEConds – for making time measurements.
- HERTz – for making frequency measurements.
- DEGRees – for making phase measurements. Use the :MARKer:XUNits:USE command to set the current X1 location as 0 degrees and the current X2 location as 360 degrees.
- PERCent – for making ratio measurements. Use the :MARKer:XUNits:USE command to set the current X1 location as 0 percent and the current X2 location as 100 percent.

Changing X units affects the input and output values of the :MARKer:X1Position, :MARKer:X2Position, and :MARKer:XDELta commands/queries.

Query Syntax :MARKer:XUNits?

The :MARKer:XUNits? query returns the current X cursors units.

Return Format <units><NL>

<units> ::= {SEC | HERT | DEGR | PERC}

- See Also**
- ["Introduction to :MARKer Commands"](#) on page 561
 - [":MARKer:XUNits:USE"](#) on page 572
 - [":MARKer:X1Y1source"](#) on page 566
 - [":MARKer:X2Y2source"](#) on page 569
 - [":MEASure:SOURce"](#) on page 661
 - [":MARKer:X1Position"](#) on page 565
 - [":MARKer:X2Position"](#) on page 568

:MARKer:XUNits:USE

N (see [page 1650](#))

Command Syntax :MARKer:XUNits:USE

When DEGRees is selected for :MARKer:XUNits, the :MARKer:XUNits:USE command sets the current X1 location as 0 degrees and the current X2 location as 360 degrees.

When PERCent is selected for :MARKer:XUNits, the :MARKer:XUNits:USE command sets the current X1 location as 0 percent and the current X2 location as 100 percent.

Once the 0 and 360 degree or 0 and 100 percent locations are set, inputs to and outputs from the :MARKer:X1Position, :MARKer:X2Position, and :MARKer:XDELta commands/queries are relative to the set locations.

- See Also
- ["Introduction to :MARKer Commands"](#) on page 561
 - [":MARKer:XUNits"](#) on page 571
 - [":MARKer:X1Y1source"](#) on page 566
 - [":MARKer:X2Y2source"](#) on page 569
 - [":MEASure:SOURce"](#) on page 661
 - [":MARKer:X1Position"](#) on page 565
 - [":MARKer:X2Position"](#) on page 568
 - [":MARKer:XDELta"](#) on page 570

:MARKer:Y1:DISPlay

N (see [page 1650](#))

Command Syntax :MARKer:Y1:DISPlay {{0 | OFF} | {1 | ON}}

The :MARKer:Y1:DISPlay command specifies whether the Y1 cursor is displayed.

Query Syntax :MARKer:Y1:DISPlay?

The :MARKer:Y1:DISPlay? query returns the Y1 cursor display setting.

Return Format <setting><NL>
<setting> ::= {0 | 1}

See Also • [":MARKer:Y1:DISPlay"](#) on page 573

:MARKer:Y1Position

N (see [page 1650](#))

Command Syntax :MARKer:Y1Position <position> [suffix]

<position> ::= Y1 cursor position in NR3 format

<suffix> ::= {mV | V | dB}

If the :MARKer:MODE is not currently set to WAVEform (see [":MARKer:MODE"](#) on page 563), the :MARKer:Y1Position command:

- Sets :MARKer:MODE to MANual.
- Sets the Y1 cursor position to the specified value.

Y cursor units are set by the :MARKer:YUNits command.

When the :MARKer:MODE is set to WAVEform, Y positions cannot be set.

Query Syntax :MARKer:Y1Position?

The :MARKer:Y1Position? query returns current Y1 cursor position. This is functionally equivalent to the obsolete :MEASure:VStart command/query.

NOTE

If the front-panel cursors are off or are set to Binary or Hex Mode, the marker position values are not defined and an error is generated. Make sure to set :MARKer:MODE to MANual or WAVEform to put the cursors in the front-panel Normal mode.

Return Format <position><NL>

<position> ::= Y1 cursor position in NR3 format

- See Also**
- ["Introduction to :MARKer Commands"](#) on page 561
 - [":MARKer:MODE"](#) on page 563
 - [":MARKer:X1Y1source"](#) on page 566
 - [":MARKer:X2Y2source"](#) on page 569
 - [":MARKer:Y2Position"](#) on page 576
 - [":MARKer:YUNits"](#) on page 578
 - [":MEASure:VStart"](#) on page 1569

:MARKer:Y2:DISPlay

N (see [page 1650](#))

Command Syntax :MARKer:Y2:DISPlay {{0 | OFF} | {1 | ON}}

The :MARKer:Y2:DISPlay command specifies whether the Y2 cursor is displayed.

Query Syntax :MARKer:Y2:DISPlay?

The :MARKer:Y2:DISPlay? query returns the Y2 cursor display setting.

Return Format <setting><NL>
<setting> ::= {0 | 1}

See Also • [":MARKer:Y2:DISPlay"](#) on page 575

:MARKer:Y2Position

N (see [page 1650](#))

Command Syntax :MARKer:Y2Position <position> [suffix]

<position> ::= Y2 cursor position in NR3 format

<suffix> ::= {mV | V | dB}

If the :MARKer:MODE is not currently set to WAVEform (see [":MARKer:MODE"](#) on page 563), the :MARKer:Y2Position command:

- Sets :MARKer:MODE to MANual.
- Sets the Y2 cursor position to the specified value.

Y cursor units are set by the :MARKer:YUNits command.

When the :MARKer:MODE is set to WAVEform, Y positions cannot be set.

Query Syntax :MARKer:Y2Position?

The :MARKer:Y2Position? query returns current Y2 cursor position. This is functionally equivalent to the obsolete :MEASure:VSTop command/query.

NOTE

If the front-panel cursors are off or are set to Binary or Hex Mode, the marker position values are not defined and an error is generated. Make sure to set :MARKer:MODE to MANual or WAVEform to put the cursors in the front-panel Normal mode.

Return Format <position><NL>

<position> ::= Y2 cursor position in NR3 format

- See Also**
- ["Introduction to :MARKer Commands"](#) on page 561
 - [":MARKer:MODE"](#) on page 563
 - [":MARKer:X1Y1source"](#) on page 566
 - [":MARKer:X2Y2source"](#) on page 569
 - [":MARKer:Y1Position"](#) on page 574
 - [":MARKer:YUNits"](#) on page 578
 - [":MEASure:VSTop"](#) on page 1570

:MARKer:YDELta

N (see [page 1650](#))

Query Syntax :MARKer:YDELta?

The :MARKer:YDELta? query returns the value difference between the current Y1 and Y2 cursor positions.

Ydelta = (Value at Y2 cursor) - (Value at Y1 cursor)

NOTE

If the front-panel cursors are off or are set to Binary or Hex Mode, the marker position values are not defined. Make sure to set :MARKer:MODE to MANual or WAVEform to put the cursors in the front-panel Normal mode.

Y cursor units are set by the :MARKer:YUNits command.

Return Format <value><NL>

<value> ::= difference value in NR3 format

- See Also**
- ["Introduction to :MARKer Commands"](#) on page 561
 - [":MARKer:MODE"](#) on page 563
 - [":MARKer:X1Y1source"](#) on page 566
 - [":MARKer:X2Y2source"](#) on page 569
 - [":MARKer:Y1Position"](#) on page 574
 - [":MARKer:Y2Position"](#) on page 576
 - [":MARKer:YUNits"](#) on page 578

:MARKer:YUNits

N (see [page 1650](#))

Command Syntax :MARKer:YUNits <units>
 <units> ::= {BASE | PERCent}

The :MARKer:YUNits command sets the Y cursors units:

- BASE – for making measurements in the units associated with the cursors source.
- PERCent – for making ratio measurements. Use the :MARKer:YUNits:USE command to set the current Y1 location as 0 percent and the current Y2 location as 100 percent.

Changing Y units affects the input and output values of the :MARKer:Y1Position, :MARKer:Y2Position, and :MARKer:YDELta commands/queries.

Query Syntax :MARKer:YUNits?

The :MARKer:YUNits? query returns the current Y cursors units.

Return Format <units><NL>
 <units> ::= {BASE | PERC}

- See Also**
- ["Introduction to :MARKer Commands"](#) on page 561
 - [":MARKer:YUNits:USE"](#) on page 579
 - [":MARKer:X1Y1source"](#) on page 566
 - [":MARKer:X2Y2source"](#) on page 569
 - [":MEASure:SOURce"](#) on page 661
 - [":MARKer:Y1Position"](#) on page 574
 - [":MARKer:Y2Position"](#) on page 576
 - [":MARKer:YDELta"](#) on page 577

:MARKer:YUNits:USE

N (see [page 1650](#))

Command Syntax :MARKer:YUNits:USE

When PERCent is selected for :MARKer:YUNits, the :MARKer:YUNits:USE command sets the current Y1 location as 0 percent and the current Y2 location as 100 percent.

Once the 0 and 100 percent locations are set, inputs to and outputs from the :MARKer:Y1Position, :MARKer:Y2Position, and :MARKer:YDELta commands/queries are relative to the set locations.

- See Also**
- ["Introduction to :MARKer Commands"](#) on page 561
 - [":MARKer:YUNits"](#) on page 578
 - [":MARKer:X1Y1source"](#) on page 566
 - [":MARKer:X2Y2source"](#) on page 569
 - [":MEASure:SOURce"](#) on page 661
 - [":MARKer:Y1Position"](#) on page 574
 - [":MARKer:Y2Position"](#) on page 576
 - [":MARKer:YDELta"](#) on page 577

26 :MEASure Commands

Select automatic measurements to be made and control time markers. See ["Introduction to :MEASure Commands"](#) on page 601.

Table 113 :MEASure Commands Summary

Command	Query	Options and Query Returns
:MEASure:ALL (see page 603)	n/a	n/a
:MEASure:AREa [<interval>][,<source>] >] (see page 604)	:MEASure:AREa? [<interval>][,<source>] >] (see page 604)	<interval> ::= {CYCLe DISPlay} <source> ::= {CHANnel<n> FUNCTION<m> MATH<m> WMemory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= area in volt-seconds, NR3 format

Table 113 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:BRATe [<source>] (see page 605)	:MEASure:BRATe? [<source>] (see page 605)	<source> ::= {<digital channels> CHANnel<n> FUNctIon<m> MATH<m> WMEMory<r>} <digital channels> ::= DIGital<d> for the MSO models <d> ::= 0 to (# digital channels - 1) in NR1 format <n> ::= 1 to (# of analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= bit rate in Hz, NR3 format
:MEASure:BWIDth [<source>] (see page 606)	:MEASure:BWIDth? [<source>] (see page 606)	<source> ::= {CHANnel<n> FUNctIon<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= burst width in seconds, NR3 format
:MEASure:CLEar (see page 607)	n/a	n/a
:MEASure:COUNter [<source>] (see page 608)	:MEASure:COUNter? [<source>] (see page 608)	<source> ::= {CHANnel<n> EXtErnal} for DSO models <source> ::= {CHANnel<n> DIGital<d> EXtErnal} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <return_value> ::= counter frequency in Hertz in NR3 format

Table 113 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:DEFine DElay, <delay spec> (see page 610)	:MEASure:DEFine? DElay (see page 612)	<delay spec> ::= <edge_spec1>,<edge_spec2> edge_spec1 ::= [<slope>]<occurrence> edge_spec2 ::= [<slope>]<occurrence> <slope> ::= {+ -} <occurrence> ::= integer
:MEASure:DEFine THResholds, <threshold spec> (see page 610)	:MEASure:DEFine? THResholds (see page 612)	<threshold spec> ::= {STANDARD} {<threshold mode>,<upper>, <middle>,<lower>} <threshold mode> ::= {PERCent ABSolute}
:MEASure:DElay [<source1>] [,<source2>] (see page 613)	:MEASure:DElay? [<source1>] [,<source2>] (see page 613)	<source1,2> ::= {CHANnel<n> FUNctIon<m> MATH<m> WMEMory<r> <digital channels>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <digital channels> ::= DIGital<d> for the MSO models <d> ::= 0 to (# digital channels - 1) in NR1 format <return_value> ::= floating-point number delay time in seconds in NR3 format
:MEASure:DElay:DEFine <source1_edge_slope>, <source1_edge_number> , <source1_edge_thresho ld>, <source2_edge_slope>, <source2_edge_number> , <source2_edge_thresho ld> (see page 615)	:MEASure:DElay:DEFine ? (see page 615)	<source1_edge_slope>, <source2_edge_slope> ::= {RISing FALLing} <source1_edge_number>, <source2_edge_number> ::= 0 to 1000 in NR1 format <source1_edge_threshold>, <source2_edge_threshold> ::= {LOWer MIDDle UPPer}

Table 113 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:DUAL:CHARge [<interval>] [,<source1>][,<source2>] (see page 616)	:MEASure:DUAL:CHARge? [<interval>] [,<source1>][,<source2>] (see page 616)	<interval> ::= {CYCLe DISPlay} <source1>,<source2> ::= CHANnel<n> with N2820A probe connected <n> ::= 1 to (# analog channels) in NR1 format <return_value> ::= area in Amp-hours, NR3 format
:MEASure:DUAL:VAMPlit ude [<source1>][,<source2>] (see page 617)	:MEASure:DUAL:VAMPlit ude? [<source1>][,<source2>] (see page 617)	<source1>,<source2> ::= CHANnel<n> with N2820A probe connected <n> ::= 1 to (# analog channels) in NR1 format <return_value> ::= the amplitude of the selected waveform in volts in NR3 format
:MEASure:DUAL:VAVerag e [<interval>] [,<source1>][,<source2>] (see page 618)	:MEASure:DUAL:VAVerag e? [<interval>] [,<source1>][,<source2>] (see page 618)	<interval> ::= {CYCLe DISPlay} <source1>,<source2> ::= CHANnel<n> with N2820A probe connected <n> ::= 1 to (# analog channels) in NR1 format <return_value> ::= calculated average voltage in NR3 format
:MEASure:DUAL:VBASe [<source1>][,<source2>] (see page 619)	:MEASure:DUAL:VBASe? [<source1>][,<source2>] (see page 619)	<source1>,<source2> ::= CHANnel<n> with N2820A probe connected <n> ::= 1 to (# analog channels) in NR1 format <base_voltage> ::= voltage at the base of the selected waveform in NR3 format
:MEASure:DUAL:VPP [<source1>][,<source2>] (see page 620)	:MEASure:DUAL:VPP? [<source1>][,<source2>] (see page 620)	<source1>,<source2> ::= CHANnel<n> with N2820A probe connected <n> ::= 1 to (# analog channels) in NR1 format <return_value> ::= voltage peak-to-peak of the selected waveform in NR3 format

Table 113 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:DUAL:VRMS [<interval>] [,<type>] [,<source1>][,<source2>] (see page 621)	:MEASure:DUAL:VRMS? [<interval>] [,<type>] [,<source1>][,<source2>] (see page 621)	<interval> ::= {CYCLE DISPLAY} <type> ::= {AC DC} <source1>,<source2> ::= CHANNEL<n> with N2820A probe connected <n> ::= 1 to (# analog channels) in NR1 format <return_value> ::= calculated RMS voltage in NR3 format
:MEASure:DUTYcycle [<source>] (see page 622)	:MEASure:DUTYcycle? [<source>] (see page 622)	<source> ::= {CHANNEL<n> FUNCTION<m> MATH<m> WMEMory<r>} for DSO models <source> ::= {CHANNEL<n> DIGital<d> FUNCTION<m> MATH<m> WMEMory<r>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <return_value> ::= ratio of positive pulse width to period in NR3 format

Table 113 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:FALLtime [<source>] (see page 623)	:MEASure:FALLtime? [<source>] (see page 623)	<source> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMemory<r>} for DSO models <source> ::= {CHANnel<n> DIGital<d> FUNction<m> MATH<m> WMEMemory<r>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <return_value> ::= time in seconds between the lower and upper thresholds in NR3 format
:MEASure:FFT:ACPR <chan_width>, <chan_spacing>, <chan>[,<source>] (see page 624)	:MEASure:FFT:ACPR? <chan_width>, <chan_spacing>, <chan>[,<source>] (see page 624)	<chan_width> ::= width of main range and sideband channels, Hz in NR3 format <chan_spacing> ::= spacing between main range and sideband channels, Hz in NR3 format <chan> ::= {CENTer HIGH<sb> LOW<sb>} <sb> ::= sideband 1 to 5 <source> ::= {FUNction<m> MATH<m> FFT} (source must be an FFT waveform) <m> ::= 1 to (# math functions) in NR1 format <return_value> ::= adjacent channel power ratio, dBV in NR3 format
:MEASure:FFT:CPOWer [<source>] (see page 625)	:MEASure:FFT:CPOWer? [<source>] (see page 625)	<source> ::= {FUNction<m> MATH<m> FFT} (source must be an FFT waveform) <m> ::= 1 to (# math functions) in NR1 format <return_value> ::= spectral channel power, dBV in NR3 format

Table 113 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:FFT:OBW <percentage>[,<source>] >] (see page 626)	:MEASure:FFT:OBW? <percentage>[,<source>] >] (see page 626)	<percentage> ::= percent of spectral power occupied bandwidth is measured for in NR3 format <source> ::= {FUNCTION<m> MATH<m> FFT} (source must be an FFT waveform) <m> ::= 1 to (# math functions) in NR1 format <return_value> ::= occupied bandwidth, Hz in NR3 format
:MEASure:FFT:THD [<source>] (see page 627)	:MEASure:FFT:THD? [<source>] (see page 627)	<source> ::= {FUNCTION<m> MATH<m> FFT} (source must be an FFT waveform) <m> ::= 1 to (# math functions) in NR1 format <return_value> ::= total harmonic distortion ratio percent in NR3 format
:MEASure:FREQuency [<source>] (see page 628)	:MEASure:FREQuency? [<source>] (see page 628)	<source> ::= {CHANnel<n> FUNCTION<m> MATH<m> WMEMemory<r>} for DSO models <source> ::= {CHANnel<n> DIGital<d> FUNCTION<m> MATH<m> WMEMemory<r>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <return_value> ::= frequency in Hertz in NR3 format
n/a	:MEASure:HISTogram:BW IDth? (see page 629)	<value> ::= bin width in NR3 format
n/a	:MEASure:HISTogram:HI TS? (see page 630)	<value> ::= # of total hits in NR3 format

Table 113 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
n/a	:MEASure:HISTogram:M1S? (see page 631)	<value> ::= % of hits within +/-1 std dev in NR3 format
n/a	:MEASure:HISTogram:M2S? (see page 632)	<value> ::= % of hits within +/-2 std devs in NR3 format
n/a	:MEASure:HISTogram:M3S? (see page 633)	<value> ::= % of hits within +/-3 std devs in NR3 format
n/a	:MEASure:HISTogram:MAXimum? (see page 634)	<value> ::= value of max bin in NR3 format
n/a	:MEASure:HISTogram:MEAN? (see page 635)	<value> ::= mean of histogram in NR3 format
n/a	:MEASure:HISTogram:MEDian? (see page 636)	<value> ::= median of histogram in NR3 format
n/a	:MEASure:HISTogram:MINimum? (see page 637)	<value> ::= value of min bin in NR3 format
n/a	:MEASure:HISTogram:MODE? (see page 638)	<value> ::= mode of histogram in NR3 format
n/a	:MEASure:HISTogram:PEAK? (see page 639)	<value> ::= # of hits in max bin in NR3 format
n/a	:MEASure:HISTogram:PPeak? (see page 640)	<value> ::= delta value between max bin and min bin in NR3 format
n/a	:MEASure:HISTogram:SDEviation? (see page 641)	<value> ::= standard deviation of histogram in NR3 format

Table 113 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:NDUTy [<source>] (see page 642)	:MEASure:NDUTy? [<source>] (see page 642)	<source> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r>} for DSO models <source> ::= {CHANnel<n> DIGital<d> FUNction<m> MATH<m> WMEMory<r>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <return_value> ::= ratio of negative pulse width to period in NR3 format
:MEASure:NEDGes [<source>] (see page 643)	:MEASure:NEDGes? [<source>] (see page 643)	<source> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= the falling edge count in NR3 format

Table 113 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:NPULses [<source>] (see page 644)	:MEASure:NPULses? [<source>] (see page 644)	<source> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r> <digital channels>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <digital channels> ::= DIGital<d> for the MSO models <d> ::= 0 to (# digital channels - 1) in NR1 format <return_value> ::= the falling pulse count in NR3 format
:MEASure:NWIDth [<source>] (see page 645)	:MEASure:NWIDth? [<source>] (see page 645)	<source> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r>} for DSO models <source> ::= {CHANnel<n> DIGital<d> FUNction<m> MATH<m> WMEMory<r>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <return_value> ::= negative pulse width in seconds-NR3 format

Table 113 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:OVERshoot [<source>] (see page 646)	:MEASure:OVERshoot? [<source>] (see page 646)	<source> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= the percent of the overshoot of the selected waveform in NR3 format
:MEASure:PEDGes [<source>] (see page 648)	:MEASure:PEDGes? [<source>] (see page 648)	<source> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= the rising edge count in NR3 format
:MEASure:PERiod [<source>] (see page 649)	:MEASure:PERiod? [<source>] (see page 649)	<source> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r>} for DSO models <source> ::= {CHANnel<n> DIGital<d> FUNction<m> MATH<m> WMEMory<r>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <return_value> ::= waveform period in seconds in NR3 format

Table 113 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:PHASe [<source1>] [,<source2>] (see page 650)	:MEASure:PHASe? [<source1>] [,<source2>] (see page 650)	<source1,2> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= the phase angle value in degrees in NR3 format
:MEASure:PPULses [<source>] (see page 651)	:MEASure:PPULses? [<source>] (see page 651)	<source> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r> <digital channels>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <digital channels> ::= DIGital<d> for the MSO models <d> ::= 0 to (# digital channels - 1) in NR1 format <return_value> ::= the rising pulse count in NR3 format
:MEASure:PREShoot [<source>] (see page 652)	:MEASure:PREShoot? [<source>] (see page 652)	<source> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= the percent of preshoot of the selected waveform in NR3 format

Table 113 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:PWIDth [<source>] (see page 653)	:MEASure:PWIDth? [<source>] (see page 653)	<source> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r>} for DSO models <source> ::= {CHANnel<n> DIGital<d> FUNction<m> MATH<m> WMEMory<r>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <return_value> ::= width of positive pulse in seconds in NR3 format
n/a	:MEASure:RESults? <result_list> (see page 654)	<result_list> ::= comma-separated list of measurement results
:MEASure:RISetime [<source>] (see page 657)	:MEASure:RISetime? [<source>] (see page 657)	<source> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= rise time in seconds in NR3 format

Table 113 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:SDEviation [<source>] (see page 658)	:MEASure:SDEviation? [<source>] (see page 658)	<source> ::= {CHANnel<n> FUNctIon<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= calculated std deviation in NR3 format
:MEASure:SHOW {{0 OFF} {1 ON}} (see page 659)	:MEASure:SHOW? (see page 659)	{0 1}
:MEASure:SLEWrate [<source>[,<slope>]] (see page 660)	:MEASure:SLEWrate? [<source>[,<slope>]] (see page 660)	<source> ::= {CHANnel<n> FUNctIon<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# of analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format
:MEASure:SOURce <source1> [,<source2>] (see page 661)	:MEASure:SOURce? (see page 661)	<source1,2> ::= {CHANnel<n> FUNctIon<m> MATH<m> WMEMory<r> EXTErnal} for DSO models <source1,2> ::= {CHANnel<n> DIGital<d> FUNctIon<m> MATH<m> WMEMory<r> EXTErnal} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <return_value> ::= {<source> NONE}

Table 113 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:STATistics <type> (see page 663)	:MEASure:STATistics? (see page 663)	<type> ::= {{ON 1} CURRENT MEAN MINimum MAXimum STDDev COUNT} ON ::= all statistics returned
:MEASure:STATistics:DISPlay {{0 OFF} {1 ON}} (see page 664)	:MEASure:STATistics:DISPlay? (see page 664)	{0 1}
:MEASure:STATistics:INCRement (see page 665)	n/a	n/a
:MEASure:STATistics:MCOUNT <setting> (see page 666)	:MEASure:STATistics:MCOUNT? (see page 666)	<setting> ::= {INFinite <count>} <count> ::= 2 to 2000 in NR1 format
:MEASure:STATistics:RESET (see page 667)	n/a	n/a
:MEASure:STATistics:RSSDeviation {{0 OFF} {1 ON}} (see page 668)	:MEASure:STATistics:RSSDeviation? (see page 668)	{0 1}

Table 113 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:TEDGE [<slope>,<occurrence>[,<source>] (see page 669)	:MEASure:TEDGE? [<slope>,<occurrence>[,<source>] (see page 670)	<slope> ::= {RISing FALLing EITHER} <occurrence> ::= [+ -]<number> <number> ::= the edge number in NR1 format <source> ::= {<digital channels> CHANNEL<n> FUNCTION<m> MATH<m> WMemory<r>} <digital channels> ::= DIGital<d> for the MSO models <n> ::= 1 to (# of analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <return_value> ::= time in seconds of the specified transition

Table 113 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
n/a	:MEASure:TVALue? <value>, [<slope>]<occurrence> [,<source>] (see page 672)	<value> ::= voltage level that the waveform must cross. <slope> ::= direction of the waveform when <value> is crossed. <occurrence> ::= transitions reported. <source> ::= {CHANnel<n> FUNctIon<m> MATH<m> WMEMory<r>} for DSO models <source> ::= {CHANnel<n> DIGital<d> FUNctIon<m> MATH<m> WMEMory<r>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <return_value> ::= time in seconds of specified voltage crossing in NR3 format
:MEASure:VAMPlitude [<source>] (see page 674)	:MEASure:VAMPlitude? [<source>] (see page 674)	<source> ::= {CHANnel<n> FUNctIon<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= the amplitude of the selected waveform in volts in NR3 format

Table 113 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:VAVerage [<interval>] [, <source>] (see page 675)	:MEASure:VAVerage? [<interval>] [, <source>] (see page 675)	<interval> ::= {CYCLe DISPlay} <source> ::= {CHANnel<n> FUNCTION<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= calculated average voltage in NR3 format
:MEASure:VBASe [<source>] (see page 676)	:MEASure:VBASe? [<source>] (see page 676)	<source> ::= {CHANnel<n> FUNCTION<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <base_voltage> ::= voltage at the base of the selected waveform in NR3 format
:MEASure:VMAX [<source>] (see page 677)	:MEASure:VMAX? [<source>] (see page 677)	<source> ::= {CHANnel<n> FUNCTION<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= maximum voltage of the selected waveform in NR3 format

Table 113 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:VMIN [<source>] (see page 678)	:MEASure:VMIN? [<source>] (see page 678)	<source> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= minimum voltage of the selected waveform in NR3 format
:MEASure:VPP [<source>] (see page 679)	:MEASure:VPP? [<source>] (see page 679)	<source> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= voltage peak-to-peak of the selected waveform in NR3 format
:MEASure:VRATio [<interval>] [,<source 1>] [,<source2>] (see page 680)	:MEASure:VRATio? [<interval>] [,<source 1>] [,<source2>] (see page 680)	<interval> ::= {CYCLe DISPlay} <source1,2> ::= {CHANnel<n> FUNction<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= the ratio value in dB in NR3 format

Table 113 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:VRMS [<interval>] [,<type>] [,<source>] (see page 681)	:MEASure:VRMS? [<interval>] [,<type>] [,<source>] (see page 681)	<interval> ::= {CYCLe DISPlay} <type> ::= {AC DC} <source> ::= {CHANnel<n> FUNctIon<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= calculated dc RMS voltage in NR3 format
:MEASure:VTOP [<source>] (see page 682)	:MEASure:VTOP? [<source>] (see page 682)	<source> ::= {CHANnel<n> FUNctIon<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= voltage at the top of the waveform in NR3 format
:MEASure:WINDow <type> (see page 683)	:MEASure:WINDow? (see page 683)	<type> ::= {MAIN ZOOM AUTO GATE}
:MEASure:XMAX [<source>] (see page 684)	:MEASure:XMAX? [<source>] (see page 684)	<source> ::= {CHANnel<n> FUNctIon<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= horizontal value of the maximum in NR3 format

Table 113 :MEASure Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:XMIN [<source>] (see page 685)	:MEASure:XMIN? [<source>] (see page 685)	<source> ::= {CHANnel<n> FUNCTION<m> MATH<m> WMEMemory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= horizontal value of the minimum in NR3 format
:MEASure:YATX <horiz_location>[,<source>] (see page 686)	:MEASure:YATX? <horiz_location>[,<source>] (see page 686)	<horiz_location> ::= displayed time from trigger in seconds in NR3 format <source> ::= {CHANnel<n> FUNCTION<m> MATH<m> WMEMemory<r>} for DSO models <source> ::= {CHANnel<n> DIGital<d> FUNCTION<m> MATH<m> WMEMemory<r>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <return_value> ::= voltage at the specified time in NR3 format

Introduction to :MEASure Commands

The commands in the MEASure subsystem are used to make parametric measurements on displayed waveforms.

Measurement Setup

To make a measurement, the portion of the waveform required for that measurement must be displayed on the oscilloscope screen.

Measurement Type	Portion of waveform that must be displayed
period, duty cycle, or frequency	at least one complete cycle
pulse width	the entire pulse
rise time	rising edge, top and bottom of pulse
fall time	falling edge, top and bottom of pulse

Measurement Error

If a measurement cannot be made (typically because the proper portion of the waveform is not displayed), the value +9.9E+37 is returned for that measurement.

Making Measurements

If more than one waveform, edge, or pulse is displayed, time measurements are made on the portion of the displayed waveform closest to the trigger reference (left, center, or right).

When making measurements in the zoomed (delayed) time base mode (:TIMebase:MODE WINDow), the oscilloscope will attempt to make the measurement inside the zoomed sweep window. If the measurement is an average and there are not three edges, the oscilloscope will revert to the mode of making the measurement at the start of the main sweep.

When the command form is used, the measurement result is displayed on the instrument. When the query form of these measurements is used, the measurement is made one time, and the measurement result is returned over the bus.

Measurements are made on the displayed waveforms specified by the :MEASure:SOURce command. The MATH source is an alias for the FUNCtion source.

Not all measurements are available on the digital channels or FFT (Fast Fourier Transform).

Reporting the Setup

Use the :MEASure? query to obtain setup information for the MEASure subsystem. (Currently, this is only :MEASure:SOURce.)

Return Format

The following is a sample response from the :MEASure? query. In this case, the query was issued following a *RST command.

```
:MEAS:SOUR CHAN1,CHAN2;STAT ON
```

:MEASure:ALL

 (see [page 1650](#))

Command Syntax :MEASure:ALL

This command installs a Snapshot All measurement on the screen.

See Also • ["Introduction to :MEASure Commands"](#) on page 601

:MEASure:AREa

N (see [page 1650](#))

Command Syntax :MEASure:AREa [<interval>][,<source>]

<interval> ::= {CYCLe | DISPlay}

<source> ::= {CHANnel<n> | FUNctIon<m> | MATH<m> | WMEMory<r>}

<n> ::= 1 to (# analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:AREa command installs an area measurement on screen. Area measurements show the area between the waveform and the ground level.

The <interval> option lets you specify the measurement interval: either an integral number of cycles, or the full screen. If <interval> is not specified, DISPlay is implied.

NOTE

This command is not available if the source is FFT (Fast Fourier Transform).

Query Syntax :MEASure:AREa? [<interval>][,<source>]

The :MEASure:AREa? query measures and returns the area value.

Return Format <value><NL>

<value> ::= the area value in volt-seconds in NR3 format

See Also

- ["Introduction to :MEASure Commands"](#) on page 601
- [":MEASure:SOURce"](#) on page 661

:MEASure:BRATe

N (see [page 1650](#))

Command Syntax :MEASure:BRATe [<source>]

<source> ::= {<digital channels> | CHANnel<n> | FUNctIon<m> | MATH<m>
| WMEMory<r>}

<digital channels> ::= DIGital<d> for the MSO models

<d> ::= 0 to (# digital channels - 1) in NR1 format

<n> ::= 1 to (# of analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:BRATe command installs a screen measurement and starts the bit rate measurement. If the optional source parameter is specified, the currently specified source is modified.

NOTE

This command is not available if the source is FFT (Fast Fourier Transform).

Query Syntax :MEASure:BRATe? [<source>]

The :MEASure:BRATe? query measures all positive and negative pulse widths on the waveform, takes the minimum value found of either width type and inverts that minimum width to give a value in Hertz.

Return Format <value><NL>

<value> ::= the bit rate value in Hertz

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:SOURce"](#) on page 661
 - [":MEASure:FREQuency"](#) on page 628
 - [":MEASure:PERiod"](#) on page 649

:MEASure:BWIDth

N (see [page 1650](#))

Command Syntax :MEASure:BWIDth [<source>]

<source> ::= {CHANnel<n> | FUNCTION<m> | MATH<m> | WMEMory<r>}

<n> ::= 1 to (# analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:BWIDth command installs a burst width measurement on screen. If the optional source parameter is not specified, the current measurement source is used.

NOTE

This command is not available if the source is FFT (Fast Fourier Transform).

Query Syntax :MEASure:BWIDth? [<source>]

The :MEASure:BWIDth? query measures and returns the width of the burst on the screen.

The burst width is calculated as follows:

$$\text{burst width} = (\text{last edge on screen} - \text{first edge on screen})$$

Return Format <value><NL>

<value> ::= burst width in seconds in NR3 format

See Also

- ["Introduction to :MEASure Commands"](#) on page 601
- [":MEASure:SOURce"](#) on page 661

:MEASure:CLEar

 (see [page 1650](#))

Command Syntax :MEASure:CLEar

This command clears all selected measurements and markers from the screen.

See Also • ["Introduction to :MEASure Commands"](#) on page 601

:MEASure:COUNter

N (see [page 1650](#))

Command Syntax :MEASure:COUNter [<source>]

<source> ::= {<digital channels> | CHANnel<n> | EXTernal}

<digital channels> ::= DIGital<d> for the MSO models

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :MEASure:COUNter command installs a screen measurement and starts a counter measurement. If the optional source parameter is specified, the current source is modified. Any channel except Math or Reference Waveforms may be selected for the source.

The counter measurement counts trigger level crossings within a certain amount of time (gate time) and displays the results in Hz.

The gate time is the horizontal range of the oscilloscope but is limited to ≥ 0.1 s and ≤ 10 s. Unlike other measurements, the Zoom horizontal timebase window does not gate the Counter measurement.

The Counter measurement can measure frequencies up to the bandwidth of the oscilloscope. The minimum frequency supported is $2.0 / \text{gateTime}$.

Only one counter measurement may be displayed at a time.

NOTE

This command is not available if the source is MATH.

Query Syntax :MEASure:COUNter? [<source>]

The :MEASure:COUNter? query measures and outputs the counter frequency of the specified source.

NOTE

The :MEASure:COUNter? query times out if the counter measurement is installed on the front panel. Use :MEASure:CLEar to remove the front-panel measurement before executing the :MEASure:COUNter? query.

Return Format <source><NL>

<source> ::= count in Hertz in NR3 format

See Also

- ["Introduction to :MEASure Commands"](#) on page 601
- [":MEASure:SOURce"](#) on page 661

- **":MEASure:FREQuency"** on page 628
- **":MEASure:CLEar"** on page 607

:MEASure:DEFine

N (see [page 1650](#))

Command Syntax :MEASure:DEFine <meas_spec>[,<source>]

<meas_spec> ::= {DELAy | THResholds}, for remaining syntax, see:

- [":MEASure:DEFine DELAy Command Syntax"](#) on page 610
- [":MEASure:DEFine THResholds Command Syntax"](#) on page 611

The :MEASure:DEFine command sets up the definition for measurements by specifying the delta time or threshold values. Changing these values may affect the results of other measure commands. The table below identifies which measurement results that can be affected by redefining the DELAy specification or the THResholds values. For example, changing the THResholds definition from the default 10%, 50%, and 90% values may change the returned measurement result.

MEASure Command	DELAy	THResholds
DUTYcycle		x
DELAy	x	x
FALLtime		x
FREQuency		x
NWIDth		x
OVERshoot		x
PERiod		x
PHASe		x
PREShoot		x
PWIDth		x
RISetime		x
VAverage		x
VRMS		x

:MEASure:DEFine
DELAy Command
Syntax

NOTE

The DELAy portion of the MEASure:DEFine command has been deprecated, and you should instead use the :MEASure:DELAy:DEFine command to specify delay measurement parameters. The delay measurement now allows an edge occurrence of zero (0) to specify automatic edge selection. The limitation of this command is that you cannot specify a negative slope with an occurrence count of zero (0).

```

:MEASure:DEFine DELay,<delay spec>[,<source>]

<delay spec> ::= <edge_spec1>,<edge_spec2>

<edge_spec1> ::= [<slope>]<occurrence>

<edge_spec2> ::= [<slope>]<occurrence>

<slope> ::= {+ | -}

<occurrence> ::= integer

<source> ::= {CHANnel<n> | FUNCTION<m> | MATH<m> | WMemory<r>}

<n> ::= 1 to (# analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

```

This command defines the behavior of the :MEASure:DELay command/query by specifying the start and stop edge to be used. <edge_spec1> specifies the slope and edge number on source1. <edge_spec2> specifies the slope and edge number on source2. The measurement is taken as:

$$\text{delay} = t(\text{<edge_spec2>}) - t(\text{<edge_spec1>})$$

:MEASure:DEFine THResholds Command Syntax

```

:MEASure:DEFine THResholds,<threshold spec>[,<source>]

<threshold spec> ::= {STANdard
                      | {<threshold mode>,<upper>,<middle>,<lower>}}

<threshold mode> ::= {PERCent | ABSolute}

```

for <threshold mode> = PERCent:

```

<upper>,<middle>,<lower> ::= A number specifying the upper, middle,
                           and lower threshold percentage values
                           between Vbase and Vtop in NR3 format.

```

for <threshold mode> = ABSolute:

```

<upper>,<middle>,<lower> ::= A number specifying the upper, middle,
                           and lower threshold absolute values in
                           NR3 format.

```

```

<source> ::= {CHANnel<n> | FUNCTION<m> | MATH<m> | WMemory<r>}

<n> ::= 1 to (# analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

```

- STANdard threshold specification sets the lower, middle, and upper measurement thresholds to 10%, 50%, and 90% values between Vbase and Vtop.
- Threshold mode PERCent sets the measurement thresholds to any user-defined percentages between 5% and 95% of values between Vbase and Vtop.

- Threshold mode ABSolute sets the measurement thresholds to absolute values. ABSolute thresholds are dependent on channel scaling (:CHANnel<n>:RANGe or :CHANnel<n>:SCALE), probe attenuation (:CHANnel<n>:PROBe), and probe units (:CHANnel<n>:UNITs). Always set these values first before setting ABSolute thresholds.

Query Syntax :MEASure:DEFine? <meas_spec>[,<source>]

<meas_spec> ::= {DELay | THResholds}

The :MEASure:DEFine? query returns the current edge specification for the delay measurements setup or the current specification for the thresholds setup.

Return Format for <meas_spec> = DELay:

{ <edge_spec1> | <edge_spec2> | <edge_spec1>,<edge_spec2>} <NL>

for <meas_spec> = THResholds and <threshold mode> = PERCent:

THR,PERC,<upper>,<middle>,<lower><NL>

<upper>, <middle>, <lower> ::= A number specifying the upper, middle, and lower threshold percentage values between Vbase and Vtop in NR3 format.

for <meas_spec> = THResholds and <threshold mode> = ABSolute:

THR,ABS,<upper>,<middle>,<lower><NL>

<upper>, <middle>, <lower> ::= A number specifying the upper, middle, and lower threshold voltages in NR3 format.

for <threshold spec> = STANDard:

THR,PERC,+90.0,+50.0,+10.0

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:DELay"](#) on page 613
 - [":MEASure:DELay:DEFine"](#) on page 615
 - [":MEASure:SOURce"](#) on page 661
 - [":CHANnel<n>:RANGe"](#) on page 353
 - [":CHANnel<n>:SCALE"](#) on page 354
 - [":CHANnel<n>:PROBe"](#) on page 337
 - [":CHANnel<n>:UNITs"](#) on page 355

:MEASure:DElay

N (see [page 1650](#))

Command Syntax :MEASure:DElay [<edge_select_mode>][,][<source1>][,<source2>]

<edge_select_mode> ::= {MANual | AUTO}

<source1>, <source2> ::= {CHANnel<n> | FUNction<m> | MATH<m>
| WMEMory<r> | <digital channels>}

<n> ::= 1 to (# analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

<digital channels> ::= DIGital<d> for the MSO models

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :MEASure:DElay command places the instrument in the continuous measurement mode and starts a delay measurement.

The measurement is taken as:

$$\text{delay} = t(\text{<edge_spec_2>}) - t(\text{<edge_spec_1>})$$

where the <edge_spec> definitions are determined by the <edge_select_mode> and the :MEASure:DElay:DEFine command.

When the <edge_select_mode> is:

- AUTO – edges are automatically selected: the source1 edge closest to the timebase reference point is used, and the source2 edge closest to the source1 edge is used.
- MANual – edge numbers are determined by the :MEASure:DElay:DEFine settings.

Edge numbers greater than zero (0) are counted from the left side of the display for both sources.

Edge thresholds and slopes are always determined by the :MEASure:DElay:DEFine settings.

If the <edge_select_mode> is not included with the command, AUTO is the default.

Query Syntax :MEASure:DElay? [<edge_select_mode>][,][<source1>][,<source2>]

The :MEASure:DElay? query measures and returns the delay between source1 and source2. Delay measurement threshold, slope, and edge count parameters are determined by the <edge_select_mode> and the :MEASure:DElay:DEFine command.

The <edge_select_mode> definitions are the same as with the command syntax.

However, if the <edge_select_mode> is not included with the query, MANual is the default.

In the :MEASure:DEFine command, you can set upper, middle, and lower threshold values. The standard upper, middle, and lower measurement thresholds are 90%, 50%, and 10% values between Vbase and Vtop.

Return Format <value><NL>

<value> ::= floating-point number delay time in seconds in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:DElay:DEFine"](#) on page 615
 - [":MEASure:DEFine"](#) on page 610
 - [":MEASure:PHASe"](#) on page 650

:MEASure:DElay:DEFine

N (see [page 1650](#))

Command Syntax :MEASure:DElay:DEFine <source1_edge_slope>, <source1_edge_number>,
<source1_edge_threshold>, <source2_edge_slope>,
<source2_edge_number>, <source2_edge_threshold>

<source1_edge_slope>,
<source2_edge_slope> ::= {RISing | FALLing}

<source1_edge_number>,
<source2_edge_number> ::= 0 to 1000 in NR1 format

<source1_edge_threshold>,
<source2_edge_threshold> ::= {LOWer | MIDDle | UPPer}

The :MEASure:DElay:DEFine command defines slope directions and edge numbers for the delay measurement started or returned by the :MEASure:DElay command. The :MEASure:DElay:DEFine command also defines edge position parameters.

A <source1_edge_number> setting of zero (0) specifies that the edge closest to the timebase reference point automatically be selected. In this case, the <source2_edge_number> setting must also be zero (0), and the source2 edge closest to the selected source1 edge is used.

When edge numbers greater than zero (0) are specified, edges are counted from the left side of the display for both sources.

The selection of the source1 and source2 waveforms is made in the :MEASure:DElay or :MEASure:SOURce commands.

Query Syntax :MEASure:DElay:DEFine?

The :MEASure:DElay:DEFine? query returns the specified delay measurement parameter definitions.

Return Format <source1_edge_slope>, <source1_edge_number>,
<source1_edge_threshold>, <source2_edge_slope>,
<source2_edge_number>, <source2_edge_threshold><NL>

<source1_edge_slope>,
<source2_edge_slope> ::= {RIS | FALL}

<source1_edge_number>,
<source2_edge_number> ::= 0 to 1000 in NR1 format

<source1_edge_threshold>,
<source2_edge_threshold> ::= {LOW | MIDD | UPP}

- See Also**
- [":MEASure:DElay"](#) on page 613
 - [":MEASure:SOURce"](#) on page 661
 - [":MEASure:DEFine"](#) on page 610

:MEASure:DUAL:CHARge

N (see [page 1650](#))

Overview This measurement is available with the N2820A high sensitivity current probe when both the Primary and Secondary probe cables are used. This measurement joins the Zoom In waveform data below the probe's clamp level with Zoom Out waveform data above the probe's clamp level to create the waveform on which the measurement is made.

Command Syntax :MEASure:DUAL:CHARge [<interval>] [,<source1>] [,<source2>]
<interval> ::= {CYCLe | DISPlay}
<source1>,<source2> ::= CHANnel<n> with N2820A probe connected
<n> ::= 1 to (# analog channels) in NR1 format

The :MEASure:DUAL:CHARge command installs a charge measurement on screen. Charge measurements show the area between the waveform and the ground level.

The <interval> option lets you specify the measurement interval: either an integral number of cycles, or the full screen. If <interval> is not specified, DISPlay is implied.

If the optional source parameter(s) are specified, the currently specified source(s) are modified.

Query Syntax :MEASure:DUAL:CHARge? [<interval>] [,<source1>] [,<source2>]

The :MEASure:DUAL:CHARge? query measures and returns the charge measurement value.

Return Format <value><NL>
<value> ::= the charge value in Amp-hours in NR3 format

- See Also**
- [":MEASure:DUAL:VAMPlitude"](#) on page 617
 - [":MEASure:DUAL:VAverage"](#) on page 618
 - [":MEASure:DUAL:VBASe"](#) on page 619
 - [":MEASure:DUAL:VPP"](#) on page 620
 - [":MEASure:DUAL:VRMS"](#) on page 621
 - ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:SOURce"](#) on page 661

:MEASure:DUAL:VAMPlitude

N (see [page 1650](#))

Overview This measurement is available with the N2820A high sensitivity current probe when both the Primary and Secondary probe cables are used. This measurement joins the Zoom In waveform data below the probe's clamp level with Zoom Out waveform data above the probe's clamp level to create the waveform on which the measurement is made.

Command Syntax :MEASure:DUAL:VAMPlitude [<source1>] [,<source2>]
 <source1>,<source2> ::= CHANnel<n> with N2820A probe connected
 <n> ::= 1 to (# analog channels) in NR1 format
 The :MEASure:DUAL:VAMPlitude command installs a screen measurement and starts a vertical amplitude measurement.
 If the optional source parameter(s) are specified, the currently specified source(s) are modified.

Query Syntax :MEASure:DUAL:VAMPlitude? [<source1>] [,<source2>]
 The :MEASure:DUAL:VAMPlitude? query measures and returns the vertical amplitude of the waveform. To determine the amplitude, the instrument measures Vtop and Vbase, then calculates the amplitude as follows:

$$\text{vertical amplitude} = V_{\text{top}} - V_{\text{base}}$$

Return Format <value><NL>
 <value> ::= the amplitude of the selected waveform in NR3 format

- See Also**
- [":MEASure:DUAL:CHARge"](#) on page 616
 - [":MEASure:DUAL:VAVerage"](#) on page 618
 - [":MEASure:DUAL:VBASe"](#) on page 619
 - [":MEASure:DUAL:VPP"](#) on page 620
 - [":MEASure:DUAL:VRMS"](#) on page 621
 - ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:SOURce"](#) on page 661
 - [":MEASure:VTOP"](#) on page 682

:MEASure:DUAL:VAverage

N (see [page 1650](#))

Overview This measurement is available with the N2820A high sensitivity current probe when both the Primary and Secondary probe cables are used. This measurement joins the Zoom In waveform data below the probe's clamp level with Zoom Out waveform data above the probe's clamp level to create the waveform on which the measurement is made.

Command Syntax :MEASure:DUAL:VAverage [<interval>] [,<source1>] [,<source2>]
<interval> ::= {CYCLE | DISPlay}
<source1>,<source2> ::= CHANnel<n> with N2820A probe connected
<n> ::= 1 to (# analog channels) in NR1 format

The :MEASure:DUAL:VAverage command installs a screen measurement and starts an average value measurement.

The <interval> option lets you specify the measurement interval: either an integral number of cycles, or the full screen. If <interval> is not specified, DISPlay is implied.

If the optional source parameter(s) are specified, the currently specified source(s) are modified.

Query Syntax :MEASure:DUAL:VAverage? [<interval>] [,<source1>] [,<source2>]

The :MEASure:DUAL:VAverage? query returns the average value measurement.

Return Format <value><NL>
<value> ::= calculated average value in NR3 format

- See Also**
- [":MEASure:DUAL:CHARge"](#) on page 616
 - [":MEASure:DUAL:VAMPlitude"](#) on page 617
 - [":MEASure:DUAL:VBASe"](#) on page 619
 - [":MEASure:DUAL:VPP"](#) on page 620
 - [":MEASure:DUAL:VRMS"](#) on page 621
 - ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:SOURce"](#) on page 661

:MEASure:DUAL:VBASe

N (see [page 1650](#))

Overview This measurement is available with the N2820A high sensitivity current probe when both the Primary and Secondary probe cables are used. This measurement joins the Zoom In waveform data below the probe's clamp level with Zoom Out waveform data above the probe's clamp level to create the waveform on which the measurement is made.

Command Syntax :MEASure:DUAL:VBASe [<source1>] [, <source2>]
 <source1>, <source2> ::= CHANnel<n> with N2820A probe connected
 <n> ::= 1 to (# analog channels) in NR1 format
 The :MEASure:DUAL:VBASe command installs a screen measurement and starts a waveform base value measurement.
 If the optional source parameter(s) are specified, the currently specified source(s) are modified.

Query Syntax :MEASure:DUAL:VBASe? [<source1>] [, <source2>]
 The :MEASure:DUAL:VBASe? query returns the vertical value at the base of the waveform. The base value of a pulse is normally not the same as the minimum value.

Return Format <base_voltage><NL>
 <base_voltage> ::= value at the base of the selected waveform in
 NR3 format

- See Also**
- [":MEASure:DUAL:CHARge"](#) on page 616
 - [":MEASure:DUAL:VAMPlitude"](#) on page 617
 - [":MEASure:DUAL:VAVerage"](#) on page 618
 - [":MEASure:DUAL:VPP"](#) on page 620
 - [":MEASure:DUAL:VRMS"](#) on page 621
 - ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:SOURce"](#) on page 661
 - [":MEASure:VTOP"](#) on page 682
 - [":MEASure:VMIN"](#) on page 678

:MEASure:DUAL:VPP

N (see [page 1650](#))

Overview This measurement is available with the N2820A high sensitivity current probe when both the Primary and Secondary probe cables are used. This measurement joins the Zoom In waveform data below the probe's clamp level with Zoom Out waveform data above the probe's clamp level to create the waveform on which the measurement is made.

Command Syntax :MEASure:DUAL:VPP [<source1>] [,<source2>]
<source1>,<source2> ::= CHANnel<n> with N2820A probe connected
<n> ::= 1 to (# analog channels) in NR1 format

The :MEASure:DUAL:VPP command installs a screen measurement and starts a vertical peak-to-peak measurement.

If the optional source parameter(s) are specified, the currently specified source(s) are modified.

Query Syntax :MEASure:DUAL:VPP? [<source1>] [,<source2>]

The :MEASure:DUAL:VPP? query measures the maximum and minimum vertical value for the selected source, then calculates the vertical peak-to-peak value and returns that value. The peak-to-peak value (Vpp) is calculated with the following formula:

$$V_{pp} = V_{max} - V_{min}$$

Vmax and Vmin are the vertical maximum and minimum values present on the selected source.

Return Format <value><NL>
<value> ::= vertical peak to peak value in NR3 format

- See Also**
- [":MEASure:DUAL:CHARge"](#) on page 616
 - [":MEASure:DUAL:VAMplitude"](#) on page 617
 - [":MEASure:DUAL:VAverage"](#) on page 618
 - [":MEASure:DUAL:VBASe"](#) on page 619
 - [":MEASure:DUAL:VRMS"](#) on page 621
 - ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:SOURce"](#) on page 661
 - [":MEASure:VMAX"](#) on page 677
 - [":MEASure:VMIN"](#) on page 678

:MEASure:DUAL:VRMS

N (see [page 1650](#))

Overview This measurement is available with the N2820A high sensitivity current probe when both the Primary and Secondary probe cables are used. This measurement joins the Zoom In waveform data below the probe's clamp level with Zoom Out waveform data above the probe's clamp level to create the waveform on which the measurement is made.

Command Syntax :MEASure:DUAL:VRMS [<interval>] [, <type>] [, <source1>] [, <source2>]

<interval> ::= {CYCLe | DISPlay}

<type> ::= {AC | DC}

<source1>, <source2> ::= CHANnel<n> with N2820A probe connected

<n> ::= 1 to (# analog channels) in NR1 format

The :MEASure:DUAL:VRMS command installs a screen measurement and starts an RMS value measurement.

The <interval> option lets you specify the measurement interval: either an integral number of cycles, or the full screen. If <interval> is not specified, DISPlay is implied.

The <type> option lets you choose between a DC RMS measurement and an AC RMS measurement. If <type> is not specified, DC is implied.

If the optional source parameter(s) are specified, the currently specified source(s) are modified.

Query Syntax :MEASure:DUAL:VRMS? [<interval>] [, <type>] [, <source1>] [, <source2>]

The :MEASure:DUAL:VRMS? query measures and outputs the RMS value measurement.

Return Format <value><NL>

<value> ::= calculated dc RMS value in NR3 format

- See Also**
- [":MEASure:DUAL:CHARge"](#) on page 616
 - [":MEASure:DUAL:VAMPlitude"](#) on page 617
 - [":MEASure:DUAL:VAVerage"](#) on page 618
 - [":MEASure:DUAL:VBASe"](#) on page 619
 - [":MEASure:DUAL:VPP"](#) on page 620
 - ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:SOURce"](#) on page 661

:MEASure:DUTYcycle

C (see [page 1650](#))

Command Syntax :MEASure:DUTYcycle [<source>]

<source> ::= {<digital channels> | CHANnel<n> | FUNction<m> | MATH<m>
| WMEMory<r>}

<digital channels> ::= DIGital<d> for the MSO models

<d> ::= 0 to (# digital channels - 1) in NR1 format

<n> ::= 1 to (# of analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:DUTYcycle command installs a screen measurement and starts a positive duty cycle measurement on the current :MEASure:SOURce. If the optional source parameter is specified, the current source is modified.

NOTE

The signal must be displayed to make the measurement. This command is not available if the source is FFT (Fast Fourier Transform).

Query Syntax :MEASure:DUTYcycle? [<source>]

The :MEASure:DUTYcycle? query measures and outputs the positive duty cycle of the signal specified by the :MEASure:SOURce command. The value returned for the duty cycle is the ratio of the positive pulse width to the period. The positive pulse width and the period of the specified signal are measured, then the duty cycle is calculated with the following formula:

$$+duty\ cycle = (+pulse\ width/period)*100$$

Return Format <value><NL>

<value> ::= ratio of positive pulse width to period in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:PERiod"](#) on page 649
 - [":MEASure:PWIDth"](#) on page 653
 - [":MEASure:SOURce"](#) on page 661
 - [":MEASure:NDUTy"](#) on page 642

Example Code • ["Example Code"](#) on page 662

:MEASure:FALLtime

C (see [page 1650](#))

Command Syntax :MEASure:FALLtime [<source>]
 <source> ::= {CHANnel<n> | FUNCTION<m> | MATH<m> | WMEMory<r>}
 <n> ::= 1 to (# analog channels) in NR1 format
 <m> ::= 1 to (# math functions) in NR1 format
 <r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:FALLtime command installs a screen measurement and starts a fall-time measurement. For highest measurement accuracy, set the sweep speed as fast as possible, while leaving the falling edge of the waveform on the display. If the optional source parameter is specified, the current source is modified.

NOTE

This command is not available if the source is FFT (Fast Fourier Transform).

Query Syntax :MEASure:FALLtime? [<source>]

The :MEASure:FALLtime? query measures and outputs the fall time of the displayed falling (negative-going) edge closest to the timebase reference. The fall time is determined by measuring the time at the upper threshold of the falling edge, then measuring the time at the lower threshold of the falling edge, and calculating the fall time with the following formula:

$$\text{fall time} = \text{time at lower threshold} - \text{time at upper threshold}$$

Return Format <value><NL>
 <value> ::= time in seconds between the lower threshold and upper threshold in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:RISetime"](#) on page 657
 - [":MEASure:SOURce"](#) on page 661
 - [":MEASure:SLEWrate"](#) on page 660

:MEASure:FFT:ACPR

 (see [page 1650](#))

Command Syntax :MEASure:FFT:ACPR <chan_width>,<chan_spacing>,<chan>[,<source>]

<chan_width> ::= of main range and sideband channels, Hz in NR3 format

<chan_spacing> ::= spacing between main range and sideband channels, Hz in NR3 format

<chan> ::= {CENTer | HIGH<sb> | LOW<sb>}

<sb> ::= sideband 1 to 5

<source> ::= {FUNctIon<m> | MATH<m> | FFT} (must be an FFT waveform)

<m> ::= 1 to (# math functions) in NR1 format

The :MEASure:FFT:ACPR command installs an FFT analysis Adjacent Channel Power Ratio (ACPR) measurement on screen.

Adjacent Channel Power Ratio (or channel leakage ratio) measures the ratio of the power in the main frequency range to the power contained in one or more sidebands.

The main range is specified by a channel width and a center frequency. The center frequency used in the measurement is the one defined for the FFT function.

Sidebands (with the same width as the main range) exist above and below the main range separated by the channel spacing width.

The sideband used for the measurement is selected with the <chan> parameter. You can select the first through fifth sidebands above or below the main range (HIGH1 through HIGH5 above and LOW1 through LOW5 below). The full sideband must be in the graticule (on screen) to be measured. Otherwise, the measurement results will be "Incomplete".

When this measurement is tracked with cursors, the cursors show the sideband being measured.

Query Syntax :MEASure:FFT:ACPR? <chan_width>,<chan_spacing>,<chan>[,<source>]

The :MEASure:FFT:ACPR? query returns the measured Adjacent Channel Power Ratio (ACPR) value.

Return Format <return_value><NL>

<return_value> ::= adjacent channel power ratio, dBV in NR3 format

- See Also**
- [":MEASure:FFT:CPOWer"](#) on page 625
 - [":MEASure:FFT:OBW"](#) on page 626
 - [":MEASure:FFT:THD"](#) on page 627

:MEASure:FFT:CPOWer

N (see [page 1650](#))

Command Syntax :MEASure:FFT:CPOWer [<source>]
 <source> ::= {FUNCTION<m> | MATH<m> | FFT} (must be an FFT waveform)
 <m> ::= 1 to (# math functions) in NR1 format

The :MEASure:FFT:CPOWer command installs an FFT analysis Channel Power measurement on screen.

Channel Power measures the spectral power across a frequency range.

The center frequency used in the measurement is the one defined for the FFT function, and the FFT span specifies the frequency range.

When this measurement is tracked with cursors, the cursors are at the left and right edges of the frequency span.

Query Syntax :MEASure:FFT:CPOWer? [<source>]

The :MEASure:FFT:CPOWer? query returns the measured Channel Power value.

Return Format <return_value><NL>
 <return_value> ::= spectral channel power, dBV in NR3 format

- See Also**
- [":MEASure:FFT:ACPR"](#) on page 624
 - [":MEASure:FFT:OBW"](#) on page 626
 - [":MEASure:FFT:THD"](#) on page 627

:MEASure:FFT:OBW

N (see [page 1650](#))

Command Syntax :MEASure:FFT:OBW <percentage>[,<source>]

<percentage> ::= percent of spectral power occupied bandwidth is measured for (in NR3 format)

<source> ::= {FUNCTION<m> | MATH<m> | FFT} (must be an FFT waveform)

<m> ::= 1 to (# math functions) in NR1 format

The :MEASure:FFT:OBW command installs an FFT analysis Occupied Bandwidth measurement on screen.

Occupied Bandwidth measures the bandwidth (frequency range) containing some percent (usually 99%) of the total spectral power. While 99% is the industry norm, you can specify the percent you want to use in the measurement.

The center frequency used in the measurement is the one defined for the FFT function, and the FFT span represents the total spectral power.

When this measurement is tracked with cursors, the cursors show the measured bandwidth (frequency range).

Query Syntax :MEASure:FFT:OBW? <percentage>[,<source>]

The :MEASure:FFT:OBW? query returns the measured Occupied Bandwidth value.

Return Format <return_value><NL>

<return_value> ::= occupied bandwidth, Hz in NR3 format

- See Also**
- [":MEASure:FFT:ACPR"](#) on page 624
 - [":MEASure:FFT:CPOWer"](#) on page 625
 - [":MEASure:FFT:THD"](#) on page 627

:MEASure:FFT:THD

N (see [page 1650](#))

Command Syntax :MEASure:FFT:THD <tracking>[,<fundamental_freq>][,<source>]
 <tracking> ::= {AUTO | MANual}
 <fundamental_freq> ::= in NR3 format, required if <tracking> is MANual
 <source> ::= {FUNCTION<m> | MATH<m> | FFT} (must be an FFT waveform)
 <m> ::= 1 to (# math functions) in NR1 format

The :MEASure:FFT:THD command installs an FFT analysis Total Harmonic Distortion measurement on screen.

Total Harmonic Distortion (THD) is the ratio of power in the fundamental frequency to the power contained in the rest of the harmonics and noise. THD is a measure of signal purity.

Total Harmonic Distortion (THD) measures the power contained in the bands surrounding each harmonic and compares it to the power in the band surrounding the fundamental frequency. The width of the bands measured is the same for the fundamental frequency and each harmonic. That width is 1/2 of the fundamental frequency.

You can either enter the fundamental frequency as a measurement parameter and have the fundamental frequency and harmonics be tracked manually, or you can allow the fundamental frequency and harmonics to be tracked automatically, where the highest peak is assumed to be the fundamental frequency.

When this measurement is tracked with cursors, the cursors show the band surrounding the fundamental frequency that is being measured (at $\pm 1/4$ of the fundamental frequency).

Query Syntax :MEASure:FFT:THD? <tracking>[,<fundamental_freq>][,<source>]

The :MEASure:FFT:THD? query returns the measured Total Harmonic Distortion value.

Return Format <return_value><NL>
 <return_value> ::= total harmonic distortion ratio percent in NR3 format

See Also

- [":MEASure:FFT:ACPR"](#) on page 624
- [":MEASure:FFT:CPOWer"](#) on page 625
- [":MEASure:FFT:OBW"](#) on page 626

:MEASure:FREQuency

C (see [page 1650](#))

Command Syntax :MEASure:FREQuency [<source>]

<source> ::= {<digital channels> | CHANnel<n> | FUNction<m> | MATH<m>
| WMEMory<r>}

<digital channels> ::= DIGital<d> for the MSO models

<d> ::= 0 to (# digital channels - 1) in NR1 format

<n> ::= 1 to (# of analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:FREQuency command installs a screen measurement and starts a frequency measurement. If the optional source parameter is specified, the current source is modified.

IF the edge on the screen closest to the trigger reference is rising:

THEN frequency = 1/(time at trailing rising edge - time at leading rising edge)

ELSE frequency = 1/(time at trailing falling edge - time at leading falling edge)

NOTE

This command is not available if the source is FFT (Fast Fourier Transform).

Query Syntax :MEASure:FREQuency? [<source>]

The :MEASure:FREQuency? query measures and outputs the frequency of the cycle on the screen closest to the trigger reference.

Return Format <source><NL>

<source> ::= frequency in Hertz in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:SOURce"](#) on page 661
 - [":MEASure:PERiod"](#) on page 649

Example Code • ["Example Code"](#) on page 662

:MEASure:HISTogram:BWIDth

N (see [page 1650](#))

Query Syntax :MEASure:HISTogram:BWIDth?

The :MEASure:HISTogram:BWIDth? query returns the width of each bin (bucket) in the histogram.

Return Format <value><NL>

<value> ::= bin width in NR3 format

- See Also**
- [Chapter 22](#), “:HISTogram Commands,” starting on page 525
 - [":MEASure:HISTogram:HITS"](#) on page 630
 - [":MEASure:HISTogram:M1S"](#) on page 631
 - [":MEASure:HISTogram:M2S"](#) on page 632
 - [":MEASure:HISTogram:M3S"](#) on page 633
 - [":MEASure:HISTogram:MAXimum"](#) on page 634
 - [":MEASure:HISTogram:MEAN"](#) on page 635
 - [":MEASure:HISTogram:MEDian"](#) on page 636
 - [":MEASure:HISTogram:MINimum"](#) on page 637
 - [":MEASure:HISTogram:MODE"](#) on page 638
 - [":MEASure:HISTogram:PEAK"](#) on page 639
 - [":MEASure:HISTogram:PPEak"](#) on page 640
 - [":MEASure:HISTogram:SDEViation"](#) on page 641

:MEASure:HISTogram:HITS

N (see [page 1650](#))

Query Syntax :MEASure:HISTogram:HITS?

The :MEASure:HISTogram:HITS? query returns the sum of all hits in all bins (buckets) in the histogram.

Return Format <value><NL>

<value> ::= # of total hits in NR1 format

- See Also**
- [Chapter 22](#), “:HISTogram Commands,” starting on page 525
 - [":MEASure:HISTogram:BWIDth"](#) on page 629
 - [":MEASure:HISTogram:M1S"](#) on page 631
 - [":MEASure:HISTogram:M2S"](#) on page 632
 - [":MEASure:HISTogram:M3S"](#) on page 633
 - [":MEASure:HISTogram:MAXimum"](#) on page 634
 - [":MEASure:HISTogram:MEAN"](#) on page 635
 - [":MEASure:HISTogram:MEDian"](#) on page 636
 - [":MEASure:HISTogram:MINimum"](#) on page 637
 - [":MEASure:HISTogram:MODE"](#) on page 638
 - [":MEASure:HISTogram:PEAK"](#) on page 639
 - [":MEASure:HISTogram:PPEak"](#) on page 640
 - [":MEASure:HISTogram:SDEViation"](#) on page 641

:MEASure:HISTogram:M1S

N (see [page 1650](#))

Query Syntax :MEASure:HISTogram:M1S?

The :MEASure:HISTogram:M1S? query returns the percentage of histogram hits that lie within ± 1 Std Dev of the mean.

Return Format <value><NL>

<value> ::= % of hits within +/-1 std dev in NR3 format

- See Also**
- [Chapter 22](#), “:HISTogram Commands,” starting on page 525
 - [":MEASure:HISTogram:BWIDth"](#) on page 629
 - [":MEASure:HISTogram:HITS"](#) on page 630
 - [":MEASure:HISTogram:M2S"](#) on page 632
 - [":MEASure:HISTogram:M3S"](#) on page 633
 - [":MEASure:HISTogram:MAXimum"](#) on page 634
 - [":MEASure:HISTogram:MEAN"](#) on page 635
 - [":MEASure:HISTogram:MEDian"](#) on page 636
 - [":MEASure:HISTogram:MINimum"](#) on page 637
 - [":MEASure:HISTogram:MODE"](#) on page 638
 - [":MEASure:HISTogram:PEAK"](#) on page 639
 - [":MEASure:HISTogram:PPEak"](#) on page 640
 - [":MEASure:HISTogram:SDEViation"](#) on page 641

:MEASure:HISTogram:M2S

N (see [page 1650](#))

Query Syntax :MEASure:HISTogram:M2S?

The :MEASure:HISTogram:M2S? query returns the percentage of histogram hits that lie within ± 2 Std Dev of the mean.

Return Format <value><NL>

<value> ::= % of hits within +/-2 std devs in NR3 format

See Also • [Chapter 22](#), “:HISTogram Commands,” starting on page 525

- [":MEASure:HISTogram:BWIDth"](#) on page 629
- [":MEASure:HISTogram:HITS"](#) on page 630
- [":MEASure:HISTogram:M1S"](#) on page 631
- [":MEASure:HISTogram:M3S"](#) on page 633
- [":MEASure:HISTogram:MAXimum"](#) on page 634
- [":MEASure:HISTogram:MEAN"](#) on page 635
- [":MEASure:HISTogram:MEDian"](#) on page 636
- [":MEASure:HISTogram:MINimum"](#) on page 637
- [":MEASure:HISTogram:MODE"](#) on page 638
- [":MEASure:HISTogram:PEAK"](#) on page 639
- [":MEASure:HISTogram:PPEak"](#) on page 640
- [":MEASure:HISTogram:SDEViation"](#) on page 641

:MEASure:HISTogram:M3S

N (see [page 1650](#))

Query Syntax :MEASure:HISTogram:M3S?

The :MEASure:HISTogram:M3S? query returns the percentage of histogram hits that lie within ± 3 Std Dev of the mean.

Return Format <value><NL>

<value> ::= % of hits within +/-3 std devs in NR3 format

See Also • [Chapter 22](#), “:HISTogram Commands,” starting on page 525

- [":MEASure:HISTogram:BWIDth"](#) on page 629
- [":MEASure:HISTogram:HITS"](#) on page 630
- [":MEASure:HISTogram:M1S"](#) on page 631
- [":MEASure:HISTogram:M2S"](#) on page 632
- [":MEASure:HISTogram:MAXimum"](#) on page 634
- [":MEASure:HISTogram:MEAN"](#) on page 635
- [":MEASure:HISTogram:MEDian"](#) on page 636
- [":MEASure:HISTogram:MINimum"](#) on page 637
- [":MEASure:HISTogram:MODE"](#) on page 638
- [":MEASure:HISTogram:PEAK"](#) on page 639
- [":MEASure:HISTogram:PPEak"](#) on page 640
- [":MEASure:HISTogram:SDEViation"](#) on page 641

:MEASure:HISTogram:MAXimum

N (see [page 1650](#))

Query Syntax :MEASure:HISTogram:MAXimum?

The :MEASure:HISTogram:MAXimum? query returns the value corresponding to the maximum bin that has any hits.

Return Format <value><NL>

<value> ::= value of max bin in NR3 format

- See Also**
- [Chapter 22](#), “:HISTogram Commands,” starting on page 525
 - [":MEASure:HISTogram:BWIDth"](#) on page 629
 - [":MEASure:HISTogram:HITS"](#) on page 630
 - [":MEASure:HISTogram:M1S"](#) on page 631
 - [":MEASure:HISTogram:M2S"](#) on page 632
 - [":MEASure:HISTogram:M3S"](#) on page 633
 - [":MEASure:HISTogram:MEAN"](#) on page 635
 - [":MEASure:HISTogram:MEDian"](#) on page 636
 - [":MEASure:HISTogram:MINimum"](#) on page 637
 - [":MEASure:HISTogram:MODE"](#) on page 638
 - [":MEASure:HISTogram:PEAK"](#) on page 639
 - [":MEASure:HISTogram:PPEak"](#) on page 640
 - [":MEASure:HISTogram:SDEViation"](#) on page 641

:MEASure:HISTogram:MEAN

N (see [page 1650](#))

Query Syntax :MEASure:HISTogram:MEAN?

The :MEASure:HISTogram:MEAN? query returns the mean value of the histogram.

Return Format <value><NL>

<value> ::= mean of histogram in NR3 format

- See Also**
- [Chapter 22](#), “:HISTogram Commands,” starting on page 525
 - [":MEASure:HISTogram:BWIDth"](#) on page 629
 - [":MEASure:HISTogram:HITS"](#) on page 630
 - [":MEASure:HISTogram:M1S"](#) on page 631
 - [":MEASure:HISTogram:M2S"](#) on page 632
 - [":MEASure:HISTogram:M3S"](#) on page 633
 - [":MEASure:HISTogram:MAXimum"](#) on page 634
 - [":MEASure:HISTogram:MEDian"](#) on page 636
 - [":MEASure:HISTogram:MINimum"](#) on page 637
 - [":MEASure:HISTogram:MODE"](#) on page 638
 - [":MEASure:HISTogram:PEAK"](#) on page 639
 - [":MEASure:HISTogram:PPEak"](#) on page 640
 - [":MEASure:HISTogram:SDEViation"](#) on page 641

:MEASure:HISTogram:MEDian

N (see [page 1650](#))

Query Syntax :MEASure:HISTogram:MEDian?

The :MEASure:HISTogram:MEDian? query returns the median value of the histogram.

Return Format <value><NL>

<value> ::= median of histogram in NR3 format

- See Also**
- [Chapter 22](#), “:HISTogram Commands,” starting on page 525
 - [":MEASure:HISTogram:BWIDth"](#) on page 629
 - [":MEASure:HISTogram:HITS"](#) on page 630
 - [":MEASure:HISTogram:M1S"](#) on page 631
 - [":MEASure:HISTogram:M2S"](#) on page 632
 - [":MEASure:HISTogram:M3S"](#) on page 633
 - [":MEASure:HISTogram:MAXimum"](#) on page 634
 - [":MEASure:HISTogram:MEAN"](#) on page 635
 - [":MEASure:HISTogram:MINimum"](#) on page 637
 - [":MEASure:HISTogram:MODE"](#) on page 638
 - [":MEASure:HISTogram:PEAK"](#) on page 639
 - [":MEASure:HISTogram:PPEak"](#) on page 640
 - [":MEASure:HISTogram:SDEViation"](#) on page 641

:MEASure:HISTogram:MINimum

N (see [page 1650](#))

Query Syntax :MEASure:HISTogram:MINimum?

The :MEASure:HISTogram:MINimum? query returns the value corresponding to the minimum bin that has any hits.

Return Format <value><NL>

<value> ::= value of min bin in NR3 format

- See Also**
- [Chapter 22](#), “:HISTogram Commands,” starting on page 525
 - [":MEASure:HISTogram:BWIDth"](#) on page 629
 - [":MEASure:HISTogram:HITS"](#) on page 630
 - [":MEASure:HISTogram:M1S"](#) on page 631
 - [":MEASure:HISTogram:M2S"](#) on page 632
 - [":MEASure:HISTogram:M3S"](#) on page 633
 - [":MEASure:HISTogram:MAXimum"](#) on page 634
 - [":MEASure:HISTogram:MEAN"](#) on page 635
 - [":MEASure:HISTogram:MEDian"](#) on page 636
 - [":MEASure:HISTogram:MODE"](#) on page 638
 - [":MEASure:HISTogram:PEAK"](#) on page 639
 - [":MEASure:HISTogram:PPEak"](#) on page 640
 - [":MEASure:HISTogram:SDEViation"](#) on page 641

:MEASure:HISTogram:MODE

N (see [page 1650](#))

Query Syntax :MEASure:HISTogram:MODE?

The :MEASure:HISTogram:MODE? query returns the mode value of the histogram.

Return Format <value><NL>

<value> ::= mode of histogram in NR3 format

- See Also**
- [Chapter 22](#), “:HISTogram Commands,” starting on page 525
 - [":MEASure:HISTogram:BWIDth"](#) on page 629
 - [":MEASure:HISTogram:HITS"](#) on page 630
 - [":MEASure:HISTogram:M1S"](#) on page 631
 - [":MEASure:HISTogram:M2S"](#) on page 632
 - [":MEASure:HISTogram:M3S"](#) on page 633
 - [":MEASure:HISTogram:MAXimum"](#) on page 634
 - [":MEASure:HISTogram:MEAN"](#) on page 635
 - [":MEASure:HISTogram:MEDian"](#) on page 636
 - [":MEASure:HISTogram:MINimum"](#) on page 637
 - [":MEASure:HISTogram:PEAK"](#) on page 639
 - [":MEASure:HISTogram:PPEak"](#) on page 640
 - [":MEASure:HISTogram:SDEViation"](#) on page 641

:MEASure:HISTogram:PEAK

N (see [page 1650](#))

Query Syntax :MEASure:HISTogram:PEAK?

The :MEASure:HISTogram:PEAK? query returns the maximum number of hits in any single bin.

Return Format <value><NL>

<value> ::= # of hits in max bin in NR1 format

- See Also**
- [Chapter 22](#), “:HISTogram Commands,” starting on page 525
 - [":MEASure:HISTogram:BWIDth"](#) on page 629
 - [":MEASure:HISTogram:HITS"](#) on page 630
 - [":MEASure:HISTogram:M1S"](#) on page 631
 - [":MEASure:HISTogram:M2S"](#) on page 632
 - [":MEASure:HISTogram:M3S"](#) on page 633
 - [":MEASure:HISTogram:MAXimum"](#) on page 634
 - [":MEASure:HISTogram:MEAN"](#) on page 635
 - [":MEASure:HISTogram:MEDian"](#) on page 636
 - [":MEASure:HISTogram:MINimum"](#) on page 637
 - [":MEASure:HISTogram:MODE"](#) on page 638
 - [":MEASure:HISTogram:PPEak"](#) on page 640
 - [":MEASure:HISTogram:SDEViation"](#) on page 641

:MEASure:HISTogram:PPEak

N (see [page 1650](#))

Query Syntax :MEASure:HISTogram:PPEak?

The :MEASure:HISTogram:PPEak? query returns the delta between the Max and Min values.

Return Format <value><NL>

<value> ::= delta value between max bin and min bin in NR3 format

See Also • [Chapter 22](#), “:HISTogram Commands,” starting on page 525

- [":MEASure:HISTogram:BWIDth"](#) on page 629
- [":MEASure:HISTogram:HITS"](#) on page 630
- [":MEASure:HISTogram:M1S"](#) on page 631
- [":MEASure:HISTogram:M2S"](#) on page 632
- [":MEASure:HISTogram:M3S"](#) on page 633
- [":MEASure:HISTogram:MAXimum"](#) on page 634
- [":MEASure:HISTogram:MEAN"](#) on page 635
- [":MEASure:HISTogram:MEDian"](#) on page 636
- [":MEASure:HISTogram:MINimum"](#) on page 637
- [":MEASure:HISTogram:MODE"](#) on page 638
- [":MEASure:HISTogram:PEAK"](#) on page 639
- [":MEASure:HISTogram:SDEViation"](#) on page 641

:MEASure:HISTogram:SDEViation

N (see [page 1650](#))

Query Syntax :MEASure:HISTogram:SDEViation?

The :MEASure:HISTogram:SDEViation? query returns the standard deviation of the histogram values.

Return Format <value><NL>

<value> ::= standard deviation of histogram in NR3 format

- See Also**
- [Chapter 22](#), “:HISTogram Commands,” starting on page 525
 - [":MEASure:HISTogram:BWIDth"](#) on page 629
 - [":MEASure:HISTogram:HITS"](#) on page 630
 - [":MEASure:HISTogram:M1S"](#) on page 631
 - [":MEASure:HISTogram:M2S"](#) on page 632
 - [":MEASure:HISTogram:M3S"](#) on page 633
 - [":MEASure:HISTogram:MAXimum"](#) on page 634
 - [":MEASure:HISTogram:MEAN"](#) on page 635
 - [":MEASure:HISTogram:MEDian"](#) on page 636
 - [":MEASure:HISTogram:MINimum"](#) on page 637
 - [":MEASure:HISTogram:MODE"](#) on page 638
 - [":MEASure:HISTogram:PEAK"](#) on page 639
 - [":MEASure:HISTogram:PPEak"](#) on page 640

:MEASure:NDUTy

N (see [page 1650](#))

Command Syntax :MEASure:NDUTy [<source>]

<source> ::= {<digital channels> | CHANnel<n> | FUNction<m> | MATH<m>
| WMEMory<r>}

<digital channels> ::= DIGital<d> for the MSO models

<d> ::= 0 to (# digital channels - 1) in NR1 format

<n> ::= 1 to (# of analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:NDUTy command installs a screen measurement and starts a negative duty cycle measurement on the current :MEASure:SOURce. If the optional source parameter is specified, the current source is modified.

NOTE

The signal must be displayed to make the measurement. This command is not available if the source is FFT (Fast Fourier Transform).

Query Syntax :MEASure:NDUTy? [<source>]

The :MEASure:NDUTy? query measures and outputs the negative duty cycle of the signal specified by the :MEASure:SOURce command. The value returned for the duty cycle is the ratio of the negative pulse width to the period. The negative pulse width and the period of the specified signal are measured, then the duty cycle is calculated with the following formula:

$$\text{-duty cycle} = (-\text{pulse width}/\text{period}) * 100$$

Return Format <value><NL>

<value> ::= ratio of negative pulse width to period in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:PERiod"](#) on page 649
 - [":MEASure:NWIDth"](#) on page 645
 - [":MEASure:SOURce"](#) on page 661
 - [":MEASure:DUTYcycle"](#) on page 622

:MEASure:NEDGes

N (see [page 1650](#))

Command Syntax :MEASure:NEDGes [<source>]

<source> ::= {CHANnel<n> | FUNction<m> | MATH<m> | WMEMory<r>}

<n> ::= 1 to (# analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:NEDGes command installs a falling edge count measurement on screen. If the optional source parameter is not specified, the current source is measured.

NOTE

This command is not available if the source is FFT (Fast Fourier Transform).

Query Syntax :MEASure:NEDGes? [<source>]

The :MEASure:NEDGes? query measures and returns the on-screen falling edge count.

Return Format <value><NL>

<value> ::= the falling edge count in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:SOURce"](#) on page 661

:MEASure:NPULses

N (see [page 1650](#))

Command Syntax :MEASure:NPULses [<source>]

<source> ::= {CHANnel<n> | FUNction<m> | MATH<m> | WMEMory<r>
| <digital channels>}

<n> ::= 1 to (# analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

<digital channels> ::= DIGital<d> for the MSO models

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :MEASure:NPULses command installs a falling pulse count measurement on screen. If the optional source parameter is not specified, the current source is measured.

NOTE

This command is not available if the source is FFT (Fast Fourier Transform).

Query Syntax :MEASure:NPULses? [<source>]

The :MEASure:NPULses? query measures and returns the on-screen falling pulse count.

Return Format <value><NL>

<value> ::= the falling pulse count in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:SOURce"](#) on page 661

:MEASure:NWIDth

C (see [page 1650](#))

Command Syntax :MEASure:NWIDth [<source>]

<source> ::= {<digital channels> | CHANnel<n> | FUNction<m> | MATH<m>
| WMEMory<r>}

<digital channels> ::= DIGital<d> for the MSO models

<d> ::= 0 to (# digital channels - 1) in NR1 format

<n> ::= 1 to (# of analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:NWIDth command installs a screen measurement and starts a negative pulse width measurement. If the optional source parameter is not specified, the current source is modified.

NOTE

This command is not available if the source is FFT (Fast Fourier Transform).

Query Syntax :MEASure:NWIDth? [<source>]

The :MEASure:NWIDth? query measures and outputs the width of the negative pulse on the screen closest to the trigger reference using the midpoint between the upper and lower thresholds.

FOR the negative pulse closest to the trigger point:

width = (time at trailing rising edge - time at leading falling edge)

Return Format <value><NL>

<value> ::= negative pulse width in seconds in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:SOURce"](#) on page 661
 - [":MEASure:PWIDth"](#) on page 653
 - [":MEASure:PERiod"](#) on page 649

:MEASure:OVERshoot

C (see [page 1650](#))

Command Syntax :MEASure:OVERshoot [<source>]

<source> ::= {CHANnel<n> | FUNCTION<m> | MATH<m> | WMEMory<r>}

<n> ::= 1 to (# analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:OVERshoot command installs a screen measurement and starts an overshoot measurement. If the optional source parameter is specified, the current source is modified.

NOTE

This command is not available if the source is FFT (Fast Fourier Transform).

Query Syntax :MEASure:OVERshoot? [<source>]

The :MEASure:OVERshoot? query measures and returns the overshoot of the edge closest to the trigger reference, displayed on the screen. The method used to determine overshoot is to make three different vertical value measurements: Vtop, Vbase, and either Vmax or Vmin, depending on whether the edge is rising or falling.

For a rising edge:

$$\text{overshoot} = ((V_{\text{max}} - V_{\text{top}}) / (V_{\text{top}} - V_{\text{base}})) \times 100$$

For a falling edge:

$$\text{overshoot} = ((V_{\text{base}} - V_{\text{min}}) / (V_{\text{top}} - V_{\text{base}})) \times 100$$

Vtop and Vbase are taken from the normal histogram of all waveform vertical values. The extremum of Vmax or Vmin is taken from the waveform interval right after the chosen edge, halfway to the next edge. This more restricted definition is used instead of the normal one, because it is conceivable that a signal may have more preshoot than overshoot, and the normal extremum would then be dominated by the preshoot of the following edge.

Return Format <overshoot><NL>

<overshoot> ::= the percent of the overshoot of the selected waveform in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:PREShoot"](#) on page 652
 - [":MEASure:SOURce"](#) on page 661

- **":MEASure:VMAX"** on page 677
- **":MEASure:VTOP"** on page 682
- **":MEASure:VBASe"** on page 676
- **":MEASure:VMIN"** on page 678

:MEASure:PEDGes

N (see [page 1650](#))

Command Syntax :MEASure:PEDGes [<source>]

<source> ::= {CHANnel<n> | FUNction<m> | MATH<m> | WMEMory<r>}

<n> ::= 1 to (# analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:PEDGes command installs a rising edge count measurement on screen. If the optional source parameter is not specified, the current source is measured.

NOTE

This command is not available if the source is FFT (Fast Fourier Transform).

Query Syntax :MEASure:PEDGes? [<source>]

The :MEASure:NEDGes? query measures and returns the on-screen rising edge count.

Return Format <value><NL>

<value> ::= the rising edge count in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:SOURce"](#) on page 661

:MEASure:PERiod

C (see [page 1650](#))

Command Syntax :MEASure:PERiod [<source>]

<source> ::= {<digital channels> | CHANnel<n> | FUNction<m> | MATH<m>
| WMEMory<r>}

<digital channels> ::= DIGital<d> for the MSO models

<d> ::= 0 to (# digital channels - 1) in NR1 format

<n> ::= 1 to (# of analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:PERiod command installs a screen measurement and starts the period measurement. If the optional source parameter is specified, the current source is modified.

NOTE

This command is not available if the source is FFT (Fast Fourier Transform).

Query Syntax :MEASure:PERiod? [<source>]

The :MEASure:PERiod? query measures and outputs the period of the cycle closest to the trigger reference on the screen. The period is measured at the midpoint of the upper and lower thresholds.

IF the edge closest to the trigger reference on screen is rising:

THEN period = (time at trailing rising edge - time at leading rising edge)

ELSE period = (time at trailing falling edge - time at leading falling edge)

Return Format <value><NL>

<value> ::= waveform period in seconds in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:SOURce"](#) on page 661
 - [":MEASure:NWIDth"](#) on page 645
 - [":MEASure:PWIDth"](#) on page 653
 - [":MEASure:FREQuency"](#) on page 628

Example Code • ["Example Code"](#) on page 662

:MEASure:PHASe

N (see [page 1650](#))

Command Syntax :MEASure:PHASe [<source1>] [,<source2>]

<source1>, <source2> ::= {CHANnel<n> | FUNction<m> | MATH<m> | WMEMory<r>}

<n> ::= 1 to (# analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:PHASe command places the instrument in the continuous measurement mode and starts a phase measurement.

Query Syntax :MEASure:PHASe? [<source1>] [,<source2>]

The :MEASure:PHASe? query measures and returns the phase between the specified sources.

A phase measurement is a combination of the period and delay measurements. First, the period is measured on source1. Then the delay is measured between source1 and source2. The edges used for delay are the source1 rising edge used for the period measurement closest to the horizontal reference and the rising edge on source 2. See :MEASure:DElay for more detail on selecting the 2nd edge.

The phase is calculated as follows:

$$\text{phase} = (\text{delay} / \text{period of input 1}) \times 360$$

Return Format <value><NL>

<value> ::= the phase angle value in degrees in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:DElay"](#) on page 613
 - [":MEASure:PERiod"](#) on page 649
 - [":MEASure:SOURce"](#) on page 661

:MEASure:PPULses

N (see [page 1650](#))

Command Syntax :MEASure:PPULses [<source>]

<source> ::= {CHANnel<n> | FUNction<m> | MATH<m> | WMEMory<r>
| <digital channels>}

<n> ::= 1 to (# analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

<digital channels> ::= DIGital<d> for the MSO models

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :MEASure:PPULses command installs a rising pulse count measurement on screen. If the optional source parameter is not specified, the current source is measured.

NOTE

This command is not available if the source is FFT (Fast Fourier Transform).

Query Syntax :MEASure:PPULses? [<source>]

The :MEASure:PPULses? query measures and returns the on-screen rising pulse count.

Return Format <value><NL>

<value> ::= the rising pulse count in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:SOURce"](#) on page 661

:MEASure:PREShoot

C (see [page 1650](#))

Command Syntax :MEASure:PREShoot [<source>]

<source> ::= {CHANnel<n> | FUNction<m> | MATH<m> | WMEMory<r>}

<n> ::= 1 to (# analog channels) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:PREShoot command installs a screen measurement and starts a preshoot measurement. If the optional source parameter is specified, the current source is modified.

Query Syntax :MEASure:PREShoot? [<source>]

The :MEASure:PREShoot? query measures and returns the preshoot of the edge closest to the trigger, displayed on the screen. The method used to determine preshoot is to make three different vertical value measurements: Vtop, Vbase, and either Vmin or Vmax, depending on whether the edge is rising or falling.

For a rising edge:

$$\text{preshoot} = ((V_{\text{min}} - V_{\text{base}}) / (V_{\text{top}} - V_{\text{base}})) \times 100$$

For a falling edge:

$$\text{preshoot} = ((V_{\text{max}} - V_{\text{top}}) / (V_{\text{top}} - V_{\text{base}})) \times 100$$

Vtop and Vbase are taken from the normal histogram of all waveform vertical values. The extremum of Vmax or Vmin is taken from the waveform interval right before the chosen edge, halfway back to the previous edge. This more restricted definition is used instead of the normal one, because it is likely that a signal may have more overshoot than preshoot, and the normal extremum would then be dominated by the overshoot of the preceding edge.

Return Format <value><NL>

<value> ::= the percent of preshoot of the selected waveform
in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:SOURce"](#) on page 661
 - [":MEASure:VMIN"](#) on page 678
 - [":MEASure:VMAX"](#) on page 677
 - [":MEASure:VTOP"](#) on page 682
 - [":MEASure:VBASe"](#) on page 676

:MEASure:PWIDth

C (see [page 1650](#))

Command Syntax :MEASure:PWIDth [<source>]

<source> ::= {<digital channels> | CHANnel<n> | FUNction<m> | MATH<m>
| WMEMory<r>}

<digital channels> ::= DIGital<d> for the MSO models

<d> ::= 0 to (# digital channels - 1) in NR1 format

<n> ::= 1 to (# of analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:PWIDth command installs a screen measurement and starts the positive pulse width measurement. If the optional source parameter is specified, the current source is modified.

NOTE

This command is not available if the source is FFT (Fast Fourier Transform).

Query Syntax :MEASure:PWIDth? [<source>]

The :MEASure:PWIDth? query measures and outputs the width of the displayed positive pulse closest to the trigger reference. Pulse width is measured at the midpoint of the upper and lower thresholds.

IF the edge on the screen closest to the trigger is falling:

THEN width = (time at trailing falling edge - time at leading rising edge)

ELSE width = (time at leading falling edge - time at leading rising edge)

Return Format <value><NL>

<value> ::= width of positive pulse in seconds in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:SOURce"](#) on page 661
 - [":MEASure:NWIDth"](#) on page 645
 - [":MEASure:PERiod"](#) on page 649

:MEASure:RESults

N (see [page 1650](#))

Query Syntax :MEASure:RESults?

The :MEASure:RESults? query returns the results of the continuously displayed measurements. The response to the MEASure:RESults? query is a list of comma-separated values.

If more than one measurement is running continuously, the :MEASure:RESults return values are duplicated for each continuous measurement from the first to last (top to bottom) result displayed. Each result returned is separated from the previous result by a comma. There is a maximum of 10 continuous measurements that can be continuously displayed at a time.

When no quick measurements are installed, the :MEASure:RESults? query returns nothing (empty string). When the count for any of the measurements is 0, the value of infinity (9.9E+37) is returned for the min, max, mean, and standard deviation.

Return Format <result_list><NL>

<result_list> ::= comma-separated list of measurement results

The following shows the order of values received for a single measurement if :MEASure:STATistics is set to ON.

Measureme nt label	current	min	max	mean	std dev	count
-----------------------	---------	-----	-----	------	---------	-------

Measurement label, current, min, max, mean, std dev, and count are only returned if :MEASure:STATistics is ON.

If :MEASure:STATistics is set to CURRent, MIN, MAX, MEAN, STDDev, or COUNT only that particular statistic value is returned for each measurement that is on.

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:STATistics"](#) on page 663

Example Code

```
' This program shows the InfiniiVision oscilloscopes' measurement
' statistics commands.
' -----
```

Option Explicit

```
Public myMgr As VisaComLib.ResourceManager
Public myScope As VisaComLib.FormattedIO488
Public varQueryResult As Variant
Public strQueryResult As String
```

```

Private Declare Sub Sleep Lib "kernel32" (ByVal dwMilliseconds As Long)

Sub Main()

    On Error GoTo VisaComError

    ' Create the VISA COM I/O resource.
    Set myMgr = New VisaComLib.ResourceManager
    Set myScope = New VisaComLib.FormattedIO488
    Set myScope.IO = myMgr.Open("TCPIP0::130.29.70.228::inst0::INSTR")

    ' Initialize.
    myScope.IO.Clear ' Clear the interface.
    myScope.WriteString "*RST" ' Reset to the defaults.
    myScope.WriteString "*CLS" ' Clear the status data structures.
    myScope.WriteString ":AUToscale"

    ' Install some measurements.
    myScope.WriteString ":MEASure:SOURce CHANnel1" ' Input source.

    Dim MeasurementArray(3) As String
    MeasurementArray(0) = "FREQuency"
    MeasurementArray(1) = "DUTYcycle"
    MeasurementArray(2) = "VAMPlitude"
    MeasurementArray(3) = "VPP"
    Dim Measurement As Variant

    For Each Measurement In MeasurementArray
        myScope.WriteString ":MEASure:" + Measurement
        myScope.WriteString ":MEASure:" + Measurement + "?"
        varQueryResult = myScope.ReadNumber ' Read measurement value.
        Debug.Print Measurement + ": " + FormatNumber(varQueryResult, 4)
    Next

    myScope.WriteString ":MEASure:STATistics:RESet" ' Reset stats.
    Sleep 5000 ' Wait for 5 seconds.

    ' Select the statistics results type.
    Dim ResultsTypeArray(6) As String
    ResultsTypeArray(0) = "CURRent"
    ResultsTypeArray(1) = "MINimum"
    ResultsTypeArray(2) = "MAXimum"
    ResultsTypeArray(3) = "MEAN"
    ResultsTypeArray(4) = "STDDev"
    ResultsTypeArray(5) = "COUNT"
    ResultsTypeArray(6) = "ON" ' All results.
    Dim ResultType As Variant

    Dim ResultsList()

    Dim ValueColumnArray(6) As String
    ValueColumnArray(0) = "Meas_Lbl"
    ValueColumnArray(1) = "Current"
    ValueColumnArray(2) = "Min"
    ValueColumnArray(3) = "Max"
    ValueColumnArray(4) = "Mean"
    ValueColumnArray(5) = "Std_Dev"

```

```

ValueColumnArray(6) = "Count"
Dim ValueColumn As Variant

For Each ResultType In ResultsTypeArray
    myScope.WriteString ":MEASure:STATistics " + ResultType

    ' Get the statistics results.
    Dim intCounter As Integer
    intCounter = 0
    myScope.WriteString ":MEASure:RESults?"
    ResultsList() = myScope.ReadList

    For Each Measurement In MeasurementArray

        If ResultType = "ON" Then ' All statistics.

            For Each ValueColumn In ValueColumnArray
                If VarType(ResultsList(intCounter)) <> vbString Then
                    Debug.Print "Measure statistics result CH1, " + _
                        Measurement + ", "; ValueColumn + ": " + _
                        FormatNumber(ResultsList(intCounter), 4)

                Else ' Result is a string (e.g., measurement label).
                    Debug.Print "Measure statistics result CH1, " + _
                        Measurement + ", "; ValueColumn + ": " + _
                        ResultsList(intCounter)

                End If

                intCounter = intCounter + 1
            Next

        Else ' Specific statistic (e.g., Current, Max, Min, etc.).

            Debug.Print "Measure statistics result CH1, " + _
                Measurement + ", "; ResultType + ": " + _
                FormatNumber(ResultsList(intCounter), 4)

            intCounter = intCounter + 1

        End If

    Next

Next

Exit Sub

VisaComError:
    MsgBox "VISA COM Error:" + vbCrLf + Err.Description

End Sub

```

:MEASure:RISetime

C (see [page 1650](#))

Command Syntax :MEASure:RISetime [<source>]
 <source> ::= {CHANnel<n> | FUNCTION<m> | MATH<m> | WMemory<r>}
 <n> ::= 1 to (# analog channels) in NR1 format
 <m> ::= 1 to (# math functions) in NR1 format
 <r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:RISetime command installs a screen measurement and starts a rise-time measurement. If the optional source parameter is specified, the current source is modified.

NOTE

This command is not available if the source is FFT (Fast Fourier Transform).

Query Syntax :MEASure:RISetime? [<source>]

The :MEASure:RISetime? query measures and outputs the rise time of the displayed rising (positive-going) edge closest to the timebase reference. For maximum measurement accuracy, set the sweep speed as fast as possible while leaving the leading edge of the waveform on the display. The rise time is determined by measuring the time at the lower threshold of the rising edge and the time at the upper threshold of the rising edge, then calculating the rise time with the following formula:

$$\text{rise time} = \text{time at upper threshold} - \text{time at lower threshold}$$

Return Format <value><NL>
 <value> ::= rise time in seconds in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:SOURce"](#) on page 661
 - [":MEASure:FALLtime"](#) on page 623
 - [":MEASure:SLEWrate"](#) on page 660

:MEASure:SDEViation

N (see [page 1650](#))

Command Syntax :MEASure:SDEViation [<source>]

<source> ::= {CHANnel<n> | FUNction<m> | MATH<m> | WMEMory<r>}

<n> ::= 1-2 or 1-4 (# of analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

NOTE

This ":MEASure:VRMS DISPlay, AC" command is the preferred syntax for making standard deviation measurements.

The :MEASure:SDEViation command installs a screen measurement and starts std deviation measurement. If the optional source parameter is specified, the current source is modified.

NOTE

This command is not available if the source is FFT (Fast Fourier Transform).

Query Syntax :MEASure:SDEViation? [<source>]

The :MEASure:SDEViation? query measures and outputs the std deviation of the selected waveform. The oscilloscope computes the std deviation on all displayed data points.

Return Format <value><NL>

<value> ::= calculated std deviation value in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:VRMS"](#) on page 681
 - [":MEASure:SOURce"](#) on page 661

:MEASure:SHOW

N (see [page 1650](#))

Command Syntax :MEASure:SHOW <on_off>

<on_off> ::= {{0 | OFF} | {1 | ON}}

The :MEASure:SHOW command enables markers for tracking measurements on the display.

Query Syntax :MEASure:SHOW?

The :MEASure:SHOW? query returns the current state of the markers.

This can return OFF when :MARKer:MODE selects a mode other than MEASurement.

Return Format <on_off><NL>

<on_off> ::= {1 | 0}

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MARKer:MODE"](#) on page 563

:MEASure:SLEWrate

N (see [page 1650](#))

Command Syntax :MEASure:SLEWrate [<source>[,<slope>]]

<source> ::= {CHANnel<n> | FUNCTION<m> | MATH<m> | WMEMory<r>}

<n> ::= 1 to (# of analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

<slope> ::= {RISing | FALLing | EITHer}

The :MEASure:SLEWrate command installs a slew rate measurement on the oscilloscope's front-panel screen.

The slew rate of an edge is the change in vertical value between the lower and upper thresholds of a waveform source (typically volts) divided by the change in time between where the waveform threshold crossings take place.

The lower and upper thresholds for the measurement are specified by the measurement threshold settings for the source waveform (see :MEASure:DEFine). If the <source> parameter is not specified, the current :MEASure:SOURce setting is used.

The on-screen edge closest to the timebase reference is measured.

You can specify that a RISing edge be measured, a FALLing edge be measured, or either a rising or a falling edge be measured (EITHer). If the <slope> is not specified, a rising edge is measured.

NOTE

This command is not available if the source is a FFT (Fast Fourier Transform) waveform.

Query Syntax :MEASure:SLEWrate? [<source>[,<slope>]]

The :MEASure:SLEWrate? query returns the measured slew rate of the specified edge closest to the timebase reference.

Return Format <value><NL>

<value> ::= slew rate in vertical units/second in NR3 format

- See Also**
- [":MEASure:DEFine"](#) on page 610
 - [":MEASure:SOURce"](#) on page 661
 - [":MEASure:RISetime"](#) on page 657
 - [":MEASure:FALLtime"](#) on page 623
 - ["Introduction to :MEASure Commands"](#) on page 601

:MEASure:SOURce

C (see [page 1650](#))

Command Syntax :MEASure:SOURce <source1>[,<source2>]

<source1>,<source2> ::= {<digital channels> | CHANnel<n> | FUNcTion<m>
| MATH<m> | WMEMory<r>}

<digital channels> ::= DIGital<d> for the MSO models

<n> ::= 1 to (# of analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1-2 in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :MEASure:SOURce command sets the default sources for measurements. The specified sources are used as the sources for the MEASure subsystem commands if the sources are not explicitly set with the command.

If a source is specified for any measurement, the current source is changed to this new value.

If :MARKer:MODE is set to OFF or MANual, setting :MEASure:SOURce to CHANnel<n>, FUNcTion<m>, or MATH<m> will also set :MARKer:X1Y1source to source1 and :MARKer:X2Y2source to source2.

Query Syntax :MEASure:SOURce?

The :MEASure:SOURce? query returns the current source selections. If source2 is not specified, the query returns "NONE" for source2. If all channels are off, the query returns "NONE,NONE". Source2 applies only to :MEASure:DElay and :MEASure:PHASe measurements.

NOTE

MATH<m> is an alias for FUNcTion<m>. The query will return FUNC<m> if the source is FUNcTion<m> or MATH<m>.

NOTE

FUNcTion or MATH (without the "<m>" math function number) is an alias for FUNcTion2 or MATH2. The query will return FUNC2 if the source is FUNcTion or MATH.

Return Format <source1>,<source2><NL>

<source1>,<source2> ::= {<digital channels> | CHAN<n> | FUNC<m>
| WMEM<r> | NONE}

- See Also:**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MARKer:MODE"](#) on page 563

- **":MARKer:X1Y1source"** on page 566
- **":MARKer:X2Y2source"** on page 569
- **":MEASure:DElay"** on page 613
- **":MEASure:PHASe"** on page 650

Example Code

```
' MEASURE - The commands in the MEASure subsystem are used to make
' measurements on displayed waveforms.
myScope.WriteString ":MEASure:SOURce CHANnel1" ' Source to measure.
myScope.WriteString ":MEASure:FREQuency?" ' Query for frequency.
varQueryResult = myScope.ReadNumber ' Read frequency.
MsgBox "Frequency:" + vbCrLf _
    + FormatNumber(varQueryResult / 1000, 4) + " kHz"
myScope.WriteString ":MEASure:DUTYcycle?" ' Query for duty cycle.
varQueryResult = myScope.ReadNumber ' Read duty cycle.
MsgBox "Duty cycle:" + vbCrLf _
    + FormatNumber(varQueryResult, 3) + "%"
myScope.WriteString ":MEASure:RISetime?" ' Query for risetime.
varQueryResult = myScope.ReadNumber ' Read risetime.
MsgBox "Risetime:" + vbCrLf _
    + FormatNumber(varQueryResult * 1000000, 4) + " us"
myScope.WriteString ":MEASure:VPP?" ' Query for Pk to Pk voltage.
varQueryResult = myScope.ReadNumber ' Read VPP.
MsgBox "Peak to peak voltage:" + vbCrLf _
    + FormatNumber(varQueryResult, 4) + " V"
myScope.WriteString ":MEASure:VMAX?" ' Query for Vmax.
varQueryResult = myScope.ReadNumber ' Read Vmax.
MsgBox "Maximum voltage:" + vbCrLf _
    + FormatNumber(varQueryResult, 4) + " V"
```

See complete example programs at: **Chapter 46**, "Programming Examples," starting on page 1659

:MEASure:STATistics

N (see [page 1650](#))

Command Syntax :MEASure:STATistics <type>

<type> ::= {{ON | 1} | CURRent | MINimum | MAXimum | MEAN | STDDev
| COUNT}

The :MEASure:STATistics command determines the type of information returned by the :MEASure:RESults? query. ON means all the statistics are on.

Query Syntax :MEASure:STATistics?

The :MEASure:STATistics? query returns the current statistics mode.

Return Format <type><NL>

<type> ::= {ON | CURR | MIN | MAX | MEAN | STDD | COUN}

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:RESults"](#) on page 654
 - [":MEASure:STATistics:DISPlay"](#) on page 664
 - [":MEASure:STATistics:RESet"](#) on page 667
 - [":MEASure:STATistics:INCRement"](#) on page 665

Example Code • ["Example Code"](#) on page 654

:MEASure:STATistics:DISPlay

N (see [page 1650](#))

Command Syntax :MEASure:STATistics:DISPlay {{0 | OFF} | {1 | ON}}

The :MEASure:STATistics:DISPlay command disables or enables the display of the measurement statistics.

Query Syntax :MEASure:STATistics:DISPlay?

The :MEASure:STATistics:DISPlay? query returns the state of the measurement statistics display.

Return Format {0 | 1}<NL>

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:RESults"](#) on page 654
 - [":MEASure:STATistics"](#) on page 663
 - [":MEASure:STATistics:MCOunt"](#) on page 666
 - [":MEASure:STATistics:RESet"](#) on page 667
 - [":MEASure:STATistics:INCRement"](#) on page 665
 - [":MEASure:STATistics:RSDeviation"](#) on page 668

:MEASure:STATistics:INCRement

N (see [page 1650](#))

Command Syntax :MEASure:STATistics:INCRement

This command updates the statistics once (incrementing the count by one) using the current measurement values. It corresponds to the front panel **Increment Statistics** softkey in the Measurement Statistics Menu. This command lets you, for example, gather statistics over multiple pulses captured in a single acquisition. To do this, change the horizontal position and enter the command for each new pulse that is measured.

This command is only allowed when the oscilloscope is stopped and quick measurements are on.

The command is allowed in segmented acquisition mode even though the corresponding front panel softkey is not available.

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:STATistics"](#) on page 663
 - [":MEASure:STATistics:DISPlay"](#) on page 664
 - [":MEASure:STATistics:RESet"](#) on page 667
 - [":MEASure:RESults"](#) on page 654

:MEASure:STATistics:MCOunt

N (see [page 1650](#))

Command Syntax :MEASure:STATistics:MCOunt <setting>

<setting> ::= {INFinite | <count>}

<count> ::= 2 to 2000 in NR1 format

The :MEASure:STATistics:MCOunt command specifies the maximum number of values used when calculating measurement statistics.

Query Syntax :MEASure:STATistics:MCOunt?

The :MEASure:STATistics:MCOunt? query returns the current measurement statistics max count setting.

Return Format <setting><NL>

<setting> ::= {INF | <count>}

<count> ::= 2 to 2000

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:RESults"](#) on page 654
 - [":MEASure:STATistics"](#) on page 663
 - [":MEASure:STATistics:DISPlay"](#) on page 664
 - [":MEASure:STATistics:RSDeviation"](#) on page 668
 - [":MEASure:STATistics:RESet"](#) on page 667
 - [":MEASure:STATistics:INCRement"](#) on page 665

:MEASure:STATistics:RESet

N (see [page 1650](#))

Command Syntax :MEASure:STATistics:RESet

This command resets the measurement statistics, zeroing the counts.

Note that the measurement (statistics) configuration is not deleted.

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:STATistics"](#) on page 663
 - [":MEASure:STATistics:DISPlay"](#) on page 664
 - [":MEASure:RESults"](#) on page 654
 - [":MEASure:STATistics:INCRement"](#) on page 665

- Example Code**
- ["Example Code"](#) on page 654

:MEASure:STATistics:RSDeviation

N (see [page 1650](#))

Command Syntax :MEASure:STATistics:RSDeviation {{0 | OFF} | {1 | ON}}

The :MEASure:STATistics:RSDeviation command disables or enables relative standard deviations, that is, standard deviation/mean, in the measurement statistics.

Query Syntax :MEASure:STATistics:RSDeviation?

The :MEASure:STATistics:RSDeviation? query returns the current relative standard deviation setting.

Return Format {0 | 1}<NL>

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:RESults"](#) on page 654
 - [":MEASure:STATistics"](#) on page 663
 - [":MEASure:STATistics:DISPlay"](#) on page 664
 - [":MEASure:STATistics:MCOunt"](#) on page 666
 - [":MEASure:STATistics:RESet"](#) on page 667
 - [":MEASure:STATistics:INCRement"](#) on page 665

:MEASure:TEDGE

N (see [page 1650](#))

Command Syntax :MEASure:TEDGE [<slope>,<occurrence>[,<source>]

<slope> ::= {RISing | FALLing | EITHER}

<occurrence> ::= [+ | -]<number>

<number> ::= the edge number in NR1 format

<source> ::= {<digital channels> | CHANnel<n> | FUNction<m> | MATH<m> | WMEMory<r>}

<digital channels> ::= DIGital<d> for the MSO models

<n> ::= 1 to (# of analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :MEASure:TEDGE command installs a screen measurement whose result is the time of the specified edge transition relative to the trigger edge (time=zero).

The threshold voltage used for this measurement is at the 50% point with a small amount of hysteresis added. You cannot change the threshold voltage used for this measurement with the :MEASure:DEFine command.

The optional <slope> parameter specifies whether to look for RISing edges or FALLing edges. If not otherwise specified, rising edges are assumed.

The <occurrence> parameter specifies the Nth edge to identify.

An <occurrence> value of zero (0) means the edge closest to the timebase reference; it is equivalent to the **Auto** selection for the **Edge #** in the front panel user interface. The EITHER (either edge) <slope> option can be used only when the <occurrence> count is zero (0).

Any other <occurrence> value means the Nth edge from the left side of the display.

The <occurrence> parameter can optionally have a plus sign (+) or a minus sign (-) to specify a rising or falling slope. Plus or minus signs cannot be used when the <slope> parameter is used.

If the optional <source> parameter is not specified, the current :MEASure:SOURce setting is used.

NOTE

This command is not available if the source is a FFT (Fast Fourier Transform) waveform.

Query Syntax :MEASure:TEDGE? [<slope>,<occurrence>[,<source>]

When the :MEASure:TEDGE query is sent, the displayed signal is searched for the specified transition.

The time interval between the trigger event and the edge occurrence is returned.

If the specified crossing cannot be found, the oscilloscope returns +9.9E+37. This value is returned if the waveform does not cross the specified vertical value, or if the waveform does not cross the specified vertical value for the specific number of times in the direction specified.

Return Format <value><NL>

<value> ::= time in seconds of the specified transition in NR3 format

:MEASure:TEDGE Code You can use multiple :MEASure:TEDGE? measurements to form delay and phase measurements:

Delay = time at the Nth rising or falling edge of the channel - time at the same edge of another channel

Phase = (delay between channels / period of channel) x 360

For example:

```
' Make a delay measurement between channel 1 and 2.
Dim dblChan1Edge1 As Double
Dim dblChan2Edge1 As Double
Dim dblChan1Edge2 As Double
Dim dblDelay As Double
Dim dblPeriod As Double
Dim dblPhase As Double

' Query time at 1st rising edge on ch1.
myScope.WriteString ":MEASURE:TEDGE? +1, CHAN1"

' Read time at edge 1 on ch 1.
dblChan1Edge1 = myScope.ReadNumber

' Query time at 1st rising edge on ch2.
myScope.WriteString ":MEASURE:TEDGE? +1, CHAN2"

' Read time at edge 1 on ch 2.
dblChan2Edge1 = myScope.ReadNumber

' Calculate delay time between ch1 and ch2.
dblDelay = dblChan2Edge1 - dblChan1Edge1

' Write calculated delay time to screen.
MsgBox "Delay = " + vbCrLf + CStr(dblDelay)

' Make a phase difference measurement between channel 1 and 2.
' Query time at 1st rising edge on ch1.
myScope.WriteString ":MEASURE:TEDGE? +2, CHAN1"

' Read time at edge 2 on ch 1.
```

```

dblChan1Edge2 = myScope.ReadNumber

' Calculate period of ch 1.
dblPeriod = dblChan1Edge2 - dblChan1Edge1

' Calculate phase difference between ch1 and ch2.
dblPhase = (dblDelay / dblPeriod) * 360
MsgBox "Phase = " + vbCrLf + CStr(dblPhase)

```

See complete example programs at: [Chapter 46](#), “Programming Examples,” starting on page 1659

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:TVALue"](#) on page 672
 - [":MEASure:VTIME"](#) on page 1571

:MEASure:TVALue

C (see [page 1650](#))

Query Syntax :MEASure:TVALue? <value>, [<slope>]<occurrence>[,<source>]

<value> ::= the vertical value that the waveform must cross. The value can be volts or a math function value such as dB, Vs, or V/s.

<slope> ::= direction of the waveform. A rising slope is indicated by a plus sign (+). A falling edge is indicated by a minus sign (-).

<occurrence> ::= the transition to be reported. If the occurrence number is one, the first crossing is reported. If the number is two, the second crossing is reported, etc.

<source> ::= {CHANnel<n> | FUNCTION<m> | MATH<m> | WMEMory<r>}

<n> ::= 1 to (# analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

When the :MEASure:TVALue? query is sent, the displayed signal is searched for the specified value level and transition. The time interval between the trigger event and this defined occurrence is returned as the response to the query.

The specified value can be negative or positive. To specify a negative value, use a minus sign (-). The sign of the slope selects a rising (+) or falling (-) edge. If no sign is specified for the slope, it is assumed to be the rising edge.

The magnitude of the occurrence defines the occurrence to be reported. For example, +3 returns the time for the third time the waveform crosses the specified value level in the positive direction. Once this value crossing is found, the oscilloscope reports the time at that crossing in seconds, with the trigger point (time zero) as the reference.

If the specified crossing cannot be found, the oscilloscope reports +9.9E+37. This value is returned if the waveform does not cross the specified value, or if the waveform does not cross the specified value for the specified number of times in the direction specified.

If the optional source parameter is specified, the current source is modified.

NOTE

This query is not available if the source is FFT (Fast Fourier Transform).

Return Format <value><NL>

<value> ::= time in seconds of the specified value crossing in
NR3 format

- See Also
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:TEDGe"](#) on page 669
 - [":MEASure:VTIMe"](#) on page 1571

:MEASure:VAMPlitude

C (see [page 1650](#))

Command Syntax :MEASure:VAMPlitude [<source>]

<source> ::= {CHANnel<n> | FUNction<m> | MATH<m> | WMEMory<r>}

<n> ::= 1 to (# analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:VAMPlitude command installs a screen measurement and starts a vertical amplitude measurement. If the optional source parameter is specified, the current source is modified.

Query Syntax :MEASure:VAMPlitude? [<source>]

The :MEASure:VAMPlitude? query measures and returns the vertical amplitude of the waveform. To determine the amplitude, the instrument measures Vtop and Vbase, then calculates the amplitude as follows:

$$\text{vertical amplitude} = V_{\text{top}} - V_{\text{base}}$$

Return Format <value><NL>

<value> ::= the amplitude of the selected waveform in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:SOURce"](#) on page 661
 - [":MEASure:VBASe"](#) on page 676
 - [":MEASure:VTOP"](#) on page 682
 - [":MEASure:VPP"](#) on page 679

:MEASure:VAVerage

C (see [page 1650](#))

Command Syntax :MEASure:VAVerage [<interval>] [,<source>]

<interval> ::= {CYCLe | DISPlay}

<source> ::= {CHANnel<n> | FUNCTION<m> | MATH<m> | WMemory<r>}

<n> ::= 1-2 or 1-4 (# of analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:VAVerage command installs a screen measurement and starts an average value measurement.

The <interval> option lets you specify the measurement interval: either an integral number of cycles, or the full screen. If <interval> is not specified, DISPlay is implied.

If the optional source parameter is specified, the current source is modified.

Query Syntax :MEASure:VAVerage? [<interval>] [,<source>]

The :MEASure:VAVerage? query returns the average value of an integral number of periods of the signal.

Return Format <value><NL>

<value> ::= calculated average value in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:SOURce"](#) on page 661

:MEASure:VBASe

C (see [page 1650](#))

Command Syntax :MEASure:VBASe [<source>]

<source> ::= {CHANnel<n> | FUNction<m> | MATH<m> | WMEMory<r>}

<n> ::= 1 to (# analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:VBASe command installs a screen measurement and starts a waveform base value measurement. If the optional source parameter is specified, the current source is modified.

NOTE

This command is not available if the source is FFT (Fast Fourier Transform).

Query Syntax :MEASure:VBASe? [<source>]

The :MEASure:VBASe? query returns the vertical value at the base of the waveform. The base value of a pulse is normally not the same as the minimum value.

Return Format <base_voltage><NL>

<base_voltage> ::= value at the base of the selected waveform in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:SOURce"](#) on page 661
 - [":MEASure:VTOP"](#) on page 682
 - [":MEASure:VAMPLitude"](#) on page 674
 - [":MEASure:VMIN"](#) on page 678

:MEASure:VMAX

C (see [page 1650](#))

Command Syntax :MEASure:VMAX [<source>]

<source> ::= {CHANnel<n> | FUNction<m> | MATH<m> | WMEMory<r>}

<n> ::= 1-2 or 1-4 (# of analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:VMAX command installs a screen measurement and starts a maximum vertical value measurement. If the optional source parameter is specified, the current source is modified.

Query Syntax :MEASure:VMAX? [<source>]

The :MEASure:VMAX? query measures and outputs the maximum vertical value present on the selected waveform.

Return Format <value><NL>

<value> ::= maximum vertical value of the selected waveform in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:SOURce"](#) on page 661
 - [":MEASure:VMIN"](#) on page 678
 - [":MEASure:VPP"](#) on page 679
 - [":MEASure:VTOP"](#) on page 682

:MEASure:VMIN

C (see [page 1650](#))

Command Syntax :MEASure:VMIN [<source>]

<source> ::= {CHANnel<n> | FUNction<m> | MATH<m> | WMEMory<r>}

<n> ::= 1 to (# analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:VMIN command installs a screen measurement and starts a minimum vertical value measurement. If the optional source parameter is specified, the current source is modified.

Query Syntax :MEASure:VMIN? [<source>]

The :MEASure:VMIN? query measures and outputs the minimum vertical value present on the selected waveform.

Return Format <value><NL>

<value> ::= minimum vertical value of the selected waveform in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:SOURce"](#) on page 661
 - [":MEASure:VBASe"](#) on page 676
 - [":MEASure:VMAX"](#) on page 677
 - [":MEASure:VPP"](#) on page 679

:MEASure:VPP

C (see [page 1650](#))

Command Syntax :MEASure:VPP [<source>]

<source> ::= {CHANnel<n> | FUNCTION<m> | MATH<m> | WMEMory<r>}

<n> ::= 1 to (# analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:VPP command installs a screen measurement and starts a vertical peak-to-peak measurement. If the optional source parameter is specified, the current source is modified.

Query Syntax :MEASure:VPP? [<source>]

The :MEASure:VPP? query measures the maximum and minimum vertical value for the selected source, then calculates the vertical peak-to-peak value and returns that value. The peak-to-peak value (Vpp) is calculated with the following formula:

$$V_{pp} = V_{max} - V_{min}$$

Vmax and Vmin are the vertical maximum and minimum values present on the selected source.

Return Format <value><NL>

<value> ::= vertical peak to peak value in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:SOURce"](#) on page 661
 - [":MEASure:VMAX"](#) on page 677
 - [":MEASure:VMIN"](#) on page 678
 - [":MEASure:VAMPLitude"](#) on page 674

:MEASure:VRATio

N (see [page 1650](#))

Command Syntax :MEASure:VRATio [<interval>] [,<source1>] [,<source2>]

<interval> ::= {CYCLe | DISPlay}

<source1,2> ::= {CHANnel<n> | FUNCTio<m> | MATH<m> | WMEMory<r>}

<n> ::= 1 to (# analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:VRATio command installs a ratio measurement on screen. Ratio measurements show the ratio of the ACRMS value of source1 to that of source2, expressed in dB.

The <interval> option lets you specify the measurement interval: either an integral number of cycles, or the full screen. If <interval> is not specified, DISPlay is implied.

NOTE

This command is not available if the source is FFT (Fast Fourier Transform).

Query Syntax :MEASure:VRATio? [<interval>] [<source1>] [,<source2>]

The :MEASure:VRATio? query measures and returns the ratio of AC RMS values of the specified sources expressed as dB.

Return Format <value><NL>

<value> ::= the ratio value in dB in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:VRMS"](#) on page 681
 - [":MEASure:SOURce"](#) on page 661

:MEASure:VRMS

C (see [page 1650](#))

Command Syntax :MEASure:VRMS [<interval>] [,<type>] [,<source>]
 <interval> ::= {CYCLe | DISPlay}
 <type> ::= {AC | DC}
 <source> ::= {CHANnel<n> | FUNCTION<m> | MATH<m> | WMEMory<r>}
 <n> ::= 1-2 or 1-4 (# of analog channels) in NR1 format
 <m> ::= 1 to (# math functions) in NR1 format
 <r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:VRMS command installs a screen measurement and starts an RMS value measurement.

The <interval> option lets you specify the measurement interval: either an integral number of cycles, or the full screen. If <interval> is not specified, DISPlay is implied.

The <type> option lets you choose between a DC RMS measurement and an AC RMS measurement. If <type> is not specified, DC is implied.

If the optional source parameter is specified, the current source is modified.

NOTE

This command is not available if the source is FFT (Fast Fourier Transform).

Query Syntax :MEASure:VRMS? [<interval>] [,<type>] [,<source>]

The :MEASure:VRMS? query measures and outputs the RMS measurement value.

Return Format <value><NL>

<value> ::= calculated dc RMS value in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:SOURce"](#) on page 661

:MEASure:VTOP

C (see [page 1650](#))

Command Syntax :MEASure:VTOP [<source>]

<source> ::= {CHANnel<n> | FUNction<m> | MATH<m>}

<n> ::= 1-2 or 1-4 (# of analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:VTOP command installs a screen measurement and starts a waveform top value measurement.

NOTE

This query is not available if the source is FFT (Fast Fourier Transform).

Query Syntax :MEASure:VTOP? [<source>]

The :MEASure:VTOP? query returns the vertical value at the top of the waveform. The top value of the pulse is normally not the same as the maximum value.

Return Format <value><NL>

<value> ::= vertical value at the top of the waveform in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:SOURce"](#) on page 661
 - [":MEASure:VMAX"](#) on page 677
 - [":MEASure:VAMPLitude"](#) on page 674
 - [":MEASure:VBASe"](#) on page 676

:MEASure:WINDow

N (see [page 1650](#))

Command Syntax :MEASure:WINDow <type>

<type> ::= {MAIN | ZOOM | AUTO | GATE}

The :MEASure:WINDow command lets you choose whether measurements are made in the Main window portion of the display, the Zoom window portion of the display (when the zoomed time base is displayed), or gated by the X1 and X2 cursors.

- MAIN – the measurement window is the Main window.
- ZOOM – the measurement window is the lower, Zoom window.
- AUTO – when the zoomed time base is displayed, the measurement is attempted in the lower, Zoom window; if it cannot be made there, or if the zoomed time base is not displayed, the Main window is used.
- GATE – the measurement window is between the X1 and X2 cursors. When the zoomed time base is displayed, the X1 and X2 cursors in the Zoom window portion of the display are used.

Query Syntax :MEASure:WINDow?

The :MEASure:WINDow? query returns the current measurement window setting.

Return Format <type><NL>

<type> ::= {MAIN | ZOOM | AUTO | GATE}

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:SOURce"](#) on page 661

:MEASure:XMAX

N (see [page 1650](#))

Command Syntax :MEASure:XMAX [<source>]

<source> ::= {CHANnel<n> | FUNction<m> | MATH<m> | WMEMory<r>}

<n> ::= 1-2 or 1-4 (# of analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:XMAX command installs a screen measurement and starts an X-at-Max-Y measurement on the selected window. If the optional source parameter is specified, the current source is modified.

NOTE

:MEASure:XMAX is an alias for :MEASure:TMAX.

Query Syntax :MEASure:XMAX? [<source>]

The :MEASure:XMAX? query measures and returns the horizontal axis value at which the maximum vertical value occurs. If the optional source is specified, the current source is modified.

Return Format <value><NL>

<value> ::= horizontal value of the maximum in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:XMIN"](#) on page 685
 - [":MEASure:TMAX"](#) on page 1562

:MEASure:XMIN

N (see [page 1650](#))

Command Syntax :MEASure:XMIN [<source>]

<source> ::= {CHANnel<n> | FUNction<m> | MATH<m> | WMEMory<r>}

<n> ::= 1-2 or 1-4 (# of analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:XMIN command installs a screen measurement and starts an X-at-Min-Y measurement on the selected window. If the optional source parameter is specified, the current source is modified.

NOTE

:MEASure:XMIN is an alias for :MEASure:TMIN.

Query Syntax :MEASure:XMIN? [<source>]

The :MEASure:XMIN? query measures and returns the horizontal axis value at which the minimum vertical value occurs. If the optional source is specified, the current source is modified.

Return Format <value><NL>

<value> ::= horizontal value of the minimum in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:XMAX"](#) on page 684
 - [":MEASure:TMIN"](#) on page 1563

:MEASure:YATX

N (see [page 1650](#))

Command Syntax :MEASure:YATX <horiz_location>[,<source>]

<horiz_location> ::= time from trigger in seconds

<source> ::= {<digital channels> | CHANnel<n> | FUNction<m> | MATH<m>
| WMEMory<r>}

<digital channels> ::= DIGital<d> for the MSO models

<d> ::= 0 to (# digital channels - 1) in NR1 format

<n> ::= 1 to (# of analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:YATX command installs a screen measurement of the vertical value at a specified horizontal value on the source specified (see also :MEASure:SOURce).

The specified horizontal value must be on the screen; when it is a time value, it is referenced to the trigger event. If the optional source parameter is specified, the measurement source is modified.

NOTE

When the source is an FFT (Fast Fourier Transform) waveform, the <horiz_location> is a frequency value instead of a time value.

Query Syntax :MEASure:YATX? <horiz_location>[,<source>]

The :MEASure:YATX? query returns the vertical value at a specified horizontal value on the source specified (see also :MEASure:SOURce).

Return Format <value><NL>

<value> ::= vertical value at the specified horizontal location
in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:SOURce"](#) on page 661
 - [":MEASure:TEDGe"](#) on page 669
 - [":MEASure:TVALue"](#) on page 672

27 :MEASure Power Commands

These :MEASure commands are available when the power measurements and analysis application is licensed and enabled.

Table 114 :MEASure Power Commands Summary

Command	Query	Options and Query Returns
:MEASure:ANGLE [<source1>] [, <source2>] >] (see page 692)	:MEASure:ANGLE? [<source1>] [, <source2>] >] (see page 692)	<source1>, <source2> ::= {CHANnel<n>} <n> ::= 1 to (# analog channels) in NR1 format <return_value> ::= the power phase angle in degrees in NR3 format
:MEASure:APParent [<source1>] [, <source2>] >] (see page 693)	:MEASure:APParent? [<source1>] [, <source2>] >] (see page 693)	<source1>, <source2> ::= {CHANnel<n>} <n> ::= 1 to (# analog channels) in NR1 format <return_value> ::= the apparent power value in NR3 format
:MEASure:CPLoss [<source1>] [, <source2>] >] (see page 694)	:MEASure:CPLoss? [<source1>] [, <source2>] >] (see page 694)	<source1>, <source2> <source1> ::= {FUNCTION<m> MATH<m>} <source2> ::= {CHANnel<n>} <m> ::= 1 to (# math functions) in NR1 format <n> ::= 1 to (# analog channels) in NR1 format <return_value> ::= the switching loss per cycle watts value in NR3 format

Table 114 :MEASure Power Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:CRESt [<source>] (see page 695)	:MEASure:CRESt? [<source>] (see page 695)	<source> ::= {CHANnel<n> FUNctIon<m> MATH<m>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <return_value> ::= the crest factor value in NR3 format
:MEASure:EFFiciency (see page 696)	:MEASure:EFFiciency? (see page 696)	<return_value> ::= percent value in NR3 format
:MEASure:ELOSs [<source>] (see page 697)	:MEASure:ELOSs? [<source>] (see page 697)	<source> ::= {CHANnel<n> FUNctIon<m> MATH<m> WMEmory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= the energy loss value in NR3 format
:MEASure:FACTOR [<source1>] [,<source2 >] (see page 698)	:MEASure:FACTOR? [<source1>] [,<source2 >] (see page 698)	<source1>, <source2> ::= {CHANnel<n>} <n> ::= 1 to (# analog channels) in NR1 format <return_value> ::= the power factor value in NR3 format
:MEASure:IPower (see page 699)	:MEASure:IPower? (see page 699)	<return_value> ::= the input power value in NR3 format
:MEASure:OFFTime [<source1>] [,<source2 >] (see page 700)	:MEASure:OFFTime? [<source1>] [,<source2 >] (see page 700)	<source1>, <source2> ::= {CHANnel<n>} <n> ::= 1 to (# analog channels) in NR1 format <return_value> ::= the time in seconds in NR3 format

Table 114 :MEASure Power Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:ONTime [<source1>] [, <source2>] >] (see page 701)	:MEASure:ONTime? [<source1>] [, <source2>] >] (see page 701)	<source1>, <source2> ::= {CHANnel<n>} <n> ::= 1 to (# analog channels) in NR1 format <return_value> ::= the time in seconds in NR3 format
:MEASure:OPower (see page 702)	:MEASure:OPower? (see page 702)	<return_value> ::= the output power value in NR3 format
:MEASure:PCURrent [<source>] (see page 703)	:MEASure:PCURrent? [<source>] (see page 703)	<source> ::= {CHANnel<n> FUNCTION<m> MATH<m> WMemory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= the peak current value in NR3 format
:MEASure:PLOSS [<source>] (see page 704)	:MEASure:PLOSS? [<source>] (see page 704)	<source> ::= {CHANnel<n> FUNCTION<m> MATH<m> WMemory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= the power loss value in NR3 format

Table 114 :MEASure Power Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:RDSon [<source1>] [, <source2>] (see page 705)	:MEASure:RDSon? [<source1>] [, <source2>] (see page 705)	<source1>, <source2> ::= {CHANnel<n> FUNCTION<m> MATH<m> WMemory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= the Rds(on) value in NR3 format
:MEASure:REACtive [<source1>] [, <source2>] (see page 706)	:MEASure:REACtive? [<source1>] [, <source2>] (see page 706)	<source1>, <source2> ::= {CHANnel<n>} <n> ::= 1 to (# analog channels) in NR1 format <return_value> ::= the reactive power value in NR3 format
:MEASure:REAL [<source>] (see page 707)	:MEASure:REAL? [<source>] (see page 707)	<source> ::= {CHANnel<n> FUNCTION<m> MATH<m>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <return_value> ::= the real power value in NR3 format
:MEASure:RIPple [<source>] (see page 708)	:MEASure:RIPple? [<source>] (see page 708)	<source> ::= {CHANnel<n> FUNCTION<m> MATH<m> WMemory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= the output ripple value in NR3 format

Table 114 :MEASure Power Commands Summary (continued)

Command	Query	Options and Query Returns
:MEASure:TREsponse [<source>] (see page 709)	:MEASure:TREsponse? [<source>] (see page 709)	<source> ::= {CHANnel<n> FUNctIon<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= time in seconds for the overshoot to settle back into the band in NR3 format
:MEASure:VCEsat [<source>] (see page 710)	:MEASure:VCEsat? [<source>] (see page 710)	<source> ::= {CHANnel<n> FUNctIon<m> MATH<m> WMEMory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <return_value> ::= the Vce(sat) value in NR3 format

:MEASure:ANGLE

N (see [page 1650](#))

Command Syntax :MEASure:ANGLE [<source1>] [, <source2>]

<source1>, <source2> ::= {CHANnel<n>}

<n> ::= 1 to (# analog channels) in NR1 format

The :MEASure:ANGLE command installs a power phase angle measurement on screen.

The <source1> parameter is the channel probing voltage and the <source2> parameter is the channel probing current. These sources can also be specified by the :MEASure:SOURce command.

Phase angle is a measure of power quality. In the *power triangle* (the right triangle where $\text{apparent_power}^2 = \text{real_power}^2 + \text{reactive_power}^2$), phase angle is the angle between the apparent power and the real power, indicating the amount of reactive power. Small phase angles equate to less reactive power.

Query Syntax :MEASure:ANGLE? [<source1>] [, <source2>]

The :MEASure:ANGLE query returns the measured power phase angle in degrees.

Return Format <return_value><NL>

<return_value> ::= the power phase angle in degrees in NR3 format

- See Also**
- [":MEASure:SOURce"](#) on page 661
 - [":POWer:QUALity:APPLy"](#) on page 820

:MEASure:APParent

N (see [page 1650](#))

Command Syntax :MEASure:APParent [<source1>] [,<source2>]

<source1>, <source2> ::= {CHANnel<n>}

<n> ::= 1 to (# analog channels) in NR1 format

The :MEASure:APParent command installs an apparent power measurement on screen.

The <source1> parameter is the channel probing voltage and the <source2> parameter is the channel probing current. These sources can also be specified by the :MEASure:SOURce command.

Apparent power is a measure of power quality. It is the portion of AC line power flow due to stored energy which returns to the source in each cycle.

$IRMS * VRMS$

Query Syntax :MEASure:APParent? [<source1>] [,<source2>]

The :MEASure:APParent query returns the measured apparent power.

Return Format <return_value><NL>

<return_value> ::= the apparent power value in NR3 format

- See Also**
- [":MEASure:SOURce"](#) on page 661
 - [":POWer:QUALity:APPLy"](#) on page 820

:MEASure:CPLoss

N (see [page 1650](#))

Command Syntax :MEASure:CPLoss [<source1>] [, <source2>]

<source1> ::= {FUNCTION<m> | MATH<m>}

<source2> ::= {CHANnel<n>}

<m> ::= 1 to (# math functions) in NR1 format

<n> ::= 1 to (# analog channels) in NR1 format

The :MEASure:CPLoss command installs a power loss per cycle measurement on screen.

The <source1> parameter is typically a math multiply waveform or other waveform that represents power (voltage * current). This source can also be specified by the :MEASure:SOURce command.

Power loss per cycle is $P_n = (V_{dsn} * I_{dn}) * (\text{Time range of zoom window}) * (\text{Counter measurement of the voltage of the switching signal})$, where n is each sample.

This measurement operates when in zoom mode and the counter measurement is installed on the voltage of the switching signal.

Query Syntax :MEASure:CPLoss? [<source1>] [, <source2>]

The :MEASure:CPLoss query returns the switching loss per cycle in watts.

Return Format <return_value><NL>

<return_value> ::= the switching loss per cycle value in NR3 format

- See Also**
- [":MEASure:SOURce"](#) on page 661
 - [":POWer:SWITCh:APPLy"](#) on page 843

:MEASure:CRESt

N (see [page 1650](#))

Command Syntax :MEASure:CRESt [<source>]

<source> ::= {CHANnel<n> | FUNction<m> | MATH<m>}

<n> ::= 1 to (# analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

The :MEASure:CRESt command installs a crest factor measurement on screen.

The <source> parameter is the channel probing current or voltage. This source can also be specified by the :MEASure:SOURce command.

Crest factor is a measure of power quality. It is the ratio between the instantaneous peak AC line current (or voltage) required by the load and the RMS current (or voltage). For example: Ipeak / IRMS or Vpeak / VRMS.

Query Syntax :MEASure:CRESt? [<source>]

The :MEASure:CRESt query returns the measured crest factor.

Return Format <return_value><NL>

<return_value> ::= the crest factor value in NR3 format

- See Also**
- [":MEASure:SOURce"](#) on page 661
 - [":POWer:QUALity:APPLy"](#) on page 820

:MEASure:EFFiciency

N (see [page 1650](#))

Command Syntax :MEASure:EFFiciency

The :MEASure:EFFiciency command installs an efficiency (output power / input power) measurement on screen.

Before sending this command or query, you must specify the channels probing the input voltage, input current, output voltage, and output current (using the :POWer:SIGNals:SOURce:VOLTage<i> and :POWer:SIGNals:SOURce:CURRent<i> commands) and you must perform the automated signals setup (using the :POWer:SIGNals:AUTosetup EFFiciency command).

Query Syntax :MEASure:EFFiciency?

The :MEASure:EFFiciency query returns the measured efficiency as a percent value.

Return Format <return_value><NL>

<return_value> ::= percent value in NR3 format

- See Also**
- [":POWer:SIGNals:SOURce:VOLTage<i>"](#) on page 840
 - [":POWer:SIGNals:SOURce:CURRent<i>"](#) on page 839
 - [":POWer:SIGNals:AUTosetup"](#) on page 822
 - [":POWer:EFFiciency:APPLY"](#) on page 778

:MEASure:ELOSs

N (see [page 1650](#))

Command Syntax :MEASure:ELOSs [<source>]

<source> ::= {CHANnel<n> | FUNCTION<m> | MATH<m> | WMemory<r>}

<n> ::= 1 to (# analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:ELOSs command installs an energy loss measurement on screen.

The <source> parameter is typically a math multiply waveform or other waveform that represents power (voltage * current). This source can also be specified by the :MEASure:SOURce command.

Energy loss = $\sum (V_{ds_n} * I_{d_n})$ * sample size, where n is each sample.

Query Syntax :MEASure:ELOSs? [<source>]

The :MEASure:ELOSs query returns the switching loss in joules.

Return Format <return_value><NL>

<return_value> ::= the energy loss value in NR3 format

- See Also**
- [":MEASure:SOURce"](#) on page 661
 - [":POWer:SWITCh:APPLy"](#) on page 843

:MEASure:FACTOR

N (see [page 1650](#))

Command Syntax :MEASure:FACTOR [<source1>] [,<source2>]

<source1>, <source2> ::= {CHANnel<n>}

<n> ::= 1 to (# analog channels) in NR1 format

The :MEASure:FACTOR command installs a power factor measurement on screen.

The <source1> parameter is the channel probing voltage and the <source2> parameter is the channel probing current. These sources can also be specified by the :MEASure:SOURce command.

Power factor is a measure of power quality. It is the ratio of the actual AC line power to the apparent power:

Real Power / Apparent Power

Query Syntax :MEASure:FACTOR? [<source1>] [,<source2>]

The :MEASure:FACTOR query returns the measured power factor.

Return Format <return_value><NL>

<return_value> ::= the power factor value in NR3 format

- See Also**
- [":MEASure:SOURce"](#) on page 661
 - [":POWer:QUALity:APPLY"](#) on page 820

:MEASure:IPower

N (see [page 1650](#))

Command Syntax :MEASure:IPower

The :MEASure:IPower command installs an input power measurement on screen.

Before sending this command or query, you must specify the channels probing the input voltage, input current, output voltage, and output current (using the :POWER:SIGNals:SOURce:VOLTage<i> and :POWER:SIGNals:SOURce:CURREnt<i> commands) and you must perform the automated signals setup (using the :POWER:SIGNals:AUTosetup EFFiciency command).

Query Syntax :MEASure:IPower?

The :MEASure:IPower query returns the measured input power.

Return Format <return_value><NL>

<return_value> ::= the input power value in NR3 format

- See Also**
- [":POWER:SIGNals:SOURce:VOLTage<i>"](#) on page 840
 - [":POWER:SIGNals:SOURce:CURREnt<i>"](#) on page 839
 - [":POWER:SIGNals:AUTosetup"](#) on page 822
 - [":POWER:EFFiciency:APPLY"](#) on page 778

:MEASure:OFFTime

N (see [page 1650](#))

Command Syntax :MEASure:OFFTime [<source1>] [,<source2>]

<source1>, <source2> ::= {CHANnel<n>}

<n> ::= 1 to (# analog channels) in NR1 format

The :MEASure:OFFTime command installs an "off time" measurement on screen.

Turn off time measures the difference of time between when the input AC Voltage last falls to 10% of its maximum amplitude to the time when the output DC Voltage last falls to 10% of its maximum amplitude.

The <source1> parameter is the AC Voltage and the <source2> parameter is the DC Voltage. These sources can also be specified by the :MEASure:SOURce command.

Query Syntax :MEASure:OFFTime? [<source1>] [,<source2>]

The :MEASure:OFFTime query returns the measured turn off time.

Return Format <return_value><NL>

<return_value> ::= the time in seconds in NR3 format

- See Also**
- [":MEASure:SOURce"](#) on page 661
 - [":POWer:ONOff:TEST"](#) on page 803
 - [":POWer:ONOff:APPLY"](#) on page 800

:MEASure:ONTime

N (see [page 1650](#))

Command Syntax :MEASure:ONTime [<source1>] [,<source2>]

<source1>, <source2> ::= {CHANnel<n>}

<n> ::= 1 to (# analog channels) in NR1 format

The :MEASure:ONTime command installs an "on time" measurement on screen.

Turn on time measures the difference of time between when the input AC Voltage first rises to 10% of its maximum amplitude to the time when the output DC Voltage rises to 90% of its maximum amplitude.

The <source1> parameter is the AC Voltage and the <source2> parameter is the DC Voltage. These sources can also be specified by the :MEASure:SOURce command.

Query Syntax :MEASure:ONTime? [<source1>] [,<source2>]

The :MEASure:ONTime query returns the measured turn off time.

Return Format <return_value><NL>

<return_value> ::= the time in seconds in NR3 format

- See Also**
- [":MEASure:SOURce"](#) on page 661
 - [":POWer:ONOff:TEST"](#) on page 803
 - [":POWer:ONOff:APPLY"](#) on page 800

:MEASure:OPower

N (see [page 1650](#))

Command Syntax

`:MEASure:OPower`

The :MEASure:OPower command installs an output power measurement on screen.

Before sending this command or query, you must specify the channels probing the input voltage, input current, output voltage, and output current (using the :POWer:SIGNals:SOURce:VOLTage<i> and :POWer:SIGNals:SOURce:CURREnt<i> commands) and you must perform the automated signals setup (using the :POWer:SIGNals:AUTosetup EFFiciency command).

Query Syntax

`:MEASure:OPower?`

The :MEASure:OPower query returns the measured output power.

Return Format `<return_value><NL>`
`<return_value>` ::= the output power value in NR3 format

- See Also**
- [":POWer:SIGNals:SOURce:VOLTage<i>"](#) on page 840
 - [":POWer:SIGNals:SOURce:CURREnt<i>"](#) on page 839
 - [":POWer:SIGNals:AUTosetup"](#) on page 822
 - [":POWer:EFFiciency:APPLY"](#) on page 778

:MEASure:PCURrent

N (see [page 1650](#))

Command Syntax :MEASure:PCURrent [<source>]

<source> ::= {CHANnel<n> | FUNCTION<m> | MATH<m> | WMemory<r>}

<n> ::= 1 to (# analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:PCURrent command installs a peak current measurement on screen.

The <source> parameter is the channel probing the current. This source can also be specified by the :MEASure:SOURce command.

This command measures the peak current when the power supply first turned on.

Query Syntax :MEASure:PCURrent? [<source>]

The :MEASure:PCURrent query returns the measured peak current.

Return Format <return_value><NL>

<return_value> ::= the peak current value in NR3 format

- See Also**
- [":MEASure:SOURce"](#) on page 661
 - [":POWer:INRush:APPLY"](#) on page 793

:MEASure:PLOSs

N (see [page 1650](#))

Command Syntax :MEASure:PLOSs [<source>]

<source> ::= {CHANnel<n> | FUNCTION<m> | MATH<m> | WMEMory<r>}

<n> ::= 1 to (# analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:PLOSs command installs a power loss measurement on screen.

The <source> parameter is typically a math multiply waveform or other waveform that represents power (voltage * current). This source can also be specified by the :MEASure:SOURce command.

Power loss is $P_n = V_{ds_n} * I_{d_n}$, where n is each sample.

Query Syntax :MEASure:PLOSs? [<source>]

The :MEASure:PLOSs query returns the switching loss in watts.

Return Format <return_value><NL>

<return_value> ::= the power loss value in NR3 format

- See Also**
- [":MEASure:SOURce"](#) on page 661
 - [":POWER:SWITCh:APPLY"](#) on page 843

:MEASure:RDson

N (see [page 1650](#))

Command Syntax :MEASure:RDson [<source1>] [,<source2>]

<source1>, <source2> ::= {CHANnel<n>| FUNction<m> | MATH<m>
| WMEMory<r>}

<n> ::= 1 to (# analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:RDson command installs a power Rds(on) measurement on screen.

Rds(on) is the ON resistance between the drain and source of MOSFET. The Rds(on) characteristic is also published in the switching device data sheet.

Query Syntax :MEASure:RDson? [<source1>] [,<source2>]

The :MEASure:RDson? query returns the measured Rds(on) value.

Return Format <return_value><NL>

<return_value> ::= the Rds(on) value in NR3 format

See Also • [":MEASure:VCESat"](#) on page 710

:MEASure:REActive

N (see [page 1650](#))

Command Syntax :MEASure:REActive [<source1>] [,<source2>]

<source1>, <source2> ::= {CHANnel<n>}

<n> ::= 1 to (# analog channels) in NR1 format

The :MEASure:REActive command installs a reactive power measurement on screen.

The <source1> parameter is the channel probing voltage and the <source2> parameter is the channel probing current. These sources can also be specified by the :MEASure:SOURce command.

Reactive power is a measure of power quality. It is the difference between apparent power and real power due to reactance. Using the *power triangle* (the right triangle where $\text{apparent_power}^2 = \text{real_power}^2 + \text{reactive_power}^2$):

$$\text{Reactive Power} = \sqrt{\text{Apparent Power}^2 - \text{Real Power}^2}$$

Reactive power is measured in VAR (Volts-Amps-Reactive).

Query Syntax :MEASure:REActive? [<source1>] [,<source2>]

The :MEASure:REActive query returns the measured reactive power.

Return Format <return_value><NL>

<return_value> ::= the reactive power value in NR3 format

- See Also**
- [":MEASure:SOURce"](#) on page 661
 - [":POWER:QUALity:APPLy"](#) on page 820

:MEASure:REAL

N (see [page 1650](#))

Command Syntax :MEASure:REAL [<source>]

<source> ::= {CHANnel<n> | FUNction<m> | MATH<m>}

<n> ::= 1 to (# analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

The :MEASure:REAL command installs a real power measurement on screen.

The <source> parameter is typically a math multiply waveform or other waveform that represents power (voltage * current). This source can also be specified by the :MEASure:SOURce command.

Real power is a measure of power quality. It is the portion of power flow that, averaged over a complete cycle of the AC waveform, results in net transfer of energy in one direction.

$$\text{Real Power} = \sqrt{\frac{1}{N} \sum_{n=0}^{N-1} V_n I_n}$$

Query Syntax :MEASure:REAL? [<source>]

The :MEASure:REAL query returns the measured real power.

Return Format <return_value><NL>

<return_value> ::= the real power value in NR3 format

- See Also**
- [":MEASure:SOURce"](#) on page 661
 - [":POWER:QUALity:APPLy"](#) on page 820

:MEASure:RIPple

N (see [page 1650](#))

Command Syntax :MEASure:RIPple [<source>]

<source> ::= {CHANnel<n> | FUNCTION<m> | MATH<m> | WMemory<r>}

<n> ::= 1 to (# analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:RIPple command installs an output ripple measurement on screen.

The <source> parameter is the channel probing the output voltage. This source can also be specified by the :MEASure:SOURce command.

Output ripple is: Vmax - Vmin.

Query Syntax :MEASure:RIPple? [<source>]

The :MEASure:RIPple query returns the measured output ripple.

Return Format <return_value><NL>

<return_value> ::= the output ripple value in NR3 format

- See Also**
- [":MEASure:SOURce"](#) on page 661
 - [":POWer:RIPple:APPLY"](#) on page 821

:MEASure:TRESponse

N (see [page 1650](#))

Command Syntax :MEASure:TRESponse [<source>]

<source> ::= {CHANnel<n> | FUNCTION<m> | MATH<m> | WMemory<r>}

<n> ::= 1 to (# analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:TRESponse command installs a transient response time measurement on screen.

The <source> parameter is the channel probing the output voltage. This source can also be specified by the :MEASure:SOURce command.

Transient response time = $t_2 - t_1$, where:

- t_1 = The first time a voltage waveform exits the settling band.
- t_2 = The last time it enters into the settling band.
- Settling band = +/-overshoot % of the steady state output voltage.

Query Syntax :MEASure:TRESponse? [<source>]

The :MEASure:TRESponse query returns the measured transient response time.

Return Format <return_value><NL>

<return_value> ::= time in seconds for the overshoot to settle back into the band in NR3 format

- See Also**
- [":MEASure:SOURce"](#) on page 661
 - [":POWER:TRANSient:APPLY"](#) on page 849

:MEASure:VCESat

N (see [page 1650](#))

Command Syntax :MEASure:VCESat [<source>]

<source> ::= {CHANnel<n> | FUNCTION<m> | MATH<m> | WMEMory<r>}

<n> ::= 1 to (# analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:VCESat command installs a power Vce(sat) measurement on screen.

Vce(sat) is the saturation voltage between the collector and emitter of a BJT. The Vce(sat) characteristic is also published in the switching device data sheet.

Query Syntax :MEASure:VCESat? [<source>]

The :MEASure:VCESat? query returns the measured Vce(sat) value.

Return Format <return_value><NL>

<return_value> ::= the VCE(sat) value in NR3 format

See Also • [":MEASure:RDSon"](#) on page 705

28 :MTESt Commands

The MTESt subsystem commands and queries control the mask test features. See ["Introduction to :MTESt Commands"](#) on page 713.

Table 115 :MTESt Commands Summary

Command	Query	Options and Query Returns
:MTESt:ALL {{0 OFF} {1 ON}} (see page 716)	:MTESt:ALL? (see page 716)	{0 1}
:MTESt:AMASk:CREate (see page 717)	n/a	n/a
:MTESt:AMASk:SOURce <source> (see page 718)	:MTESt:AMASk:SOURce? (see page 718)	<source> ::= CHANnel<n> <n> ::= {1 2 3 4} for 4ch models <n> ::= {1 2} for 2ch models
:MTESt:AMASk:UNITs <units> (see page 719)	:MTESt:AMASk:UNITs? (see page 719)	<units> ::= {CURRent DIVisions}
:MTESt:AMASk:XDELta <value> (see page 720)	:MTESt:AMASk:XDELta? (see page 720)	<value> ::= X delta value in NR3 format
:MTESt:AMASk:YDELta <value> (see page 721)	:MTESt:AMASk:YDELta? (see page 721)	<value> ::= Y delta value in NR3 format
n/a	:MTESt:COUNT:FWAVEfor ms? [CHANnel<n>] (see page 722)	<failed> ::= number of failed waveforms in NR1 format
:MTESt:COUNT:RESet (see page 723)	n/a	n/a
n/a	:MTESt:COUNT:TIME? (see page 724)	<time> ::= elapsed seconds in NR3 format
n/a	:MTESt:COUNT:WAVEform s? (see page 725)	<count> ::= number of waveforms in NR1 format
:MTESt:DATA <mask> (see page 726)	:MTESt:DATA? (see page 726)	<mask> ::= data in IEEE 488.2 # format.

Table 115 :MTESt Commands Summary (continued)

Command	Query	Options and Query Returns
:MTESt:DELeTe (see page 727)	n/a	n/a
:MTESt:ENABLe {{0 OFF} {1 ON}} (see page 728)	:MTESt:ENABLe? (see page 728)	{0 1}
:MTESt:LOCK {{0 OFF} {1 ON}} (see page 729)	:MTESt:LOCK? (see page 729)	{0 1}
:MTESt:RMODE <rmode> (see page 730)	:MTESt:RMODE? (see page 730)	<rmode> ::= {FORever TIME SIGMa WAVEforms}
:MTESt:RMODE:FACTION:MEASure {{0 OFF} {1 ON}} (see page 731)	:MTESt:RMODE:FACTION:MEASure? (see page 731)	{0 1}
:MTESt:RMODE:FACTION:PRINT {{0 OFF} {1 ON}} (see page 732)	:MTESt:RMODE:FACTION:PRINT? (see page 732)	{0 1}
:MTESt:RMODE:FACTION:SAVE {{0 OFF} {1 ON}} (see page 733)	:MTESt:RMODE:FACTION:SAVE? (see page 733)	{0 1}
:MTESt:RMODE:FACTION:STOP {{0 OFF} {1 ON}} (see page 734)	:MTESt:RMODE:FACTION:STOP? (see page 734)	{0 1}
:MTESt:RMODE:SIGMa <level> (see page 735)	:MTESt:RMODE:SIGMa? (see page 735)	<level> ::= from 0.1 to 9.3 in NR3 format
:MTESt:RMODE:TIME <seconds> (see page 736)	:MTESt:RMODE:TIME? (see page 736)	<seconds> ::= from 1 to 86400 in NR3 format
:MTESt:RMODE:WAVEform s <count> (see page 737)	:MTESt:RMODE:WAVEform s? (see page 737)	<count> ::= number of waveforms in NR1 format
:MTESt:SCALE:BIND {{0 OFF} {1 ON}} (see page 738)	:MTESt:SCALE:BIND? (see page 738)	{0 1}
:MTESt:SCALE:X1 <x1_value> (see page 739)	:MTESt:SCALE:X1? (see page 739)	<x1_value> ::= X1 value in NR3 format
:MTESt:SCALE:XDELta <xdelta_value> (see page 740)	:MTESt:SCALE:XDELta? (see page 740)	<xdelta_value> ::= X delta value in NR3 format

Table 115 :MTESt Commands Summary (continued)

Command	Query	Options and Query Returns
:MTESt:SCALe:Y1 <y1_value> (see page 741)	:MTESt:SCALe:Y1? (see page 741)	<y1_value> ::= Y1 value in NR3 format
:MTESt:SCALe:Y2 <y2_value> (see page 742)	:MTESt:SCALe:Y2? (see page 742)	<y2_value> ::= Y2 value in NR3 format
:MTESt:SOURce <source> (see page 743)	:MTESt:SOURce? (see page 743)	<source> ::= {CHANnel<n> NONE} <n> ::= {1 2 3 4} for 4ch models <n> ::= {1 2} for 2ch models
n/a	:MTESt:TITLe? (see page 744)	<title> ::= a string of up to 128 ASCII characters

Introduction to :MTESt Commands

Mask testing automatically compares the current displayed waveform with the boundaries of a set of polygons that you define. Any waveform or sample that falls within the boundaries of one or more polygons is recorded as a failure.

Reporting the Setup

Use :MTESt? to query setup information for the MTESt subsystem.

Return Format

The following is a sample response from the :MTESt? query. In this case, the query was issued following a *RST command.

```
:MTES:SOUR CHAN1;ENAB 0;LOCK 1;:MTES:AMAS:SOUR CHAN1;UNIT DIV;XDEL
+2.50000000E-001;YDEL +2.50000000E-001;:MTES:SCAL:X1 +200.000E-06;XDEL
+400.000E-06;Y1 -3.00000E+00;Y2 +3.00000E+00;BIND 0;:MTES:RMOD
FOR;RMOD:TIME +1E+00;WAV 1000;SIGM +6.0E+00;:MTES:RMOD:FACT:STOP
0;PRIN 0;SAVE 0
```

Example Code

```
' Mask testing commands example.
' -----

Option Explicit

Public myMgr As VisaComLib.ResourceManager
Public myScope As VisaComLib.FormattedIO488
Public varQueryResult As Variant
Public strQueryResult As String

Private Declare Sub Sleep Lib "kernel32" (ByVal dwMilliseconds As Long)

Sub Main()
```

```

On Error GoTo VisaComError

' Create the VISA COM I/O resource.
Set myMgr = New VisaComLib.ResourceManager
Set myScope = New VisaComLib.FormattedIO488
Set myScope.IO = _
    myMgr.Open("USB0::0x0957::0x17A6::US50210029::0::INSTR")
myScope.IO.Clear ' Clear the interface.

' Make sure oscilloscope is running.
myScope.WriteString ":RUN"

' Set mask test termination conditions.
myScope.WriteString ":MTESt:RMODE SIGMa"
myScope.WriteString ":MTESt:RMODE?"
strQueryResult = myScope.ReadString
Debug.Print "Mask test termination mode: " + strQueryResult

myScope.WriteString ":MTESt:RMODE:SIGMa 4.2"
myScope.WriteString ":MTESt:RMODE:SIGMa?"
varQueryResult = myScope.ReadNumber
Debug.Print "Mask test termination 'test sigma': " + _
    FormatNumber(varQueryResult)

' Use auto-mask to create mask.
myScope.WriteString ":MTESt:AMASK:SOURce CHANnel1"
myScope.WriteString ":MTESt:AMASK:SOURce?"
strQueryResult = myScope.ReadString
Debug.Print "Mask test auto-mask source: " + strQueryResult

myScope.WriteString ":MTESt:AMASK:UNITs DIVisions"
myScope.WriteString ":MTESt:AMASK:UNITs?"
strQueryResult = myScope.ReadString
Debug.Print "Mask test auto-mask units: " + strQueryResult

myScope.WriteString ":MTESt:AMASK:XDELta 0.1"
myScope.WriteString ":MTESt:AMASK:XDELta?"
varQueryResult = myScope.ReadNumber
Debug.Print "Mask test auto-mask X delta: " + _
    FormatNumber(varQueryResult)

myScope.WriteString ":MTESt:AMASK:YDELta 0.1"
myScope.WriteString ":MTESt:AMASK:YDELta?"
varQueryResult = myScope.ReadNumber
Debug.Print "Mask test auto-mask Y delta: " + _
    FormatNumber(varQueryResult)

' Enable "Auto Mask Created" event (bit 10, &H400)
myScope.WriteString "*CLS"
myScope.WriteString ":MTEenable " + CStr(CInt("&H400"))

' Create mask.
myScope.WriteString ":MTESt:AMASK:CREate"
Debug.Print "Auto-mask created, mask test automatically enabled."

' Set up timeout variables.

```

```

Dim lngTimeout As Long    ' Max millisecs to wait.
Dim lngElapsed As Long
lngTimeout = 60000        ' 60 seconds.

' Wait until mask is created.
lngElapsed = 0
Do While lngElapsed <= lngTimeout
    myScope.WriteString ":OPERRegister:CONDition?"
    varQueryResult = myScope.ReadNumber
    ' Operation Status Condition Register MTE bit (bit 9, &H200).
    If (varQueryResult And &H200) <> 0 Then
        Exit Do
    Else
        Sleep 100        ' Small wait to prevent excessive queries.
        lngElapsed = lngElapsed + 100
    End If
Loop

' Look for RUN bit = stopped (mask test termination).
lngElapsed = 0
Do While lngElapsed <= lngTimeout
    myScope.WriteString ":OPERRegister:CONDition?"
    varQueryResult = myScope.ReadNumber
    ' Operation Status Condition Register RUN bit (bit 3, &H8).
    If (varQueryResult And &H8) = 0 Then
        Exit Do
    Else
        Sleep 100        ' Small wait to prevent excessive queries.
        lngElapsed = lngElapsed + 100
    End If
Loop

' Get total waveforms, failed waveforms, and test time.
myScope.WriteString ":MTESt:COUNT:WAVEforms?"
strQueryResult = myScope.ReadString
Debug.Print "Mask test total waveforms: " + strQueryResult

myScope.WriteString ":MTESt:COUNT:FWAVEforms?"
strQueryResult = myScope.ReadString
Debug.Print "Mask test failed waveforms: " + strQueryResult

myScope.WriteString ":MTESt:COUNT:TIME?"
strQueryResult = myScope.ReadString
Debug.Print "Mask test elapsed seconds: " + strQueryResult

Exit Sub

VisaComError:
MsgBox "VISA COM Error:" + vbCrLf + Err.Description

End Sub

```

:MTESt:ALL

N (see [page 1650](#))

Command Syntax `:MTESt:ALL <on_off>`
 `<on_off> ::= {{1 | ON} | {0 | OFF}}`

The :MTESt:ALL command specifies the channel(s) that are included in the mask test:

- ON – All displayed analog channels are included in the mask test.
- OFF – Just the selected source channel is included in the test.

Query Syntax `:MTESt:ALL?`

The :MTESt:ALL? query returns the current setting.

Return Format `<on_off><NL>`
 `<on_off> ::= {1 | 0}`

See Also • ["Introduction to :MTESt Commands"](#) on page 713

:MTESt:AMASk:CREate

N (see [page 1650](#))

Command Syntax :MTESt:AMASk:CREate

The :MTESt:AMASk:CREate command automatically constructs a mask around the current selected channel, using the tolerance parameters defined by the :MTESt:AMASk:XDELta, :MTESt:AMASk:YDELta, and :MTESt:AMASk:UNITs commands. The mask only encompasses the portion of the waveform visible on the display, so you must ensure that the waveform is acquired and displayed consistently to obtain repeatable results.

The :MTESt:SOURce command selects the channel and should be set before using this command.

- See Also**
- ["Introduction to :MTESt Commands"](#) on page 713
 - [":MTESt:AMASk:XDELta"](#) on page 720
 - [":MTESt:AMASk:YDELta"](#) on page 721
 - [":MTESt:AMASk:UNITs"](#) on page 719
 - [":MTESt:AMASk:SOURce"](#) on page 718
 - [":MTESt:SOURce"](#) on page 743

- Example Code**
- ["Example Code"](#) on page 713

:MTESt:AMASk:SOURce

N (see [page 1650](#))

Command Syntax :MTESt:AMASk:SOURce <source>

<source> ::= CHANnel<n>

<n> ::= 1 to (# analog channels) in NR1 format

The :MTESt:AMASk:SOURce command selects the source for the interpretation of the :MTESt:AMASk:XDELta and :MTESt:AMASk:YDELta parameters when :MTESt:AMASk:UNITs is set to CURRent.

When UNITs are CURRent, the XDELta and YDELta parameters are defined in terms of the channel units, as set by the :CHANnel<n>:UNITs command, of the selected source.

Suppose that UNITs are CURRent and that you set SOURce to CHANNEL1, which is using units of volts. Then you can define AMASk:XDELta in terms of volts and AMASk:YDELta in terms of seconds.

This command is the same as the :MTESt:SOURce command.

Query Syntax :MTESt:AMASk:SOURce?

The :MTESt:AMASk:SOURce? query returns the currently set source.

Return Format <source> ::= CHAN<n>

<n> ::= 1 to (# analog channels) in NR1 format

- See Also**
- ["Introduction to :MTESt Commands"](#) on page 713
 - [":MTESt:AMASk:XDELta"](#) on page 720
 - [":MTESt:AMASk:YDELta"](#) on page 721
 - [":MTESt:AMASk:UNITs"](#) on page 719
 - [":MTESt:SOURce"](#) on page 743

Example Code

- ["Example Code"](#) on page 713

:MTESt:AMASK:UNITs

N (see [page 1650](#))

Command Syntax :MTESt:AMASK:UNITs <units>

<units> ::= {CURRENT | DIVisions}

The :MTESt:AMASK:UNITs command alters the way the mask test subsystem interprets the tolerance parameters for automasking as defined by :MTESt:AMASK:XDELta and :MTESt:AMASK:YDELta commands.

- CURRENT – the mask test subsystem uses the units as set by the :CHANnel<n>:UNITs command, usually time for ΔX and voltage for ΔY .
- DIVisions – the mask test subsystem uses the graticule as the measurement system, so tolerance settings are specified as parts of a screen division. The mask test subsystem maintains separate XDELta and YDELta settings for CURRENT and DIVisions. Thus, XDELta and YDELta are not converted to new values when the UNITs setting is changed.

Query Syntax :MTESt:AMASK:UNITs?

The :MTESt:AMASK:UNITs query returns the current measurement units setting for the mask test automask feature.

Return Format <units><NL>

<units> ::= {CURR | DIV}

- See Also**
- ["Introduction to :MTESt Commands"](#) on page 713
 - [":MTESt:AMASK:XDELta"](#) on page 720
 - [":MTESt:AMASK:YDELta"](#) on page 721
 - [":CHANnel<n>:UNITs"](#) on page 355
 - [":MTESt:AMASK:SOURce"](#) on page 718
 - [":MTESt:SOURce"](#) on page 743

Example Code • ["Example Code"](#) on page 713

:MTESt:AMASk:XDELta

 (see [page 1650](#))

Command Syntax :MTESt:AMASk:XDELta <value>
 <value> ::= X delta value in NR3 format

The :MTESt:AMASk:XDELta command sets the tolerance in the X direction around the waveform for the automasking feature. The absolute value of the tolerance will be added and subtracted to horizontal values of the waveform to determine the boundaries of the mask.

The horizontal tolerance value is interpreted based on the setting specified by the :MTESt:AMASk:UNITs command; thus, if you specify 250-E3, the setting for :MTESt:AMASk:UNITs is CURRent, and the current setting specifies time in the horizontal direction, the tolerance will be ± 250 ms. If the setting for :MTESt:AMASk:UNITs is DIVisions, the same X delta value will set the tolerance to ± 250 millidivisions, or 1/4 of a division.

Query Syntax :MTESt:AMASk:XDELta?

The :MTESt:AMASk:XDELta? query returns the current setting of the ΔX tolerance for automasking. If your computer program will interpret this value, it should also request the current measurement system using the :MTESt:AMASk:UNITs query.

Return Format <value><NL>
 <value> ::= X delta value in NR3 format

- See Also**
- ["Introduction to :MTESt Commands"](#) on page 713
 - [":MTESt:AMASk:UNITs"](#) on page 719
 - [":MTESt:AMASk:YDELta"](#) on page 721
 - [":MTESt:AMASk:SOURce"](#) on page 718
 - [":MTESt:SOURce"](#) on page 743

Example Code • ["Example Code"](#) on page 713

:MTESt:AMASk:YDELta

N (see [page 1650](#))

Command Syntax :MTESt:AMASk:YDELta <value>
 <value> ::= Y delta value in NR3 format

The :MTESt:AMASk:YDELta command sets the vertical tolerance around the waveform for the automasking feature. The absolute value of the tolerance will be added and subtracted to vertical values of the waveform to determine the boundaries of the mask.

The vertical tolerance value is interpreted based on the setting specified by the :MTESt:AMASk:UNITs command; thus, if you specify 250-E3, the setting for :MTESt:AMASk:UNITs is CURRent, and the current setting specifies voltage in the vertical direction, the tolerance will be ± 250 mV. If the setting for :MTESt:AMASk:UNITs is DIVisions, the same Y delta value will set the tolerance to ± 250 millidivisions, or 1/4 of a division.

Query Syntax :MTESt:AMASk:YDELta?

The :MTESt:AMASk:YDELta? query returns the current setting of the ΔY tolerance for automasking. If your computer program will interpret this value, it should also request the current measurement system using the :MTESt:AMASk:UNITs query.

Return Format <value><NL>
 <value> ::= Y delta value in NR3 format

- See Also**
- ["Introduction to :MTESt Commands"](#) on page 713
 - [":MTESt:AMASk:UNITs"](#) on page 719
 - [":MTESt:AMASk:XDELta"](#) on page 720
 - [":MTESt:AMASk:SOURce"](#) on page 718
 - [":MTESt:SOURce"](#) on page 743

Example Code • ["Example Code"](#) on page 713

:MTESt:COUNT:FWAVEforms

N (see [page 1650](#))

Query Syntax :MTESt:COUNT:FWAVEforms? [CHANnel<n>]

<n> ::= 1 to (# analog channels) in NR1 format

The :MTESt:COUNT:FWAVEforms? query returns the total number of failed waveforms in the current mask test run. This count is for all regions and all waveforms collected on the channel specified by the optional parameter or collected on the currently specified source channel (:MTESt:SOURce) if there is no parameter.

Return Format <failed><NL>

<failed> ::= number of failed waveforms in NR1 format.

- See Also**
- ["Introduction to :MTESt Commands"](#) on page 713
 - [":MTESt:COUNT:WAVEforms"](#) on page 725
 - [":MTESt:COUNT:TIME"](#) on page 724
 - [":MTESt:COUNT:RESet"](#) on page 723
 - [":MTESt:SOURce"](#) on page 743

Example Code • ["Example Code"](#) on page 713

:MTESt:COUNT:RESet

N (see [page 1650](#))

Command Syntax :MTESt:COUNT:RESet

The :MTESt:COUNT:RESet command resets the mask statistics.

- See Also**
- ["Introduction to :MTESt Commands"](#) on page 713
 - [":MTESt:COUNT:WAVEforms"](#) on page 725
 - [":MTESt:COUNT:FWAVEforms"](#) on page 722
 - [":MTESt:COUNT:TIME"](#) on page 724

:MTESt:COUNT:TIME

N (see [page 1650](#))

Query Syntax :MTESt:COUNT:TIME?

The :MTESt:COUNT:TIME? query returns the elapsed time in the current mask test run.

Return Format <time><NL>

<time> ::= elapsed seconds in NR3 format.

- See Also**
- ["Introduction to :MTESt Commands"](#) on page 713
 - [":MTESt:COUNT:WAVEforms"](#) on page 725
 - [":MTESt:COUNT:FWAVEforms"](#) on page 722
 - [":MTESt:COUNT:RESet"](#) on page 723

Example Code • ["Example Code"](#) on page 713

:MTESt:COUNT:WAVEforms

N (see [page 1650](#))

Query Syntax :MTESt:COUNT:WAVEforms?

The :MTESt:COUNT:WAVEforms? query returns the total number of waveforms acquired in the current mask test run.

Return Format <count><NL>

<count> ::= number of waveforms in NR1 format.

- See Also**
- ["Introduction to :MTESt Commands"](#) on page 713
 - [":MTESt:COUNT:FWAVEforms"](#) on page 722
 - [":MTESt:COUNT:TIME"](#) on page 724
 - [":MTESt:COUNT:RESet"](#) on page 723

Example Code • ["Example Code"](#) on page 713

:MTESt:DATA

N (see [page 1650](#))

Command Syntax :MTESt:DATA <mask>

<mask> ::= binary block data in IEEE 488.2 # format.

The :MTESt:DATA command loads a mask from binary block data.

Query Syntax :MTESt:DATA?

The :MTESt:DATA? query returns a mask in binary block data format. The format for the data transmission is the # format defined in the IEEE 488.2 specification.

Return Format <mask><NL>

<mask> ::= binary block data in IEEE 488.2 # format

- See Also**
- [":SAVE:MASK\[:START\]"](#) on page 880
 - [":RECall:MASK\[:START\]"](#) on page 862

:MTESt:DELeTe

N (see [page 1650](#))

Command Syntax :MTESt:DELeTe

The :MTESt:DELeTe command clears the currently loaded mask.

- See Also**
- ["Introduction to :MTESt Commands"](#) on page 713
 - [":MTESt:AMASk:CREate"](#) on page 717

:MTESt:ENABle

N (see [page 1650](#))

Command Syntax :MTESt:ENABle <on_off>

<on_off> ::= {{1 | ON} | {0 | OFF}}

The :MTESt:ENABle command enables or disables the mask test features.

- ON – Enables the mask test features.
- OFF – Disables the mask test features.

Query Syntax :MTESt:ENABle?

The :MTESt:ENABle? query returns the current state of mask test features.

Return Format <on_off><NL>

<on_off> ::= {1 | 0}

See Also • ["Introduction to :MTESt Commands"](#) on page 713

:MTESt:LOCK

N (see [page 1650](#))

Command Syntax :MTESt:LOCK <on_off>
 <on_off> ::= {{1 | ON} | {0 | OFF}}

The :MTESt:LOCK command enables or disables the mask lock feature:

- ON – Locks a mask to the SOURce. As the vertical or horizontal scaling or position of the SOURce changes, the mask is redrawn accordingly.
- OFF – The mask is static and does not move.

Query Syntax :MTESt:LOCK?

The :MTESt:LOCK? query returns the current mask lock setting.

Return Format <on_off><NL>
 <on_off> ::= {1 | 0}

- See Also**
- ["Introduction to :MTESt Commands"](#) on page 713
 - [":MTESt:SOURce"](#) on page 743

:MTESt:RMODE

N (see [page 1650](#))

Command Syntax :MTESt:RMODE <rmode>

<rmode> ::= {FOREver | SIGMa | TIME | WAVeforms}

The :MTESt:RMODE command specifies the termination conditions for the mask test:

- FOREver – the mask test runs until it is turned off.
- SIGMa – the mask test runs until the Sigma level is reached. This level is set by the [":MTESt:RMODE:SIGMa"](#) on page 735 command.
- TIME – the mask test runs for a fixed amount of time. The amount of time is set by the [":MTESt:RMODE:TIME"](#) on page 736 command.
- WAVeforms – the mask test runs until a fixed number of waveforms are acquired. The number of waveforms is set by the [":MTESt:RMODE:WAVeforms"](#) on page 737 command.

Query Syntax :MTESt:RMODE?

The :MTESt:RMODE? query returns the currently set termination condition.

Return Format <rmode><NL>

<rmode> ::= {FOR | SIGM | TIME | WAV}

- See Also**
- ["Introduction to :MTESt Commands"](#) on page 713
 - [":MTESt:RMODE:SIGMa"](#) on page 735
 - [":MTESt:RMODE:TIME"](#) on page 736
 - [":MTESt:RMODE:WAVeforms"](#) on page 737

Example Code • ["Example Code"](#) on page 713

:MTESt:RMODe:FACTion:MEASure

N (see [page 1650](#))

Command Syntax :MTESt:RMODe:FACTion:MEASure <on_off>

<on_off> ::= {{1 | ON} | {0 | OFF}}

The :MTESt:RMODe:FACTion:MEASure command sets measuring only mask failures on or off.

When ON, measurements and measurement statistics run only on waveforms that contain a mask violation; passing waveforms do not affect measurements and measurement statistics.

This mode is not available when the acquisition mode is set to Averaging.

Query Syntax :MTESt:RMODe:FACTion:MEASure?

The :MTESt:RMODe:FACTion:MEASure? query returns the current mask failure measure setting.

Return Format <on_off><NL>

<on_off> ::= {1 | 0}

- See Also**
- ["Introduction to :MTESt Commands"](#) on page 713
 - [":MTESt:RMODe:FACTion:PRINT"](#) on page 732
 - [":MTESt:RMODe:FACTion:SAVE"](#) on page 733
 - [":MTESt:RMODe:FACTion:STOP"](#) on page 734

:MTESt:RMODe:FACTion:PRINt

N (see [page 1650](#))

Command Syntax :MTESt:RMODe:FACTion:PRINt <on_off>

<on_off> ::= {{1 | ON} | {0 | OFF}}

The :MTESt:RMODe:FACTion:PRINt command sets printing on mask failures on or off.

NOTE

Setting :MTESt:RMODe:FACTion:PRINt ON automatically sets :MTESt:RMODe:FACTion:SAVE OFF.

See [Chapter 21](#), “:HARDcopy/:HCOPY Commands,” starting on page 505 for more information on setting the hardcopy device and formatting options.

Query Syntax :MTESt:RMODe:FACTion:PRINt?

The :MTESt:RMODe:FACTion:PRINt? query returns the current mask failure print setting.

Return Format <on_off><NL>

<on_off> ::= {1 | 0}

- See Also**
- ["Introduction to :MTESt Commands"](#) on page 713
 - [":MTESt:RMODe:FACTion:MEASure"](#) on page 731
 - [":MTESt:RMODe:FACTion:SAVE"](#) on page 733
 - [":MTESt:RMODe:FACTion:STOP"](#) on page 734

:MTESt:RMODe:FACTion:SAVE

N (see [page 1650](#))

Command Syntax :MTESt:RMODe:FACTion:SAVE <on_off>

<on_off> ::= {{1 | ON} | {0 | OFF}}

The :MTESt:RMODe:FACTion:SAVE command sets saving on mask failures on or off.

NOTE

Setting :MTESt:RMODe:FACTion:SAVE ON automatically sets :MTESt:RMODe:FACTion:PRINT OFF.

See [Chapter 32](#), “:SAVE Commands,” starting on page 867 for more information on save options.

Query Syntax :MTESt:RMODe:FACTion:SAVE?

The :MTESt:RMODe:FACTion:SAVE? query returns the current mask failure save setting.

Return Format <on_off><NL>

<on_off> ::= {1 | 0}

- See Also**
- ["Introduction to :MTESt Commands"](#) on page 713
 - [":MTESt:RMODe:FACTion:MEASure"](#) on page 731
 - [":MTESt:RMODe:FACTion:PRINT"](#) on page 732
 - [":MTESt:RMODe:FACTion:STOP"](#) on page 734

:MTESt:RMODe:FACTion:STOP

N (see [page 1650](#))

Command Syntax :MTESt:RMODe:FACTion:STOP <on_off>

<on_off> ::= {{1 | ON} | {0 | OFF}}

The :MTESt:RMODe:FACTion:STOP command sets stopping on a mask failure on or off. When this setting is ON and a mask violation is detected, the mask test is stopped and the acquisition system is stopped.

Query Syntax :MTESt:RMODe:FACTion:STOP?

The :MTESt:RMODe:FACTion:STOP? query returns the current mask failure stop setting.

Return Format <on_off><NL>

<on_off> ::= {1 | 0}

- See Also**
- ["Introduction to :MTESt Commands"](#) on page 713
 - [":MTESt:RMODe:FACTion:MEASure"](#) on page 731
 - [":MTESt:RMODe:FACTion:PRINt"](#) on page 732
 - [":MTESt:RMODe:FACTion:SAVE"](#) on page 733

:MTESt:RMODE:SIGMa

N (see [page 1650](#))

Command Syntax :MTESt:RMODE:SIGMa <level>

<level> ::= from 0.1 to 9.3 in NR3 format

When the :MTESt:RMODE command is set to SIGMa, the :MTESt:RMODE:SIGMa command sets the *test sigma* level to which a mask test runs. *Test sigma* is the best achievable process sigma, assuming no failures. (*Process sigma* is calculated using the number of failures per test.) The *test sigma* level indirectly specifies the number of waveforms that must be tested (in order to reach the sigma level).

Query Syntax :MTESt:RMODE:SIGMa?

The :MTESt:RMODE:SIGMa? query returns the current Sigma level setting.

Return Format <level><NL>

<level> ::= from 0.1 to 9.3 in NR3 format

See Also • ["Introduction to :MTESt Commands"](#) on page 713

• [":MTESt:RMODE"](#) on page 730

Example Code • ["Example Code"](#) on page 713

:MTESt:RMODE:TIME

N (see [page 1650](#))

Command Syntax :MTESt:RMODE:TIME <seconds>

<seconds> ::= from 1 to 86400 in NR3 format

When the :MTESt:RMODE command is set to TIME, the :MTESt:RMODE:TIME command sets the number of seconds for a mask test to run.

Query Syntax :MTESt:RMODE:TIME?

The :MTESt:RMODE:TIME? query returns the number of seconds currently set.

Return Format <seconds><NL>

<seconds> ::= from 1 to 86400 in NR3 format

- See Also**
- ["Introduction to :MTESt Commands"](#) on page 713
 - [":MTESt:RMODE"](#) on page 730

:MTESt:RMODE:WAVEforms

N (see [page 1650](#))

Command Syntax :MTESt:RMODE:WAVEforms <count>

<count> ::= number of waveforms in NR1 format
from 1 to 2,000,000,000

When the :MTESt:RMODE command is set to WAVEforms, the :MTESt:RMODE:WAVEforms command sets the number of waveform acquisitions that are mask tested.

Query Syntax :MTESt:RMODE:WAVEforms?

The :MTESt:RMODE:WAVEforms? query returns the number of waveforms currently set.

Return Format <count><NL>

<count> ::= number of waveforms in NR1 format
from 1 to 2,000,000,000

- See Also**
- ["Introduction to :MTESt Commands"](#) on page 713
 - [":MTESt:RMODE"](#) on page 730

:MTESt:SCALe:BIND

N (see [page 1650](#))

Command Syntax :MTESt:SCALe:BIND <on_off>

<on_off> ::= {{1 | ON} | {0 | OFF}}

The :MTESt:SCALe:BIND command enables or disables Bind 1 & 0 Levels (Bind -1 & 0 Levels for inverted masks) control:

- ON —

If the Bind 1 & 0 Levels control is enabled, the 1 Level and the 0 Level controls track each other. Adjusting either the 1 Level or the 0 Level control shifts the position of the mask up or down without changing its size.

If the Bind -1 & 0 Levels control is enabled, the -1 Level and the 0 Level controls track each other. Adjusting either the -1 Level or the 0 Level control shifts the position of the mask up or down without changing its size.

- OFF —

If the Bind 1 & 0 Levels control is disabled, adjusting either the 1 Level or the 0 Level control changes the vertical height of the mask.

If the Bind -1 & 0 Levels control is disabled, adjusting either the -1 Level or the 0 Level control changes the vertical height of the mask.

Query Syntax :MTESt:SCALe:BIND?

The :MTESt:SCALe:BIND? query returns the value of the Bind 1&0 control (Bind -1&0 for inverted masks).

Return Format <on_off><NL>

<on_off> ::= {1 | 0}

- See Also**
- ["Introduction to :MTESt Commands"](#) on page 713
 - [":MTESt:SCALe:X1"](#) on page 739
 - [":MTESt:SCALe:XDELta"](#) on page 740
 - [":MTESt:SCALe:Y1"](#) on page 741
 - [":MTESt:SCALe:Y2"](#) on page 742

:MTESt:SCALE:X1

N (see [page 1650](#))

Command Syntax :MTESt:SCALE:X1 <x1_value>
 <x1_value> ::= X1 value in NR3 format

The :MTESt:SCALE:X1 command defines where X=0 in the base coordinate system used for mask testing. The other X-coordinate is defined by the :MTESt:SCALE:XDELta command. Once the X1 and XDELta coordinates are set, all X values of vertices in the mask regions are defined with respect to this value, according to the equation:

$$X = (X * \Delta X) + X1$$

Thus, if you set X1 to 100 ms, and XDELta to 100 ms, an X value of 0.100 is a vertex at 110 ms.

The oscilloscope uses this equation to normalize vertices. This simplifies reprogramming to handle different data rates. For example, if you halve the period of the waveform of interest, you need only to adjust the XDELta value to set up the mask for the new waveform.

The X1 value is a time value specifying the location of the X1 coordinate, which will then be treated as X=0 for mask regions coordinates.

Query Syntax :MTESt:SCALE:X1?

The :MTESt:SCALE:X1? query returns the current X1 coordinate setting.

Return Format <x1_value><NL>
 <x1_value> ::= X1 value in NR3 format

- See Also**
- ["Introduction to :MTESt Commands"](#) on page 713
 - [":MTESt:SCALE:BIND"](#) on page 738
 - [":MTESt:SCALE:XDELta"](#) on page 740
 - [":MTESt:SCALE:Y1"](#) on page 741
 - [":MTESt:SCALE:Y2"](#) on page 742

:MTESt:SCALE:XDELta

N (see [page 1650](#))

Command Syntax :MTESt:SCALE:XDELta <xdelta_value>

<xdelta_value> ::= X delta value in NR3 format

The :MTESt:SCALE:XDELta command defines the position of the X2 marker with respect to the X1 marker. In the mask test coordinate system, the X1 marker defines where $X=0$; thus, the X2 marker defines where $X=1$.

Because all X vertices of the regions defined for mask testing are normalized with respect to X1 and ΔX , redefining ΔX also moves those vertices to stay in the same locations with respect to X1 and ΔX . Thus, in many applications, it is best if you define XDELta as a pulse width or bit period. Then, a change in data rate without corresponding changes in the waveform can easily be handled by changing ΔX .

The X-coordinate of polygon vertices is normalized using this equation:

$$X = (X * \Delta X) + X1$$

The X delta value is a time value specifying the distance of the X2 marker with respect to the X1 marker.

For example, if the period of the waveform you wish to test is 1 ms, setting ΔX to 1 ms ensures that the waveform's period is between the X1 and X2 markers.

Query Syntax :MTESt:SCALE:XDELta?

The :MTESt:SCALE:XDELta? query returns the current value of ΔX .

Return Format <xdelta_value><NL>

<xdelta_value> ::= X delta value in NR3 format

- See Also**
- ["Introduction to :MTESt Commands"](#) on page 713
 - [":MTESt:SCALE:BIND"](#) on page 738
 - [":MTESt:SCALE:X1"](#) on page 739
 - [":MTESt:SCALE:Y1"](#) on page 741
 - [":MTESt:SCALE:Y2"](#) on page 742

:MTESt:SCALe:Y1

N (see [page 1650](#))

Command Syntax :MTESt:SCALe:Y1 <y1_value>

<y1_value> ::= Y1 value in NR3 format

The :MTESt:SCALe:Y1 command defines where Y=0 in the coordinate system for mask testing. All Y values of vertices in the coordinate system are defined with respect to the boundaries set by SCALe:Y1 and SCALe:Y2 according to the equation:

$$Y = (Y * (Y2 - Y1)) + Y1$$

Thus, if you set Y1 to 100 mV, and Y2 to 1 V, a Y value of 0.100 in a vertex is at 190 mV.

The Y1 value is a voltage value specifying the point at which Y=0.

Query Syntax :MTESt:SCALe:Y1?

The :MTESt:SCALe:Y1? query returns the current setting of the Y1 marker.

Return Format <y1_value><NL>

<y1_value> ::= Y1 value in NR3 format

- See Also**
- ["Introduction to :MTESt Commands"](#) on page 713
 - [":MTESt:SCALe:BIND"](#) on page 738
 - [":MTESt:SCALe:X1"](#) on page 739
 - [":MTESt:SCALe:XDELta"](#) on page 740
 - [":MTESt:SCALe:Y2"](#) on page 742

:MTESt:SCALe:Y2

N (see [page 1650](#))

Command Syntax :MTESt:SCALe:Y2 <y2_value>

<y2_value> ::= Y2 value in NR3 format

The :MTESt:SCALe:Y2 command defines the Y2 marker in the coordinate system for mask testing. All Y values of vertices in the coordinate system are defined with respect to the boundaries defined by SCALe:Y1 and SCALe:Y2 according to the following equation:

$$Y = (Y * (Y2 - Y1)) + Y1$$

Thus, if you set Y1 to 100 mV, and Y2 to 1 V, a Y value of 0.100 in a vertex is at 190 mV.

The Y2 value is a voltage value specifying the location of the Y2 marker.

Query Syntax :MTESt:SCALe:Y2?

The :MTESt:SCALe:Y2? query returns the current setting of the Y2 marker.

Return Format <y2_value><NL>

<y2_value> ::= Y2 value in NR3 format

- See Also**
- ["Introduction to :MTESt Commands"](#) on page 713
 - [":MTESt:SCALe:BIND"](#) on page 738
 - [":MTESt:SCALe:X1"](#) on page 739
 - [":MTESt:SCALe:XDELta"](#) on page 740
 - [":MTESt:SCALe:Y1"](#) on page 741

:MTESt:SOURce

N (see [page 1650](#))

Command Syntax :MTESt:SOURce <source>

<source> ::= CHANnel<n>

<n> ::= 1 to (# analog channels) in NR1 format

The :MTESt:SOURce command selects the channel which is configured by the commands contained in a mask file when it is loaded.

Query Syntax :MTESt:SOURce?

The :MTESt:SOURce? query returns the channel which is configured by the commands contained in the current mask file.

Return Format <source><NL>

<source> ::= {CHAN<n> | NONE}

<n> ::= 1 to (# analog channels) in NR1 format

- See Also**
- ["Introduction to :MTESt Commands"](#) on page 713
 - [":MTESt:AMASK:SOURce"](#) on page 718

:MTESt:TITLe

N (see [page 1650](#))

Query Syntax :MTESt:TITLe?

The :MTESt:TITLe? query returns the mask title which is a string of up to 128 characters. The title is displayed in the mask test dialog box and mask test tab when a mask file is loaded.

Return Format <title><NL>

<title> ::= a string of up to 128 ASCII characters.

See Also • ["Introduction to :MTESt Commands"](#) on page 713

29 :POD Commands

Control all oscilloscope functions associated with groups of digital channels. See ["Introduction to :POD<n> Commands"](#) on page 745.

Table 116 :POD<n> Commands Summary

Command	Query	Options and Query Returns
:POD<n>:DISPlay { {0 OFF} {1 ON} } (see page 747)	:POD<n>:DISPlay? (see page 747)	{0 1} <n> ::= 1-2 in NR1 format
:POD<n>:SIZE <value> (see page 748)	:POD<n>:SIZE? (see page 748)	<value> ::= {SMALL MEDIUM LARGE}
:POD<n>:THReshold <type>[suffix] (see page 749)	:POD<n>:THReshold? (see page 749)	<n> ::= 1-2 in NR1 format <type> ::= {CMOS ECL TTL <user defined value>} <user defined value> ::= value in NR3 format [suffix] ::= {V mV uV }

Introduction to :POD<n> Commands <n> ::= {1 | 2}

The POD subsystem commands control the viewing and threshold of groups of digital channels.

POD1 ::= D0-D7

POD2 ::= D8-D15

NOTE

These commands are only valid for the MSO models.

Reporting the Setup

Use :POD1? or :POD2? to query setup information for the POD subsystem.

Return Format

The following is a sample response from the :POD1? query. In this case, the query was issued following a *RST command.

```
:POD1:DISP 0;THR +1.40E+00
```

:POD<n>:DISPlay

N (see [page 1650](#))

Command Syntax :POD<n>:DISPlay <display>

<display> ::= {{1 | ON} | {0 | OFF}}

<n> ::= An integer, 1 or 2, is attached as a suffix to the command and defines the group of channels that are affected by the command.

POD1 ::= D0-D7

POD2 ::= D8-D15

The :POD<n>:DISPlay command turns displaying of the specified group of channels on or off.

NOTE

This command is only valid for the MSO models.

Query Syntax :POD<n>:DISPlay?

The :POD<n>:DISPlay? query returns the current display setting of the specified group of channels.

Return Format <display><NL>

<display> ::= {0 | 1}

- See Also**
- ["Introduction to :POD<n> Commands"](#) on page 745
 - [":DIGital<d>:DISPlay"](#) on page 385
 - [":CHANnel<n>:DISPlay"](#) on page 332
 - [":VIEW"](#) on page 280
 - [":BLANK"](#) on page 248
 - [":STATus"](#) on page 277

:POD<n>:SIZE

N (see [page 1650](#))

Command Syntax :POD<n>:SIZE <value>

<n> ::= An integer, 1 or 2, is attached as a suffix to the command and defines the group of channels that are affected by the command.

POD1 ::= D0-D7

POD2 ::= D8-D15

<value> ::= {SMALl | MEDium | LARGe}

The :POD<n>:SIZE command specifies the size of digital channels on the display. Sizes are set for all pods. Therefore, if you set the size on pod 1 (for example), the same size is set on pod 2 as well.

NOTE

This command is only valid for the MSO models.

Query Syntax :POD<n>:SIZE?

The :POD<n>:SIZE? query returns the digital channels size setting.

Return Format <size_value><NL>

<size_value> ::= {SMAL | MED | LARG}

- See Also**
- ["Introduction to :POD<n> Commands"](#) on page 745
 - [":DIGital<d>:SIZE"](#) on page 388
 - [":DIGital<d>:POSition"](#) on page 387

:POD<n>:THReshold

N (see [page 1650](#))

Command Syntax :POD<n>:THReshold <type>[<suffix>]

<n> ::= An integer, 1 or 2, is attached as a suffix to the command and defines the group of channels that are affected by the command.

<type> ::= {CMOS | ECL | TTL | <user defined value>}

<user defined value> ::= -8.00 to +8.00 in NR3 format

<suffix> ::= {V | mV | uV}

POD1 ::= D0-D7

POD2 ::= D8-D15

TTL ::= 1.4V

CMOS ::= 2.5V

ECL ::= -1.3V

The :POD<n>:THReshold command sets the threshold for the specified group of channels. The threshold is used for triggering purposes and for displaying the digital data as high (above the threshold) or low (below the threshold).

NOTE

This command is only valid for the MSO models.

Query Syntax :POD<n>:THReshold?

The :POD<n>:THReshold? query returns the threshold value for the specified group of channels.

Return Format <threshold><NL>

<threshold> ::= Floating point number in NR3 format

- See Also**
- ["Introduction to :POD<n> Commands"](#) on page 745
 - [":DIGital<d>:THReshold"](#) on page 389
 - [":TRIGger\[:EDGE\]:LEVel"](#) on page 1376

Example Code

```
' THRESHOLD - This command is used to set the voltage threshold for
' the waveforms. There are three preset values (TTL, CMOS, and ECL)
' and you can also set a user-defined threshold value between
' -8.0 volts and +8.0 volts.
'
' In this example, we set channels 0-7 to CMOS, then set channels
' 8-15 to a user-defined 2.0 volts, and then set the external trigger
' to TTL. Of course, you only need to set the thresholds for the
' channels you will be using in your program.
```

```
' Set channels 0-7 to CMOS threshold.  
myScope.WriteString ":POD1:THRESHOLD CMOS"  
  
' Set channels 8-15 to 2.0 volts.  
myScope.WriteString ":POD2:THRESHOLD 2.0"  
  
' Set external channel to TTL threshold (short form).  
myScope.WriteString ":TRIG:LEV TTL,EXT"
```

See complete example programs at: [Chapter 46](#), “Programming Examples,” starting on page 1659

30 :POWer Commands

These :POWer commands are available when the power measurements and analysis application is licensed and enabled.

Table 117 :POWer Commands Summary

Command	Query	Options and Query Returns
n/a	:POWer:CLResponse? (see page 759)	n/a
:POWer:CLResponse:APPLy (see page 760)	n/a	n/a
n/a	:POWer:CLResponse:DATA? [SWEep SINGLE] (see page 761)	<binary_block> ::= comma-separated data with newlines at the end of each row
n/a	:POWer:CLResponse:DATA:GMARgin? (see page 762)	<gain_margin> ::= gain margin in dB in NR3 format.
n/a	:POWer:CLResponse:DATA:GMARgin:FREQuency? (see page 763)	<frequency> ::= 0 degrees phase crossover frequency in Hz in NR3 format
n/a	:POWer:CLResponse:DATA:PMARgin? (see page 764)	<phase_margin> ::= phase margin in degrees in NR3 format.
n/a	:POWer:CLResponse:DATA:PMARgin:FREQuency? (see page 765)	<frequency> ::= 0dB gain crossover frequency in Hz in NR3 format.
:POWer:CLResponse:FREQuency:MODE <mode> (see page 766)	:POWer:CLResponse:FREQuency:MODE? (see page 766)	<mode> ::= {SWEep SINGLE}
:POWer:CLResponse:FREQuency:SINGLE <value>[suffix] (see page 767)	:POWer:CLResponse:FREQuency:SINGLE? (see page 767)	<value> ::= {20 100 1000 10000 100000 1000000 10000000 20000000} [suffix] ::= {Hz kHz MHz}

Table 117 :POWer Commands Summary (continued)

Command	Query	Options and Query Returns
:POWer:CLResponse:FREQuency:STARt <value>[suffix] (see page 768)	:POWer:CLResponse:FREQuency:STARt? (see page 768)	<value> ::= {20 100 1000 10000 100000 1000000 10000000} [suffix] ::= {Hz kHz MHz}
:POWer:CLResponse:FREQuency:STOP <value>[suffix] (see page 769)	:POWer:CLResponse:FREQuency:STOP? (see page 769)	<value> ::= {100 1000 10000 100000 1000000 10000000 20000000} [suffix] ::= {Hz kHz MHz}
:POWer:CLResponse:PPDeade <pts> (see page 770)	:POWer:CLResponse:PPDeade? (see page 770)	<pts> ::= {10 20 30 40 50 60 70 80 90 100}
:POWer:CLResponse:SOURce:INPut <source> (see page 771)	:POWer:CLResponse:SOURce:INPut? (see page 771)	<source> ::= CHANnel<n> <n> ::= 1 to (# analog channels) in NR1 format
:POWer:CLResponse:SOURce:OUTPut <source> (see page 772)	:POWer:CLResponse:SOURce:OUTPut? (see page 772)	<source> ::= CHANnel<n> <n> ::= 1 to (# analog channels) in NR1 format
:POWer:CLResponse:TRACe <selection> (see page 773)	:POWer:CLResponse:TRACe? (see page 773)	<selection> ::= {NONE ALL GAIN PHASe}[, {GAIN PHASe}]
:POWer:CLResponse:WGEN:LOAD <impedance> (see page 774)	:POWer:CLResponse:WGEN:LOAD? (see page 774)	<impedance> ::= {ONEMeg FIFTy}
:POWer:CLResponse:WGEN:VOLTagE <amplitude>[, <range>] (see page 775)	:POWer:CLResponse:WGEN:VOLTagE? [<range>] (see page 775)	<amplitude> ::= amplitude in volts in NR3 format <range> ::= {F20HZ F100HZ F1KHZ F10KHZ F100KHZ F1MHZ F10MHZ F20MHZ}
:POWer:CLResponse:WGEN:VOLTagE:PROFile { {0 OFF} {1 ON} } (see page 776)	:POWer:CLResponse:WGEN:VOLTagE:PROFile? (see page 776)	{0 1}
:POWer:DESKew (see page 777)	n/a	n/a
:POWer:EFFiciency:APPLy (see page 778)	n/a	n/a
:POWer:EFFiciency:TYPE <type> (see page 779)	:POWer:EFFiciency:TYPE? (see page 779)	<type> ::= {DCDC DCAC ACDC ACAC}

Table 117 :POWer Commands Summary (continued)

Command	Query	Options and Query Returns
:POWer:ENABle {{0 OFF} {1 ON}} (see page 780)	:POWer:ENABle? (see page 780)	{0 1}
:POWer:HARMonics:APPLy (see page 781)	n/a	n/a
n/a	:POWer:HARMonics:DATA? (see page 782)	<binary_block> ::= comma-separated data with newlines at the end of each row
:POWer:HARMonics:DISP lay <display> (see page 783)	:POWer:HARMonics:DISP lay? (see page 783)	<display> ::= {TABLE BAR OFF}
n/a	:POWer:HARMonics:FAIL count? (see page 784)	<count> ::= integer in NR1 format
:POWer:HARMonics:LINE <frequency> (see page 785)	:POWer:HARMonics:LINE ? (see page 785)	<frequency> ::= {F50 F60 F400 AUTO}
n/a	:POWer:HARMonics:POWe rfactor? (see page 786)	<value> ::= Class C power factor in NR3 format
:POWer:HARMonics:RPOW er <source> (see page 787)	:POWer:HARMonics:RPOW er? (see page 787)	<source> ::= {MEASured USER}
:POWer:HARMonics:RPOW er:USER <value> (see page 788)	:POWer:HARMonics:RPOW er:USER? (see page 788)	<value> ::= Watts from 1.0 to 600.0 in NR3 format
n/a	:POWer:HARMonics:RUNC out? (see page 789)	<count> ::= integer in NR1 format
:POWer:HARMonics:STAN dard <class> (see page 790)	:POWer:HARMonics:STAN dard? (see page 790)	<class> ::= {A B C D}
n/a	:POWer:HARMonics:STAT us? (see page 791)	<status> ::= {PASS FAIL UNTested}
n/a	:POWer:HARMonics:THD? (see page 792)	<value> ::= Total Harmonics Distortion in NR3 format
:POWer:INRush:APPLy (see page 793)	n/a	n/a
:POWer:INRush:EXIT (see page 794)	n/a	n/a

Table 117 :POWer Commands Summary (continued)

Command	Query	Options and Query Returns
:POWer:INRush:NEXT (see page 795)	n/a	n/a
:POWer:ITYPE <type> (see page 796)	:POWer:ITYPE? (see page 796)	<type> ::= {DC AC}
:POWer:MODulation:APPLy (see page 797)	n/a	n/a
:POWer:MODulation:SOURce <source> (see page 798)	:POWer:MODulation:SOURce? (see page 798)	<source> ::= {V I}
:POWer:MODulation:TYPE <modulation> (see page 799)	:POWer:MODulation:TYPE? (see page 799)	<modulation> ::= {VAverage ACRMs VRATio PERiod FREQuency PWIDith NWIDth DUTYcycle RISetime FALLtime}
:POWer:ONOff:APPLy (see page 800)	n/a	n/a
:POWer:ONOff:EXIT (see page 801)	n/a	n/a
:POWer:ONOff:NEXT (see page 802)	n/a	n/a
:POWer:ONOff:TEST { {0 OFF} {1 ON} } (see page 803)	:POWer:ONOff:TEST? (see page 803)	{0 1}
:POWer:ONOff:THReshols <type>, <input_thr>, <output_thr> (see page 804)	:POWer:ONOff:THReshols? <type> (see page 804)	<type> ::= {0 1} <input_thr> ::= percent from 0-100 in NR1 format <output_thr> ::= percent from 0-100 in NR1 format
n/a	:POWer:PSRR? (see page 806)	n/a
:POWer:PSRR:APPLy (see page 807)	n/a	n/a
n/a	:POWer:PSRR:DATA? [SWEep SINGLE] (see page 808)	<binary_block> ::= comma-separated data with newlines at the end of each row
:POWer:PSRR:FREQuency:MAXimum <value>[suffix] (see page 809)	:POWer:PSRR:FREQuency:MAXimum? (see page 809)	<value> ::= {10 100 1000 10000 100000 1000000 10000000 20000000} [suffix] ::= {Hz kHz MHz}

Table 117 :POWer Commands Summary (continued)

Command	Query	Options and Query Returns
:POWer:PSRR:FREQuency :MINimum <value>[suffix] (see page 810)	:POWer:PSRR:FREQuency :MINimum? (see page 810)	<value> ::= {1 10 100 1000 10000 100000 1000000 10000000} [suffix] ::= {Hz kHz MHz}
:POWer:PSRR:FREQuency :MODE <mode> (see page 811)	:POWer:PSRR:FREQuency :MODE? (see page 811)	<mode> ::= {SWEep SINGLE}
:POWer:PSRR:FREQuency :SINGLE <value>[suffix] (see page 812)	:POWer:PSRR:FREQuency :SINGLE? (see page 812)	<value> ::= {1 10 100 1000 10000 100000 1000000 10000000 20000000} [suffix] ::= {Hz kHz MHz}
:POWer:PSRR:PPDecade <pts> (see page 813)	:POWer:PSRR:PPDecade? (see page 813)	<pts> ::= {10 20 30 40 50 60 70 80 90 100}
:POWer:PSRR:SOURce:IN Put <source> (see page 814)	:POWer:PSRR:SOURce:IN Put? (see page 814)	<source> ::= CHANNEL<n> <n> ::= 1 to (# analog channels) in NR1 format
:POWer:PSRR:SOURce:OU TPut <source> (see page 815)	:POWer:PSRR:SOURce:OU TPut? (see page 815)	<source> ::= CHANNEL<n> <n> ::= 1 to (# analog channels) in NR1 format
:POWer:PSRR:TRACe <selection> (see page 816)	:POWer:PSRR:TRACe? (see page 816)	<selection> ::= {NONE GAIN}
:POWer:PSRR:WGEN:LOAD <impedance> (see page 817)	:POWer:PSRR:WGEN:LOAD ? (see page 817)	<impedance> ::= {ONEMeg FIFTy}
:POWer:PSRR:WGEN:VOLT age <amplitude>[,<range>] (see page 818)	:POWer:PSRR:WGEN:VOLT age? [<range>] (see page 818)	<amplitude> ::= amplitude in volts in NR3 format <range> ::= {F20HZ F100HZ F1KHZ F10KHZ F100KHZ F1MHZ F10MHZ F20MHZ}
:POWer:PSRR:WGEN:VOLT age:PROFile {{0 OFF} {1 ON}} (see page 819)	:POWer:PSRR:WGEN:VOLT age:PROFile? (see page 819)	{0 1}
:POWer:QUALity:APPLY (see page 820)	n/a	n/a
:POWer:RIPPlE:APPLY (see page 821)	n/a	n/a

Table 117 :POWer Commands Summary (continued)

Command	Query	Options and Query Returns
:POWer:SIGNals:AUTose tup <analysis> (see page 822)	n/a	<analysis> ::= {HARMonics EFFiciency RIPple MODulation QUALity SLEW SWITch RDSVce}
:POWer:SIGNals:CYCLes :HARMonics <count> (see page 823)	:POWer:SIGNals:CYCLes :HARMonics? (see page 823)	<count> ::= integer in NR1 format Legal values are 1 to 100.
:POWer:SIGNals:CYCLes :QUALity <count> (see page 824)	:POWer:SIGNals:CYCLes :QUALity? (see page 824)	<count> ::= integer in NR1 format Legal values are 1 to 100.
:POWer:SIGNals:DURati on:EFFiciency <value>[suffix] (see page 825)	:POWer:SIGNals:DURati on:EFFiciency? (see page 825)	<value> ::= value in NR3 format [suffix] ::= {s ms us ns}
:POWer:SIGNals:DURati on:MODulation <value>[suffix] (see page 826)	:POWer:SIGNals:DURati on:MODulation? (see page 826)	<value> ::= value in NR3 format [suffix] ::= {s ms us ns}
:POWer:SIGNals:DURati on:ONOff:OFF <value>[suffix] (see page 827)	:POWer:SIGNals:DURati on:ONOff:OFF? (see page 827)	<value> ::= value in NR3 format [suffix] ::= {s ms us ns}
:POWer:SIGNals:DURati on:ONOff:ON <value>[suffix] (see page 828)	:POWer:SIGNals:DURati on:ONOff:ON? (see page 828)	<value> ::= value in NR3 format [suffix] ::= {s ms us ns}
:POWer:SIGNals:DURati on:RIPple <value>[suffix] (see page 829)	:POWer:SIGNals:DURati on:RIPple? (see page 829)	<value> ::= value in NR3 format [suffix] ::= {s ms us ns}
:POWer:SIGNals:DURati on:TRANsient <value>[suffix] (see page 830)	:POWer:SIGNals:DURati on:TRANsient? (see page 830)	<value> ::= value in NR3 format [suffix] ::= {s ms us ns}
:POWer:SIGNals:IEXpec ted <value>[suffix] (see page 831)	:POWer:SIGNals:IEXpec ted? (see page 831)	<value> ::= Expected current value in NR3 format [suffix] ::= {A mA}
:POWer:SIGNals:OVERsh oot <percent> (see page 832)	:POWer:SIGNals:OVERsh oot? (see page 832)	<percent> ::= percent of overshoot value in NR1 format [suffix] ::= {V mV}}

Table 117 :POWer Commands Summary (continued)

Command	Query	Options and Query Returns
:POWer:SIGNals:VMAXimum:INRush <value>[suffix] (see page 833)	:POWer:SIGNals:VMAXimum:INRush? (see page 833)	<value> ::= Maximum expected input Voltage in NR3 format [suffix] ::= {V mV}
:POWer:SIGNals:VMAXimum:ONOff:OFF <value>[suffix] (see page 834)	:POWer:SIGNals:VMAXimum:ONOff:OFF? (see page 834)	<value> ::= Maximum expected input Voltage in NR3 format [suffix] ::= {V mV}
:POWer:SIGNals:VMAXimum:ONOff:ON <value>[suffix] (see page 835)	:POWer:SIGNals:VMAXimum:ONOff:ON? (see page 835)	<value> ::= Maximum expected input Voltage in NR3 format [suffix] ::= {V mV}
:POWer:SIGNals:VSTeady:ONOff:OFF <value>[suffix] (see page 836)	:POWer:SIGNals:VSTeady:ONOff:OFF? (see page 836)	<value> ::= Expected steady stage output Voltage value in NR3 format [suffix] ::= {V mV}
:POWer:SIGNals:VSTeady:ONOff:ON <value>[suffix] (see page 837)	:POWer:SIGNals:VSTeady:ONOff:ON? (see page 837)	<value> ::= Expected steady stage output Voltage value in NR3 format [suffix] ::= {V mV}
:POWer:SIGNals:VSTeady:TRANSient <value>[suffix] (see page 838)	:POWer:SIGNals:VSTeady:TRANSient? (see page 838)	<value> ::= Expected steady stage output Voltage value in NR3 format [suffix] ::= {V mV}
:POWer:SIGNals:SOURce:CURRENT<i> <source> (see page 839)	:POWer:SIGNals:SOURce:CURRENT<i>? (see page 839)	<i> ::= 1, 2 in NR1 format <source> ::= CHANnel<n> <n> ::= 1 to (# analog channels) in NR1 format
:POWer:SIGNals:SOURce:VOLTage<i> <source> (see page 840)	:POWer:SIGNals:SOURce:VOLTage<i>? (see page 840)	<i> ::= 1, 2 in NR1 format <source> ::= CHANnel<n> <n> ::= 1 to (# analog channels) in NR1 format
:POWer:SLEW:APPLy (see page 841)	n/a	n/a
:POWer:SLEW:SOURce <source> (see page 842)	:POWer:SLEW:SOURce? (see page 842)	<source> ::= {V I}
:POWer:SWITCh:APPLy (see page 843)	n/a	n/a

Table 117 :POWer Commands Summary (continued)

Command	Query	Options and Query Returns
:POWer:SWITCh:CONduct ion <conduction> (see page 844)	:POWer:SWITCh:CONduct ion? (see page 844)	<conduction> ::= {WAVEform RDS VCE}
:POWer:SWITCh:IREFere nce <percent> (see page 845)	:POWer:SWITCh:IREFere nce? (see page 845)	<percent> ::= percent in NR1 format
:POWer:SWITCh:RDS <value>[suffix] (see page 846)	:POWer:SWITCh:RDS? (see page 846)	<value> ::= Rds(on) value in NR3 format [suffix] ::= {OHM mOHM}
:POWer:SWITCh:VCE <value>[suffix] (see page 847)	:POWer:SWITCh:VCE? (see page 847)	<value> ::= Vce(sat) value in NR3 format [suffix] ::= {V mV}
:POWer:SWITCh:VREFere nce <percent> (see page 848)	:POWer:SWITCh:VREFere nce? (see page 848)	<percent> ::= percent in NR1 format
:POWer:TRANsient:APPL y (see page 849)	n/a	n/a
:POWer:TRANsient:EXIT (see page 850)	n/a	n/a
:POWer:TRANsient:IINi tial <value>[suffix] (see page 851)	:POWer:TRANsient:IINi tial? (see page 851)	<value> ::= Initial current value in NR3 format [suffix] ::= {A mA}
:POWer:TRANsient:INEW <value>[suffix] (see page 852)	:POWer:TRANsient:INEW ? (see page 852)	<value> ::= New current value in NR3 format [suffix] ::= {A mA}
:POWer:TRANsient:NEXT (see page 853)	n/a	n/a

:POWer:CLResponse

N (see [page 1650](#))

Query Syntax :POWer:CLResponse?

The :POWer:CLResponse? query returns the Control Loop Response (Bode) power analysis settings.

Return Format <settings_string><NL>

For example, the query returns the following string when issued after the *RST command.

```
:POW:CLR:SOUR:INP CHAN1;OUTP CHAN2;:POW:CLR:FREQ:STAR +100E+00;
STOP +20.000000E+06;:POW:CLR:WGEN:VOLT +200.0E-03;LOAD FIFT
```

- See Also**
- [":POWer:CLResponse:APPLY"](#) on page 760
 - [":POWer:CLResponse:DATA"](#) on page 761
 - [":POWer:CLResponse:FREQuency:MODE"](#) on page 766
 - [":POWer:CLResponse:FREQuency:START"](#) on page 768
 - [":POWer:CLResponse:FREQuency:STOP"](#) on page 769
 - [":POWer:CLResponse:PPDecade"](#) on page 770
 - [":POWer:CLResponse:SOURce:INPut"](#) on page 771
 - [":POWer:CLResponse:SOURce:OUTPut"](#) on page 772
 - [":POWer:CLResponse:WGEN:LOAD"](#) on page 774
 - [":POWer:CLResponse:WGEN:VOLTage"](#) on page 775
 - [":POWer:CLResponse:WGEN:VOLTage:PROFile"](#) on page 776

:POWer:CLResponse:APPLy

N (see [page 1650](#))

Command Syntax :POWer:CLResponse:APPLy

The :POWer:CLResponse:APPLy command performs the control loop response (Bode) analysis to help you determine the margin of a control loop.

A Bode plot measurement plots gain and/or phase as a function of frequency.

You can use the :POWer:CLResponse:TRACe command to specify whether to include gain, phase, both gain and phase, or neither in the frequency response analysis results.

This control loop response analysis requires an input sine wave (from the oscilloscope's waveform generator, Vi) be swept from a low to a high frequency while measuring Vi and Vo RMS voltages at each step frequency, using two channels of the oscilloscope.

For a gain plot, gain (A, in dB units) at each step frequency is computed as $20\log(V_o/V_i)$ and plotted using a math function waveform.

For a phase plot, the phase difference between the channels is measured at each step frequency. Phase measurements and plots are only possible if the input and output waveforms exceed 1 division peak-to-peak (>1 mVpp).

It takes some time for the frequency sweep analysis to complete. You can query bit 0 of the Standard Event Status Register (*ESR?) to find out when the analysis is complete.

- See Also**
- ["*ESR \(Standard Event Status Register\)"](#) on page 218
 - [":POWer:CLResponse"](#) on page 759
 - [":POWer:CLResponse:DATA"](#) on page 761
 - [":POWer:CLResponse:FREQuency:MODE"](#) on page 766
 - [":POWer:CLResponse:FREQuency:SINGle"](#) on page 767
 - [":POWer:CLResponse:FREQuency:STARt"](#) on page 768
 - [":POWer:CLResponse:FREQuency:STOP"](#) on page 769
 - [":POWer:CLResponse:PPDecade"](#) on page 770
 - [":POWer:CLResponse:SOURce:INPut"](#) on page 771
 - [":POWer:CLResponse:SOURce:OUTPut"](#) on page 772
 - [":POWer:CLResponse:TRACe"](#) on page 773
 - [":POWer:CLResponse:WGEN:LOAD"](#) on page 774
 - [":POWer:CLResponse:WGEN:VOLTage"](#) on page 775
 - [":POWer:CLResponse:WGEN:VOLTage:PROFile"](#) on page 776

:POWer:CLResponse:DATA

N (see [page 1650](#))

Query Syntax :POWer:CLResponse:DATA? [SWEep | SINGle]

The :POWer:CLResponse:DATA? query returns data from the Control Loop Response (Bode) power analysis.

The comma-separated value format is suitable for spreadsheet analysis.

You can use the :POWer:CLResponse:TRACe command to specify whether to include gain, phase, both gain and phase, or neither in the frequency response analysis results.

The SWEep or SINGle option specifies whether to get the data from a sweep or single-frequency analysis (see :POWer:CLResponse:FREQuency:MODE). If this option is not specified, the data from the sweep analysis is returned by default.

Return Format <binary_block><NL>

<binary_block> ::= comma-separated data with newlines at the end of each row

- See Also**
- [":POWer:CLResponse"](#) on page 759
 - [":POWer:CLResponse:APPLy"](#) on page 760
 - [":POWer:CLResponse:DATA:GMARgin"](#) on page 762
 - [":POWer:CLResponse:DATA:GMARgin:FREQuency"](#) on page 763
 - [":POWer:CLResponse:DATA:PMARgin"](#) on page 764
 - [":POWer:CLResponse:DATA:PMARgin:FREQuency"](#) on page 765
 - [":POWer:CLResponse:FREQuency:MODE"](#) on page 766
 - [":POWer:CLResponse:FREQuency:SINGle"](#) on page 767
 - [":POWer:CLResponse:FREQuency:STARt"](#) on page 768
 - [":POWer:CLResponse:FREQuency:STOP"](#) on page 769
 - [":POWer:CLResponse:PPDecade"](#) on page 770
 - [":POWer:CLResponse:SOURce:INPut"](#) on page 771
 - [":POWer:CLResponse:SOURce:OUTPut"](#) on page 772
 - [":POWer:CLResponse:TRACe"](#) on page 773
 - [":POWer:CLResponse:WGEN:LOAD"](#) on page 774
 - [":POWer:CLResponse:WGEN:VOLTage"](#) on page 775
 - [":POWer:CLResponse:WGEN:VOLTage:PROFile"](#) on page 776

:POWer:CLResponse:DATA:GMARgin

N (see [page 1650](#))

Query Syntax :POWer:CLResponse:DATA:GMARgin?

After the Control Loop Response (Bode) power analysis has been performed (see :POWer:CLResponse:APPLY), the :POWer:CLResponse:DATA:GMARgin? query returns the gain margin in dB.

Return Format <gain_margin><NL>

<gain_margin> ::= gain margin in dB in NR3 format.

The query returns +9.9E+37 if the value cannot be calculated from the last sweep (that is, if there are no crossover points).

- See Also**
- [":POWer:CLResponse:APPLY"](#) on page 760
 - [":POWer:CLResponse:DATA"](#) on page 761
 - [":POWer:CLResponse:DATA:GMARgin:FREQuency"](#) on page 763
 - [":POWer:CLResponse:DATA:PMARgin"](#) on page 764
 - [":POWer:CLResponse:DATA:PMARgin:FREQuency"](#) on page 765

:POWer:CLResponse:DATA:GMARgin:FREQuency

N (see [page 1650](#))

Query Syntax :POWer:CLResponse:DATA:GMARgin:FREQuency?

After the Control Loop Response (Bode) power analysis has been performed (see :POWer:CLResponse:APPLy), the :POWer:CLResponse:DATA:GMARgin:FREQuency? query returns the 0° phase crossover frequency in Hz.

Return Format <frequency><NL>

<frequency> ::= 0 degrees phase crossover frequency in Hz in NR3 format

The query returns +9.9E+37 if the value cannot be calculated from the last sweep (that is, if there are no crossover points).

- See Also**
- [":POWer:CLResponse:APPLy"](#) on page 760
 - [":POWer:CLResponse:DATA"](#) on page 761
 - [":POWer:CLResponse:DATA:GMARgin"](#) on page 762
 - [":POWer:CLResponse:DATA:PMARgin"](#) on page 764
 - [":POWer:CLResponse:DATA:PMARgin:FREQuency"](#) on page 765

:POWer:CLResponse:DATA:PMARgin

N (see [page 1650](#))

Query Syntax :POWer:CLResponse:DATA:PMARgin?

After the Control Loop Response (Bode) power analysis has been performed (see :POWer:CLResponse:APPLY), the :POWer:CLResponse:DATA:PMARgin? query returns the phase margin in degrees.

Return Format <phase_margin><NL>

<phase_margin> ::= phase margin in degrees in NR3 format.

The query returns +9.9E+37 if the value cannot be calculated from the last sweep (that is, if there are no crossover points).

- See Also**
- [":POWer:CLResponse:APPLY"](#) on page 760
 - [":POWer:CLResponse:DATA"](#) on page 761
 - [":POWer:CLResponse:DATA:GMARgin"](#) on page 762
 - [":POWer:CLResponse:DATA:GMARgin:FREQuency"](#) on page 763
 - [":POWer:CLResponse:DATA:PMARgin:FREQuency"](#) on page 765

:POWer:CLResponse:DATA:PMARgin:FREQuency

N (see [page 1650](#))

Query Syntax :POWer:CLResponse:DATA:PMARgin:FREQuency?

After the Control Loop Response (Bode) power analysis has been performed (see :POWer:CLResponse:APPLy), the :POWer:CLResponse:DATA:PMARgin:FREQuency? query returns the 0 dB gain crossover frequency in Hz.

Return Format <frequency><NL>

<frequency> ::= 0dB gain crossover frequency in Hz in NR3 format.

The query returns +9.9E+37 if the value cannot be calculated from the last sweep (that is, if there are no crossover points).

- See Also**
- [":POWer:CLResponse:APPLy"](#) on page 760
 - [":POWer:CLResponse:DATA"](#) on page 761
 - [":POWer:CLResponse:DATA:GMARgin"](#) on page 762
 - [":POWer:CLResponse:DATA:GMARgin:FREQuency"](#) on page 763
 - [":POWer:CLResponse:DATA:PMARgin"](#) on page 764

:POWer:CLResponse:FREQuency:MODE

N (see [page 1650](#))

Command Syntax :POWer:CLResponse:FREQuency:MODE <mode>

<mode> ::= {SWEep | SINGle}

The :POWer:CLResponse:FREQuency:MODE command specifies whether the analysis should be performed by sweeping through a range of frequencies (SWEep) or at a single frequency (SINGle).

The SINGle mode is useful for evaluating amplitudes at a single frequency, for example, near the expected 0 dB cross-over frequency. After running the test at a single frequency, you can manually adjust (increase) the waveform generator's amplitude until you begin to observe distortion in the waveforms on the oscilloscope's display. You can then use that amplitude at all frequencies in SWEep mode, or you can evaluate amplitudes at other frequencies in order to determine an optimized amplitude profile (see :POWer:CLResponse:WGEN:VOLTage:PROFile).

Query Syntax :POWer:CLResponse:FREQuency:MODE?

The :POWer:CLResponse:FREQuency:MODE? query returns the frequency mode setting.

Return Format <mode><NL>

<mode> ::= {SWE | SING}

- See Also**
- [":POWer:CLResponse"](#) on page 759
 - [":POWer:CLResponse:APPLY"](#) on page 760
 - [":POWer:CLResponse:DATA"](#) on page 761
 - [":POWer:CLResponse:FREQuency:SINGle"](#) on page 767
 - [":POWer:CLResponse:FREQuency:START"](#) on page 768
 - [":POWer:CLResponse:FREQuency:STOP"](#) on page 769
 - [":POWer:CLResponse:PPDecade"](#) on page 770
 - [":POWer:CLResponse:SOURce:INPut"](#) on page 771
 - [":POWer:CLResponse:SOURce:OUTPut"](#) on page 772
 - [":POWer:CLResponse:WGEN:LOAD"](#) on page 774
 - [":POWer:CLResponse:WGEN:VOLTage"](#) on page 775
 - [":POWer:CLResponse:WGEN:VOLTage:PROFile"](#) on page 776

:POWer:CLResponse:FREQuency:SINGle

N (see [page 1650](#))

Command Syntax :POWer:CLResponse:FREQuency:SINGle <value>[suffix]
 <value> ::= {20 | 100 | 1000 | 10000 | 100000 | 1000000 | 10000000
 | 20000000}
 [suffix] ::= {Hz | kHz | MHz}

The :POWer:CLResponse:FREQuency:SINGle command sets the single frequency value. The control loop response analysis is displayed on a log scale Bode plot, so you can select from decade values in addition to the minimum frequency of 20 Hz.

Query Syntax :POWer:CLResponse:FREQuency:SINGle?

The :POWer:CLResponse:FREQuency:SINGle? query returns the single frequency setting.

Return Format <value><NL>
 <value> ::= {20 | 100 | 1000 | 10000 | 100000 | 1000000 | 10000000
 | 20000000}

- See Also**
- [":POWer:CLResponse"](#) on page 759
 - [":POWer:CLResponse:APPLy"](#) on page 760
 - [":POWer:CLResponse:DATA"](#) on page 761
 - [":POWer:CLResponse:FREQuency:MODE"](#) on page 766
 - [":POWer:CLResponse:PPDecade"](#) on page 770
 - [":POWer:CLResponse:SOURce:INPut"](#) on page 771
 - [":POWer:CLResponse:SOURce:OUTPut"](#) on page 772
 - [":POWer:CLResponse:WGEN:LOAD"](#) on page 774
 - [":POWer:CLResponse:WGEN:VOLTage"](#) on page 775
 - [":POWer:CLResponse:WGEN:VOLTage:PROFile"](#) on page 776

:POWer:CLResponse:FREQuency:START

N (see [page 1650](#))

Command Syntax :POWer:CLResponse:FREQuency:START <value>[suffix]

<value> ::= {20 | 100 | 1000 | 10000 | 100000 | 1000000 | 10000000}

[suffix] ::= {Hz | kHz | MHz}

The :POWer:CLResponse:FREQuency:START command sets the frequency sweep start value. The control loop response analysis is displayed on a log scale Bode plot, so you can select from decade values in addition to the minimum frequency of 20 Hz.

Query Syntax :POWer:CLResponse:FREQuency:START?

The :POWer:CLResponse:FREQuency:START? query returns the frequency sweep start setting.

Return Format <value><NL>

<value> ::= {20 | 100 | 1000 | 10000 | 100000 | 1000000 | 10000000}

- See Also**
- [":POWer:CLResponse"](#) on page 759
 - [":POWer:CLResponse:APPLY"](#) on page 760
 - [":POWer:CLResponse:DATA"](#) on page 761
 - [":POWer:CLResponse:FREQuency:MODE"](#) on page 766
 - [":POWer:CLResponse:FREQuency:STOP"](#) on page 769
 - [":POWer:CLResponse:PPDecade"](#) on page 770
 - [":POWer:CLResponse:SOURce:INPut"](#) on page 771
 - [":POWer:CLResponse:SOURce:OUTPut"](#) on page 772
 - [":POWer:CLResponse:WGEN:LOAD"](#) on page 774
 - [":POWer:CLResponse:WGEN:VOLTage"](#) on page 775
 - [":POWer:CLResponse:WGEN:VOLTage:PROFile"](#) on page 776

:POWer:CLResponse:FREQuency:STOP

N (see [page 1650](#))

Command Syntax :POWer:CLResponse:FREQuency:STOP <value>[suffix]

<value> ::= {100 | 1000 | 10000 | 100000 | 1000000 | 10000000 | 20000000}

[suffix] ::= {Hz | kHz | MHz}

The :POWer:CLResponse:FREQuency:STOP command sets the frequency sweep stop value. The control loop response analysis is displayed on a log scale Bode plot, so you can select from decade values in addition to the maximum frequency of 20 MHz.

Query Syntax :POWer:CLResponse:FREQuency:STOP?

The :POWer:CLResponse:FREQuency:STOP? query returns the frequency sweep stop setting.

Return Format <value><NL>

<value> ::= {100 | 1000 | 10000 | 100000 | 1000000 | 10000000 | 20000000}

- See Also**
- [":POWer:CLResponse"](#) on page 759
 - [":POWer:CLResponse:APPLY"](#) on page 760
 - [":POWer:CLResponse:DATA"](#) on page 761
 - [":POWer:CLResponse:FREQuency:MODE"](#) on page 766
 - [":POWer:CLResponse:FREQuency:START"](#) on page 768
 - [":POWer:CLResponse:PPDecade"](#) on page 770
 - [":POWer:CLResponse:SOURce:INPut"](#) on page 771
 - [":POWer:CLResponse:SOURce:OUTPut"](#) on page 772
 - [":POWer:CLResponse:WGEN:LOAD"](#) on page 774
 - [":POWer:CLResponse:WGEN:VOLTage"](#) on page 775
 - [":POWer:CLResponse:WGEN:VOLTage:PROFile"](#) on page 776

:POWer:CLResponse:PPDecade

N (see [page 1650](#))

Command Syntax :POWer:CLResponse:PPDecade <pts>

<pts> ::= {10 | 20 | 30 | 40 | 50 | 60 | 70 | 80 | 90 | 100}

The :POWer:CLResponse:PPDecade command selects the number of frequency test points per decade (in the log scale).

Query Syntax :POWer:CLResponse:PPDecade?

The :POWer:CLResponse:PPDecade? query returns the points per decade setting.

Return Format <pts><NL>

<pts> ::= {10 | 20 | 30 | 40 | 50 | 60 | 70 | 80 | 90 | 100}

- See Also**
- [":POWer:CLResponse"](#) on page 759
 - [":POWer:CLResponse:APPLY"](#) on page 760
 - [":POWer:CLResponse:DATA"](#) on page 761
 - [":POWer:CLResponse:FREQuency:MODE"](#) on page 766
 - [":POWer:CLResponse:FREQuency:START"](#) on page 768
 - [":POWer:CLResponse:FREQuency:STOP"](#) on page 769
 - [":POWer:CLResponse:SOURce:INPut"](#) on page 771
 - [":POWer:CLResponse:SOURce:OUTPut"](#) on page 772
 - [":POWer:CLResponse:WGEN:LOAD"](#) on page 774
 - [":POWer:CLResponse:WGEN:VOLTage"](#) on page 775
 - [":POWer:CLResponse:WGEN:VOLTage:PROFile"](#) on page 776

:POWer:CLResponse:SOURce:INPut

N (see [page 1650](#))

Command Syntax :POWer:CLResponse:SOURce:INPut <source>

<source> ::= CHANnel<n>

<n> ::= 1 to (# analog channels) in NR1 format

The :POWer:CLResponse:SOURce:INPut command selects the oscilloscope channel that is probing the power supply input.

Query Syntax :POWer:CLResponse:SOURce:INPut?

The :POWer:CLResponse:SOURce:INPut? query returns the channel selection.

Return Format <source><NL>

<source> ::= CHAN<n>

- See Also**
- [":POWer:CLResponse"](#) on page 759
 - [":POWer:CLResponse:APPLY"](#) on page 760
 - [":POWer:CLResponse:DATA"](#) on page 761
 - [":POWer:CLResponse:FREQuency:MODE"](#) on page 766
 - [":POWer:CLResponse:FREQuency:START"](#) on page 768
 - [":POWer:CLResponse:FREQuency:STOP"](#) on page 769
 - [":POWer:CLResponse:PPDecade"](#) on page 770
 - [":POWer:CLResponse:SOURce:OUTPut"](#) on page 772
 - [":POWer:CLResponse:WGEN:LOAD"](#) on page 774
 - [":POWer:CLResponse:WGEN:VOLTage"](#) on page 775
 - [":POWer:CLResponse:WGEN:VOLTage:PROFile"](#) on page 776

:POWer:CLResponse:SOURce:OUTPut

N (see [page 1650](#))

Command Syntax :POWer:CLResponse:SOURce:OUTPut <source>

<source> ::= CHANnel<n>

<n> ::= 1 to (# analog channels) in NR1 format

The :POWer:CLResponse:SOURce:OUTPut command selects the oscilloscope channel that is probing the power supply output.

Query Syntax :POWer:CLResponse:SOURce:OUTPut?

The :POWer:CLResponse:SOURce:OUTPut? query returns the channel selection.

Return Format <source><NL>

<source> ::= CHAN<n>

- See Also**
- [":POWer:CLResponse"](#) on page 759
 - [":POWer:CLResponse:APPLY"](#) on page 760
 - [":POWer:CLResponse:DATA"](#) on page 761
 - [":POWer:CLResponse:FREQuency:MODE"](#) on page 766
 - [":POWer:CLResponse:FREQuency:START"](#) on page 768
 - [":POWer:CLResponse:FREQuency:STOP"](#) on page 769
 - [":POWer:CLResponse:PPDecade"](#) on page 770
 - [":POWer:CLResponse:SOURce:INPut"](#) on page 771
 - [":POWer:CLResponse:WGEN:LOAD"](#) on page 774
 - [":POWer:CLResponse:WGEN:VOLTage"](#) on page 775
 - [":POWer:CLResponse:WGEN:VOLTage:PROFile"](#) on page 776

:POWer:CLResponse:TRACe

N (see [page 1650](#))

Command Syntax :POWer:CLResponse:TRACe <selection>

<selection> ::= {NONE | ALL | GAIN | PHASe}[, {GAIN | PHASe}]

The :POWer:CLResponse:TRACe command specifies whether to include gain, phase, both gain and phase, or neither in the control loop response analysis results.

NOTE

This command affects the oscilloscope's front panel graphical user interface (plot and table) as well as when saving analysis data.

Query Syntax :POWer:CLResponse:TRACe?

The :POWer:CLResponse:TRACe? query returns a comma-separated list of the types of data that are currently included in the control loop response analysis results, or "NONE" if neither gain nor phase data is included.

Return Format <selection_list><NL>

<selection_list> ::= {"NONE" | "GAIN" | "PHASe" | "GAIN, PHASe"}

- See Also**
- [":POWer:CLResponse:APPLy"](#) on page 760
 - [":POWer:CLResponse:DATA"](#) on page 761

:POWer:CLResponse:WGEN:LOAD

N (see [page 1650](#))

Command Syntax :POWer:CLResponse:WGEN:LOAD <impedance>

<impedance> ::= {ONEMeg | FIFTy}

The :POWer:CLResponse:WGEN:LOAD command sets the waveform generator expected output load impedance.

The output impedance of the Gen Out signal is fixed at 50 ohms. However, the output load selection lets the waveform generator display the correct amplitude and offset levels for the expected output load. If the actual load impedance is different than the selected value, the displayed amplitude and offset levels will be incorrect.

Query Syntax :POWer:CLResponse:WGEN:LOAD?

The :POWer:CLResponse:WGEN:LOAD? query returns the waveform generator expected output load impedance setting.

Return Format <impedance><NL>

<impedance> ::= {ONEM | FIFT}

- See Also**
- [":POWer:CLResponse"](#) on page 759
 - [":POWer:CLResponse:APPLY"](#) on page 760
 - [":POWer:CLResponse:DATA"](#) on page 761
 - [":POWer:CLResponse:FREQuency:MODE"](#) on page 766
 - [":POWer:CLResponse:FREQuency:START"](#) on page 768
 - [":POWer:CLResponse:FREQuency:STOP"](#) on page 769
 - [":POWer:CLResponse:PPDecade"](#) on page 770
 - [":POWer:CLResponse:SOURce:INPut"](#) on page 771
 - [":POWer:CLResponse:SOURce:OUTPut"](#) on page 772
 - [":POWer:CLResponse:WGEN:VOLTage"](#) on page 775
 - [":POWer:CLResponse:WGEN:VOLTage:PROFile"](#) on page 776

:POWer:CLResponse:WGEN:VOLTage

N (see [page 1650](#))

Command Syntax :POWer:CLResponse:WGEN:VOLTage <amplitude>[,<range>]

<amplitude> ::= amplitude in volts in NR3 format

<range> ::= {F20HZ | F100HZ | F1KHZ | F10KHZ | F100KHZ | F1MHZ
| F10MHZ | F20MHZ}

The :POWer:CLResponse:WGEN:VOLTage command sets the waveform generator output amplitude(s).

When the waveform generator amplitude profile is enabled (with the :POWer:CLResponse:WGEN:VOLTage:PROFile command), you can set an initial ramp amplitude for each frequency range.

Without the <range> parameter, this command sets the waveform generator output amplitude used when the amplitude profile is disabled.

Query Syntax :POWer:PSRR:WGEN:VOLTage? [<range>]

The :POWer:CLResponse:WGEN:VOLTage? query returns the waveform generator output amplitude setting(s).

Return Format <amplitude><NL>

<amplitude> ::= amplitude in volts in NR3 format

- See Also**
- [":POWer:CLResponse"](#) on page 759
 - [":POWer:CLResponse:APPLY"](#) on page 760
 - [":POWer:CLResponse:DATA"](#) on page 761
 - [":POWer:CLResponse:FREQuency:MODE"](#) on page 766
 - [":POWer:CLResponse:FREQuency:START"](#) on page 768
 - [":POWer:CLResponse:FREQuency:STOP"](#) on page 769
 - [":POWer:CLResponse:PPDecade"](#) on page 770
 - [":POWer:CLResponse:SOURce:INPut"](#) on page 771
 - [":POWer:CLResponse:SOURce:OUTPut"](#) on page 772
 - [":POWer:CLResponse:WGEN:LOAD"](#) on page 774
 - [":POWer:CLResponse:WGEN:VOLTage:PROFile"](#) on page 776

:POWer:CLResponse:WGEN:VOLTage:PROFile

N (see [page 1650](#))

Command Syntax :POWer:CLResponse:WGEN:VOLTage:PROFile {{0 | OFF} | {1 | ON}}

The :POWer:CLResponse:WGEN:VOLTage:PROFile command enables or disables the ability to set initial waveform generator ramp amplitudes for each frequency range.

With amplitude profiling, you can use lower amplitudes at frequencies where the device under test (DUT) is sensitive to distortion and use higher amplitudes where the DUT is less sensitive to distortion. Power supply feedback networks are typically most sensitive near the 0 dB cross-over frequency.

You can often observe distortions during the test. If the input test sine wave begins to look lopsided, clipped, or somewhat triangular in shape (nonsinusoidal), you are probably encountering distortion due to overdriving your DUT. Optimizing test amplitudes to achieve the best dynamic range measurements is often an iterative process of running your frequency response measurements multiple times.

Query Syntax :POWer:CLResponse:WGEN:VOLTage:PROFile?

The :POWer:CLResponse:WGEN:VOLTage:PROFile? query returns the voltage profile setting.

Return Format <setting><NL>
<setting> ::= {0 | 1}

- See Also**
- [":POWer:CLResponse"](#) on page 759
 - [":POWer:CLResponse:APPLY"](#) on page 760
 - [":POWer:CLResponse:DATA"](#) on page 761
 - [":POWer:CLResponse:FREQuency:MODE"](#) on page 766
 - [":POWer:CLResponse:FREQuency:START"](#) on page 768
 - [":POWer:CLResponse:FREQuency:STOP"](#) on page 769
 - [":POWer:CLResponse:PPDecade"](#) on page 770
 - [":POWer:CLResponse:SOURce:INPut"](#) on page 771
 - [":POWer:CLResponse:SOURce:OUTPut"](#) on page 772
 - [":POWer:CLResponse:WGEN:LOAD"](#) on page 774
 - [":POWer:CLResponse:WGEN:VOLTage"](#) on page 775

:POWer:DESKew

N (see [page 1650](#))

Command Syntax :POWer:DESKew

The :POWer:DESKew command launches the auto deskew process on the oscilloscope.

Before sending this command:

- 1 Demagnetize and zero-adjust the current probe.
Refer to the current probe's documentation for instructions on how to do this.
- 2 Make connections to the U1880A deskew fixture as described in the oscilloscope's connection dialog or in the *Power Measurement Application User's Guide*.
- 3 Make sure the voltage probe and current probe channels are specified appropriately using the :POWer:SIGNals:SOURce:VOLTage1 and :POWer:SIGNals:SOURce:CURREnt1 commands.

NOTE

Use the lowest attenuation setting on the high voltage differential probes whenever possible because the voltage levels on the deskew fixture are very small. Using a higher attenuation setting may yield inaccurate skew values (and affect the measurements made) because the noise level is magnified as well.

The deskew values are saved in the oscilloscope until a factory default or secure erase is performed. The next time you run the Power Application, you can use the saved deskew values or perform the deskew again.

Generally, you need to perform the deskew again when part of the test setup changes (for example, a different probe, different oscilloscope channel, etc.) or when the ambient temperature has changed.

- See Also**
- [":POWer:SIGNals:SOURce:VOLTage<i>"](#) on page 840
 - [":POWer:SIGNals:SOURce:CURREnt<i>"](#) on page 839

:POWer:EFFiciency:APPLy

N (see [page 1650](#))

Command Syntax :POWer:EFFiciency:APPLy

The :POWer:EFFiciency:APPLy command applies the efficiency power analysis. Efficiency analysis tests the overall efficiency of the power supply by measuring the output power over the input power.

NOTE

Efficiency analysis requires a 4-channel oscilloscope because input voltage, input current, output voltage, and output current are measured.

-
- See Also
- [":POWer:EFFiciency:TYPE"](#) on page 779
 - [":MEASure:EFFiciency"](#) on page 696
 - [":MEASure:IPOWer"](#) on page 699
 - [":MEASure:OPOWer"](#) on page 702

:POWer:EFFiciency:TYPE

N (see [page 1650](#))

Command Syntax :POWer:EFFiciency:TYPE <type>

<type> ::= {DCDC | DCAC | ACDC | ACAC}

The :POWer:EFFiciency:TYPE command specifies the type of power that is being converted from the input to the output. This selection affects how the efficiency is measured.

Query Syntax :POWer:EFFiciency:TYPE?

The :POWer:EFFiciency:TYPE? query returns the currently specified type setting.

Return Format <type><NL>

<type> ::= {DCDC | DCAC | ACDC | ACAC}

- See Also**
- [":POWer:EFFiciency:APPLY"](#) on page 778
 - [":MEASure:EFFiciency"](#) on page 696
 - [":MEASure:IPOWer"](#) on page 699
 - [":MEASure:OPOWer"](#) on page 702

:POWer:ENABle

N (see [page 1650](#))

Command Syntax :POWer:ENABle {{0 | OFF} | {1 | ON}}

The :POWer:ENABle command enables or disables power analysis.

Query Syntax :POWer:ENABle?

The :POWer:ENABle query returns a 1 or a 0 showing whether power analysis is enabled or disabled, respectively.

Return Format {0 | 1}

See Also • [Chapter 27](#), “:MEASure Power Commands,” starting on page 687

:POWer:HARMonics:APPLy

N (see [page 1650](#))

Command Syntax :POWer:HARMonics:APPLy

The :POWer:HARMonics:APPLy command applies the current harmonics analysis.

Switching power supplies draw a range of harmonics from the AC mains.

Standard limits are set for these harmonics because these harmonics can travel back to the supply grid and cause problems with other devices on the grid.

Use the Current Harmonics analysis to test a switching power supply's current harmonics to pre-compliance standard of IEC61000-3-2 (Class A, B, C, or D). The analysis presents up to 40 harmonics.

- See Also**
- [":POWer:HARMonics:DATA"](#) on page 782
 - [":POWer:HARMonics:DISPlay"](#) on page 783
 - [":POWer:HARMonics:FAILcount"](#) on page 784
 - [":POWer:HARMonics:LINE"](#) on page 785
 - [":POWer:HARMonics:POWerfactor"](#) on page 786
 - [":POWer:HARMonics:STANdard"](#) on page 790
 - [":POWer:HARMonics:STATus"](#) on page 791
 - [":POWer:HARMonics:RUNCount"](#) on page 789
 - [":POWer:HARMonics:THD"](#) on page 792

:POWer:HARMonics:DATA

N (see [page 1650](#))

Query Syntax :POWer:HARMonics:DATA?

The :POWer:HARMonics:DATA query returns the power harmonics results table data.

Return Format <binary_block> ::= comma-separated data with newlines at the end of each row

- See Also**
- [":POWer:HARMonics:APPLy"](#) on page 781
 - [":POWer:HARMonics:DISPlay"](#) on page 783
 - [":POWer:HARMonics:FAILcount"](#) on page 784
 - [":POWer:HARMonics:LINE"](#) on page 785
 - [":POWer:HARMonics:POWerfactor"](#) on page 786
 - [":POWer:HARMonics:RUNCount"](#) on page 789
 - [":POWer:HARMonics:STANdard"](#) on page 790
 - [":POWer:HARMonics:STATus"](#) on page 791
 - [":POWer:HARMonics:THD"](#) on page 792

:POWer:HARMonics:DISPlay

N (see [page 1650](#))

Command Syntax :POWer:HARMonics:DISPlay <display>

<display> ::= {TABLE | BAR | OFF}

The :POWer:HARMonics:DISPlay command specifies how to display the current harmonics analysis results:

- TABLE
- BAR — Bar chart.
- OFF — Harmonics measurement results are not displayed.

Query Syntax :POWer:HARMonics:DISPlay?

The :POWer:HARMonics:DISPlay query returns the display setting.

Return Format <display><NL>

<display> ::= {TABL | BAR | OFF}

- See Also**
- [":POWer:HARMonics:APPLy"](#) on page 781
 - [":POWer:HARMonics:DATA"](#) on page 782
 - [":POWer:HARMonics:FAILcount"](#) on page 784
 - [":POWer:HARMonics:LINE"](#) on page 785
 - [":POWer:HARMonics:POWerfactor"](#) on page 786
 - [":POWer:HARMonics:RUNCount"](#) on page 789
 - [":POWer:HARMonics:STANdard"](#) on page 790
 - [":POWer:HARMonics:STATus"](#) on page 791
 - [":POWer:HARMonics:THD"](#) on page 792

:POWer:HARMonics:FAILcount

N (see [page 1650](#))

Query Syntax :POWer:HARMonics:FAILcount?

Returns the current harmonics analysis' fail count. Non Spec values (that is, harmonics values not specified by the selected standard) are not counted.

Return Format <count><NL>

<count> ::= integer in NR1 format

- See Also**
- [":POWer:HARMonics:RUNCount"](#) on page 789
 - [":POWer:HARMonics:APPLy"](#) on page 781
 - [":POWer:HARMonics:DATA"](#) on page 782
 - [":POWer:HARMonics:DISPlay"](#) on page 783
 - [":POWer:HARMonics:LINE"](#) on page 785
 - [":POWer:HARMonics:POWerfactor"](#) on page 786
 - [":POWer:HARMonics:STANdard"](#) on page 790
 - [":POWer:HARMonics:STATus"](#) on page 791
 - [":POWer:HARMonics:THD"](#) on page 792

:POWer:HARMonics:LINE

N (see [page 1650](#))

Command Syntax :POWer:HARMonics:LINE <frequency>

<frequency> ::= {F50 | F60 | F400 | AUTO}

The :POWer:HARMonics:LINE command specifies the line frequency setting for the current harmonics analysis:

- F50 – 50 Hz.
- F60 – 60 Hz.
- F400 – 400 Hz.
- AUTO – Automatically determines the frequency of the Current waveform.

Query Syntax :POWer:HARMonics:LINE?

The :POWer:HARMonics:LINE query returns the line frequency setting.

Return Format <frequency><NL>

<frequency> ::= {F50 | F60 | F400 | AUTO}

- See Also**
- [":POWer:HARMonics:APPLy"](#) on page 781
 - [":POWer:HARMonics:DATA"](#) on page 782
 - [":POWer:HARMonics:DISPlay"](#) on page 783
 - [":POWer:HARMonics:FAILcount"](#) on page 784
 - [":POWer:HARMonics:POWerfactor"](#) on page 786
 - [":POWer:HARMonics:RUNCount"](#) on page 789
 - [":POWer:HARMonics:STANdard"](#) on page 790
 - [":POWer:HARMonics:STATus"](#) on page 791
 - [":POWer:HARMonics:THD"](#) on page 792

:POWer:HARMonics:POWerfactor

N (see [page 1650](#))

Query Syntax :POWer:HARMonics:POWerfactor?

The :POWer:HARMonics:POWerfactor query returns the power factor for IEC 61000-3-2 Standard Class C power factor value.

Return Format <value> ::= Class C power factor in NR3 format

- See Also**
- [":POWer:HARMonics:APPLY"](#) on page 781
 - [":POWer:HARMonics:DATA"](#) on page 782
 - [":POWer:HARMonics:DISPlay"](#) on page 783
 - [":POWer:HARMonics:FAILcount"](#) on page 784
 - [":POWer:HARMonics:LINE"](#) on page 785
 - [":POWer:HARMonics:RUNCount"](#) on page 789
 - [":POWer:HARMonics:STANdard"](#) on page 790
 - [":POWer:HARMonics:STATus"](#) on page 791
 - [":POWer:HARMonics:THD"](#) on page 792

:POWer:HARMonics:RPOWer

N (see [page 1650](#))

Command Syntax :POWer:HARMonics:RPOWer <source>

<source> ::= {MEASured | USER}

When Class D is selected as the current harmonics analysis standard, the :POWer:HARMonics:RPOWer command specifies whether the Real Power value used for the current-per-watt measurement is measured by the oscilloscope or is defined by the user.

When USER is selected, use the :POWer:HARMonics:RPOWer:USER command to enter the user-defined value.

Query Syntax :POWer:HARMonics:RPOWer?

The :POWer:HARMonics:RPOWer? query returns the Real Power source setting.

Return Format <source><NL>

<source> ::= {MEAS | USER}

- See Also**
- [":POWer:HARMonics:STANdard"](#) on page 790
 - [":POWer:HARMonics:RPOWer:USER"](#) on page 788

:POWer:HARMonics:RPOWer:USER

N (see [page 1650](#))

Command Syntax :POWer:HARMonics:RPOWer:USER <value>

<value> ::= Watts from 1.0 to 600.0 in NR3 format

When Class D is selected as the current harmonics analysis standard and you have chosen to use a user-defined Real Power value (see :POWer:HARMonics:RPOWer), the :POWer:HARMonics:RPOWer:USER command specifies the Real Power value used in the current-per-watt measurement.

Query Syntax :POWer:HARMonics:RPOWer:USER?

The :POWer:HARMonics:RPOWer:USER? query returns the user-defined Real Power value.

Return Format <value><NL>

<value> ::= Watts from 1.0 to 600.0 in NR3 format

- See Also**
- [":POWer:HARMonics:STANdard"](#) on page 790
 - [":POWer:HARMonics:RPOWer"](#) on page 787

:POWer:HARMonics:RUNCount

N (see [page 1650](#))

Query Syntax :POWer:HARMonics:RUNCount?

Returns the current harmonics analysis' run iteration count. Non Spec values (that is, harmonics values not specified by the selected standard) are not counted.

Return Format <count><NL>

<count> ::= integer in NR1 format

- See Also**
- [":POWer:HARMonics:FAILcount"](#) on page 784
 - [":POWer:HARMonics:APPLy"](#) on page 781
 - [":POWer:HARMonics:DATA"](#) on page 782
 - [":POWer:HARMonics:DISPlay"](#) on page 783
 - [":POWer:HARMonics:LINE"](#) on page 785
 - [":POWer:HARMonics:POWerfactor"](#) on page 786
 - [":POWer:HARMonics:STANdard"](#) on page 790
 - [":POWer:HARMonics:STATus"](#) on page 791
 - [":POWer:HARMonics:THD"](#) on page 792

:POWer:HARMonics:STANdard

N (see [page 1650](#))

Command Syntax :POWer:HARMonics:STANdard <class>

<class> ::= {A | B | C | D}

The :POWer:HARMonics:STANdard command selects the standard to perform current harmonics compliance testing on.

- A – IEC 61000-3-2 Class A – for balanced three-phase equipment, household appliances (except equipment identified as Class D), tools excluding portable tools, dimmers for incandescent lamps, and audio equipment.
- B – IEC 61000-3-2 Class B – for portable tools.
- C – IEC 61000-3-2 Class C – for lighting equipment.
- D – IEC 61000-3-2 Class D – for equipment having a specified power according less than or equal to 600 W, of the following types: personal computers and personal computer monitors, television receivers.

Query Syntax :POWer:HARMonics:STANdard?

The :POWer:HARMonics:STANdard query returns the currently set IEC 61000-3-2 standard.

Return Format <class><NL>

<class> ::= {A | B | C | D}

- See Also**
- [":POWer:HARMonics:APPLy"](#) on page 781
 - [":POWer:HARMonics:DATA"](#) on page 782
 - [":POWer:HARMonics:DISPlay"](#) on page 783
 - [":POWer:HARMonics:FAILcount"](#) on page 784
 - [":POWer:HARMonics:LINE"](#) on page 785
 - [":POWer:HARMonics:POWerfactor"](#) on page 786
 - [":POWer:HARMonics:RUNCount"](#) on page 789
 - [":POWer:HARMonics:STATus"](#) on page 791
 - [":POWer:HARMonics:THD"](#) on page 792

:POWer:HARMonics:STATus

N (see [page 1650](#))

Query Syntax :POWer:HARMonics:STATus?

The :POWer:HARMonics:STATus query returns the overall pass/fail status of the current harmonics analysis.

Return Format <status> ::= {PASS | FAIL | UNTested}

- See Also**
- [":POWer:HARMonics:RUNCount"](#) on page 789
 - [":POWer:HARMonics:FAILcount"](#) on page 784
 - [":POWer:HARMonics:APPLY"](#) on page 781
 - [":POWer:HARMonics:DATA"](#) on page 782
 - [":POWer:HARMonics:DISPlay"](#) on page 783
 - [":POWer:HARMonics:LINE"](#) on page 785
 - [":POWer:HARMonics:POWerfactor"](#) on page 786
 - [":POWer:HARMonics:STANdard"](#) on page 790
 - [":POWer:HARMonics:THD"](#) on page 792

:POWer:HARMonics:THD

N (see [page 1650](#))

Query Syntax :POWer:HARMonics:THD?

The :POWer:HARMonics:THD query returns the Total Harmonics Distortion (THD) results of the current harmonics analysis.

Return Format <value> ::= Total Harmonics Distortion in NR3 format

- See Also**
- [":POWer:HARMonics:APPLy"](#) on page 781
 - [":POWer:HARMonics:DATA"](#) on page 782
 - [":POWer:HARMonics:DISPlay"](#) on page 783
 - [":POWer:HARMonics:FAILcount"](#) on page 784
 - [":POWer:HARMonics:LINE"](#) on page 785
 - [":POWer:HARMonics:POWerfactor"](#) on page 786
 - [":POWer:HARMonics:RUNCount"](#) on page 789
 - [":POWer:HARMonics:STANdard"](#) on page 790
 - [":POWer:HARMonics:STATus"](#) on page 791

:POWer:INRush:APPLy

N (see [page 1650](#))

Command Syntax :POWer:INRush:APPLy

The :POWer:INRush:APPLy command applies the inrush current analysis.

The Inrush current analysis measures the peak inrush current of the power supply when the power supply is first turned on.

- See Also**
- [":POWer:ITYPE"](#) on page 796
 - [":POWer:INRush:EXIT"](#) on page 794
 - [":POWer:INRush:NEXT"](#) on page 795
 - [":MEASure:PCURrent"](#) on page 703

:POWer:INRush:EXIT

N (see [page 1650](#))

Command Syntax :POWer:INRush:EXIT

The :POWer:INRush:EXIT command exits (stops) the inrush current power analysis.

This command is equivalent to pressing the **Exit** softkey on the oscilloscope front panel during the analysis.

- See Also**
- [":POWer:INRush:APPLy"](#) on page 793
 - [":POWer:INRush:NEXT"](#) on page 795
 - [":POWer:ITYPE"](#) on page 796

:POWer:INRush:NEXT

N (see [page 1650](#))

Command Syntax :POWer:INRush:NEXT

The :POWer:INRush:NEXT command goes to the next step of the inrush current analysis.

This command is equivalent to pressing the **Next** softkey on the oscilloscope front panel when prompted during the analysis.

- See Also**
- [":POWer:INRush:APPLY"](#) on page 793
 - [":POWer:INRush:EXIT"](#) on page 794
 - [":POWer:ITYPE"](#) on page 796

:POWer:ITYPE

N (see [page 1650](#))

Command Syntax :POWer:ITYPE <type>

<type> ::= {DC | AC}

The :POWer:ITYPE command specifies the type of power that is being converted from the input (DC or AC). Your selection affects how the measurements are made.

This setting is used in the Inrush Current and Turn On/Turn Off tests.

Query Syntax :POWer:ITYPE?

The :POWer:ITYPE? query returns the input power type setting.

Return Format <type><NL>

<type> ::= {DC | AC}

- See Also**
- [":POWer:INRush:APPLY"](#) on page 793
 - [":POWer:ONOff:APPLY"](#) on page 800

:POWer:MODulation:APPLy

N (see [page 1650](#))

Command Syntax :POWer:MODulation:APPLy

The :POWer:MODulation:APPLy command applies the selected modulation analysis type (:POWer:MODulation:TYPE).

The Modulation analysis measures the control pulse signal to a switching device (MOSFET) and observes the trending of the pulse width, duty cycle, period, frequency, etc. of the control pulse signal.

- See Also**
- [":POWer:MODulation:SOURce"](#) on page 798
 - [":POWer:MODulation:TYPE"](#) on page 799
 - [":MEASure:VAverage"](#) on page 675
 - [":MEASure:VRMS"](#) on page 681
 - [":MEASure:VRATio"](#) on page 680
 - [":MEASure:PERiod"](#) on page 649
 - [":MEASure:FREQuency"](#) on page 628
 - [":MEASure:PWIDth"](#) on page 653
 - [":MEASure:NWIDth"](#) on page 645
 - [":MEASure:DUTYcycle"](#) on page 622
 - [":MEASure:RISetime"](#) on page 657
 - [":MEASure:FALLtime"](#) on page 623

:POWer:MODulation:SOURce

N (see [page 1650](#))

Command Syntax :POWer:MODulation:SOURce <source>

<source> ::= {V | I}

The :POWer:MODulation:SOURce command selects either the voltage source or the current source as the source for the modulation analysis.

Query Syntax :POWer:MODulation:SOURce?

The :POWer:MODulation:SOURce query returns the selected source for the modulation analysis.

Return Format <source><NL>

<source> ::= {V | I}

- See Also**
- [":POWer:MODulation:APPLY"](#) on page 797
 - [":POWer:MODulation:TYPE"](#) on page 799

:POWer:MODulation:TYPE

N (see [page 1650](#))

Command Syntax :POWer:MODulation:TYPE <modulation>

```
<modulation> ::= {VAVerage | ACRMs | VRATio | PERiod | FREQuency
                  | PWIDth | NWIDth | DUTYcycle | RISetime | FALLtime}
```

The :POWer:MODulation:TYPE command selects the type of measurement to make in the modulation analysis:

- VAVerage
- ACRMs
- VRATio
- PERiod
- FREQuency
- PWIDth (positive pulse width)
- NWIDth (negative pulse width)
- DUTYcycle
- RISetime
- FALLtime

Query Syntax :POWer:MODulation:TYPE?

The :POWer:MODulation:TYPE query returns the modulation type setting.

Return Format <modulation><NL>

```
<modulation> ::= {VAV | ACRM | VRAT | PER | FREQ | PWID | NWID | DUTY
                  | RIS | FALL}
```

- See Also**
- [":POWer:MODulation:SOURce"](#) on page 798
 - [":POWer:MODulation:APPLY"](#) on page 797
 - [":MEASure:VAVerage"](#) on page 675
 - [":MEASure:VRMS"](#) on page 681
 - [":MEASure:VRATio"](#) on page 680
 - [":MEASure:PERiod"](#) on page 649
 - [":MEASure:FREQuency"](#) on page 628
 - [":MEASure:PWIDth"](#) on page 653
 - [":MEASure:NWIDth"](#) on page 645
 - [":MEASure:DUTYcycle"](#) on page 622
 - [":MEASure:RISetime"](#) on page 657
 - [":MEASure:FALLtime"](#) on page 623

:POWer:ONOFF:APPLy

N (see [page 1650](#))

Command Syntax :POWer:ONOFF:APPLy

The :POWer:ONOFF:APPLy command applies the selected turn on/off analysis test (:POWer:ONOFF:TEST).

- See Also**
- [":POWer:SIGNals:VSTeady:ONOFF:OFF"](#) on page 836
 - [":POWer:SIGNals:VSTeady:ONOFF:ON"](#) on page 837
 - [":POWer:ITYPE"](#) on page 796
 - [":POWer:ONOFF:THResholds"](#) on page 804
 - [":POWer:ONOFF:TEST"](#) on page 803
 - [":POWer:ONOFF:EXIT"](#) on page 801
 - [":POWer:ONOFF:NEXT"](#) on page 802
 - [":MEASure:ONTime"](#) on page 701
 - [":MEASure:OFFTime"](#) on page 700

:POWer:ONOFF:EXIT

N (see [page 1650](#))

Command Syntax :POWer:ONOFF:EXIT

The :POWer:ONOFF:EXIT command exits (stops) the turn on time/turn off time analysis.

This command is equivalent to pressing the **Exit** softkey on the oscilloscope front panel during the analysis.

- See Also**
- [":POWer:ONOFF:THResholds"](#) on page 804
 - [":POWer:ITYPE"](#) on page 796
 - [":POWer:ONOFF:APPLY"](#) on page 800
 - [":POWer:ONOFF:NEXT"](#) on page 802
 - [":POWer:ONOFF:TEST"](#) on page 803

:POWer:ONOFF:NEXT

N (see [page 1650](#))

Command Syntax :POWer:ONOFF:NEXT

The :POWer:ONOFF:NEXT command goes to the next step of the turn on/turn off analysis.

This command is equivalent to pressing the **Next** softkey on the oscilloscope front panel when prompted during the analysis.

- See Also
- [":POWer:ONOFF:THResholds"](#) on page 804
 - [":POWer:ITYPE"](#) on page 796
 - [":POWer:ONOFF:APPLY"](#) on page 800
 - [":POWer:ONOFF:EXIT"](#) on page 801
 - [":POWer:ONOFF:TEST"](#) on page 803

:POWer:ONOFF:TEST

N (see [page 1650](#))

Command Syntax :POWer:ONOFF:TEST {{0 | OFF} | {1 | ON}}

The :POWer:ONOFF:TEST command selects whether turn on or turn off analysis is performed:

- ON – Turn On – measures the time taken to get the output voltage of the power supply after the input voltage is applied.
- OFF – Turn Off – measures the time taken for the output voltage of the power supply to turn off after the input voltage is removed.

Query Syntax :POWer:ONOFF:TEST?

The :POWer:ONOFF:TEST query returns the selected test type.

Return Format {0 | 1}

- See Also**
- [":POWer:ONOFF:THResholds"](#) on page 804
 - [":POWer:ITYPE"](#) on page 796
 - [":POWer:ONOFF:APPLY"](#) on page 800
 - [":POWer:ONOFF:EXIT"](#) on page 801
 - [":POWer:ONOFF:NEXT"](#) on page 802

:POWer:ONOff:THResholds

N (see [page 1650](#))

Command Syntax :POWer:ONOff:THResholds <type>,<input_thr>,<output_thr>

<type> ::= {ON | OFF}

<input_thr> ::= percent from 0-100 in NR1 format

<output_thr> ::= percent from 0-100 in NR1 format

The :POWer:ONOff:THResholds command specifies the input and output thresholds used in the Turn On/Turn Off analysis.

Turn On analysis determines how fast a turned on power supply takes to reach some percent of its steady state output. Turn on time is the time between T2 and T1 where:

- T1 = when the input voltage first rises to some percent (typically the 10% threshold) of its maximum amplitude.
- T2 = when the output DC voltage rises to some percent (typically the 90% threshold) of its maximum amplitude.

Turn Off analysis determines how fast a turned off power supply takes to reduce its output voltage to some percent of maximum. Turn off time is the time between T2 and T1 where:

- T1 = when the input voltage last falls to some percent (typically the 10% threshold) of its maximum amplitude.
- T2 = when the output DC voltage last falls to some percent (typically the 10% threshold) of its maximum amplitude.

Query Syntax :POWer:ONOff:THResholds? <type>

The :POWer:ONOff:THResholds? query returns the input and output threshold settings for the turn on/turn off analysis type.

Return Format <input_thr>,<output_thr><NL>

<input_thr> ::= percent from 0-100 in NR1 format

<output_thr> ::= percent from 0-100 in NR1 format

- See Also**
- [":POWer:SIGNals:VSTeady:ONOff:OFF"](#) on page 836
 - [":POWer:SIGNals:VSTeady:ONOff:ON"](#) on page 837
 - [":POWer:ITYPE"](#) on page 796
 - [":POWer:ONOff:APPLY"](#) on page 800
 - [":POWer:ONOff:TEST"](#) on page 803
 - [":POWer:ONOff:EXIT"](#) on page 801
 - [":POWer:ONOff:NEXT"](#) on page 802

- **":MEASure:ONTime"** on page 701
- **":MEASure:OFFTime"** on page 700

:POWer:PSRR

N (see [page 1650](#))

Query Syntax :POWer:PSRR?

The :POWer:PSRR? query returns the Power Supply Rejection Ratio (PSRR) power analysis settings.

Return Format <settings_string><NL>

For example, the query returns the following string when issued after the *RST command.

```
:POW:PSRR:SOUR:INP CHAN1;OUTP CHAN2;:POW:PSRR:FREQ:STAR +100E+00;
STOP +20.000000E+06;:POW:PSRR:WGEN:VOLT +200.0E-03;LOAD FIFT
```

- See Also**
- [":POWer:PSRR:APPLY"](#) on page 807
 - [":POWer:PSRR:DATA"](#) on page 808
 - [":POWer:PSRR:FREQuency:MAXimum"](#) on page 809
 - [":POWer:PSRR:FREQuency:MINimum"](#) on page 810
 - [":POWer:PSRR:FREQuency:MODE"](#) on page 811
 - [":POWer:PSRR:PPDecade"](#) on page 813
 - [":POWer:PSRR:SOURce:INPut"](#) on page 814
 - [":POWer:PSRR:SOURce:OUTPut"](#) on page 815
 - [":POWer:PSRR:WGEN:LOAD"](#) on page 817
 - [":POWer:PSRR:WGEN:VOLTage"](#) on page 818
 - [":POWer:PSRR:WGEN:VOLTage:PROFile"](#) on page 819

:POWer:PSRR:APPLy

N (see [page 1650](#))

Command Syntax :POWer:PSRR:APPLy

The :POWer:PSRR:APPLy command applies the power supply rejection ratio (PSRR) analysis.

The Power Supply Rejection Ratio (PSRR) test is used to determine how well a voltage regulator rejects ripple noise over different frequency range.

This analysis provides a signal from the oscilloscope's waveform generator that sweeps its frequency. This signal is used to inject ripple to the DC voltage that feeds the voltage regulator.

The AC RMS ratio of the input over the output is measured and is plotted over the range of frequencies.

It takes some time for the frequency sweep analysis to complete. You can query bit 0 of the Standard Event Status Register (*ESR?) to find out when the analysis is complete.

You can use the :POWer:PSRR:TRACe command to specify whether to include gain data in the PSRR analysis results.

- See Also**
- ["*ESR \(Standard Event Status Register\)"](#) on page 218
 - [":POWer:PSRR"](#) on page 806
 - [":POWer:PSRR:DATA"](#) on page 808
 - [":POWer:PSRR:FREQuency:MAXimum"](#) on page 809
 - [":POWer:PSRR:FREQuency:MINimum"](#) on page 810
 - [":POWer:PSRR:FREQuency:MODE"](#) on page 811
 - [":POWer:PSRR:FREQuency:SINGLE"](#) on page 812
 - [":POWer:PSRR:PPDecade"](#) on page 813
 - [":POWer:PSRR:SOURce:INPut"](#) on page 814
 - [":POWer:PSRR:SOURce:OUTPut"](#) on page 815
 - [":POWer:PSRR:TRACe"](#) on page 816
 - [":POWer:PSRR:WGEN:LOAD"](#) on page 817
 - [":POWer:PSRR:WGEN:VOLTage"](#) on page 818
 - [":POWer:PSRR:WGEN:VOLTage:PROFile"](#) on page 819

:POWer:PSRR:DATA

N (see [page 1650](#))

Query Syntax :POWer:PSRR:DATA? [SWEep | SINGLE]

The :POWer:PSRR:DATA? query returns data from the Power Supply Rejection Ratio (PSRR) power analysis.

The comma-separated value format is suitable for spreadsheet analysis.

You can use the :POWer:PSRR:TRACe command to specify whether to include gain data in the PSRR analysis results.

The SWEep or SINGLE option specifies whether to get the data from a sweep or single-frequency analysis (see :POWer:PSRR:FREQuency:MODE). If this option is not specified, the data from the sweep analysis is returned by default.

Return Format <binary_block><NL>

<binary_block> ::= comma-separated data with newlines at the end of each row

- See Also**
- [":POWer:PSRR"](#) on page 806
 - [":POWer:PSRR:APPLy"](#) on page 807
 - [":POWer:PSRR:FREQuency:MAXimum"](#) on page 809
 - [":POWer:PSRR:FREQuency:MINimum"](#) on page 810
 - [":POWer:PSRR:FREQuency:MODE"](#) on page 811
 - [":POWer:PSRR:FREQuency:SINGLE"](#) on page 812
 - [":POWer:PSRR:PPDecade"](#) on page 813
 - [":POWer:PSRR:SOURce:INPut"](#) on page 814
 - [":POWer:PSRR:SOURce:OUTPut"](#) on page 815
 - [":POWer:PSRR:TRACe"](#) on page 816
 - [":POWer:PSRR:WGEN:LOAD"](#) on page 817
 - [":POWer:PSRR:WGEN:VOLTage"](#) on page 818
 - [":POWer:PSRR:WGEN:VOLTage:PROFile"](#) on page 819

:POWer:PSRR:FREQuency:MAXimum

N (see [page 1650](#))

Command Syntax :POWer:PSRR:FREQuency:MAXimum <value>[suffix]

<value> ::= {10 | 100 | 1000 | 10000 | 100000 | 1000000 | 10000000
| 20000000}

[suffix] ::= {Hz | kHz | MHz}

The :POWer:PSRR:FREQuency:MAXimum command sets the end sweep frequency value. The PSRR measurement is displayed on a log scale, so you can select from decade values in addition to the maximum frequency of 20 MHz.

Query Syntax :POWer:PSRR:FREQuency:MAXimum?

The :POWer:PSRR:FREQuency:MAXimum query returns the maximum sweep frequency setting.

Return Format <value><NL>

<value> ::= {10 | 100 | 1000 | 10000 | 100000 | 1000000 | 10000000
| 20000000}

- See Also**
- [":POWer:PSRR"](#) on page 806
 - [":POWer:PSRR:APPLy"](#) on page 807
 - [":POWer:PSRR:DATA"](#) on page 808
 - [":POWer:PSRR:FREQuency:MINimum"](#) on page 810
 - [":POWer:PSRR:FREQuency:MODE"](#) on page 811
 - [":POWer:PSRR:PPDecade"](#) on page 813
 - [":POWer:PSRR:SOURce:INPut"](#) on page 814
 - [":POWer:PSRR:SOURce:OUTPut"](#) on page 815
 - [":POWer:PSRR:WGEN:LOAD"](#) on page 817
 - [":POWer:PSRR:WGEN:VOLTage"](#) on page 818
 - [":POWer:PSRR:WGEN:VOLTage:PROFile"](#) on page 819

:POWer:PSRR:FREQuency:MINimum

N (see [page 1650](#))

Command Syntax	<pre>:POWer:PSRR:FREQuency:MINimum <value>[suffix]</pre> <pre><value> ::= {1 10 100 1000 10000 100000 1000000 10000000}</pre> <pre>[suffix] ::= {Hz kHz MHz}</pre> The :POWer:PSRR:FREQuency:MINimum command sets the start sweep frequency value. The measurement is displayed on a log scale, so you can select from decade values.
Query Syntax	<pre>:POWer:PSRR:FREQuency:MINimum?</pre> The :POWer:PSRR:FREQuency:MINimum query returns the minimum sweep frequency setting.
Return Format	<pre><value><NL></pre> <pre><value> ::= {1 10 100 1000 10000 100000 1000000 10000000}</pre>
See Also	• ":POWer:PSRR" on page 806 • ":POWer:PSRR:APPLy" on page 807 • ":POWer:PSRR:DATA" on page 808 • ":POWer:PSRR:FREQuency:MAXimum" on page 809 • ":POWer:PSRR:FREQuency:MODE" on page 811 • ":POWer:PSRR:PPDecade" on page 813 • ":POWer:PSRR:SOURce:INPut" on page 814 • ":POWer:PSRR:SOURce:OUTPut" on page 815 • ":POWer:PSRR:WGEN:LOAD" on page 817 • ":POWer:PSRR:WGEN:VOLTage" on page 818 • ":POWer:PSRR:WGEN:VOLTage:PROFile" on page 819

:POWer:PSRR:FREQuency:MODE

N (see [page 1650](#))

Command Syntax :POWer:PSRR:FREQuency:MODE <mode>

<mode> ::= {SWEep | SINGle}

The :POWer:PSRR:FREQuency:MODE command specifies whether the analysis should be performed by sweeping through a range of frequencies (SWEep) or at a single frequency (SINGle).

The SINGle mode is useful for evaluating amplitudes at a single frequency. After running the test at a single frequency, you can manually adjust (increase) the waveform generator's amplitude until you begin to observe distortion in the waveforms on the oscilloscope's display. You can then use that amplitude at all frequencies in SWEep mode, or you can evaluate amplitudes at other frequencies in order to determine an optimized amplitude profile (see :POWer:PSRR:WGEN:VOLTage:PROFile).

Query Syntax :POWer:CLResponse:FREQuency:MODE?

The :POWer:PSRR:FREQuency:MODE? query returns the frequency mode setting.

Return Format <mode><NL>

<mode> ::= {SWE | SING}

- See Also**
- [":POWer:PSRR"](#) on page 806
 - [":POWer:PSRR:APPLy"](#) on page 807
 - [":POWer:PSRR:DATA"](#) on page 808
 - [":POWer:PSRR:FREQuency:MAXimum"](#) on page 809
 - [":POWer:PSRR:FREQuency:MINimum"](#) on page 810
 - [":POWer:PSRR:FREQuency:SINGle"](#) on page 812
 - [":POWer:PSRR:PPDecade"](#) on page 813
 - [":POWer:PSRR:SOURce:INPut"](#) on page 814
 - [":POWer:PSRR:SOURce:OUTPut"](#) on page 815
 - [":POWer:PSRR:WGEN:LOAD"](#) on page 817
 - [":POWer:PSRR:WGEN:VOLTage"](#) on page 818
 - [":POWer:PSRR:WGEN:VOLTage:PROFile"](#) on page 819

:POWer:PSRR:FREQuency:SINGle

N (see [page 1650](#))

Command Syntax :POWer:PSRR:FREQuency:SINGle <value>[suffix]

<value> ::= {1 | 10 | 100 | 1000 | 10000 | 100000 | 1000000 | 10000000
| 2000000}

[suffix] ::= {Hz | kHz | MHz}

The :POWer:PSRR:FREQuency:SINGle command sets the single frequency value. The measurement is displayed on a log scale, so you can select from decade values.

Query Syntax :POWer:PSRR:FREQuency:SINGle?

The :POWer:PSRR:FREQuency:SINGle query returns the single frequency setting.

Return Format <value><NL>

<value> ::= {1 | 10 | 100 | 1000 | 10000 | 100000 | 1000000 | 10000000
| 2000000}

- See Also**
- [":POWer:PSRR"](#) on page 806
 - [":POWer:PSRR:APPLy"](#) on page 807
 - [":POWer:PSRR:DATA"](#) on page 808
 - [":POWer:PSRR:FREQuency:MODE"](#) on page 811
 - [":POWer:PSRR:PPDecade"](#) on page 813
 - [":POWer:PSRR:SOURce:INPut"](#) on page 814
 - [":POWer:PSRR:SOURce:OUTPut"](#) on page 815
 - [":POWer:PSRR:WGEN:LOAD"](#) on page 817
 - [":POWer:PSRR:WGEN:VOLTage"](#) on page 818
 - [":POWer:PSRR:WGEN:VOLTage:PROFile"](#) on page 819

:POWer:PSRR:PPDecade

N (see [page 1650](#))

Command Syntax :POWer:PSRR:PPDecade <pts>

<pts> ::= {10 | 20 | 30 | 40 | 50 | 60 | 70 | 80 | 90 | 100}

The :POWer:PSRR:PPDecade command selects the number of frequency test points per decade (in the log scale).

Query Syntax :POWer:CLResponse:PPDecade?

The :POWer:PSRR:PPDecade? query returns the points per decade setting.

Return Format <pts><NL>

<pts> ::= {10 | 20 | 30 | 40 | 50 | 60 | 70 | 80 | 90 | 100}

- See Also**
- [":POWer:PSRR"](#) on page 806
 - [":POWer:PSRR:APPLy"](#) on page 807
 - [":POWer:PSRR:DATA"](#) on page 808
 - [":POWer:PSRR:FREQuency:MAXimum"](#) on page 809
 - [":POWer:PSRR:FREQuency:MINimum"](#) on page 810
 - [":POWer:PSRR:FREQuency:MODE"](#) on page 811
 - [":POWer:PSRR:SOURce:INPut"](#) on page 814
 - [":POWer:PSRR:SOURce:OUTPut"](#) on page 815
 - [":POWer:PSRR:WGEN:LOAD"](#) on page 817
 - [":POWer:PSRR:WGEN:VOLTage"](#) on page 818
 - [":POWer:PSRR:WGEN:VOLTage:PROFile"](#) on page 819

:POWer:PSRR:SOURce:INPut

N (see [page 1650](#))

Command Syntax :POWer:PSRR:SOURce:INPut <source>

<source> ::= CHANnel<n>

<n> ::= 1 to (# analog channels) in NR1 format

The :POWer:PSRR:SOURce:INPut command selects the oscilloscope channel that is probing the power supply input.

Query Syntax :POWer:PSRR:SOURce:INPut?

The :POWer:PSRR:SOURce:INPut? query returns the channel selection.

Return Format <source><NL>

<source> ::= CHAN<n>

- See Also**
- [":POWer:PSRR"](#) on page 806
 - [":POWer:PSRR:APPLy"](#) on page 807
 - [":POWer:PSRR:DATA"](#) on page 808
 - [":POWer:PSRR:FREQuency:MAXimum"](#) on page 809
 - [":POWer:PSRR:FREQuency:MINimum"](#) on page 810
 - [":POWer:PSRR:FREQuency:MODE"](#) on page 811
 - [":POWer:PSRR:PPDecade"](#) on page 813
 - [":POWer:PSRR:SOURce:OUTPut"](#) on page 815
 - [":POWer:PSRR:WGEN:LOAD"](#) on page 817
 - [":POWer:PSRR:WGEN:VOLTage"](#) on page 818
 - [":POWer:PSRR:WGEN:VOLTage:PROFile"](#) on page 819

:POWer:PSRR:SOURce:OUTPut

N (see [page 1650](#))

Command Syntax :POWer:PSRR:SOURce:OUTPut <source>

<source> ::= CHANnel<n>

<n> ::= 1 to (# analog channels) in NR1 format

The :POWer:PSRR:SOURce:OUTPut command selects the oscilloscope channel that is probing the power supply output.

Query Syntax :POWer:PSRR:SOURce:OUTPut?

The :POWer:PSRR:SOURce:OUTPut? query returns the channel selection.

Return Format <source><NL>

<source> ::= CHAN<n>

- See Also**
- [":POWer:PSRR"](#) on page 806
 - [":POWer:PSRR:APPLy"](#) on page 807
 - [":POWer:PSRR:DATA"](#) on page 808
 - [":POWer:PSRR:FREQuency:MAXimum"](#) on page 809
 - [":POWer:PSRR:FREQuency:MINimum"](#) on page 810
 - [":POWer:PSRR:FREQuency:MODE"](#) on page 811
 - [":POWer:PSRR:PPDecade"](#) on page 813
 - [":POWer:PSRR:SOURce:INPut"](#) on page 814
 - [":POWer:PSRR:WGEN:LOAD"](#) on page 817
 - [":POWer:PSRR:WGEN:VOLTage"](#) on page 818
 - [":POWer:PSRR:WGEN:VOLTage:PROFile"](#) on page 819

:POWer:PSRR:TRACe

N (see [page 1650](#))

Command Syntax :POWer:PSRR:TRACe <selection>

<selection> ::= {NONE | GAIN}

The :POWer:PSRR:TRACe command specifies whether to include gain (PSRR) data in the power supply rejection ratio analysis results.

NOTE

This command affects the oscilloscope's front panel graphical user interface (plot and table) as well as when saving analysis data.

Query Syntax :POWer:PSRR:TRACe?

The :POWer:PSRR:TRACe? query returns the type of data that is currently included in the PSRR analysis results, or "NONE" if the gain data is not included.

Return Format <selection_list><NL>

<selection_list> ::= {"NONE" | "GAIN"}

- See Also**
- [":POWer:PSRR:APPLy"](#) on page 807
 - [":POWer:PSRR:DATA"](#) on page 808

:POWer:PSRR:WGEN:LOAD

N (see [page 1650](#))

Command Syntax :POWer:PSRR:WGEN:LOAD <impedance>

<impedance> ::= {ONEMeg | FIFTy}

The :POWer:PSRR:WGEN:LOAD command sets the waveform generator expected output load impedance.

The output impedance of the Gen Out signal is fixed at 50 ohms. However, the output load selection lets the waveform generator display the correct amplitude and offset levels for the expected output load. If the actual load impedance is different than the selected value, the displayed amplitude and offset levels will be incorrect.

Query Syntax :POWer:PSRR:WGEN:LOAD?

The :POWer:PSRR:WGEN:LOAD? query returns the waveform generator expected output load impedance setting.

Return Format <impedance><NL>

<impedance> ::= {ONEM | FIFT}

- See Also**
- [":POWer:PSRR"](#) on page 806
 - [":POWer:PSRR:APPLy"](#) on page 807
 - [":POWer:PSRR:DATA"](#) on page 808
 - [":POWer:PSRR:FREQuency:MAXimum"](#) on page 809
 - [":POWer:PSRR:FREQuency:MINimum"](#) on page 810
 - [":POWer:PSRR:FREQuency:MODE"](#) on page 811
 - [":POWer:PSRR:PPDecade"](#) on page 813
 - [":POWer:PSRR:SOURce:INPut"](#) on page 814
 - [":POWer:PSRR:SOURce:OUTPut"](#) on page 815
 - [":POWer:PSRR:WGEN:VOLTage"](#) on page 818
 - [":POWer:PSRR:WGEN:VOLTage:PROFile"](#) on page 819

:POWer:PSRR:WGEN:VOLTage

N (see [page 1650](#))

Command Syntax :POWer:PSRR:WGEN:VOLTage <amplitude>[,<range>]

<amplitude> ::= amplitude in volts in NR3 format

<range> ::= {F20HZ | F100HZ | F1KHZ | F10KHZ | F100KHZ | F1MHZ
| F10MHZ | F20MHZ}

The :POWer:PSRR:WGEN:VOLTage command sets the waveform generator output amplitude(s).

When the waveform generator amplitude profile is enabled (with the :POWer:PSRR:WGEN:VOLTage:PROFile command), you can set an initial ramp amplitude for each frequency range.

Without the <range> parameter, this command sets the waveform generator output amplitude used when the amplitude profile is disabled.

Query Syntax :POWer:PSRR:WGEN:VOLTage? [<range>]

The :POWer:PSRR:WGEN:VOLTage? query returns the waveform generator output amplitude setting(s).

Return Format <amplitude><NL>

<amplitude> ::= amplitude in volts in NR3 format

- See Also**
- [":POWer:PSRR"](#) on page 806
 - [":POWer:PSRR:APPLy"](#) on page 807
 - [":POWer:PSRR:DATA"](#) on page 808
 - [":POWer:PSRR:FREQuency:MAXimum"](#) on page 809
 - [":POWer:PSRR:FREQuency:MINimum"](#) on page 810
 - [":POWer:PSRR:FREQuency:MODE"](#) on page 811
 - [":POWer:PSRR:PPDecade"](#) on page 813
 - [":POWer:PSRR:SOURce:INPut"](#) on page 814
 - [":POWer:PSRR:SOURce:OUTPut"](#) on page 815
 - [":POWer:PSRR:WGEN:LOAD"](#) on page 817
 - [":POWer:PSRR:WGEN:VOLTage:PROFile"](#) on page 819

:POWer:PSRR:WGEN:VOLTage:PROFile

N (see [page 1650](#))

Command Syntax :POWer:PSRR:WGEN:VOLTage:PROFile {{0 | OFF} | {1 | ON}}

The :POWer:PSRR:WGEN:VOLTage:PROFile command enables or disables the ability to set initial waveform generator ramp amplitudes for each frequency range.

With amplitude profiling, you can use lower amplitudes at frequencies where the device under test (DUT) is sensitive to distortion and use higher amplitudes where the DUT is less sensitive to distortion.

You can often observe distortions during the test. If the input test sine wave begins to look lopsided, clipped, or somewhat triangular in shape (nonsinusoidal), you are probably encountering distortion due to overdriving your DUT. Optimizing test amplitudes to achieve the best dynamic range measurements is often an iterative process of running your frequency response measurements multiple times.

Query Syntax :POWer:CLResponse:WGEN:VOLTage:PROFile?

The :POWer:PSRR:WGEN:VOLTage:PROFile? query returns the voltage profile setting.

Return Format <setting><NL>
<setting> ::= {0 | 1}

- See Also**
- [":POWer:PSRR"](#) on page 806
 - [":POWer:PSRR:APPLy"](#) on page 807
 - [":POWer:PSRR:DATA"](#) on page 808
 - [":POWer:PSRR:FREQuency:MAXimum"](#) on page 809
 - [":POWer:PSRR:FREQuency:MINimum"](#) on page 810
 - [":POWer:PSRR:FREQuency:MODE"](#) on page 811
 - [":POWer:PSRR:PPDecade"](#) on page 813
 - [":POWer:PSRR:SOURce:INPut"](#) on page 814
 - [":POWer:PSRR:SOURce:OUTPut"](#) on page 815
 - [":POWer:PSRR:WGEN:LOAD"](#) on page 817
 - [":POWer:PSRR:WGEN:VOLTage"](#) on page 818

:POWer:QUALity:APPLy

N (see [page 1650](#))

Command Syntax :POWer:QUALity:APPLy

The :POWer:QUALity:APPLy command applies the selected power quality analysis type (:POWer:QUALity:TYPE).

The power quality analysis shows the quality of the AC input line.

Some AC current may flow back into and back out of the load without delivering energy. This current, called reactive or harmonic current, gives rise to an "apparent" power which is larger than the actual power consumed. Power quality is gauged by these measurements: power factor, apparent power, true power, reactive power, crest factor, and phase angle of the current and voltage of the AC line.

- See Also
- [":MEASure:FACTor"](#) on page 698
 - [":MEASure:REAL"](#) on page 707
 - [":MEASure:APParent"](#) on page 693
 - [":MEASure:REACTive"](#) on page 706
 - [":MEASure:CRESt"](#) on page 695
 - [":MEASure:ANGLE"](#) on page 692

:POWer:RIPPlE:APPLy

N (see [page 1650](#))

Command Syntax :POWer:RIPPlE:APPLy

The :POWer:RIPPlE:APPLy command applies the output ripple analysis.

See Also • [":MEASure:RIPPlE"](#) on page 708

:POWer:SIGNals:AUTosetup

N (see [page 1650](#))

Command Syntax :POWer:SIGNals:AUTosetup <analysis>

```
<analysis> ::= {HARMonics | EFFiciency | RIPPlE | MODulation | QUALity  
               | SLEW | SWITCh | RDSVce}
```

The :POWer:SIGNals:AUTosetup command performs automated oscilloscope setup for the signals in the specified type of power analysis.

- See Also**
- [":POWer:HARMonics:DISPlay"](#) on page 783
 - [":POWer:EFFiciency:APPLy"](#) on page 778
 - [":POWer:RIPPlE:APPLy"](#) on page 821
 - [":POWer:MODulation:APPLy"](#) on page 797
 - [":POWer:QUALity:APPLy"](#) on page 820
 - [":POWer:SLEW:APPLy"](#) on page 841
 - [":POWer:SWITCh:APPLy"](#) on page 843
 - [":POWer:SIGNals:CYCLes:HARMonics"](#) on page 823
 - [":POWer:SIGNals:CYCLes:QUALity"](#) on page 824
 - [":POWer:SIGNals:DURation:EFFiciency"](#) on page 825
 - [":POWer:SIGNals:DURation:MODulation"](#) on page 826
 - [":POWer:SIGNals:DURation:RIPPlE"](#) on page 829
 - [":POWer:SIGNals:IEXpected"](#) on page 831
 - [":POWer:SIGNals:OVERshoot"](#) on page 832
 - [":POWer:SIGNals:SOURce:CURRENT<i>"](#) on page 839
 - [":POWer:SIGNals:SOURce:VOLTage<i>"](#) on page 840

:POWer:SIGNals:CYCLes:HARMonics

N (see [page 1650](#))

Command Syntax :POWer:SIGNals:CYCLes:HARMonics <count>

<count> ::= integer in NR1 format

Legal values are 1 to 100.

The :POWer:SIGNals:CYCLes:HARMonics command specifies the number of cycles to include in the current harmonics analysis.

Query Syntax :POWer:SIGNals:CYCLes:HARMonics?

The :POWer:SIGNals:CYCLes:HARMonics query returns the number of cycles currently set.

Return Format <count><NL>

<count> ::= integer in NR1 format

- See Also**
- [":POWer:HARMonics:DISPlay"](#) on page 783
 - [":POWer:HARMonics:APPLy"](#) on page 781
 - [":POWer:SIGNals:AUTosetup"](#) on page 822
 - [":POWer:SIGNals:IEXPeCted"](#) on page 831
 - [":POWer:SIGNals:OVERshoot"](#) on page 832
 - [":POWer:SIGNals:SOURce:CURRent<i>"](#) on page 839
 - [":POWer:SIGNals:SOURce:VOLTage<i>"](#) on page 840

:POWer:SIGNals:CYCLes:QUALity

N (see [page 1650](#))

Command Syntax :POWer:SIGNals:CYCLes:QUALity <count>

<count> ::= integer in NR1 format

Legal values are 1 to 100.

The :POWer:SIGNals:CYCLes:QUALity command specifies the number of cycles to include in the power quality analysis.

Query Syntax :POWer:SIGNals:CYCLes:QUALity?

The :POWer:SIGNals:CYCLes:QUALity query returns the number of cycles currently set.

Return Format <count><NL>

<count> ::= integer in NR1 format

- See Also**
- [":POWer:QUALity:APPLy"](#) on page 820
 - [":POWer:SIGNals:AUTosetup"](#) on page 822
 - [":POWer:SIGNals:IEXPeCted"](#) on page 831
 - [":POWer:SIGNals:OVERshoot"](#) on page 832
 - [":POWer:SIGNals:SOURce:CURRent<i>"](#) on page 839
 - [":POWer:SIGNals:SOURce:VOLTage<i>"](#) on page 840

:POWer:SIGNals:DURation:EFFiciency

N (see [page 1650](#))

Command Syntax :POWer:SIGNals:DURation:EFFiciency <value>[suffix]

<value> ::= value in NR3 format

[suffix] ::= {s | ms | us | ns}

The :POWer:SIGNals:DURation:EFFiciency command specifies the duration of the efficiency analysis.

Query Syntax :POWer:SIGNals:DURation:EFFiciency?

The :POWer:SIGNals:DURation:EFFiciency query returns the set duration time value.

Return Format <value><NL>

<value> ::= value in NR3 format

- See Also**
- [":POWer:EFFiciency:APPLy"](#) on page 778
 - [":POWer:SIGNals:AUTosetup"](#) on page 822
 - [":POWer:SIGNals:IEXPeCted"](#) on page 831
 - [":POWer:SIGNals:OVERshoot"](#) on page 832
 - [":POWer:SIGNals:SOURce:CURREnt<i>"](#) on page 839
 - [":POWer:SIGNals:SOURce:VOLTage<i>"](#) on page 840

:POWer:SIGNals:DURation:MODulation

N (see [page 1650](#))

Command Syntax :POWer:SIGNals:DURation:MODulation <value>[suffix]

<value> ::= value in NR3 format

[suffix] ::= {s | ms | us | ns}

The :POWer:SIGNals:DURation:MODulation command specifies the duration of the modulation analysis.

Query Syntax :POWer:SIGNals:DURation:MODulation?

The :POWer:SIGNals:DURation:MODulation query returns the set duration time value.

Return Format <value><NL>

<value> ::= value in NR3 format

- See Also**
- [":POWer:MODulation:APPLy"](#) on page 797
 - [":POWer:SIGNals:AUTosetup"](#) on page 822
 - [":POWer:SIGNals:IEXpected"](#) on page 831
 - [":POWer:SIGNals:OVERshoot"](#) on page 832
 - [":POWer:SIGNals:SOURce:CURREnt<i>"](#) on page 839
 - [":POWer:SIGNals:SOURce:VOLTage<i>"](#) on page 840

:POWer:SIGNals:DURation:ONOff:OFF

N (see [page 1650](#))

Command Syntax :POWer:SIGNals:DURation:ONOff:OFF <value>[suffix]

<value> ::= value in NR3 format

[suffix] ::= {s | ms | us | ns}

The :POWer:SIGNals:DURation:ONOff:OFF command specifies the duration of the turn off analysis.

Query Syntax :POWer:SIGNals:DURation:ONOff:OFF?

The :POWer:SIGNals:DURation:ONOff:OFF query returns the set duration time value.

Return Format <value><NL>

<value> ::= value in NR3 format

- See Also**
- [":POWer:ONOff:APPLy"](#) on page 800
 - [":POWer:SIGNals:AUTosetup"](#) on page 822
 - [":POWer:SIGNals:IEXPeCted"](#) on page 831
 - [":POWer:SIGNals:OVERshoot"](#) on page 832
 - [":POWer:SIGNals:VMAXimum:ONOff:OFF"](#) on page 834
 - [":POWer:SIGNals:VSTeady:ONOff:OFF"](#) on page 836
 - [":POWer:SIGNals:SOURce:CURREnt<i>"](#) on page 839
 - [":POWer:SIGNals:SOURce:VOLTage<i>"](#) on page 840

:POWer:SIGNals:DURation:ONOff:ON

N (see [page 1650](#))

Command Syntax :POWer:SIGNals:DURation:ONOff:ON <value>[suffix]

<value> ::= value in NR3 format

[suffix] ::= {s | ms | us | ns}

The :POWer:SIGNals:DURation:ONOff:ON command specifies the duration of the turn on analysis.

Query Syntax :POWer:SIGNals:DURation:ONOff:ON?

The :POWer:SIGNals:DURation:ONOff:ON query returns the set duration time value.

Return Format <value><NL>

<value> ::= value in NR3 format

- See Also**
- [":POWer:ONOff:APPLy"](#) on page 800
 - [":POWer:SIGNals:AUTosetup"](#) on page 822
 - [":POWer:SIGNals:IEXPeCted"](#) on page 831
 - [":POWer:SIGNals:OVERshoot"](#) on page 832
 - [":POWer:SIGNals:VMAXimum:ONOff:ON"](#) on page 835
 - [":POWer:SIGNals:VSTeady:ONOff:ON"](#) on page 837
 - [":POWer:SIGNals:SOURce:CURREnt<i>"](#) on page 839
 - [":POWer:SIGNals:SOURce:VOLTage<i>"](#) on page 840

:POWer:SIGNals:DURation:RIPPlE

N (see [page 1650](#))

Command Syntax :POWer:SIGNals:DURation:RIPPlE <value>[suffix]

<value> ::= value in NR3 format

[suffix] ::= {s | ms | us | ns}

The :POWer:SIGNals:DURation:RIPPlE command specifies the duration of the output ripple analysis.

Query Syntax :POWer:SIGNals:DURation:RIPPlE?

The :POWer:SIGNals:DURation:RIPPlE query returns the set duration time value.

Return Format <value><NL>

<value> ::= value in NR3 format

- See Also**
- [":POWer:RIPPlE:APPLy"](#) on page 821
 - [":POWer:SIGNals:AUTosetup"](#) on page 822
 - [":POWer:SIGNals:IEXPeCted"](#) on page 831
 - [":POWer:SIGNals:OVERshoot"](#) on page 832
 - [":POWer:SIGNals:SOURce:CURREnt<i>"](#) on page 839
 - [":POWer:SIGNals:SOURce:VOLTage<i>"](#) on page 840

:POWer:SIGNals:DURation:TRANsient

N (see [page 1650](#))

Command Syntax :POWer:SIGNals:DURation:TRANsient <value>[suffix]

<value> ::= value in NR3 format

[suffix] ::= {s | ms | us | ns}

The :POWer:SIGNals:DURation:TRANsient command specifies the duration of the transient response analysis.

Query Syntax :POWer:SIGNals:DURation:TRANsient?

The :POWer:SIGNals:DURation:TRANsient query returns the set duration time value.

Return Format <value><NL>

<value> ::= value in NR3 format

- See Also**
- [":POWer:TRANsient:APPLy"](#) on page 849
 - [":POWer:SIGNals:AUTosetup"](#) on page 822
 - [":POWer:SIGNals:IEXPeCted"](#) on page 831
 - [":POWer:SIGNals:OVERshoot"](#) on page 832
 - [":POWer:SIGNals:VSTeady:TRANsient"](#) on page 838
 - [":POWer:SIGNals:SOURce:CURREnt<i>"](#) on page 839
 - [":POWer:SIGNals:SOURce:VOLTage<i>"](#) on page 840

:POWer:SIGNals:IEXPected

N (see [page 1650](#))

Command Syntax :POWer:SIGNals:IEXPected <value>[suffix]
 <value> ::= Expected current value in NR3 format
 [suffix] ::= {A | mA}

The :POWer:SIGNals:IEXPected command specifies the expected inrush current amplitude. This value is used to set the vertical scale of the channel probing current.

Query Syntax :POWer:SIGNals:IEXPected?

The :POWer:SIGNals:IEXPected query returns the expected inrush current setting.

Return Format <value><NL>
 <value> ::= Expected current value in NR3 format

- See Also**
- [":POWer:INRush:APPLY"](#) on page 793
 - [":POWer:SIGNals:AUTosetup"](#) on page 822
 - [":POWer:SIGNals:OVERshoot"](#) on page 832
 - [":POWer:SIGNals:VMAXimum:INRush"](#) on page 833
 - [":POWer:SIGNals:SOURce:CURRent<i>"](#) on page 839
 - [":POWer:SIGNals:SOURce:VOLTage<i>"](#) on page 840

:POWer:SIGNals:OVERshoot

N (see [page 1650](#))

Command Syntax :POWer:SIGNals:OVERshoot <percent>

<percent> ::= percent of overshoot value in NR1 format

The :POWer:SIGNals:OVERshoot command specifies the percent of overshoot of the output voltage. This value is used to determine the settling band value for the transient response and to adjust the vertical scale of the oscilloscope.

Query Syntax :POWer:SIGNals:OVERshoot?

The :POWer:SIGNals:OVERshoot query returns the overshoot percent setting.

Return Format <percent><NL>

<percent> ::= percent of overshoot value in NR1 format

- See Also**
- [":POWer:TRANsient:APPLy"](#) on page 849
 - [":POWer:SIGNals:AUTosetup"](#) on page 822
 - [":POWer:SIGNals:DURation:TRANsient"](#) on page 830
 - [":POWer:SIGNals:IEXPected"](#) on page 831
 - [":POWer:SIGNals:VSTeady:TRANsient"](#) on page 838
 - [":POWer:SIGNals:SOURce:CURREnt<i>"](#) on page 839
 - [":POWer:SIGNals:SOURce:VOLTage<i>"](#) on page 840

:POWer:SIGNals:VMAximum:INRush

N (see [page 1650](#))

Command Syntax :POWer:SIGNals:VMAximum:INRush <value>[suffix]

<value> ::= Maximum expected input Voltage in NR3 format

[suffix] ::= {V | mV}

The :POWer:SIGNals:VMAximum:INRush command specifies the maximum expected input voltage. This value is used to set the vertical scale of the channel probing voltage for inrush current analysis.

When the :POWer:ITYPE is DC, this command defines the maximum DC input voltage amplitude value. The values can be negative.

When the :POWer:ITYPE is AC, this command defines the maximum peak-to-peak input voltage. Only positive values are allowed.

Query Syntax :POWer:SIGNals:VMAximum:INRush?

The :POWer:SIGNals:VMAximum:INRush query returns the expected maximum input voltage setting.

Return Format <value><NL>

<value> ::= Maximum expected input Voltage in NR3 format

- See Also**
- [":POWer:ITYPE"](#) on page 796
 - [":POWer:INRush:APPLY"](#) on page 793
 - [":POWer:SIGNals:AUTosetup"](#) on page 822
 - [":POWer:SIGNals:IEXpected"](#) on page 831
 - [":POWer:SIGNals:OVERshoot"](#) on page 832
 - [":POWer:SIGNals:SOURce:CURREnt<i>"](#) on page 839
 - [":POWer:SIGNals:SOURce:VOLTage<i>"](#) on page 840

:POWer:SIGNals:VMAXimum:ONOff:OFF

N (see [page 1650](#))

Command Syntax :POWer:SIGNals:VMAXimum:ONOff:OFF <value>[suffix]

<value> ::= Maximum expected input Voltage in NR3 format

[suffix] ::= {V | mV}

The :POWer:SIGNals:VMAXimum:ONOff:OFF command specifies the maximum expected input voltage. This value is used to set the vertical scale of the channel probing voltage for turn off analysis.

When the :POWer:ITYPE is DC, this command defines the maximum DC input voltage amplitude value. The values can be negative.

When the :POWer:ITYPE is AC, this command defines the maximum peak-to-peak input voltage. Only positive values are allowed.

Query Syntax :POWer:SIGNals:VMAXimum:ONOff:OFF?

The :POWer:SIGNals:VMAXimum:ONOff:OFF query returns the expected maximum input voltage setting.

Return Format <value><NL>

<value> ::= Maximum expected input Voltage in NR3 format

- See Also**
- [":POWer:ITYPE"](#) on page 796
 - [":POWer:ONOff:APPLY"](#) on page 800
 - [":POWer:SIGNals:AUTosetup"](#) on page 822
 - [":POWer:SIGNals:DURation:ONOff:OFF"](#) on page 827
 - [":POWer:SIGNals:IEXpected"](#) on page 831
 - [":POWer:SIGNals:OVERshoot"](#) on page 832
 - [":POWer:SIGNals:VSTeady:ONOff:OFF"](#) on page 836
 - [":POWer:SIGNals:SOURce:CURREnt<i>"](#) on page 839
 - [":POWer:SIGNals:SOURce:VOLTage<i>"](#) on page 840

:POWer:SIGNals:VMAXimum:ONOff:ON

N (see [page 1650](#))

Command Syntax :POWer:SIGNals:VMAXimum:ONOff:ON <value>[suffix]
 <value> ::= Maximum expected input Voltage in NR3 format
 [suffix] ::= {V | mV}

The :POWer:SIGNals:VMAXimum:ONOff:ON command specifies the maximum expected input voltage. This value is used to set the vertical scale of the channel probing voltage for turn on analysis.

When the :POWer:ITYPE is DC, this command defines the maximum DC input voltage amplitude value. The values can be negative.

When the :POWer:ITYPE is AC, this command defines the maximum peak-to-peak input voltage. Only positive values are allowed.

Query Syntax :POWer:SIGNals:VMAXimum:ONOff:ON?

The :POWer:SIGNals:VMAXimum:ONOff:ON query returns the expected maximum input voltage setting.

Return Format <value><NL>
 <value> ::= Maximum expected input Voltage in NR3 format

- See Also**
- [":POWer:ITYPE"](#) on page 796
 - [":POWer:ONOff:APPLY"](#) on page 800
 - [":POWer:SIGNals:AUTosetup"](#) on page 822
 - [":POWer:SIGNals:DURation:ONOff:ON"](#) on page 828
 - [":POWer:SIGNals:IEXpected"](#) on page 831
 - [":POWer:SIGNals:OVERshoot"](#) on page 832
 - [":POWer:SIGNals:VSTeady:ONOff:ON"](#) on page 837
 - [":POWer:SIGNals:SOURce:CURREnt<i>"](#) on page 839
 - [":POWer:SIGNals:SOURce:VOLTage<i>"](#) on page 840

:POWer:SIGNals:VSTeady:ONOff:OFF

N (see [page 1650](#))

Command Syntax :POWer:SIGNals:VSTeady:ONOff:OFF <value>[suffix]

<value> ::= Expected steady state output Voltage value in NR3 format

[suffix] ::= {V | mV}

The :POWer:SIGNals:VSTeady:ONOff:OFF command specifies the expected steady state output DC voltage of the power supply for turn off analysis.

Query Syntax :POWer:SIGNals:VSTeady:ONOff:OFF?

The :POWer:SIGNals:VSTeady:ONOff:OFF query returns the expected steady state voltage setting.

Return Format <value><NL>

<value> ::= Expected steady state output Voltage value in NR3 format

- See Also**
- [":POWer:ONOff:APPLy"](#) on page 800
 - [":POWer:SIGNals:AUTosetup"](#) on page 822
 - [":POWer:SIGNals:DURation:ONOff:OFF"](#) on page 827
 - [":POWer:SIGNals:IEXpected"](#) on page 831
 - [":POWer:SIGNals:OVERshoot"](#) on page 832
 - [":POWer:SIGNals:VMAXimum:ONOff:OFF"](#) on page 834
 - [":POWer:SIGNals:SOURce:CURREnt<i>"](#) on page 839
 - [":POWer:SIGNals:SOURce:VOLTage<i>"](#) on page 840

:POWer:SIGNals:VSTeady:ONOff:ON

N (see [page 1650](#))

Command Syntax :POWer:SIGNals:VSTeady:ONOff:ON <value>[suffix]

<value> ::= Expected steady state output Voltage value in NR3 format

[suffix] ::= {V | mV}

The :POWer:SIGNals:VSTeady:ONOff:ON command specifies the expected steady state output DC voltage of the power supply for turn on analysis.

Query Syntax :POWer:SIGNals:VSTeady:ONOff:ON?

The :POWer:SIGNals:VSTeady:ONOff:ON query returns the expected steady state voltage setting.

Return Format <value><NL>

<value> ::= Expected steady state output Voltage value in NR3 format

- See Also**
- [":POWer:ONOff:APPLy"](#) on page 800
 - [":POWer:SIGNals:AUTosetup"](#) on page 822
 - [":POWer:SIGNals:DURation:ONOff:ON"](#) on page 828
 - [":POWer:SIGNals:IEXpected"](#) on page 831
 - [":POWer:SIGNals:OVERshoot"](#) on page 832
 - [":POWer:SIGNals:VMAXimum:ONOff:ON"](#) on page 835
 - [":POWer:SIGNals:SOURce:CURREnt<i>"](#) on page 839
 - [":POWer:SIGNals:SOURce:VOLTage<i>"](#) on page 840

:POWer:SIGNals:VSTeady:TRANsient

N (see [page 1650](#))

Command Syntax :POWer:SIGNals:VSTeady:TRANsient <value>[suffix]

<value> ::= Expected steady state output Voltage value in NR3 format

[suffix] ::= {V | mV}

The :POWer:SIGNals:VSTeady:TRANsient command specifies the expected steady state output DC voltage of the power supply for transient response analysis.

This value is used along with the overshoot percentage to specify the settling band for the transient response and to adjust the vertical scale of the oscilloscope.

Query Syntax :POWer:SIGNals:VSTeady:TRANsient?

The :POWer:SIGNals:VSTeady:TRANsient query returns the expected steady state voltage setting.

Return Format <value><NL>

<value> ::= Expected steady state output Voltage value in NR3 format

- See Also**
- [":POWer:TRANsient:APPLy"](#) on page 849
 - [":POWer:SIGNals:AUTosetup"](#) on page 822
 - [":POWer:SIGNals:DURation:TRANsient"](#) on page 830
 - [":POWer:SIGNals:IEXpected"](#) on page 831
 - [":POWer:SIGNals:OVERshoot"](#) on page 832
 - [":POWer:SIGNals:SOURce:CURREnt<i>"](#) on page 839
 - [":POWer:SIGNals:SOURce:VOLTage<i>"](#) on page 840

:POWer:SIGNals:SOURce:CURRent<i>

N (see [page 1650](#))

Command Syntax :POWer:SIGNals:SOURce:CURRent<i> <source>

<i> ::= 1, 2 in NR1 format

<source> ::= CHANnel<n>

<n> ::= 1 to (# analog channels) in NR1 format

The :POWer:SIGNals:SOURce:CURRent<i> command specifies the first, and perhaps second, current source channel to be used in the power analysis.

Query Syntax :POWer:SIGNals:SOURce:CURRent<i>?

The :POWer:SIGNals:SOURce:CURRent<i> query returns the current source channel setting.

Return Format <source><NL>

<source> ::= CHANnel<n>

<n> ::= 1 to (# analog channels) in NR1 format

- See Also**
- [":POWer:SIGNals:AUTosetup"](#) on page 822
 - [":POWer:SIGNals:CYCLes:HARMonics"](#) on page 823
 - [":POWer:SIGNals:CYCLes:QUALity"](#) on page 824
 - [":POWer:SIGNals:DURation:EFFiciency"](#) on page 825
 - [":POWer:SIGNals:DURation:MODulation"](#) on page 826
 - [":POWer:SIGNals:DURation:ONOff:OFF"](#) on page 827
 - [":POWer:SIGNals:DURation:ONOff:ON"](#) on page 828
 - [":POWer:SIGNals:DURation:RIPple"](#) on page 829
 - [":POWer:SIGNals:DURation:TRANsient"](#) on page 830
 - [":POWer:SIGNals:IEXpected"](#) on page 831
 - [":POWer:SIGNals:OVERshoot"](#) on page 832
 - [":POWer:SIGNals:VMAXimum:INRush"](#) on page 833
 - [":POWer:SIGNals:VMAXimum:ONOff:OFF"](#) on page 834
 - [":POWer:SIGNals:VMAXimum:ONOff:ON"](#) on page 835
 - [":POWer:SIGNals:VSTeady:ONOff:OFF"](#) on page 836
 - [":POWer:SIGNals:VSTeady:ONOff:ON"](#) on page 837
 - [":POWer:SIGNals:VSTeady:TRANsient"](#) on page 838
 - [":POWer:SIGNals:SOURce:VOLTage<i>"](#) on page 840

:POWer:SIGNals:SOURce:VOLTage<i>

N (see [page 1650](#))

Command Syntax :POWer:SIGNals:SOURce:VOLTage<i> <source>

<i> ::= 1, 2 in NR1 format

<source> ::= CHANnel<n>

<n> ::= 1 to (# analog channels) in NR1 format

The :POWer:SIGNals:SOURce:VOLTage<i> command specifies the first, and perhaps second, voltage source channel to be used in the power analysis.

Query Syntax :POWer:SIGNals:SOURce:VOLTage<i>?

The :POWer:SIGNals:SOURce:VOLTage<i> query returns the voltage source channel setting.

Return Format <source><NL>

<source> ::= CHANnel<n>

<n> ::= 1 to (# analog channels) in NR1 format

- See Also**
- [":POWer:SIGNals:AUTosetup"](#) on page 822
 - [":POWer:SIGNals:CYCLes:HARMonics"](#) on page 823
 - [":POWer:SIGNals:CYCLes:QUALity"](#) on page 824
 - [":POWer:SIGNals:DURation:EFFiciency"](#) on page 825
 - [":POWer:SIGNals:DURation:MODulation"](#) on page 826
 - [":POWer:SIGNals:DURation:ONOff:OFF"](#) on page 827
 - [":POWer:SIGNals:DURation:ONOff:ON"](#) on page 828
 - [":POWer:SIGNals:DURation:RIPPlE"](#) on page 829
 - [":POWer:SIGNals:DURation:TRANsient"](#) on page 830
 - [":POWer:SIGNals:IEXPeCted"](#) on page 831
 - [":POWer:SIGNals:OVERshoot"](#) on page 832
 - [":POWer:SIGNals:VMAXimum:INRush"](#) on page 833
 - [":POWer:SIGNals:VMAXimum:ONOff:OFF"](#) on page 834
 - [":POWer:SIGNals:VMAXimum:ONOff:ON"](#) on page 835
 - [":POWer:SIGNals:VSTeady:ONOff:OFF"](#) on page 836
 - [":POWer:SIGNals:VSTeady:ONOff:ON"](#) on page 837
 - [":POWer:SIGNals:VSTeady:TRANsient"](#) on page 838
 - [":POWer:SIGNals:SOURce:CURREnt<i>"](#) on page 839

:POWer:SLEW:APPLY

N (see [page 1650](#))

Command Syntax :POWer:SLEW:APPLY

The :POWer:SLEW:APPLY command applies the slew rate analysis.

See Also • [":POWer:SLEW:SOURce"](#) on page 842

:POWer:SLEW:SOURce

N (see [page 1650](#))

Command Syntax `:POWer:SLEW:SOURce <source>`

`<source> ::= {V | I}`

The :POWer:SLEW:SOURce command selects either the voltage source or the current source as the source for the slew rate analysis.

Query Syntax `:POWer:SLEW:SOURce?`

The :POWer:SLEW:SOURce query returns the selected source for the slew rate analysis.

Return Format `<source><NL>`

`<source> ::= {V | I}`

See Also • [":POWer:SLEW:APPLy"](#) on page 841

:POWer:SWITCh:APPLy

N (see [page 1650](#))

Command Syntax :POWer:SWITCh:APPLy

The :POWer:SWITCh:APPLy command applies the switching loss analysis using the conduction calculation method, V reference, and I reference settings.

- See Also**
- [":POWer:SWITCh:CONDUction"](#) on page 844
 - [":POWer:SWITCh:IREFERENCE"](#) on page 845
 - [":POWer:SWITCh:RDS"](#) on page 846
 - [":POWer:SWITCh:VCE"](#) on page 847
 - [":POWer:SWITCh:VREFERENCE"](#) on page 848
 - [":MEASure:ELOSs"](#) on page 697
 - [":MEASure:PLOSs"](#) on page 704

:POWer:SWITCh:CONDUction

N (see [page 1650](#))

Command Syntax :POWer:SWITCh:CONDUction <conduction>

<conduction> ::= {WAVeform | RDS | VCE}

The :POWer:SWITCh:CONDUction command specifies the conduction calculation method:

- WAVeform – The Power waveform uses the original voltage waveform data, and the calculation is: $P = V \times I$
- RDS – Rds(on) – The Power waveform includes error correction:
 - In the On Zone (where the voltage level is below V Ref) – the Power calculation is: $P = Id2 \times Rds(on)$
Specify Rds(on) using the :POWer:SWITCh:RDS command.
 - In the Off Zone (where the current level is below I Ref) – the Power calculation is: $P = 0$ Watt.
- VCE – Vce(sat) – The Power waveform includes error correction:
 - In the On Zone (where the voltage level is below V Ref) – the Power calculation is: $P = Vce(sat) \times Ic$
Specify Vce(sat) using the :POWer:SWITCh:VCE command.
 - In the Off Zone (where the current level is below I Ref) – the Power calculation is: $P = 0$ Watt.

Query Syntax :POWer:SWITCh:CONDUction?

The :POWer:SWITCh:CONDUction query returns the conduction calculation method.

Return Format <conduction><NL>

<conduction> ::= {WAV | RDS | VCE}

- See Also**
- [":POWer:SWITCh:APPLY"](#) on page 843
 - [":POWer:SWITCh:IREference"](#) on page 845
 - [":POWer:SWITCh:RDS"](#) on page 846
 - [":POWer:SWITCh:VCE"](#) on page 847
 - [":POWer:SWITCh:VREference"](#) on page 848

:POWer:SWITCh:IREFERENCE

N (see [page 1650](#))

Command Syntax :POWer:SWITCh:IREFERENCE <percent>

<percent> ::= percent in NR1 format

The :POWer:SWITCh:IREFERENCE command to specify the current switching level for the start of switching edges. The value is in percentage of the maximum switch current.

You can adjust this value to ignore noise floors or null offset that is difficult to eliminate in current probes.

This value specifies the threshold that is used to determine the switching edges.

Query Syntax :POWer:SWITCh:IREFERENCE?

The :POWer:SWITCh:IREFERENCE query returns the current switching level percent value.

Return Format <percent><NL>

<percent> ::= percent in NR1 format

- See Also**
- [":POWer:SWITCh:APPLY"](#) on page 843
 - [":POWer:SWITCh:CONDUCTION"](#) on page 844
 - [":POWer:SWITCh:RDS"](#) on page 846
 - [":POWer:SWITCh:VCE"](#) on page 847
 - [":POWer:SWITCh:VREFERENCE"](#) on page 848

:POWer:SWITCh:RDS

N (see [page 1650](#))

Command Syntax :POWer:SWITCh:RDS <value>[suffix]

<value> ::= Rds(on) value in NR3 format

[suffix] ::= {OHM | mOHM}

The :POWer:SWITCh:RDS command specifies the Rds(on) value when the RDS conduction calculation method is chosen (by :POWer:SWITCh:CONDUction).

Query Syntax :POWer:SWITCh:RDS?

The :POWer:SWITCh:RDS query returns the Rds(on) value.

Return Format <value><NL>

<value> ::= Rds(on) value in NR3 format

- See Also**
- [":POWer:SWITCh:APPLy"](#) on page 843
 - [":POWer:SWITCh:CONDUction"](#) on page 844
 - [":POWer:SWITCh:IREFERENCE"](#) on page 845
 - [":POWer:SWITCh:VCE"](#) on page 847
 - [":POWer:SWITCh:VREFERENCE"](#) on page 848

:POWer:SWITCh:VCE

N (see [page 1650](#))

Command Syntax :POWer:SWITCh:VCE <value>[suffix]

<value> ::= Vce(sat) value in NR3 format

[suffix] ::= {V | mV}

The :POWer:SWITCh:VCE command specifies the Vce(sat) value when the VCE conduction calculation method is chosen (by :POWer:SWITCh:CONDUction).

Query Syntax :POWer:SWITCh:VCE?

The :POWer:SWITCh:VCE query returns the Vce(sat) value.

Return Format <value><NL>

<value> ::= Vce(sat) value in NR3 format

- See Also**
- [":POWer:SWITCh:APPLy"](#) on page 843
 - [":POWer:SWITCh:CONDUction"](#) on page 844
 - [":POWer:SWITCh:IREFERENCE"](#) on page 845
 - [":POWer:SWITCh:RDS"](#) on page 846
 - [":POWer:SWITCh:VREFERENCE"](#) on page 848

:POWer:SWITCh:VREference

N (see [page 1650](#))

Command Syntax :POWer:SWITCh:VREference <percent>

<percent> ::= percent in NR1 format

The :POWer:SWITCh:VREference command to specify the voltage switching level for the switching edges. The value is in percentage of the maximum switch voltage.

You can adjust this value to ignore noise floors.

This value specifies the threshold that is used to determine the switching edges.

Query Syntax :POWer:SWITCh:VREference?

The :POWer:SWITCh:VREference query returns the voltage switching level percent value.

Return Format <percent><NL>

<percent> ::= percent in NR1 format

- See Also**
- [":POWer:SWITCh:APPLY"](#) on page 843
 - [":POWer:SWITCh:CONDUCTION"](#) on page 844
 - [":POWer:SWITCh:IREference"](#) on page 845
 - [":POWer:SWITCh:RDS"](#) on page 846
 - [":POWer:SWITCh:VCE"](#) on page 847

:POWer:TRANsient:APPLy

N (see [page 1650](#))

Command Syntax :POWer:TRANsient:APPLy

The :POWer:TRANsient:APPLy command applies the transient analysis using the initial current and new current settings.

- See Also**
- [":POWer:TRANsient:EXIT"](#) on page 850
 - [":POWer:TRANsient:IInitial"](#) on page 851
 - [":POWer:TRANsient:INew"](#) on page 852
 - [":POWer:TRANsient:NEXT"](#) on page 853
 - [":MEASure:TRESponse"](#) on page 709

:POWer:TRANsient:EXIT

N (see [page 1650](#))

Command Syntax :POWer:TRANsient:EXIT

The :POWer:TRANsient:EXIT command exits (stops) the transient analysis.

This command is equivalent to pressing the **Exit** softkey on the oscilloscope front panel during the analysis.

- See Also**
- [":POWer:TRANsient:APPLy"](#) on page 849
 - [":POWer:TRANsient:IIInitial"](#) on page 851
 - [":POWer:TRANsient:INEW"](#) on page 852
 - [":POWer:TRANsient:NEXT"](#) on page 853

:POWer:TRANsient:IINitIAL

N (see [page 1650](#))

Command Syntax :POWer:TRANsient:IINitIAL <value>[suffix]

<value> ::= Initial current value in NR3 format

[suffix] ::= {A | mA}

The :POWer:TRANsient:IINitIAL command to specify the initial load current value. The initial load current will be used as a reference and to trigger the oscilloscope.

Query Syntax :POWer:TRANsient:IINitIAL?

The :POWer:TRANsient:IINitIAL query returns the initial load current value.

Return Format <value><NL>

<value> ::= Initial current value in NR3 format

- See Also**
- [":POWer:SIGNals:VSTeady:TRANsient"](#) on page 838
 - [":POWer:TRANsient:APPLy"](#) on page 849
 - [":POWer:TRANsient:EXIT"](#) on page 850
 - [":POWer:TRANsient:INew"](#) on page 852
 - [":POWer:TRANsient:NEXT"](#) on page 853

:POWer:TRANsient:INEW

N (see [page 1650](#))

Command Syntax :POWer:TRANsient:INEW <value>[suffix]

<value> ::= New current value in NR3 format

[suffix] ::= {A | mA}

The :POWer:TRANsient:INEW command to specify the new load current value. The new load current will be used as a reference and to trigger the oscilloscope.

Query Syntax :POWer:TRANsient:INEW?

The :POWer:TRANsient:INEW query returns the new load current value.

Return Format <value><NL>

<value> ::= New current value in NR3 format

- See Also**
- [":POWer:TRANsient:APPLy"](#) on page 849
 - [":POWer:TRANsient:EXIT"](#) on page 850
 - [":POWer:TRANsient:IInitial"](#) on page 851
 - [":POWer:TRANsient:NEXT"](#) on page 853

:POWer:TRANsient:NEXT

 (see [page 1650](#))

Command Syntax :POWer:TRANsient:NEXT

The :POWer:TRANsient:NEXT command goes to the next step of the transient analysis.

This command is equivalent to pressing the **Next** softkey on the oscilloscope front panel when prompted during the analysis.

- See Also**
- [":POWer:TRANsient:APPLY"](#) on page 849
 - [":POWer:TRANsient:EXIT"](#) on page 850
 - [":POWer:TRANsient:IInitial"](#) on page 851
 - [":POWer:TRANsient:INEW"](#) on page 852

31 :RECall Commands

Recall previously saved oscilloscope setups, reference waveforms, and masks.

Table 118 :RECall Commands Summary

Command	Query	Options and Query Returns
:RECall:ARBitrary:[ST ARt] [<file_spec>] [, <column>] [, <wavegen_id>] (see page 858)	n/a	<file_spec> ::= {<internal_loc> <file_name>} <column> ::= Column in CSV file to load. Column number starts from 1. <internal_loc> ::= 0-3; an integer in NR1 format <file_name> ::= quoted ASCII string <wavegen_id> ::= {WGEN1 WGEN2}
:RECall:DBC[:START] [<file_name>] [, <serialbus>] (see page 859)	n/a	<file_name> ::= quoted ASCII string If extension included in file name, it must be ".dbc". <serialbus> ::= {SBUS<n>} <n> ::= 1 to (# of serial bus) in NR1 format
:RECall:FIleName <base_name> (see page 860)	:RECall:FIleName? (see page 860)	<base_name> ::= quoted ASCII string
:RECall:LDF[:START] [<file_name>] [, <serialbus>] (see page 861)	n/a	<file_name> ::= quoted ASCII string If extension included in file name, it must be ".ldf". <serialbus> ::= {SBUS<n>} <n> ::= 1 to (# of serial bus) in NR1 format

Table 118 :RECall Commands Summary (continued)

Command	Query	Options and Query Returns
:RECall:MASK[:START] [<file_spec>] (see page 862)	n/a	<file_spec> ::= {<internal_loc> <file_name>} <internal_loc> ::= 0-3; an integer in NR1 format <file_name> ::= quoted ASCII string
:RECall:PWD <path_name> (see page 863)	:RECall:PWD? (see page 863)	<path_name> ::= quoted ASCII string
:RECall:SETup[:START] [<file_spec>] (see page 864)	n/a	<file_spec> ::= {<internal_loc> <file_name>} <internal_loc> ::= 0-9; an integer in NR1 format <file_name> ::= quoted ASCII string
:RECall:WMEemory<r>[:S TART] [<file_name> <data>] (see page 865)	n/a	<r> ::= 1 to (# ref waveforms) in NR1 format <file_name> ::= quoted ASCII string If extension included in file name, it must be ".h5". <data> ::= binary block data in IEEE 488.2 # format

Introduction to :RECall Commands The :RECall subsystem provides commands to recall previously saved oscilloscope setups, reference waveforms, and masks.

Reporting the Setup

Use :RECall? to query setup information for the RECall subsystem.

Return Format

The following is a sample response from the :RECall? query. In this case, the query was issued following the *RST command.

```
:REC:FIL "scope_0"
```

Recalling Files From a USB Storage Device

When :RECall commands have a "quoted ASCII string" <file_name> parameter, you can recall files from a connected USB storage device. For example:

```
' To recall a setup file from a connected USB storage device:  
myScope.WriteString ":RECall:SETup:START ""\usb\my_setup_file.scp"""
```

:RECall:ARbitrary[:START]

N (see [page 1650](#))

Command Syntax :RECall:ARbitrary:[START] [<file_spec>][, <column>][, <wavegen_id>]

<file_spec> ::= {<internal_loc> | <file_name>}

<column> ::= Column in CSV file to load. Column number starts from 1.

<wavegen_id> ::= {WGEN1 | WGEN2} - specifies which wavegen

<internal_loc> ::= 0-3; an integer in NR1 format

<file_name> ::= quoted ASCII string

The :RECall:ARbitrary:[START] command recalls an arbitrary waveform.

NOTE

If a file extension is provided as part of a specified <file_name>, it must be ".csv".

For internal locations, the <column> parameter is ignored.

For external (USB storage device) files, the column parameter is optional. If no <column> parameter is entered, and it is a 2-column file, the 2nd column (assumed to be voltage) is automatically be selected. If the <column> parameter is entered, and that column does not exist in the file, the operation fails.

When recalling arbitrary waveforms (from an external USB storage device) that were not saved from the oscilloscope, be aware that the oscilloscope uses a maximum of 8192 points for an arbitrary waveform. For more efficient recalls, make sure your arbitrary waveforms are 8192 points or less.

The <wavegen_id> parameter specifies which waveform generator to recall the arbitrary waveform into.

- See Also**
- ["Introduction to :RECall Commands"](#) on page 856
 - [":RECall:FILENAME"](#) on page 860
 - [":RECall:PWD"](#) on page 863
 - [":SAVE:ARbitrary\[:START\]"](#) on page 871

:RECall:DBC[:START]

N (see [page 1650](#))

Command Syntax :RECall:DBC[:START] [<file_name>] [, <serialbus>]

<file_name> ::= quoted ASCII string

<serialbus> ::= {SBUS<n>}

<n> ::= 1 to (# of serial bus) in NR1 format

The :RECall:DBC[:START] command loads a CAN DBC (communication database) symbolic data file into the oscilloscope.

NOTE

If a file extension is provided as part of a specified <file_name>, it must be ".dbc".

The <serialbus> parameter specifies which serial decode waveform the CAN symbolic data will be loaded for.

- See Also**
- ["Introduction to :RECall Commands"](#) on page 856
 - [":RECall:FILENAME"](#) on page 860
 - [":SBUS<n>:CAN:TRIGger"](#) on page 940
 - [":SBUS<n>:CAN:TRIGger:SYMBOLic:MESSAge"](#) on page 950
 - [":SBUS<n>:CAN:TRIGger:SYMBOLic:SIGNAL"](#) on page 951
 - [":SBUS<n>:CAN:TRIGger:SYMBOLic:VALue"](#) on page 952
 - [":SEARCH:SERial:CAN:MODE"](#) on page 1233
 - [":SEARCH:SERial:CAN:SYMBOLic:MESSAge"](#) on page 1239
 - [":SEARCH:SERial:CAN:SYMBOLic:SIGNAL"](#) on page 1240
 - [":SEARCH:SERial:CAN:SYMBOLic:VALue"](#) on page 1241

:RECall:FIlename

N (see [page 1650](#))

Command Syntax :RECall:FIlename <base_name>

<base_name> ::= quoted ASCII string

The :RECall:FIlename command specifies the source for any RECall operations.

NOTE

This command specifies a file's base name only, without path information or an extension.

Query Syntax :RECall:FIlename?

The :RECall:FIlename? query returns the current RECall filename.

Return Format <base_name><NL>

<base_name> ::= quoted ASCII string

- See Also**
- ["Introduction to :RECall Commands"](#) on page 856
 - [":RECall:SETup\[:START\]"](#) on page 864
 - [":SAVE:FIlename"](#) on page 873

:RECall:LDF[:START]

N (see [page 1650](#))

Command Syntax :RECall:LDF[:START] [<file_name>] [, <serialbus>]

<file_name> ::= quoted ASCII string

<serialbus> ::= {SBUS<n>}

<n> ::= 1 to (# of serial bus) in NR1 format

The :RECall:LDF[:START] command loads a LIN description file (LDF) symbolic data file into the oscilloscope.

NOTE

If a file extension is provided as part of a specified <file_name>, it must be ".ldf".

The <serialbus> parameter specifies which serial decode waveform the LIN symbolic data will be loaded for.

- See Also**
- ["Introduction to :RECall Commands"](#) on page 856
 - [":RECall:FILENAME"](#) on page 860
 - [":SBUS<n>:LIN:TRIGger"](#) on page 1028
 - [":SBUS<n>:LIN:TRIGger:SYMBOLic:FRAME"](#) on page 1035
 - [":SBUS<n>:LIN:TRIGger:SYMBOLic:SIGNAL"](#) on page 1036
 - [":SBUS<n>:LIN:TRIGger:SYMBOLic:VALue"](#) on page 1037
 - [":SEARCH:SERial:LIN:MODE"](#) on page 1263
 - [":SEARCH:SERial:LIN:SYMBOLic:FRAME"](#) on page 1267
 - [":SEARCH:SERial:LIN:SYMBOLic:SIGNAL"](#) on page 1268
 - [":SEARCH:SERial:LIN:SYMBOLic:VALue"](#) on page 1269

:RECall:MASK[:START]

N (see [page 1650](#))

Command Syntax :RECall:MASK[:START] [<file_spec>]
 <file_spec> ::= {<internal_loc> | <file_name>}
 <internal_loc> ::= 0-3; an integer in NR1 format
 <file_name> ::= quoted ASCII string

The :RECall:MASK[:START] command recalls a mask.

NOTE

If a file extension is provided as part of a specified <file_name>, it must be ".msk".

-
- See Also**
- ["Introduction to :RECall Commands"](#) on page 856
 - [":RECall:FILENAME"](#) on page 860
 - [":RECall:PWD"](#) on page 863
 - [":SAVE:MASK\[:START\]"](#) on page 880
 - [":MTEST:DATA"](#) on page 726

:RECall:PWD

N (see [page 1650](#))

Command Syntax :RECall:PWD <path_name>
 <path_name> ::= quoted ASCII string

The :RECall:PWD command sets the present working directory for recall operations.

NOTE

Presently, the internal "/User Files" directory you see in the oscilloscope's front panel user interface is the "\Agilent Flash" directory you see in the remote interface.

Query Syntax :RECall:PWD?

The :RECall:PWD? query returns the currently set working directory for recall operations.

Return Format <path_name><NL>
 <path_name> ::= quoted ASCII string

- See Also**
- ["Introduction to :RECall Commands"](#) on page 856
 - [":SAVE:PWD"](#) on page 883

:RECall:SETup[:START]

N (see [page 1650](#))

Command Syntax :RECall:SETup[:START] [<file_spec>]
 <file_spec> ::= {<internal_loc> | <file_name>}
 <internal_loc> ::= 0-9; an integer in NR1 format
 <file_name> ::= quoted ASCII string

The :RECall:SETup[:START] command recalls an oscilloscope setup.

NOTE

If a file extension is provided as part of a specified <file_name>, it must be ".scp".

- See Also**
- ["Introduction to :RECall Commands"](#) on page 856
 - [":RECall:FILENAME"](#) on page 860
 - [":RECall:PWD"](#) on page 863
 - [":SAVE\[:SETup\[:START\]\]"](#) on page 891

:RECall:WMEemory<r>[:START]

N (see [page 1650](#))

Command Syntax :RECall:WMEemory<r>[:START] [<file_name> | <data>]
 <r> ::= 1 to (# ref waveforms) in NR1 format
 <file_name> ::= quoted ASCII string
 <data> ::= binary block data in IEEE 488.2 # format

The :RECall:WMEemory<r>[:START] command recalls a reference waveform.

NOTE

If a file extension is provided as part of a specified <file_name>, it must be ".h5".

The <data> option lets you recall a reference waveform from a local file on the controller PC (instead of from a USB storage device connected to the oscilloscope). In this case, your remote program reads data from the local ".h5" format reference waveform file in the same way that setup files are restored to the oscilloscope (see [":SYSTEM:SETup"](#) on page 1326).

- See Also**
- ["Introduction to :RECall Commands"](#) on page 856
 - [":RECall:FILENAME"](#) on page 860
 - [":SAVE:WMEemory\[:START\]"](#) on page 898

32 :SAVE Commands

Save oscilloscope setups, screen images, and data. See "[Introduction to :SAVE Commands](#)" on page 870.

Table 119 :SAVE Commands Summary

Command	Query	Options and Query Returns
:SAVE:ARbitrary:[STARt] [<file_spec>] [, <wavegen_id>] (see page 871)	n/a	<file_spec> ::= {<internal_loc> <file_name>} <internal_loc> ::= 0-3; an integer in NR1 format <file_name> ::= quoted ASCII string <wavegen_id> ::= {WGEN1 WGEN2}
:SAVE:COMpliance:USB[:START] [<file_name>] (see page 872)	n/a	<file_name> ::= quoted ASCII string
:SAVE:FILENAME <base_name> (see page 873)	:SAVE:FILENAME? (see page 873)	<base_name> ::= quoted ASCII string
:SAVE:IMAGe[:START] [<file_name>] (see page 874)	n/a	<file_name> ::= quoted ASCII string
:SAVE:IMAGe:FACTors {{0 OFF} {1 ON}} (see page 875)	:SAVE:IMAGe:FACTors? (see page 875)	{0 1}
:SAVE:IMAGe:FORMat <format> (see page 876)	:SAVE:IMAGe:FORMat? (see page 876)	<format> ::= {{BMP BMP24bit} BMP8bit PNG NONE}
:SAVE:IMAGe:INKSaver {{0 OFF} {1 ON}} (see page 877)	:SAVE:IMAGe:INKSaver? (see page 877)	{0 1}

Table 119 :SAVE Commands Summary (continued)

Command	Query	Options and Query Returns
:SAVE:IMAGe:PALette <palette> (see page 878)	:SAVE:IMAGe:PALette? (see page 878)	<palette> ::= {COLor GRAYscale}
:SAVE:LISTer[:START] [<file_name>] (see page 879)	n/a	<file_name> ::= quoted ASCII string
:SAVE:MASK[:START] [<file_spec>] (see page 880)	n/a	<file_spec> ::= {<internal_loc> <file_name>} <internal_loc> ::= 0-3; an integer in NR1 format <file_name> ::= quoted ASCII string
:SAVE:MULTi[:START] [<file_name>] (see page 881)	n/a	<file_name> ::= quoted ASCII string
:SAVE:POWER[:START] [<file_name>] (see page 882)	n/a	<file_name> ::= quoted ASCII string
:SAVE:PWD <path_name> (see page 883)	:SAVE:PWD? (see page 883)	<path_name> ::= quoted ASCII string
:SAVE:RESults:[START] [<file_spec>] (see page 884)	n/a	<file_name> ::= quoted ASCII string
:SAVE:RESults:FORMat: CURSor {{0 OFF} {1 ON}} (see page 885)	:SAVE:RESults:FORMat: CURSor? (see page 885)	{0 1}
:SAVE:RESults:FORMat: HISTogram {{0 OFF} {1 ON}} (see page 886)	:SAVE:RESults:FORMat: HISTogram? (see page 886)	{0 1}
:SAVE:RESults:FORMat: MASK {{0 OFF} {1 ON}} (see page 887)	:SAVE:RESults:FORMat: MASK? (see page 887)	{0 1}
:SAVE:RESults:FORMat: MEASurement {{0 OFF} {1 ON}} (see page 888)	:SAVE:RESults:FORMat: MEASurement? (see page 888)	{0 1}

Table 119 :SAVE Commands Summary (continued)

Command	Query	Options and Query Returns
:SAVE:RESults:FORMat:SEARCh {{0 OFF} {1 ON}} (see page 889)	:SAVE:RESults:FORMat:SEARCh? (see page 889)	{0 1}
:SAVE:RESults:FORMat:SEGMENTed {{0 OFF} {1 ON}} (see page 890)	:SAVE:RESults:FORMat:SEGMENTed? (see page 890)	{0 1}
:SAVE:SETup[:START] [<file_spec>] (see page 891)	n/a	<file_spec> ::= {<internal_loc> <file_name>} <internal_loc> ::= 0-9; an integer in NR1 format <file_name> ::= quoted ASCII string
:SAVE:WAVEform[:START] [<file_name>] (see page 892)	n/a	<file_name> ::= quoted ASCII string
:SAVE:WAVEform:FORMat<format> (see page 893)	:SAVE:WAVEform:FORMat? (see page 893)	<format> ::= {ASCIixy CSV BINary NONE}
:SAVE:WAVEform:LENGth<length> (see page 894)	:SAVE:WAVEform:LENGth? (see page 894)	<length> ::= 100 to max. length; an integer in NR1 format
:SAVE:WAVEform:LENGth:MAX {{0 OFF} {1 ON}} (see page 895)	:SAVE:WAVEform:LENGth:MAX? (see page 895)	{0 1}
:SAVE:WAVEform:SEGMENTed <option> (see page 896)	:SAVE:WAVEform:SEGMENTed? (see page 896)	<option> ::= {ALL CURRENT}

Table 119 :SAVE Commands Summary (continued)

Command	Query	Options and Query Returns
:SAVE:WMEemory:SOURce <source> (see page 897)	:SAVE:WMEemory:SOURce? (see page 897)	<source> ::= {CHANnel<n> FUNCTION<m> MATH<m> WMEemory<r>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format NOTE: Only ADD or SUBtract math operations can be saved as reference waveforms. <return_value> ::= <source>
:SAVE:WMEemory[:START] [<file_name>] (see page 898)	n/a	<file_name> ::= quoted ASCII string If extension included in file name, it must be ".h5".

Introduction to :SAVE Commands

The :SAVE subsystem provides commands to save oscilloscope setups, screen images, and data.

:SAV is an acceptable short form for :SAVE.

Reporting the Setup

Use :SAVE? to query setup information for the SAVE subsystem.

Return Format

The following is a sample response from the :SAVE? query. In this case, the query was issued following the *RST command.

```
:SAVE:FIL " ";:SAVE:IMAG:AREA GRAT;FACT 0;FORM TIFF;INKS 0;PAL
MON;:SAVE:PWD "C:/setups/";:SAVE:WAV:FORM NONE;LENG 1000;SEGM CURR
```

Saving Files to a USB Storage Device

When :SAVE commands have a "quoted ASCII string" <file_name> parameter, you can save files to a connected USB storage device. For example:

```
' To save a setup file to a connected USB storage device:
myScope.WriteString ":SAVE:SETup:START ""\usb\my_setup_file.scp"""
```

:SAVE:ARbitrary[:START]

N (see [page 1650](#))

Command Syntax :SAVE:ARbitrary:[START] [<file_spec>] [, <wavegen_id>]

<file_spec> ::= {<internal_loc> | <file_name>}

<internal_loc> ::= 0-3; an integer in NR1 format

<file_name> ::= quoted ASCII string

<wavegen_id> ::= {WGEN1 | WGEN2}

The :SAVE:ARbitrary:[START] command saves the current arbitrary waveform to an internal location or a file on a USB storage device.

The <wavegen_id> parameter specifies which waveform generator to save the arbitrary waveform from.

NOTE

If a file extension is provided as part of a specified <file_name>, it must be ".csv".

- See Also**
- ["Introduction to :SAVE Commands"](#) on page 870
 - [":SAVE:FILENAME"](#) on page 873
 - [":SAVE:PWD"](#) on page 883
 - [":RECall:ARbitrary\[:START\]"](#) on page 858

:SAVE:COMpliance:USB[:START]

N (see [page 1650](#))

Command Syntax :SAVE:COMpliance:USB[:START] [<file_name>]

<file_name> ::= quoted ASCII string

The :SAVE:COMpliance:USB[:START] command saves USB 2.0 signal quality test results to a file.

NOTE

If a file extension is provided as part of a specified <file_name>, it must be ".html".

-
- See Also**
- ["Introduction to :SAVE Commands"](#) on page 870
 - [":SAVE:FILENAME"](#) on page 873
 - [":SAVE:PWD"](#) on page 883
 - [Chapter 12](#), ":COMpliance Commands," starting on page 357

:SAVE:FiLenamE

N (see [page 1650](#))

Command Syntax :SAVE:FiLenamE <base_name>
 <base_name> ::= quoted ASCII string

The :SAVE:FiLenamE command specifies the source for any SAVE operations.

NOTE

This command specifies a file's base name only, without path information or an extension.

Query Syntax :SAVE:FiLenamE?

The :SAVE:FiLenamE? query returns the current SAVE filename.

Return Format <base_name><NL>
 <base_name> ::= quoted ASCII string

- See Also**
- ["Introduction to :SAVE Commands"](#) on page 870
 - [":SAVE:IMAGe\[:START\]"](#) on page 874
 - [":SAVE\[:SETup\[:START\]\]"](#) on page 891
 - [":SAVE:WAVEform\[:START\]"](#) on page 892
 - [":SAVE:PWD"](#) on page 883
 - [":RECall:FiLenamE"](#) on page 860

:SAVE:IMAGe[:START]

N (see [page 1650](#))

Command Syntax :SAVE:IMAGe[:START] [<file_name>]

<file_name> ::= quoted ASCII string

The :SAVE:IMAGe[:START] command saves an image.

NOTE

Be sure to set the :SAVE:IMAGe:FORMat before saving an image. If the format is NONE, the save image command will not succeed.

NOTE

If a file extension is provided as part of a specified <file_name>, and it does not match the extension expected by the format specified in :SAVE:IMAGe:FORMat, the format will be changed if the extension is a valid image file extension.

NOTE

If the extension ".bmp" is used and the current :SAVE:IMAGe:FORMat is not BMP or BMP8, the format will be changed to BMP.

- See Also**
- ["Introduction to :SAVE Commands"](#) on page 870
 - [":SAVE:IMAGe:FACTors"](#) on page 875
 - [":SAVE:IMAGe:FORMat"](#) on page 876
 - [":SAVE:IMAGe:INKSaver"](#) on page 877
 - [":SAVE:IMAGe:PALette"](#) on page 878
 - [":SAVE:FILEname"](#) on page 873

:SAVE:IMAGe:FACTors

N (see [page 1650](#))

Command Syntax :SAVE:IMAGe:FACTors <factors>
 <factors> ::= {{OFF | 0} | {ON | 1}}

The :SAVE:IMAGe:FACTors command controls whether the oscilloscope factors are output along with the image.

NOTE

Factors are written to a separate file with the same path and base name but with the ".txt" extension.

Query Syntax :SAVE:IMAGe:FACTors?

The :SAVE:IMAGe:FACTors? query returns a flag indicating whether oscilloscope factors are output along with the image.

Return Format <factors><NL>
 <factors> ::= {0 | 1}

- See Also**
- ["Introduction to :SAVE Commands"](#) on page 870
 - [":SAVE:IMAGe\[:START\]"](#) on page 874
 - [":SAVE:IMAGe:FORMat"](#) on page 876
 - [":SAVE:IMAGe:INKSaver"](#) on page 877
 - [":SAVE:IMAGe:PALette"](#) on page 878

:SAVE:IMAGe:FORMat

N (see [page 1650](#))

Command Syntax :SAVE:IMAGe:FORMat <format>

<format> ::= {{BMP | BMP24bit} | BMP8bit | PNG}

The :SAVE:IMAGe:FORMat command sets the image format type.

Query Syntax :SAVE:IMAGe:FORMat?

The :SAVE:IMAGe:FORMat? query returns the selected image format type.

Return Format <format><NL>

<format> ::= {BMP | BMP8 | PNG | NONE}

When NONE is returned, it indicates that a waveform data file format is currently selected.

- See Also**
- ["Introduction to :SAVE Commands"](#) on page 870
 - [":SAVE:IMAGe\[:START\]"](#) on page 874
 - [":SAVE:IMAGe:FACTors"](#) on page 875
 - [":SAVE:IMAGe:INKSaver"](#) on page 877
 - [":SAVE:IMAGe:PALette"](#) on page 878
 - [":SAVE:WAVEform:FORMat"](#) on page 893

:SAVE:IMAGe:INKSaver

N (see [page 1650](#))

Command Syntax :SAVE:IMAGe:INKSaver <value>

<value> ::= {{OFF | 0} | {ON | 1}}

The :SAVE:IMAGe:INKSaver command controls whether the graticule colors are inverted or not.

Query Syntax :SAVE:IMAGe:INKSaver?

The :SAVE:IMAGe:INKSaver? query returns a flag indicating whether graticule colors are inverted or not.

Return Format <value><NL>

<value> ::= {0 | 1}

- See Also**
- ["Introduction to :SAVE Commands"](#) on page 870
 - [":SAVE:IMAGe\[:START\]"](#) on page 874
 - [":SAVE:IMAGe:FACTors"](#) on page 875
 - [":SAVE:IMAGe:FORMat"](#) on page 876
 - [":SAVE:IMAGe:PALette"](#) on page 878

:SAVE:IMAGe:PALette

N (see [page 1650](#))

Command Syntax :SAVE:IMAGe:PALette <palette>

<palette> ::= {COLor | GRAYscale}

The :SAVE:IMAGe:PALette command sets the image palette color.

Query Syntax :SAVE:IMAGe:PALette?

The :SAVE:IMAGe:PALette? query returns the selected image palette color.

Return Format <palette><NL>

<palette> ::= {COL | GRAY}

- See Also**
- ["Introduction to :SAVE Commands"](#) on page 870
 - [":SAVE:IMAGe\[:START\]"](#) on page 874
 - [":SAVE:IMAGe:FACTors"](#) on page 875
 - [":SAVE:IMAGe:FORMat"](#) on page 876
 - [":SAVE:IMAGe:INKSaver"](#) on page 877

:SAVE:LISTer[:START]

N (see [page 1650](#))

Command Syntax :SAVE:LISTer[:START] [<file_name>]

<file_name> ::= quoted ASCII string

The :SAVE:LISTer[:START] command saves the Lister display data to a file.

NOTE

If a file extension is provided as part of a specified <file_name>, it must be ".csv".

-
- See Also**
- ["Introduction to :SAVE Commands"](#) on page 870
 - [":SAVE:FILENAME"](#) on page 873
 - [Chapter 23](#), ":LISTer Commands," starting on page 541

:SAVE:MASK[:START]

N (see [page 1650](#))

Command Syntax :SAVE:MASK[:START] [<file_spec>]
 <file_spec> ::= {<internal_loc> | <file_name>}
 <internal_loc> ::= 0-3; an integer in NR1 format
 <file_name> ::= quoted ASCII string

The :SAVE:MASK[:START] command saves a mask.

NOTE

If a file extension is provided as part of a specified <file_name>, it must be ".msk".

- See Also**
- ["Introduction to :SAVE Commands"](#) on page 870
 - [":SAVE:FILENAME"](#) on page 873
 - [":SAVE:PWD"](#) on page 883
 - [":RECall:MASK\[:START\]"](#) on page 862
 - [":MTEST:DATA"](#) on page 726

:SAVE:MULTi[:START]

N (see [page 1650](#))

Command Syntax :SAVE:MULTi[:START] [<file_name>]

<file_name> ::= quoted ASCII string

The :SAVE:MULTi[:START] command saves multi-channel waveform data to a file. This file can be opened by the N8900A Infiniium Offline oscilloscope analysis software.

NOTE

If a file extension is provided as part of a specified <file_name>, it must be ".h5".

-
- See Also**
- ["Introduction to :SAVE Commands"](#) on page 870
 - [":SAVE:FILENAME"](#) on page 873
 - [":SAVE:PWD"](#) on page 883

:SAVE:POWer[:START]

N (see [page 1650](#))

Command Syntax :SAVE:POWer[:START] [<file_name>]

<file_name> ::= quoted ASCII string

The :SAVE:POWer[:START] command saves the power measurement application's current harmonics analysis results to a file.

NOTE

If a file extension is provided as part of a specified <file_name>, it must be ".csv".

-
- See Also**
- ["Introduction to :SAVE Commands"](#) on page 870
 - [":SAVE:FILENAME"](#) on page 873
 - [Chapter 30](#), ":POWer Commands," starting on page 751

:SAVE:PWD

N (see [page 1650](#))

Command Syntax :SAVE:PWD <path_name>
 <path_name> ::= quoted ASCII string

The :SAVE:PWD command sets the present working directory for save operations.

NOTE

Presently, the internal "/User Files" directory you see in the oscilloscope's front panel user interface is the "\Agilent Flash" directory you see in the remote interface.

Query Syntax :SAVE:PWD?

The :SAVE:PWD? query returns the currently set working directory for save operations.

Return Format <path_name><NL>
 <path_name> ::= quoted ASCII string

- See Also**
- ["Introduction to :SAVE Commands"](#) on page 870
 - [":SAVE:FILENAME"](#) on page 873
 - [":RECALL:PWD"](#) on page 863

:SAVE:RESults:[START]

N (see [page 1650](#))

Command Syntax :SAVE:RESults:[START] [<file_spec>]

<file_name> ::= quoted ASCII string

The :SAVE:RESults:[START] command saves analysis results to a comma-separated values (*.csv) file on a USB storage device.

Use the :SAVE:RESults:FORMat commands to specify the analysis types whose results are saved to the file.

When multiple types of analysis results are selected, they are all saved to the same file and separated by a blank line.

- See Also**
- [":SAVE:RESults:FORMat:CURSor"](#) on page 885
 - [":SAVE:RESults:FORMat:MASK"](#) on page 887
 - [":SAVE:RESults:FORMat:MEASurement"](#) on page 888
 - [":SAVE:RESults:FORMat:SEARch"](#) on page 889
 - [":SAVE:RESults:FORMat:SEGmented"](#) on page 890

:SAVE:RESuLts:FORMat:CURSor

N (see [page 1650](#))

Command Syntax :SAVE:RESuLts:FORMat:CURSor {{0 | OFF} | {1 | ON}}

The :SAVE:RESuLts:FORMat:CURSor command specifies whether cursor values will be included when analysis results are saved.

Analysis results are saved using the :SAVE:RESuLts:[START] command.

Other :SAVE:RESuLts:FORMat commands specify whether other types of analysis results are also saved.

When multiple types of analysis results are saved, they are all saved to the same file and separated by a blank line.

Query Syntax :SAVE:RESuLts:FORMat:CURSor?

The :SAVE:RESuLts:FORMat:CURSor? query returns whether cursor values will be included when analysis results are saved.

Return Format <off_on><NL>

{0 | 1}

- See Also**
- [":SAVE:RESuLts:\[START\]"](#) on page 884
 - [":SAVE:RESuLts:FORMat:MASK"](#) on page 887
 - [":SAVE:RESuLts:FORMat:MEASurement"](#) on page 888
 - [":SAVE:RESuLts:FORMat:SEARCh"](#) on page 889
 - [":SAVE:RESuLts:FORMat:SEGMENTed"](#) on page 890

:SAVE:RESuLts:FORMat:HISTogram

N (see [page 1650](#))

Command Syntax :SAVE:RESuLts:FORMat:HISTogram {{0 | OFF} | {1 | ON}}

The :SAVE:RESuLts:FORMat:HISTogram command specifies whether histogram statistics will be included when analysis results are saved.

Analysis results are saved using the :SAVE:RESuLts:[START] command.

Other :SAVE:RESuLts:FORMat commands specify whether other types of analysis results are also saved.

When multiple types of analysis results are saved, they are all saved to the same file and separated by a blank line.

Query Syntax :SAVE:RESuLts:FORMat:HISTogram?

The :SAVE:RESuLts:FORMat:HISTogram? query returns whether histogram statistics will be included when analysis results are saved.

Return Format <off_on><NL>
{0 | 1}

- See Also**
- [":SAVE:RESuLts:\[START\]"](#) on page 884
 - [":SAVE:RESuLts:FORMat:CURSor"](#) on page 885
 - [":SAVE:RESuLts:FORMat:MASK"](#) on page 887
 - [":SAVE:RESuLts:FORMat:MEASurement"](#) on page 888
 - [":SAVE:RESuLts:FORMat:SEARch"](#) on page 889
 - [":SAVE:RESuLts:FORMat:SEGMENTed"](#) on page 890

:SAVE:RESuLts:FORMat:MASK

N (see [page 1650](#))

Command Syntax :SAVE:RESuLts:FORMat:MASK {{0 | OFF} | {1 | ON}}

The :SAVE:RESuLts:FORMat:MASK command specifies whether mask statistics will be included when analysis results are saved.

Analysis results are saved using the :SAVE:RESuLts:[START] command.

Other :SAVE:RESuLts:FORMat commands specify whether other types of analysis results are also saved.

When multiple types of analysis results are saved, they are all saved to the same file and separated by a blank line.

Query Syntax :SAVE:RESuLts:FORMat:MASK?

The :SAVE:RESuLts:FORMat:MASK? query returns whether mask statistics will be included when analysis results are saved.

Return Format <off_on><NL>

{0 | 1}

- See Also**
- [":SAVE:RESuLts:\[START\]"](#) on page 884
 - [":SAVE:RESuLts:FORMat:CURSor"](#) on page 885
 - [":SAVE:RESuLts:FORMat:MEASurement"](#) on page 888
 - [":SAVE:RESuLts:FORMat:SEARch"](#) on page 889
 - [":SAVE:RESuLts:FORMat:SEGMENTed"](#) on page 890

:SAVE:RESuLts:FORMat:MEASurement

N (see [page 1650](#))

Command Syntax :SAVE:RESuLts:FORMat:MEASurement {{0 | OFF} | {1 | ON}}

The :SAVE:RESuLts:FORMat:MEASurement command specifies whether measurement results will be included when analysis results are saved.

Analysis results are saved using the :SAVE:RESuLts:[START] command.

Other :SAVE:RESuLts:FORMat commands specify whether other types of analysis results are also saved.

When multiple types of analysis results are saved, they are all saved to the same file and separated by a blank line.

Query Syntax :SAVE:RESuLts:FORMat:MEASurement?

The :SAVE:RESuLts:FORMat:MEASurement? query returns whether measurement results will be included when analysis results are saved.

Return Format <off_on><NL>
{0 | 1}

- See Also**
- [":SAVE:RESuLts:\[START\]"](#) on page 884
 - [":SAVE:RESuLts:FORMat:CURSor"](#) on page 885
 - [":SAVE:RESuLts:FORMat:MASK"](#) on page 887
 - [":SAVE:RESuLts:FORMat:SEARCh"](#) on page 889
 - [":SAVE:RESuLts:FORMat:SEGMented"](#) on page 890

:SAVE:RESuLts:FORMat:SEARch

N (see [page 1650](#))

Command Syntax :SAVE:RESuLts:FORMat:SEARch {{0 | OFF} | {1 | ON}}

The :SAVE:RESuLts:FORMat:SEARch command specifies whether found search event times will be included when analysis results are saved.

Analysis results are saved using the :SAVE:RESuLts:[START] command.

Other :SAVE:RESuLts:FORMat commands specify whether other types of analysis results are also saved.

When multiple types of analysis results are saved, they are all saved to the same file and separated by a blank line.

Query Syntax :SAVE:RESuLts:FORMat:SEARch?

The :SAVE:RESuLts:FORMat:SEARch? query returns whether found search event times will be included when analysis results are saved.

Return Format <off_on><NL>

{0 | 1}

- See Also**
- [":SAVE:RESuLts:\[START\]"](#) on page 884
 - [":SAVE:RESuLts:FORMat:CURSOr"](#) on page 885
 - [":SAVE:RESuLts:FORMat:MASK"](#) on page 887
 - [":SAVE:RESuLts:FORMat:MEASurement"](#) on page 888
 - [":SAVE:RESuLts:FORMat:SEGMENTed"](#) on page 890

:SAVE:RESuLts:FORMat:SEGmented

N (see [page 1650](#))

Command Syntax :SAVE:RESuLts:FORMat:SEGmented {{0 | OFF} | {1 | ON}}

The :SAVE:RESuLts:FORMat:SEGmented command specifies whether segmented memory acquisition times will be included when analysis results are saved.

Analysis results are saved using the :SAVE:RESuLts:[START] command.

Other :SAVE:RESuLts:FORMat commands specify whether other types of analysis results are also saved.

When multiple types of analysis results are saved, they are all saved to the same file and separated by a blank line.

Query Syntax :SAVE:RESuLts:FORMat:SEGmented?

The :SAVE:RESuLts:FORMat:SEGmented? query returns whether segmented memory acquisition times will be included when analysis results are saved.

Return Format <off_on><NL>
{0 | 1}

- See Also**
- [":SAVE:RESuLts:\[START\]"](#) on page 884
 - [":SAVE:RESuLts:FORMat:CURSor"](#) on page 885
 - [":SAVE:RESuLts:FORMat:MASK"](#) on page 887
 - [":SAVE:RESuLts:FORMat:MEASurement"](#) on page 888
 - [":SAVE:RESuLts:FORMat:SEARch"](#) on page 889

:SAVE[:SETup[:START]]

N (see [page 1650](#))

Command Syntax :SAVE[:SETup[:START]] [<file_spec>]
 <file_spec> ::= {<internal_loc> | <file_name>}
 <internal_loc> ::= 0-9; an integer in NR1 format
 <file_name> ::= quoted ASCII string

The :SAVE[:SETup[:START]] command saves an oscilloscope setup.

NOTE

If a file extension is provided as part of a specified <file_name>, it must be ".scp".

-
- See Also**
- ["Introduction to :SAVE Commands"](#) on page 870
 - [":SAVE:FILENAME"](#) on page 873
 - [":SAVE:PWD"](#) on page 883
 - [":RECall:SETup\[:START\]"](#) on page 864

:SAVE:WAVEform[:START]

N (see [page 1650](#))

Command Syntax :SAVE:WAVEform[:START] [<file_name>]

<file_name> ::= quoted ASCII string

The :SAVE:WAVEform[:START] command saves oscilloscope waveform data to a file.

NOTE

Be sure to set the :SAVE:WAVEform:FORMat before saving waveform data. If the format is NONE, the save waveform command will not succeed.

NOTE

If a file extension is provided as part of a specified <file_name>, and it does not match the extension expected by the format specified in :SAVE:WAVEform:FORMat, the format will be changed if the extension is a valid waveform file extension.

- See Also**
- ["Introduction to :SAVE Commands"](#) on page 870
 - [":SAVE:WAVEform:FORMat"](#) on page 893
 - [":SAVE:WAVEform:LENGth"](#) on page 894
 - [":SAVE:FILEname"](#) on page 873
 - [":RECall:SETup\[:START\]"](#) on page 864

:SAVE:WAVEform:FORMat

N (see [page 1650](#))

Command Syntax :SAVE:WAVEform:FORMat <format>

<format> ::= {ASCIixy | CSV | BINary}

The :SAVE:WAVEform:FORMat command sets the waveform data format type:

- ASCIixy – creates comma-separated value files for each analog channel that is displayed (turned on). The proper file extension for this format is ".csv".
- CSV – creates one comma-separated value file that contains information for all analog channels that are displayed (turned on). The proper file extension for this format is ".csv".
- BINary – creates an oscilloscope binary data format file. See the *User's Guide* for a description of this format. The proper file extension for this format is ".bin".

Query Syntax :SAVE:WAVEform:FORMat?

The :SAVE:WAVEform:FORMat? query returns the selected waveform data format type.

Return Format <format><NL>

<format> ::= {ASC | CSV | BIN | NONE}

When NONE is returned, it indicates that an image file format is currently selected.

- See Also**
- ["Introduction to :SAVE Commands"](#) on page 870
 - [":SAVE:WAVEform\[:START\]"](#) on page 892
 - [":SAVE:WAVEform:LENGth"](#) on page 894
 - [":SAVE:IMAGe:FORMat"](#) on page 876

:SAVE:WAVEform:LENGth

N (see [page 1650](#))

Command Syntax :SAVE:WAVEform:LENGth <length>

<length> ::= 100 to max. length; an integer in NR1 format

When the :SAVE:WAVEform:LENGth:MAX setting is OFF, the :SAVE:WAVEform:LENGth command sets the waveform data length (that is, the number of points saved).

When the :SAVE:WAVEform:LENGth:MAX setting is ON, the :SAVE:WAVEform:LENGth setting has no effect.

Query Syntax :SAVE:WAVEform:LENGth?

The :SAVE:WAVEform:LENGth? query returns the current waveform data length setting.

Return Format <length><NL>

<length> ::= 100 to max. length; an integer in NR1 format

- See Also**
- ["Introduction to :SAVE Commands"](#) on page 870
 - [":SAVE:WAVEform:LENGth:MAX"](#) on page 895
 - [":SAVE:WAVEform\[:START\]"](#) on page 892
 - [":WAVEform:POINts"](#) on page 1446
 - [":SAVE:WAVEform:FORMat"](#) on page 893

:SAVE:WAVEform:LENGth:MAX

N (see [page 1650](#))

Command Syntax :SAVE:WAVEform:LENGth:MAX <setting>

<setting> ::= {{OFF | 0} | {ON | 1}}

The :SAVE:WAVEform:LENGth:MAX command specifies whether maximum number of waveform data points is saved.

When OFF, the :SAVE:WAVEform:LENGth command specifies the number of waveform data points saved.

Query Syntax :SAVE:WAVEform:LENGth:MAX?

The :SAVE:WAVEform:LENGth:MAX? query returns the current setting.

Return Format <setting><NL>

<setting> ::= {0 | 1}

- See Also**
- ["Introduction to :SAVE Commands"](#) on page 870
 - [":SAVE:WAVEform\[:START\]"](#) on page 892
 - [":SAVE:WAVEform:LENGth"](#) on page 894

:SAVE:WAVEform:SEGMented

N (see [page 1650](#))

Command Syntax :SAVE:WAVEform:SEGMented <option>

<option> ::= {ALL | CURRent}

When segmented memory is used for acquisitions, the :SAVE:WAVEform:SEGMented command specifies which segments are included when the waveform is saved:

- ALL – all acquired segments are saved.
- CURRent – only the currently selected segment is saved.

Query Syntax :SAVE:WAVEform:SEGMented?

The :SAVE:WAVEform:SEGMented? query returns the current segmented waveform save option setting.

Return Format <option><NL>

<option> ::= {ALL | CURR}

- See Also**
- ["Introduction to :SAVE Commands"](#) on page 870
 - [":SAVE:WAVEform\[:START\]"](#) on page 892
 - [":SAVE:WAVEform:FORMat"](#) on page 893
 - [":SAVE:WAVEform:LENGth"](#) on page 894

:SAVE:WMEemory:SOURce

N (see [page 1650](#))

Command Syntax :SAVE:WMEemory:SOURce <source>

<source> ::= {CHANnel<n> | FUNCTION<m> | MATH<m> | WMEemory<r>}

<n> ::= 1 to (# analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :SAVE:WMEemory:SOURce command selects the source to be saved as a reference waveform file.

NOTE

Only ADD or SUBtract math operations can be saved as reference waveforms.

NOTE

MATH is an alias for FUNCTION. The query will return FUNC if the source is FUNCTION or MATH.

Query Syntax :SAVE:WMEemory:SOURce?

The :SAVE:WMEemory:SOURce? query returns the source to be saved as a reference waveform file.

Return Format <source><NL>

<source> ::= {CHAN<n> | FUNC | WMEM<r> | NONE}

- See Also**
- ["Introduction to :SAVE Commands"](#) on page 870
 - [":SAVE:WMEemory\[:START\]"](#) on page 898
 - [":RECall:WMEemory<r>\[:START\]"](#) on page 865

:SAVE:WMEemory[:START]

N (see [page 1650](#))

Command Syntax :SAVE:WMEemory[:START] [<file_name>]

<file_name> ::= quoted ASCII string

The :SAVE:WMEemory[:START] command saves oscilloscope waveform data to a reference waveform file.

NOTE

If a file extension is provided as part of a specified <file_name>, it must be ".h5".

-
- See Also**
- ["Introduction to :SAVE Commands"](#) on page 870
 - [":SAVE:WMEemory:SOURce"](#) on page 897
 - [":RECall:WMEemory<r>\[:START\]"](#) on page 865

33 :SBUS<n> Commands

Control the modes and parameters for each serial bus decode/trigger type. See:

- **"Introduction to :SBUS<n> Commands"** on page 899
- **"General :SBUS<n> Commands"** on page 901
- **":SBUS<n>:A429 Commands"** on page 904
- **":SBUS<n>:CAN Commands"** on page 923
- **":SBUS<n>:CXPI Commands"** on page 953
- **":SBUS<n>:FLEXray Commands"** on page 970
- **":SBUS<n>:I2S Commands"** on page 989
- **":SBUS<n>:IIC Commands"** on page 1008
- **":SBUS<n>:LIN Commands"** on page 1019
- **":SBUS<n>:M1553 Commands"** on page 1038
- **":SBUS<n>:MANchester Commands"** on page 1045
- **":SBUS<n>:NRZ Commands"** on page 1064
- **":SBUS<n>:SENT Commands"** on page 1083
- **":SBUS<n>:SPI Commands"** on page 1119
- **":SBUS<n>:UART Commands"** on page 1136
- **":SBUS<n>:USB Commands"** on page 1158
- **":SBUS<n>:USBPd Commands"** on page 1183

Introduction to :SBUS<n> Commands

The :SBUS subsystem commands control the serial decode bus viewing, mode, and other options.

NOTE

These commands are only valid on oscilloscope models when a serial decode option has been licensed.

The following serial bus decode/trigger types are available (see **":TRIGger:MODE"** on page 1358).

- **CAN (Controller Area Network) triggering** – will trigger on CAN version 2.0A and 2.0B signals. Setup consists of connecting the oscilloscope to a CAN signal. Baud rate, signal source, and signal polarity, and type of data to trigger on can be specified. You can trigger on CAN data and identifier patterns and you can set the bit sample point.
- **CXPI triggering** – lets you trigger on CXPI serial data.
- **I2S (Inter-IC Sound or Integrated Interchip Sound bus) triggering** – consists of connecting the oscilloscope to the serial clock, word select, and serial data lines, then triggering on a data value.
- **IIC (Inter-IC bus) triggering** – consists of connecting the oscilloscope to the serial data (SDA) line and the serial clock (SCL) line, then triggering on a stop/start condition, a restart, a missing acknowledge, or on a read/write frame with a specific device address and data value.
- **LIN (Local Interconnect Network) triggering** – will trigger on LIN sync break at the beginning of a message frame. You can trigger on Sync Break, Frame IDs, or Frame IDs and Data.
- **SPI (Serial Peripheral Interface) triggering** – consists of connecting the oscilloscope to a clock, data (MOSI or MISO), and framing signal. You can then trigger on a data pattern during a specific framing period. The serial data string can be specified to be from 4 to 64 bits long.
- **UART/RS-232 triggering** – lets you trigger on RS-232 serial data.
- **SENT triggering** – lets you trigger on SENT serial data.
- **USB 2.0 triggering** – lets you trigger on USB 2.0 serial data.
- **USB PD triggering** – lets you trigger on USB PD serial data.

NOTE

Two I2S buses or two SPI buses cannot be decoded on both SBUS1 and SBUS2 at the same time.

Reporting the Setup

Use :SBUS<n>? to query setup information for the :SBUS<n> subsystem.

Return Format

The following is a sample response from the :SBUS1? query. In this case, the query was issued following a *RST command.

```
:SBUS1:DISP 0;MODE IIC;:SBUS1:IIC:ASIZ BIT7;:SBUS1:IIC:TRIG:TYPE
STAR;QUAL EQU;:SBUS1:IIC:SOUR:CLOC CHAN1;DATA
CHAN2;:SBUS1:IIC:TRIG:PATT:ADDR -1;DATA -1;DATA2 -1
```


General :SBUS<n> Commands

Table 120 General :SBUS<n> Commands Summary

Command	Query	Options and Query Returns
:SBUS<n>:DISPlay { {0 OFF} {1 ON} } (see page 902)	:SBUS<n>:DISPlay? (see page 902)	{0 1}
:SBUS<n>:MODE <mode> (see page 903)	:SBUS<n>:MODE? (see page 903)	<mode> ::= {A429 CAN CXPI FLEXray I2S IIC LIN M1553 MANchester NRZ SENT SPI UART USB USBPd}

:SBUS<n>:DISPlay

N (see [page 1650](#))

Command Syntax :SBUS<n>:DISPlay <display>
<display> ::= {{1 | ON} | {0 | OFF}}

The :SBUS<n>:DISPlay command turns displaying of the serial decode bus on or off.

NOTE

This command is only valid when a serial decode option has been licensed.

NOTE

Two I2S buses or two SPI buses cannot be decoded on both SBUS1 and SBUS2 at the same time.

Query Syntax :SBUS<n>:DISPlay?

The :SBUS<n>:DISPlay? query returns the current display setting of the serial decode bus.

Return Format <display><NL>
<display> ::= {0 | 1}

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • ["Introduction to :SBUS<n> Commands"](#) on page 899
• [":CHANnel<n>:DISPlay"](#) on page 332
• [":DIGital<d>:DISPlay"](#) on page 385
• [":POD<n>:DISPlay"](#) on page 747
• [":VIEW"](#) on page 280
• [":BLANK"](#) on page 248
• [":STATus"](#) on page 277

:SBUS<n>:MODE

N (see [page 1650](#))

Command Syntax :SBUS<n>:MODE <mode>

<mode> ::= {A429 | FLEXray | CAN | CXPI | I2S | IIC | LIN | M1553
| MANchester | NRZ | SENT | SPI | UART | USB | USBPd}

The :SBUS<n>:MODE command determines the decode mode for the serial bus.

NOTE

This command is only valid when a serial decode option has been licensed.

Query Syntax :SBUS<n>:MODE?

The :SBUS<n>:MODE? query returns the current serial bus decode mode setting.

Return Format <mode><NL>

<mode> ::= {A429 | FLEX | CAN | CXPI | I2S | IIC | LIN | M1553 | MANC
| NRZ | SENT | SPI | UART | USB | USBP | NONE}

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • ["Introduction to :SBUS<n> Commands"](#) on page 899

- [":SBUS<n>:A429 Commands"](#) on page 904
- [":SBUS<n>:CAN Commands"](#) on page 923
- [":SBUS<n>:FLEXray Commands"](#) on page 970
- [":SBUS<n>:I2S Commands"](#) on page 989
- [":SBUS<n>:IIC Commands"](#) on page 1008
- [":SBUS<n>:LIN Commands"](#) on page 1019
- [":SBUS<n>:M1553 Commands"](#) on page 1038
- [":SBUS<n>:SENT Commands"](#) on page 1083
- [":SBUS<n>:SPI Commands"](#) on page 1119
- [":SBUS<n>:UART Commands"](#) on page 1136
- [":SBUS<n>:USB Commands"](#) on page 1158

:SBUS<n>:A429 Commands

NOTE

These commands are valid when the MIL-STD-1553 and ARINC 429 triggering and serial decode license has been enabled.

Table 121 :SBUS<n>:A429 Commands Summary

Command	Query	Options and Query Returns
:SBUS<n>:A429:AUTOset up (see page 906)	n/a	n/a
:SBUS<n>:A429:BASE <base> (see page 907)	:SBUS<n>:A429:BASE? (see page 907)	<base> ::= {BINary HEX}
:SBUS<n>:A429:BAUDrate <baudrate> (see page 908)	:SBUS<n>:A429:BAUDrate? (see page 908)	<baudrate> ::= integer from 10000 to 1000000
n/a	:SBUS<n>:A429:COUNT:ERRor? (see page 909)	<error_count> ::= integer in NR1 format
:SBUS<n>:A429:COUNT:RESet (see page 910)	n/a	n/a
n/a	:SBUS<n>:A429:COUNT:WORD? (see page 911)	<word_count> ::= integer in NR1 format
:SBUS<n>:A429:FORMat <format> (see page 912)	:SBUS<n>:A429:FORMat? (see page 912)	<format> ::= {LDSDi LDSSm LDATa}
:SBUS<n>:A429:SIGNal <signal> (see page 913)	:SBUS<n>:A429:SIGNal? (see page 913)	<signal> ::= {A B DIFFerential}
:SBUS<n>:A429:SOURce <source> (see page 914)	:SBUS<n>:A429:SOURce? (see page 914)	<source> ::= {CHANnel<n>} <n> ::= 1 to (# analog channels) in NR1 format
:SBUS<n>:A429:SPEed <speed> (see page 915)	:SBUS<n>:A429:SPEed? (see page 915)	<speed> ::= {LOW HIGH USER}

Table 121 :SBUS<n>:A429 Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:A429:TRIGger :LABel <value> (see page 916)	:SBUS<n>:A429:TRIGger :LABel? (see page 916)	<value> ::= 8-bit integer in decimal, <hex>, <octal>, or <string> from 0-255 or "0xXX" (don't care) <hex> ::= #Hnn where n ::= {0,...,9 A,...,F} <octal> ::= #Qnnn where n ::= {0,...,7} <string> ::= "0xnn" where n ::= {0,...,9 A,...,F}
:SBUS<n>:A429:TRIGger :PATtern:DATA <string> (see page 917)	:SBUS<n>:A429:TRIGger :PATtern:DATA? (see page 917)	<string> ::= "nn...n" where n ::= {0 1 X}, length depends on FORMat
:SBUS<n>:A429:TRIGger :PATtern:SDI <string> (see page 918)	:SBUS<n>:A429:TRIGger :PATtern:SDI? (see page 918)	<string> ::= "nn" where n ::= {0 1 X}, length always 2 bits
:SBUS<n>:A429:TRIGger :PATtern:SSM <string> (see page 919)	:SBUS<n>:A429:TRIGger :PATtern:SSM? (see page 919)	<string> ::= "nn" where n ::= {0 1 X}, length always 2 bits
:SBUS<n>:A429:TRIGger :RANGe <min>, <max> (see page 920)	:SBUS<n>:A429:TRIGger :RANGe? (see page 920)	<min> ::= 8-bit integer in decimal, <hex>, <octal>, or <string> from 0-255 <max> ::= 8-bit integer in decimal, <hex>, <octal>, or <string> from 0-255 <hex> ::= #Hnn where n ::= {0,...,9 A,...,F} <octal> ::= #Qnnn where n ::= {0,...,7} <string> ::= "0xnn" where n ::= {0,...,9 A,...,F}
:SBUS<n>:A429:TRIGger :TYPE <condition> (see page 921)	:SBUS<n>:A429:TRIGger :TYPE? (see page 921)	<condition> ::= {WSTArt WSTOp LABel LBITs PERror WERRor GERRor WGERrors ALLerrors LRANGe ABITs AOBits AZBits}

:SBUS<n>:A429:AUTosetup

N (see [page 1650](#))

Command Syntax :SBUS<n>:A429:AUTosetup

The :SBUS<n>:A429:AUTosetup command automatically sets these options for decoding and triggering on ARINC 429 signals:

- High Trigger Threshold: 3.0 V.
- Low Trigger Threshold: -3.0 V.
- Noise Reject: Off.
- Probe Attenuation: 10.0.
- Vertical Scale: 4 V/div.
- Serial Decode: On.
- Base (:SBUS<n>:A429:BASE): HEX.
- Word Format (:SBUS<n>:A429:FORMat): LDSDi (Label/SDI/Data/SSM).
- Trigger: the specified serial bus (n of SBUS<n>).
- Trigger Mode (:SBUS<n>:A429:TRIGger:TYPE): WSTArt.

Errors • ["-241, Hardware missing"](#) on page 1591

- See Also**
- [":SBUS<n>:A429:BASE"](#) on page 907
 - [":SBUS<n>:A429:FORMat"](#) on page 912
 - [":SBUS<n>:A429:TRIGger:TYPE"](#) on page 921
 - ["Introduction to :SBUS<n> Commands"](#) on page 899
 - [":SBUS<n>:MODE"](#) on page 903
 - [":SBUS<n>:A429 Commands"](#) on page 904

:SBUS<n>:A429:BASE

N (see [page 1650](#))

Command Syntax :SBUS<n>:A429:BASE <base>

<base> ::= {BINary | HEX}

The :SBUS<n>:A429:BASE command selects between hexadecimal and binary display of the decoded data.

The BASE command has no effect on the SDI and SSM fields, which are always displayed in binary, nor the Label field, which is always displayed in octal.

Query Syntax :SBUS<n>:A429:BASE?

The :SBUS<n>:A429:BASE? query returns the current ARINC 429 base setting.

Return Format <base><NL>

<base> ::= {BIN | HEX}

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • ["Introduction to :SBUS<n> Commands"](#) on page 899
 • [":SBUS<n>:MODE"](#) on page 903
 • [":SBUS<n>:A429:FORMat"](#) on page 912

:SBUS<n>:A429:BAUDrate

N (see [page 1650](#))

Command Syntax :SBUS<n>:A429:BAUDrate <baudrate>

<baudrate> ::= integer from 10000 to 1000000

When a user-defined baud rate is selected (with the ":SBUS<n>:A429:SPEEd USER" command), the :SBUS<n>:A429:BAUDrate command specifies the user-defined baud rate. The baud rate can be set in 100 b/s increments between 10000 and 100000 and in 1000 b/s increments between 100000 and 1000000.

Query Syntax :SBUS<n>:A429:BAUDrate?

The :SBUS<n>:A429:BAUDrate? query returns the user-defined baud rate setting.

Return Format <baudrate><NL>

See Also • [":SBUS<n>:A429:SPEEd"](#) on page 915

:SBUS<n>:A429:COUNT:ERRor

N (see [page 1650](#))

Query Syntax :SBUS<n>:A429:COUNT:ERRor?

Returns the error count.

Return Format <error_count><NL>

<error_count> ::= integer in NR1 format

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • [":SBUS<n>:A429:COUNT:RESet"](#) on page 910
 • [":SBUS<n>:A429:COUNT:WORD"](#) on page 911
 • ["Introduction to :SBUS<n> Commands"](#) on page 899
 • [":SBUS<n>:MODE"](#) on page 903
 • [":SBUS<n>:A429 Commands"](#) on page 904

:SBUS<n>:A429:COUNT:RESet

N (see [page 1650](#))

Command Syntax :SBUS<n>:A429:COUNT:RESet

Resets the word and error counters.

- Errors**
- ["-241, Hardware missing"](#) on page 1591
- See Also**
- [":SBUS<n>:A429:COUNT:WORD"](#) on page 911
 - [":SBUS<n>:A429:COUNT:ERRor"](#) on page 909
 - ["Introduction to :SBUS<n> Commands"](#) on page 899
 - [":SBUS<n>:MODE"](#) on page 903
 - [":SBUS<n>:A429 Commands"](#) on page 904

:SBUS<n>:A429:COUNT:WORD

N (see [page 1650](#))

Query Syntax :SBUS<n>:A429:COUNT:WORD?

Returns the word count.

Return Format <word_count><NL>

<word_count> ::= integer in NR1 format

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • [":SBUS<n>:A429:COUNT:RESet"](#) on page 910
 • [":SBUS<n>:A429:COUNT:ERRor"](#) on page 909
 • ["Introduction to :SBUS<n> Commands"](#) on page 899
 • [":SBUS<n>:MODE"](#) on page 903
 • [":SBUS<n>:A429 Commands"](#) on page 904

:SBUS<n>:A429:FORMat

N (see [page 1650](#))

Command Syntax :SBUS<n>:A429:FORMat <format>

<format> ::= {LDSDi | LDSSm | LDATa}

The :SBUS<n>:A429:FORMat command specifies the word decode format:

- LDSDi:
 - Label – 8 bits.
 - SDI – 2 bits.
 - Data – 19 bits.
 - SSM – 2 bits.
- LDSSm:
 - Label – 8 bits.
 - Data – 21 bits.
 - SSM – 2 bits.
- LDATa:
 - Label – 8 bits.
 - Data – 23 bits.

Query Syntax :SBUS<n>:A429:FORMat?

The :SBUS<n>:A429:FORMat? query returns the current ARINC 429 word decode format setting.

Return Format <format><NL>

<format> ::= {LDSD | LDSS | LDAT}

Errors • ["-241, Hardware missing"](#) on page 1591

- See Also**
- ["Introduction to :SBUS<n> Commands"](#) on page 899
 - [":SBUS<n>:MODE"](#) on page 903
 - [":SBUS<n>:A429:TRIGger:PATtern:DATA"](#) on page 917
 - [":SBUS<n>:A429:TRIGger:PATtern:SDI"](#) on page 918
 - [":SBUS<n>:A429:TRIGger:PATtern:SSM"](#) on page 919
 - [":SBUS<n>:A429:TRIGger:TYPE"](#) on page 921
 - [":SBUS<n>:A429:SIGNal"](#) on page 913
 - [":SBUS<n>:A429:SPEed"](#) on page 915
 - [":SBUS<n>:A429:BASE"](#) on page 907
 - [":SBUS<n>:A429:SOURce"](#) on page 914

:SBUS<n>:A429:SIGNal

N (see [page 1650](#))

Command Syntax :SBUS<n>:A429:SIGNal <signal>

<signal> ::= {A | B | DIFFerential}

The :SBUS<n>:A429:SIGNal command specifies the signal type:

- A – Line A (non-inverted).
- B – Line B (inverted).
- DIFFerential – Differential (A-B).

Query Syntax :SBUS<n>:A429:SIGNal?

The :SBUS<n>:A429:SIGNal? query returns the current ARINC 429 signal type setting.

Return Format <signal><NL>

<signal> ::= {A | B | DIFF}

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • ["Introduction to :SBUS<n> Commands"](#) on page 899

- [":SBUS<n>:MODE"](#) on page 903
- [":SBUS<n>:A429:FORMat"](#) on page 912
- [":SBUS<n>:A429:SPEed"](#) on page 915
- [":SBUS<n>:A429:SOURce"](#) on page 914

:SBUS<n>:A429:SOURce

N (see [page 1650](#))

Command Syntax :SBUS<n>:A429:SOURce <source>

<source> ::= {CHANnel<n>}

<n> ::= 1 to (# analog channels) in NR1 format

The :SBUS<n>:A429:SOURce command sets the source of the ARINC 429 signal.

Query Syntax :SBUS<n>:A429:SOURce?

The :SBUS<n>:A429:SOURce? query returns the currently set source of the ARINC 429 signal.

Use the :TRIGger:LEVel:HIGH and :TRIGger:LEVel:LOW commands to set the threshold levels for the selected source.

Return Format <source><NL>

- See Also**
- [":TRIGger:LEVel:HIGH"](#) on page 1356
 - [":TRIGger:LEVel:LOW"](#) on page 1357
 - [":TRIGger:MODE"](#) on page 1358
 - [":SBUS<n>:MODE"](#) on page 903
 - [":SBUS<n>:A429:TRIGger:TYPE"](#) on page 921
 - [":SBUS<n>:A429:SIGNal"](#) on page 913
 - [":SBUS<n>:A429:SPEed"](#) on page 915
 - [":SBUS<n>:A429:FORMat"](#) on page 912
 - ["Introduction to :TRIGger Commands"](#) on page 1345

:SBUS<n>:A429:SPEEd

N (see [page 1650](#))

Command Syntax :SBUS<n>:A429:SPEEd <speed>
 <speed> ::= {LOW | HIGH | USER}

The :SBUS<n>:A429:SPEEd command specifies the signal speed:

- LOW – 12.5 kb/s.
- HIGH – 100 kb/s.
- USER – lets you specify a user-defined baud rate using the :SBUS<n>:A429:BAUDRate command.

Query Syntax :SBUS<n>:A429:SPEEd?

The :SBUS<n>:A429:SPEEd? query returns the current ARINC 429 signal speed setting.

Return Format <speed><NL>
 <speed> ::= {LOW | HIGH | USER}

Errors • ["-241, Hardware missing"](#) on page 1591

- See Also**
- [":SBUS<n>:A429:BAUDRate"](#) on page 908
 - ["Introduction to :SBUS<n> Commands"](#) on page 899
 - [":SBUS<n>:MODE"](#) on page 903
 - [":SBUS<n>:A429:SIGNal"](#) on page 913
 - [":SBUS<n>:A429:FORMat"](#) on page 912
 - [":SBUS<n>:A429:SOURce"](#) on page 914

:SBUS<n>:A429:TRIGger:LABel

N (see [page 1650](#))

Command Syntax :SBUS<n>:A429:TRIGger:LABel <value>

<value> ::= 8-bit integer in decimal, <hex>, <octal>, or <string>
from 0-255 or "0xXX" (don't care)

<hex> ::= #Hnn where n ::= {0,...,9 | A,...,F}

<octal> ::= #Qnnn where n ::= {0,...,7}

<string> ::= "0xnn" where n ::= {0,...,9 | A,...,F}

The :SBUS<n>:A429:TRIGger:LABel command defines the ARINC 429 label value when labels are used in the selected trigger type.

To set the label value to don't cares (0xXX), set the value to -1.

Query Syntax :SBUS<n>:A429:TRIGger:LABel?

The :SBUS<n>:A429:TRIGger:LABel? query returns the current label value in decimal format.

Return Format <value><NL> in decimal format

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • ["Introduction to :TRIGger Commands"](#) on page 1345
• [":SBUS<n>:A429:TRIGger:TYPE"](#) on page 921

:SBUS<n>:A429:TRIGger:PATtern:DATA

N (see [page 1650](#))

Command Syntax :SBUS<n>:A429:TRIGger:PATtern:DATA <string>
 <string> ::= "nn...n" where n ::= {0 | 1 | X}, length depends on FORMat
 The :SBUS<n>:A429:TRIGger:PATtern:DATA command defines the ARINC 429 data pattern resource according to the string parameter. This pattern controls the data pattern searched for in each ARINC 429 word.

NOTE

If more bits are sent for <string> than specified by the :SBUS<n>:A429:FORMat command, the most significant bits will be truncated.

Query Syntax :SBUS<n>:A429:TRIGger:PATtern:DATA?

The :SBUS<n>:A429:TRIGger:PATtern:DATA? query returns the current settings of the specified ARINC 429 data pattern resource in the binary string format.

Return Format <string><NL> in nondecimal format

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • ["Introduction to :TRIGger Commands"](#) on page 1345
 • [":SBUS<n>:A429:TRIGger:TYPE"](#) on page 921
 • [":SBUS<n>:A429:TRIGger:PATtern:SDI"](#) on page 918
 • [":SBUS<n>:A429:TRIGger:PATtern:SSM"](#) on page 919

:SBUS<n>:A429:TRIGger:PATtern:SDI

N (see [page 1650](#))

Command Syntax :SBUS<n>:A429:TRIGger:PATtern:SDI <string>

<string> ::= "nn" where n ::= {0 | 1 | X}, length always 2 bits

The :SBUS<n>:A429:TRIGger:PATtern:SDI command defines the ARINC 429 two-bit SDI pattern resource according to the string parameter. This pattern controls the SDI pattern searched for in each ARINC 429 word.

The specified SDI is only used if the :SBUS<n>:A429:FORMat includes the SDI field.

Query Syntax :SBUS<n>:A429:TRIGger:PATtern:SDI?

The :SBUS<n>:A429:TRIGger:PATtern:SDI? query returns the current settings of the specified ARINC 429 two-bit SDI pattern resource in the binary string format.

Return Format <string><NL> in nondecimal format

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • ["Introduction to :TRIGger Commands"](#) on page 1345
 • [":SBUS<n>:A429:FORMat"](#) on page 912
 • [":SBUS<n>:A429:TRIGger:TYPE"](#) on page 921
 • [":SBUS<n>:A429:TRIGger:PATtern:DATA"](#) on page 917
 • [":SBUS<n>:A429:TRIGger:PATtern:SSM"](#) on page 919

:SBUS<n>:A429:TRIGger:PATtern:SSM

N (see [page 1650](#))

Command Syntax :SBUS<n>:A429:TRIGger:PATtern:SSM <string>

<string> ::= "nn" where n ::= {0 | 1 | X}, length always 2 bits

The :SBUS<n>:A429:TRIGger:PATtern:SSM command defines the ARINC 429 two-bit SSM pattern resource according to the string parameter. This pattern controls the SSM pattern searched for in each ARINC 429 word.

The specified SSM is only used if the :SBUS<n>:A429:FORMat includes the SSM field.

Query Syntax :SBUS<n>:A429:TRIGger:PATtern:SSM?

The :SBUS<n>:A429:TRIGger:PATtern:SSM? query returns the current settings of the specified ARINC 429 two-bit SSM pattern resource in the binary string format.

Return Format <string><NL> in nondecimal format

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • ["Introduction to :TRIGger Commands"](#) on page 1345
 • [":SBUS<n>:A429:FORMat"](#) on page 912
 • [":SBUS<n>:A429:TRIGger:TYPE"](#) on page 921
 • [":SBUS<n>:A429:TRIGger:PATtern:DATA"](#) on page 917
 • [":SBUS<n>:A429:TRIGger:PATtern:SDI"](#) on page 918

:SBUS<n>:A429:TRIGger:RANGe

N (see [page 1650](#))

Command Syntax :SBUS<n>:A429:TRIGger:RANGe <min>,<max>

<min> ::= 8-bit integer in decimal, <hex>, <octal>, or <string>
from 0-255

<max> ::= 8-bit integer in decimal, <hex>, <octal>, or <string>
from 0-255

<hex> ::= #Hnn where n ::= {0,...,9 | A,...,F}

<octal> ::= #Qnnn where n ::= {0,...,7}

<string> ::= "0xnn" where n ::= {0,...,9 | A,...,F}

The :SBUS<n>:A429:TRIGger:RANGe command defines a range of ARINC 429 label values. This range is used when the LRange trigger type is selected.

Query Syntax :SBUS<n>:A429:TRIGger:RANGe?

The :SBUS<n>:A429:TRIGger:RANGe? query returns the current label values in decimal format.

Return Format <min>,<max><NL> in decimal format

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • ["Introduction to :TRIGger Commands"](#) on page 1345
• [":SBUS<n>:A429:TRIGger:TYPE"](#) on page 921

:SBUS<n>:A429:TRIGger:TYPE

N (see [page 1650](#))

Command Syntax :SBUS<n>:A429:TRIGger:TYPE <condition>

```
<condition> ::= {WSTArt | WSTOp | LABel | LBITs | PERRor | WERRor
                  | GERRor | WGERrors | ALLerrors | LRANge | ABITs
                  | AOBits | AZBits}
```

The :SBUS<n>:A429:TRIGger command sets the ARINC 429 trigger on condition:

- WSTArt – triggers on the start of a word.
- WSTOp – triggers at the end of a word.
- LABel – triggers on the specified label value.
- LBITs – triggers on the label and the other word fields as specified.
- LRANge – triggers on a label within a min/max range.
- PERRor – triggers on words with a parity error.
- WERRor – triggers on an intra-word coding error.
- GERRor – triggers on an inter-word gap error.
- WGERrors – triggers on either a Word or Gap Error.
- ALLerrors – triggers on any of the above errors.
- ABITs – triggers on any bit, which will therefore form an eye diagram.
- AZBits – triggers on any bit with a value of zero.
- AOBits – triggers on any bit with a value of one.

Query Syntax :SBUS<n>:A429:TRIGger:TYPE?

The :SBUS<n>:A429:TRIGger:TYPE? query returns the current ARINC 429 trigger on condition.

Return Format <condition><NL>

```
<condition> ::= {WSTA | WSTO | LAB | LBIT | PERR | WERR | GERR | WGER
                  | ALL | LRAN | ABIT | AOB | AZB}
```

Errors • ["-241, Hardware missing"](#) on page 1591

- See Also** • ["Introduction to :SBUS<n> Commands"](#) on page 899
- [":SBUS<n>:MODE"](#) on page 903
 - [":SBUS<n>:A429:TRIGger:LABel"](#) on page 916
 - [":SBUS<n>:A429:TRIGger:PATtern:DATA"](#) on page 917
 - [":SBUS<n>:A429:TRIGger:PATtern:SDI"](#) on page 918
 - [":SBUS<n>:A429:TRIGger:PATtern:SSM"](#) on page 919
 - [":SBUS<n>:A429:TRIGger:RANge"](#) on page 920

- **":SBUS<n>:A429:SOURce"** on page 914

:SBUS<n>:CAN Commands

NOTE

These commands are valid when the CAN and LIN serial decode license has been enabled.

Table 122 :SBUS<n>:CAN Commands Summary

Command	Query	Options and Query Returns
n/a	:SBUS<n>:CAN:COUNT:ER Ror? (see page 926)	<frame_count> ::= integer in NR1 format
n/a	:SBUS<n>:CAN:COUNT:OV ERload? (see page 927)	<frame_count> ::= 0 in NR1 format
:SBUS<n>:CAN:COUNT:RE Set (see page 928)	n/a	n/a
n/a	:SBUS<n>:CAN:COUNT:SP EC? (see page 929)	<spec_error_count> ::= integer in NR1 format
n/a	:SBUS<n>:CAN:COUNT:TO Tal? (see page 930)	<frame_count> ::= integer in NR1 format
n/a	:SBUS<n>:CAN:COUNT:UT ILization? (see page 931)	<percent> ::= floating-point in NR3 format
:SBUS<n>:CAN:DISPlay <type> (see page 932)	:SBUS<n>:CAN:DISPlay? (see page 932)	<type> ::= {HEXadecimal SYMBOLic}
:SBUS<n>:CAN:FDSPoint <value> (see page 933)	:SBUS<n>:CAN:FDSPoint ? (see page 933)	<value> ::= even numbered percentages from 30 to 90 in NR3 format.
:SBUS<n>:CAN:FDSTanda rd <std> (see page 934)	:SBUS<n>:CAN:FDSTanda rd? (see page 934)	<std> ::= {ISO NISO}
:SBUS<n>:CAN:SAMPlapo int <percent> (see page 935)	:SBUS<n>:CAN:SAMPlapo int? (see page 935)	<percent> ::= 30.0 to 90.0 in NR3 format
:SBUS<n>:CAN:SIGNAL:B AUDrate <baudrate> (see page 936)	:SBUS<n>:CAN:SIGNAL:B AUDrate? (see page 936)	<baudrate> ::= integer from 10000 to 4000000 in 100 b/s increments, or 5000000
:SBUS<n>:CAN:SIGNAL:D EFinition <value> (see page 937)	:SBUS<n>:CAN:SIGNAL:D EFinition? (see page 937)	<value> ::= {CANH CANL RX TX DIFFerential DIFL DIFH}
:SBUS<n>:CAN:SIGNAL:F DBaudrate <baudrate> (see page 938)	:SBUS<n>:CAN:SIGNAL:F DBaudrate? (see page 938)	<baudrate> ::= integer from 10000 to 10000000 in 100 b/s increments.

Table 122 :SBUS<n>:CAN Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:CAN:SOURce <source> (see page 939)	:SBUS<n>:CAN:SOURce? (see page 939)	<source> ::= {CHANnel<n> EXTErnal} for DSO models <source> ::= {CHANnel<n> DIGital<d> } for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:SBUS<n>:CAN:TRIGger <condition> (see page 940)	:SBUS<n>:CAN:TRIGger? (see page 941)	<condition> ::= {SOF EOF IDData DATA FDData IDRemote IDEither ERRor ACKerror FORMerror STUFFerror CRCerror SPECerror ALLerrors BRSBit CRCDbit EBActive EBPAssive OVERload MESSage MSIGnal FDMSignal}
:SBUS<n>:CAN:TRIGger:IDFilter {{0 OFF} {1 ON}} (see page 943)	:SBUS<n>:CAN:TRIGger:IDFilter? (see page 943)	{0 1}
:SBUS<n>:CAN:TRIGger:PATtern:DATA <string> (see page 944)	:SBUS<n>:CAN:TRIGger:PATtern:DATA? (see page 944)	<string> ::= "nn...n" where n ::= {0 1 X \$} <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X \$}
:SBUS<n>:CAN:TRIGger:PATtern:DATA:DLC <dlc> (see page 945)	:SBUS<n>:CAN:TRIGger:PATtern:DATA:DLC? (see page 945)	<dlc> ::= integer between -1 (don't care) and 64, in NR1 format.
:SBUS<n>:CAN:TRIGger:PATtern:DATA:LENGth <length> (see page 946)	:SBUS<n>:CAN:TRIGger:PATtern:DATA:LENGth? (see page 946)	<length> ::= integer from 1 to 8 in NR1 format
:SBUS<n>:CAN:TRIGger:PATtern:DATA:START <start> (see page 947)	:SBUS<n>:CAN:TRIGger:PATtern:DATA:START? (see page 947)	<start> ::= integer between 0 and 63, in NR1 format.
:SBUS<n>:CAN:TRIGger:PATtern:ID <string> (see page 948)	:SBUS<n>:CAN:TRIGger:PATtern:ID? (see page 948)	<string> ::= "nn...n" where n ::= {0 1 X \$} <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X \$}
:SBUS<n>:CAN:TRIGger:PATtern:ID:MODE <value> (see page 949)	:SBUS<n>:CAN:TRIGger:PATtern:ID:MODE? (see page 949)	<value> ::= {STANDard EXTended}

Table 122 :SBUS<n>:CAN Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:CAN:TRIGger: SYMBOLic:MESSAge <name> (see page 950)	:SBUS<n>:CAN:TRIGger: SYMBOLic:MESSAge? (see page 950)	<name> ::= quoted ASCII string
:SBUS<n>:CAN:TRIGger: SYMBOLic:SIGNAL <name> (see page 951)	:SBUS<n>:CAN:TRIGger: SYMBOLic:SIGNAL? (see page 951)	<name> ::= quoted ASCII string
:SBUS<n>:CAN:TRIGger: SYMBOLic:VALue <data> (see page 952)	:SBUS<n>:CAN:TRIGger: SYMBOLic:VALue? (see page 952)	<data> ::= value in NR3 format

:SBUS<n>:CAN:COUNT:ERRor

N (see [page 1650](#))

Query Syntax :SBUS<n>:CAN:COUNT:ERRor?

Returns the error frame count.

Return Format <frame_count><NL>

<frame_count> ::= integer in NR1 format

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • [":SBUS<n>:CAN:COUNT:RESet"](#) on page 928
 • ["Introduction to :SBUS<n> Commands"](#) on page 899
 • [":SBUS<n>:MODE"](#) on page 903
 • [":SBUS<n>:CAN Commands"](#) on page 923

:SBUS<n>:CAN:COUNT:OVERload

N (see [page 1650](#))

Query Syntax :SBUS<n>:CAN:COUNT:OVERload?

Returns the overload frame count.

Return Format <frame_count><NL>

<frame_count> ::= 0 in NR1 format

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • [":SBUS<n>:CAN:COUNT:RESet"](#) on page 928
 • ["Introduction to :SBUS<n> Commands"](#) on page 899
 • [":SBUS<n>:MODE"](#) on page 903
 • [":SBUS<n>:CAN Commands"](#) on page 923

:SBUS<n>:CAN:COUNT:RESet

N (see [page 1650](#))

Command Syntax :SBUS<n>:CAN:COUNT:RESet

Resets the frame counters.

- Errors**
- ["-241, Hardware missing"](#) on page 1591
- See Also**
- [":SBUS<n>:CAN:COUNT:ERRor"](#) on page 926
 - [":SBUS<n>:CAN:COUNT:OVERload"](#) on page 927
 - [":SBUS<n>:CAN:COUNT:TOTal"](#) on page 930
 - [":SBUS<n>:CAN:COUNT:UTILization"](#) on page 931
 - ["Introduction to :SBUS<n> Commands"](#) on page 899
 - [":SBUS<n>:MODE"](#) on page 903
 - [":SBUS<n>:CAN Commands"](#) on page 923

:SBUS<n>:CAN:COUNT:SPEC

N (see [page 1650](#))

Query Syntax :SBUS<n>:CAN:COUNT:SPEC?

Returns the Spec error (Ack + Form + Stuff + CRC errors) count.

Return Format <spec_error_count><NL>

<spec_error_count> ::= integer in NR1 format

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • [":SBUS<n>:CAN:COUNT:RESet"](#) on page 928
 • ["Introduction to :SBUS<n> Commands"](#) on page 899
 • [":SBUS<n>:MODE"](#) on page 903
 • [":SBUS<n>:CAN Commands"](#) on page 923

:SBUS<n>:CAN:COUNT:TOTal

N (see [page 1650](#))

Query Syntax :SBUS<n>:CAN:COUNT:TOTal?

Returns the total frame count.

Return Format <frame_count><NL>

<frame_count> ::= integer in NR1 format

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • [":SBUS<n>:CAN:COUNT:RESet"](#) on page 928
 • ["Introduction to :SBUS<n> Commands"](#) on page 899
 • [":SBUS<n>:MODE"](#) on page 903
 • [":SBUS<n>:CAN Commands"](#) on page 923

:SBUS<n>:CAN:COUNT:UTILization

N (see [page 1650](#))

Query Syntax :SBUS<n>:CAN:COUNT:UTILization?

Returns the percent utilization.

Return Format <percent><NL>

<percent> ::= floating-point in NR3 format

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • [":SBUS<n>:CAN:COUNT:RESet"](#) on page 928
 • ["Introduction to :SBUS<n> Commands"](#) on page 899
 • [":SBUS<n>:MODE"](#) on page 903
 • [":SBUS<n>:CAN Commands"](#) on page 923

:SBUS<n>:CAN:DISPlay

N (see [page 1650](#))

Command Syntax :SBUS<n>:CAN:DISPlay <type>
 <type> ::= {HEXadecimal | SYMBolic}

The :SBUS<n>:CAN:DISPlay command specifies, when CAN symbolic data is loaded into the oscilloscope, whether symbolic values (from the DBC file) or hexadecimal values are displayed in the decode waveform and the Lister window.

Query Syntax :SBUS<n>:CAN:DISPlay?

The :SBUS<n>:CAN:DISPlay? query returns the CAN decode display type.

Return Format <type><NL>
 <type> ::= {HEX | SYMB}

See Also • [":RECall:DBC\[:START\]"](#) on page 859

:SBUS<n>:CAN:FDSPoint

N (see [page 1650](#))

Command Syntax :SBUS<n>:CAN:FDSPoint <value>

<value> ::= even numbered percentages from 30 to 90 in NR3 format.

The :SBUS<n>:CAN:FDSPoint command sets the point during the bit time where the bit level is sampled to determine whether the bit is dominant or recessive. The sample point represents the percentage of time between the beginning of the bit time to the end of the bit time.

Query Syntax :SBUS<n>:CAN:FDSPoint?

The :SBUS<n>:CAN:FDSPoint? query returns the current CAN FD sample point setting.

Return Format <value><NL>

<value> ::= even numbered percentages from 30 to 90 in NR3 format.

See Also • [":SBUS<n>:CAN:SIGNal:FDbaudrate"](#) on page 938

:SBUS<n>:CAN:FDStandard

N (see [page 1650](#))

Command Syntax :SBUS<n>:CAN:FDStandard <std>

<std> ::= {ISO | NISO}

The :SBUS<n>:CAN:FDStandard command lets you pick the standard that will be used when decoding or triggering on FD frames, ISO, or non-ISO.

This setting has no effect on the processing of non-FD (classical) frames.

Query Syntax :SBUS<n>:CAN:FDStandard?

The :SBUS<n>:CAN:FDStandard? query returns the selected CAN FD frame decode standard.

Return Format <std><NL>

<std> ::= {ISO | NISO}

See Also • [":SBUS<n>:CAN:FDSPoint"](#) on page 933

:SBUS<n>:CAN:SAMPlEpoint

N (see [page 1650](#))

Command Syntax :SBUS<n>:CAN:SAMPlEpoint <percent>

<percent><NL>

<percent> ::= 30.0 to 90.0 in NR3 format

The :SBUS<n>:CAN:SAMPlEpoint command sets the point during the bit time where the bit level is sampled to determine whether the bit is dominant or recessive. The sample point represents the percentage of time between the beginning of the bit time to the end of the bit time.

Query Syntax :SBUS<n>:CAN:SAMPlEpoint?

The :SBUS<n>:CAN:SAMPlEpoint? query returns the current CAN sample point setting.

Return Format <percent><NL>

<percent> ::= 30.0 to 90.0 in NR3 format

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:MODE"](#) on page 903
 - [":SBUS<n>:CAN:TRIGger"](#) on page 940

:SBUS<n>:CAN:SIGNal:BAUDRate

N (see [page 1650](#))

Command Syntax :SBUS<n>:CAN:SIGNal:BAUDRate <baudrate>

<baudrate> ::= integer from 10000 to 4000000 in 100 b/s increments,
or 5000000

The :SBUS<n>:CAN:SIGNal:BAUDRate command sets the standard baud rate of the CAN signal from 10 kb/s to 4 Mb/s in 100 b/s increments. If you enter a baud rate that is not divisible by 100 b/s, the baud rate is set to the nearest baud rate divisible by 100 b/s.

You can also set the baud rate of the CAN signal to 5 Mb/s. Fractional baud rates between 4 Mb/s and 5 Mb/s are not allowed.

If the baud rate you select does not match the system baud rate, false triggers may occur.

Query Syntax :SBUS<n>:CAN:SIGNal:BAUDRate?

The :SBUS<n>:CAN:SIGNal:BAUDRate? query returns the current CAN baud rate setting.

Return Format <baudrate><NL>

<baudrate> ::= integer from 10000 to 4000000 in 100 b/s increments,
or 5000000

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:MODE"](#) on page 903
 - [":SBUS<n>:CAN:TRIGger"](#) on page 940
 - [":SBUS<n>:CAN:SIGNal:DEFinition"](#) on page 937
 - [":SBUS<n>:CAN:SOURce"](#) on page 939

:SBUS<n>:CAN:SIGNal:DEFinition

N (see [page 1650](#))

Command Syntax :SBUS<n>:CAN:SIGNal:DEFinition <value>

<value> ::= {CANH | CANL | RX | TX | DIFFerential | DIFL | DIFH}

The :SBUS<n>:CAN:SIGNal:DEFinition command sets the CAN signal type when :SBUS<n>:CAN:TRIGger is set to SOF (start of frame). These signals can be set to:

Dominant high signals:

- CANH – the actual CAN_H differential bus signal.
- DIFH – the CAN differential (H-L) bus signal connected to an analog source channel using a differential probe.

Dominant low signals:

- CANL – the actual CAN_L differential bus signal.
- RX – the Receive signal from the CAN bus transceiver.
- TX – the Transmit signal to the CAN bus transceiver.
- DIFL – the CAN differential (L-H) bus signal connected to an analog source channel using a differential probe.
- DIFFerential – the CAN differential bus signal connected to an analog source channel using a differential probe. This is the same as DIFL.

Query Syntax :SBUS<n>:CAN:SIGNal:DEFinition?

The :SBUS<n>:CAN:SIGNal:DEFinition? query returns the current CAN signal type.

Return Format <value><NL>

<value> ::= {CANH | CANL | RX | TX | DIFL | DIFH}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:MODE"](#) on page 903
 - [":SBUS<n>:CAN:SIGNal:BAUDrate"](#) on page 936
 - [":SBUS<n>:CAN:SOURce"](#) on page 939
 - [":SBUS<n>:CAN:TRIGger"](#) on page 940

:SBUS<n>:CAN:SIGNal:FDBaudrate

N (see [page 1650](#))

Command Syntax :SBUS<n>:CAN:SIGNal:FDBaudrate <baudrate>

<baudrate> ::= integer from 10000 to 100000000 in 100 b/s increments.

The :SBUS<n>:CAN:SIGNal:FDBaudrate command sets the CAN FD baud rate from 10 kb/s to 10 Mb/s in 100 b/s increments. If you enter a baud rate that is not divisible by 100 b/s, the baud rate is set to the nearest baud rate divisible by 100 b/s.

For CAN FD, both the standard rate settings (see :SBUS<n>:CAN:SIGNal:BAUDrate) and the FD rate settings must be set correctly; otherwise, false triggers may occur.

Query Syntax :SBUS<n>:CAN:SIGNal:FDBaudrate?

The :SBUS<n>:CAN:SIGNal:FDBaudrate? query returns the current CAN FD baud rate setting.

Return Format <baudrate><NL>

<baudrate> ::= integer from 10000 to 100000000 in 100 b/s increments.

- See Also**
- [":SBUS<n>:CAN:FDSPoint"](#) on page 933
 - [":SBUS<n>:CAN:SIGNal:BAUDrate"](#) on page 936

:SBUS<n>:CAN:SOURce

N (see [page 1650](#))

Command Syntax :SBUS<n>:CAN:SOURce <source>

<source> ::= {CHANnel<n> | EXTernal} for the DSO models

<source> ::= {CHANnel<n> | DIGital<d>} for the MSO models

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :SBUS<n>:CAN:SOURce command sets the source for the CAN signal.

Query Syntax :SBUS<n>:CAN:SOURce?

The :SBUS<n>:CAN:SOURce? query returns the current source for the CAN signal.

Return Format <source><NL>

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:MODE"](#) on page 903
 - [":SBUS<n>:CAN:TRIGger"](#) on page 940
 - [":SBUS<n>:CAN:SIGNAL:DEFinition"](#) on page 937

:SBUS<n>:CAN:TRIGger

N (see [page 1650](#))

Command Syntax :SBUS<n>:CAN:TRIGger <condition>

```
<condition> ::= {SOF | EOF | IDData | DATA | FDData | IDRemote
| IDEither | ERRor | ACKerror | FORMerror | STUFFerror | CRCerror
| SPECerror | ALLerrors | BRSBit | CRCDbit | EBActive | EBPassive
| OVERload | MESSage | MSIGnal | FDMSignal}
```

The :SBUS<n>:CAN:TRIGger command sets the CAN trigger on condition:

Condition	Front-panel name	Description	Filter by ID*
SOF	SOF - Start of Frame	Triggers at the start bit for both data and overload frames.	
EOF	EOF - End of Frame	Triggers at the end of any frame.	X
IDEither	Frame ID	Triggers on any standard CAN (data or remote) or CAN FD frame at the end of the 11- or 29-bit ID field.	
IDData	Data Frame ID (non-FD)	Triggers on standard CAN data frames at the end of the 11- or 29-bit ID field.	
DATA	Data Frame ID and Data (non-FD)	Triggers on any standard CAN data frame at the end of the last data byte defined in the trigger. The DLC of the packet must match the number of bytes specified.	
FDData	Data Frame ID and Data (FD)	triggers on CAN FD frames at the end of the last data byte defined in the trigger. You can trigger on up to 8 bytes of data anywhere within the CAN FD data, which can be up to 64 bytes long.	
IDRemote	Remote Frame ID	Triggers on standard CAN remote frames at the end of the 11- or 29-bit ID field.	
ERRor	Error Frame	Triggers after 6 consecutive 0s while in a data frame, at the EOF.	X
ACKerror	Acknowledge Error	Triggers on the acknowledge bit if the polarity is incorrect.	X
FORMerror	Form Error	Triggers on reserved bit errors.	X
STUFFerror	Stuff Error	Triggers on 6 consecutive 1s or 6 consecutive 0s, while in a non-error or non overload frame.	X

Condition	Front-panel name	Description	Filter by ID*
CRCError	CRC Field Error	Triggers when the calculated CRC does not match the transmitted CRC. In addition, for FD frames, will also trigger if the Stuff Count is in error.	X
SPECError	Spec Error (Ack or Form or Stuff or CRC)	Triggers on Ack, Form, Stuff, or CRC errors.	X
ALLerrors	All Errors	Triggers on all Spec errors and error frames.	X
BRSBit	BRS Bit (FD)	Triggers on the BRS bit of CAN FD frames.	X
CRCDbit	CRC Delimiter Bit (FD)	Triggers on the CRC delimiter bit in CAN FD frames.	X
EBAActive	ESI Bit Active (FD)	Triggers on the ESI bit if set active.	X
EBPassive	ESI Bit Passive (FD)	Triggers on the ESI bit if set passive.	X
OVERload	Overload Frame	Triggers on an overload frame.	
MESSage	Message	Triggers on a symbolic message.	
MSIGnal	Message and Signal (non-FD)	Triggers on a symbolic message and a signal value.	
FDMSignal	Message and Signal (FD, first 8 bytes only)	Triggers on a symbolic message and a signal value, limited to the first 8 bytes of FD data.	
* Filtering by CAN IDs is available for these trigger conditions (see :SBUS<n>:CAN:TRIGger:IDFilter).			

CAN Id specification is set by the :SBUS<n>:CAN:TRIGger:PATtern:ID and :SBUS<n>:CAN:TRIGger:PATtern:ID:MODE commands.

CAN Data specification is set by the :SBUS<n>:CAN:TRIGger:PATtern:DATA command.

CAN Data Length Code is set by the :SBUS<n>:CAN:TRIGger:PATtern:DATA:LENGth command.

Query Syntax :SBUS<n>:CAN:TRIGger?

The :SBUS<n>:CAN:TRIGger? query returns the current CAN trigger on condition.

Return Format <condition><NL>

```
<condition> ::= {SOF | EOF | IDD | DATA | FDD | IDR | IDE | ERR | ACK
                | FORM | STUF | CRC | SPEC | ALL | BRSB | CRCD | EBA | EBP | OVER
                | MESS | MSIG | FDMS}
```

Errors • **"-241, Hardware missing"** on page 1591

See Also • **"Introduction to :SBUS<n> Commands"** on page 899

- **":SBUS<n>:MODE"** on page 903
- **":SBUS<n>:CAN:TRIGger:PATtern:DATA"** on page 944
- **":SBUS<n>:CAN:TRIGger:PATtern:DATA:LENGth"** on page 946
- **":SBUS<n>:CAN:TRIGger:PATtern:ID"** on page 948
- **":SBUS<n>:CAN:TRIGger:PATtern:ID:MODE"** on page 949
- **":SBUS<n>:CAN:TRIGger:IDFilter"** on page 943
- **":SBUS<n>:CAN:SIGNal:DEFinition"** on page 937
- **":SBUS<n>:CAN:SOURce"** on page 939
- **":RECall:DBC[:STARt]"** on page 859
- **":SBUS<n>:CAN:TRIGger:SYMBolic:MESSage"** on page 950
- **":SBUS<n>:CAN:TRIGger:SYMBolic:SIGNal"** on page 951
- **":SBUS<n>:CAN:TRIGger:SYMBolic:VALue"** on page 952

:SBUS<n>:CAN:TRIGger:IDFilter

N (see [page 1650](#))

Command Syntax :SBUS<n>:CAN:TRIGger:IDFilter {{0 | OFF} | {1 | ON}}

The :SBUS<n>:CAN:TRIGger:IDFilter command specifies, in certain error and bit trigger modes, whether triggers are filtered by CAN IDs.

Query Syntax :SBUS<n>:CAN:TRIGger:IDFilter?

The :SBUS<n>:CAN:TRIGger:IDFilter? query returns the CAN trigger ID filter setting.

Return Format <setting><NL>
 <setting> ::= {0 | 1}

See Also • [":SBUS<n>:CAN:TRIGger"](#) on page 940

:SBUS<n>:CAN:TRIGger:PATtern:DATA

N (see [page 1650](#))

Command Syntax :SBUS<n>:CAN:TRIGger:PATtern:DATA <string>

<string> ::= "nn...n" where n ::= {0 | 1 | X | \$}

<string> ::= "0xnn...n" where n ::= {0,...,9 | A,...,F | X | \$}

The :SBUS<n>:CAN:TRIGger:PATtern:DATA command defines the CAN data pattern resource according to the string parameter. This pattern, along with the data length (set by the :SBUS<n>:CAN:TRIGger:PATtern:DATA:LENGth command), control the data pattern searched for in each CAN message.

If the string parameter starts with "0x", it is a hexadecimal string made up of hexadecimal and X (don't care) characters; otherwise, it is a binary string made up of 0, 1, and X (don't care) characters.

NOTE

If more bits are sent for <string> than specified by the :SBUS<n>:CAN:TRIGger:PATtern:DATA:LENGth command, the most significant bits will be truncated. If the data length is changed after the <string> is programmed, the added or deleted bits will be added to or deleted from the least significant bits.

Query Syntax :SBUS<n>:CAN:TRIGger:PATtern:DATA?

The :SBUS<n>:CAN:TRIGger:PATtern:DATA? query returns the current settings of the specified CAN data pattern resource in the binary string format.

Return Format <string><NL> in nondecimal format

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • ["Introduction to :TRIGger Commands"](#) on page 1345
 • [":SBUS<n>:CAN:TRIGger:PATtern:DATA:LENGth"](#) on page 946
 • [":SBUS<n>:CAN:TRIGger:PATtern:ID"](#) on page 948

:SBUS<n>:CAN:TRIGger:PATtern:DATA:DLC

N (see [page 1650](#))

Command Syntax :SBUS<n>:CAN:TRIGger:PATtern:DATA:DLC <dlc>

<dlc> ::= integer between -1 (don't care) and 64, in NR1 format.

The :SBUS<n>:CAN:TRIGger:PATtern:DATA:DLC command specifies the DLC value to be used in the CAN FD data trigger mode. A specific valid FD value can be specified, or -1 can be specified to indicate "don't care".

Query Syntax :SBUS<n>:CAN:TRIGger:PATtern:DATA:START?

The :SBUS<n>:CAN:TRIGger:PATtern:DATA:DLC? query returns the currently set DLC value.

Return Format <dlc><NL>

<dlc> ::= integer between -1 (don't care) and 64, in NR1 format.

See Also • [":SBUS<n>:CAN:TRIGger:PATtern:DATA"](#) on page 944

:SBUS<n>:CAN:TRIGger:PATtern:DATA:LENGth

N (see [page 1650](#))

Command Syntax :SBUS<n>:CAN:TRIGger:PATtern:DATA:LENGth <length>

<length> ::= integer from 1 to 8 in NR1 format

The :SBUS<n>:CAN:TRIGger:PATtern:DATA:LENGth command sets the number of 8-bit bytes in the CAN data string. The number of bytes in the string can be anywhere from 1 bytes to 8 bytes (64 bits). The value for these bytes is set by the :SBUS<n>:CAN:TRIGger:PATtern:DATA command.

Query Syntax :SBUS<n>:CAN:TRIGger:PATtern:DATA:LENGth?

The :SBUS<n>:CAN:TRIGger:PATtern:DATA:LENGth? query returns the current CAN data pattern length setting.

Return Format <count><NL>

<count> ::= integer from 1 to 8 in NR1 format

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • ["Introduction to :TRIGger Commands"](#) on page 1345
 • [":SBUS<n>:CAN:TRIGger:PATtern:DATA"](#) on page 944
 • [":SBUS<n>:CAN:SOURce"](#) on page 939

:SBUS<n>:CAN:TRIGger:PATtern:DATA:START

N (see [page 1650](#))

Command Syntax :SBUS<n>:CAN:TRIGger:PATtern:DATA:START <start>

<start> ::= integer between 0 and 63, in NR1 format.

The :SBUS<n>:CAN:TRIGger:PATtern:DATA:START command specifies the starting byte position for CAN FD data triggers.

CAN FD frames can have up to 64 bytes of data. You can trigger on up to 8 bytes of data. The starting byte position setting lets you trigger on data anywhere within the frame.

Query Syntax :SBUS<n>:CAN:TRIGger:PATtern:DATA:START?

The :SBUS<n>:CAN:TRIGger:PATtern:DATA:START? query returns the starting byte position setting.

Return Format <start><NL>

<start> ::= integer between 0 and 63, in NR1 format.

- See Also**
- [":SBUS<n>:CAN:TRIGger:PATtern:DATA"](#) on page 944
 - [":SBUS<n>:CAN:TRIGger:PATtern:DATA:LENGth"](#) on page 946

:SBUS<n>:CAN:TRIGger:PATtern:ID

N (see [page 1650](#))

Command Syntax :SBUS<n>:CAN:TRIGger:PATtern:ID <string>

<string> ::= "nn...n" where n ::= {0 | 1 | X | \$}

<string> ::= "0xnn...n" where n ::= {0,...,9 | A,...,F | X | \$}

The :SBUS<n>:CAN:TRIGger:PATtern:ID command defines the CAN identifier pattern resource according to the string parameter. This pattern, along with the identifier mode (set by the :SBUS<n>:CAN:TRIGger:PATtern:ID:MODE command), control the identifier pattern searched for in each CAN message.

If the string parameter starts with "0x", it is a hexadecimal string made up of hexadecimal and X (don't care) characters; otherwise, it is a binary string made up of 0, 1, and X (don't care) characters.

NOTE

The ID pattern resource string is always 29 bits. Only 11 of these bits are used when the :SBUS<n>:CAN:TRIGger:PATtern:ID:MODE is STANDard.

A string longer than 29 bits is truncated to 29 bits when setting the ID pattern resource.

Query Syntax :SBUS<n>:CAN:TRIGger:PATtern:ID?

The :SBUS<n>:CAN:TRIGger:PATtern:ID? query returns the current settings of the specified CAN identifier pattern resource in the 29-bit binary string format.

Return Format <string><NL> in 29-bit binary string format

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • ["Introduction to :TRIGger Commands"](#) on page 1345
 • [":SBUS<n>:CAN:TRIGger:PATtern:ID:MODE"](#) on page 949
 • [":SBUS<n>:CAN:TRIGger:PATtern:DATA"](#) on page 944

:SBUS<n>:CAN:TRIGger:PATtern:ID:MODE

N (see [page 1650](#))

Command Syntax :SBUS<n>:CAN:TRIGger:PATtern:ID:MODE <value>

<value> ::= {STANdard | EXTended}

The :SBUS<n>:CAN:TRIGger:PATtern:ID:MODE command sets the CAN identifier mode. STANdard selects the standard 11-bit identifier. EXTended selects the extended 29-bit identifier. The CAN identifier is set by the :SBUS<n>:CAN:TRIGger:PATtern:ID command.

Query Syntax :SBUS<n>:CAN:TRIGger:PATtern:ID:MODE?

The :SBUS<n>:CAN:TRIGger:PATtern:ID:MODE? query returns the current setting of the CAN identifier mode.

Return Format <value><NL>

<value> ::= {STAN | EXT}

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • ["Introduction to :TRIGger Commands"](#) on page 1345

- [":SBUS<n>:MODE"](#) on page 903
- [":SBUS<n>:CAN:TRIGger:PATtern:DATA"](#) on page 944
- [":SBUS<n>:CAN:TRIGger:PATtern:DATA:LENGth"](#) on page 946
- [":SBUS<n>:CAN:TRIGger:PATtern:ID"](#) on page 948

:SBUS<n>:CAN:TRIGger:SYMBolic:MESSage

N (see [page 1650](#))

Command Syntax :SBUS<n>:CAN:TRIGger:SYMBolic:MESSage <name>

<name> ::= quoted ASCII string

The :SBUS<n>:CAN:TRIGger:SYMBolic:MESSage command specifies the message to trigger on when CAN symbolic data has been loaded (recalled) into the oscilloscope and the CAN trigger mode is set to MESSage or MSIGnal.

Query Syntax :SBUS<n>:CAN:TRIGger:SYMBolic:MESSage?

The :SBUS<n>:CAN:TRIGger:SYMBolic:MESSage? query returns the specified message.

Return Format <name><NL>

<name> ::= quoted ASCII string

- See Also**
- [":RECall:DBC\[:START\]"](#) on page 859
 - [":SBUS<n>:CAN:TRIGger"](#) on page 940
 - [":SBUS<n>:CAN:TRIGger:SYMBolic:SIGNal"](#) on page 951
 - [":SBUS<n>:CAN:TRIGger:SYMBolic:VALue"](#) on page 952

:SBUS<n>:CAN:TRIGger:SYMBolic:SIGNaL

N (see [page 1650](#))

Command Syntax :SBUS<n>:CAN:TRIGger:SYMBolic:SIGNaL <name>

<name> ::= quoted ASCII string

The :SBUS<n>:CAN:TRIGger:SYMBolic:SIGNaL command specifies the signal to trigger on when CAN symbolic data has been loaded (recalled) into the oscilloscope and the CAN trigger mode is set to MSIGNAL.

Query Syntax :SBUS<n>:CAN:TRIGger:SYMBolic:SIGNaL?

The :SBUS<n>:CAN:TRIGger:SYMBolic:SIGNaL? query returns the specified signal.

Return Format <name><NL>

<name> ::= quoted ASCII string

- See Also**
- [":RECall:DBC\[:START\]"](#) on page 859
 - [":SBUS<n>:CAN:TRIGger"](#) on page 940
 - [":SBUS<n>:CAN:TRIGger:SYMBolic:MESSage"](#) on page 950
 - [":SBUS<n>:CAN:TRIGger:SYMBolic:VALue"](#) on page 952

:SBUS<n>:CAN:TRIGger:SYMBolic:VALue

N (see [page 1650](#))

Command Syntax :SBUS<n>:CAN:TRIGger:SYMBolic:VALue <data>

<data> ::= value in NR3 format

The :SBUS<n>:CAN:TRIGger:SYMBolic:VALue command specifies the signal value to trigger on when CAN symbolic data has been loaded (recalled) into the oscilloscope and the CAN trigger mode is set to MSIGnal.

NOTE

Encoded signal values are not supported in the remote interface (even though they can be used in the front panel graphical interface).

Query Syntax :SBUS<n>:CAN:TRIGger:SYMBolic:VALue?

The :SBUS<n>:CAN:TRIGger:SYMBolic:VALue? query returns the specified signal value.

Return Format <data><NL>

<data> ::= value in NR3 format

- See Also**
- [":RECall:DBC\[:START\]"](#) on page 859
 - [":SBUS<n>:CAN:TRIGger"](#) on page 940
 - [":SBUS<n>:CAN:TRIGger:SYMBolic:MESSage"](#) on page 950
 - [":SBUS<n>:CAN:TRIGger:SYMBolic:SIGNal"](#) on page 951

:SBUS<n>:CXPI Commands

NOTE

These commands are valid when the CXPI (Clock Extension Peripheral Interface) serial decode and triggering option has been licensed.

Table 123 :SBUS<n>:CXPI Commands Summary

Command	Query	Options and Query Returns
:SBUS<n>:CXPI:BAUDRate <baudrate> (see page 955)	:SBUS<n>:CXPI:BAUDRate? (see page 955)	<baudrate> ::= integer from 9600 to 40000 in 100 b/s increments.
:SBUS<n>:CXPI:PARity {{0 OFF} {1 ON}} (see page 956)	:SBUS<n>:CXPI:PARity? (see page 956)	{0 1}
:SBUS<n>:CXPI:SOURce <source> (see page 957)	:SBUS<n>:CXPI:SOURce? (see page 957)	<source> ::= {CHANnel<n>} <n> ::= 1 to (# analog channels) in NR1 format
:SBUS<n>:CXPI:TOLerance <percent> (see page 958)	:SBUS<n>:CXPI:TOLerance? (see page 958)	<percent> ::= from 1-30, in NR1 format.
:SBUS<n>:CXPI:TRIGger <mode> (see page 959)	:SBUS<n>:CXPI:TRIGger? (see page 960)	<mode> ::= {SOF EOF PTYPE ID DATA LDATA CRCerror PARityerror IBSError IFSerror FRAMingerror DLENgtherror SAMPlerror ALLerrors SLEepframe WAKEuppulse}
:SBUS<n>:CXPI:TRIGger:IDFilter {{0 OFF} {1 ON}} (see page 961)	:SBUS<n>:CXPI:TRIGger:IDFilter? (see page 961)	{0 1}
:SBUS<n>:CXPI:TRIGger:PTYPE {{0 OFF} {1 ON}} (see page 962)	:SBUS<n>:CXPI:TRIGger:PTYPE? (see page 962)	{0 1}
:SBUS<n>:CXPI:TRIGger:PATtern:DATA <string> (see page 963)	:SBUS<n>:CXPI:TRIGger:PATtern:DATA? (see page 963)	<string> ::= "nn...n" where n ::= {0 1 X} <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X}
:SBUS<n>:CXPI:TRIGger:PATtern:DATA:LENGth <length> (see page 964)	:SBUS<n>:CXPI:TRIGger:PATtern:DATA:LENGth? (see page 964)	<start> ::= integer between 0 and 12, in NR1 format.

Table 123 :SBUS<n>:CXPI Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:CXPI:TRIGger :PATtern:DATA:START <start> (see page 965)	:SBUS<n>:CXPI:TRIGger :PATtern:DATA:START? (see page 965)	<start> ::= integer between 0 and 124, in NR1 format.
:SBUS<n>:CXPI:TRIGger :PATtern:ID <string> (see page 966)	:SBUS<n>:CXPI:TRIGger :PATtern:ID? (see page 966)	<string> ::= "nn...n" where n ::= {0 1 X} <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X}
:SBUS<n>:CXPI:TRIGger :PATtern:INFO:CT <string> (see page 967)	:SBUS<n>:CXPI:TRIGger :PATtern:INFO:CT? (see page 967)	<string> ::= "nn" where n ::= {0 1 X}
:SBUS<n>:CXPI:TRIGger :PATtern:INFO:DLC <dlc> (see page 968)	:SBUS<n>:CXPI:TRIGger :PATtern:INFO:DLC? (see page 968)	<dlc> ::= integer between -1 (don't care) and 15, in NR1 format, when trigger is in DATA mode. <dlc> ::= integer between -1 (don't care) and 255, in NR1 format, when trigger is in LDATA mode.
:SBUS<n>:CXPI:TRIGger :PATtern:INFO:NM <string> (see page 969)	:SBUS<n>:CXPI:TRIGger :PATtern:INFO:NM? (see page 969)	<string> ::= "nn" where n ::= {0 1 X}

:SBUS<n>:CXPI:BAUDrate

N (see [page 1650](#))

Command Syntax :SBUS<n>:CXPI:BAUDrate <baudrate>

<baudrate> ::= integer from 9600 to 40000 in 100 b/s increments.

The :SBUS<n>:CXPI:BAUDrate command specifies the baud rate of the CXPI signal from your device under test.

The CXPI baud rate can be set from 9600 b/s to 40000 b/s in 100 b/s increments.

You must set the baud rate to match your device under test.

The default baud rate is 20 kb/s.

Query Syntax :SBUS<n>:CXPI:BAUDrate?

The :SBUS<n>:CXPI:BAUDrate? query returns the baud rate setting.

Return Format <baudrate><NL>

<baudrate> ::= integer from 9600 to 40000 in 100 b/s increments.

- See Also**
- [":SBUS<n>:CXPI:PARity"](#) on page 956
 - [":SBUS<n>:CXPI:SOURce"](#) on page 957
 - [":SBUS<n>:CXPI:TOLerance"](#) on page 958
 - [":SBUS<n>:CXPI:TRIGger"](#) on page 959

:SBUS<n>:CXPI:PARity

N (see [page 1650](#))

Command Syntax :SBUS<n>:CXPI:PARity {{0 | OFF} | {1 | ON}}

The :SBUS<n>:CXPI:PARity command specifies whether the parity bit should be displayed in the identifier field.

When OFF, the upper bit is masked. The parity is still checked, but it is not displayed unless a parity error occurs.

Query Syntax :SBUS<n>:CXPI:PARity?

The :SBUS<n>:CXPI:PARity? query returns the parity display setting.

Return Format <setting><NL>
 <setting> ::= {0 | 1}

- See Also**
- [":SBUS<n>:CXPI:BAUDrate"](#) on page 955
 - [":SBUS<n>:CXPI:SOURce"](#) on page 957
 - [":SBUS<n>:CXPI:TOLerance"](#) on page 958
 - [":SBUS<n>:CXPI:TRIGger"](#) on page 959

:SBUS<n>:CXPI:SOURce

N (see [page 1650](#))

Command Syntax :SBUS<n>:CXPI:SOURce <source>

<source> ::= {CHANnel<n>}

<n> ::= 1 to (# analog channels) in NR1 format

The :SBUS<n>:CXPI:SOURce command selects the oscilloscope channel connected to the CXPI signal line.

Query Syntax :SBUS<n>:CXPI:SOURce?

The :SBUS<n>:CXPI:SOURce? query returns the selected oscilloscope channel source.

Return Format <source><NL>

<source> ::= {CHAN<n>}

- See Also**
- [":SBUS<n>:CXPI:BAUDRate"](#) on page 955
 - [":SBUS<n>:CXPI:PARity"](#) on page 956
 - [":SBUS<n>:CXPI:TOLerance"](#) on page 958
 - [":SBUS<n>:CXPI:TRIGger"](#) on page 959

:SBUS<n>:CXPI:TOLerance

N (see [page 1650](#))

Command Syntax :SBUS<n>:CXPI:TOLerance <percent>

<percent> ::= from 1-30, in NR1 format.

The :SBUS<n>:CXPI:TOLerance command specifies the tolerance as a percentage of the Tbit width.

Query Syntax :SBUS<n>:CXPI:TOLerance?

The :SBUS<n>:CXPI:TOLerance? query returns the tolerance setting.

Return Format <percent><NL>

<percent> ::= from 1-30, in NR1 format.

- See Also**
- [":SBUS<n>:CXPI:BAUDrate"](#) on page 955
 - [":SBUS<n>:CXPI:PARity"](#) on page 956
 - [":SBUS<n>:CXPI:SOURce"](#) on page 957
 - [":SBUS<n>:CXPI:TRIGger"](#) on page 959

:SBUS<n>:CXPI:TRIGger

N (see [page 1650](#))

Command Syntax :SBUS<n>:CXPI:TRIGger <mode>

```
<mode> ::= {SOF | EOF | PTYPe | ID | DATA | LDATa | CRCerror
            | PARityerror | IBSerror | IFSerror | FRAMingerror | DLENgtherror
            | SAMPlerror | ALLerrors | SLEepframe | WAKEuppulse}
```

The :SBUS<n>:CXPI:TRIGger command selects the CXPI trigger type:

- SOF – (Start of Frame) triggers at the start bit of any frame.
- EOF – (End of Frame) triggers at the end of any frame.
- PTYPe – triggers on any frame that starts with the special PTYPe byte.

PTYPE frames begins with an extra PID byte with a Frame ID of 0000000b (reserved for only PTYPe frames). The PTYPe PID byte is then followed by a regular PID byte and the rest of the normal frame. The extra PTYPe byte is never included in the CRC calculation.

- ID – (Frame ID) triggers on a user-defined Frame ID at the end of the PID byte. The Frame ID value is user-defined, 7 bits, and has bitwise don't-cares. You can specify whether to trigger on PTYPe present or no PTYPe present.
- DATA – (Frame ID, Info and Data) triggers on CXPI frames at the end of the last data byte defined in the trigger. In addition to the PID value, you can specify the contents of the Frame Info byte with bitwise don't-cares. You can specify up to 12 data bytes on which to trigger with bitwise don't-cares.
- LDATa – (Frame ID, Info and Data (Long Frame)) triggers on CXPI frames at the end of the last data byte defined in the trigger. The standard DLC field will be locked to 1111b. You can specify up to 12 bytes of data on which to trigger and specify the start byte number as an offset. The offset can be up to 255.
- CRCerror – (CRC Field Error) triggers when the calculated CRC does not match the transmitted CRC. You can optionally filter by Frame ID and PTYPe as in the Frame ID trigger.
- PARityerror – triggers when the parity bit in the PID or PTYPe field is not correct.
- IBSerror – (Inter-Byte Space Error) triggers when there are more than 9 bits between consecutive bytes in a frame. You can optionally filter by Frame ID and PTYPe as in the Frame ID trigger.
- IFSerror – (Inter-Frame Space Error) triggers when there are fewer than 10 idle bits before a new frame begins.
- FRAMingerror – triggers when the stop bit of a byte is not logical 1. You can optionally filter by Frame ID and PTYPe as in the Frame ID trigger.

- **DLENgtherror** – (Data Length Error) triggers when there are more data bytes in a frame than is indicated by the DLC or Extended DLC field. You can optionally filter by Frame ID and PTYPE as in the Frame ID trigger.
- **SAMPlerror** – triggers when 10 consecutive logical 0s are detected.
- **ALLerrors** – triggers on all CRC, Parity, IBS, Stop Bit, Data Length, and Sample errors.
- **SLEepframe** – triggers when a normal frame is transmitted matching the definition of a sleep frame in the CXPI specification.
- **WAKEuppulse** – triggers when a wakeup pulse is detected.

Query Syntax :SBUS<n>:CXPI:TRIGger?

The :SBUS<n>:CXPI:TRIGger? query returns the CXPI trigger type setting.

Return Format <mode><NL>

```
<mode> ::= {SOF | EOF | PTyp | ID | DATA | LDAT | CRC | PAR | IBS
           | IFS | FRAM | DLEN | SAMP | ALL | SLE | WAK}
```

- See Also**
- **":SBUS<n>:CXPI:BAUDRate"** on page 955
 - **":SBUS<n>:CXPI:PARity"** on page 956
 - **":SBUS<n>:CXPI:SOURce"** on page 957
 - **":SBUS<n>:CXPI:TOLerance"** on page 958
 - **":SBUS<n>:CXPI:TRIGger:IDFilter"** on page 961
 - **":SBUS<n>:CXPI:TRIGger:PTYPE"** on page 962
 - **":SBUS<n>:CXPI:TRIGger:PATtern:DATA"** on page 963
 - **":SBUS<n>:CXPI:TRIGger:PATtern:DATA:LENGth"** on page 964
 - **":SBUS<n>:CXPI:TRIGger:PATtern:DATA:START"** on page 965
 - **":SBUS<n>:CXPI:TRIGger:PATtern:ID"** on page 966
 - **":SBUS<n>:CXPI:TRIGger:PATtern:INFO:CT"** on page 967
 - **":SBUS<n>:CXPI:TRIGger:PATtern:INFO:DLC"** on page 968
 - **":SBUS<n>:CXPI:TRIGger:PATtern:INFO:NM"** on page 969

:SBUS<n>:CXPI:TRIGger:IDFilter

N (see [page 1650](#))

Command Syntax :SBUS<n>:CXPI:TRIGger:IDFilter {{0 | OFF} | {1 | ON}}

When triggering on CRC Field Errors, Inter-Byte Space Errors, Framing Errors, or Data Length Errors, the :SBUS<n>:CXPI:TRIGger:IDFilter command lets you enable/disable modification of the trigger so that it occurs only for a specified ID.

Query Syntax :SBUS<n>:CXPI:TRIGger:IDFilter?

The :SBUS<n>:CXPI:TRIGger:IDFilter? query returns the ID filter setting.

Return Format <setting><NL>
<setting> ::= {0 | 1}

- See Also**
- [":SBUS<n>:CXPI:TRIGger"](#) on page 959
 - [":SBUS<n>:CXPI:TRIGger:PTYPE"](#) on page 962
 - [":SBUS<n>:CXPI:TRIGger:PATTERN:DATA"](#) on page 963
 - [":SBUS<n>:CXPI:TRIGger:PATTERN:DATA:LENGTH"](#) on page 964
 - [":SBUS<n>:CXPI:TRIGger:PATTERN:DATA:START"](#) on page 965
 - [":SBUS<n>:CXPI:TRIGger:PATTERN:ID"](#) on page 966
 - [":SBUS<n>:CXPI:TRIGger:PATTERN:INFO:CT"](#) on page 967
 - [":SBUS<n>:CXPI:TRIGger:PATTERN:INFO:DLC"](#) on page 968
 - [":SBUS<n>:CXPI:TRIGger:PATTERN:INFO:NM"](#) on page 969

:SBUS<n>:CXPI:TRIGger:PTYPE

N (see [page 1650](#))

Command Syntax :SBUS<n>:CXPI:TRIGger:PTYPE {{0 | OFF} | {1 | ON}}

For the trigger types that let you trigger on data, the :SBUS<n>:CXPI:TRIGger:PTYPE command specifies whether you want to trigger when the special PTYPE byte is present (ON) or not present (OFF).

Query Syntax :SBUS<n>:CXPI:TRIGger:PTYPE?

The :SBUS<n>:CXPI:TRIGger:PTYPE? query returns the PTYPE trigger setting.

Return Format <setting><NL>
<setting> ::= {0 | 1}

- See Also**
- [":SBUS<n>:CXPI:TRIGger"](#) on page 959
 - [":SBUS<n>:CXPI:TRIGger:IDFilter"](#) on page 961
 - [":SBUS<n>:CXPI:TRIGger:PATtern:DATA"](#) on page 963
 - [":SBUS<n>:CXPI:TRIGger:PATtern:DATA:LENGth"](#) on page 964
 - [":SBUS<n>:CXPI:TRIGger:PATtern:DATA:STARt"](#) on page 965
 - [":SBUS<n>:CXPI:TRIGger:PATtern:ID"](#) on page 966
 - [":SBUS<n>:CXPI:TRIGger:PATtern:INFO:CT"](#) on page 967
 - [":SBUS<n>:CXPI:TRIGger:PATtern:INFO:DLC"](#) on page 968
 - [":SBUS<n>:CXPI:TRIGger:PATtern:INFO:NM"](#) on page 969

:SBUS<n>:CXPI:TRIGger:PATtern:DATA

N (see [page 1650](#))

Command Syntax :SBUS<n>:CXPI:TRIGger:PATtern:DATA <string>

<string> ::= "nn...n" where n ::= {0 | 1 | X}

<string> ::= "0xnn...n" where n ::= {0,...,9 | A,...,F | X}

For the trigger types that let you trigger on data, the :SBUS<n>:CXPI:TRIGger:PATtern:DATA command lets you specify the data value.

The :SBUS<n>:CXPI:TRIGger:PATtern:DATA:LENGth command specifies the length of the data to trigger on, from 0 to 12 bytes, limited by the data length code (DLC) setting of the :SBUS<n>:CXPI:TRIGger:PATtern:INFO:DLC command.

When triggering on long frames (with the LDATa trigger type) that can have up to 255 data bytes, the maximum number of data bytes you can include in the trigger specification is still only 12 bytes. In this case, you can use the :SBUS<n>:CXPI:TRIGger:PATtern:DATA:START command to specify the starting byte location where the data value should be found.

Query Syntax :SBUS<n>:CXPI:TRIGger:PATtern:DATA?

The :SBUS<n>:CXPI:TRIGger:PATtern:DATA? query returns the specified data value.

Returned data values are always quoted binary format strings.

Return Format <string><NL>

<string> ::= "nn...n" where n ::= {0 | 1 | X}

- See Also**
- [":SBUS<n>:CXPI:TRIGger"](#) on page 959
 - [":SBUS<n>:CXPI:TRIGger:IDFilter"](#) on page 961
 - [":SBUS<n>:CXPI:TRIGger:PTYPE"](#) on page 962
 - [":SBUS<n>:CXPI:TRIGger:PATtern:DATA:LENGth"](#) on page 964
 - [":SBUS<n>:CXPI:TRIGger:PATtern:DATA:START"](#) on page 965
 - [":SBUS<n>:CXPI:TRIGger:PATtern:ID"](#) on page 966
 - [":SBUS<n>:CXPI:TRIGger:PATtern:INFO:CT"](#) on page 967
 - [":SBUS<n>:CXPI:TRIGger:PATtern:INFO:DLC"](#) on page 968
 - [":SBUS<n>:CXPI:TRIGger:PATtern:INFO:NM"](#) on page 969

:SBUS<n>:CXPI:TRIGger:PATtern:DATA:LENGth

N (see [page 1650](#))

Command Syntax :SBUS<n>:CXPI:TRIGger:PATtern:DATA:LENGth <length>
 <length> ::= integer between 0 and 12, in NR1 format.

For the trigger types that let you trigger on data, the :SBUS<n>:CXPI:TRIGger:PATtern:DATA:LENGth command specifies the length of the data to trigger on, from 0 to 12 bytes, limited by the data length code (DLC) setting of the :SBUS<n>:CXPI:TRIGger:PATtern:INFO:DLC command.

The :SBUS<n>:CXPI:TRIGger:PATtern:DATA command lets you specify the data value to trigger on.

Query Syntax :SBUS<n>:CXPI:TRIGger:PATtern:DATA:LENGth?

The :SBUS<n>:CXPI:TRIGger:PATtern:DATA:LENGth? query returns the data length setting.

Return Format <length><NL>
 <length> ::= integer between 0 and 12, in NR1 format.

- See Also**
- [":SBUS<n>:CXPI:TRIGger"](#) on page 959
 - [":SBUS<n>:CXPI:TRIGger:IDFilter"](#) on page 961
 - [":SBUS<n>:CXPI:TRIGger:PTYPE"](#) on page 962
 - [":SBUS<n>:CXPI:TRIGger:PATtern:DATA"](#) on page 963
 - [":SBUS<n>:CXPI:TRIGger:PATtern:DATA:START"](#) on page 965
 - [":SBUS<n>:CXPI:TRIGger:PATtern:ID"](#) on page 966
 - [":SBUS<n>:CXPI:TRIGger:PATtern:INFO:CT"](#) on page 967
 - [":SBUS<n>:CXPI:TRIGger:PATtern:INFO:DLC"](#) on page 968
 - [":SBUS<n>:CXPI:TRIGger:PATtern:INFO:NM"](#) on page 969

:SBUS<n>:CXPI:TRIGger:PATtern:DATA:START

N (see [page 1650](#))

Command Syntax :SBUS<n>:CXPI:TRIGger:PATtern:DATA:START <start>

<start> ::= integer between 0 and 124, in NR1 format.

When triggering on long frames (with the LDATA trigger type) that can have up to 255 data bytes, the maximum number of data bytes you can include in the trigger specification is still only 12 bytes. In this case, you can use the :SBUS<n>:CXPI:TRIGger:PATtern:DATA:START command to specify the starting byte location where the data value should be found.

The starting byte location must be within the first 123 bytes when PTYPE is present or 124 bytes when PTYPE is not present.

The :SBUS<n>:CXPI:TRIGger:PATtern:DATA:LENGth command lets you specify the length of the data value to trigger on.

The :SBUS<n>:CXPI:TRIGger:PATtern:DATA command lets you specify the data value to trigger on.

Query Syntax :SBUS<n>:CXPI:TRIGger:PATtern:DATA:START?

The :SBUS<n>:CXPI:TRIGger:PATtern:DATA:START? query returns the start byte setting.

Return Format <start><NL>

<start> ::= integer between 0 and 124, in NR1 format.

- See Also**
- [":SBUS<n>:CXPI:TRIGger"](#) on page 959
 - [":SBUS<n>:CXPI:TRIGger:IDFilter"](#) on page 961
 - [":SBUS<n>:CXPI:TRIGger:PTYPE"](#) on page 962
 - [":SBUS<n>:CXPI:TRIGger:PATtern:DATA"](#) on page 963
 - [":SBUS<n>:CXPI:TRIGger:PATtern:DATA:LENGth"](#) on page 964
 - [":SBUS<n>:CXPI:TRIGger:PATtern:ID"](#) on page 966
 - [":SBUS<n>:CXPI:TRIGger:PATtern:INFO:CT"](#) on page 967
 - [":SBUS<n>:CXPI:TRIGger:PATtern:INFO:DLC"](#) on page 968
 - [":SBUS<n>:CXPI:TRIGger:PATtern:INFO:NM"](#) on page 969

:SBUS<n>:CXPI:TRIGger:PATtern:ID

N (see [page 1650](#))

Command Syntax :SBUS<n>:CXPI:TRIGger:PATtern:ID <string>

<string> ::= "nn...n" where n ::= {0 | 1 | x}

<string> ::= "0xnn...n" where n ::= {0,...,9 | A,...,F | x}

For the trigger types that let you specify frame ID values in the trigger or allow filtering by the frame ID, the :SBUS<n>:CXPI:TRIGger:PATtern:ID command lets you specify the frame ID value.

Query Syntax :SBUS<n>:CXPI:TRIGger:PATtern:ID?

The :SBUS<n>:CXPI:TRIGger:PATtern:ID? query returns the specified frame ID value.

Returned frame ID values are always quoted binary format strings.

Return Format <string><NL>

<string> ::= "nn...n" where n ::= {0 | 1 | x}

- See Also**
- [":SBUS<n>:CXPI:TRIGger"](#) on page 959
 - [":SBUS<n>:CXPI:TRIGger:IDFilter"](#) on page 961
 - [":SBUS<n>:CXPI:TRIGger:PTYPE"](#) on page 962
 - [":SBUS<n>:CXPI:TRIGger:PATtern:DATA"](#) on page 963
 - [":SBUS<n>:CXPI:TRIGger:PATtern:DATA:LENGth"](#) on page 964
 - [":SBUS<n>:CXPI:TRIGger:PATtern:DATA:START"](#) on page 965
 - [":SBUS<n>:CXPI:TRIGger:PATtern:INFO:CT"](#) on page 967
 - [":SBUS<n>:CXPI:TRIGger:PATtern:INFO:DLC"](#) on page 968
 - [":SBUS<n>:CXPI:TRIGger:PATtern:INFO:NM"](#) on page 969

:SBUS<n>:CXPI:TRIGger:PATtern:INFO:CT

N (see [page 1650](#))

Command Syntax :SBUS<n>:CXPI:TRIGger:PATtern:INFO:CT <string>

<string> ::= "nn" where n ::= {0 | 1 | x}

The command ...

For the trigger types that let you trigger on data, as well as frame ID and frame information bits, the :SBUS<n>:CXPI:TRIGger:PATtern:INFO:CT command lets you specify the Count (CT) value of the CXPI frame you wish to trigger on. This is a two-bit binary value.

Query Syntax :SBUS<n>:CXPI:TRIGger:PATtern:INFO:CT?

The :SBUS<n>:CXPI:TRIGger:PATtern:INFO:CT? query returns the specified CT bits included in the trigger.

Return Format <string><NL>

<string> ::= "nn" where n ::= {0 | 1 | x}

- See Also**
- [":SBUS<n>:CXPI:TRIGger"](#) on page 959
 - [":SBUS<n>:CXPI:TRIGger:IDFilter"](#) on page 961
 - [":SBUS<n>:CXPI:TRIGger:PTYPE"](#) on page 962
 - [":SBUS<n>:CXPI:TRIGger:PATtern:DATA"](#) on page 963
 - [":SBUS<n>:CXPI:TRIGger:PATtern:DATA:LENGth"](#) on page 964
 - [":SBUS<n>:CXPI:TRIGger:PATtern:DATA:START"](#) on page 965
 - [":SBUS<n>:CXPI:TRIGger:PATtern:ID"](#) on page 966
 - [":SBUS<n>:CXPI:TRIGger:PATtern:INFO:DLC"](#) on page 968
 - [":SBUS<n>:CXPI:TRIGger:PATtern:INFO:Nm"](#) on page 969

:SBUS<n>:CXPI:TRIGger:PATtern:INFO:DLC

N (see [page 1650](#))

Command Syntax :SBUS<n>:CXPI:TRIGger:PATtern:INFO:DLC <dlc>

<dlc> ::= integer between -1 (don't care) and 15, in NR1 format,
when trigger is in DATA mode.

<dlc> ::= integer between -1 (don't care) and 255, in NR1 format,
when trigger is in LDATa mode.

For the trigger types that let you trigger on data, the :SBUS<n>:CXPI:TRIGger:PATtern:INFO:DLC command specifies the data length code of the CXPI frame you wish to trigger on.

This will also affect the number of data bytes you can specify in the trigger.

Query Syntax :SBUS<n>:CXPI:TRIGger:PATtern:INFO:DLC?

The :SBUS<n>:CXPI:TRIGger:PATtern:INFO:DLC? query returns the DLC trigger value setting.

Return Format <dlc><NL>

<dlc> ::= integer between -1 (don't care) and 15, in NR1 format,
when trigger is in DATA mode.

<dlc> ::= integer between -1 (don't care) and 255, in NR1 format,
when trigger is in LDATa mode.

- See Also**
- [":SBUS<n>:CXPI:TRIGger"](#) on page 959
 - [":SBUS<n>:CXPI:TRIGger:IDFilter"](#) on page 961
 - [":SBUS<n>:CXPI:TRIGger:PTYPE"](#) on page 962
 - [":SBUS<n>:CXPI:TRIGger:PATtern:DATA"](#) on page 963
 - [":SBUS<n>:CXPI:TRIGger:PATtern:DATA:LENGth"](#) on page 964
 - [":SBUS<n>:CXPI:TRIGger:PATtern:DATA:START"](#) on page 965
 - [":SBUS<n>:CXPI:TRIGger:PATtern:ID"](#) on page 966
 - [":SBUS<n>:CXPI:TRIGger:PATtern:INFO:CT"](#) on page 967
 - [":SBUS<n>:CXPI:TRIGger:PATtern:INFO:NM"](#) on page 969

:SBUS<n>:CXPI:TRIGger:PATtern:INFO:NM

N (see [page 1650](#))

Command Syntax :SBUS<n>:CXPI:TRIGger:PATtern:INFO:NM <string>

<string> ::= "nn" where n ::= {0 | 1 | X}

For the trigger types that let you trigger on data, as well as frame ID and frame information bits, the :SBUS<n>:CXPI:TRIGger:PATtern:INFO:NM command lets you specify the Network Management (NM) value of the CXPI frame you wish to trigger on. This is a two-bit binary value.

Query Syntax :SBUS<n>:CXPI:TRIGger:PATtern:INFO:NM?

The :SBUS<n>:CXPI:TRIGger:PATtern:INFO:NM? query returns the specified NM bits included in the trigger.

Return Format <string><NL>

<string> ::= "nn" where n ::= {0 | 1 | X}

- See Also**
- [":SBUS<n>:CXPI:TRIGger"](#) on page 959
 - [":SBUS<n>:CXPI:TRIGger:IDFilter"](#) on page 961
 - [":SBUS<n>:CXPI:TRIGger:PTYPE"](#) on page 962
 - [":SBUS<n>:CXPI:TRIGger:PATtern:DATA"](#) on page 963
 - [":SBUS<n>:CXPI:TRIGger:PATtern:DATA:LENGTH"](#) on page 964
 - [":SBUS<n>:CXPI:TRIGger:PATtern:DATA:START"](#) on page 965
 - [":SBUS<n>:CXPI:TRIGger:PATtern:ID"](#) on page 966
 - [":SBUS<n>:CXPI:TRIGger:PATtern:INFO:CT"](#) on page 967
 - [":SBUS<n>:CXPI:TRIGger:PATtern:INFO:DLC"](#) on page 968

:SBUS<n>:FLEXray Commands

NOTE

These commands are only valid when the FLEXray triggering and serial decode option has been licensed.

Table 124 :SBUS<n>:FLEXray Commands Summary

Command	Query	Options and Query Returns
:SBUS<n>:FLEXray:AUTO setup (see page 972)	n/a	n/a
:SBUS<n>:FLEXray:BAUD rate <baudrate> (see page 973)	:SBUS<n>:FLEXray:BAUD rate? (see page 973)	<baudrate> ::= {2500000 5000000 10000000}
:SBUS<n>:FLEXray:CHAN nel <channel> (see page 974)	:SBUS<n>:FLEXray:CHAN nel? (see page 974)	<channel> ::= {A B}
n/a	:SBUS<n>:FLEXray:COUN t:NULL? (see page 975)	<frame_count> ::= integer in NR1 format
:SBUS<n>:FLEXray:COUN t:RESet (see page 976)	n/a	n/a
n/a	:SBUS<n>:FLEXray:COUN t:SYNC? (see page 977)	<frame_count> ::= integer in NR1 format
n/a	:SBUS<n>:FLEXray:COUN t:TOTal? (see page 978)	<frame_count> ::= integer in NR1 format
:SBUS<n>:FLEXray:SOUR ce <source> (see page 979)	:SBUS<n>:FLEXray:SOUR ce? (see page 979)	<source> ::= {CHANnel<n>} <n> ::= 1-2 or 1-4 in NR1 format
:SBUS<n>:FLEXray:TRIG ger <condition> (see page 980)	:SBUS<n>:FLEXray:TRIG ger? (see page 980)	<condition> ::= {FRAME ERRor EVENT}
:SBUS<n>:FLEXray:TRIG ger:ERRor:TYPE <error_type> (see page 981)	:SBUS<n>:FLEXray:TRIG ger:ERRor:TYPE? (see page 981)	<error_type> ::= {ALL HCRC FCRC}
:SBUS<n>:FLEXray:TRIG ger:EVENT:AUtoSet (see page 982)	n/a	n/a
:SBUS<n>:FLEXray:TRIG ger:EVENT:BSS:ID <frame_id> (see page 983)	:SBUS<n>:FLEXray:TRIG ger:EVENT:BSS:ID? (see page 983)	<frame_id> ::= {ALL <frame #>} <frame #> ::= integer from 1-2047

Table 124 :SBUS<n>:FLEXray Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:FLEXray:TRIGGER:EVENT:TYPE <event> (see page 984)	:SBUS<n>:FLEXray:TRIGGER:EVENT:TYPE? (see page 984)	<event> ::= {WAKEup TSS {FES DTS} BSS}
:SBUS<n>:FLEXray:TRIGGER:FRAME:CCBase <cycle_count_base> (see page 985)	:SBUS<n>:FLEXray:TRIGGER:FRAME:CCBase? (see page 985)	<cycle_count_base> ::= integer from 0-63
:SBUS<n>:FLEXray:TRIGGER:FRAME:CCRepetition <cycle_count_repetition> (see page 986)	:SBUS<n>:FLEXray:TRIGGER:FRAME:CCRepetition? (see page 986)	<cycle_count_repetition> ::= {ALL <rep #>} <rep #> ::= integer values 2, 4, 8, 16, 32, or 64
:SBUS<n>:FLEXray:TRIGGER:FRAME:ID <frame_id> (see page 987)	:SBUS<n>:FLEXray:TRIGGER:FRAME:ID? (see page 987)	<frame_id> ::= {ALL <frame #>} <frame #> ::= integer from 1-2047
:SBUS<n>:FLEXray:TRIGGER:FRAME:TYPE <frame_type> (see page 988)	:SBUS<n>:FLEXray:TRIGGER:FRAME:TYPE? (see page 988)	<frame_type> ::= {NORMAL STARTup NULL SYNC NSTartup NNULl NSYNc ALL}

:SBUS<n>:FLEXray:AUTosetup

N (see [page 1650](#))

Command Syntax :SBUS<n>:FLEXray:AUTosetup

The :SBUS<n>:FLEXray:AUTosetup command automatically configures oscilloscope settings to facilitate FlexRay triggering and serial decode.

- Sets the selected source channel's impedance to 50 Ohms.
- Sets the selected source channel's probe attenuation to 10:1.
- Sets the trigger level (on the selected source channel) to -300 mV.
- Turns on trigger Noise Reject.
- Turns on Serial Decode.
- Sets the trigger to the specified serial bus (n of SBUS<n>).

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:FLEXray:TRIGger"](#) on page 980
 - [":SBUS<n>:FLEXray:BAUDrate"](#) on page 973
 - [":TRIGger\[:EDGE\]:LEVel"](#) on page 1376
 - [":SBUS<n>:FLEXray:SOURce"](#) on page 979

:SBUS<n>:FLEXray:BAUDrate

N (see [page 1650](#))

Command Syntax :SBUS<n>:FLEXray:BAUDrate <baudrate>

<baudrate> ::= {2500000 | 5000000 | 10000000}

The :SBUS<n>:FLEXray:BAUDrate command specifies the baud rate as 2.5 Mb/s, 5 Mb/s, or 10 Mb/s.

Query Syntax :SBUS<n>:FLEXray:BAUDrate?

The :SBUS<n>:FLEXray:BAUDrate? query returns the current baud rate setting.

Return Format <baudrate><NL>

<baudrate> ::= {2500000 | 5000000 | 10000000}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:FLEXray Commands"](#) on page 970

:SBUS<n>:FLEXray:CHANnel

N (see [page 1650](#))

Command Syntax :SBUS<n>:FLEXray:CHANnel <channel>

<channel> ::= {A | B}

The :SBUS<n>:FLEXray:CHANnel command specifies the bus channel, A or B, of the FlexRay signal.

Query Syntax :SBUS<n>:FLEXray:CHANnel?

The :SBUS<n>:FLEXray:CHANnel? query returns the current bus channel setting.

Return Format <channel><NL>

<channel> ::= {A | B}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:FLEXray Commands"](#) on page 970

:SBUS<n>:FLEXray:COUNT:NULL

N (see [page 1650](#))

Query Syntax :SBUS<n>:FLEXray:COUNT:NULL?

Returns the FlexRay null frame count.

Return Format <frame_count><NL>

<frame_count> ::= integer in NR1 format

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • [":SBUS<n>:FLEXray:COUNT:RESet"](#) on page 976
 • [":SBUS<n>:FLEXray:COUNT:TOTal"](#) on page 978
 • [":SBUS<n>:FLEXray:COUNT:SYNC"](#) on page 977
 • ["Introduction to :SBUS<n> Commands"](#) on page 899
 • [":SBUS<n>:MODE"](#) on page 903
 • [":SBUS<n>:FLEXray Commands"](#) on page 970

:SBUS<n>:FLEXray:COUNt:RESet

N (see [page 1650](#))

Command Syntax :SBUS<n>:FLEXray:COUNt:RESet

Resets the FlexRay frame counters.

- Errors**
- ["-241, Hardware missing"](#) on page 1591
- See Also**
- [":SBUS<n>:FLEXray:COUNt:NULL"](#) on page 975
 - [":SBUS<n>:FLEXray:COUNt:TOTal"](#) on page 978
 - [":SBUS<n>:FLEXray:COUNt:SYNC"](#) on page 977
 - ["Introduction to :SBUS<n> Commands"](#) on page 899
 - [":SBUS<n>:MODE"](#) on page 903
 - [":SBUS<n>:FLEXray Commands"](#) on page 970

:SBUS<n>:FLEXray:COUNT:SYNC

N (see [page 1650](#))

Query Syntax :SBUS<n>:FLEXray:COUNT:SYNC?

Returns the FlexRay sync frame count.

Return Format <frame_count><NL>

<frame_count> ::= integer in NR1 format

Errors • ["-241, Hardware missing"](#) on page 1591

- See Also**
- [":SBUS<n>:FLEXray:COUNT:RESet"](#) on page 976
 - [":SBUS<n>:FLEXray:COUNT:TOTal"](#) on page 978
 - [":SBUS<n>:FLEXray:COUNT:NULL"](#) on page 975
 - ["Introduction to :SBUS<n> Commands"](#) on page 899
 - [":SBUS<n>:MODE"](#) on page 903
 - [":SBUS<n>:FLEXray Commands"](#) on page 970

:SBUS<n>:FLEXray:COUNT:TOTal

N (see [page 1650](#))

Query Syntax :SBUS<n>:FLEXray:COUNT:TOTal?

Returns the FlexRay total frame count.

Return Format <frame_count><NL>

<frame_count> ::= integer in NR1 format

Errors • ["-241, Hardware missing"](#) on page 1591

- See Also**
- [":SBUS<n>:FLEXray:COUNT:RESet"](#) on page 976
 - [":SBUS<n>:FLEXray:COUNT:TOTal"](#) on page 978
 - [":SBUS<n>:FLEXray:COUNT:NULL"](#) on page 975
 - [":SBUS<n>:FLEXray:COUNT:SYNC"](#) on page 977
 - ["Introduction to :SBUS<n> Commands"](#) on page 899
 - [":SBUS<n>:MODE"](#) on page 903
 - [":SBUS<n>:FLEXray Commands"](#) on page 970

:SBUS<n>:FLEXray:SOURce

N (see [page 1650](#))

Command Syntax :SBUS<n>:FLEXray:SOURce <source>

<source> ::= {CHANnel<n>}

<n> ::= {1 | 2 | 3 | 4}

The :SBUS<n>:FLEXray:SOURce command specifies the input source for the FlexRay signal.

Query Syntax :SBUS<n>:FLEXray:SOURce?

The :SBUS<n>:FLEXray:SOURce? query returns the current source for the FlexRay signal.

Return Format <source><NL>

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:FLEXray:TRIGger"](#) on page 980
 - [":SBUS<n>:FLEXray:TRIGger:EVENT:TYPE"](#) on page 984
 - [":SBUS<n>:FLEXray:AUTosetup"](#) on page 972

:SBUS<n>:FLEXray:TRIGger

N (see [page 1650](#))

Command Syntax :SBUS<n>:FLEXray:TRIGger <condition>

<condition> ::= {FRAME | ERRor | EVENT}

The :SBUS<n>:FLEXray:TRIGger:TRIGger command sets the FLEXray trigger on condition:

- FRAME – triggers on specified frames (without errors).
- ERRor – triggers on selected active error frames and unknown bus conditions.
- EVENT – triggers on specified FlexRay event/symbol.

Query Syntax :SBUS<n>:FLEXray:TRIGger?

The :SBUS<n>:FLEXray:TRIGger? query returns the current FLEXray trigger on condition.

Return Format <condition><NL>

<condition> ::= {FRAM | ERR | EVEN}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":SBUS<n>:FLEXray:TRIGger:ERRor:TYPE"](#) on page 981
 - [":SBUS<n>:FLEXray:TRIGger:EVENT:AUToset"](#) on page 982
 - [":SBUS<n>:FLEXray:TRIGger:EVENT:BSS:ID"](#) on page 983
 - [":SBUS<n>:FLEXray:TRIGger:EVENT:TYPE"](#) on page 984
 - [":SBUS<n>:FLEXray:TRIGger:FRAME:CCBase"](#) on page 985
 - [":SBUS<n>:FLEXray:TRIGger:FRAME:CCRepetition"](#) on page 986
 - [":SBUS<n>:FLEXray:TRIGger:FRAME:ID"](#) on page 987
 - [":SBUS<n>:FLEXray:TRIGger:FRAME:TYPE"](#) on page 988

:SBUS<n>:FLEXray:TRIGger:ERRor:TYPE

N (see [page 1650](#))

Command Syntax :SBUS<n>:FLEXray:TRIGger:ERRor:TYPE <error_type>

<error_type> ::= {ALL | HCRC | FCRC}

Selects the FlexRay error type to trigger on. The error type setting is only valid when the FlexRay trigger mode is set to ERRor.

- ALL – triggers on ALL errors.
- HCRC – triggers on only Header CRC errors.
- FCRC – triggers on only Frame CRC errors.

Query Syntax :SBUS<n>:FLEXray:TRIGger:ERRor:TYPE?

The :SBUS<n>:FLEXray:TRIGger:ERRor:TYPE? query returns the currently selected FLEXray error type.

Return Format <error_type><NL>

<error_type> ::= {ALL | HCRC | FCRC}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:FLEXray:TRIGger"](#) on page 980

:SBUS<n>:FLEXray:TRIGger:EVENT:AUToset

N (see [page 1650](#))

Command Syntax :SBUS<n>:FLEXray:TRIGger:EVENT:AUToset

The :SBUS<n>:FLEXray:TRIGger:EVENT:AUToset command automatically configures oscilloscope settings (as shown on the display) for the selected event trigger.

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:FLEXray:TRIGger:EVENT:TYPE"](#) on page 984
 - [":SBUS<n>:FLEXray:TRIGger:EVENT:BSS:ID"](#) on page 983
 - [":SBUS<n>:FLEXray:TRIGger"](#) on page 980
 - [":SBUS<n>:FLEXray:BAUDrate"](#) on page 973
 - [":TRIGger\[:EDGE\]:LEVel"](#) on page 1376
 - [":SBUS<n>:FLEXray:SOURce"](#) on page 979

:SBUS<n>:FLEXray:TRIGger:EVENT:BSS:ID

N (see [page 1650](#))

Command Syntax :SBUS<n>:FLEXray:TRIGger:EVENT:BSS:ID <frame_id>

<frame_id> ::= {ALL | <frame #>}

<frame #> ::= integer from 1-2047

The :SBUS<N>:FLEXray:TRIGger:EVENT:BSS:ID command sets the frame ID used by the Byte Start Sequence (BSS) event trigger. This setting is only valid if the trigger mode is EVENT and the EVENT:TYPE is BSS.

Query Syntax :SBUS<n>:FLEXray:TRIGger:EVENT:BSS:ID?

The :SBUS<n>:FLEXray:TRIGger:EVENT:BSS:ID? query returns the current frame ID setting for the Byte Start Sequence (BSS) event trigger setup.

Return Format <frame_id><NL>

<frame_id> ::= {ALL | <frame #>}

<frame #> ::= integer from 1-2047

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:FLEXray:TRIGger:EVENT:TYPE"](#) on page 984
 - [":SBUS<n>:FLEXray:TRIGger:EVENT:AUToset"](#) on page 982
 - [":TRIGger:MODE"](#) on page 1358
 - [":SBUS<n>:FLEXray:TRIGger"](#) on page 980

:SBUS<n>:FLEXray:TRIGger:EVENT:TYPE

N (see [page 1650](#))

Command Syntax :SBUS<n>:FLEXray:TRIGger:EVENT:TYPE <event>

<event> ::= {WAKEup | TSS | {FES | DTS} | BSS}

Selects the FlexRay event to trigger on. The event setting is only valid when the FlexRay trigger mode is set to EVENT.

- WAKEup – triggers on Wake-Up event.
- TSS – triggers on Transmission Start Sequence event.
- FES – triggers on either Frame End or Dynamic Trailing Sequence event.
- DTS – triggers on either Frame End or Dynamic Trailing Sequence event.
- BSS – triggers on Byte Start Sequence event.

Query Syntax :SBUS<n>:FLEXray:TRIGger:EVENT:TYPE?

The :SBUS<n>:FLEXray:TRIGger:EVENT:TYPE? query returns the currently selected FLEXray event.

Return Format <event><NL>

<event> ::= {WAK | TSS | {FES | DTS} | BSS}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:FLEXray:TRIGger:EVENT:AUToset"](#) on page 982
 - [":SBUS<n>:FLEXray:TRIGger:EVENT:BSS:ID"](#) on page 983
 - [":SBUS<n>:FLEXray:TRIGger"](#) on page 980
 - [":SBUS<n>:FLEXray:AUTosetup"](#) on page 972
 - [":SBUS<n>:FLEXray:SOURce"](#) on page 979

:SBUS<n>:FLEXray:TRIGger:FRAME:CCBase

N (see [page 1650](#))

Command Syntax :SBUS<n>:FLEXray:TRIGger:FRAME:CCBase <cycle_count_base>

<cycle_count_base> ::= integer from 0-63

The :SBUS<n>:FLEXray:TRIGger:FRAME:CCBase command sets the base of the FlexRay cycle count (in the frame header) to trigger on. The cycle count base setting is only valid when the FlexRay trigger mode is set to FRAME.

Query Syntax :SBUS<n>:FLEXray:TRIGger:FRAME:CCBase?

The :SBUS<n>:FLEXray:TRIGger:FRAME:CCBase? query returns the current cycle count base setting for the FlexRay frame trigger setup.

Return Format <cycle_count_base><NL>

<cycle_count_base> ::= integer from 0-63

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:FLEXray:TRIGger"](#) on page 980

:SBUS<n>:FLEXray:TRIGger:FRAME:CCRepetition

N (see [page 1650](#))

Command Syntax :SBUS<n>:FLEXray:TRIGger:FRAME:CCRepetition <cycle_count_repetition>

<cycle_count_repetition> ::= {ALL | <rep #>}

<rep #> ::= integer values 2, 4, 8, 16, 32, or 64

The :SBUS<n>:FLEXray:TRIGger:FRAME:CCRepetition command sets the repetition number of the FlexRay cycle count (in the frame header) to trigger on. The cycle count repetition setting is only valid when the FlexRay trigger mode is set to FRAME.

Query Syntax :SBUS<n>:FLEXray:TRIGger:FRAME:CCRepetition?

The :SBUS<n>:FLEXray:TRIGger:FRAME:CCRepetition? query returns the current cycle count repetition setting for the FlexRay frame trigger setup.

Return Format <cycle_count_repetition><NL>

<cycle_count_repetition> ::= {ALL | <rep #>}

<rep #> ::= integer values 2, 4, 8, 16, 32, or 64

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:FLEXray:TRIGger"](#) on page 980

:SBUS<n>:FLEXray:TRIGger:FRAME:ID

N (see [page 1650](#))

Command Syntax :SBUS<n>:FLEXray:TRIGger:FRAME:ID <frame_id>

<frame_id> ::= {ALL | <frame #>}

<frame #> ::= integer from 1-2047

The :SBUS<n>:FLEXray:TRIGger:FRAME:ID command sets the FlexRay frame ID to trigger on. The frame ID setting is only valid when the FlexRay trigger mode is set to FRAME.

Query Syntax :SBUS<n>:FLEXray:TRIGger:FRAME:ID?

The :SBUS<n>:FLEXray:TRIGger:FRAME:ID? query returns the current frame ID setting for the FlexRay frame trigger setup.

Return Format <frame_id><NL>

<frame_id> ::= {ALL | <frame #>}

<frame #> ::= integer from 1-2047

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":SBUS<n>:FLEXray:TRIGger"](#) on page 980

:SBUS<n>:FLEXray:TRIGger:FRAME:TYPE

N (see [page 1650](#))

Command Syntax :SBUS<n>:FLEXray:TRIGger:FRAME:TYPE <frame_type>

<frame_type> ::= {NORMal | STARTup | NULL | SYNC | NSTArtup | NNULl |
NSYNc | ALL}

The :SBUS<n>:FLEXray:TRIGger:FRAME:TYPE command sets the FlexRay frame type to trigger on. The frame type setting is only valid when the FlexRay trigger mode is set to FRAME.

- NORMal – will trigger on only normal (NSTArtup & NNULl & NSYNc) frames.
- STARTup – will trigger on only startup frames.
- NULL – will trigger on only null frames.
- SYNC – will trigger on only sync frames.
- NSTArtup – will trigger on frames other than startup frames.
- NNULl – will trigger on frames other than null frames.
- NSYNc – will trigger on frames other than sync frames.
- ALL – will trigger on all FlexRay frame types.

Query Syntax :SBUS<n>:FLEXray:TRIGger:FRAME:TYPE?

The :SBUS<n>:FLEXray:TRIGger:FRAME:TYPE? query returns the current frame type setting for the FlexRay frame trigger setup.

Return Format <frame_type><NL>

<frame_type> ::= {NORM | STAR | NULL | SYNC | NSTA | NNUL |
NSYN | ALL}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":SBUS<n>:FLEXray:TRIGger"](#) on page 980

:SBUS<n>:I2S Commands

NOTE

These commands are only valid when the I2S serial decode option has been licensed.

Table 125 :SBUS<n>:I2S Commands Summary

Command	Query	Options and Query Returns
:SBUS<n>:I2S:ALIGnment <setting> (see page 992)	:SBUS<n>:I2S:ALIGnment? (see page 992)	<setting> ::= {I2S LJ RJ}
:SBUS<n>:I2S:BASE <base> (see page 993)	:SBUS<n>:I2S:BASE? (see page 993)	<base> ::= {DECimal HEX}
:SBUS<n>:I2S:CLOCK:SLOPe <slope> (see page 994)	:SBUS<n>:I2S:CLOCK:SLOPe? (see page 994)	<slope> ::= {NEGative POSitive}
:SBUS<n>:I2S:RWIDth <receiver> (see page 995)	:SBUS<n>:I2S:RWIDth? (see page 995)	<receiver> ::= 4-32 in NR1 format
:SBUS<n>:I2S:SOURce:CLOCK <source> (see page 996)	:SBUS<n>:I2S:SOURce:CLOCK? (see page 996)	<source> ::= {CHANnel<n> EXTernal} for DSO models <source> ::= {CHANnel<n> DIGital<d> } for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:SBUS<n>:I2S:SOURce:DATA <source> (see page 997)	:SBUS<n>:I2S:SOURce:DATA? (see page 997)	<source> ::= {CHANnel<n> EXTernal} for DSO models <source> ::= {CHANnel<n> DIGital<d> } for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format

Table 125 :SBUS<n>:I2S Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:I2S:SOURce:WSElect <source> (see page 998)	:SBUS<n>:I2S:SOURce:WSElect? (see page 998)	<source> ::= {CHANnel<n> EXTErnal} for DSO models <source> ::= {CHANnel<n> DIGital<d> } for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:SBUS<n>:I2S:TRIGger<operator> (see page 999)	:SBUS<n>:I2S:TRIGger? (see page 999)	<operator> ::= {EQUal NOTequal LESSthan GREaterthan INRange OUTRange INCReasing DECReasing}
:SBUS<n>:I2S:TRIGger:AUDio <audio_ch> (see page 1001)	:SBUS<n>:I2S:TRIGger:AUDio? (see page 1001)	<audio_ch> ::= {RIGHT LEFT EITHER}
:SBUS<n>:I2S:TRIGger:PATtern:DATA <string> (see page 1002)	:SBUS<n>:I2S:TRIGger:PATtern:DATA? (see page 1003)	<string> ::= "n" where n ::= 32-bit integer in signed decimal when <base> = DECimal <string> ::= "nn...n" where n ::= {0 1 X \$} when <base> = BINary <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X \$} when <base> = HEX
:SBUS<n>:I2S:TRIGger:PATtern:FORMat <base> (see page 1004)	:SBUS<n>:I2S:TRIGger:PATtern:FORMat? (see page 1004)	<base> ::= {BINary HEX DECimal}
:SBUS<n>:I2S:TRIGger:RANGe <lower>,<upper> (see page 1005)	:SBUS<n>:I2S:TRIGger:RANGe? (see page 1005)	<lower> ::= 32-bit integer in signed decimal, <nondecimal>, or <string> <upper> ::= 32-bit integer in signed decimal, <nondecimal>, or <string> <nondecimal> ::= #Hnn...n where n ::= {0,...,9 A,...,F} for hexadecimal <nondecimal> ::= #Bnn...n where n ::= {0 1} for binary <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F} for hexadecimal

Table 125 :SBUS<n>:I2S Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:I2S:TWIDth <word_size> (see page 1006)	:SBUS<n>:I2S:TWIDth? (see page 1006)	<word_size> ::= 4-32 in NR1 format
:SBUS<n>:I2S:WSLow <low_def> (see page 1007)	:SBUS<n>:I2S:WSLow? (see page 1007)	<low_def> ::= {LEFT RIGHT}

:SBUS<n>:I2S:ALIGNment

N (see [page 1650](#))

Command Syntax :SBUS<n>:I2S:ALIGNment <setting>

<setting> ::= { I2S | LJ | RJ }

The :SBUS<n>:I2S:ALIGNment command selects the data alignment of the I2S bus for the serial decoder and/or trigger when in I2S mode:

- I2S – standard.
- LJ – left justified.
- RJ – right justified.

Note that the word select (WS) polarity is specified separately with the :SBUS<n>:I2S:WSLow command.

Query Syntax :SBUS<n>:I2S:ALIGNment?

The :SBUS<n>:I2S:ALIGNment? query returns the currently selected I2S data alignment.

Return Format <setting><NL>

<setting> ::= { I2S | LJ | RJ }

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:I2S:CLOCK:SLOPe"](#) on page 994
 - [":SBUS<n>:I2S:RWIDth"](#) on page 995
 - [":SBUS<n>:I2S:TWIDth"](#) on page 1006
 - [":SBUS<n>:I2S:WSLow"](#) on page 1007

:SBUS<n>:I2S:BASE

N (see [page 1650](#))

Command Syntax :SBUS<n>:I2S:BASE <base>

<base> ::= {DECimal | HEX}

The :SBUS<n>:I2S:BASE command determines the base to use for the I2S decode display.

Query Syntax :SBUS<n>:I2S:BASE?

The :SBUS<n>:I2S:BASE? query returns the current I2S display decode base.

Return Format <base><NL>

<base> ::= {DECimal | HEX}

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • ["Introduction to :SBUS<n> Commands"](#) on page 899

• [":SBUS<n>:I2S Commands"](#) on page 989

:SBUS<n>:I2S:CLOCK:SLOPe

N (see [page 1650](#))

Command Syntax :SBUS<n>:I2S:CLOCK:SLOPe <slope>

<slope> ::= {NEGative | POSitive}

The :SBUS<n>:I2S:CLOCK:SLOPe command specifies which edge of the I2S serial clock signal clocks in data.

- NEGative – Falling edge.
- POSitive – Rising edge.

Query Syntax :SBUS<n>:I2S:CLOCK:SLOPe?

The :SBUS<n>:I2S:CLOCK:SLOPe? query returns the current I2S clock slope setting.

Return Format <slope><NL>

<slope> ::= {NEG | POS}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:I2S:ALIGNment"](#) on page 992
 - [":SBUS<n>:I2S:RWIDth"](#) on page 995
 - [":SBUS<n>:I2S:TWIDth"](#) on page 1006
 - [":SBUS<n>:I2S:WSLow"](#) on page 1007

:SBUS<n>:I2S:RWIDth

N (see [page 1650](#))

Command Syntax :SBUS<n>:I2S:RWIDth <receiver>

<receiver> ::= 4-32 in NR1 format

The :SBUS<n>:I2S:RWIDth command sets the width of the receiver (decoded) data word in I2S anywhere from 4 bits to 32 bits.

Query Syntax :SBUS<n>:I2S:RWIDth?

The :SBUS<n>:I2S:RWIDth? query returns the currently set I2S receiver data word width.

Return Format <receiver><NL>

<receiver> ::= 4-32 in NR1 format

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:I2S:ALIGNment"](#) on page 992
 - [":SBUS<n>:I2S:CLOCK:SLOPe"](#) on page 994
 - [":SBUS<n>:I2S:TWIDth"](#) on page 1006
 - [":SBUS<n>:I2S:WSLow"](#) on page 1007

:SBUS<n>:I2S:SOURce:CLOCK

N (see [page 1650](#))

Command Syntax :SBUS<n>:I2S:SOURce:CLOCK <source>

<source> ::= {CHANnel<n> | EXternal} for the DSO models

<source> ::= {CHANnel<n> | DIGital<d>} for the MSO models

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :SBUS<n>:I2S:SOURce:CLOCK controls which signal is used as the serial clock (SCLK) source by the serial decoder and/or trigger when in I2S mode.

Query Syntax :SBUS<n>:I2S:SOURce:CLOCK?

The :SBUS<n>:I2S:SOURce:CLOCK? query returns the current source for the I2S serial clock (SCLK).

Return Format <source><NL>

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:I2S:SOURce:DATA"](#) on page 997
 - [":SBUS<n>:I2S:SOURce:WSElect"](#) on page 998

:SBUS<n>:I2S:SOURce:DATA

N (see [page 1650](#))

Command Syntax :SBUS<n>:I2S:SOURce:DATA <source>

<source> ::= {CHANnel<n> | EXternal} for the DSO models

<source> ::= {CHANnel<n> | DIGital<d>} for the MSO models

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :SBUS<n>:I2S:SOURce:DATA command controls which signal is used as the serial data (SDATA) source by the serial decoder and/or trigger when in I2S mode.

Query Syntax :SBUS<n>:I2S:SOURce:DATA?

The :SBUS<n>:I2S:SOURce:DATA? query returns the current source for the I2S serial data (SDATA).

Return Format <source><NL>

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:I2S:SOURce:CLOCK"](#) on page 996
 - [":SBUS<n>:I2S:SOURce:WSElect"](#) on page 998

:SBUS<n>:I2S:SOURce:WSElect

N (see [page 1650](#))

Command Syntax :SBUS<n>:I2S:SOURce:WSElect <source>

<source> ::= {CHANnel<n> | EXternal} for the DSO models

<source> ::= {CHANnel<n> | DIGital<d>} for the MSO models

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :SBUS<n>:I2S:SOURce:WSElect command controls which signal is used as the word select (WS) source by the serial decoder and/or trigger when in I2S mode.

Query Syntax :SBUS<n>:I2S:SOURce:WSElect?

The :SBUS<n>:I2S:SOURce:WSElect? query returns the current source for I2S word select (WS).

Return Format <source><NL>

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:I2S:SOURce:CLOCK"](#) on page 996
 - [":SBUS<n>:I2S:SOURce:DATA"](#) on page 997

:SBUS<n>:I2S:TRIGger

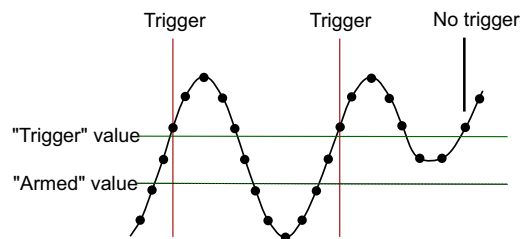
N (see [page 1650](#))

Command Syntax :SBUS<n>:I2S:TRIGger <operator>

<operator> ::= {EQUal | NOTequal | LESSthan | GREaterthan | INRange
| OUTRange | INCReasing | DECReasing}

The :SBUS<n>:I2S:TRIGger command sets the I2S trigger operator:

- EQUal— triggers on the specified audio channel's data word when it equals the specified word.
- NOTequal — triggers on any word other than the specified word.
- LESSthan — triggers when the channel's data word is less than the specified value.
- GREaterthan — triggers when the channel's data word is greater than the specified value.
- INRange — enter upper and lower values to specify the range in which to trigger.
- OUTRange — enter upper and lower values to specify range in which trigger will not occur.
- INCReasing — triggers when the data value makes a certain increase over time and the specified value is met or exceeded. Use the :SBUS<n>:I2S:TRIGger:RANGe command to set "Trigger" and "Armed" values. The "Trigger" value is the value that must be met or exceeded to cause the trigger. The "Armed" value is the value the data must go below in order to re-arm the oscilloscope (ready it to trigger again).



- DECReasing — similar to INCReasing except the trigger occurs on a certain decrease over time and the "Trigger" data value is less than the "Armed" data value.

Query Syntax :SBUS<n>:I2S:TRIGger?

The :SBUS<n>:I2S:TRIGger? query returns the current I2S trigger operator.

Return Format <operator><NL>

<operator> ::= {EQU | NOT | LESS | GRE | INR | OUTR | INCR | DECR}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:I2S:TRIGger:AUDio"](#) on page 1001
 - [":SBUS<n>:I2S:TRIGger:RANGe"](#) on page 1005
 - [":SBUS<n>:I2S:TRIGger:PATtern:FORMat"](#) on page 1004

:SBUS<n>:I2S:TRIGger:AUDio

N (see [page 1650](#))

Command Syntax :SBUS<n>:I2S:TRIGger:AUDio <audio_ch>

<audio_ch> ::= {RIGHT | LEFT | EITHER}

The :SBUS<n>:I2S:TRIGger:AUDio command specifies the audio channel to trigger on:

- RIGHT – right channel.
- LEFT – left channel.
- EITHER – right or left channel.

Query Syntax :SBUS<n>:I2S:TRIGger:AUDio?

The :SBUS<n>:I2S:TRIGger:AUDio? query returns the current audio channel for the I2S trigger.

Return Format <audio_ch><NL>

<audio_ch> ::= {RIGH | LEFT | EITH}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:I2S:TRIGger"](#) on page 999

:SBUS<n>:I2S:TRIGger:PATtern:DATA

N (see [page 1650](#))

Command Syntax :SBUS<n>:I2S:TRIGger:PATtern:DATA <string>

<string> ::= "n" where n ::= 32-bit integer in signed decimal when
<base> = DECimal

<string> ::= "nn...n" where n ::= {0 | 1 | X | \$} when
<base> = BINary

<string> ::= "0xnn...n" where n ::= {0,...,9 | A,...,F | X | \$} when
<base> = HEX

NOTE

<base> is specified with the :SBUS<n>:I2S:TRIGger:PATtern:FORMat command. The default <base> is DECimal.

The :SBUS<n>:I2S:TRIGger:PATtern:DATA command specifies the I2S trigger data pattern searched for in each I2S message.

Set a <string> bit to "0" or "1" to set the corresponding bit in the data pattern to low or high, respectively.

Set a <string> bit to "X" to ignore (mask off) that bit in the data pattern.

Use the "\$" character to indicate that the value of the corresponding bit will not be changed (the existing bit value is used).

When <base> = DECimal, the "X" and "\$" characters cannot be entered. When queried, the "\$" character is returned when any bits in the pattern have the value of "X" and <base> = DECimal. When any bits in a given nibble have the value of "X" and <base> = HEX, the "\$" character is returned for the corresponding nibble.

NOTE

The :SBUS<n>:I2S:TRIGger:PATtern:DATA command specifies the I2S trigger data pattern used by the EQUal, NOTequal, GREaterthan, and LESSthan trigger conditions. If the GREaterthan or LESSthan trigger condition is selected, the bits specified to be masked off ("X") will be interpreted as 0's.

NOTE

The length of the trigger data value is determined by the :SBUS<n>:I2S:RWIDth and :SBUS<n>:I2S:TWIDth commands. When the receiver word size is less than the transmitter word size, the data length is equal to the receiver word size. When the receiver word size is greater than the transmitter word size, the data length is equal to the transmitter word size.

NOTE

If more bits are sent for <string> than the specified trigger data length, the most significant bits will be truncated. If the word size is changed after the <string> is programmed, the added or deleted bits will be added to or deleted from the least significant bits.

Query Syntax :SBUS<n>:I2S:TRIGger:PATtern:DATA?

The :SBUS<n>:I2S:TRIGger:PATtern:DATA? query returns the currently specified I2S trigger data pattern.

Return Format <string><NL>

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:I2S:TRIGger:PATtern:FORMat"](#) on page 1004
 - [":SBUS<n>:I2S:TRIGger"](#) on page 999
 - [":SBUS<n>:I2S:RWIDth"](#) on page 995
 - [":SBUS<n>:I2S:TWIDth"](#) on page 1006
 - [":SBUS<n>:I2S:TRIGger:AUDio"](#) on page 1001

:SBUS<n>:I2S:TRIGger:PATtern:FORMat

N (see [page 1650](#))

Command Syntax :SBUS<n>:I2S:TRIGger:PATtern:FORMat <base>

<base> ::= {BINary | HEX | DECimal}

The :SBUS<n>:I2S:TRIGger:PATtern:FORMat command sets the entry (and query) number base used by the :SBUS<n>:I2S:TRIGger:PATtern:DATA command. The default <base> is DECimal.

Query Syntax :SBUS<n>:I2S:TRIGger:PATtern:FORMat?

The :SBUS<n>:I2S:TRIGger:PATtern:FORMat? query returns the currently set number base for I2S pattern data.

Return Format <base><NL>

<base> ::= {BIN | HEX | DEC}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:I2S:TRIGger:AUDio"](#) on page 1001
 - [":SBUS<n>:I2S:TRIGger"](#) on page 999

:SBUS<n>:I2S:TRIGger:RANGe

N (see [page 1650](#))

Command Syntax :SBUS<n>:I2S:TRIGger:RANGe <lower>,<upper>

<lower> ::= 32-bit integer in signed decimal, <nondecimal>
or <string>

<upper> ::= 32-bit integer in signed decimal, <nondecimal>,
or <string>

<nondecimal> ::= #Hnn...n where n ::= {0,...,9 | A,...,F}
for hexadecimal

<nondecimal> ::= #Bnn...n where n ::= {0 | 1} for binary

<string> ::= "0xnn...n" where n ::= {0,...,9 | A,...,F} for hexadecimal

The :SBUS<n>:I2S:TRIGger:RANGe command sets the lower and upper range boundaries used by the INRange, OUTRange, INCReasing, and DECReasing trigger conditions. You can enter the parameters in any order – the smaller value becomes the <lower> and the larger value becomes the <upper>.

Note that for INCReasing and DECReasing, the <lower> and <upper> values correspond to the "Armed" and "Trigger" softkeys.

NOTE

The length of the <lower> and <upper> values is determined by the :SBUS<n>:I2S:RWIDth and :SBUS<n>:I2S:TWIDth commands. When the receiver word size is less than the transmitter word size, the length is equal to the receiver word size. When the receiver word size is greater than the transmitter word size, the length is equal to the transmitter word size.

Query Syntax :SBUS<n>:I2S:TRIGger:RANGe?

The :SBUS<n>:I2S:TRIGger:RANGe? query returns the currently set lower and upper range boundaries.

Return Format <lower>,<upper><NL>

<lower> ::= 32-bit integer in signed decimal

<upper> ::= 32-bit integer in signed decimal

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:I2S:TRIGger"](#) on page 999
 - [":SBUS<n>:I2S:RWIDth"](#) on page 995
 - [":SBUS<n>:I2S:TWIDth"](#) on page 1006
 - [":SBUS<n>:I2S:WSLow"](#) on page 1007

:SBUS<n>:I2S:TWIDth

N (see [page 1650](#))

Command Syntax :SBUS<n>:I2S:TWIDth <word_size>

<word_size> ::= 4-32 in NR1 format

The :SBUS<n>:I2S:TWIDth command sets the width of the transmitted data word in I2S anywhere from 4 bits to 32 bits.

Query Syntax :SBUS<n>:I2S:TWIDth?

The :SBUS<n>:I2S:TWIDth? query returns the currently set I2S transmitted data word width.

Return Format <word_size><NL>

<word_size> ::= 4-32 in NR1 format

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:I2S:ALIGNment"](#) on page 992
 - [":SBUS<n>:I2S:CLOCK:SLOPe"](#) on page 994
 - [":SBUS<n>:I2S:RWIDth"](#) on page 995
 - [":SBUS<n>:I2S:WSLow"](#) on page 1007

:SBUS<n>:I2S:WSLow

N (see [page 1650](#))

Command Syntax :SBUS<n>:I2S:WSLow <low_def>

<low_def> ::= {LEFT | RIGHT}

The :SBUS<n>:I2S:WSLow command selects the polarity of the word select (WS) signal:

- LEFT— a word select (WS) state of low indicates left channel data is active on the I2S bus, and a WS state of high indicates right channel data is active on the bus.
- RIGHT — a word select (WS) state of low indicates right channel data is active on the I2S bus, and a WS state of high indicates left channel data is active on the bus.

Query Syntax :SBUS<n>:I2S:WSLow?

The :SBUS<n>:I2S:WSLow? query returns the currently selected I2S word select (WS) polarity.

Return Format <low_def><NL>

<low_def> ::= {LEFT | RIGHT}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:I2S:ALIGNment"](#) on page 992
 - [":SBUS<n>:I2S:CLOCK:SLOPe"](#) on page 994
 - [":SBUS<n>:I2S:RWIDth"](#) on page 995
 - [":SBUS<n>:I2S:TWIDth"](#) on page 1006

:SBUS<n>:IIC Commands

NOTE

These commands are only valid when the low-speed IIC and SPI serial decode option has been licensed.

Table 126 :SBUS<n>:IIC Commands Summary

Command	Query	Options and Query Returns
:SBUS<n>:IIC:ASize <size> (see page 1010)	:SBUS<n>:IIC:ASize? (see page 1010)	<size> ::= {BIT7 BIT8}
:SBUS<n>:IIC[:SOURce] :CLOCK <source> (see page 1011)	:SBUS<n>:IIC[:SOURce] :CLOCK? (see page 1011)	<source> ::= {CHANnel<n> EXTernal} for DSO models <source> ::= {CHANnel<n> DIGital<d> } for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:SBUS<n>:IIC[:SOURce] :DATA <source> (see page 1012)	:SBUS<n>:IIC[:SOURce] :DATA? (see page 1012)	<source> ::= {CHANnel<n> EXTernal} for DSO models <source> ::= {CHANnel<n> DIGital<d> } for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:SBUS<n>:IIC:TRIGger: PATtern:ADDRESS <value> (see page 1013)	:SBUS<n>:IIC:TRIGger: PATtern:ADDRESS? (see page 1013)	<value> ::= integer or <string> <string> ::= "0xnn" n ::= {0,...,9 A,...,F}
:SBUS<n>:IIC:TRIGger: PATtern:DATA <value> (see page 1014)	:SBUS<n>:IIC:TRIGger: PATtern:DATA? (see page 1014)	<value> ::= integer or <string> <string> ::= "0xnn" n ::= {0,...,9 A,...,F}
:SBUS<n>:IIC:TRIGger: PATtern:DATA2 <value> (see page 1015)	:SBUS<n>:IIC:TRIGger: PATtern:DATA2? (see page 1015)	<value> ::= integer or <string> <string> ::= "0xnn" n ::= {0,...,9 A,...,F}

Table 126 :SBUS<n>:IIC Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:IIC:TRIGger:QUALifier <value> (see page 1016)	:SBUS<n>:IIC:TRIGger:QUALifier? (see page 1016)	<value> ::= {EQUAL NOTequal LESSthan GREATERthan}
:SBUS<n>:IIC:TRIGger[:TYPE] <type> (see page 1017)	:SBUS<n>:IIC:TRIGger[:TYPE]? (see page 1017)	<type> ::= {START STOP REStart ADDRESS ANACK DNACK NACKnowledge READEprom READ7 WRITe7 R7Data2 W7Data2 WRITe10}

:SBUS<n>:IIC:ASIZe

N (see [page 1650](#))

Command Syntax :SBUS<n>:IIC:ASIZe <size>

<size> ::= {BIT7 | BIT8}

The :SBUS<n>:IIC:ASIZe command determines whether the Read/Write bit is included as the LSB in the display of the IIC address field of the decode bus.

Query Syntax :SBUS<n>:IIC:ASIZe?

The :SBUS<n>:IIC:ASIZe? query returns the current IIC address width setting.

Return Format <mode><NL>

<mode> ::= {BIT7 | BIT8}

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • ["Introduction to :SBUS<n> Commands"](#) on page 899
• [":SBUS<n>:IIC Commands"](#) on page 1008

:SBUS<n>:IIC[:SOURce]:CLOCK

N (see [page 1650](#))

Command Syntax :SBUS<n>:IIC[:SOURce]:CLOCK <source>

<source> ::= {CHANnel<n> | EXternal} for the DSO models

<source> ::= {CHANnel<n> | DIGital<d>} for the MSO models

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :SBUS<n>:IIC[:SOURce]:CLOCK command sets the source for the IIC serial clock (SCL).

Query Syntax :SBUS<n>:IIC[:SOURce]:CLOCK?

The :SBUS<n>:IIC[:SOURce]:CLOCK? query returns the current source for the IIC serial clock.

Return Format <source><NL>

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:IIC\[:SOURce\]:DATA"](#) on page 1012

:SBUS<n>:IIC[:SOURce]:DATA

N (see [page 1650](#))

Command Syntax :SBUS<n>:IIC[:SOURce]:DATA <source>

<source> ::= {CHANnel<n> | EXTernal} for the DSO models

<source> ::= {CHANnel<n> | DIGital<d>} for the MSO models

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :SBUS<n>:IIC[:SOURce]:DATA command sets the source for IIC serial data (SDA).

Query Syntax :SBUS<n>:IIC[:SOURce]:DATA?

The :SBUS<n>:IIC[:SOURce]:DATA? query returns the current source for IIC serial data.

Return Format <source><NL>

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:IIC\[:SOURce\]:CLOCK"](#) on page 1011

:SBUS<n>:IIC:TRIGger:PATtern:ADDress

N (see [page 1650](#))

Command Syntax :SBUS<n>:IIC:TRIGger:PATtern:ADDress <value>

<value> ::= integer or <string>

<string> ::= "0xnn" where n ::= {0,...,9 | A,...,F}

The :SBUS<n>:IIC:TRIGger:PATtern:ADDress command sets the address for IIC data. The address can range from 0x00 to 0x7F (7-bit) or 0x3FF (10-bit) hexadecimal. Use the don't care address (-1 or 0xFFFFFFFF) to ignore the address value.

Query Syntax :SBUS<n>:IIC:TRIGger:PATtern:ADDress?

The :SBUS<n>:IIC:TRIGger:PATtern:ADDress? query returns the current address for IIC data.

Return Format <value><NL>

<value> ::= integer

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:IIC:TRIGger:PATtern:DATA"](#) on page 1014
 - [":SBUS<n>:IIC:TRIGger:PATtern:DATA2"](#) on page 1015
 - [":SBUS<n>:IIC:TRIGger\[:TYPE\]"](#) on page 1017

:SBUS<n>:IIC:TRIGger:PATtern:DATA

N (see [page 1650](#))

Command Syntax :SBUS<n>:IIC:TRIGger:PATtern:DATA <value>

<value> ::= integer or <string>

<string> ::= "0xnn" where n ::= {0,...,9 | A,...,F}

The :SBUS<n>:IIC:TRIGger:PATtern:DATA command sets IIC data. The data value can range from 0x00 to 0xFF (hexadecimal). Use the don't care data pattern (-1 or 0xFFFFFFFF) to ignore the data value.

Query Syntax :SBUS<n>:IIC:TRIGger:PATtern:DATA?

The :SBUS<n>:IIC:TRIGger:PATtern:DATA? query returns the current pattern for IIC data.

Return Format <value><NL>

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:IIC:TRIGger:PATtern:ADDRes"](#) on page 1013
 - [":SBUS<n>:IIC:TRIGger:PATtern:DATA2"](#) on page 1015
 - [":SBUS<n>:IIC:TRIGger\[:TYPE\]"](#) on page 1017

:SBUS<n>:IIC:TRIGger:PATtern:DATa2

N (see [page 1650](#))

Command Syntax :SBUS<n>:IIC:TRIGger:PATtern:DATa2 <value>

<value> ::= integer or <string>

<string> ::= "0xnn" where n ::= {0,...,9 | A,...,F}

The :SBUS<n>:IIC:TRIGger:PATtern:DATa2 command sets IIC data 2. The data value can range from 0x00 to 0xFF (hexadecimal). Use the don't care data pattern (-1 or 0xFFFFFFFF) to ignore the data value.

Query Syntax :SBUS<n>:IIC:TRIGger:PATtern:DATa2?

The :SBUS<n>:IIC:TRIGger:PATtern:DATa2? query returns the current pattern for IIC data 2.

Return Format <value><NL>

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:IIC:TRIGger:PATtern:ADDRes"](#) on page 1013
 - [":SBUS<n>:IIC:TRIGger:PATtern:DATA"](#) on page 1014
 - [":SBUS<n>:IIC:TRIGger\[:TYPE\]"](#) on page 1017

:SBUS<n>:IIC:TRIGger:QUALifier

N (see [page 1650](#))

Command Syntax :SBUS<n>:IIC:TRIGger:QUALifier <value>

<value> ::= {EQUAL | NOTequal | LESSthan | GREATERthan}

The :SBUS<n>:IIC:TRIGger:QUALifier command sets the IIC data qualifier when TRIGger:IIC:TRIGger[:TYPE] is set to READEprom.

Query Syntax :SBUS<n>:IIC:TRIGger:QUALifier?

The :SBUS<n>:IIC:TRIGger:QUALifier? query returns the current IIC data qualifier value.

Return Format <value><NL>

<value> ::= {EQUAL | NOTequal | LESSthan | GREATERthan}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":SBUS<n>:IIC:TRIGger\[:TYPE\]"](#) on page 1017

:SBUS<n>:IIC:TRIGger[:TYPE]

N (see [page 1650](#))

Command Syntax :SBUS<n>:IIC:TRIGger[:TYPE] <value>

```
<value> ::= {START | STOP | REStart | ADDRess | ANACk | DNACk
            | NACKnowledge | READEprom | READ7 | WRITe7 | R7Data2 | W7Data2
            | WRITe10}
```

The :SBUS<n>:IIC:TRIGger[:TYPE] command sets the IIC trigger type:

- START – Start condition.
- STOP – Stop condition.
- REStart – Another start condition occurs before a stop condition.
- ADDRess – Triggers on the selected address. The R/W bit is ignored.
- ANACk – Address with no acknowledge.
- DNACk – Write data with no acknowledge.
- NACKnowledge – Missing acknowledge.
- READEprom – EEPROM data read.
- READ7 – 7-bit address frame containing (Start:Address7:Read:Ack:Data). The value READ is also accepted for READ7.
- WRITe7 – 7-bit address frame containing (Start:Address7:Write:Ack:Data). The value WRITe is also accepted for WRITe7.
- R7Data2 – 7-bit address frame containing (Start:Address7:Read:Ack:Data:Ack:Data2).
- W7Data2 – 7-bit address frame containing (Start:Address7:Write:Ack:Data:Ack:Data2).
- WRITe10 – 10-bit address frame containing (Start:Address byte1:Write:Ack:Address byte 2:Data).

NOTE

The short form of READ7 (READ7), READEprom (READE), WRITe7 (WRIT7), and WRITe10 (WRIT10) do not follow the defined Long Form to Short Form Truncation Rules (see [page 1652](#)).

Query Syntax :SBUS<n>:IIC:TRIGger[:TYPE] ?

The :SBUS<n>:IIC:TRIGger[:TYPE]? query returns the current IIC trigger type value.

Return Format <value><NL>

```
<value> ::= {STAR | STOP | REST | ADDR | ANAC | DNAC | NACK | READE
            | READ7 | WRIT7 | R7D2 | W7D2 | WRIT10}
```

See Also • ["Introduction to :TRIGger Commands"](#) on page 1345

- **":TRIGger:MODE"** on page 1358
- **":SBUS<n>:IIC:TRIGger:PATtern:ADDReSS"** on page 1013
- **":SBUS<n>:IIC:TRIGger:PATtern:DATA"** on page 1014
- **":SBUS<n>:IIC:TRIGger:PATtern:DATa2"** on page 1015
- **":SBUS<n>:IIC:TRIGger:QUALifier"** on page 1016
- **"Long Form to Short Form Truncation Rules"** on page 1652

:SBUS<n>:LIN Commands

NOTE

These commands are valid when the CAN and LIN serial decode license has been enabled.

Table 127 :SBUS<n>:LIN Commands Summary

Command	Query	Options and Query Returns
:SBUS<n>:LIN:DISPlay <type> (see page 1021)	:SBUS<n>:LIN:DISPlay? (see page 1021)	<type> ::= {HEXadecimal SYMBOLic}
:SBUS<n>:LIN:PARity { {0 OFF} {1 ON} } (see page 1022)	:SBUS<n>:LIN:PARity? (see page 1022)	{0 1}
:SBUS<n>:LIN:SAMplepo int <value> (see page 1023)	:SBUS<n>:LIN:SAMplepo int? (see page 1023)	<value> ::= {60 62.5 68 70 75 80 87.5} in NR3 format
:SBUS<n>:LIN:SIGNAL:B AUDrate <baudrate> (see page 1024)	:SBUS<n>:LIN:SIGNAL:B AUDrate? (see page 1024)	<baudrate> ::= integer from 2400 to 625000 in 100 b/s increments
:SBUS<n>:LIN:SOURce <source> (see page 1025)	:SBUS<n>:LIN:SOURce? (see page 1025)	<source> ::= {CHANnel<n> EXTErnal} for DSO models <source> ::= {CHANnel<n> DIGital<d>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:SBUS<n>:LIN:STANdard <std> (see page 1026)	:SBUS<n>:LIN:STANdard ? (see page 1026)	<std> ::= {LIN13 LIN13NLC LIN20}
:SBUS<n>:LIN:SYNCbrea k <value> (see page 1027)	:SBUS<n>:LIN:SYNCbrea k? (see page 1027)	<value> ::= integer = {11 12 13}
:SBUS<n>:LIN:TRIGger <condition> (see page 1028)	:SBUS<n>:LIN:TRIGger? (see page 1028)	<condition> ::= {SYNCbreak ID DATA}

Table 127 :SBUS<n>:LIN Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:LIN:TRIGger:ID <value> (see page 1030)	:SBUS<n>:LIN:TRIGger:ID? (see page 1030)	<value> ::= 7-bit integer in decimal, <nondecimal>, or <string> from 0-63 or 0x00-0x3f <nondecimal> ::= #Hnn where n ::= {0,...,9 A,...,F} for hexadecimal <nondecimal> ::= #Bnn...n where n ::= {0 1} for binary <string> ::= "0xnn" where n ::= {0,...,9 A,...,F} for hexadecimal
:SBUS<n>:LIN:TRIGger:PATtern:DATA <string> (see page 1031)	:SBUS<n>:LIN:TRIGger:PATtern:DATA? (see page 1031)	<string> ::= "n" where n ::= 32-bit integer in unsigned decimal when <base> = DECimal <string> ::= "nn...n" where n ::= {0 1 X \$} when <base> = BINary <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X \$} when <base> = HEX
:SBUS<n>:LIN:TRIGger:PATtern:DATA:LENGth <length> (see page 1033)	:SBUS<n>:LIN:TRIGger:PATtern:DATA:LENGth? (see page 1033)	<length> ::= integer from 1 to 8 in NR1 format
:SBUS<n>:LIN:TRIGger:PATtern:FORMat <base> (see page 1034)	:SBUS<n>:LIN:TRIGger:PATtern:FORMat? (see page 1034)	<base> ::= {BINary HEX DECimal}
:SBUS<n>:LIN:TRIGger:SYMBolic:FRAMe <name> (see page 1035)	:SBUS<n>:LIN:TRIGger:SYMBolic:FRAMe? (see page 1035)	<name> ::= quoted ASCII string
:SBUS<n>:LIN:TRIGger:SYMBolic:SIGNal <name> (see page 1036)	:SBUS<n>:LIN:TRIGger:SYMBolic:SIGNal? (see page 1036)	<name> ::= quoted ASCII string
:SBUS<n>:LIN:TRIGger:SYMBolic:VALue <data> (see page 1037)	:SBUS<n>:LIN:TRIGger:SYMBolic:VALue? (see page 1037)	<data> ::= value in NR3 format

:SBUS<n>:LIN:DISPlay

N (see [page 1650](#))

Command Syntax :SBUS<n>:LIN:DISPlay <type>
 <type> ::= {HEXadecimal | SYMBolic}

The :SBUS<n>:LIN:DISPlay command specifies, when LIN symbolic data is loaded into the oscilloscope, whether symbolic values (from the LDF file) or hexadecimal values are displayed in the decode waveform and the Lister window.

Query Syntax :SBUS<n>:LIN:DISPlay?

The :SBUS<n>:LIN:DISPlay? query returns the LIN decode display type.

Return Format <type><NL>
 <type> ::= {HEX | SYMB}

See Also • [":RECall:LDF\[:START\]"](#) on page 861

:SBUS<n>:LIN:PARity

N (see [page 1650](#))

Command Syntax :SBUS<n>:LIN:PARity <display>

<display> ::= {{1 | ON} | {0 | OFF}}

The :SBUS<n>:LIN:PARity command determines whether the parity bits are included as the most significant bits (MSB) in the display of the Frame Id field in the LIN decode bus.

Query Syntax :SBUS<n>:LIN:PARity?

The :SBUS<n>:LIN:PARity? query returns the current LIN parity bits display setting of the serial decode bus.

Return Format <display><NL>

<display> ::= {0 | 1}

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • ["Introduction to :SBUS<n> Commands"](#) on page 899

• [":SBUS<n>:LIN Commands"](#) on page 1019

:SBUS<n>:LIN:SAMPlEpoint

N (see [page 1650](#))

Command Syntax :SBUS<n>:LIN:SAMPlEpoint <value>

<value><NL>

<value> ::= {60 | 62.5 | 68 | 70 | 75 | 80 | 87.5} in NR3 format

The :SBUS<n>:LIN:SAMPlEpoint command sets the point during the bit time where the bit level is sampled to determine whether the bit is dominant or recessive. The sample point represents the percentage of time between the beginning of the bit time to the end of the bit time.

NOTE

The sample point values are not limited by the baud rate.

Query Syntax :SBUS<n>:LIN:SAMPlEpoint?

The :SBUS<n>:LIN:SAMPlEpoint? query returns the current LIN sample point setting.

Return Format <value><NL>

<value> ::= {60 | 62.5 | 68 | 70 | 75 | 80 | 87.5} in NR3 format

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":SBUS<n>:LIN:TRIGger"](#) on page 1028

:SBUS<n>:LIN:SIGNal:BAUDrate

N (see [page 1650](#))

Command Syntax :SBUS<n>:LIN:SIGNal:BAUDrate <baudrate>

<baudrate> ::= integer from 2400 to 625000 in 100 b/s increments

The :SBUS<n>:LIN:SIGNal:BAUDrate command sets the standard baud rate of the LIN signal from 2400 b/s to 625 kb/s in 100 b/s increments. If you enter a baud rate that is not divisible by 100 b/s, the baud rate is set to the nearest baud rate divisible by 100 b/s.

Query Syntax :SBUS<n>:LIN:SIGNal:BAUDrate?

The :SBUS<n>:LIN:SIGNal:BAUDrate? query returns the current LIN baud rate setting.

Return Format <baudrate><NL>

<baudrate> ::= integer from 2400 to 625000 in 100 b/s increments

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":SBUS<n>:LIN:TRIGger"](#) on page 1028
 - [":SBUS<n>:LIN:SIGNal:DEFinition"](#) on page 1583
 - [":SBUS<n>:LIN:SOURce"](#) on page 1025

:SBUS<n>:LIN:SOURce

N (see [page 1650](#))

Command Syntax :SBUS<n>:LIN:SOURce <source>

<source> ::= {CHANnel<n> | EXternal} for the DSO models

<source> ::= {CHANnel<n> | DIGital<d>} for the MSO models

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :SBUS<n>:LIN:SOURce command sets the source for the LIN signal.

Query Syntax :SBUS<n>:LIN:SOURce?

The :SBUS<n>:LIN:SOURce? query returns the current source for the LIN signal.

Return Format <source><NL>

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":SBUS<n>:LIN:TRIGger"](#) on page 1028
 - [":SBUS<n>:LIN:SIGNAL:DEFinition"](#) on page 1583

:SBUS<n>:LIN:STANdard

N (see [page 1650](#))

Command Syntax :SBUS<n>:LIN:STANdard <std>

<std> ::= {LIN13 | LIN13NLC | LIN20}

The :SBUS<n>:LIN:STANdard command sets the LIN standard in effect for triggering and decoding:

- LIN13 – LIN 1.3.
- LIN13NLC – LIN 1.3 (no length control). Select this for systems where length control is not used and all nodes have knowledge of the data packet size. In LIN 1.3, the ID may or may not be used to indicate the number of bytes. (In LIN 2.X, there is no length control.)
- LIN20 – LIN 2.X.

For LIN 1.2 signals, use the LIN 1.3 setting. The LIN 1.3 setting assumes the signal follows the "Table of Valid ID Values" as shown in section A.2 of the LIN Specification dated December 12, 2002. If your signal does not comply with the table, use the LIN 2.X setting.

Query Syntax :SBUS<n>:LIN:STANdard?

The :SBUS<n>:LIN:STANdard? query returns the current LIN standard setting.

Return Format <std><NL>

<std> ::= {LIN13 | LIN13NLC | LIN20}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":SBUS<n>:LIN:SIGNal:DEFinition"](#) on page 1583
 - [":SBUS<n>:LIN:SOURce"](#) on page 1025

:SBUS<n>:LIN:SYNCbreak

N (see [page 1650](#))

Command Syntax :SBUS<n>:LIN:SYNCbreak <value>

<value> ::= integer = {11 | 12 | 13}

The :SBUS<n>:LIN:SYNCbreak command sets the length of the LIN sync break to be greater than or equal to 11, 12, or 13 clock lengths. The sync break is the idle period in the bus activity at the beginning of each packet that distinguishes one information packet from the previous one.

Query Syntax :SBUS<n>:LIN:SYNCbreak?

The :SBUS<n>:LIN:SYNCbreak? query returns the current LIN sync break setting.

Return Format <value><NL>

<value> ::= {11 | 12 | 13}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":SBUS<n>:LIN:SIGNAL:DEFinition"](#) on page 1583
 - [":SBUS<n>:LIN:SOURce"](#) on page 1025

:SBUS<n>:LIN:TRIGger

N (see [page 1650](#))

Command Syntax :SBUS<n>:LIN:TRIGger <condition>

<condition> ::= {SYNCbreak | ID | DATA | PARityerror | CSUMerror
| FRAME | FSignal}

The :SBUS<n>:LIN:TRIGger command sets the LIN trigger condition to be:

- SYNCbreak – Sync Break.
- ID – Frame ID.

Use the :SBUS<n>:LIN:TRIGger:ID command to specify the frame ID.

- DATA – Frame ID and Data.

Use the :SBUS<n>:LIN:TRIGger:ID command to specify the frame ID.

Use the :SBUS<n>:LIN:TRIGger:PATtern:DATA:LENGth and :SBUS<n>:LIN:TRIGger:PATtern:DATA commands to specify the data string length and value.

- PARityerror – parity errors.
- CSUMerror – checksum errors.
- FRAME – Triggers on a symbolic frame.
- FSignal – Triggers on a symbolic frame and a signal value.

Query Syntax :SBUS<n>:LIN:TRIGger?

The :SBUS<n>:LIN:TRIGger? query returns the current LIN trigger value.

Return Format <condition><NL>

<condition> ::= {SYNC | ID | DATA | PAR | CSUM | FRAM | FSIG}

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • ["Introduction to :TRIGger Commands"](#) on page 1345

- [":TRIGger:MODE"](#) on page 1358
- [":SBUS<n>:LIN:TRIGger:ID"](#) on page 1030
- [":SBUS<n>:LIN:TRIGger:PATtern:DATA:LENGth"](#) on page 1033
- [":SBUS<n>:LIN:TRIGger:PATtern:DATA"](#) on page 1031
- [":SBUS<n>:LIN:SIGnal:DEFinition"](#) on page 1583
- [":SBUS<n>:LIN:SOURce"](#) on page 1025
- [":RECall:LDF\[:START\]"](#) on page 861
- [":SBUS<n>:LIN:TRIGger:SYMBolic:FRAME"](#) on page 1035
- [":SBUS<n>:LIN:TRIGger:SYMBolic:SIGNAL"](#) on page 1036

- **":SBUS<n>:LIN:TRIGger:SYMBolic:VALue"** on page 1037

:SBUS<n>:LIN:TRIGger:ID

N (see [page 1650](#))

Command Syntax :SBUS<n>:LIN:TRIGger:ID <value>

<value> ::= 7-bit integer in decimal, <nondecimal>, or <string>
from 0-63 or 0x00-0x3f

<nondecimal> ::= #Hnn where n ::= {0,...,9 | A,...,F} for hexadecimal

<nondecimal> ::= #Bnn...n where n ::= {0 | 1} for binary

<string> ::= "0xnn" where n ::= {0,...,9 | A,...,F} for hexadecimal

The :SBUS<n>:LIN:TRIGger:ID command defines the LIN identifier searched for in each CAN message when the LIN trigger mode is set to frame ID.

Setting the ID to a value of "-1" results in "0xXX" which is equivalent to all IDs.

Query Syntax :SBUS<n>:LIN:TRIGger:ID?

The :SBUS<n>:LIN:TRIGger:ID? query returns the current LIN identifier setting.

Return Format <value><NL>

<value> ::= integer in decimal

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • ["Introduction to :TRIGger Commands"](#) on page 1345
• [":TRIGger:MODE"](#) on page 1358
• [":SBUS<n>:LIN:TRIGger"](#) on page 1028
• [":SBUS<n>:LIN:SIGNAL:DEFinition"](#) on page 1583
• [":SBUS<n>:LIN:SOURce"](#) on page 1025

- See Also
- [":Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:LIN:TRIGger:PATtern:FORMat"](#) on page 1034
 - [":SBUS<n>:LIN:TRIGger"](#) on page 1028
 - [":SBUS<n>:LIN:TRIGger:PATtern:DATA:LENGth"](#) on page 1033

:SBUS<n>:LIN:TRIGger:PATtern:DATA:LENGth

N (see [page 1650](#))

Command Syntax :SBUS<n>:LIN:TRIGger:PATtern:DATA:LENGth <length>

<length> ::= integer from 1 to 8 in NR1 format

The :SBUS<n>:LIN:TRIGger:PATtern:DATA:LENGth command sets the number of 8-bit bytes in the LIN data string. The number of bytes in the string can be anywhere from 1 bytes to 8 bytes (64 bits). The value for these bytes is set by the :SBUS<n>:LIN:TRIGger:PATtern:DATA command.

Query Syntax :SBUS<n>:LIN:TRIGger:PATtern:DATA:LENGth?

The :SBUS<n>:LIN:TRIGger:PATtern:DATA:LENGth? query returns the current LIN data pattern length setting.

Return Format <count><NL>

<count> ::= integer from 1 to 8 in NR1 format

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • ["Introduction to :TRIGger Commands"](#) on page 1345

• [":SBUS<n>:LIN:TRIGger:PATtern:DATA"](#) on page 1031

• [":SBUS<n>:LIN:SOURce"](#) on page 1025

:SBUS<n>:LIN:TRIGger:PATtern:FORMat

N (see [page 1650](#))

Command Syntax :SBUS<n>:LIN:TRIGger:PATtern:FORMat <base>

<base> ::= {BINary | HEX | DECimal}

The :SBUS<n>:LIN:TRIGger:PATtern:FORMat command sets the entry (and query) number base used by the :SBUS<n>:LIN:TRIGger:PATtern:DATA command. The default <base> is BINary.

Query Syntax :SBUS<n>:LIN:TRIGger:PATtern:FORMat?

The :SBUS<n>:LIN:TRIGger:PATtern:FORMat? query returns the currently set number base for LIN pattern data.

Return Format <base><NL>

<base> ::= {BIN | HEX | DEC}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:LIN:TRIGger:PATtern:DATA"](#) on page 1031
 - [":SBUS<n>:LIN:TRIGger:PATtern:DATA:LENGth"](#) on page 1033

:SBUS<n>:LIN:TRIGger:SYMBolic:FRAMe

N (see [page 1650](#))

Command Syntax :SBUS<n>:LIN:TRIGger:SYMBolic:FRAMe <name>

<name> ::= quoted ASCII string

The :SBUS<n>:LIN:TRIGger:SYMBolic:FRAMe command specifies the message to trigger on when LIN symbolic data has been loaded (recalled) into the oscilloscope and the LIN trigger mode is set to FRAMe or FSIGnal.

Query Syntax :SBUS<n>:LIN:TRIGger:SYMBolic:FRAMe?

The :SBUS<n>:LIN:TRIGger:SYMBolic:FRAMe? query returns the specified message.

Return Format <name><NL>

<name> ::= quoted ASCII string

- See Also**
- [":RECall:LDF\[:STARt\]"](#) on page 861
 - [":SBUS<n>:LIN:TRIGger"](#) on page 1028
 - [":SBUS<n>:LIN:TRIGger:SYMBolic:SIGNaL"](#) on page 1036
 - [":SBUS<n>:LIN:TRIGger:SYMBolic:VALue"](#) on page 1037

:SBUS<n>:LIN:TRIGger:SYMBolic:SIGNal

N (see [page 1650](#))

Command Syntax :SBUS<n>:LIN:TRIGger:SYMBolic:SIGNal <name>

<name> ::= quoted ASCII string

The :SBUS<n>:LIN:TRIGger:SYMBolic:SIGNal command specifies the signal to trigger on when LIN symbolic data has been loaded (recalled) into the oscilloscope and the LIN trigger mode is set to FSIGnal.

Query Syntax :SBUS<n>:LIN:TRIGger:SYMBolic:SIGNal?

The :SBUS<n>:LIN:TRIGger:SYMBolic:SIGNal? query returns the specified signal.

Return Format <name><NL>

<name> ::= quoted ASCII string

- See Also**
- [":RECall:LDF\[:START\]"](#) on page 861
 - [":SBUS<n>:LIN:TRIGger"](#) on page 1028
 - [":SBUS<n>:LIN:TRIGger:SYMBolic:FRAMe"](#) on page 1035
 - [":SBUS<n>:LIN:TRIGger:SYMBolic:VALue"](#) on page 1037

:SBUS<n>:LIN:TRIGger:SYMBolic:VALue

N (see [page 1650](#))

Command Syntax :SBUS<n>:LIN:TRIGger:SYMBolic:VALue <data>

<data> ::= value in NR3 format

The :SBUS<n>:LIN:TRIGger:SYMBolic:VALue command specifies the signal value to trigger on when LIN symbolic data has been loaded (recalled) into the oscilloscope and the LIN trigger mode is set to FSIgnal.

NOTE

Encoded signal values are not supported in the remote interface (even though they can be used in the front panel graphical interface).

Query Syntax :SBUS<n>:LIN:TRIGger:SYMBolic:VALue?

The :SBUS<n>:LIN:TRIGger:SYMBolic:VALue? query returns the specified signal value.

Return Format <data><NL>

<data> ::= value in NR3 format

- See Also**
- [":RECall:LDF\[:START\]"](#) on page 861
 - [":SBUS<n>:LIN:TRIGger"](#) on page 1028
 - [":SBUS<n>:LIN:TRIGger:SYMBolic:FRAME"](#) on page 1035
 - [":SBUS<n>:LIN:TRIGger:SYMBolic:SIGNAL"](#) on page 1036

:SBUS<n>:M1553 Commands

NOTE

These commands are valid when the MIL-STD-1553 and ARINC 429 triggering and serial decode license has been enabled.

Table 128 :SBUS<n>:M1553 Commands Summary

Command	Query	Options and Query Returns
:SBUS<n>:M1553:AUTOsetup (see page 1039)	n/a	n/a
:SBUS<n>:M1553:BASE<base> (see page 1040)	:SBUS<n>:M1553:BASE? (see page 1040)	<base> ::= {BINary HEX}
:SBUS<n>:M1553:SOURce<source> (see page 1041)	:SBUS<n>:M1553:SOURce? (see page 1041)	<source> ::= {CHANnel<n>} <n> ::= 1 to (# analog channels) in NR1 format
:SBUS<n>:M1553:TRIGger:PATtern:DATA<string> (see page 1042)	:SBUS<n>:M1553:TRIGger:PATtern:DATA? (see page 1042)	<string> ::= "nn...n" where n ::= {0 1 X}
:SBUS<n>:M1553:TRIGger:RTA<value> (see page 1043)	:SBUS<n>:M1553:TRIGger:RTA? (see page 1043)	<value> ::= 5-bit integer in decimal, <nondecimal>, or <string> from 0-31 <nondecimal> ::= #Hnn where n ::= {0,...,9 A,...,F} <string> ::= "0xnn" where n ::= {0,...,9 A,...,F}
:SBUS<n>:M1553:TRIGger:TYPE<type> (see page 1044)	:SBUS<n>:M1553:TRIGger:TYPE? (see page 1044)	<type> ::= {DStArt DStOp CStArt CStOp RTA PERRor SError MError RTA11}

:SBUS<n>:M1553:AUTosetup

N (see [page 1650](#))

Command Syntax :SBUS<n>:M1553:TRIGger:AUTosetup

The :SBUS<n>:M1553:AUTosetup command automatically sets these options for decoding and triggering on MIL-STD-1553 signals:

- High/Low Trigger Thresholds: to a voltage value equal to $\pm 1/3$ division based on the source channel's current V/div setting.
- Noise Reject: Off.
- Probe Attenuation: 10.0.
- Serial Decode: On.
- Trigger: the specified serial bus (n of SBUS<n>).

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":SBUS<n>:M1553:SOURce"](#) on page 1041

:SBUS<n>:M1553:BASE

N (see [page 1650](#))

Command Syntax :SBUS<n>:M1553:BASE <base>

<base> ::= {BINary | HEX}

The :SBUS<n>:M1553:BASE command determines the base to use for the MIL-STD-1553 decode display.

Query Syntax :SBUS<n>:M1553:BASE?

The :SBUS<n>:M1553:BASE? query returns the current MIL-STD-1553 display decode base.

Return Format <base><NL>

<base> ::= {BIN | HEX}

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • ["Introduction to :SBUS<n> Commands"](#) on page 899
 • [":SBUS<n>:M1553 Commands"](#) on page 1038

:SBUS<n>:M1553:SOURce

N (see [page 1650](#))

Command Syntax :SBUS<n>:M1553:SOURce <source>

<source> ::= {CHANnel<n>}

<n> ::= 1 to (# analog channels) in NR1 format

The :SBUS<n>:M1553:SOURce command sets the source of the MIL-STD 1553 signal.

Use the :TRIGger:LEVel:HIGH and :TRIGger:LEVel:LOW commands to set the threshold levels for the selected source.

Query Syntax :SBUS<n>:M1553:TRIGger:SOURce?

The :SBUS<n>:M1553:SOURce? query returns the currently set source of the MIL-STD 1553 signal.

Return Format <source><NL>

<source> ::= {CHAN<n>}

<n> ::= 1 to (# analog channels) in NR1 format

- See Also**
- [":TRIGger:LEVel:HIGH"](#) on page 1356
 - [":TRIGger:LEVel:LOW"](#) on page 1357
 - [":TRIGger:MODE"](#) on page 1358
 - ["Introduction to :TRIGger Commands"](#) on page 1345

:SBUS<n>:M1553:TRIGger:PATtern:DATA

N (see [page 1650](#))

Command Syntax :SBUS<n>:M1553:TRIGger:PATtern:DATA <string>

<string> ::= "nn...n" where n ::= {0 | 1 | x}

The :SBUS<n>:M1553:TRIGger:PATtern:DATA command sets the 11 bits to trigger on if the trigger type has been set to RTA11 (RTA + 11 Bits) using the :SBUS<n>:M1553:TRIGger:TYPE command.

Query Syntax :SBUS<n>:M1553:TRIGger:PATtern:DATA?

The :SBUS<n>:M1553:TRIGger:PATtern:DATA? query returns the current 11-bit setting.

Return Format <string><NL>

<string> ::= "nn...n" where n ::= {0 | 1 | x}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:M1553:TRIGger:TYPE"](#) on page 1044
 - [":SBUS<n>:M1553:TRIGger:RTA"](#) on page 1043

:SBUS<n>:M1553:TRIGger:RTA

N (see [page 1650](#))

Command Syntax :SBUS<n>:M1553:TRIGger:RTA <value>

<value> ::= 5-bit integer in decimal, <nondecimal>, or
<string> from 0-31

<nondecimal> ::= #Hnn where n ::= {0,...,9|A,...,F}

<string> ::= "0xnn" where n ::= {0,...,9|A,...,F}

The :SBUS<n>:M1553:TRIGger:RTA command sets the Remote Terminal Address (RTA) to trigger on when the trigger type has been set to RTA or RTA11 (using the :SBUS<n>:M1553:TRIGger:TYPE command).

To set the RTA value to don't cares (0xxx), set the value to -1.

Query Syntax :SBUS<n>:M1553:TRIGger:RTA?

The :SBUS<n>:M1553:TRIGger:RTA? query returns the RTA value.

Return Format <value><NL> in decimal format

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:M1553:TRIGger:TYPE"](#) on page 1044

:SBUS<n>:M1553:TRIGger:TYPE

N (see [page 1650](#))

Command Syntax :SBUS<n>:M1553:TRIGger:TYPE <type>

<type> ::= {DSTArt | DSTOp | CSTArt | CSTOp | RTA | PERRor | SERRor
| MERRor | RTA11}

The :SBUS<n>:M1553:TRIGger:TYPE command specifies the type of MIL-STD-1553 trigger to be used:

- DSTArt – (Data Word Start) triggers on the start of a Data word (at the end of a valid Data Sync pulse).
- DSTOp – (Data Word Stop) triggers on the end of a Data word.
- CSTArt – (Command/Status Word Start) triggers on the start of Command/Status word (at the end of a valid C/S Sync pulse).
- CSTOp – (Command/Status Word Stop) triggers on the end of a Command/Status word.
- RTA – (Remote Terminal Address) triggers if the RTA of the Command/Status word matches the specified value. The value is specified in hex.
- RTA11 – (RTA + 11 Bits) triggers if the RTA and the remaining 11 bits match the specified criteria. The RTA can be specified as a hex value, and the remaining 11 bits can be specified as a 1, 0, or X (don't care).
- PERRor – (Parity Error) triggers if the (odd) parity bit is incorrect for the data in the word.
- MERRor – (Manchester Error) triggers if a Manchester encoding error is detected.
- SERRor – (Sync Error) triggers if an invalid Sync pulse is found.

Query Syntax :SBUS<n>:M1553:TRIGger:TYPE?

The :SBUS<n>:M1553:TRIGger:TYPE? query returns the currently set MIL-STD-1553 trigger type.

Return Format <type><NL>

<type> ::= {DSTA | DSTO | CSTA | CSTO | RTA | PERR | SERR
| MERR | RTA11}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:M1553:TRIGger:RTA"](#) on page 1043
 - [":SBUS<n>:M1553:TRIGger:PATtern:DATA"](#) on page 1042
 - [":TRIGger:MODE"](#) on page 1358

:SBUS<n>:MANChester Commands

NOTE

These commands are valid when the automotive MANChester serial decode and triggering option has been licensed.

Table 129 :SBUS<n>:MANChester Commands Summary

Command	Query	Options and Query Returns
:SBUS<n>:MANChester:BASE <base> (see page 1047)	:SBUS<n>:MANChester:BASE? (see page 1047)	<base> ::= {HEX DECimal ASCii}
:SBUS<n>:MANChester:BAUDRate <baudrate> (see page 1048)	:SBUS<n>:MANChester:BAUDRate? (see page 1048)	<baudrate> ::= integer from 500 to 5000000 in 100 b/s increments
:SBUS<n>:MANChester:BITOrder <bitorder> (see page 1049)	:SBUS<n>:MANChester:BITOrder? (see page 1049)	<bitorder> ::= {MSBFirst LSBFirst}
:SBUS<n>:MANChester:DISPlay <format> (see page 1050)	:SBUS<n>:MANChester:DISPlay? (see page 1050)	<format> ::= {BIT WORD}
:SBUS<n>:MANChester:DISize {AUTO <#words>} (see page 1051)	:SBUS<n>:MANChester:DISize? (see page 1051)	<#words> ::= from 1-255, in NR1 format
:SBUS<n>:MANChester:HSize <#bits> (see page 1052)	:SBUS<n>:MANChester:HSize? (see page 1052)	<#bits> ::= from 0-32, in NR1 format
:SBUS<n>:MANChester:IDLE:BITS <#bits> (see page 1053)	:SBUS<n>:MANChester:IDLE:BITS? (see page 1053)	<#bits> ::= minimum idle time in terms of bit width, from 1.50 to 32.0, in NR3 format.
:SBUS<n>:MANChester:LOGic <logic> (see page 1054)	:SBUS<n>:MANChester:LOGic? (see page 1054)	<logic> ::= {FALLing RISing}
:SBUS<n>:MANChester:SOURce <source> (see page 1055)	:SBUS<n>:MANChester:SOURce? (see page 1055)	<source> ::= {CHANnel<n> DIGital<d>}
:SBUS<n>:MANChester:SSize <#bits> (see page 1056)	:SBUS<n>:MANChester:SSize? (see page 1056)	<#bits> ::= from 0-255, in NR1 format
:SBUS<n>:MANChester:STARt <edge#> (see page 1057)	:SBUS<n>:MANChester:STARt? (see page 1057)	<edge#> ::= from 1-256, in NR1 format

Table 129 :SBUS<n>:MANchester Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:MANchester:TOLerance <percent> (see page 1058)	:SBUS<n>:MANchester:TOLerance? (see page 1058)	<percent> ::= from 1-30, in NR1 format
:SBUS<n>:MANchester:TRIGger <mode> (see page 1059)	:SBUS<n>:MANchester:TRIGger? (see page 1059)	<mode> ::= {SOF VALue MERRor}
:SBUS<n>:MANchester:TRIGger:PATtern:VALue:DATA <string> (see page 1060)	:SBUS<n>:MANchester:TRIGger:PATtern:VALue:DATA? (see page 1060)	<string> ::= "nn...n" where n ::= {0 1 X \$} <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X \$}
:SBUS<n>:MANchester:TRIGger:PATtern:VALue:WIDTh <width> (see page 1061)	:SBUS<n>:MANchester:TRIGger:PATtern:VALue:WIDTh? (see page 1061)	<width> ::= integer from 4 to 128 in NR1 format
:SBUS<n>:MANchester:TSIZE <#bits> (see page 1062)	:SBUS<n>:MANchester:TSIZE? (see page 1062)	<#bits> ::= from 0-32, in NR1 format
:SBUS<n>:MANchester:WSIZE <#bits> (see page 1063)	:SBUS<n>:MANchester:WSIZE? (see page 1063)	<#bits> ::= from 2-32, in NR1 format

:SBUS<n>:MANChesTer:BASE

N (see [page 1650](#))

Command Syntax :SBUS<n>:MANChesTer:BASE <base>

<base> ::= {HEX | DECimal | ASCii | BINary}

When the display format is WORD (see :SBUS<n>:MANChesTer:DISPlay), the :SBUS<n>:MANChesTer:BASE command specifies the base for the Manchester bus decode and Lister display.

- HEX – hexadecimal
- DECimal – unsigned decimal
- ASCii

When the display format is BIT, the only legal decode base value is BINary.

Query Syntax :SBUS<n>:MANChesTer:BASE?

The :SBUS<n>:MANChesTer:BASE? query returns the decode number base setting.

Return Format <base><NL>

<base> ::= {HEX | DEC | ASC | BIN}

- See Also**
- [":SBUS<n>:MANChesTer:BITorder"](#) on page 1049
 - [":SBUS<n>:MANChesTer:IDLE:BITS"](#) on page 1053
 - [":SBUS<n>:MANChesTer:LOGic"](#) on page 1054
 - [":SBUS<n>:MANChesTer:START"](#) on page 1057

:SBUS<n>:MANChesTer:BAUDrate

N (see [page 1650](#))

Command Syntax :SBUS<n>:MANChesTer:BAUDrate <baudrate>

<baudrate> ::= integer from 500 to 5000000 in 100 b/s increments

The :SBUS<n>:MANChesTer:BAUDrate command specifies the baud rate of the Manchester signal.

Query Syntax :SBUS<n>:MANChesTer:BAUDrate?

The :SBUS<n>:MANChesTer:BAUDrate? query returns the specified baud rate.

Return Format <baudrate><NL>

- See Also**
- [":SBUS<n>:MANChesTer:SOURce"](#) on page 1055
 - [":SBUS<n>:MANChesTer:TOLerance"](#) on page 1058

:SBUS<n>:MANChester:BITOrder

N (see [page 1650](#))

Command Syntax :SBUS<n>:MANChester:BITOrder <bitorder>

<bitorder> ::= {MSBFirst | LSBFirst}

When the display format is WORD (see :SBUS<n>:MANChester:DISPlay), the :SBUS<n>:MANChester:BITOrder command specifies the order of transmission on the Manchester bus:

- MSBFirst – specifies the most significant bit is transmitted first.
- LSBFirst – specifies the least significant bit is transmitted first.

Query Syntax :SBUS<n>:MANChester:BITOrder?

The :SBUS<n>:MANChester:BITOrder? query returns the bit order setting.

Return Format <bitorder><NL>

<bitorder> ::= {MSBF | LSBF}

- See Also**
- [":SBUS<n>:MANChester:BASE"](#) on page 1047
 - [":SBUS<n>:MANChester:IDLE:BITS"](#) on page 1053
 - [":SBUS<n>:MANChester:LOGic"](#) on page 1054
 - [":SBUS<n>:MANChester:START"](#) on page 1057

:SBUS<n>:MANChester:DISPlay

N (see [page 1650](#))

Command Syntax :SBUS<n>:MANChester:DISPlay <format>

<format> ::= {BIT | WORD}

The :SBUS<n>:MANChester:DISPlay command specifies the format of the Manchester bus display.

Query Syntax :SBUS<n>:MANChester:DISPlay?

The :SBUS<n>:MANChester:DISPlay? query returns the bus display format setting.

Return Format <format><NL>

<format> ::= {BIT | WORD}

- See Also**
- [":SBUS<n>:MANChester:DSIZE"](#) on page 1051
 - [":SBUS<n>:MANChester:HSIZE"](#) on page 1052
 - [":SBUS<n>:MANChester:SSIZE"](#) on page 1056
 - [":SBUS<n>:MANChester:TSIZE"](#) on page 1062
 - [":SBUS<n>:MANChester:WSIZE"](#) on page 1063

:SBUS<n>:MANChester:DSIZe

N (see [page 1650](#))

Command Syntax :SBUS<n>:MANChester:DSIZe {AUTO | <#words>}

<#words> ::= from 1-255, in NR1 format

When the display format is WORD (see :SBUS<n>:MANChester:DISPlay), the :SBUS<n>:MANChester:DSIZe command specifies the number of words in the data field of your Manchester protocol definition.

AUTO is available as a selection only when the trailer field size is 0 (see :SBUS<n>:MANChester:TSIZe).

Query Syntax :SBUS<n>:MANChester:DSIZe?

The :SBUS<n>:MANChester:DSIZe? query returns the number of data field words setting.

Return Format <#words><NL>

<#words> ::= from 0-255, in NR1 format

In AUTO mode, the query returns 0.

In BIT format the only legal value is 0 (for AUTO).

- See Also**
- [":SBUS<n>:MANChester:DISPlay"](#) on page 1050
 - [":SBUS<n>:MANChester:HSIZe"](#) on page 1052
 - [":SBUS<n>:MANChester:SSIZe"](#) on page 1056
 - [":SBUS<n>:MANChester:TSIZe"](#) on page 1062
 - [":SBUS<n>:MANChester:WSIZe"](#) on page 1063

:SBUS<n>:MANChester:HSIZe

N (see [page 1650](#))

Command Syntax :SBUS<n>:MANChester:HSIZe <#bits>

<#bits> ::= from 0-32, in NR1 format

When the display format is WORD (see :SBUS<n>:MANChester:DISPlay), the :SBUS<n>:MANChester:HSIZe command specifies the number of bits in the header field of your Manchester protocol definition.

Query Syntax :SBUS<n>:MANChester:HSIZe?

The :SBUS<n>:MANChester:HSIZe? query returns the number of header field bits setting.

Return Format <#bits><NL>

- See Also**
- [":SBUS<n>:MANChester:DISPlay"](#) on page 1050
 - [":SBUS<n>:MANChester:DSIZe"](#) on page 1051
 - [":SBUS<n>:MANChester:SSIZe"](#) on page 1056
 - [":SBUS<n>:MANChester:TSIZe"](#) on page 1062
 - [":SBUS<n>:MANChester:WSIZe"](#) on page 1063

:SBUS<n>:MANChester:IDLE:BITS

N (see [page 1650](#))

Command Syntax :SBUS<n>:MANChester:IDLE:BITS <#bits>

<#bits> ::= minimum idle time, from 1.50 to 32.00 in 0.25 increments,
in NR3 format.

The :SBUS<n>:MANChester:IDLE:BITS command specifies the minimum idle time or inter-frame gap time in terms of the number of bits.

Query Syntax :SBUS<n>:MANChester:IDLE:BITS?

The :SBUS<n>:MANChester:IDLE:BITS? query returns the specified idle time in terms of the number of bits.

Return Format <#bits><NL>

- See Also**
- [":SBUS<n>:MANChester:BASE"](#) on page 1047
 - [":SBUS<n>:MANChester:BITOrder"](#) on page 1049
 - [":SBUS<n>:MANChester:LOGic"](#) on page 1054
 - [":SBUS<n>:MANChester:START"](#) on page 1057

:SBUS<n>:MANchester:LOGic

N (see [page 1650](#))

Command Syntax :SBUS<n>:MANchester:LOGic <logic>

<logic> ::= {FALLing | RISing}

The :SBUS<n>:MANchester:LOGic command specifies the polarity of the Manchester signal:

- FALLing – specifies that a falling edge is used to encode a bit value of logic 1 (and a rising edge encodes a bit value of logic 0).
- RISing – specifies that a rising edge is used to encode a bit value of logic 1.

Query Syntax :SBUS<n>:MANchester:LOGic?

The :SBUS<n>:MANchester:LOGic? query returns the polarity setting.

Return Format <logic><NL>

<logic> ::= {FALL | RIS}

- See Also**
- [":SBUS<n>:MANchester:BASE"](#) on page 1047
 - [":SBUS<n>:MANchester:BITorder"](#) on page 1049
 - [":SBUS<n>:MANchester:IDLE:BITS"](#) on page 1053
 - [":SBUS<n>:MANchester:START"](#) on page 1057

:SBUS<n>:MANChester:SOURce

N (see [page 1650](#))

Command Syntax :SBUS<n>:MANChester:SOURce <source>

<source> ::= {CHANnel<n> | DIGital<d>}

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :SBUS<n>:MANChester:SOURce command selects the oscilloscope channel connected to the Manchester signal line.

Query Syntax :SBUS<n>:MANChester:SOURce?

The :SBUS<n>:MANChester:SOURce? query returns the selected oscilloscope channel source.

Return Format <source><NL>

<source> ::= {CHAN<n> | DIG<d>}

- See Also**
- [":SBUS<n>:MANChester:BAUDrate"](#) on page 1048
 - [":SBUS<n>:MANChester:TOLerance"](#) on page 1058

:SBUS<n>:MANchester:SSIZe

N (see [page 1650](#))

Command Syntax :SBUS<n>:MANchester:SSIZe <#bits>

<#bits> ::= from 0-255, in NR1 format

The :SBUS<n>:MANchester:SSIZe command specifies the number of sync bits for the Manchester signal.

Query Syntax :SBUS<n>:MANchester:SSIZe?

The :SBUS<n>:MANchester:SSIZe? query returns the number of sync bits setting.

Return Format <#bits><NL>

- See Also**
- [":SBUS<n>:MANchester:DISPlay"](#) on page 1050
 - [":SBUS<n>:MANchester:DSIZe"](#) on page 1051
 - [":SBUS<n>:MANchester:HSIZe"](#) on page 1052
 - [":SBUS<n>:MANchester:TSIZe"](#) on page 1062
 - [":SBUS<n>:MANchester:WSIZe"](#) on page 1063

:SBUS<n>:MANChesTer:STARt

N (see [page 1650](#))

Command Syntax :SBUS<n>:MANChesTer:STARt <edge#>

<edge#> ::= from 1-256, in NR1 format

The :SBUS<n>:MANChesTer:STARt command specifies the starting edge of the Manchester signal.

Query Syntax :SBUS<n>:MANChesTer:STARt?

The :SBUS<n>:MANChesTer:STARt? query returns the starting edge number setting.

Return Format <edge#><NL>

- See Also**
- [":SBUS<n>:MANChesTer:BASE"](#) on page 1047
 - [":SBUS<n>:MANChesTer:BITOrder"](#) on page 1049
 - [":SBUS<n>:MANChesTer:IDLE:BITS"](#) on page 1053
 - [":SBUS<n>:MANChesTer:LOGic"](#) on page 1054

:SBUS<n>:MANChester:TOLerance

N (see [page 1650](#))

Command Syntax :SBUS<n>:MANChester:TOLerance <percent>

<percent> ::= from 5-30, in NR1 format

The :SBUS<n>:MANChester:TOLerance command specifies the tolerance for the Manchester signal in terms of the percentage of the bit period.

Query Syntax :SBUS<n>:MANChester:TOLerance?

The :SBUS<n>:MANChester:TOLerance? query returns the tolerance setting.

Return Format <percent><NL>

<percent> ::= from 5-30, in NR1 format

- See Also**
- [":SBUS<n>:MANChester:BAUDrate"](#) on page 1048
 - [":SBUS<n>:MANChester:SOURce"](#) on page 1055

:SBUS<n>:MANChester:TRIGger

N (see [page 1650](#))

Command Syntax :SBUS<n>:MANChester:TRIGger <mode>

<mode> ::= {SOF | VALue | MERRor}

The :SBUS<n>:MANChester:TRIGger command specifies the trigger mode:

- SOF (Start Of Frame) – triggers at the start of a Manchester frame, after the starting edge.
- VALue – triggers on the specified bit values.
- MERRor – triggers on a Manchester error.

Query Syntax :SBUS<n>:MANChester:TRIGger?

The :SBUS<n>:MANChester:TRIGger? query returns the trigger mode setting.

Return Format <mode><NL>

<mode> ::= {SOF | VAL | MERR}

- See Also**
- [":SBUS<n>:MANChester:TRIGger:PATtern:VALue:DATA"](#) on page 1060
 - [":SBUS<n>:MANChester:TRIGger:PATtern:VALue:WIDTh"](#) on page 1061

:SBUS<n>:MANChester:TRIGger:PATtern:VALue:DATA

N (see [page 1650](#))

Command Syntax :SBUS<n>:MANChester:TRIGger:PATtern:VALue:DATA <string>

<string> ::= "nn...n" where n ::= {0 | 1 | X | \$}

<string> ::= "0xnn...n" where n ::= {0,...,9 | A,...,F | X | \$}

When the VALue trigger mode is selected (:SBUS<n>:MANChester:TRIGger), the :SBUS<n>:MANChester:TRIGger:PATtern:VALue:DATA command specifies the value to trigger on.

Note that the trigger value bit order is always for bits as they arrive (that is, MSB first) regardless of the serial decode bit order setting (in :SBUS<n>:MANChester:BITorder).

The bit width (length) of the value is set with the :SBUS<n>:MANChester:TRIGger:PATtern:VALue:WIDTH command.

Query Syntax :SBUS<n>:MANChester:TRIGger:PATtern:VALue:DATA?

The :SBUS<n>:MANChester:TRIGger:PATtern:VALue:DATA? query returns the specified trigger value as a string of binary digits.

Return Format <string><NL>

<string> ::= "nn...n" where n ::= {0 | 1 | X | \$}

- See Also**
- [":SBUS<n>:MANChester:BITorder"](#) on page 1049
 - [":SBUS<n>:MANChester:TRIGger"](#) on page 1059
 - [":SBUS<n>:MANChester:TRIGger:PATtern:VALue:WIDTH"](#) on page 1061

:SBUS<n>:MANChester:TRIGger:PATtern:VALue:WIDTh

N (see [page 1650](#))

Command Syntax :SBUS<n>:MANChester:TRIGger:PATtern:VALue:WIDTh <width>

<width> ::= integer from 4 to 128 in NR1 format

When the VALue trigger mode is selected (:SBUS<n>:MANChester:TRIGger), the :SBUS<n>:MANChester:TRIGger:PATtern:VALue:WIDTh command specifies the bit width (length) of the value to trigger on.

The actual value to trigger on is set with the :SBUS<n>:MANChester:TRIGger:PATtern:VALue:DATA command.

Query Syntax :SBUS<n>:MANChester:TRIGger:PATtern:VALue:WIDTh?

The :SBUS<n>:MANChester:TRIGger:PATtern:VALue:WIDTh? query returns the specified trigger value bit width (length).

Return Format <width><NL>

- See Also**
- [":SBUS<n>:MANChester:BITorder"](#) on page 1049
 - [":SBUS<n>:MANChester:TRIGger"](#) on page 1059
 - [":SBUS<n>:MANChester:TRIGger:PATtern:VALue:DATA"](#) on page 1060

:SBUS<n>:MANChester:TSIZe

N (see [page 1650](#))

Command Syntax :SBUS<n>:MANChester:TSIZe <#bits>

<#bits> ::= from 0-32, in NR1 format

When the display format is WORD (see :SBUS<n>:MANChester:DISPlay), the :SBUS<n>:MANChester:TSIZe command specifies the number of bits in the trailer field of your Manchester protocol definition.

Query Syntax :SBUS<n>:MANChester:TSIZe?

The :SBUS<n>:MANChester:TSIZe? query returns the number of trailer field bits setting.

Return Format <#bits><NL>

<#bits> ::= from 0-32, in NR1 format

- See Also**
- [":SBUS<n>:MANChester:DISPlay"](#) on page 1050
 - [":SBUS<n>:MANChester:DSIZe"](#) on page 1051
 - [":SBUS<n>:MANChester:HSIZe"](#) on page 1052
 - [":SBUS<n>:MANChester:SSIZe"](#) on page 1056
 - [":SBUS<n>:MANChester:WSIZe"](#) on page 1063

:SBUS<n>:MANChester:WSize

N (see [page 1650](#))

Command Syntax :SBUS<n>:MANChester:WSize <#bits>

<#bits> ::= from 2-32, in NR1 format

When the display format is WORD (see :SBUS<n>:MANChester:DISPlay), the :SBUS<n>:MANChester:WSize command the number of bits per word in the data field of your Manchester protocol definition.

Query Syntax :SBUS<n>:MANChester:WSize?

The :SBUS<n>:MANChester:WSize? query returns the number of bits per word setting.

Return Format <#bits><NL>

- See Also**
- [":SBUS<n>:MANChester:DISPlay"](#) on page 1050
 - [":SBUS<n>:MANChester:DSIZE"](#) on page 1051
 - [":SBUS<n>:MANChester:HSIZE"](#) on page 1052
 - [":SBUS<n>:MANChester:SSIZE"](#) on page 1056
 - [":SBUS<n>:MANChester:TSIZE"](#) on page 1062

:SBUS<n>:NRZ Commands

NOTE

These commands are valid when the automotive NRZ serial decode and triggering option has been licensed.

Table 130 :SBUS<n>:NRZ Commands Summary

Command	Query	Options and Query Returns
:SBUS<n>:NRZ:BASE <base> (see page 1066)	:SBUS<n>:NRZ:BASE? (see page 1066)	<base> ::= {HEX DECimal ASCii}
:SBUS<n>:NRZ:BAUDrate <baudrate> (see page 1067)	:SBUS<n>:NRZ:BAUDrate? (see page 1067)	<baudrate> ::= integer from 5000 to 5000000 in 100 b/s increments
:SBUS<n>:NRZ:BITorder <bitorder> (see page 1068)	:SBUS<n>:NRZ:BITorder? (see page 1068)	<bitorder> ::= {MSBFirst LSBFirst}
:SBUS<n>:NRZ:DISPlay <format> (see page 1069)	:SBUS<n>:NRZ:DISPlay? (see page 1069)	<format> ::= {BIT WORD}
:SBUS<n>:NRZ:DSIZE <#words> (see page 1070)	:SBUS<n>:NRZ:DSIZE? (see page 1070)	<#words> ::= from 1-255, in NR1 format
:SBUS<n>:NRZ:FSize <#bits> (see page 1071)	:SBUS<n>:NRZ:FSize? (see page 1071)	<#bits> ::= from 2-255, in NR1 format
:SBUS<n>:NRZ:HSize <#bits> (see page 1072)	:SBUS<n>:NRZ:HSize? (see page 1072)	<#bits> ::= from 0-32, in NR1 format
:SBUS<n>:NRZ:IDLE:BIT S <#bits> (see page 1073)	:SBUS<n>:NRZ:IDLE:BIT S? (see page 1073)	<#bits> ::= minimum idle time in terms of bit width, from 1.50 to 32.0, in NR3 format.
:SBUS<n>:NRZ:IDLE:STA Te <state> (see page 1074)	:SBUS<n>:NRZ:IDLE:STA Te? (see page 1074)	<state> ::= {LOW HIGH}
:SBUS<n>:NRZ:LOGic <logic> (see page 1075)	:SBUS<n>:NRZ:LOGic? (see page 1075)	<logic> ::= {HIGH LOW}
:SBUS<n>:NRZ:SOURce <source> (see page 1076)	:SBUS<n>:NRZ:SOURce? (see page 1076)	<source> ::= {CHANnel<n> DIGital<d>}

Table 130 :SBUS<n>:NRZ Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:NRZ:START <#bits> (see page 1077)	:SBUS<n>:NRZ:START? (see page 1077)	<#bits> ::= from 0-255, in NR1 format
:SBUS<n>:NRZ:TRIGger <mode> (see page 1078)	:SBUS<n>:NRZ:TRIGger? (see page 1078)	<mode> ::= {SOF VALue}
:SBUS<n>:NRZ:TRIGger: PATtern:VALue:DATA <string> (see page 1079)	:SBUS<n>:NRZ:TRIGger: PATtern:VALue:DATA? (see page 1079)	<string> ::= "nn...n" where n ::= {0 1 X \$} <string ::= "0xnn...n" where n ::= {0,...,9 A,...,F X \$}
:SBUS<n>:NRZ:TRIGger: PATtern:VALue:WIDTh <width> (see page 1080)	:SBUS<n>:NRZ:TRIGger: PATtern:VALue:WIDTh? (see page 1080)	<width> ::= integer from 4 to 128 in NR1 format
:SBUS<n>:NRZ:TSIZe <#bits> (see page 1081)	:SBUS<n>:NRZ:TSIZe? (see page 1081)	<#bits> ::= from 0-32, in NR1 format
:SBUS<n>:NRZ:WSIZe <#bits> (see page 1082)	:SBUS<n>:NRZ:WSIZe? (see page 1082)	<#bits> ::= from 2-32, in NR1 format

:SBUS<n>:NRZ:BASE

N (see [page 1650](#))

Command Syntax :SBUS<n>:NRZ:BASE <base>

<base> ::= {HEX | DECimal | ASCii | BINary}

When the display format is WORD (see :SBUS<n>:NRZ:DISPlay), the :SBUS<n>:NRZ:BASE command specifies the base for the NRZ bus decode and Lister display.

- HEX – hexadecimal
- DECimal – unsigned decimal
- ASCii

When the display format is BIT, the only legal decode base value is BINary.

Query Syntax :SBUS<n>:NRZ:BASE?

The :SBUS<n>:NRZ:BASE? query returns the decode number base setting.

Return Format <base><NL>

<base> ::= {HEX | DEC | ASC | BIN}

- See Also**
- **":SBUS<n>:NRZ:BITorder"** on page 1068
 - **":SBUS<n>:NRZ:IDLE:BITS"** on page 1073
 - **":SBUS<n>:NRZ:IDLE:STATe"** on page 1074
 - **":SBUS<n>:NRZ:LOGic"** on page 1075

:SBUS<n>:NRZ:BAUDrate

N (see [page 1650](#))

Command Syntax :SBUS<n>:NRZ:BAUDrate <baudrate>

<baudrate> ::= integer from 5000 to 5000000 in 100 b/s increments

The :SBUS<n>:NRZ:BAUDrate command specifies the baud rate of the NRZ signal.

Query Syntax :SBUS<n>:NRZ:BAUDrate?

The :SBUS<n>:NRZ:BAUDrate? query returns the specified baud rate.

Return Format <baudrate><NL>

See Also • [":SBUS<n>:NRZ:SOURce"](#) on page 1076

:SBUS<n>:NRZ:BITOrder

N (see [page 1650](#))

Command Syntax :SBUS<n>:NRZ:BITOrder <bitorder>

<bitorder> ::= {MSBFirst | LSBFirst}

When the display format is WORD (see :SBUS<n>:NRZ:DISPlay), the :SBUS<n>:NRZ:BITOrder command specifies the order of transmission on the NRZ bus:

- MSBFirst – specifies the most significant bit is transmitted first.
- LSBFirst – specifies the least significant bit is transmitted first.

Query Syntax :SBUS<n>:NRZ:BITOrder?

The :SBUS<n>:NRZ:BITOrder? query returns the bit order setting.

Return Format <bitorder><NL>

<bitorder> ::= {MSBF | LSBF}

- See Also**
- [":SBUS<n>:NRZ:BASE"](#) on page 1066
 - [":SBUS<n>:NRZ:IDLE:BITS"](#) on page 1073
 - [":SBUS<n>:NRZ:IDLE:STATe"](#) on page 1074
 - [":SBUS<n>:NRZ:LOGic"](#) on page 1075

:SBUS<n>:NRZ:DISPlay

N (see [page 1650](#))

Command Syntax :SBUS<n>:NRZ:DISPlay <format>

<format> ::= {BIT | WORD}

The :SBUS<n>:NRZ:DISPlay command specifies the format of the NRZ bus display.

Query Syntax :SBUS<n>:NRZ:DISPlay?

The :SBUS<n>:NRZ:DISPlay? query returns the bus display format setting.

Return Format <format><NL>

<format> ::= {BIT | WORD}

- See Also**
- [":SBUS<n>:NRZ:DSIZE"](#) on page 1070
 - [":SBUS<n>:NRZ:FSize"](#) on page 1071
 - [":SBUS<n>:NRZ:HSize"](#) on page 1072
 - [":SBUS<n>:NRZ:STArT"](#) on page 1077
 - [":SBUS<n>:NRZ:TSize"](#) on page 1081
 - [":SBUS<n>:NRZ:WSize"](#) on page 1082

:SBUS<n>:NRZ:DSIZe

N (see [page 1650](#))

Command Syntax :SBUS<n>:NRZ:DSIZe <#words>
 <#words> ::= from 1-255, in NR1 format

The :SBUS<n>:NRZ:DSIZe command ...

When the display format is WORD (see :SBUS<n>:NRZ:DISPlay), the :SBUS<n>:NRZ:DSIZe command specifies the number of words in the data field of your NRZ protocol definition.

AUTO is available as a selection only when the trailer field size is 0 (see :SBUS<n>:NRZ:TSIZe).

Query Syntax :SBUS<n>:NRZ:DSIZe?

The :SBUS<n>:NRZ:DSIZe? query returns the number of data field words setting.

Return Format <#words><NL>
 <#words> ::= from 1-255, in NR1 format

In AUTO mode, the query returns 0.

In BIT format the only legal value is 0 (for AUTO).

- See Also**
- [":SBUS<n>:NRZ:DISPlay"](#) on page 1069
 - [":SBUS<n>:NRZ:FSIZe"](#) on page 1071
 - [":SBUS<n>:NRZ:HSIZe"](#) on page 1072
 - [":SBUS<n>:NRZ:START"](#) on page 1077
 - [":SBUS<n>:NRZ:TSIZe"](#) on page 1081
 - [":SBUS<n>:NRZ:WSIZe"](#) on page 1082

:SBUS<n>:NRZ:FSIZe

N (see [page 1650](#))

Command Syntax :SBUS<n>:NRZ:FSIZe <#bits>

<#bits> ::= from 2-255, in NR1 format

When the NRZ bus display format (:SBUS<n>:NRZ:DISPlay) is BIT, the :SBUS<n>:NRZ:FSIZe command lets you specify the total frame size of the NRZ signal from 2 to 255 bits. This would be equivalent to the sum of the number of bits in the header, data, and trailer fields in WORD format.

Query Syntax :SBUS<n>:NRZ:FSIZe?

The :SBUS<n>:NRZ:FSIZe? query returns the specified total frame size.

Return Format <#bits><NL>

- See Also**
- [":SBUS<n>:NRZ:DISPlay"](#) on page 1069
 - [":SBUS<n>:NRZ:DSIZe"](#) on page 1070
 - [":SBUS<n>:NRZ:HSIZe"](#) on page 1072
 - [":SBUS<n>:NRZ:START"](#) on page 1077
 - [":SBUS<n>:NRZ:TSIZe"](#) on page 1081
 - [":SBUS<n>:NRZ:WSIZe"](#) on page 1082

:SBUS<n>:NRZ:HSIZe

N (see [page 1650](#))

Command Syntax :SBUS<n>:NRZ:HSIZe <#bits>

<#bits> ::= from 0-32, in NR1 format

When the display format is WORD (see :SBUS<n>:NRZ:DISPlay), the :SBUS<n>:NRZ:HSIZe command specifies the number of bits in the header field of your NRZ protocol definition.

Query Syntax :SBUS<n>:NRZ:HSIZe?

The :SBUS<n>:NRZ:HSIZe? query returns the number of header field bits setting.

Return Format <#bits><NL>

- See Also**
- [":SBUS<n>:NRZ:DISPlay"](#) on page 1069
 - [":SBUS<n>:NRZ:DSIZe"](#) on page 1070
 - [":SBUS<n>:NRZ:FSIZe"](#) on page 1071
 - [":SBUS<n>:NRZ:STARt"](#) on page 1077
 - [":SBUS<n>:NRZ:TSIZe"](#) on page 1081
 - [":SBUS<n>:NRZ:WSIZe"](#) on page 1082

:SBUS<n>:NRZ:IDLE:BITS

N (see [page 1650](#))

Command Syntax :SBUS<n>:NRZ:IDLE:BITS <#bits>

<#bits> ::= minimum idle time, from 1.50 to 32.00 in 0.25 increments,
in NR3 format.

The :SBUS<n>:NRZ:IDLE:BITS command specifies the minimum idle time or inter-frame gap time in terms of the number of bits.

Query Syntax :SBUS<n>:NRZ:IDLE:BITS?

The :SBUS<n>:NRZ:IDLE:BITS? query returns the specified idle time in terms of the number of bits.

Return Format <#bits><NL>

- See Also**
- [":SBUS<n>:NRZ:BASE"](#) on page 1066
 - [":SBUS<n>:NRZ:BITorder"](#) on page 1068
 - [":SBUS<n>:NRZ:IDLE:STATe"](#) on page 1074
 - [":SBUS<n>:NRZ:LOGic"](#) on page 1075

:SBUS<n>:NRZ:IDLE:STATe

N (see [page 1650](#))

Command Syntax :SBUS<n>:NRZ:IDLE:STATe <state>

<state> ::= {LOW | HIGH}

The :SBUS<n>:NRZ:IDLE:STATe command specifies the idle state of the NRZ signal.

Query Syntax :SBUS<n>:NRZ:IDLE:STATe?

The :SBUS<n>:NRZ:IDLE:STATe? query returns the idle state setting.

Return Format <state><NL>

<state> ::= {LOW | HIGH}

- See Also**
- [":SBUS<n>:NRZ:BASE"](#) on page 1066
 - [":SBUS<n>:NRZ:BITorder"](#) on page 1068
 - [":SBUS<n>:NRZ:IDLE:BITS"](#) on page 1073
 - [":SBUS<n>:NRZ:LOGic"](#) on page 1075

:SBUS<n>:NRZ:LOGic

N (see [page 1650](#))

Command Syntax :SBUS<n>:NRZ:LOGic <logic>

<logic> ::= {HIGH | LOW}

The :SBUS<n>:NRZ:LOGic command specifies the polarity of the NRZ signal:

- HIGH – specifies that a positive voltage is used to encode a bit value of logic 1 (and a negative voltage encodes a bit value of logic 0).
- LOW – specifies that a negative voltage is used to encode a bit value of logic 1.

Query Syntax :SBUS<n>:NRZ:LOGic?

The :SBUS<n>:NRZ:LOGic? query returns the polarity setting.

Return Format <logic><NL>

<logic> ::= {HIGH | LOW}

- See Also**
- [":SBUS<n>:NRZ:BASE"](#) on page 1066
 - [":SBUS<n>:NRZ:BITorder"](#) on page 1068
 - [":SBUS<n>:NRZ:IDLE:BITS"](#) on page 1073
 - [":SBUS<n>:NRZ:IDLE:STATe"](#) on page 1074

:SBUS<n>:NRZ:SOURce

N (see [page 1650](#))

Command Syntax :SBUS<n>:NRZ:SOURce <source>

<source> ::= {CHANnel<n> | DIGital<d>}

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :SBUS<n>:NRZ:SOURce command selects the oscilloscope channel connected to the NRZ signal

Query Syntax :SBUS<n>:NRZ:SOURce?

The :SBUS<n>:NRZ:SOURce? query returns the selected oscilloscope channel source.

Return Format <source><NL>

<source> ::= {CHAN<n> | DIG<d>}

See Also • [":SBUS<n>:NRZ:BAUDrate"](#) on page 1067

:SBUS<n>:NRZ:START

N (see [page 1650](#))

Command Syntax :SBUS<n>:NRZ:START <#bits>

<#bits> ::= from 0-255, in NR1 format

The :SBUS<n>:NRZ:START command specifies the number of start bits for the NRZ signal.

Query Syntax :SBUS<n>:NRZ:START?

The :SBUS<n>:NRZ:START? query returns the number of start bits setting.

Return Format <#bits><NL>

- See Also**
- [":SBUS<n>:NRZ:DISPlay"](#) on page 1069
 - [":SBUS<n>:NRZ:DSIZe"](#) on page 1070
 - [":SBUS<n>:NRZ:FSIZE"](#) on page 1071
 - [":SBUS<n>:NRZ:HSIZe"](#) on page 1072
 - [":SBUS<n>:NRZ:TSIZe"](#) on page 1081
 - [":SBUS<n>:NRZ:WSIZe"](#) on page 1082

:SBUS<n>:NRZ:TRIGger

N (see [page 1650](#))

Command Syntax :SBUS<n>:NRZ:TRIGger <mode>

<mode> ::= {SOF | VALue}

The :SBUS<n>:NRZ:TRIGger command specifies the trigger mode:

- SOF (Start Of Frame) – triggers at the start of a NRZ frame, before the header field.
- VALue – triggers on the specified bit values (see :SBUS<n>:NRZ:TRIGger:PATtern:VALue:WIDTh and :SBUS<n>:NRZ:TRIGger:PATtern:VALue:DATA), up to 128 bits, after the specified number of starting bits.

Note that the trigger value is always for bits as they arrive (that is, MSB first).

When the decode bit order (see :SBUS<n>:NRZ:BITorder) is MSBFirst, the decoded value bit order matches the trigger value bit order.

When the serial decode bit order is LSBFirst, the decoded value bit order is opposite of the trigger value bit order.

Query Syntax :SBUS<n>:NRZ:TRIGger?

The :SBUS<n>:NRZ:TRIGger? query returns the trigger mode setting.

Return Format <mode><NL>

<mode> ::= {SOF | VAL}

- See Also**
- [":SBUS<n>:NRZ:TRIGger:PATtern:VALue:DATA"](#) on page 1079
 - [":SBUS<n>:NRZ:TRIGger:PATtern:VALue:WIDTh"](#) on page 1080

:SBUS<n>:NRZ:TRIGger:PATtern:VALue:DATA

N (see [page 1650](#))

Command Syntax :SBUS<n>:NRZ:TRIGger:PATtern:VALue:DATA <string>

<string> ::= "nn...n" where n ::= {0 | 1 | X | \$}

<string> ::= "0xnn...n" where n ::= {0,...,9 | A,...,F | X | \$}

When the VALue trigger mode is selected (:SBUS<n>:NRZ:TRIGger), the :SBUS<n>:NRZ:TRIGger:PATtern:VALue:DATA command specifies the value to trigger on.

Note that the trigger value bit order is always for bits as they arrive (that is, MSB first) regardless of the serial decode bit order setting (in :SBUS<n>:NRZ:BITorder).

The bit width (length) of the value is set with the :SBUS<n>:NRZ:TRIGger:PATtern:VALue:WIDTH command.

Query Syntax :SBUS<n>:NRZ:TRIGger:PATtern:VALue:DATA?

The :SBUS<n>:NRZ:TRIGger:PATtern:VALue:DATA? query returns the specified trigger value as a string of binary digits.

Return Format <string><NL>

<string> ::= "nn...n" where n ::= {0 | 1 | X | \$}

- See Also**
- [":SBUS<n>:NRZ:BITorder"](#) on page 1068
 - [":SBUS<n>:NRZ:TRIGger"](#) on page 1078
 - [":SBUS<n>:NRZ:TRIGger:PATtern:VALue:WIDTH"](#) on page 1080

:SBUS<n>:NRZ:TRIGger:PATtern:VALue:WIDTh

N (see [page 1650](#))

Command Syntax :SBUS<n>:NRZ:TRIGger:PATtern:VALue:WIDTh <width>

<width> ::= integer from 4 to 128 in NR1 format

When the VALue trigger mode is selected (:SBUS<n>:NRZ:TRIGger), the :SBUS<n>:NRZ:TRIGger:PATtern:VALue:WIDTh command specifies the bit width (length) of the value to trigger on.

The actual value to trigger on is set with the :SBUS<n>:NRZ:TRIGger:PATtern:VALue:DATA command.

Query Syntax :SBUS<n>:NRZ:TRIGger:PATtern:VALue:WIDTh?

The :SBUS<n>:NRZ:TRIGger:PATtern:VALue:WIDTh? query returns the specified trigger value bit width (length).

Return Format <width><NL>

- See Also**
- [":SBUS<n>:NRZ:BITorder"](#) on page 1068
 - [":SBUS<n>:NRZ:TRIGger"](#) on page 1078
 - [":SBUS<n>:NRZ:TRIGger:PATtern:VALue:DATA"](#) on page 1079

:SBUS<n>:NRZ:TSIZe

N (see [page 1650](#))

Command Syntax :SBUS<n>:NRZ:TSIZe <#bits>

<#bits> ::= from 0-32, in NR1 format

When the display format is WORD (see :SBUS<n>:NRZ:DISPlay), the :SBUS<n>:NRZ:TSIZe command specifies the number of bits in the trailer field of your NRZ protocol definition.

Query Syntax :SBUS<n>:NRZ:TSIZe?

The :SBUS<n>:NRZ:TSIZe? query returns the number of trailer field bits setting.

Return Format <opt><NL>

<#bits> ::= from 0-32, in NR1 format

- See Also**
- [":SBUS<n>:NRZ:DISPlay"](#) on page 1069
 - [":SBUS<n>:NRZ:DSIZe"](#) on page 1070
 - [":SBUS<n>:NRZ:FSIZe"](#) on page 1071
 - [":SBUS<n>:NRZ:HSIZe"](#) on page 1072
 - [":SBUS<n>:NRZ:START"](#) on page 1077
 - [":SBUS<n>:NRZ:WSIZe"](#) on page 1082

:SBUS<n>:NRZ:WSize

N (see [page 1650](#))

Command Syntax :SBUS<n>:NRZ:WSize <#bits>

<#bits> ::= from 2-32, in NR1 format

When the display format is WORD (see :SBUS<n>:NRZ:DISPlay), the :SBUS<n>:NRZ:WSize command specifies the number of bits per word in the data field of your NRZ protocol definition.

Query Syntax :SBUS<n>:NRZ:WSize?

The :SBUS<n>:NRZ:WSize? query returns the number of bits per word setting.

Return Format <#bits><NL>

- See Also**
- [":SBUS<n>:NRZ:DISPlay"](#) on page 1069
 - [":SBUS<n>:NRZ:DSIZE"](#) on page 1070
 - [":SBUS<n>:NRZ:FSIZE"](#) on page 1071
 - [":SBUS<n>:NRZ:HSIZE"](#) on page 1072
 - [":SBUS<n>:NRZ:START"](#) on page 1077
 - [":SBUS<n>:NRZ:TSIZE"](#) on page 1081

:SBUS<n>:SENT Commands

NOTE

These commands are valid when the automotive SENT serial decode and triggering option has been licensed.

Table 131 :SBUS<n>:SENT Commands Summary

Command	Query	Options and Query Returns
:SBUS<n>:SENT:CLOCK <period> (see page 1086)	:SBUS<n>:SENT:CLOCK? (see page 1086)	<period> ::= the nominal clock period (tick), from 500 ns to 300 us, in NR3 format.
:SBUS<n>:SENT:CRC <format> (see page 1087)	:SBUS<n>:SENT:CRC? (see page 1087)	<format> ::= {LEGacy RECommended}
:SBUS<n>:SENT:DISPlay <base> (see page 1088)	:SBUS<n>:SENT:DISPlay ? (see page 1088)	<base> ::= {HEX DECimal SYMBolic}
:SBUS<n>:SENT:FORMat <decode> (see page 1090)	:SBUS<n>:SENT:FORMat? (see page 1090)	<decode> ::= {NIBBles FSIGnal FSSerial FESerial SSERial ESERial}
:SBUS<n>:SENT:IDLE <state> (see page 1092)	:SBUS<n>:SENT:IDLE? (see page 1092)	<state> ::= {LOW HIGH}
:SBUS<n>:SENT:LENGth <#_nibbles> (see page 1093)	:SBUS<n>:SENT:LENGth? (see page 1093)	<#_nibbles> ::= from 1-6, in NR1 format.
:SBUS<n>:SENT:PPULse { {0 OFF} {1 ON} SPC } (see page 1094)	:SBUS<n>:SENT:PPULse? (see page 1094)	{0 1 SPC}
:SBUS<n>:SENT:SIGnal<s>:DISPlay { {0 OFF} {1 ON} } (see page 1096)	:SBUS<n>:SENT:SIGnal<s>:DISPlay? (see page 1096)	<s> ::= 1-6, in NR1 format. {0 1}
:SBUS<n>:SENT:SIGnal<s>:LENGth <length> (see page 1097)	:SBUS<n>:SENT:SIGnal<s>:LENGth? (see page 1097)	<s> ::= 1-6, in NR1 format. <length> ::= from 1-24, in NR1 format.
:SBUS<n>:SENT:SIGnal<s>:MULTiplier <multiplier> (see page 1099)	:SBUS<n>:SENT:SIGnal<s>:MULTiplier? (see page 1099)	<s> ::= 1-6, in NR1 format. <multiplier> ::= from 1-24, in NR3 format.

Table 131 :SBUS<n>:SENT Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:SENT:SIGnAl<s>:OFFSet <offset> (see page 1101)	:SBUS<n>:SENT:SIGnAl<s>:OFFSet? (see page 1101)	<s> ::= 1-6, in NR1 format. <offset> ::= from 1-24, in NR3 format.
:SBUS<n>:SENT:SIGnAl<s>:ORDer <order> (see page 1103)	:SBUS<n>:SENT:SIGnAl<s>:ORDer? (see page 1103)	<s> ::= 1-6, in NR1 format. <order> ::= {MSNFirst LSNFirst}
:SBUS<n>:SENT:SIGnAl<s>:STARt <position> (see page 1105)	:SBUS<n>:SENT:SIGnAl<s>:STARt? (see page 1105)	<s> ::= 1-6, in NR1 format. <position> ::= from 0-23, in NR1 format.
:SBUS<n>:SENT:SOURce <source> (see page 1107)	:SBUS<n>:SENT:SOURce? (see page 1107)	<source> ::= {CHANnel<n> DIGital<d>} <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:SBUS<n>:SENT:TOLerance <percent> (see page 1109)	:SBUS<n>:SENT:TOLerance? (see page 1109)	<percent> ::= from 3-30, in NR1 format.
:SBUS<n>:SENT:TRIGger <mode> (see page 1110)	:SBUS<n>:SENT:TRIGger? (see page 1110)	<mode> ::= {SFCMessage SSCMessage FCData SCMid SCData FCCerror SCCerror CRCerror TOLerror PPErrror SSPerror}
:SBUS<n>:SENT:TRIGger:FAST:DATA <string> (see page 1112)	:SBUS<n>:SENT:TRIGger:FAST:DATA? (see page 1112)	<string> ::= "nnnn..." where n ::= {0 1 X} <string> ::= "0xn..." where n ::= {0,...,9 A,...,F X \$}
:SBUS<n>:SENT:TRIGger:SLOW:DATA <data> (see page 1113)	:SBUS<n>:SENT:TRIGger:SLOW:DATA? (see page 1113)	<data> ::= when ILENgth = SHORt, from -1 (don't care) to 65535, in NR1 format. <data> ::= when ILENgth = LONG, from -1 (don't care) to 4095, in NR1 format.
:SBUS<n>:SENT:TRIGger:SLOW:ID <id> (see page 1115)	:SBUS<n>:SENT:TRIGger:SLOW:ID? (see page 1115)	<id> ::= when ILENgth = SHORt, from -1 (don't care) to 15, in NR1 format. <id> ::= when ILENgth = LONG, from -1 (don't care) to 255, in NR1 format.

Table 131 :SBUS<n>:SENT Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:SENT:TRIGger :SLOW:ILENgtH <length> (see page 1117)	:SBUS<n>:SENT:TRIGger :SLOW:ILENgtH? (see page 1117)	<length> ::= {SHORT LONG}
:SBUS<n>:SENT:TRIGger :TOLerance <percent> (see page 1118)	:SBUS<n>:SENT:TRIGger :TOLerance? (see page 1118)	<percent> ::= from 1-18, in NR1 format.

:SBUS<n>:SENT:CLOCK

N (see [page 1650](#))

Command Syntax :SBUS<n>:SENT:CLOCK <period>

<period> ::= the nominal clock period (tick), from 500 ns to 300 us, in NR3 format.

The :SBUS<n>:SENT:CLOCK command specifies the nominal clock period (tick), from 500 ns to 300 μs.

Query Syntax :SBUS<n>:SENT:CLOCK?

The :SBUS<n>:SENT:CLOCK? query returns the clock period setting.

Return Format <period><NL>

<period> ::= the nominal clock period (tick), from 500 ns to 300 us, in NR3 format.

- See Also**
- [":SBUS<n>:SENT:CRC"](#) on page 1087
 - [":SBUS<n>:SENT:DISPlay"](#) on page 1088
 - [":SBUS<n>:SENT:FORMat"](#) on page 1090
 - [":SBUS<n>:SENT:IDLE"](#) on page 1092
 - [":SBUS<n>:SENT:LENGth"](#) on page 1093
 - [":SBUS<n>:SENT:PPULse"](#) on page 1094
 - [":SBUS<n>:SENT:SIGNal<s>:DISPlay"](#) on page 1096
 - [":SBUS<n>:SENT:SIGNal<s>:LENGth"](#) on page 1097
 - [":SBUS<n>:SENT:SIGNal<s>:MULTIplier"](#) on page 1099
 - [":SBUS<n>:SENT:SIGNal<s>:OFFSet"](#) on page 1101
 - [":SBUS<n>:SENT:SIGNal<s>:ORDer"](#) on page 1103
 - [":SBUS<n>:SENT:SIGNal<s>:START"](#) on page 1105
 - [":SBUS<n>:SENT:SOURce"](#) on page 1107
 - [":SBUS<n>:SENT:TOLerance"](#) on page 1109
 - [":SBUS<n>:SENT:TRIGger"](#) on page 1110
 - [":SBUS<n>:SENT:TRIGger:FAST:DATA"](#) on page 1112
 - [":SBUS<n>:SENT:TRIGger:SLOW:DATA"](#) on page 1113
 - [":SBUS<n>:SENT:TRIGger:SLOW:ID"](#) on page 1115
 - [":SBUS<n>:SENT:TRIGger:SLOW:ILENGth"](#) on page 1117
 - [":SBUS<n>:SENT:TRIGger:TOLerance"](#) on page 1118

:SBUS<n>:SENT:CRC

N (see [page 1650](#))

Command Syntax :SBUS<n>:SENT:CRC <format>

<format> ::= {LEGacy | RECommended}

The :SBUS<n>:SENT:CRC command specifies the format of the CRC. Either Legacy (2008) or Recommended (2010).

Enhanced Serial Message CRCs are always calculated using the 2010 format, but for the Fast Channel Messages, and for Short Serial Message CRCs, this setting is used.

Query Syntax :SBUS<n>:SENT:CRC?

The :SBUS<n>:SENT:CRC? query returns the CRC format setting.

Return Format <format><NL>

<format> ::= {LEG | REC}

- See Also**
- [":SBUS<n>:SENT:CLOCK"](#) on page 1086
 - [":SBUS<n>:SENT:DISPlay"](#) on page 1088
 - [":SBUS<n>:SENT:FORMat"](#) on page 1090
 - [":SBUS<n>:SENT:IDLE"](#) on page 1092
 - [":SBUS<n>:SENT:LENGth"](#) on page 1093
 - [":SBUS<n>:SENT:PPULse"](#) on page 1094
 - [":SBUS<n>:SENT:SIGNal<s>:DISPlay"](#) on page 1096
 - [":SBUS<n>:SENT:SIGNal<s>:LENGth"](#) on page 1097
 - [":SBUS<n>:SENT:SIGNal<s>:MULTIplier"](#) on page 1099
 - [":SBUS<n>:SENT:SIGNal<s>:OFFSet"](#) on page 1101
 - [":SBUS<n>:SENT:SIGNal<s>:ORDer"](#) on page 1103
 - [":SBUS<n>:SENT:SIGNal<s>:START"](#) on page 1105
 - [":SBUS<n>:SENT:SOURce"](#) on page 1107
 - [":SBUS<n>:SENT:TOLerance"](#) on page 1109
 - [":SBUS<n>:SENT:TRIGger"](#) on page 1110
 - [":SBUS<n>:SENT:TRIGger:FAST:DATA"](#) on page 1112
 - [":SBUS<n>:SENT:TRIGger:SLOW:DATA"](#) on page 1113
 - [":SBUS<n>:SENT:TRIGger:SLOW:ID"](#) on page 1115
 - [":SBUS<n>:SENT:TRIGger:SLOW:ILENgtH"](#) on page 1117
 - [":SBUS<n>:SENT:TRIGger:TOLerance"](#) on page 1118

:SBUS<n>:SENT:DISPlay

N (see [page 1650](#))

Command Syntax :SBUS<n>:SENT:DISPlay <base>

<base> ::= {HEX | DECimal | SYMBolic}

The :SBUS<n>:SENT:DISPlay command specifies the number base used by the decoder. The chosen base is used for the data nibbles in Raw decode format, the defined Signals in the other formats, and for the data field of the Serial Messages.

This selection is used for both the Lister and the decode line displays.

When SYMBolic is selected, Fast Channel Signals display a calculated physical value based on the specified multiplier and offset:

- $PhysicalValue = (Multiplier * SignalValueAsUnsignedInteger) + Offset$

When SYMBolic is selected, the CRC and Slow Channel information is displayed in hex.

Query Syntax :SBUS<n>:SENT:DISPlay?

The :SBUS<n>:SENT:DISPlay? query returns the SENT decode number base setting.

Return Format <base><NL>

<base> ::= {HEX | DEC | SYMB}

- See Also**
- [":SBUS<n>:SENT:CLOCK"](#) on page 1086
 - [":SBUS<n>:SENT:CRC"](#) on page 1087
 - [":SBUS<n>:SENT:FORMat"](#) on page 1090
 - [":SBUS<n>:SENT:IDLE"](#) on page 1092
 - [":SBUS<n>:SENT:LENGth"](#) on page 1093
 - [":SBUS<n>:SENT:PPULse"](#) on page 1094
 - [":SBUS<n>:SENT:SIGNal<s>:DISPlay"](#) on page 1096
 - [":SBUS<n>:SENT:SIGNal<s>:LENGth"](#) on page 1097
 - [":SBUS<n>:SENT:SIGNal<s>:MULTiplier"](#) on page 1099
 - [":SBUS<n>:SENT:SIGNal<s>:OFFSet"](#) on page 1101
 - [":SBUS<n>:SENT:SIGNal<s>:ORDer"](#) on page 1103
 - [":SBUS<n>:SENT:SIGNal<s>:START"](#) on page 1105
 - [":SBUS<n>:SENT:SOURce"](#) on page 1107
 - [":SBUS<n>:SENT:TOLerance"](#) on page 1109
 - [":SBUS<n>:SENT:TRIGger"](#) on page 1110
 - [":SBUS<n>:SENT:TRIGger:FAST:DATA"](#) on page 1112

- `":SBUS<n>:SENT:TRIGger:SLOW:DATA"` on page 1113
- `":SBUS<n>:SENT:TRIGger:SLOW:ID"` on page 1115
- `":SBUS<n>:SENT:TRIGger:SLOW:ILENgtH"` on page 1117
- `":SBUS<n>:SENT:TRIGger:TOLerance"` on page 1118

:SBUS<n>:SENT:FORMat

N (see [page 1650](#))

Command Syntax :SBUS<n>:SENT:FORMat <decode>

<decode> ::= {NIBBles | FSIGnal | FSSerial | FESerial | SSERial | ESERial}

The :SBUS<n>:SENT:FORMat command specifies the message decode/triggering format:

- NIBBles – displays the raw transmitted nibble values.
- FSIGnal – displays Fast Channel Message Signals.
- FSSerial – displays both Fast and Slow Messages (Short format) simultaneously.
- FESerial – displays both Fast and Slow Messages (Enhanced format) simultaneously.
- SSERial – displays Slow Channel Messages in Short format.
- ESERial – displays Slow Channel Messages in Enhanced format.

This selection affects both decoding and triggering. The decode is affected both in how the system interprets the data, and what will be displayed. The trigger is affected in that the trigger hardware needs to be configured to trigger on serial messages correctly.

You can specify the nibble display order for Fast Channel Message Signals (see :SBUS<n>:SENT:SIGNa<s>:ORDer). Raw transmitted nibble values are displayed in the order received.

Note that for the Slow Channel, the proper format, Short or Enhanced, must be chosen for proper decoding and triggering to occur.

Slow Channel Serial Messages are always displayed as defined by the SENT specification.

Query Syntax :SBUS<n>:SENT:FORMat?

The :SBUS<n>:SENT:FORMat? query returns the message decode/triggering format setting.

Return Format <decode><NL>

<decode> ::= {NIBB | FSIG | FSS | FES | SSER | ESER}

- See Also**
- **"[:SBUS<n>:SENT:CLOCK](#)"** on page 1086
 - **"[:SBUS<n>:SENT:CRC](#)"** on page 1087
 - **"[:SBUS<n>:SENT:DISPlay](#)"** on page 1088
 - **"[:SBUS<n>:SENT:IDLE](#)"** on page 1092

- `":SBUS<n>:SENT:LENGth"` on page 1093
- `":SBUS<n>:SENT:PPULse"` on page 1094
- `":SBUS<n>:SENT:SIGNal<s>:DISPlay"` on page 1096
- `":SBUS<n>:SENT:SIGNal<s>:LENGth"` on page 1097
- `":SBUS<n>:SENT:SIGNal<s>:MULTIplier"` on page 1099
- `":SBUS<n>:SENT:SIGNal<s>:OFFSet"` on page 1101
- `":SBUS<n>:SENT:SIGNal<s>:ORDer"` on page 1103
- `":SBUS<n>:SENT:SIGNal<s>:STARt"` on page 1105
- `":SBUS<n>:SENT:SOURce"` on page 1107
- `":SBUS<n>:SENT:TOLerance"` on page 1109
- `":SBUS<n>:SENT:TRIGger"` on page 1110
- `":SBUS<n>:SENT:TRIGger:FAST:DATA"` on page 1112
- `":SBUS<n>:SENT:TRIGger:SLOW:DATA"` on page 1113
- `":SBUS<n>:SENT:TRIGger:SLOW:ID"` on page 1115
- `":SBUS<n>:SENT:TRIGger:SLOW:ILENgtH"` on page 1117
- `":SBUS<n>:SENT:TRIGger:TOLerance"` on page 1118

:SBUS<n>:SENT:IDLE

N (see [page 1650](#))

Command Syntax :SBUS<n>:SENT:IDLE <state>

<state> ::= {LOW | HIGH}

The :SBUS<n>:SENT:IDLE command specifies the idle state of the SENT bus.

Query Syntax :SBUS<n>:SENT:IDLE?

The :SBUS<n>:SENT:IDLE? query returns the idle state setting.

Return Format <state><NL>

<state> ::= {LOW | HIGH}

- See Also**
- [":SBUS<n>:SENT:CLOCK"](#) on page 1086
 - [":SBUS<n>:SENT:CRC"](#) on page 1087
 - [":SBUS<n>:SENT:DISPlay"](#) on page 1088
 - [":SBUS<n>:SENT:FORMat"](#) on page 1090
 - [":SBUS<n>:SENT:LENGth"](#) on page 1093
 - [":SBUS<n>:SENT:PPULse"](#) on page 1094
 - [":SBUS<n>:SENT:SIGNal<s>:DISPlay"](#) on page 1096
 - [":SBUS<n>:SENT:SIGNal<s>:LENGth"](#) on page 1097
 - [":SBUS<n>:SENT:SIGNal<s>:MULTiplier"](#) on page 1099
 - [":SBUS<n>:SENT:SIGNal<s>:OFFSet"](#) on page 1101
 - [":SBUS<n>:SENT:SIGNal<s>:ORDer"](#) on page 1103
 - [":SBUS<n>:SENT:SIGNal<s>:START"](#) on page 1105
 - [":SBUS<n>:SENT:SOURce"](#) on page 1107
 - [":SBUS<n>:SENT:TOLerance"](#) on page 1109
 - [":SBUS<n>:SENT:TRIGger"](#) on page 1110
 - [":SBUS<n>:SENT:TRIGger:FAST:DATA"](#) on page 1112
 - [":SBUS<n>:SENT:TRIGger:SLOW:DATA"](#) on page 1113
 - [":SBUS<n>:SENT:TRIGger:SLOW:ID"](#) on page 1115
 - [":SBUS<n>:SENT:TRIGger:SLOW:ILENgth"](#) on page 1117
 - [":SBUS<n>:SENT:TRIGger:TOLerance"](#) on page 1118

:SBUS<n>:SENT:LENGth

N (see [page 1650](#))

Command Syntax :SBUS<n>:SENT:LENGth <#_nibbles>

<#_nibbles> ::= from 1-6, in NR1 format.

The :SBUS<n>:SENT:LENGth command specifies the number of nibbles in a SENT message, from 1 to 6.

Query Syntax :SBUS<n>:SENT:LENGth?

The :SBUS<n>:SENT:LENGth? query returns the number of nibbles setting.

Return Format <#_nibbles><NL>

<#_nibbles> ::= from 1-6, in NR1 format.

- See Also**
- [":SBUS<n>:SENT:CLOCK"](#) on page 1086
 - [":SBUS<n>:SENT:CRC"](#) on page 1087
 - [":SBUS<n>:SENT:DISPlay"](#) on page 1088
 - [":SBUS<n>:SENT:FORMat"](#) on page 1090
 - [":SBUS<n>:SENT:IDLE"](#) on page 1092
 - [":SBUS<n>:SENT:PPULse"](#) on page 1094
 - [":SBUS<n>:SENT:SIGNal<s>:DISPlay"](#) on page 1096
 - [":SBUS<n>:SENT:SIGNal<s>:LENGth"](#) on page 1097
 - [":SBUS<n>:SENT:SIGNal<s>:MULTiplier"](#) on page 1099
 - [":SBUS<n>:SENT:SIGNal<s>:OFFSet"](#) on page 1101
 - [":SBUS<n>:SENT:SIGNal<s>:ORDer"](#) on page 1103
 - [":SBUS<n>:SENT:SIGNal<s>:START"](#) on page 1105
 - [":SBUS<n>:SENT:SOURce"](#) on page 1107
 - [":SBUS<n>:SENT:TOLerance"](#) on page 1109
 - [":SBUS<n>:SENT:TRIGger"](#) on page 1110
 - [":SBUS<n>:SENT:TRIGger:FAST:DATA"](#) on page 1112
 - [":SBUS<n>:SENT:TRIGger:SLOW:DATA"](#) on page 1113
 - [":SBUS<n>:SENT:TRIGger:SLOW:ID"](#) on page 1115
 - [":SBUS<n>:SENT:TRIGger:SLOW:ILENgtH"](#) on page 1117
 - [":SBUS<n>:SENT:TRIGger:TOLerance"](#) on page 1118

:SBUS<n>:SENT:PPULse

N (see [page 1650](#))

Command Syntax :SBUS<n>:SENT:PPULse {{0 | OFF} | {1 | ON} | SPC}

The :SBUS<n>:SENT:PPULse command specifies whether there is a pause pulse between Fast Channel Messages:

- OFF – There is no pause pulse between Fast Channel Messages.

Note that a SENT serial bus with no pause pulse is never idle. This means, during normal operation, the fast channel decode line will show a continuous stream of packets, with a new packet opening as soon as the previous one has closed.

- ON – Pause pulses are added between Fast Channel Messages so that frames come at a regular interval.

If there is a pause pulse (and **Pause Pulse** is on), idle time is shown between messages.

- SPC (Short PWM Code) – In SENT SPC, there are no pause pulses. Instead, the message event is triggered by the master when it wants to receive data. SENT SPC ends the transmission after the CRC so it almost appears as if there is a pause pulse from the end until the next master trigger.

Query Syntax :SBUS<n>:SENT:PPULse?

The :SBUS<n>:SENT:PPULse? query returns the pause mode setting.

Return Format <setting><NL>

<setting> ::= {0 | 1 | SPC}

- See Also**
- [":SBUS<n>:SENT:CLOCK"](#) on page 1086
 - [":SBUS<n>:SENT:CRC"](#) on page 1087
 - [":SBUS<n>:SENT:DISPlay"](#) on page 1088
 - [":SBUS<n>:SENT:FORMat"](#) on page 1090
 - [":SBUS<n>:SENT:IDLE"](#) on page 1092
 - [":SBUS<n>:SENT:LENGth"](#) on page 1093
 - [":SBUS<n>:SENT:SIGNal<s>:DISPlay"](#) on page 1096
 - [":SBUS<n>:SENT:SIGNal<s>:LENGth"](#) on page 1097
 - [":SBUS<n>:SENT:SIGNal<s>:MULTiplier"](#) on page 1099
 - [":SBUS<n>:SENT:SIGNal<s>:OFFSet"](#) on page 1101
 - [":SBUS<n>:SENT:SIGNal<s>:ORDer"](#) on page 1103
 - [":SBUS<n>:SENT:SIGNal<s>:START"](#) on page 1105
 - [":SBUS<n>:SENT:SOURce"](#) on page 1107

- `":SBUS<n>:SENT:TOLerance"` on page 1109
- `":SBUS<n>:SENT:TRIGger"` on page 1110
- `":SBUS<n>:SENT:TRIGger:FAST:DATA"` on page 1112
- `":SBUS<n>:SENT:TRIGger:SLOW:DATA"` on page 1113
- `":SBUS<n>:SENT:TRIGger:SLOW:ID"` on page 1115
- `":SBUS<n>:SENT:TRIGger:SLOW:ILENgtH"` on page 1117
- `":SBUS<n>:SENT:TRIGger:TOLerance"` on page 1118

:SBUS<n>:SENT:SIGNAl<s>:DISPlay

N (see [page 1650](#))

Command Syntax :SBUS<n>:SENT:SIGNAl<s>:DISPlay {{0 | OFF} | {1 | ON}}

<s> ::= 1-6, in NR1 format.

The :SBUS<n>:SENT:SIGNAl<s>:DISPlay command specifies whether the given signal is on or off.

Query Syntax :SBUS<n>:SENT:SIGNAl<s>:DISPlay?

The :SBUS<n>:SENT:SIGNAl<s>:DISPlay? query returns the signal on/off setting.

Return Format <setting><NL>

<setting> ::= {0 | 1}

- See Also**
- [":SBUS<n>:SENT:CLOCK"](#) on page 1086
 - [":SBUS<n>:SENT:CRC"](#) on page 1087
 - [":SBUS<n>:SENT:DISPlay"](#) on page 1088
 - [":SBUS<n>:SENT:FORMat"](#) on page 1090
 - [":SBUS<n>:SENT:IDLE"](#) on page 1092
 - [":SBUS<n>:SENT:LENGth"](#) on page 1093
 - [":SBUS<n>:SENT:PPULse"](#) on page 1094
 - [":SBUS<n>:SENT:SIGNAl<s>:LENGth"](#) on page 1097
 - [":SBUS<n>:SENT:SIGNAl<s>:MULTIplier"](#) on page 1099
 - [":SBUS<n>:SENT:SIGNAl<s>:OFFSet"](#) on page 1101
 - [":SBUS<n>:SENT:SIGNAl<s>:ORDer"](#) on page 1103
 - [":SBUS<n>:SENT:SIGNAl<s>:START"](#) on page 1105
 - [":SBUS<n>:SENT:SOURce"](#) on page 1107
 - [":SBUS<n>:SENT:TOLerance"](#) on page 1109
 - [":SBUS<n>:SENT:TRIGger"](#) on page 1110
 - [":SBUS<n>:SENT:TRIGger:FAST:DATA"](#) on page 1112
 - [":SBUS<n>:SENT:TRIGger:SLOW:DATA"](#) on page 1113
 - [":SBUS<n>:SENT:TRIGger:SLOW:ID"](#) on page 1115
 - [":SBUS<n>:SENT:TRIGger:SLOW:ILENgth"](#) on page 1117
 - [":SBUS<n>:SENT:TRIGger:TOLerance"](#) on page 1118

:SBUS<n>:SENT:SIGNAl<s>:LENGth

N (see [page 1650](#))

Command Syntax :SBUS<n>:SENT:SIGNAl<s>:LENGth <length>

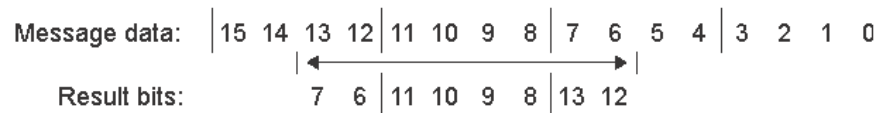
<s> ::= 1-6, in NR1 format.

<length> ::= from 1-24, in NR1 format.

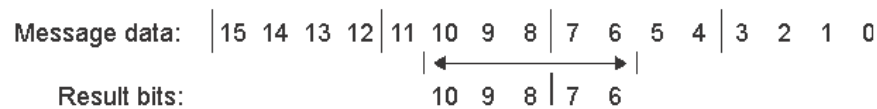
The :SBUS<n>:SENT:SIGNAl<s>:LENGth command specifies the bit length of the signal being defined.

Fast Signal definition examples:

Example 1: Start Bit # = 13, # of bits = 8, Nibble Order = LSN First



Example 2: Start Bit # = 10, # of bits = 5, Nibble Order = MSN First



Query Syntax :SBUS<n>:SENT:SIGNAl<s>:LENGth?

The :SBUS<n>:SENT:SIGNAl<s>:LENGth? query returns the signal bit length setting.

Return Format <length><NL>

<length> ::= from 1-24, in NR1 format.

- See Also**
- **":SBUS<n>:SENT:CLOCK"** on page 1086
 - **":SBUS<n>:SENT:CRC"** on page 1087
 - **":SBUS<n>:SENT:DISPlay"** on page 1088
 - **":SBUS<n>:SENT:FORMat"** on page 1090
 - **":SBUS<n>:SENT:IDLE"** on page 1092
 - **":SBUS<n>:SENT:LENGth"** on page 1093
 - **":SBUS<n>:SENT:PPULse"** on page 1094
 - **":SBUS<n>:SENT:SIGNAl<s>:DISPlay"** on page 1096
 - **":SBUS<n>:SENT:SIGNAl<s>:MULTIplier"** on page 1099
 - **":SBUS<n>:SENT:SIGNAl<s>:OFFSet"** on page 1101

- **":SBUS<n>:SENT:SIGNAl<s>:ORDer"** on page 1103
- **":SBUS<n>:SENT:SIGNAl<s>:START"** on page 1105
- **":SBUS<n>:SENT:SOURce"** on page 1107
- **":SBUS<n>:SENT:TOLerance"** on page 1109
- **":SBUS<n>:SENT:TRIGger"** on page 1110
- **":SBUS<n>:SENT:TRIGger:FAST:DATA"** on page 1112
- **":SBUS<n>:SENT:TRIGger:SLOW:DATA"** on page 1113
- **":SBUS<n>:SENT:TRIGger:SLOW:ID"** on page 1115
- **":SBUS<n>:SENT:TRIGger:SLOW:ILENgtH"** on page 1117
- **":SBUS<n>:SENT:TRIGger:TOLerance"** on page 1118

:SBUS<n>:SENT:SIGNAl<s>:MULTIplier

N (see [page 1650](#))

Command Syntax :SBUS<n>:SENT:SIGNAl<s>:MULTIplier <multiplier>

<s> ::= 1-6, in NR1 format.

<multiplier> ::= from 1-24, in NR3 format.

When the display mode setting is SYMBolic (see :SBUS<n>:SENT:DISPlay), the :SBUS<n>:SENT:SIGNAl<s>:MULTIplier command specifies the multiplier to be used in calculating a physical value displayed for a Fast Channel Signal.

- $\text{PhysicalValue} = (\text{Multiplier} * \text{SignalValueAsUnsignedInteger}) + \text{Offset}$

Query Syntax :SBUS<n>:SENT:SIGNAl<s>:MULTIplier?

The :SBUS<n>:SENT:SIGNAl<s>:MULTIplier? query returns the multiplier value for the Fast Channel Signal.

Return Format <multiplier><NL>

<multiplier> ::= from 1-24, in NR3 format.

- See Also**
- [":SBUS<n>:SENT:CLOCK"](#) on page 1086
 - [":SBUS<n>:SENT:CRC"](#) on page 1087
 - [":SBUS<n>:SENT:DISPlay"](#) on page 1088
 - [":SBUS<n>:SENT:FORMat"](#) on page 1090
 - [":SBUS<n>:SENT:IDLE"](#) on page 1092
 - [":SBUS<n>:SENT:LENGth"](#) on page 1093
 - [":SBUS<n>:SENT:PPULse"](#) on page 1094
 - [":SBUS<n>:SENT:SIGNAl<s>:DISPlay"](#) on page 1096
 - [":SBUS<n>:SENT:SIGNAl<s>:LENGth"](#) on page 1097
 - [":SBUS<n>:SENT:SIGNAl<s>:OFFSet"](#) on page 1101
 - [":SBUS<n>:SENT:SIGNAl<s>:ORDer"](#) on page 1103
 - [":SBUS<n>:SENT:SIGNAl<s>:START"](#) on page 1105
 - [":SBUS<n>:SENT:SOURce"](#) on page 1107
 - [":SBUS<n>:SENT:TOLerance"](#) on page 1109
 - [":SBUS<n>:SENT:TRIGger"](#) on page 1110
 - [":SBUS<n>:SENT:TRIGger:FAST:DATA"](#) on page 1112
 - [":SBUS<n>:SENT:TRIGger:SLOW:DATA"](#) on page 1113
 - [":SBUS<n>:SENT:TRIGger:SLOW:ID"](#) on page 1115
 - [":SBUS<n>:SENT:TRIGger:SLOW:ILENgth"](#) on page 1117

- **":SBUS<n>:SENT:TRIGger:TOLerance"** on page 1118

:SBUS<n>:SENT:SIGNAl<s>:OFFSet

N (see [page 1650](#))

Command Syntax :SBUS<n>:SENT:SIGNAl<s>:OFFSet <offset>

<s> ::= 1-6, in NR1 format.

<offset> ::= from 1-24, in NR3 format.

When the display mode setting is SYMBolic (see :SBUS<n>:SENT:DISPlay), the :SBUS<n>:SENT:SIGNAl<s>:OFFSet command is used in calculating a physical value displayed for the Fast Channel Signal:

- $\text{PhysicalValue} = (\text{Multiplier} * \text{SignalValueAsUnsignedInteger}) + \text{Offset}$

Query Syntax :SBUS<n>:SENT:SIGNAl<s>:OFFSet?

The :SBUS<n>:SENT:SIGNAl<s>:OFFSet? query returns the offset value for the Fast Channel Signal.

Return Format <offset><NL>

<offset> ::= from 1-24, in NR3 format.

- See Also**
- [":SBUS<n>:SENT:CLOCK"](#) on page 1086
 - [":SBUS<n>:SENT:CRC"](#) on page 1087
 - [":SBUS<n>:SENT:DISPlay"](#) on page 1088
 - [":SBUS<n>:SENT:FORMat"](#) on page 1090
 - [":SBUS<n>:SENT:IDLE"](#) on page 1092
 - [":SBUS<n>:SENT:LENGth"](#) on page 1093
 - [":SBUS<n>:SENT:PPULse"](#) on page 1094
 - [":SBUS<n>:SENT:SIGNAl<s>:DISPlay"](#) on page 1096
 - [":SBUS<n>:SENT:SIGNAl<s>:LENGth"](#) on page 1097
 - [":SBUS<n>:SENT:SIGNAl<s>:MULTIplier"](#) on page 1099
 - [":SBUS<n>:SENT:SIGNAl<s>:ORDer"](#) on page 1103
 - [":SBUS<n>:SENT:SIGNAl<s>:START"](#) on page 1105
 - [":SBUS<n>:SENT:SOURce"](#) on page 1107
 - [":SBUS<n>:SENT:TOLerance"](#) on page 1109
 - [":SBUS<n>:SENT:TRIGger"](#) on page 1110
 - [":SBUS<n>:SENT:TRIGger:FAST:DATA"](#) on page 1112
 - [":SBUS<n>:SENT:TRIGger:SLOW:DATA"](#) on page 1113
 - [":SBUS<n>:SENT:TRIGger:SLOW:ID"](#) on page 1115
 - [":SBUS<n>:SENT:TRIGger:SLOW:ILENgth"](#) on page 1117

- **":SBUS<n>:SENT:TRIGger:TOLerance"** on page 1118

:SBUS<n>:SENT:SIGNal<s>:ORDER

N (see [page 1650](#))

Command Syntax :SBUS<n>:SENT:SIGNal<s>:ORDER <order>

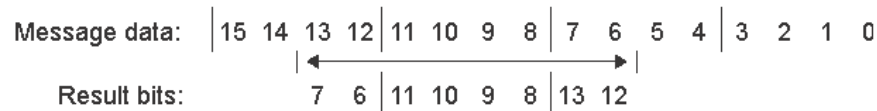
<s> ::= 1-6, in NR1 format.

<order> ::= {MSNFirst | LSNFirst}

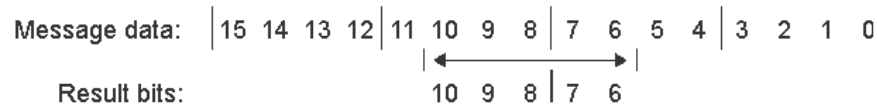
The :SBUS<n>:SENT:SIGNal<s>:ORDER command specifies the nibble order of the signal being defined, either Most Significant Nibble first, or Least Significant Nibble first.

Fast Signal definition examples:

Example 1: Start Bit # = 13, # of bits = 8, Nibble Order = LSN First



Example 2: Start Bit # = 10, # of bits = 5, Nibble Order = MSN First



Query Syntax :SBUS<n>:SENT:SIGNal<s>:ORDER?

The :SBUS<n>:SENT:SIGNal<s>:ORDER? query returns the nibble order setting.

Return Format <order><NL>

<order> ::= {MSNF | LSNF}

- See Also**
- [":SBUS<n>:SENT:CLOCK"](#) on page 1086
 - [":SBUS<n>:SENT:CRC"](#) on page 1087
 - [":SBUS<n>:SENT:DISPlay"](#) on page 1088
 - [":SBUS<n>:SENT:FORMat"](#) on page 1090
 - [":SBUS<n>:SENT:IDLE"](#) on page 1092
 - [":SBUS<n>:SENT:LENGth"](#) on page 1093
 - [":SBUS<n>:SENT:PPULse"](#) on page 1094
 - [":SBUS<n>:SENT:SIGNal<s>:DISPlay"](#) on page 1096
 - [":SBUS<n>:SENT:SIGNal<s>:LENGth"](#) on page 1097
 - [":SBUS<n>:SENT:SIGNal<s>:MULTiplier"](#) on page 1099

- **":SBUS<n>:SENT:SIGNAl<s>:OFFSet"** on page 1101
- **":SBUS<n>:SENT:SIGNAl<s>:START"** on page 1105
- **":SBUS<n>:SENT:SOURce"** on page 1107
- **":SBUS<n>:SENT:TOLerance"** on page 1109
- **":SBUS<n>:SENT:TRIGger"** on page 1110
- **":SBUS<n>:SENT:TRIGger:FAST:DATA"** on page 1112
- **":SBUS<n>:SENT:TRIGger:SLOW:DATA"** on page 1113
- **":SBUS<n>:SENT:TRIGger:SLOW:ID"** on page 1115
- **":SBUS<n>:SENT:TRIGger:SLOW:ILENgtH"** on page 1117
- **":SBUS<n>:SENT:TRIGger:TOLerance"** on page 1118

:SBUS<n>:SENT:SIGNal<s>:START

N (see [page 1650](#))

Command Syntax :SBUS<n>:SENT:SIGNal<s>:START <position>

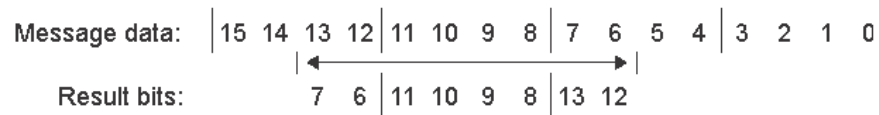
<s> ::= 1-6, in NR1 format.

<position> ::= from 0-23, in NR1 format.

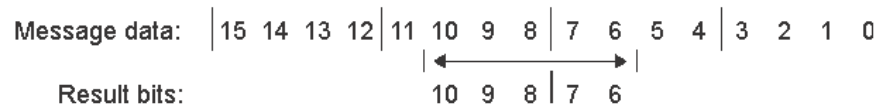
The :SBUS<n>:SENT:SIGNal<s>:START command specifies the starting bit of the Fast Signal being defined.

Fast Signal definition examples:

Example 1: Start Bit # = 13, # of bits = 8, Nibble Order = LSN First



Example 2: Start Bit # = 10, # of bits = 5, Nibble Order = MSN First



Query Syntax :SBUS<n>:SENT:SIGNal<s>:START?

The :SBUS<n>:SENT:SIGNal<s>:START? query returns the Fast Signal starting bit setting.

Return Format <position><NL>

<position> ::= from 0-23, in NR1 format.

- See Also**
- **":SBUS<n>:SENT:CLOCK"** on page 1086
 - **":SBUS<n>:SENT:CRC"** on page 1087
 - **":SBUS<n>:SENT:DISPlay"** on page 1088
 - **":SBUS<n>:SENT:FORMat"** on page 1090
 - **":SBUS<n>:SENT:IDLE"** on page 1092
 - **":SBUS<n>:SENT:LENGth"** on page 1093
 - **":SBUS<n>:SENT:PPULse"** on page 1094
 - **":SBUS<n>:SENT:SIGNal<s>:DISPlay"** on page 1096
 - **":SBUS<n>:SENT:SIGNal<s>:LENGth"** on page 1097
 - **":SBUS<n>:SENT:SIGNal<s>:MULTiplier"** on page 1099

- **":SBUS<n>:SENT:SIGNAl<s>:OFFSet"** on page 1101
- **":SBUS<n>:SENT:SIGNAl<s>:ORDer"** on page 1103
- **":SBUS<n>:SENT:SOURce"** on page 1107
- **":SBUS<n>:SENT:TOLerance"** on page 1109
- **":SBUS<n>:SENT:TRIGger"** on page 1110
- **":SBUS<n>:SENT:TRIGger:FAST:DATA"** on page 1112
- **":SBUS<n>:SENT:TRIGger:SLOW:DATA"** on page 1113
- **":SBUS<n>:SENT:TRIGger:SLOW:ID"** on page 1115
- **":SBUS<n>:SENT:TRIGger:SLOW:ILENgtH"** on page 1117
- **":SBUS<n>:SENT:TRIGger:TOLerance"** on page 1118

:SBUS<n>:SENT:SOURce

N (see [page 1650](#))

Command Syntax :SBUS<n>:SENT:SOURce <source>

<source> ::= {CHANnel<n> | DIGital<d>}

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :SBUS<n>:SENT:SOURce command specifies the input channel for SENT decode and triggering.

Query Syntax :SBUS<n>:SENT:SOURce?

The :SBUS<n>:SENT:SOURce? query returns the specified SENT input source.

Return Format <source><NL>

<source> ::= {CHANnel<n> | DIGital<d>}

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

- See Also**
- [":SBUS<n>:SENT:CLOCK"](#) on page 1086
 - [":SBUS<n>:SENT:CRC"](#) on page 1087
 - [":SBUS<n>:SENT:DISPlay"](#) on page 1088
 - [":SBUS<n>:SENT:FORMat"](#) on page 1090
 - [":SBUS<n>:SENT:IDLE"](#) on page 1092
 - [":SBUS<n>:SENT:LENGth"](#) on page 1093
 - [":SBUS<n>:SENT:PPULse"](#) on page 1094
 - [":SBUS<n>:SENT:SIGNAL<s>:DISPlay"](#) on page 1096
 - [":SBUS<n>:SENT:SIGNAL<s>:LENGth"](#) on page 1097
 - [":SBUS<n>:SENT:SIGNAL<s>:MULTIplier"](#) on page 1099
 - [":SBUS<n>:SENT:SIGNAL<s>:OFFSet"](#) on page 1101
 - [":SBUS<n>:SENT:SIGNAL<s>:ORDer"](#) on page 1103
 - [":SBUS<n>:SENT:SIGNAL<s>:START"](#) on page 1105
 - [":SBUS<n>:SENT:TOLerance"](#) on page 1109
 - [":SBUS<n>:SENT:TRIGger"](#) on page 1110
 - [":SBUS<n>:SENT:TRIGger:FAST:DATA"](#) on page 1112
 - [":SBUS<n>:SENT:TRIGger:SLOW:DATA"](#) on page 1113
 - [":SBUS<n>:SENT:TRIGger:SLOW:ID"](#) on page 1115
 - [":SBUS<n>:SENT:TRIGger:SLOW:ILENgth"](#) on page 1117

- **":SBUS<n>:SENT:TRIGger:TOLerance"** on page 1118

:SBUS<n>:SENT:TOLerance

N (see [page 1650](#))

Command Syntax :SBUS<n>:SENT:TOLerance <percent>

<percent> ::= from 3-30, in NR1 format.

The :SBUS<n>:SENT:TOLerance command specifies the tolerance for determining whether the sync pulse is valid. Valid values range from 3% to 30%.

Query Syntax :SBUS<n>:SENT:TOLerance?

The :SBUS<n>:SENT:TOLerance? query returns the tolerance setting.

Return Format <percent><NL>

<percent> ::= from 3-30, in NR1 format.

- See Also**
- [":SBUS<n>:SENT:CLOCK"](#) on page 1086
 - [":SBUS<n>:SENT:CRC"](#) on page 1087
 - [":SBUS<n>:SENT:DISPlay"](#) on page 1088
 - [":SBUS<n>:SENT:FORMat"](#) on page 1090
 - [":SBUS<n>:SENT:IDLE"](#) on page 1092
 - [":SBUS<n>:SENT:LENGth"](#) on page 1093
 - [":SBUS<n>:SENT:PPULse"](#) on page 1094
 - [":SBUS<n>:SENT:SIGNal<s>:DISPlay"](#) on page 1096
 - [":SBUS<n>:SENT:SIGNal<s>:LENGth"](#) on page 1097
 - [":SBUS<n>:SENT:SIGNal<s>:MULTiplier"](#) on page 1099
 - [":SBUS<n>:SENT:SIGNal<s>:OFFSet"](#) on page 1101
 - [":SBUS<n>:SENT:SIGNal<s>:ORDer"](#) on page 1103
 - [":SBUS<n>:SENT:SIGNal<s>:START"](#) on page 1105
 - [":SBUS<n>:SENT:SOURce"](#) on page 1107
 - [":SBUS<n>:SENT:TRIGger"](#) on page 1110
 - [":SBUS<n>:SENT:TRIGger:FAST:DATA"](#) on page 1112
 - [":SBUS<n>:SENT:TRIGger:SLOW:DATA"](#) on page 1113
 - [":SBUS<n>:SENT:TRIGger:SLOW:ID"](#) on page 1115
 - [":SBUS<n>:SENT:TRIGger:SLOW:ILENgtH"](#) on page 1117
 - [":SBUS<n>:SENT:TRIGger:TOLerance"](#) on page 1118

:SBUS<n>:SENT:TRIGger

N (see [page 1650](#))

Command Syntax :SBUS<n>:SENT:TRIGger <mode>

```
<mode> ::= {SFCMessage | SSCMessage | FCData | SCMid | SCData
            | TOLerror | FCCerror | SCCerror | CRCerror | PPERror | SSPerror}
```

The :SBUS<n>:SENT:TRIGger command specifies the SENT trigger mode:

- SFCMessage – triggers on the start of any Fast Channel Message (after 56 Synchronization/Calibration ticks).
- SSCMessage – trigger on the start of any Slow Channel Message.
- FCData – triggers on a Fast Channel Message when the Status & Communication nibble and the data nibbles match the values entered using additional softkeys.
- SCMid – triggers when a Slow Channel Message ID matches the value entered using additional softkeys.
- SCData – triggers when a Slow Channel Message ID and Data field both match the values entered using additional softkeys.
- TOLerror – triggers when the sync pulse width varies from the nominal value by greater than the entered percentage.
- FCCerror – triggers on any Fast Channel Message CRC error.
- SCCerror – triggers on any Slow Channel Message CRC error.
- CRCerror – triggers on any CRC error, Fast or Slow.
- PPERror – triggers if a nibble is either too wide or too narrow (for example, data nibble < 12 (11.5) or > 27 (27.5) ticks wide). Sync, S&C, data, or checksum pulse periods are checked.
- SSPerror – triggers on a sync pulse whose width varies from the previous sync pulse's width by greater than 1/64 (1.5625%, as defined in the SENT specification).

Query Syntax :SBUS<n>:SENT:TRIGger?

The :SBUS<n>:SENT:TRIGger? query returns the trigger mode setting.

Return Format <mode><NL>

```
<mode> ::= {SFCM | SSCM | FCD | SCM | SCD | TOL | FCC | SCC | CRC
            | PPER | SSP}
```

- See Also**
- [":SBUS<n>:SENT:CLOCK"](#) on page 1086
 - [":SBUS<n>:SENT:CRC"](#) on page 1087
 - [":SBUS<n>:SENT:DISPlay"](#) on page 1088
 - [":SBUS<n>:SENT:FORMat"](#) on page 1090

- **":SBUS<n>:SENT:IDLE"** on page 1092
- **":SBUS<n>:SENT:LENGth"** on page 1093
- **":SBUS<n>:SENT:PPULse"** on page 1094
- **":SBUS<n>:SENT:SIGNal<s>:DISPlay"** on page 1096
- **":SBUS<n>:SENT:SIGNal<s>:LENGth"** on page 1097
- **":SBUS<n>:SENT:SIGNal<s>:MULTiplier"** on page 1099
- **":SBUS<n>:SENT:SIGNal<s>:OFFSet"** on page 1101
- **":SBUS<n>:SENT:SIGNal<s>:ORDer"** on page 1103
- **":SBUS<n>:SENT:SIGNal<s>:START"** on page 1105
- **":SBUS<n>:SENT:SOURce"** on page 1107
- **":SBUS<n>:SENT:TOLerance"** on page 1109
- **":SBUS<n>:SENT:TRIGger:FAST:DATA"** on page 1112
- **":SBUS<n>:SENT:TRIGger:SLOW:DATA"** on page 1113
- **":SBUS<n>:SENT:TRIGger:SLOW:ID"** on page 1115
- **":SBUS<n>:SENT:TRIGger:SLOW:ILENgtH"** on page 1117
- **":SBUS<n>:SENT:TRIGger:TOLerance"** on page 1118

:SBUS<n>:SENT:TRIGger:FAST:DATA

N (see [page 1650](#))

Command Syntax :SBUS<n>:SENT:TRIGger:FAST:DATA <string>

<string> ::= "nnnn..." where n ::= {0 | 1 | X}

<string> ::= "0xn..." where n ::= {0,...,9 | A,...,F | X | \$}

The :SBUS<n>:SENT:TRIGger:FAST:DATA command specifies the status and data nibbles that will be triggered on when the FCData trigger mode is chosen.

Query Syntax :SBUS<n>:SENT:TRIGger:FAST:DATA?

The :SBUS<n>:SENT:TRIGger:FAST:DATA? query returns the fast channel data trigger setting.

Return Format <string><NL>

<string> ::= "nnnn..." where n ::= {0 | 1 | X}

<string> ::= "0xn..." where n ::= {0,...,9 | A,...,F | X | \$}

- See Also**
- [":SBUS<n>:SENT:CLOCK"](#) on page 1086
 - [":SBUS<n>:SENT:CRC"](#) on page 1087
 - [":SBUS<n>:SENT:DISPlay"](#) on page 1088
 - [":SBUS<n>:SENT:FORMat"](#) on page 1090
 - [":SBUS<n>:SENT:IDLE"](#) on page 1092
 - [":SBUS<n>:SENT:LENGth"](#) on page 1093
 - [":SBUS<n>:SENT:PPULse"](#) on page 1094
 - [":SBUS<n>:SENT:SIGNal<s>:DISPlay"](#) on page 1096
 - [":SBUS<n>:SENT:SIGNal<s>:LENGth"](#) on page 1097
 - [":SBUS<n>:SENT:SIGNal<s>:MULTIplier"](#) on page 1099
 - [":SBUS<n>:SENT:SIGNal<s>:OFFSet"](#) on page 1101
 - [":SBUS<n>:SENT:SIGNal<s>:ORDer"](#) on page 1103
 - [":SBUS<n>:SENT:SIGNal<s>:START"](#) on page 1105
 - [":SBUS<n>:SENT:SOURce"](#) on page 1107
 - [":SBUS<n>:SENT:TOLerance"](#) on page 1109
 - [":SBUS<n>:SENT:TRIGger"](#) on page 1110
 - [":SBUS<n>:SENT:TRIGger:SLOW:DATA"](#) on page 1113
 - [":SBUS<n>:SENT:TRIGger:SLOW:ID"](#) on page 1115
 - [":SBUS<n>:SENT:TRIGger:SLOW:ILENGth"](#) on page 1117
 - [":SBUS<n>:SENT:TRIGger:TOLerance"](#) on page 1118

:SBUS<n>:SENT:TRIGger:SLOW:DATA

N (see [page 1650](#))

Command Syntax :SBUS<n>:SENT:TRIGger:SLOW:DATA <data>

<data> ::= when ILENgth = SHORt, from -1 (don't care) to 65535, in NR1 format.

<data> ::= when ILENgth = LONG, from -1 (don't care) to 4095, in NR1 format.

The :SBUS<n>:SENT:TRIGger:SLOW:DATA command specifies the data to trigger on for the Slow Channel Message ID and Data trigger mode.

Query Syntax :SBUS<n>:SENT:TRIGger:SLOW:DATA?

The :SBUS<n>:SENT:TRIGger:SLOW:DATA? query returns the data value setting for the slow channel ID and data trigger.

Return Format <data><NL>

<data> ::= when ILENgth = SHORt, from -1 (don't care) to 65535, in NR1 format.

<data> ::= when ILENgth = LONG, from -1 (don't care) to 4095, in NR1 format.

- See Also**
- [":SBUS<n>:SENT:CLOCK"](#) on page 1086
 - [":SBUS<n>:SENT:CRC"](#) on page 1087
 - [":SBUS<n>:SENT:DISPlay"](#) on page 1088
 - [":SBUS<n>:SENT:FORMat"](#) on page 1090
 - [":SBUS<n>:SENT:IDLE"](#) on page 1092
 - [":SBUS<n>:SENT:LENgth"](#) on page 1093
 - [":SBUS<n>:SENT:PPULse"](#) on page 1094
 - [":SBUS<n>:SENT:SIGNal<s>:DISPlay"](#) on page 1096
 - [":SBUS<n>:SENT:SIGNal<s>:LENgth"](#) on page 1097
 - [":SBUS<n>:SENT:SIGNal<s>:MULTiplier"](#) on page 1099
 - [":SBUS<n>:SENT:SIGNal<s>:OFFSet"](#) on page 1101
 - [":SBUS<n>:SENT:SIGNal<s>:ORDer"](#) on page 1103
 - [":SBUS<n>:SENT:SIGNal<s>:STARt"](#) on page 1105
 - [":SBUS<n>:SENT:SOURce"](#) on page 1107
 - [":SBUS<n>:SENT:TOLerance"](#) on page 1109
 - [":SBUS<n>:SENT:TRIGger"](#) on page 1110
 - [":SBUS<n>:SENT:TRIGger:FAST:DATA"](#) on page 1112
 - [":SBUS<n>:SENT:TRIGger:SLOW:ID"](#) on page 1115

- [":SBUS<n>:SENT:TRIGger:SLOW:ILENgtH"](#) on page 1117
- [":SBUS<n>:SENT:TRIGger:TOLerance"](#) on page 1118

:SBUS<n>:SENT:TRIGger:SLOW:ID

N (see [page 1650](#))

Command Syntax :SBUS<n>:SENT:TRIGger:SLOW:ID <id>

<id> ::= when ILENgth = SHORt, from -1 (don't care) to 15, in NR1 format
.

<id> ::= when ILENgth = LONG, from -1 (don't care) to 255, in NR1 format
.

The :SBUS<n>:SENT:TRIGger:SLOW:ID command specifies the ID to trigger on for the "Slow Channel Message ID" and "Slow Channel Message ID & Data" trigger modes. The ID can be from -1 (don't care) to 255 (depending on the message length).

Query Syntax :SBUS<n>:SENT:TRIGger:SLOW:ID?

The :SBUS<n>:SENT:TRIGger:SLOW:ID? query returns the slow channel ID setting.

Return Format <id><NL>

<id> ::= when ILENgth = SHORt, from -1 (don't care) to 15, in NR1 format
.

<id> ::= when ILENgth = LONG, from -1 (don't care) to 255, in NR1 format
.

- See Also**
- [":SBUS<n>:SENT:CLOCK"](#) on page 1086
 - [":SBUS<n>:SENT:CRC"](#) on page 1087
 - [":SBUS<n>:SENT:DISPlay"](#) on page 1088
 - [":SBUS<n>:SENT:FORMat"](#) on page 1090
 - [":SBUS<n>:SENT:IDLE"](#) on page 1092
 - [":SBUS<n>:SENT:LENgth"](#) on page 1093
 - [":SBUS<n>:SENT:PPULse"](#) on page 1094
 - [":SBUS<n>:SENT:SIGNal<s>:DISPlay"](#) on page 1096
 - [":SBUS<n>:SENT:SIGNal<s>:LENgth"](#) on page 1097
 - [":SBUS<n>:SENT:SIGNal<s>:MULTiplier"](#) on page 1099
 - [":SBUS<n>:SENT:SIGNal<s>:OFFSet"](#) on page 1101
 - [":SBUS<n>:SENT:SIGNal<s>:ORDer"](#) on page 1103
 - [":SBUS<n>:SENT:SIGNal<s>:STARt"](#) on page 1105
 - [":SBUS<n>:SENT:SOURce"](#) on page 1107
 - [":SBUS<n>:SENT:TOLerance"](#) on page 1109
 - [":SBUS<n>:SENT:TRIGger"](#) on page 1110
 - [":SBUS<n>:SENT:TRIGger:FAST:DATA"](#) on page 1112

- **":SBUS<n>:SENT:TRIGger:SLOW:DATA"** on page 1113
- **":SBUS<n>:SENT:TRIGger:SLOW:ILENgtH"** on page 1117
- **":SBUS<n>:SENT:TRIGger:TOLerance"** on page 1118

:SBUS<n>:SENT:TRIGger:SLOW:ILENgtH

N (see [page 1650](#))

Command Syntax :SBUS<n>:SENT:TRIGger:SLOW:ILENgtH <length>

<length> ::= {SHORT | LONG}

The :SBUS<n>:SENT:TRIGger:SLOW:ILENgtH command specifies the ID and data lengths for the Slow Message Enhanced messages. Either "SHORT" for the 4-bit ID, 16-bit data format, or "LONG" for the 8-bit ID, 12-bit data format.

Query Syntax :SBUS<n>:SENT:TRIGger:SLOW:ILENgtH?

The :SBUS<n>:SENT:TRIGger:SLOW:ILENgtH? query returns the ID and data length setting.

Return Format <length><NL>

<length> ::= {SHOR | LONG}

- See Also**
- [":SBUS<n>:SENT:CLOCK"](#) on page 1086
 - [":SBUS<n>:SENT:CRC"](#) on page 1087
 - [":SBUS<n>:SENT:DISPlay"](#) on page 1088
 - [":SBUS<n>:SENT:FORMat"](#) on page 1090
 - [":SBUS<n>:SENT:IDLE"](#) on page 1092
 - [":SBUS<n>:SENT:LENGth"](#) on page 1093
 - [":SBUS<n>:SENT:PPULse"](#) on page 1094
 - [":SBUS<n>:SENT:SIGNal<s>:DISPlay"](#) on page 1096
 - [":SBUS<n>:SENT:SIGNal<s>:LENGth"](#) on page 1097
 - [":SBUS<n>:SENT:SIGNal<s>:MULTIplier"](#) on page 1099
 - [":SBUS<n>:SENT:SIGNal<s>:OFFSet"](#) on page 1101
 - [":SBUS<n>:SENT:SIGNal<s>:ORDer"](#) on page 1103
 - [":SBUS<n>:SENT:SIGNal<s>:START"](#) on page 1105
 - [":SBUS<n>:SENT:SOURce"](#) on page 1107
 - [":SBUS<n>:SENT:TOLerance"](#) on page 1109
 - [":SBUS<n>:SENT:TRIGger"](#) on page 1110
 - [":SBUS<n>:SENT:TRIGger:FAST:DATA"](#) on page 1112
 - [":SBUS<n>:SENT:TRIGger:SLOW:DATA"](#) on page 1113
 - [":SBUS<n>:SENT:TRIGger:SLOW:ID"](#) on page 1115
 - [":SBUS<n>:SENT:TRIGger:TOLerance"](#) on page 1118

:SBUS<n>:SENT:TRIGger:TOLerance

N (see [page 1650](#))

Command Syntax :SBUS<n>:SENT:TRIGger:TOLerance <percent>

<percent> ::= from 1-28, in NR1 format.

The :SBUS<n>:SENT:TRIGger:TOLerance command specifies the tolerance variation that is considered a violation.

The trigger tolerance can be up to the :SBUS<n>:SENT:TOLerance setting minus two percent. For example, after sending ":SBUS1:SENT:TOLerance 20", the :SBUS1:SENT:TRIGger:TOLerance setting can be from 1 to 18.

Query Syntax :SBUS<n>:SENT:TRIGger:TOLerance?

The :SBUS<n>:SENT:TRIGger:TOLerance? query returns tolerance variation percent setting.

Return Format <percent><NL>

<percent> ::= from 1-28, in NR1 format.

- See Also**
- [":SBUS<n>:SENT:CLOCK"](#) on page 1086
 - [":SBUS<n>:SENT:CRC"](#) on page 1087
 - [":SBUS<n>:SENT:DISPlay"](#) on page 1088
 - [":SBUS<n>:SENT:FORMat"](#) on page 1090
 - [":SBUS<n>:SENT:IDLE"](#) on page 1092
 - [":SBUS<n>:SENT:LENGth"](#) on page 1093
 - [":SBUS<n>:SENT:PPULse"](#) on page 1094
 - [":SBUS<n>:SENT:SIGNal<s>:DISPlay"](#) on page 1096
 - [":SBUS<n>:SENT:SIGNal<s>:LENGth"](#) on page 1097
 - [":SBUS<n>:SENT:SIGNal<s>:MULTiplier"](#) on page 1099
 - [":SBUS<n>:SENT:SIGNal<s>:OFFSet"](#) on page 1101
 - [":SBUS<n>:SENT:SIGNal<s>:ORDer"](#) on page 1103
 - [":SBUS<n>:SENT:SIGNal<s>:START"](#) on page 1105
 - [":SBUS<n>:SENT:SOURce"](#) on page 1107
 - [":SBUS<n>:SENT:TOLerance"](#) on page 1109
 - [":SBUS<n>:SENT:TRIGger"](#) on page 1110
 - [":SBUS<n>:SENT:TRIGger:FAST:DATA"](#) on page 1112
 - [":SBUS<n>:SENT:TRIGger:SLOW:DATA"](#) on page 1113
 - [":SBUS<n>:SENT:TRIGger:SLOW:ID"](#) on page 1115
 - [":SBUS<n>:SENT:TRIGger:SLOW:ILENgtH"](#) on page 1117

:SBUS<n>:SPI Commands

NOTE

These commands are only valid when the low-speed IIC and SPI serial decode option has been licensed.

Table 132 :SBUS<n>:SPI Commands Summary

Command	Query	Options and Query Returns
:SBUS<n>:SPI:BITOrder<order> (see page 1121)	:SBUS<n>:SPI:BITOrder? (see page 1121)	<order> ::= {LSBFirst MSBFirst}
:SBUS<n>:SPI:CLOCK:SLOPe <slope> (see page 1122)	:SBUS<n>:SPI:CLOCK:SLOPe? (see page 1122)	<slope> ::= {NEGative POSitive}
:SBUS<n>:SPI:CLOCK:TIMeout <time_value> (see page 1123)	:SBUS<n>:SPI:CLOCK:TIMeout? (see page 1123)	<time_value> ::= time in seconds in NR3 format
:SBUS<n>:SPI:DELAy <value> (see page 1124)	:SBUS<n>:SPI:DELAy? (see page 1124)	<value> ::= {OFF 2-63}
:SBUS<n>:SPI:FRAMing <value> (see page 1125)	:SBUS<n>:SPI:FRAMing? (see page 1125)	<value> ::= {CHIPselect {NCHIPselect NOTC} TIMEout}
:SBUS<n>:SPI:SOURce:CLOCK <source> (see page 1126)	:SBUS<n>:SPI:SOURce:CLOCK? (see page 1126)	<value> ::= {CHANnel<n> EXTERNAL} for the DSO models <value> ::= {CHANnel<n> DIGital<d>} for the MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:SBUS<n>:SPI:SOURce:FRAME <source> (see page 1127)	:SBUS<n>:SPI:SOURce:FRAME? (see page 1127)	<value> ::= {CHANnel<n> EXTERNAL} for the DSO models <value> ::= {CHANnel<n> DIGital<d>} for the MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format

Table 132 :SBUS<n>:SPI Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:SPI:SOURce:MISO <source> (see page 1128)	:SBUS<n>:SPI:SOURce:MISO? (see page 1128)	<value> ::= {CHANnel<n> EXTeRnal} for the DSO models <value> ::= {CHANnel<n> DIGital<d>} for the MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:SBUS<n>:SPI:SOURce:MOSI <source> (see page 1129)	:SBUS<n>:SPI:SOURce:MOSI? (see page 1129)	<value> ::= {CHANnel<n> EXTeRnal} for the DSO models <value> ::= {CHANnel<n> DIGital<d>} for the MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:SBUS<n>:SPI:TRIGger:PATtern:MISO:DATA <string> (see page 1130)	:SBUS<n>:SPI:TRIGger:PATtern:MISO:DATA? (see page 1130)	<string> ::= "nn...n" where n ::= {0 1 X \$} <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X \$}
:SBUS<n>:SPI:TRIGger:PATtern:MISO:WIDTh <width> (see page 1131)	:SBUS<n>:SPI:TRIGger:PATtern:MISO:WIDTh? (see page 1131)	<width> ::= integer from 4 to 64 in NR1 format
:SBUS<n>:SPI:TRIGger:PATtern:MOSI:DATA <string> (see page 1132)	:SBUS<n>:SPI:TRIGger:PATtern:MOSI:DATA? (see page 1132)	<string> ::= "nn...n" where n ::= {0 1 X \$} <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X \$}
:SBUS<n>:SPI:TRIGger:PATtern:MOSI:WIDTh <width> (see page 1133)	:SBUS<n>:SPI:TRIGger:PATtern:MOSI:WIDTh? (see page 1133)	<width> ::= integer from 4 to 64 in NR1 format
:SBUS<n>:SPI:TRIGger:TYPE <value> (see page 1134)	:SBUS<n>:SPI:TRIGger:TYPE? (see page 1134)	<value> ::= {MOSI MISO}
:SBUS<n>:SPI:WIDTh <word_width> (see page 1135)	:SBUS<n>:SPI:WIDTh? (see page 1135)	<word_width> ::= integer 4-16 in NR1 format

:SBUS<n>:SPI:BITOrder

N (see [page 1650](#))

Command Syntax :SBUS<n>:SPI:BITOrder <order>

<order> ::= {LSBFirst | MSBFirst}

The :SBUS<n>:SPI:BITOrder command selects the bit order, most significant bit first (MSB) or least significant bit first (LSB), used when displaying data in the serial decode waveform and in the Lister.

Query Syntax :SBUS<n>:SPI:BITOrder?

The :SBUS<n>:SPI:BITOrder? query returns the current SPI decode bit order.

Return Format <order><NL>

<order> ::= {LSBF | MSBF}

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • ["Introduction to :SBUS<n> Commands"](#) on page 899
 • [":SBUS<n>:MODE"](#) on page 903
 • [":SBUS<n>:SPI Commands"](#) on page 1119

:SBUS<n>:SPI:CLOCK:SLOPe

N (see [page 1650](#))

Command Syntax :SBUS<n>:SPI:CLOCK:SLOPe <slope>

<slope> ::= {NEGative | POSitive}

The :SBUS<n>:SPI:CLOCK:SLOPe command specifies the rising edge (POSitive) or falling edge (NEGative) of the SPI clock source that will clock in the data.

Query Syntax :SBUS<n>:SPI:CLOCK:SLOPe?

The :SBUS<n>:SPI:CLOCK:SLOPe? query returns the current SPI clock source slope.

Return Format <slope><NL>

<slope> ::= {NEG | POS}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:SPI:CLOCK:TIMEout"](#) on page 1123
 - [":SBUS<n>:SPI:SOURce:CLOCK"](#) on page 1126

:SBUS<n>:SPI:CLOCK:TIMEout

N (see [page 1650](#))

Command Syntax :SBUS<n>:SPI:CLOCK:TIMEout <time_value>

<time_value> ::= time in seconds in NR3 format

The :SBUS<n>:SPI:CLOCK:TIMEout command sets the SPI signal clock timeout resource in seconds from 100 ns to 10 s when the :SBUS<n>:SPI:FRAMing command is set to TIMEout. The timer is used to frame a signal by a clock timeout.

Query Syntax :SBUS<n>:SPI:CLOCK:TIMEout?

The :SBUS<n>:SPI:CLOCK:TIMEout? query returns current SPI clock timeout setting.

Return Format <time_value><NL>

<time_value> ::= time in seconds in NR3 format

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:SPI:CLOCK:SLOPe"](#) on page 1122
 - [":SBUS<n>:SPI:SOURce:CLOCK"](#) on page 1126
 - [":SBUS<n>:SPI:FRAMing"](#) on page 1125

:SBUS<n>:SPI:DELaY

N (see [page 1650](#))

Command Syntax :SBUS<n>:SPI:DELaY <value>

<value> ::= {OFF | 2-63}

The :SBUS<n>:SPI:DELaY command specifies the number of bits to ignore (delay) before decoding the MISO stream.

Query Syntax :SBUS<n>:SPI:DELaY?

The :SBUS<n>:SPI:DELaY? query returns the specified number of delay bits or "OFF" if there are none.

Return Format <value><NL>

<value> ::= {OFF | 2-63}

See Also • [":SBUS<n>:SPI:SOURce:MISO"](#) on page 1128

:SBUS<n>:SPI:FRAMing

N (see [page 1650](#))

Command Syntax :SBUS<n>:SPI:FRAMing <value>

<value> ::= {CHIPselect | {NCHipselect | NOTC} | TIMEout}

The :SBUS<n>:SPI:FRAMing command sets the SPI trigger framing value. If TIMEout is selected, the timeout value is set by the :SBUS<n>:SPI:CLOCK:TIMEout command.

NOTE

The NOTC value is deprecated. It is the same as NCHipselect.

Query Syntax :SBUS<n>:SPI:FRAMing?

The :SBUS<n>:SPI:FRAMing? query returns the current SPI framing value.

Return Format <value><NL>

<value> ::= {CHIP | NCH | TIM}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":SBUS<n>:SPI:CLOCK:TIMEout"](#) on page 1123
 - [":SBUS<n>:SPI:SOURce:FRAME"](#) on page 1127

:SBUS<n>:SPI:SOURce:CLOCK

N (see [page 1650](#))

Command Syntax :SBUS<n>:SPI:SOURce:CLOCK <source>

<source> ::= {CHANnel<n> | EXternal} for the DSO models

<source> ::= {CHANnel<n> | DIGital<d>} for the MSO models

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :SBUS<n>:SPI:SOURce:CLOCK command sets the source for the SPI serial clock.

Query Syntax :SBUS<n>:SPI:SOURce:CLOCK?

The :SBUS<n>:SPI:SOURce:CLOCK? query returns the current source for the SPI serial clock.

Return Format <source><NL>

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:SPI:CLOCK:SLOPe"](#) on page 1122
 - [":SBUS<n>:SPI:CLOCK:TIMEout"](#) on page 1123
 - [":SBUS<n>:SPI:SOURce:FRAME"](#) on page 1127
 - [":SBUS<n>:SPI:SOURce:MOSI"](#) on page 1129
 - [":SBUS<n>:SPI:SOURce:MISO"](#) on page 1128

:SBUS<n>:SPI:SOURce:FRAME

N (see [page 1650](#))

Command Syntax :SBUS<n>:SPI:SOURce:FRAME <source>

<source> ::= {CHANnel<n> | EXTernal} for the DSO models

<source> ::= {CHANnel<n> | DIGital<d>} for the MSO models

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :SBUS<n>:SPI:SOURce:FRAME command sets the frame source when :SBUS<n>:SPI:FRAMing is set to CHIPselect or NOTChipselect.

Query Syntax :SBUS<n>:SPI:SOURce:FRAME?

The :SBUS<n>:SPI:SOURce:FRAME? query returns the current frame source for the SPI serial frame.

Return Format <source><NL>

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:SPI:SOURce:CLOCK"](#) on page 1126
 - [":SBUS<n>:SPI:SOURce:MOSI"](#) on page 1129
 - [":SBUS<n>:SPI:SOURce:MISO"](#) on page 1128
 - [":SBUS<n>:SPI:FRAMing"](#) on page 1125

:SBUS<n>:SPI:SOURce:MISO

N (see [page 1650](#))

Command Syntax :SBUS<n>:SPI:SOURce:MISO <source>

<source> ::= {CHANnel<n> | EXternal} for the DSO models

<source> ::= {CHANnel<n> | DIGital<d>} for the MSO models

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :SBUS<n>:SPI:SOURce:MISO command sets the source for the SPI serial MISO data.

Query Syntax :SBUS<n>:SPI:SOURce:MISO?

The :SBUS<n>:SPI:SOURce:MISO? query returns the current source for the SPI serial MISO data.

Return Format <source><NL>

- See Also**
- [":SBUS<n>:SPI:DELaY"](#) on page 1124
 - ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:SPI:SOURce:MOSI"](#) on page 1129
 - [":SBUS<n>:SPI:SOURce:CLOCK"](#) on page 1126
 - [":SBUS<n>:SPI:SOURce:FRAMe"](#) on page 1127
 - [":SBUS<n>:SPI:TRIGger:PATtern:MISO:DATA"](#) on page 1130
 - [":SBUS<n>:SPI:TRIGger:PATtern:MOSI:DATA"](#) on page 1132
 - [":SBUS<n>:SPI:TRIGger:PATtern:MISO:WIDTh"](#) on page 1131
 - [":SBUS<n>:SPI:TRIGger:PATtern:MOSI:WIDTh"](#) on page 1133

:SBUS<n>:SPI:SOURce:MOSI

N (see [page 1650](#))

Command Syntax :SBUS<n>:SPI:SOURce:MOSI <source>

<source> ::= {CHANnel<n> | EXTernal} for the DSO models

<source> ::= {CHANnel<n> | DIGital<d>} for the MSO models

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :SBUS<n>:SPI:SOURce:MOSI command sets the source for the SPI serial MOSI data.

Query Syntax :SBUS<n>:SPI:SOURce:MOSI?

The :SBUS<n>:SPI:SOURce:MOSI? query returns the current source for the SPI serial MOSI data.

Return Format <source><NL>

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:SPI:SOURce:MISO"](#) on page 1128
 - [":SBUS<n>:SPI:SOURce:CLOCK"](#) on page 1126
 - [":SBUS<n>:SPI:SOURce:FRAME"](#) on page 1127
 - [":SBUS<n>:SPI:TRIGger:PATtern:MISO:DATA"](#) on page 1130
 - [":SBUS<n>:SPI:TRIGger:PATtern:MOSI:DATA"](#) on page 1132
 - [":SBUS<n>:SPI:TRIGger:PATtern:MISO:WIDTH"](#) on page 1131
 - [":SBUS<n>:SPI:TRIGger:PATtern:MOSI:WIDTH"](#) on page 1133

:SBUS<n>:SPI:TRIGger:PATtern:MISO:DATA

N (see [page 1650](#))

Command Syntax :SBUS<n>:SPI:TRIGger:PATtern:MISO:DATA <string>

<string> ::= "nn...n" where n ::= {0 | 1 | X | \$}

<string ::= "0xnn...n" where n ::= {0,...,9 | A,...,F | X | \$}

The :SBUS<n>:SPI:TRIGger:PATtern:MISO:DATA command defines the SPI data pattern resource according to the string parameter. This pattern, along with the data width, control the data pattern searched for in the data stream.

If the string parameter starts with "0x", it is a hexadecimal string made up of hexadecimal and X (don't care) characters; otherwise, it is a binary string made up of 0, 1, and X (don't care) characters.

NOTE

The :SBUS<n>:SPI:TRIGger:PATtern:MISO:WIDTH should be set before :SBUS<n>:SPI:TRIGger:PATtern:MISO:DATA.

Query Syntax :SBUS<n>:SPI:TRIGger:PATtern:MISO:DATA?

The :SBUS<n>:SPI:TRIGger:PATtern:MISO:DATA? query returns the current settings of the specified SPI data pattern resource in the binary string format.

Return Format <string><NL>

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:SPI:TRIGger:PATtern:MISO:WIDTH"](#) on page 1131
 - [":SBUS<n>:SPI:SOURce:MISO"](#) on page 1128

:SBUS<n>:SPI:TRIGger:PATtern:MISO:WIDTh

N (see [page 1650](#))

Command Syntax :SBUS<n>:SPI:TRIGger:PATtern:MISO:WIDTh <width>

<width> ::= integer from 4 to 64 in NR1 format

The :SBUS<n>:SPI:TRIGger:PATtern:MISO:WIDTh command sets the width of the SPI data pattern anywhere from 4 bits to 64 bits.

NOTE

The :SBUS<n>:SPI:TRIGger:PATtern:MISO:WIDTh should be set before :SBUS<n>:SPI:TRIGger:PATtern:MISO:DATA.

Query Syntax :SBUS<n>:SPI:TRIGger:PATtern:MISO:WIDTh?

The :SBUS<n>:SPI:TRIGger:PATtern:MISO:WIDTh? query returns the current SPI data pattern width setting.

Return Format <width><NL>

<width> ::= integer from 4 to 64 in NR1 format

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:SPI:TRIGger:PATtern:MISO:DATA"](#) on page 1130
 - [":SBUS<n>:SPI:SOURce:MISO"](#) on page 1128

:SBUS<n>:SPI:TRIGger:PATtern:MOSI:DATA

N (see [page 1650](#))

Command Syntax :SBUS<n>:SPI:TRIGger:PATtern:MOSI:DATA <string>

<string> ::= "nn...n" where n ::= {0 | 1 | X | \$}

<string> ::= "0xnn...n" where n ::= {0,...,9 | A,...,F | X | \$}

The :SBUS<n>:SPI:TRIGger:PATtern:MOSI:DATA command defines the SPI data pattern resource according to the string parameter. This pattern, along with the data width, control the data pattern searched for in the data stream.

If the string parameter starts with "0x", it is a hexadecimal string made up of hexadecimal and X (don't care) characters; otherwise, it is a binary string made up of 0, 1, and X (don't care) characters.

NOTE

The :SBUS<n>:SPI:TRIGger:PATtern:MOSI:WIDTH should be set before :SBUS<n>:SPI:TRIGger:PATtern:MOSI:DATA.

Query Syntax :SBUS<n>:SPI:TRIGger:PATtern:MOSI:DATA?

The :SBUS<n>:SPI:TRIGger:PATtern:MOSI:DATA? query returns the current settings of the specified SPI data pattern resource in the binary string format.

Return Format <string><NL>

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:SPI:TRIGger:PATtern:MOSI:WIDTH"](#) on page 1133
 - [":SBUS<n>:SPI:SOURce:MOSI"](#) on page 1129

:SBUS<n>:SPI:TRIGger:PATtern:MOSI:WIDTh

N (see [page 1650](#))

Command Syntax :SBUS<n>:SPI:TRIGger:PATtern:MOSI:WIDTh <width>

<width> ::= integer from 4 to 64 in NR1 format

The :SBUS<n>:SPI:TRIGger:PATtern:MOSI:WIDTh command sets the width of the SPI data pattern anywhere from 4 bits to 64 bits.

NOTE

The :SBUS<n>:SPI:TRIGger:PATtern:MOSI:WIDTh should be set before :SBUS<n>:SPI:TRIGger:PATtern:MOSI:DATA.

Query Syntax :SBUS<n>:SPI:TRIGger:PATtern:MOSI:WIDTh?

The :SBUS<n>:SPI:TRIGger:PATtern:MOSI:WIDTh? query returns the current SPI data pattern width setting.

Return Format <width><NL>

<width> ::= integer from 4 to 64 in NR1 format

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:SPI:TRIGger:PATtern:MOSI:DATA"](#) on page 1132
 - [":SBUS<n>:SPI:SOURce:MOSI"](#) on page 1129

:SBUS<n>:SPI:TRIGger:TYPE

N (see [page 1650](#))

Command Syntax :SBUS<n>:SPI:TRIGger:TYPE <value>

<value> ::= {MOSI | MISO}

The :SBUS<n>:SPI:TRIGger:TYPE command specifies whether the SPI trigger will be on the MOSI data or the MISO data.

When triggering on MOSI data, the data value is specified by the :SBUS<n>:SPI:TRIGger:PATtern:MOSI:DATA and :SBUS<n>:SPI:TRIGger:PATtern:MOSI:WIDTh commands.

When triggering on MISO data, the data value is specified by the :SBUS<n>:SPI:TRIGger:PATtern:MISO:DATA and :SBUS<n>:SPI:TRIGger:PATtern:MISO:WIDTh commands.

Query Syntax :SBUS<n>:SPI:TRIGger:TYPE?

The :SBUS<n>:SPI:TRIGger:TYPE? query returns the current SPI trigger type setting.

Return Format <value><NL>

<value> ::= {MOSI | MISO}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:SPI:SOURce:MOSI"](#) on page 1129
 - [":SBUS<n>:SPI:SOURce:MISO"](#) on page 1128
 - [":SBUS<n>:SPI:TRIGger:PATtern:MISO:DATA"](#) on page 1130
 - [":SBUS<n>:SPI:TRIGger:PATtern:MOSI:DATA"](#) on page 1132
 - [":SBUS<n>:SPI:TRIGger:PATtern:MISO:WIDTh"](#) on page 1131
 - [":SBUS<n>:SPI:TRIGger:PATtern:MOSI:WIDTh"](#) on page 1133
 - [":TRIGger:MODE"](#) on page 1358

:SBUS<n>:SPI:WIDTh

N (see [page 1650](#))

Command Syntax :SBUS<n>:SPI:WIDTh <word_width>

<word_width> ::= integer 4-16 in NR1 format

The :SBUS<n>:SPI:WIDTh command determines the number of bits in a word of data for SPI.

Query Syntax :SBUS<n>:SPI:WIDTh?

The :SBUS<n>:SPI:WIDTh? query returns the current SPI decode word width.

Return Format <word_width><NL>

<word_width> ::= integer 4-16 in NR1 format

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • ["Introduction to :SBUS<n> Commands"](#) on page 899

• [":SBUS<n>:MODE"](#) on page 903

• [":SBUS<n>:SPI Commands"](#) on page 1119

:SBUS<n>:UART Commands

NOTE

These commands are only valid when the UART/RS-232 triggering and serial decode option has been licensed.

Table 133 :SBUS<n>:UART Commands Summary

Command	Query	Options and Query Returns
:SBUS<n>:UART:BASE<base> (see page 1139)	:SBUS<n>:UART:BASE? (see page 1139)	<base> ::= {ASCIi BINary HEX}
:SBUS<n>:UART:BAUDRate <baudrate> (see page 1140)	:SBUS<n>:UART:BAUDRate? (see page 1140)	<baudrate> ::= integer from 100 to 8000000, 10000000, or 12000000
:SBUS<n>:UART:BITOrder <bitorder> (see page 1141)	:SBUS<n>:UART:BITOrder? (see page 1141)	<bitorder> ::= {LSBFirst MSBFirst}
n/a	:SBUS<n>:UART:COUNT:ERRor? (see page 1142)	<frame_count> ::= integer in NR1 format
:SBUS<n>:UART:COUNT:RESet (see page 1143)	n/a	n/a
n/a	:SBUS<n>:UART:COUNT:RXFrames? (see page 1144)	<frame_count> ::= integer in NR1 format
n/a	:SBUS<n>:UART:COUNT:TXFrames? (see page 1145)	<frame_count> ::= integer in NR1 format
:SBUS<n>:UART:FRAMing<value> (see page 1146)	:SBUS<n>:UART:FRAMing? (see page 1146)	<value> ::= {OFF <decimal> <nondecimal>} <decimal> ::= 8-bit integer from 0-255 (0x00-0xff) <nondecimal> ::= #Hnn where n ::= {0,...,9 A,...,F} for hexadecimal <nondecimal> ::= #Bnn...n where n ::= {0 1} for binary
:SBUS<n>:UART:PARity<parity> (see page 1147)	:SBUS<n>:UART:PARity? (see page 1147)	<parity> ::= {EVEN ODD NONE}
:SBUS<n>:UART:POLarity <polarity> (see page 1148)	:SBUS<n>:UART:POLarity? (see page 1148)	<polarity> ::= {HIGH LOW}

Table 133 :SBUS<n>:UART Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:UART:SOURce:RX <source> (see page 1149)	:SBUS<n>:UART:SOURce:RX? (see page 1149)	<source> ::= {CHANnel<n> EXTernal} for DSO models <source> ::= {CHANnel<n> DIGital<d>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:SBUS<n>:UART:SOURce:TX <source> (see page 1150)	:SBUS<n>:UART:SOURce:TX? (see page 1150)	<source> ::= {CHANnel<n> EXTernal} for DSO models <source> ::= {CHANnel<n> DIGital<d>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:SBUS<n>:UART:TRIGger:BASE <base> (see page 1151)	:SBUS<n>:UART:TRIGger:BASE? (see page 1151)	<base> ::= {ASCii HEX}
:SBUS<n>:UART:TRIGger:BURSt <value> (see page 1152)	:SBUS<n>:UART:TRIGger:BURSt? (see page 1152)	<value> ::= {OFF 1 to 4096 in NR1 format}
:SBUS<n>:UART:TRIGger:DATA <value> (see page 1153)	:SBUS<n>:UART:TRIGger:DATA? (see page 1153)	<value> ::= 8-bit integer from 0-255 (0x00-0xff) in decimal, <hexadecimal>, <binary>, or <quoted_string> format <hexadecimal> ::= #Hnn where n ::= {0,...,9 A,...,F} for hexadecimal <binary> ::= #Bnn...n where n ::= {0 1} for binary <quoted_string> ::= any of the 128 valid 7-bit ASCII characters (or standard abbreviations)
:SBUS<n>:UART:TRIGger:IDLE <time_value> (see page 1154)	:SBUS<n>:UART:TRIGger:IDLE? (see page 1154)	<time_value> ::= time from 1 us to 10 s in NR3 format
:SBUS<n>:UART:TRIGger:QUALifier <value> (see page 1155)	:SBUS<n>:UART:TRIGger:QUALifier? (see page 1155)	<value> ::= {EQUal NOTequal GREATERthan LESSthan}

Table 133 :SBUS<n>:UART Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:UART:TRIGger :TYPE <value> (see page 1156)	:SBUS<n>:UART:TRIGger :TYPE? (see page 1156)	<value> ::= {RStArt RStOp RDATA RD1 RD0 RDX PARityerror TStArt TStOp TDATA TD1 TD0 TDX}
:SBUS<n>:UART:WIDTh <width> (see page 1157)	:SBUS<n>:UART:WIDTh? (see page 1157)	<width> ::= {5 6 7 8 9}

:SBUS<n>:UART:BASE

N (see [page 1650](#))

Command Syntax :SBUS<n>:UART:BASE <base>

<base> ::= {ASCIi | BINary | HEX}

The :SBUS<n>:UART:BASE command determines the base to use for the UART decode and Lister display.

Query Syntax :SBUS<n>:UART:BASE?

The :SBUS<n>:UART:BASE? query returns the current UART decode and Lister base setting.

Return Format <base><NL>

<base> ::= {ASCIi | BINary | HEX}

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • ["Introduction to :SBUS<n> Commands"](#) on page 899
 • [":SBUS<n>:UART Commands"](#) on page 1136

:SBUS<n>:UART:BAUDrate

N (see [page 1650](#))

Command Syntax :SBUS<n>:UART:BAUDrate <baudrate>

<baudrate> ::= integer from 100 to 8000000, 10000000, or 12000000

The :SBUS<n>:UART:BAUDrate command selects the bit rate (in bps) for the serial decoder and/or trigger when in UART mode. The baud rate can be set in the range from 100 b/s to 8 Mb/s or to the specific values of 10 Mb/s or 12 Mb/s.

If the baud rate you select does not match the system baud rate, false triggers may occur.

Query Syntax :SBUS<n>:UART:BAUDrate?

The :SBUS<n>:UART:BAUDrate? query returns the current UART baud rate setting.

Return Format <baudrate><NL>

<baudrate> ::= integer from 100 to 8000000, 10000000, or 12000000

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":SBUS<n>:UART:TRIGger:TYPE"](#) on page 1156

:SBUS<n>:UART:BITOrder

N (see [page 1650](#))

Command Syntax :SBUS<n>:UART:BITOrder <bitorder>

<bitorder> ::= {LSBFirst | MSBFirst}

The :SBUS<n>:UART:BITOrder command specifies the order of transmission used by the physical Tx and Rx input signals for the serial decoder and/or trigger when in UART mode. LSBFirst sets the least significant bit of each message "byte" as transmitted first. MSBFirst sets the most significant bit as transmitted first.

Query Syntax :SBUS<n>:UART:BITOrder?

The :SBUS<n>:UART:BITOrder? query returns the current UART bit order setting.

Return Format <bitorder><NL>

<bitorder> ::= {LSBF | MSBF}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":SBUS<n>:UART:TRIGger:TYPE"](#) on page 1156
 - [":SBUS<n>:UART:SOURce:RX"](#) on page 1149
 - [":SBUS<n>:UART:SOURce:TX"](#) on page 1150

:SBUS<n>:UART:COUNT:ERRor

N (see [page 1650](#))

Query Syntax :SBUS<n>:UART:COUNT:ERRor?

Returns the UART error frame count.

Return Format <frame_count><NL>

<frame_count> ::= integer in NR1 format

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • [":SBUS<n>:UART:COUNT:RESet"](#) on page 1143
 • ["Introduction to :SBUS<n> Commands"](#) on page 899
 • [":SBUS<n>:MODE"](#) on page 903
 • [":SBUS<n>:UART Commands"](#) on page 1136

:SBUS<n>:UART:COUNT:RESet

N (see [page 1650](#))

Command Syntax :SBUS<n>:UART:COUNT:RESet

Resets the UART frame counters.

Errors • ["-241, Hardware missing"](#) on page 1591

- See Also** • [":SBUS<n>:UART:COUNT:ERRor"](#) on page 1142
- [":SBUS<n>:UART:COUNT:RXFRames"](#) on page 1144
 - [":SBUS<n>:UART:COUNT:TXFRames"](#) on page 1145
 - ["Introduction to :SBUS<n> Commands"](#) on page 899
 - [":SBUS<n>:MODE"](#) on page 903
 - [":SBUS<n>:UART Commands"](#) on page 1136

:SBUS<n>:UART:COUNT:RXFRames

N (see [page 1650](#))

Query Syntax :SBUS<n>:UART:COUNT:RXFRames?

Returns the UART Rx frame count.

Return Format <frame_count><NL>

<frame_count> ::= integer in NR1 format

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • [":SBUS<n>:UART:COUNT:RESet"](#) on page 1143
 • ["Introduction to :SBUS<n> Commands"](#) on page 899
 • [":SBUS<n>:MODE"](#) on page 903
 • [":SBUS<n>:UART Commands"](#) on page 1136

:SBUS<n>:UART:COUNT:TXFRames

N (see [page 1650](#))

Query Syntax :SBUS<n>:UART:COUNT:TXFRames?

Returns the UART Tx frame count.

Return Format <frame_count><NL>

<frame_count> ::= integer in NR1 format

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • [":SBUS<n>:UART:COUNT:RESet"](#) on page 1143
 • ["Introduction to :SBUS<n> Commands"](#) on page 899
 • [":SBUS<n>:MODE"](#) on page 903
 • [":SBUS<n>:UART Commands"](#) on page 1136

:SBUS<n>:UART:FRAMing

N (see [page 1650](#))

Command Syntax :SBUS<n>:UART:FRAMing <value>

<value> ::= {OFF | <decimal> | <nondecimal>}

<decimal> ::= 8-bit integer in decimal from 0-255 (0x00-0xff)

<nondecimal> ::= #Hnn where n ::= {0,...,9 | A,...,F} for hexadecimal

<nondecimal> ::= #Bnn...n where n ::= {0 | 1} for binary

The :SBUS<n>:UART:FRAMing command determines the byte value to use for framing (end of packet) or to turn off framing for UART decode.

Query Syntax :SBUS<n>:UART:FRAMing?

The :SBUS<n>:UART:FRAMing? query returns the current UART decode base setting.

Return Format <value><NL>

<value> ::= {OFF | <decimal>}

<decimal> ::= 8-bit integer in decimal from 0-255

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • ["Introduction to :SBUS<n> Commands"](#) on page 899

• [":SBUS<n>:UART Commands"](#) on page 1136

:SBUS<n>:UART:PARity

N (see [page 1650](#))

Command Syntax :SBUS<n>:UART:PARity <parity>

<parity> ::= {EVEN | ODD | NONE}

The :SBUS<n>:UART:PARity command selects the parity to be used with each message "byte" for the serial decoder and/or trigger when in UART mode.

Query Syntax :SBUS<n>:UART:PARity?

The :SBUS<n>:UART:PARity? query returns the current UART parity setting.

Return Format <parity><NL>

<parity> ::= {EVEN | ODD | NONE}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":SBUS<n>:UART:TRIGger:TYPE"](#) on page 1156

:SBUS<n>:UART:POLarity

N (see [page 1650](#))

Command Syntax :SBUS<n>:UART:POLarity <polarity>

<polarity> ::= {HIGH | LOW}

The :SBUS<n>:UART:POLarity command selects the polarity as idle low or idle high for the serial decoder and/or trigger when in UART mode.

Query Syntax :SBUS<n>:UART:POLarity?

The :SBUS<n>:UART:POLarity? query returns the current UART polarity setting.

Return Format <polarity><NL>

<polarity> ::= {HIGH | LOW}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":SBUS<n>:UART:TRIGger:TYPE"](#) on page 1156

:SBUS<n>:UART:SOURce:RX

N (see [page 1650](#))

Command Syntax :SBUS<n>:UART:SOURce:RX <source>

<source> ::= {CHANnel<n> | EXTernal} for the DSO models

<source> ::= {CHANnel<n> | DIGital<d>} for the MSO models

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :SBUS<n>:UART:SOURce:RX command controls which signal is used as the Rx source by the serial decoder and/or trigger when in UART mode.

Query Syntax :SBUS<n>:UART:SOURce:RX?

The :SBUS<n>:UART:SOURce:RX? query returns the current source for the UART Rx signal.

Return Format <source><NL>

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":SBUS<n>:UART:TRIGger:TYPE"](#) on page 1156
 - [":SBUS<n>:UART:BITOrder"](#) on page 1141

:SBUS<n>:UART:SOURce:TX

N (see [page 1650](#))

Command Syntax :SBUS<n>:UART:SOURce:TX <source>

<source> ::= {CHANnel<n> | EXternal} for the DSO models

<source> ::= {CHANnel<n> | DIGital<d>} for the MSO models

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :SBUS<n>:UART:SOURce:TX command controls which signal is used as the Tx source by the serial decoder and/or trigger when in UART mode.

Query Syntax :SBUS<n>:UART:SOURce:TX?

The :SBUS<n>:UART:SOURce:TX? query returns the current source for the UART Tx signal.

Return Format <source><NL>

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":SBUS<n>:UART:TRIGger:TYPE"](#) on page 1156
 - [":SBUS<n>:UART:BITOrder"](#) on page 1141

:SBUS<n>:UART:TRIGger:BASE

N (see [page 1650](#))

Command Syntax :SBUS<n>:UART:TRIGger:BASE <base>

<base> ::= {ASCii | HEX}

The :SBUS<n>:UART:TRIGger:BASE command sets the front panel UART/RS232 trigger setup data selection option:

- ASCii – front panel data selection is from ASCII values.
- HEX – front panel data selection is from hexadecimal values.

The :SBUS<n>:UART:TRIGger:BASE setting does not affect the :SBUS<n>:UART:TRIGger:DATA command which can always set data values using ASCII or hexadecimal values.

NOTE

The :SBUS<n>:UART:TRIGger:BASE command is independent of the :SBUS<n>:UART:BASE command which affects decode and Lister only.

Query Syntax :SBUS<n>:UART:TRIGger:BASE?

The :SBUS<n>:UART:TRIGger:BASE? query returns the current UART base setting.

Return Format <base><NL>

<base> ::= {ASC | HEX}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":SBUS<n>:UART:TRIGger:DATA"](#) on page 1153

:SBUS<n>:UART:TRIGger:BURSt

N (see [page 1650](#))

Command Syntax :SBUS<n>:UART:TRIGger:BURSt <value>

<value> ::= {OFF | 1 to 4096 in NR1 format}

The :SBUS<n>:UART:TRIGger:BURSt command selects the burst value (Nth frame after idle period) in the range 1 to 4096 or OFF, for the trigger when in UART mode.

Query Syntax :SBUS<n>:UART:TRIGger:BURSt?

The :SBUS<n>:UART:TRIGger:BURSt? query returns the current UART trigger burst value.

Return Format <value><NL>

<value> ::= {OFF | 1 to 4096 in NR1 format}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":SBUS<n>:UART:TRIGger:IDLE"](#) on page 1154
 - [":SBUS<n>:UART:TRIGger:TYPE"](#) on page 1156

:SBUS<n>:UART:TRIGger:DATA

N (see [page 1650](#))

Command Syntax :SBUS<n>:UART:TRIGger:DATA <value>

<value> ::= 8-bit integer from 0-255 (0x00-0xff) in decimal,
 <hexadecimal>, <binary>, or <quoted_string> format

<hexadecimal> ::= #Hnn where n ::= {0,...,9 | A,...,F} for hexadecimal

<binary> ::= #Bnn...n where n ::= {0 | 1} for binary

<quoted_string> ::= any of the 128 valid 7-bit ASCII characters
 (or standard abbreviations)

The :SBUS<n>:UART:TRIGger:DATA command selects the data byte value (0x00 to 0xFF) for the trigger QUALifier when in UART mode. The data value is used when one of the RD or TD trigger types is selected.

When entering an ASCII character via the quoted string, it must be one of the 128 valid characters (case-sensitive): "NUL", "SOH", "STX", "ETX", "EOT", "ENQ", "ACK", "BEL", "BS", "HT", "LF", "VT", "FF", "CR", "SO", "SI", "DLE", "DC1", "DC2", "DC3", "DC4", "NAK", "SYN", "ETB", "CAN", "EM", "SUB", "ESC", "FS", "GS", "RS", "US", "SP", "!", "\"", "#", "\$", "%", "&", "\", "(", ")", "*", "+", ",", "-", ".", ":", "/", "0", "1", "2", "3", "4", "5", "6", "7", "8", "9", ":", ";", "<", "=", ">", "?", "@", "A", "B", "C", "D", "E", "F", "G", "H", "I", "J", "K", "L", "M", "N", "O", "P", "Q", "R", "S", "T", "U", "V", "W", "X", "Y", "Z", "[", "\\", "]", "^", "_", "`", "a", "b", "c", "d", "e", "f", "g", "h", "i", "j", "k", "l", "m", "n", "o", "p", "q", "r", "s", "t", "u", "v", "w", "x", "y", "z", "{", "|", "}", "~", or "DEL".

Query Syntax :SBUS<n>:UART:TRIGger:DATA?

The :SBUS<n>:UART:TRIGger:DATA? query returns the current UART trigger data value.

Return Format <value><NL>

<value> ::= 8-bit integer in decimal from 0-255

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":SBUS<n>:UART:TRIGger:BASE"](#) on page 1151
 - [":SBUS<n>:UART:TRIGger:TYPE"](#) on page 1156

:SBUS<n>:UART:TRIGger:IDLE

N (see [page 1650](#))

Command Syntax :SBUS<n>:UART:TRIGger:IDLE <time_value>

<time_value> ::= time from 1 us to 10 s in NR3 format

The :SBUS<n>:UART:TRIGger:IDLE command selects the value of the idle period for burst trigger in the range from 1 us to 10 s when in UART mode.

Query Syntax :SBUS<n>:UART:TRIGger:IDLE?

The :SBUS<n>:UART:TRIGger:IDLE? query returns the current UART trigger idle period time.

Return Format <time_value><NL>

<time_value> ::= time from 1 us to 10 s in NR3 format

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":SBUS<n>:UART:TRIGger:BURSt"](#) on page 1152
 - [":SBUS<n>:UART:TRIGger:TYPE"](#) on page 1156

:SBUS<n>:UART:TRIGger:QUALifier

N (see [page 1650](#))

Command Syntax :SBUS<n>:UART:TRIGger:QUALifier <value>

<value> ::= {EQUAL | NOTequal | GREATERthan | LESSthan}

The :SBUS<n>:UART:TRIGger:QUALifier command selects the data qualifier when :TYPE is set to RDATA, RD1, RD0, RDX, TDATA, TD1, TD0, or TDX for the trigger when in UART mode.

Query Syntax :SBUS<n>:UART:TRIGger:QUALifier?

The :SBUS<n>:UART:TRIGger:QUALifier? query returns the current UART trigger qualifier.

Return Format <value><NL>

<value> ::= {EQU | NOT | GRE | LESS}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":SBUS<n>:UART:TRIGger:TYPE"](#) on page 1156

:SBUS<n>:UART:TRIGger:TYPE

N (see [page 1650](#))

Command Syntax :SBUS<n>:UART:TRIGger:TYPE <value>

```
<value> ::= {RSTArt | RSTOp | RDATA | RD1 | RD0 | RDX | PARityerror
             | TSTArt | TSTOp | TDATA | TD1 | TD0 | TDX}
```

The :SBUS<n>:UART:TRIGger:TYPE command selects the UART trigger type.

When one of the RD or TD types is selected, the :SBUS<n>:UART:TRIGger:DATA and :SBUS<n>:UART:TRIGger:QUALifier commands are used to specify the data value and comparison operator.

The RD1, RD0, RDX, TD1, TD0, and TDX types (for triggering on data and alert bit values) are only valid when a 9-bit width has been selected.

Query Syntax :SBUS<n>:UART:TRIGger:TYPE?

The :SBUS<n>:UART:TRIGger:TYPE? query returns the current UART trigger data value.

Return Format <value><NL>

```
<value> ::= {RSTA | RSTO | RDATA | RD1 | RD0 | RDX | PAR | TSTA |
             TSTO | TDATA | TD1 | TD0 | TDX}
```

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":SBUS<n>:UART:TRIGger:DATA"](#) on page 1153
 - [":SBUS<n>:UART:TRIGger:QUALifier"](#) on page 1155
 - [":SBUS<n>:UART:WIDTH"](#) on page 1157

:SBUS<n>:UART:WIDTh

N (see [page 1650](#))

Command Syntax :SBUS<n>:UART:WIDTh <width>
 <width> ::= { 5 | 6 | 7 | 8 | 9 }

The :SBUS<n>:UART:WIDTh command determines the number of bits (5-9) for each message "byte" for the serial decoder and/or trigger when in UART mode.

Query Syntax :SBUS<n>:UART:WIDTh?

The :SBUS<n>:UART:WIDTh? query returns the current UART width setting.

Return Format <width><NL>
 <width> ::= { 5 | 6 | 7 | 8 | 9 }

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":SBUS<n>:UART:TRIGger:TYPE"](#) on page 1156

:SBUS<n>:USB Commands

NOTE

These commands are only valid when a USB 2.0 triggering and serial decode option has been licensed.

Table 134 :SBUS<n>:USB Commands Summary

Command	Query	Options and Query Returns
:SBUS<n>:USB:BASE <base> (see page 1161)	:SBUS<n>:USB:BASE? (see page 1161)	<base> ::= {ASCIi BINary DECimal HEX}
:SBUS<n>:USB:SOURce:D MINus <source> (see page 1162)	:SBUS<n>:USB:SOURce:D MINus? (see page 1162)	<source> ::= {CHANnel<n>} for DSO models <source> ::= {CHANnel<n> DIGital<d>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:SBUS<n>:USB:SOURce:D PLus <source> (see page 1163)	:SBUS<n>:USB:SOURce:D PLus? (see page 1163)	<source> ::= {CHANnel<n>} for DSO models <source> ::= {CHANnel<n> DIGital<d>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:SBUS<n>:USB:SOURce:D IFFerential <source> (see page 1164)	:SBUS<n>:USB:SOURce:D IFFerential? (see page 1164)	<source> ::= {CHANnel<n>} <n> ::= 1 to (# analog channels) in NR1 format
:SBUS<n>:USB:SPEEd <speed> (see page 1165)	:SBUS<n>:USB:SPEEd? (see page 1165)	<speed> ::= {LOW FULL HIGH}
:SBUS<n>:USB:TRIGger <condition> (see page 1166)	:SBUS<n>:USB:TRIGger? (see page 1166)	<condition> ::= {SOP EOP ENTersuspend EXITsuspend RESet TOKEN DATA HANDshake SPECial ALLerrors PIDerror CRC5error CRC16error GLITCherror STUFFerror SE1error}

Table 134 :SBUS<n>:USB Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:USB:TRIGger:ADDRESS <string> (see page 1167)	:SBUS<n>:USB:TRIGger:ADDRESS? (see page 1167)	<string> ::= "nnnnnnnn" where n ::= {0 1 X} <string> ::= "0xnn" where n ::= {0,...,9 A,...,F X \$}
:SBUS<n>:USB:TRIGger:CRC <string> (see page 1168)	:SBUS<n>:USB:TRIGger:CRC? (see page 1168)	<string> ::= "nnnnnn" where n ::= {0 1 X} <string> ::= "0xnn" where n ::= {0,...,9 A,...,F X \$}
:SBUS<n>:USB:TRIGger:DATA <string> (see page 1169)	:SBUS<n>:USB:TRIGger:DATA? (see page 1169)	<string> ::= "nnnn..." where n ::= {0 1 X} <string> ::= "0xn..." where n ::= {0,...,9 A,...,F X \$}
:SBUS<n>:USB:TRIGger:DATA:LENGth <value> (see page 1170)	:SBUS<n>:USB:TRIGger:DATA:LENGth? (see page 1170)	<length> ::= data length between 1-20
:SBUS<n>:USB:TRIGger:ENDPoint <string> (see page 1171)	:SBUS<n>:USB:TRIGger:ENDPoint? (see page 1171)	<string> ::= "nnnn" where n ::= {0 1 X} <string> ::= "0xn" where n ::= {0,...,9 A,...,F X \$}
:SBUS<n>:USB:TRIGger:ET <string> (see page 1172)	:SBUS<n>:USB:TRIGger:ET? (see page 1172)	<string> ::= "nn" where n ::= {0 1 X} <string> ::= "0xn" where n ::= {0 1 2 3 X \$}
:SBUS<n>:USB:TRIGger:FRaME <string> (see page 1173)	:SBUS<n>:USB:TRIGger:FRaME? (see page 1173)	<string> ::= "nnnnnnnnnnnn" where n ::= {0 1 X} <string> ::= "0xnnnn" where n ::= {0,...,9 A,...,F X \$}
:SBUS<n>:USB:TRIGger:HADDRESS <string> (see page 1174)	:SBUS<n>:USB:TRIGger:HADDRESS? (see page 1174)	<string> ::= "nnnnnnnn" where n ::= {0 1 X} <string> ::= "0xnn" where n ::= {0,...,9 A,...,F X \$}
:SBUS<n>:USB:TRIGger:PCHeck <string> (see page 1175)	:SBUS<n>:USB:TRIGger:PCHeck? (see page 1175)	<string> ::= "nnnn" where n ::= {0 1 X} <string> ::= "0xn" where n ::= {0,...,9 A,...,F X \$}
:SBUS<n>:USB:TRIGger:PID:DATA <pid> (see page 1176)	:SBUS<n>:USB:TRIGger:PID:DATA? (see page 1176)	<pid> ::= {DATA0 DATA1 DATA2 MDATa}

Table 134 :SBUS<n>:USB Commands Summary (continued)

Command	Query	Options and Query Returns
:SBUS<n>:USB:TRIGger:PID:HANDshake <pid> (see page 1177)	:SBUS<n>:USB:TRIGger:PID:HANDshake? (see page 1177)	<pid> ::= {ACK NAK STALL NYET}
:SBUS<n>:USB:TRIGger:PID:SPECIAL <pid> (see page 1178)	:SBUS<n>:USB:TRIGger:PID:SPECIAL? (see page 1178)	<pid> ::= {PING PRE ERR SPLit}
:SBUS<n>:USB:TRIGger:PID:TOKEN <pid> (see page 1179)	:SBUS<n>:USB:TRIGger:PID:TOKEN? (see page 1179)	<pid> ::= {OUT IN SETUp SOF}
:SBUS<n>:USB:TRIGger:PORT <string> (see page 1180)	:SBUS<n>:USB:TRIGger:PORT? (see page 1180)	<string> ::= "nnnnnnnn" where n ::= {0 1 X} <string> ::= "0xnn" where n ::= {0,...,9 A,...,F X \$}
:SBUS<n>:USB:TRIGger:SC <string> (see page 1181)	:SBUS<n>:USB:TRIGger:SC? (see page 1181)	<string> ::= "n" where n ::= {0 1 X} <string> ::= "0xn" where n ::= {0 1 X \$}
:SBUS<n>:USB:TRIGger:SEU <string> (see page 1182)	:SBUS<n>:USB:TRIGger:SEU? (see page 1182)	<string> ::= "nn" where n ::= {0 1 X} <string> ::= "0xn" where n ::= {0 1 2 3 X \$}

:SBUS<n>:USB:BASE

N (see [page 1650](#))

Command Syntax :SBUS<n>:USB:BASE <base>

<base> ::= {ASCIi | BINary | DECimal | HEX}

The :SBUS<n>:USB:BASE command determines the base to use for the USB decode and Lister display, controlling how the data or payload field is displayed.

All other fields are displayed in hex

Query Syntax :SBUS<n>:USB:BASE?

The :SBUS<n>:USB:BASE? query returns the currently specified base.

Return Format <base><NL>

<base> ::= {ASC | BIN | DEC | HEX}

:SBUS<n>:USB:SOURce:DMINus

N (see [page 1650](#))

Command Syntax :SBUS<n>:USB:SOURce:DMINus <source>

<source> ::= {CHANnel<n>} for DSO models

<source> ::= {CHANnel<n> | DIGital<d>} for MSO models

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :SBUS<n>:USB:SOURce:DMINus command specifies which signal is used as the USB D- source for the selected bus.

The D- source is only used for Low and Full Speed USB.

Query Syntax :SBUS<n>:USB:SOURce:DMINus?

The :SBUS<n>:USB:SOURce:DMINus? query returns the specified D- source.

Return Format <source><NL>

<source> ::= {CHAN<n> | DIG<d>}

- See Also**
- [":SBUS<n>:USB:SOURce:DPLus"](#) on page 1163
 - [":SBUS<n>:USB:SPEed"](#) on page 1165

:SBUS<n>:USB:SOURce:DPLus

N (see [page 1650](#))

Command Syntax :SBUS<n>:USB:SOURce:DPLus <source>

<source> ::= {CHANnel<n>} for DSO models

<source> ::= {CHANnel<n> | DIGital<d>} for MSO models

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :SBUS<n>:USB:SOURce:DPLus command specifies which signal is used as the USB D+ source for the selected bus.

The D+ source is only used for Low and Full Speed USB.

Query Syntax :SBUS<n>:USB:SOURce:DPLus?

The :SBUS<n>:USB:SOURce:DPLus? query returns the specified D+ source.

Return Format <source><NL>

<source> ::= {CHAN<n> | DIG<d>}

- See Also**
- [":SBUS<n>:USB:SOURce:DMINus"](#) on page 1162
 - [":SBUS<n>:USB:SPEed"](#) on page 1165

:SBUS<n>:USB:SOURce:DIFFerential

N (see [page 1650](#))

Command Syntax :SBUS<n>:USB:SOURce:DIFFerential <source>

<source> ::= {CHANnel<n>}

<n> ::= 1 to (# analog channels) in NR1 format

The :SBUS<n>:USB:SOURce:DIFFerential command specifies which signal is used as the differential source for the selected bus.

Differential sources are only used for High Speed USB.

Query Syntax :SBUS<n>:USB:SOURce:DIFFerential?

The :SBUS<n>:USB:SOURce:DIFFerential? query returns the specified differential source.

Return Format <source><NL>

<source> ::= {CHAN<n>}

See Also • [":SBUS<n>:USB:SPEed"](#) on page 1165

:SBUS<n>:USB:SPEEd

N (see [page 1650](#))

Command Syntax :SBUS<n>:USB:SPEEd <speed>

<speed> ::= {LOW | FULL | HIGH}

The :SBUS<n>:USB:SPEEd command specifies the speed of the USB interface for the selected bus.

Query Syntax :SBUS<n>:USB:SPEEd?

The :SBUS<n>:USB:SPEEd? query returns the speed setting.

Return Format <speed><NL>

<speed> ::= {LOW | FULL | HIGH}

- See Also**
- [":SBUS<n>:USB:SOURce:DPLus"](#) on page 1163
 - [":SBUS<n>:USB:SOURce:DMINus"](#) on page 1162
 - [":SBUS<n>:USB:SOURce:DIFFerential"](#) on page 1164

:SBUS<n>:USB:TRIGger

N (see [page 1650](#))

Command Syntax :SBUS<n>:USB:TRIGger <condition>

```
<condition> ::= {SOP | EOP | ENTersuspend | EXITsuspend | RESet
                | TOKen | DATA | HANDshake | SPECial | ALLerrors
                | PIDerror | CRC5error | CRC16error | GLITCherror
                | STUFFerror | SE1error}
```

The :SBUS<n>:USB:TRIGger command specifies the USB trigger mode for the selected bus.

Query Syntax :SBUS<n>:USB:TRIGger?

The :SBUS<n>:USB:TRIGger? query returns the specified USB trigger mode.

Return Format <condition><NL>

```
<condition> ::= {SOP | EOP | ENT | EXIT | RES | TOK | DATA | HAND
                | SPEC | ALL | PID | CRC5 | CRC16 | GLIT | STUFF
                | SE1}
```

- See Also**
- [":SBUS<n>:USB:TRIGger:PID:DATA"](#) on page 1176
 - [":SBUS<n>:USB:TRIGger:PID:HANDshake"](#) on page 1177
 - [":SBUS<n>:USB:TRIGger:PID:SPECial"](#) on page 1178
 - [":SBUS<n>:USB:TRIGger:PID:TOKen"](#) on page 1179

:SBUS<n>:USB:TRIGger:ADDRess

N (see [page 1650](#))

Command Syntax :SBUS<n>:USB:TRIGger:ADDRess <string>

<string> ::= "nnnnnnnn" where n ::= {0 | 1 | X}

<string> ::= "0xnn" where n ::= {0,...,9 | A,...,F | X | \$}

The :SBUS<n>:USB:TRIGger:ADDRess command specifies the 7-bit Address portion of the trigger for the selected bus, in binary or hex.

Query Syntax :SBUS<n>:USB:TRIGger:ADDRess?

The :SBUS<n>:USB:TRIGger:ADDRess? query returns the specified Address portion of the trigger.

Return Format <string><NL>

<string> ::= "nnnnnnnn" where n ::= {0 | 1 | X}

See Also

- [":SBUS<n>:USB:TRIGger:PID:SPECial"](#) on page 1178
- [":SBUS<n>:USB:TRIGger:PID:TOKen"](#) on page 1179

:SBUS<n>:USB:TRIGger:CRC

N (see [page 1650](#))

Command Syntax :SBUS<n>:USB:TRIGger:CRC <string>

<string> ::= "nnnnn" where n ::= {0 | 1 | X}

<string> ::= "0xnn" where n ::= {0,...,9 | A,...,F | X | \$}

The :SBUS<n>:USB:TRIGger:CRC command specifies the CRC portion of the trigger for the selected bus, in binary or hex.

Query Syntax :SBUS<n>:USB:TRIGger:CRC?

The :SBUS<n>:USB:TRIGger:CRC? query returns the specified CRC portion of the trigger.

Return Format <string><NL>

<string> ::= "nnnnn" where n ::= {0 | 1 | X}

- See Also**
- [":SBUS<n>:USB:TRIGger:PID:SPECial"](#) on page 1178
 - [":SBUS<n>:USB:TRIGger:PID:TOKEN"](#) on page 1179

:SBUS<n>:USB:TRIGger:DATA

N (see [page 1650](#))

Command Syntax :SBUS<n>:USB:TRIGger:DATA <string>

<string> ::= "nnnn..." where n ::= {0 | 1 | X}

<string> ::= "0xn..." where n ::= {0,...,9 | A,...,F | X | \$}

The :SBUS<n>:USB:TRIGger:DATA command specifies the Data portion of the trigger for the selected bus, in binary or hex.

See the :SBUS<n>:USB:TRIGger:DATA:LENGth command for setting the length of the data value.

Query Syntax :SBUS<n>:USB:TRIGger:DATA?

The :SBUS<n>:USB:TRIGger:DATA? query returns the specified Data portion of the trigger.

Return Format <string><NL>

<string> ::= "nnnn..." where n ::= {0 | 1 | X}

See Also • [":SBUS<n>:USB:TRIGger:DATA:LENGth"](#) on page 1170

:SBUS<n>:USB:TRIGger:DATA:LENGth

N (see [page 1650](#))

Command Syntax :SBUS<n>:USB:TRIGger:DATA:LENGth <length>

<length> ::= data length between 1-20

The :SBUS<n>:USB:TRIGger:DATA:LENGth command specifies the data length in bytes.

Query Syntax :SBUS<n>:USB:TRIGger:DATA:LENGth?

The :SBUS<n>:USB:TRIGger:DATA:LENGth? query returns the specified data length.

Return Format <length><NL>

<length> ::= data length between 1-20

See Also • [":SBUS<n>:USB:TRIGger:DATA"](#) on page 1169

:SBUS<n>:USB:TRIGger:ENDPoint

N (see [page 1650](#))

Command Syntax :SBUS<n>:USB:TRIGger:ENDPoint <string>

<string> ::= "nnnn" where n ::= {0 | 1 | X}

<string> ::= "0xn" where n ::= {0,...,9 | A,...,F | X | \$}

The :SBUS<n>:USB:TRIGger:ENDPoint command specifies the 4-bit Endpoint portion of the trigger for the selected bus, in binary or hex.

Query Syntax :SBUS<n>:USB:TRIGger:ENDPoint?

The :SBUS<n>:USB:TRIGger:ENDPoint? query returns the specified Endpoint portion of the trigger.

Return Format <string><NL>

<string> ::= "nnnn" where n ::= {0 | 1 | X}

- See Also**
- [":SBUS<n>:USB:TRIGger:PID:SPECial"](#) on page 1178
 - [":SBUS<n>:USB:TRIGger:PID:TOKen"](#) on page 1179

:SBUS<n>:USB:TRIGger:ET

N (see [page 1650](#))

Command Syntax :SBUS<n>:USB:TRIGger:ET <string>

<string> ::= "nn" where n ::= {0 | 1 | X}

<string> ::= "0xn" where n ::= {0 | 1 | 2 | 3 | X | \$}

The :SBUS<n>:USB:TRIGger:ET command specifies the 2-bit ET portion of the trigger for the selected bus, in binary or hex.

Query Syntax :SBUS<n>:USB:TRIGger:ET?

The :SBUS<n>:USB:TRIGger:ET? query returns the specified ET portion of the trigger.

Return Format <string><NL>

<string> ::= "nn" where n ::= {0 | 1 | X}

See Also • [":SBUS<n>:USB:TRIGger:PID:SPECial"](#) on page 1178

:SBUS<n>:USB:TRIGger:FRAME

N (see [page 1650](#))

Command Syntax :SBUS<n>:USB:TRIGger:FRAME <string>

<string> ::= "nnnnnnnnnnnn" where n ::= {0 | 1 | X}

<string> ::= "0xnnn" where n ::= {0,...,9 | A,...,F | X | \$}

The :SBUS<n>:USB:TRIGger:FRAME command specifies the 11-bit Frame portion of the trigger for the selected bus, in binary or hex.

Query Syntax :SBUS<n>:USB:TRIGger:FRAME?

The :SBUS<n>:USB:TRIGger:FRAME? query returns the specified Frame portion of the trigger.

Return Format <string><NL>

<string> ::= "nnnnnnnnnnnn" where n ::= {0 | 1 | X}

See Also • [":SBUS<n>:USB:TRIGger:PID:TOKen"](#) on page 1179

:SBUS<n>:USB:TRIGger:HADDress

N (see [page 1650](#))

Command Syntax :SBUS<n>:USB:TRIGger:HADDress <string>

<string> ::= "nnnnnnnn" where n ::= {0 | 1 | X}

<string> ::= "0xnn" where n ::= {0,...,9 | A,...,F | X | \$}

The :SBUS<n>:USB:TRIGger:HADDress command specifies the 7-bit Hub Address portion of the trigger for the selected bus, in binary or hex.

Query Syntax :SBUS<n>:USB:TRIGger:HADDress?

The :SBUS<n>:USB:TRIGger:HADDress? query returns the specified Hub Address portion of the trigger.

Return Format <string><NL>

<string> ::= "nnnnnnnn" where n ::= {0 | 1 | X}

See Also • [":SBUS<n>:USB:TRIGger:PID:SPECial"](#) on page 1178

:SBUS<n>:USB:TRIGger:PCHeck

N (see [page 1650](#))

Command Syntax :SBUS<n>:USB:TRIGger:PCHeck <string>

<string> ::= "nnnn" where n ::= {0 | 1 | x}

<string> ::= "0xn" where n ::= {0,...,9 | A,...,F | x | \$}

The :SBUS<n>:USB:TRIGger:PCHeck command specifies the 4-bit PID check portion of the trigger for the selected bus, in binary or hex.

See the :SBUS<n>:USB:TRIGger:PID commands for setting the PID.

Query Syntax :SBUS<n>:USB:TRIGger:PCHeck?

The :SBUS<n>:USB:TRIGger:PCHeck? query returns the specified PID check portion of the trigger.

Return Format <string><NL>

<string> ::= "nnnn" where n ::= {0 | 1 | x}

- See Also**
- [":SBUS<n>:USB:TRIGger:PID:DATA"](#) on page 1176
 - [":SBUS<n>:USB:TRIGger:PID:HANDshake"](#) on page 1177
 - [":SBUS<n>:USB:TRIGger:PID:SPECial"](#) on page 1178
 - [":SBUS<n>:USB:TRIGger:PID:TOKen"](#) on page 1179

:SBUS<n>:USB:TRIGger:PID:DATA

N (see [page 1650](#))

Command Syntax :SBUS<n>:USB:TRIGger:PID:DATA <pid>

<pid> ::= {DATA0 | DATA1 | DATA2 | MDATA}

The :SBUS<n>:USB:TRIGger:PID:DATA command specifies the USB data PID to trigger on for the selected bus.

The specified PID does not include the PID check value, which is specified using the :SBUS<n>:USB:TRIGger:PIDCheck command.

The DATA USB trigger mode can be selected using the :SBUS<n>:USB:TRIGger command.

Query Syntax :SBUS<n>:USB:TRIGger:PID:DATA?

The :SBUS<n>:USB:TRIGger:PID:DATA? query returns the specified data PID.

Return Format <pid><NL>

<pid> ::= {DATA0 | DATA1 | DATA2 | MDATA}

- See Also**
- [":SBUS<n>:USB:TRIGger:PIDCheck"](#) on page 1175
 - [":SBUS<n>:USB:TRIGger"](#) on page 1166

:SBUS<n>:USB:TRIGger:PID:HANDshake

N (see [page 1650](#))

Command Syntax :SBUS<n>:USB:TRIGger:PID:HANDshake <pid>

<pid> ::= {ACK | NAK | STAL1 | NYET}

The :SBUS<n>:USB:TRIGger:PID:HANDshake command specifies the USB handshake PID to trigger on for the selected bus.

The specified PID does not include the PID check value, which is specified using the :SBUS<n>:USB:TRIGger:PIDCheck command.

The HANDshake USB trigger mode can be selected using the :SBUS<n>:USB:TRIGger command.

Query Syntax :SBUS<n>:USB:TRIGger:PID:HANDshake?

The :SBUS<n>:USB:TRIGger:PID:HANDshake? query returns the specified handshake PID.

Return Format <pid><NL>

<pid> ::= {ACK | NAK | STAL | NYET}

- See Also**
- [":SBUS<n>:USB:TRIGger:PCHeck"](#) on page 1175
 - [":SBUS<n>:USB:TRIGger"](#) on page 1166

:SBUS<n>:USB:TRIGger:PID:SPEcial

N (see [page 1650](#))

Command Syntax :SBUS<n>:USB:TRIGger:PID:SPEcial <pid>

<pid> ::= {PING | PRE | ERR | SPLit}

The :SBUS<n>:USB:TRIGger:PID:SPEcial command specifies the USB special PID to trigger on for the selected bus.

The specified PID does not include the PID check value, which is specified using the :SBUS<n>:USB:TRIGger:PIDCheck command.

The SPEcial USB trigger mode can be selected using the :SBUS<n>:USB:TRIGger command.

Query Syntax :SBUS<n>:USB:TRIGger:PID:SPEcial?

The :SBUS<n>:USB:TRIGger:PID:SPEcial? query returns the specified special PID.

Return Format <pid><NL>

<pid> ::= {PING | PRE | ERR | SPL}

- See Also**
- [":SBUS<n>:USB:TRIGger:PCheck"](#) on page 1175
 - [":SBUS<n>:USB:TRIGger"](#) on page 1166

:SBUS<n>:USB:TRIGger:PID:TOKen

N (see [page 1650](#))

Command Syntax :SBUS<n>:USB:TRIGger:PID:TOKen <pid>

<pid> ::= {OUT | IN | SETup | SOF}

The :SBUS<n>:USB:TRIGger:PID:TOKen command specifies the USB token PID to trigger on for the selected bus.

The specified PID does not include the PID check value, which is specified using the :SBUS<n>:USB:TRIGger:PIDCheck command.

The TOKen USB trigger mode can be selected using the :SBUS<n>:USB:TRIGger command.

Query Syntax :SBUS<n>:USB:TRIGger:PID:TOKen?

The :SBUS<n>:USB:TRIGger:PID:TOKen? query returns the specified token PID.

Return Format <pid><NL>

<pid> ::= {OUT | IN | SETup | SOF}

- See Also**
- [":SBUS<n>:USB:TRIGger:PCHeck"](#) on page 1175
 - [":SBUS<n>:USB:TRIGger"](#) on page 1166

:SBUS<n>:USB:TRIGger:PORT

N (see [page 1650](#))

Command Syntax :SBUS<n>:USB:TRIGger:PORT <string>

<string> ::= "nnnnnnnn" where n ::= {0 | 1 | X}

<string> ::= "0xnn" where n ::= {0,...,9 | A,...,F | X | \$}

The :SBUS<n>:USB:TRIGger:PORT command specifies the 7-bit Port portion of the trigger for the selected bus, in binary or hex.

Query Syntax :SBUS<n>:USB:TRIGger:PORT?

The :SBUS<n>:USB:TRIGger:PORT? query returns the specified Port portion of the trigger.

Return Format <string><NL>

<string> ::= "nnnnnnnn" where n ::= {0 | 1 | X}

See Also • [":SBUS<n>:USB:TRIGger:PID:SPECial"](#) on page 1178

:SBUS<n>:USB:TRIGger:SC

N (see [page 1650](#))

Command SyntaxS :SBUS<n>:USB:TRIGger:SC <string>

<string> ::= "n" where n ::= {0 | 1 | X}

<string> ::= "0xn" where n ::= {0 | 1 | X | \$}

The :SBUS<n>:USB:TRIGger:SC command specifies the 1-bit SC portion of the trigger for the selected bus, in binary or hex.

Query Syntax :SBUS<n>:USB:TRIGger:SC?

The :SBUS<n>:USB:TRIGger:SC? query returns the specified SC portion of the trigger.

Return Format <string><NL>

<string> ::= "nnnn" where n ::= {0 | 1 | X}

See Also • [":SBUS<n>:USB:TRIGger:PID:SPECial"](#) on page 1178

:SBUS<n>:USB:TRIGger:SEU

N (see [page 1650](#))

Command Syntax :SBUS<n>:USB:TRIGger:SEU <string>

<string> ::= "nn" where n ::= {0 | 1 | X}

<string> ::= "0xn" where n ::= {0 | 1 | 2 | 3 | X | \$}

The :SBUS<n>:USB:TRIGger:SEU command specifies the 2-bit S and E or U portion of the trigger for the selected bus, in binary or hex.

Query Syntax :SBUS<n>:USB:TRIGger:SEU?

The :SBUS<n>:USB:TRIGger:SEU? query returns the specified S and E or U portion of the trigger.

Return Format <string><NL>

<string> ::= "nn" where n ::= {0 | 1 | X}

See Also • [":SBUS<n>:USB:TRIGger:PID:SPECial"](#) on page 1178

:SBUS<n>:USBPd Commands

NOTE

These commands are valid when the USB PD (Power Delivery) serial decode and triggering option has been licensed.

Table 135 :SBUS<n>:USBPd Commands Summary

Command	Query	Options and Query Returns
:SBUS<n>:USBPd:SOURce <source> (see page 1184)	:SBUS<n>:USBPd:SOURce ? (see page 1184)	<source> ::= {CHANnel<n>} <n> ::= 1 to (# analog channels) in NR1 format
:SBUS<n>:USBPd:TRIGger <mode> (see page 1185)	:SBUS<n>:USBPd:TRIGger ? (see page 1185)	<mode> ::= {PSTart EOP SOP SPRime SDPRime SPDebug SDPDebug HRST CRST CRCerror PERRor HEADer}
:SBUS<n>:USBPd:TRIGger:HEADer <type> (see page 1186)	:SBUS<n>:USBPd:TRIGger:HEADer? (see page 1186)	<type> ::= {CMESsage DMESsage EMESsage VALue}
:SBUS<n>:USBPd:TRIGger:HEADer:CMESsage <type> (see page 1188)	:SBUS<n>:USBPd:TRIGger:HEADer:CMESsage? (see page 1188)	<type> ::= {GOODcrc GOTOmin ACCEpt REJect PING PSRDy GSRCap GSNCap DRSwap PRSWap VCSwap WAIT SRST GSCX GSTatus FRSwap GPSTatus GCCodes}
:SBUS<n>:USBPd:TRIGger:HEADer:DMESsage <type> (see page 1190)	:SBUS<n>:USBPd:TRIGger:HEADer:DMESsage? (see page 1190)	<type> ::= {SRCap REQuest BIST SNCap BSTatus ALERt GCINfo VDEfined}
:SBUS<n>:USBPd:TRIGger:HEADer:EMESsage <type> (see page 1191)	:SBUS<n>:USBPd:TRIGger:HEADer:EMESsage? (see page 1191)	<type> ::= {SCX STATus GBCap GBSTatus BCAP GMINfo MINfo SREQuest SRESponse FREQuest FRESponse PSTatus CINfo CCODEs}
:SBUS<n>:USBPd:TRIGger:HEADer:VALue <string> (see page 1193)	:SBUS<n>:USBPd:TRIGger:HEADer:VALue? (see page 1193)	<string> ::= "nn...n" where n ::= {0 1 X} <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X}
:SBUS<n>:USBPd:TRIGger:HEADer:QUALifier <type> (see page 1194)	:SBUS<n>:USBPd:TRIGger:HEADer:QUALifier? (see page 1194)	<type> ::= {NONE SOP SPRime SDPRime}

:SBUS<n>:USBPd:SOURce

N (see [page 1650](#))

Command Syntax :SBUS<n>:USBPd:SOURce <source>

<source> ::= {CHANnel<n>}

<n> ::= 1 to (# analog channels) in NR1 format

The :SBUS<n>:USBPd:SOURce command selects the USB PD waveform source. You can use analog channels.

Query Syntax :SBUS<n>:USBPd:SOURce?

The :SBUS<n>:USBPd:SOURce? query returns the selected analog input channel.

Return Format <source><NL>

<source> ::= {CHAN<n>}

- See Also**
- [":SBUS<n>:USBPd:TRIGger"](#) on page 1185
 - [":SBUS<n>:USBPd:TRIGger:HEADer"](#) on page 1186
 - [":SBUS<n>:USBPd:TRIGger:HEADer:CMESsage"](#) on page 1188
 - [":SBUS<n>:USBPd:TRIGger:HEADer:DMESsage"](#) on page 1190
 - [":SBUS<n>:USBPd:TRIGger:HEADer:EMESsage"](#) on page 1191
 - [":SBUS<n>:USBPd:TRIGger:HEADer:VALue"](#) on page 1193
 - [":SBUS<n>:USBPd:TRIGger:HEADer:QUALifier"](#) on page 1194

:SBUS<n>:USBPd:TRIGger

N (see [page 1650](#))

Command Syntax :SBUS<n>:USBPd:TRIGger <mode>

```
<mode> ::= {PSTart | EOP | SOP | SPRime | SDPRime | SPDebug | SDPDebug
            | HRST | CRST | CRCerror | PERRor | HEADer}
```

The :SBUS<n>:USBPd:TRIGger command selects the USB PD trigger mode:

- PSTart – **Preamble Start**, triggers at the start of a preamble, which starts with 0.
- EOP – **EOP**, triggers at the end of packet.
- SOP – **SOP**, triggers on ordered set Sync-1, Sync-1, Sync-1, Sync-2.
- SPRime – **SOP'**, triggers on ordered set Sync-1, Sync-1, Sync-3, Sync-3.
- SDPRime – **SOP"**, triggers on ordered set Sync-1, Sync-3, Sync-1, Sync-3.
- SPDebug – **SOP' Debug**, triggers on ordered set Sync-1, RST-2, RST-2, Sync-3.
- SDPDebug – **SOP" Debug**, triggers on ordered set Sync-1, RST-2, Sync-3, Sync-2.
- HRST – **Hard Reset**, triggers on ordered set RST-1, RST-1, RST-1, RST-2.
- CRST – **Cable Reset**, triggers on ordered set RST-1, Sync-1, RST-1, Sync-3.
- CRCerror – **CRC Error**, triggers when an error is detected on a 32-bit CRC.
- PERRor – **Preamble Error**, triggers when an error is detected on a 64-bit sequence of alternating 0 and 1.
- HEADer – **Header Content**, triggers on a user-defined 16-bit value.

In this mode, use the :SBUS<n>:USBPd:TRIGger:HEADer command to select the header type.

Query Syntax :SBUS<n>:USBPd:TRIGger?

The :SBUS<n>:USBPd:TRIGger? query returns the selected USB PD trigger mode.

Return Format <mode><NL>

```
<mode> ::= {PST | EOP | SOP | SPR | SDPR | SPD | SDPD | HRST | CRST
            | CRC | PERR | HEAD}
```

- See Also**
- [":SBUS<n>:USBPd:SOURce"](#) on page 1184
 - [":SBUS<n>:USBPd:TRIGger:HEADer"](#) on page 1186
 - [":SBUS<n>:USBPd:TRIGger:HEADer:CMESsage"](#) on page 1188
 - [":SBUS<n>:USBPd:TRIGger:HEADer:DMESsage"](#) on page 1190
 - [":SBUS<n>:USBPd:TRIGger:HEADer:EMESsage"](#) on page 1191
 - [":SBUS<n>:USBPd:TRIGger:HEADer:VALue"](#) on page 1193
 - [":SBUS<n>:USBPd:TRIGger:HEADer:QUALifier"](#) on page 1194

:SBUS<n>:USBPd:TRIGger:HEADer

N (see [page 1650](#))

Command Syntax :SBUS<n>:USBPd:TRIGger:HEADer <type>

<type> ::= {CMESsage | DMESsage | EMESsage | VALue}

When the **Header Content** trigger mode is selected (with the ":SBUS<n>:USBPd:TRIGger:HEADer" command), the :SBUS<n>:USBPd:TRIGger:HEADer command selects the header type:

- CMESsage – **Control Message**, triggers on control message types (0 data object).

When the CMESsage header type is selected, use the :SBUS<n>:USBPd:TRIGger:HEADer:CMESsage command to select the control message type.

- DMESsage – **Data Message**, triggers on data message types (1 or more data objects).

When the DMESsage header type is selected, use the :SBUS<n>:USBPd:TRIGger:HEADer:DMESsage command to select the data message type.

- EMESsage – **Extended Message**, triggers on extended message types (bit 15 is set).

When the EMESsage header type is selected, use the :SBUS<n>:USBPd:TRIGger:HEADer:EMESsage command to select the extended message type.

- VALue – **Value**, triggers on a user-defined header value.

When the CMESsage header type is selected, use the :SBUS<n>:USBPd:TRIGger:HEADer:VALue command to specify the user-defined header value.

Query Syntax :SBUS<n>:USBPd:TRIGger:HEADer?

The :SBUS<n>:USBPd:TRIGger:HEADer? query returns the selected header type.

Return Format <type><NL>

<type> ::= {CMES | DMES | EMES | VAL}

- See Also**
- [":SBUS<n>:USBPd:SOURce"](#) on page 1184
 - [":SBUS<n>:USBPd:TRIGger"](#) on page 1185
 - [":SBUS<n>:USBPd:TRIGger:HEADer:CMESsage"](#) on page 1188
 - [":SBUS<n>:USBPd:TRIGger:HEADer:DMESsage"](#) on page 1190
 - [":SBUS<n>:USBPd:TRIGger:HEADer:EMESsage"](#) on page 1191
 - [":SBUS<n>:USBPd:TRIGger:HEADer:VALue"](#) on page 1193

- **":SBUS<n>:USBPd:TRIGger:HEADer:QUALifier"** on page 1194

:SBUS<n>:USBPd:TRIGger:HEADer:CMESsage

N (see [page 1650](#))

Command Syntax :SBUS<n>:USBPd:TRIGger:HEADer:CMESsage <type>

```
<type> ::= {GOODcrc | GOTOmin | ACcept | REJect | PING | PSRDy
            | GSRCap | GSNCap | DRSwap | PRSwap | VCSwap | WAIT | SRST | GSCX
            | GSTatus | FRSwap | GPSTatus | GCCodes}
```

When the **Header Content** trigger mode is selected (with the ":SBUS<n>:USBPd:TRIGger:HEADer" command) and the **Control Message** header type is selected (with the ":SBUS<n>:USBPd:TRIGger:HEADer:CMESsage" command), the :SBUS<n>:USBPd:TRIGger:HEADer:CMESsage command selects the control message type:

- GOODcrc – GoodCRC
- GOTOmin – GotoMin
- ACcept – Accept
- REJect – Reject
- PING – Ping
- PSRDy – PS_RDY
- GSRCap – Get_Source_Cap
- GSNCap – Get_Sink_Cap
- DRSwap – DR_Swap
- PRSwap – PR_Swap
- VCSwap – VCONN_Swap
- WAIT – Wait
- SRST – Soft_Reset
- GSCX – Get_Source_Cap_Extended
- GSTatus – Get_Status
- FRSwap – FR_Swap
- GPSTatus – Get_PPS_Status
- GCCodes – Get_Country_Codes

Query Syntax :SBUS<n>:USBPd:TRIGger:HEADer:CMESsage?

The :SBUS<n>:USBPd:TRIGger:HEADer:CMESsage? query returns the selected control message type.

Return Format <type><NL>

```
<type> ::= {GOOD | GOTO | ACC | REJ | PING | PSRD | GSRC | GSNC | DRSW
            | PRSW | VCSW | WAIT | SRST | GSCX | GST | FRSW | GPST | GCC}
```


- See Also
- [":SBUS<n>:USBPd:SOURce"](#) on page 1184
 - [":SBUS<n>:USBPd:TRIGger"](#) on page 1185
 - [":SBUS<n>:USBPd:TRIGger:HEADer"](#) on page 1186
 - [":SBUS<n>:USBPd:TRIGger:HEADer:DMESsage"](#) on page 1190
 - [":SBUS<n>:USBPd:TRIGger:HEADer:EMESsage"](#) on page 1191
 - [":SBUS<n>:USBPd:TRIGger:HEADer:VALue"](#) on page 1193
 - [":SBUS<n>:USBPd:TRIGger:HEADer:QUALifier"](#) on page 1194

:SBUS<n>:USBPd:TRIGger:HEADer:DMESsage

N (see [page 1650](#))

Command Syntax :SBUS<n>:USBPd:TRIGger:HEADer:DMESsage <type>

<type> ::= {SRCap | REQuest | BIST | SNCap | BStatus | ALERt | GCINfo
| VDEFined}

When the **Header Content** trigger mode is selected (with the ":SBUS<n>:USBPd:TRIGger:HEADer" command) and the **Data Message** header type is selected (with the ":SBUS<n>:USBPd:TRIGger:HEADer:DMESsage" command), the :SBUS<n>:USBPd:TRIGger:HEADer:DMESsage command selects the data message type:

- SRCap – Source_Capabilities
- REQuest – Request
- BIST – BIST
- SNCap – Sink_Capabilities
- BStatus – Battery_Status
- ALERt – Alert
- GCINfo – Get_Country_Info
- VDEFined – Vendor_Defined

Query Syntax :SBUS<n>:USBPd:TRIGger:HEADer:DMESsage?

The :SBUS<n>:USBPd:TRIGger:HEADer:DMESsage? query returns the selected data message type.

Return Format <type><NL>

<type> ::= {SRC | REQ | BIST | SNC | BST | ALER | GCIN | VDEF}

- See Also**
- [":SBUS<n>:USBPd:SOURce"](#) on page 1184
 - [":SBUS<n>:USBPd:TRIGger"](#) on page 1185
 - [":SBUS<n>:USBPd:TRIGger:HEADer"](#) on page 1186
 - [":SBUS<n>:USBPd:TRIGger:HEADer:CMESsage"](#) on page 1188
 - [":SBUS<n>:USBPd:TRIGger:HEADer:EMESsage"](#) on page 1191
 - [":SBUS<n>:USBPd:TRIGger:HEADer:VALue"](#) on page 1193
 - [":SBUS<n>:USBPd:TRIGger:HEADer:QUALifier"](#) on page 1194

:SBUS<n>:USBPd:TRIGger:HEADer:EMESsage

N (see [page 1650](#))

Command Syntax :SBUS<n>:USBPd:TRIGger:HEADer:EMESsage <type>

```
<type> ::= {SCX | STATus | GBCap | GBSTatus | BCAP | GMINfo | MINFo
           | SREQuest | SRESponse | FREQuest | FRESponse | PSTatus | CINFo
           | CCODes}
```

When the **Header Content** trigger mode is selected (with the ":SBUS<n>:USBPd:TRIGger:HEADer" command) and the **Extended Message** header type is selected (with the ":SBUS<n>:USBPd:TRIGger:HEADer:EMESsage" command), the :SBUS<n>:USBPd:TRIGger:HEADer:EMESsage command selects the extended message type:

- SCX – Source_Capabilities_Extended
- STATus – Status
- GBCap – Get_Battery_Cap
- GBSTatus – Get_Battery_Status
- BCAP – Battery_Capabilities
- GMINfo – Get_Manufacturer_Info
- MINFo – Manufacturer_Info
- SREQuest – Security_Request
- SRESponse – Security_Response
- FREQuest – Firmware_Update_Request
- FRESponse – Firmware_Update_Response
- PSTatus – PPS_Status
- CINFo – Country_Info
- CCODes – Country_Codes

Query Syntax :SBUS<n>:USBPd:TRIGger:HEADer:EMESsage?

The :SBUS<n>:USBPd:TRIGger:HEADer:EMESsage? query returns the selected extended message type

Return Format <type><NL>

```
<type> ::= {SCX | STATus | GBC | GBST | BCAP | GMIN | MINF | SREQ
           | SRES | FREQ | FRES | PSTatus | CINF | CCOD}
```

- See Also**
- [":SBUS<n>:USBPd:SOURce"](#) on page 1184
 - [":SBUS<n>:USBPd:TRIGger"](#) on page 1185
 - [":SBUS<n>:USBPd:TRIGger:HEADer"](#) on page 1186
 - [":SBUS<n>:USBPd:TRIGger:HEADer:CMESsage"](#) on page 1188

- [":SBUS<n>:USBPd:TRIGger:HEADer:DMESsage"](#) on page 1190
- [":SBUS<n>:USBPd:TRIGger:HEADer:VALue"](#) on page 1193
- [":SBUS<n>:USBPd:TRIGger:HEADer:QUALifier"](#) on page 1194

:SBUS<n>:USBPd:TRIGger:HEADer:VALue

N (see [page 1650](#))

Command Syntax :SBUS<n>:USBPd:TRIGger:HEADer:VALue <string>

<string> ::= "nn...n" where n ::= {0 | 1 | X}

<string> ::= "0xnn...n" where n ::= {0,...,9 | A,...,F | X}

When the **Header Content** trigger mode is selected (with the ":SBUS<n>:USBPd:TRIGger:HEADer" command) and the user-defined **Value** type is selected (with the ":SBUS<n>:USBPd:TRIGger:HEADer:VALue" command), the :SBUS<n>:USBPd:TRIGger:HEADer:VALue command specifies the user-defined header value.

Query Syntax :SBUS<n>:USBPd:TRIGger:HEADer:VALue?

The :SBUS<n>:USBPd:TRIGger:HEADer:VALue? query returns the specified user-defined header value.

Returned data values are always quoted binary format strings.

Return Format <string><NL>

<string> ::= "nn...n" where n ::= {0 | 1 | X}

- See Also**
- [":SBUS<n>:USBPd:SOURce"](#) on page 1184
 - [":SBUS<n>:USBPd:TRIGger"](#) on page 1185
 - [":SBUS<n>:USBPd:TRIGger:HEADer"](#) on page 1186
 - [":SBUS<n>:USBPd:TRIGger:HEADer:CMESsage"](#) on page 1188
 - [":SBUS<n>:USBPd:TRIGger:HEADer:DMESsage"](#) on page 1190
 - [":SBUS<n>:USBPd:TRIGger:HEADer:EMESsage"](#) on page 1191
 - [":SBUS<n>:USBPd:TRIGger:HEADer:QUALifier"](#) on page 1194

:SBUS<n>:USBPd:TRIGger:HEADer:QUALifier

N (see [page 1650](#))

Command Syntax :SBUS<n>:USBPd:TRIGger:HEADer:QUALifier <type>

<type> ::= {NONE | SOP | SPRime | SDPRime}

When the **Header Content** trigger mode is selected (with the ":SBUS<n>:USBPd:TRIGger:HEADer" command), the :SBUS<n>:USBPd:TRIGger:HEADer:QUALifier command selects an additional qualifier for the Header Content trigger:

- NONE — **None**. There is no additional qualifier for the trigger.
- SOP — **SOP**. The trigger occurs on Sync-1, Sync-1, Sync-1, Sync-2 ordered sets only.
- SPRime — **SOP'**. The trigger occurs on Sync-1, Sync-1, Sync-3, Sync-3 ordered sets only.
- SDPRime — **SOP"**. The trigger occurs on Sync-1, Sync-3, Sync-1, Sync-3 ordered sets only.

Query Syntax :SBUS<n>:USBPd:TRIGger:HEADer:QUALifier?

The :SBUS<n>:USBPd:TRIGger:HEADer:QUALifier? query returns the selected header content trigger qualifier.

Return Format <type><NL>

<type> ::= {NONE | SOP | SPRime | SDPRime}

- See Also**
- [":SBUS<n>:USBPd:SOURce"](#) on page 1184
 - [":SBUS<n>:USBPd:TRIGger"](#) on page 1185
 - [":SBUS<n>:USBPd:TRIGger:HEADer"](#) on page 1186
 - [":SBUS<n>:USBPd:TRIGger:HEADer:CMESsage"](#) on page 1188
 - [":SBUS<n>:USBPd:TRIGger:HEADer:DMESsage"](#) on page 1190
 - [":SBUS<n>:USBPd:TRIGger:HEADer:EMESsage"](#) on page 1191
 - [":SBUS<n>:USBPd:TRIGger:HEADer:VALue"](#) on page 1193

34 :SEARCh Commands

Control the event search modes and parameters for each search type. See:

- **"General :SEARCh Commands"** on page 1196
- **":SEARCh:EDGE Commands"** on page 1201
- **":SEARCh:GLITCh Commands"** on page 1204 (Pulse Width search)
- **":SEARCh:PEAK Commands"** on page 1211
- **":SEARCh:RUNT Commands"** on page 1216
- **":SEARCh:TRANSition Commands"** on page 1221
- **":SEARCh:SERial:A429 Commands"** on page 1226
- **":SEARCh:SERial:CAN Commands"** on page 1232
- **":SEARCh:SERial:FLEXray Commands"** on page 1242
- **":SEARCh:SERial:I2S Commands"** on page 1248
- **":SEARCh:SERial:IIC Commands"** on page 1254
- **":SEARCh:SERial:LIN Commands"** on page 1261
- **":SEARCh:SERial:M1553 Commands"** on page 1270
- **":SEARCh:SERial:SENT Commands"** on page 1274
- **":SEARCh:SERial:SPI Commands"** on page 1279
- **":SEARCh:SERial:UART Commands"** on page 1283
- **":SEARCh:SERial:USB Commands"** on page 1287

General :SEARCH Commands

Table 136 General :SEARCH Commands Summary

Command	Query	Options and Query Returns
n/a	:SEARCH:COUNT? (see page 1197)	<count> ::= an integer count value
:SEARCH:EVENT <event_number> (see page 1198)	:SEARCH:EVENT? (see page 1198)	<event_number> ::= the integer number of a found search event
:SEARCH:MODE <value> (see page 1199)	:SEARCH:MODE? (see page 1199)	<value> ::= {EDGE GLITCh RUNT TRANSition SERIAL{1 2} PEAK}
:SEARCH:STATE <value> (see page 1200)	:SEARCH:STATE? (see page 1200)	<value> ::= {{0 OFF} {1 ON}}

:SEARCh:COUNT

N (see [page 1650](#))

Query Syntax :SEARCh:COUNT?

The :SEARCh:COUNT? query returns the number of search events found.

Return Format <count><NL>

<count> ::= an integer count value

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:EVENT"](#) on page 1198
 - [":SEARCh:STATe"](#) on page 1200
 - [":SEARCh:MODE"](#) on page 1199

:SEARCh:EVENT

N (see [page 1650](#))

Command Syntax :SEARCh:EVENT <event_number>

<event_number> ::= the integer number of a found search event

The :SEARCh:EVENT command navigates to a found search event.

If the :SEARCh:STATE is ON, the horizontal position is changed so that the specified event is located at the time reference.

Query Syntax :SEARCh:EVENT?

The :SEARCh:EVENT? query returns the currently selected event number.

Return Format <event_number><NL>

<event_number> ::= the integer number of a found search event

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:COUNt"](#) on page 1197
 - [":SEARCh:STAtE"](#) on page 1200
 - [":SEARCh:MODE"](#) on page 1199

:SEARCh:MODE

N (see [page 1650](#))

Command Syntax :SEARCh:MODE <value>

<value> ::= {EDGE | GLITCh | RUNT | TRANSition | SERIAL{1 | 2} | PEAK}

The :SEARCh:MODE command selects the search mode.

The command is only valid when the :SEARCh:STAtE is ON.

Query Syntax :SEARCh:MODE?

The :SEARCh:MODE? query returns the currently selected mode or OFF if the :SEARCh:STAtE is OFF.

Return Format <value><NL>

<value> ::= {EDGE | GLIT | RUNT | TRAN | SER{1 | 2} | PEAK | OFF}

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:STAtE"](#) on page 1200
 - [":SEARCh:COUNt"](#) on page 1197
 - [":SEARCh:EVENT"](#) on page 1198

:SEARCh:STATe

N (see [page 1650](#))

Command Syntax :SEARCh:STATe <value>

<value> ::= {{0 | OFF} | {1 | ON}}

The :SEARCh:STATe command enables or disables the search feature.

Query Syntax :SEARCh:STATe?

The :SEARCh:STATe? query returns the current setting.

Return Format <value><NL>

<value> ::= {0 | 1}

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:MODE"](#) on page 1199
 - [":SEARCh:COUNt"](#) on page 1197
 - [":SEARCh:EVENT"](#) on page 1198

:SEARCh:EDGE Commands

Table 137 :SEARCh:EDGE Commands Summary

Command	Query	Options and Query Returns
:SEARCh:EDGE:SLOPe <slope> (see page 1202)	:SEARCh:EDGE:SLOPe? (see page 1202)	<slope> ::= {POSitive NEGative EITHer}
:SEARCh:EDGE:SOURce <source> (see page 1203)	:SEARCh:EDGE:SOURce? (see page 1203)	<source> ::= CHANnel<n> <n> ::= 1 to (# analog channels) in NR1 format

:SEARCh:EDGE:SLOPe

N (see [page 1650](#))

Command Syntax :SEARCh:EDGE:SLOPe <slope>

<slope> ::= {NEGative | POSitive | EITHer}

The :SEARCh:EDGE:SLOPe command specifies the slope of the edge for the search.

Query Syntax :SEARCh:EDGE:SLOPe?

The :SEARCh:EDGE:SLOPe? query returns the current slope setting.

Return Format <slope><NL>

<slope> ::= {NEG | POS | EITH}

See Also • [Chapter 34](#), “:SEARCh Commands,” starting on page 1195

:SEARCh:EDGE:SOURce

N (see [page 1650](#))

Command Syntax :SEARCh:EDGE:SOURce <source>

<source> ::= CHANnel<n>

<n> ::= 1 to (# analog channels) in NR1 format

The :SEARCh:EDGE:SOURce command selects the channel on which to search for edges.

Query Syntax :SEARCh:EDGE:SOURce?

The :SEARCh:EDGE:SOURce? query returns the current source.

Return Format <source><NL>

<source> ::= CHAN<n>

See Also • [Chapter 34](#), “:SEARCh Commands,” starting on page 1195

:SEARCH:GLITCh Commands

Table 138 :SEARCH:GLITCh Commands Summary

Command	Query	Options and Query Returns
:SEARCH:GLITCh:GREate rthan <greater_than_time>[s uffix] (see page 1205)	:SEARCH:GLITCh:GREate rthan? (see page 1205)	<greater_than_time> ::= floating-point number in NR3 format [suffix] ::= {s ms us ns ps}
:SEARCH:GLITCh:LESSth an <less_than_time>[suff ix] (see page 1206)	:SEARCH:GLITCh:LESSth an? (see page 1206)	<less_than_time> ::= floating-point number in NR3 format [suffix] ::= {s ms us ns ps}
:SEARCH:GLITCh:POLari ty <polarity> (see page 1207)	:SEARCH:GLITCh:POLari ty? (see page 1207)	<polarity> ::= {POSitive NEGative}
:SEARCH:GLITCh:QUALif ier <qualifier> (see page 1208)	:SEARCH:GLITCh:QUALif ier? (see page 1208)	<qualifier> ::= {GREaterthan LESSthan RANGE}
:SEARCH:GLITCh:RANGE <less_than_time>[suff ix], <greater_than_time>[s uffix] (see page 1209)	:SEARCH:GLITCh:RANGE? (see page 1209)	<less_than_time> ::= 15 ns to 10 seconds in NR3 format <greater_than_time> ::= 10 ns to 9.99 seconds in NR3 format [suffix] ::= {s ms us ns ps}
:SEARCH:GLITCh:SOURce <source> (see page 1210)	:SEARCH:GLITCh:SOURce ? (see page 1210)	<source> ::= CHANNEL<n> <n> ::= 1 to (# analog channels) in NR1 format

:SEARCh:GLITCh:GREaterthan

N (see [page 1650](#))

- Command Syntax** :SEARCh:GLITCh:GREaterthan <greater_than_time> [<suffix>]
 <greater_than_time> ::= floating-point number in NR3 format
 <suffix> ::= {s | ms | us | ns | ps}
- The :SEARCh:GLITCh:GREaterthan command sets the minimum pulse width duration for the selected :SEARCh:GLITCh:SOURce.
- Query Syntax** :SEARCh:GLITCh:GREaterthan?
- The :SEARCh:GLITCh:GREaterthan? query returns the minimum pulse width duration time for :SEARCh:GLITCh:SOURce.
- Return Format** <greater_than_time><NL>
 <greater_than_time> ::= floating-point number in NR3 format.
- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:GLITCh:SOURce"](#) on page 1210
 - [":SEARCh:GLITCh:QUALifier"](#) on page 1208
 - [":SEARCh:MODE"](#) on page 1199

:SEARCh:GLITCh:LESSthan

N (see [page 1650](#))

Command Syntax :SEARCh:GLITCh:LESSthan <less_than_time>[<suffix>]
 <less_than_time> ::= floating-point number in NR3 format
 <suffix> ::= {s | ms | us | ns | ps}
 The :SEARCh:GLITCh:LESSthan command sets the maximum pulse width duration for the selected :SEARCh:GLITCh:SOURce.

Query Syntax :SEARCh:GLITCh:LESSthan?
 The :SEARCh:GLITCh:LESSthan? query returns the pulse width duration time for :SEARCh:GLITCh:SOURce.

Return Format <less_than_time><NL>
 <less_than_time> ::= floating-point number in NR3 format.

See Also

- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
- [":SEARCh:GLITCh:SOURce"](#) on page 1210
- [":SEARCh:GLITCh:QUALifier"](#) on page 1208
- [":SEARCh:MODE"](#) on page 1199

:SEARCh:GLITCh:POLarity

N (see [page 1650](#))

Command Syntax :SEARCh:GLITCh:POLarity <polarity>
 <polarity> ::= {POSitive | NEGative}

The :SEARCh:GLITCh:POLarity command sets the polarity for the glitch (pulse width) search.

Query Syntax :SEARCh:GLITCh:POLarity?

The :SEARCh:GLITCh:POLarity? query returns the current polarity setting for the glitch (pulse width) search.

Return Format <polarity><NL>
 <polarity> ::= {POS | NEG}

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:MODE"](#) on page 1199
 - [":SEARCh:GLITCh:SOURce"](#) on page 1210

:SEARCh:GLITCh:QUALifier

N (see [page 1650](#))

Command Syntax :SEARCh:GLITCh:QUALifier <operator>

<operator> ::= {GREATERthan | LESSthan | RANGE}

This command sets the mode of operation of the glitch (pulse width) search. The oscilloscope can search for a pulse width that is greater than a time value, less than a time value, or within a range of time values.

Query Syntax :SEARCh:GLITCh:QUALifier?

The :SEARCh:GLITCh:QUALifier? query returns the glitch (pulse width) qualifier.

Return Format <operator><NL>

<operator> ::= {GRE | LESS | RANG}

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:GLITCh:SOURce"](#) on page 1210
 - [":SEARCh:MODE"](#) on page 1199

:SEARCh:GLITCh:SOURce

N (see [page 1650](#))

Command Syntax :SEARCh:GLITCh:SOURce <source>

<source> ::= CHANnel<n>

<n> ::= 1 to (# analog channels) in NR1 format

The :SEARCh:GLITCh:SOURce command selects the channel on which to search for glitches (pulse widths).

Query Syntax :SEARCh:GLITCh:SOURce?

The :SEARCh:GLITCh:SOURce? query returns the current pulse width source.

If all channels are off, the query returns "NONE."

Return Format <source><NL>

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:MODE"](#) on page 1199
 - [":SEARCh:GLITCh:POLarity"](#) on page 1207
 - [":SEARCh:GLITCh:QUALifier"](#) on page 1208
 - [":SEARCh:GLITCh:RANGe"](#) on page 1209

:SEARCh:PEAK Commands

Table 139 :SEARCh:PEAK Commands Summary

Command	Query	Options and Query Returns
:SEARCh:PEAK:EXCURsion <delta_level> (see page 1212)	:SEARCh:PEAK:EXCURsion? (see page 1212)	<delta_level> ::= required change in level to be recognized as a peak, in NR3 format.
:SEARCh:PEAK:NPEaks <number> (see page 1213)	:SEARCh:PEAK:NPEaks? (see page 1213)	<number> ::= max number of peaks to find, 1-11 in NR1 format.
:SEARCh:PEAK:SOURce <source> (see page 1214)	:SEARCh:PEAK:SOURce? (see page 1214)	<source> ::= {FUNCTION<m> MATH<m>} (must be an FFT waveform) <m> ::= 1 to 4 in NR1 format
:SEARCh:PEAK:THReshold <level> (see page 1215)	:SEARCh:PEAK:THReshold? (see page 1215)	<level> ::= necessary level to be considered a peak, in NR3 format.

:SEARCh:PEAK:EXCursion

N (see [page 1650](#))

Command Syntax :SEARCh:PEAK:EXCursion <delta_level>

<delta_level> ::= required change in level to be recognized as a peak,
in NR3 format.

The :SEARCh:PEAK:EXCursion command specifies the change in level that must occur (in other words, hysteresis) to be recognized as a peak.

The threshold level units are specified by the :FUNCTion<m>[:FFT]:VTYPE command.

Query Syntax :SEARCh:PEAK:EXCursion?

The :SEARCh:PEAK:EXCursion? query returns the specified excursion delta level value.

Return Format <delta_level><NL>

<delta_level> ::= in NR3 format.

- See Also**
- [":FUNCTion<m>\[:FFT\]:VTYPE"](#) on page 476
 - [":SEARCh:PEAK:NPEaks"](#) on page 1213
 - [":SEARCh:PEAK:SOURce"](#) on page 1214
 - [":SEARCh:PEAK:THReshold"](#) on page 1215

:SEARCh:PEAK:NPEaks

N (see [page 1650](#))

Command Syntax :SEARCh:PEAK:NPEaks <number>

<number> ::= max number of peaks to find, 1-11 in NR1 format.

The :SEARCh:PEAK:NPEaks command specifies the maximum number of FFT peaks to find. This number can be from 1 to 11.

Query Syntax :SEARCh:PEAK:NPEaks?

The :SEARCh:PEAK:NPEaks? query returns the specified maximum number of FFT peaks to find.

Return Format <number><NL>

<number> ::= in NR1 format.

- See Also**
- [":SEARCh:PEAK:EXCursion"](#) on page 1212
 - [":SEARCh:PEAK:SOURce"](#) on page 1214
 - [":SEARCh:PEAK:THReshold"](#) on page 1215

:SEARCh:PEAK:SOURce

N (see [page 1650](#))

Command Syntax :SEARCh:PEAK:SOURce <source>

<source> ::= {FUNCtion<m> | MATH<m>} (must be an FFT waveform)

<m> ::= 1 to 4 in NR1 format

The :SEARCh:PEAK:SOURce command selects the FFT math function waveform to search.

Query Syntax :SEARCh:PEAK:SOURce?

The :SEARCh:PEAK:SOURce? query returns the FFT math function waveform that is being searched.

Return Format <source><NL>

<source> ::= FUNC<m> (must be FFT)

<m> ::= 1 to 4 in NR1 format

- See Also**
- [":SEARCh:PEAK:EXCursion"](#) on page 1212
 - [":SEARCh:PEAK:NPEaks"](#) on page 1213
 - [":SEARCh:PEAK:THReshold"](#) on page 1215

:SEARCh:PEAK:THReshold

N (see [page 1650](#))

Command Syntax :SEARCh:PEAK:THReshold <level>

<level> ::= necessary level to be considered a peak, in NR3 format.

The :SEARCh:PEAK:THReshold command specifies the threshold level necessary to be considered a peak.

The threshold level units are specified by the :FUNCTion<m>[:FFT]:VTYPe command.

Query Syntax :SEARCh:PEAK:THReshold?

The :SEARCh:PEAK:THReshold? query returns the specified threshold level for FFT peak search.

Return Format <level><NL>

<level> ::= in NR3 format.

- See Also**
- [":FUNCTion<m>\[:FFT\]:VTYPe"](#) on page 476
 - [":SEARCh:PEAK:EXCursion"](#) on page 1212
 - [":SEARCh:PEAK:NPEaks"](#) on page 1213
 - [":SEARCh:PEAK:SOURce"](#) on page 1214

:SEARCH:RUNT Commands

Table 140 :SEARCH:RUNT Commands Summary

Command	Query	Options and Query Returns
:SEARCH:RUNT:POLarity <polarity> (see page 1217)	:SEARCH:RUNT:POLarity? (see page 1217)	<polarity> ::= {POSitive NEGative EITHer}
:SEARCH:RUNT:QUALifier <qualifier> (see page 1218)	:SEARCH:RUNT:QUALifier? (see page 1218)	<qualifier> ::= {GREaterthan LESSthan NONE}
:SEARCH:RUNT:SOURce <source> (see page 1219)	:SEARCH:RUNT:SOURce? (see page 1219)	<source> ::= CHANnel<n> <n> ::= 1 to (# analog channels) in NR1 format
:SEARCH:RUNT:TIME <time>[suffix] (see page 1220)	:SEARCH:RUNT:TIME? (see page 1220)	<time> ::= floating-point number in NR3 format [suffix] ::= {s ms us ns ps}

:SEARCh:RUNT:POLarity

N (see [page 1650](#))

Command Syntax :SEARCh:RUNT:POLarity <slope>

<polarity> ::= {POSitive | NEGative | EITHer}

The :SEARCh:RUNT:POLarity command sets the polarity for the runt search.

Query Syntax :SEARCh:RUNT:POLarity?

The :SEARCh:RUNT:POLarity? query returns the currently set runt polarity.

Return Format <slope><NL>

<polarity> ::= {POS | NEG | EITH}

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:MODE"](#) on page 1199
 - [":SEARCh:RUNT:SOURce"](#) on page 1219

:SEARCh:RUNT:QUALifier

N (see [page 1650](#))

Command Syntax :SEARCh:RUNT:QUALifier <qualifier>

<qualifier> ::= {GREATERthan | LESSthan | NONE}

The :SEARCh:RUNT:QUALifier command specifies whether to search for a runt that is greater than a time value, less than a time value, or any time value.

Query Syntax :SEARCh:RUNT:QUALifier?

The :SEARCh:RUNT:QUALifier? query returns the current runt search qualifier.

Return Format <qualifier><NL>

<qualifier> ::= {GRE | LESS NONE}

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:MODE"](#) on page 1199

:SEARCh:RUNT:SOURce

N (see [page 1650](#))

Command Syntax :SEARCh:RUNT:SOURce <source>

<source> ::= CHANnel<n>

<n> ::= 1 to (# analog channels) in NR1 format

The :SEARCh:RUNT:SOURce command selects the channel on which to search for the runt pulse.

Query Syntax :SEARCh:RUNT:SOURce?

The :SEARCh:RUNT:SOURce? query returns the current runt search source.

Return Format <source><NL>

<source> ::= CHAN<n>

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:RUNT:POLarity"](#) on page 1217

:SEARCh:RUNT:TIME

N (see [page 1650](#))

Command Syntax :SEARCh:RUNT:TIME <time>[suffix]

<time> ::= floating-point number in NR3 format

[suffix] ::= {s | ms | us | ns | ps}

When searching for runt pulses whose widths are greater than or less than a time (see :SEARCh:RUNT:QUALifier), the :SEARCh:RUNT:TIME command specifies the time value.

Query Syntax :SEARCh:RUNT:TIME?

The :SEARCh:RUNT:TIME? query returns the currently specified runt time value.

Return Format <time><NL>

<time> ::= floating-point number in NR3 format

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:RUNT:QUALifier"](#) on page 1218

:SEARCh:TRANsition Commands

Table 141 :SEARCh:TRANsition Commands Summary

Command	Query	Options and Query Returns
:SEARCh:TRANsition:QU ALifier <qualifier> (see page 1222)	:SEARCh:TRANsition:QU ALifier? (see page 1222)	<qualifier> ::= {GREATERthan LESSthan}
:SEARCh:TRANsition:SL OPe <slope> (see page 1223)	:SEARCh:TRANsition:SL OPe? (see page 1223)	<slope> ::= {NEGative POSitive}
:SEARCh:TRANsition:SO URce <source> (see page 1224)	:SEARCh:TRANsition:SO URce? (see page 1224)	<source> ::= CHANnel<n> <n> ::= 1 to (# analog channels) in NR1 format
:SEARCh:TRANsition:TI ME <time>[suffix] (see page 1225)	:SEARCh:TRANsition:TI ME? (see page 1225)	<time> ::= floating-point number in NR3 format [suffix] ::= {s ms us ns ps}

:SEARCh:TRANSition:QUALifier

N (see [page 1650](#))

Command Syntax :SEARCh:TRANSition:QUALifier <qualifier>

<qualifier> ::= {GREATERthan | LESSthan}

The :SEARCh:TRANSition:QUALifier command specifies whether to search for edge transitions greater than or less than a time.

Query Syntax :SEARCh:TRANSition:QUALifier?

The :SEARCh:TRANSition:QUALifier? query returns the current transition search qualifier.

Return Format <qualifier><NL>

<qualifier> ::= {GRE | LESS}

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:MODE"](#) on page 1199
 - [":SEARCh:TRANSition:TIME"](#) on page 1225

:SEARCh:TRANSition:SLOPe

N (see [page 1650](#))

Command Syntax :SEARCh:TRANSition:SLOPe <slope>

<slope> ::= {NEGative | POSitive}

The :SEARCh:TRANSition:SLOPe command selects whether to search for rising edge (POSitive slope) transitions or falling edge (NEGative slope) transitions.

Query Syntax :SEARCh:TRANSition:SLOPe?

The :SEARCh:TRANSition:SLOPe? query returns the current transition search slope setting.

Return Format <slope><NL>

<slope> ::= {NEG | POS}

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:MODE"](#) on page 1199
 - [":SEARCh:TRANSition:SOURce"](#) on page 1224
 - [":SEARCh:TRANSition:TIME"](#) on page 1225

:SEARCh:TRANSition:SOURce

N (see [page 1650](#))

Command Syntax :SEARCh:TRANSition:SOURce <source>

<source> ::= CHANnel<n>

<n> ::= 1 to (# analog channels) in NR1 format

The :SEARCh:TRANSition:SOURce command selects the channel on which to search for edge transitions.

Query Syntax :SEARCh:TRANSition:SOURce?

The :SEARCh:TRANSition:SOURce? query returns the current transition search source.

Return Format <source><NL>

<source> ::= CHAN<n>

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:MODE"](#) on page 1199
 - [":SEARCh:TRANSition:SLOPe"](#) on page 1223

:SEARCh:TRANsition:TIME

N (see [page 1650](#))

Command Syntax :SEARCh:TRANsition:TIME <time>[suffix]

<time> ::= floating-point number in NR3 format

[suffix] ::= {s | ms | us | ns | ps}

The :SEARCh:TRANsition:TIME command sets the time of the transition to search for. You can search for transitions greater than or less than this time.

Query Syntax :SEARCh:TRANsition:TIME?

The :SEARCh:TRANsition:TIME? query returns the current transition time value.

Return Format <time><NL>

<time> ::= floating-point number in NR3 format

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:TRANsition:QUALifier"](#) on page 1222

:SEARCH:SERIAL:A429 Commands

Table 142 :SEARCH:SERIAL:A429 Commands Summary

Command	Query	Options and Query Returns
:SEARCH:SERIAL:A429:LABel <value> (see page 1227)	:SEARCH:SERIAL:A429:LABel? (see page 1227)	<value> ::= 8-bit integer in decimal, <hex>, <octal>, or <string> from 0-255 <hex> ::= #Hnn where n ::= {0,...,9 A,...,F} <octal> ::= #Qnnn where n ::= {0,...,7} <string> ::= "0xnn" where n ::= {0,...,9 A,...,F}
:SEARCH:SERIAL:A429:MODE <condition> (see page 1228)	:SEARCH:SERIAL:A429:MODE? (see page 1228)	<condition> ::= {LABel LBITs PERRor WERRor GERRor WGERrors ALLerrors}
:SEARCH:SERIAL:A429:PATTern:DATA <string> (see page 1229)	:SEARCH:SERIAL:A429:PATTern:DATA? (see page 1229)	<string> ::= "nn...n" where n ::= {0 1}, length depends on FORMat
:SEARCH:SERIAL:A429:PATTern:SDI <string> (see page 1230)	:SEARCH:SERIAL:A429:PATTern:SDI? (see page 1230)	<string> ::= "nn" where n ::= {0 1}, length always 2 bits
:SEARCH:SERIAL:A429:PATTern:SSM <string> (see page 1231)	:SEARCH:SERIAL:A429:PATTern:SSM? (see page 1231)	<string> ::= "nn" where n ::= {0 1}, length always 2 bits

:SEARCh:SERial:A429:LABel

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:A429:LABel <value>

<value> ::= 8-bit integer in decimal, <hex>, <octal>, or <string>
from 0-255

<hex> ::= #Hnn where n ::= {0,...,9 | A,...,F}

<octal> ::= #Qnnn where n ::= {0,...,7}

<string> ::= "0xnn" where n ::= {0,...,9 | A,...,F}

The :SEARCh:SERial:A429:LABel command defines the ARINC 429 label value when labels are used in the selected search mode.

Query Syntax :SEARCh:SERial:A429:LABel?

The :SEARCh:SERial:A429:LABel? query returns the current label value in decimal format.

Return Format <value><NL> in decimal format

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • ["Introduction to :TRIGger Commands"](#) on page 1345

• [":SEARCh:SERial:A429:MODE"](#) on page 1228

:SEARCh:SERial:A429:MODE

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:A429:MODE <condition>
 <condition> ::= {LAbel | LBITs | PERRor | WERRor | GERRor | WGERrors
 | ALLerrors}

The :SEARCh:SERial:A429:MODE command selects the type of ARINC 429 information to find in the Lister display:

- LAbel – finds the specified label value.
- LBITs – finds the label and the other word fields as specified.
- PERRor – finds words with a parity error.
- WERRor – finds an intra-word coding error.
- GERRor – finds an inter-word gap error.
- WGERrors – finds either a Word or Gap Error.
- ALLerrors – finds any of the above errors.

Query Syntax :SEARCh:SERial:A429:MODE?

The :SEARCh:SERial:A429:MODE? query returns the current ARINC 429 search mode condition.

Return Format <condition><NL>
 <condition> ::= {LAB | LBIT | PERR | WERR | GERR | WGER | ALL}

Errors • ["-241, Hardware missing"](#) on page 1591

- See Also**
- ["Introduction to :SBUS<n> Commands"](#) on page 899
 - [":SBUS<n>:MODE"](#) on page 903
 - [":SEARCh:SERial:A429:LAbel"](#) on page 1227
 - [":SEARCh:SERial:A429:PATtern:DATA"](#) on page 1229
 - [":SEARCh:SERial:A429:PATtern:SDI"](#) on page 1230
 - [":SEARCh:SERial:A429:PATtern:SSM"](#) on page 1231
 - [":SBUS<n>:A429:SOURce"](#) on page 914

:SEARCh:SERIal:A429:PATTerN:DATA

N (see [page 1650](#))

Command Syntax :SEARCh:SERIal:A429:PATTerN:DATA <string>
 <string> ::= "nn...n" where n ::= {0 | 1}, length depends on FORMat
 The :SEARCh:SERIal:A429:PATTerN:DATA command defines the ARINC 429 data pattern resource according to the string parameter. This pattern controls the data pattern searched for in each ARINC 429 word.

NOTE

If more bits are sent for <string> than specified by the :SBUS<n>:A429:FORMat command, the most significant bits will be truncated.

Query Syntax :SEARCh:SERIal:A429:PATTerN:DATA?

The :SEARCh:SERIal:A429:PATTerN:DATA? query returns the current settings of the specified ARINC 429 data pattern resource in the binary string format.

Return Format <string><NL>

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • ["Introduction to :TRIGger Commands"](#) on page 1345
 • [":SEARCh:SERIal:A429:MODE"](#) on page 1228
 • [":SEARCh:SERIal:A429:PATTerN:SDI"](#) on page 1230
 • [":SEARCh:SERIal:A429:PATTerN:SSM"](#) on page 1231

:SEARCh:SERial:A429:PATtern:SDI

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:A429:PATtern:SDI <string>

<string> ::= "nn" where n ::= {0 | 1}, length always 2 bits

The :SEARCh:SERial:A429:PATtern:SDI command defines the ARINC 429 two-bit SDI pattern resource according to the string parameter. This pattern controls the SDI pattern searched for in each ARINC 429 word.

The specified SDI is only used if the :SBUS<n>:A429:FORMat includes the SDI field.

Query Syntax :SEARCh:SERial:A429:PATtern:SDI?

The :SEARCh:SERial:A429:PATtern:SDI? query returns the current settings of the specified ARINC 429 two-bit SDI pattern resource in the binary string format.

Return Format <string><NL>

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • ["Introduction to :TRIGger Commands"](#) on page 1345
 • [":SBUS<n>:A429:FORMat"](#) on page 912
 • [":SEARCh:SERial:A429:MODE"](#) on page 1228
 • [":SEARCh:SERial:A429:PATtern:DATA"](#) on page 1229
 • [":SEARCh:SERial:A429:PATtern:SSM"](#) on page 1231

:SEARCh:SERIal:A429:PATtern:SSM

N (see [page 1650](#))

Command Syntax :SEARCh:SERIal:A429:PATtern:SSM <string>

<string> ::= "nn" where n ::= {0 | 1}, length always 2 bits

The :SEARCh:SERIal:A429:PATtern:SSM command defines the ARINC 429 two-bit SSM pattern resource according to the string parameter. This pattern controls the SSM pattern searched for in each ARINC 429 word.

The specified SSM is only used if the :SBUS<n>:A429:FORMat includes the SSM field.

Query Syntax :SEARCh:SERIal:A429:PATtern:SSM?

The :SEARCh:SERIal:A429:PATtern:SSM? query returns the current settings of the specified ARINC 429 two-bit SSM pattern resource in the binary string format.

Return Format <string><NL>

Errors • ["-241, Hardware missing"](#) on page 1591

See Also • ["Introduction to :TRIGger Commands"](#) on page 1345
 • [":SBUS<n>:A429:FORMat"](#) on page 912
 • [":SEARCh:SERIal:A429:MODE"](#) on page 1228
 • [":SEARCh:SERIal:A429:PATtern:DATA"](#) on page 1229
 • [":SEARCh:SERIal:A429:PATtern:SDI"](#) on page 1230

:SEARCH:SERIAL:CAN Commands

Table 143 :SEARCH:SERIAL:CAN Commands Summary

Command	Query	Options and Query Returns
:SEARCH:SERIAL:CAN:MODE <value> (see page 1233)	:SEARCH:SERIAL:CAN:MODE? (see page 1233)	<value> ::= {IDEither IDData DATA IDRemote ERROR ACKError FORMError STUFFerror CRCError ALLerrors OVERload MESSAGE MSIGNAL}
:SEARCH:SERIAL:CAN:PA TTern:DATA <string> (see page 1235)	:SEARCH:SERIAL:CAN:PA TTern:DATA? (see page 1235)	<string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X} for hexadecimal
:SEARCH:SERIAL:CAN:PA TTern:DATA:LENGTH <length> (see page 1236)	:SEARCH:SERIAL:CAN:PA TTern:DATA:LENGTH? (see page 1236)	<length> ::= integer from 1 to 8 in NR1 format
:SEARCH:SERIAL:CAN:PA TTern:ID <string> (see page 1237)	:SEARCH:SERIAL:CAN:PA TTern:ID? (see page 1237)	<string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X} for hexadecimal
:SEARCH:SERIAL:CAN:PA TTern:ID:MODE <value> (see page 1238)	:SEARCH:SERIAL:CAN:PA TTern:ID:MODE? (see page 1238)	<value> ::= {STANDARD EXTENDED}
:SEARCH:SERIAL:CAN:SY MBolic:MESSAGE <name> (see page 1239)	:SEARCH:SERIAL:CAN:SY MBolic:MESSAGE? (see page 1239)	<name> ::= quoted ASCII string
:SEARCH:SERIAL:CAN:SY MBolic:SIGNAL <name> (see page 1240)	:SEARCH:SERIAL:CAN:SY MBolic:SIGNAL? (see page 1240)	<name> ::= quoted ASCII string
:SEARCH:SERIAL:CAN:SY MBolic:VALUE <data> (see page 1241)	:SEARCH:SERIAL:CAN:SY MBolic:VALUE? (see page 1241)	<data> ::= value in NR3 format

:SEARCH:SERIAL:CAN:MODE

N (see [page 1650](#))

Command Syntax :SEARCH:SERIAL:CAN:MODE <value>

```
<value> ::= {IDEither | IDData | DATA | IDRemote | ERRor | ACKerror
            | FORMerror | STUFFerror | CRCerror | ALLerrors | OVERload
            | MESSage | MSIGnal}
```

The :SEARCH:SERIAL:CAN:MODE command selects the type of CAN information to find in the Lister display:

Condition	Front-panel name	Description
IDEither	Frame ID	Finds remote or data frames matching the specified ID.
IDData	Data Frame ID	Finds data frames matching the specified ID.
DATA	Data Frame ID and Data	Finds data frames matching the specified ID and data.
IDRemote	Remote Frame ID	Finds remote frames with the specified ID.
ERRor	Error Frame	Finds CAN active error frames.
ACKerror	Acknowledge Error	Finds the acknowledge bit if the polarity is incorrect.
FORMerror	Form Error	Finds reserved bit errors.
STUFFerror	Stuff Error	Finds 6 consecutive 1s or 6 consecutive 0s, while in a non-error or non overload frame.
CRCerror	CRC Field Error	Finds when the calculated CRC does not match the transmitted CRC.
ALLerrors	All Errors	Finds any form error or active error.
OVERload	Overload Frame	Finds CAN overload frames.
MESSage	Message	Finds a symbolic message.
MSIGnal	Message and Signal (non-FD)	Finds a symbolic message and a signal value.

Query Syntax :SEARCH:SERIAL:CAN:MODE?

The :SEARCH:SERIAL:CAN:MODE? query returns the currently selected mode.

Return Format <value><NL>

```
<value> ::= {IDE | IDD | DATA | IDR | ERR | ACK | FORM | STUF | CRC
            | ALL | OVER | MESS | MSIG}
```

- See Also**
- [Chapter 34](#), “:SEARCH Commands,” starting on page 1195
 - [":SEARCH:SERIAL:CAN:PATTERN:DATA"](#) on page 1235
 - [":SEARCH:SERIAL:CAN:PATTERN:ID"](#) on page 1237

- **":RECall:DBC[:START]"** on page 859
- **":SEARch:SERial:CAN:SYMBolic:MESSage"** on page 1239
- **":SEARch:SERial:CAN:SYMBolic:SIGNal"** on page 1240
- **":SEARch:SERial:CAN:SYMBolic:VALue"** on page 1241

:SEARCh:SErIal:CAN:PATtern:DATA

N (see [page 1650](#))

Command Syntax :SEARCh:SErIal:CAN:PATtern:DATA <string>

<string> ::= "0xnn...n" where n ::= {0,...,9 | A,...,F | X}
for hexadecimal

The :SEARCh:SErIal:CAN:PATtern:DATA command specifies the data value when searching for Data Frame ID and Data.

The length of the data value is specified using the :SEARCh:SErIal:CAN:PATtern:DATA:LENGth command.

Query Syntax :SEARCh:SErIal:CAN:PATtern:DATA?

The :SEARCh:SErIal:CAN:PATtern:DATA? query returns the current data value setting.

Return Format <string><NL>

<string> ::= "0xnn...n" where n ::= {0,...,9 | A,...,F | X}
for hexadecimal

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:SErIal:CAN:MODE"](#) on page 1233
 - [":SEARCh:SErIal:CAN:PATtern:DATA:LENGth"](#) on page 1236

:SEARCh:SERial:CAN:PATtern:DATA:LENGth

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:CAN:PATtern:DATA:LENGth <length>

<length> ::= integer from 1 to 8 in NR1 format

The :SEARCh:SERial:CAN:PATtern:DATA:LENGth command specifies the length of the data value when searching for Data Frame ID and Data.

The data value is specified using the :SEARCh:SERial:CAN:PATtern:DATA command.

Query Syntax :SEARCh:SERial:CAN:PATtern:DATA:LENGth?

The :SEARCh:SERial:CAN:PATtern:DATA:LENGth? query returns the current data length setting.

Return Format <length><NL>

<length> ::= integer from 1 to 8 in NR1 format

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:SERial:CAN:MODE"](#) on page 1233
 - [":SEARCh:SERial:CAN:PATtern:DATA"](#) on page 1235

:SEARCh:SERIal:CAN:PATTerN:ID

N (see [page 1650](#))

Command Syntax :SEARCh:SERIal:CAN:PATTerN:ID <string>

<string> ::= "0xnn...n" where n ::= {0,...,9 | A,...,F | X}
for hexadecimal

The :SEARCh:SERIal:CAN:PATTerN:ID command specifies the ID value when searching for a CAN event.

The value can be a standard ID or an extended ID, depending on the :SEARCh:SERIal:CAN:PATTerN:ID:MODE command's setting.

Query Syntax :SEARCh:SERIal:CAN:PATTerN:ID?

The :SEARCh:SERIal:CAN:PATTerN:ID? query returns the current ID value setting.

Return Format <string><NL>

<string> ::= "0xnn...n" where n ::= {0,...,9 | A,...,F | X}
for hexadecimal

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:SERIal:CAN:MODE"](#) on page 1233
 - [":SEARCh:SERIal:CAN:PATTerN:ID:MODE"](#) on page 1238

:SEARCh:SERIal:CAN:PATTeRn:ID:MODE

N (see [page 1650](#))

Command Syntax :SEARCh:SERIal:CAN:PATTeRn:ID:MODE <value>

<value> ::= {STANdard | EXTended}

The :SEARCh:SERIal:CAN:PATTeRn:ID:MODE command specifies whether a standard ID value or an extended ID value is used when searching for a CAN event.

The ID value is specified using the :SEARCh:SERIal:CAN:PATTeRn:ID command.

Query Syntax :SEARCh:SERIal:CAN:PATTeRn:ID:MODE?

The :SEARCh:SERIal:CAN:PATTeRn:ID:MODE? query returns the current setting.

Return Format <value><NL>

<value> ::= {STAN | EXT}

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:SERIal:CAN:MODE"](#) on page 1233
 - [":SEARCh:SERIal:CAN:PATTeRn:ID"](#) on page 1237

:SEARCh:SERIal:CAN:SYMBolic:MESSage

N (see [page 1650](#))

Command Syntax :SEARCh:SERIal:CAN:SYMBolic:MESSage <name>

<name> ::= quoted ASCII string

The :SEARCh:SERIal:CAN:SYMBolic:MESSage command specifies the message to search for when CAN symbolic data has been loaded (recalled) into the oscilloscope and the CAN serial search mode is set to MESSage or MSIGnal.

Query Syntax :SEARCh:SERIal:CAN:SYMBolic:MESSage?

The :SEARCh:SERIal:CAN:SYMBolic:MESSage? query returns the specified message.

Return Format <name><NL>

<name> ::= quotes ASCII string

- See Also**
- [":RECall:DBC\[:STARt\]"](#) on page 859
 - [":SEARCh:SERIal:CAN:MODE"](#) on page 1233
 - [":SEARCh:SERIal:CAN:SYMBolic:SIGNaI"](#) on page 1240
 - [":SEARCh:SERIal:CAN:SYMBolic:VALue"](#) on page 1241

:SEARCh:SERIal:CAN:SYMBolic:SIGNal

N (see [page 1650](#))

Command Syntax :SEARCh:SERIal:CAN:SYMBolic:SIGNal <name>

<name> ::= quoted ASCII string

The :SEARCh:SERIal:CAN:SYMBolic:SIGNal command specifies the signal to search for when CAN symbolic data has been loaded (recalled) into the oscilloscope and the CAN serial search mode is set to MSIGnal.

Query Syntax :SEARCh:SERIal:CAN:SYMBolic:SIGNal?

The :SEARCh:SERIal:CAN:SYMBolic:SIGNal? query returns the specified signal.

Return Format <name><NL>

<name> ::= quoted ASCII string

- See Also**
- [":RECall:DBC\[:START\]"](#) on page 859
 - [":SEARCh:SERIal:CAN:MODE"](#) on page 1233
 - [":SEARCh:SERIal:CAN:SYMBolic:MESSage"](#) on page 1239
 - [":SEARCh:SERIal:CAN:SYMBolic:VALue"](#) on page 1241

:SEARCh:SERial:CAN:SYMBolic:VALue

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:CAN:SYMBolic:VALue <data>

<data> ::= value in NR3 format

The :SEARCh:SERial:CAN:SYMBolic:VALue command specifies the signal value to search for when CAN symbolic data has been loaded (recalled) into the oscilloscope and the CAN serial search mode is set to MSIGnal.

NOTE

Encoded signal values are not supported in the remote interface (even though they can be used in the front panel graphical interface).

Query Syntax :SEARCh:SERial:CAN:SYMBolic:VALue?

The :SEARCh:SERial:CAN:SYMBolic:VALue? query returns the specified signal value.

Return Format <data><NL>

<data> ::= value in NR3 format

- See Also**
- [":RECall:DBC\[:START\]"](#) on page 859
 - [":SEARCh:SERial:CAN:MODE"](#) on page 1233
 - [":SEARCh:SERial:CAN:SYMBolic:MESSAge"](#) on page 1239
 - [":SEARCh:SERial:CAN:SYMBolic:SIGNal"](#) on page 1240

:SEARCH:SERIAL:FLEXray Commands

Table 144 :SEARCH:SERIAL:FLEXray Commands Summary

Command	Query	Options and Query Returns
:SEARCH:SERIAL:FLEXray:CYCLE <cycle> (see page 1243)	:SEARCH:SERIAL:FLEXray:CYCLE? (see page 1243)	<cycle> ::= {ALL <cycle #>} <cycle #> ::= integer from 0-63
:SEARCH:SERIAL:FLEXray:DATA <string> (see page 1244)	:SEARCH:SERIAL:FLEXray:DATA? (see page 1244)	<string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X }
:SEARCH:SERIAL:FLEXray:DATA:LENGTH <length> (see page 1245)	:SEARCH:SERIAL:FLEXray:DATA:LENGTH? (see page 1245)	<length> ::= integer from 1 to 12 in NR1 format
:SEARCH:SERIAL:FLEXray:FRAME <frame id> (see page 1246)	:SEARCH:SERIAL:FLEXray:FRAME? (see page 1246)	<frame_id> ::= {ALL <frame #>} <frame #> ::= integer from 1-2047
:SEARCH:SERIAL:FLEXray:MODE <value> (see page 1247)	:SEARCH:SERIAL:FLEXray:MODE? (see page 1247)	<value> ::= {FRAME CYCLE DATA HERRor FERRor AERRor}

:SEARCh:SERial:FLEXray:CYCLe

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:FLEXray:CYCLe <cycle>

<cycle> ::= {ALL | <cycle #>}

<cycle #> ::= integer from 0-63

The :SEARCh:SERial:FLEXray:CYCLe command specifies the cycle value to find when searching for FlexRay frames.

A cycle value of -1 is the same as ALL.

Query Syntax :SEARCh:SERial:FLEXray:CYCLe?

The :SEARCh:SERial:FLEXray:CYCLe? query returns the current cycle value setting.

Return Format <cycle><NL>

<cycle> ::= {ALL | <cycle #>}

<cycle #> ::= integer from 0-63

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:SERial:FLEXray:MODE"](#) on page 1247

:SEARCh:SERial:FLEXray:DATA

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:FLEXray:DATA <string>

<string> ::= "0xnn...n" where n ::= {0,...,9 | A,...,F | X }

The :SEARCh:SERial:FLEXray:DATA command specifies the data value to find when searching for FlexRay frames.

The length of the data value is specified by the :SEARCh:SERial:FLEXray:DATA:LENGth command.

Query Syntax :SEARCh:SERial:FLEXray:DATA?

The :SEARCh:SERial:FLEXray:DATA? query returns the current data value setting.

Return Format <string><NL>

<string> ::= "0xnn...n" where n ::= {0,...,9 | A,...,F | X }

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:SERial:FLEXray:MODE"](#) on page 1247
 - [":SEARCh:SERial:FLEXray:DATA:LENGth"](#) on page 1245

:SEARCh:SERIal:FLEXray:DATA:LENGth

N (see [page 1650](#))

Command Syntax :SEARCh:SERIal:FLEXray:DATA:LENGth <length>

<length> ::= integer from 1 to 12 in NR1 format

The :SEARCh:SERIal:FLEXray:DATA:LENGth command specifies the length of data values when searching for FlexRay frames.

The data value is specified using the :SEARCh:SERIal:FLEXray:DATA command.

Query Syntax :SEARCh:SERIal:FLEXray:DATA:LENGth?

The :SEARCh:SERIal:FLEXray:DATA:LENGth? query returns the current data length setting.

Return Format <length><NL>

<length> ::= integer from 1 to 12 in NR1 format

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:SERIal:FLEXray:MODE"](#) on page 1247
 - [":SEARCh:SERIal:FLEXray:DATA"](#) on page 1244

:SEARCh:SERial:FLEXray:FRAME

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:FLEXray:FRAME <frame_id>

<frame_id> ::= {ALL | <frame #>}

<frame #> ::= integer from 1-2047

The :SEARCh:SERial:FLEXray:FRAME command specifies the frame ID value to find when searching for FlexRay frames.

Query Syntax :SEARCh:SERial:FLEXray:FRAME?

The :SEARCh:SERial:FLEXray:FRAME? query returns the current frame ID setting.

Return Format <frame_id><NL>

<frame_id> ::= {ALL | <frame #>}

<frame #> ::= integer from 1-2047

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:SERial:FLEXray:MODE"](#) on page 1247

:SEARCh:SERial:FLEXray:MODE

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:FLEXray:MODE <value>

<value> := {FRAME | CYCLe | DATA | HERRor | FERRor | AERRor}

The :SEARCh:SERial:FLEXray:MODE command selects the type of FlexRay information to find in the Lister display:

- FRAME – searches for FlexRay frames with the specified frame ID.
- CYCLe – searches for FlexRay frames with the specified cycle number and frame ID.
- DATA – searches for FlexRay frames with the specified data, cycle number, and frame ID.
- HERRor – searches for header CRC errors.
- FERRor – searches for frame CRC errors.
- AERRor – searches for all errors.

Query Syntax :SEARCh:SERial:FLEXray:MODE?

The :SEARCh:SERial:FLEXray:MODE? query returns the currently selected mode.

Return Format <value><NL>

<value> := {FRAM | CYCL | DATA | HERR | FERR | AERR}

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:SERial:FLEXray:FRAME"](#) on page 1246
 - [":SEARCh:SERial:FLEXray:CYCLe"](#) on page 1243
 - [":SEARCh:SERial:FLEXray:DATA"](#) on page 1244

:SEARCh:SERial:I2S Commands

Table 145 :SEARCh:SERial:I2S Commands Summary

Command	Query	Options and Query Returns
:SEARCh:SERial:I2S:AUDio <audio_ch> (see page 1249)	:SEARCh:SERial:I2S:AUDio? (see page 1249)	<audio_ch> ::= {RIGHT LEFT EITHER}
:SEARCh:SERial:I2S:MODE <value> (see page 1250)	:SEARCh:SERial:I2S:MODE? (see page 1250)	<value> ::= {EQUAL NOTequal LESSthan GREATERthan INRange OUTRange}
:SEARCh:SERial:I2S:PA TTern:DATA <string> (see page 1251)	:SEARCh:SERial:I2S:PA TTern:DATA? (see page 1251)	<string> ::= "n" where n ::= 32-bit integer in signed decimal when <base> = DECimal <string> ::= "nn...n" where n ::= {0 1 X} when <base> = BINary <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X} when <base> = HEX
:SEARCh:SERial:I2S:PA TTern:FORMat <base> (see page 1252)	:SEARCh:SERial:I2S:PA TTern:FORMat? (see page 1252)	<base> ::= {BINary HEX DECimal}
:SEARCh:SERial:I2S:RANGe <lower>, <upper> (see page 1253)	:SEARCh:SERial:I2S:RANGe? (see page 1253)	<lower> ::= 32-bit integer in signed decimal, <nondecimal>, or <string> <upper> ::= 32-bit integer in signed decimal, <nondecimal>, or <string> <nondecimal> ::= #Hnn...n where n ::= {0,...,9 A,...,F} for hexadecimal <nondecimal> ::= #Bnn...n where n ::= {0 1} for binary <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F} for hexadecimal

:SEARCh:SERIal:I2S:AUDio

N (see [page 1650](#))

Command Syntax :SEARCh:SERIal:I2S:AUDio <audio_ch>

<audio_ch> ::= {RIGHT | LEFT | EITHER}

The :SEARCh:SERIal:I2S:AUDio command specifies the channel on which to search for I2S events: right, left, or either channel.

Query Syntax :SEARCh:SERIal:I2S:AUDio?

The :SEARCh:SERIal:I2S:AUDio? query returns the current channel setting.

Return Format <audio_ch><NL>

<audio_ch> ::= {RIGH | LEFT | EITH}

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:SERIal:I2S:MODE"](#) on page 1250

:SEARCh:SERial:I2S:MODE

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:I2S:MODE <value>

<value> ::= {EQUal | NOTequal | LESSthan | GREaterthan | INRange
| OUTRange}

The :SEARCh:SERial:I2S:MODE command selects the type of I2S information to find in the Lister display:

- EQUal— searches for the specified audio channel's data word when it equals the specified word.
- NOTequal — searches for any word other than the specified word.
- LESSthan — searches for channel data words less than the specified value.
- GREaterthan — searches for channel data words greater than the specified value.
- INRange — searches for channel data words in the range.
- OUTRange — searches for channel data words outside the range.

Data word values are specified using the :SEARCh:SERial:I2S:PATtern:DATA command.

Value ranges are specified using the :SEARCh:SERial:I2S:RANGe command.

Query Syntax :SEARCh:SERial:I2S:MODE?

The :SEARCh:SERial:I2S:MODE? query returns the currently selected mode.

Return Format <value><NL>

<value> ::= {EQU | NOT | LESS | GRE | INR | OUTR}

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:SERial:I2S:PATtern:DATA"](#) on page 1251
 - [":SEARCh:SERial:I2S:RANGe"](#) on page 1253
 - [":SEARCh:SERial:I2S:AUDio"](#) on page 1249

:SEARCh:SERial:I2S:PATtern:DATA

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:I2S:PATtern:DATA <string>

<string> ::= "n" where n ::= 32-bit integer in signed decimal
when <base> = DECimal

<string> ::= "nn...n" where n ::= {0 | 1 | X} when <base> = BINary

<string> ::= "0xnn...n" where n ::= {0,...,9 | A,...,F | X}
when <base> = HEX

The :SEARCh:SERial:I2S:PATtern:DATA command specifies the data word value when searching for I2S events.

The base of the value entered with this command is specified using the :SEARCh:SERial:I2S:PATtern:FORMat command.

Query Syntax :SEARCh:SERial:I2S:PATtern:DATA?

The :SEARCh:SERial:I2S:PATtern:DATA? query returns the current data word value setting.

Return Format <string><NL>

<string> ::= "n" where n ::= 32-bit integer in signed decimal
when <base> = DECimal

<string> ::= "nn...n" where n ::= {0 | 1 | X} when <base> = BINary

<string> ::= "0xnn...n" where n ::= {0,...,9 | A,...,F | X}
when <base> = HEX

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:SERial:I2S:MODE"](#) on page 1250
 - [":SEARCh:SERial:I2S:PATtern:FORMat"](#) on page 1252

:SEARCh:SERial:I2S:PATtern:FORMat

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:I2S:PATtern:FORMat <base>

<base> ::= {BINary | HEX | DECimal}

The :SEARCh:SERial:I2S:PATtern:FORMat command specifies the number base used with the :SEARCh:SERial:I2S:PATtern:DATA command.

Query Syntax :SEARCh:SERial:I2S:PATtern:FORMat?

The :SEARCh:SERial:I2S:PATtern:FORMat? query returns the current number base setting.

Return Format <base><NL>

<base> ::= {BIN | HEX | DEC}

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:SERial:I2S:PATtern:DATA"](#) on page 1251

:SEARCh:SERial:I2S:RANGe

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:I2S:RANGe <lower>, <upper>

<lower> ::= 32-bit integer in signed decimal, <nondecimal>, or <string>

<upper> ::= 32-bit integer in signed decimal, <nondecimal>, or <string>

<nondecimal> ::= #Hnn...n where n ::= {0,...,9 | A,...,F} for hexadecimal

<nondecimal> ::= #Bnn...n where n ::= {0 | 1} for binary

<string> ::= "0xnn...n" where n ::= {0,...,9 | A,...,F} for hexadecimal

The :SEARCh:SERial:I2S:RANGe command specifies the data value range when searching for I2S events in the INRange and OUTRange search modes (set by the :SEARCh:SERial:I2S:MODE command).

You can enter the parameters in any order – the smaller value becomes the <lower> and the larger value becomes the <upper>.

Query Syntax :SEARCh:SERial:I2S:RANGe?

The :SEARCh:SERial:I2S:RANGe? query returns the current data value range setting.

Return Format <lower>, <upper><NL>

<lower> ::= 32-bit integer in signed decimal

<upper> ::= 32-bit integer in signed decimal

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:SERial:I2S:MODE"](#) on page 1250

:SEARCh:SERial:IIC Commands

Table 146 :SEARCh:SERial:IIC Commands Summary

Command	Query	Options and Query Returns
:SEARCh:SERial:IIC:MODE <value> (see page 1255)	:SEARCh:SERial:IIC:MODE? (see page 1255)	<value> ::= {REStart ADDRESS ANACK NACKnowledge READEprom READ7 WRITE7 R7Data2 W7Data2}
:SEARCh:SERial:IIC:PA TTern:ADDRESS <value> (see page 1257)	:SEARCh:SERial:IIC:PA TTern:ADDRESS? (see page 1257)	<value> ::= integer or <string> <string> ::= "0xnn" n ::= {0,...,9 A,...,F}
:SEARCh:SERial:IIC:PA TTern:DATA <value> (see page 1258)	:SEARCh:SERial:IIC:PA TTern:DATA? (see page 1258)	<value> ::= integer or <string> <string> ::= "0xnn" n ::= {0,...,9 A,...,F}
:SEARCh:SERial:IIC:PA TTern:DATA2 <value> (see page 1259)	:SEARCh:SERial:IIC:PA TTern:DATA2? (see page 1259)	<value> ::= integer or <string> <string> ::= "0xnn" n ::= {0,...,9 A,...,F}
:SEARCh:SERial:IIC:QUALifier <value> (see page 1260)	:SEARCh:SERial:IIC:QUALifier? (see page 1260)	<value> ::= {EQUAL NOTequal LESSthan GREATERthan}

:SEARCh:SERial:IIC:MODE

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:IIC:MODE <value>

<value> ::= {REStart | ADDRESS | ANACk | NACKnowledge | READEprom
| READ7 | WRITe7 | R7Data2 | W7Data2}

The :SEARCh:SERial:IIC:MODE command selects the type of IIC information to find in the Lister display:

- REStart – searches for another start condition occurring before a stop condition.
- ADDRESS – searches for a packet with the specified address, ignoring the R/W bit.
- ANACk – searches for address with no acknowledge events.
- NACKnowledge – searches for missing acknowledge events.
- READEprom – searches for EEPROM data reads.
- READ7 – searches for 7-bit address frames containing Start:Address7:Read:Ack:Data. The value READ is also accepted for READ7.
- WRITe7 – searches for 7-bit address frames containing Start:Address7:Write:Ack:Data. The value WRITe is also accepted for WRITe7.
- R7Data2 – searches for 7-bit address frames containing Start:Address7:Read:Ack:Data:Ack:Data2.
- W7Data2 – searches for 7-bit address frames containing Start:Address7:Write:Ack:Data:Ack:Data2.

NOTE

The short form of READ7 (READ7), READEprom (READE), and WRITe7 (WRIT7) do not follow the defined Long Form to Short Form Truncation Rules (see [page 1652](#)).

When searching for events containing addresses, address values are specified using the :SEARCh:SERial:IIC:PATtern:ADDReSS command.

When searching for events containing data, data values are specified using the :SEARCh:SERial:IIC:PATtern:DATA and :SEARCh:SERial:IIC:PATtern:DATA2 commands.

Query Syntax :SEARCh:SERial:IIC:MODE?

The :SEARCh:SERial:IIC:MODE? query returns the currently selected mode.

Return Format <value><NL>

<value> ::= {REST | ADDR | ANAC | NACK | READE | READ7 | WRITe7
| R7D2 | W7D2}

- See Also**
- **Chapter 34**, “:SEARCh Commands,” starting on page 1195
 - **“:SEARCh:SERial:IIC:PATtern:ADDReSS”** on page 1257
 - **“:SEARCh:SERial:IIC:PATtern:DATA”** on page 1258
 - **“:SEARCh:SERial:IIC:PATtern:DATA2”** on page 1259
 - **“:SEARCh:SERial:IIC:QUALifier”** on page 1260

:SEARCh:SERial:IIC:PATtern:ADDReSS

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:IIC:PATtern:ADDReSS <value>

<value> ::= integer or <string>

<string> ::= "0xnn" n ::= {0,...,9 | A,...,F}

The :SEARCh:SERial:IIC:PATtern:ADDReSS command specifies address values when searching for IIC events.

To set don't care values, use the integer -1.

Query Syntax :SEARCh:SERial:IIC:PATtern:ADDReSS?

The :SEARCh:SERial:IIC:PATtern:ADDReSS? query returns the current address value setting.

Return Format <value><NL>

<value> ::= integer

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:SERial:IIC:MODE"](#) on page 1255

:SEARCh:SERial:IIC:PATtern:DATA

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:IIC:PATtern:DATA <value>

<value> ::= integer or <string>

<string> ::= "0xnn" n ::= {0,...,9 | A,...,F}

The :SEARCh:SERial:IIC:PATtern:DATA command specifies data values when searching for IIC events.

To set don't care values, use the integer -1.

When searching for IIC EEPROM data read events, you specify the data value qualifier using the :SEARCh:SERial:IIC:QUALifier command.

Query Syntax :SEARCh:SERial:IIC:PATtern:DATA?

The :SEARCh:SERial:IIC:PATtern:DATA? query returns the current data value setting.

Return Format <value><NL>

<value> ::= integer

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:SERial:IIC:MODE"](#) on page 1255
 - [":SEARCh:SERial:IIC:QUALifier"](#) on page 1260
 - [":SEARCh:SERial:IIC:PATtern:DATA2"](#) on page 1259

:SEARCh:SERial:IIC:PATTerN:DATA2

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:IIC:PATTerN:DATA2 <value>

<value> ::= integer or <string>

<string> ::= "0xnn" n ::= {0,...,9 | A,...,F}

The :SEARCh:SERial:IIC:PATTerN:DATA2 command specifies the second data value when searching for IIC events with two data values.

To set don't care values, use the integer -1.

Query Syntax :SEARCh:SERial:IIC:PATTerN:DATA2?

The :SEARCh:SERial:IIC:PATTerN:DATA2? query returns the current second data value setting.

Return Format <value><NL>

<value> ::= integer

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:SERial:IIC:MODE"](#) on page 1255
 - [":SEARCh:SERial:IIC:PATTerN:DATA"](#) on page 1258

:SEARCh:SERial:IIC:QUALifier

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:IIC:QUALifier <value>

<value> ::= {EQUAL | NOTequal | LESSthan | GREATERthan}

The :SEARCh:SERial:IIC:QUALifier command specifies the data value qualifier used when searching for IIC EEPROM data read events.

Query Syntax :SEARCh:SERial:IIC:QUALifier?

The :SEARCh:SERial:IIC:QUALifier? query returns the current data value qualifier setting.

Return Format <value><NL>

<value> ::= {EQU | NOT | LESS | GRE}

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:SERial:IIC:MODE"](#) on page 1255
 - [":SEARCh:SERial:IIC:PATtern:DATA"](#) on page 1258

:SEARCh:SERial:LIN Commands

Table 147 :SEARCh:SERial:LIN Commands Summary

Command	Query	Options and Query Returns
:SEARCh:SERial:LIN:ID <value> (see page 1262)	:SEARCh:SERial:LIN:ID ? (see page 1262)	<value> ::= 7-bit integer in decimal, <nondecimal>, or <string> from 0-63 or 0x00-0x3f <nondecimal> ::= #Hnn where n ::= {0,...,9 A,...,F} for hexadecimal <nondecimal> ::= #Bnn...n where n ::= {0 1} for binary <string> ::= "0xnn" where n ::= {0,...,9 A,...,F} for hexadecimal
:SEARCh:SERial:LIN:MO DE <value> (see page 1263)	:SEARCh:SERial:LIN:MO DE? (see page 1263)	<value> ::= {ID DATA ERRor}
:SEARCh:SERial:LIN:PA TTern:DATA <string> (see page 1264)	:SEARCh:SERial:LIN:PA TTern:DATA? (see page 1264)	When :SEARCh:SERial:LIN:PA TTern:FORMa t DECimal, <string> ::= "n" where n ::= 32-bit integer in unsigned decimal, returns "\$" if data has any don't cares When :SEARCh:SERial:LIN:PA TTern:FORMa t HEX, <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X }
:SEARCh:SERial:LIN:PA TTern:DATA:LENGth <length> (see page 1265)	:SEARCh:SERial:LIN:PA TTern:DATA:LENGth? (see page 1265)	<length> ::= integer from 1 to 8 in NR1 format
:SEARCh:SERial:LIN:PA TTern:FORMat <base> (see page 1266)	:SEARCh:SERial:LIN:PA TTern:FORMat? (see page 1266)	<base> ::= {HEX DECimal}
:SEARCh:SERial:LIN:SY MBolic:FRAME <name> (see page 1267)	:SEARCh:SERial:LIN:SY MBolic:FRAME? (see page 1267)	<name> ::= quoted ASCII string
:SEARCh:SERial:LIN:SY MBolic:SIGNAL <name> (see page 1268)	:SEARCh:SERial:LIN:SY MBolic:SIGNAL? (see page 1268)	<name> ::= quoted ASCII string
:SEARCh:SERial:LIN:SY MBolic:VALue <data> (see page 1269)	:SEARCh:SERial:LIN:SY MBolic:VALue? (see page 1269)	<data> ::= value in NR3 format

:SEARCh:SERial:LIN:ID

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:LIN:ID <value>

<value> ::= 7-bit integer in decimal, <nondecimal>, or <string>
from 0-63 or 0x00-0x3f

<nondecimal> ::= #Hnn where n ::= {0,...,9 | A,...,F} for hexadecimal

<nondecimal> ::= #Bnn...n where n ::= {0 | 1} for binary

<string> ::= "0xnn" where n ::= {0,...,9 | A,...,F} for hexadecimal

The :SEARCh:SERial:LIN:ID command specifies the frame ID value when searching for LIN events.

Query Syntax :SEARCh:SERial:LIN:ID?

The :SEARCh:SERial:LIN:ID? query returns the current frame ID setting.

Return Format <value><NL>

<value> ::= 7-bit integer in decimal

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:SERial:LIN:MODE"](#) on page 1263

:SEARCh:SERial:LIN:MODE

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:LIN:MODE <value>

<value> ::= {ID | DATA | ERRor | FRAMe | FSIGnal}

The :SEARCh:SERial:LIN:MODE command selects the type of LIN information to find in the Lister display:

- ID – searches for a frame ID.
- DATA – searches for a frame ID and data.
- ERRor – searches for errors.
- FRAMe – searches for symbolic frames.
- FSIGnal – searched for symbolic frames and a signal values.

Frame IDs are specified using the :SEARCh:SERial:LIN:ID command.

Data values are specified using the :SEARCh:SERial:LIN:PATtern:DATA command.

Frames, signals, and signal values are specified using the :SEARCh:SERial:LIN:SYMBolic:FRAMe, :SEARCh:SERial:LIN:SYMBolic:SIGNaL, and :SEARCh:SERial:LIN:SYMBolic:VALue commands. LIN symbolic data files are loaded (recalled) using the :RECall:LDF[:STARt] command.

Query Syntax :SEARCh:SERial:LIN:MODE?

The :SEARCh:SERial:LIN:MODE? query returns the currently selected mode.

Return Format <value><NL>

<value> ::= {ID | DATA | ERR | FRAM | FSIG}

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:SERial:LIN:ID"](#) on page 1262
 - [":SEARCh:SERial:LIN:PATtern:DATA"](#) on page 1264
 - [":RECall:LDF\[:STARt\]"](#) on page 861
 - [":SEARCh:SERial:LIN:SYMBolic:FRAMe"](#) on page 1267
 - [":SEARCh:SERial:LIN:SYMBolic:SIGNaL"](#) on page 1268
 - [":SEARCh:SERial:LIN:SYMBolic:VALue"](#) on page 1269

:SEARCH:SERIAL:LIN:PATTERN:DATA

N (see [page 1650](#))

Command Syntax :SEARCH:SERIAL:LIN:PATTERN:DATA <string>

When :SEARCH:SERIAL:LIN:PATTERN:FORMAT DECIMAL,
<string> ::= "n" where n ::= 32-bit integer in unsigned decimal

When :SEARCH:SERIAL:LIN:PATTERN:FORMAT HEX,
<string> ::= "0xnn...n" where n ::= {0,...,9 | A,...,F | X }

The :SEARCH:SERIAL:LIN:PATTERN:DATA command specifies the data value when searching for LIN events.

The number base of the value entered with this command is specified using the :SEARCH:SERIAL:LIN:PATTERN:FORMAT command. To set don't care values with the DATA command, the FORMAT must be HEX.

The length of the data value entered is specified using the :SEARCH:SERIAL:LIN:PATTERN:DATA:LENGTH command.

Query Syntax :SEARCH:SERIAL:LIN:PATTERN:DATA?

The :SEARCH:SERIAL:LIN:PATTERN:DATA? query returns the current data value setting.

Return Format <string><NL>

When :SEARCH:SERIAL:LIN:PATTERN:FORMAT DECIMAL,
<string> ::= "n" where n ::= 32-bit integer in unsigned decimal or
"\$" if data has any don't cares

When :SEARCH:SERIAL:LIN:PATTERN:FORMAT HEX,
<string> ::= "0xnn...n" where n ::= {0,...,9 | A,...,F | X }

- See Also**
- [Chapter 34](#), “:SEARCH Commands,” starting on page 1195
 - [":SEARCH:SERIAL:LIN:MODE"](#) on page 1263
 - [":SEARCH:SERIAL:LIN:PATTERN:FORMAT"](#) on page 1266
 - [":SEARCH:SERIAL:LIN:PATTERN:DATA:LENGTH"](#) on page 1265

:SEARCh:SERIal:LIN:PATTeRn:DATA:LENGTh

N (see [page 1650](#))

Command Syntax :SEARCh:SERIal:LIN:PATTeRn:DATA:LENGTh <length>

<length> ::= integer from 1 to 8 in NR1 format

The :SEARCh:SERIal:LIN:PATTeRn:DATA:LENGTh command specifies the the length of the data value when searching for LIN events.

The data value is specified using the :SEARCh:SERIal:LIN:PATTeRn:DATA command.

Query Syntax :SEARCh:SERIal:LIN:PATTeRn:DATA:LENGTh?

The :SEARCh:SERIal:LIN:PATTeRn:DATA:LENGTh? query returns the current data value length setting.

Return Format <length><NL>

<length> ::= integer from 1 to 8 in NR1 format

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:SERIal:LIN:PATTeRn:DATA"](#) on page 1264

:SEARCh:SERial:LIN:PATtern:FORMat

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:LIN:PATtern:FORMat <base>

<base> ::= {HEX | DECimal}

The :SEARCh:SERial:LIN:PATtern:FORMat command specifies the number base used with the :SEARCh:SERial:LIN:PATtern:DATA command.

Query Syntax :SEARCh:SERial:LIN:PATtern:FORMat?

The :SEARCh:SERial:LIN:PATtern:FORMat? query returns the current number base setting.

Return Format <base><NL>

<base> ::= {HEX | DEC}

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:SERial:LIN:PATtern:DATA"](#) on page 1264

:SEARCh:SERial:LIN:SYMBolic:FRAME

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:LIN:SYMBolic:FRAME <name>

<name> ::= quoted ASCII string

The :SEARCh:SERial:LIN:SYMBolic:FRAME command specifies the message to search for when LIN symbolic data has been loaded (recalled) into the oscilloscope and the LIN serial search mode is set to FRAME or FSIGNAL.

Query Syntax :SEARCh:SERial:LIN:SYMBolic:FRAME?

The :SEARCh:SERial:LIN:SYMBolic:FRAME? query returns the specified message.

Return Format <name><NL>

<name> ::= quotes ASCII string

- See Also**
- [":RECall:LDF\[:START\]"](#) on page 861
 - [":SEARCh:SERial:LIN:MODE"](#) on page 1263
 - [":SEARCh:SERial:LIN:SYMBolic:SIGNal"](#) on page 1268
 - [":SEARCh:SERial:LIN:SYMBolic:VALue"](#) on page 1269

:SEARCh:SERial:LIN:SYMBolic:SIGNa1

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:LIN:SYMBolic:SIGNa1 <name>

<name> ::= quoted ASCII string

The :SEARCh:SERial:LIN:SYMBolic:SIGNa1 command specifies the signal to search for when LIN symbolic data has been loaded (recalled) into the oscilloscope and the LIN serial search mode is set to FSiGna1.

Query Syntax :SEARCh:SERial:LIN:SYMBolic:SIGNa1?

The :SEARCh:SERial:LIN:SYMBolic:SIGNa1? query returns the specified signal.

Return Format <name><NL>

<name> ::= quoted ASCII string

- See Also**
- [":RECall:LDF\[:START\]"](#) on page 861
 - [":SEARCh:SERial:LIN:MODE"](#) on page 1263
 - [":SEARCh:SERial:LIN:SYMBolic:FRAMe"](#) on page 1267
 - [":SEARCh:SERial:LIN:SYMBolic:VALue"](#) on page 1269

:SEARCh:SERial:LIN:SYMBolic:VALue

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:LIN:SYMBolic:VALue <data>

<data> ::= value in NR3 format

The :SEARCh:SERial:LIN:SYMBolic:VALue command specifies the signal value to search for when LIN symbolic data has been loaded (recalled) into the oscilloscope and the LIN serial search mode is set to FSignal.

NOTE

Encoded signal values are not supported in the remote interface (even though they can be used in the front panel graphical interface).

Query Syntax :SEARCh:SERial:LIN:SYMBolic:VALue?

The :SEARCh:SERial:LIN:SYMBolic:VALue? query returns the specified signal value.

Return Format <data><NL>

<data> ::= value in NR3 format

- See Also**
- [":RECall:LDF\[:START\]"](#) on page 861
 - [":SEARCh:SERial:LIN:MODE"](#) on page 1263
 - [":SEARCh:SERial:LIN:SYMBolic:FRAMe"](#) on page 1267
 - [":SEARCh:SERial:LIN:SYMBolic:SIGNal"](#) on page 1268

:SEARCh:SERial:M1553 Commands

Table 148 :SEARCh:SERial:M1553 Commands Summary

Command	Query	Options and Query Returns
:SEARCh:SERial:M1553:MODE <value> (see page 1271)	:SEARCh:SERial:M1553:MODE? (see page 1271)	<value> ::= {DStArt CStArt RTA RTA11 PERRor SERRor MERRor}
:SEARCh:SERial:M1553:PATtern:DATA <string> (see page 1272)	:SEARCh:SERial:M1553:PATtern:DATA? (see page 1272)	<string> ::= "nn...n" where n ::= {0 1}
:SEARCh:SERial:M1553:RTA <value> (see page 1273)	:SEARCh:SERial:M1553:RTA? (see page 1273)	<value> ::= 5-bit integer in decimal, <hexadecimal>, <binary>, or <string> from 0-31 < hexadecimal > ::= #Hnn where n ::= {0,...,9 A,...,F} <binary> ::= #Bnn...n where n ::= {0 1} for binary <string> ::= "0xnn" where n ::= {0,...,9 A,...,F}

:SEARCh:SERial:M1553:MODE

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:M1553:MODE <value>

<value> ::= {DSTArt | CSTArt | RTA | RTA11 | PERRor | SERRor | MERRor}

The :SEARCh:SERial:M1553:MODE command selects the type of MIL-STD-1553 information to find in the Lister display:

- DSTArt – searches for the start of a Data word (at the end of a valid Data Sync pulse).
- CSTArt – searches for the start of a Command/Status word (at the end of a valid C/S Sync pulse).
- RTA – searches for the Remote Terminal Address (RTA) of a Command/Status word.
- RTA11 – searches for the Remote Terminal Address (RTA) and the additional 11 bits of a Command/Status word.
- PERRor – searches for (odd) parity errors for the data in the word.
- SERRor – searches for invalid Sync pulses.
- MERRor – searches for Manchester encoding errors.

In the RTA or RTA11 modes, the Remote Terminal Address is specified using the :SEARCh:SERial:M1553:RTA command.

In the RTA11 mode, the additional 11 bits are specified using the :SEARCh:SERial:M1553:PATtern:DATA command.

Query Syntax :SEARCh:SERial:M1553:MODE?

The :SEARCh:SERial:M1553:MODE? query returns the currently selected mode.

Return Format <value><NL>

<value> ::= {DSTA | CSTA | RTA | RTA11 | PERR | SERR | MERR}

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:SERial:M1553:RTA"](#) on page 1273
 - [":SEARCh:SERial:M1553:PATtern:DATA"](#) on page 1272

:SEARCh:SERial:M1553:PATtern:DATA

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:M1553:PATtern:DATA <string>

<string> ::= "nn...n" where n ::= {0 | 1}

The :SEARCh:SERial:M1553:PATtern:DATA command specifies the additional 11 bits when searching for the MIL-STD-1553 Remote Terminal Address + 11 Bits.

Query Syntax :SEARCh:SERial:M1553:PATtern:DATA?

The :SEARCh:SERial:M1553:PATtern:DATA? query returns the current value setting for the additional 11 bits.

Return Format <string><NL>

<string> ::= "nn...n" where n ::= {0 | 1}

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:SERial:M1553:MODE"](#) on page 1271

:SEARCh:SERial:M1553:RTA

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:M1553:RTA <value>

<value> ::= 5-bit integer in decimal, <hexadecimal>, <binary>, or <string> from 0-31

<hexadecimal> ::= #Hnn where n ::= {0,...,9|A,...,F}

<binary> ::= #Bnn...n where n ::= {0 | 1} for binary

<string> ::= "0xnn" where n ::= {0,...,9|A,...,F}

The :SEARCh:SERial:M1553:RTA command specifies the Remote Terminal Address (RTA) value when searching for MIL-STD-1553 events.

Query Syntax :SEARCh:SERial:M1553:RTA?

The :SEARCh:SERial:M1553:RTA? query returns the current Remote Terminal Address value setting.

Return Format <value><NL>

<value> ::= 5-bit integer in decimal from 0-31

See Also • [Chapter 34](#), “:SEARCh Commands,” starting on page 1195

:SEARCH:SERIAL:SENT Commands

Table 149 :SEARCH:SERIAL:SENT Commands Summary

Command	Query	Options and Query Returns
:SEARCH:SERIAL:SENT:F AST:DATA <string> (see page 1275)	:SEARCH:SERIAL:SENT:F AST:DATA? (see page 1275)	<string> ::= "0xn..." where n ::= {0,...,9 A,...,F X \$}
:SEARCH:SERIAL:SENT:M ODE <mode> (see page 1276)	:SEARCH:SERIAL:SENT:M ODE? (see page 1276)	<mode> ::= {FCData SCMid SCData CRCerror PPErrors}
:SEARCH:SERIAL:SENT:S LOW:DATA <data> (see page 1277)	:SEARCH:SERIAL:SENT:S LOW:DATA? (see page 1277)	<data> ::= from -1 (don't care) to 65535, in NR1 format.
:SEARCH:SERIAL:SENT:S LOW:ID <id> (see page 1278)	:SEARCH:SERIAL:SENT:S LOW:ID? (see page 1278)	<id> ::= from -1 (don't care) to 255, in NR1 format.

:SEARCh:SErIal:SENT:FAST:DATA

N (see [page 1650](#))

Command Syntax :SEARCh:SErIal:SENT:FAST:DATA <string>

<string> ::= "0xn..." where n ::= {0,...,9 | A,...,F | X | \$}

The :SEARCh:SErIal:SENT:FAST:DATA command specifies the status and data nibbles that will be searched for when the FCData search mode is chosen.

Query Syntax :SEARCh:SErIal:SENT:FAST:DATA?

The :SEARCh:SErIal:SENT:FAST:DATA? query returns the fast channel data search value setting.

Return Format <string><NL>

<string> ::= "0xn..." where n ::= {0,...,9 | A,...,F | X | \$}

- See Also**
- [":SEARCh:SErIal:SENT:MODE"](#) on page 1276
 - [":SEARCh:SErIal:SENT:SLOW:DATA"](#) on page 1277
 - [":SEARCh:SErIal:SENT:SLOW:ID"](#) on page 1278

:SEARCh:SERial:SENT:MODE

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:SENT:MODE <mode>

<mode> ::= {FCDData | SCMid | SCData | CRCerror | PPERror}

When SENT serial decode is turned on and displayed in the Lister, the :SEARCh:SERial:SENT:MODE command specifies what to search for in the decoded data:

- FCDData – finds Fast Channel data nibbles that match the values entered using additional softkeys.
- SCMid – finds Slow Channel Message IDs that match the value entered using additional softkeys.
- SCData – finds Slow Channel Message IDs and Data that match the values entered using additional softkeys.
- CRCerror – finds any CRC error, Fast or Slow.
- PPERror – finds where a nibble is either too wide or too narrow (for example, data nibble < 12 (11.5) or > 27 (27.5) ticks wide). Sync, S&C, data, or checksum pulse periods are checked.

Query Syntax :SEARCh:SERial:SENT:MODE?

The :SEARCh:SERial:SENT:MODE? query returns the search mode setting.

Return Format <mode><NL>

<mode> ::= {FCD | SCM | SCD | CRC | PPER}

- See Also**
- [":SEARCh:SERial:SENT:FAST:DATA"](#) on page 1275
 - [":SEARCh:SERial:SENT:SLOW:DATA"](#) on page 1277
 - [":SEARCh:SERial:SENT:SLOW:ID"](#) on page 1278

:SEARCh:SERIal:SENT:SLOW:DATA

N (see [page 1650](#))

Command Syntax :SEARCh:SERIal:SENT:SLOW:DATA <data>

<data> ::= from -1 (don't care) to 65535, in NR1 format.

The :SEARCh:SERIal:SENT:SLOW:DATA command specifies the data to search for in the Slow Channel Message ID and Data search mode.

Query Syntax :SEARCh:SERIal:SENT:SLOW:DATA?

The :SEARCh:SERIal:SENT:SLOW:DATA? query returns the slow channel data search value setting.

Return Format <data><NL>

<data> ::= from -1 (don't care) to 65535, in NR1 format.

- See Also**
- [":SEARCh:SERIal:SENT:FAST:DATA"](#) on page 1275
 - [":SEARCh:SERIal:SENT:MODE"](#) on page 1276
 - [":SEARCh:SERIal:SENT:SLOW:ID"](#) on page 1278

:SEARCh:SERIal:SENT:SLOW:ID

N (see [page 1650](#))

Command Syntax :SEARCh:SERIal:SENT:SLOW:ID <id>

<id> ::= from -1 (don't care) to 255, in NR1 format.

The :SEARCh:SERIal:SENT:SLOW:ID command specifies the ID to search for in the "Slow Channel Message ID" and "Slow Channel Message ID & Data" trigger modes. The ID can be from -1 (don't care) to 255 (depending on the message length).

Query Syntax :SEARCh:SERIal:SENT:SLOW:ID?

The :SEARCh:SERIal:SENT:SLOW:ID? query returns the slow channel ID search value setting.

Return Format <id><NL>

<id> ::= from -1 (don't care) to 255, in NR1 format.

- See Also**
- [":SEARCh:SERIal:SENT:FAST:DATA"](#) on page 1275
 - [":SEARCh:SERIal:SENT:MODE"](#) on page 1276
 - [":SEARCh:SERIal:SENT:SLOW:DATA"](#) on page 1277

:SEARCh:SERial:SPI Commands

Table 150 :SEARCh:SERial:SPI Commands Summary

Command	Query	Options and Query Returns
:SEARCh:SERial:SPI:MODE <value> (see page 1280)	:SEARCh:SERial:SPI:MODE? (see page 1280)	<value> ::= {MOSI MISO}
:SEARCh:SERial:SPI:PA TTern:DATA <string> (see page 1281)	:SEARCh:SERial:SPI:PA TTern:DATA? (see page 1281)	<string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X}
:SEARCh:SERial:SPI:PA TTern:WIDTh <width> (see page 1282)	:SEARCh:SERial:SPI:PA TTern:WIDTh? (see page 1282)	<width> ::= integer from 1 to 10

:SEARCh:SERial:SPI:MODE

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:SPI:MODE <value>

<value> ::= {MOSI | MISO}

The :SEARCh:SERial:SPI:MODE command specifies whether the SPI search will be on the MOSI data or the MISO data.

Data values are specified using the :SEARCh:SERial:SPI:PATtern:DATA command.

Query Syntax :SEARCh:SERial:SPI:MODE?

The :SEARCh:SERial:SPI:MODE? query returns the current SPI search mode setting.

Return Format <value><NL>

<value> ::= {MOSI | MISO}

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:SERial:SPI:PATtern:DATA"](#) on page 1281

:SEARCh:SERIal:SPI:PATtern:DATA

N (see [page 1650](#))

Command Syntax :SEARCh:SERIal:SPI:PATtern:DATA <string>

<string> ::= "0xnn...n" where n ::= {0,...,9 | A,...,F | X}

The :SEARCh:SERIal:SPI:PATtern:DATA command specifies the data value when searching for SPI events.

The width of the data value is specified using the :SEARCh:SERIal:SPI:PATtern:WIDTh command.

Query Syntax :SEARCh:SERIal:SPI:PATtern:DATA?

The :SEARCh:SERIal:SPI:PATtern:DATA? query returns the current data value setting.

Return Format <string><NL>

<string> ::= "0xnn...n" where n ::= {0,...,9 | A,...,F | X}

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:SERIal:SPI:PATtern:WIDTh"](#) on page 1282

:SEARCh:SERIal:SPI:PATtern:WIDTh

N (see [page 1650](#))

Command Syntax :SEARCh:SERIal:SPI:PATtern:WIDTh <width>

<width> ::= integer from 1 to 10

The :SEARCh:SERIal:SPI:PATtern:WIDTh command specifies the width of the data value (in bytes) when searching for SPI events.

The data value is specified using the :SEARCh:SERIal:SPI:PATtern:DATA command.

Query Syntax :SEARCh:SERIal:SPI:PATtern:WIDTh?

The :SEARCh:SERIal:SPI:PATtern:WIDTh? query returns the current data width setting.

Return Format <width><NL>

<width> ::= integer from 1 to 10

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:SERIal:SPI:PATtern:DATA"](#) on page 1281

:SEARCh:SERial:UART Commands

Table 151 :SEARCh:SERial:UART Commands Summary

Command	Query	Options and Query Returns
:SEARCh:SERial:UART:D ATA <value> (see page 1284)	:SEARCh:SERial:UART:D ATA? (see page 1284)	<value> ::= 8-bit integer from 0-255 (0x00-0xff) in decimal, <hexadecimal>, <binary>, or <quoted_string> format <hexadecimal> ::= #Hnn where n ::= {0,...,9 A,...,F} for hexadecimal <binary> ::= #Bnn...n where n ::= {0 1} for binary <quoted_string> ::= any of the 128 valid 7-bit ASCII characters (or standard abbreviations)
:SEARCh:SERial:UART:M ODE <value> (see page 1285)	:SEARCh:SERial:UART:M ODE? (see page 1285)	<value> ::= {RDATA RD1 RD0 RDX TDATA TD1 TD0 TDX PARityerror AERRor}
:SEARCh:SERial:UART:Q UALifier <value> (see page 1286)	:SEARCh:SERial:UART:Q UALifier? (see page 1286)	<value> ::= {EQUal NOTequal GREaterthan LESSthan}

:SEARCh:SERial:UART:DATA

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:UART:DATA <value>

<value> ::= 8-bit integer from 0-255 (0x00-0xff) in decimal,
 <hexadecimal>, <binary>, or <quoted_string> format

<hexadecimal> ::= #Hnn where n ::= {0,...,9 | A,...,F} for hexadecimal

<binary> ::= #Bnn...n where n ::= {0 | 1} for binary

<quoted_string> ::= any of the 128 valid 7-bit ASCII characters (or
 standard abbreviations)

The :SEARCh:SERial:UART:DATA command specifies a data value when searching for UART/RS232 events.

The data value qualifier is specified using the :SEARCh:SERial:UART:QUALifier command.

Query Syntax :SEARCh:SERial:UART:DATA?

The :SEARCh:SERial:UART:DATA? query returns the current data value setting.

Return Format <value><NL>

<value> ::= 8-bit integer from 0-255 (0x00-0xff) in decimal format

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:SERial:UART:MODE"](#) on page 1285
 - [":SEARCh:SERial:UART:QUALifier"](#) on page 1286

:SEARCh:SERial:UART:MODE

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:UART:MODE <value>

```
<value> ::= {RDATa | RD1 | RD0 | RDX | TDATa | TD1 | TD0 | TDX
             | PARityerror | AERRor}
```

The :SEARCh:SERial:UART:MODE command selects the type of UART/RS232 information to find in the Lister display:

- RDATa – searches for a receive data value when data words are from 5 to 8 bits long.
- RD1 – searches for a receive data value when data words are 9 bits long and the 9th (alert) bit is 1.
- RD0 – searches for a receive data value when data words are 9 bits long and the 9th (alert) bit is 0.
- RDX – searches for a receive data value when data words are 9 bits long and the 9th (alert) bit is a don't care (X).
- TDATa – searches for a transmit data value when data words are from 5 to 8 bits long.
- TD1 – searches for a transmit data value when data words are 9 bits long and the 9th (alert) bit is 1.
- TD0 – searches for a transmit data value when data words are 9 bits long and the 9th (alert) bit is 0.
- TDX – searches for a transmit data value when data words are 9 bits long and the 9th (alert) bit is a don't care (X).
- PARityerror – searches for parity errors.
- AERRor – searches for any error.

Data values are specified using the :SEARCh:SERial:UART:DATA command.

Data value qualifiers are specified using the :SEARCh:SERial:UART:QUALifier command.

Query Syntax :SEARCh:SERial:UART:MODE?

The :SEARCh:SERial:UART:MODE? query returns ...

Return Format <value><NL>

```
<value> ::= {RDAT | RD1 | RD0 | RDX | TDAT | TD1 | TD0 | TDX | PAR
             | AERR}
```

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:SERial:UART:DATA"](#) on page 1284
 - [":SEARCh:SERial:UART:QUALifier"](#) on page 1286

:SEARCh:SERial:UART:QUALifier

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:UART:QUALifier <value>

<value> ::= {EQUal | NOTequal | GREaterthan | LESSthan}

The :SEARCh:SERial:UART:QUALifier command specifies the data value qualifier when searching for UART/RS232 events.

Query Syntax :SEARCh:SERial:UART:QUALifier?

The :SEARCh:SERial:UART:QUALifier? query returns the current data value qualifier setting.

Return Format <value><NL>

<value> ::= {EQU | NOT | GRE | LESS}

- See Also**
- [Chapter 34](#), “:SEARCh Commands,” starting on page 1195
 - [":SEARCh:SERial:UART:DATA"](#) on page 1284

:SEARCh:SERial:USB Commands

Table 152 :SEARCh:SERial:USB Commands Summary

Command	Query	Options and Query Returns
:SEARCh:SERial:USB:MO DE <condition> (see page 1289)	:SEARCh:SERial:USB:MO DE? (see page 1289)	<event> ::= {Token DATA HANDshake SPECial ALLerrors PIDerror CRC5error CRC16error GLITCherror STUFFerror SE1error}
:SEARCh:SERial:USB:AD DRess <string> (see page 1290)	:SEARCh:SERial:USB:AD DRess? (see page 1290)	<string> ::= "0xnn" where n ::= {0,...,9 A,...,F X \$}
:SEARCh:SERial:USB:CR C <string> (see page 1291)	:SEARCh:SERial:USB:CR C? (see page 1291)	<string> ::= "0xnn" where n ::= {0,...,9 A,...,F X \$}
:SEARCh:SERial:USB:DA TA <string> (see page 1292)	:SEARCh:SERial:USB:DA TA? (see page 1292)	<string> ::= "0xn..." where n ::= {0,...,9 A,...,F X \$}
:SEARCh:SERial:USB:DA TA:LENGth <value> (see page 1293)	:SEARCh:SERial:USB:DA TA:LENGth? (see page 1293)	<length> ::= data length between 1-20
:SEARCh:SERial:USB:EN DPoint <string> (see page 1294)	:SEARCh:SERial:USB:EN DPoint? (see page 1294)	<string> ::= "0xn" where n ::= {0,...,9 A,...,F X \$}
:SEARCh:SERial:USB:ET <string> (see page 1295)	:SEARCh:SERial:USB:ET ? (see page 1295)	<string> ::= "0xn" where n ::= {0 1 2 3 X \$}
:SEARCh:SERial:USB:FR AMe <string> (see page 1296)	:SEARCh:SERial:USB:FR AMe? (see page 1296)	<string> ::= "0xnnn" where n ::= {0,...,9 A,...,F X \$}
:SEARCh:SERial:USB:HA DDress <string> (see page 1297)	:SEARCh:SERial:USB:HA DDress? (see page 1297)	<string> ::= "0xnn" where n ::= {0,...,9 A,...,F X \$}
:SEARCh:SERial:USB:PI D:DATA <pid> (see page 1298)	:SEARCh:SERial:USB:PI D:DATA? (see page 1298)	<pid> ::= {DATA0 DATA1 DATA2 MDATa}
:SEARCh:SERial:USB:PI D:HANDshake <pid> (see page 1299)	:SEARCh:SERial:USB:PI D:HANDshake? (see page 1299)	<pid> ::= {ACK NAK STAl1 NYET}
:SEARCh:SERial:USB:PI D:SPECial <pid> (see page 1300)	:SEARCh:SERial:USB:PI D:SPECial? (see page 1300)	<pid> ::= {PING PRE ERR SPLit}

Table 152 :SEARCh:SERIal:USB Commands Summary (continued)

Command	Query	Options and Query Returns
:SEARCh:SERIal:USB:PID:TOKE n <pid> (see page 1301)	:SEARCh:SERIal:USB:PID:TOKE n? (see page 1301)	<pid> ::= {OUT IN SETup SOF}
:SEARCh:SERIal:USB:PORT <string> (see page 1302)	:SEARCh:SERIal:USB:PORT? (see page 1302)	<string> ::= "0xnn" where n ::= {0,...,9 A,...,F X \$}
:SEARCh:SERIal:USB:SC <string> (see page 1303)	:SEARCh:SERIal:USB:SC ? (see page 1303)	<string> ::= "0xn" where n ::= {0 1 X \$}
:SEARCh:SERIal:USB:SEU <string> (see page 1304)	:SEARCh:SERIal:USB:SEU? (see page 1304)	<string> ::= "0xn" where n ::= {0 1 2 3 X \$}

:SEARCh:SERial:USB:MODE

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:USB:MODE <mode>

```
<mode> ::= {TOKen | DATA | HANDshake | SPECial | ALLerrors
            | PIDerror | CRC5error | CRC16error | GLITCherror
            | STUFFerror | SE1error}
```

The :SEARCh:SERial:USB:MODE command specifies the USB search mode.

Query Syntax :SEARCh:SERial:USB:MODE?

The :SEARCh:SERial:USB:MODE? query returns the specified USB search mode.

Return Format <mode><NL>

```
<mode> ::= {TOK | DATA | HAND | SPEC | ALL | PID | CRC5
            | CRC16 | GLIT | STUFF | SE1}
```

- See Also**
- [":SEARCh:SERial:USB:PID:DATA"](#) on page 1298
 - [":SEARCh:SERial:USB:PID:HANDshake"](#) on page 1299
 - [":SEARCh:SERial:USB:PID:SPECial"](#) on page 1300
 - [":SEARCh:SERial:USB:PID:TOKen"](#) on page 1301

:SEARCh:SERial:USB:ADDReSS

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:USB:ADDReSS <string>

<string> ::= "0xnn" where n ::= {0,...,9 | A,...,F | X | \$}

The :SEARCh:SERial:USB:ADDReSS command specifies the 7-bit Address portion of the search value, in hex.

Query Syntax :SEARCh:SERial:USB:ADDReSS?

The :SEARCh:SERial:USB:ADDReSS? query returns the specified Address portion of the search value.

Return Format <string><NL>

<string> ::= "0xnn" where n ::= {0,...,9 | A,...,F | X | \$}

- See Also**
- [":SEARCh:SERial:USB:PID:SPECial"](#) on page 1300
 - [":SEARCh:SERial:USB:PID:TOKen"](#) on page 1301

:SEARCh:SERial:USB:CRC

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:USB:CRC <string>

<string> ::= "0xnn" where n ::= {0,...,9 | A,...,F | X | \$}

The :SEARCh:SERial:USB:CRC command specifies the CRC portion of the search value, in hex.

Query Syntax :SEARCh:SERial:USB:CRC?

The :SEARCh:SERial:USB:CRC? query returns the specified CRC portion of the search value.

Return Format <string><NL>

<string> ::= "0xnn" where n ::= {0,...,9 | A,...,F | X | \$}

- See Also**
- [":SEARCh:SERial:USB:PID:SPECial"](#) on page 1300
 - [":SEARCh:SERial:USB:PID:TOKen"](#) on page 1301

:SEARCh:SERial:USB:DATA

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:USB:DATA <string>

<string> ::= "0xn..." where n ::= {0,...,9 | A,...,F | X | \$}

The :SEARCh:SERial:USB:DATA command specifies the Data portion of the search value, in hex.

See the :SEARCh:SERial:USB:DATA:LENGth command for setting the length of the data value.

Query Syntax :SEARCh:SERial:USB:DATA?

The :SEARCh:SERial:USB:DATA? query returns the specified Data portion of the search value.

Return Format <string><NL>

<string> ::= "0xn..." where n ::= {0,...,9 | A,...,F | X | \$}

See Also • [":SEARCh:SERial:USB:DATA:LENGth"](#) on page 1293

:SEARCh:SERial:USB:DATA:LENGth

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:USB:DATA:LENGth <length>

<length> ::= data length between 1-20

The :SEARCh:SERial:USB:DATA:LENGth command specifies the data length in bytes.

Query Syntax :SEARCh:SERial:USB:DATA:LENGth?

The :SEARCh:SERial:USB:DATA:LENGth? query returns the specified data length.

Return Format <length><NL>

<length> ::= data length between 1-20

See Also • [":SEARCh:SERial:USB:DATA"](#) on page 1292

:SEARCh:SERial:USB:ENDPoint

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:USB:ENDPoint <string>

<string> ::= "0xn" where n ::= {0,...,9 | A,...,F | X | \$}

The :SEARCh:SERial:USB:ENDPoint command specifies the 4-bit Endpoint portion of the search value, in hex.

Query Syntax :SEARCh:SERial:USB:ENDPoint?

The :SEARCh:SERial:USB:ENDPoint? query returns the specified Endpoint portion of the search value.

Return Format <string><NL>

<string> ::= "0xn" where n ::= {0,...,9 | A,...,F | X | \$}

- See Also**
- [":SEARCh:SERial:USB:PID:SPECial"](#) on page 1300
 - [":SEARCh:SERial:USB:PID:TOKen"](#) on page 1301

:SEARCh:SERial:USB:ET

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:USB:ET <string>

<string> ::= "0xn" where n ::= {0 | 1 | 2 | 3 | X | \$}

The :SEARCh:SERial:USB:ET command specifies the 2-bit ET portion of the search value, in hex.

Query Syntax :SEARCh:SERial:USB:ET?

The :SEARCh:SERial:USB:ET? query returns the specified ET portion of the search value.

Return Format <string><NL>

<string> ::= "0xn" where n ::= {0 | 1 | 2 | 3 | X | \$}

See Also • [":SEARCh:SERial:USB:PID:SPECial"](#) on page 1300

:SEARCh:SERial:USB:FRAMe

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:USB:FRAMe <string>

<string> ::= "0xnnn" where n ::= {0,...,9 | A,...,F | X | \$}

The :SEARCh:SERial:USB:FRAMe command specifies the 11-bit Frame portion of the search value, in hex.

Query Syntax :SEARCh:SERial:USB:FRAMe?

The :SEARCh:SERial:USB:FRAMe? query returns the specified Frame portion of the search value.

Return Format <string><NL>

<string> ::= "0xnnn" where n ::= {0,...,9 | A,...,F | X | \$}

See Also • [":SEARCh:SERial:USB:PID:TOKen"](#) on page 1301

:SEARCh:SERIal:USB:HADDress

N (see [page 1650](#))

Command Syntax :SEARCh:SERIal:USB:HADDress <string>

<string> ::= "0xnn" where n ::= {0,...,9 | A,...,F | X | \$}

The :SEARCh:SERIal:USB:HADDress command specifies the 7-bit Hub Address portion of the search value, in hex.

Query Syntax :SEARCh:SERIal:USB:HADDress?

The :SEARCh:SERIal:USB:HADDress? query returns the specified Hub Address portion of the search value.

Return Format <string><NL>

<string> ::= "0xnn" where n ::= {0,...,9 | A,...,F | X | \$}

See Also • [":SEARCh:SERIal:USB:PID:SPECial"](#) on page 1300

:SEARCh:SERial:USB:PID:DATA

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:USB:PID:DATA <pid>

<pid> ::= {DATA0 | DATA1 | DATA2 | MDATA}

The :SEARCh:SERial:USB:PID:DATA command specifies the USB data PID to find.

The DATA USB search mode can be selected using the :SEARCh:SERial:USB:MODE command.

Query Syntax :SEARCh:SERial:USB:PID:DATA?

The :SEARCh:SERial:USB:PID:DATA? query returns the specified data PID.

Return Format <pid><NL>

<pid> ::= {DATA0 | DATA1 | DATA2 | MDATA}

See Also • [":SEARCh:SERial:USB:MODE"](#) on page 1289

:SEARCh:SERial:USB:PID:HANDshake

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:USB:PID:HANDshake <pid>

<pid> ::= {ACK | NAK | STAl1 | NYET}

The :SEARCh:SERial:USB:PID:HANDshake command specifies the USB handshake PID to find.

The HANDshake USB search mode can be selected using the :SEARCh:SERial:USB:MODE command.

Query Syntax :SEARCh:SERial:USB:PID:HANDshake?

The :SEARCh:SERial:USB:PID:HANDshake? query returns the specified handshake PID.

Return Format <pid><NL>

<pid> ::= {ACK | NAK | STAl | NYET}

See Also • [":SEARCh:SERial:USB:MODE"](#) on page 1289

:SEARCh:SERial:USB:PID:SPECial

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:USB:PID:SPECial <pid>

<pid> ::= {PING | PRE | ERR | SPLit}

The :SEARCh:SERial:USB:PID:SPECial command specifies the USB special PID to find.

The SPECial USB search mode can be selected using the :SEARCh:SERial:USB:MODE command.

Query Syntax :SEARCh:SERial:USB:PID:SPECial?

The :SEARCh:SERial:USB:PID:SPECial? query returns the specified special PID.

Return Format <pid><NL>

<pid> ::= {PING | PRE | ERR | SPL}

See Also • [":SEARCh:SERial:USB:MODE"](#) on page 1289

:SEARCh:SERial:USB:PID:TOKen

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:USB:PID:TOKen <pid>

<pid> ::= {OUT | IN | SETup | SOF}

The :SEARCh:SERial:USB:PID:TOKen command specifies the USB token PID to find.

The TOKen USB search mode can be selected using the :SEARCh:SERial:USB:MODE command.

Query Syntax :SEARCh:SERial:USB:PID:TOKen?

The :SEARCh:SERial:USB:PID:TOKen? query returns the specified token PID.

Return Format <pid><NL>

<pid> ::= {OUT | IN | SETup | SOF}

See Also • [":SEARCh:SERial:USB:MODE"](#) on page 1289

:SEARCh:SERial:USB:PORT

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:USB:PORT <string>

<string> ::= "0xnn" where n ::= {0,...,9 | A,...,F | X | \$}

The :SEARCh:SERial:USB:PORT command specifies the 7-bit Port portion of the search value, in hex.

Query Syntax :SEARCh:SERial:USB:PORT?

The :SEARCh:SERial:USB:PORT? query returns the specified Port portion of the search value.

Return Format <string><NL>

<string> ::= "0xnn" where n ::= {0,...,9 | A,...,F | X | \$}

See Also • [":SEARCh:SERial:USB:PID:SPECial"](#) on page 1300

:SEARCh:SERial:USB:SC

N (see [page 1650](#))

Command SyntaxS :SEARCh:SERial:USB:SC <string>

<string> ::= "0xn" where n ::= {0 | 1 | X | \$}

The :SEARCh:SERial:USB:SC command specifies the 1-bit SC portion of the search value, in hex.

Query Syntax :SEARCh:SERial:USB:SC?

The :SEARCh:SERial:USB:SC? query returns the specified SC portion of the search value.

Return Format <string><NL>

<string> ::= "0xn" where n ::= {0 | 1 | X | \$}

See Also • [":SEARCh:SERial:USB:PID:SPECial"](#) on page 1300

:SEARCh:SERial:USB:SEU

N (see [page 1650](#))

Command Syntax :SEARCh:SERial:USB:SEU <string>

<string> ::= "0xn" where n ::= {0 | 1 | 2 | 3 | X | \$}

The :SEARCh:SERial:USB:SEU command specifies the 2-bit S and E or U portion of the search value, in hex.

Query Syntax :SEARCh:SERial:USB:SEU?

The :SEARCh:SERial:USB:SEU? query returns the specified S and E or U portion of the search value.

Return Format <string><NL>

<string> ::= "0xn" where n ::= {0 | 1 | 2 | 3 | X | \$}

See Also • [":SEARCh:SERial:USB:PID:SPECial"](#) on page 1300

35 :SYSTem Commands

Control basic system functions of the oscilloscope. See ["Introduction to :SYSTem Commands"](#) on page 1307.

Table 153 :SYSTem Commands Summary

Command	Query	Options and Query Returns
:SYSTem:DATE <date> (see page 1308)	:SYSTem:DATE? (see page 1308)	<date> ::= <year>,<month>,<day> <year> ::= 4-digit year in NR1 format <month> ::= {1,...,12 JANuary FEBruary MARch APRil MAY JUNe JULy AUGust SEPtember OCTober NOVember DECember} <day> ::= {1,...31}
n/a	:SYSTem:DIDentifier? (see page 1309)	n/a
:SYSTem:DSP <string> (see page 1310)	n/a	<string> ::= up to 75 characters as a quoted ASCII string
n/a	:SYSTem:ERRor? (see page 1311)	<error> ::= an integer error code <error string> ::= quoted ASCII string. See Error Messages (see page 1589) .
:SYSTem:LOCK <value> (see page 1312)	:SYSTem:LOCK? (see page 1312)	<value> ::= {{1 ON} {0 OFF}}
:SYSTem:PERSonA[:MANu facturer] <manufacturer_string> (see page 1313)	:SYSTem:PERSonA[:MANu facturer]? (see page 1313)	<manufacturer_string> ::= quoted ASCII string, up to 63 characters
:SYSTem:PERSonA[:MANu facturer]:DEFault (see page 1314)	n/a	Sets manufacturer string to "KEYSIGHT TECHNOLOGIES"

Table 153 :SYSTem Commands Summary (continued)

Command	Query	Options and Query Returns
:SYSTem:PRESet (see page 1315)	n/a	See :SYSTem:PRESet (see page 1315)
:SYSTem:PROTection:LOCK <value> (see page 1318)	:SYSTem:PROTection:LOCK? (see page 1318)	<value> ::= {{1 ON} {0 OFF}}
:SYSTem:RLOGger <setting>[,<file_name>[,<write_mode>]] (see page 1319)	n/a	<setting> ::= {{0 OFF} {1 ON}} <file_name> ::= quoted ASCII string <write_mode> ::= {CREate APPend}
:SYSTem:RLOGger:DESTination <dest> (see page 1320)	:SYSTem:RLOGger:DESTination? (see page 1320)	<dest> ::= {FILE SCReen BOTH}
:SYSTem:RLOGger:DISPlay {{0 OFF} {1 ON}} (see page 1321)	:SYSTem:RLOGger:DISPlay? (see page 1321)	<setting> ::= {0 1}
:SYSTem:RLOGger:FNAME <file_name> (see page 1322)	:SYSTem:RLOGger:FNAME? (see page 1322)	<file_name> ::= quoted ASCII string
:SYSTem:RLOGger:STATe {{0 OFF} {1 ON}} (see page 1323)	:SYSTem:RLOGger:STATe? (see page 1323)	<setting> ::= {0 1}
:SYSTem:RLOGger:TRANsparent {{0 OFF} {1 ON}} (see page 1324)	:SYSTem:RLOGger:TRANsparent? (see page 1324)	<setting> ::= {0 1}
:SYSTem:RLOGger:WMODE <write_mode> (see page 1325)	:SYSTem:RLOGger:WMODE? (see page 1325)	<write_mode> ::= {CREate APPend}
:SYSTem:SETup <setup_data> (see page 1326)	:SYSTem:SETup? (see page 1326)	<setup_data> ::= data in IEEE 488.2 # format.
:SYSTem:TIME <time> (see page 1328)	:SYSTem:TIME? (see page 1328)	<time> ::= hours, minutes, seconds in NR1 format
:SYSTem:TOUCh {{1 ON} {0 OFF}} (see page 1329)	:SYSTem:TOUCh? (see page 1329)	{1 0}

**Introduction to
:SYSTem
Commands** SYSTem subsystem commands enable writing messages to the display, setting and reading both the time and the date, querying for errors, and saving and recalling setups.

:SYSTem:DATE

N (see [page 1650](#))

Command Syntax :SYSTem:DATE <date>

<date> ::= <year>,<month>,<day>

<year> ::= 4-digit year in NR1 format

<month> ::= {1,...,12 | JANuary | FEBruary | MARCH | APRil | MAY | JUNE
| JULy | AUGust | SEPTember | OCTober | NOVember | DECember}

<day> ::= {1,...,31}

The :SYSTem:DATE command sets the date. Validity checking is performed to ensure that the date is valid.

Query Syntax :SYSTem:DATE?

The SYSTem:DATE? query returns the date.

Return Format <year>,<month>,<day><NL>

- See Also**
- ["Introduction to :SYSTem Commands"](#) on page 1307
 - [":SYSTem:TIME"](#) on page 1328

:SYSTem:DIIdentifier

N (see [page 1650](#))

Query Syntax :SYSTem:DIIdentifier?

The :SYSTem:DIIdentifier? query returns the oscilloscope's Host ID as (part of) a quoted string.

The oscilloscope's Host ID is needed when redeeming licenses for oscilloscope upgrades or other licensed features.

The exact format of returned string are product-specific. This example returns the model number, serial number, and host ID in a comma-separated format:

```
"X12345A,US12345678,G1EFDNLPAF2YPLRN"
```

Portable programs should not attempt to parse the contents of the returned string. Use the *IDN? query instead to get the model number and/or serial number in a defined portable format.

Return Format <host_id><NL>

```
<host_id> ::= quoted ASCII string
```

See Also • ["*IDN \(Identification Number\)"](#) on page 220

:SYSTem:DSP

N (see [page 1650](#))

Command Syntax :SYSTem:DSP <string>

<string> ::= quoted ASCII string (up to 75 characters)

The :SYSTem:DSP command writes the quoted string (excluding quotation marks) to a text box on-screen.

Use :SYSTem:DSP "" to remotely remove the message from the display. (Two sets of quote marks without a space between them creates a NULL string.)

Press any menu key to manually remove the message from the display.

See Also • ["Introduction to :SYSTem Commands"](#) on page 1307

:SYSTem:ERRor

C (see [page 1650](#))

Query Syntax :SYSTem:ERRor?

The :SYSTem:ERRor? query outputs the next error number and text from the error queue. The instrument has an error queue that is 30 errors deep and operates on a first-in, first-out basis. Repeatedly sending the :SYSTem:ERRor? query returns the errors in the order that they occurred until the queue is empty. Any further queries then return zero until another error occurs.

When remote logging is enabled (using the oscilloscope's front panel), additional debug information can be included in the returned error string. If the error is detected by the SCPI command parser, such as a header error or other syntax error, the extra debug information is generated and included. But if the error is detected by the oscilloscope system, such as when an out-of-range value is sent, then no extra debug information is included.

Return Format

<error number>,<error string><NL>

<error number> ::= an integer error code in NR1 format

<error string> ::= quoted ASCII string containing the error message

Error messages are listed in [Chapter 42](#), “Error Messages,” starting on page 1589.

See Also

- ["Introduction to :SYSTem Commands"](#) on page 1307
- ["*ESR \(Standard Event Status Register\)"](#) on page 218
- ["*CLS \(Clear Status\)"](#) on page 215

:SYSTem:LOCK

N (see [page 1650](#))

Command Syntax :SYSTem:LOCK <on_off>

<on_off> ::= {{1 | ON} | {0 | OFF}}

The :SYSTem:LOCK command disables the front panel. LOCK ON is the equivalent of sending a local lockout message over the programming interface.

Query Syntax :SYSTem:LOCK?

The :SYSTem:LOCK? query returns the lock status of the front panel.

Return Format <on_off><NL>

<on_off> ::= {1 | 0}

See Also • ["Introduction to :SYSTem Commands"](#) on page 1307

:SYSTem:PERSonA[:MANufacturer]

N (see [page 1650](#))

Command Syntax :SYSTem:PERSonA[:MANufacturer] <manufacturer_string>
 <manufacturer_string> ::= ::= quoted ASCII string, up to 63 characters

The :SYSTem:PERSonA[:MANufacturer] command lets you change the manufacturer string portion of the identification string returned by the *IDN? query.

The default manufacturer string is "KEYSIGHT TECHNOLOGIES".

If your remote programs depend on a legacy manufacturer string, for example, you could use this command to set the manufacturer string to "AGILENT TECHNOLOGIES".

Query Syntax :SYSTem:PERSonA[:MANufacturer]?

The :SYSTem:PERSonA[:MANufacturer]? query returns the currently set manufacturer string.

Return Format <manufacturer_string><NL>

- See Also**
- ["*IDN \(Identification Number\)"](#) on page 220
 - [":SYSTem:PERSonA\[:MANufacturer\]:DEFault"](#) on page 1314
 - ["Introduction to :SYSTem Commands"](#) on page 1307

:SYSTem:PERSonA[:MANufacturer]:DEFault

N (see [page 1650](#))

Command Syntax :SYSTem:PERSonA[:MANufacturer]:DEFault

The :SYSTem:PERSonA[:MANufacturer]:DEFault command sets the manufacturer string to "KEYSIGHT TECHNOLOGIES".

- See Also**
- ["*IDN \(Identification Number\)"](#) on page 220
 - [":SYSTem:PERSonA\[:MANufacturer\]"](#) on page 1313
 - ["Introduction to :SYSTem Commands"](#) on page 1307

:SYSTem:PRESet

C (see [page 1650](#))

Command Syntax :SYSTem:PRESet

The :SYSTem:PRESet command places the instrument in a known state. This is the same as pressing the **[Default Setup]** key or **[Save/Recall] > Default/Erase > Default Setup** on the front panel.

When you perform a default setup, some user settings (like preferences) remain unchanged. To reset all user settings to their factory defaults, use the *RST command.

Reset conditions are:

Acquire Menu	
Mode	Normal
Averaging	Off
# Averages	8

Analog Channel Menu	
Channel 1	On
Channel 2	Off
Volts/division	5.00 V
Offset	0.00
Coupling	DC
Probe attenuation	10:1
Vernier	Off
Invert	Off
BW limit	Off
Impedance	1 M Ohm (cannot be changed)
Units	Volts
Skew	0

Cursor Menu	
Source	Channel 1

Digital Channel Menu (MSO models only)	
Channel 0 - 7	Off
Labels	Off
Threshold	TTL (1.4 V)

Display Menu	
Persistence	Off
Grid	20%

Quick Meas Menu	
Source	Channel 1

Run Control	
	Scope is running

Time Base Menu	
Main time/division	100 us
Main time base delay	0.00 s
Delay time/division	500 ns
Delay time base delay	0.00 s
Reference	center
Mode	main
Vernier	Off

Trigger Menu	
Type	Edge
Mode	Auto
Coupling	dc
Source	Channel 1
Level	0.0 V
Slope	Positive

Trigger Menu	
HF Reject and noise reject	Off
Holdoff	40 ns
External probe attenuation	10:1
External Units	Volts
External Impedance	1 M Ohm (cannot be changed)

- See Also
- **"Introduction to Common (*) Commands"** on page 213
 - **"*RST (Reset)"** on page 226

:SYSTem:PROTection:LOCK

N (see [page 1650](#))

Command Syntax :SYSTem:PROTection:LOCK <on_off>
 <on_off> ::= {{1 | ON} | {0 | OFF}}

The :SYSTem:PROTection:LOCK command disables the fifty ohm impedance setting for all analog channels.

NOTE

Be careful when turning ON the :SYSTem:PROTection:LOCK because there is no visual indication of this setting in the front-panel user interface (other than a disabled 50 Ω channel input impedance selection), and a user could think the oscilloscope is not working properly.

Query Syntax :SYSTem:PROTection:LOCK?

The :SYSTem:PROTection:LOCK? query returns the analog channel protection lock status.

Return Format <on_off><NL>
 <on_off> ::= {1 | 0}

See Also • ["Introduction to :SYSTem Commands"](#) on page 1307

:SYSTem:RLOGger

N (see [page 1650](#))

Command Syntax :SYSTem:RLOGger <setting>[,<file_name>[,<write_mode>]]

<setting> ::= {{0 | OFF} | {1 | ON}}

<file_name> ::= quoted ASCII string

<write_mode> ::= {CREate | APPend}

The :SYSTem:RLOGger command enables or disables remote command logging, optionally specifying the log file name and write mode.

- See Also**
- [":SYSTem:RLOGger:DESTination"](#) on page 1320
 - [":SYSTem:RLOGger:DISPlay"](#) on page 1321
 - [":SYSTem:RLOGger:FNAME"](#) on page 1322
 - [":SYSTem:RLOGger:STATE"](#) on page 1323
 - [":SYSTem:RLOGger:TRANSPARENT"](#) on page 1324
 - [":SYSTem:RLOGger:WMODE"](#) on page 1325

:SYSTem:RLOGger:DESTination

N (see [page 1650](#))

Command Syntax :SYSTem:RLOGger:DESTination <dest>

<dest> ::= {FILE | SCReen | BOTH}

The :SYSTem:RLOGger:DESTination command specifies whether remote commands are logged to a text file (on a connected USB storage device), logged to the screen, or both.

NOTE

If the destination is changed while remote command logging is running, remote command logging is turned off.

Query Syntax :SYSTem:RLOGger:DESTination?

The :SYSTem:RLOGger:DESTination? query returns the remote command logging destination.

Return Format <dest><NL>

<dest> ::= {FILE | SCR | BOTH}

- See Also**
- [":SYSTem:RLOGger"](#) on page 1319
 - [":SYSTem:RLOGger:DISPlay"](#) on page 1321
 - [":SYSTem:RLOGger:FNAME"](#) on page 1322
 - [":SYSTem:RLOGger:STAtE"](#) on page 1323
 - [":SYSTem:RLOGger:TRANSPARENT"](#) on page 1324
 - [":SYSTem:RLOGger:WMODe"](#) on page 1325

:SYSTem:RLOGger:DISPlay

N (see [page 1650](#))

Command Syntax :SYSTem:RLOGger:DISPlay {{0 | OFF} | {1 | ON}}

The :SYSTem:RLOGger:DISPlay command enables or disables the screen display of logged remote commands and their return values (if applicable).

Query Syntax :SYSTem:RLOGger:DISPlay?

The :SYSTem:RLOGger:DISPlay? query returns whether the screen display for remote command logging is enabled or disabled.

Return Format <setting><NL>
<setting> ::= {0 | 1}

- See Also**
- [":SYSTem:RLOGger"](#) on page 1319
 - [":SYSTem:RLOGger:DESTination"](#) on page 1320
 - [":SYSTem:RLOGger:FNAME"](#) on page 1322
 - [":SYSTem:RLOGger:STATE"](#) on page 1323
 - [":SYSTem:RLOGger:TRANSPARENT"](#) on page 1324
 - [":SYSTem:RLOGger:WMODE"](#) on page 1325

:SYSTem:RLOGger:FNAME

N (see [page 1650](#))

Command Syntax :SYSTem:RLOGger:FNAME <file_name>

<file_name> ::= quoted ASCII string

The :SYSTem:RLOGger:FNAME command specifies the remote command log file name.

Because log files are ASCII text files, the ".txt" extension is automatically added to the name specified.

Query Syntax :SYSTem:RLOGger:FNAME?

The :SYSTem:RLOGger:FNAME? query returns the remote command log file name.

Return Format <file_name><NL>

- See Also**
- [":SYSTem:RLOGger"](#) on page 1319
 - [":SYSTem:RLOGger:DESTination"](#) on page 1320
 - [":SYSTem:RLOGger:DISPlay"](#) on page 1321
 - [":SYSTem:RLOGger:STAtE"](#) on page 1323
 - [":SYSTem:RLOGger:TRANSPARENT"](#) on page 1324
 - [":SYSTem:RLOGger:WMODE"](#) on page 1325

:SYSTem:RLOGger:STATe

N (see [page 1650](#))

Command Syntax :SYSTem:RLOGger:STATe {{0 | OFF} | {1 | ON}}

The :SYSTem:RLOGger:STATe command enables or disables remote command logging.

Query Syntax :SYSTem:RLOGger:STATe?

The :SYSTem:RLOGger:STATe? query returns the remote command logging state.

Return Format <setting><NL>

<setting> ::= {0 | 1}

- See Also**
- [":SYSTem:RLOGger"](#) on page 1319
 - [":SYSTem:RLOGger:DESTination"](#) on page 1320
 - [":SYSTem:RLOGger:DISPlay"](#) on page 1321
 - [":SYSTem:RLOGger:FNAME"](#) on page 1322
 - [":SYSTem:RLOGger:TRANsparent"](#) on page 1324
 - [":SYSTem:RLOGger:WMODE"](#) on page 1325

:SYSTem:RLOGger:TRANSPARENT

N (see [page 1650](#))

Command Syntax :SYSTem:RLOGger:TRANSPARENT {{0 | OFF} | {1 | ON}}

The :SYSTem:RLOGger:TRANSPARENT command specifies whether the screen display background for remote command logging is transparent or solid.

Query Syntax :SYSTem:RLOGger:TRANSPARENT?

The :SYSTem:RLOGger:TRANSPARENT? query returns the setting for transparent screen display background.

Return Format <setting><NL>
<setting> ::= {0 | 1}

- See Also**
- [":SYSTem:RLOGger"](#) on page 1319
 - [":SYSTem:RLOGger:DESTination"](#) on page 1320
 - [":SYSTem:RLOGger:DISPlay"](#) on page 1321
 - [":SYSTem:RLOGger:FNAME"](#) on page 1322
 - [":SYSTem:RLOGger:STATE"](#) on page 1323
 - [":SYSTem:RLOGger:WMODE"](#) on page 1325

:SYSTem:RLOGger:WMODe

N (see [page 1650](#))

Command Syntax :SYSTem:RLOGger:WMODe <write_mode>

<write_mode> ::= {CREate | APPend}

The :SYSTem:RLOGger:WMODe command specifies the remote command logging write mode.

Query Syntax :SYSTem:RLOGger:WMODe?

The :SYSTem:RLOGger:WMODe? query returns the remote command logging write mode.

Return Format <write_mode><NL>

<write_mode> ::= {CRE | APP}

- See Also**
- [":SYSTem:RLOGger"](#) on page 1319
 - [":SYSTem:RLOGger:DESTination"](#) on page 1320
 - [":SYSTem:RLOGger:DISPlay"](#) on page 1321
 - [":SYSTem:RLOGger:FNAME"](#) on page 1322
 - [":SYSTem:RLOGger:STAtE"](#) on page 1323
 - [":SYSTem:RLOGger:TRANSPARENT"](#) on page 1324

:SYSTem:SETup

C (see [page 1650](#))

Command Syntax :SYSTem:SETup <setup_data>

<setup_data> ::= binary block data in IEEE 488.2 # format.

The :SYSTem:SETup command sets the oscilloscope as defined by the data in the setup (learn) string sent from the controller. The setup string does not change the interface mode or interface address.

Query Syntax :SYSTem:SETup?

The :SYSTem:SETup? query operates the same as the *LRN? query. It outputs the current oscilloscope setup in the form of a learn string to the controller. The setup (learn) string is sent and received as a binary block of data. The format for the data transmission is the # format defined in the IEEE 488.2 specification.

Return Format <setup_data><NL>

<setup_data> ::= binary block data in IEEE 488.2 # format

- See Also**
- ["Introduction to :SYSTem Commands"](#) on page 1307
 - ["*LRN \(Learn Device Setup\)"](#) on page 221

Example Code

```
' SAVE_SYSTEM_SETUP - The :SYSTEM:SETUP? query returns a program
' message that contains the current state of the instrument. Its
' format is a definite-length binary block, for example,
' #800075595<setup string><NL>
' where the setup string is 75595 bytes in length.
myScope.WriteString ":SYSTEM:SETUP?"
varQueryResult = myScope.ReadIEEEBlock(BinaryType_UI1)
CheckForInstrumentErrors ' After reading query results.

' Output setup string to a file:
Dim strPath As String
strPath = "c:\scope\config\setup.dat"

' Open file for output.
Close #1 ' If #1 is open, close it.
Open strPath For Binary Access Write Lock Write As #1
Put #1, , varQueryResult ' Write data.
Close #1 ' Close file.

' RESTORE_SYSTEM_SETUP - Read the setup string from a file and
' write it back to the oscilloscope.
Dim varSetupString As Variant
strPath = "c:\scope\config\setup.dat"

' Open file for input.
Open strPath For Binary Access Read As #1
Get #1, , varSetupString ' Read data.
Close #1 ' Close file.
```

```
' Write setup string back to oscilloscope using ":SYSTEM:SETUP"  
' command:  
myScope.WriteIEEEBlock ":SYSTEM:SETUP ", varSetupString  
CheckForInstrumentErrors
```

See complete example programs at: [Chapter 46](#), “Programming Examples,” starting on page 1659

:SYSTem:TIME

N (see [page 1650](#))

Command Syntax :SYSTem:TIME <time>

<time> ::= hours,minutes,seconds in NR1 format

The :SYSTem:TIME command sets the system time, using a 24-hour format. Commas are used as separators. Validity checking is performed to ensure that the time is valid.

Query Syntax :SYSTem:TIME? <time>

The :SYSTem:TIME? query returns the current system time.

Return Format <time><NL>

<time> ::= hours,minutes,seconds in NR1 format

- See Also**
- ["Introduction to :SYSTem Commands"](#) on page 1307
 - [":SYSTem:DATE"](#) on page 1308

:SYSTem:TOUCh

N (see [page 1650](#))

Command Syntax :SYSTem:TOUCh <on_off>

<on_off> ::= {{1 | ON} | {0 | OFF}}

The :SYSTem:TOUCh command disables or enables the touchscreen.

Query Syntax :SYSTem:TOUCh?

The :SYSTem:TOUCh? query returns the touchscreen's on/off status.

Return Format <on_off><NL>

<on_off> ::= {1 | 0}

See Also • ["Introduction to :SYSTem Commands"](#) on page 1307

36 :TIMebase Commands

Control all horizontal sweep functions. See "[Introduction to :TIMebase Commands](#)" on page 1332.

Table 154 :TIMebase Commands Summary

Command	Query	Options and Query Returns
:TIMebase:MODE <value> (see page 1333)	:TIMebase:MODE? (see page 1333)	<value> ::= {MAIN WINDow XY ROLL}
:TIMebase:POSition <pos> (see page 1334)	:TIMebase:POSition? (see page 1334)	<pos> ::= time from the trigger event to the display reference point in NR3 format
:TIMebase:RANGe <range_value> (see page 1335)	:TIMebase:RANGe? (see page 1335)	<range_value> ::= time for 10 div in seconds in NR3 format
:TIMebase:REFClock { {0 OFF} {1 ON} } (see page 1336)	:TIMebase:REFClock? (see page 1336)	{0 1}
:TIMebase:REFerence {LEFT CENTer RIGHT CUSTom} (see page 1337)	:TIMebase:REFerence? (see page 1337)	<return_value> ::= {LEFT CENTer RIGHT CUSTom}
:TIMebase:REFerence:L OCation <loc> (see page 1338)	:TIMebase:REFerence:L OCation? (see page 1338)	<loc> ::= 0.0 to 1.0 in NR3 format
:TIMebase:SCALe <scale_value> (see page 1339)	:TIMebase:SCALe? (see page 1339)	<scale_value> ::= time/div in seconds in NR3 format
:TIMebase:VERNier { {0 OFF} {1 ON} } (see page 1340)	:TIMebase:VERNier? (see page 1340)	{0 1}
:TIMebase:WINDow:POSi tion <pos> (see page 1341)	:TIMebase:WINDow:POSi tion? (see page 1341)	<pos> ::= time from the trigger event to the zoomed view reference point in NR3 format

Table 154 :TImebase Commands Summary (continued)

Command	Query	Options and Query Returns
:TImebase:WINDow:RANGe <range_value> (see page 1342)	:TImebase:WINDow:RANGe? (see page 1342)	<range_value> ::= range value in seconds in NR3 format for the zoomed window
:TImebase:WINDow:SCALe <scale_value> (see page 1343)	:TImebase:WINDow:SCALe? (see page 1343)	<scale_value> ::= scale value in seconds in NR3 format for the zoomed window

Introduction to :TImebase Commands The TImebase subsystem commands control the horizontal (X-axis) functions and set the oscilloscope to X-Y mode (where channel 1 becomes the X input and channel 2 becomes the Y input). The time per division, delay, vernier control, and reference can be controlled for the main and window (zoomed) time bases.

Reporting the Setup

Use :TImebase? to query setup information for the TImebase subsystem.

Return Format

The following is a sample response from the :TImebase? query. In this case, the query was issued following a *RST command.

```
:TIM:MODE MAIN;REF CENT;MAIN:RANG +1.00E-03;POS +0.0E+00
```


:TIMEbase:MODE

C (see [page 1650](#))

Command Syntax :TIMEbase:MODE <value>
 <value> ::= {MAIN | WINDow | XY | ROLL}

The :TIMEbase:MODE command sets the current time base. There are four time base modes:

- MAIN – The normal time base mode is the main time base. It is the default time base mode after the *RST (Reset) command.
- WINDow – In the WINDow (zoomed or delayed) time base mode, measurements are made in the zoomed time base if possible; otherwise, the measurements are made in the main time base.
- XY – In the XY mode, the :TIMEbase:RANGe, :TIMEbase:POSition, and :TIMEbase:REFerence commands are not available. No measurements are available in this mode.
- ROLL – In the ROLL mode, data moves continuously across the display from left to right. The oscilloscope runs continuously and is untriggered. The :TIMEbase:REFerence selection changes to RIGHT.

Query Syntax :TIMEbase:MODE?

The :TIMEbase:MODE query returns the current time base mode.

Return Format <value><NL>
 <value> ::= {MAIN | WIND | XY | ROLL}

- See Also**
- ["Introduction to :TIMEbase Commands"](#) on page 1332
 - ["*RST \(Reset\)"](#) on page 226
 - [":TIMEbase:RANGe"](#) on page 1335
 - [":TIMEbase:POSition"](#) on page 1334
 - [":TIMEbase:REFerence"](#) on page 1337

Example Code

```
' TIMEBASE_MODE - (not executed in this example)
' Set the time base mode to MAIN, DELAYED, XY, or ROLL.

' Set time base mode to main.
myScope.WriteString ":TIMEBASE:MODE MAIN"
```

See complete example programs at: [Chapter 46](#), "Programming Examples," starting on page 1659

:TIMebase:POSition

C (see [page 1650](#))

Command Syntax :TIMebase:POSition <pos>

<pos> ::= time in seconds from the trigger to the display reference
in NR3 format

The :TIMebase:POSition command sets the time interval between the trigger event and the display reference point on the screen. The display reference point is either left, right, or center and is set with the :TIMebase:REFerence command. The maximum position value depends on the time/division settings.

NOTE

This command is an alias for the :TIMebase:DELay command.

Query Syntax :TIMebase:POSition?

The :TIMebase:POSition? query returns the current time from the trigger to the display reference in seconds.

Return Format <pos><NL>

<pos> ::= time in seconds from the trigger to the display reference
in NR3 format

- See Also**
- ["Introduction to :TIMebase Commands"](#) on page 1332
 - [":TIMebase:REFerence"](#) on page 1337
 - [":TIMebase:RANGe"](#) on page 1335
 - [":TIMebase:SCALE"](#) on page 1339
 - [":TIMebase:WINDow:POSition"](#) on page 1341
 - [":TIMebase:DELay"](#) on page 1586

:TIMebase:RANGe

C (see [page 1650](#))

Command Syntax :TIMebase:RANGe <range_value>

<range_value> ::= time for 10 div in seconds in NR3 format

The :TIMebase:RANGe command sets the full-scale horizontal time in seconds for the main window. The range is 10 times the current time-per-division setting.

Query Syntax :TIMebase:RANGe?

The :TIMebase:RANGe query returns the current full-scale range value for the main window.

Return Format <range_value><NL>

<range_value> ::= time for 10 div in seconds in NR3 format

- See Also**
- ["Introduction to :TIMebase Commands"](#) on page 1332
 - [":TIMebase:MODE"](#) on page 1333
 - [":TIMebase:SCALE"](#) on page 1339
 - [":TIMebase:WINDow:RANGe"](#) on page 1342

Example Code

```
' TIME_RANGE - Sets the full scale horizontal time in seconds. The
' range value is 10 times the time per division.
myScope.WriteString ":TIM:RANG 2e-3" ' Set the time range to 0.002
seconds.
```

See complete example programs at: [Chapter 46](#), "Programming Examples," starting on page 1659

:TImebase:REFClock

N (see [page 1650](#))

Command Syntax :TImebase:REFClock <value>
 <value> ::= {{1 | ON} | {0 | OFF}}

The :TImebase:REFClock command enables or disables the 10 MHz REF BNC located on the rear panel of the oscilloscope.

The 10 MHz REF BNC can be used as an input for the oscilloscope's reference clock (instead of the internal 10 MHz reference), or it can be used to output the internal 10 MHz reference clock when synchronizing multiple instruments (see [":ACQuire:RSIGnal"](#) on page 292).

The :TImebase:REFClock ON command enables the 10 MHz REF BNC and sets the reference signal mode to IN. The :TImebase:REFClock OFF command disables the 10 MHz REF BNC (the same as setting the reference signal mode to OFF).

Query Syntax :TImebase:REFClock?

The :TImebase:REFClock? query returns the current state of the 10 MHz reference signal mode. A "1" indicates that the 10 MHz REF input is enabled (on), and a "0" indicates that either the 10 MHz REF BNC is disabled (off) or that it is set as an output (by the :ACQuire:RSIGnal command).

Return Format <value><NL>
 <value> ::= {0 | 1}

See Also • [":ACQuire:RSIGnal"](#) on page 292

:TImebase:REFeRence

C (see [page 1650](#))

Command Syntax :TImebase:REFeRence <reference>

<reference> ::= {LEFT | CENTer | RIGHT | CUSTom}

The :TImebase:REFeRence command sets the time reference to:

- LEFT – one division from the left side of the screen.
- CENTer – the center of the screen.
- RIGHT – one division from the right side of the screen.
- CUSTom – lets you use the :TImebase:REFeRence:LOCation command to place the time reference location at a percent of the graticule width (where 0.0 is the left edge and 1.0 is the right edge).

The time reference is the point on the display where the trigger point is referenced.

Query Syntax :TImebase:REFeRence?

The :TImebase:REFeRence? query returns the current display reference for the main window.

Return Format <reference><NL>

<reference> ::= {LEFT | CENT | RIGH | CUST}

- See Also**
- ["Introduction to :TImebase Commands"](#) on page 1332
 - [":TImebase:REFeRence:LOCation"](#) on page 1338
 - [":TImebase:MODE"](#) on page 1333

Example Code myScope.WriteString ":TImebase:REFeRence CENTer" ' Set reference to center.

See complete example programs at: [Chapter 46](#), "Programming Examples," starting on page 1659

:TImebase:REFeRence:LOCation

N (see [page 1650](#))

Command Syntax :TImebase:REFeRence:LOCation <loc>

<loc> ::= 0.0 to 1.0 in NR3 format

When the :TImebase:REFeRence is set to CUSTom, the :TImebase:REFeRence:LOCation command lets you place the time reference location at a percent of the graticule width (where 0.0 is the left edge and 1.0 is the right edge).

Query Syntax :TImebase:REFeRence:LOCation?

The :TImebase:REFeRence:LOCation? query returns the time base reference custom location setting.

Return Format <loc><NL>

<loc> ::= 0.0 to 1.0 in NR3 format

See Also • [":TImebase:REFeRence"](#) on page 1337

:TImebase:SCALe

N (see [page 1650](#))

Command Syntax :TImebase:SCALe <scale_value>

<scale_value> ::= time/div in seconds in NR3 format

The :TImebase:SCALe command sets the horizontal scale or units per division for the main window.

Query Syntax :TImebase:SCALe?

The :TImebase:SCALe? query returns the current horizontal scale setting in seconds per division for the main window.

Return Format <scale_value><NL>

<scale_value> ::= time/div in seconds in NR3 format

- See Also**
- ["Introduction to :TImebase Commands"](#) on page 1332
 - [":TImebase:RANGe"](#) on page 1335
 - [":TImebase:WINDow:SCALe"](#) on page 1343
 - [":TImebase:WINDow:RANGe"](#) on page 1342

:TImebase:VERNier

N (see [page 1650](#))

Command Syntax :TImebase:VERNier <vernier value>

<vernier value> ::= {{1 | ON} | {0 | OFF}}

The :TImebase:VERNier command specifies whether the time base control's vernier (fine horizontal adjustment) setting is ON (1) or OFF (0).

Query Syntax :TImebase:VERNier?

The :TImebase:VERNier? query returns the current state of the time base control's vernier setting.

Return Format <vernier value><NL>

<vernier value> ::= {0 | 1}

See Also • ["Introduction to :TImebase Commands"](#) on page 1332

:TImebase:WINDow:POSition

C (see [page 1650](#))

Command Syntax :TImebase:WINDow:POSition <pos value>

<pos value> ::= time from the trigger event to the zoomed (delayed) view reference point in NR3 format

The :TImebase:WINDow:POSition command sets the horizontal position in the zoomed (delayed) view of the main sweep. The main sweep range and the main sweep horizontal position determine the range for this command. The value for this command must keep the zoomed view window within the main sweep range.

Query Syntax :TImebase:WINDow:POSition?

The :TImebase:WINDow:POSition? query returns the current horizontal window position setting in the zoomed view.

Return Format <value><NL>

<value> ::= position value in seconds

- See Also**
- ["Introduction to :TImebase Commands"](#) on page 1332
 - [":TImebase:MODE"](#) on page 1333
 - [":TImebase:POSition"](#) on page 1334
 - [":TImebase:RANGe"](#) on page 1335
 - [":TImebase:SCALE"](#) on page 1339
 - [":TImebase:WINDow:RANGe"](#) on page 1342
 - [":TImebase:WINDow:SCALE"](#) on page 1343

:TIMEbase:WINDow:RANGe

C (see [page 1650](#))

Command Syntax :TIMEbase:WINDow:RANGe <range value>

<range value> ::= range value in seconds in NR3 format

The :TIMEbase:WINDow:RANGe command sets the full-scale horizontal time in seconds for the zoomed (delayed) window. The range is 10 times the current zoomed view window seconds per division setting. The main sweep range determines the range for this command. The maximum value is one half of the :TIMEbase:RANGe value.

Query Syntax :TIMEbase:WINDow:RANGe?

The :TIMEbase:WINDow:RANGe? query returns the current window timebase range setting.

Return Format <value><NL>

<value> ::= range value in seconds

- See Also**
- ["Introduction to :TIMEbase Commands"](#) on page 1332
 - [":TIMEbase:RANGe"](#) on page 1335
 - [":TIMEbase:POSition"](#) on page 1334
 - [":TIMEbase:SCALe"](#) on page 1339

:TIMebase:WINDow:SCALE

N (see [page 1650](#))

Command Syntax :TIMebase:WINDow:SCALE <scale_value>

<scale_value> ::= scale value in seconds in NR3 format

The :TIMebase:WINDow:SCALE command sets the zoomed (delayed) window horizontal scale (seconds/division). The main sweep scale determines the range for this command. The maximum value is one half of the :TIMebase:SCALE value.

Query Syntax :TIMebase:WINDow:SCALE?

The :TIMebase:WINDow:SCALE? query returns the current zoomed window scale setting.

Return Format <scale_value><NL>

<scale_value> ::= current seconds per division for the zoomed window

- See Also**
- ["Introduction to :TIMebase Commands"](#) on page 1332
 - [":TIMebase:RANGe"](#) on page 1335
 - [":TIMebase:POSition"](#) on page 1334
 - [":TIMebase:SCALE"](#) on page 1339
 - [":TIMebase:WINDow:RANGe"](#) on page 1342

37 :TRIGger Commands

Control the trigger modes and parameters for each trigger type. See:

- **"Introduction to :TRIGger Commands"** on page 1345
- **"General :TRIGger Commands"** on page 1347
- **":TRIGger:DElay Commands"** on page 1361
- **":TRIGger:EBURst Commands"** on page 1368
- **":TRIGger[:EDGE] Commands"** on page 1373
- **":TRIGger:GLITch Commands"** on page 1380 (Pulse Width trigger)
- **":TRIGger:OR Commands"** on page 1389
- **":TRIGger:PATtern Commands"** on page 1391
- **":TRIGger:RUNT Commands"** on page 1399
- **":TRIGger:SHOLd Commands"** on page 1404
- **":TRIGger:TRANSition Commands"** on page 1410
- **":TRIGger:TV Commands"** on page 1415
- **":TRIGger:ZONE Commands"** on page 1425

Introduction to :TRIGger Commands

The commands in the TRIGger subsystem define the conditions for an internal trigger. Many of these commands are valid in multiple trigger modes.

The default trigger mode is :EDGE.

The trigger subsystem controls the trigger sweep mode and the trigger specification. The trigger sweep (see **":TRIGger:SWEep"** on page 1360) can be AUTO or NORMal.

- **NORMal** mode – displays a waveform only if a trigger signal is present and the trigger conditions are met. Otherwise the oscilloscope does not trigger and the display is not updated. This mode is useful for low-repetitive-rate signals.
- **AUTO** trigger mode – generates an artificial trigger event if the trigger specification is not satisfied within a preset time, acquires unsynchronized data and displays it.

AUTO mode is useful for signals other than low-repetitive-rate signals. You must use this mode to display a DC signal because there are no edges on which to trigger.

The following trigger types are available (see **":TRIGger:MODE"** on page 1358).

- **Edge triggering** – identifies a trigger by looking for a specified slope and voltage level on a waveform.
- **Nth Edge Burst triggering** – lets you trigger on the Nth edge of a burst that occurs after an idle time.
- **Pulse width triggering** – (:TRIGger:GLITCh commands) sets the oscilloscope to trigger on a positive pulse or on a negative pulse of a specified width.
- **Pattern triggering** – identifies a trigger condition by looking for a specified pattern. This pattern is a logical AND combination of the channels. You can also trigger on a specified time duration of a pattern.
- **TV triggering** – is used to capture the complicated waveforms of television equipment. The trigger circuitry detects the vertical and horizontal interval of the waveform and produces triggers based on the TV trigger settings you selected. TV triggering requires greater than ¼ division of sync amplitude with any analog channel as the trigger source.
- **USB (Universal Serial Bus) triggering** – will trigger on a Start of Packet (SOP), End of Packet (EOP), Reset Complete, Enter Suspend, or Exit Suspend signal on the differential USB data lines. USB Low Speed and Full Speed are supported by this trigger.

Reporting the Setup

Use :TRIGger? to query setup information for the TRIGger subsystem.

Return Format

The return format for the TRIGger? query varies depending on the current mode. The following is a sample response from the :TRIGger? query. In this case, the query was issued following a *RST command.

```
:TRIG:MODE EDGE;SWE AUTO;NREJ 0;HFR 0;HOLD +60.0000000000000E-09;
:TRIG:EDGE:SOUR CHAN1;LEV +0.00000E+00;SLOP POS;REJ OFF;COUP DC;
:TRIG:ZONE:STAT 0
```

General :TRIGger Commands

Table 155 General :TRIGger Commands Summary

Command	Query	Options and Query Returns
:TRIGger:FORCE (see page 1349)	n/a	n/a
:TRIGger:HFReject { {0 OFF} {1 ON} } (see page 1350)	:TRIGger:HFReject? (see page 1350)	{0 1}
:TRIGger:HOLDoff <holdoff_time> (see page 1351)	:TRIGger:HOLDoff? (see page 1351)	<holdoff_time> ::= 60 ns to 10 s in NR3 format
:TRIGger:HOLDoff:MAXimum <max_holdoff> (see page 1352)	:TRIGger:HOLDoff:MAXimum? (see page 1352)	<max_holdoff> ::= maximum holdoff time in seconds in NR3 format
:TRIGger:HOLDoff:MINimum <min_holdoff> (see page 1353)	:TRIGger:HOLDoff:MINimum? (see page 1353)	<min_holdoff> ::= minimum holdoff time in seconds in NR3 format
:TRIGger:HOLDoff:RANDOM { {0 OFF} {1 ON} } (see page 1354)	:TRIGger:HOLDoff:RANDOM? (see page 1354)	<setting> ::= {0 1}
:TRIGger:LEVel:ASETup (see page 1355)	n/a	n/a
:TRIGger:LEVel:HIGH <level>, <source> (see page 1356)	:TRIGger:LEVel:HIGH? <source> (see page 1356)	<level> ::= .75 x full-scale voltage from center screen in NR3 format. <source> ::= CHANNEL<n> <n> ::= 1 to (# analog channels) in NR1 format
:TRIGger:LEVel:LOW <level>, <source> (see page 1357)	:TRIGger:LEVel:LOW? <source> (see page 1357)	<level> ::= .75 x full-scale voltage from center screen in NR3 format. <source> ::= CHANNEL<n> <n> ::= 1 to (# analog channels) in NR1 format

Table 155 General :TRIGger Commands Summary (continued)

Command	Query	Options and Query Returns
:TRIGger:MODE <mode> (see page 1358)	:TRIGger:MODE? (see page 1358)	<mode> ::= {EDGE GLITCh PATtern TV DELay EBURst OR RUNT SHOLd TRANSition SBUS{1 2}} <return_value> ::= {<mode> <none>} <none> ::= query returns "NONE" if the :TIMEbase:MODE is ROLL or XY
:TRIGger:NREJect {{0 OFF} {1 ON}} (see page 1359)	:TRIGger:NREJect? (see page 1359)	{0 1}
:TRIGger:SWEep <sweep> (see page 1360)	:TRIGger:SWEep? (see page 1360)	<sweep> ::= {AUTO NORMal}

:TRIGger:FORCe

N (see [page 1650](#))

Command Syntax :TRIGger:FORCe

The :TRIGger:FORCe command causes an acquisition to be captured even though the trigger condition has not been met. This command is equivalent to the front panel **[Force Trigger]** key.

See Also • ["Introduction to :TRIGger Commands"](#) on page 1345

:TRIGger:HFReject

C (see [page 1650](#))

Command Syntax :TRIGger:HFReject <value>
 <value> ::= {{0 | OFF} | {1 | ON}}

The :TRIGger:HFReject command turns the high frequency reject filter off and on. The high frequency reject filter adds a 50 kHz low-pass filter in the trigger path to remove high frequency components from the trigger waveform. Use this filter to remove high-frequency noise, such as AM or FM broadcast stations, from the trigger path.

Query Syntax :TRIGger:HFReject?

The :TRIGger:HFReject? query returns the current high frequency reject filter mode.

Return Format <value><NL>
 <value> ::= {0 | 1}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger\[:EDGE\]:REJect"](#) on page 1377

:TRIGger:HOLDoff

C (see [page 1650](#))

Command Syntax :TRIGger:HOLDoff <holdoff_time>

<holdoff_time> ::= 40 ns to 10 s in NR3 format

The :TRIGger:HOLDoff command defines the holdoff time value in seconds. Holdoff keeps a trigger from occurring until after a certain amount of time has passed since the last trigger. This feature is valuable when a waveform crosses the trigger level multiple times during one period of the waveform. Without holdoff, the oscilloscope could trigger on each of the crossings, producing a confusing waveform. With holdoff set correctly, the oscilloscope always triggers on the same crossing. The correct holdoff setting is typically slightly less than one period.

Query Syntax :TRIGger:HOLDoff?

The :TRIGger:HOLDoff? query returns the holdoff time value for the current trigger mode.

Return Format <holdoff_time><NL>

<holdoff_time> ::= the holdoff time value in seconds in NR3 format.

See Also • ["Introduction to :TRIGger Commands"](#) on page 1345

:TRIGger:HOLDoff:MAXimum

N (see [page 1650](#))

Command Syntax :TRIGger:HOLDoff:MAXimum <max_holdoff>

<max_holdoff> ::= maximum holdoff time in seconds in NR3 format

When the random trigger holdoff mode is enabled (see :TRIGger:HOLDoff:RANDom), the :TRIGger:HOLDoff:MAXimum command specifies the maximum trigger holdoff time.

Query Syntax :TRIGger:HOLDoff:MAXimum?

The :TRIGger:HOLDoff:MAXimum? query returns the maximum random trigger holdoff time setting.

Return Format <max_holdoff><NL>

- See Also**
- [":TRIGger:HOLDoff:MINimum"](#) on page 1353
 - [":TRIGger:HOLDoff:RANDom"](#) on page 1354

:TRIGger:HOLDoff:MINimum

N (see [page 1650](#))

Command Syntax :TRIGger:HOLDoff:MINimum <min_holdoff>

<min_holdoff> ::= minimum holdoff time in seconds in NR3 format

When the random trigger holdoff mode is enabled (see :TRIGger:HOLDoff:RANDom), the :TRIGger:HOLDoff:MINimum command specifies the minimum trigger holdoff time.

Query Syntax :TRIGger:HOLDoff:MINimum?

The :TRIGger:HOLDoff:MINimum? query returns the minimum random trigger holdoff time setting.

Return Format <min_holdoff><NL>

- See Also**
- [":TRIGger:HOLDoff:MAXimum"](#) on page 1352
 - [":TRIGger:HOLDoff:RANDom"](#) on page 1354

:TRIGger:HOLDoff:RANDom

N (see [page 1650](#))

Command Syntax :TRIGger:HOLDoff:RANDom {{0 | OFF} | {1 | ON}}

The :TRIGger:HOLDoff:RANDom command enables or disables the random trigger holdoff mode. This mode randomizes the holdoff time from one acquisition to the next. The randomized holdoff time values will be between the values specified by the :TRIGger:HOLDoff:MINimum and :TRIGger:HOLDoff:MAXimum commands.

The random trigger holdoff mode ensures that the oscilloscope re-arms after each acquisition in a manner that minimizes or eliminates the likelihood of triggering at the beginning of a DDR burst. Randomizing the holdoff time increases the likelihood that the oscilloscope will trigger on different data phases of a multi-phase (8 data transfer) burst. This mode mixes up the traffic pattern the oscilloscope triggers on and is very effective when used on repeating patterns.

Query Syntax :TRIGger:HOLDoff:RANDom?

The :TRIGger:HOLDoff:RANDom? query returns random trigger holdoff mode setting.

Return Format <setting><NL>

<setting> ::= {0 | 1}

- See Also**
- [":TRIGger:HOLDoff:MAXimum"](#) on page 1352
 - [":TRIGger:HOLDoff:MINimum"](#) on page 1353

:TRIGger:LEVel:ASETup

N (see [page 1650](#))

Command Syntax :TRIGger:LEVel:ASETup

The :TRIGger:LEVel:ASETup command automatically sets the trigger levels of all displayed analog channels to their waveforms' 50% values.

If AC coupling is used, the trigger levels are set to 0 V.

When High and Low (dual) trigger levels are used (as with Rise/Fall Time and Runt triggers, for example), this command has no effect.

See Also • [":TRIGger\[:EDGE\]:LEVel"](#) on page 1376

:TRIGger:LEVel:HIGH

N (see [page 1650](#))

Command Syntax :TRIGger:LEVel:HIGH <level>, <source>

<level> ::= 0.75 x full-scale voltage from center screen in NR3 format
for internal triggers

<source> ::= CHANnel<n>

<n> ::= 1 to (# analog channels) in NR1 format

The :TRIGger:LEVel:HIGH command sets the high trigger voltage level voltage for the specified source.

High and low trigger levels are used with runt triggers and rise/fall time (transition) triggers.

Query Syntax :TRIGger:LEVel:HIGH? <source>

The :TRIGger:LEVel:HIGH? query returns the high trigger voltage level for the specified source.

Return Format <level><NL>

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:LEVel:LOW"](#) on page 1357
 - [":TRIGger:RUNT Commands"](#) on page 1399
 - [":TRIGger:TRANSition Commands"](#) on page 1410
 - [":TRIGger\[:EDGE\]:SOURce"](#) on page 1379

:TRIGger:LEVel:LOW

N (see [page 1650](#))

Command Syntax :TRIGger:LEVel:LOW <level>, <source>

<level> ::= 0.75 x full-scale voltage from center screen in NR3 format
for internal triggers

<source> ::= CHANnel<n>

<n> ::= 1 to (# analog channels) in NR1 format

The :TRIGger:LEVel:LOW command sets the low trigger voltage level voltage for the specified source.

High and low trigger levels are used with runt triggers and rise/fall time (transition) triggers.

Query Syntax :TRIGger:LEVel:LOW? <source>

The :TRIGger:LEVel:LOW? query returns the low trigger voltage level for the specified source.

Return Format <level><NL>

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:LEVel:HIGH"](#) on page 1356
 - [":TRIGger:RUNT Commands"](#) on page 1399
 - [":TRIGger:TRANSition Commands"](#) on page 1410
 - [":TRIGger\[:EDGE\]:SOURce"](#) on page 1379

:TRIGger:MODE

C (see [page 1650](#))

Command Syntax :TRIGger:MODE <mode>

<mode> ::= {EDGE | GLITCh | PATTeRn | TV | DELay | EBURst | OR | RUNT
| SHOLd | TRANsition | SBUS{1 | 2}}

The :TRIGger:MODE command selects the trigger mode (trigger type).

Query Syntax :TRIGger:MODE?

The :TRIGger:MODE? query returns the current trigger mode. If the :TIMebase:MODE is ROLL or XY, the query returns "NONE".

Return Format <mode><NL>

<mode> ::= {EDGE | GLIT | PATT | TV | DEL | EBUR | OR | RUNT | SHOL
| TRAN | SBUS{1 | 2}}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:SWEep"](#) on page 1360
 - [":TIMebase:MODE"](#) on page 1333

Example Code

```
' TRIGGER_MODE - Set the trigger mode to EDGE.  
myScope.WriteString ":TRIGger:MODE EDGE"
```

See complete example programs at: [Chapter 46](#), "Programming Examples," starting on page 1659

:TRIGger:NREJect

C (see [page 1650](#))

Command Syntax :TRIGger:NREJect <value>
 <value> ::= {{0 | OFF} | {1 | ON}}

The :TRIGger:NREJect command turns the noise reject filter off and on. When the noise reject filter is on, the trigger circuitry is less sensitive to noise but may require a greater amplitude waveform to trigger the oscilloscope. This command is not valid in TV trigger mode.

Query Syntax :TRIGger:NREJect?

The :TRIGger:NREJect? query returns the current noise reject filter mode.

Return Format <value><NL>
 <value> ::= {0 | 1}

See Also • ["Introduction to :TRIGger Commands"](#) on page 1345

:TRIGger:SWEep

C (see [page 1650](#))

Command Syntax :TRIGger:SWEep <sweep>

<sweep> ::= {AUTO | NORMal}

The :TRIGger:SWEep command selects the trigger sweep mode.

When AUTO sweep mode is selected, a baseline is displayed in the absence of a signal. If a signal is present but the oscilloscope is not triggered, the unsynchronized signal is displayed instead of a baseline.

When NORMal sweep mode is selected and no trigger is present, the instrument does not sweep, and the data acquired on the previous trigger remains on the screen.

NOTE

This feature is called "Mode" on the instrument's front panel.

Query Syntax :TRIGger:SWEep?

The :TRIGger:SWEep? query returns the current trigger sweep mode.

Return Format <sweep><NL>

<sweep> ::= current trigger sweep mode

See Also • ["Introduction to :TRIGger Commands"](#) on page 1345

:TRIGger:DElay Commands

Table 156 :TRIGger:DElay Commands Summary

Command	Query	Options and Query Returns
:TRIGger:DElay:ARM:SL OPe <slope> (see page 1362)	:TRIGger:DElay:ARM:SL OPe? (see page 1362)	<slope> ::= {NEGative POSitive}
:TRIGger:DElay:ARM:SO URce <source> (see page 1363)	:TRIGger:DElay:ARM:SO URce? (see page 1363)	<source> ::= {CHANnel<n> DIGital<d>} <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:TRIGger:DElay:TDElay :TIME <time_value> (see page 1364)	:TRIGger:DElay:TDElay :TIME? (see page 1364)	<time_value> ::= time in seconds in NR3 format
:TRIGger:DElay:TRIGge r:COUNT <count> (see page 1365)	:TRIGger:DElay:TRIGge r:COUNT? (see page 1365)	<count> ::= integer in NR1 format
:TRIGger:DElay:TRIGge r:SLOPe <slope> (see page 1366)	:TRIGger:DElay:TRIGge r:SLOPe? (see page 1366)	<slope> ::= {NEGative POSitive}
:TRIGger:DElay:TRIGge r:SOURce <source> (see page 1367)	:TRIGger:DElay:TRIGge r:SOURce? (see page 1367)	<source> ::= {CHANnel<n> DIGital<d>} <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format

The :TRIGger:DElay:ARM:SOURce and :TRIGger:DElay:TRIGger:SOURce commands are used to specify the source channel for the arming edge and the trigger edge in the Edge Then Edge trigger.

If an analog channel is selected as a source, the :TRIGger:EDGE:LEVel command is used to set the trigger level.

If a digital channel is selected as the source, the :DIGital<n>:THReshold or :POD<n>:THReshold command is used to set the trigger level.

:TRIGger:DElay:ARM:SLOPe

N (see [page 1650](#))

Command Syntax :TRIGger:DElay:ARM:SLOPe <slope>

<slope> ::= {NEGative | POSitive}

The :TRIGger:DElay:ARM:SLOPe command specifies rising (POSitive) or falling (NEGative) for the arming edge in the Edge Then Edge trigger.

Query Syntax :TRIGger:DElay:ARM:SLOPe?

The :TRIGger:DElay:ARM:SLOPe? query returns the current arming edge slope setting.

Return Format <slope><NL>

<slope> ::= {NEG | POS}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:DElay:ARM:SOURce"](#) on page 1363
 - [":TRIGger:DElay:TDElay:TIME"](#) on page 1364

:TRIGger:DELay:ARM:SOURce

N (see [page 1650](#))

Command Syntax :TRIGger:DELay:ARM:SOURce <source>

<source> ::= {CHANnel<n> | DIGital<d>}

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :TRIGger:DELay:ARM:SOURce command selects the input used for the arming edge in the Edge Then Edge trigger.

Query Syntax :TRIGger:DELay:ARM:SOURce?

The :TRIGger:DELay:ARM:SOURce? query returns the current arming edge source.

Return Format <source><NL>

<source> ::= {CHAN<n> | DIG<d>}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:DELay:ARM:SLOPe"](#) on page 1362
 - [":TRIGger:DELay:TDELay:TIME"](#) on page 1364
 - [":TRIGger:MODE"](#) on page 1358

:TRIGger:DElay:TDElay:TIME

N (see [page 1650](#))

Command Syntax :TRIGger:DElay:TDElay:TIME <time_value>

<time_value> ::= time in seconds in NR3 format

The :TRIGger:DElay:TDElay:TIME command sets the delay time between the arming edge and the trigger edge in the Edge Then Edge trigger. The time is in seconds and must be from 4 ns to 10 s.

Query Syntax :TRIGger:DElay:TDElay:TIME?

The :TRIGger:DElay:TDElay:TIME? query returns current delay time setting.

Return Format <time_value><NL>

<time_value> ::= time in seconds in NR3 format

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:DElay:TRIGger:SLOPe"](#) on page 1366
 - [":TRIGger:DElay:TRIGger:COUNt"](#) on page 1365

:TRIGger:DElay:TRIGger:COUNT

N (see [page 1650](#))

Command Syntax :TRIGger:DElay:TRIGger:COUNT <count>

<count> ::= integer in NR1 format

The :TRIGger:DElay:TRIGger:COUNT command sets the Nth edge of the trigger source to trigger on.

Query Syntax :TRIGger:DElay:TRIGger:COUNT?

The :TRIGger:DElay:TRIGger:COUNT? query returns the current Nth trigger edge setting.

Return Format <count><NL>

<count> ::= integer in NR1 format

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:DElay:TRIGger:SLOPe"](#) on page 1366
 - [":TRIGger:DElay:TRIGger:SOURce"](#) on page 1367
 - [":TRIGger:DElay:TDElay:TIME"](#) on page 1364

:TRIGger:DElay:TRIGger:SLOPe

N (see [page 1650](#))

Command Syntax :TRIGger:DElay:TRIGger:SLOPe <slope>

<slope> ::= {NEGative | POSitive}

The :TRIGger:DElay:TRIGger:SLOPe command specifies rising (POSitive) or falling (NEGative) for the trigger edge in the Edge Then Edge trigger.

Query Syntax :TRIGger:DElay:TRIGger:SLOPe?

The :TRIGger:DElay:TRIGger:SLOPe? query returns the current trigger edge slope setting.

Return Format <slope><NL>

<slope> ::= {NEG | POS}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:DElay:TRIGger:SOURce"](#) on page 1367
 - [":TRIGger:DElay:TDElay:TIME"](#) on page 1364
 - [":TRIGger:DElay:TRIGger:COUNt"](#) on page 1365

:TRIGger:DElay:TRIGger:SOURce

N (see [page 1650](#))

Command Syntax :TRIGger:DElay:TRIGger:SOURce <source>

<source> ::= {CHANnel<n> | DIGital<d>}

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :TRIGger:DElay:TRIGger:SOURce command selects the input used for the trigger edge in the Edge Then Edge trigger.

Query Syntax :TRIGger:DElay:TRIGger:SOURce?

The :TRIGger:DElay:TRIGger:SOURce? query returns the current trigger edge source.

Return Format <source><NL>

<source> ::= {CHAN<n> | DIG<d>}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:DElay:TRIGger:SLOPe"](#) on page 1366
 - [":TRIGger:DElay:TDElay:TIME"](#) on page 1364
 - [":TRIGger:DElay:TRIGger:COUNt"](#) on page 1365
 - [":TRIGger:MODE"](#) on page 1358

:TRIGger:EBURst Commands

Table 157 :TRIGger:EBURst Commands Summary

Command	Query	Options and Query Returns
:TRIGger:EBURst:COUNT <count> (see page 1369)	:TRIGger:EBURst:COUNT ? (see page 1369)	<count> ::= integer in NR1 format
:TRIGger:EBURst:IDLE <time_value> (see page 1370)	:TRIGger:EBURst:IDLE? (see page 1370)	<time_value> ::= time in seconds in NR3 format
:TRIGger:EBURst:SLOPe <slope> (see page 1371)	:TRIGger:EBURst:SLOPe ? (see page 1371)	<slope> ::= {NEGative POSitive}
:TRIGger:EBURst:SOURc e <source> (see page 1372)	:TRIGger:EBURst:SOURc e? (see page 1372)	<source> ::= {CHANnel<n> DIGital<d>} <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format

The :TRIGger:EBURst:SOURce command is used to specify the source channel for the Nth Edge Burst trigger. If an analog channel is selected as the source, the :TRIGger:EDGE:LEVEL command is used to set the Nth Edge Burst trigger level. If a digital channel is selected as the source, the :DIGital<n>:THReshold or :POD<n>:THReshold command is used to set the Nth Edge Burst trigger level.

:TRIGger:EBURst:COUNT

N (see [page 1650](#))

Command Syntax :TRIGger:EBURst:COUNT <count>

<count> ::= integer in NR1 format

The :TRIGger:EBURst:COUNT command sets the Nth edge at burst counter resource. The edge counter is used in the trigger stage to determine which edge in a burst will generate a trigger.

Query Syntax :TRIGger:EBURst:COUNT?

The :TRIGger:EBURst:COUNT? query returns the current Nth edge of burst edge counter setting.

Return Format <count><NL>

<count> ::= integer in NR1 format

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:EBURst:SLOPe"](#) on page 1371
 - [":TRIGger:EBURst:IDLE"](#) on page 1370

:TRIGger:EBURst:IDLE

N (see [page 1650](#))

Command Syntax :TRIGger:EBURst:IDLE <time_value>

<time_value> ::= time in seconds in NR3 format

The :TRIGger:EBURst:IDLE command sets the Nth edge in a burst idle resource in seconds from 10 ns to 10 s. The timer is used to set the minimum time before the next burst.

Query Syntax :TRIGger:EBURst:IDLE?

The :TRIGger:EBURst:IDLE? query returns current Nth edge in a burst idle setting.

Return Format <time_value><NL>

<time_value> ::= time in seconds in NR3 format

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:EBURst:SLOPe"](#) on page 1371
 - [":TRIGger:EBURst:COUNt"](#) on page 1369

:TRIGger:EBURst:SLOPe

N (see [page 1650](#))

Command Syntax :TRIGger:EBURst:SLOPe <slope>

<slope> ::= {NEGative | POSitive}

The :TRIGger:EBURst:SLOPe command specifies whether the rising edge (POSitive) or falling edge (NEGative) of the Nth edge in a burst will generate a trigger.

Query Syntax :TRIGger:EBURst:SLOPe?

The :TRIGger:EBURst:SLOPe? query returns the current Nth edge in a burst slope.

Return Format <slope><NL>

<slope> ::= {NEG | POS}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:EBURst:IDLE"](#) on page 1370
 - [":TRIGger:EBURst:COUNt"](#) on page 1369

:TRIGger:EBURst:SOURce

C (see [page 1650](#))

Command Syntax :TRIGger:EBURst:SOURce <source>

<source> ::= {CHANnel<n> | DIGital<d>}

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :TRIGger:EBURst:SOURce command selects the input that produces the Nth edge burst trigger.

Query Syntax :TRIGger:EBURst:SOURce?

The :TRIGger:EBURst:SOURce? query returns the current Nth edge burst trigger source. If all channels are off, the query returns "NONE."

Return Format <source><NL>

<source> ::= {CHAN<n> | DIG<d>}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358

:TRIGger[:EDGE] Commands

Table 158 :TRIGger[:EDGE] Commands Summary

Command	Query	Options and Query Returns
:TRIGger[:EDGE]:COUPling {AC DC LFReject} (see page 1375)	:TRIGger[:EDGE]:COUPling? (see page 1375)	{AC DC LFReject}
:TRIGger[:EDGE]:LEVel <level> [, <source>] (see page 1376)	:TRIGger[:EDGE]:LEVel? [<source>] (see page 1376)	For internal triggers, <level> ::= .75 x full-scale voltage from center screen in NR3 format. For external triggers, <level> ::= ±(external range setting) in NR3 format. For digital channels (MSO models), <level> ::= ±8 V. <source> ::= {CHANnel<n> EXTERNAL} for DSO models <source> ::= {CHANnel<n> DIGital<d> EXTERNAL} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:TRIGger[:EDGE]:REJect {OFF LFReject HFReject} (see page 1377)	:TRIGger[:EDGE]:REJect? (see page 1377)	{OFF LFReject HFReject}

Table 158 :TRIGger[:EDGE] Commands Summary (continued)

Command	Query	Options and Query Returns
:TRIGger[:EDGE]:SLOPe <polarity> (see page 1378)	:TRIGger[:EDGE]:SLOPe ? (see page 1378)	<polarity> ::= {POSitive NEGative EITHER ALternate}
:TRIGger[:EDGE]:SOURc e <source> (see page 1379)	:TRIGger[:EDGE]:SOURc e? (see page 1379)	<source> ::= {CHANnel<n> EXtErnal LINE WGEN WGEN1 WGEN2 WMOD} for the DSO models <source> ::= {CHANnel<n> DIGital<d> EXtErnal LINE WGEN WGEN1 WGEN2 WMOD} for the MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format Note: WGEN and WGEN1 are equivalent. Note: WGEN2 only available on models with 2 WaveGen outputs.

:TRIGger[:EDGE]:COUPling

C (see [page 1650](#))

Command Syntax :TRIGger[:EDGE]:COUPling <coupling>

<coupling> ::= {AC | DC | LFReject}

The :TRIGger[:EDGE]:COUPling command sets the input coupling for the selected trigger sources. The coupling can be set to AC, DC, or LFReject.

- AC coupling places a high-pass filter (10 Hz for analog channels, and 3.5 Hz for all External trigger inputs) in the trigger path, removing dc offset voltage from the trigger waveform. Use AC coupling to get a stable edge trigger when your waveform has a large dc offset.
- LFReject coupling places a 50 kHz high-pass filter in the trigger path.
- DC coupling allows dc and ac signals into the trigger path.

NOTE

The :TRIGger[:EDGE]:COUPling and the :TRIGger[:EDGE]:REJect selections are coupled. Changing the setting of the :TRIGger[:EDGE]:REJect can change the COUPling setting.

Query Syntax :TRIGger[:EDGE]:COUPling?

The :TRIGger[:EDGE]:COUPling? query returns the current coupling selection.

Return Format <coupling><NL>

<coupling> ::= {AC | DC | LFR}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":TRIGger\[:EDGE\]:REJect"](#) on page 1377

:TRIGger[:EDGE]:LEVel

C (see [page 1650](#))

Command Syntax :TRIGger[:EDGE]:LEVel <level>

<level> ::= <level>[,<source>]

<level> ::= 0.75 x full-scale voltage from center screen in NR3 format for internal triggers

<level> ::= ±(external range setting) in NR3 format for external triggers

<level> ::= ±8 V for digital channels (MSO models)

<source> ::= {CHANnel<n> | EXTernal} for the DSO models

<source> ::= {CHANnel<n> | DIGital<d> | EXTernal} for the MSO models

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :TRIGger[:EDGE]:LEVel command sets the trigger level voltage for the active trigger source.

NOTE

If the optional source is specified and is not the active source, the level on the active source is not affected and the active source is not changed.

Query Syntax :TRIGger[:EDGE]:LEVel? [<source>]

The :TRIGger[:EDGE]:LEVel? query returns the trigger level of the current trigger source.

Return Format <level><NL>

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger\[:EDGE\]:SOURce"](#) on page 1379
 - [":EXTernal:RANGe"](#) on page 426
 - [":POD<n>:THReshold"](#) on page 749
 - [":DIGital<d>:THReshold"](#) on page 389

:TRIGger[:EDGE]:REJect

C (see [page 1650](#))

Command Syntax :TRIGger[:EDGE]:REJect <reject>
 <reject> ::= {OFF | LFReject | HFReject}

The :TRIGger[:EDGE]:REJect command turns the low-frequency or high-frequency reject filter on or off. You can turn on one of these filters at a time.

- The high frequency reject filter adds a 50 kHz low-pass filter in the trigger path to remove high frequency components from the trigger waveform. Use the high frequency reject filter to remove high-frequency noise, such as AM or FM broadcast stations, from the trigger path.
- The low frequency reject filter adds a 50 kHz high-pass filter in series with the trigger waveform to remove any unwanted low frequency components from a trigger waveform, such as power line frequencies, that can interfere with proper triggering.

NOTE

The :TRIGger[:EDGE]:REJect and the :TRIGger[:EDGE]:COUpling selections are coupled. Changing the setting of the :TRIGger[:EDGE]:COUpling can change the COUpling setting.

Query Syntax :TRIGger[:EDGE]:REJect?

The :TRIGger[:EDGE]:REJect? query returns the current status of the reject filter.

Return Format <reject><NL>
 <reject> ::= {OFF | LFR | HFR}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:HFReject"](#) on page 1350
 - [":TRIGger\[:EDGE\]:COUpling"](#) on page 1375

:TRIGger[:EDGE]:SLOPe

C (see [page 1650](#))

Command Syntax :TRIGger[:EDGE]:SLOPe <slope>

<slope> ::= {NEGative | POSitive | EITHer | ALTerNate}

The :TRIGger[:EDGE]:SLOPe command specifies the slope of the edge for the trigger. The SLOPe command is not valid in TV trigger mode. Instead, use :TRIGger:TV:POLarity to set the polarity in TV trigger mode.

Query Syntax :TRIGger[:EDGE]:SLOPe?

The :TRIGger[:EDGE]:SLOPe? query returns the current trigger slope.

Return Format <slope><NL>

<slope> ::= {NEG | POS | EITH | ALT}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":TRIGger:TV:POLarity"](#) on page 1418

Example Code

```
' TRIGGER_EDGE_SLOPE - Sets the slope of the edge for the trigger.

' Set the slope to positive.
myScope.WriteString ":TRIGGER:EDGE:SLOPE POSITIVE"
```

See complete example programs at: [Chapter 46](#), "Programming Examples," starting on page 1659

:TRIGger[:EDGE]:SOURce

C (see [page 1650](#))

Command Syntax :TRIGger[:EDGE]:SOURce <source>

<source> ::= {CHANnel<n> | EXTernal | LINE | WGEN | WGEN1 | WGEN2
| WMOD} for the DSO models

<source> ::= {CHANnel<n> | DIGital<d> | EXTernal | LINE | WGEN | WGEN1
| WGEN2 | WMOD} for the MSO models

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

Note: WAVE and WGEN1 are equivalent.

Note: WGEN2 only available on models with 2 WaveGen outputs.

The :TRIGger[:EDGE]:SOURce command selects the input that produces the trigger.

- EXTernal – triggers on the rear panel EXT TRIG IN signal.
- LINE – triggers at the 50% level of the rising or falling edge of the AC power source signal.
- WGEN, WGEN1, WGEN2 – triggers at the 50% level of the rising edge of the waveform generator output signal. This option is not available when the DC, NOISe, or CARDiac waveforms are selected.
- WMOD – when waveform generator FSK or FM modulation is used, triggers at the 50% level of the rising edge of the modulating signal.

Query Syntax :TRIGger[:EDGE]:SOURce?

The :TRIGger[:EDGE]:SOURce? query returns the current source. If all channels are off, the query returns "NONE."

Return Format <source><NL>

<source> ::= {CHAN<n> | EXT | LINE | WGEN | WGEN1 | WGEN2 | WMOD
| NONE} for the DSO models

<source> ::= {CHAN<n> | DIG<d> | EXTernal | LINE | WGEN | WGEN1
| WGEN2 | WMOD | NONE} for the MSO models

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358

Example Code

```
' TRIGGER_EDGE_SOURCE - Selects the channel that actually produces the
edge trigger. Any channel can be selected.
myScope.WriteString ":TRIGger:EDGE:SOURce CHANnel1"
```

See complete example programs at: [Chapter 46](#), "Programming Examples," starting on page 1659

:TRIGger:GLITch Commands

Table 159 :TRIGger:GLITch Commands Summary

Command	Query	Options and Query Returns
:TRIGger:GLITch:GREAt erthan <greater_than_time>[s uffix] (see page 1382)	:TRIGger:GLITch:GREAt erthan? (see page 1382)	<greater_than_time> ::= floating-point number in NR3 format [suffix] ::= {s ms us ns ps}
:TRIGger:GLITch:LESt han <less_than_time>[suff ix] (see page 1383)	:TRIGger:GLITch:LESt han? (see page 1383)	<less_than_time> ::= floating-point number in NR3 format [suffix] ::= {s ms us ns ps}
:TRIGger:GLITch:LEVel <level> [<source>] (see page 1384)	:TRIGger:GLITch:LEVel ? (see page 1384)	For internal triggers, <level> ::= .75 x full-scale voltage from center screen in NR3 format. For external triggers (DSO models), <level> ::= ±(external range setting) in NR3 format. For digital channels (MSO models), <level> ::= ±8 V. <source> ::= {CHANnel<n> EXTErnal} for DSO models <source> ::= {CHANnel<n> DIGital<d>} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:TRIGger:GLITch:POLar ity <polarity> (see page 1385)	:TRIGger:GLITch:POLar ity? (see page 1385)	<polarity> ::= {POSitive NEGative}
:TRIGger:GLITch:QUALi fier <qualifier> (see page 1386)	:TRIGger:GLITch:QUALi fier? (see page 1386)	<qualifier> ::= {GREATERthan LESSthan RANGE}

Table 159 :TRIGger:GLITCh Commands Summary (continued)

Command	Query	Options and Query Returns
:TRIGger:GLITCh:RANGe <less_than_time>[suffix], <greater_than_time>[suffix] (see page 1387)	:TRIGger:GLITCh:RANGe? (see page 1387)	<less_than_time> ::= 15 ns to 10 seconds in NR3 format <greater_than_time> ::= 10 ns to 9.99 seconds in NR3 format [suffix] ::= {s ms us ns ps}
:TRIGger:GLITCh:SOURCe <source> (see page 1388)	:TRIGger:GLITCh:SOURCe? (see page 1388)	<source> ::= {CHANnel<n> DIGital<d>} <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format

:TRIGger:GLITch:GREaterthan

N (see [page 1650](#))

Command Syntax :TRIGger:GLITch:GREaterthan <greater_than_time> [<suffix>]

<greater_than_time> ::= floating-point number in NR3 format

<suffix> ::= {s | ms | us | ns | ps}

The :TRIGger:GLITch:GREaterthan command sets the minimum pulse width duration for the selected :TRIGger:GLITch:SOURce.

Query Syntax :TRIGger:GLITch:GREaterthan?

The :TRIGger:GLITch:GREaterthan? query returns the minimum pulse width duration time for :TRIGger:GLITch:SOURce.

Return Format <greater_than_time><NL>

<greater_than_time> ::= floating-point number in NR3 format.

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:GLITch:SOURce"](#) on page 1388
 - [":TRIGger:GLITch:QUALifier"](#) on page 1386
 - [":TRIGger:MODE"](#) on page 1358

:TRIGger:GLITch:LESSthan

N (see [page 1650](#))

Command Syntax :TRIGger:GLITch:LESSthan <less_than_time> [<suffix>]

<less_than_time> ::= floating-point number in NR3 format

<suffix> ::= {s | ms | us | ns | ps}

The :TRIGger:GLITch:LESSthan command sets the maximum pulse width duration for the selected :TRIGger:GLITch:SOURce.

Query Syntax :TRIGger:GLITch:LESSthan?

The :TRIGger:GLITch:LESSthan? query returns the pulse width duration time for :TRIGger:GLITch:SOURce.

Return Format <less_than_time><NL>

<less_than_time> ::= floating-point number in NR3 format.

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:GLITch:SOURce"](#) on page 1388
 - [":TRIGger:GLITch:QUALifier"](#) on page 1386
 - [":TRIGger:MODE"](#) on page 1358

:TRIGger:GLITch:LEVel

N (see [page 1650](#))

Command Syntax :TRIGger:GLITch:LEVel <level>[,<source>]

<level> ::= .75 x full-scale voltage from center screen in NR3 format
for internal triggers

<level> ::= ±(external range setting) in NR3 format
for external triggers (DSO models)

<level> ::= ±8 V for digital channels (MSO models)

<source> ::= {CHANnel<n> | EXTernal} for DSO models

<source> ::= {CHANnel<n> | DIGital<d>} for MSO models

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :TRIGger:GLITch:LEVel command sets the trigger level voltage for the active pulse width trigger.

Query Syntax :TRIGger:GLITch:LEVel?

The :TRIGger:GLITch:LEVel? query returns the trigger level of the current pulse width trigger mode. If all channels are off, the query returns "NONE."

Return Format <level><NL>

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":TRIGger:GLITch:SOURce"](#) on page 1388
 - [":EXTernal:RANGe"](#) on page 426

:TRIGger:GLITch:POLarity

N (see [page 1650](#))

Command Syntax :TRIGger:GLITch:POLarity <polarity>
 <polarity> ::= {POSitive | NEGative}

The :TRIGger:GLITch:POLarity command sets the polarity for the glitch pulse width trigger.

Query Syntax :TRIGger:GLITch:POLarity?

The :TRIGger:GLITch:POLarity? query returns the glitch pulse width trigger polarity.

Return Format <polarity><NL>
 <polarity> ::= {POS | NEG}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":TRIGger:GLITch:SOURce"](#) on page 1388

:TRIGger:GLITch:QUALifier

N (see [page 1650](#))

Command Syntax :TRIGger:GLITch:QUALifier <operator>

<operator> ::= {GREATERthan | LESSthan | RANGE}

This command sets the mode of operation of the glitch pulse width trigger. The oscilloscope can trigger on a pulse width that is greater than a time value, less than a time value, or within a range of time values.

Query Syntax :TRIGger:GLITch:QUALifier?

The :TRIGger:GLITch:QUALifier? query returns the glitch pulse width qualifier.

Return Format <operator><NL>

<operator> ::= {GRE | LESS | RANG}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:GLITch:SOURce"](#) on page 1388
 - [":TRIGger:MODE"](#) on page 1358

:TRIGger:GLITch:SOURce

N (see [page 1650](#))

Command Syntax :TRIGger:GLITch:SOURce <source>

<source> ::= {DIGital<d> | CHANnel<n>}

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :TRIGger:GLITch:SOURce command selects the channel that produces the pulse width trigger.

Query Syntax :TRIGger:GLITch:SOURce?

The :TRIGger:GLITch:SOURce? query returns the current pulse width source. If all channels are off, the query returns "NONE".

Return Format <source><NL>

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":TRIGger:GLITch:LEVel"](#) on page 1384
 - [":TRIGger:GLITch:POLarity"](#) on page 1385
 - [":TRIGger:GLITch:QUALifier"](#) on page 1386
 - [":TRIGger:GLITch:RANGe"](#) on page 1387

Example Code • ["Example Code"](#) on page 1379

:TRIGger:OR Commands

Table 160 :TRIGger:OR Commands Summary

Command	Query	Options and Query Returns
:TRIGger:OR <string> (see page 1390)	:TRIGger:OR? (see page 1390)	<string> ::= "nn...n" where n ::= {R F E X} R = rising edge, F = falling edge, E = either edge, X = don't care. Each character in the string is for an analog or digital channel as shown on the front panel display.

:TRIGger:OR

N (see [page 1650](#))

Command Syntax :TRIGger:OR <string>

<string> ::= "nn...n" where n ::= {R | F | E | X}

R = rising edge, F = falling edge, E = either edge, X = don't care.

The :TRIGger:OR command specifies the edges to include in the OR'ed edge trigger.

In the <string> parameter, each bit corresponds to a channel as described in the following table:

Oscilloscope Models	Value and Mask Bit Assignments
4 analog + 16 digital channels (mixed-signal)	Bits 0 through 15 - digital channels 0 through 15. Bits 16 through 19 - analog channels 4 through 1.
2 analog + 16 digital channels (mixed-signal)	Bits 0 through 15 - digital channels 0 through 15. Bits 16 and 17 - analog channels 2 and 1.
4 analog channels only	Bits 0 through 3 - analog channels 4 through 1.
2 analog channels only	Bits 0 and 1 - analog channels 2 and 1.

Query Syntax :TRIGger:OR?

The :TRIGger:OR? query returns the current OR'ed edge trigger string.

Return Format <string><NL>

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358

:TRIGger:PATtern Commands

Table 161 :TRIGger:PATtern Commands Summary

Command	Query	Options and Query Returns
:TRIGger:PATtern <string>[,<edge_source>,<edge>] (see page 1392)	:TRIGger:PATtern? (see page 1393)	<string> ::= "nn...n" where n ::= {0 1 X R F} when <base> = ASCII <string> ::= "0xnn...n" where n ::= {0,...,9 A,...,F X \$} when <base> = HEX <edge_source> ::= {CHANnel<n> NONE} for DSO models <edge_source> ::= {CHANnel<n> DIGital<d> NONE} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format <edge> ::= {POSitive NEGative}
:TRIGger:PATtern:FORM at <base> (see page 1394)	:TRIGger:PATtern:FORM at? (see page 1394)	<base> ::= {ASCII HEX}
:TRIGger:PATtern:GREa terthan <greater_than_time>[suffix] (see page 1395)	:TRIGger:PATtern:GREa terthan? (see page 1395)	<greater_than_time> ::= floating-point number in NR3 format [suffix] ::= {s ms us ns ps}
:TRIGger:PATtern:LESS than <less_than_time>[suffix] (see page 1396)	:TRIGger:PATtern:LESS than? (see page 1396)	<less_than_time> ::= floating-point number in NR3 format [suffix] ::= {s ms us ns ps}
:TRIGger:PATtern:QUAL ifier <qualifier> (see page 1397)	:TRIGger:PATtern:QUAL ifier? (see page 1397)	<qualifier> ::= {ENTERed GREaterthan LESSthan INRange OUTRange TIMEout}
:TRIGger:PATtern:RANG e <less_than_time>[suffix], <greater_than_time>[suffix] (see page 1398)	:TRIGger:PATtern:RANG e? (see page 1398)	<less_than_time> ::= 15 ns to 10 seconds in NR3 format <greater_than_time> ::= 10 ns to 9.99 seconds in NR3 format [suffix] ::= {s ms us ns ps}

:TRIGger:PATtern

C (see [page 1650](#))

Command Syntax :TRIGger:PATtern <pattern>

<pattern> ::= <string>[,<edge_source>,<edge>]

<string> ::= "nn...n" where n ::= {0 | 1 | X | R | F} when
<base> = ASCii

<string> ::= "0xnn...n" where n ::= {0,...,9 | A,...,F | X | \$} when
<base> = HEX

<edge_source> ::= {CHANnel<n> | NONE} for DSO models

<edge_source> ::= {CHANnel<n> | DIGital<d>
| NONE} for MSO models

<n> ::= 1 to (# of analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

<edge> ::= {POSitive | NEGative}

The :TRIGger:PATtern command specifies the channel values to be used in the pattern trigger.

In the <string> parameter, each bit corresponds to a channel as described in the following table:

Oscilloscope Models	Value and Mask Bit Assignments
4 analog + 16 digital channels (mixed-signal)	Bits 0 through 15 - digital channels 0 through 15. Bits 16 through 19 - analog channels 4 through 1.
2 analog + 16 digital channels (mixed-signal)	Bits 0 through 15 - digital channels 0 through 15. Bits 16 and 17 - analog channels 2 and 1.
4 analog channels only	Bits 0 through 3 - analog channels 4 through 1.
2 analog channels only	Bits 0 and 1 - analog channels 2 and 1.

The format of the <string> parameter depends on the :TRIGger:PATtern:FORMat command setting:

- When the format is ASCii, the string looks just like the string you see on the oscilloscope's front panel, made up of 0, 1, X (don't care), R (rising edge), and F (falling edge) characters.
- When the format is HEX, the string begins with "0x" and contains hex digit characters or X (don't care for all four bits in the nibble).

With the hex format string, you can use the <edge_source> and <edge> parameters to specify an edge on one of the channels.

NOTE

The optional <edge_source> and <edge> parameters should be sent together or not at all. The edge can be specified in the ASCII <string> parameter. If the edge source and edge parameters are used, they take precedence.

You can only specify an edge on one channel. When an edge is specified, the :TRIGger:PATtern:QUALifier does not apply.

Query Syntax :TRIGger:PATtern?

The :TRIGger:PATtern? query returns the pattern string, edge source, and edge.

Return Format <string>,<edge_source>,<edge><NL>

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:PATtern:FORMat"](#) on page 1394
 - [":TRIGger:PATtern:QUALifier"](#) on page 1397
 - [":TRIGger:MODE"](#) on page 1358

:TRIGger:PATtern:FORMat

N (see [page 1650](#))

Command Syntax :TRIGger:PATtern:FORMat <base>

<base> ::= {ASCii | HEX}

The :TRIGger:PATtern:FORMat command sets the entry (and query) number base used by the :TRIGger:PATtern command. The default <base> is ASCii.

Query Syntax :TRIGger:PATtern:FORMat?

The :TRIGger:PATtern:FORMat? query returns the currently set number base for pattern trigger patterns.

Return Format <base><NL>

<base> ::= {ASC | HEX}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:PATtern"](#) on page 1392

:TRIGger:PATtern:GREaterthan

N (see [page 1650](#))

```

Command Syntax      :TRIGger:PATtern:GREATERthan <greater_than_time>[<suffix>]
                        <greater_than_time> ::= minimum trigger duration in seconds
                                                in NR3 format
                        <suffix> ::= { s | ms | us | ns | ps }

```

The `:TRIGger:PATtern:GREaterthan` command sets the minimum duration for the defined pattern when `:TRIGger:PATtern:QUALifier` is set to `GREaterthan`. The command also sets the timeout value when the `:TRIGger:PATtern:QUALifier` is set to `TIMEout`.

Query Syntax

The :TRIGger:PATtern:GREaterthan? query returns the minimum duration time for the defined pattern.

Return Format <greater than time><NL>

See Also

- ["Introduction to :TRIGger Commands"](#) on page 1345
- [":TRIGger:PATtern"](#) on page 1392
- [":TRIGger:PATtern:QUALifier"](#) on page 1397
- [":TRIGger:MODE"](#) on page 1358

:TRIGger:PATTern:LESSthan

N (see [page 1650](#))

Command Syntax :TRIGger:PATtern:LESSthan <less_than_time>[<suffix>]

```
<less_than_time> ::= maximum trigger duration in seconds
                        in NR3 format
```

$$\langle \text{suffix} \rangle ::= \{ \text{s} \mid \text{ms} \mid \text{us} \mid \text{ns} \mid \text{ps} \}$$

The `:TRIGger:PATtern:LESSthan` command sets the maximum duration for the defined pattern when `:TRIGger:PATtern:QUALifier` is set to `LESSthan`.

Query Syntax :TRIGger:PATtern:LESSthan?

The `:TRIGger:PATtern:LESSthan?` query returns the duration time for the defined pattern.

Return Format <less_than_time><NL>

See Also

- ["Introduction to :TRIGger Commands"](#) on page 1345
- [":TRIGger:PATtern"](#) on page 1392
- [":TRIGger:PATtern:QUALifier"](#) on page 1397
- [":TRIGger:MODE"](#) on page 1358

:TRIGger:PATtern:QUALifier

N (see [page 1650](#))

Command Syntax :TRIGger:PATtern:QUALifier <qualifier>

<qualifier> ::= {ENTERed | GREaterthan | LESSthan | INRange | OUTRange
| TIMEout}

The :TRIGger:PATtern:QUALifier command qualifies when the trigger occurs:

- ENTERed – when the pattern is entered.
- LESSthan – when the pattern is present for less than a time value.
- GREaterthan – when the pattern is present for greater than a time value. The trigger occurs when the pattern exits (not when the GREaterthan time value is exceeded).
- TIMEout – when the pattern is present for greater than a time value. In this case, the trigger occurs when the GREaterthan time value is exceeded (not when the pattern exits).
- INRange – when the pattern is present for a time within a range of values.
- OUTRange – when the pattern is present for a time outside of range of values.

Pattern durations are evaluated using a timer. The timer starts on the last edge that makes the pattern (logical AND) true. Except when the TIMEout qualifier is selected, the trigger occurs on the first edge that makes the pattern false, provided the time qualifier criteria has been met.

Set the GREaterthan qualifier value with the :TRIGger:PATtern:GREaterthan command.

Set the LESSthan qualifier value with the :TRIGger:PATtern:LESSthan command.

Set the INRange and OUTRange qualifier values with the :TRIGger:PATtern:RANGE command.

Set the TIMEout qualifier value with the :TRIGger:PATtern:GREaterthan command.

Query Syntax :TRIGger:PATtern:QUALifier?

The :TRIGger:PATtern:QUALifier? query returns the trigger duration qualifier.

Return Format <qualifier><NL>

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:PATtern:GREaterthan"](#) on page 1395
 - [":TRIGger:PATtern:LESSthan"](#) on page 1396
 - [":TRIGger:PATtern:RANGE"](#) on page 1398

:TRIGger:PATTern:RANGe

N (see [page 1650](#))

```

Command Syntax      :TRIGger:PATtern:RANGE <less_than_time>[<suffix>],
                        <greater_than_time>[<suffix>]

<greater_than_time> ::= 10 ns to 9.99 seconds in NR3 format

<less_than_time>    ::= 15 ns to 10 seconds in NR3 format

<suffix>            ::= {s | ms | us | ns | ps}

```

The `:TRIGger:Pattern:Range` command sets the duration for the defined pattern when the `:TRIGger:Pattern:Qualifier` command is set to `INRange` or `OUTRange`. You can enter the parameters in any order – the smaller value becomes the `<greater_than_time>` and the larger value becomes the `<less_than_time>`.

Query Syntax

The `:TRIGger:PATtern:RANGe?` query returns the duration time for the defined pattern.

Return Format <less_than_time>,<greater_than_time><NL>

See Also

- ["Introduction to :TRIGger Commands"](#) on page 1345
- [":TRIGger:PATtern"](#) on page 1392
- [":TRIGger:PATtern:QUALifier"](#) on page 1397
- [":TRIGger:MODE"](#) on page 1358

:TRIGger:RUNT Commands

Table 162 :TRIGger:RUNT Commands Summary

Command	Query	Options and Query Returns
:TRIGger:RUNT:POLarity <polarity> (see page 1400)	:TRIGger:RUNT:POLarity? (see page 1400)	<polarity> ::= {POSitive NEGative EITHer}
:TRIGger:RUNT:QUALifier <qualifier> (see page 1401)	:TRIGger:RUNT:QUALifier? (see page 1401)	<qualifier> ::= {GREaterthan LESSthan NONE}
:TRIGger:RUNT:SOURce <source> (see page 1402)	:TRIGger:RUNT:SOURce? (see page 1402)	<source> ::= CHANnel<n> <n> ::= 1 to (# analog channels) in NR1 format
:TRIGger:RUNT:TIME <time>[suffix] (see page 1403)	:TRIGger:RUNT:TIME? (see page 1403)	<time> ::= floating-point number in NR3 format [suffix] ::= {s ms us ns ps}

:TRIGger:RUNT:POLarity

N (see [page 1650](#))

Command Syntax :TRIGger:RUNT:POLarity <polarity>

<polarity> ::= {POSitive | NEGative | EITHer}

The :TRIGger:RUNT:POLarity command sets the polarity for the runt trigger:

- POSitive – positive runt pulses.
- NEGative – negative runt pulses.
- EITHer – either positive or negative runt pulses.

Query Syntax :TRIGger:RUNT:POLarity?

The :TRIGger:RUNT:POLarity? query returns the runt trigger polarity.

Return Format <polarity><NL>

<polarity> ::= {POS | NEG | EITH}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":TRIGger:LEVel:HIGH"](#) on page 1356
 - [":TRIGger:LEVel:LOW"](#) on page 1357
 - [":TRIGger:RUNT:SOURce"](#) on page 1402

:TRIGger:RUNT:QUALifier

N (see [page 1650](#))

Command Syntax :TRIGger:RUNT:QUALifier <qualifier>

<qualifier> ::= {GREaterthan | LESSthan | NONE}

The :TRIGger:RUNT:QUALifier command selects the qualifier used for specifying runt pulse widths:

- GREaterthan – triggers on runt pulses whose width is greater than the :TRIGger:RUNT:TIME.
- LESSthan – triggers on runt pulses whose width is less than the :TRIGger:RUNT:TIME.
- NONE – triggers on runt pulses of any width.

Query Syntax :TRIGger:RUNT:QUALifier?

The :TRIGger:RUNT:QUALifier? query returns the runt trigger qualifier setting.

Return Format <qualifier><NL>

<qualifier> ::= {GRE | LESS NONE}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":TRIGger:RUNT:TIME"](#) on page 1403

:TRIGger:RUNT:SOURce

N (see [page 1650](#))

Command Syntax :TRIGger:RUNT:SOURce <source>

<source> ::= CHANnel<n>

<n> ::= 1 to (# analog channels) in NR1 format

The :TRIGger:RUNT:SOURce command selects the channel used to produce the trigger.

Query Syntax :TRIGger:RUNT:SOURce?

The :TRIGger:RUNT:SOURce? query returns the current runt trigger source.

Return Format <source><NL>

<source> ::= CHAN<n>

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:RUNT:POLarity"](#) on page 1400

:TRIGger:RUNT:TIME

N (see [page 1650](#))

Command Syntax :TRIGger:RUNT:TIME <time>[suffix]

<time> ::= floating-point number in NR3 format

[suffix] ::= {s | ms | us | ns | ps}

When triggering on runt pulses whose width is greater than or less than a certain value (see :TRIGger:RUNT:QUALifier), the :TRIGger:RUNT:TIME command specifies the time used with the qualifier.

Query Syntax :TRIGger:RUNT:TIME?

The :TRIGger:RUNT:TIME? query returns the current runt pulse qualifier time setting.

Return Format <time><NL>

<time> ::= floating-point number in NR3 format

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:RUNT:QUALifier"](#) on page 1401

:TRIGger:SHOLd Commands

Table 163 :TRIGger:SHOLd Commands Summary

Command	Query	Options and Query Returns
:TRIGger:SHOLd:SLOPe <slope> (see page 1405)	:TRIGger:SHOLd:SLOPe? (see page 1405)	<slope> ::= {NEGative POSitive}
:TRIGger:SHOLd:SOURce :CLOCK <source> (see page 1406)	:TRIGger:SHOLd:SOURce :CLOCK? (see page 1406)	<source> ::= {CHANnel<n> DIGital<d>} <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:TRIGger:SHOLd:SOURce :DATA <source> (see page 1407)	:TRIGger:SHOLd:SOURce :DATA? (see page 1407)	<source> ::= {CHANnel<n> DIGital<d>} <n> ::= 1 to (# analog channels) in NR1 format <d> ::= 0 to (# digital channels - 1) in NR1 format
:TRIGger:SHOLd:TIME:H OLD <time>[suffix] (see page 1408)	:TRIGger:SHOLd:TIME:H OLD? (see page 1408)	<time> ::= floating-point number in NR3 format [suffix] ::= {s ms us ns ps}
:TRIGger:SHOLd:TIME:S ETup <time>[suffix] (see page 1409)	:TRIGger:SHOLd:TIME:S ETup? (see page 1409)	<time> ::= floating-point number in NR3 format [suffix] ::= {s ms us ns ps}

:TRIGger:SHOLd:SLOPe

N (see [page 1650](#))

Command Syntax :TRIGger:SHOLd:SLOPe <slope>

<slope> ::= {NEGative | POSitive}

The :TRIGger:SHOLd:SLOPe command specifies whether the rising edge or the falling edge of the clock signal is used.

Query Syntax :TRIGger:SHOLd:SLOPe?

The :TRIGger:SHOLd:SLOPe? query returns the current rising or falling edge setting.

Return Format <slope><NL>

<slope> ::= {NEG | POS}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":TRIGger:SHOLd:SOURce:CLOCK"](#) on page 1406
 - [":TRIGger:SHOLd:SOURce:DATA"](#) on page 1407

:TRIGger:SHOLd:SOURce:CLOCK

N (see [page 1650](#))

Command Syntax :TRIGger:SHOLd:SOURce:CLOCK <source>

<source> ::= {CHANnel<n> | DIGital<d>}

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :TRIGger:SHOLd:SOURce:CLOCK command selects the input channel probing the clock signal.

Query Syntax :TRIGger:SHOLd:SOURce:CLOCK?

The :TRIGger:SHOLd:SOURce:CLOCK? query returns the currently set clock signal source.

Return Format <source><NL>

<source> ::= {CHAN<n> | DIG<d>}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":TRIGger:SHOLd:SLOPe"](#) on page 1405

:TRIGger:SHOLd:SOURce:DATA

N (see [page 1650](#))

Command Syntax :TRIGger:SHOLd:SOURce:DATA <source>

<source> ::= {CHANnel<n> | DIGital<d>}

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :TRIGger:SHOLd:SOURce:DATA command selects the input channel probing the data signal.

Query Syntax :TRIGger:SHOLd:SOURce:DATA?

The :TRIGger:SHOLd:SOURce:DATA? query returns the currently set data signal source.

Return Format <source><NL>

<source> ::= {CHAN<n> | DIG<d>}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":TRIGger:SHOLd:SLOPe"](#) on page 1405

:TRIGger:SHOLd:TIME:HOLD

N (see [page 1650](#))

Command Syntax :TRIGger:SHOLd:TIME:HOLD <time>[suffix]

<time> ::= floating-point number in NR3 format

[suffix] ::= {s | ms | us | ns | ps}

The :TRIGger:SHOLd:TIME:HOLD command sets the hold time.

Query Syntax :TRIGger:SHOLd:TIME:HOLD?

The :TRIGger:SHOLd:TIME:HOLD? query returns the currently specified hold time.

Return Format <time><NL>

<time> ::= floating-point number in NR3 format

See Also • ["Introduction to :TRIGger Commands"](#) on page 1345

:TRIGger:SHOLd:TIME:SETup

N (see [page 1650](#))

Command Syntax :TRIGger:SHOLd:TIME:SETup <time>[suffix]

<time> ::= floating-point number in NR3 format

[suffix] ::= {s | ms | us | ns | ps}

The :TRIGger:SHOLd:TIME:SETup command sets the setup time.

Query Syntax :TRIGger:SHOLd:TIME:SETup?

The :TRIGger:SHOLd:TIME:SETup? query returns the currently specified setup time.

Return Format <time><NL>

<time> ::= floating-point number in NR3 format

See Also • ["Introduction to :TRIGger Commands"](#) on page 1345

:TRIGger:TRANSition Commands

The :TRIGger:TRANSition commands set the rise/fall time trigger options.

Table 164 :TRIGger:TRANSition Commands Summary

Command	Query	Options and Query Returns
:TRIGger:TRANSition:QUALifier <qualifier> (see page 1411)	:TRIGger:TRANSition:QUALifier? (see page 1411)	<qualifier> ::= {GREATERthan LESSthan}
:TRIGger:TRANSition:SLOPe <slope> (see page 1412)	:TRIGger:TRANSition:SLOPe? (see page 1412)	<slope> ::= {NEGative POSitive}
:TRIGger:TRANSition:SOURce <source> (see page 1413)	:TRIGger:TRANSition:SOURce? (see page 1413)	<source> ::= CHANNEL<n> <n> ::= 1 to (# analog channels) in NR1 format
:TRIGger:TRANSition:TIME <time>[suffix] (see page 1414)	:TRIGger:TRANSition:TIME? (see page 1414)	<time> ::= floating-point number in NR3 format [suffix] ::= {s ms us ns ps}

:TRIGger:TRANSition:QUALifier

N (see [page 1650](#))

Command Syntax :TRIGger:TRANSition:QUALifier <qualifier>

<qualifier> ::= {GREATERthan | LESSthan}

The :TRIGger:TRANSition:QUALifier command specifies whether you are looking for rise/fall times greater than or less than a certain time value. The time value is set using the :TRIGger:TRANSition:TIME command.

Query Syntax :TRIGger:TRANSition:QUALifier?

The :TRIGger:TRANSition:QUALifier? query returns the current rise/fall time trigger qualifier setting.

Return Format <qualifier><NL>

<qualifier> ::= {GRE | LESS}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:TRANSition:TIME"](#) on page 1414
 - [":TRIGger:MODE"](#) on page 1358

:TRIGger:TRANSition:SLOPe

N (see [page 1650](#))

Command Syntax :TRIGger:TRANSition:SLOPe <slope>

<slope> ::= {NEGative | POSitive}

The :TRIGger:TRANSition:SLOPe command specifies a POSitive rising edge or a NEGative falling edge.

Query Syntax :TRIGger:TRANSition:SLOPe?

The :TRIGger:TRANSition:SLOPe? query returns the current rise/fall time trigger slope setting.

Return Format <slope><NL>

<slope> ::= {NEG | POS}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":TRIGger:TRANSition:SOURce"](#) on page 1413

:TRIGger:TRANSition:SOURce

N (see [page 1650](#))

Command Syntax :TRIGger:TRANSition:SOURce <source>

<source> ::= CHANnel<n>

<n> ::= 1 to (# analog channels) in NR1 format

The :TRIGger:TRANSition:SOURce command selects the channel used to produce the trigger.

Query Syntax :TRIGger:TRANSition:SOURce?

The :TRIGger:TRANSition:SOURce? query returns the current transition trigger source.

Return Format <source><NL>

<source> ::= CHAN<n>

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":TRIGger:TRANSition:SLOPe"](#) on page 1412

:TRIGger:TRANSition:TIME

N (see [page 1650](#))

Command Syntax :TRIGger:TRANSition:TIME <time>[suffix]

<time> ::= floating-point number in NR3 format

[suffix] ::= {s | ms | us | ns | ps}

The :TRIGger:TRANSition:TIME command sets the time value for rise/fall time triggers. You also use the :TRIGger:TRANSition:QUALifier command to specify whether you are triggering on times greater than or less than this time value.

Query Syntax :TRIGger:TRANSition:TIME?

The :TRIGger:TRANSition:TIME? query returns the current rise/fall time trigger time value.

Return Format <time><NL>

<time> ::= floating-point number in NR3 format

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:TRANSition:QUALifier"](#) on page 1411

:TRIGger:TV Commands

Table 165 :TRIGger:TV Commands Summary

Command	Query	Options and Query Returns
:TRIGger:TV:LINE <line number> (see page 1416)	:TRIGger:TV:LINE? (see page 1416)	<line number> ::= integer in NR1 format
:TRIGger:TV:MODE <tv mode> (see page 1417)	:TRIGger:TV:MODE? (see page 1417)	<tv mode> ::= {FIELD1 FIELD2 AFIELDS ALINES LINE LFIELD1 LFIELD2 LALTERNATE}
:TRIGger:TV:POLarity <polarity> (see page 1418)	:TRIGger:TV:POLarity? (see page 1418)	<polarity> ::= {POSitive NEGative}
:TRIGger:TV:SOURce <source> (see page 1419)	:TRIGger:TV:SOURce? (see page 1419)	<source> ::= {CHANnel<n>} <n> ::= 1 to (# analog channels) in NR1 format
:TRIGger:TV:STANdard <standard> (see page 1420)	:TRIGger:TV:STANdard? (see page 1420)	<standard> ::= {NTSC PAL PALM SECam} <standard> ::= {GENeric {P480L60HZ P480} {P720L60HZ P720} {P1080L24HZ P1080} P1080L25HZ P1080L50HZ P1080L60HZ {I1080L50HZ I1080} I1080L60HZ} with extended video triggering license
:TRIGger:TV:UDTV:ENUM ber <count> (see page 1421)	:TRIGger:TV:UDTV:ENUM ber? (see page 1421)	<count> ::= edge number in NR1 format
:TRIGger:TV:UDTV:HSYN c {{0 OFF} {1 ON}} (see page 1422)	:TRIGger:TV:UDTV:HSYN c? (see page 1422)	{0 1}
:TRIGger:TV:UDTV:HTIM e <time> (see page 1423)	:TRIGger:TV:UDTV:HTIM e? (see page 1423)	<time> ::= seconds in NR3 format
:TRIGger:TV:UDTV:PGTH an <min_time> (see page 1424)	:TRIGger:TV:UDTV:PGTH an? (see page 1424)	<min_time> ::= seconds in NR3 format

:TRIGger:TV:LINE

N (see [page 1650](#))

Command Syntax :TRIGger:TV:LINE <line_number>
 <line_number> ::= integer in NR1 format

The :TRIGger:TV:LINE command allows triggering on a specific line of video. The line number limits vary with the standard and mode, as shown in the following table.

Table 166 TV Trigger Line Number Limits

TV Standard	Mode				
	LINE	LField1	LField2	LALternate	VERTical
NTSC		1 to 263	1 to 262	1 to 262	
PAL		1 to 313	314 to 625	1 to 312	
PAL-M		1 to 263	264 to 525	1 to 262	
SECAM		1 to 313	314 to 625	1 to 312	
GENERIC		1 to 1024	1 to 1024		1 to 1024
P480L60HZ	1 to 525				
P720L60HZ	1 to 750				
P1080L24HZ	1 to 1125				
P1080L25HZ	1 to 1125				
P1080L50HZ	1 to 1125				
P1080L60HZ	1 to 1125				
I1080L50HZ	1 to 1125				
I1080L60HZ	1 to 1125				

Query Syntax :TRIGger:TV:LINE?

The :TRIGger:TV:LINE? query returns the current TV trigger line number setting.

Return Format <line_number><NL>
 <line_number> ::= integer in NR1 format

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:TV:STANdard"](#) on page 1420
 - [":TRIGger:TV:MODE"](#) on page 1417

:TRIGger:TV:MODE

N (see [page 1650](#))

Command Syntax :TRIGger:TV:MODE <mode>

<mode> ::= {FIEld1 | FIEld2 | AFIElds | ALINes | LINE | LFIEld1
 | LFIEld2 | LALTernate}

The :TRIGger:TV:MODE command selects the TV trigger mode and field. The LALTernate parameter is not available when :TRIGger:TV:STANdard is GENeric.

Old forms for <mode> are accepted:

<mode>	Old Forms Accepted
FIEld1	F1
FIEld2	F2
AFIElds	ALLFields, ALLFLDS
ALINes	ALLLines
LFIEld1	LINEF1, LINEFIELD1
LFIEld2	LINEF2, LINEFIELD2
LALTernate	LINEAlt

Query Syntax :TRIGger:TV:MODE?

The :TRIGger:TV:MODE? query returns the TV trigger mode.

Return Format <value><NL>

<value> ::= {FIE1 | FIE2 | AFI | ALIN | LINE | LFI1 | LFI2 | LALT}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:TV:STANdard"](#) on page 1420
 - [":TRIGger:MODE"](#) on page 1358

:TRIGger:TV:POLarity

N (see [page 1650](#))

Command Syntax :TRIGger:TV:POLarity <polarity>

<polarity> ::= {POSitive | NEGative}

The :TRIGger:TV:POLarity command sets the polarity for the TV trigger.

Query Syntax :TRIGger:TV:POLarity?

The :TRIGger:TV:POLarity? query returns the TV trigger polarity.

Return Format <polarity><NL>

<polarity> ::= {POS | NEG}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":TRIGger:TV:SOURce"](#) on page 1419

:TRIGger:TV:SOURce

N (see [page 1650](#))

Command Syntax :TRIGger:TV:SOURce <source>

<source> ::= {CHANnel<n>}

<n> ::= 1 to (# analog channels) in NR1 format

The :TRIGger:TV:SOURce command selects the channel used to produce the trigger.

Query Syntax :TRIGger:TV:SOURce?

The :TRIGger:TV:SOURce? query returns the current TV trigger source.

Return Format <source><NL>

<source> ::= {CHAN<n>}

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":TRIGger:TV:POLarity"](#) on page 1418

Example Code • ["Example Code"](#) on page 1379

:TRIGger:TV:STANdard

N (see [page 1650](#))

Command Syntax :TRIGger:TV:STANdard <standard>

```

<standard> ::= {GENeric | NTSC | PALM | PAL | SECam
                | {P480L60HZ | P480} | {P720L60HZ | P720}
                | {P1080L24HZ | P1080} | P1080L25HZ
                | P1080L50HZ | P1080L60HZ
                | {I1080L50HZ | I1080} | I1080L60HZ}

```

The :TRIGger:TV:STANdard command selects the video standard:

- NTSC
- PAL
- PAL-M
- SECAM

With an extended Video triggering license, the oscilloscope additionally supports these standards:

- Generic – GENeric mode is non-interlaced.
- EDTV 480p/60
- HDTV 720p/60
- HDTV 1080p/24
- HDTV 1080p/25
- HDTV 1080i/50
- HDTV 1080i/60

Query Syntax :TRIGger:TV:STANdard?

The :TRIGger:TV:STANdard? query returns the current TV trigger standard setting.

Return Format <standard><NL>

```

<standard> ::= {GEN | NTSC | PALM | PAL | SEC | P480L60HZ | P760L60HZ
                | P1080L24HZ | P1080L25HZ | P1080L50HZ | P1080L60HZ
                | I1080L50HZ | I1080L60HZ}

```


:TRIGger:TV:UDTV:ENUMber

N (see [page 1650](#))

Command Syntax :TRIGger:TV:UDTV:ENUMber <count>

<count> ::= edge number in NR1 format

The :TRIGger:TV:UDTV:ENUMber command specifies the Generic video trigger's Nth edge to trigger on after synchronizing with the vertical sync.

Query Syntax :TRIGger:TV:UDTV:ENUMber?

The :TRIGger:TV:UDTV:ENUMber query returns the edge count setting.

Return Format <count><NL>

<count> ::= edge number in NR1 format

- See Also**
- [":TRIGger:TV:STANdard"](#) on page 1420
 - [":TRIGger:TV:UDTV:PGTHan"](#) on page 1424
 - [":TRIGger:TV:UDTV:HSYNc"](#) on page 1422

:TRIGger:TV:UDTV:HSYNc

N (see [page 1650](#))

Command Syntax :TRIGger:TV:UDTV:HSYNc {{0 | OFF} | {1 | ON}}

The :TRIGger:TV:UDTV:HSYNc command enables or disables the horizontal sync control in the Generic video trigger.

For interleaved video, enabling the HSYNc control and setting the HTIME adjustment to the sync time of the probed video signal allows the ENUMber function to count only lines and not double count during equalization. Additionally, the Field Holdoff can be adjusted so that the oscilloscope triggers once per frame.

Similarly, for progressive video with a tri-level sync, enabling the HSYNc control and setting the HTIME adjustment to the sync time of the probed video signal allows the ENUMber function to count only lines and not double count during vertical sync.

Query Syntax :TRIGger:TV:UDTV:HSYNc?

The :TRIGger:TV:UDTV:HSYNc query returns the horizontal sync control setting.

Return Format {0 | 1}

- See Also**
- [":TRIGger:TV:STANdard"](#) on page 1420
 - [":TRIGger:TV:UDTV:HTIME"](#) on page 1423
 - [":TRIGger:TV:UDTV:ENUMber"](#) on page 1421
 - [":TRIGger:TV:UDTV:PGTHan"](#) on page 1424

:TRIGger:TV:UDTV:HTIME

N (see [page 1650](#))

Command Syntax :TRIGger:TV:UDTV:HTIME <time>

<time> ::= seconds in NR3 format

When the Generic video trigger's horizontal sync control is enabled, the :TRIGger:TV:UDTV:HTIME command sets the minimum time the horizontal sync pulse must be present to be considered valid.

Query Syntax :TRIGger:TV:UDTV:HTIME?

The :TRIGger:TV:UDTV:HTIME query returns the horizontal sync time setting.

Return Format <time><NL>

<time> ::= seconds in NR3 format

- See Also**
- [":TRIGger:TV:STANdard"](#) on page 1420
 - [":TRIGger:TV:UDTV:HSYNc"](#) on page 1422

:TRIGger:TV:UDTV:PGTHan

N (see [page 1650](#))

Command Syntax :TRIGger:TV:UDTV:PGTHan <min_time>
 <min_time> ::= seconds in NR3 format

The :TRIGger:TV:UDTV:PGTHan command specifies the "greater than the sync pulse width" time in the Generic video trigger. This setting allows oscilloscope synchronization to the vertical sync.

Query Syntax :TRIGger:TV:UDTV:PGTHan?

The :TRIGger:TV:UDTV:PGTHan query returns the "greater than the sync pulse width" time setting.

Return Format <min_time><NL>
 <min_time> ::= seconds in NR3 format

- See Also**
- [":TRIGger:TV:STANdard"](#) on page 1420
 - [":TRIGger:TV:UDTV:ENUMber"](#) on page 1421
 - [":TRIGger:TV:UDTV:HSYNc"](#) on page 1422

:TRIGger:ZONE Commands

Table 167 :TRIGger:ZONE Commands Summary

Command	Query	Options and Query Returns
:TRIGger:ZONE:SOURce <source> (see page 1426)	:TRIGger:ZONE:SOURce? (see page 1426)	<source> ::= {CHANnel<n>} <n> ::= 1 to (# analog channels) in NR1 format
:TRIGger:ZONE:STATe {{0 OFF} {1 ON}} (see page 1427)	:TRIGger:ZONE:STATe? (see page 1427)	{0 1}
:TRIGger:ZONE<n>:MODE <mode> (see page 1428)	:TRIGger:ZONE<n>:MODE ? (see page 1428)	<mode> ::= {INTERsect NOTintersect} <n> ::= 1-2 in NR1 format
:TRIGger:ZONE<n>:PLAC ement <width>, <height>, <x_center>, <y_center> (see page 1429)	:TRIGger:ZONE<n>:PLAC ement? (see page 1429)	<width> ::= width of zone in seconds <height> ::= height of zone in volts <x_center> ::= center of zone in seconds <y_center> ::= center of zone in volts <n> ::= 1-2 in NR1 format
n/a	:TRIGger:ZONE<n>:VALi dity? (see page 1430)	<value> ::= {VALid INValid OSCRreen} <n> ::= 1-2 in NR1 format
:TRIGger:ZONE<n>:STAT e {{0 OFF} {1 ON}} (see page 1431)	:TRIGger:ZONE<n>:STAT e? (see page 1431)	{0 1} <n> ::= 1-2 in NR1 format

:TRIGger:ZONE:SOURce

N (see [page 1650](#))

Command Syntax :TRIGger:ZONE:SOURce <source>

<source> ::= {CHANnel<n>}

<n> ::= 1 to (# analog channels) in NR1 format

The :TRIGger:ZONE:SOURce command sets the analog source channel shared by all zones.

Query Syntax :TRIGger:ZONE:SOURce?

The :TRIGger:ZONE:SOURce? query returns the analog source channel specified for zone qualified triggers.

Return Format <source><NL>

<source> ::= {CHAN<n>}

See Also • [":TRIGger:ZONE:STAtE"](#) on page 1427

:TRIGger:ZONE:STATe

N (see [page 1650](#))

Command Syntax :TRIGger:ZONE:STATe <on_off>

<on_off> ::= {{0 | OFF} | {1 | ON}}

The :TRIGger:ZONE:STATe command enables or disables the zone qualified trigger feature.

When the zone qualified trigger is on, the zone(s) are actively being used to qualify the trigger.

Note that the :TRIGger:ZONE<n>:STATe setting must also be ON for a zone to be active.

Note that :TRIGger:ZONE:STATe mimics the behavior of the **[Zone]** key on the front panel, and :TRIGger:ZONE<n>:STATe mimics the behavior of the **Zone 1 On** and **Zone 2 On** softkeys. At least one zone's state must be on for the Zone Trigger feature (:TRIGger:ZONE:STATe) to be on. When the states of both individual zones are turned off, Zone Trigger is automatically turned off. In this case, when Zone Trigger is turned back on Zone 1 is forced to on. Otherwise, if at least one zone was on when Zone Trigger was turned off, the same configuration of individual zone on/off states will be restored when Zone Trigger is turned back on.

Query Syntax :TRIGger:ZONE:STATe?

The :TRIGger:ZONE:STATe? query returns whether the zone qualified trigger feature is enabled or disabled.

Return Format <on_off><NL>

<on_off> ::= {0 | 1}

- See Also**
- **":TRIGger:ZONE<n>:STATe"** on page 1431
 - **":TRIGger:ZONE:SOURce"** on page 1426

:TRIGger:ZONE<n>:MODE

N (see [page 1650](#))

Command Syntax :TRIGger:ZONE<n>:MODE <mode>

<mode> ::= {INTersect | NOTintersect}

<n> ::= 1-2 in NR1 format

The :TRIGger:ZONE<n>:MODE command sets the zone qualifying condition for Zone 1 or Zone 2 as either "Must Intersect" or "Must Not Intersect".

Query Syntax :TRIGger:ZONE<n>:MODE?

The :TRIGger:ZONE<n>:MODE? query returns the zone qualifying condition for Zone 1 or Zone 2.

Return Format <mode><NL>

<mode> ::= {INT | NOT}

- See Also**
- [":TRIGger:ZONE<n>:STATe"](#) on page 1431
 - [":TRIGger:ZONE<n>:PLACement"](#) on page 1429
 - [":TRIGger:ZONE<n>:VALidity"](#) on page 1430

:TRIGger:ZONE<n>:PLACement

N (see [page 1650](#))

Command Syntax :TRIGger:ZONE<n>:PLACement <width>, <height>, <x_center>, <y_center>

<width> ::= width of zone in seconds

<height> ::= height of zone in volts

<x_center> ::= center of zone in seconds

<y_center> ::= center of zone in volts

<n> ::= 1-2 in NR1 format

The :TRIGger:ZONE<n>:PLACement command sets the size and location of Zone 1 or Zone 2.

No error is returned if the zone is placed off-screen, or if the zones overlap such that Zone 2 becomes invalid. The :TRIGger:ZONE<n>:VALidity? query is used to retrieve this information.

Query Syntax :TRIGger:ZONE<n>:PLACement?

The :TRIGger:ZONE<n>:PLACement? query returns the size and location of Zone 1 or Zone 2.

Return Format <opt><NL>

<opt> ::= <width>, <height>, <x_center>, <y_center>

- See Also**
- [":TRIGger:ZONE<n>:STATe"](#) on page 1431
 - [":TRIGger:ZONE<n>:MODE"](#) on page 1428
 - [":TRIGger:ZONE<n>:VALidity"](#) on page 1430

:TRIGger:ZONE<n>:VALidity

N (see [page 1650](#))

Query Syntax :TRIGger:ZONE<n>:VALidity?

<n> ::= 1-2 in NR1 format

The :TRIGger:ZONE<n>:VALidity? query returns the validity of Zone 1 or Zone 2.

- INValid is returned (for Zone 2 only) when Zone 1 and Zone 2 overlap and have opposing qualifying conditions (modes). Zone 1 can never be invalid.
- OSCReen (off-screen) is returned when the associated zone is off-screen, and thus not being used to qualify the trigger.
- A zone is valid when it is neither invalid nor off-screen.

The validity of a zone is not affected by the zone's state. For example, a zone can be valid and off. You cannot directly set the validity of a zone.

Return Format <validity><NL>

<validity> ::= {VALid | INValid | OSCReen}

- See Also**
- [":TRIGger:ZONE<n>:STATe"](#) on page 1431
 - [":TRIGger:ZONE<n>:MODE"](#) on page 1428
 - [":TRIGger:ZONE<n>:PLACement"](#) on page 1429

:TRIGger:ZONE<n>:STATe

N (see [page 1650](#))

Command Syntax :TRIGger:ZONE<n>:STATe <on_off>

<n> ::= {1 | 2}

<on_off> ::= {{0 | OFF} | {1 | ON}}

<n> ::= 1-2 in NR1 format

The :TRIGger:ZONE<n>:STATe command sets the state for Zone 1 or Zone 2.

- When a zone's state is on, and the Zone Trigger feature is on (see [":TRIGger:ZONE:STATe"](#) on page 1427), that zone is actively being used to qualify the trigger if it is not invalid or off-screen (see [":TRIGger:ZONE<n>:VALidity"](#) on page 1430).
- When the Zone Trigger feature is off, no zones are being used to qualify the trigger, regardless of their individual states.

Note that :TRIGger:ZONE:STATe mimics the behavior of the **[Zone]** key on the front panel, and :TRIGger:ZONE<n>:STATe mimics the behavior of the **Zone 1 On** and **Zone 2 On** softkeys. At least one zone's state must be on for the Zone Trigger feature (:TRIGger:ZONE:STATe) to be on. When the states of both individual zones are turned off, Zone Trigger is automatically turned off. In this case, when Zone Trigger is turned back on Zone 1 is forced to on. Otherwise, if at least one zone was on when Zone Trigger was turned off, the same configuration of individual zone on/off states will be restored when Zone Trigger is turned back on.

Query Syntax :TRIGger:ZONE<n>:STATe?

The :TRIGger:ZONE<n>:STATe? query returns the state of Zone 1 or Zone 2.

Return Format <on_off><NL>

<on_off> ::= {0 | 1}

- See Also**
- [":TRIGger:ZONE:STATe"](#) on page 1427
 - [":TRIGger:ZONE<n>:VALidity"](#) on page 1430

38 :WAVeform Commands

Provide access to waveform data. See ["Introduction to :WAVeform Commands"](#) on page 1436.

Table 168 :WAVeform Commands Summary

Command	Query	Options and Query Returns
:WAVeform:BYTeorder <value> (see page 1441)	:WAVeform:BYTeorder? (see page 1441)	<value> ::= {LSBFirst MSBFirst}
n/a	:WAVeform:COUNT? (see page 1442)	<count> ::= an integer from 1 to 65536 in NR1 format
n/a	:WAVeform:DATA? (see page 1443)	<binary block length bytes>, <binary data> For example, to transmit 1000 bytes of data, the syntax would be: #800001000<1000 bytes of data><NL> 8 is the number of digits that follow 00001000 is the number of bytes to be transmitted <1000 bytes of data> is the actual data
:WAVeform:FORMat <value> (see page 1445)	:WAVeform:FORMat? (see page 1445)	<value> ::= {WORD BYTE ASCII}
:WAVeform:POINTs <wfm_points> (see page 1446)	:WAVeform:POINTs? (see page 1446)	<wfm_points> ::= {<#_points> <points_mode>} <#_points> ::= an integer between 100 and 10000000 <points_mode> ::= {NORMal MAXimum RAW}

Table 168 :WAVEform Commands Summary (continued)

Command	Query	Options and Query Returns
:WAVEform:POINTs:MODE <points_mode> (see page 1448)	:WAVEform:POINTs:MODE ? (see page 1449)	<points_mode> ::= {NORMAL MAXimum RAW}
n/a	:WAVEform:PREamble? (see page 1450)	<pre><preamble_block> ::= <format NR1>, <type NR1>,<points NR1>,<count NR1>, <xincrement NR3>, <xorigin NR3>, <xreference NR1>,<yincrement NR3>, <yorigin NR3>, <yreference NR1></pre> <pre><format> ::= an integer in NR1 format:</pre> • 0 for BYTE format • 1 for WORD format • 2 for ASCII format <pre><type> ::= an integer in NR1 format:</pre> • 0 for NORMAL type • 1 for PEAK detect type • 3 for AVERAGE type • 4 for HRESolution type <pre><count> ::= Average count, or 1 if PEAK detect type or NORMAL; an integer in NR1 format</pre>
:WAVEform:SEGmented:ALL {0 OFF} {1 ON}} (see page 1453)	:WAVEform:SEGmented:ALL? (see page 1453)	<setting> ::= {0 1}
n/a	:WAVEform:SEGmented:COUNT? (see page 1454)	<count> ::= an integer from 2 to 1000 in NR1 format
n/a	:WAVEform:SEGmented:TAG? (see page 1455)	<time_tag> ::= in NR3 format
n/a	:WAVEform:SEGmented:XLIST? <xlist_type> (see page 1456)	<pre><xlist_type> ::= {RELXorigin ABSXorigin TTAG}</pre> <pre><return_value> ::= X-info for all segments</pre>

Table 168 :WAVeform Commands Summary (continued)

Command	Query	Options and Query Returns
:WAVeform:SOURce <source> (see page 1457)	:WAVeform:SOURce? (see page 1457)	<source> ::= {CHANnel<n> FUNCTION<m> MATH<m> SBUS} for DSO models <source> ::= {CHANnel<n> POD{1 2} BUS{1 2} FUNCTION<m> MATH<m> SBUS} for MSO models <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format
:WAVeform:SOURce:SUBS ource <subsource> (see page 1461)	:WAVeform:SOURce:SUBS ource? (see page 1461)	<subsource> ::= {{SUB0 RX MOSI} {SUB1 TX MISO}}
n/a	:WAVeform:TYPE? (see page 1462)	<return_mode> ::= {NORM PEAK AVER HRES}
:WAVeform:UNSigned {0 OFF} {1 ON} (see page 1463)	:WAVeform:UNSigned? (see page 1463)	{0 1}
:WAVeform:VIEW <view> (see page 1464)	:WAVeform:VIEW? (see page 1464)	<view> ::= {MAIN ALL}
n/a	:WAVeform:XINCrement? (see page 1465)	<return_value> ::= x-increment in the current preamble in NR3 format
n/a	:WAVeform:XORigin? (see page 1466)	<return_value> ::= x-origin value in the current preamble in NR3 format
n/a	:WAVeform:XREFerence? (see page 1467)	<return_value> ::= 0 (x-reference value in the current preamble in NR1 format)
n/a	:WAVeform:YINCrement? (see page 1468)	<return_value> ::= y-increment value in the current preamble in NR3 format
n/a	:WAVeform:YORigin? (see page 1469)	<return_value> ::= y-origin in the current preamble in NR3 format
n/a	:WAVeform:YREFerence? (see page 1470)	<return_value> ::= y-reference value in the current preamble in NR1 format

Introduction to :WAVEform Commands The WAVEform subsystem is used to transfer data to a controller from the oscilloscope waveform memories. The queries in this subsystem will only operate when the channel selected by :WAVEform:SOURce is on.

Waveform Data and Preamble

The waveform record is actually contained in two portions: the preamble and waveform data. The waveform record must be read from the oscilloscope by the controller using two separate commands, :WAVEform:DATA (see [page 1443](#)) and :WAVEform:PREAmble (see [page 1450](#)). The waveform data is the actual data acquired for each point in the specified source. The preamble contains the information for interpreting the waveform data, which includes the number of points acquired, the format of acquired data, and the type of acquired data. The preamble also contains the X and Y increments, origins, and references for the acquired data, so that word and byte data can be translated to time and voltage values.

Data Acquisition Types

There are four types of waveform acquisitions that can be selected for analog channels with the :ACQuire:TYPE command (see [page 300](#)): NORMal, AVERage, PEAK, and HRESolution. Digital channels are always acquired using NORMal. When the data is acquired using the :DIGitize command (see [page 250](#)) or :RUN command (see [page 274](#)), the data is placed in the channel buffer of the specified source.

Once you have acquired data with the :DIGitize command, the instrument is stopped. If the instrument is restarted (via the programming interface or the front panel), or if any instrument setting is changed, the data acquired with the :DIGitize command may be overwritten. You should first acquire the data with the :DIGitize command, then immediately read the data with the :WAVEform:DATA? query (see [page 1443](#)) before changing any instrument setup.

A waveform record consists of either all of the acquired points or a subset of the acquired points. The number of points acquired may be queried using :ACQuire:POINTS[:ANALog]? (see [page 290](#)).

Helpful Hints:

The number of points transferred to the computer is controlled using the :WAVEform:POINTS command (see [page 1446](#)). If :WAVEform:POINTS MAXimum is specified and the instrument is not running (stopped), all of the points that are displayed are transferred. This can be as many as 4,000,000 in some operating modes or as many as 8,000,000 for a digital channel on the mixed signal oscilloscope. Fewer points may be specified to speed data transfers and minimize controller analysis time. The :WAVEform:POINTS may be varied even after data on a channel is acquired. However, this decimation may result in lost pulses and transitions. The number of points selected for transfer using :WAVEform:POINTS

must be an even divisor of 1,000 or be set to MAXimum. :WAVEform:POINTS determines the increment between time buckets that will be transferred. If POINTs = MAXimum, the data cannot be decimated. For example:

- :WAVEform:POINTS 1000 — returns time buckets 0, 1, 2, 3, 4 ..., 999.
- :WAVEform:POINTS 500 — returns time buckets 0, 2, 4, 6, 8 ..., 998.
- :WAVEform:POINTS 250 — returns time buckets 0, 4, 8, 12, 16 ..., 996.
- :WAVEform:POINTS 100 — returns time buckets 0, 10, 20, 30, 40 ..., 990.

Analog Channel Data

NORMAL Data

Normal data consists of the last data point (hit) in each time bucket. This data is transmitted over the programming interface in a linear fashion starting with time bucket 0 and going through time bucket $n - 1$, where n is the number returned by the :WAVEform:POINTS? query (see [page 1446](#)). Only the magnitude values of each data point are transmitted. The first voltage value corresponds to the first time bucket on the left side of the screen and the last value corresponds to the next-to-last time bucket on the right side of the screen. Time buckets without data return 0. The time values for each data point correspond to the position of the data point in the data array. These time values are not transmitted.

AVERAge Data

AVERAge data consists of the average of the first n hits in a time bucket, where n is the value returned by the :ACquire:COUNT query (see [page 286](#)). Time buckets that have fewer than n hits return the average of the data they do have. If a time bucket does not have any data in it, it returns 0.

This data is transmitted over the interface linearly, starting with time bucket 0 and proceeding through time bucket $n-1$, where n is the number returned by the :WAVEform:POINTS? query (see [page 1446](#)). The first value corresponds to a point at the left side of the screen and the last value corresponds to one point away from the right side of the screen. The maximum number of points that can be returned in average mode is 1000 unless ACquire:COUNT has been set to 1.

PEAK Data

Peak detect display mode is used to detect glitches for time base settings of 500 us/div and slower. In this mode, the oscilloscope can sample more data than it can store and display. So, when peak detect is turned on, the oscilloscope scans through the extra data, picks up the minimum and maximum for each time bucket, then stores the data in an array. Each time bucket contains two data sample.

The array is transmitted over the interface bus linearly, starting with time bucket 0 proceeding through time bucket $n-1$, where n is the number returned by the :WAVEform:POINTS? query (see [page 1446](#)). In each time bucket, two values are transmitted, first the minimum, followed by the maximum. The first pair of values

corresponds to the time bucket at the leftmost side of the screen. The last pair of values corresponds to the time bucket at the far right side of the screen. In :ACQUIRE:TYPE PEAK mode (see [page 300](#)), the value returned by the :WAVEform:XINCrement query (see [page 1465](#)) should be doubled to find the time difference between the min-max pairs.

HRESolution Data

The high resolution (*smoothing*) mode is used to reduce noise at slower sweep speeds where the digitizer samples faster than needed to fill memory for the displayed time range.

Data Conversion

Word or byte data sent from the oscilloscope must be scaled for useful interpretation. The values used to interpret the data are the X and Y references, X and Y origins, and X and Y increments. These values are read from the waveform preamble. Each channel has its own waveform preamble.

In converting a data value to a voltage value, the following formula is used:

$$\text{voltage} = [(\text{data value} - \text{yreference}) * \text{yincrement}] + \text{yorigin}$$

If the :WAVEform:FORMat data format is ASCII (see [page 1445](#)), the data values are converted internally and sent as floating point values separated by commas.

In converting a data value to time, the time value of a data point can be determined by the position of the data point. For example, the fourth data point sent with :WAVEform:XORigin = 16 ns, :WAVEform:XREFerence = 0, and :WAVEform:XINCrement = 2 ns, can be calculated using the following formula:

$$\text{time} = [(\text{data point number} - \text{xreference}) * \text{xincrement}] + \text{xorigin}$$

This would result in the following calculation for time bucket 3:

$$\text{time} = [(3 - 0) * 2 \text{ ns}] + 16 \text{ ns} = 22 \text{ ns}$$

In :ACQUIRE:TYPE PEAK mode (see [page 300](#)), because data is acquired in max-min pairs, modify the previous time formula to the following:

$$\text{time} = [(\text{data pair number} - \text{xreference}) * \text{xincrement} * 2] + \text{xorigin}$$

Data Format for Transfer

There are three formats for transferring waveform data over the interface: BYTE, WORD and ASCII (see [":WAVEform:FORMat"](#) on page 1445). BYTE, WORD and ASCII formatted waveform records are transmitted using the arbitrary block program data format specified in IEEE 488.2.

When you use the block data format, the ASCII character string "#8<DD...D>" is sent prior to sending the actual data. The 8 indicates how many Ds follow. The Ds are ASCII numbers that indicate how many data bytes follow.

For example, if 1000 points will be transferred, and the WORD format was specified, the block header "#800001000" would be sent. The 8 indicates that eight length bytes follow, and 00001000 indicates that 1000 binary data bytes follow.

Use the :WAVEform:UNSIGNED command (see [page 1463](#)) to control whether data values are sent as unsigned or signed integers. This command can be used to match the instrument's internal data type to the data type used by the programming language. This command has no effect if the data format is ASCII.

Data Format for Transfer - ASCII format

The ASCII format (see [":WAVEform:FORMat"](#) on page 1445) provides access to the waveform data as real Y-axis values without using Y origin, Y reference, and Y increment to convert the binary data. Values are transferred as ASCII digits in floating point format separated by commas. In ASCII format, holes are represented by the value 9.9e+37. The setting of :WAVEform:BYTeorder (see [page 1441](#)) and :WAVEform:UNSIGNED (see [page 1463](#)) have no effect when the format is ASCII.

Data Format for Transfer - WORD format

WORD format (see [":WAVEform:FORMat"](#) on page 1445) provides 16-bit access to the waveform data. In the WORD format, the number of data bytes is twice the number of data points. The number of data points is the value returned by the :WAVEform:POINts? query (see [page 1446](#)). If the data intrinsically has less than 16 bits of resolution, the data is left-shifted to provide 16 bits of resolution and the least significant bits are set to 0. Currently, the greatest intrinsic resolution of any data is 12 bits, so at least the lowest 4 bits of data will be 0. If there is a hole in the data, the hole is represented by a 16 bit value equal to 0.

Use :WAVEform:BYTeorder (see [page 1441](#)) to determine if the least significant byte or most significant byte is to be transferred first. The :BYTeorder command can be used to alter the transmit sequence to match the storage sequence of an integer in the programming language being used.

Data Format for Transfer - BYTE format

The BYTE format (see [":WAVEform:FORMat"](#) on page 1445) allows 8-bit access to the waveform data. If the data intrinsically has more than 8 bits of resolution (averaged data), the data is right-shifted (truncated) to fit into 8 bits. If there is a hole in the data, the hole is represented by a value of 0. The BYTE-formatted data transfers over the programming interface faster than ASCII or WORD-formatted data, because in ASCII format, as many as 13 bytes per point are transferred, in BYTE format one byte per point is transferred, and in WORD format two bytes per point are transferred.

The :WAVEform:BYTeorder command (see [page 1441](#)) has no effect when the data format is BYTE.

Digital Channel Data (MSO models only)

The waveform record for digital channels is similar to that of analog channels. The main difference is that the data points represent either DIGital0,...,7 (POD1), DIGital8,...,15 (POD2), or any grouping of digital channels (BUS1 or BUS2).

For digital channels, :WAVEform:UNSigned (see [page 1463](#)) must be set to ON.

Digital Channel POD Data Format

Data for digital channels is only available in groups of 8 bits (Pod1 = D0 - D7, Pod2 = D8 - D15). The bytes are organized as:

:WAVEform:SOURce	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
POD1	D7	D6	D5	D4	D3	D2	D1	D0
POD2	D15	D14	D13	D12	D11	D10	D9	D8

If the :WAVEform:FORMat is WORD (see [page 1445](#)) is WORD, every other data byte will be 0. The setting of :WAVEform:BYTeorder (see [page 1441](#)) controls which byte is 0.

If a digital channel is not displayed, its bit value in the pod data byte is not defined.

Digital Channel BUS Data Format

Digital channel BUS definitions can include any or all of the digital channels. Therefore, data is always returned as 16-bit values. :BUS commands (see [page 303](#)) are used to select the digital channels for a bus.

Reporting the Setup

The following is a sample response from the :WAVEform? query. In this case, the query was issued following a *RST command.

```
:WAV:UNS 1;VIEW MAIN;BYT MSBF;FORM BYTE;POIN +1000;SOUR CHAN1;SOUR:SUBS
NONE
```

:WAVeform:BYTeorder

C (see [page 1650](#))

Command Syntax :WAVeform:BYTeorder <value>
 <value> ::= {LSBFirst | MSBFirst}

The :WAVeform:BYTeorder command sets the output sequence of the WORD data.

- MSBFirst – sets the most significant byte to be transmitted first.
- LSBFirst – sets the least significant byte to be transmitted first.

This command affects the transmitting sequence only when :WAVeform:FORMat WORD is selected.

The default setting is MSBFirst.

Query Syntax :WAVeform:BYTeorder?

The :WAVeform:BYTeorder query returns the current output sequence.

Return Format <value><NL>
 <value> ::= {LSBF | MSBF}

- See Also**
- ["Introduction to :WAVeform Commands"](#) on page 1436
 - [":WAVeform:DATA"](#) on page 1443
 - [":WAVeform:FORMat"](#) on page 1445
 - [":WAVeform:PREamble"](#) on page 1450

- Example Code**
- ["Example Code"](#) on page 1458
 - ["Example Code"](#) on page 1451

:WAVeform:COUNT

C (see [page 1650](#))

Query Syntax :WAVeform:COUNT?

The :WAVeform:COUNT? query returns the count used to acquire the current waveform. This may differ from current values if the unit has been stopped and its configuration modified. For all acquisition types except average, this value is 1.

Return Format <count_argument><NL>

<count_argument> ::= an integer from 1 to 65536 in NR1 format

- See Also**
- ["Introduction to :WAVeform Commands"](#) on page 1436
 - [":ACQuire:COUNT"](#) on page 286
 - [":ACQuire:TYPE"](#) on page 300

:WAVEform:DATA

C (see [page 1650](#))

Query Syntax :WAVEform:DATA?

The :WAVEform:DATA query returns the binary block of sampled data points transmitted using the IEEE 488.2 arbitrary block data format. The binary data is formatted according to the settings of the :WAVEform:UNSIGNED, :WAVEform:BYTEorder, :WAVEform:FORMat, and :WAVEform:SOURce commands. The number of points returned is controlled by the :WAVEform:POINTS command.

In BYTE or WORD waveform formats, these data values have special meaning:

- 0x00 or 0x0000 – Hole. Holes are locations where data has not yet been acquired.

Another situation where there can be zeros in the data, incorrectly, is when programming over telnet port 5024. Port 5024 provides a command prompt and is intended for ASCII transfers. Use telnet port 5025 instead.

- 0x01 or 0x0100 – Clipped low. These are locations where the waveform is clipped at the bottom of the oscilloscope display.
- 0xFF or 0xFF00 – Clipped high. These are locations where the waveform is clipped at the top of the oscilloscope display.

Return Format <binary block data><NL>

- See Also**
- For a more detailed description of the data returned for different acquisition types, see: "[Introduction to :WAVEform Commands](#)" on page 1436
 - "[:WAVEform:UNSIGNED](#)" on page 1463
 - "[:WAVEform:BYTEorder](#)" on page 1441
 - "[:WAVEform:FORMat](#)" on page 1445
 - "[:WAVEform:POINTS](#)" on page 1446
 - "[:WAVEform:PREamble](#)" on page 1450
 - "[:WAVEform:SOURce](#)" on page 1457
 - "[:WAVEform:TYPE](#)" on page 1462

Example Code

```
' QUERY_WAVE_DATA - Outputs waveform data that is stored in a buffer.

' Query the oscilloscope for the waveform data.
myScope.WriteString ":WAV:DATA?"

' READ_WAVE_DATA - The wave data consists of two parts: the header,
' and the actual waveform data followed by a new line (NL) character.
' The query data has the following format:
'
'     <header><waveform_data><NL>
'
' Where:
```

```

' <header> = #800001000 (This is an example header)
' The "#8" may be stripped off of the header and the remaining
' numbers are the size, in bytes, of the waveform data block. The
' size can vary depending on the number of points acquired for the
' waveform. You can then read that number of bytes from the
' oscilloscope and the terminating NL character.
'
Dim lngI As Long
Dim lngDataValue As Long

varQueryResult = myScope.ReadIEEEBlock(BinaryType_UI1)
' Unsigned integer bytes.
For lngI = 0 To UBound(varQueryResult) - 1
    Step (UBound(varQueryResult) / 20) ' 20 points.
    If intBytesPerData = 2 Then
        lngDataValue = varQueryResult(lngI) * 256 _
            + varQueryResult(lngI + 1) ' 16-bit value.
    Else
        lngDataValue = varQueryResult(lngI) ' 8-bit value.
    End If
    strOutput = strOutput + "Data point " + _
        CStr(lngI / intBytesPerData) + ", " + _
        FormatNumber((lngDataValue - lngYReference) _
            * sngYIncrement + sngYOrigin) + " V, " + _
        FormatNumber(((lngI / intBytesPerData - lngXReference) _
            * sngXIncrement + dblXOrigin) * 1000000) + " us" + vbCrLf
Next lngI
MsgBox "Waveform data:" + vbCrLf + strOutput

```

See complete example programs at: [Chapter 46](#), "Programming Examples," starting on page 1659

:WAVEform:FORMat

C (see [page 1650](#))

Command Syntax :WAVEform:FORMat <value>

<value> ::= {WORD | BYTE | ASCii}

The :WAVEform:FORMat command sets the data transmission mode for waveform data points. This command controls how the data is formatted when sent from the oscilloscope.

- ASCII formatted data converts the internal integer data values to real Y-axis values. Values are transferred as ASCII digits in floating point notation, separated by commas.
ASCII formatted data is transferred ASCII text.
- WORD formatted data transfers 16-bit data as two bytes. The :WAVEform:BYTEorder command can be used to specify whether the upper or lower byte is transmitted first. The default (no command sent) is that the upper byte transmitted first.
- BYTE formatted data is transferred as 8-bit bytes.

When the :WAVEform:SOURce is the serial decode bus (SBUS1 or SBUS2), ASCII is the only waveform format allowed.

When the :WAVEform:SOURce is one of the digital channel buses (BUS1 or BUS2), ASCII and WORD are the only waveform formats allowed.

Query Syntax :WAVEform:FORMat?

The :WAVEform:FORMat query returns the current output format for the transfer of waveform data.

Return Format <value><NL>

<value> ::= {WORD | BYTE | ASC}

- See Also**
- ["Introduction to :WAVEform Commands"](#) on page 1436
 - [":WAVEform:BYTEorder"](#) on page 1441
 - [":WAVEform:SOURce"](#) on page 1457
 - [":WAVEform:DATA"](#) on page 1443
 - [":WAVEform:PREamble"](#) on page 1450

Example Code • ["Example Code"](#) on page 1458

:WAVEform:POINts

C (see [page 1650](#))

Command Syntax :WAVEform:POINts <wfm_points>

 <wfm_points> ::= {<#_points> | <points_mode>}

 <#_points> ::= an integer between 100 and 100000000

 <points_mode> ::= {NORMal | MAXimum | RAW}

NOTE

The <points_mode> option is deprecated. Use the :WAVEform:POINts:MODE command instead.

The :WAVEform:POINts command sets the desired number of waveform points to be transferred with the :WAVEform:DATA? query. This value represents the points contained in the waveform selected with the :WAVEform:SOURce command.

For the analog or digital sources, the records that can be transferred depend on the waveform points mode. The maximum number of points returned for math (function) waveforms is determined by the NORMal waveform points mode. See the :WAVEform:POINts:MODE command (see [page 1448](#)) for more information.

To get the maximum number of points possible, you can set the waveform points mode to MAXimum and ask for 10,000,000 points, which is the command's request limit (not the maximum memory depth of the oscilloscope).

Only data visible on the display will be returned.

When the :WAVEform:SOURce is the serial decode bus (SBUS1 or SBUS2), this command is ignored, and all available serial decode bus data is returned.

Query Syntax :WAVEform:POINts?

The :WAVEform:POINts query returns the number of waveform points to be transferred when using the :WAVEform:DATA? query. Setting the points mode will affect what data is transferred (see the :WAVEform:POINts:MODE command (see [page 1448](#)) for more information).

In any waveform points mode, the oscilloscope returns as close to the requested number of points as possible, but the actual number of points available depends on oscilloscope settings.

When the :WAVEform:SOURce is the serial decode bus (SBUS1 or SBUS2), this query returns the number of messages that were decoded.

Return Format <#_points><NL>

 <#_points> ::= an integer number of waveform points

See Also • ["Introduction to :WAVEform Commands"](#) on page 1436

- **":ACQuire:POINts[:ANALog]"** on page 290
- **":WAVeform:DATA"** on page 1443
- **":WAVeform:SOURce"** on page 1457
- **":WAVeform:VIEW"** on page 1464
- **":WAVeform:PREamble"** on page 1450
- **":WAVeform:POINts:MODE"** on page 1448

Example Code

```
' WAVE_POINTS - Specifies the number of points to be transferred
' using the ":WAVeform:DATA?" query.
myScope.WriteString ":WAVeform:POINTS 1000"
```

See complete example programs at: **Chapter 46**, "Programming Examples," starting on page 1659

:WAVEform:POINts:MODE

N (see [page 1650](#))

Command Syntax :WAVEform:POINts:MODE <points_mode>

<points_mode> ::= {NORMal | MAXimum | RAW}

The :WAVEform:POINts:MODE command sets the data record to be transferred with the :WAVEform:DATA? query.

For the analog or digital sources, there are two different records that can be transferred:

- The first is the raw acquisition record. The maximum number of points available in this record is returned by the :ACquire:POINts? query. The raw acquisition record can only be retrieved from the analog or digital sources.
- The second is referred to as the *measurement record* and is a 62,500-point (maximum) representation of the raw acquisition record. The measurement record can be retrieved from any source.

If the <points_mode> is NORMal the measurement record is retrieved.

If the <points_mode> is RAW, the raw acquisition record is used. Under some conditions, this data record is unavailable.

If the <points_mode> is MAXimum, whichever record contains the maximum amount of points is used. Usually, this is the raw acquisition record. But, the measurement record may have more data. If data is being retrieved as the oscilloscope is stopped and as the data displayed is changing, the data being retrieved can switch between the measurement and raw acquisition records.

NOTE

The decision about which record has the MAXimum number of points happens before any decimation that may result from a :WAVEform:POINts request. This means it is possible for the measurement record to have more points than the raw acquisition record, but then a :WAVEform:POINts request may cause an integer decimation that results in fewer points being returned by :WAVEform:DATA? than are in the raw acquisition record.

NOTE

If the :WAVEform:SOURce is not an analog or digital source, the only valid parameters for WAVEform:POINts:MODE is NORMal or MAXimum.

Considerations for MAXimum or RAW data retrieval

- The instrument must be stopped (see the :STOP command (see [page 278](#)) or the :DIGitize command (see [page 250](#)) in the root subsystem) in order to return more than the *measurement record*.
- :TIMEbase:MODE must be set to MAIN.
- :ACquire:TYPE must be set to NORMal, AVERage, or HRESolution.

- MAXimum or RAW will allow up to 4,000,000 points to be returned. The number of points returned will vary as the instrument's configuration is changed. Use the :WAVEform:POINts? MAXimum query to determine the maximum number of points that can be retrieved at the current settings.

Query Syntax :WAVEform:POINts:MODE?

The :WAVEform:POINts:MODE? query returns the current points mode. Setting the points mode will affect what data is transferred. See the discussion above.

Return Format <points_mode><NL>

<points_mode> ::= {NORMal | MAXimum | RAW}

- See Also**
- ["Introduction to :WAVEform Commands"](#) on page 1436
 - [":WAVEform:DATA"](#) on page 1443
 - [":ACQuire:POINts\[:ANALog\]"](#) on page 290
 - [":WAVEform:VIEW"](#) on page 1464
 - [":WAVEform:PREamble"](#) on page 1450
 - [":WAVEform:POINts"](#) on page 1446
 - [":TIMebase:MODE"](#) on page 1333
 - [":ACQuire:TYPE"](#) on page 300
 - [":ACQuire:COUNt"](#) on page 286

:WAVEform:PREamble

C (see [page 1650](#))

Query Syntax :WAVEform:PREamble?

The :WAVEform:PREamble query requests the preamble information for the selected waveform source. The preamble data contains information concerning the vertical and horizontal scaling of the data of the corresponding channel.

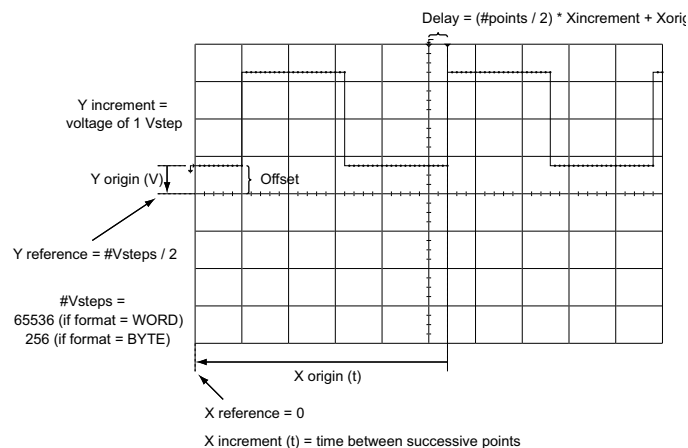
Return Format <preamble_block><NL>

```
<preamble_block> ::= <format 16-bit NR1>,
                    <type 16-bit NR1>,
                    <points 32-bit NR1>,
                    <count 32-bit NR1>,
                    <xincrement 64-bit floating point NR3>,
                    <xorigin 64-bit floating point NR3>,
                    <xreference 32-bit NR1>,
                    <yincrement 32-bit floating point NR3>,
                    <yorigin 32-bit floating point NR3>,
                    <yreference 32-bit NR1>
```

<format> ::= 0 for BYTE format, 1 for WORD format, 4 for ASCII format;
an integer in NR1 format (format set by :WAVEform:FORMat).

<type> ::= 3 for HRESolution type, 2 for AVERage type, 0 for NORMal
type, 1 for PEAK detect type; an integer in NR1 format
(type set by :ACQuire:TYPE).

<count> ::= Average count or 1 if PEAK or NORMal; an integer in NR1
format (count set by :ACQuire:COUNt).



- See Also**
- ["Introduction to :WAVEform Commands"](#) on page 1436
 - [":ACQuire:COUNt"](#) on page 286
 - [":ACQuire:POINts\[:ANALog\]"](#) on page 290

- **":ACquire:TYPE"** on page 300
- **":DIGitize"** on page 250
- **":WAVEform:COUNT"** on page 1442
- **":WAVEform:DATA"** on page 1443
- **":WAVEform:FORMat"** on page 1445
- **":WAVEform:POINts"** on page 1446
- **":WAVEform:TYPE"** on page 1462
- **":WAVEform:XINCrement"** on page 1465
- **":WAVEform:XORigin"** on page 1466
- **":WAVEform:XREFerence"** on page 1467
- **":WAVEform:YINCrement"** on page 1468
- **":WAVEform:YORigin"** on page 1469
- **":WAVEform:YREFerence"** on page 1470

Example Code

```
' GET_PREAMBLE - The preamble block contains all of the current
' WAVEFORM settings. It is returned in the form <preamble_block><NL>
' where <preamble_block> is:
'   FORMAT      : int16 - 0 = BYTE, 1 = WORD, 4 = ASCII.
'   TYPE        : int16 - 0 = NORM, 1 = PEAK, 2 = AVER, 3 = HRES
'   POINTS      : int32 - number of data points transferred.
'   COUNT       : int32 - 1 and is always 1.
'   XINCREMENT  : float64 - time difference between data points.
'   XORIGIN     : float64 - always the first data point in memory.
'   XREFERENCE  : int32 - specifies the data point associated with
'                   x-origin.
'   YINCREMENT  : float32 - voltage diff between data points.
'   YORIGIN     : float32 - value is the voltage at center screen.
'   YREFERENCE  : int32 - specifies the data point where y-origin
'                   occurs.

Dim Preamble()
Dim intFormat As Integer
Dim intType As Integer
Dim lngPoints As Long
Dim lngCount As Long
Dim dblXIncrement As Double
Dim dblXOrigin As Double
Dim lngXReference As Long
Dim sngYIncrement As Single
Dim sngYOrigin As Single
Dim lngYReference As Long
Dim strOutput As String

myScope.WriteString ":WAVEFORM:PREAMBLE?" ' Query for the preamble.
Preamble() = myScope.ReadList ' Read preamble information.
intFormat = Preamble(0)
intType = Preamble(1)
lngPoints = Preamble(2)
lngCount = Preamble(3)
dblXIncrement = Preamble(4)
```

```
dblXOrigin = Preamble(5)
lngXReference = Preamble(6)
sngYIncrement = Preamble(7)
sngYOrigin = Preamble(8)
lngYReference = Preamble(9)
```

See complete example programs at: [Chapter 46](#), “Programming Examples,” starting on page 1659

:WAVEform:SEGMented:ALL

N (see [page 1650](#))

Command Syntax :WAVEform:SEGMented:ALL {{0 | OFF} | {1 | ON}}

The :WAVEform:SEGMented:ALL command enables or disables the "waveform data for all segments" setting:

- ON – The :WAVEform:DATA? query returns data for all segments.
ON is available only when the :WAVEform:SOURce is an analog input channel.
When ON, the :WAVEform:DATA? query returns data for the number of segments multiplied by the number of points in a segment (which is the points value returned by the :WAVEform:POINts? or :WAVEform:PREamble? queries).
- OFF – The :WAVEform:DATA? query returns data for the current segment.

The ability to get waveform data for all segments is faster and more convenient than iterating over multiple segments and retrieving each one separately.

The performance improvement comes primarily when raw acquisition record data is being retrieved (:WAVEform:POINts:MODE RAW) instead of the measurement record data (:WAVEform:POINts:MODE NORMal).

One corner case where you may want to retrieve the measurement record data is when the acquired data for a segment is less than the screen resolution and the measurement record is interpolated.

Most often though, the acquired data for a segment is enough to fill the screen, and less than the maximum measurement record size, so the segmented raw acquisition data and the measurement record data are the same.

If the number of segments is small, the raw acquisition segment size is greater than the measurement record size.

Query Syntax :WAVEform:SEGMented:ALL?

The :WAVEform:SEGMented:ALL? query returns the "waveform data for all segments" setting.

Return Format <setting><NL>

<setting> ::= {0 | 1}

- See Also**
- [":WAVEform:DATA"](#) on page 1443
 - [":WAVEform:SEGMented:XLISt"](#) on page 1456
 - [":WAVEform:POINts:MODE"](#) on page 1448
 - [":WAVEform:POINts"](#) on page 1446
 - [":WAVEform:PREamble"](#) on page 1450

:WAVEform:SEGMented:COUNT

N (see [page 1650](#))

Query Syntax :WAVEform:SEGMented:COUNT?

The :WAVEform:SEGMented:COUNT query returns the number of memory segments in the acquired data. You can use the :WAVEform:SEGMented:COUNT? query while segments are being acquired (although :DIGitize blocks subsequent queries until the full segmented acquisition is complete).

The segmented memory acquisition mode is enabled with the :ACQUIRE:MODE command. The number of segments to acquire is set using the :ACQUIRE:SEGMented:COUNT command, and data is acquired using the :DIGitize, :SINGLE, or :RUN commands.

Return Format <count> ::= an integer from 2 to 1000 in NR1 format (count set by :ACQUIRE:SEGMented:COUNT).

- See Also**
- [":ACQUIRE:MODE"](#) on page 289
 - [":ACQUIRE:SEGMented:COUNT"](#) on page 294
 - [":DIGitize"](#) on page 250
 - [":SINGLE"](#) on page 276
 - [":RUN"](#) on page 274
 - ["Introduction to :WAVEform Commands"](#) on page 1436

Example Code • ["Example Code"](#) on page 295

:WAVeform:SEGMented:TTAG

N (see [page 1650](#))

Query Syntax :WAVeform:SEGMented:TTAG?

The :WAVeform:SEGMented:TTAG? query returns the time tag of the currently selected segmented memory index. The index is selected using the :ACQuire:SEGMented:INDex command.

Return Format <time_tag> ::= in NR3 format

- See Also**
- [":ACQuire:SEGMented:INDex"](#) on page 295
 - ["Introduction to :WAVeform Commands"](#) on page 1436

Example Code

- ["Example Code"](#) on page 295

:WAVEform:SEGMented:XLISt

N (see [page 1650](#))

Query Syntax :WAVEform:SEGMented:XLISt? <xlist_type>

<xlist_type> ::= {RELXorigin | ABSXorigin | TTAG}

The :WAVEform:SEGMented:XLISt? query returns the X (time) information for all segments at once. The <xlist_type> option specifies the type of information that is returned:

- RELXorigin – The relative X-origin for each segment (the value returned by the :WAVEform:XORigin? query) is returned.
- TTAG – The time tag for each segment (the value returned by the :WAVEform:SEGMented:TTAG? query) will be returned.
- ABSXorigin – The sum of the values of the RELXorigin and TTAG types is returned for each segment.

This command is useful when getting the waveform data for all segments at once (see :WAVEform:SEGMented:ALL).

Return Format <return_value><NL>

<return_value> ::= binary block data in IEEE 488.2 # format, contains comma-separated string with X-info for all segments

- See Also**
- [":WAVEform:XORigin"](#) on page 1466
 - [":WAVEform:SEGMented:TTAG"](#) on page 1455
 - [":WAVEform:SEGMented:ALL"](#) on page 1453

:WAVEform:SOURce

C (see [page 1650](#))

Command Syntax :WAVEform:SOURce <source>

```
<source> ::= {CHANnel<n> | FUNCTION<m> | MATH<m> | WMEMory<r>
              | SBUS{1 | 2}} for DSO models

<source> ::= {CHANnel<n> | POD{1 | 2} | BUS{1 | 2} | FUNCTION<m>
              | MATH<m> | WMEMory<r> | SBUS{1 | 2}}
              for MSO models

<n> ::= 1 to (# analog channels) in NR1 format
<m> ::= 1 to (# math functions) in NR1 format
<r> ::= 1 to (# ref waveforms) in NR1 format
```

The :WAVEform:SOURce command selects the analog channel, function, digital pod, digital bus, reference waveform, or serial decode bus to be used as the source for the :WAVEform commands.

Function capabilities include add, subtract, multiply, integrate, differentiate, and FFT (Fast Fourier Transform) operations.

When the :WAVEform:SOURce is the serial decode bus (SBUS1 or SBUS2), ASCII is the only waveform format allowed, and the :WAVEform:DATA? query returns a string with timestamps and associated bus decode information.

With MSO oscilloscope models, you can choose a POD or BUS as the waveform source. There are some differences between POD and BUS when formatting and getting data from the oscilloscope:

- When POD1 or POD2 is selected as the waveform source, you can choose the BYTE, WORD, or ASCII formats (see [":WAVEform:FORMat"](#) on page 1445).

When the WORD format is chosen, every other data byte will be 0. The setting of :WAVEform:BYTeorder controls which byte is 0.

When the ASCII format is chosen, the :WAVEform:DATA? query returns a string with unsigned decimal values separated by commas.

- When BUS1 or BUS2 is selected as the waveform source, you can choose the WORD or ASCII formats (but not BYTE because bus values are always returned as 16-bit values).

When the ASCII format is chosen, the :WAVEform:DATA? query returns a string with hexadecimal bus values, for example: 0x1938,0xff38,...

Query Syntax :WAVEform:SOURce?

The :WAVEform:SOURce? query returns the currently selected source for the WAVEform commands.

NOTE

MATH<m> is an alias for FUNCtion<m>. The :WAVEform:SOURce? query returns FUNC<m> if the source is FUNCtion<m> or MATH<m>.

Return Format

<source><NL>

<source> ::= {CHAN<n> | FUNC<m> | WMEM<r> | SBUS{1 | 2}} for DSO models

<source> ::= {CHAN<n> | POD{1 | 2} | BUS{1 | 2} | FUNC<m>
| WMEM<r> | SBUS{1 | 2}} for MSO models

<n> ::= 1 to (# analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

See Also

- ["Introduction to :WAVEform Commands"](#) on page 1436
- [":DIGitize"](#) on page 250
- [":WAVEform:FORMat"](#) on page 1445
- [":WAVEform:BYTeorder"](#) on page 1441
- [":WAVEform:DATA"](#) on page 1443
- [":WAVEform:PREamble"](#) on page 1450

Example Code

```
' WAVEFORM_DATA - To obtain waveform data, you must specify the
' WAVEFORM parameters for the waveform data prior to sending the
' ":WAVEFORM:DATA?" query. Once these parameters have been sent,
' the waveform data and the preamble can be read.
'
' WAVE_SOURCE - Selects the channel to be used as the source for
' the waveform commands.
myScope.WriteString ":WAVEFORM:SOURCE CHAN1"

' WAVE_POINTS - Specifies the number of points to be transferred
' using the ":WAVEFORM:DATA?" query.
myScope.WriteString ":WAVEFORM:POINTS 1000"

' WAVE_FORMAT - Sets the data transmission mode for the waveform
' data output. This command controls whether data is formatted in
' a word or byte format when sent from the oscilloscope.
Dim lngVSteps As Long
Dim intBytesPerData As Integer

' Data in range 0 to 65535.
myScope.WriteString ":WAVEFORM:FORMAT WORD"
lngVSteps = 65536
intBytesPerData = 2

' Data in range 0 to 255.
myScope.WriteString ":WAVEFORM:FORMAT BYTE"
lngVSteps = 256
intBytesPerData = 1

' GET_PREAMBLE - The preamble block contains all of the current
```

```

' WAVEFORM settings. It is returned in the form <preamble_block><NL>
' where <preamble_block> is:
'   FORMAT      : int16 - 0 = BYTE, 1 = WORD, 4 = ASCII.
'   TYPE        : int16 - 0 = NORMAL, 1 = PEAK DETECT, 2 = AVERAGE
'   POINTS      : int32 - number of data points transferred.
'   COUNT       : int32 - 1 and is always 1.
'   XINCREMENT  : float64 - time difference between data points.
'   XORIGIN     : float64 - always the first data point in memory.
'   XREFERENCE  : int32 - specifies the data point associated with
'                   x-origin.
'   YINCREMENT  : float32 - voltage diff between data points.
'   YORIGIN     : float32 - value is the voltage at center screen.
'   YREFERENCE  : int32 - specifies the data point where y-origin
'                   occurs.
Dim Preamble()
Dim intFormat As Integer
Dim intType As Integer
Dim lngPoints As Long
Dim lngCount As Long
Dim dblXIncrement As Double
Dim dblXOrigin As Double
Dim lngXReference As Long
Dim sngYIncrement As Single
Dim sngYOrigin As Single
Dim lngYReference As Long
Dim strOutput As String

myScope.WriteString ":WAVEFORM:PREAMBLE?" ' Query for the preamble.
Preamble() = myScope.ReadList ' Read preamble information.
intFormat = Preamble(0)
intType = Preamble(1)
lngPoints = Preamble(2)
lngCount = Preamble(3)
dblXIncrement = Preamble(4)
dblXOrigin = Preamble(5)
lngXReference = Preamble(6)
sngYIncrement = Preamble(7)
sngYOrigin = Preamble(8)
lngYReference = Preamble(9)
strOutput = ""
'strOutput = strOutput + "Format = " + CStr(intFormat) + vbCrLf
'strOutput = strOutput + "Type = " + CStr(intType) + vbCrLf
'strOutput = strOutput + "Points = " + CStr(lngPoints) + vbCrLf
'strOutput = strOutput + "Count = " + CStr(lngCount) + vbCrLf
'strOutput = strOutput + "X increment = " + _
'   FormatNumber(dblXIncrement * 1000000) + " us" + vbCrLf
'strOutput = strOutput + "X origin = " + _
'   FormatNumber(dblXOrigin * 1000000) + " us" + vbCrLf
'strOutput = strOutput + "X reference = " + _
'   CStr(lngXReference) + vbCrLf
'strOutput = strOutput + "Y increment = " + _
'   FormatNumber(sngYIncrement * 1000) + " mV" + vbCrLf
'strOutput = strOutput + "Y origin = " + _
'   FormatNumber(sngYOrigin) + " V" + vbCrLf
'strOutput = strOutput + "Y reference = " + _
'   CStr(lngYReference) + vbCrLf
strOutput = strOutput + "Volts/Div = " + _

```

```

        FormatNumber(lngVSteps * sngYIncrement / 8) + _
        " V" + vbCrLf
strOutput = strOutput + "Offset = " + _
        FormatNumber((lngVSteps / 2 - lngYReference) * _
        sngYIncrement + sngYOrigin) + " V" + vbCrLf
strOutput = strOutput + "Sec/Div = " + _
        FormatNumber(lngPoints * dblXIncrement / 10 * _
        1000000) + " us" + vbCrLf
strOutput = strOutput + "Delay = " + _
        FormatNumber(((lngPoints / 2 - lngXReference) * _
        dblXIncrement + dblXOrigin) * 1000000) + " us" + vbCrLf

' QUERY_WAVE_DATA - Outputs waveform data that is stored in a buffer.

' Query the oscilloscope for the waveform data.
myScope.WriteString ":WAV:DATA?"

' READ_WAVE_DATA - The wave data consists of two parts: the header,
' and the actual waveform data followed by a new line (NL) character.
' The query data has the following format:
'
'   <header><waveform_data><NL>
'
' Where:
'   <header> = #800001000 (This is an example header)
' The "#8" may be stripped off of the header and the remaining
' numbers are the size, in bytes, of the waveform data block. The
' size can vary depending on the number of points acquired for the
' waveform. You can then read that number of bytes from the
' oscilloscope and the terminating NL character.
'
Dim lngI As Long
Dim lngDataValue As Long

' Unsigned integer bytes.
varQueryResult = myScope.ReadIEEEBlock(BinaryType_UI1)

For lngI = 0 To UBound(varQueryResult) _
    Step (UBound(varQueryResult) / 20) ' 20 points.
    If intBytesPerData = 2 Then
        lngDataValue = varQueryResult(lngI) * 256 _
            + varQueryResult(lngI + 1) ' 16-bit value.
    Else
        lngDataValue = varQueryResult(lngI) ' 8-bit value.
    End If
    strOutput = strOutput + "Data point " + _
        CStr(lngI / intBytesPerData) + ", " + _
        FormatNumber((lngDataValue - lngYReference) _
            * sngYIncrement + sngYOrigin) + " V, " + _
        FormatNumber(((lngI / intBytesPerData - lngXReference) _
            * sngXIncrement + dblXOrigin) * 1000000) + " us" + vbCrLf
Next lngI
MsgBox "Waveform data:" + vbCrLf + strOutput

```

See complete example programs at: [Chapter 46](#), "Programming Examples," starting on page 1659

:WAVEform:SOURce:SUBSource

C (see [page 1650](#))

Command Syntax :WAVEform:SOURce:SUBSource <subsource>

```
<subsource> ::= { {SUB0 | RX | MOSI | FAST}
                  | {SUB1 | TX | MISO | SLOW} }
```

If the :WAVEform:SOURce is SBUS<n> (serial decode), more than one data set may be available, and this command lets you choose from the available data sets.

When using UART serial decode, this option lets you get "TX" data. (TX is an alias for SUB1.) The default, SUB0, specifies "RX" data. (RX is an alias for SUB0.)

When using SPI serial decode, this option lets you get "MISO" data. (MISO is an alias for SUB1.) The default, SUB0, specifies "MOSI" data. (MOSI is an alias for SUB0.)

When using SENT serial decode, this option lets you get "SLOW" data. (SLOW is an alias for SUB1.) The default, SUB0, specifies "FAST" data. (FAST is an alias for SUB0.)

If the :WAVEform:SOURce is not SBUS, or the :SBUS<n>:MODE is not UART, SPI, or SENT, the only valid subsource is SUB0.

Query Syntax :WAVEform:SOURce:SUBSource?

The :WAVEform:SOURce:SUBSource? query returns the current waveform subsource setting.

Return Format <subsource><NL>

```
<subsource> ::= {SUB0 | SUB1}
```

- See Also**
- ["Introduction to :WAVEform Commands"](#) on page 1436
 - [":WAVEform:SOURce"](#) on page 1457

:WAVeform:TYPE

C (see [page 1650](#))

Query Syntax :WAVeform:TYPE?

The :WAVeform:TYPE? query returns the acquisition mode associated with the currently selected waveform. The acquisition mode is set by the :ACQuire:TYPE command.

Return Format <mode><NL>

<mode> ::= { NORM | PEAK | AVER | HRES }

NOTE

If the :WAVeform:SOURce is POD1, POD2, or SBUS1, SBUS2, the type is always NORM.

-
- See Also**
- ["Introduction to :WAVeform Commands"](#) on page 1436
 - [":ACQuire:TYPE"](#) on page 300
 - [":WAVeform:DATA"](#) on page 1443
 - [":WAVeform:PREamble"](#) on page 1450
 - [":WAVeform:SOURce"](#) on page 1457

:WAVEform:UNSigned

C (see [page 1650](#))

Command Syntax :WAVEform:UNSigned <unsigned>

<unsigned> ::= {{0 | OFF} | {1 | ON}}

The :WAVEform:UNSigned command turns unsigned mode on or off for the currently selected waveform. Use the WAVEform:UNSigned command to control whether data values are sent as unsigned or signed integers. This command can be used to match the instrument's internal data type to the data type used by the programming language. This command has no effect if the data format is ASCII.

If :WAVEform:SOURce is set to POD1, POD2, BUS1, or BUS2, WAVEform:UNSigned must be set to ON.

Query Syntax :WAVEform:UNSigned?

The :WAVEform:UNSigned? query returns the status of unsigned mode for the currently selected waveform.

Return Format <unsigned><NL>

<unsigned> ::= {0 | 1}

- See Also**
- ["Introduction to :WAVEform Commands"](#) on page 1436
 - [":WAVEform:SOURce"](#) on page 1457

:WAVEform:VIEW

C (see [page 1650](#))

Command Syntax :WAVEform:VIEW <view>

<view> ::= {MAIN | ALL}

The :WAVEform:VIEW command sets the view setting associated with the currently selected waveform:

- MAIN – This view specifies the data you see in the oscilloscope's main waveform display area.
- ALL – Available only when Digitizer mode is on (see :ACQUIRE:DIGitizer), this view specifies all the captured data, which may extend beyond the edges of the oscilloscope's main waveform display area depending on the settings for sample rate, memory depth, and horizontal time/div.

Query Syntax :WAVEform:VIEW?

The :WAVEform:VIEW? query returns the view setting associated with the currently selected waveform.

Return Format <view><NL>

<view> ::= {MAIN | ALL}

- See Also**
- ["Introduction to :WAVEform Commands"](#) on page 1436
 - [":WAVEform:POINts"](#) on page 1446
 - [":ACQUIRE:DIGitizer"](#) on page 288

:WAVEform:XINCrement

C (see [page 1650](#))

Query Syntax :WAVEform:XINCrement?

The :WAVEform:XINCrement? query returns the x-increment value for the currently specified source. This value is the time difference between consecutive data points in seconds.

Return Format <value><NL>

<value> ::= x-increment in the current preamble in 64-bit floating point NR3 format

See Also

- ["Introduction to :WAVEform Commands"](#) on page 1436
- [":WAVEform:PREamble"](#) on page 1450

Example Code

- ["Example Code"](#) on page 1451

:WAVeform:XORigin

C (see [page 1650](#))

Query Syntax `:WAVeform:XORigin?`

The `:WAVeform:XORigin?` query returns the x-origin value for the currently specified source. XORigin is the X-axis value of the data point specified by the `:WAVeform:XREference` value. In this product, that is always the X-axis value of the first data point (`XREference = 0`).

Return Format `<value><NL>`

`<value>` ::= x-origin value in the current preamble in 64-bit floating point NR3 format

- See Also**
- ["Introduction to :WAVeform Commands"](#) on page 1436
 - [":WAVeform:PREamble"](#) on page 1450
 - [":WAVeform:XREference"](#) on page 1467

Example Code • ["Example Code"](#) on page 1451

:WAVeform:XREFerence

C (see [page 1650](#))

Query Syntax :WAVeform:XREFerence?

The :WAVeform:XREFerence? query returns the x-reference value for the currently specified source. This value specifies the index of the data point associated with the x-origin data value. In this product, the x-reference point is the first point displayed and XREFerence is always 0.

Return Format <value><NL>

<value> ::= x-reference value = 0 in 32-bit NR1 format

- See Also**
- ["Introduction to :WAVeform Commands"](#) on page 1436
 - [":WAVeform:PREamble"](#) on page 1450
 - [":WAVeform:XORigin"](#) on page 1466

Example Code • ["Example Code"](#) on page 1451

:WAVeform:YINCrement

C (see [page 1650](#))

Query Syntax :WAVeform:YINCrement?

The :WAVeform:YINCrement? query returns the y-increment value in volts for the currently specified source. This value is the voltage difference between consecutive data values. The y-increment for digital waveforms is always "1".

Return Format <value><NL>

<value> ::= y-increment value in the current preamble in 32-bit floating point NR3 format

See Also

- ["Introduction to :WAVeform Commands"](#) on page 1436
- [":WAVeform:PREamble"](#) on page 1450

Example Code

- ["Example Code"](#) on page 1451

:WAVEform:YORigin

C (see [page 1650](#))

Query Syntax :WAVEform:YORigin?

The :WAVEform:YORigin? query returns the y-origin value for the currently specified source. This value is the Y-axis value of the data value specified by the :WAVEform:YREFerence value. For this product, this is the Y-axis value of the center of the screen.

Return Format <value><NL>

<value> ::= y-origin in the current preamble in 32-bit floating point NR3 format

- See Also**
- ["Introduction to :WAVEform Commands"](#) on page 1436
 - [":WAVEform:PREamble"](#) on page 1450
 - [":WAVEform:YREFerence"](#) on page 1470

Example Code • ["Example Code"](#) on page 1451

:WAVeform:YREFerence

C (see [page 1650](#))

Query Syntax :WAVeform:YREFerence?

The :WAVeform:YREFerence? query returns the y-reference value for the currently specified source. This value specifies the data point value where the y-origin occurs. In this product, this is the data point value of the center of the screen. It is undefined if the format is ASCII.

Return Format <value><NL>

<value> ::= y-reference value in the current preamble in 32-bit
NR1 format

- See Also**
- ["Introduction to :WAVeform Commands"](#) on page 1436
 - [":WAVeform:PREamble"](#) on page 1450
 - [":WAVeform:YORigin"](#) on page 1469

Example Code • ["Example Code"](#) on page 1451

39 :WGEN<w> Commands

When the built-in waveform generator is licensed (WAVEGEN license), you can use it to output sine, square, ramp, pulse, DC, noise, sine cardinal, exponential rise, exponential fall, cardiac, and gaussian pulse waveforms. The :WGEN<w> commands are used to select the waveform function and parameters. See ["Introduction to :WGEN<w> Commands"](#) on page 1475.

Table 169 :WGEN<w> Commands Summary

Command	Query	Options and Query Returns
:WGEN<w>:ARbitrary:BYTeorder <order> (see page 1476)	:WGEN<w>:ARbitrary:BYTeorder? (see page 1476)	<order> ::= {MSBFirst LSBFirst} <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:ARbitrary:DATA {<binary> <value>, <value> ...} (see page 1477)	n/a	<binary> ::= floating point values between -1.0 to +1.0 in IEEE 488.2 binary block format <value> ::= floating point values between -1.0 to +1.0 in comma-separated format <w> ::= 1 to (# WaveGen outputs) in NR1 format
n/a	:WGEN<w>:ARbitrary:DATA:ATTRibute:POINTs? (see page 1480)	<points> ::= number of points in NR1 format <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:ARbitrary:DATA:CLear (see page 1481)	n/a	<w> ::= 1 to (# WaveGen outputs) in NR1 format

Table 169 :WGEN<w> Commands Summary (continued)

Command	Query	Options and Query Returns
:WGEN<w>:ARBitrary:DA TA:DAC {<binary> <value>, <value> ...} (see page 1482)	n/a	<binary> ::= decimal 16-bit integer values between -512 to +511 in IEEE 488.2 binary block format <value> ::= decimal integer values between -512 to +511 in comma-separated NR1 format <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:ARBitrary:IN Terpolate {{0 OFF} {1 ON}} (see page 1483)	:WGEN<w>:ARBitrary:IN Terpolate? (see page 1483)	{0 1} <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:ARBitrary:ST ORe <source> (see page 1484)	n/a	<source> ::= {CHANnel<n> WMEMemory<r> FUNctIon<m> MATH<m>} <n> ::= 1 to (# analog channels) in NR1 format <r> ::= 1 to (# ref waveforms) in NR1 format <m> ::= 1 to (# math functions) in NR1 format <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:FREQuency <frequency> (see page 1485)	:WGEN<w>:FREQuency? (see page 1485)	<frequency> ::= frequency in Hz in NR3 format <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:FUNctIon <signal> (see page 1486)	:WGEN<w>:FUNctIon? (see page 1489)	<signal> ::= {SINusoid SQUare RAMP PULSe NOISe DC SINC EXPRise EXPFall CARDiac GAUSSian ARBitrary} <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:FUNctIon:PUL Se:WIDTh <width> (see page 1490)	:WGEN<w>:FUNctIon:PUL Se:WIDTh? (see page 1490)	<width> ::= pulse width in seconds in NR3 format <w> ::= 1 to (# WaveGen outputs) in NR1 format

Table 169 :WGEN<w> Commands Summary (continued)

Command	Query	Options and Query Returns
:WGEN<w>:FUNCTION:RAM P:SYMMetry <percent> (see page 1491)	:WGEN<w>:FUNCTION:RAM P:SYMMetry? (see page 1491)	<percent> ::= symmetry percentage from 0% to 100% in NR1 format <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:FUNCTION:SQU are:DCYcle <percent> (see page 1492)	:WGEN<w>:FUNCTION:SQU are:DCYcle? (see page 1492)	<percent> ::= duty cycle percentage from 20% to 80% in NR1 format <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:MODulation:A M:DEPTh <percent> (see page 1493)	:WGEN<w>:MODulation:A M:DEPTh? (see page 1493)	<percent> ::= AM depth percentage from 0% to 100% in NR1 format <w> ::= 1 in NR1 format
:WGEN<w>:MODulation:A M:FREQuency <frequency> (see page 1494)	:WGEN<w>:MODulation:A M:FREQuency? (see page 1494)	<frequency> ::= modulating waveform frequency in Hz in NR3 format <w> ::= 1 in NR1 format
:WGEN<w>:MODulation:F M:DEVIation <frequency> (see page 1495)	:WGEN<w>:MODulation:F M:DEVIation? (see page 1495)	<frequency> ::= frequency deviation in Hz in NR3 format <w> ::= 1 in NR1 format
:WGEN<w>:MODulation:F M:FREQuency <frequency> (see page 1496)	:WGEN<w>:MODulation:F M:FREQuency? (see page 1496)	<frequency> ::= modulating waveform frequency in Hz in NR3 format <w> ::= 1 in NR1 format
:WGEN<w>:MODulation:F SKey:FREQuency <percent> (see page 1497)	:WGEN<w>:MODulation:F SKey:FREQuency? (see page 1497)	<frequency> ::= hop frequency in Hz in NR3 format <w> ::= 1 in NR1 format
:WGEN<w>:MODulation:F SKey:RATE <rate> (see page 1498)	:WGEN<w>:MODulation:F SKey:RATE? (see page 1498)	<rate> ::= FSK modulation rate in Hz in NR3 format <w> ::= 1 in NR1 format
:WGEN<w>:MODulation:F UNCTION <shape> (see page 1499)	:WGEN<w>:MODulation:F UNCTION? (see page 1499)	<shape> ::= {SINusoid SQUare RAMP} <w> ::= 1 in NR1 format
:WGEN<w>:MODulation:F UNCTION:RAMP:SYMMetry <percent> (see page 1500)	:WGEN<w>:MODulation:F UNCTION:RAMP:SYMMetry ? (see page 1500)	<percent> ::= symmetry percentage from 0% to 100% in NR1 format <w> ::= 1 in NR1 format

Table 169 :WGEN<w> Commands Summary (continued)

Command	Query	Options and Query Returns
:WGEN<w>:MODulation:NOISe <percent> (see page 1501)	:WGEN<w>:MODulation:NOISe? (see page 1501)	<percent> ::= 0 to 100 <w> ::= 1 in NR1 format
:WGEN<w>:MODulation:STATe {{0 OFF} {1 ON}} (see page 1502)	:WGEN<w>:MODulation:STATe? (see page 1502)	{0 1} <w> ::= 1 in NR1 format
:WGEN<w>:MODulation:TYPE <type> (see page 1503)	:WGEN<w>:MODulation:TYPE? (see page 1503)	<type> ::= {AM FM FSK} <w> ::= 1 in NR1 format
:WGEN<w>:OUTPut {{0 OFF} {1 ON}} (see page 1505)	:WGEN<w>:OUTPut? (see page 1505)	{0 1} <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:OUTPut:LOAD <impedance> (see page 1506)	:WGEN<w>:OUTPut:LOAD? (see page 1506)	<impedance> ::= {ONEMeg FIFTy} <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:OUTPut:MODE <mode> (see page 1507)	:WGEN<w>:OUTPut:MODE? (see page 1507)	<mode> ::= {NORMAl SINGLE}
:WGEN<w>:OUTPut:POLarity <polarity> (see page 1508)	:WGEN<w>:OUTPut:POLarity? (see page 1508)	<polarity> ::= {NORMAl INVerted} <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:OUTPut:SINGLe (see page 1509)	n/a	n/a
:WGEN<w>:PERiod <period> (see page 1510)	:WGEN<w>:PERiod? (see page 1510)	<period> ::= period in seconds in NR3 format <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:RST (see page 1511)	n/a	<w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:TRACk {{0 OFF} {1 ON}} (see page 1512)	:WGEN<w>:TRACk? (see page 1512)	{0 1} <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:TRACk:AMPLitUde {{0 OFF} {1 ON}} (see page 1513)	:WGEN<w>:TRACk:AMPLitUde? (see page 1513)	{0 1} <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:TRACk:CSIGnaL (see page 1514)	n/a	<w> ::= 1 to (# WaveGen outputs) in NR1 format

Table 169 :WGEN<w> Commands Summary (continued)

Command	Query	Options and Query Returns
:WGEN<w>:TRACk:FREQue ncy {{0 OFF} {1 ON}} (see page 1515)	:WGEN<w>:TRACk:FREQue ncy? (see page 1515)	{0 1} <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:TRACk:PHASe <phase> (see page 1516)	:WGEN<w>:TRACk:PHASe? (see page 1516)	<w> ::= 1 to (# WaveGen outputs) in NR1 format <phase> ::= -360 to +360 in NR1 format
:WGEN<w>:VOLTage <amplitude> (see page 1517)	:WGEN<w>:VOLTage? (see page 1517)	<amplitude> ::= amplitude in volts in NR3 format <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:VOLTage:HIGH <high> (see page 1518)	:WGEN<w>:VOLTage:HIGH ? (see page 1518)	<high> ::= high-level voltage in volts, in NR3 format <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:VOLTage:LOW <low> (see page 1519)	:WGEN<w>:VOLTage:LOW? (see page 1519)	<low> ::= low-level voltage in volts, in NR3 format <w> ::= 1 to (# WaveGen outputs) in NR1 format
:WGEN<w>:VOLTage:OFFS et <offset> (see page 1520)	:WGEN<w>:VOLTage:OFFS et? (see page 1520)	<offset> ::= offset in volts in NR3 format <w> ::= 1 to (# WaveGen outputs) in NR1 format

Introduction to :WGEN<w> Commands

The :WGEN<w> subsystem provides commands to select the waveform generator function and parameters.

In the :WGEN<w> commands, the <w> can be 1 or 2, and :WGEN is equivalent to :WGEN1

Reporting the Setup

Use :WGEN<w>? to query setup information for the WGEN<w> subsystem.

Return Format

The following is a sample response from the :WGEN? query. In this case, the query was issued following the *RST command.

```
:WGEN1:FUNC SIN;OUTP 0;FREQ +1.0000E+03;VOLT +500.0E-03;VOLT:OFFS  
+0.0E+00;;WGEN1:OUTP:LOAD ONEM
```

:WGEN<w>:ARbitrary:BYTeorder

N (see [page 1650](#))

Command Syntax :WGEN<w>:ARbitrary:BYTeorder <order>

<w> ::= 1 to (# WaveGen outputs) in NR1 format

<order> ::= {MSBFirst | LSBFirst}

The :WGEN<w>:ARbitrary:BYTeorder command selects the byte order for binary transfers.

Query Syntax :WGEN<w>:ARbitrary:BYTeorder?

The :WGEN<w>:ARbitrary:BYTeorder query returns the current byte order selection.

Return Format <order><NL>

<order> ::= {MSBFirst | LSBFirst}

- See Also**
- [":WGEN<w>:ARbitrary:DATA"](#) on page 1477
 - [":WGEN<w>:ARbitrary:DATA:DAC"](#) on page 1482

:WGEN<w>:ARbitrary:DATA

N (see [page 1650](#))

Command Syntax :WGEN<w>:ARbitrary:DATA {<binary> | <value>, <value> ...}

<w> ::= 1 to (# WaveGen outputs) in NR1 format

<binary> ::= floating point values between -1.0 to +1.0
in IEEE 488.2 binary block format

<value> ::= floating point values between -1.0 to +1.0
in comma-separated format

The :WGEN<w>:ARbitrary:DATA command downloads an arbitrary waveform in floating-point values format.

- See Also**
- [":WGEN<w>:ARbitrary:DATA:DAC"](#) on page 1482
 - [":SAVE:ARbitrary\[:START\]"](#) on page 871
 - [":RECall:ARbitrary\[:START\]"](#) on page 858

Example Code ' Waveform generator arbitrary data commands example.

```
' -----
Option Explicit

Public myMgr As VisaComLib.ResourceManager
Public myScope As VisaComLib.FormattedIO488
Public varQueryResult As Variant
Public strQueryResult As String

Private Declare Sub Sleep Lib "kernel32" (ByVal dwMilliseconds As Long)
Private Declare Sub CopyMemory Lib "kernel32" Alias "RtlMoveMemory" ( _
    dest As Any, _
    source As Any, _
    ByVal bytes As Long)

Sub Main()

    On Error GoTo VisaComError

    ' Create the VISA COM I/O resource.
    Set myMgr = New VisaComLib.ResourceManager
    Set myScope = New VisaComLib.FormattedIO488
    Set myScope.IO = _
        myMgr.Open("TCPIP0::a-mx4154a-60014.cos.is.keysight.com::inst0::
INSTR")
    myScope.IO.Clear ' Clear the interface.

    ' Turn on arbitrary waveform generator function.
    myScope.WriteString ":WGEN1:OUTPut ON"
    myScope.WriteString ":WGEN1:FUNCTION ARbitrary"
    myScope.WriteString ":WGEN1:FUNCTION?"
    strQueryResult = myScope.ReadString
    Debug.Print "WaveGen1 function: " + strQueryResult
```

```

DefaultArbitraryWaveform

' Download comma-separated floating-point values.
myScope.WriteString ":WGEN1:ARbitrary:DATA 0.0, 0.5, 1.0, 0.5, 0.0, -0
.5, -1.0, -0.5"
Debug.Print "WaveGen1 CSV floating-point values downloaded."
Sleep 5000

DefaultArbitraryWaveform

' Download comma-separated 16-bit integer (DAC) values.
myScope.WriteString ":WGEN1:ARbitrary:DATA:DAC 0, 255, 511, 255, 0, -2
56, -512, -256"
Debug.Print "WaveGen1 CSV 16-bit integer (DAC) values downloaded."
Sleep 5000

' Set the byte order for binary data.
myScope.WriteString ":WGEN1:ARbitrary:BYTeorder LSBFirst"
myScope.WriteString ":WGEN1:ARbitrary:BYTeorder?"
strQueryResult = myScope.ReadString
Debug.Print "WaveGen1 byte order for binary data: " + strQueryResult

DefaultArbitraryWaveform

' Download binary floating-point values.
Dim mySingleArray(8) As Single
mySingleArray(0) = 0!
mySingleArray(1) = 0.5!
mySingleArray(2) = 1!
mySingleArray(3) = 0.5!
mySingleArray(4) = 0!
mySingleArray(5) = -0.5!
mySingleArray(6) = -1!
mySingleArray(7) = -0.5!

Dim myByteArray(32) As Byte
CopyMemory myByteArray(0), mySingleArray(0), 32 * LenB(myByteArray(0))

myScope.WriteIEEEBlock ":WGEN1:ARbitrary:DATA", myByteArray, True
Debug.Print "WaveGen1 binary floating-point values downloaded."
Sleep 5000

DefaultArbitraryWaveform

' Download binary 16-bit integer (DAC) values.
Dim myIntegerArray(8) As Integer
myIntegerArray(0) = 0
myIntegerArray(1) = 255
myIntegerArray(2) = 511
myIntegerArray(3) = 255
myIntegerArray(4) = 0
myIntegerArray(5) = -256
myIntegerArray(6) = -512
myIntegerArray(7) = -256

Dim myByteArray2(16) As Byte

```

```

CopyMemory myByteArray2(0), myIntegerArray(0), 16 * LenB(myByteArray2(
0))

myScope.WriteIEEEBlock ":WGEN1:ARbitrary:DATA:DAC", myByteArray2, True
Debug.Print "WaveGen1 binary 16-bit integer (DAC) values downloaded."
Sleep 5000

Exit Sub

VisaComError:
MsgBox "VISA COM Error:" + vbCrLf + Err.Description

End Sub

'
' Initialize WaveGen1 to a known state.
' -----
Private Sub DefaultArbitraryWaveform()

On Error GoTo VisaComError

' Load default arbitrary waveform.
myScope.WriteString ":WGEN1:ARbitrary:DATA:CLear"
Debug.Print "WaveGen1 default arbitrary waveform loaded."
Sleep 5000

Exit Sub

VisaComError:
MsgBox "VISA COM Error:" + vbCrLf + Err.Description
End

End Sub

```

:WGEN<w>:ARbitrary:DATA:ATTRibute:POINts

N (see [page 1650](#))

Query Syntax :WGEN<w>:ARbitrary:DATA:ATTRibute:POINts?

<w> ::= 1 to (# WaveGen outputs) in NR1 format

The :WGEN<w>:ARbitrary:DATA:ATTRibute:POINts query returns the number of points used by the current arbitrary waveform.

Return Format <points> ::= number of points in NR1 format

- See Also**
- [":WGEN<w>:ARbitrary:DATA"](#) on page 1477
 - [":WGEN<w>:ARbitrary:DATA:DAC"](#) on page 1482
 - [":SAVE:ARbitrary\[:START\]"](#) on page 871
 - [":RECall:ARbitrary\[:START\]"](#) on page 858

:WGEN<w>:ARbitrary:DATA:CLEar

N (see [page 1650](#))

Command Syntax :WGEN<w>:ARbitrary:DATA:CLEar

<w> ::= 1 to (# WaveGen outputs) in NR1 format

The :WGEN<w>:ARbitrary:DATA:CLEar command clears the arbitrary waveform memory and loads it with the default waveform.

- See Also**
- [":WGEN<w>:ARbitrary:DATA"](#) on page 1477
 - [":WGEN<w>:ARbitrary:DATA:DAC"](#) on page 1482
 - [":SAVE:ARbitrary\[:START\]"](#) on page 871
 - [":RECall:ARbitrary\[:START\]"](#) on page 858

- Example Code**
- ["Example Code"](#) on page 1477

:WGEN<w>:ARbitrary:DATA:DAC

N (see [page 1650](#))

Command Syntax :WGEN<w>:ARbitrary:DATA:DAC {<binary> | <value>, <value> ...}

<w> ::= 1 to (# WaveGen outputs) in NR1 format

<binary> ::= decimal 16-bit integer values between -512 to +511
in IEEE 488.2 binary block format

<value> ::= decimal integer values between -512 to +511
in comma-separated NR1 format

The :WGEN<w>:ARbitrary:DATA:DAC command downloads an arbitrary waveform using 16-bit integer (DAC) values.

- See Also**
- [":WGEN<w>:ARbitrary:DATA"](#) on page 1477
 - [":SAVE:ARbitrary\[:START\]"](#) on page 871
 - [":RECall:ARbitrary\[:START\]"](#) on page 858

- Example Code**
- ["Example Code"](#) on page 1477

:WGEN<w>:ARbitrary:INTerpolate

N (see [page 1650](#))

Command Syntax :WGEN<w>:ARbitrary:INTerpolate {{0 | OFF} | {1 | ON}}

<w> ::= 1 to (# WaveGen outputs) in NR1 format

The :WGEN<w>:ARbitrary:INTerpolate command enables or disables the Interpolation control.

Interpolation specifies how lines are drawn between arbitrary waveform points:

- When ON, lines are drawn between points in the arbitrary waveform. Voltage levels change linearly between one point and the next.
- When OFF, all line segments in the arbitrary waveform are horizontal. The voltage level of one point remains until the next point.

Query Syntax :WGEN<w>:ARbitrary:INTerpolate?

The :WGEN<w>:ARbitrary:INTerpolate query returns the current interpolation setting.

Return Format {0 | 1}

- See Also**
- [":WGEN<w>:ARbitrary:DATA"](#) on page 1477
 - [":WGEN<w>:ARbitrary:DATA:DAC"](#) on page 1482
 - [":SAVE:ARbitrary\[:START\]"](#) on page 871
 - [":RECall:ARbitrary\[:START\]"](#) on page 858

:WGEN<w>:ARbitrary:STORe

N (see [page 1650](#))

Command Syntax :WGEN<w>:ARbitrary:STORe <source>

<w> ::= 1 to (# WaveGen outputs) in NR1 format

<source> ::= {CHANnel<n> | WMEMory<r> | FUNction<m> | MATH<m>}

<n> ::= 1 to (# analog channels) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

The :WGEN<w>:ARbitrary:STORe command stores the source's waveform into the arbitrary waveform memory.

- See Also**
- [":WGEN<w>:ARbitrary:DATA"](#) on page 1477
 - [":WGEN<w>:ARbitrary:DATA:DAC"](#) on page 1482
 - [":SAVE:ARbitrary\[:START\]"](#) on page 871
 - [":RECall:ARbitrary\[:START\]"](#) on page 858

:WGEN<w>:FREQuency

N (see [page 1650](#))

Command Syntax :WGEN<w>:FREQuency <frequency>

<w> ::= 1 to (# WaveGen outputs) in NR1 format

<frequency> ::= frequency in Hz in NR3 format

For all waveforms except Noise and DC, the :WGEN<w>:FREQuency command specifies the frequency of the waveform.

You can also specify the frequency indirectly using the :WGEN<w>:PERiod command.

Query Syntax :WGEN<w>:FREQuency?

The :WGEN<w>:FREQuency? query returns the currently set waveform generator frequency.

Return Format <frequency><NL>

<frequency> ::= frequency in Hz in NR3 format

- See Also**
- ["Introduction to :WGEN<w> Commands"](#) on page 1475
 - [":WGEN<w>:FUNction"](#) on page 1486
 - [":WGEN<w>:PERiod"](#) on page 1510

:WGEN<w>:FUNCTION

N (see [page 1650](#))

Command Syntax :WGEN<w>:FUNCTION <signal>

<w> ::= 1 to (# WaveGen outputs) in NR1 format

<signal> ::= {SINusoid | SQUare | RAMP | PULSe | DC | NOISe | SINC
| EXPRise | EXPFall | CARDiac | GAUSSian | ARBitary}

The :WGEN<w>:FUNCTION command selects the type of waveform:

Waveform Type	Characteristics	Frequency Range	Max. Amplitude ² (High-Z) ¹	Offset ² (High-Z) ¹
SINusoid	Use these commands to set the sine signal parameters: ▪ ":WGEN<w>:FREQuency" on page 1485 ▪ ":WGEN<w>:PERiod" on page 1510 ▪ ":WGEN<w>:VOLTage" on page 1517 ▪ ":WGEN<w>:VOLTage:OFFSet" on page 1520 ▪ ":WGEN<w>:VOLTage:HIGH" on page 1518 ▪ ":WGEN<w>:VOLTage:LOW" on page 1519	100 mHz to 20 MHz	20 mVpp to 10 Vpp	±4.00 V
SQUare	Use these commands to set the square wave signal parameters: ▪ ":WGEN<w>:FREQuency" on page 1485 ▪ ":WGEN<w>:PERiod" on page 1510 ▪ ":WGEN<w>:VOLTage" on page 1517 ▪ ":WGEN<w>:VOLTage:OFFSet" on page 1520 ▪ ":WGEN<w>:VOLTage:HIGH" on page 1518 ▪ ":WGEN<w>:VOLTage:LOW" on page 1519 ▪ ":WGEN<w>:FUNCTION:SQUare:DCYCLE" on page 1492 The duty cycle can be adjusted from 20% to 80%.	100 mHz to 10 MHz	20 mVpp to 10 Vpp	±5.00 V

Waveform Type	Characteristics	Frequency Range	Max. Amplitude ² (High-Z) ¹	Offset ² (High-Z) ¹
RAMP	Use these commands to set the ramp signal parameters: ▪ ":WGEN<w>:FREQuency" on page 1485 ▪ ":WGEN<w>:PERiod" on page 1510 ▪ ":WGEN<w>:VOLTage" on page 1517 ▪ ":WGEN<w>:VOLTage:OFFSet" on page 1520 ▪ ":WGEN<w>:VOLTage:HIGH" on page 1518 ▪ ":WGEN<w>:VOLTage:LOW" on page 1519 ▪ ":WGEN<w>:FUNctio:n:RAMP:SYMMetry" on page 1491 Symmetry represents the amount of time per cycle that the ramp waveform is rising and can be adjusted from 0% to 100%.	100 mHz to 200 kHz	20 mVpp to 10 Vpp	±5.00 V
PULSe	Use these commands to set the pulse signal parameters: ▪ ":WGEN<w>:FREQuency" on page 1485 ▪ ":WGEN<w>:PERiod" on page 1510 ▪ ":WGEN<w>:VOLTage" on page 1517 ▪ ":WGEN<w>:VOLTage:OFFSet" on page 1520 ▪ ":WGEN<w>:VOLTage:HIGH" on page 1518 ▪ ":WGEN<w>:VOLTage:LOW" on page 1519 ▪ ":WGEN<w>:FUNctio:n:PULSe:WIDTh" on page 1490 The pulse width can be adjusted from 20 ns to the period minus 20 ns.	100 mHz to 10 MHz.	20 mVpp to 10 Vpp	±5.00 V
DC	Use this command to set the DC level: ▪ ":WGEN<w>:VOLTage:OFFSet" on page 1520	n/a	n/a	±10.00 V
NOISe	Use these commands to set the noise signal parameters: ▪ ":WGEN<w>:VOLTage" on page 1517 ▪ ":WGEN<w>:VOLTage:OFFSet" on page 1520 ▪ ":WGEN<w>:VOLTage:HIGH" on page 1518 ▪ ":WGEN<w>:VOLTage:LOW" on page 1519	n/a	20 mVpp to 10 Vpp	±5.00 V

Waveform Type	Characteristics	Frequency Range	Max. Amplitude ² (High-Z) ¹	Offset ² (High-Z) ¹
SINC	Use these commands to set the sine cardinal signal parameters: ▪ <code>":WGEN<w>:FREQuency"</code> on page 1485 ▪ <code>":WGEN<w>:PERiod"</code> on page 1510 ▪ <code>":WGEN<w>:VOLTage"</code> on page 1517 ▪ <code>":WGEN<w>:VOLTage:OFFSet"</code> on page 1520	100 mHz to 1 MHz	20 mVpp to 9 Vpp	±2.50 V
EXPRise	Use these commands to set the exponential rise signal parameters: ▪ <code>":WGEN<w>:FREQuency"</code> on page 1485 ▪ <code>":WGEN<w>:PERiod"</code> on page 1510 ▪ <code>":WGEN<w>:VOLTage"</code> on page 1517 ▪ <code>":WGEN<w>:VOLTage:OFFSet"</code> on page 1520 ▪ <code>":WGEN<w>:VOLTage:HIGH"</code> on page 1518 ▪ <code>":WGEN<w>:VOLTage:LOW"</code> on page 1519	100 mHz to 5 MHz	20 mVpp to 10 Vpp	±5.00 V
EXPFall	Use these commands to set the exponential fall signal parameters: ▪ <code>":WGEN<w>:FREQuency"</code> on page 1485 ▪ <code>":WGEN<w>:PERiod"</code> on page 1510 ▪ <code>":WGEN<w>:VOLTage"</code> on page 1517 ▪ <code>":WGEN<w>:VOLTage:OFFSet"</code> on page 1520 ▪ <code>":WGEN<w>:VOLTage:HIGH"</code> on page 1518 ▪ <code>":WGEN<w>:VOLTage:LOW"</code> on page 1519	100 mHz to 5 MHz	20 mVpp to 10 Vpp	±5.00 V
CARDiac	Use these commands to set the cardiac signal parameters: ▪ <code>":WGEN<w>:FREQuency"</code> on page 1485 ▪ <code>":WGEN<w>:PERiod"</code> on page 1510 ▪ <code>":WGEN<w>:VOLTage"</code> on page 1517 ▪ <code>":WGEN<w>:VOLTage:OFFSet"</code> on page 1520	100 mHz to 200 kHz	20 mVpp to 9 Vpp	±2.50 V

Waveform Type	Characteristics	Frequency Range	Max. Amplitude ² (High-Z) ¹	Offset ² (High-Z) ¹
GAUSSian	Use these commands to set the gaussian pulse signal parameters: ▪ ":WGEN<w>:FREQuency" on page 1485 ▪ ":WGEN<w>:PERiod" on page 1510 ▪ ":WGEN<w>:VOLTage" on page 1517 ▪ ":WGEN<w>:VOLTage:OFFSet" on page 1520	100 mHz to 5 MHz	20 mVpp to 7.5 Vpp	±2.50 V
ARBitrary	Use these commands to set the arbitrary signal parameters: ▪ ":WGEN<w>:FREQuency" on page 1485 ▪ ":WGEN<w>:PERiod" on page 1510 ▪ ":WGEN<w>:VOLTage" on page 1517 ▪ ":WGEN<w>:VOLTage:OFFSet" on page 1520 ▪ ":WGEN<w>:VOLTage:HIGH" on page 1518 ▪ ":WGEN<w>:VOLTage:LOW" on page 1519	100 mHz to 12 MHz	20 mVpp to 10 Vpp	±5.00 V
¹ When the output load is 50 Ω , these values are halved. ² The minimum amplitude is limited to 40 mVpp if the offset is greater than 500 mV or less than -500 mV. Likewise, the offset is limited to +/-500 mV if the amplitude is less than 40 mVpp.				

Query Syntax :WGEN<w>:FUNctIon?

The :WGEN<w>:FUNctIon? query returns the currently selected signal type.

Return Format <signal><NL>

<signal> ::= {SIN | SQU | RAMP | PULS | DC | NOIS | SINC | EXPR | EXPF
| CARD | GAUS | ARB}

See Also

- **"Introduction to :WGEN<w> Commands"** on page 1475
- **":WGEN<w>:MODulation:NOISe"** on page 1501

:WGEN<w>:FUNCTION:PULSE:WIDTH

N (see [page 1650](#))

Command Syntax :WGEN<w>:FUNCTION:PULSE:WIDTH <width>

<w> ::= 1 to (# WaveGen outputs) in NR1 format

<width> ::= pulse width in seconds in NR3 format

For Pulse waveforms, the :WGEN<w>:FUNCTION:PULSE:WIDTH command specifies the width of the pulse.

The pulse width can be adjusted from 20 ns to the period minus 20 ns.

Query Syntax :WGEN<w>:FUNCTION:PULSE:WIDTH?

The :WGEN<w>:FUNCTION:PULSE:WIDTH? query returns the currently set pulse width.

Return Format <width><NL>

<width> ::= pulse width in seconds in NR3 format

- See Also**
- ["Introduction to :WGEN<w> Commands"](#) on page 1475
 - [":WGEN<w>:FUNCTION"](#) on page 1486

:WGEN<w>:FUNCTION:RAMP:SYMMetry

N (see [page 1650](#))

Command Syntax :WGEN<w>:FUNCTION:RAMP:SYMMetry <percent>

<w> ::= 1 to (# WaveGen outputs) in NR1 format

<percent> ::= symmetry percentage from 0% to 100% in NR1 format

For Ramp waveforms, the :WGEN<w>:FUNCTION:RAMP:SYMMetry command specifies the symmetry of the waveform.

Symmetry represents the amount of time per cycle that the ramp waveform is rising.

Query Syntax :WGEN<w>:FUNCTION:RAMP:SYMMetry?

The :WGEN<w>:FUNCTION:RAMP:SYMMetry? query returns the currently set ramp symmetry.

Return Format <percent><NL>

<percent> ::= symmetry percentage from 0% to 100% in NR1 format

- See Also**
- ["Introduction to :WGEN<w> Commands"](#) on page 1475
 - [":WGEN<w>:FUNCTION"](#) on page 1486

:WGEN<w>:FUNCTION:SQUare:DCYCl

N (see [page 1650](#))

Command Syntax :WGEN<w>:FUNCTION:SQUare:DCYCl <percent>

<w> ::= 1 to (# WaveGen outputs) in NR1 format

<percent> ::= duty cycle percentage from 20% to 80% in NR1 format

For Square waveforms, the :WGEN<w>:FUNCTION:SQUare:DCYCl command specifies the square wave duty cycle.

Duty cycle is the percentage of the period that the waveform is high.

Query Syntax :WGEN<w>:FUNCTION:SQUare:DCYCl?

The :WGEN<w>:FUNCTION:SQUare:DCYCl? query returns the currently set square wave duty cycle.

Return Format <percent><NL>

<percent> ::= duty cycle percentage from 20% to 80% in NR1 format

- See Also**
- ["Introduction to :WGEN<w> Commands"](#) on page 1475
 - [":WGEN<w>:FUNCTION"](#) on page 1486

:WGEN<w>:MODulation:AM:DEPTH

N (see [page 1650](#))

Command Syntax :WGEN<w>:MODulation:AM:DEPTH <percent>

<w> ::= 1 in NR1 format

<percent> ::= AM depth percentage from 0% to 100% in NR1 format

The :WGEN<w>:MODulation:AM:DEPTH command specifies the amount of amplitude modulation.

AM Depth refers to the portion of the amplitude range that will be used by the modulation. For example, a depth setting of 80% causes the output amplitude to vary from 10% to 90% ($90\% - 10\% = 80\%$) of the original amplitude as the modulating signal goes from its minimum to maximum amplitude.

Query Syntax :WGEN<w>:MODulation:AM:DEPTH?

The :WGEN<w>:MODulation:AM:DEPTH? query returns the AM depth percentage setting.

Return Format <percent><NL>

<percent> ::= AM depth percentage from 0% to 100% in NR1 format

- See Also**
- [":WGEN<w>:MODulation:AM:FREQuency"](#) on page 1494
 - [":WGEN<w>:MODulation:FM:DEVIation"](#) on page 1495
 - [":WGEN<w>:MODulation:FM:FREQuency"](#) on page 1496
 - [":WGEN<w>:MODulation:FSKey:FREQuency"](#) on page 1497
 - [":WGEN<w>:MODulation:FSKey:RATE"](#) on page 1498
 - [":WGEN<w>:MODulation:FUNCTion"](#) on page 1499
 - [":WGEN<w>:MODulation:FUNCTion:RAMP:SYMMetry"](#) on page 1500
 - [":WGEN<w>:MODulation:STATe"](#) on page 1502
 - [":WGEN<w>:MODulation:TYPE"](#) on page 1503

:WGEN<w>:MODulation:AM:FREQuency

N (see [page 1650](#))

Command Syntax :WGEN<w>:MODulation:AM:FREQuency <frequency>

<w> ::= 1 in NR1 format

<frequency> ::= modulating waveform frequency in Hz in NR3 format

The :WGEN<w>:MODulation:AM:FREQuency command specifies the frequency of the modulating signal.

Query Syntax :WGEN<w>:MODulation:AM:FREQuency?

The :WGEN<w>:MODulation:AM:FREQuency? query returns the frequency of the modulating signal.

Return Format <frequency><NL>

<frequency> ::= modulating waveform frequency in Hz in NR3 format

- See Also**
- [":WGEN<w>:MODulation:AM:DEPTH"](#) on page 1493
 - [":WGEN<w>:MODulation:FM:DEViation"](#) on page 1495
 - [":WGEN<w>:MODulation:FM:FREQuency"](#) on page 1496
 - [":WGEN<w>:MODulation:FSKey:FREQuency"](#) on page 1497
 - [":WGEN<w>:MODulation:FSKey:RATE"](#) on page 1498
 - [":WGEN<w>:MODulation:FUNCTion"](#) on page 1499
 - [":WGEN<w>:MODulation:FUNCTion:RAMP:SYMMetry"](#) on page 1500
 - [":WGEN<w>:MODulation:STATe"](#) on page 1502
 - [":WGEN<w>:MODulation:TYPE"](#) on page 1503

:WGEN<w>:MODulation:FM:DEVIation

N (see [page 1650](#))

Command Syntax :WGEN<w>:MODulation:FM:DEVIation <frequency>

<w> ::= 1 in NR1 format

<frequency> ::= frequency deviation in Hz in NR3 format

The :WGEN<w>:MODulation:FM:DEVIation command specifies the frequency deviation from the original carrier signal frequency.

When the modulating signal is at its maximum amplitude, the output frequency is the carrier signal frequency plus the deviation amount, and when the modulating signal is at its minimum amplitude, the output frequency is the carrier signal frequency minus the deviation amount.

The frequency deviation cannot be greater than the original carrier signal frequency.

Also, the sum of the original carrier signal frequency and the frequency deviation must be less than or equal to the maximum frequency for the selected waveform generator function plus 100 kHz.

Query Syntax :WGEN<w>:MODulation:FM:DEVIation?

The :WGEN<w>:MODulation:FM:DEVIation? query returns the frequency deviation setting.

Return Format <frequency><NL>

<frequency> ::= frequency deviation in Hz in NR3 format

- See Also**
- [":WGEN<w>:MODulation:AM:DEPTH"](#) on page 1493
 - [":WGEN<w>:MODulation:AM:FREQuency"](#) on page 1494
 - [":WGEN<w>:MODulation:FM:FREQuency"](#) on page 1496
 - [":WGEN<w>:MODulation:FSKey:FREQuency"](#) on page 1497
 - [":WGEN<w>:MODulation:FSKey:RATE"](#) on page 1498
 - [":WGEN<w>:MODulation:FUNCTion"](#) on page 1499
 - [":WGEN<w>:MODulation:FUNCTion:RAMP:SYMMetry"](#) on page 1500
 - [":WGEN<w>:MODulation:STATe"](#) on page 1502
 - [":WGEN<w>:MODulation:TYPE"](#) on page 1503

:WGEN<w>:MODulation:FM:FREQuency

N (see [page 1650](#))

Command Syntax :WGEN<w>:MODulation:FM:FREQuency <frequency>

<w> ::= 1 in NR1 format

<frequency> ::= modulating waveform frequency in Hz in NR3 format

The :WGEN<w>:MODulation:FM:FREQuency command specifies the frequency of the modulating signal.

Query Syntax :WGEN<w>:MODulation:FM:FREQuency?

The :WGEN<w>:MODulation:FM:FREQuency? query returns the frequency of the modulating signal.

Return Format <frequency><NL>

<frequency> ::= modulating waveform frequency in Hz in NR3 format

- See Also**
- [":WGEN<w>:MODulation:AM:DEPTH"](#) on page 1493
 - [":WGEN<w>:MODulation:AM:FREQuency"](#) on page 1494
 - [":WGEN<w>:MODulation:FM:DEViation"](#) on page 1495
 - [":WGEN<w>:MODulation:FSKey:FREQuency"](#) on page 1497
 - [":WGEN<w>:MODulation:FSKey:RATE"](#) on page 1498
 - [":WGEN<w>:MODulation:FUNCTion"](#) on page 1499
 - [":WGEN<w>:MODulation:FUNCTion:RAMP:SYMMetry"](#) on page 1500
 - [":WGEN<w>:MODulation:STATe"](#) on page 1502
 - [":WGEN<w>:MODulation:TYPE"](#) on page 1503

:WGEN<w>:MODulation:FSKey:FREQuency

N (see [page 1650](#))

Command Syntax :WGEN<w>:MODulation:FSKey:FREQuency <frequency>

<w> ::= 1 in NR1 format

<frequency> ::= hop frequency in Hz in NR3 format

The :WGEN<w>:MODulation:FSKey:FREQuency command specifies the "hop frequency".

The output frequency "shifts" between the original carrier frequency and this "hop frequency".

Query Syntax :WGEN<w>:MODulation:FSKey:FREQuency?

The :WGEN<w>:MODulation:FSKey:FREQuency? query returns the "hop frequency" setting.

Return Format <frequency><NL>

<frequency> ::= hop frequency in Hz in NR3 format

- See Also**
- [":WGEN<w>:MODulation:AM:DEPTH"](#) on page 1493
 - [":WGEN<w>:MODulation:AM:FREQuency"](#) on page 1494
 - [":WGEN<w>:MODulation:FM:DEViation"](#) on page 1495
 - [":WGEN<w>:MODulation:FM:FREQuency"](#) on page 1496
 - [":WGEN<w>:MODulation:FSKey:RATE"](#) on page 1498
 - [":WGEN<w>:MODulation:FUNCTion"](#) on page 1499
 - [":WGEN<w>:MODulation:FUNCTion:RAMP:SYMMetry"](#) on page 1500
 - [":WGEN<w>:MODulation:STATe"](#) on page 1502
 - [":WGEN<w>:MODulation:TYPE"](#) on page 1503

:WGEN<w>:MODulation:FSKey:RATE

N (see [page 1650](#))

Command Syntax :WGEN<w>:MODulation:FSKey:RATE <rate>

<w> ::= 1 in NR1 format

<rate> ::= FSK modulation rate in Hz in NR3 format

The :WGEN<w>:MODulation:FSKey:RATE command specifies the rate at which the output frequency "shifts".

The FSK rate specifies a digital square wave modulating signal.

Query Syntax :WGEN<w>:MODulation:FSKey:RATE?

The :WGEN<w>:MODulation:FSKey:RATE? query returns the FSK rate setting.

Return Format <rate><NL>

<rate> ::= FSK modulation rate in Hz in NR3 format

- See Also**
- [":WGEN<w>:MODulation:AM:DEPTH"](#) on page 1493
 - [":WGEN<w>:MODulation:AM:FREQuency"](#) on page 1494
 - [":WGEN<w>:MODulation:FM:DEViation"](#) on page 1495
 - [":WGEN<w>:MODulation:FM:FREQuency"](#) on page 1496
 - [":WGEN<w>:MODulation:FSKey:FREQuency"](#) on page 1497
 - [":WGEN<w>:MODulation:FUNCTion"](#) on page 1499
 - [":WGEN<w>:MODulation:FUNCTion:RAMP:SYMMetry"](#) on page 1500
 - [":WGEN<w>:MODulation:STATe"](#) on page 1502
 - [":WGEN<w>:MODulation:TYPE"](#) on page 1503

:WGEN<w>:MODulation:FUNCTion

N (see [page 1650](#))

Command Syntax :WGEN<w>:MODulation:FUNCTion <shape>

<w> ::= 1 in NR1 format

<shape> ::= {SINusoid | SQUare | RAMP}

The :WGEN<w>:MODulation:FUNCTion command specifies the shape of the modulating signal.

When the RAMP shape is selected, you can specify the amount of time per cycle that the ramp waveform is rising with the :WGEN<w>:MODulation:FUNCTion:RAMP:SYMMetry command.

This command applies to AM and FM modulation. (The FSK modulation signal is a square wave shape.)

Query Syntax :WGEN<w>:MODulation:FUNCTion?

The :WGEN<w>:MODulation:FUNCTion? query returns the specified modulating signal shape.

Return Format <shape><NL>

<shape> ::= {SIN | SQU | RAMP}

- See Also**
- [":WGEN<w>:MODulation:AM:DEPTH"](#) on page 1493
 - [":WGEN<w>:MODulation:AM:FREQuency"](#) on page 1494
 - [":WGEN<w>:MODulation:FM:DEViation"](#) on page 1495
 - [":WGEN<w>:MODulation:FM:FREQuency"](#) on page 1496
 - [":WGEN<w>:MODulation:FSKey:FREQuency"](#) on page 1497
 - [":WGEN<w>:MODulation:FSKey:RATE"](#) on page 1498
 - [":WGEN<w>:MODulation:FUNCTion:RAMP:SYMMetry"](#) on page 1500
 - [":WGEN<w>:MODulation:STATe"](#) on page 1502
 - [":WGEN<w>:MODulation:TYPE"](#) on page 1503

:WGEN<w>:MODulation:FUNCtion:RAMP:SYMMetry

N (see [page 1650](#))

Command Syntax :WGEN<w>:MODulation:FUNCtion:RAMP:SYMMetry <percent>

<w> ::= 1 in NR1 format

<percent> ::= symmetry percentage from 0% to 100% in NR1 format

The :WGEN<w>:MODulation:FUNCtion:RAMP:SYMMetry command specifies the amount of time per cycle that the ramp waveform is rising. The ramp modulating waveform shape is specified with the :WGEN<w>:MODulation:FUNCtion command.

Query Syntax :WGEN<w>:MODulation:FUNCtion:RAMP:SYMMetry?

The :WGEN<w>:MODulation:FUNCtion:RAMP:SYMMetry? query returns ramp symmetry percentage setting.

Return Format <percent><NL>

<percent> ::= symmetry percentage from 0% to 100% in NR1 format

- See Also**
- [":WGEN<w>:MODulation:AM:DEPTH"](#) on page 1493
 - [":WGEN<w>:MODulation:AM:FREQuency"](#) on page 1494
 - [":WGEN<w>:MODulation:FM:DEViation"](#) on page 1495
 - [":WGEN<w>:MODulation:FM:FREQuency"](#) on page 1496
 - [":WGEN<w>:MODulation:FSKey:FREQuency"](#) on page 1497
 - [":WGEN<w>:MODulation:FSKey:RATE"](#) on page 1498
 - [":WGEN<w>:MODulation:FUNCtion"](#) on page 1499
 - [":WGEN<w>:MODulation:STATe"](#) on page 1502
 - [":WGEN<w>:MODulation:TYPE"](#) on page 1503

:WGEN<w>:MODulation:NOISe

N (see [page 1650](#))

Command Syntax :WGEN<w>:MODulation:NOISe <percent>

<w> ::= 1 in NR1 format

<percent> ::= 0 to 100

The :WGEN<w>:MODulation:NOISe command adds noise to the currently selected signal. The sum of the amplitude between the original signal and injected noise is limited to the regular amplitude limit (for example, 5 Vpp in 1 MOhm), so the range for <percent> varies according to current amplitude.

Note that adding noise affects edge triggering on the waveform generator source as well as the waveform generator sync pulse output signal (which can be sent to TRIG OUT). This is because the trigger comparator is located after the noise source.

Query Syntax :WGEN<w>:MODulation:NOISe?

The :WGEN<w>:MODulation:NOISe query returns the percent of added noise.

Return Format <percent><NL>

<percent> ::= 0 to 100

See Also • [":WGEN<w>:FUNCTION"](#) on page 1486

:WGEN<w>:MODulation:STATe

N (see [page 1650](#))

Command Syntax :WGEN<w>:MODulation:STATe <setting>

<w> ::= 1 in NR1 format

<setting> ::= {{OFF | 0} | {ON | 1}}

The :WGEN<w>:MODulation:STATe command enables or disables modulated waveform generator output.

You can enable modulation for all waveform generator function types except pulse, DC, and noise.

Query Syntax :WGEN<w>:MODulation:STATe?

The :WGEN<w>:MODulation:STATe? query returns whether the modulated waveform generator output is enabled or disabled.

Return Format <setting><NL>

<setting> ::= {0 | 1}

- See Also**
- [":WGEN<w>:MODulation:AM:DEPTH"](#) on page 1493
 - [":WGEN<w>:MODulation:AM:FREQuency"](#) on page 1494
 - [":WGEN<w>:MODulation:FM:DEViation"](#) on page 1495
 - [":WGEN<w>:MODulation:FM:FREQuency"](#) on page 1496
 - [":WGEN<w>:MODulation:FSKey:FREQuency"](#) on page 1497
 - [":WGEN<w>:MODulation:FSKey:RATE"](#) on page 1498
 - [":WGEN<w>:MODulation:FUNCTion"](#) on page 1499
 - [":WGEN<w>:MODulation:FUNCTion:RAMP:SYMMetry"](#) on page 1500
 - [":WGEN<w>:MODulation:TYPE"](#) on page 1503

:WGEN<w>:MODulation:TYPE

N (see [page 1650](#))

Command Syntax :WGEN<w>:MODulation:TYPE <type>

<w> ::= 1 in NR1 format

<type> ::= {AM | FM | FSK}

The :WGEN<w>:MODulation:TYPE command selects the modulation type:

- AM (amplitude modulation) – the amplitude of the original carrier signal is modified according to the amplitude of the modulating signal.

Use the :WGEN<w>:MODulation:AM:FREQuency command to set the modulating signal frequency.

Use the :WGEN<w>:MODulation:AM:DEPTH command to specify the amount of amplitude modulation.

- FM (frequency modulation) – the frequency of the original carrier signal is modified according to the amplitude of the modulating signal.

Use the :WGEN<w>:MODulation:FM:FREQuency command to set the modulating signal frequency.

Use the :WGEN<w>:MODulation:FM:DEVIation command to specify the frequency deviation from the original carrier signal frequency.

- FSK (frequency-shift keying modulation) – the output frequency "shifts" between the original carrier frequency and a "hop frequency" at the specified FSK rate.

The FSK rate specifies a digital square wave modulating signal.

Use the :WGEN<w>:MODulation:FSKey:FREQuency command to specify the "hop frequency".

Use the :WGEN<w>:MODulation:FSKey:RATE command to specify the rate at which the output frequency "shifts".

Query Syntax :WGEN<w>:MODulation:TYPE?

The :WGEN<w>:MODulation:TYPE? query returns the selected modulation type.

Return Format <type><NL>

<type> ::= {AM | FM | FSK}

- See Also**
- [":WGEN<w>:MODulation:AM:DEPTH"](#) on page 1493
 - [":WGEN<w>:MODulation:AM:FREQuency"](#) on page 1494
 - [":WGEN<w>:MODulation:FM:DEVIation"](#) on page 1495
 - [":WGEN<w>:MODulation:FM:FREQuency"](#) on page 1496

- [":WGEN<w>:MODulation:FSKey:FREQuency"](#) on page 1497
- [":WGEN<w>:MODulation:FSKey:RATE"](#) on page 1498
- [":WGEN<w>:MODulation:FUNcTion"](#) on page 1499
- [":WGEN<w>:MODulation:FUNcTion:RAMP:SYMMetry"](#) on page 1500
- [":WGEN<w>:MODulation:STATe"](#) on page 1502

:WGEN<w>:OUTPut

N (see [page 1650](#))

Command Syntax :WGEN<w>:OUTPut <on_off>

<w> ::= 1 to (# WaveGen outputs) in NR1 format

<on_off> ::= {{1 | ON} | {0 | OFF}}

The :WGEN<w>:OUTPut command specifies whether the waveform generator signal output is ON (1) or OFF (0).

Query Syntax :WGEN<w>:OUTPut?

The :WGEN<w>:OUTPut? query returns the current state of the waveform generator output setting.

Return Format <on_off><NL>

<on_off> ::= {1 | 0}

See Also • ["Introduction to :WGEN<w> Commands"](#) on page 1475

:WGEN<w>:OUTPut:LOAD

N (see [page 1650](#))

Command Syntax :WGEN<w>:OUTPut:LOAD <impedance>

<w> ::= 1 to (# WaveGen outputs) in NR1 format

<impedance> ::= {ONEMeg | FIFTy}

The :WGEN<w>:OUTPut:LOAD command selects the expected output load impedance.

The output impedance of the Gen Out BNC is fixed at 50 ohms. However, the output load selection lets the waveform generator display the correct amplitude and offset levels for the expected output load.

If the actual load impedance is different than the selected value, the displayed amplitude and offset levels will be incorrect.

Query Syntax :WGEN<w>:OUTPut:LOAD?

The :WGEN<w>:OUTPut:LOAD? query returns the current expected output load impedance.

Return Format <impedance><NL>

<impedance> ::= {ONEM | FIFT}

See Also • ["Introduction to :WGEN<w> Commands"](#) on page 1475

:WGEN<w>:OUTPut:MODE

N (see [page 1650](#))

Command Syntax :WGEN<w>:OUTPut:MODE <mode>

<mode> ::= {NORMal | SINGle}

The :WGEN<w>:OUTPut:MODE command specifies whether the defined waveform is output continuously or as a single cycle (single-shot):

- NORMal – the defined waveform is output continuously.
- SINGle – one cycle of the defined waveform is output when you send the :WGEN<w>:OUTPut:SINGle command.

Not all waveform types allow single-shot output.

Query Syntax :WGEN<w>:OUTPut:MODE?

The :WGEN<w>:OUTPut:MODE? query returns the output mode setting.

Return Format <mode><NL>

<mode> ::= {NORMal | SINGle}

See Also • [":WGEN<w>:OUTPut:SINGle"](#) on page 1509

:WGEN<w>:OUTPut:POLarity

N (see [page 1650](#))

Command Syntax :WGEN<w>:OUTPut:POLarity <polarity>

<w> ::= 1 to (# WaveGen outputs) in NR1 format

<polarity> ::= {NORMal | INVerted}

The :WGEN<w>:OUTPut:POLarity command specifies whether the waveform generator output is inverted..

Query Syntax :WGEN<w>:OUTPut:POLarity?

The :WGEN<w>:OUTPut:POLarity? query returns the specified output polarity.

Return Format <polarity><NL>

<polarity> ::= {NORM | INV}

See Also • ["Introduction to :WGEN<w> Commands"](#) on page 1475

:WGEN<w>:OUTPut:SINGle

N (see [page 1650](#))

Command Syntax :WGEN<w>:OUTPut:SINGle

When the single-shot output mode is selected (by the :WGEN<w>:OUTPut:MODE command), the :WGEN<w>:OUTPut:SINGle command causes a single cycle of the defined waveform to be output.

Sending this command multiple times will interrupt a slow signal output before the cycle is completed.

See Also • [":WGEN<w>:OUTPut:MODE"](#) on page 1507

:WGEN<w>:PERiod

N (see [page 1650](#))

Command Syntax :WGEN<w>:PERiod <period>

<w> ::= 1 to (# WaveGen outputs) in NR1 format

<period> ::= period in seconds in NR3 format

For all waveforms except Noise and DC, the :WGEN<w>:PERiod command specifies the period of the waveform.

You can also specify the period indirectly using the :WGEN<w>:FREQuency command.

Query Syntax :WGEN<w>:PERiod?

The :WGEN<w>:PERiod? query returns the currently set waveform generator period.

Return Format <period><NL>

<period> ::= period in seconds in NR3 format

- See Also**
- ["Introduction to :WGEN<w> Commands"](#) on page 1475
 - [":WGEN<w>:FUNction"](#) on page 1486
 - [":WGEN<w>:FREQuency"](#) on page 1485

:WGEN<w>:RST

N (see [page 1650](#))

Command Syntax :WGEN<w>:RST

<w> ::= 1 to (# WaveGen outputs) in NR1 format

The :WGEN<w>:RST command restores the waveform generator factory default settings (1 kHz sine wave, 500 mVpp, 0 V offset).

- See Also**
- ["Introduction to :WGEN<w> Commands"](#) on page 1475
 - [":WGEN<w>:FUNCTION"](#) on page 1486
 - [":WGEN<w>:FREQUENCY"](#) on page 1485

:WGEN<w>:TRACk

N (see [page 1650](#))

Command Syntax :WGEN<w>:TRACk <on_off>

<w> ::= 1 to (# WaveGen outputs) in NR1 format

<on_off> ::= {{1 | ON} | {0 | OFF}}

The :WGEN<w>:TRACk ON command causes frequency, amplitude, offset, and duty cycle adjustments to this waveform generator output signal to be tracked by the other waveform generator output.

Not all waveform shapes can be frequency tracked. When :WGEN<w>:TRACk or :WGEN<w>:TRACk:FREQuency is enabled, the other generator's waveform selections (:WGEN<w>:FUNCTion) must be in the same group as this generator's waveform:

Group 1	Group 2	Group 3	Group 4
SINusoid RAMP SINC CARDiac GAUSSian	SQUare PULSe	EXPRise EXPFall	ARBitrary

When tracking is enabled, adjustments for tracked settings in the other waveform generator become unavailable.

Query Syntax :WGEN<w>:TRACk?

The :WGEN<w>:TRACk? query returns whether tracking is on or off.

Return Format <on_off><NL>

<on_off> ::= {1 | 0}

- See Also**
- [":WGEN<w>:FUNCTion"](#) on page 1486
 - [":WGEN<w>:TRACk:AMPLitude"](#) on page 1513
 - [":WGEN<w>:TRACk:FREQuency"](#) on page 1515
 - [":WGEN<w>:TRACk:PHASe"](#) on page 1516
 - [":WGEN<w>:TRACk:CSIGnal"](#) on page 1514

:WGEN<w>:TRACk:AMPLitude

N (see [page 1650](#))

Command Syntax :WGEN<w>:TRACk:AMPLitude <on_off>

<w> ::= 1 to (# WaveGen outputs) in NR1 format

<on_off> ::= {{1 | ON} | {0 | OFF}}

The :WGEN<w>:TRACk:AMPLitude ON command causes amplitude and offset adjustments to this waveform generator output signal to be tracked by the other waveform generator output.

Also, when tracking is enabled, adjustments for tracked settings in the other waveform generator become unavailable (ghosted).

Query Syntax :WGEN<w>:TRACk:AMPLitude?

The :WGEN<w>:TRACk:AMPLitude? query returns whether amplitude tracking is on or off.

Return Format <on_off><NL>

<on_off> ::= {1 | 0}

- See Also**
- [":WGEN<w>:TRACk"](#) on page 1512
 - [":WGEN<w>:TRACk:FREQuency"](#) on page 1515
 - [":WGEN<w>:TRACk:PHASe"](#) on page 1516
 - [":WGEN<w>:TRACk:CSIGnal"](#) on page 1514

:WGEN<w>:TRACk:CSIGnal

N (see [page 1650](#))

Command Syntax :WGEN<w>:TRACk:CSIGnal

The :WGEN<w>:TRACk:CSIGnal command sets the other waveform generator output to be identical to this waveform generator output (except that the shape of either output may be inverted).

If the signal selected is ARB, the ARB waveform is copied as well.

- See Also**
- [":WGEN<w>:TRACk"](#) on page 1512
 - [":WGEN<w>:TRACk:AMPLitude"](#) on page 1513
 - [":WGEN<w>:TRACk:FREQuency"](#) on page 1515
 - [":WGEN<w>:TRACk:PHASe"](#) on page 1516

:WGEN<w>:TRACk:FREQuency

N (see [page 1650](#))

Command Syntax :WGEN<w>:TRACk:FREQuency <on_off>

<w> ::= 1 to (# WaveGen outputs) in NR1 format

<on_off> ::= {{1 | ON} | {0 | OFF}}

The :WGEN<w>:TRACk:FREQuency ON command causes frequency adjustments to this waveform generator output signal to be tracked by the other waveform generator output.

Not all waveform shapes can be frequency tracked. When :WGEN<w>:TRACk or :WGEN<w>:TRACk:FREQuency is enabled, the other generator's waveform selections (:WGEN<w>:FUNCTioN) must be in the same group as this generator's waveform:

Group 1	Group 2	Group 3	Group 4
SINusoid RAMP SINC CARDiac GAUSSian	SQUare PULSe	EXPRise EXPFall	ARBitrary

When tracking is enabled, adjustments for tracked settings in the other waveform generator become unavailable.

Query Syntax :WGEN<w>:TRACk:FREQuency?

The :WGEN<w>:TRACk:FREQuency? query returns whether frequency tracking is on or off.

Return Format <on_off><NL>

<on_off> ::= {1 | 0}

- See Also**
- [":WGEN<w>:TRACk:PHASe"](#) on page 1516
 - [":WGEN<w>:FUNCTioN"](#) on page 1486
 - [":WGEN<w>:TRACk"](#) on page 1512
 - [":WGEN<w>:TRACk:AMPLitude"](#) on page 1513
 - [":WGEN<w>:TRACk:CSIGnal"](#) on page 1514

:WGEN<w>:TRACk:PHASe

N (see [page 1650](#))

Command Syntax :WGEN<w>:TRACk:PHASe <phase>

<w> ::= 1 to (# WaveGen outputs) in NR1 format

<phase> ::= -360 to +360 in NR1 format

The :WGEN<w>:TRACk:PHASe command sets the phase relationship of a frequency-tracked waveform generator output.

Query Syntax :WGEN<w>:TRACk:PHASe?

The :WGEN<w>:TRACk:PHASe? query returns the frequency-tracked waveform phase relationship setting.

Return Format <phase><NL>

<phase> ::= -360 to +360 in NR1 format

- See Also**
- [":WGEN<w>:TRACk"](#) on page 1512
 - [":WGEN<w>:TRACk:AMPLitude"](#) on page 1513
 - [":WGEN<w>:TRACk:FREQuency"](#) on page 1515
 - [":WGEN<w>:TRACk:CSIGnal"](#) on page 1514

:WGEN<w>:VOLTage

N (see [page 1650](#))

Command Syntax :WGEN<w>:VOLTage <amplitude>

<w> ::= 1 to (# WaveGen outputs) in NR1 format

<amplitude> ::= amplitude in volts in NR3 format

For all waveforms except DC, the :WGEN<w>:VOLTage command specifies the waveform's amplitude. Use the :WGEN<w>:VOLTage:OFFSet command to specify the offset voltage or DC level.

You can also specify the amplitude and offset indirectly using the :WGEN<w>:VOLTage:HIGH and :WGEN<w>:VOLTage:LOW commands. For example, an amplitude of 5 V and an offset of 1 V is the same as a high-level voltage of 4 V and a low-level voltage of -1 V.

NOTE

When the amplitude is below 40 mV, the offset is limited to ± 500 mV.

Query Syntax :WGEN<w>:VOLTage?

The :WGEN<w>:VOLTage? query returns the currently specified waveform amplitude.

Return Format <amplitude><NL>

<amplitude> ::= amplitude in volts in NR3 format

- See Also**
- ["Introduction to :WGEN<w> Commands"](#) on page 1475
 - [":WGEN<w>:FUNCTION"](#) on page 1486
 - [":WGEN<w>:VOLTage:OFFSet"](#) on page 1520
 - [":WGEN<w>:VOLTage:HIGH"](#) on page 1518
 - [":WGEN<w>:VOLTage:LOW"](#) on page 1519

:WGEN<w>:VOLTage:HIGH

N (see [page 1650](#))

Command Syntax :WGEN<w>:VOLTage:HIGH <high>

<w> ::= 1 to (# WaveGen outputs) in NR1 format

<high> ::= high-level voltage in volts, in NR3 format

For all waveforms except DC, the :WGEN<w>:VOLTage:HIGH command specifies the waveform's high-level voltage. Use the :WGEN<w>:VOLTage:LOW command to specify the low-level voltage.

You can also specify the high-level and low-level voltages indirectly using the :WGEN<w>:VOLTage and :WGEN<w>:VOLTage:OFFSet commands. For example, a high-level voltage of 4 V and a low-level voltage of -1 V is the same as an amplitude of 5 V and an offset of 1 V.

NOTE

When the amplitude is below 40 mV, the offset is limited to ± 500 mV.

Query Syntax :WGEN<w>:VOLTage:HIGH?

The :WGEN<w>:VOLTage:HIGH? query returns the currently specified waveform high-level voltage.

Return Format <high><NL>

<high> ::= high-level voltage in volts, in NR3 format

- See Also**
- ["Introduction to :WGEN<w> Commands"](#) on page 1475
 - [":WGEN<w>:FUNCTION"](#) on page 1486
 - [":WGEN<w>:VOLTage:LOW"](#) on page 1519
 - [":WGEN<w>:VOLTage"](#) on page 1517
 - [":WGEN<w>:VOLTage:OFFSet"](#) on page 1520

:WGEN<w>:VOLTage:LOW

N (see [page 1650](#))

Command Syntax :WGEN<w>:VOLTage:LOW <low>

<w> ::= 1 to (# WaveGen outputs) in NR1 format

<low> ::= low-level voltage in volts, in NR3 format

For all waveforms except DC, the :WGEN<w>:VOLTage:LOW command specifies the waveform's low-level voltage. Use the :WGEN<w>:VOLTage:HIGH command to specify the high-level voltage.

You can also specify the high-level and low-level voltages indirectly using the :WGEN<w>:VOLTage and :WGEN<w>:VOLTage:OFFSet commands. For example, a high-level voltage of 4 V and a low-level voltage of -1 V is the same as an amplitude of 5 V and an offset of 1 V.

NOTE

When the amplitude is below 40 mV, the offset is limited to ± 500 mV.

Query Syntax :WGEN<w>:VOLTage:LOW?

The :WGEN<w>:VOLTage:LOW? query returns the currently specified waveform low-level voltage.

Return Format <low><NL>

<low> ::= low-level voltage in volts, in NR3 format

- See Also**
- ["Introduction to :WGEN<w> Commands"](#) on page 1475
 - [":WGEN<w>:FUNCTION"](#) on page 1486
 - [":WGEN<w>:VOLTage:LOW"](#) on page 1519
 - [":WGEN<w>:VOLTage"](#) on page 1517
 - [":WGEN<w>:VOLTage:OFFSet"](#) on page 1520

:WGEN<w>:VOLTage:OFFSet

N (see [page 1650](#))

Command Syntax :WGEN<w>:VOLTage:OFFSet <offset>

<w> ::= 1 to (# WaveGen outputs) in NR1 format

<offset> ::= offset in volts in NR3 format

The :WGEN<w>:VOLTage:OFFSet command specifies the waveform's offset voltage or the DC level. Use the :WGEN<w>:VOLTage command to specify the amplitude.

You can also specify the amplitude and offset indirectly using the :WGEN<w>:VOLTage:HIGH and :WGEN<w>:VOLTage:LOW commands. For example, an amplitude of 5 V and an offset of 1 V is the same as a high-level voltage of 4 V and a low-level voltage of -1 V.

NOTE

When the amplitude is below 40 mV, the offset is limited to ± 500 mV.

Query Syntax :WGEN<w>:VOLTage:OFFSet?

The :WGEN<w>:VOLTage:OFFSet? query returns the currently specified waveform offset voltage.

Return Format <offset><NL>

<offset> ::= offset in volts in NR3 format

- See Also**
- ["Introduction to :WGEN<w> Commands"](#) on page 1475
 - [":WGEN<w>:FUNCTION"](#) on page 1486
 - [":WGEN<w>:VOLTage"](#) on page 1517
 - [":WGEN<w>:VOLTage:HIGH"](#) on page 1518
 - [":WGEN<w>:VOLTage:LOW"](#) on page 1519

40 :WMEMemory<r> Commands

Control reference waveforms.

Table 170 :WMEMemory<r> Commands Summary

Command	Query	Options and Query Returns
:WMEMemory<r>:CLEar (see page 1523)	n/a	<r> ::= 1 to (# ref waveforms) in NR1 format
:WMEMemory<r>:DISPlay { {0 OFF} {1 ON} } (see page 1524)	:WMEMemory<r>:DISPlay? (see page 1524)	<r> ::= 1 to (# ref waveforms) in NR1 format {0 1}
:WMEMemory<r>:LABel <string> (see page 1525)	:WMEMemory<r>:LABel? (see page 1525)	<r> ::= 1 to (# ref waveforms) in NR1 format <string> ::= any series of 10 or less ASCII characters enclosed in quotation marks
:WMEMemory<r>:SAVE <source> (see page 1526)	n/a	<r> ::= 1 to (# ref waveforms) in NR1 format <source> ::= {CHANnel<n> FUNCTION<m> MATH<m>} <n> ::= 1 to (# analog channels) in NR1 format <m> ::= 1 to (# math functions) in NR1 format NOTE: Math functions whose x-axis is not frequency can be saved as reference waveforms.
:WMEMemory<r>:SKEW <skew> (see page 1527)	:WMEMemory<r>:SKEW? (see page 1527)	<r> ::= 1 to (# ref waveforms) in NR1 format <skew> ::= time in seconds in NR3 format

Table 170 :WMEemory<r> Commands Summary (continued)

Command	Query	Options and Query Returns
:WMEemory<r>:YOffset <offset>[suffix] (see page 1528)	:WMEemory<r>:YOffset? (see page 1528)	<r> ::= 1 to (# ref waveforms) in NR1 format <offset> ::= vertical offset value in NR3 format [suffix] ::= {V mV}
:WMEemory<r>:YRange <range> (see page 1529)	:WMEemory<r>:YRange? (see page 1529)	<r> ::= 1 to (# ref waveforms) in NR1 format <range> ::= vertical full-scale range value in NR3 format
:WMEemory<r>:YScale <scale> (see page 1530)	:WMEemory<r>:YScale? (see page 1530)	<r> ::= 1 to (# ref waveforms) in NR1 format <scale> ::= vertical units per division value in NR3 format

:WMEemory<r>:CLEar

N (see [page 1650](#))

Command Syntax :WMEemory<r>:CLEar

<r> ::= 1 to (# ref waveforms) in NR1 format

The :WMEemory<r>:CLEar command clears the specified reference waveform location.

- See Also**
- [Chapter 40](#), “:WMEemory<r> Commands,” starting on page 1521
 - [":WMEemory<r>:SAVE"](#) on page 1526
 - [":WMEemory<r>:DISPlay"](#) on page 1524

:WMEemory<r>:DISPlay

N (see [page 1650](#))

Command Syntax :WMEemory<r>:DISPlay <on_off>

<r> ::= 1 to (# ref waveforms) in NR1 format

<on_off> ::= {{1 | ON} | {0 | OFF}}

The :WMEemory<r>:DISPlay command turns the display of the specified reference waveform on or off.

There are two reference waveform locations, but only one reference waveform can be displayed at a time. That means, if :WMEemory1:DISPlay is ON, sending the :WMEemory2:DISPlay ON command will automatically set :WMEemory1:DISPlay OFF.

Query Syntax :WMEemory<r>:DISPlay?

The :WMEemory<r>:DISPlay? query returns the current display setting for the reference waveform.

Return Format <on_off><NL>

<on_off> ::= {1 | 0}

- See Also**
- [Chapter 40](#), “:WMEemory<r> Commands,” starting on page 1521
 - [":WMEemory<r>:CLEar"](#) on page 1523
 - [":WMEemory<r>:LABel"](#) on page 1525

:WMEemory<r>:LABel

N (see [page 1650](#))

Command Syntax :WMEemory<r>:LABel <string>
 <r> ::= 1 to (# ref waveforms) in NR1 format
 <string> ::= quoted ASCII string

NOTE

Label strings are 10 characters or less, and may contain any commonly used ASCII characters. Labels with more than 10 characters are truncated to 10 characters. Lower case characters are converted to upper case.

The :WMEemory<r>:LABel command sets the reference waveform label to the string that follows.

Setting a label for a reference waveform also adds the name to the label list in non-volatile memory (replacing the oldest label in the list).

Query Syntax :WMEemory<r>:LABel?

The :WMEemory<r>:LABel? query returns the label associated with a particular reference waveform.

Return Format <string><NL>
 <string> ::= quoted ASCII string

See Also

- [Chapter 40](#), “:WMEemory<r> Commands,” starting on page 1521
- [":WMEemory<r>:DISPlay"](#) on page 1524

:WMEemory<r>:SAVE

N (see [page 1650](#))

Command Syntax :WMEemory<r>:SAVE <source>

<r> ::= 1 to (# ref waveforms) in NR1 format

<source> ::= {CHANnel<n> | FUNction<m> | MATH<m>}

<n> ::= 1 to (# analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

The :WMEemory<r>:SAVE command copies the analog channel or math function waveform to the specified reference waveform location.

NOTE

Math functions whose x-axis is not frequency can be saved as reference waveforms.

-
- See Also**
- [Chapter 40](#), “:WMEemory<r> Commands,” starting on page 1521
 - [":WMEemory<r>:DISPlay"](#) on page 1524

:WMEemory<r>:SKEW

N (see [page 1650](#))

Command Syntax :WMEemory<r>:SKEW <skew>

<r> ::= 1 to (# ref waveforms) in NR1 format

<skew> ::= time in seconds in NR3 format

The :WMEemory<r>:SKEW command sets the skew factor for the specified reference waveform.

Query Syntax :WMEemory<r>:SKEW?

The :WMEemory<r>:SKEW? query returns the current skew setting for the selected reference waveform.

Return Format <skew><NL>

<skew> ::= time in seconds in NR3 format

- See Also**
- [Chapter 40](#), “:WMEemory<r> Commands,” starting on page 1521
 - [":WMEemory<r>:DISPlay"](#) on page 1524
 - [":WMEemory<r>:YOFFset"](#) on page 1528
 - [":WMEemory<r>:YRANge"](#) on page 1529
 - [":WMEemory<r>:YSCale"](#) on page 1530

:WMEMory<r>:YOFFset

N (see [page 1650](#))

Command Syntax :WMEMory<r>:YOFFset <offset> [<suffix>]

<r> ::= 1 to (# ref waveforms) in NR1 format

<offset> ::= vertical offset value in NR3 format

<suffix> ::= {V | mV}

The :WMEMory<r>:YOFFset command sets the value that is represented at center screen for the selected reference waveform.

The range of legal values varies with the value set by the :WMEMory<r>:YRANge or :WMEMory<r>:YSCale commands. If you set the offset to a value outside of the legal range, the offset value is automatically set to the nearest legal value. Legal values are affected by the probe attenuation setting.

Query Syntax :WMEMory<r>:YOFFset?

The :WMEMory<r>:YOFFset? query returns the current offset value for the selected reference waveform.

Return Format <offset><NL>

<offset> ::= vertical offset value in NR3 format

- See Also**
- [Chapter 40](#), “:WMEMory<r> Commands,” starting on page 1521
 - [":WMEMory<r>:DISPlay"](#) on page 1524
 - [":WMEMory<r>:YRANge"](#) on page 1529
 - [":WMEMory<r>:YSCale"](#) on page 1530
 - [":WMEMory<r>:SKEW"](#) on page 1527

:WMEemory<r>:YRANge

N (see [page 1650](#))

Command Syntax :WMEemory<r>:YRANge <range>

<r> ::= 1 to (# ref waveforms) in NR1 format

<range> ::= vertical full-scale range value in NR3 format

The :WMEemory<r>:YRANge command defines the full-scale vertical axis of the selected reference waveform.

Legal values for the range are copied from the original source waveform (that is, the analog channel or math function waveform that was originally saved as a reference waveform).

Query Syntax :WMEemory<r>:YRANge?

The :WMEemory<r>:YRANge? query returns the current full-scale range setting for the specified reference waveform.

Return Format <range><NL>

<range> ::= vertical full-scale range value in NR3 format

- See Also**
- [Chapter 40](#), “:WMEemory<r> Commands,” starting on page 1521
 - [":WMEemory<r>:DISPlay"](#) on page 1524
 - [":WMEemory<r>:YOFFset"](#) on page 1528
 - [":WMEemory<r>:SKEW"](#) on page 1527
 - [":WMEemory<r>:YSCale"](#) on page 1530

:WMEMory<r>:YScale

N (see [page 1650](#))

Command Syntax :WMEMory<r>:YScale <scale>

<r> ::= 1 to (# ref waveforms) in NR1 format

<scale> ::= vertical units per division in NR3 format

The :WMEMory<r>:YScale command sets the vertical scale, or units per division, of the selected reference waveform.

Legal values for the scale are copied from the original source waveform (that is, the analog channel or math function waveform that was originally saved as a reference waveform).

Query Syntax :WMEMory<r>:YScale?

The :WMEMory<r>:YScale? query returns the current scale setting for the specified reference waveform.

Return Format <scale><NL>

<scale> ::= vertical units per division in NR3 format

- See Also**
- [Chapter 40](#), “:WMEMory<r> Commands,” starting on page 1521
 - [":WMEMory<r>:DISPlay"](#) on page 1524
 - [":WMEMory<r>:YOFFset"](#) on page 1528
 - [":WMEMory<r>:YRANge"](#) on page 1529
 - [":WMEMory<r>:SKEW"](#) on page 1527

41 Obsolete and Discontinued Commands

Obsolete commands are older forms of commands that are provided to reduce customer rework for existing systems and programs (see "**Obsolete Commands**" on page 1650).

Obsolete Command	Current Command Equivalent	Behavior Differences
ANALog<n>:BWLimit	:CHANnel<n>:BWLimit (see page 330)	
ANALog<n>:COUPling	:CHANnel<n>:COUPling (see page 331)	
ANALog<n>:INVert	:CHANnel<n>:INVert (see page 334)	
ANALog<n>:LABel	:CHANnel<n>:LABel (see page 335)	
ANALog<n>:OFFSet	:CHANnel<n>:OFFSet (see page 336)	
ANALog<n>:PROBe	:CHANnel<n>:PROBe (see page 337)	
ANALog<n>:PMODE	none	
ANALog<n>:RANGe	:CHANnel<n>:RANGe (see page 353)	
:CHANnel:ACTivity (see page 1538)	:ACTivity (see page 241)	
:CHANnel:LABel (see page 1539)	:CHANnel<n>:LABel (see page 335) or :DIGital<n>:LABel (see page 386)	use CHANnel<n>:LABel for analog channels and use DIGital<n>:LABel for digital channels

Obsolete Command	Current Command Equivalent	Behavior Differences
:CHANnel:THReshold (see page 1540)	:POD<n>:THReshold (see page 749) or :DIGital<d>:THReshold (see page 389)	
:CHANnel2:SKEW (see page 1541)	:CHANnel<n>:PROBe:SKEW (see page 349)	
:CHANnel<n>:INPut (see page 1542)	:CHANnel<n>:IMPedance (see page 333)	
:CHANnel<n>:PMODE (see page 1543)	none	
:DISPlay:CONNect (see page 1544)	:DISPlay:VECTors (see page 416)	
:DISPlay:ORDer (see page 1545)	none	
:ERASe (see page 1546)	:DISPlay:CLEar (see page 401)	
:EXTernal:PMODE (see page 1547)	none	
FUNCTION1, FUNCTION2	:FUNCTION Commands (see page 449)	ADD not included
:FUNCTION Commands	:FUNCTION2 Commands (see page 449)	:FUNCTION commands (with no <m> number) map to :FUNCTION2. This allows legacy programs to work without change.
:FUNCTION:GOFT:OPERation (see page 1548)	:FUNCTION1:OPERation (see page 487)	GOFT maps to FUNCTION1.
:FUNCTION:GOFT:SOURce1 (see page 1549)	:FUNCTION1:SOURce1 (see page 501)	GOFT maps to FUNCTION1.
:FUNCTION:GOFT:SOURce2 (see page 1550)	:FUNCTION1:SOURce2 (see page 503)	GOFT maps to FUNCTION1.
:FUNCTION:SOURce (see page 1551)	:FUNCTION:SOURce1 (see page 501)	Obsolete command has ADD, SUBTract, and MULTiPLY parameters; current command has GOFT parameter.
:FUNCTION:VIEW (see page 1552)	:FUNCTION:DISPlay (see page 463)	
:HARDcopy:DESTination (see page 1553)	:HARDcopy:FILEname (see page 1554)	

Obsolete Command	Current Command Equivalent	Behavior Differences
:HARDcopy:FILENAME (see page 1554)	:RECall:FILENAME (see page 860) :SAVE:FILENAME (see page 860)	
:HARDcopy:GRAYscale (see page 1555)	:HARDcopy:PALETTE (see page 520)	
:HARDcopy:IGColors (see page 1556)	:HARDcopy:INKSaver (see page 512)	
:HARDcopy:PDRiver (see page 1557)	:HARDcopy:APRinter (see page 509)	
:MEASure:LOWer (see page 1558)	:MEASure:DEFine:THResholds (see page 610)	MEASure:DEFine:THResholds can define absolute values or percentage
:MEASure:SCRatch (see page 1559)	:MEASure:CLEar (see page 607)	
:MEASure:TDELta (see page 1560)	:MARKer:XDELta (see page 570)	
:MEASure:THResholds (see page 1561)	:MEASure:DEFine:THResholds (see page 610)	MEASure:DEFine:THResholds can define absolute values or percentage
:MEASure:TMAX (see page 1562)	:MEASure:XMAX (see page 684)	
:MEASure:TMIN (see page 1563)	:MEASure:XMIN (see page 685)	
:MEASure:TSTArt (see page 1564)	:MARKer:X1Position (see page 565)	
:MEASure:TSTOp (see page 1565)	:MARKer:X2Position (see page 568)	
:MEASure:TVOLt (see page 1566)	:MEASure:TVALue (see page 672)	TVALue measures additional values such as db, Vs, etc.
:MEASure:UPPer (see page 1567)	:MEASure:DEFine:THResholds (see page 610)	MEASure:DEFine:THResholds can define absolute values or percentage
:MEASure:VDELta (see page 1568)	:MARKer:YDELta (see page 577)	
:MEASure:VSTArt (see page 1569)	:MARKer:Y1Position (see page 574)	
:MEASure:VSTOp (see page 1570)	:MARKer:Y2Position (see page 576)	

Obsolete Command	Current Command Equivalent	Behavior Differences
:MEASure:VTime (see page 1571)	:MEASure:YATX (see page 686)	With :MEASure:YATX, there is a command for installing the measurement on the oscilloscope's display. :MEASure:VTime had no command, only a query.
:MTESt:AMASK:{SAVE STORe} (see page 1572)	:SAVE:MASK[:START] (see page 880)	
:MTESt:AVERage (see page 1573)	:ACQuire:TYPE AVERage (see page 300)	
:MTESt:AVERage:COUNT (see page 1574)	:ACQuire:COUNT (see page 286)	
:MTESt:LOAD (see page 1575)	:RECall:MASK[:START] (see page 862)	
:MTESt:RUMode (see page 1576)	:MTESt:RMODE (see page 730)	
:MTESt:RUMode:SOFailure (see page 1577)	:MTESt:RMODE:FACTion:STOP (see page 734)	
:MTESt:{START STOP} (see page 1578)	:RUN (see page 274) or :STOP (see page 278)	
:MTESt:TRIGger:SOURce (see page 1579)	:TRIGger Commands (see page 1345)	There are various commands for setting the source with different types of triggers.
:PRINT? (see page 1580)	:DISPlay:DATA? (see page 402)	
:SAVE:IMAGe:AREA (see page 1582)	none	
:TIMEbase:DELaY (see page 1586)	:TIMEbase:POSition (see page 1334) or :TIMEbase:WINDow:POSition (see page 1341)	TIMEbase:POSition is position value of main time base; TIMEbase:WINDow:POSition is position value of zoomed (delayed) time base window.
:SBUS<n>:LIN:SIGNal:DEFinitio n (see page 1583)	none	
:SBUS<n>:SPI:SOURce:DATA (see page 1584)	:SBUS<n>:SPI:SOURce:MOSI (see page 1129)	
:SYSTem:MENU (see page 1585)	:DISPlay:MENU (see page 410)	No change in behavior.

Obsolete Command	Current Command Equivalent	Behavior Differences
:TRIGger:THReshold (see page 1587)	:POD<n>:THReshold (see page 749) or :DIGital<d>:THReshold (see page 389)	
:TRIGger:TV:TVMODE (see page 1588)	:TRIGger:TV:MODE (see page 1417)	

Discontinued Commands Discontinued commands are commands that were used by previous oscilloscopes, but are not supported by the InfiniiVision 4000G X-Series oscilloscopes. Listed below are the Discontinued commands and the nearest equivalent command available (if any).

Discontinued Command	Current Command Equivalent	Comments
ASToRe	:DISPlay:PERSiStence INFinite (see page 412)	
CHANnel:MATH	:FUNctIon:OPERation (see page 487)	ADD not included
CHANnel<n>:PROTect	:CHANnel<n>:PROTectIon (see page 352)	Previous form of this command was used to enable/disable 50Ω protection. The new command resets a tripped protect and the query returns the status of TRIPed or NORMal.
DISPlay:INVerse	none	
DISPlay:COLumn	none	
DISPlay:FREeze	none	
DISPlay:GRID	none	
DISPlay:LINE	none	
DISPlay:PIXel	none	
DISPlay:POSiTion	none	
DISPlay:ROW	none	
DISPlay:TEXT	none	
FUNctIon:MOVE	none	
FUNctIon:PEAKs	none	
HARDcopy:ADDReSS	none	Only parallel printer port is supported. GPIB printing not supported

Discontinued Command	Current Command Equivalent	Comments
MASK	none	All commands discontinued, feature not available
:POWer:SIGNals:CYCLes	:POWer:SIGNals:CYCLes:HARMonics (see page 823) :POWer:SIGNals:CYCLes:QUALiTy (see page 824)	This command was separated into several other commands for specific types of power analysis.
:POWer:SIGNals:DURation	:POWer:SIGNals:DURation:EFFiCiency (see page 825) :POWer:SIGNals:DURation:MOdulatioN (see page 826) :POWer:SIGNals:DURation:ONOff:OFF (see page 827) :POWer:SIGNals:DURation:ONOff:ON (see page 828) :POWer:SIGNals:DURation:RIPPlE (see page 829) :POWer:SIGNals:DURation:TRANsient (see page 830)	This command was separated into several other commands for specific types of power analysis.
:POWer:SIGNals:VMAXimum	:POWer:SIGNals:VMAXimum:INRush (see page 833) :POWer:SIGNals:VMAXimum:ONOff:OFF (see page 834) :POWer:SIGNals:VMAXimum:ONOff:ON (see page 835)	This command was separated into several other commands for specific types of power analysis.
:POWer:SIGNals:VSTeady	:POWer:SIGNals:VSTeady:ONOff:OFF (see page 836) :POWer:SIGNals:VSTeady:ONOff:ON (see page 837) :POWer:SIGNals:VSTeady:TRANsient (see page 838)	This command was separated into several other commands for specific types of power analysis.
:POWer:SLEW:VALue	none	Slew rate values are now displayed using max and min measurements of a differentiate math function signal.
:PWRenable	none	The Power Event Enable Register does not exist in the 4000G X-Series oscilloscopes.
:PWRRegister[:EVENT]	none	The Power Event Event Register does not exist in the 4000G X-Series oscilloscopes.

Discontinued Command	Current Command Equivalent	Comments
SYSTem:KEY	none	
TEST:ALL	*TST (Self Test) (see page 235)	
TRACE subsystem	none	All commands discontinued, feature not available
TRIGger:ADVanced subsystem		Use new GLITCh, PATtern, or TV trigger modes
TRIGger:TV:FIELD	:TRIGger:TV:MODE (see page 1417)	
TRIGger:TV:TVHFrj		
TRIGger:TV:VIR	none	
:TRIGger:USB:SOURce:DMINus	none	These USB trigger commands are superseded by commands that are available with the USB 2.0 Full/Low Speed or USB 2.0 High Speed serial decode and triggering licenses. See ":SBUS<n>:USB Commands" on page 1158.
:TRIGger:USB:SOURce:DPLus	none	
:TRIGger:USB:SPEed	none	
:TRIGger:USB:TRIGger	none	
VAUToscale	none	

Discontinued Parameters

Some previous oscilloscope queries returned control setting values of OFF and ON. The InfiniiVision 4000G X-Series oscilloscopes only return the enumerated values 0 (for off) and 1 (for on).

:CHANnel:ACTivity

O (see [page 1650](#))

Command Syntax :CHANnel:ACTivity

The :CHANnel:ACTivity command clears the cumulative edge variables for the next activity query.

NOTE

The :CHANnel:ACTivity command is an obsolete command provided for compatibility to previous oscilloscopes. Use the :ACTivity command (see [page 241](#)) instead.

Query Syntax :CHANnel:ACTivity?

The :CHANnel:ACTivity? query returns the active edges since the last clear, and returns the current logic levels.

Return Format <edges>,<levels><NL>

<edges> ::= presence of edges (32-bit integer in NR1 format).

<levels> ::= logical highs or lows (32-bit integer in NR1 format).

NOTE

A bit equal to zero indicates that no edges were detected at the specified threshold since the last clear on that channel. Edges may have occurred that were not detected because of the threshold setting.

A bit equal to one indicates that edges have been detected at the specified threshold since the last clear on that channel.

:CHANnel:LABel

O (see [page 1650](#))

Command Syntax :CHANnel:LABel <source_text><string>
 <source_text> ::= {CHANnel1 | CHANnel2 | DIGital<d>}
 <d> ::= 0 to (# digital channels - 1) in NR1 format
 <string> ::= quoted ASCII string

The :CHANnel:LABel command sets the source text to the string that follows. Setting a channel will also result in the name being added to the label list.

NOTE

The :CHANnel:LABel command is an obsolete command provided for compatibility to previous oscilloscopes. Use the :CHANnel<n>:LABel command (see [page 335](#)) or :DIGital<n>:LABel command (see [page 386](#)).

Query Syntax :CHANnel:LABel?

The :CHANnel:LABel? query returns the label associated with a particular analog channel.

Return Format <string><NL>
 <string> ::= quoted ASCII string

:CHANnel:THReshold

O (see [page 1650](#))

Command Syntax :CHANnel:THReshold <channel group>, <threshold type> [, <value>]

<channel group> ::= {POD1 | POD2}

<threshold type> ::= {CMOS | ECL | TTL | USERdef}

<value> ::= voltage for USERdef in NR3 format [volt_type]

[volt_type] ::= {V | mV (-3) | uV (-6)}

The :CHANnel:THReshold command sets the threshold for a group of channels. The threshold is either set to a predefined value or to a user-defined value. For the predefined value, the voltage parameter is ignored.

NOTE

The :CHANnel:THReshold command is an obsolete command provided for compatibility to previous oscilloscopes. Use the :POD<n>:THReshold command (see [page 749](#)) or :DIGital<n>:THReshold command (see [page 389](#)).

Query Syntax :CHANnel:THReshold? <channel group>

The :CHANnel:THReshold? query returns the voltage and threshold text for a specific group of channels.

Return Format <threshold type> [, <value>]<NL>

<threshold type> ::= {CMOS | ECL | TTL | USERdef}

<value> ::= voltage for USERdef (float 32 NR3)

NOTE

- CMOS = 2.5V
- TTL = 1.5V
- ECL = -1.3V
- USERdef ::= -6.0V to 6.0V

:CHANnel2:SKEW

O (see [page 1650](#))

Command Syntax :CHANnel2:SKEW <skew value>
 <skew value> ::= skew time in NR3 format
 <skew value> ::= -100 ns to +100 ns

The :CHANnel2:SKEW command sets the skew between channels 1 and 2. The maximum skew is +/-100 ns. You can use the oscilloscope's analog probe skew control to remove cable delay errors between channel 1 and channel 2.

NOTE

The :CHANnel2:SKEW command is an obsolete command provided for compatibility to previous oscilloscopes. Use the :CHANnel<n>:PROBe:SKEW command (see [page 349](#)) instead.

NOTE

This command is only valid for the two channel oscilloscope models.

Query Syntax :CHANnel2:SKEW?

The :CHANnel2:SKEW? query returns the current probe skew setting for the selected channel.

Return Format <skew value><NL>
 <skew value> ::= skew value in NR3 format

See Also • ["Introduction to :CHANnel<n> Commands"](#) on page 328

:CHANnel<n>:INPut

O (see [page 1650](#))

Command Syntax :CHANnel<n>:INPut <impedance>

<impedance> ::= {ONEMeg | FIFTy}

<n> ::= 1 to (# analog channels) in NR1 format

The :CHANnel<n>:INPut command selects the input impedance setting for the specified channel. The legal values for this command are ONEMeg (1 M Ω) and FIFTy (50 Ω).

NOTE

The :CHANnel<n>:INPut command is an obsolete command provided for compatibility to previous oscilloscopes. Use the :CHANnel<n>:IMPedance command (see [page 333](#)) instead.

Query Syntax :CHANnel<n>:INPut?

The :CHANnel<n>:INPut? query returns the current input impedance setting for the specified channel.

Return Format <impedance value><NL>

<impedance value> ::= {ONEM | FIFT}

:CHANnel<n>:PMODE

O (see [page 1650](#))

Command Syntax :CHANnel<n>:PMODE <pmode value>

<pmode value> ::= {AUTO | MANual}

<n> ::= 1 to (# analog channels) in NR1 format

The probe sense mode is controlled internally and cannot be set. If a probe with sense is connected to the specified channel, auto sensing is enabled; otherwise, the mode is manual.

If the PMODE sent matches the oscilloscope's setting, the command will be accepted. Otherwise, a setting conflict error is generated.

NOTE

The :CHANnel<n>:PMODE command is an obsolete command provided for compatibility to previous oscilloscopes.

Query Syntax :CHANnel<n>:PMODE?

The :CHANnel<n>:PMODE? query returns AUT if an autosense probe is attached and MAN otherwise.

Return Format <pmode value><NL>

<pmode value> ::= {AUT | MAN}

:DISPlay:CONNect

O (see [page 1650](#))

Command Syntax :DISPlay:CONNect <connect>
 <connect> ::= {{ 1 | ON } | { 0 | OFF }}

The :DISPlay:CONNect command turns vectors on and off. When vectors are turned on, the oscilloscope displays lines connecting sampled data points. When vectors are turned off, only the sampled data is displayed.

NOTE

The :DISPlay:CONNect command is an obsolete command provided for compatibility to previous oscilloscopes. Use the :DISPlay:VECTors command (see [page 416](#)) instead.

Query Syntax :DISPlay:CONNect?

The :DISPlay:CONNect? query returns the current state of the vectors setting.

Return Format <connect><NL>
 <connect> ::= { 1 | 0 }

See Also • [":DISPlay:VECTors"](#) on page 416

:DISPlay:ORDer

O (see [page 1650](#))

Query Syntax :DISPlay:ORDer?

The :DISPlay:ORDer? query returns a list of digital channel numbers in screen order, from top to bottom, separated by commas. Busing is displayed as digital channels with no separator. For example, in the following list, the bus consists of digital channels 4 and 5: DIG1, DIG4 DIG5, DIG7.

NOTE

The :DISPlay:ORDer command is an obsolete command provided for compatibility to previous oscilloscopes. This command is only available on the MSO models.

Return Format <order><NL>

<order> ::= Unquoted ASCII string

NOTE

A return value is included for each digital channel. A return value of NONE indicates that a channel is turned off.

See Also • [":DIGital<d>:POSition"](#) on page 387

Example Code

```
' DISP_ORDER - Set the order the channels are displayed on the
' analyzer. You can enter between 1 and 32 channels at one time.
' If you leave out channels, they will not be displayed.

' Display ONLY channel 0 and channel 10 in that order.
myScope.WriteString ":DISPLAY:ORDER 0,10"
```

See complete example programs at: [Chapter 46](#), "Programming Examples," starting on page 1659

:ERASe

 (see [page 1650](#))

Command Syntax :ERASe

The :ERASe command erases the screen.

NOTE

The :ERASe command is an obsolete command provided for compatibility to previous oscilloscopes. Use the :DISplay:CLEar command (see [page 401](#)) instead.

:EXternal:PMODE

O (see [page 1650](#))

Command Syntax :EXternal:PMODE <pmode value>

<pmode value> ::= {AUTO | MANual}

The probe sense mode is controlled internally and cannot be set. If a probe with sense is connected to the specified channel, auto sensing is enabled; otherwise, the mode is manual.

If the pmode sent matches the oscilloscope's setting, the command will be accepted. Otherwise, a setting conflict error is generated.

NOTE

The :EXternal:PMODE command is an obsolete command provided for compatibility to previous oscilloscopes.

Query Syntax :EXternal:PMODE?

The :EXternal:PMODE? query returns AUT if an autosense probe is attached and MAN otherwise.

Return Format <pmode value><NL>

<pmode value> ::= {AUT | MAN}

:FUNction:GOFT:OPERation

O (see [page 1650](#))

Command Syntax :FUNction:GOFT:OPERation <operation>

<operation> ::= {ADD | SUBTract | MULTiply}

The :FUNction:GOFT:OPERation command sets the math operation for the g(t) source that can be used as the input to transform or filter functions (if available):

- ADD — Source1 + source2.
- SUBTract — Source1 - source2.
- MULTiply — Source1 * source2.

The :FUNction:GOFT:SOURce1 and :FUNction:GOFT:SOURce2 commands are used to select source1 and source2.

:FUNction:GOFT:SOURce1

O (see [page 1650](#))

Command Syntax :FUNction:GOFT:SOURce1 <value>

<value> ::= CHANnel<n>

<n> ::= {1 | 2 | 3 | 4} for 4ch models

<n> ::= {1 | 2} for 2ch models

The :FUNction:GOFT:SOURce1 command selects the first input channel for the g(t) source that can be used as the input to transform or filter functions (if available).

Query Syntax :FUNction:GOFT:SOURce1?

The :FUNction:GOFT:SOURce1? query returns the current selection for the first input channel for the g(t) source.

Return Format <value><NL>

<value> ::= CHAN<n>

<n> ::= {1 | 2 | 3 | 4} for the 4ch models

<n> ::= {1 | 2} for the 2ch models

- See Also**
- ["Introduction to :FUNction<m> Commands"](#) on page 455
 - [":FUNction:GOFT:SOURce2"](#) on page 1550
 - [":FUNction:GOFT:OPERation"](#) on page 1548

:FUNction:GOFT:SOURce2

O (see [page 1650](#))

Command Syntax :FUNction:GOFT:SOURce2 <value>

<value> ::= CHANnel<n>

<n> ::= {1 | 2 | 3 | 4} for 4ch models

<n> ::= {1 | 2} for 2ch models

The :FUNction:GOFT:SOURce2 command selects the second input channel for the g(t) source that can be used as the input to transform or filter functions (if available).

Query Syntax :FUNction:GOFT:SOURce2?

The :FUNction:GOFT:SOURce2? query returns the current selection for the second input channel for the g(t) source.

Return Format <value><NL>

<value> ::= CHAN<n>

<n> ::= {1 | 2 | 3 | 4} for 4ch models

<n> ::= {1 | 2} for 2ch models

- See Also**
- ["Introduction to :FUNction<m> Commands"](#) on page 455
 - [":FUNction:GOFT:SOURce1"](#) on page 1549
 - [":FUNction:GOFT:OPERation"](#) on page 1548

:FUNCTION:SOURce

O (see [page 1650](#))

Command Syntax :FUNCTION:SOURce <value>

<value> ::= {CHANnel<n> | ADD | SUBTract | MULTiply}

<n> ::= 1 to (# analog channels) in NR1 format

The :FUNCTION:SOURce command is only used when an FFT (Fast Fourier Transform), DIFF, or INT operation is selected (see the :FUNCTION:OPERation command for more information about selecting an operation). The :FUNCTION:SOURce command selects the source for function operations. Choose CHANnel<n>, or ADD, SUBT, or MULT to specify the desired source for function DIFF (differentiate), INTegrate, and FFT operations specified by the :FUNCTION:OPERation command.

NOTE

The :FUNCTION:SOURce command is an obsolete command provided for compatibility to previous oscilloscopes. Use the :FUNCTION:SOURce1 command (see [page 501](#)) instead.

Query Syntax :FUNCTION:SOURce?

The :FUNCTION:SOURce? query returns the current source for function operations.

Return Format <value><NL>

<value> ::= {CHAN<n> | ADD | SUBT | MULT}

<n> ::= 1 to (# analog channels) in NR1 format

- See Also**
- ["Introduction to :FUNCTION<m> Commands"](#) on page 455
 - [":FUNCTION<m>:OPERation"](#) on page 487

:FUNCTION:VIEW

O (see [page 1650](#))

Command Syntax :FUNCTION:VIEW <view>
 <view> ::= {{1 | ON} | (0 | OFF)}

The :FUNCTION:VIEW command turns the selected function on or off. When ON is selected, the function performs as specified using the other FUNCTION commands. When OFF is selected, function is neither calculated nor displayed.

NOTE

The :FUNCTION:VIEW command is provided for backward compatibility to previous oscilloscopes. Use the :FUNCTION:DISPlay command (see [page 463](#)) instead.

Query Syntax :FUNCTION:VIEW?

The :FUNCTION:VIEW? query returns the current state of the selected function.

Return Format <view><NL>
 <view> ::= {1 | 0}

:HARDcopy:DESTination

O (see [page 1650](#))

Command Syntax :HARDcopy:DESTination <destination>
 <destination> ::= {CENTronics | FLOppy}

The :HARDcopy:DESTination command sets the hardcopy destination.

NOTE

The :HARDcopy:DESTination command is an obsolete command provided for compatibility to previous oscilloscopes. Use the :HARDcopy:FILENAME command (see [page 1554](#)) instead.

Query Syntax :HARDcopy:DESTination?

The :HARDcopy:DESTination? query returns the selected hardcopy destination.

Return Format <destination><NL>
 <destination> ::= {CENT | FLOP}

See Also • ["Introduction to :HARDcopy Commands"](#) on page 506

:HARDcopy:FILENAME

O (see [page 1650](#))

Command Syntax :HARDcopy:FILENAME <string>
 <string> ::= quoted ASCII string

The :HARDcopy:FILENAME command sets the output filename for those print formats whose output is a file.

NOTE

The :HARDcopy:FILENAME command is an obsolete command provided for compatibility to previous oscilloscopes. Use the :SAVE:FILENAME command (see [page 873](#)) and :RECall:FILENAME command (see [page 860](#)) instead.

Query Syntax :HARDcopy:FILENAME?

The :HARDcopy:FILENAME? query returns the current hardcopy output filename.

Return Format <string><NL>
 <string> ::= quoted ASCII string

See Also • ["Introduction to :HARDcopy Commands"](#) on page 506

:HARDcopy:GRAYscale

O (see [page 1650](#))

Command Syntax :HARDcopy:GRAYscale <gray>
 <gray> ::= {{OFF | 0} | {ON | 1}}

The :HARDcopy:GRAYscale command controls whether grayscaling is performed in the hardcopy dump.

NOTE

The :HARDcopy:GRAYscale command is an obsolete command provided for compatibility to previous oscilloscopes. Use the :HARDcopy:PALETTE command (see [page 520](#)) instead. (":HARDcopy:GRAYscale ON" is the same as ":HARDcopy:PALETTE GRAYscale" and ":HARDcopy:GRAYscale OFF" is the same as ":HARDcopy:PALETTE COLOR".)

Query Syntax :HARDcopy:GRAYscale?

The :HARDcopy:GRAYscale? query returns a flag indicating whether grayscaling is performed in the hardcopy dump.

Return Format <gray><NL>
 <gray> ::= {0 | 1}

See Also • ["Introduction to :HARDcopy Commands"](#) on page 506

:HARDcopy:IGColors

O (see [page 1650](#))

Command Syntax :HARDcopy:IGColors <value>
 <value> ::= {{OFF | 0} | {ON | 1}}

The :HARDcopy:IGColors command controls whether the graticule colors are inverted or not.

NOTE

The :HARDcopy:IGColors command is an obsolete command provided for compatibility to previous oscilloscopes. Use the :HARDcopy:INKSaver (see [page 512](#)) command instead.

Query Syntax :HARDcopy:IGColors?

The :HARDcopy:IGColors? query returns a flag indicating whether graticule colors are inverted or not.

Return Format <value><NL>
 <value> ::= {0 | 1}

See Also • ["Introduction to :HARDcopy Commands"](#) on page 506

:HARDcopy:PDRiver

O (see [page 1650](#))

Command Syntax :HARDcopy:PDRiver <driver>

```
<driver> ::= {AP2Xxx | AP21xx | {AP2560 | AP25} | {DJ350 | DJ35} |
             DJ6xx | {DJ630 | DJ63} | DJ6Special | DJ6Photo |
             DJ8Special | DJ8xx | DJ9Vip | OJPRokx50 | DJ9xx | GVIP |
             DJ55xx | {PS470 | PS47} {PS100 | PS10} | CLASer |
             MLASer | LJFastraster | POSTscript}
```

The :HARDcopy:PDRiver command sets the hardcopy printer driver used for the selected printer.

If the correct driver for the selected printer can be identified, it will be selected and cannot be changed.

NOTE

The :HARDcopy:PDRiver command is an obsolete command provided for compatibility to previous oscilloscopes. Use the :HARDcopy:APRinter (see [page 509](#)) command instead.

Query Syntax :HARDcopy:PDRiver?

The :HARDcopy:PDRiver? query returns the selected hardcopy printer driver.

Return Format <driver><NL>

```
<driver> ::= {AP2X | AP21 | AP25 | DJ35 | DJ6 | DJ63 | DJ6S | DJ6P |
             DJ8S | DJ8 | DJ9V | OJPR | DJ9 | GVIP | DJ55 | PS10 |
             PS47 | CLAS | MLAS | LJF | POST}
```

See Also • ["Introduction to :HARDcopy Commands"](#) on page 506

:MEASure:LOWer

O (see [page 1650](#))

Command Syntax :MEASure:LOWer <voltage>

The :MEASure:LOWer command sets the lower measurement threshold value. This value and the UPPer value represent absolute values when the thresholds are ABSolute and percentage when the thresholds are PERCent as defined by the :MEASure:DEFine THResholds command.

NOTE

The :MEASure:LOWer command is obsolete and is provided for backward compatibility to previous oscilloscopes. Use the :MEASure:DEFine THResholds command (see [page 610](#)) instead.

Query Syntax :MEASure:LOWer?

The :MEASure:LOWer? query returns the current lower threshold level.

Return Format <voltage><NL>

<voltage> ::= the user-defined lower threshold in volts in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:THResholds"](#) on page 1561
 - [":MEASure:UPPer"](#) on page 1567

:MEASure:SCRatch

O (see [page 1650](#))

Command Syntax :MEASure:SCRatch

The :MEASure:SCRatch command clears all selected measurements and markers from the screen.

NOTE

The :MEASure:SCRatch command is obsolete and is provided for backward compatibility to previous oscilloscopes. Use the :MEASure:CLEar command (see [page 607](#)) instead.

:MEASure:TDELta

O (see [page 1650](#))

Query Syntax :MEASure:TDELta?

The :MEASure:TDELta? query returns the time difference between the Tstop marker (X2 cursor) and the Tstart marker (X1 cursor).

$T_{\text{delta}} = T_{\text{stop}} - T_{\text{start}}$

Tstart is the time at the start marker (X1 cursor) and Tstop is the time at the stop marker (X2 cursor). No measurement is made when the :MEASure:TDELta? query is received by the oscilloscope. The delta time value that is output is the current value. This is the same value as the front-panel cursors delta X value.

NOTE

The :MEASure:TDELta command is an obsolete command provided for compatibility to previous oscilloscopes. Use the :MARKer:XDELta command (see [page 570](#)) instead.

Return Format <value><NL>

<value> ::= time difference between start and stop markers in NR3 format

- See Also**
- ["Introduction to :MARKer Commands"](#) on page 561
 - ["Introduction to :MEASure Commands"](#) on page 601
 - [":MARKer:X1Position"](#) on page 565
 - [":MARKer:X2Position"](#) on page 568
 - [":MARKer:XDELta"](#) on page 570
 - [":MEASure:TSTArt"](#) on page 1564
 - [":MEASure:TSTOp"](#) on page 1565

:MEASure:THResholds

O (see [page 1650](#))

Command Syntax :MEASure:THResholds {T1090 | T2080 | VOLTage}

The :MEASure:THResholds command selects the thresholds used when making time measurements.

NOTE

The :MEASure:THResholds command is obsolete and is provided for backward compatibility to previous oscilloscopes. Use the :MEASure:DEFine THResholds command (see [page 610](#)) instead.

Query Syntax :MEASure:THResholds?

The :MEASure:THResholds? query returns the current thresholds selected when making time measurements.

Return Format {T1090 | T2080 | VOLTage}<NL>

{T1090} uses the 10% and 90% levels of the selected waveform.

{T2080} uses the 20% and 80% levels of the selected waveform.

{VOLTage} uses the upper and lower voltage thresholds set by the UPPer and LOWer commands on the selected waveform.

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:LOWer"](#) on page 1558
 - [":MEASure:UPPer"](#) on page 1567

:MEASure:TMAX

O (see [page 1650](#))

Command Syntax :MEASure:TMAX [<source>]

<source> ::= {CHANnel<n> | FUNCTION | MATH}

<n> ::= 1 to (# analog channels) in NR1 format

The :MEASure:TMAX command installs a screen measurement and starts an X-at-Max-Y measurement on the selected waveform. If the optional source is specified, the current source is modified.

NOTE

The :MEASure:TMAX command is obsolete and is provided for backward compatibility to previous oscilloscopes. Use the :MEASure:XMAX command (see [page 684](#)) instead.

Query Syntax :MEASure:TMAX? [<source>]

The :MEASure:TMAX? query returns the horizontal axis value at which the maximum vertical value occurs on the current source. If the optional source is specified, the current source is modified.

Return Format <value><NL>

<value> ::= time at maximum in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:TMIN"](#) on page 1563
 - [":MEASure:XMAX"](#) on page 684
 - [":MEASure:XMIN"](#) on page 685

:MEASure:TMIN

O (see [page 1650](#))

Command Syntax :MEASure:TMIN [<source>]

<source> ::= {CHANnel<n> | FUNCTION | MATH}

<n> ::= 1 to (# analog channels) in NR1 format

The :MEASure:TMIN command installs a screen measurement and starts an X-at-Min-Y measurement on the selected waveform. If the optional source is specified, the current source is modified.

NOTE

The :MEASure:TMIN command is obsolete and is provided for backward compatibility to previous oscilloscopes. Use the :MEASure:XMIN command (see [page 685](#)) instead.

Query Syntax :MEASure:TMIN? [<source>]

The :MEASure:TMIN? query returns the horizontal axis value at which the minimum vertical value occurs on the current source. If the optional source is specified, the current source is modified.

Return Format <value><NL>

<value> ::= time at minimum in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:TMAX"](#) on page 1562
 - [":MEASure:XMAX"](#) on page 684
 - [":MEASure:XMIN"](#) on page 685

:MEASure:TSTArt

O (see [page 1650](#))

Command Syntax :MEASure:TSTArt <value> [suffix]

<value> ::= time at the start marker in seconds

[suffix] ::= {s | ms | us | ns | ps}

The :MEASure:TSTArt command moves the start marker (X1 cursor) to the specified time with respect to the trigger time.

NOTE

The short form of this command, TSTA, does not follow the defined Long Form to Short Form Truncation Rules (see [page 1652](#)). The normal short form "TST" would be the same for both TSTArt and TSTOp, so sending TST for the TSTArt command produces an error.

NOTE

The :MEASure:TSTArt command is an obsolete command provided for compatibility to previous oscilloscopes. Use the :MARKer:X1Position command (see [page 565](#)) instead.

Query Syntax :MEASure:TSTArt?

The :MEASure:TSTArt? query returns the time at the start marker (X1 cursor).

Return Format <value><NL>

<value> ::= time at the start marker in NR3 format

- See Also**
- ["Introduction to :MARKer Commands"](#) on page 561
 - ["Introduction to :MEASure Commands"](#) on page 601
 - [":MARKer:X1Position"](#) on page 565
 - [":MARKer:X2Position"](#) on page 568
 - [":MARKer:XDELta"](#) on page 570
 - [":MEASure:TDELta"](#) on page 1560
 - [":MEASure:TSTOp"](#) on page 1565

:MEASure:TSTOp

O (see [page 1650](#))

Command Syntax :MEASure:TSTOp <value> [suffix]

<value> ::= time at the stop marker in seconds

[suffix] ::= {s | ms | us | ns | ps}

The :MEASure:TSTOp command moves the stop marker (X2 cursor) to the specified time with respect to the trigger time.

NOTE

The short form of this command, TSTO, does not follow the defined Long Form to Short Form Truncation Rules (see [page 1652](#)). The normal short form "TST" would be the same for both TSTArt and TSTOp, so sending TST for the TSTOp command produces an error.

NOTE

The :MEASure:TSTOp command is an obsolete command provided for compatibility to previous oscilloscopes. Use the :MARKer:X2Position command (see [page 568](#)) instead.

Query Syntax :MEASure:TSTOp?

The :MEASure:TSTOp? query returns the time at the stop marker (X2 cursor).

Return Format <value><NL>

<value> ::= time at the stop marker in NR3 format

- See Also**
- ["Introduction to :MARKer Commands"](#) on page 561
 - ["Introduction to :MEASure Commands"](#) on page 601
 - [":MARKer:X1Position"](#) on page 565
 - [":MARKer:X2Position"](#) on page 568
 - [":MARKer:XDELta"](#) on page 570
 - [":MEASure:TDELta"](#) on page 1560
 - [":MEASure:TSTArt"](#) on page 1564

:MEASure:TVOLt

O (see [page 1650](#))

Query Syntax :MEASure:TVOLt? <value>, [<slope>]<occurrence>[,<source>]

<value> ::= the voltage level that the waveform must cross.

<slope> ::= direction of the waveform. A rising slope is indicated by a plus sign (+). A falling edge is indicated by a minus sign (-).

<occurrence> ::= the transition to be reported. If the occurrence number is one, the first crossing is reported. If the number is two, the second crossing is reported, etc.

<source> ::= {<digital channels> | CHANnel<n> | FUNction | MATH}

<digital channels> ::= {DIGital<d>} for the MSO models

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

When the :MEASure:TVOLt? query is sent, the displayed signal is searched for the specified voltage level and transition. The time interval between the trigger event and this defined occurrence is returned as the response to the query.

The specified voltage can be negative or positive. To specify a negative voltage, use a minus sign (-). The sign of the slope selects a rising (+) or falling (-) edge. If no sign is specified for the slope, it is assumed to be the rising edge.

The magnitude of the occurrence defines the occurrence to be reported. For example, +3 returns the time for the third time the waveform crosses the specified voltage level in the positive direction. Once this voltage crossing is found, the oscilloscope reports the time at that crossing in seconds, with the trigger point (time zero) as the reference.

If the specified crossing cannot be found, the oscilloscope reports +9.9E+37. This value is returned if the waveform does not cross the specified voltage, or if the waveform does not cross the specified voltage for the specified number of times in the direction specified.

If the optional source parameter is specified, the current source is modified.

NOTE

The :MEASure:TVOLt command is obsolete and is provided for backward compatibility to previous oscilloscopes. Use the :MEASure:TVALue command (see [page 672](#)).

Return Format <value><NL>

<value> ::= time in seconds of the specified voltage crossing
in NR3 format

:MEASure:UPPer

O (see [page 1650](#))

Command Syntax :MEASure:UPPer <value>

The :MEASure:UPPer command sets the upper measurement threshold value. This value and the LOWer value represent absolute values when the thresholds are ABSolute and percentage when the thresholds are PERCent as defined by the :MEASure:DEFine THResholds command.

NOTE

The :MEASure:UPPer command is obsolete and is provided for backward compatibility to previous oscilloscopes. Use the :MEASure:DEFine THResholds command (see [page 610](#)) instead.

Query Syntax :MEASure:UPPer?

The :MEASure:UPPer? query returns the current upper threshold level.

Return Format <value><NL>

<value> ::= the user-defined upper threshold in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:LOWer"](#) on page 1558
 - [":MEASure:THResholds"](#) on page 1561

:MEASure:VDELta

O (see [page 1650](#))

Query Syntax :MEASure:VDELta?

The :MEASure:VDELta? query returns the voltage difference between vertical marker 1 (Y1 cursor) and vertical marker 2 (Y2 cursor). No measurement is made when the :MEASure:VDELta? query is received by the oscilloscope. The delta value that is returned is the current value. This is the same value as the front-panel cursors delta Y value.

VDELta = value at marker 2 - value at marker 1

NOTE

The :MEASure:VDELta command is an obsolete command provided for compatibility to previous oscilloscopes. Use the :MARKer:YDELta command (see [page 577](#)) instead.

Return Format <value><NL>

<value> ::= delta V value in NR1 format

- See Also**
- ["Introduction to :MARKer Commands"](#) on page 561
 - ["Introduction to :MEASure Commands"](#) on page 601
 - [":MARKer:Y1Position"](#) on page 574
 - [":MARKer:Y2Position"](#) on page 576
 - [":MARKer:YDELta"](#) on page 577
 - [":MEASure:TDELta"](#) on page 1560
 - [":MEASure:TSTArt"](#) on page 1564

:MEASure:VSTArt

O (see [page 1650](#))

Command Syntax :MEASure:VSTArt <vstart_argument>

<vstart_argument> ::= value for vertical marker 1

The :MEASure:VSTArt command moves the vertical marker (Y1 cursor) to the specified value corresponding to the selected source. The source can be selected by the MARKer:X1Y1source command.

NOTE

The short form of this command, VSTA, does not follow the defined Long Form to Short Form Truncation Rules (see [page 1652](#)). The normal short form, VST, would be the same for both VSTArt and VSTOp, so sending VST for the VSTArt command produces an error.

NOTE

The :MEASure:VSTArt command is an obsolete command provided for compatibility to previous oscilloscopes. Use the :MARKer:Y1Position command (see [page 574](#)) instead.

Query Syntax :MEASure:VSTArt?

The :MEASure:VSTArt? query returns the current value of the Y1 cursor.

Return Format <value><NL>

<value> ::= voltage at voltage marker 1 in NR3 format

- See Also**
- ["Introduction to :MARKer Commands"](#) on page 561
 - ["Introduction to :MEASure Commands"](#) on page 601
 - [":MARKer:Y1Position"](#) on page 574
 - [":MARKer:Y2Position"](#) on page 576
 - [":MARKer:YDELta"](#) on page 577
 - [":MARKer:X1Y1source"](#) on page 566
 - [":MEASure:SOURce"](#) on page 661
 - [":MEASure:TDELta"](#) on page 1560
 - [":MEASure:TSTArt"](#) on page 1564

:MEASure:VSTOp

O (see [page 1650](#))

Command Syntax :MEASure:VSTOp <vstop_argument>
 <vstop_argument> ::= value for Y2 cursor

The :MEASure:VSTOp command moves the vertical marker 2 (Y2 cursor) to the specified value corresponding to the selected source. The source can be selected by the MARKer:X2Y2source command.

NOTE

The short form of this command, VSTO, does not follow the defined Long Form to Short Form Truncation Rules (see [page 1652](#)). The normal short form, VST, would be the same for both VSTArt and VSTOp, so sending VST for the VSTOp command produces an error.

NOTE

The :MEASure:VSTOp command is an obsolete command provided for compatibility to previous oscilloscopes. Use the :MARKer:Y2Position command (see [page 576](#)) instead.

Query Syntax :MEASure:VSTOp?

The :MEASure:VSTOp? query returns the current value of the Y2 cursor.

Return Format <value><NL>
 <value> ::= value of the Y2 cursor in NR3 format

- See Also**
- ["Introduction to :MARKer Commands"](#) on page 561
 - ["Introduction to :MEASure Commands"](#) on page 601
 - [":MARKer:Y1Position"](#) on page 574
 - [":MARKer:Y2Position"](#) on page 576
 - [":MARKer:YDELta"](#) on page 577
 - [":MARKer:X2Y2source"](#) on page 569
 - [":MEASure:SOURce"](#) on page 661
 - [":MEASure:TDELta"](#) on page 1560
 - [":MEASure:TSTArt"](#) on page 1564

:MEASure:VTIME

O (see [page 1650](#))

Query Syntax :MEASure:VTIME? <vtime_argument>[,<source>]

<vtime_argument> ::= time from trigger in seconds

<source> ::= {<digital channels> | CHANnel<n> | FUNction<m> | MATH<m>
| WMEMory<r>}

<digital channels> ::= DIGital<d> for the MSO models

<d> ::= 0 to (# digital channels - 1) in NR1 format

<n> ::= 1 to (# of analog channels) in NR1 format

<m> ::= 1 to (# math functions) in NR1 format

<r> ::= 1 to (# ref waveforms) in NR1 format

The :MEASure:VTIME? query returns the vertical value at a specified horizontal value on the source specified (see also :MEASure:SOURce). The specified horizontal value must be on the screen; when it is a time value, it is referenced to the trigger event. If the optional source parameter is specified, the measurement source is modified.

NOTE

When the source is an FFT (Fast Fourier Transform) waveform, the <vtime_argument> is a frequency value instead of a time value.

Return Format <value><NL>

<value> ::= vertical value at the specified horizontal location
in NR3 format

- See Also**
- ["Introduction to :MEASure Commands"](#) on page 601
 - [":MEASure:SOURce"](#) on page 661
 - [":MEASure:TEDGe"](#) on page 669
 - [":MEASure:TVALue"](#) on page 672

:MTESt:AMASk:{SAVE | STORe}

O (see [page 1650](#))

Command Syntax :MTESt:AMASk:{SAVE | STORe} "<filename>"

The :MTESt:AMASk:SAVE command saves the automask generated mask to a file. If an automask has not been generated, an error occurs.

The <filename> parameter is an MS-DOS compatible name of the file, a maximum of 254 characters long (including the path name, if used). The filename assumes the present working directory if a path does not precede the file name.

NOTE

The :MTESt:AMASk:{SAVE | STORe} command is obsolete and is provided for backward compatibility to previous oscilloscopes. Use the :SAVE:MASK[:START] command (see [page 880](#)) instead.

See Also • ["Introduction to :MTESt Commands"](#) on page 713

:MTESt:AVERage

O (see [page 1650](#))

Command Syntax :MTESt:AVERage <on_off>
 <on_off> ::= {{1 | ON} | {0 | OFF}}

The :MTESt:AVERage command enables or disables averaging. When ON, the oscilloscope acquires multiple data values for each time bucket, and averages them. When OFF, averaging is disabled. To set the number of averages, use the :MTESt:AVERage:COUNT command described next.

NOTE

The :MTESt:AVERage command is obsolete and is provided for backward compatibility to previous oscilloscopes. Use the :ACQuire:TYPE AVERage command (see [page 300](#)) instead.

Query Syntax :MTESt:AVERage?

The :MTESt:AVERage? query returns the current setting for averaging.

Return Format <on_off><NL>
 <on_off> ::= {1 | 0}

- See Also**
- ["Introduction to :MTESt Commands"](#) on page 713
 - [":MTESt:AVERage:COUNT"](#) on page 1574

:MTESt:AVERAge:COUnT

O (see [page 1650](#))

Command Syntax :MTESt:AVERAge:COUnT <count>
<count> ::= an integer from 2 to 65536 in NR1 format

The :MTESt:AVERAge:COUnT command sets the number of averages for the waveforms. With the AVERAge acquisition type, the :MTESt:AVERAge:COUnT command specifies the number of data values to be averaged for each time bucket before the acquisition is considered complete for that time bucket.

NOTE

The :MTESt:AVERAge:COUnT command is obsolete and is provided for backward compatibility to previous oscilloscopes. Use the :ACQuire:COUnT command (see [page 286](#)) instead.

Query Syntax :MTESt:AVERAge:COUnT?
The :MTESt:AVERAge:COUnT? query returns the currently selected count value.

Return Format <count><NL>
<count> ::= an integer from 2 to 65536 in NR1 format

- See Also**
- ["Introduction to :MTESt Commands"](#) on page 713
 - [":MTESt:AVERAge"](#) on page 1573

:MTEST:LOAD

O (see [page 1650](#))

Command Syntax :MTEST:LOAD "<filename>"

The :MTEST:LOAD command loads the specified mask file.

The <filename> parameter is an MS-DOS compatible name of the file, a maximum of 254 characters long (including the path name, if used).

NOTE

The :MTEST:LOAD command is obsolete and is provided for backward compatibility to previous oscilloscopes. Use the :RECall:MASK[:START] command (see [page 862](#)) instead.

- See Also**
- ["Introduction to :MTEST Commands"](#) on page 713
 - [":MTEST:AMASK:{SAVE | STORE}"](#) on page 1572

:MTEST:RUMode

O (see [page 1650](#))

Command Syntax :MTEST:RUMode {FOREver | TIME,<seconds> | {WAVEforms,<wfm_count>}}

<seconds> ::= from 1 to 86400 in NR3 format

<wfm_count> ::= number of waveforms in NR1 format
from 1 to 1,000,000,000

The :MTEST:RUMode command determines the termination conditions for the mask test. The choices are FOREver, TIME, or WAVEforms.

- FOREver – runs the Mask Test until the test is turned off.
- TIME – sets the amount of time in seconds that a mask test will run before it terminates. The <seconds> parameter is a real number from 1 to 86400 seconds.
- WAVEforms – sets the maximum number of waveforms that are required before the mask test terminates. The <wfm_count> parameter indicates the number of waveforms that are to be acquired; it is an integer from 1 to 1,000,000,000.

NOTE

The :MTEST:RUMode command is obsolete and is provided for backward compatibility to previous oscilloscopes. Use the :MTEST:RMODE command (see [page 730](#)) instead.

Query Syntax :MTEST:RUMode?

The :MTEST:RUMode? query returns the currently selected termination condition and value.

Return Format {FOR | TIME,<seconds> | {WAV,<wfm_count>}}<NL>

<seconds> ::= from 1 to 86400 in NR3 format

<wfm_count> ::= number of waveforms in NR1 format
from 1 to 1,000,000,000

- See Also**
- ["Introduction to :MTEST Commands"](#) on page 713
 - [":MTEST:RUMode:SOFailure"](#) on page 1577

:MTESt:RUMode:SOFailure

O (see [page 1650](#))

Command Syntax :MTESt:RUMode:SOFailure <on_off>
 <on_off> ::= {{1 | ON} | {0 | OFF}}

The :MTESt:RUMode:SOFailure command enables or disables the Stop On Failure run until criteria. When a mask test is run and a mask violation is detected, the mask test is stopped and the acquisition system is stopped.

NOTE

The :MTESt:RUMode:SOFailure command is obsolete and is provided for backward compatibility to previous oscilloscopes. Use the :MTESt:RMODE:FACTION:STOP command (see [page 734](#)) instead.

Query Syntax :MTESt:RUMode:SOFailure?

The :MTESt:RUMode:SOFailure? query returns the current state of the Stop on Failure control.

Return Format <on_off><NL>
 <on_off> ::= {1 | 0}

- See Also**
- ["Introduction to :MTESt Commands"](#) on page 713
 - [":MTESt:RUMode"](#) on page 1576

:MTEST:{START | STOP}

O (see [page 1650](#))

Command Syntax :MTEST:{START | STOP}

The :MTEST:{START | STOP} command starts or stops the acquisition system.

NOTE

The :MTEST:START and :MTEST:STOP commands are obsolete and are provided for backward compatibility to previous oscilloscopes. Use the :RUN command (see [page 274](#)) and :STOP command (see [page 278](#)) instead.

See Also • ["Introduction to :MTEST Commands"](#) on page 713

:MTESt:TRIGger:SOURce

O (see [page 1650](#))

Command Syntax :MTESt:TRIGger:SOURce <source>

<source> ::= CHANnel<n>

<n> ::= 1 to (# analog channels) in NR1 format

The :MTESt:TRIGger:SOURce command sets the channel to use as the trigger.

NOTE

The :MTESt:TRIGger:SOURce command is obsolete and is provided for backward compatibility to previous oscilloscopes. Use the trigger source commands (see [page 1345](#)) instead.

Query Syntax :MTESt:TRIGger:SOURce?

The :MTESt:TRIGger:SOURce? query returns the currently selected trigger source.

Return Format <source> ::= CHAN<n>

<n> ::= 1 to (# analog channels) in NR1 format

See Also • ["Introduction to :MTESt Commands"](#) on page 713

:PRINT?

O (see [page 1650](#))

Query Syntax :PRINT? [<options>]

<options> ::= [<print option>][,...,<print option>]

<print option> ::= {COLor | GRAYscale | BMP8bit | BMP}

The :PRINT? query pulls image data back over the bus for storage.

NOTE

The :PRINT command is an obsolete command provided for compatibility to previous oscilloscopes. Use the :DISPlay:DATA command (see [page 402](#)) instead.

Print Option	:PRINt command	:PRINt? query	Query Default
COLor	Sets palette=COLor		
GRAYscale	Sets palette=GRAYscale		palette=COLor
PRINter0,1	Causes the USB printer #0,1 to be selected as destination (if connected)	Not used	N/A
BMP8bit	Sets print format to 8-bit BMP	Selects 8-bit BMP formatting for query	N/A
BMP	Sets print format to BMP	Selects BMP formatting for query	N/A
FACTors	Selects outputting of additional settings information for :PRINT	Not used	N/A
NOFactors	Deselects outputting of additional settings information for :PRINT	Not used	N/A

Old Print Option:	Is Now:
HIRes	COLor
LORes	GRAYscale
PARallel	PRINter0
DISK	invalid
PCL	invalid

NOTE

The PRINT? query is not a core command.

-
- See Also**
- ["Introduction to Root \(:\) Commands"](#) on page 240
 - ["Introduction to :HARDcopy Commands"](#) on page 506
 - [":HARDcopy:FACTors"](#) on page 510
 - [":HARDcopy:GRAYscale"](#) on page 1555
 - [":DISPlay:DATA"](#) on page 402

:SAVE:IMAGe:AREA

O (see [page 1650](#))

Query Syntax :SAVE:IMAGe:AREA?

The :SAVE:IMAGe:AREA? query returns the selected image area.

When saving images, this query returns SCR (screen). When saving setups or waveform data, this query returns GRAT (graticule) even though graticule images are not saved.

Return Format <area><NL>

<area> ::= {GRAT | SCR}

- See Also**
- ["Introduction to :SAVE Commands"](#) on page 870
 - [":SAVE:IMAGe\[:START\]"](#) on page 874
 - [":SAVE:IMAGe:FACTors"](#) on page 875
 - [":SAVE:IMAGe:FORMat"](#) on page 876
 - [":SAVE:IMAGe:INKSaver"](#) on page 877
 - [":SAVE:IMAGe:PALette"](#) on page 878

:SBUS<n>:LIN:SIGNal:DEFinition

O (see [page 1650](#))

Command Syntax :SBUS<n>:LIN:SIGNal:DEFinition <value>

<value> ::= {LIN | RX | TX}

The :SBUS<n>:LIN:SIGNal:DEFinition command sets the LIN signal type. These signals can be set to:

Dominant low signals:

- LIN – the actual LIN single-end bus signal line.
- RX – the Receive signal from the LIN bus transceiver.
- TX – the Transmit signal to the LIN bus transceiver.

NOTE

This command is available, but the only legal value is LIN.

Query Syntax :SBUS<n>:LIN:SIGNal:DEFinition?

The :SBUS<n>:LIN:SIGNal:DEFinition? query returns the current LIN signal type.

Return Format <value><NL>

<value> ::= LIN

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":TRIGger:MODE"](#) on page 1358
 - [":SBUS<n>:LIN:SIGNal:BAUDrate"](#) on page 1024
 - [":SBUS<n>:LIN:SOURce"](#) on page 1025

:SBUS<n>:SPI:SOURce:DATA

O (see [page 1650](#))

Command Syntax :SBUS<n>:SPI:SOURce:DATA <source>

<source> ::= {CHANnel<n> | EXternal} for the DSO models

<source> ::= {CHANnel<n> | DIGital<d>} for the MSO models

<n> ::= 1 to (# analog channels) in NR1 format

<d> ::= 0 to (# digital channels - 1) in NR1 format

The :SBUS<n>:SPI:SOURce:DATA command sets the source for the SPI serial MOSI data.

This command is the same as the :SBUS<n>:SPI:SOURce:MOSI command.

Query Syntax :SBUS<n>:SPI:SOURce:DATA?

The :SBUS<n>:SPI:SOURce:DATA? query returns the current source for the SPI serial MOSI data.

Return Format <source><NL>

- See Also**
- ["Introduction to :TRIGger Commands"](#) on page 1345
 - [":SBUS<n>:SPI:SOURce:MOSI"](#) on page 1129
 - [":SBUS<n>:SPI:SOURce:MISO"](#) on page 1128
 - [":SBUS<n>:SPI:SOURce:CLOCK"](#) on page 1126
 - [":SBUS<n>:SPI:SOURce:FRAME"](#) on page 1127
 - [":SBUS<n>:SPI:TRIGger:PATtern:MISO:DATA"](#) on page 1130
 - [":SBUS<n>:SPI:TRIGger:PATtern:MOSI:DATA"](#) on page 1132
 - [":SBUS<n>:SPI:TRIGger:PATtern:MISO:WIDTH"](#) on page 1131
 - [":SBUS<n>:SPI:TRIGger:PATtern:MOSI:WIDTH"](#) on page 1133

:SYSTem:MENU

 (see [page 1650](#))

Command Syntax :SYSTem:MENU <menu>

<menu> ::= {MASK | MEASure | SEGmented | LISTer | POWER}

The :SYSTem:MENU command changes the front panel softkey menu.

:TIMEbase:DElay

O (see [page 1650](#))

Command Syntax :TIMEbase:DElay <delay_value>

<delay_value> ::= time in seconds from trigger to the delay reference point on the screen.

The valid range for delay settings depends on the time/division setting for the main time base.

The :TIMEbase:DElay command sets the main time base delay. This delay is the time between the trigger event and the delay reference point on the screen. The delay reference point is set with the :TIMEbase:REference command (see [page 1337](#)).

NOTE

The :TIMEbase:DElay command is obsolete and is provided for backward compatibility to previous oscilloscopes. Use the :TIMEbase:POSition command (see [page 1334](#)) instead.

Query Syntax :TIMEbase:DElay?

The :TIMEbase:DElay query returns the current delay value.

Return Format <delay_value><NL>

<delay_value> ::= time from trigger to display reference in seconds in NR3 format.

Example Code

```
' TIMEBASE_DELAY - Sets the time base delay. This delay
' is the internal time between the trigger event and the
' onscreen delay reference point.
```

```
' Set time base delay to 0.0.
myScope.WriteString ":TIMEBASE:DELAY 0.0"
```

See complete example programs at: [Chapter 46](#), "Programming Examples," starting on page 1659

:TRIGger:THReshold

O (see [page 1650](#))

Command Syntax :TRIGger:THReshold <channel group>, <threshold type> [, <value>]
 <channel group> ::= {POD1 | POD2}
 <threshold type> ::= {CMOS | ECL | TTL | USERdef}
 <value> ::= voltage for USERdef (floating-point number) [Volt type]
 [Volt type] ::= {V | mV | uV}

The :TRIGger:THReshold command sets the threshold (trigger level) for a pod of 8 digital channels (either digital channels 0 through 7 or 8 through 15). The threshold can be set to a predefined value or to a user-defined value. For the predefined value, the voltage parameter is not required.

NOTE

This command is only available on the MSO models.

NOTE

The :TRIGger:THReshold command is an obsolete command provided for compatibility to previous oscilloscopes. Use the :POD<n>:THReshold command (see [page 749](#)), :DIGital<d>:THReshold command (see [page 389](#)), or :TRIGger[:EDGE]:LEVel command (see [page 1376](#)).

Query Syntax :TRIGger:THReshold? <channel group>

The :TRIGger:THReshold? query returns the voltage and threshold text for analog channel 1 or 2, or POD1 or POD2.

Return Format <threshold type>[, <value>]<NL>
 <threshold type> ::= {CMOS | ECL | TTL | USER}
 CMOS ::= 2.5V
 TTL ::= 1.5V
 ECL ::= -1.3V
 USERdef ::= range from -8.0V to +8.0V.
 <value> ::= voltage for USERdef (a floating-point number in NR1).

:TRIGger:TV:TVMode

O (see [page 1650](#))

Command Syntax :TRIGger:TV:TVMode <mode>

```
<mode> ::= {FIEld1 | FIEld2 | AFIElds | ALINes | LINE | VERTical
            | LFIeld1 | LFIeld2 | LALTernate | LVERTical}
```

The :TRIGger:TV:MODE command selects the TV trigger mode and field. The LVERTical parameter is only available when :TRIGger:TV:STANdard is GENeric. The LALTernate parameter is not available when :TRIGger:TV:STANdard is GENeric (see [page 1420](#)).

Old forms for <mode> are accepted:

<mode>	Old Forms Accepted
FIEld1	F1
FIEld2	F2
AFIEld	ALLFields, ALLFLDS
ALINes	ALLLines
LFIeld1	LINEF1, LINEFIELD1
LFIeld2	LINEF2, LINEFIELD2
LALTernate	LINEAlt
LVERTical	LINEVert

NOTE

The :TRIGger:TV:TVMode command is an obsolete command provided for compatibility to previous oscilloscopes. Use the :TRIGger:TV:MODE command (see [page 1417](#)) instead.

Query Syntax :TRIGger:TV:TVMode?

The :TRIGger:TV:TVMode? query returns the TV trigger mode.

Return Format <value><NL>

```
<value> ::= {FIE1 | FIE2 | AF1 | ALIN | LINE | VERT | LFI1 | LFI2
            | LALT | LVER}
```


42 Error Messages

-440, Query UNTERMINATED after indefinite response

-430, Query DEADLOCKED

-420, Query UNTERMINATED

-410, Query INTERRUPTED

-400, Query error

-340, Calibration failed

-330, Self-test failed

-321, Out of memory

-320, Storage fault

-315, Configuration memory lost

-314, Save/recall memory lost

-313, Calibration memory lost

-311, Memory error

-310, System error

-300, Device specific error

-278, Macro header not found

-277, Macro redefinition not allowed

-276, Macro recursion error

-273, Illegal macro label

-272, Macro execution error

-258, Media protected

-257, File name error

-256, File name not found

-255, Directory full

-254, Media full

-253, Corrupt media

-252, Missing media

-251, Missing mass storage

-250, Mass storage error

-241, Hardware missing

This message can occur when a feature is unavailable or unlicensed.

-240, Hardware error

-231, Data questionable

-230, Data corrupt or stale

-224, Illegal parameter value

-223, Too much data

-222, Data out of range

-221, Settings conflict

-220, Parameter error

-200, Execution error

-183, Invalid inside macro definition

-181, Invalid outside macro definition

-178, Expression data not allowed

-171, Invalid expression

-170, Expression error

-168, Block data not allowed

-161, Invalid block data

-158, String data not allowed

-151, Invalid string data

- 150, String data error
- 148, Character data not allowed
- 138, Suffix not allowed
- 134, Suffix too long
- 131, Invalid suffix
- 128, Numeric data not allowed
- 124, Too many digits
- 123, Exponent too large
- 121, Invalid character in number
- 120, Numeric data error
- 114, Header suffix out of range
- 113, Undefined header
- 112, Program mnemonic too long

-109, Missing parameter

-108, Parameter not allowed

-105, GET not allowed

-104, Data type error

-103, Invalid separator

-102, Syntax error

-101, Invalid character

-100, Command error

+10, Software Fault Occurred

+100, File Exists

+101, End-Of-File Found

+102, Read Error

+103, Write Error

+104, Illegal Operation

+105, Print Canceled

+106, Print Initialization Failed

+107, Invalid Trace File

+108, Compression Error

+109, No Data For Operation

A remote operation wants some information, but there is no information available. For example, you may request a stored TIFF image using the :DISPlay:DATA? query, but there may be no image stored.

+112, Unknown File Type

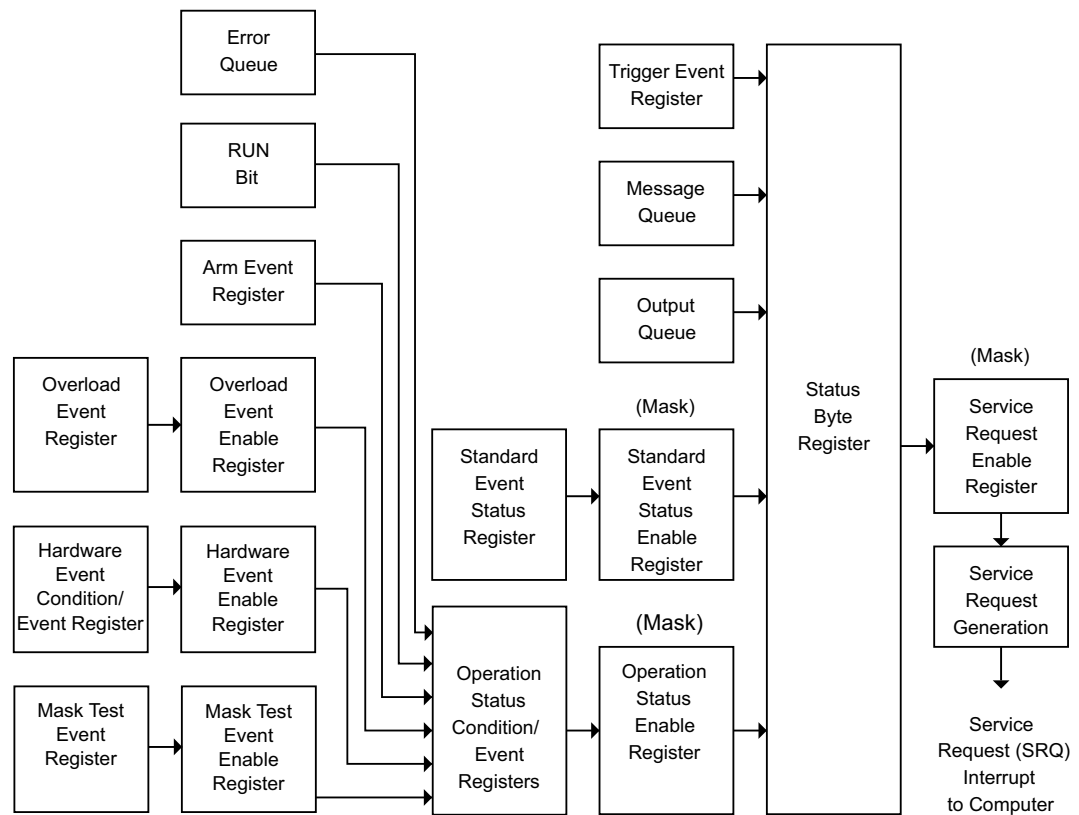
+113, Directory Not Supported

43 Status Reporting

Status Reporting Data Structures / 1599
Status Byte Register (STB) / 1602
Service Request Enable Register (SRE) / 1604
Trigger Event Register (TER) / 1605
Output Queue / 1606
Message Queue / 1607
(Standard) Event Status Register (ESR) / 1608
(Standard) Event Status Enable Register (ESE) / 1609
Error Queue / 1610
Operation Status Event Register (:OPERRegister[:EVENT]) / 1611
Operation Status Condition Register (:OPERRegister:CONDition) / 1613
Arm Event Register (AER) / 1614
Overload Event Register (:OVLRegister) / 1615
Hardware Event Event Register (:HWERRegister[:EVENT]) / 1616
Hardware Event Condition Register (:HWERRegister:CONDition) / 1617
Mask Test Event Event Register (:MTERRegister[:EVENT]) / 1618
Clearing Registers and Queues / 1619
Status Reporting Decision Chart / 1620
Example: Checking for Armed Status / 1621

IEEE 488.2 defines data structures, commands, and common bit definitions for status reporting (for example, the Status Byte Register and the Standard Event Status Register). There are also instrument-defined structures and bits (for example, the Operation Status Event Register and the Overload Event Register).

An overview of the oscilloscope's status reporting structure is shown in the following block diagram. The status reporting structure allows monitoring specified events in the oscilloscope. The ability to monitor and report these events allows determination of such things as the status of an operation, the availability and reliability of the measured data, and more.



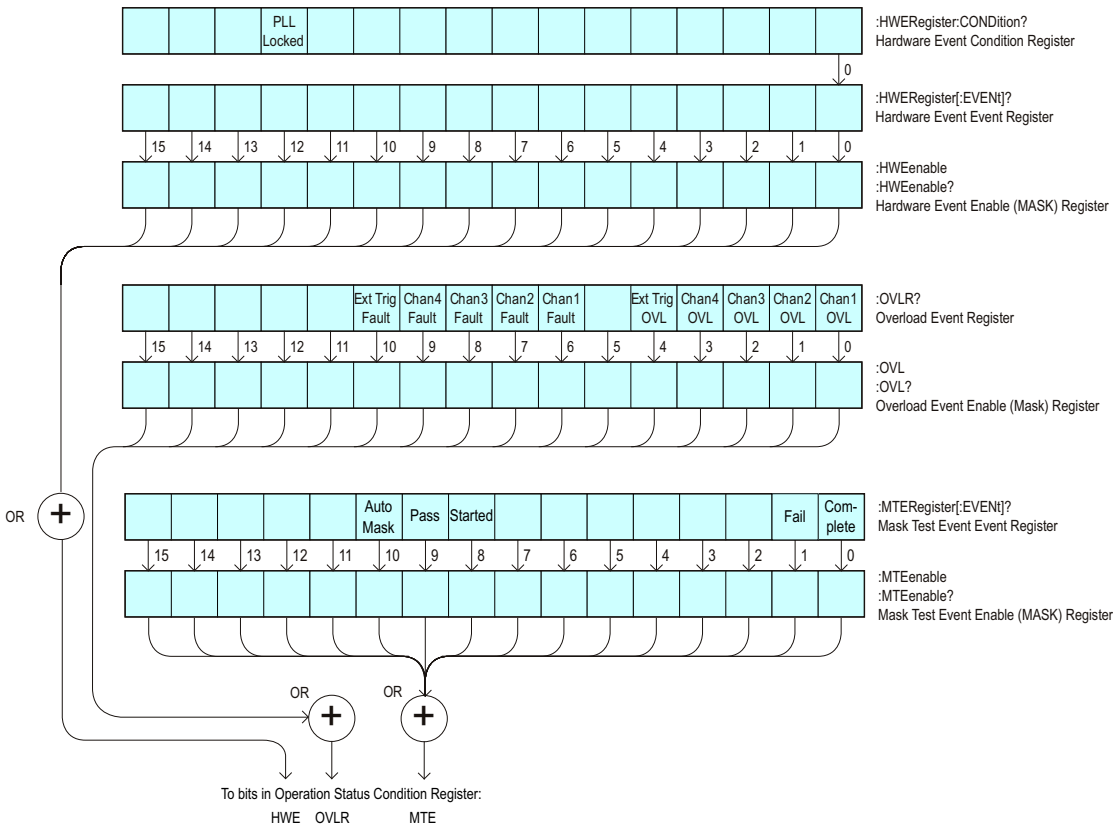
- To monitor an event, first clear the event; then, enable the event. All of the events are cleared when you initialize the instrument.
- To allow a service request (SRQ) interrupt to an external controller, enable at least one bit in the Status Byte Register (by setting, or unmasking, the bit in the Service Request Enable register).

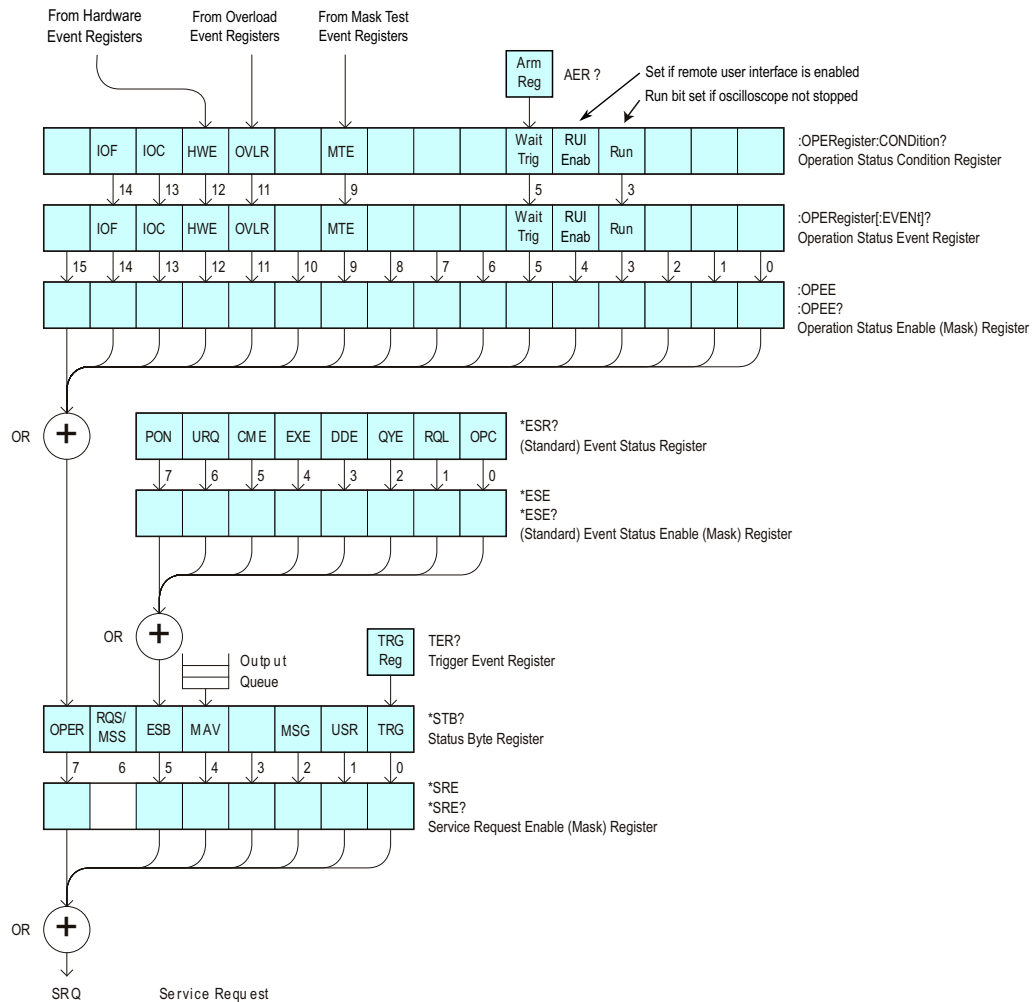
The Status Byte Register, the Standard Event Status Register group, and the Output Queue are defined as the Standard Status Data Structure Model in IEEE 488.2-1987.

The bits in the status byte act as summary bits for the data structures residing behind them. In the case of queues, the summary bit is set if the queue is not empty. For registers, the summary bit is set if any enabled bit in the event register is set. The events are enabled with the corresponding event enable register. Events captured by an event register remain set until the register is read or cleared. Registers are read with their associated commands. The *CLS command clears all event registers and all queues except the output queue. If you send *CLS immediately after a program message terminator, the output queue is also cleared.

Status Reporting Data Structures

The following figure shows how the status register bits are masked and logically OR'ed to generate service requests (SRQ) on particular events.





The status register bits are described in more detail in the following tables:

- [Table 83](#)
- [Table 81](#)
- [Table 91](#)
- [Table 92](#)
- [Table 94](#)
- [Table 86](#)
- [Table 87](#)
- [Table 89](#)

The status registers picture above shows how the different status reporting data structures work together. To make it possible for any of the Standard Event Status Register bits to generate a summary bit, the bits must be enabled. These bits are enabled by using the *ESE common command to set the corresponding bit in the Standard Event Status Enable Register.

To generate a service request (SRQ) interrupt to an external controller, at least one bit in the Status Byte Register must be enabled. These bits are enabled by using the *SRE common command to set the corresponding bit in the Service Request Enable Register. These enabled bits can then set RQS and MSS (bit 6) in the Status Byte Register.

Status Byte Register (STB)

The Status Byte Register is the summary-level register in the status reporting structure. It contains summary bits that monitor activity in the other status registers and queues. The Status Byte Register is a live register. That is, its summary bits are set and cleared by the presence and absence of a summary bit from other event registers or queues.

If the Status Byte Register is to be used with the Service Request Enable Register to set bit 6 (RQS/MSS) and to generate an SRQ, at least one of the summary bits must be enabled, then set. Also, event bits in all other status registers must be specifically enabled to generate the summary bit that sets the associated summary bit in the Status Byte Register.

The Status Byte Register can be read using either the *STB? Common Command or the programming interface serial poll command. Both commands return the decimal-weighted sum of all set bits in the register. The difference between the two methods is that the serial poll command reads bit 6 as the Request Service (RQS) bit and clears the bit which clears the SRQ interrupt. The *STB? command reads bit 6 as the Master Summary Status (MSS) and does not clear the bit or have any affect on the SRQ interrupt. The value returned is the total bit weights of all of the bits that are set at the present time.

The use of bit 6 can be confusing. This bit was defined to cover all possible computer interfaces, including a computer that could not do a serial poll. The important point to remember is that, if you are using an SRQ interrupt to an external computer, the serial poll command clears bit 6. Clearing bit 6 allows the oscilloscope to generate another SRQ interrupt when another enabled event occurs.

No other bits in the Status Byte Register are cleared by either the *STB? query or the serial poll, except the Message Available bit (bit 4). If there are no other messages in the Output Queue, bit 4 (MAV) can be cleared as a result of reading the response to the *STB? command.

If bit 4 (weight = 16) and bit 5 (weight = 32) are set, the program prints the sum of the two weights. Since these bits were not enabled to generate an SRQ, bit 6 (weight = 64) is not set.

The following example uses the *STB? query to read the contents of the oscilloscope's Status Byte Register.

```
myScope.WriteString "*STB?"
varQueryResult = myScope.ReadNumber
MsgBox "Status Byte Register, Read: 0x" + Hex(varQueryResult)
```

The next program prints 0xD1 and clears bit 6 (RQS) and bit 4 (MAV) of the Status Byte Register. The difference in the output value between this example and the previous one is the value of bit 6 (weight = 64). Bit 6 is set when the first enabled summary bit is set and is cleared when the Status Byte Register is read by the serial poll command.

Example The following example uses the resource session object's ReadSTB method to read the contents of the oscilloscope's Status Byte Register.

```
varQueryResult = myScope.IO.ReadSTB  
MsgBox "Status Byte Register, Serial Poll: 0x" + Hex(varQueryResult)
```

NOTE

Use Serial Polling to Read Status Byte Register. Serial polling is the preferred method to read the contents of the Status Byte Register because it resets bit 6 and allows the next enabled event that occurs to generate a new SRQ interrupt.

See Also • **"*STB (Read Status Byte)"** on page 232

Service Request Enable Register (SRE)

Setting the Service Request Enable Register bits enable corresponding bits in the Status Byte Register. These enabled bits can then set RQS and MSS (bit 6) in the Status Byte Register.

Bits are set in the Service Request Enable Register using the *SRE command and the bits that are set are read with the *SRE? query.

Example The following example sets bit 4 (MAV) and bit 5 (ESB) in the Service Request Enable Register.

```
myScope.WriteString "*SRE " + CStr(CInt("&H30"))
```

This example uses the decimal parameter value of 48, the string returned by CStr(CInt("&H30")), to enable the oscilloscope to generate an SRQ interrupt under the following conditions:

- When one or more bytes in the Output Queue set bit 4 (MAV).
- When an enabled event in the Standard Event Status Register generates a summary bit that sets bit 5 (ESB).

See Also • **"*SRE (Service Request Enable)"** on page 230

Trigger Event Register (TER)

This register sets the TRG bit in the status byte when a trigger event occurs.

The TER event register stays set until it is cleared by reading the register or using the *CLS command. If your application needs to detect multiple triggers, the TER event register must be cleared after each one.

If you are using the Service Request to interrupt a program or controller operation, you must clear the event register each time the trigger bit is set.

See Also • **":TER (Trigger Event Register)"** on page 279

Output Queue

The output queue stores the oscilloscope-to-controller responses that are generated by certain instrument commands and queries. The output queue generates the Message Available summary bit when the output queue contains one or more bytes. This summary bit sets the MAV bit (bit 4) in the Status Byte Register.

When using the Keysight VISA COM library, the output queue may be read with the FormattedIO488 object's ReadString, ReadNumber, ReadList, or ReadIEEEBlock methods.

Message Queue

The message queue contains the text of the last message written to the advisory line on the screen of the oscilloscope. The length of the oscilloscope's message queue is 1. Note that messages sent with the :SYSTem:DSP command do not set the MSG status bit in the Status Byte Register.

(Standard) Event Status Register (ESR)

The (Standard) Event Status Register (ESR) monitors the following oscilloscope status events:

- PON - Power On
- URQ - User Request
- CME - Command Error
- EXE - Execution Error
- DDE - Device Dependent Error
- QYE - Query Error
- RQC - Request Control
- OPC - Operation Complete

When one of these events occur, the event sets the corresponding bit in the register. If the bits are enabled in the Standard Event Status Enable Register, the bits set in this register generate a summary bit to set bit 5 (ESB) in the Status Byte Register.

You can read the contents of the Standard Event Status Register and clear the register by sending the *ESR? query. The value returned is the total bit weights of all of the bits that are set at the present time.

Example The following example uses the *ESR query to read the contents of the Standard Event Status Register.

```
myScope.WriteString "*ESR?"
varQueryResult = myScope.ReadNumber
MsgBox "Standard Event Status Register: 0x" + Hex(varQueryResult)
```

If bit 4 (weight = 16) and bit 5 (weight = 32) are set, the program prints the sum of the two weights.

See Also • ["*ESR \(Standard Event Status Register\)"](#) on page 218

(Standard) Event Status Enable Register (ESE)

To allow any of the (Standard) Event Status Register (ESR) bits to generate a summary bit, you must first enable that bit. Enable the bit by using the *ESE (Event Status Enable) common command to set the corresponding bit in the (Standard) Event Status Enable Register (ESE).

Set bits are read with the *ESE? query.

Example Suppose your application requires an interrupt whenever any type of error occurs. The error related bits in the (Standard) Event Status Register are bits 2 through 5 (hexadecimal value 0x3C). Therefore, you can enable any of these bits to generate the summary bit by sending:

```
myScope.WriteString "*ESE " + CStr(CInt("&H3C"))
```

Whenever an error occurs, it sets one of these bits in the (Standard) Event Status Register. Because all the error related bits are enabled, a summary bit is generated to set bit 5 (ESB) in the Status Byte Register.

If bit 5 (ESB) in the Status Byte Register is enabled (via the *SRE command), an SRQ service request interrupt is sent to the controller PC.

NOTE

Disabled (Standard) Event Status Register bits respond but do not generate a summary bit. (Standard) Event Status Register bits that are not enabled still respond to their corresponding conditions (that is, they are set if the corresponding event occurs). However, because they are not enabled, they do not generate a summary bit to the Status Byte Register.

See Also • ["*ESE \(Standard Event Status Enable\)"](#) on page 216

Error Queue

As errors are detected, they are placed in an error queue. This queue is first in, first out. If the error queue overflows, the last error in the queue is replaced with error 350, Queue overflow. Any time the queue overflows, the least recent errors remain in the queue, and the most recent error is discarded. The length of the oscilloscope's error queue is 30 (29 positions for the error messages, and 1 position for the Queue overflow message).

The error queue is read with the :SYSTem:ERRor? query. Executing this query reads and removes the oldest error from the head of the queue, which opens a position at the tail of the queue for a new error. When all the errors have been read from the queue, subsequent error queries return "0, No error".

The error queue is cleared when:

- the instrument is powered up,
- the instrument receives the *CLS common command, or
- the last item is read from the error queue.

Operation Status Event Register (:OPERRegister[:EVENT])

The Operation Status Event Register register hosts these bits:

Name	Location	Description
RUN bit	bit 3	Is set whenever the instrument goes from a stop state to a single or running state.
RUI Enab bit	bit 4	The remote user interface has gone from a disabled state to an enabled state. The front-panel graphical user interface can disable most of the remote interface, including the *OPC syntax, for example when: ▪ a modal dialog box is open ▪ the user is being prompted ▪ all segments are being analyzed ▪ there is a channel overload ▪ certain compliance applications are running When disabled, commands or queries are accepted, but "settings conflict" errors are returned. The status model is the only part of the remote user interface that is enabled. To determine when the remote interface is re-enabled, you can read this bit or wait for the event that gets generated when its status goes from disabled to enabled.
WAIT TRIG bit	bit 5	Is set by the Trigger Armed Event Register and indicates that the trigger is armed.
MTE bit	bit 9	Comes from the Mask Test Event Registers.
OVLRL bit	bit 11	Is set whenever a 50Ω input overload occurs.
HWE bit	bit 12	Comes from the Hardware Event Registers.
IOC bit	bit 13	Is set when any IO operation completes. IO operations are any remote data request using any interface (USB, LAN, or GPIB). For example, if you connect to an oscilloscope using the USB interface and then request waveform data, the IOC bit will be set when the IO operation completes.
IOF bit	bit 14	Is set only when something causes the IO operation to fail, like disconnecting a USB device or interrupting the operation from the oscilloscope's front panel.

If any of these bits are set, the OPER bit (bit 7) of the Status Byte Register is set. The Operation Status Event Register is read and cleared with the :OPERRegister[:EVENT]? query. The register output is enabled or disabled using the mask value supplied with the OPEE command.

See Also • **":OPERRegister[:EVENT] (Operation Status Event Register)"** on page 265

Operation Status Condition Register (:OPERRegister:CONDition)

The Operation Status Condition Register register hosts these bits:

Name	Location	Description
RUN bit	bit 3	Is set whenever the instrument is not stopped.
RUI Enab bit	bit 4	Shows whether the remote user interface is enabled. The front-panel graphical user interface can disable most of the remote interface, including the *OPC syntax, for example when: - a modal dialog box is open - the user is being prompted - all segments are being analyzed - there is a channel overload - certain compliance applications are running When disabled, commands or queries are accepted, but "settings conflict" errors are returned. The status model is the only part of the remote user interface that is enabled. To determine when the remote interface is re-enabled, you can read this bit or wait for the event that gets generated when its status goes from disabled to enabled.
WAIT TRIG bit	bit 5	Is set by the Trigger Armed Event Register and indicates that the trigger is armed.
MTE bit	bit 9	Comes from the Mask Test Event Registers.
OVLr bit	bit 11	Is set whenever a 50Ω input overload occurs.
HWE bit	bit 12	Comes from the Hardware Event Registers.
IOC bit	bit 13	Is set when any IO operation completes. IO operations are any remote data request using any interface (USB, LAN, or GPIB). For example, if you connect to an oscilloscope using the USB interface and then request waveform data, the IOC bit will be set when the IO operation completes.
IOF bit	bit 14	Is set only when something causes the IO operation to fail, like disconnecting a USB device or interrupting the operation from the oscilloscope's front panel.

The :OPERRegister:CONDition? query returns the value of the Operation Status Condition Register.

See Also • [":OPERRegister:CONDition \(Operation Status Condition Register\)"](#) on page 262

Arm Event Register (AER)

This register sets bit 5 (Wait Trig bit) in the Operation Status Register and the OPER bit (bit 7) in the Status Byte Register when the instrument becomes armed.

The ARM event register stays set until it is cleared by reading the register with the AER? query or using the *CLS command. If your application needs to detect multiple triggers, the ARM event register must be cleared after each one.

If you are using the Service Request to interrupt a program or controller operation when the trigger bit is set, then you must clear the event register after each time it has been set.

See Also • **":AER (Arm Event Register)"** on page 242

Overload Event Register (:OVLRegister)

The Overload Event Register register hosts these bits:

Name	Location	Description
Channel 1 OVL	bit 0	Overload has occurred on Channel 1 input.
Channel 2 OVL	bit 1	Overload has occurred on Channel 2 input.
Channel 3 OVL	bit 2	Overload has occurred on Channel 3 input.
Channel 4 OVL	bit 3	Overload has occurred on Channel 4 input.
External Trigger OVL	bit 4	Overload has occurred on External Trigger input.
Channel 1 Fault	bit 6	Fault has occurred on Channel 1 input.
Channel 2 Fault	bit 7	Fault has occurred on Channel 2 input.
Channel 3 Fault	bit 8	Fault has occurred on Channel 3 input.
Channel 4 Fault	bit 9	Fault has occurred on Channel 4 input.
External Trigger Fault	bit 10	Fault has occurred on External Trigger input.

See Also • [":OVLRegister \(Overload Event Register\)"](#) on page 270

Hardware Event Event Register (:HWERegister[:EVENT])

This register hosts the PLL LOCKED bit (bit 12).

- The PLL LOCKED bit (bit 12) is for internal use and is not intended for general use.

See Also • [":HWERegister\[:EVENT\] \(Hardware Event Event Register\)"](#) on page 255

Hardware Event Condition Register (:HWERegister:CONDition)

This register hosts the PLL LOCKED bit (bit 12).

- The :HWERegister:CONDition? query returns the value of the Hardware Event Condition Register.
- The PLL LOCKED bit (bit 12) is for internal use and is not intended for general use.

See Also • [":HWERegister:CONDition \(Hardware Event Condition Register\)"](#) on page 254

Mask Test Event Event Register (:MTERegister[:EVENT])

The Mask Test Event Event Register register hosts these bits:

Name	Location	Description
Complete	bit 0	Is set when the mask test is complete.
Fail	bit 1	Is set when there is a mask test failure.
Started	bit 8	Is set when mask testing is started.
Pass	bit 9	Is set when the mask test passes.
Auto Mask	bit 10	Is set when auto mask creation is completed.

The :MTERegister[:EVENT]? query returns the value of, and clears, the Mask Test Event Event Register.

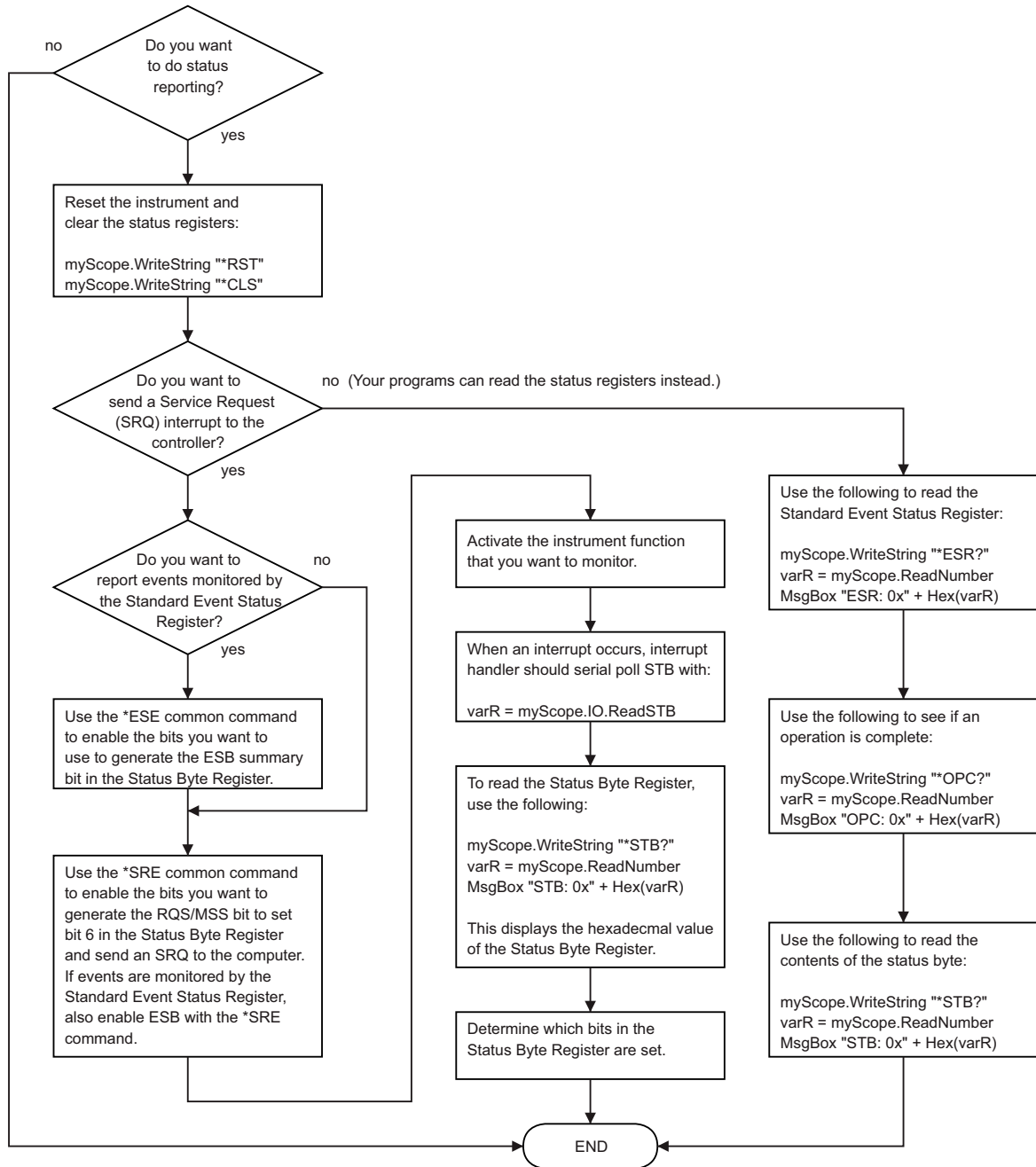
See Also • **[":MTERegister\[:EVENT\] \(Mask Test Event Event Register\)"](#)** on page 258

Clearing Registers and Queues

The *CLS common command clears all event registers and all queues except the output queue. If *CLS is sent immediately after a program message terminator, the output queue is also cleared.

See Also • **"*CLS (Clear Status)"** on page 215

Status Reporting Decision Chart



Example: Checking for Armed Status

```

#!/python3
# -*- coding: utf-8 -*-

# *****
# This script using the Python language (http://www.python.org/) and
# the PyVISA package (http://pyvisa.readthedocs.org/) shows three
# methods to tell whether a Keysight InfiniiVision oscilloscope is
# armed.
# *****

# Import modules
# -----
import sys
import pyvisa
import time

# Initialization constants
# -----
# Get VISA address from Keysight IO Libraries Connection Expert
VISA_ADDRESS = \
    "TCPIP0::k-mx3104g-10104.cos.is.keysight.com::inst0::INSTR"
GLOBAL_TOUT = 20000 # IO timeout in milliseconds

# =====
# Method 1: Query the Armed Event Register with :AER?
# -----
# This method reads the 1-bit Armed Event Register using the :AER?
# query.
#
# The Armed Event Register bit goes low (0) when it is read using
# :AER? or when a *CLS command is issued.
# =====
def method_1():

    # Stop the oscilloscope.
    KsInfiniiVisionX.query(":STOP;*OPC?")

    # Method 1: Initiate capture using :SINGLE
    # -----
    print("Acquiring signal (Method 1, using :SINGLE)...\n")
    now = time.perf_counter()

    # Clear all status registers before checking for new events.
    KsInfiniiVisionX.write("*CLS")

    # Because the :AER? query will not work with :DIGitize (which is
    # blocking), use the :SINGLE command to start the acquisition.
    KsInfiniiVisionX.write(":SINGLE")

    # Method 1: Determine if armed using :AER? query.
    # -----
    # Define armed criteria.

```

```

ARMED = 1

# Test for armed.
ARMED_STATUS = int(KsInfiniiVisionX.query(":AER?"))

# Wait indefinitely until armed.
while ARMED_STATUS != ARMED:
    # Check the status again after small delay.
    time.sleep(0.1)    # 100 ms delay to prevent excessive queries.
    ARMED_STATUS = int(KsInfiniiVisionX.query(":AER?"))

print("Oscilloscope is armed (method 1, using :AER? query)!")
print("It took " + str(time.perf_counter() - now) + \
      " seconds to arm.\n")

# =====
# Method 2: Read the Status Byte
# -----
# This method reads the Status Byte register's OPER bit (bit 7) using
# the "read status byte" function in VISA, which works during blocking
# commands and can therefore be used with the :DIGitize command.
#
# The Status Byte bits do NOT go low (0) when the register is read.
#
# The *CLS command will clear the Status Byte bits.
#
# CAUTION: The oscilloscope's status registers are not updated until
# the :DIGitize completes. So, while the ARM may go true midway
# through the :DIGitize, it does not get reported to the status model
# until the :DIGitize completes, and therefore the STB does not
# reflect it as true until the :DIGitize completes.
# =====
def method_2():

    # Stop the oscilloscope.
    KsInfiniiVisionX.query(":STOP;*OPC?")

    # Method 2: Initiate capture using :DIGitize or :SINGle
    # -----
    print("Acquiring signal (Method 2, using :DIGitize)...\n")
    now = time.perf_counter()

    # Clear all status registers before checking for new events.
    KsInfiniiVisionX.write("*CLS")

    # Mask out all bits in the Operation Status Event Register except
    # for the ARM bit.
    KsInfiniiVisionX.write(":OPEE 32")    # "Unmask" only the arm bit

    # Use the :DIGitize command to start the acquisition.
    KsInfiniiVisionX.write(":DIGitize")

    # Method 2: Determine if armed by reading the Status Byte.
    # -----
    # Define register bit masks for the Status Byte Register
    ARM_BIT = 7

```

```

# 1 leftshift 7 = 128 (bit 7 in the Status Byte Register)
ARM_MASK = 1 << ARM_BIT

# Define armed criteria.
ARMED = 1 << ARM_BIT # 1 leftshift 7 = 128

# Test for armed.
STATUS_BYTE = int(KsInfiniiVisionX.read_stb())
ARMED_STATUS = STATUS_BYTE & ARM_MASK
# Note that you could also do:
# ARMED_STATUS = int(KsInfiniiVisionX.query("*STB?"))
# BUT *STB? does not work with the blocking :DIGitize.

# Wait indefinitely until armed.
while ARMED_STATUS != ARMED:
    # Check the status again after small delay.
    time.sleep(0.1) # 100 ms delay to prevent excessive queries.
    STATUS_BYTE = int(KsInfiniiVisionX.read_stb())
    ARMED_STATUS = STATUS_BYTE & ARM_MASK

print("Oscilloscope is armed (method 2, using Read STB function)!")
print("It took " + str(time.perf_counter() - now) + \
      " seconds to arm.\n")

# =====
# Method 3: Query the Operation Status Event Register with :OPER?
# -----
# This method reads the Operation Status Event Register's Wait Trig
# bit (bit 5) using the :OPER? query.
#
# The Operation Status Event Register bits are cleared (0) when the
# register is read.
#
# Also, the Wait Trig bit does NOT go low (0) when the oscilloscope
# becomes unarmed by starting or stopping another acquisition (before
# the first one finishes) or by changing the time scale.
#
# The Wait Trig bit is cleared by a *CLS command, by reading the
# Operation Status Event Register with the :OPER? query, or by reading
# the Armed Event Register with the :AER? query.
# =====
def method_3():

    # Stop the oscilloscope.
    KsInfiniiVisionX.query(":STOP;*OPC?")

    # Method 3: Initiate capture using :SINGle
    # -----
    print("Acquiring signal (Method 3, using :SINGle)...\n")
    now = time.perf_counter()

    # Clear all status registers before checking for new events.
    KsInfiniiVisionX.write("*CLS")

    # Because the :OPER? query will not work with :DIGitize (which is
    # blocking), use the :SINGle command to start the acquisition.

```

```

KsInfiniiVisionX.write(":SINGLE")

# Method 3: Determine if armed using :OPER? query.
# -----
# Define bit masks for the Operation Status Event Register
ARM_BIT = 5
# 1 leftshift 5 = 32 (bit 5 in the Operation Status Event
# Register)
ARM_MASK = 1 << ARM_BIT

# Define armed criteria.
ARMED = 1 << ARM_BIT # 1 leftshift 5 = 32

# Test for armed.
STATUS_REGISTER = int(KsInfiniiVisionX.query(":OPER?"))
ARMED_STATUS = STATUS_REGISTER & ARM_MASK

# Wait indefinitely until armed.
while ARMED_STATUS != ARMED:
    # Check the status again after small delay.
    time.sleep(0.1) # 100 ms delay to prevent excessive queries.
    STATUS_REGISTER = int(KsInfiniiVisionX.query(":OPER?"))
    ARMED_STATUS = STATUS_REGISTER & ARM_MASK

print("Oscilloscope is armed (method 3, using :OPER? query)!")
print("It took " + str(time.perf_counter() - now) + \
      " seconds to arm.\n")

# =====
# Main
# =====

# Connect and initialize oscilloscope
# -----
# Define VISA Resource Manager & Install directory
rm = pyvisa.ResourceManager('C:\\Windows\\System32\\visa64.dll')

# Define and open the oscilloscope using the VISA address
KsInfiniiVisionX = rm.open_resource(VISA_ADDRESS)

# Set the Global Timeout
KsInfiniiVisionX.timeout = GLOBAL_TOUT

# Clear the instrument bus
KsInfiniiVisionX.clear()

# Reset the oscilloscope.
KsInfiniiVisionX.write("*RST")

# Autoscale to set up vertical scale and trigger level on Probe Comp.
KsInfiniiVisionX.write(":CHANnel1:DISPlay OFF")
KsInfiniiVisionX.write(":CHANnel2:DISPlay ON")
KsInfiniiVisionX.write(":AUToscale:CHANnels DISPlayed")
KsInfiniiVisionX.write(":AUToscale")

# Ensure a "long" time to arm (5 seconds) and not trigger immediately.

```

```

# -----
# 10 second total capture, with trigger point in the middle = 5s to arm
KsInfiniiVisionX.write(":TIMEbase:RANGe 10")
# Prevent Auto trigger.
KsInfiniiVisionX.write(":TRIGger:SWEep NORMal")

# Use the three methods to check whether the oscilloscope is armed.
# -----
method_1()
method_2()
method_3()

# End of Script
# -----
KsInfiniiVisionX.clear()    # Clear communications interface
KsInfiniiVisionX.close()    # Close communications interface
print("All done.")

```

Example: Waiting for IO Operation Complete

NOTE

The IOC (IO Operation Complete) and IOF (IO Operation Failed) bits in the Operation Status Event Register identify when :WMEMory<r>:SAVE and other :SAVE and :RECall commands are completely done. To determine when a IO operation is complete, you should use these bits instead of the OPC (Operation Complete) bit in the Standard Event Status Register or the *OPC? query.

```
#!/python3
# -*- coding: utf-8 -*-

# *****
# This script using the Python language (http://www.python.org/) and
# the PyVISA package (http://pyvisa.readthedocs.org/) shows how to
# wait for IO operation completion in a Keysight InfiniiVision
# oscilloscope.
# *****

# Import modules
# -----
import sys
import pyvisa
import time

# Initialization constants
# -----
# Get VISA address from Keysight IO Libraries Connection Expert
VISA_ADDRESS = \
    "TCPIP0::k-mx3104g-10104.cos.is.keysight.com::inst0::INSTR"
GLOBAL_TOUT = 20000 # IO timeout in milliseconds

# =====
# Check for IO Operation Complete using :OPER? query.
# -----
# This method reads the Operation Status Register's IOC bit
# (bit 13) using the :OPER? query.
#
# The Operation Status Event Register bits are cleared (0) when the
# register is read.
#
# All status bits are cleared by a *CLS command.
# =====
def wait_io_operation():

    # -----
    now = time.perf_counter()
    # Define bit masks for the Operation Status Event Register
    IOC_BIT = 13
    # 1 leftshift 13 = 8192 (bit 13 in the Operation Status Event
    # Register)
    IOC_MASK = 1 << IOC_BIT
```

```

# Define IO complete criteria.
IO_COMPLETE = 1 << IOC_BIT    # 1 leftshift 13 = 8192

# Test for armed.
STATUS_REGISTER = int(KsInfiniiVisionX.query(":OPER?"))
IO_COMPLETE_STATUS = STATUS_REGISTER & IOC_MASK

# Wait indefinitely until armed.
while IO_COMPLETE_STATUS != IO_COMPLETE:
    # Check the status again after small delay.
    time.sleep(0.1)    # 100 ms delay to prevent excessive queries.
    STATUS_REGISTER = int(KsInfiniiVisionX.query(":OPER?"))
    IO_COMPLETE_STATUS = STATUS_REGISTER & IOC_MASK

time_in_seconds = str(time.perf_counter() - now)
print("IO Operation Complete (from :OPER? query).")
print("It took %s seconds for IO operation to complete.\n" % \
      time_in_seconds)

# =====
# Main
# =====

# Connect and initialize oscilloscope
# -----
# Define VISA Resource Manager & Install directory
rm = pyvisa.ResourceManager('C:\\Windows\\System32\\visa64.dll')

# Define and open the oscilloscope using the VISA address
KsInfiniiVisionX = rm.open_resource(VISA_ADDRESS)

# Set the Global Timeout
KsInfiniiVisionX.timeout = GLOBAL_TOUT

# Clear the instrument bus
KsInfiniiVisionX.clear()

# Reset the oscilloscope.
# KsInfiniiVisionX.write("*RST")
# Or comment out to use the current oscilloscope setup.

# Autoscale to set up vertical scale and trigger level on channels 1 and
# 2.
KsInfiniiVisionX.write(":CHANnel1:DISPlay ON")
KsInfiniiVisionX.write(":CHANnel2:DISPlay ON")
KsInfiniiVisionX.write(":AUToscale:CHANnels DISPlayed")
KsInfiniiVisionX.write(":AUToscale")

# Between :WMEemory<r>:SAVE commands, wait for IO complete.
# -----
# Clear all status registers before checking for new events.
KsInfiniiVisionX.write("*CLS")

KsInfiniiVisionX.write(":WMEemory1:SAVE CHANnel1")
wait_io_operation()
KsInfiniiVisionX.write(":WMEemory2:SAVE CHANnel2")

```

```
wait_io_operation()

# End of Script
# -----
KsInfiniiVisionX.clear()    # Clear communications interface
KsInfiniiVisionX.close()   # Close communications interface
print("All done.")
```


44 Synchronizing Acquisitions

Synchronization in the Programming Flow / 1630

Blocking Synchronization / 1631

Polling Synchronization With Timeout / 1632

Synchronizing with a Single-Shot Device Under Test (DUT) / 1634

Synchronization with an Averaging Acquisition / 1636

Example: Blocking and Polling Synchronization / 1638

When remotely controlling an oscilloscope with programming commands, it is often necessary to know when the oscilloscope has finished the previous operation and is ready for the next command. The most common example is when an acquisition is started using the :DIGitize, :RUN, or :SINGle commands. Before a measurement result can be queried, the acquisition must complete. Too often fixed delays are used to accomplish this wait, but fixed delays often use excessive time or the time may not be long enough. A better solution is to use synchronous commands and status to know when the oscilloscope is ready for the next request.

Synchronization in the Programming Flow

Most remote programming follows these three general steps:

- 1 Set up the oscilloscope and device under test (see [page 1630](#)).
- 2 Acquire a waveform (see [page 1630](#)).
- 3 Retrieve results (see [page 1630](#)).

Set Up the Oscilloscope

Before making changes to the oscilloscope setup, it is best to make sure it is stopped using the :STOP command followed by the *OPC? query.

NOTE

It is not necessary to use *OPC?, hard coded waits, or status checking when setting up the oscilloscope. After the oscilloscope is configured, it is ready for an acquisition.

Acquire a Waveform

When acquiring a waveform there are two possible methods used to wait for the acquisition to complete. These methods are blocking and polling. The table below details when each method should be chosen and why.

	Blocking Wait	Polling Wait
Use When	You know the oscilloscope <i>will</i> trigger based on the oscilloscope setup and device under test.	You know the oscilloscope <i>may or may not</i> trigger on the oscilloscope setup and device under test.
Advantages	No need for polling. Fastest method.	Remote interface will not timeout No need for device clear if no trigger.
Disadvantages	Remote interface may timeout. Device clear only way to get control of oscilloscope if there is no trigger.	Slower method. Requires polling loop. Requires known maximum wait time.
Implementation Details	See " Blocking Synchronization " on page 1631.	See " Polling Synchronization With Timeout " on page 1632.

Retrieve Results

Once the acquisition is complete, it is safe to retrieve measurements and statistics.

Blocking Synchronization

Use the :DIGitize command to start the acquisition. This blocks subsequent queries until the acquisition and processing is complete. For example:

```
'
' Synchronizing acquisition using blocking.
' =====

Option Explicit

Public myMgr As VisaComLib.ResourceManager
Public myScope As VisaComLib.FormattedIO488
Public varQueryResult As Variant
Public strQueryResult As String

Sub Main()

    On Error GoTo VisaComError

    ' Create the VISA COM I/O resource.
    Set myMgr = New VisaComLib.ResourceManager
    Set myScope = New VisaComLib.FormattedIO488
    Set myScope.IO = myMgr.Open("TCPIP0::130.29.69.12::inst0::INSTR")
    myScope.IO.Clear ' Clear the interface.

    ' Set up.
    ' -----
    myScope.WriteString ":TRIGger:MODE EDGE"
    myScope.WriteString ":TRIGger:EDGE:LEVel 2"
    myScope.WriteString ":TIMEbase:SCALe 5e-8"

    ' Acquire.
    ' -----
    myScope.WriteString ":DIGitize"

    ' Get results.
    ' -----
    myScope.WriteString ":MEASure:RISetime"
    myScope.WriteString ":MEASure:RISetime?"
    varQueryResult = myScope.ReadNumber ' Read risetime.
    Debug.Print "Risetime: " + _
        FormatNumber(varQueryResult * 1000000000, 1) + " ns"

    Exit Sub

VisaComError:
    MsgBox "VISA COM Error:" + vbCrLf + Err.Description

End Sub
```

Polling Synchronization With Timeout

This example requires a timeout value so the operation can abort if an acquisition does not occur within the timeout period:

```
'
' Synchronizing acquisition using polling.
' =====

Option Explicit

Public myMgr As VisaComLib.ResourceManager
Public myScope As VisaComLib.FormattedIO488
Public varQueryResult As Variant
Public strQueryResult As String

Private Declare Sub Sleep Lib "kernel32" (ByVal dwMilliseconds As Long)

Sub Main()

    On Error GoTo VisaComError

    ' Create the VISA COM I/O resource.
    Set myMgr = New VisaComLib.ResourceManager
    Set myScope = New VisaComLib.FormattedIO488
    Set myScope.IO = myMgr.Open("TCPIP0::130.29.69.12::inst0::INSTR")
    myScope.IO.Clear ' Clear the interface.

    ' Set up.
    ' -----
    ' Set up the trigger and horizontal scale.
    myScope.WriteString ":TRIGger:MODE EDGE"
    myScope.WriteString ":TRIGger:EDGE:LEVel 2"
    myScope.WriteString ":TIMEbase:SCALe 5e-8"

    ' Stop acquisitions and wait for the operation to complete.
    myScope.WriteString ":STOP"
    myScope.WriteString "*OPC?"
    strQueryResult = myScope.ReadString

    ' Acquire.
    ' -----
    ' Start a single acquisition.
    myScope.WriteString ":SINGle"

    ' Oscilloscope is armed and ready, enable DUT here.
    Debug.Print "Oscilloscope is armed and ready, enable DUT."

    ' Look for RUN bit = stopped (acquisition complete).
    Dim lngTimeout As Long ' Max millisecs to wait for single-shot.
    Dim lngElapsed As Long
    lngTimeout = 10000 ' 10 seconds.
    lngElapsed = 0

    Do While lngElapsed <= lngTimeout
```

```

myScope.WriteString ":OPERRegister:CONDition?"
varQueryResult = myScope.ReadNumber
' Mask RUN bit (bit 3, &H8).
If (varQueryResult And &H8) = 0 Then
    Exit Do
Else
    Sleep 100 ' Small wait to prevent excessive queries.
    lngElapsed = lngElapsed + 100
End If
Loop

' Get results.
' -----
If lngElapsed < lngTimeout Then
    myScope.WriteString ":MEASure:RISetime"
    myScope.WriteString ":MEASure:RISetime?"
    varQueryResult = myScope.ReadNumber ' Read risetime.
    Debug.Print "Risetime: " + _
        FormatNumber(varQueryResult * 1000000000, 1) + " ns"
Else
    Debug.Print "Timeout waiting for single-shot trigger."
End If

Exit Sub

VisaComError:
    MsgBox "VISA COM Error:" + vbCrLf + Err.Description

End Sub

```

Synchronizing with a Single-Shot Device Under Test (DUT)

The examples in **"Blocking Synchronization"** on page 1631 and **"Polling Synchronization With Timeout"** on page 1632 assume the DUT is continually running and therefore the oscilloscope will have more than one opportunity to trigger. With a single shot DUT, there is only one opportunity for the oscilloscope to trigger, so it is necessary for the oscilloscope to be armed and ready before the DUT is enabled.

NOTE

The blocking :DIGitize command cannot be used for a single shot DUT because once the :DIGitize command is issued, the oscilloscope is blocked from any further commands until the acquisition is complete.

This example is the same **"Polling Synchronization With Timeout"** on page 1632 with the addition of checking for the armed event status.

```
'
' Synchronizing single-shot acquisition using polling.
' =====

Option Explicit

Public myMgr As VisaComLib.ResourceManager
Public myScope As VisaComLib.FormattedIO488
Public varQueryResult As Variant
Public strQueryResult As String

Private Declare Sub Sleep Lib "kernel32" (ByVal dwMilliseconds As Long)

Sub Main()

    On Error GoTo VisaComError

    ' Create the VISA COM I/O resource.
    Set myMgr = New VisaComLib.ResourceManager
    Set myScope = New VisaComLib.FormattedIO488
    Set myScope.IO = myMgr.Open("TCPIP0::130.29.69.12::inst0::INSTR")
    myScope.IO.Clear ' Clear the interface.

    ' Set up.
    ' -----
    ' Set up the trigger and horizontal scale.
    myScope.WriteString ":TRIGger:MODE EDGE"
    myScope.WriteString ":TRIGger:EDGE:LEVel 2"
    myScope.WriteString ":TIMEbase:SCALe 5e-8"

    ' Stop acquisitions and wait for the operation to complete.
    myScope.WriteString ":STOP"
    myScope.WriteString "*OPC?"
    strQueryResult = myScope.ReadString

    ' Acquire.
```

```

' -----
' Start a single acquisition.
myScope.WriteString ":SINGLE"

' Wait until the trigger system is armed.
Do
    Sleep 100 ' Small wait to prevent excessive queries.
    myScope.WriteString ":AER?"
    varQueryResult = myScope.ReadNumber
Loop Until varQueryResult = 1

' Oscilloscope is armed and ready, enable DUT here.
Debug.Print "Oscilloscope is armed and ready, enable DUT."

' Now, look for RUN bit = stopped (acquisition complete).
Dim lngTimeout As Long ' Max millisecs to wait for single-shot.
Dim lngElapsed As Long
lngTimeout = 10000 ' 10 seconds.
lngElapsed = 0

Do While lngElapsed <= lngTimeout
    myScope.WriteString ":OPERRegister:CONDition?"
    varQueryResult = myScope.ReadNumber
    ' Mask RUN bit (bit 3, &H8).
    If (varQueryResult And &H8) = 0 Then
        Exit Do
    Else
        Sleep 100 ' Small wait to prevent excessive queries.
        lngElapsed = lngElapsed + 100
    End If
Loop

' Get results.
' -----
If lngElapsed < lngTimeout Then
    myScope.WriteString ":MEASure:RISetime"
    myScope.WriteString ":MEASure:RISetime?"
    varQueryResult = myScope.ReadNumber ' Read risetime.
    Debug.Print "Risetime: " + _
        FormatNumber(varQueryResult * 1000000000, 1) + " ns"
Else
    Debug.Print "Timeout waiting for single-shot trigger."
End If

Exit Sub

VisaComError:
MsgBox "VISA COM Error:" + vbCrLf + Err.Description

End Sub

```

Synchronization with an Averaging Acquisition

When averaging, it is necessary to know when the average count has been reached. The :SINGle command does not average.

If it is known that a trigger will occur, a :DIGitize will acquire the complete number of averages, but if the number of averages is large, a timeout on the connection can occur.

The example below polls during the :DIGitize to prevent a timeout on the connection.

```
'
' Synchronizing in averaging acquisition mode.
' =====

Option Explicit

Public myMgr As VisaComLib.ResourceManager
Public myScope As VisaComLib.FormattedIO488
Public varQueryResult As Variant
Public strQueryResult As String

Private Declare Sub Sleep Lib "kernel32" (ByVal dwMilliseconds As Long)

Sub Main()

    On Error GoTo VisaComError

    ' Create the VISA COM I/O resource.
    Set myMgr = New VisaComLib.ResourceManager
    Set myScope = New VisaComLib.FormattedIO488
    Set myScope.IO = myMgr.Open("TCPIP0::130.29.69.12::inst0::INSTR")
    myScope.IO.Clear ' Clear the interface.
    myScope.IO.Timeout = 5000

    ' Set up.
    ' -----
    ' Set up the trigger and horizontal scale.
    myScope.WriteString ":TRIGger:SWEep NORMal"
    myScope.WriteString ":TRIGger:MODE EDGE"
    myScope.WriteString ":TRIGger:EDGE:LEVel 2"
    myScope.WriteString ":TIMEbase:SCALE 5e-8"

    ' Stop acquisitions and wait for the operation to complete.
    myScope.WriteString ":STOP"
    myScope.WriteString "*OPC?"
    strQueryResult = myScope.ReadString

    ' Set up average acquisition mode.
    Dim lngAverages As Long
    lngAverages = 256
    myScope.WriteString ":ACquire:COUNT " + CStr(lngAverages)
    myScope.WriteString ":ACquire:TYPE AVERage"
```



```

' Save *ESE (Standard Event Status Enable register) mask
' (so it can be restored later).
Dim varInitialESE As Variant
myScope.WriteString "*ESE?"
varInitialESE = myScope.ReadNumber

' Set *ESE mask to allow only OPC (Operation Complete) bit.
myScope.WriteString "*ESE " + CStr(CInt("&H01"))

' Acquire using :DIGitize. Set up OPC bit to be set when the
' operation is complete.
' -----
myScope.WriteString ":DIGitize"
myScope.WriteString "*OPC"

' Assume the oscilloscope will trigger, if not put a check here.

' Wait until OPC becomes true (bit 5 of Status Byte register, STB,
' from Standard Event Status register, ESR, is set). STB can be
' read during :DIGitize without generating a timeout.
Do
    Sleep 4000 ' Poll more often than the timeout setting.
    varQueryResult = myScope.IO.ReadSTB
Loop While (varQueryResult And &H20) = 0

' Clear ESR and restore previously saved *ESE mask.
myScope.WriteString "*ESR?" ' Clear ESR by reading it.
varQueryResult = myScope.ReadNumber
myScope.WriteString "*ESE " + CStr(varInitialESE)

' Get results.
' -----
myScope.WriteString ":WAVEform:COUNT?"
varQueryResult = myScope.ReadNumber
Debug.Print "Averaged waveforms: " + CStr(varQueryResult)

myScope.WriteString ":MEASure:RISetime"
myScope.WriteString ":MEASure:RISetime?"
varQueryResult = myScope.ReadNumber ' Read risetime.
Debug.Print "Risetime: " + _
    FormatNumber(varQueryResult * 1000000000, 1) + " ns"

Exit Sub

VisaComError:
    MsgBox "VISA COM Error:" + vbCrLf + Err.Description

End Sub

```

Example: Blocking and Polling Synchronization

```

#!/python3
# -*- coding: utf-8 -*-

# *****
# This script using the Python language (http://www.python.org/) and
# the PyVISA package (http://pyvisa.readthedocs.org/) shows the two
# best synchronization methods for InfiniiVision oscilloscopes.
# Benefits and drawbacks of each method are described. No error
# handling is provided except in the actual synchronization methods.
# *****

# Import modules
# -----
import sys
import pyvisa
import time

# Initialization constants
# -----
# Get VISA address from Keysight IO Libraries Connection Expert
VISA_ADDRESS = \
    "TCPIP0::k-mx3104g-10104.cos.is.keysight.com::inst0::INSTR"
GLOBAL_TOUT = 10000 # IO timeout in milliseconds

TIME_TO_TRIGGER = 10 # Time in seconds
# -----
# This is the time until the FIRST trigger event.
#
# While the script calculates a general timeout for the given setup,
# it cannot know when a trigger event will occur. Thus, you must
# still set this value.
#
# This time is in addition to the calculated minimum timeout... so, if
# an oscilloscope might take say, 1 us to arm and acquire data, the
# signal might take 100 seconds before it occurs... this accounts for
# that.
#
# The SCOPE_ACQUISITION_TIME_OUT calculation pads this by 1.1.
# -----

TIME_BETWEEN_TRIGGERS = 0.025 # Time in seconds - for Average,
# Segmented, and Equivalent Time modes, else set to 0
# -----
# In Average, Segmented, and Equivalent Time modes, the oscilloscope
# makes repeated acquisitions. This is similar to the above
# TIME_TO_TRIGGER, but it is the time BETWEEN triggers. For example,
# it might take 10 seconds for the first trigger event, and then they
# might start occurring regularly at say, 1 ms intervals. In that
# scenario, 15 seconds (a conservative number for 10s) would be good
# for TIME_TO_TRIGGER, and 2 ms (again conservative) would be good for
# TIME_BETWEEN_TRIGGERS.
#
# The default in this sample script is 0.025 seconds. This is to make

```

```

# the sample work for the LINE trigger used in this script when the
# oscilloscope is in Average, Segmented, and Equivalent Time modes to
# force a trigger off of the AC input line (:TRIGger:EDGE:SOURce LINE)
# which runs at 50 or 60 Hz in most of the world (1/50 Hz -> 20 ms, so
# use 25 ms to be conservative).
#
# The SCOPE_ACQUISITION_TIME_OUT calculation pads this by 1.1.
# -----

# =====
# Define a simple and fast function utilizing the blocking :DIGitize
# command in conjunction with *OPC?.
# -----
#
# Benefits of this method:
#
# - Fastest, compact
# - Only way for Average Acquisition type:
#   - The :SINGLe command does not do a complete average.
#   - Counting triggers with :RUN is much too slow.
# - Allows for easy synchronization with math functions
# - Don't have to deal with the status registers, which can be
#   confusing.
# - Potentially faster than polling_method(), for better throughput
# - Because it's faster, one can retrieve more accurate acquisition
#   times than with a polling method.
# - Works best for segmented memory if any post processing is done
#   on the oscilloscope, for example, measurements, lister, math,
#   as this does not come back until the processing is all done
#   - In this scenario, :DIGitize does not reduce the sample
#     rate or memory depth.
#
# Drawbacks of this method:
#
# - Usually does not fill acquisition memory to the maximum
#   available, usually only on-screen data.
#
# - May not be at the highest sample rate (compared with the
#   polling_method).
#
# - Requires a well-chosen, hard-set timeout that will cover the
#   time to arm, trigger, and finish acquisition.
#
# - Requires Exception handling and a device_clear() for a
#   possible timeout (no trigger event).
#
# - Socket connection cannot do device_clear()
#
# - Because :DIGitize is a "specialized form of the :RUN" command,
#   on these oscilloscopes, that results in:
#
#   - the sample rate MAY be reduced from using :SINGLe -
#     usually at longer time scales - typically only acquires
#     what is on screen, though at the fastest time scales,
#     more than on screen data may be acquired. Thus, for max
#     memory and max sample rate, use the polling_method(),

```

```

#           which uses :SINGLE.
#
# How it works:
#
# - The :DIGitize command is a blocking command, and thus, all
#   other SCPI commands are blocked until :DIGitize is completely
#   done. This includes any subsequent processing that is already
#   set up, such as math and measurements.
#
# KEY POINT: However, :DIGitize does not prevent additional
# commands from being sent to the queue or cause the remote
# program to wait. For example, if your program does something
# like:
#
#     KsInfiniiVisionX.write(":DIGitize")
#     print("Signal acquired.\n")
#
# The "Signal acquired" message will be written immediately
# after the :DIGitize is sent, not after the acquisition and
# processing is complete.
#
# To pause the program until the :DIGitize is complete, you must
# wait for a query result after the :DIGitize. For example, in
# this case:
#
#     query_result = KsInfiniiVisionX.query(":DIGitize;*OPC?")
#     print("Signal acquired.\n")
#
# The "Signal acquired" message will be written after the
# acquisition and processing is complete. The *OPC? query is
# appended to :DIGitize with a semi-colon (;), which
# essentially ties it to the same thread in the parser. It is
# immediately dealt with once :DIGitize finishes and gives a "1"
# back to the program (whether the program uses it or not),
# allowing the program to move on.
#
# Other Notes:
#
# - If you DO NOT know when a trigger will occur, you should set a
#   very long timeout.
#
# - The timeout should be adjusted before and after the :DIGitize
#   operation, though this is not absolutely required.
#
# - A :DIGitize can be aborted with a device clear, which also
#   stops the oscilloscope:
#   KsInfiniiVisionX.clear()
#
# - :DIGitize disables the anti-aliasing feature (sample rate
#   dither) on all InfiniiVision and InfiniiVision X-Series
#   oscilloscopes.
#
# - :DIGitize temporarily blocks the front panel, and all front
#   panel presses are queued until :DIGitize is done. So if you
#   change the vertical scale, it will not happen until the
#   acquisition is done.
#

```

```

# The exception is that the Run/Stop button on the front panel
# is NOT blocked (unless the front panel is otherwise locked by
# ":SYSTem:LOCK ON").
# =====
def blocking_method():

    KsInfiniiVisionX.timeout = SCOPE_ACQUISITION_TIME_OUT
    # Time in milliseconds (PyVISA uses ms) to wait for the
    # oscilloscope to arm, trigger, finish acquisition, and finish
    # any processing.
    #
    # Note that this is a property of the device interface,
    # KsInfiniiVisionX.
    #
    # If doing repeated acquisitions, this should be done BEFORE the
    # loop, and changed again after the loop if the goal is to
    # achieve the best throughput.

    print("Acquiring signal(s)...")
    # Set up a try/except block to catch a possible timeout and exit.
    try:
        KsInfiniiVisionX.query(":DIGitize;*OPC?")
        # Acquire the signal(s) with :DIGitize (blocking) and wait
        # until *OPC? comes back with a one.
        print("Signal acquired.")

        # Reset timeout back to what it was, GLOBAL_TOUT.
        KsInfiniiVisionX.timeout = GLOBAL_TOUT

    # Catch a possible timeout and exit.
    except Exception:
        print("The acquisition timed out, most likely due to no " \
              "trigger or improper setup causing no trigger. " \
              "Properly closing the oscilloscope connection and " \
              "exiting script.\n")
        KsInfiniiVisionX.clear() # Clear communications interface;
                                # A device clear also aborts digitize.
        KsInfiniiVisionX.close() # Close communications interface
        sys.exit("Exiting script.")

# =====
# Define a function using the non-blocking :SINGle command and polling
# on the Operation Status Condition Register
# -----
#
# Benefits of this method:
#
# - Don't have to worry about interface timeouts.
# - Easy to expand to know when the oscilloscope is armed.
# - Don't have to worry about interface timeouts
# - Easy to expand to know when scope is armed, and triggered
# - MAY result in a higher sample rate than the blocking method
# - Always fills max available memory
# - Can use with a socket connection if desired
#
#

```

```

# Drawbacks of this method:
#
# - Slow, as you don't want to poll the oscilloscope too fast.
#
# - DOES NOT work for Equivalent time mode. MUST use the blocking
#   method.
#
# - Slow
#
# - Does NOT work for Average Acquisition type
#   - :SINGLE does not do a complete average
#     - It does a single acquisition as if it were in NORMAL
#       acq. type
#     - Counting triggers in :RUN is much too slow
#
# - Works for Segmented Memory, BUT if any post processing is done
#   on the oscilloscope, for example measurements, lister, math, as
#   this reports that the acquisition is done, which is correct,
#   BUT the processing is NOT done, and it will take an indefinite
#   amount of time to wait for that, though there is no way to tell
#   if it is done. Use the blocking_method for Segmented Memory.
#
# - Can't be used effectively for synchronizing math functions
#
#   It can be done by applying an additional hard coded wait after
#   the acquisition is done. At least 200 ms is suggested, more
#   may be required.
#
#   However, as long as the timeout is not excessively short, the
#   math happens fast enough that once :OPERRegister:CONDition?
#   comes back as done that one can just wait for it when it is
#   time to pull the math waveform. The exception would be for eye
#   or jitter mode on a 6000 X-Series oscilloscope, where the
#   processing time can be long.
#
# - Still need some maximum timeout (here MAX_TIME_TO_WAIT),
#   ideally, or the script will sit in the while loop forever if
#   there is no trigger event
#
#   Max timeout (here MAX_TIME_TO_WAIT) must also account for any
#   processing done (see comments on math above)
#
#   Max timeout (here MAX_TIME_TO_WAIT) must also account for time
#   to arm the scope and finish the acquisition
#
#   This arm/trigger/finish part is accounted for in the main script.
#
# How it works:
#
# - What really matters is the RUN bit in the Operation Condition
#   (not Event) Register. This bit changes based on the
#   oscilloscope state.
#
#   If the oscilloscope is running, it is high (8), and low (0) if
#   it is stopped.
#
#   The only (best) way to get at this bit is with the

```

```

# :OPERation:CONDition? query. The Operation Condition Register
# can reflect states for other oscilloscope properties, for
# example, if the oscilloscope is armed, thus it can produce
# values other than 0 (stopped) or 8 (running).
#
# To handle that, the result of :OPERation:Condition? is bitwise
# ANDed (& in Python) with an 8. This is called "unmasking".
#
# Here, the "unmasking" is done in the script. On the other
# hand, it is possible to "mask" which bits get passed to the
# summary bit to the next register below on the instrument
# itself. However, this method it typically only used when
# working with the Status Byte, and not used here.
#
# - Why 8 = running = not done?
#
# The Run bit is the 4th bit of the Operation Status Condition
# (and Event) Registers.
#
# The registers are binary and start counting at zero, thus the
# 4th bit is bit number 3, and  $2^3 = 8$ , and thus it returns an
# 8 for high and a 0 for low.
#
# - Why the CONDITION and NOT the EVENT register?
#
# The Condition register reflects the CURRENT state, while the
# EVENT register reflects the first event that occurred since it
# was cleared or read (as in: has it EVER happened?), thus the
# CONDITION register is used.
#
# Note that with this method using :SINGle, for InfiniiVision
# X-Series oscilloscopes only, :SINGle itself forces the trigger
# sweep mode into NORMal. This does not happen with the blocking
# method, using :DIGitize, or on the InfiniiVision notXs.
# =====
def polling_method():

    MAX_TIME_TO_WAIT = SCOPE_ACQUISITION_TIME_OUT/float(1000)
    # Time in seconds to wait for the oscilloscope to arm, trigger,
    # and finish acquisition.
    #
    # Note that this is NOT a property of the device interface,
    # KsInfiniiVisionX, but rather some constant in the script to be
    # used later with the Python module "time", and will be used with
    # time.perf_counter().

    # Define "mask" bits.
    #
    # Mask condition for Run state in the Operation Status Condition
    # (and Event) Register.
    #
    # Refer to Programmer's Guide chapters on Status Reporting and
    # Synchronizing Acquisitions.
    RUN_BIT = 3
    # The run bit is the 4th bit (see next set of comments @
    # Completion Criteria).
    RUN_MASK = 1<<RUN_BIT

```

```

# This basically means: 2^3 = 8, or rather, in Python 2**3
# (<< is a left shift; left shift is fastest); this is used later
# to "unmask" the result of the Operation Status Event Register
# as there is no direct access to the RUN bit.

# Define completion criterion:
ACQ_DONE = 0
# Means the oscilloscope is stopped
ACQ_NOT_DONE = 1<<RUN_BIT
# Means the oscilloscope is running; value is 8. This is the
# 4th bit of the Operation Status Condition (and Event) Register.
# The registers are binary and start counting at zero, thus the
# 4th bit (4th position in a binary representation of decimal
# 8 = 2^3 = (1 left shift 3). This is either High (running = 8)
# or low (stopped and therefore done with acquisition = 0).

print("Acquiring signal(s)...")

# Define acquisition start time. This is in seconds.
StartTime = time.perf_counter()

# Begin Acquisition with *CLS and the non-blocking :SINGLe
# command, concatenated together. The *CLS clears all (non-mask)
# registers & sets them to 0;
KsInfiniiVisionX.write("*CLS;:SINGLe")

# Initialize the loop entry condition (assume Acq. is not done).
Acq_State = ACQ_NOT_DONE

# Poll the oscilloscope until Acq_State is a one. (This is NOT a
# "Serial Poll.")
while Acq_State == ACQ_NOT_DONE and (time.perf_counter() - StartTime
                                     <= MAX_TIME_TO_WAIT):

    # Ask oscilloscope if it is done with the acquisition via the
    # Operation Status Condition (not Event) Register.
    # The Condition register reflects the CURRENT state, while
    # the EVENT register reflects the first event that occurred
    # since it was cleared or read, thus the CONDITION register
    # is used.
    #
    # DO NOT do:
    # KsInfiniiVisionX.query("*CLS;SINGLe;OPERRegister:CONDition?")
    # as putting :OPERRegister:CONDition? right after :SINGLe
    # does not work reliably
    #
    # The oscilloscope SHOULD trigger, but it sits there with the
    # Single hard key on the oscilloscope lit yellow; pressing
    # this key causes a trigger.
    Status = int(KsInfiniiVisionX.query(":OPERRegister:CONDition?"))

    # Bitwise AND of the Status and RUN_MASK. This exposes ONLY
    # the 4th bit, which is either High (running = 8) or low
    # (stopped and therefore done with acquisition = 0)
    Acq_State = (Status & RUN_MASK)
    if Acq_State == ACQ_DONE:
        break # Break out of while loop so that the 100 ms pause

```



```

        # below is not incurred if done.
time.sleep(0.1)    # Pause 100 ms to prevent excessive queries
# This can actually be set a little faster, at 0.045.  The
# point here is that:
#
# 1.  If there are other things being controlled, going too
#     fast can tie up the bus.
# 2.  Going faster does not work on all scopes.  The
#     symptom of this not working is:
#
# The oscilloscope SHOULD trigger, but it sits there with the
# Single hard key on the oscilloscope lit yellow; pressing
# this key causes a trigger.
#
# The pause should be at the end of the loop, so that the
# oscilloscope is immediately asked if it is done.
#
# Loop exits when Acq_State != NOT_DONE, that is, it exits
# the loop when it is DONE or if the max wait time is
# exceeded.

if Acq_State == ACQ_DONE: # Acquisition fully completed
    print("Signal acquired.")
else: # Acquisition failed for some reason
    print("Max wait time exceeded.")
    print("This happens if there was no trigger event.")
    print("Adjust settings accordingly.\n")
    print("Properly closing oscilloscope connection and exiting " \
          "script.\n")
    KsInfiniiVisionX.clear() # Clear communications interface
    KsInfiniiVisionX.query(":STOP;*OPC?")
    # Stop the oscilloscope
    KsInfiniiVisionX.close() # Close communications interface
    sys.exit("Exiting script.")

# =====
# Do Something with data... save, export, additional analysis...
# =====
def do_something_with_data():

    # For example, make a peak-peak voltage measurement on channel 1:
    Vpp_Ch1 = \
        str(KsInfiniiVisionX.query("MEASure:VPP? CHANnel1")).strip("\n")
    # The result has a newline, so remove it with .strip("\n")
    print("Vpp Ch1 = " + Vpp_Ch1 + " V\n")

# =====
# Main code
# =====

# Connect and initialize oscilloscope
# -----
# Define VISA Resource Manager & Install directory
rm = pyvisa.ResourceManager('C:\\Windows\\System32\\visa64.dll')
```

```

# Define and open the oscilloscope using the VISA address
try:
    KsInfiniiVisionX = rm.open_resource(VISA_ADDRESS)
except Exception:
    print("Unable to connect to oscilloscope at " + str(VISA_ADDRESS)\
          + ". Aborting script.\n")
    sys.exit()

# Set the Global Timeout
KsInfiniiVisionX.timeout = GLOBAL_TOUT

# Clear the instrument bus
KsInfiniiVisionX.clear()

# Clear all status registers and errors
KsInfiniiVisionX.write("*CLS")

try:

    # Set up the oscilloscope
    # -----
    # Note that you would normally perform a reset (default setup) if
    # you were to create the setup from scratch... But here we will
    # use the oscilloscope "as is" for the most part.
    # KsInfiniiVisionX.query("*RST;*OPC?")    # Resets the oscilloscope

    # Always stop the oscilloscope when making any changes.
    KsInfiniiVisionX.query(":STOP;*OPC?")

    # For this example, the oscilloscope will be forced to trigger on
    # the # (AC input power) LINE voltage so something happens.
    # Always use normal trigger sweep, never auto.
    KsInfiniiVisionX.write(":TRIGger:SWEep NORMal")
    # This line simply gives the oscilloscope something to trigger on.
    KsInfiniiVisionX.query(":TRIGger:EDGE:SOURce LINE;*OPC?")

    # Clear the display (so you can see the waveform being acquired -
    # otherwise, there is no need for this).
    KsInfiniiVisionX.write(":CDISplay")

    # Calculate acquisition timeout/wait time by short, overestimate
    # method
    # -----

    # Create some default variables
    N_AVERAGES = 1
    N_SEGMENTS = 1

    # Get some info about the scope time base setup
    HO          = float(KsInfiniiVisionX.query(":TRIGger:HOLDoff?"))
    T_RANGE     = float(KsInfiniiVisionX.query(":TIMEbase:RANGe?"))
    T_POSITION  = float(KsInfiniiVisionX.query(":TIMEbase:POSition?"))

    # Determine Acquisition Type and Mode:
    ACQ_TYPE = str(KsInfiniiVisionX.query(":ACquire:TYPE?").strip("\n"))
    ACQ_MODE = str(KsInfiniiVisionX.query(":ACquire:MODE?").strip("\n"))

```

```

if ACQ_MODE == "SEGM":
    N_SEGMENTS = \
        float(KsInfiniiVisionX.query(":ACquire:SEGmented:COUNT?"))
    # Note that if there are a lot of analysis associated segments,
    # for example, serial data decode, the timeout will likely need
    # to be longer than calculated.
    #
    # You are encouraged to manually set up the oscilloscope in
    # this case, as it will be used, time it, and use that, with
    # a little overhead.
    #
    # Blocking method is recommended for Segmented Memory mode.
elif ACQ_TYPE == "AVER":
    N_AVERAGES = float(KsInfiniiVisionX.query(":ACquire:COUNT?"))

# Calculate acquisition timeout by overestimate method:
# Recall that PyVISA timeouts are in ms, so multiply by 1000.
SCOPE_ACQUISITION_TIME_OUT = (float(TIME_TO_TRIGGER)*1.1 +
    (T_RANGE*2.0 + abs(T_POSITION)*2.0 + HO*1.1 +
    float(TIME_BETWEEN_TRIGGERS)*1.1)*N_SEGMENTS*N_AVERAGES)*1000.0

## Ensure the timeout is no less than 10 seconds
if SCOPE_ACQUISITION_TIME_OUT < 10000.0:
    SCOPE_ACQUISITION_TIME_OUT = 10000.0

# What about Equivalent Time Mode and other odd modes such as
# Jitter or Eye (the last two only being found on the
# 6000 X-Series), and math functions?
#
# In most cases, the padding and 10 second minimum timeout will
# take care of this.
#
# Equivalent Time Mode has effects only at the fastest time
# scales, so it really does not make a difference as long as a
# trigger signal is present. If trigger signal occurs rarely,
# adjust the TIME_BETWEEN_TRIGGERS constant accordingly.
#
# For math, the math will happen fast enough that the "padding"
# in the timeout calculation takes care of this.
#
# For jitter mode on the 6000 X-Series, you can try this method,
# typically there is always a signal present, and the 10 second
# minimum should work out. If not, make it bigger, or increase
# padding.
#
# For Eye mode on the 6000 X-Series, none of this works anyway,
# and you have to use :RUN (or :RTEye:ACquire) and :STOP.

# Acquire Signal
# -----
# Choose blocking_method or polling_method. These were defined as
# functions in case you want to use them repeatedly.
blocking_method()
# If Acquisition Type is Average, always use blocking_method() to
# get complete average.
do_something_with_data()

```

```

polling_method()
do_something_with_data()

# Done - cleanup
KsInfiniiVisionX.clear() # Clear communications interface
KsInfiniiVisionX.close() # Close communications interface
except KeyboardInterrupt:
    KsInfiniiVisionX.clear()
    KsInfiniiVisionX.query(":STOP;*OPC?")
    KsInfiniiVisionX.write(":SYSTem:LOCK OFF")
    KsInfiniiVisionX.clear()
    KsInfiniiVisionX.close()
    sys.exit("User Interrupt. Properly closing oscilloscope and "
            "aborting script.")
except Exception:
    KsInfiniiVisionX.clear()
    KsInfiniiVisionX.query(":STOP;*OPC?")
    KsInfiniiVisionX.write(":SYSTem:LOCK OFF")
    KsInfiniiVisionX.clear()
    KsInfiniiVisionX.close()
    sys.exit("Something went wrong. Properly closing oscilloscope "
            "and aborting script.")

# End of Script
# -----
print("Done.")

```

45 More About Oscilloscope Commands

Command Classifications / 1650

Valid Command/Query Strings / 1651

Query Return Values / 1657

Command Classifications

To help you use existing programs with your oscilloscope, or use current programs with the next generation of Keysight InfiniiVision oscilloscopes, commands are classified by the following categories:

- **"Core Commands"** on page 1650
- **"Non-Core Commands"** on page 1650
- **"Obsolete Commands"** on page 1650

C Core Commands

Core commands are a common set of commands that provide basic oscilloscope functionality on this oscilloscope and future Keysight InfiniiVision oscilloscopes. Core commands are unlikely to be modified in the future. If you restrict your programs to core commands, the programs should work across product offerings in the future, assuming appropriate programming methods are employed.

N Non-Core Commands

Non-core commands are commands that provide specific features, but are not universal across all Keysight InfiniiVision oscilloscope models. Non-core commands may be modified or deleted in the future. With a command structure as complex as the one for your oscilloscope, some evolution over time is inevitable. Keysight's intent is to continue to expand command subsystems, such as the rich and evolving trigger feature set.

O Obsolete Commands

Obsolete commands are older forms of commands that are provided to reduce customer rework for existing systems and programs. Generally, these commands are mapped onto some of the Core and Non-core commands, but may not strictly have the same behavior as the new command. None of the obsolete commands are guaranteed to remain functional in future products. New systems and programs should use the Core (and Non-core) commands. Obsolete commands are listed in:

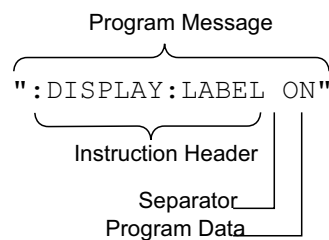
- **Chapter 41**, "Obsolete and Discontinued Commands," starting on page 1531

Valid Command/Query Strings

- **"Program Message Syntax"** on page 1651
- **"Duplicate Mnemonics"** on page 1655
- **"Tree Traversal Rules and Multiple Commands"** on page 1655

Program Message Syntax

To program the instrument remotely, you must understand the command format and structure expected by the instrument. The IEEE 488.2 syntax rules govern how individual elements such as headers, separators, program data, and terminators may be grouped together to form complete instructions. Syntax definitions are also given to show how query responses are formatted. The following figure shows the main syntactical parts of a typical program statement.



Instructions (both commands and queries) normally appear as a string embedded in a statement of your host language, such as Visual Basic or C/C++. The only time a parameter is not meant to be expressed as a string is when the instruction's syntax definition specifies <block data>, such as <learn string>. There are only a few instructions that use block data.

Program messages can have long or short form commands (and data in some cases – see **"Long Form to Short Form Truncation Rules"** on page 1652), and upper and/or lower case ASCII characters may be used. (Query responses, however, are always returned in upper case.)

Instructions are composed of two main parts:

- The header, which specifies the command or query to be sent.
- The program data, which provide additional information needed to clarify the meaning of the instruction.

Instruction Header

The instruction header is one or more mnemonics separated by colons (:) that represent the operation to be performed by the instrument.

`:DISPlay:LABel ON` is a command. Queries are indicated by adding a question mark (?) to the end of the header, for example, `:DISPlay:LABel?`. Many instructions can be used as either commands or queries, depending on whether or

not you have included the question mark. The command and query forms of an instruction usually have different program data. Many queries do not use any program data.

There are three types of headers:

- **"Simple Command Headers"** on page 1653
- **"Compound Command Headers"** on page 1653
- **"Common Command Headers"** on page 1653

White Space (Separator) White space is used to separate the instruction header from the program data. If the instruction does not require any program data parameters, you do not need to include any white space. White space is defined as one or more space characters. ASCII defines a space to be character 32 (in decimal).

Program Data Program data are used to clarify the meaning of the command or query. They provide necessary information, such as whether a function should be on or off, or which waveform is to be displayed. Each instruction's syntax definition shows the program data, as well as the values they accept. **"Program Data Syntax Rules"** on page 1654 describes all of the general rules about acceptable values.

When there is more than one data parameter, they are separated by commas(,). Spaces can be added around the commas to improve readability.

Program Message Terminator The program instructions within a data message are executed after the program message terminator is received. The terminator may be either an NL (New Line) character, an EOI (End-Of-Identify) asserted in the programming interface, or a combination of the two. Asserting the EOI sets the EOI control line low on the last byte of the data message. The NL character is an ASCII linefeed (decimal 10).

NOTE

New Line Terminator Functions. The NL (New Line) terminator has the same function as an EOS (End Of String) and EOT (End Of Text) terminator.

Long Form to Short Form Truncation Rules

To get the short form of a command/keyword:

- When the command/keyword is longer than four characters, use the first four characters of the command/keyword unless the fourth character is a vowel; when the fourth character is a vowel, use the first three characters of the command/keyword.
- When the command/keyword is four or fewer characters, use all of the characters.

Long Form	Short form
RANGe	RANG
PATTerN	PATT

Long Form	Short form
TIMEbase	TIM
DElay	DEL
TYPE	TYPE

In the oscilloscope programmer's documentation, the short form of a command is indicated by uppercase characters.

Programs written in long form are easily read and are almost self-documenting. The short form syntax conserves the amount of controller memory needed for program storage and reduces I/O activity.

Simple Command Headers

Simple command headers contain a single mnemonic. :AUToscale and :DIGitize are examples of simple command headers typically used in the oscilloscope. The syntax is:

```
<program mnemonic><terminator>
```

Simple command headers must occur at the beginning of a program message; if not, they must be preceded by a colon.

When program data must be included with the simple command header (for example, :DIGitize CHANnel1), white space is added to separate the data from the header. The syntax is:

```
<program mnemonic><separator><program data><terminator>
```

Compound Command Headers

Compound command headers are a combination of two or more program mnemonics. The first mnemonic selects the subsystem, and the second mnemonic selects the function within that subsystem. The mnemonics within the compound message are separated by colons. For example, to execute a single function within a subsystem:

```
:<subsystem>:<function><separator><program data><terminator>
```

For example, :CHANnel1:BWLimit ON

Common Command Headers

Common command headers control IEEE 488.2 functions within the instrument (such as clear status). Their syntax is:

```
*<command header><terminator>
```

No space or separator is allowed between the asterisk (*) and the command header. *CLS is an example of a common command header.

Program Data Syntax Rules

Program data is used to convey a parameter information related to the command header. At least one space must separate the command header or query header from the program data.

```
<program mnemonic><separator><data><terminator>
```

When a program mnemonic or query has multiple program data, a comma separates sequential program data.

```
<program mnemonic><separator><data>,<data><terminator>
```

For example, :MEASure:DElay CHANnel1,CHANnel2 has two program data: CHANnel1 and CHANnel2.

Two main types of program data are used in commands: character and numeric.

Character Program Data

Character program data is used to convey parameter information as alpha or alphanumeric strings. For example, the :TIMEbase:MODE command can be set to normal, zoomed (delayed), XY, or ROLL. The character program data in this case may be MAIN, WINDow, XY, or ROLL. The command :TIMEbase:MODE WINDow sets the time base mode to zoomed.

The available mnemonics for character program data are always included with the command's syntax definition.

When sending commands, you may either the long form or short form (if one exists). Uppercase and lowercase letters may be mixed freely.

When receiving query responses, uppercase letters are used exclusively.

Numeric Program Data

Some command headers require program data to be expressed numerically. For example, :TIMEbase:RANGe requires the desired full scale range to be expressed numerically.

For numeric program data, you have the option of using exponential notation or using suffix multipliers to indicate the numeric value. The following numbers are all equal:

```
28 = 0.28E2 = 280e-1 = 28000m = 0.028K = 28e-3K.
```

When a syntax definition specifies that a number is an integer, that means that the number should be whole. Any fractional part will be ignored, truncating the number. Numeric data parameters accept fractional values are called real numbers.

All numbers must be strings of ASCII characters. Thus, when sending the number 9, you would send a byte representing the ASCII code for the character 9 (which is 57). A three-digit number like 102 would take up three bytes (ASCII codes 49, 48, and 50). This is handled automatically when you include the entire instruction in a string.

Duplicate Mnemonics

Identical function mnemonics can be used in more than one subsystem. For example, the function mnemonic RANGE may be used to change the vertical range or to change the horizontal range:

```
:CHANnel1:RANGe .4
```

Sets the vertical range of channel 1 to 0.4 volts full scale.

```
:TIMebase:RANGe 1
```

Sets the horizontal time base to 1 second full scale.

:CHANnel1 and :TIMebase are subsystem selectors and determine which range is being modified.

Tree Traversal Rules and Multiple Commands

Command headers are created by traversing down the command tree. A legal command header would be :TIMebase:RANGe. This is referred to as a *compound header*. A compound header is a header made of two or more mnemonics separated by colons. The mnemonic created contains no spaces.

The following rules apply to traversing the tree:

- A leading colon (<NL> or EOI true on the last byte) places the parser at the root of the command tree. A leading colon is a colon that is the first character of a program header. Executing a subsystem command lets you access that subsystem until a leading colon or a program message terminator (<NL>) or EOI true is found.
- In the command tree, use the last mnemonic in the compound header as the reference point (for example, RANGE). Then find the last colon above that mnemonic (TIMebase:). That is the point where the parser resides. Any command below that point can be sent within the current program message without sending the mnemonics which appear above them (for example, POSition).

The output statements in the examples are written using the Keysight VISA COM library in Visual Basic. The quoted string is placed on the bus, followed by a carriage return and linefeed (CRLF).

To execute more than one function within the same subsystem, separate the functions with a semicolon (;):

```
:<subsystem>:<function><separator><data>;<function><separator><data><terminator>
```

For example:

```
myScope.WriteString ":TIMebase:RANGe 0.5;POSition 0"
```

NOTE

The colon between TIMEbase and RANGE is necessary because TIMEbase:RANGE is a compound command. The semicolon between the RANGE command and the POSition command is the required program message unit separator. The POSition command does not need TIMEbase preceding it because the TIMEbase:RANGE command sets the parser to the TIMEbase node in the tree.

Example 2:
Program Message
Terminator Sets
Parser Back to
Root

```
myScope.WriteString ":TIMEbase:REfERENCE CENTER;POSition 0.00001"
or
myScope.WriteString ":TIMEbase:REfERENCE CENTER"
myScope.WriteString ":TIMEbase:POSition 0.00001"
```

NOTE

In the first line of example 2, the subsystem selector is implied for the POSition command in the compound command. The POSition command must be in the same program message as the REfERENCE command because the program message terminator places the parser back at the root of the command tree.

A second way to send these commands is by placing TIMEbase: before the POSition command as shown in the second part of example 2. The space after POSition is required.

Example 3:
Selecting Multiple
Subsystems

You can send multiple program commands and program queries for different subsystems on the same line by separating each command with a semicolon. The colon following the semicolon enables you to enter a new subsystem. For example:

```
<program mnemonic><data>;:<program mnemonic><data><terminator>
```

For example:

```
myScope.WriteString ":TIMEbase:REfERENCE CENTER;:DISPlay:VECTors ON"
```

NOTE

The leading colon before DISPlay:VECTors ON tells the parser to go back to the root of the command tree. The parser can then see the DISPlay:VECTors ON command. The space between REfERENCE and CENTer is required; so is the space between VECTors and ON.

Multiple commands may be any combination of compound and simple commands.

Query Return Values

Command headers immediately followed by a question mark (?) are queries. Queries are used to get results of measurements made by the instrument or to find out how the instrument is currently configured.

After receiving a query, the instrument interrogates the requested function and places the answer in its output queue. The answer remains in the output queue until it is read or another command is issued.

When read, the answer is transmitted across the bus to the designated listener (typically a controller). For example, the query :TIMEbase:RANGe? places the current time base setting in the output queue. When using the Keysight VISA COM library in Visual Basic, the controller statements:

```
Dim strQueryResult As String
myScope.WriteString ":TIMEbase:RANGe?"
strQueryResult = myScope.ReadString
```

pass the value across the bus to the controller and place it in the variable strQueryResult.

NOTE

Read Query Results Before Sending Another Command. Sending another command or query before reading the result of a query clears the output buffer (the current response) and places a Query INTERRUPTED error in the error queue.

Infinity Representation

The representation of infinity is +9.9E+37. This is also the value returned when a measurement cannot be made.

46 Programming Examples

VISA COM Examples / 1660

VISA Examples / 1694

VISA.NET Examples / 1741

SICL Examples / 1760

SCPI.NET Examples / 1780

The example programs in this manual are ASCII text files that can be cut from the help file and pasted into your favorite text editor.

- See Also**
- You can find additional programming examples for the InfiniiVision 4000G X-Series oscilloscopes on the Keysight Technologies website at:
www.keysight.com/find/4000GX-Series-examples

VISA COM Examples

- ["VISA COM Example in Visual Basic"](#) on page 1660
- ["VISA COM Example in C#" on page 1669](#)
- ["VISA COM Example in Visual Basic .NET"](#) on page 1678
- ["VISA COM Example in Python"](#) on page 1686

VISA COM Example in Visual Basic

To run this example in Visual Basic for Applications (VBA):

- 1 Start the application that provides Visual Basic for Applications (for example, Microsoft Excel).
- 2 Press ALT+F11 to launch the Visual Basic editor.
- 3 Reference the Keysight VISA COM library:
 - a Choose **Tools > References...** from the main menu.
 - b In the References dialog, check the "VISA COM 5.9 Type Library".
 - c Click **OK**.
- 4 Choose **Insert > Module**.
- 5 Cut-and-paste the code that follows into the editor.
- 6 Edit the program to use the VISA address of your oscilloscope, and save the changes.
- 7 Run the program.

```
'
' Keysight VISA COM Example in Visual Basic
' -----
' This program illustrates a few commonly-used programming
' features of your Keysight oscilloscope.
' -----

Option Explicit

Public myMgr As VisaComLib.ResourceManager
Public myScope As VisaComLib.FormattedIO488
Public varQueryResult As Variant
Public strQueryResult As String

' For Sleep subroutine.
Private Declare Sub Sleep Lib "kernel32" (ByVal dwMilliseconds As Long)

'
' Main Program
' -----

Sub Main()
```



```

On Error GoTo VisaComError

' Create the VISA COM I/O resource.
Set myMgr = New VisaComLib.ResourceManager
Set myScope = New VisaComLib.FormattedIO488
Set myScope.IO = _
    myMgr.Open("USB0::0x0957::0x17A6::US50210029::0::INSTR")
myScope.IO.Clear ' Clear the interface.
myScope.IO.Timeout = 10000 ' Set I/O communication timeout.

' Initialize - start from a known state.
Initialize

' Capture data.
Capture

' Analyze the captured waveform.
Analyze

Exit Sub

VisaComError:
    MsgBox "VISA COM Error:" + vbCrLf + Err.Description
    End

End Sub

'
' Initialize the oscilloscope to a known state.
' -----

Private Sub Initialize()

    On Error GoTo VisaComError

    ' Get and display the device's *IDN? string.
    strQueryResult = DoQueryString("*IDN?")
    Debug.Print "Identification string: " + strQueryResult

    ' Clear status and load the default setup.
    DoCommand "*CLS"
    DoCommand "*RST"

    Exit Sub

VisaComError:
    MsgBox "VISA COM Error:" + vbCrLf + Err.Description
    End

End Sub

'
' Capture the waveform.
' -----

Private Sub Capture()

```

```

On Error GoTo VisaComError

' Use auto-scale to automatically configure oscilloscope.
' -----
DoCommand ":AUToscale"

' Set trigger mode (EDGE, PULSe, PATtern, etc., and input source.
DoCommand ":TRIGger:MODE EDGE"
Debug.Print "Trigger mode: " + _
    DoQueryString(":TRIGger:MODE?")

' Set EDGE trigger parameters.
DoCommand ":TRIGger:EDGE:SOURce CHANnel1"
Debug.Print "Trigger edge source: " + _
    DoQueryString(":TRIGger:EDGE:SOURce?")

DoCommand ":TRIGger:EDGE:LEVel 1.5"
Debug.Print "Trigger edge level: " + _
    DoQueryString(":TRIGger:EDGE:LEVel?")

DoCommand ":TRIGger:EDGE:SLOPe POSitive"
Debug.Print "Trigger edge slope: " + _
    DoQueryString(":TRIGger:EDGE:SLOPe?")

' Save oscilloscope configuration.
' -----
varQueryResult = DoQueryIEEEBlock_UI1(":SYSTem:SETup?")

' Output setup string to a file:
Dim strPath As String
strPath = "c:\scope\config\setup.dat"
Dim hFile As Long
hFile = FreeFile
Open strPath For Binary Access Write Lock Write As hFile
Put hFile, , varQueryResult ' Write data.
Close hFile ' Close file.
Debug.Print "Setup bytes saved: " + CStr(LenB(varQueryResult))

' Change settings with individual commands:
' -----

' Set vertical scale and offset.
DoCommand ":CHANnel1:SCALe 0.05"
Debug.Print "Channel 1 vertical scale: " + _
    DoQueryString(":CHANnel1:SCALe?")

DoCommand ":CHANnel1:OFFSet -1.5"
Debug.Print "Channel 1 vertical offset: " + _
    DoQueryString(":CHANnel1:OFFSet?")

' Set horizontal scale and offset.
DoCommand ":TIMEbase:SCALe 0.0002"
Debug.Print "Timebase scale: " + _
    DoQueryString(":TIMEbase:SCALe?")

DoCommand ":TIMEbase:POSition 0.0"
Debug.Print "Timebase position: " + _

```

```

        DoQueryString(":TIMEbase:POSition?")

' Set the acquisition type (NORMAL, PEAK, AVERage, or HRESolution).
DoCommand ":ACQuire:TYPE NORMAL"
Debug.Print "Acquire type: " + _
    DoQueryString(":ACQuire:TYPE?")

' Or, configure by loading a previously saved setup.
' -----
Dim varSetupString As Variant
strPath = "c:\scope\config\setup.dat"
Open strPath For Binary Access Read As hFile ' Open file for input.
Get hFile, , varSetupString ' Read data.
Close hFile ' Close file.
' Write learn string back to oscilloscope using ":SYSTem:SETup"
' command:
DoCommandIEEEBlock ":SYSTem:SETup", varSetupString
Debug.Print "Setup bytes restored: " + CStr(LenB(varSetupString))

' Capture an acquisition using :DIGitize.
' -----
DoCommand ":DIGitize CHANnel1"

Exit Sub

VisaComError:
    MsgBox "VISA COM Error:" + vbCrLf + Err.Description
End

End Sub

'
' Analyze the captured waveform.
' -----

Private Sub Analyze()

    On Error GoTo VisaComError

' Make a couple of measurements.
' -----
DoCommand ":MEASure:SOURce CHANnel1"
Debug.Print "Measure source: " + _
    DoQueryString(":MEASure:SOURce?")

DoCommand ":MEASure:FREquency"
varQueryResult = DoQueryNumber(":MEASure:FREquency?")
MsgBox "Frequency:" + vbCrLf + _
    FormatNumber(varQueryResult / 1000, 4) + " kHz"

DoCommand ":MEASure:VAMPlitude"
varQueryResult = DoQueryNumber(":MEASure:VAMPlitude?")
MsgBox "Vertical amplitude:" + vbCrLf + _
    FormatNumber(varQueryResult, 4) + " V"

' Download the screen image.

```

```

' -----
' Get screen image.
DoCommand ":HARDcopy:INKSaver OFF"
Dim byteData() As Byte
byteData = DoQueryIEEEBlock_UI1(":DISPlay:DATA? PNG, COlor")

' Save screen image to a file.
Dim strPath As String
strPath = "c:\scope\data\screen.png"
If Len(Dir(strPath)) Then
    Kill strPath ' Remove file if it exists.
End If

Dim hFile As Long
hFile = FreeFile
Open strPath For Binary Access Write Lock Write As hFile
Put hFile, , byteData ' Write data.
Close hFile ' Close file.
MsgBox "Screen image (" + CStr(UBound(byteData) + 1) + _
    " bytes) written to " + strPath

' Download waveform data.
' -----

' Set the waveform points mode.
DoCommand ":WAVeform:POINts:MODE RAW"
Debug.Print "Waveform points mode: " + _
    DoQueryString(":WAVeform:POINts:MODE?")

' Get the number of waveform points available.
Debug.Print "Waveform points available: " + _
    DoQueryString(":WAVeform:POINts?")

' Set the waveform source.
DoCommand ":WAVeform:SOURce CHANnel1"
Debug.Print "Waveform source: " + _
    DoQueryString(":WAVeform:SOURce?")

' Choose the format of the data returned (WORD, BYTE, ASCII):
DoCommand ":WAVeform:FORMat BYTE"
Debug.Print "Waveform format: " + _
    DoQueryString(":WAVeform:FORMat?")

' Display the waveform settings:
Dim Preamble()
Dim intFormat As Integer
Dim intType As Integer
Dim lngPoints As Long
Dim lngCount As Long
Dim dblXIncrement As Double
Dim dblXOrigin As Double
Dim lngXReference As Long
Dim sngYIncrement As Single
Dim sngYOrigin As Single
Dim lngYReference As Long

```

```

Preamble() = DoQueryNumbers(":WAVEform:PREamble?")

intFormat = Preamble(0)
intType = Preamble(1)
lngPoints = Preamble(2)
lngCount = Preamble(3)
dblXIncrement = Preamble(4)
dblXOrigin = Preamble(5)
lngXReference = Preamble(6)
sngYIncrement = Preamble(7)
sngYOrigin = Preamble(8)
lngYReference = Preamble(9)

If intFormat = 0 Then
    Debug.Print "Waveform format: BYTE"
ElseIf intFormat = 1 Then
    Debug.Print "Waveform format: WORD"
ElseIf intFormat = 4 Then
    Debug.Print "Waveform format: ASCII"
End If

If intType = 0 Then
    Debug.Print "Acquisition type: NORMAl"
ElseIf intType = 1 Then
    Debug.Print "Acquisition type: PEAK"
ElseIf intType = 2 Then
    Debug.Print "Acquisition type: AVERage"
ElseIf intType = 3 Then
    Debug.Print "Acquisition type: HRESolution"
End If

Debug.Print "Waveform points: " + _
    FormatNumber(lngPoints, 0)

Debug.Print "Waveform average count: " + _
    FormatNumber(lngCount, 0)

Debug.Print "Waveform X increment: " + _
    Format(dblXIncrement, "Scientific")

Debug.Print "Waveform X origin: " + _
    Format(dblXOrigin, "Scientific")

Debug.Print "Waveform X reference: " + _
    FormatNumber(lngXReference, 0)

Debug.Print "Waveform Y increment: " + _
    Format(sngYIncrement, "Scientific")

Debug.Print "Waveform Y origin: " + _
    FormatNumber(sngYOrigin, 0)

Debug.Print "Waveform Y reference: " + _
    FormatNumber(lngYReference, 0)

' Get the waveform data
varQueryResult = DoQueryIEEEBlock_UI1(":WAVEform:DATA?")

```

```

Debug.Print "Number of data values: " + _
    CStr(UBound(varQueryResult) + 1)

' Set up output file:
strPath = "c:\scope\data\waveform_data.csv"

' Open file for output.
Open strPath For Output Access Write Lock Write As hFile

' Output waveform data in CSV format.
Dim lngDataValue As Long
Dim lngI As Long

For lngI = 0 To UBound(varQueryResult)
    lngDataValue = varQueryResult(lngI)

    ' Write time value, voltage value.
    Print #hFile, _
        FormatNumber(dblXOrigin + (lngI * dblXIncrement), 9) + _
        ", " + _
        FormatNumber(((lngDataValue - lngYReference) * _
            sngYIncrement) + sngYOrigin)

Next lngI

' Close output file.
Close hFile ' Close file.
MsgBox "Waveform format BYTE data written to " + _
    "c:\scope\data\waveform_data.csv."

Exit Sub

VisaComError:
MsgBox "VISA COM Error:" + vbCrLf + Err.Description
End

End Sub

Private Sub DoCommand(command As String)

    On Error GoTo VisaComError

    myScope.WriteString command
    CheckInstrumentErrors

Exit Sub

VisaComError:
MsgBox "VISA COM Error: " + vbCrLf + CStr(Err.Number) + ", " + _
    Err.Source + ", " + _
    Err.Description, vbExclamation, "VISA COM Error"
End

End Sub

Private Sub DoCommandIEEEBlock(command As String, data As Variant)

```

```

    On Error GoTo VisaComError

    Dim strErrors As String

    myScope.WriteIEEEBlock command, data
    CheckInstrumentErrors

    Exit Sub

VisaComError:
    MsgBox "VISA COM Error: " + vbCrLf + CStr(Err.Number) + ", " + _
        Err.Source + ", " + _
        Err.Description, vbExclamation, "VISA COM Error"
    End

End Sub

Private Function DoQueryString(query As String) As String

    On Error GoTo VisaComError

    myScope.WriteString query
    DoQueryString = myScope.ReadString
    CheckInstrumentErrors

    Exit Function

VisaComError:
    MsgBox "VISA COM Error: " + vbCrLf + CStr(Err.Number) + ", " + _
        Err.Source + ", " + _
        Err.Description, vbExclamation, "VISA COM Error"
    End

End Function

Private Function DoQueryNumber(query As String) As Variant

    On Error GoTo VisaComError

    myScope.WriteString query
    DoQueryNumber = myScope.ReadNumber
    CheckInstrumentErrors

    Exit Function

VisaComError:
    MsgBox "VISA COM Error: " + vbCrLf + CStr(Err.Number) + ", " + _
        Err.Source + ", " + _
        Err.Description, vbExclamation, "VISA COM Error"
    End

End Function

Private Function DoQueryNumbers(query As String) As Variant()

    On Error GoTo VisaComError

```

```

    Dim strErrors As String

    myScope.WriteString query
    DoQueryNumbers = myScope.ReadList
    CheckInstrumentErrors

    Exit Function

VisaComError:
    MsgBox "VISA COM Error: " + vbCrLf + CStr(Err.Number) + ", " + _
        Err.Source + ", " + _
        Err.Description, vbExclamation, "VISA COM Error"
    End

End Function

Private Function DoQueryIEEEBlock_UI1(query As String) As Variant

    On Error GoTo VisaComError

    myScope.WriteString query
    DoQueryIEEEBlock_UI1 = myScope.ReadIEEEBlock(BinaryType_UI1)
    CheckInstrumentErrors

    Exit Function

VisaComError:
    MsgBox "VISA COM Error: " + vbCrLf + CStr(Err.Number) + ", " + _
        Err.Source + ", " + _
        Err.Description, vbExclamation, "VISA COM Error"
    End

End Function

Private Sub CheckInstrumentErrors()

    On Error GoTo VisaComError

    Dim strErrVal As String
    Dim strOut As String

    myScope.WriteString ":SYSTem:ERror?" ' Query any errors data.
    strErrVal = myScope.ReadString ' Read: Errnum,"Error String".
    While Val(strErrVal) <> 0 ' End if find: 0,"No Error".
        strOut = strOut + "INST Error: " + strErrVal
        myScope.WriteString ":SYSTem:ERror?" ' Request error message.
        strErrVal = myScope.ReadString ' Read error message.
    Wend

    If Not strOut = "" Then
        MsgBox strOut, vbExclamation, "INST Error Messages"
        myScope.FlushWrite (False)
        myScope.FlushRead
    End If

    Exit Sub

```



```

VisaComError:
    MsgBox "VISA COM Error: " + vbCrLf + Err.Description

End Sub

```

VISA COM Example in C#

To compile and run this example in Microsoft Visual Studio 2008:

- 1 Open Visual Studio.
- 2 Create a new Visual C#, Windows, Console Application project.
- 3 Cut-and-paste the code that follows into the C# source file.
- 4 Edit the program to use the VISA address of your oscilloscope.
- 5 Add a reference to the VISA COM 5.9 Type Library:
 - a Right-click the project you wish to modify (not the solution) in the Solution Explorer window of the Microsoft Visual Studio environment.
 - b Choose **Add Reference...**
 - c In the Add Reference dialog, select the **COM** tab.
 - d Select **VISA COM 5.9 Type Library**; then click **OK**.
- 6 Build and run the program.

For more information, see the VISA COM Help that comes with Keysight IO Libraries Suite.

```

/*
 * Keysight VISA COM Example in C#
 * -----
 * This program illustrates a few commonly used programming
 * features of your Keysight oscilloscope.
 * -----
 */

using System;
using System.IO;
using System.Text;
using Ivi.Visa.Interop;
using System.Runtime.InteropServices;

namespace InfiniiVision
{
    class VisaComInstrumentApp
    {
        private static VisaComInstrument myScope;

        public static void Main(string[] args)
        {
            try
            {
                myScope = new

```

```

        VisaComInstrument("USB0::0x0957::0x17A6::US50210029::0::INSTR"
);
myScope.SetTimeoutSeconds(10);

// Initialize - start from a known state.
Initialize();

// Capture data.
Capture();

// Analyze the captured waveform.
Analyze();
}
catch (System.ApplicationException err)
{
    Console.WriteLine("*** VISA COM Error : " + err.Message);
}
catch (System.SystemException err)
{
    Console.WriteLine("*** System Error Message : " + err.Message);
}
catch (System.Exception err)
{
    System.Diagnostics.Debug.Fail("Unexpected Error");
    Console.WriteLine("*** Unexpected Error : " + err.Message);
}
finally
{
    myScope.Close();
}
}

/*
 * Initialize the oscilloscope to a known state.
 * -----
 */
private static void Initialize()
{
    string strResults;

    // Get and display the device's *IDN? string.
    strResults = myScope.DoQueryString("*IDN?");
    Console.WriteLine("*IDN? result is: {0}", strResults);

    // Clear status and load the default setup.
    myScope.DoCommand("*CLS");
    myScope.DoCommand("*RST");
}

/*
 * Capture the waveform.
 * -----
 */
private static void Capture()
{
    // Use auto-scale to automatically configure oscilloscope.
    myScope.DoCommand(":AUTOScale");
}

```

```

// Set trigger mode (EDGE, PULSe, PATtern, etc., and input source.
myScope.DoCommand(":TRIGger:MODE EDGE");
Console.WriteLine("Trigger mode: {0}",
    myScope.DoQueryString(":TRIGger:MODE?"));

// Set EDGE trigger parameters.
myScope.DoCommand(":TRIGger:EDGE:SOURce CHANnel1");
Console.WriteLine("Trigger edge source: {0}",
    myScope.DoQueryString(":TRIGger:EDGE:SOURce?"));

myScope.DoCommand(":TRIGger:EDGE:LEVel 1.5");
Console.WriteLine("Trigger edge level: {0}",
    myScope.DoQueryString(":TRIGger:EDGE:LEVel?"));

myScope.DoCommand(":TRIGger:EDGE:SLOPe POSitive");
Console.WriteLine("Trigger edge slope: {0}",
    myScope.DoQueryString(":TRIGger:EDGE:SLOPe?"));

// Save oscilloscope configuration.
byte[] ResultsArray; // Results array.
int nLength; // Number of bytes returned from instrument.
string strPath;

// Query and read setup string.
ResultsArray = myScope.DoQueryIEEEBlock(":SYSTem:SETup?");
nLength = ResultsArray.Length;

// Write setup string to file.
strPath = "c:\\scope\\config\\setup.stp";
FileStream fStream = File.Open(strPath, FileMode.Create);
fStream.Write(ResultsArray, 0, nLength);
fStream.Close();
Console.WriteLine("Setup bytes saved: {0}", nLength);

// Change settings with individual commands:

// Set vertical scale and offset.
myScope.DoCommand(":CHANnel1:SCALe 0.05");
Console.WriteLine("Channel 1 vertical scale: {0}",
    myScope.DoQueryString(":CHANnel1:SCALe?"));

myScope.DoCommand(":CHANnel1:OFFSet -1.5");
Console.WriteLine("Channel 1 vertical offset: {0}",
    myScope.DoQueryString(":CHANnel1:OFFSet?"));

// Set horizontal scale and offset.
myScope.DoCommand(":TIMebase:SCALe 0.0002");
Console.WriteLine("Timebase scale: {0}",
    myScope.DoQueryString(":TIMebase:SCALe?"));

myScope.DoCommand(":TIMebase:POSition 0.0");
Console.WriteLine("Timebase position: {0}",
    myScope.DoQueryString(":TIMebase:POSition?"));

// Set the acquisition type (NORMal, PEAK, AVERage, or HRESolution
).

```

```

myScope.DoCommand(":ACquire:TYPE NORMal");
Console.WriteLine("Acquire type: {0}",
    myScope.DoQueryString(":ACquire:TYPE?"));

// Or, configure by loading a previously saved setup.
byte[] DataArray;
int nBytesWritten;

// Read setup string from file.
strPath = "c:\\scope\\config\\setup.stp";
DataArray = File.ReadAllBytes(strPath);
nBytesWritten = DataArray.Length;

// Restore setup string.
myScope.DoCommandIEEEBlock(":SYSTem:SETup", DataArray);
Console.WriteLine("Setup bytes restored: {0}", nBytesWritten);

// Capture an acquisition using :DIGitize.
myScope.DoCommand(":DIGitize CHANnel1");
}

/*
 * Analyze the captured waveform.
 * -----
 */
private static void Analyze()
{
    byte[] ResultsArray;    // Results array.
    int nLength;           // Number of bytes returned from instrument.
    string strPath;

    // Make a couple of measurements.
    // -----
    myScope.DoCommand(":MEASure:SOURce CHANnel1");
    Console.WriteLine("Measure source: {0}",
        myScope.DoQueryString(":MEASure:SOURce?"));

    double fResult;
    myScope.DoCommand(":MEASure:FREQuency");
    fResult = myScope.DoQueryNumber(":MEASure:FREQuency?");
    Console.WriteLine("Frequency: {0:F4} kHz", fResult / 1000);

    myScope.DoCommand(":MEASure:VAMplitude");
    fResult = myScope.DoQueryNumber(":MEASure:VAMplitude?");
    Console.WriteLine("Vertical amplitude: {0:F2} V", fResult);

    // Download the screen image.
    // -----
    myScope.DoCommand(":HARDcopy:INKSaver OFF");

    // Get the screen data.
    ResultsArray =
        myScope.DoQueryIEEEBlock(":DISPlay:DATA? PNG, COlor");
    nLength = ResultsArray.Length;

    // Store the screen data to a file.
    strPath = "c:\\scope\\data\\screen.png";

```

```

FileStream fStream = File.Open(strPath, FileMode.Create);
fStream.Write(ResultsArray, 0, nLength);
fStream.Close();
Console.WriteLine("Screen image ({0} bytes) written to {1}",
    nLength, strPath);

// Download waveform data.
// -----

// Set the waveform points mode.
myScope.DoCommand(":WAVEform:POINts:MODE RAW");
Console.WriteLine("Waveform points mode: {0}",
    myScope.DoQueryString(":WAVEform:POINts:MODE?"));

// Get the number of waveform points available.
Console.WriteLine("Waveform points available: {0}",
    myScope.DoQueryString(":WAVEform:POINts?"));

// Set the waveform source.
myScope.DoCommand(":WAVEform:SOURce CHANnel1");
Console.WriteLine("Waveform source: {0}",
    myScope.DoQueryString(":WAVEform:SOURce?"));

// Choose the format of the data returned (WORD, BYTE, ASCII):
myScope.DoCommand(":WAVEform:FORMat BYTE");
Console.WriteLine("Waveform format: {0}",
    myScope.DoQueryString(":WAVEform:FORMat?"));

// Display the waveform settings:
double[] fResultsArray;
fResultsArray = myScope.DoQueryNumbers(":WAVEform:PREamble?");

double fFormat = fResultsArray[0];
if (fFormat == 0.0)
{
    Console.WriteLine("Waveform format: BYTE");
}
else if (fFormat == 1.0)
{
    Console.WriteLine("Waveform format: WORD");
}
else if (fFormat == 2.0)
{
    Console.WriteLine("Waveform format: ASCii");
}

double fType = fResultsArray[1];
if (fType == 0.0)
{
    Console.WriteLine("Acquire type: NORMal");
}
else if (fType == 1.0)
{
    Console.WriteLine("Acquire type: PEAK");
}
else if (fType == 2.0)
{

```

```

        Console.WriteLine("Acquire type: AVERage");
    }
    else if (fType == 3.0)
    {
        Console.WriteLine("Acquire type: HRESolution");
    }

    double fPoints = fResultsArray[2];
    Console.WriteLine("Waveform points: {0:e}", fPoints);

    double fCount = fResultsArray[3];
    Console.WriteLine("Waveform average count: {0:e}", fCount);

    double fXincrement = fResultsArray[4];
    Console.WriteLine("Waveform X increment: {0:e}", fXincrement);

    double fXorigin = fResultsArray[5];
    Console.WriteLine("Waveform X origin: {0:e}", fXorigin);

    double fXreference = fResultsArray[6];
    Console.WriteLine("Waveform X reference: {0:e}", fXreference);

    double fYincrement = fResultsArray[7];
    Console.WriteLine("Waveform Y increment: {0:e}", fYincrement);

    double fYorigin = fResultsArray[8];
    Console.WriteLine("Waveform Y origin: {0:e}", fYorigin);

    double fYreference = fResultsArray[9];
    Console.WriteLine("Waveform Y reference: {0:e}", fYreference);

    // Read waveform data.
    ResultsArray = myScope.DoQueryIEEEBlock(":WAVEform:DATA?");
    nLength = ResultsArray.Length;
    Console.WriteLine("Number of data values: {0}", nLength);

    // Set up output file:
    strPath = "c:\\scope\\data\\waveform_data.csv";
    if (File.Exists(strPath)) File.Delete(strPath);

    // Open file for output.
    StreamWriter writer = File.CreateText(strPath);

    // Output waveform data in CSV format.
    for (int i = 0; i < nLength - 1; i++)
        writer.WriteLine("{0:f9}, {1:f6}",
            fXorigin + ((float)i * fXincrement),
            (((float)ResultsArray[i] - fYreference)
            * fYincrement) + fYorigin);

    // Close output file.
    writer.Close();
    Console.WriteLine("Waveform format BYTE data written to {0}",
        strPath);
}
}

```

```

class VisaComInstrument
{
    private ResourceManagerClass m_ResourceManager;
    private FormattedIO488Class m_IoObject;
    private string m_strVisaAddress;

    // Constructor.
    public VisaComInstrument(string strVisaAddress)
    {
        // Save VISA address in member variable.
        m_strVisaAddress = strVisaAddress;

        // Open the default VISA COM IO object.
        OpenIo();

        // Clear the interface.
        m_IoObject.IO.Clear();
    }

    public void DoCommand(string strCommand)
    {
        // Send the command.
        m_IoObject.WriteString(strCommand, true);

        // Check for inst errors.
        CheckInstrumentErrors(strCommand);
    }

    public void DoCommandIEEEBlock(string strCommand,
        byte[] DataArray)
    {
        // Send the command to the device.
        m_IoObject.WriteIEEEBlock(strCommand, DataArray, true);

        // Check for inst errors.
        CheckInstrumentErrors(strCommand);
    }

    public string DoQueryString(string strQuery)
    {
        // Send the query.
        m_IoObject.WriteString(strQuery, true);

        // Get the result string.
        string strResults;
        strResults = m_IoObject.ReadString();

        // Check for inst errors.
        CheckInstrumentErrors(strQuery);

        // Return results string.
        return strResults;
    }

    public double DoQueryNumber(string strQuery)
    {
        // Send the query.

```

```

        m_IoObject.WriteString(strQuery, true);

        // Get the result number.
        double fResult;
        fResult = (double)m_IoObject.ReadNumber(
            IEEEASCIIType.ASCIIType_R8, true);

        // Check for inst errors.
        CheckInstrumentErrors(strQuery);

        // Return result number.
        return fResult;
    }

    public double[] DoQueryNumbers(string strQuery)
    {
        // Send the query.
        m_IoObject.WriteString(strQuery, true);

        // Get the result numbers.
        double[] fResultsArray;
        fResultsArray = (double[])m_IoObject.ReadList(
            IEEEASCIIType.ASCIIType_R8, ",;");

        // Check for inst errors.
        CheckInstrumentErrors(strQuery);

        // Return result numbers.
        return fResultsArray;
    }

    public byte[] DoQueryIEEEBlock(string strQuery)
    {
        // Send the query.
        m_IoObject.WriteString(strQuery, true);

        // Get the results array.
        System.Threading.Thread.Sleep(2000); // Delay before reading.
        byte[] ResultsArray;
        ResultsArray = (byte[])m_IoObject.ReadIEEEBlock(
            IEEEBinaryType.BinaryType_UI1, false, true);

        // Check for inst errors.
        CheckInstrumentErrors(strQuery);

        // Return results array.
        return ResultsArray;
    }

    private void CheckInstrumentErrors(string strCommand)
    {
        // Check for instrument errors.
        string strInstrumentError;
        bool bFirstError = true;

        do    // While not "0,No error".
        {

```



```

        m_IoObject.WriteString(":SYSTem:ERRor?", true);
        strInstrumentError = m_IoObject.ReadString();

        if (!strInstrumentError.ToString().StartsWith("+0,"))
        {
            if (bFirstError)
            {
                Console.WriteLine("ERROR(s) for command '{0}': ",
                                   strCommand);
                bFirstError = false;
            }
            Console.Write(strInstrumentError);
        }
    } while (!strInstrumentError.ToString().StartsWith("+0,"));
}

private void OpenIo()
{
    m_ResourceManager = new ResourceManagerClass();
    m_IoObject = new FormattedIO488Class();

    // Open the default VISA COM IO object.
    try
    {
        m_IoObject.IO =
            (IMessage)m_ResourceManager.Open(m_strVisaAddress,
                                             AccessMode.NO_LOCK, 0, "");
    }
    catch (Exception e)
    {
        Console.WriteLine("An error occurred: {0}", e.Message);
    }
}

public void SetTimeoutSeconds(int nSeconds)
{
    m_IoObject.IO.Timeout = nSeconds * 1000;
}

public void Close()
{
    try
    {
        m_IoObject.IO.Close();
    }
    catch { }

    try
    {
        Marshal.ReleaseComObject(m_IoObject);
    }
    catch { }

    try
    {
        Marshal.ReleaseComObject(m_ResourceManager);
    }
}

```

```

        catch { }
    }
}

```

VISA COM Example in Visual Basic .NET

To compile and run this example in Microsoft Visual Studio 2008:

- 1 Open Visual Studio.
- 2 Create a new Visual Basic, Windows, Console Application project.
- 3 Cut-and-paste the code that follows into the Visual Basic .NET source file.
- 4 Edit the program to use the VISA address of your oscilloscope.
- 5 Add a reference to the VISA COM 5.9 Type Library:
 - a Right-click the project you wish to modify (not the solution) in the Solution Explorer window of the Microsoft Visual Studio environment.
 - b Choose **Add Reference....**
 - c In the Add Reference dialog, select the **COM** tab.
 - d Select **VISA COM 5.9 Type Library**; then click **OK**.
 - e Right-click the project you wish to modify (not the solution) in the Solution Explorer window of the Microsoft Visual Studio environment and choose **Properties**; then, select "InfiniiVision.VisaComInstrumentApp" as the **Startup object**.
- 6 Build and run the program.

For more information, see the VISA COM Help that comes with Keysight IO Libraries Suite.

```

'
' Keysight VISA COM Example in Visual Basic .NET
' -----
' This program illustrates a few commonly used programming
' features of your Keysight oscilloscope.
' -----

Imports System
Imports System.IO
Imports System.Text
Imports Ivi.Visa.Interop
Imports System.Runtime.InteropServices

Namespace InfiniiVision
    Class VisaComInstrumentApp
        Private Shared myScope As VisaComInstrument

        Public Shared Sub Main(ByVal args As String())
            Try
                myScope = New _
                    VisaComInstrument("USB0::0x0957::0x17A6::US50210029::0::INSTR"

```

```

)

    myScope.SetTimeoutSeconds(10)

    ' Initialize - start from a known state.
    Initialize()

    ' Capture data.
    Capture()

    ' Analyze the captured waveform.
    Analyze()

Catch err As System.ApplicationException
    Console.WriteLine("*** VISA Error Message : " + err.Message)
Catch err As System.SystemException
    Console.WriteLine("*** System Error Message : " + err.Message)
Catch err As System.Exception
    System.Diagnostics.Debug.Fail("Unexpected Error")
    Console.WriteLine("*** Unexpected Error : " + err.Message)
Finally
    myScope.Close()
End Try
End Sub

' Initialize the oscilloscope to a known state.
' -----

Private Shared Sub Initialize()
    Dim strResults As String

    ' Get and display the device's *IDN? string.
    strResults = myScope.DoQueryString("*IDN?")
    Console.WriteLine("*IDN? result is: {0}", strResults)

    ' Clear status and load the default setup.
    myScope.DoCommand("*CLS")
    myScope.DoCommand("*RST")

End Sub

' Capture the waveform.
' -----

Private Shared Sub Capture()

    ' Use auto-scale to automatically configure oscilloscope.
    myScope.DoCommand(":AUTOScale")

    ' Set trigger mode (EDGE, PULSe, PATtern, etc., and input source.
    myScope.DoCommand(":TRIGger:MODE EDGE")
    Console.WriteLine("Trigger mode: {0}", _
        myScope.DoQueryString(":TRIGger:MODE?"))

    ' Set EDGE trigger parameters.
    myScope.DoCommand(":TRIGger:EDGE:SOURce CHANnel1")
    Console.WriteLine("Trigger edge source: {0}", _
        myScope.DoQueryString(":TRIGger:EDGE:SOURce?"))

```

```

myScope.DoCommand(":TRIGger:EDGE:LEVel 1.5")
Console.WriteLine("Trigger edge level: {0}", _
    myScope.DoQueryString(":TRIGger:EDGE:LEVel?"))

myScope.DoCommand(":TRIGger:EDGE:SLOPe POSitive")
Console.WriteLine("Trigger edge slope: {0}", _
    myScope.DoQueryString(":TRIGger:EDGE:SLOPe?"))

' Save oscilloscope configuration.
Dim ResultsArray As Byte() ' Results array.
Dim nLength As Integer ' Number of bytes returned from inst.
Dim strPath As String
Dim fStream As FileStream

' Query and read setup string.
ResultsArray = myScope.DoQueryIEEEBlock(":SYSTem:SETup?")
nLength = ResultsArray.Length

' Write setup string to file.
strPath = "c:\scope\config\setup.stp"
fStream = File.Open(strPath, FileMode.Create)
fStream.Write(ResultsArray, 0, nLength)
fStream.Close()
Console.WriteLine("Setup bytes saved: {0}", nLength)

' Change settings with individual commands:

' Set vertical scale and offset.
myScope.DoCommand(":CHANnel1:SCALe 0.05")
Console.WriteLine("Channel 1 vertical scale: {0}", _
    myScope.DoQueryString(":CHANnel1:SCALe?"))

myScope.DoCommand(":CHANnel1:OFFSet -1.5")
Console.WriteLine("Channel 1 vertical offset: {0}", _
    myScope.DoQueryString(":CHANnel1:OFFSet?"))

' Set horizontal scale and offset.
myScope.DoCommand(":TIMebase:SCALe 0.0002")
Console.WriteLine("Timebase scale: {0}", _
    myScope.DoQueryString(":TIMebase:SCALe?"))

myScope.DoCommand(":TIMebase:POSition 0.0")
Console.WriteLine("Timebase position: {0}", _
    myScope.DoQueryString(":TIMebase:POSition?"))

' Set the acquisition type (NORMal, PEAK, AVERage, or HRESolution)

myScope.DoCommand(":ACQuire:TYPE NORMal")
Console.WriteLine("Acquire type: {0}", _
    myScope.DoQueryString(":ACQuire:TYPE?"))

' Or, configure by loading a previously saved setup.
Dim DataArray As Byte()
Dim nBytesWritten As Integer

' Read setup string from file.

```

```

strPath = "c:\scope\config\setup.stp"
dataArray = File.ReadAllBytes(strPath)
nBytesWritten = dataArray.Length

' Restore setup string.
myScope.DoCommandIEEEBlock(":SYSTem:SETup", dataArray)
Console.WriteLine("Setup bytes restored: {0}", nBytesWritten)

' Capture an acquisition using :DIGitize.
myScope.DoCommand(":DIGitize CHANnel1")

End Sub

' Analyze the captured waveform.
' -----

Private Shared Sub Analyze()

    Dim fResult As Double
    Dim ResultsArray As Byte() ' Results array.
    Dim nLength As Integer ' Number of bytes returned from inst.
    Dim strPath As String

    ' Make a couple of measurements.
    ' -----
    myScope.DoCommand(":MEASure:SOURce CHANnel1")
    Console.WriteLine("Measure source: {0}", _
        myScope.DoQueryString(":MEASure:SOURce?"))

    myScope.DoCommand(":MEASure:FREQuency")
    fResult = myScope.DoQueryNumber(":MEASure:FREQuency?")
    Console.WriteLine("Frequency: {0:F4} kHz", fResult / 1000)

    myScope.DoCommand(":MEASure:VAMPlitude")
    fResult = myScope.DoQueryNumber(":MEASure:VAMPlitude?")
    Console.WriteLine("Vertical amplitude: {0:F2} V", fResult)

    ' Download the screen image.
    ' -----
    myScope.DoCommand(":HARDcopy:INKSaver OFF")

    ' Get the screen data.
    ResultsArray = myScope.DoQueryIEEEBlock(":DISPlay:DATA? PNG, COLor
")
    nLength = ResultsArray.Length

    ' Store the screen data to a file.
    strPath = "c:\scope\data\screen.png"
    Dim fStream As FileStream
    fStream = File.Open(strPath, FileMode.Create)
    fStream.Write(ResultsArray, 0, nLength)
    fStream.Close()
    Console.WriteLine("Screen image ({0} bytes) written to {1}", _
        nLength, strPath)

    ' Download waveform data.
    ' -----

```

```

' Set the waveform points mode.
myScope.DoCommand(":WAVEform:POINTs:MODE RAW")
Console.WriteLine("Waveform points mode: {0}", _
    myScope.DoQueryString(":WAVEform:POINTs:MODE?"))

' Get the number of waveform points available.
Console.WriteLine("Waveform points available: {0}", _
    myScope.DoQueryString(":WAVEform:POINTs?"))

' Set the waveform source.
myScope.DoCommand(":WAVEform:SOURce CHANnel1")
Console.WriteLine("Waveform source: {0}", _
    myScope.DoQueryString(":WAVEform:SOURce?"))

' Choose the format of the data returned (WORD, BYTE, ASCII):
myScope.DoCommand(":WAVEform:FORMat BYTE")
Console.WriteLine("Waveform format: {0}", _
    myScope.DoQueryString(":WAVEform:FORMat?"))

' Display the waveform settings:
Dim fResultsArray As Double()
fResultsArray = myScope.DoQueryNumbers(":WAVEform:PREamble?")

Dim fFormat As Double = fResultsArray(0)
If fFormat = 0 Then
    Console.WriteLine("Waveform format: BYTE")
ElseIf fFormat = 1 Then
    Console.WriteLine("Waveform format: WORD")
ElseIf fFormat = 2 Then
    Console.WriteLine("Waveform format: ASCii")
End If

Dim fType As Double = fResultsArray(1)
If fType = 0 Then
    Console.WriteLine("Acquire type: NORMAl")
ElseIf fType = 1 Then
    Console.WriteLine("Acquire type: PEAK")
ElseIf fType = 2 Then
    Console.WriteLine("Acquire type: AVERAge")
ElseIf fType = 3 Then
    Console.WriteLine("Acquire type: HRESolution")
End If

Dim fPoints As Double = fResultsArray(2)
Console.WriteLine("Waveform points: {0:e}", fPoints)

Dim fCount As Double = fResultsArray(3)
Console.WriteLine("Waveform average count: {0:e}", fCount)

Dim fXincrement As Double = fResultsArray(4)
Console.WriteLine("Waveform X increment: {0:e}", fXincrement)

Dim fXorigin As Double = fResultsArray(5)
Console.WriteLine("Waveform X origin: {0:e}", fXorigin)

Dim fXreference As Double = fResultsArray(6)

```

```

Console.WriteLine("Waveform X reference: {0:e}", fXreference)

Dim fYincrement As Double = fResultsArray(7)
Console.WriteLine("Waveform Y increment: {0:e}", fYincrement)

Dim fYorigin As Double = fResultsArray(8)
Console.WriteLine("Waveform Y origin: {0:e}", fYorigin)

Dim fYreference As Double = fResultsArray(9)
Console.WriteLine("Waveform Y reference: {0:e}", fYreference)

' Get the waveform data.
ResultsArray = myScope.DoQueryIEEEBlock(":WAVEform:DATA?")
nLength = ResultsArray.Length
Console.WriteLine("Number of data values: {0}", nLength)

' Set up output file:
strPath = "c:\scope\data\waveform_data.csv"
If File.Exists(strPath) Then
    File.Delete(strPath)
End If

' Open file for output.
Dim writer As StreamWriter = File.CreateText(strPath)

' Output waveform data in CSV format.
For index As Integer = 0 To nLength - 1
    ' Write time value, voltage value.
    writer.WriteLine("{0:f9}, {1:f6}", _
        fXorigin + (CSng(index) * fXincrement), _
        ((CSng(ResultsArray(index)) - fYreference) _
        * fYincrement) + fYorigin)
Next

' Close output file.
writer.Close()
Console.WriteLine("Waveform format BYTE data written to {0}", _
    strPath)

End Sub

End Class

Class VisaComInstrument
    Private m_ResourceManager As ResourceManagerClass
    Private m_IOException As FormattedIO488Class
    Private m_strVisaAddress As String

    ' Constructor.
    Public Sub New(ByVal strVisaAddress As String)

        ' Save VISA address in member variable.
        m_strVisaAddress = strVisaAddress

        ' Open the default VISA COM IO object.
        OpenIo()
    End Sub
End Class

```

```

        ' Clear the interface.
        m_IoObject.IO.Clear()

End Sub

Public Sub DoCommand(ByVal strCommand As String)

    ' Send the command.
    m_IoObject.WriteString(strCommand, True)

    ' Check for inst errors.
    CheckInstrumentErrors(strCommand)

End Sub

Public Sub DoCommandIEEEBlock(ByVal strCommand As String, _
    ByVal dataArray As Byte())

    ' Send the command to the device.
    m_IoObject.WriteIEEEBlock(strCommand, dataArray, True)

    ' Check for inst errors.
    CheckInstrumentErrors(strCommand)

End Sub

Public Function DoQueryString(ByVal strQuery As String) As String
    ' Send the query.
    m_IoObject.WriteString(strQuery, True)

    ' Get the result string.
    Dim strResults As String
    strResults = m_IoObject.ReadString()

    ' Check for inst errors.
    CheckInstrumentErrors(strQuery)

    ' Return results string.
    Return strResults
End Function

Public Function DoQueryNumber(ByVal strQuery As String) As Double
    ' Send the query.
    m_IoObject.WriteString(strQuery, True)

    ' Get the result number.
    Dim fResult As Double
    fResult = _
        CDbl(m_IoObject.ReadNumber(IEEEASCIIType.ASCIIType_R8, True))

    ' Check for inst errors.
    CheckInstrumentErrors(strQuery)

    ' Return result number.
    Return fResult
End Function

```



```

Public Function DoQueryNumbers(ByVal strQuery As String) As _
    Double()
    ' Send the query.
    m_IoObject.WriteString(strQuery, True)

    ' Get the result numbers.
    Dim fResultsArray As Double()
    fResultsArray = _
        m_IoObject.ReadList(IEEEASCIIType.ASCIIType_R8, ",;")

    ' Check for inst errors.
    CheckInstrumentErrors(strQuery)

    ' Return result numbers.
    Return fResultsArray
End Function

Public _
    Function DoQueryIEEEBlock(ByVal strQuery As String) As Byte()
    ' Send the query.
    m_IoObject.WriteString(strQuery, True)

    ' Get the results array.
    System.Threading.Thread.Sleep(2000) ' Delay before reading data.
    Dim ResultsArray As Byte()
    ResultsArray = _
        m_IoObject.ReadIEEEBlock(IEEEBinaryType.BinaryType_UI1, _
            False, True)

    ' Check for inst errors.
    CheckInstrumentErrors(strQuery)

    ' Return results array.
    Return ResultsArray
End Function

Private Sub CheckInstrumentErrors(ByVal strCommand As String)
    ' Check for instrument errors.
    Dim strInstrumentError As String
    Dim bFirstError As Boolean = True
    Do ' While not "0,No error".
        m_IoObject.WriteString(":SYSTem:ERRor?", True)
        strInstrumentError = m_IoObject.ReadString()

        If Not strInstrumentError.ToString().StartsWith("+0,") Then
            If bFirstError Then
                Console.WriteLine("ERROR(s) for command '{0}': ", _
                    strCommand)
                bFirstError = False
            End If
            Console.WriteLine(strInstrumentError)
        End If
    Loop While Not strInstrumentError.ToString().StartsWith("+0,")
End Sub

Private Sub OpenIo()
    m_ResourceManager = New ResourceManagerClass()

```

```

m_IOException = New FormattedIOException()

' Open the default VISA COM IO object.
Try
    m_IOException.IO = _
        DirectCast(m_ResourceManager.Open(m_strVisaAddress, _
            AccessMode.NO_LOCK, 0, ""), IMessage)
Catch e As Exception
    Console.WriteLine("An error occurred: {0}", e.Message)
End Try
End Sub

Public Sub SetTimeoutSeconds(ByVal nSeconds As Integer)
    m_IOException.IO.Timeout = nSeconds * 1000
End Sub

Public Sub Close()
    Try
        m_IOException.IO.Close()
    Catch
    End Try

    Try
        Marshal.ReleaseComObject(m_IOException)
    Catch
    End Try

    Try
        Marshal.ReleaseComObject(m_ResourceManager)
    Catch
    End Try
End Sub
End Class
End Namespace

```

VISA COM Example in Python

You can use the Python programming language with the "comtypes" package to control Keysight oscilloscopes.

The Python language and "comtypes" package can be downloaded from the web at <http://www.python.org/> and <https://pypi.org/project/comtypes/>, respectively.

To run this example with Python and "comtypes":

- 1 Cut-and-paste the code that follows into a file named "example.py".
- 2 Edit the program to use the VISA address of your oscilloscope.
- 3 If "python.exe" can be found via your PATH environment variable, open a Command Prompt window; then, change to the folder that contains the "example.py" file, and enter:

```

python example.py

#!python3
# -*- coding: utf-8 -*-

```

```

# Keysight VISA COM Example in Python using "comtypes"
# *****
# This program illustrates a few commonly-used programming
# features of your Keysight oscilloscope.
# *****

# Import Python modules.
# -----
import sys
import array

from comtypes.client import GetModule
from comtypes.client import CreateObject

# Run GetModule once to generate comtypes.gen.VisaComLib.
if not hasattr(sys, "frozen"):
    GetModule("C:\Program Files (x86)\IVI Foundation\VISA\VisaCom\
GlobMgr.dll")

import comtypes.gen.VisaComLib as VisaComLib

# Global variables.
# -----
input_channel = "CHANnel1"
input_channel = "CHANnel2"
setup_file_name = "setup.scp"
screen_image_file_name = "screen_image.png"
waveform_data_file_name = "waveform_data.csv"
# wfm_fmt = "BYTE"
wfm_fmt = "WORD"

# =====
# Initialize:
# =====
def initialize():

    # Get and display the device's *IDN? string.
    idn_string = do_query_string("*IDN?")
    print(f"Identification string: '{idn_string}'")

    # Clear status and load the default setup.
    do_command("*CLS")
    do_command("*RST")

# =====
# Capture:
# =====
def capture():

    # Use auto-scale to automatically set up oscilloscope.
    print("Autoscale.")
    do_command(":AUToscale")

```

```

# Set trigger mode.
do_command(":TRIGger:MODE EDGE")
qresult = do_query_string(":TRIGger:MODE?")
print(f"Trigger mode: {qresult}")

# Set EDGE trigger parameters.
do_command(f":TRIGger:EDGE:SOURce {input_channel}")
qresult = do_query_string(":TRIGger:EDGE:SOURce?")
print(f"Trigger edge source: {qresult}")

do_command(":TRIGger:EDGE:LEVel 1.5")
qresult = do_query_string(":TRIGger:EDGE:LEVel?")
print(f"Trigger edge level: {qresult}")

do_command(":TRIGger:EDGE:SLOPe POSitive")
qresult = do_query_string(":TRIGger:EDGE:SLOPe?")
print(f"Trigger edge slope: {qresult}")

# Save oscilloscope setup.
setup_bytes = do_query_ieee_block(":SYSTem:SETup?")
setup_bytes_length = len(setup_bytes)
f = open(setup_file_name, "wb")
f.write(bytearray(setup_bytes))
f.close()
print(f"Setup bytes saved: {setup_bytes_length}")

# Change oscilloscope settings with individual commands:

# Set vertical scale and offset.
do_command(f":{input_channel}:SCALE 0.05")
qresult = do_query_string(f":{input_channel}:SCALE?")
print(f"{input_channel} vertical scale: {qresult}")

do_command(f":{input_channel}:OFFSet -1.5")
qresult = do_query_string(f":{input_channel}:OFFSet?")
print(f"{input_channel} offset: {qresult}")

# Set horizontal scale and offset.
do_command(":TIMEbase:SCALE 0.0002")
qresult = do_query_string(":TIMEbase:SCALE?")
print(f"Timebase scale: {qresult}")

do_command(":TIMEbase:POSition 0.0")
qresult = do_query_string(":TIMEbase:POSition?")
print(f"Timebase position: {qresult}")

# Set the acquisition type.
do_command(":ACQuire:TYPE NORMal")
qresult = do_query_string(":ACQuire:TYPE?")
print(f"Acquire type: {qresult}")

# Or, set up oscilloscope by loading a previously saved setup.
f = open(setup_file_name, "rb")
setup_bytes = f.read()
f.close()
do_command_ieee_block(":SYSTem:SETup", array.array('B', setup_bytes))
print(f"Setup bytes restored: {len(setup_bytes)}")

```

```

# Capture an acquisition using :DIGitize.
do_command(f":DIGitize {input_channel}")

# =====
# Analyze:
# =====
def analyze():

    # Make measurements.
    # -----
    do_command(f":MEASure:SOURce {input_channel}")
    gresult = do_query_string(":MEASure:SOURce?")
    print(f"Measure source: {gresult}")

    do_command(":MEASure:FREQuency")
    gresult = do_query_string(":MEASure:FREQuency?")
    print(f"Measured frequency on {input_channel}: {gresult}")

    do_command(":MEASure:VAMPlitude")
    gresult = do_query_string(":MEASure:VAMPlitude?")
    print(f"Measured vertical amplitude on {input_channel}: {gresult}")

    # Download the screen image.
    # -----
    do_command(":HARDcopy:INKSaver OFF")

    screen_bytes = do_query_ieee_block(":DISPlay:DATA? PNG, COlor")

    # Save display data values to file.
    f = open(screen_image_file_name, "wb")
    f.write(bytearray(screen_bytes))
    f.close()
    print(f"Screen image written to {screen_image_file_name}.")

    # Download waveform data.
    # -----

    # Set the waveform source.
    do_command(f":WAVEform:SOURce {input_channel}")
    gresult = do_query_string(":WAVEform:SOURce?")
    print(f"Waveform source: {gresult}")

    # Set the waveform points mode.
    do_command(":WAVEform:POINts:MODE RAW")
    gresult = do_query_string(":WAVEform:POINts:MODE?")
    print(f"Waveform points mode: {gresult}")

    # Get the number of waveform points available.
    do_command(":WAVEform:POINts 10240")
    gresult = do_query_string(":WAVEform:POINts?")
    print(f"Waveform points available: {gresult}")

    # Choose the format of the data returned (BYTE or WORD):
    do_command(f":WAVEform:FORMat {wfm_fmt}")
    gresult = do_query_string(":WAVEform:FORMat?")

```

```

print(f"Waveform format: {qresult}")

# Display the waveform settings from preamble:
wav_form_dict = {
    0 : "BYTE",
    1 : "WORD",
    4 : "AScii",
}
acq_type_dict = {
    0 : "NORMAl",
    1 : "PEAK",
    2 : "AVERage",
    3 : "HRESolution",
}

(
    wav_form,
    acq_type,
    wfmpmts,
    avgcnt,
    x_increment,
    x_origin,
    x_reference,
    y_increment,
    y_origin,
    y_reference
) = do_query_numbers(":WAVeform:PREamble?")

print(f"Waveform format: {wav_form_dict[int(wav_form)]}")
print(f"Acquire type: {acq_type_dict[int(acq_type)]}")
print(f"Waveform points desired: {wfmpmts}")
print(f"Waveform average count: {avgcnt}")
print(f"Waveform X reference: {x_reference}")    # Always 0.

# Get numeric values for later calculations.
x_increment = do_query_number(":WAVeform:XINCrement?")
print(f"Waveform X increment: {x_increment}")
x_origin = do_query_number(":WAVeform:XORigin?")
print(f"Waveform X origin: {x_origin}")
y_increment = do_query_number(":WAVeform:YINCrement?")
print(f"Waveform Y increment: {y_increment}")
y_origin = do_query_number(":WAVeform:YORigin?")
print(f"Waveform Y origin: {y_origin}")
y_reference = do_query_number(":WAVeform:YREference?")
print(f"Waveform Y reference: {y_reference}")

# Specify the byte order in WORD data.
if wfm_fmt == "WORD":
    do_command(":WAVeform:BYTeorder LSBF")
    qresult = do_query_string(":WAVeform:BYTeorder?")
    print(f"Waveform byte order for WORD data: {qresult}")

# Get the waveform data bytes.
data_bytes = do_query_ieee_block(":WAVeform:DATA?")
data_bytes_length = len(data_bytes)
print(f"Byte count: {data_bytes_length}")

```

```

# Get the waveform data values.
if wfm_fmt == "BYTE":
    values = data_bytes
elif wfm_fmt == "WORD":
    data_bytes_array = array.array('B', data_bytes)    # Array of unsigned
bytes.
    values = array.array('H')    # Interpret as array of unsigned shorts.
    values.frombytes(data_bytes_array)
print(f"Number of data values: {len(values)}")

# Save waveform data values to CSV file.
f = open(waveform_data_file_name, "w")

for i in range(0, len(values) - 1):
    time_val = x_origin + (i * x_increment)
    voltage = ((values[i] - y_reference) * y_increment) + y_origin
    f.write(f"{time_val:E}, {voltage:f}\n")

# Close output file.
f.close()
print(f"Waveform format {wfm_fmt} data written to {waveform_data_file_n
ame}.")

# =====
# Send a command and check for errors:
# =====
def do_command(command):
    myScope.WriteString(f"{command}", True)
    check_instrument_errors(command)

# =====
# Send a command and check for errors:
# =====
def do_command_ieee_block(command, data):
    myScope.WriteIEEEBlock(command, data, True)
    check_instrument_errors(command)

# =====
# Send a query, check for errors, return string:
# =====
def do_query_string(query):
    myScope.WriteString(f"{query}", True)
    result = myScope.ReadString()
    check_instrument_errors(query)
    return result.strip()

# =====
# Send a query, check for errors, return string:
# =====
def do_query_ieee_block(query):
    myScope.WriteString(f"{query}", True)
    result = myScope.ReadIEEEBlock(VisaComLib.BinaryType_UI1, \
        False, True)

```

```

    check_instrument_errors(query)
    return result

# =====
# Send a query, check for errors, return values:
# =====
def do_query_number(query):
    myScope.WriteString(f"{query}", True)
    result = myScope.ReadNumber(VisaComLib.ASCIIType_R8, True)
    check_instrument_errors(query)
    return result

# =====
# Send a query, check for errors, return values:
# =====
def do_query_numbers(query):
    myScope.WriteString(f"{query}", True)
    result = myScope.ReadList(VisaComLib.ASCIIType_R8, ",;")
    check_instrument_errors(query)
    return result

# =====
# Check for instrument errors:
# =====
def check_instrument_errors(command):

    while True:
        myScope.WriteString(":SYSTem:ERRor?", True)
        error_string = myScope.ReadString()
        if error_string:    # If there is an error string value.

            if error_string.find("+0,", 0, 3) == -1:    # Not "No error".
                print(f"ERROR: {error_string}, command: '{command}'")
                print("Exited because of error.")
                sys.exit(1)

            else:    # "No error"
                break

        else:    # :SYSTem:ERRor? should always return string.
            print("ERROR: :SYSTem:ERRor? returned nothing, command: '%s' \" \
                % command)
            print("Exited because of error.")
            sys.exit(1)

# =====
# Main program:
# =====
rm = CreateObject("VISA.GlobalRM", \
    interface=VisaComLib.IResourceManager)
myScope = CreateObject("VISA.BasicFormattedIO", \
    interface=VisaComLib.IFormattedIO488)
myScope.IO = \

```



```
rm.Open("TCPIP0::k-mx3104g-10104.cos.is.keysight.com::inst0::INSTR")

# Clear the interface.
myScope.IO.Clear()
print("Interface cleared.")

# Set the Timeout to 15 seconds.
myScope.IO.Timeout = 15000 # 15 seconds.
print("Timeout set to 15000 milliseconds.")

# Initialize the oscilloscope, capture data, and analyze.
initialize()
capture()
analyze()

myScope.IO.Close()
print("End of program.")
sys.exit()
```

VISA Examples

- "VISA Example in C" on page 1694
- "VISA Example in Visual Basic" on page 1703
- "VISA Example in C#" on page 1713
- "VISA Example in Visual Basic .NET" on page 1724
- "VISA Example in Python" on page 1734

VISA Example in C

To compile and run this example in Microsoft Visual Studio 2008:

- 1 Open Visual Studio.
- 2 Create a new Visual C++, Win32, Win32 Console Application project.
- 3 In the Win32 Application Wizard, click **Next >**. Then, check **Empty project**, and click **Finish**.
- 4 Cut-and-paste the code that follows into a file named "example.c" in the project directory.
- 5 In Visual Studio 2008, right-click the Source Files folder, choose **Add > Add Existing Item...**, select the example.c file, and click **Add**.
- 6 Edit the program to use the VISA address of your oscilloscope.
- 7 Choose **Project > Properties....** In the Property Pages dialog, update these project settings:
 - a Under Configuration Properties, Linker, Input, add "visa32.lib" to the Additional Dependencies field.
 - b Under Configuration Properties, C/C++, Code Generation, select Multi-threaded DLL for the Runtime Library field.
 - c Click **OK** to close the Property Pages dialog.
- 8 Add the include files and library files search paths:
 - a Choose **Tools > Options....**
 - b In the Options dialog, under Projects and Solutions, select **VC++ Directories**.
 - c Show directories for **Include files**, and add the include directory (for example, Program Files (x86)\IVI Foundation\VISA\WinNT\Include).
 - d Show directories for **Library files**, and add the library files directory (for example, Program Files (x86)\IVI Foundation\VISA\WinNT\lib\msc).
 - e Click **OK** to close the Options dialog.
- 9 Build and run the program.

```
/*
 * Keysight VISA Example in C
 * -----
```

```

* This program illustrates a few commonly-used programming
* features of your Keysight oscilloscope.
*/

#include <stdio.h>           /* For printf(). */
#include <string.h>          /* For strcpy(), strcat(). */
#include <time.h>            /* For clock(). */
#include <visa.h>            /* Keysight VISA routines. */

#define VISA_ADDRESS "USB0::0x0957::0x17A6::US50210029::0::INSTR"
#define IEEEBLOCK_SPACE 5000000

/* Function prototypes */
void initialize(void);      /* Initialize to known state. */
void capture(void);        /* Capture the waveform. */
void analyze(void);        /* Analyze the captured waveform. */

void do_command(char *command); /* Send command. */
int do_command_ieeeblock(char *command); /* Command w/IEEE block. */
void do_query_string(char *query); /* Query for string. */
void do_query_number(char *query); /* Query for number. */
void do_query_numbers(char *query); /* Query for numbers. */
int do_query_ieeeblock(char *query); /* Query for IEEE block. */
void check_instrument_errors(); /* Check for inst errors. */
void error_handler(); /* VISA error handler. */

/* Global variables */
ViSession defaultRM, vi; /* Device session ID. */
ViStatus err; /* VISA function return value. */
char str_result[256] = {0}; /* Result from do_query_string(). */
double num_result; /* Result from do_query_number(). */
unsigned char ieeeblock_data[IEEBLOCK_SPACE]; /* Result from
do_query_ieeeblock(). */
double dbl_results[10]; /* Result from do_query_numbers(). */

/* Main Program
* ----- */
void main(void)
{
    /* Open the default resource manager session. */
    err = viOpenDefaultRM(&defaultRM);
    if (err != VI_SUCCESS) error_handler();

    /* Open the session using the oscilloscope's VISA address. */
    err = viOpen(defaultRM, VISA_ADDRESS, VI_NULL, VI_NULL, &vi);
    if (err != VI_SUCCESS) error_handler();

    /* Set the I/O timeout to fifteen seconds. */
    err = viSetAttribute(vi, VI_ATTR_TMO_VALUE, 15000);
    if (err != VI_SUCCESS) error_handler();

    /* Initialize - start from a known state. */
    initialize();

    /* Capture data. */
    capture();

```

```

    /* Analyze the captured waveform. */
    analyze();

    /* Close the vi session and the resource manager session. */
    viClose(vi);
    viClose(defaultRM);
}

/* Initialize the oscilloscope to a known state.
 * ----- */
void initialize (void)
{
    /* Clear the interface. */
    err = viClear(vi);
    if (err != VI_SUCCESS) error_handler();

    /* Get and display the device's *IDN? string. */
    do_query_string("*IDN?");
    printf("Oscilloscope *IDN? string: %s\n", str_result);

    /* Clear status and load the default setup. */
    do_command("*CLS");
    do_command("*RST");
}

/* Capture the waveform.
 * ----- */
void capture (void)
{
    int num_bytes;
    FILE *fp;

    /* Use auto-scale to automatically configure oscilloscope. */
    do_command(":AUToscale");

    /* Set trigger mode (EDGE, PULSe, PATtern, etc., and input source. */
    do_command(":TRIGger:MODE EDGE");
    do_query_string(":TRIGger:MODE?");
    printf("Trigger mode: %s\n", str_result);

    /* Set EDGE trigger parameters. */
    do_command(":TRIGger:EDGE:SOURce CHANnel1");
    do_query_string(":TRIGger:EDGE:SOURce?");
    printf("Trigger edge source: %s\n", str_result);

    do_command(":TRIGger:EDGE:LEVel 1.5");
    do_query_string(":TRIGger:EDGE:LEVel?");
    printf("Trigger edge level: %s\n", str_result);

    do_command(":TRIGger:EDGE:SLOPe POSitive");
    do_query_string(":TRIGger:EDGE:SLOPe?");
    printf("Trigger edge slope: %s\n", str_result);

    /* Save oscilloscope configuration. */

    /* Read system setup. */
    num_bytes = do_query_ieeeblock(":SYSTem:SETup?");

```

```

printf("Read setup string query (%d bytes).\n", num_bytes);

/* Write setup string to file. */
fp = fopen ("c:\\scope\\config\\setup.stp", "wb");
num_bytes = fwrite(ieeeblock_data, sizeof(unsigned char), num_bytes,
    fp);
fclose (fp);
printf("Wrote setup string (%d bytes) to ", num_bytes);
printf("c:\\scope\\config\\setup.stp.\n");

/* Change settings with individual commands:

/* Set vertical scale and offset. */
do_command(":CHANnel1:SCALE 0.05");
do_query_string(":CHANnel1:SCALE?");
printf("Channel 1 vertical scale: %s\n", str_result);

do_command(":CHANnel1:OFFSet -1.5");
do_query_string(":CHANnel1:OFFSet?");
printf("Channel 1 offset: %s\n", str_result);

/* Set horizontal scale and offset. */
do_command(":TIMEbase:SCALE 0.0002");
do_query_string(":TIMEbase:SCALE?");
printf("Timebase scale: %s\n", str_result);

do_command(":TIMEbase:POSition 0.0");
do_query_string(":TIMEbase:POSition?");
printf("Timebase position: %s\n", str_result);

/* Set the acquisition type (NORMAL, PEAK, AVERage, or HRESolution). */
/
do_command(":ACQuire:TYPE NORMAl");
do_query_string(":ACQuire:TYPE?");
printf("Acquire type: %s\n", str_result);

/* Or, configure by loading a previously saved setup. */

/* Read setup string from file. */
fp = fopen ("c:\\scope\\config\\setup.stp", "rb");
num_bytes = fread (ieeeblock_data, sizeof(unsigned char),
    IEEEBLOCK_SPACE, fp);
fclose (fp);
printf("Read setup string (%d bytes) from file ", num_bytes);
printf("c:\\scope\\config\\setup.stp.\n");

/* Restore setup string. */
num_bytes = do_command_ieeeblock(":SYSTem:SETup", num_bytes);
printf("Restored setup string (%d bytes).\n", num_bytes);

/* Capture an acquisition using :DIGitize. */
do_command(":DIGitize CHANnel1");
}

/* Analyze the captured waveform.
* ----- */
void analyze (void)

```

```

{
    double wav_format;
    double acq_type;
    double wav_points;
    double avg_count;
    double x_increment;
    double x_origin;
    double x_reference;
    double y_increment;
    double y_origin;
    double y_reference;

    FILE *fp;
    int num_bytes;    /* Number of bytes returned from instrument. */
    int i;

    /* Make a couple of measurements.
     * ----- */
    do_command(":MEASure:SOURce CHANnel1");
    do_query_string(":MEASure:SOURce?");
    printf("Measure source: %s\n", str_result);

    do_command(":MEASure:FREQuency");
    do_query_number(":MEASure:FREQuency?");
    printf("Frequency: %.4f kHz\n", num_result / 1000);

    do_command(":MEASure:VAMPlitude");
    do_query_number(":MEASure:VAMPlitude?");
    printf("Vertical amplitude: %.2f V\n", num_result);

    /* Download the screen image.
     * ----- */
    do_command(":HARDcopy:INKSaver OFF");

    /* Read screen image. */
    num_bytes = do_query_ieeeblock(":DISPlay:DATA? PNG, COLOr");
    printf("Screen image bytes: %d\n", num_bytes);

    /* Write screen image bytes to file. */
    fp = fopen("c:\\scope\\data\\screen.png", "wb");
    num_bytes = fwrite(ieeeblock_data, sizeof(unsigned char), num_bytes,
        fp);
    fclose(fp);
    printf("Wrote screen image (%d bytes) to ", num_bytes);
    printf("c:\\scope\\data\\screen.png.\n");

    /* Download waveform data.
     * ----- */

    /* Set the waveform points mode. */
    do_command(":WAVEform:POINts:MODE RAW");
    do_query_string(":WAVEform:POINts:MODE?");
    printf("Waveform points mode: %s\n", str_result);

    /* Get the number of waveform points available. */
    do_query_string(":WAVEform:POINts?");
    printf("Waveform points available: %s\n", str_result);

```

```

/* Set the waveform source. */
do_command(":WAVEform:SOURce CHANnel");
do_query_string(":WAVEform:SOURce?");
printf("Waveform source: %s\n", str_result);

/* Choose the format of the data returned (WORD, BYTE, ASCII): */
do_command(":WAVEform:FORMat BYTE");
do_query_string(":WAVEform:FORMat?");
printf("Waveform format: %s\n", str_result);

/* Display the waveform settings: */
do_query_numbers(":WAVEform:PREAmble?");

wav_format = dbl_results[0];
if (wav_format == 0.0)
{
    printf("Waveform format: BYTE\n");
}
else if (wav_format == 1.0)
{
    printf("Waveform format: WORD\n");
}
else if (wav_format == 2.0)
{
    printf("Waveform format: ASCii\n");
}

acq_type = dbl_results[1];
if (acq_type == 0.0)
{
    printf("Acquire type: NORMal\n");
}
else if (acq_type == 1.0)
{
    printf("Acquire type: PEAK\n");
}
else if (acq_type == 2.0)
{
    printf("Acquire type: AVERage\n");
}
else if (acq_type == 3.0)
{
    printf("Acquire type: HRESolution\n");
}

wav_points = dbl_results[2];
printf("Waveform points: %e\n", wav_points);

avg_count = dbl_results[3];
printf("Waveform average count: %e\n", avg_count);

x_increment = dbl_results[4];
printf("Waveform X increment: %e\n", x_increment);

x_origin = dbl_results[5];
printf("Waveform X origin: %e\n", x_origin);

```

```

x_reference = dbl_results[6];
printf("Waveform X reference: %e\n", x_reference);

y_increment = dbl_results[7];
printf("Waveform Y increment: %e\n", y_increment);

y_origin = dbl_results[8];
printf("Waveform Y origin: %e\n", y_origin);

y_reference = dbl_results[9];
printf("Waveform Y reference: %e\n", y_reference);

/* Read waveform data. */
num_bytes = do_query_ieeeblock(":WAVEform:DATA?");
printf("Number of data values: %d\n", num_bytes);

/* Open file for output. */
fp = fopen("c:\\scope\\data\\waveform_data.csv", "wb");

/* Output waveform data in CSV format. */
for (i = 0; i < num_bytes - 1; i++)
{
    /* Write time value, voltage value. */
    fprintf(fp, "%9f, %6f\n",
        x_origin + ((float)i * x_increment),
        (((float)ieeeblock_data[i] - y_reference) * y_increment)
        + y_origin);
}

/* Close output file. */
fclose(fp);
printf("Waveform format BYTE data written to ");
printf("c:\\scope\\data\\waveform_data.csv.\n");
}

/* Send a command to the instrument.
 * ----- */
void do_command(command)
char *command;
{
    char message[80];

    strcpy(message, command);
    strcat(message, "\n");
    err = viPrintf(vi, message);
    if (err != VI_SUCCESS) error_handler();

    check_instrument_errors();
}

/* Command with IEEE definite-length block.
 * ----- */
int do_command_ieeeblock(command, num_bytes)
char *command;
int num_bytes;
{

```



```

    char message[80];
    int data_length;

    strcpy(message, command);
    strcat(message, " #8%08d");
    err = viPrintf(vi, message, num_bytes);
    if (err != VI_SUCCESS) error_handler();

    err = viBufWrite(vi, ieeeblock_data, num_bytes, &data_length);
    if (err != VI_SUCCESS) error_handler();

    check_instrument_errors();

    return(data_length);
}

/* Query for a string result.
 * ----- */
void do_query_string(query)
char *query;
{
    char message[80];

    strcpy(message, query);
    strcat(message, "\n");

    err = viPrintf(vi, message);
    if (err != VI_SUCCESS) error_handler();

    err = viScanf(vi, "%t", str_result);
    if (err != VI_SUCCESS) error_handler();

    check_instrument_errors();
}

/* Query for a number result.
 * ----- */
void do_query_number(query)
char *query;
{
    char message[80];

    strcpy(message, query);
    strcat(message, "\n");

    err = viPrintf(vi, message);
    if (err != VI_SUCCESS) error_handler();

    err = viScanf(vi, "%lf", &num_result);
    if (err != VI_SUCCESS) error_handler();

    check_instrument_errors();
}

/* Query for numbers result.
 * ----- */
void do_query_numbers(query)

```

```

char *query;
{
    char message[80];

    strcpy(message, query);
    strcat(message, "\n");

    err = viPrintf(vi, message);
    if (err != VI_SUCCESS) error_handler();

    err = viScanf(vi, "%,10lf\n", dbl_results);
    if (err != VI_SUCCESS) error_handler();

    check_instrument_errors();
}

/* Query for an IEEE definite-length block result.
 * ----- */
int do_query_ieeeblock(query)
char *query;
{
    char message[80];
    int data_length;

    strcpy(message, query);
    strcat(message, "\n");
    err = viPrintf(vi, message);
    if (err != VI_SUCCESS) error_handler();

    data_length = IEEEBLOCK_SPACE;
    err = viScanf(vi, "%#b\n", &data_length, ieeeblock_data);
    if (err != VI_SUCCESS) error_handler();

    if (data_length == IEEEBLOCK_SPACE )
    {
        printf("IEEE block buffer full: ");
        printf("May not have received all data.\n");
    }

    check_instrument_errors();

    return(data_length);
}

/* Check for instrument errors.
 * ----- */
void check_instrument_errors()
{
    char str_err_val[256] = {0};
    char str_out[800] = "";

    err = viQueryf(vi, ":SYSTem:ERRor?\n", "%t", str_err_val);
    if (err != VI_SUCCESS) error_handler();
    while(strncmp(str_err_val, "+0,No error", 3) != 0 )
    {
        strcat(str_out, ", ");
        strcat(str_out, str_err_val);
    }
}

```

```

    err = viQueryf(vi, ":SYSTem:ERRor?\n", "%t", str_err_val);
    if (err != VI_SUCCESS) error_handler();
}

if (strcmp(str_out, "") != 0)
{
    printf("INST Error%s\n", str_out);
    err = viFlush(vi, VI_READ_BUF);
    if (err != VI_SUCCESS) error_handler();
    err = viFlush(vi, VI_WRITE_BUF);
    if (err != VI_SUCCESS) error_handler();
}
}

/* Handle VISA errors.
 * ----- */
void error_handler()
{
    char err_msg[1024] = {0};

    viStatusDesc(vi, err, err_msg);
    printf("VISA Error: %s\n", err_msg);
    if (err < VI_SUCCESS)
    {
        exit(1);
    }
}

```

VISA Example in Visual Basic

To run this example in Visual Basic for Applications:

- 1 Start the application that provides Visual Basic for Applications (for example, Microsoft Excel).
- 2 Press ALT+F11 to launch the Visual Basic editor.
- 3 Add the visa32.bas file to your project:
 - a Choose **File > Import File...**
 - b Navigate to the header file, visa32.bas (installed with Keysight IO Libraries Suite and found in the Program Files (x86)\IVI Foundation\VISA\WinNT\Include), select it, and click **Open**.
- 4 Choose **Insert > Module**.
- 5 Cut-and-paste the code that follows into the editor.
- 6 Edit the program to use the VISA address of your oscilloscope, and save the changes.
- 7 Run the program.

```

'
' Keysight VISA Example in Visual Basic
' -----
' This program illustrates a few commonly-used programming

```

```

' features of your Keysight oscilloscope.
' -----

Option Explicit

Public err As Long    ' Error returned by VISA function calls.
Public drm As Long    ' Session to Default Resource Manager.
Public vi As Long     ' Session to instrument.

' Declare variables to hold numeric values returned by
' viVScanf/viVQueryf.
Public dblQueryResult As Double
Public Const ByteArraySize = 5000000
Public retCount As Long
Public byteArray(ByteArraySize) As Byte
Public paramsArray(2) As Long
Public Const DblArraySize = 20
Public dblArray(DblArraySize) As Double

' Declare fixed length string variable to hold string value returned
' by viVScanf/viVQueryf.
Public strQueryResult As String * 200

' For Sleep subroutine.
Private Declare Sub Sleep Lib "kernel32" (ByVal dwMilliseconds As Long)

'
' Main Program
' -----

Sub Main()

    ' Open the default resource manager session.
    err = viOpenDefaultRM(drm)
    If (err <> VI_SUCCESS) Then HandleVISAError drm

    ' Open the session using the oscilloscope's VISA address.
    err = viOpen(drm, _
        "USB0::0x0957::0x17A6::US50210029::0::INSTR", 0, 15000, vi)
    If (err <> VI_SUCCESS) Then HandleVISAError drm

    ' Set the I/O timeout to ten seconds.
    err = viSetAttribute(vi, VI_ATTR_TMO_VALUE, 10000)
    If (err <> VI_SUCCESS) Then HandleVISAError vi

    ' Initialize - start from a known state.
    Initialize

    ' Capture data.
    Capture

    ' Analyze the captured waveform.
    Analyze

    ' Close the vi session and the resource manager session.
    err = viClose(vi)
    err = viClose(drm)

```

```

End Sub

'
' Initialize the oscilloscope to a known state.
' -----

Private Sub Initialize()

    ' Clear the interface.
    err = viClear(vi)
    If Not (err = VI_SUCCESS) Then HandleVISAError vi

    ' Get and display the device's *IDN? string.
    strQueryResult = DoQueryString("*IDN?")
    MsgBox "*IDN? string: " + strQueryResult, vbOKOnly, "*IDN? Result"

    ' Clear status and load the default setup.
    DoCommand "*CLS"
    DoCommand "*RST"

End Sub

'
' Capture the waveform.
' -----

Private Sub Capture()

    ' Use auto-scale to automatically configure oscilloscope.
    ' -----
    DoCommand ":AUToscale"

    ' Set trigger mode (EDGE, PULSe, PATtern, etc., and input source.
    DoCommand ":TRIGger:MODE EDGE"
    Debug.Print "Trigger mode: " + _
        DoQueryString(":TRIGger:MODE?")

    ' Set EDGE trigger parameters.
    DoCommand ":TRIGger:EDGE:SOURce CHANnel1"
    Debug.Print "Trigger edge source: " + _
        DoQueryString(":TRIGger:EDGE:SOURce?")

    DoCommand ":TRIGger:EDGE:LEVel 1.5"
    Debug.Print "Trigger edge level: " + _
        DoQueryString(":TRIGger:EDGE:LEVel?")

    DoCommand ":TRIGger:EDGE:SLOPe POSitive"
    Debug.Print "Trigger edge slope: " + _
        DoQueryString(":TRIGger:EDGE:SLOPe?")

    ' Save oscilloscope configuration.
    ' -----
    Dim lngSetupStringSize As Long
    lngSetupStringSize = DoQueryIEEEBlock_Bytes(":SYSTem:SETup?")
    Debug.Print "Setup bytes saved: " + CStr(lngSetupStringSize)

```

```

' Output setup string to a file:
Dim strPath As String
strPath = "c:\scope\config\setup.dat"
If Len(Dir(strPath)) Then
    Kill strPath ' Remove file if it exists.
End If

' Open file for output.
Dim hFile As Long
hFile = FreeFile
Open strPath For Binary Access Write Lock Write As hFile
Dim lngI As Long
For lngI = 0 To lngSetupStringSize - 1
    Put hFile, , byteArray(lngI) ' Write data.
Next lngI
Close hFile ' Close file.

' Change settings with individual commands:
' -----

' Set vertical scale and offset.
DoCommand ":CHANnel1:SCALE 0.05"
Debug.Print "Channel 1 vertical scale: " + _
    DoQueryString(":CHANnel1:SCALE?")

DoCommand ":CHANnel1:OFFSet -1.5"
Debug.Print "Channel 1 vertical offset: " + _
    DoQueryString(":CHANnel1:OFFSet?")

' Set horizontal scale and position.
DoCommand ":TIMEbase:SCALE 0.0002"
Debug.Print "Timebase scale: " + _
    DoQueryString(":TIMEbase:SCALE?")

DoCommand ":TIMEbase:POSition 0.0"
Debug.Print "Timebase position: " + _
    DoQueryString(":TIMEbase:POSition?")

' Set the acquisition type (NORMAL, PEAK, AVERage, or HRESolution).
DoCommand ":ACQuire:TYPE NORMAL"
Debug.Print "Acquire type: " + _
    DoQueryString(":ACQuire:TYPE?")

' Or, configure by loading a previously saved setup.
' -----
strPath = "c:\scope\config\setup.dat"
Open strPath For Binary Access Read As hFile ' Open file for input.
Dim lngSetupFileSize As Long
lngSetupFileSize = LOF(hFile) ' Length of file.
Get hFile, , byteArray ' Read data.
Close hFile ' Close file.
' Write learn string back to oscilloscope using ":SYSTem:SETup"
' command:
Dim lngRestored As Long
lngRestored = DoCommandIEEEBlock(":SYSTem:SETup", lngSetupFileSize)
Debug.Print "Setup bytes restored: " + CStr(lngRestored)

```

```

' Capture an acquisition using :DIGitize.
' -----
DoCommand ":DIGitize CHANnel1"

End Sub

'
' Analyze the captured waveform.
' -----

Private Sub Analyze()

' Make a couple of measurements.
' -----
DoCommand ":MEASure:SOURce CHANnel1"
Debug.Print "Measure source: " + _
    DoQueryString(":MEASure:SOURce?")

DoCommand ":MEASure:FREQuency"
dblQueryResult = DoQueryNumber(":MEASure:FREQuency?")
MsgBox "Frequency:" + vbCrLf + _
    FormatNumber(dblQueryResult / 1000, 4) + " kHz"

DoCommand ":MEASure:VAMPlitude"
dblQueryResult = DoQueryNumber(":MEASure:VAMPlitude?")
MsgBox "Vertical amplitude:" + vbCrLf + _
    FormatNumber(dblQueryResult, 4) + " V"

' Download the screen image.
' -----
DoCommand ":HARDcopy:INKSaver OFF"

' Get screen image.
Dim lngBlockSize As Long
lngBlockSize = DoQueryIEEEBlock_Bytes(":DISPlay:DATA? PNG, COlor")
Debug.Print "Screen image bytes: " + CStr(lngBlockSize)

' Save screen image to a file:
Dim strPath As String
strPath = "c:\scope\data\screen.png"
If Len(Dir(strPath)) Then
    Kill strPath ' Remove file if it exists.
End If
Dim hFile As Long
hFile = FreeFile
Open strPath For Binary Access Write Lock Write As hFile
Dim lngI As Long
For lngI = 0 To lngBlockSize - 1
    Put hFile, , byteArray(lngI) ' Write data.
Next lngI
Close hFile ' Close file.
MsgBox "Screen image written to " + strPath

' Download waveform data.
' -----

```

```

' Set the waveform points mode.
DoCommand ":WAVEform:POINts:MODE RAW"
Debug.Print "Waveform points mode: " + _
    DoQueryString(":WAVEform:POINts:MODE?")

' Get the number of waveform points available.
Debug.Print "Waveform points available: " + _
    DoQueryString(":WAVEform:POINts?")

' Set the waveform source.
DoCommand ":WAVEform:SOURce CHANnel1"
Debug.Print "Waveform source: " + _
    DoQueryString(":WAVEform:SOURce?")

' Choose the format of the data returned (WORD, BYTE, ASCII):
DoCommand ":WAVEform:FORMat BYTE"
Debug.Print "Waveform format: " + _
    DoQueryString(":WAVEform:FORMat?")

' Display the waveform settings:
Dim intFormat As Integer
Dim intType As Integer
Dim lngPoints As Long
Dim lngCount As Long
Dim dblXIncrement As Double
Dim dblXOrigin As Double
Dim lngXReference As Long
Dim sngYIncrement As Single
Dim lngYOrigin As Long
Dim lngYReference As Long
Dim strOutput As String

Dim lngNumNumbers As Long
lngNumNumbers = DoQueryNumbers(":WAVEform:PREamble?")

intFormat = dblArray(0)
intType = dblArray(1)
lngPoints = dblArray(2)
lngCount = dblArray(3)
dblXIncrement = dblArray(4)
dblXOrigin = dblArray(5)
lngXReference = dblArray(6)
sngYIncrement = dblArray(7)
lngYOrigin = dblArray(8)
lngYReference = dblArray(9)

If intFormat = 0 Then
    Debug.Print "Waveform format: BYTE"
ElseIf intFormat = 1 Then
    Debug.Print "Waveform format: WORD"
ElseIf intFormat = 2 Then
    Debug.Print "Waveform format: ASCii"
End If

If intType = 0 Then
    Debug.Print "Acquisition type: NORMal"

```



```

ElseIf intType = 1 Then
    Debug.Print "Acquisition type: PEAK"
ElseIf intType = 2 Then
    Debug.Print "Acquisition type: AVERage"
ElseIf intType = 3 Then
    Debug.Print "Acquisition type: HRESolution"
End If

Debug.Print "Waveform points: " + _
    FormatNumber(lngPoints, 0)

Debug.Print "Waveform average count: " + _
    FormatNumber(lngCount, 0)

Debug.Print "Waveform X increment: " + _
    Format(dblXIncrement, "Scientific")

Debug.Print "Waveform X origin: " + _
    Format(dblXOrigin, "Scientific")

Debug.Print "Waveform X reference: " + _
    FormatNumber(lngXReference, 0)

Debug.Print "Waveform Y increment: " + _
    Format(sngYIncrement, "Scientific")

Debug.Print "Waveform Y origin: " + _
    FormatNumber(lngYOrigin, 0)

Debug.Print "Waveform Y reference: " + _
    FormatNumber(lngYReference, 0)

' Get the waveform data
Dim lngNumBytes As Long
lngNumBytes = DoQueryIEEEBlock_Bytes(":WAVEform:DATA?")
Debug.Print "Number of data values: " + CStr(lngNumBytes)

' Set up output file:
strPath = "c:\scope\data\waveform_data.csv"

' Open file for output.
Open strPath For Output Access Write Lock Write As hFile

' Output waveform data in CSV format.
Dim lngDataValue As Long

For lngI = 0 To lngNumBytes - 1
    lngDataValue = CLng(byteArray(lngI))

    ' Write time value, voltage value.
    Print #hFile, _
        FormatNumber(dblXOrigin + (lngI * dblXIncrement), 9) + _
        ", " + _
        FormatNumber(((lngDataValue - lngYReference) _
            * sngYIncrement) + lngYOrigin)
Next lngI

```

```

        ' Close output file.
        Close hFile      ' Close file.
        MsgBox "Waveform format BYTE data written to " + _
            "c:\scope\data\waveform_data.csv."

End Sub

Private Sub DoCommand(command As String)

    err = viVPrintf(vi, command + vbLf, 0)
    If (err <> VI_SUCCESS) Then HandleVISAError vi

    CheckInstrumentErrors

End Sub

Private Function DoCommandIEEEBlock(command As String, _
    lngBlockSize As Long)

    retCount = lngBlockSize

    Dim strCommandAndLength As String
    strCommandAndLength = command + " %#" + _
        Format(lngBlockSize) + "b"

    err = viVPrintf(vi, strCommandAndLength + vbLf, paramsArray(1))
    If (err <> VI_SUCCESS) Then HandleVISAError vi

    DoCommandIEEEBlock = retCount

    CheckInstrumentErrors

End Function

Private Function DoQueryString(query As String) As String

    Dim strResult As String * 200

    err = viVPrintf(vi, query + vbLf, 0)
    If (err <> VI_SUCCESS) Then HandleVISAError vi

    err = viVScanf(vi, "%t", strResult)
    If (err <> VI_SUCCESS) Then HandleVISAError vi

    DoQueryString = strResult

    CheckInstrumentErrors

End Function

Private Function DoQueryNumber(query As String) As Variant

    Dim dblResult As Double

    err = viVPrintf(vi, query + vbLf, 0)
    If (err <> VI_SUCCESS) Then HandleVISAError vi

```

```

    err = viVScanf(vi, "%lf" + vbLf, VarPtr(dblResult))
    If (err <> VI_SUCCESS) Then HandleVISAError vi

    DoQueryNumber = dblResult

    CheckInstrumentErrors

End Function

Private Function DoQueryNumbers(query As String) As Long

    Dim dblResult As Double

    ' Send query.
    err = viVPrintf(vi, query + vbLf, 0)
    If (err <> VI_SUCCESS) Then HandleVISAError vi

    ' Set up paramsArray for multiple parameter query returning array.
    paramsArray(0) = VarPtr(retCount)
    paramsArray(1) = VarPtr(dblArray(0))

    ' Set retCount to max number of elements array can hold.
    retCount = DblArraySize

    ' Read numbers.
    err = viVScanf(vi, "%,#lf" + vbLf, paramsArray(0))
    If (err <> VI_SUCCESS) Then HandleVISAError vi

    ' retCount is now actual number of values returned by query.
    DoQueryNumbers = retCount

    CheckInstrumentErrors

End Function

Private Function DoQueryIEEEBlock_Bytes(query As String) As Long

    ' Send query.
    err = viVPrintf(vi, query + vbLf, 0)
    If (err <> VI_SUCCESS) Then HandleVISAError vi

    ' Set up paramsArray for multiple parameter query returning array.
    paramsArray(0) = VarPtr(retCount)
    paramsArray(1) = VarPtr(byteArray(0))

    ' Set retCount to max number of elements array can hold.
    retCount = ByteArraySize

    ' Get unsigned integer bytes.
    err = viVScanf(vi, "%#b" + vbLf, paramsArray(0))
    If (err <> VI_SUCCESS) Then HandleVISAError vi

    err = viFlush(vi, VI_READ_BUF)
    If (err <> VI_SUCCESS) Then HandleVISAError vi

    err = viFlush(vi, VI_WRITE_BUF)

```

```

    If (err <> VI_SUCCESS) Then HandleVISAError vi

    ' retCount is now actual number of bytes returned by query.
    DoQueryIEEEBlock_Bytes = retCount

    CheckInstrumentErrors

End Function

Private Sub CheckInstrumentErrors()

    On Error GoTo ErrorHandler

    Dim strErrVal As String * 200
    Dim strOut As String

    err = viVPrintf(vi, ":SYSTem:ERror?" + vbLf, 0)    ' Query any errors.
    If (err <> VI_SUCCESS) Then HandleVISAError vi

    err = viVScanf(vi, "%t", strErrVal)    ' Read: Errnum,"Error String".
    If (err <> VI_SUCCESS) Then HandleVISAError vi

    While Val(strErrVal) <> 0                ' End if find: 0,"No Error".
        strOut = strOut + "INST Error: " + strErrVal

        err = viVPrintf(vi, ":SYSTem:ERror?" + vbLf, 0)    ' Request error.
        If (err <> VI_SUCCESS) Then HandleVISAError vi

        err = viVScanf(vi, "%t", strErrVal)    ' Read error message.
        If (err <> VI_SUCCESS) Then HandleVISAError vi

    Wend

    If Not strOut = "" Then
        MsgBox strOut, vbExclamation, "INST Error Messages"

        err = viFlush(vi, VI_READ_BUF)
        If (err <> VI_SUCCESS) Then HandleVISAError vi

        err = viFlush(vi, VI_WRITE_BUF)
        If (err <> VI_SUCCESS) Then HandleVISAError vi

    End If

    Exit Sub

ErrorHandler:

    MsgBox "*** Error : " + Error, vbExclamation
    End

End Sub

Private Sub HandleVISAError(session As Long)

    Dim strVisaErr As String * 200
    Call viStatusDesc(session, err, strVisaErr)

```

```

MsgBox "*** VISA Error : " + strVisaErr, vbExclamation

' If the error is not a warning, close the session.
If err < VI_SUCCESS Then
    If session <> 0 Then Call viClose(session)
    End
End If

End Sub

```

VISA Example in C#

To compile and run this example in Microsoft Visual Studio 2008:

- 1 Open Visual Studio.
- 2 Create a new Visual C#, Windows, Console Application project.
- 3 Cut-and-paste the code that follows into the C# source file.
- 4 Edit the program to use the VISA address of your oscilloscope.
- 5 Add Keysight's VISA header file to your project:
 - a Right-click the project you wish to modify (not the solution) in the Solution Explorer window of the Microsoft Visual Studio environment.
 - b Click **Add** and then click **Add Existing Item...**
 - c Navigate to the header file, visa32.cs (installed with Keysight IO Libraries Suite and found in the Program Files (x86)\IVI Foundation\VISA\WinNT\Include directory), select it, but *do not click the Open button*.
 - d Click the down arrow to the right of the **Add** button, and choose **Add as Link**.

You should now see the file underneath your project in the Solution Explorer. It will have a little arrow icon in its lower left corner, indicating that it is a link.
- 6 Build and run the program.

For more information, see the tutorial on using VISA in Microsoft .NET in the VISA Help that comes with Keysight IO Libraries Suite.

```

/*
 * Keysight VISA Example in C#
 * -----
 * This program illustrates a few commonly used programming
 * features of your Keysight oscilloscope.
 * -----
 */

using System;
using System.IO;
using System.Text;

namespace InfiniiVision
{
    class VisaInstrumentApp

```

```

{
    private static VisaInstrument myScope;

    public static void Main(string[] args)
    {
        try
        {
            myScope = new
                VisaInstrument("USB0::0x0957::0x17A6::US50210029::0::INSTR");
            myScope.SetTimeoutSeconds(10);

            // Initialize - start from a known state.
            Initialize();

            // Capture data.
            Capture();

            // Analyze the captured waveform.
            Analyze();

        }
        catch (System.ApplicationException err)
        {
            Console.WriteLine("*** VISA Error Message : " + err.Message);
        }
        catch (System.SystemException err)
        {
            Console.WriteLine("*** System Error Message : " + err.Message);
        }
        catch (System.Exception err)
        {
            System.Diagnostics.Debug.Fail("Unexpected Error");
            Console.WriteLine("*** Unexpected Error : " + err.Message);
        }
        finally
        {
            myScope.Close();
        }
    }

    /*
    * Initialize the oscilloscope to a known state.
    * -----
    */
    private static void Initialize()
    {
        StringBuilder strResults;

        // Get and display the device's *IDN? string.
        strResults = myScope.DoQueryString("*IDN?");
        Console.WriteLine("*IDN? result is: {0}", strResults);

        // Clear status and load the default setup.
        myScope.DoCommand("*CLS");
        myScope.DoCommand("*RST");
    }
}

```

```

/*
 * Capture the waveform.
 * -----
 */
private static void Capture()
{
    // Use auto-scale to automatically configure oscilloscope.
    myScope.DoCommand(":AUToscale");

    // Set trigger mode (EDGE, PULSe, PATtern, etc., and input source.
    myScope.DoCommand(":TRIGger:MODE EDGE");
    Console.WriteLine("Trigger mode: {0}",
        myScope.DoQueryString(":TRIGger:MODE?"));

    // Set EDGE trigger parameters.
    myScope.DoCommand(":TRIGger:EDGE:SOURce CHANnel1");
    Console.WriteLine("Trigger edge source: {0}",
        myScope.DoQueryString(":TRIGger:EDGE:SOURce?"));

    myScope.DoCommand(":TRIGger:EDGE:LEVel 1.5");
    Console.WriteLine("Trigger edge level: {0}",
        myScope.DoQueryString(":TRIGger:EDGE:LEVel?"));

    myScope.DoCommand(":TRIGger:EDGE:SLOPe POSitive");
    Console.WriteLine("Trigger edge slope: {0}",
        myScope.DoQueryString(":TRIGger:EDGE:SLOPe?"));

    // Save oscilloscope configuration.
    byte[] ResultsArray;    // Results array.
    int nLength;    // Number of bytes returned from instrument.
    string strPath;

    // Query and read setup string.
    nLength = myScope.DoQueryIEEEBlock(":SYSTem:SETup?",
        out ResultsArray);

    // Write setup string to file.
    strPath = "c:\\scope\\config\\setup.stp";
    FileStream fStream = File.Open(strPath, FileMode.Create);
    fStream.Write(ResultsArray, 0, nLength);
    fStream.Close();
    Console.WriteLine("Setup bytes saved: {0}", nLength);

    // Change settings with individual commands:

    // Set vertical scale and offset.
    myScope.DoCommand(":CHANnel1:SCALe 0.05");
    Console.WriteLine("Channel 1 vertical scale: {0}",
        myScope.DoQueryString(":CHANnel1:SCALe?"));

    myScope.DoCommand(":CHANnel1:OFFSet -1.5");
    Console.WriteLine("Channel 1 vertical offset: {0}",
        myScope.DoQueryString(":CHANnel1:OFFSet?"));

    // Set horizontal scale and position.
    myScope.DoCommand(":TIMEbase:SCALe 0.0002");
    Console.WriteLine("Timebase scale: {0}",

```

```

        myScope.DoQueryString(":TIMEbase:SCALE?"));

myScope.DoCommand(":TIMEbase:POSition 0.0");
Console.WriteLine("Timebase position: {0}",
    myScope.DoQueryString(":TIMEbase:POSition?"));

// Set the acquisition type (NORMal, PEAK, AVERage, or HRESolution
).
myScope.DoCommand(":ACquire:TYPE NORMal");
Console.WriteLine("Acquire type: {0}",
    myScope.DoQueryString(":ACquire:TYPE?"));

// Or, configure by loading a previously saved setup.
byte[] DataArray;
int nBytesWritten;

// Read setup string from file.
strPath = "c:\\scope\\config\\setup.stp";
DataArray = File.ReadAllBytes(strPath);

// Restore setup string.
nBytesWritten = myScope.DoCommandIEEEBlock(":SYSTem:SETup",
    DataArray);
Console.WriteLine("Setup bytes restored: {0}", nBytesWritten);

// Capture an acquisition using :DIGitize.
myScope.DoCommand(":DIGitize CHANnel1");
}

/*
 * Analyze the captured waveform.
 * -----
 */
private static void Analyze()
{
    byte[] ResultsArray; // Results array.
    int nLength; // Number of bytes returned from instrument.
    string strPath;

    // Make a couple of measurements.
    // -----
    myScope.DoCommand(":MEASure:SOURce CHANnel1");
    Console.WriteLine("Measure source: {0}",
        myScope.DoQueryString(":MEASure:SOURce?"));

    double fResult;
    myScope.DoCommand(":MEASure:FREQuency");
    fResult = myScope.DoQueryNumber(":MEASure:FREQuency?");
    Console.WriteLine("Frequency: {0:F4} kHz", fResult / 1000);

    myScope.DoCommand(":MEASure:VAMPlitude");
    fResult = myScope.DoQueryNumber(":MEASure:VAMPlitude?");
    Console.WriteLine("Vertical amplitude: {0:F2} V", fResult);

    // Download the screen image.
    // -----
    myScope.DoCommand(":HARDcopy:INKSaver OFF");

```



```

// Get the screen data.
nLength = myScope.DoQueryIEEEBlock(":DISPlay:DATA? PNG, COlOr",
    out ResultsArray);

// Store the screen data to a file.
strPath = "c:\\scope\\data\\screen.png";
FileStream fStream = File.Open(strPath, FileMode.Create);
fStream.Write(ResultsArray, 0, nLength);
fStream.Close();
Console.WriteLine("Screen image ({0} bytes) written to {1}",
    nLength, strPath);

// Download waveform data.
// -----

// Set the waveform points mode.
myScope.DoCommand(":WAVEform:POINts:MODE RAW");
Console.WriteLine("Waveform points mode: {0}",
    myScope.DoQueryString(":WAVEform:POINts:MODE?"));

// Get the number of waveform points available.
myScope.DoCommand(":WAVEform:POINts 10240");
Console.WriteLine("Waveform points available: {0}",
    myScope.DoQueryString(":WAVEform:POINts?"));

// Set the waveform source.
myScope.DoCommand(":WAVEform:SOURce CHANnel1");
Console.WriteLine("Waveform source: {0}",
    myScope.DoQueryString(":WAVEform:SOURce?"));

// Choose the format of the data returned (WORD, BYTE, ASCII):
myScope.DoCommand(":WAVEform:FORMat BYTE");
Console.WriteLine("Waveform format: {0}",
    myScope.DoQueryString(":WAVEform:FORMat?"));

// Display the waveform settings:
double[] fResultsArray;
fResultsArray = myScope.DoQueryNumbers(":WAVEform:PREAmble?");

double fFormat = fResultsArray[0];
if (fFormat == 0.0)
{
    Console.WriteLine("Waveform format: BYTE");
}
else if (fFormat == 1.0)
{
    Console.WriteLine("Waveform format: WORD");
}
else if (fFormat == 2.0)
{
    Console.WriteLine("Waveform format: ASCii");
}

double fType = fResultsArray[1];
if (fType == 0.0)
{

```

```

        Console.WriteLine("Acquire type: NORMAl");
    }
    else if (fType == 1.0)
    {
        Console.WriteLine("Acquire type: PEAK");
    }
    else if (fType == 2.0)
    {
        Console.WriteLine("Acquire type: AVERAge");
    }
    else if (fType == 3.0)
    {
        Console.WriteLine("Acquire type: HRESolution");
    }

    double fPoints = fResultsArray[2];
    Console.WriteLine("Waveform points: {0:e}", fPoints);

    double fCount = fResultsArray[3];
    Console.WriteLine("Waveform average count: {0:e}", fCount);

    double fXincrement = fResultsArray[4];
    Console.WriteLine("Waveform X increment: {0:e}", fXincrement);

    double fXorigin = fResultsArray[5];
    Console.WriteLine("Waveform X origin: {0:e}", fXorigin);

    double fXreference = fResultsArray[6];
    Console.WriteLine("Waveform X reference: {0:e}", fXreference);

    double fYincrement = fResultsArray[7];
    Console.WriteLine("Waveform Y increment: {0:e}", fYincrement);

    double fYorigin = fResultsArray[8];
    Console.WriteLine("Waveform Y origin: {0:e}", fYorigin);

    double fYreference = fResultsArray[9];
    Console.WriteLine("Waveform Y reference: {0:e}", fYreference);

    // Read waveform data.
    nLength = myScope.DoQueryIEEEBlock(":WAVEform:DATA?",
        out ResultsArray);
    Console.WriteLine("Number of data values: {0}", nLength);

    // Set up output file:
    strPath = "c:\\scope\\data\\waveform_data.csv";
    if (File.Exists(strPath)) File.Delete(strPath);

    // Open file for output.
    StreamWriter writer = File.CreateText(strPath);

    // Output waveform data in CSV format.
    for (int i = 0; i < nLength - 1; i++)
        writer.WriteLine("{0:f9}, {1:f6}",
            fXorigin + ((float)i * fXincrement),
            (((float)ResultsArray[i] - fYreference) *
            fYincrement) + fYorigin);

```

```

        // Close output file.
        writer.Close();
        Console.WriteLine("Waveform format BYTE data written to {0}",
            strPath);
    }
}

class VisaInstrument
{
    private int m_nResourceManager;
    private int m_nSession;
    private string m_strVisaAddress;

    // Constructor.
    public VisaInstrument(string strVisaAddress)
    {
        // Save VISA address in member variable.
        m_strVisaAddress = strVisaAddress;

        // Open the default VISA resource manager.
        OpenResourceManager();

        // Open a VISA resource session.
        OpenSession();

        // Clear the interface.
        int nViStatus;
        nViStatus = visa32.viClear(m_nSession);
    }

    public void DoCommand(string strCommand)
    {
        // Send the command.
        VisaSendCommandOrQuery(strCommand);

        // Check for inst errors.
        CheckInstrumentErrors(strCommand);
    }

    public int DoCommandIEEEBlock(string strCommand,
        byte[] DataArray)
    {
        // Send the command to the device.
        string strCommandAndLength;
        int nViStatus, nLength, nBytesWritten;

        nLength = DataArray.Length;
        strCommandAndLength = String.Format("{0} #8%08d",
            strCommand);

        // Write first part of command to formatted I/O write buffer.
        nViStatus = visa32.viPrintf(m_nSession, strCommandAndLength,
            nLength);
        CheckVisaStatus(nViStatus);

        // Write the data to the formatted I/O write buffer.
    }
}

```

```

        nViStatus = visa32.viBufWrite(m_nSession, dataArray, nLength,
            out nBytesWritten);
        CheckVisaStatus(nViStatus);

        // Check for inst errors.
        CheckInstrumentErrors(strCommand);

        return nBytesWritten;
    }

    public StringBuilder DoQueryString(string strQuery)
    {
        // Send the query.
        VisaSendCommandOrQuery(strQuery);

        // Get the result string.
        StringBuilder strResults = new StringBuilder(1000);
        strResults = VisaGetResultString();

        // Check for inst errors.
        CheckInstrumentErrors(strQuery);

        // Return string results.
        return strResults;
    }

    public double DoQueryNumber(string strQuery)
    {
        // Send the query.
        VisaSendCommandOrQuery(strQuery);

        // Get the result string.
        double fResults;
        fResults = VisaGetResultNumber();

        // Check for inst errors.
        CheckInstrumentErrors(strQuery);

        // Return string results.
        return fResults;
    }

    public double[] DoQueryNumbers(string strQuery)
    {
        // Send the query.
        VisaSendCommandOrQuery(strQuery);

        // Get the result string.
        double[] fResultsArray;
        fResultsArray = VisaGetResultNumbers();

        // Check for inst errors.
        CheckInstrumentErrors(strQuery);

        // Return string results.
        return fResultsArray;
    }
}

```

```

public int DoQueryIEEEBlock(string strQuery,
    out byte[] ResultsArray)
{
    // Send the query.
    VisaSendCommandOrQuery(strQuery);

    // Get the result string.
    int length;    // Number of bytes returned from instrument.
    length = VisaGetResultIEEEBlock(out ResultsArray);

    // Check for inst errors.
    CheckInstrumentErrors(strQuery);

    // Return string results.
    return length;
}

private void VisaSendCommandOrQuery(string strCommandOrQuery)
{
    // Send command or query to the device.
    string strWithNewline;
    strWithNewline = String.Format("{0}\n", strCommandOrQuery);
    int nViStatus;
    nViStatus = visa32.viPrintf(m_nSession, strWithNewline);
    CheckVisaStatus(nViStatus);
}

private StringBuilder VisaGetResultString()
{
    StringBuilder strResults = new StringBuilder(1000);

    // Read return value string from the device.
    int nViStatus;
    nViStatus = visa32.viScanf(m_nSession, "%1000t", strResults);
    CheckVisaStatus(nViStatus);

    return strResults;
}

private double VisaGetResultNumber()
{
    double fResults = 0;

    // Read return value string from the device.
    int nViStatus;
    nViStatus = visa32.viScanf(m_nSession, "%lf", out fResults);
    CheckVisaStatus(nViStatus);

    return fResults;
}

private double[] VisaGetResultNumbers()
{
    double[] fResultsArray;
    fResultsArray = new double[10];
}

```

```

        // Read return value string from the device.
        int nViStatus;
        nViStatus = visa32.viScanf(m_nSession, "%10lf\n",
            fResultsArray);
        CheckVisaStatus(nViStatus);

        return fResultsArray;
    }

private int VisaGetResultIEEEBlock(out byte[] ResultsArray)
{
    // Results array, big enough to hold a PNG.
    ResultsArray = new byte[300000];
    int length;    // Number of bytes returned from instrument.

    // Set the default number of bytes that will be contained in
    // the ResultsArray to 300,000 (300kB).
    length = 300000;

    // Read return value string from the device.
    int nViStatus;
    nViStatus = visa32.viScanf(m_nSession, "%#b", ref length,
        ResultsArray);
    CheckVisaStatus(nViStatus);

    // Write and read buffers need to be flushed after IEEE block?
    nViStatus = visa32.viFlush(m_nSession, visa32.VI_WRITE_BUF);
    CheckVisaStatus(nViStatus);

    nViStatus = visa32.viFlush(m_nSession, visa32.VI_READ_BUF);
    CheckVisaStatus(nViStatus);

    return length;
}

private void CheckInstrumentErrors(string strCommand)
{
    // Check for instrument errors.
    StringBuilder strInstrumentError = new StringBuilder(1000);
    bool bFirstError = true;

    do    // While not "0,No error"
    {
        VisaSendCommandOrQuery(":SYSTem:ERRor?");
        strInstrumentError = VisaGetResultString();

        if (!strInstrumentError.ToString().StartsWith("+0,"))
        {
            if (bFirstError)
            {
                Console.WriteLine("ERROR(s) for command '{0}': ",
                    strCommand);
                bFirstError = false;
            }
            Console.Write(strInstrumentError);
        }
    } while (!strInstrumentError.ToString().StartsWith("+0,"));
}

```

```

    }

    private void OpenResourceManager()
    {
        int nViStatus;
        nViStatus =
            visa32.viOpenDefaultRM(out this.m_nResourceManager);
        if (nViStatus < visa32.VI_SUCCESS)
            throw new
                ApplicationException("Failed to open Resource Manager");
    }

    private void OpenSession()
    {
        int nViStatus;
        nViStatus = visa32.viOpen(this.m_nResourceManager,
            this.m_strVisaAddress, visa32.VI_NO_LOCK,
            visa32.VI_TMO_IMMEDIATE, out this.m_nSession);
        CheckVisaStatus(nViStatus);
    }

    public void SetTimeoutSeconds(int nSeconds)
    {
        int nViStatus;
        nViStatus = visa32.viSetAttribute(this.m_nSession,
            visa32.VI_ATTR_TMO_VALUE, nSeconds * 1000);
        CheckVisaStatus(nViStatus);
    }

    public void CheckVisaStatus(int nViStatus)
    {
        // If VISA error, throw exception.
        if (nViStatus < visa32.VI_SUCCESS)
        {
            StringBuilder strError = new StringBuilder(256);
            visa32.viStatusDesc(this.m_nResourceManager, nViStatus,
                strError);
            throw new ApplicationException(strError.ToString());
        }
    }

    public void Close()
    {
        if (m_nSession != 0)
            visa32.viClose(m_nSession);
        if (m_nResourceManager != 0)
            visa32.viClose(m_nResourceManager);
    }
}

```

VISA Example in Visual Basic .NET

To compile and run this example in Microsoft Visual Studio 2008:

- 1 Open Visual Studio.
- 2 Create a new Visual Basic, Windows, Console Application project.
- 3 Cut-and-paste the code that follows into the Visual Basic .NET source file.
- 4 Edit the program to use the VISA address of your oscilloscope.
- 5 Add Keysight's VISA header file to your project:
 - a Right-click the project you wish to modify (not the solution) in the Solution Explorer window of the Microsoft Visual Studio environment.
 - b Choose **Add** and then choose **Add Existing Item...**
 - c Navigate to the header file, visa32.vb (installed with Keysight IO Libraries Suite and found in the Program Files (x86)\IVI Foundation\VISA\WinNT\Include directory), select it, but *do not click the Open button*.
 - d Click the down arrow to the right of the **Add** button, and choose **Add as Link**.

You should now see the file underneath your project in the Solution Explorer. It will have a little arrow icon in its lower left corner, indicating that it is a link.

 - e Right-click the project again and choose **Properties**; then, select "InfiniiVision.VisaInstrumentApp" as the **Startup object**.
- 6 Build and run the program.

For more information, see the tutorial on using VISA in Microsoft .NET in the VISA Help that comes with Keysight IO Libraries Suite.

```
'
' Keysight VISA Example in Visual Basic .NET
' -----
' This program illustrates a few commonly-used programming
' features of your Keysight oscilloscope.
' -----

Imports System
Imports System.IO
Imports System.Text

Namespace InfiniiVision
  Class VisaInstrumentApp
    Private Shared myScope As VisaInstrument

    Public Shared Sub Main(ByVal args As String())
      Try
        myScope = _
          New VisaInstrument("USB0::0x0957::0x17A6::US50210029::0::INSTR
")
        myScope.SetTimeoutSeconds(10)
      
```



```

' Initialize - start from a known state.
Initialize()

' Capture data.
Capture()

' Analyze the captured waveform.
Analyze()

Catch err As System.ApplicationException
    Console.WriteLine("*** VISA Error Message : " + err.Message)
Catch err As System.SystemException
    Console.WriteLine("*** System Error Message : " + err.Message)
Catch err As System.Exception
    Debug.Fail("Unexpected Error")
    Console.WriteLine("*** Unexpected Error : " + err.Message)
End Try
End Sub

'
' Initialize the oscilloscope to a known state.
' -----

Private Shared Sub Initialize()
    Dim strResults As StringBuilder

    ' Get and display the device's *IDN? string.
    strResults = myScope.DoQueryString("*IDN?")
    Console.WriteLine("*IDN? result is: {0}", strResults)

    ' Clear status and load the default setup.
    myScope.DoCommand("*CLS")
    myScope.DoCommand("*RST")

End Sub

'
' Capture the waveform.
' -----

Private Shared Sub Capture()

    ' Use auto-scale to automatically configure oscilloscope.
    myScope.DoCommand(":AUToscale")

    ' Set trigger mode (EDGE, PULSe, PATtern, etc., and input source.
    myScope.DoCommand(":TRIGger:MODE EDGE")
    Console.WriteLine("Trigger mode: {0}", _
        myScope.DoQueryString(":TRIGger:MODE?"))

    ' Set EDGE trigger parameters.
    myScope.DoCommand(":TRIGger:EDGE:SOURce CHANnel1")
    Console.WriteLine("Trigger edge source: {0}", _
        myScope.DoQueryString(":TRIGger:EDGE:SOURce?"))

    myScope.DoCommand(":TRIGger:EDGE:LEVel 1.5")
    Console.WriteLine("Trigger edge level: {0}", _

```

```

myScope.DoQueryString(":TRIGger:EDGE:LEVel?"))

myScope.DoCommand(":TRIGger:EDGE:SLOPe POSitive")
Console.WriteLine("Trigger edge slope: {0}", _
    myScope.DoQueryString(":TRIGger:EDGE:SLOPe?"))

' Save oscilloscope configuration.
Dim ResultsArray As Byte() ' Results array.
Dim nLength As Integer ' Number of bytes returned from inst.
Dim strPath As String
Dim fStream As FileStream

' Query and read setup string.
nLength = myScope.DoQueryIEEEBlock(":SYSTem:SETUp?", _
    ResultsArray)

' Write setup string to file.
strPath = "c:\scope\config\setup.stp"
fStream = File.Open(strPath, FileMode.Create)
fStream.Write(ResultsArray, 0, nLength)
fStream.Close()
Console.WriteLine("Setup bytes saved: {0}", nLength)

' Change settings with individual commands:

' Set vertical scale and offset.
myScope.DoCommand(":CHANnel1:SCALe 0.05")
Console.WriteLine("Channel 1 vertical scale: {0}", _
    myScope.DoQueryString(":CHANnel1:SCALe?"))

myScope.DoCommand(":CHANnel1:OFFSet -1.5")
Console.WriteLine("Channel 1 vertical offset: {0}", _
    myScope.DoQueryString(":CHANnel1:OFFSet?"))

' Set horizontal scale and position.
myScope.DoCommand(":TIMebase:SCALe 0.0002")
Console.WriteLine("Timebase scale: {0}", _
    myScope.DoQueryString(":TIMebase:SCALe?"))

myScope.DoCommand(":TIMebase:POSition 0.0")
Console.WriteLine("Timebase position: {0}", _
    myScope.DoQueryString(":TIMebase:POSition?"))

' Set the acquisition type (NORMal, PEAK, AVERage, or HRESolution)

myScope.DoCommand(":ACQuire:TYPE NORMal")
Console.WriteLine("Acquire type: {0}", _
    myScope.DoQueryString(":ACQuire:TYPE?"))

' Or, configure by loading a previously saved setup.
Dim DataArray As Byte()
Dim nBytesWritten As Integer

' Read setup string from file.
strPath = "c:\scope\config\setup.stp"
DataArray = File.ReadAllBytes(strPath)

```

```

' Restore setup string.
nBytesWritten = myScope.DoCommandIEEEBlock(":SYSTem:SETup", _
    dataArray)
Console.WriteLine("Setup bytes restored: {0}", nBytesWritten)

' Capture an acquisition using :DIGitize.
myScope.DoCommand(":DIGitize CHANnel1")

End Sub

'
' Analyze the captured waveform.
' -----

Private Shared Sub Analyze()

    Dim fResult As Double
    Dim ResultsArray As Byte() ' Results array.
    Dim nLength As Integer ' Number of bytes returned from inst.
    Dim strPath As String

    ' Make a couple of measurements.
    ' -----
    myScope.DoCommand(":MEASure:SOURce CHANnel1")
    Console.WriteLine("Measure source: {0}", _
        myScope.DoQueryString(":MEASure:SOURce?"))

    myScope.DoCommand(":MEASure:FREQuency")
    fResult = myScope.DoQueryNumber(":MEASure:FREQuency?")
    Console.WriteLine("Frequency: {0:F4} kHz", fResult / 1000)

    myScope.DoCommand(":MEASure:VAMplitude")
    fResult = myScope.DoQueryNumber(":MEASure:VAMplitude?")
    Console.WriteLine("Vertical amplitude: {0:F2} V", fResult)

    ' Download the screen image.
    ' -----
    myScope.DoCommand(":HARDcopy:INKSaver OFF")

    ' Get the screen data.
    nLength = myScope.DoQueryIEEEBlock(":DISPlay:DATA? PNG, COLor", _
        ResultsArray)

    ' Store the screen data to a file.
    strPath = "c:\scope\data\screen.png"
    Dim fStream As FileStream
    fStream = File.Open(strPath, FileMode.Create)
    fStream.Write(ResultsArray, 0, nLength)
    fStream.Close()
    Console.WriteLine("Screen image ({0} bytes) written to {1}", _
        nLength, strPath)

    ' Download waveform data.
    ' -----

    ' Set the waveform points mode.
    myScope.DoCommand(":WAVEform:POINTs:MODE RAW")

```

```

Console.WriteLine("Waveform points mode: {0}", _
    myScope.DoQueryString(":WAVEform:POINTS:MODE?"))

' Get the number of waveform points available.
myScope.DoCommand(":WAVEform:POINTS 10240")
Console.WriteLine("Waveform points available: {0}", _
    myScope.DoQueryString(":WAVEform:POINTS?"))

' Set the waveform source.
myScope.DoCommand(":WAVEform:SOURce CHANnel1")
Console.WriteLine("Waveform source: {0}", _
    myScope.DoQueryString(":WAVEform:SOURce?"))

' Choose the format of the data returned (WORD, BYTE, ASCII):
myScope.DoCommand(":WAVEform:FORMat BYTE")
Console.WriteLine("Waveform format: {0}", _
    myScope.DoQueryString(":WAVEform:FORMat?"))

' Display the waveform settings:
Dim fResultsArray As Double()
fResultsArray = myScope.DoQueryNumbers(":WAVEform:PREAmble?")

Dim fFormat As Double = fResultsArray(0)
If fFormat = 0 Then
    Console.WriteLine("Waveform format: BYTE")
ElseIf fFormat = 1 Then
    Console.WriteLine("Waveform format: WORD")
ElseIf fFormat = 2 Then
    Console.WriteLine("Waveform format: ASCii")
End If

Dim fType As Double = fResultsArray(1)
If fType = 0 Then
    Console.WriteLine("Acquire type: NORMAl")
ElseIf fType = 1 Then
    Console.WriteLine("Acquire type: PEAK")
ElseIf fType = 2 Then
    Console.WriteLine("Acquire type: AVERAge")
ElseIf fType = 3 Then
    Console.WriteLine("Acquire type: HRESolution")
End If

Dim fPoints As Double = fResultsArray(2)
Console.WriteLine("Waveform points: {0:e}", fPoints)

Dim fCount As Double = fResultsArray(3)
Console.WriteLine("Waveform average count: {0:e}", fCount)

Dim fXincrement As Double = fResultsArray(4)
Console.WriteLine("Waveform X increment: {0:e}", fXincrement)

Dim fXorigin As Double = fResultsArray(5)
Console.WriteLine("Waveform X origin: {0:e}", fXorigin)

Dim fXreference As Double = fResultsArray(6)
Console.WriteLine("Waveform X reference: {0:e}", fXreference)

```

```

Dim fYincrement As Double = fResultsArray(7)
Console.WriteLine("Waveform Y increment: {0:e}", fYincrement)

Dim fYorigin As Double = fResultsArray(8)
Console.WriteLine("Waveform Y origin: {0:e}", fYorigin)

Dim fYreference As Double = fResultsArray(9)
Console.WriteLine("Waveform Y reference: {0:e}", fYreference)

' Get the waveform data.
nLength = myScope.DoQueryIEEEBlock(":WAVEform:DATA?", _
    ResultsArray)
Console.WriteLine("Number of data values: {0}", nLength)

' Set up output file:
strPath = "c:\scope\data\waveform_data.csv"
If File.Exists(strPath) Then
    File.Delete(strPath)
End If

' Open file for output.
Dim writer As StreamWriter = File.CreateText(strPath)

' Output waveform data in CSV format.
For index As Integer = 0 To nLength - 1
    ' Write time value, voltage value.
    writer.WriteLine("{0:f9}, {1:f6}", _
        fXorigin + (CSng(index) * fXincrement), _
        ((CSng(ResultsArray(index)) - fYreference) _
        * fYincrement) + fYorigin)
Next

' Close output file.
writer.Close()
Console.WriteLine("Waveform format BYTE data written to {0}", _
    strPath)

End Sub

End Class

Class VisaInstrument
    Private m_nResourceManager As Integer
    Private m_nSession As Integer
    Private m_strVisaAddress As String

    ' Constructor.
    Public Sub New(ByVal strVisaAddress As String)
        ' Save VISA address in member variable.
        m_strVisaAddress = strVisaAddress

        ' Open the default VISA resource manager.
        OpenResourceManager()

        ' Open a VISA resource session.
        OpenSession()
    End Sub
End Class

```

```

    ' Clear the interface.
    Dim nViStatus As Integer
    nViStatus = visa32.viClear(m_nSession)
End Sub

Public Sub DoCommand(ByVal strCommand As String)
    ' Send the command.
    VisaSendCommandOrQuery(strCommand)

    ' Check for inst errors.
    CheckInstrumentErrors(strCommand)
End Sub

Public Function DoCommandIEEEBlock(ByVal strCommand As String, _
    ByVal dataArray As Byte()) As Integer

    ' Send the command to the device.
    Dim strCommandAndLength As String
    Dim nViStatus As Integer
    Dim nLength As Integer
    Dim nBytesWritten As Integer

    nLength = dataArray.Length
    strCommandAndLength = [String].Format("{0} #8{1:D8}", _
        strCommand, nLength)

    ' Write first part of command to formatted I/O write buffer.
    nViStatus = visa32.viPrintf(m_nSession, strCommandAndLength)
    CheckVisaStatus(nViStatus)

    ' Write the data to the formatted I/O write buffer.
    nViStatus = visa32.viBufWrite(m_nSession, dataArray, nLength, _
        nBytesWritten)
    CheckVisaStatus(nViStatus)

    ' Check for inst errors.
    CheckInstrumentErrors(strCommand)

    Return nBytesWritten
End Function

Public Function DoQueryString(ByVal strQuery As String) _
    As StringBuilder
    ' Send the query.
    VisaSendCommandOrQuery(strQuery)

    ' Get the result string.
    Dim strResults As New StringBuilder(1000)
    strResults = VisaGetResultString()

    ' Check for inst errors.
    CheckInstrumentErrors(strQuery)

    ' Return string results.
    Return strResults
End Function

```

```

Public Function DoQueryNumber(ByVal strQuery As String) As Double
    ' Send the query.
    VisaSendCommandOrQuery(strQuery)

    ' Get the result string.
    Dim fResults As Double
    fResults = VisaGetResultNumber()

    ' Check for inst errors.
    CheckInstrumentErrors(strQuery)

    ' Return string results.
    Return fResults
End Function

Public Function DoQueryNumbers(ByVal strQuery As String) _
    As Double()
    ' Send the query.
    VisaSendCommandOrQuery(strQuery)

    ' Get the result string.
    Dim fResultsArray As Double()
    fResultsArray = VisaGetResultNumbers()

    ' Check for instrument errors (another command and result).
    CheckInstrumentErrors(strQuery)

    ' Return string results.
    Return fResultsArray
End Function

Public Function DoQueryIEEEBlock(ByVal strQuery As String, _
    ByRef ResultsArray As Byte()) As Integer
    ' Send the query.
    VisaSendCommandOrQuery(strQuery)

    ' Get the result string.
    System.Threading.Thread.Sleep(2000) ' Delay before reading data.
    Dim length As Integer
    ' Number of bytes returned from instrument.
    length = VisaGetResultIEEEBlock(ResultsArray)

    ' Check for inst errors.
    CheckInstrumentErrors(strQuery)

    ' Return string results.
    Return length
End Function

Private Sub VisaSendCommandOrQuery(ByVal strCommandOrQuery _
    As String)
    ' Send command or query to the device.
    Dim strWithNewline As String
    strWithNewline = [String].Format("{0}" & Chr(10) & "", _
        strCommandOrQuery)
    Dim nViStatus As Integer

```

```

        nViStatus = visa32.viPrintf(m_nSession, strWithNewline)
        CheckVisaStatus(nViStatus)
    End Sub

Private Function VisaGetResultString() As StringBuilder
    Dim strResults As New StringBuilder(1000)

    ' Read return value string from the device.
    Dim nViStatus As Integer
    nViStatus = visa32.viScanf(m_nSession, "%1000t", strResults)
    CheckVisaStatus(nViStatus)

    Return strResults
End Function

Private Function VisaGetResultNumber() As Double
    Dim fResults As Double = 0

    ' Read return value string from the device.
    Dim nViStatus As Integer
    nViStatus = visa32.viScanf(m_nSession, "%lf", fResults)
    CheckVisaStatus(nViStatus)

    Return fResults
End Function

Private Function VisaGetResultNumbers() As Double()
    Dim fResultsArray As Double()
    fResultsArray = New Double(9) {}

    ' Read return value string from the device.
    Dim nViStatus As Integer
    nViStatus = visa32.viScanf(m_nSession, _
        "%,10lf" & Chr(10) & "", fResultsArray)
    CheckVisaStatus(nViStatus)

    Return fResultsArray
End Function

Private Function VisaGetResultIEEEBlock(ByRef ResultsArray _
    As Byte()) As Integer
    ' Results array, big enough to hold a PNG.
    ResultsArray = New Byte(299999) {}
    Dim length As Integer
    ' Number of bytes returned from instrument.
    ' Set the default number of bytes that will be contained in
    ' the ResultsArray to 300,000 (300kB).
    length = 300000

    ' Read return value string from the device.
    Dim nViStatus As Integer
    nViStatus = visa32.viScanf(m_nSession, "%#b", length, _
        ResultsArray)
    CheckVisaStatus(nViStatus)

    ' Write and read buffers need to be flushed after IEEE block?
    nViStatus = visa32.viFlush(m_nSession, visa32.VI_WRITE_BUF)

```



```

CheckVisaStatus(nViStatus)

nViStatus = visa32.viFlush(m_nSession, visa32.VI_READ_BUF)
CheckVisaStatus(nViStatus)

Return length
End Function

Private Sub CheckInstrumentErrors(ByVal strCommand As String)
    ' Check for instrument errors.
    Dim strInstrumentError As New StringBuilder(1000)
    Dim bFirstError As Boolean = True
    Do ' While not "0,No error"
        VisaSendCommandOrQuery(":SYSTem:ERRor?")
        strInstrumentError = VisaGetResultString()

        If Not strInstrumentError.ToString().StartsWith("+0,") Then
            If bFirstError Then
                Console.WriteLine("ERROR(s) for command '{0}': ", _
                    strCommand)
                bFirstError = False
            End If
            Console.Write(strInstrumentError)
        End If
    Loop While Not strInstrumentError.ToString().StartsWith("+0,")
End Sub

Private Sub OpenResourceManager()
    Dim nViStatus As Integer
    nViStatus = visa32.viOpenDefaultRM(Me.m_nResourceManager)
    If nViStatus < visa32.VI_SUCCESS Then
        Throw New _
            ApplicationException("Failed to open Resource Manager")
    End If
End Sub

Private Sub OpenSession()
    Dim nViStatus As Integer
    nViStatus = visa32.viOpen(Me.m_nResourceManager, _
        Me.m_strVisaAddress, visa32.VI_NO_LOCK, _
        visa32.VI_TMO_IMMEDIATE, Me.m_nSession)
    CheckVisaStatus(nViStatus)
End Sub

Public Sub SetTimeoutSeconds(ByVal nSeconds As Integer)
    Dim nViStatus As Integer
    nViStatus = visa32.viSetAttribute(Me.m_nSession, _
        visa32.VI_ATTR_TMO_VALUE, nSeconds * 1000)
    CheckVisaStatus(nViStatus)
End Sub

Public Sub CheckVisaStatus(ByVal nViStatus As Integer)
    ' If VISA error, throw exception.
    If nViStatus < visa32.VI_SUCCESS Then
        Dim strError As New StringBuilder(256)
        visa32.viStatusDesc(Me.m_nResourceManager, nViStatus, strError)
        Throw New ApplicationException(strError.ToString())
    End If
End Sub

```

```

        End If
    End Sub

    Public Sub Close()
        If m_nSession <> 0 Then
            visa32.viClose(m_nSession)
        End If
        If m_nResourceManager <> 0 Then
            visa32.viClose(m_nResourceManager)
        End If
    End Sub
End Class
End Namespace

```

VISA Example in Python

You can use the Python programming language with the PyVISA package to control Keysight InfiniiVision Series oscilloscopes.

The Python language and PyVISA package can be downloaded from the web at <http://www.python.org/> and <http://pyvisa.readthedocs.io/>, respectively.

To run this example with Python and PyVISA:

- 1 Cut-and-paste the code that follows into a file named "example.py".
- 2 Edit the program to use the VISA address of your oscilloscope.
- 3 If "python.exe" can be found via your PATH environment variable, open a Command Prompt window; then, change to the folder that contains the "example.py" file, and enter:

```

python example.py

#!python3
# -*- coding: utf-8 -*-

# *****
# This program illustrates a few commonly-used programming
# features of your Keysight oscilloscope.
# *****

# Import modules.
# -----
import pyvisa
import struct
import sys

# Global variables.
# -----
input_channel = "CHANnel1"
input_channel = "CHANnel2"
setup_file_name = "setup.scp"
screen_image_file_name = "screen_image.png"
waveform_data_file_name = "waveform_data.csv"
wfm_fmt = "BYTE"
wfm_fmt = "WORD"

```

```

debug = False

# =====
# Initialize:
# =====
def initialize():

    # Get and display the device's *IDN? string.
    idn_string = do_query_string("*IDN?")
    print(f"Identification string: '{idn_string}'")

    # Clear status and load the default setup.
    do_command("*CLS")
    do_command("*RST")

# =====
# Capture:
# =====
def capture():

    # Use auto-scale to automatically set up oscilloscope.
    print("Autoscale.")
    do_command(":AUToscale")

    # Set trigger mode.
    do_command(":TRIGger:MODE EDGE")
    qresult = do_query_string(":TRIGger:MODE?")
    print(f"Trigger mode: {qresult}")

    # Set EDGE trigger parameters.
    do_command(f":TRIGger:EDGE:SOURce {input_channel}")
    qresult = do_query_string(":TRIGger:EDGE:SOURce?")
    print(f"Trigger edge source: {qresult}")

    do_command(":TRIGger:EDGE:LEVel 1.5")
    qresult = do_query_string(":TRIGger:EDGE:LEVel?")
    print(f"Trigger edge level: {qresult}")

    do_command(":TRIGger:EDGE:SLOPe POSitive")
    qresult = do_query_string(":TRIGger:EDGE:SLOPe?")
    print(f"Trigger edge slope: {qresult}")

    # Save oscilloscope setup.
    setup_bytes = do_query_ieee_block(":SYSTem:SETup?")

    f = open(setup_file_name, "wb")
    f.write(setup_bytes)
    f.close()
    print(f"Setup bytes saved: {len(setup_bytes)}")

    # Change oscilloscope settings with individual commands:

    # Set vertical scale and offset.
    do_command(f":{input_channel}:SCALE 0.05")
    qresult = do_query_string(f":{input_channel}:SCALE?")

```

```

print(f"{input_channel} vertical scale: {qresult}")

do_command(f":{input_channel}:OFFSet -1.5")
qresult = do_query_string(f":{input_channel}:OFFSet?")
print(f"{input_channel} offset: {qresult}")

# Set horizontal scale and offset.
do_command(":TIMEbase:SCALe 0.0002")
qresult = do_query_string(":TIMEbase:SCALe?")
print(f"Timebase scale: {qresult}")

do_command(":TIMEbase:POSition 0.0")
qresult = do_query_string(":TIMEbase:POSition?")
print(f"Timebase position: {qresult}")

# Set the acquisition type.
do_command(":ACQuire:TYPE NORMal")
qresult = do_query_string(":ACQuire:TYPE?")
print(f"Acquire type: {qresult}")

# Or, set up oscilloscope by loading a previously saved setup.
setup_bytes = ""
f = open(setup_file_name, "rb")
setup_bytes = f.read()
f.close()
do_command_ieee_block(":SYSTem:SETup", setup_bytes)
print(f"Setup bytes restored: {len(setup_bytes)}")

# Capture an acquisition using :DIGitize.
do_command(f":DIGitize {input_channel}")

# =====
# Analyze:
# =====
def analyze():

    # Make measurements.
    # -----
    do_command(f":MEASure:SOURce {input_channel}")
    qresult = do_query_string(":MEASure:SOURce?")
    print(f"Measure source: {qresult}")

    do_command(":MEASure:FREQuency")
    qresult = do_query_string(":MEASure:FREQuency?")
    print(f"Measured frequency on {input_channel}: {qresult}")

    do_command(":MEASure:VAMPlitude")
    qresult = do_query_string(":MEASure:VAMPlitude?")
    print(f"Measured vertical amplitude on {input_channel}: {qresult}")

    # Download the screen image.
    # -----
    do_command(":HARDcopy:INKSaver OFF")

    screen_bytes = do_query_ieee_block(":DISPlay:DATA? PNG, COLor")

```

```

# Save display data values to file.
f = open(screen_image_file_name, "wb")
f.write(screen_bytes)
f.close()
print(f"Screen image written to {screen_image_file_name}.")

# Download waveform data.
# -----

# Set the waveform source.
do_command(f":WAVEform:SOURce {input_channel}")
qresult = do_query_string(":WAVEform:SOURce?")
print(f"Waveform source: {qresult}")

# Set the waveform points mode.
do_command(":WAVEform:POINts:MODE RAW")
qresult = do_query_string(":WAVEform:POINts:MODE?")
print(f"Waveform points mode: {qresult}")

# Get the number of waveform points available.
do_command(":WAVEform:POINts 10240")
qresult = do_query_string(":WAVEform:POINts?")
print(f"Waveform points available: {qresult}")

# Choose the format of the data returned (BYTE or WORD):
do_command(f":WAVEform:FORMat {wfm_fmt}")
qresult = do_query_string(":WAVEform:FORMat?")
print(f"Waveform format: {qresult}")

# Specify the byte order in WORD data.
if wfm_fmt == "WORD":
    do_command(":WAVEform:BYTeorder LSBF")
    qresult = do_query_string(":WAVEform:BYTeorder?")
    print(f"Waveform byte order for WORD data: {qresult}")

# Display the waveform settings from preamble:
wav_form_dict = {
    0 : "BYTE",
    1 : "WORD",
    4 : "ASCIi",
}
acq_type_dict = {
    0 : "NORMal",
    1 : "PEAK",
    2 : "AVERage",
    3 : "HRESolution",
}

preamble_string = do_query_string(":WAVEform:PREamble?")
(
    wav_form, acq_type, wfmpmts, avgcnt, x_increment, x_origin,
    x_reference, y_increment, y_origin, y_reference
) = preamble_string.split(",")

print(f"Waveform format: {wav_form_dict[int(wav_form)]}")
print(f"Acquire type: {acq_type_dict[int(acq_type)]}")
print(f"Waveform points desired: {wfmpmts}")

```

```

print(f"Waveform average count: {avgcnt}")
print(f"Waveform X increment: {x_increment}")
print(f"Waveform X origin: {x_origin}")
print(f"Waveform X reference: {x_reference}")    # Always 0.
print(f"Waveform Y increment: {y_increment}")
print(f"Waveform Y origin: {y_origin}")
print(f"Waveform Y reference: {y_reference}")

# Get numeric values for later calculations.
x_increment = do_query_number(":WAVEform:XINCrement?")
x_origin = do_query_number(":WAVEform:XORigin?")
y_increment = do_query_number(":WAVEform:YINCrement?")
y_origin = do_query_number(":WAVEform:YORigin?")
y_reference = do_query_number(":WAVEform:YREFerence?")

# Get the waveform data.
data_bytes = do_query_ieee_block(":WAVEform:DATA?")
data_bytes_length = len(data_bytes)
print(f"Byte count: {data_bytes_length}")

if wfm_fmt == "BYTE":
    block_points = data_bytes_length
elif wfm_fmt == "WORD":
    block_points = data_bytes_length / 2

# Unpack or split into list of data values.
if wfm_fmt == "BYTE":
    values = struct.unpack("%dB" % block_points, data_bytes)
elif wfm_fmt == "WORD":
    values = struct.unpack("%dH" % block_points, data_bytes)
print(f"Number of data values: {len(values)}")

# Save waveform data values to CSV file.
f = open(waveform_data_file_name, "w")

for i in range(0, len(values) - 1):
    time_val = x_origin + (i * x_increment)
    voltage = ((values[i] - y_reference) * y_increment) + y_origin
    f.write(f"{time_val:E}, {voltage:f}\n")

# Close output file.
f.close()
print(f"Waveform format {wfm_fmt} data written to {waveform_data_file_name}.")

# =====
# Send a command and check for errors:
# =====
def do_command(command, hide_params=False):

    if hide_params:
        (header, data) = command.split(" ", 1)
        if debug:
            print(f"Cmd = '{header}'")
    else:
        if debug:

```

```

    print(f"Cmd = '{command}'")

InfiniiVision.write(f"{command}")

if hide_params:
    check_instrument_errors(header)
else:
    check_instrument_errors(command)

# =====
# Send a command and binary values and check for errors:
# =====
def do_command_ieee_block(command, values):
    if debug:
        print(f"Cmb = '{command}'")
    InfiniiVision.write_binary_values(f"{command} ", values, datatype='B')
    check_instrument_errors(command)

# =====
# Send a query, check for errors, return string:
# =====
def do_query_string(query):
    if debug:
        print(f"Qys = '{query}'")
    result = InfiniiVision.query(f"{query}")
    check_instrument_errors(query)
    return result.strip()

# =====
# Send a query, check for errors, return floating-point value:
# =====
def do_query_number(query):
    if debug:
        print(f"Qyn = '{query}'")
    results = InfiniiVision.query(f"{query}")
    check_instrument_errors(query)
    return float(results)

# =====
# Send a query, check for errors, return binary values:
# =====
def do_query_ieee_block(query):
    if debug:
        print(f"Qyb = '{query}'")
    result = InfiniiVision.query_binary_values(f"{query}", datatype='s', container=bytes)
    check_instrument_errors(query)
    return result

# =====
# Check for instrument errors:
# =====

```

```

def check_instrument_errors(command):

    while True:
        error_string = InfiniiVision.query(":SYSTem:ERRor?")
        if error_string:    # If there is an error string value.

            if error_string.find("+0,", 0, 3) == -1:    # Not "No error".

                print(f"ERROR: {error_string}, command: '{command}'")
                print("Exited because of error.")
                sys.exit(1)

            else:    # "No error"
                break

        else:    # :SYSTem:ERRor? should always return string.
            print(f"ERROR: :SYSTem:ERRor? returned nothing, command: '{command}'")
    )
    print("Exited because of error.")
    sys.exit(1)

# =====
# Main program:
# =====

rm = pyvisa.ResourceManager("C:\\Windows\\System32\\visa64.dll")
InfiniiVision = rm.open_resource("TCPIP0::k-mx3104g-10104.cos.is.keysigh
t.com::inst0::INSTR")

InfiniiVision.timeout = 15000
InfiniiVision.clear()

# Initialize the oscilloscope, capture data, and analyze.
initialize()
capture()
analyze()

InfiniiVision.close()
print("End of program.")
sys.exit()

```


VISA.NET Examples

These programming examples show how to use the VISA.NET drivers that come with Keysight IO Libraries Suite.

- **"VISA.NET Example in C#" on page 1741**
- **"VISA.NET Example in Visual Basic .NET" on page 1747**
- **"VISA.NET Example in Python" on page 1753**

VISA.NET Example in C#

To compile and run this example in Microsoft Visual Studio 2013:

- 1 Open Visual Studio.
- 2 Choose **FILE > New > Project....**
- 3 In the New Project dialog box, select **.NET Framework 4.5.2**.
- 4 Create a new Visual C#, Console Application project.
- 5 Cut-and-paste the code that follows into the C# source file.
- 6 Edit the program to use the VISA address of your oscilloscope.
- 7 Add a reference to the VISA.NET driver:
 - a Right-click the project you wish to modify (not the solution) in the Solution Explorer window of the Microsoft Visual Studio environment.
 - b Choose **Add Reference....**
 - c In the Reference Manager dialog box, under **Assemblies**, select **Extensions**.
 - d In the "Targeting: .NET Framework 4.5.2" list, select the **Ivi.Visa Assembly** check box; then, click **OK**.
- 8 Build and run the program.

For more information, see the VISA.NET Help that comes with Keysight IO Libraries Suite.

```

/*
 * Keysight VISA.NET Example in C#
 * -----
 * This program illustrates a few commonly used programming
 * features of your Keysight InfiniiVision oscilloscope.
 * -----
 */

using System;
using System.IO;
using System.Collections.Generic;
using System.Text;

using Ivi.Visa;
using Ivi.Visa.FormattedIO;

```

```

namespace Example
{
    class Program
    {
        static void Main(string[] args)
        {
            // Change this variable to the address of your instrument
            string VISA_ADDRESS = "TCPIP0::141.121.230.6::inst0::INSTR";

            // Create a connection (session) to the instrument
            IMessageBasedSession session;
            try
            {
                session = GlobalResourceManager.Open(VISA_ADDRESS) as
                    IMessageBasedSession;
            }
            catch (NativeVisaException visaException)
            {
                Console.WriteLine("Couldn't connect.");
                Console.WriteLine("Error is:\r\n{0}\r\n", visaException);
                Console.WriteLine("Press any key to exit...");
                Console.ReadKey();
                return;
            }

            // Create a formatted I/O object which will help us format the
            // data we want to send/receive to/from the instrument
            MessageBasedFormattedIO myScope =
                new MessageBasedFormattedIO(session);

            // For Serial and TCP/IP socket connections enable the read
            // Termination Character, or read's will timeout
            if (session.ResourceName.Contains("ASRL") ||
                session.ResourceName.Contains("SOCKET"))
                session.TerminationCharacterEnabled = true;

            session.TimeoutMilliseconds = 20000;

            // Initialize - start from a known state.
            // =====
            string strResults;
            FileStream fStream;

            // Get and display the device's *IDN? string.
            myScope.WriteLine("*IDN?");
            strResults = myScope.ReadLine();
            Console.WriteLine("*IDN? result is: {0}", strResults);

            // Clear status and load the default setup.
            myScope.WriteLine("*CLS");
            myScope.WriteLine("*RST");

            // Capture data.
            // =====

```

```

// Use auto-scale to automatically configure oscilloscope.
myScope.WriteLine(":AUToscale");

// Set trigger mode (EDGE, PULSe, PATtern, etc., and input source.
myScope.WriteLine(":TRIGger:MODE EDGE");
myScope.WriteLine(":TRIGger:MODE?");
strResults = myScope.ReadLine();
Console.WriteLine("Trigger mode: {0}", strResults);

// Set EDGE trigger parameters.
myScope.WriteLine(":TRIGger:EDGE:SOURce CHANnel1");
myScope.WriteLine(":TRIGger:EDGE:SOURce?");
strResults = myScope.ReadLine();
Console.WriteLine("Trigger edge source: {0}", strResults);

myScope.WriteLine(":TRIGger:EDGE:LEVel 1.5");
myScope.WriteLine(":TRIGger:EDGE:LEVel?");
strResults = myScope.ReadLine();
Console.WriteLine("Trigger edge level: {0}", strResults);

myScope.WriteLine(":TRIGger:EDGE:SLOPe POSitive");
myScope.WriteLine(":TRIGger:EDGE:SLOPe?");
strResults = myScope.ReadLine();
Console.WriteLine("Trigger edge slope: {0}", strResults);

// Save oscilloscope configuration.
byte[] ResultsArray; // Results array.
int nLength; // Number of bytes returned from instrument.
string strPath;

// Query and read setup string.
myScope.WriteLine(":SYSTem:SETup?");
ResultsArray = myScope.ReadLineBinaryBlockOfByte();
nLength = ResultsArray.Length;

// Write setup string to file.
strPath = "c:\\scope\\config\\setup.stp";
fStream = File.Open(strPath, FileMode.Create);
fStream.Write(ResultsArray, 0, nLength);
fStream.Close();
Console.WriteLine("Setup bytes saved: {0}", nLength);

// Change settings with individual commands:

// Set vertical scale and offset.
myScope.WriteLine(":CHANnel1:SCALe 0.05");
myScope.WriteLine(":CHANnel1:SCALe?");
strResults = myScope.ReadLine();
Console.WriteLine("Channel 1 vertical scale: {0}", strResults);

myScope.WriteLine(":CHANnel1:OFFSet -1.5");
myScope.WriteLine(":CHANnel1:OFFSet?");
strResults = myScope.ReadLine();
Console.WriteLine("Channel 1 vertical offset: {0}", strResults);

// Set horizontal scale and offset.
myScope.WriteLine(":TIMEbase:SCALe 0.0002");

```

```

myScope.WriteLine(":TIMEbase:SCALE?");
strResults = myScope.ReadLine();
Console.WriteLine("Timebase scale: {0}", strResults);

myScope.WriteLine(":TIMEbase:POSition 0.0");
myScope.WriteLine(":TIMEbase:POSition?");
strResults = myScope.ReadLine();
Console.WriteLine("Timebase position: {0}", strResults);

// Set the acquisition type (NORMal, PEAK, AVERage, or HRESolution
).
myScope.WriteLine(":ACQuire:TYPE NORMal");
myScope.WriteLine(":ACQuire:TYPE?");
strResults = myScope.ReadLine();
Console.WriteLine("Acquire type: {0}", strResults);

// Or, configure by loading a previously saved setup.
byte[] DataArray;
int nBytesWritten;

// Read setup string from file.
strPath = "c:\\scope\\config\\setup.stp";
DataArray = File.ReadAllBytes(strPath);
nBytesWritten = DataArray.Length;

// Restore setup string.
myScope.Write(":SYSTem:SETup ");
myScope.WriteBinary(DataArray);
myScope.WriteLine("");
Console.WriteLine("Setup bytes restored: {0}", nBytesWritten);

// Capture an acquisition using :DIGitize.
myScope.WriteLine(":DIGitize CHANnel1");

// Analyze the captured waveform.
// =====

// Make a couple of measurements.
// -----
myScope.WriteLine(":MEASure:SOURce CHANnel1");
myScope.WriteLine(":MEASure:SOURce?");
strResults = myScope.ReadLine();
Console.WriteLine("Measure source: {0}", strResults);

double fResult;
myScope.WriteLine(":MEASure:FREQuency");
myScope.WriteLine(":MEASure:FREQuency?");
fResult = myScope.ReadLineDouble();
Console.WriteLine("Frequency: {0:F4} kHz", fResult / 1000);

myScope.WriteLine(":MEASure:VAMPlitude");
myScope.WriteLine(":MEASure:VAMPlitude?");
fResult = myScope.ReadLineDouble();
Console.WriteLine("Vertical amplitude: {0:F2} V", fResult);

// Download the screen image.
// -----

```

```

myScope.WriteLine(":HARDcopy:INKSaver OFF");

// Get the screen data.
myScope.WriteLine(":DISPlay:DATA? PNG, COlor");
ResultsArray = myScope.ReadLineBinaryBlockOfByte();
nLength = ResultsArray.Length;

// Store the screen data to a file.
strPath = "c:\\scope\\data\\screen.png";
fStream = File.Open(strPath, FileMode.Create);
fStream.Write(ResultsArray, 0, nLength);
fStream.Close();
Console.WriteLine("Screen image ({0} bytes) written to {1}",
    nLength, strPath);

// Download waveform data.
// -----

// Set the waveform points mode.
myScope.WriteLine(":WAVEform:POINts:MODE RAW");
myScope.WriteLine(":WAVEform:POINts:MODE?");
strResults = myScope.ReadLine();
Console.WriteLine("Waveform points mode: {0}", strResults);

// Get the number of waveform points available.
myScope.WriteLine(":WAVEform:POINts?");
strResults = myScope.ReadLine();
Console.WriteLine("Waveform points available: {0}", strResults);

// Set the waveform source.
myScope.WriteLine(":WAVEform:SOURce CHANnel1");
myScope.WriteLine(":WAVEform:SOURce?");
strResults = myScope.ReadLine();
Console.WriteLine("Waveform source: {0}", strResults);

// Choose the format of the data returned (WORD, BYTE, ASCII):
myScope.WriteLine(":WAVEform:FORMat BYTE");
myScope.WriteLine(":WAVEform:FORMat?");
strResults = myScope.ReadLine();
Console.WriteLine("Waveform format: {0}", strResults);

// Display the waveform settings:
double[] fResultsArray;
myScope.WriteLine(":WAVEform:PREamble?");
fResultsArray = myScope.ReadLineListofDouble();

double fFormat = fResultsArray[0];
if (fFormat == 0.0)
{
    Console.WriteLine("Waveform format: BYTE");
}
else if (fFormat == 1.0)
{
    Console.WriteLine("Waveform format: WORD");
}
else if (fFormat == 2.0)
{

```

```

        Console.WriteLine("Waveform format: ASCii");
    }

    double fType = fResultsArray[1];
    if (fType == 0.0)
    {
        Console.WriteLine("Acquire type: NORMAl");
    }
    else if (fType == 1.0)
    {
        Console.WriteLine("Acquire type: PEAK");
    }
    else if (fType == 2.0)
    {
        Console.WriteLine("Acquire type: AVERAge");
    }
    else if (fType == 3.0)
    {
        Console.WriteLine("Acquire type: HRESolution");
    }

    double fPoints = fResultsArray[2];
    Console.WriteLine("Waveform points: {0:e}", fPoints);

    double fCount = fResultsArray[3];
    Console.WriteLine("Waveform average count: {0:e}", fCount);

    double fXincrement = fResultsArray[4];
    Console.WriteLine("Waveform X increment: {0:e}", fXincrement);

    double fXorigin = fResultsArray[5];
    Console.WriteLine("Waveform X origin: {0:e}", fXorigin);

    double fXreference = fResultsArray[6];
    Console.WriteLine("Waveform X reference: {0:e}", fXreference);

    double fYincrement = fResultsArray[7];
    Console.WriteLine("Waveform Y increment: {0:e}", fYincrement);

    double fYorigin = fResultsArray[8];
    Console.WriteLine("Waveform Y origin: {0:e}", fYorigin);

    double fYreference = fResultsArray[9];
    Console.WriteLine("Waveform Y reference: {0:e}", fYreference);

    // Read waveform data.
    myScope.WriteLine(":WAVEform:DATA?");
    ResultsArray = myScope.ReadLineBinaryBlockOfByte();
    nLength = ResultsArray.Length;
    Console.WriteLine("Number of data values: {0}", nLength);

    // Set up output file:
    strPath = "c:\\scope\\data\\waveform_data.csv";
    if (File.Exists(strPath)) File.Delete(strPath);

    // Open file for output.
    StreamWriter writer = File.CreateText(strPath);

```

```

// Output waveform data in CSV format.
for (int i = 0; i < nLength - 1; i++)
    writer.WriteLine("{0:f9}, {1:f6}",
        fXorigin + ((float)i * fXincrement),
        (((float)ResultsArray[i] - fYreference)
        * fYincrement) + fYorigin);

// Close output file.
writer.Close();
Console.WriteLine("Waveform format BYTE data written to {0}",
    strPath);

// Close the connection to the instrument
// -----
session.Dispose();

Console.WriteLine("Press any key to exit...");
Console.ReadKey();

    }
}

```

VISA.NET Example in Visual Basic .NET

To compile and run this example in Microsoft Visual Studio 2013:

- 1 Open Visual Studio.
- 2 Choose **FILE > New > Project...**
- 3 In the New Project dialog box, select **.NET Framework 4.5.2**.
- 4 Create a new Visual Basic, Console Application project.
- 5 Cut-and-paste the code that follows into the Visual Basic .NET source file.
- 6 Edit the program to use the VISA address of your oscilloscope.
- 7 Add a reference to the VISA.NET driver:
 - a Right-click the project you wish to modify (not the solution) in the Solution Explorer window of the Microsoft Visual Studio environment.
 - b Choose **Add Reference...**
 - c In the Reference Manager dialog box, under **Assemblies**, select **Extensions**.
 - d In the "Targeting: .NET Framework 4.5.2" list, select the **Ivi.Visa Assembly** check box; then, click **OK**.

- 8 Specify the Startup object:
 - a Right-click the project you wish to modify (not the solution) in the Solution Explorer window of the Microsoft Visual Studio environment.
 - b Choose **Properties**.
 - c In the Properties dialog box, under **Application**, select the **Startup object:** field and choose **Sub Main**.
 - d Save your change and close the Properties dialog box.
- 9 Build and run the program.

For more information, see the VISA.NET driver help that comes with Keysight Command Expert.

```
'
' Keysight VISA.NET Example in VB.NET
' -----
' This program illustrates a few commonly used programming
' features of your Keysight InfiniiVision oscilloscope.
' -----

Imports System
Imports System.IO
Imports System.Collections.Generic
Imports System.Text

Imports Ivi.Visa
Imports Ivi.Visa.FormattedIO

Namespace Example
  Class Program

    Public Shared Sub Main(args As String())
      ' Change this variable to the address of your instrument
      Dim VISA_ADDRESS As String = "TCPIP0::141.121.230.6::inst0::INSTR"

      ' Create a connection (session) to the instrument
      Dim session As IMessageBasedSession
      Try
        session = TryCast(GlobalResourceManager.Open(VISA_ADDRESS), _
          IMessageBasedSession)
      Catch visaException As NativeVisaException
        Console.WriteLine("Couldn't connect.")
        Console.WriteLine("Error is:" & vbCrLf & "{0}" _
          & vbCrLf & vbCrLf, visaException)
        Console.WriteLine("Press any key to exit...")
        Console.ReadKey()
        Return
      End Try

      ' Create a formatted I/O object which will help us format the
      ' data we want to send/receive to/from the instrument
      Dim myScope As New MessageBasedFormattedIO(session)
```



```

' For Serial and TCP/IP socket connections enable the read
' Termination Character, or read's will timeout
If session.ResourceName.Contains("ASRL") OrElse _
    session.ResourceName.Contains("SOCKET") Then
    session.TerminationCharacterEnabled = True
End If

session.TimeoutMilliseconds = 20000

' Initialize - start from a known state.
' =====
Dim strResults As String
Dim fStream As FileStream

' Get and display the device's *IDN? string.
myScope.WriteLine("*IDN?")
strResults = myScope.ReadLine()
Console.WriteLine("*IDN? result is: {0}", strResults)

' Clear status and load the default setup.
myScope.WriteLine("*CLS")
myScope.WriteLine("*RST")

' Capture data.
' =====
' Use auto-scale to automatically configure oscilloscope.
myScope.WriteLine(":AUToscale")

' Set trigger mode (EDGE, PULSe, PATtern, etc., and input source.
myScope.WriteLine(":TRIGger:MODE EDGE")
myScope.WriteLine(":TRIGger:MODE?")
strResults = myScope.ReadLine()
Console.WriteLine("Trigger mode: {0}", strResults)

' Set EDGE trigger parameters.
myScope.WriteLine(":TRIGger:EDGE:SOURce CHANnel1")
myScope.WriteLine(":TRIGger:EDGE:SOURce?")
strResults = myScope.ReadLine()
Console.WriteLine("Trigger edge source: {0}", strResults)

myScope.WriteLine(":TRIGger:EDGE:LEVel 1.5")
myScope.WriteLine(":TRIGger:EDGE:LEVel?")
strResults = myScope.ReadLine()
Console.WriteLine("Trigger edge level: {0}", strResults)

myScope.WriteLine(":TRIGger:EDGE:SLOPe POSitive")
myScope.WriteLine(":TRIGger:EDGE:SLOPe?")
strResults = myScope.ReadLine()
Console.WriteLine("Trigger edge slope: {0}", strResults)

' Save oscilloscope configuration.
Dim ResultsArray As Byte()
' Results array.
Dim nLength As Integer
' Number of bytes returned from instrument.
Dim strPath As String

```

```

' Query and read setup string.
myScope.WriteLine(":SYSTem:SETup?")
ResultsArray = myScope.ReadLineBinaryBlockOfByte()
nLength = ResultsArray.Length

' Write setup string to file.
strPath = "c:\scope\config\setup.stp"
fStream = File.Open(strPath, FileMode.Create)
fStream.Write(ResultsArray, 0, nLength)
fStream.Close()
Console.WriteLine("Setup bytes saved: {0}", nLength)

' Change settings with individual commands:

' Set vertical scale and offset.
myScope.WriteLine(":CHANnel1:SCALe 0.05")
myScope.WriteLine(":CHANnel1:SCALe?")
strResults = myScope.ReadLine()
Console.WriteLine("Channel 1 vertical scale: {0}", strResults)

myScope.WriteLine(":CHANnel1:OFFSet -1.5")
myScope.WriteLine(":CHANnel1:OFFSet?")
strResults = myScope.ReadLine()
Console.WriteLine("Channel 1 vertical offset: {0}", strResults)

' Set horizontal scale and offset.
myScope.WriteLine(":TIMebase:SCALe 0.0002")
myScope.WriteLine(":TIMebase:SCALe?")
strResults = myScope.ReadLine()
Console.WriteLine("Timebase scale: {0}", strResults)

myScope.WriteLine(":TIMebase:POSition 0.0")
myScope.WriteLine(":TIMebase:POSition?")
strResults = myScope.ReadLine()
Console.WriteLine("Timebase position: {0}", strResults)

' Set the acquisition type (NORMal, PEAK, AVERage, or HRESolution)

myScope.WriteLine(":ACQuire:TYPE NORMal")
myScope.WriteLine(":ACQuire:TYPE?")
strResults = myScope.ReadLine()
Console.WriteLine("Acquire type: {0}", strResults)

' Or, configure by loading a previously saved setup.
Dim dataArray As Byte()
Dim nBytesWritten As Integer

' Read setup string from file.
strPath = "c:\scope\config\setup.stp"
dataArray = File.ReadAllBytes(strPath)
nBytesWritten = dataArray.Length

' Restore setup string.
myScope.Write(":SYSTem:SETup ")
myScope.WriteBinary(dataArray)
myScope.WriteLine("")
Console.WriteLine("Setup bytes restored: {0}", nBytesWritten)

```

```

' Capture an acquisition using :DIGitize.
myScope.WriteLine(":DIGitize CHANnel1")

' Analyze the captured waveform.
' =====

' Make a couple of measurements.
' -----
myScope.WriteLine(":MEASure:SOURce CHANnel1")
myScope.WriteLine(":MEASure:SOURce?")
strResults = myScope.ReadLine()
Console.WriteLine("Measure source: {0}", strResults)

Dim fResult As Double
myScope.WriteLine(":MEASure:FREQuency")
myScope.WriteLine(":MEASure:FREQuency?")
fResult = myScope.ReadLineDouble()
Console.WriteLine("Frequency: {0:F4} kHz", fResult / 1000)

myScope.WriteLine(":MEASure:VAMPlitude")
myScope.WriteLine(":MEASure:VAMPlitude?")
fResult = myScope.ReadLineDouble()
Console.WriteLine("Vertical amplitude: {0:F2} V", fResult)

' Download the screen image.
' -----
myScope.WriteLine(":HARDcopy:INKSaver OFF")

' Get the screen data.
myScope.WriteLine(":DISPlay:DATA? PNG, COLOr")
ResultsArray = myScope.ReadLineBinaryBlockOfByte()
nLength = ResultsArray.Length

' Store the screen data to a file.
strPath = "c:\scope\data\screen.png"
fStream = File.Open(strPath, FileMode.Create)
fStream.Write(ResultsArray, 0, nLength)
fStream.Close()
Console.WriteLine("Screen image ({0} bytes) written to {1}", _
    nLength, strPath)

' Download waveform data.
' -----

' Set the waveform points mode.
myScope.WriteLine(":WAVEform:POINts:MODE RAW")
myScope.WriteLine(":WAVEform:POINts:MODE?")
strResults = myScope.ReadLine()
Console.WriteLine("Waveform points mode: {0}", strResults)

' Get the number of waveform points available.
myScope.WriteLine(":WAVEform:POINts?")
strResults = myScope.ReadLine()
Console.WriteLine("Waveform points available: {0}", strResults)

' Set the waveform source.

```

```

myScope.WriteLine(":WAVEform:SOURce CHANnel1")
myScope.WriteLine(":WAVEform:SOURce?")
strResults = myScope.ReadLine()
Console.WriteLine("Waveform source: {0}", strResults)

' Choose the format of the data returned (WORD, BYTE, ASCII):
myScope.WriteLine(":WAVEform:FORMat BYTE")
myScope.WriteLine(":WAVEform:FORMat?")
strResults = myScope.ReadLine()
Console.WriteLine("Waveform format: {0}", strResults)

' Display the waveform settings:
Dim fResultsArray As Double()
myScope.WriteLine(":WAVEform:PREAmble?")
fResultsArray = myScope.ReadLineListOfDouble()

Dim fFormat As Double = fResultsArray(0)
If fFormat = 0.0 Then
    Console.WriteLine("Waveform format: BYTE")
ElseIf fFormat = 1.0 Then
    Console.WriteLine("Waveform format: WORD")
ElseIf fFormat = 2.0 Then
    Console.WriteLine("Waveform format: ASCii")
End If

Dim fType As Double = fResultsArray(1)
If fType = 0.0 Then
    Console.WriteLine("Acquire type: NORMAl")
ElseIf fType = 1.0 Then
    Console.WriteLine("Acquire type: PEAK")
ElseIf fType = 2.0 Then
    Console.WriteLine("Acquire type: AVERAge")
ElseIf fType = 3.0 Then
    Console.WriteLine("Acquire type: HRESolution")
End If

Dim fPoints As Double = fResultsArray(2)
Console.WriteLine("Waveform points: {0:e}", fPoints)

Dim fCount As Double = fResultsArray(3)
Console.WriteLine("Waveform average count: {0:e}", fCount)

Dim fXincrement As Double = fResultsArray(4)
Console.WriteLine("Waveform X increment: {0:e}", fXincrement)

Dim fXorigin As Double = fResultsArray(5)
Console.WriteLine("Waveform X origin: {0:e}", fXorigin)

Dim fXreference As Double = fResultsArray(6)
Console.WriteLine("Waveform X reference: {0:e}", fXreference)

Dim fYincrement As Double = fResultsArray(7)
Console.WriteLine("Waveform Y increment: {0:e}", fYincrement)

Dim fYorigin As Double = fResultsArray(8)
Console.WriteLine("Waveform Y origin: {0:e}", fYorigin)

```

```

Dim fYreference As Double = fResultsArray(9)
Console.WriteLine("Waveform Y reference: {0:e}", fYreference)

' Read waveform data.
myScope.WriteLine(":WAVEform:DATA?")
ResultsArray = myScope.ReadLineBinaryBlockOfByte()
nLength = ResultsArray.Length
Console.WriteLine("Number of data values: {0}", nLength)

' Set up output file:
strPath = "c:\scope\data\waveform_data.csv"
If File.Exists(strPath) Then
    File.Delete(strPath)
End If

' Open file for output.
Dim writer As StreamWriter = File.CreateText(strPath)

' Output waveform data in CSV format.
For i As Integer = 0 To nLength - 2
    writer.WriteLine("{0:f9}, {1:f6}", _
        fXorigin + (CSng(i) * fXincrement), _
        ((CSng(ResultsArray(i)) - fYreference) * fYincrement) + fYorigin)
Next

' Close output file.
writer.Close()
Console.WriteLine("Waveform format BYTE data written to {0}", _
    strPath)

' Close the connection to the instrument
' -----
session.Dispose()

Console.WriteLine("Press any key to exit...")
Console.ReadKey()

End Sub
End Class
End Namespace

```

VISA.NET Example in Python

You can use the Python programming language with the "Python.NET" package (or IronPython) to control Keysight oscilloscopes.

The Python language and "Python.NET" package can be downloaded from the web at <http://www.python.org/> and <http://pythonnet.github.io/>, respectively.

To run this example with Python and "Python.NET":

- 1 Cut-and-paste the code that follows into a file named "example.py".
- 2 Edit the program to use the VISA address of your oscilloscope.

- 3 If "python.exe" can be found via your PATH environment variable, open a Command Prompt window; then, change to the folder that contains the "example.py" file, and enter:

```
python example.py

#!python3
# -*- coding: utf-8 -*-

# Keysight VISA.NET Example in Python.NET or IronPython
# *****
# This program illustrates a few commonly-used programming
# features of your Keysight InfiniiVision oscilloscope.
# *****

# Import Python modules.
# -----
import sys
sys.path.append("C:\\Program Files\\IVI Foundation\\VISA\\Microsoft.NET\\
\\Framework64\\v2.0.50727\\VISA.NET Shared Components 5.8.0")

# Import .NET modules.
# -----
import clr
clr.AddReference("Ivi.Visa")
from Ivi.Visa import *
from Ivi.Visa.FormattedIO import *
from System import *
from System.IO import *

# Global variables.
# -----
visa_addr = "TCPIP0::k-mx3104g-10104.cos.is.keysight.com::inst0::INSTR"
io_timeout_ms = 20000
# input_channel = "CHANnel1"
input_channel = "CHANnel2"
setup_file_name = "setup.scp"
screen_image_file_name = "screen_image.png"
waveform_data_file_name = "waveform_data.csv"
# wfm_fmt = "BYTE"
wfm_fmt = "WORD"

# =====
# Check for instrument errors:
# =====
def check_instrument_errors(when):

    errors_found = False
    while True:
        # Keep reading errors until "No error".
        myScope.WriteLine(":SYSTem:ERRor?")
        error_string = myScope.ReadLine().strip()
        if error_string: # If there is an error string value.

            if error_string.find("+0,", 0, 3) == -1: # Not "No error".
                errors_found = True
```

```

        print(f"ERROR: {error_string}")

    else:    # "No error"
        break

    else:    # :SYSTem:ERROr? should always return string.
        errors_found = True
        print("ERROR: ':SYSTem:ERROr?' empty.")
        break

if errors_found:
    print(f"Exited because error(s) found when: '{when}'")
    sys.exit(1)

# =====
# Initialize:
# =====
def initialize():

    # Get and display the device's *IDN? string.
    myScope.WriteLine("*IDN?")
    idn_string = myScope.ReadLine().strip()
    print(f"Identification string: '{idn_string}'")

    # Clear status and load the default setup.
    myScope.WriteLine("*CLS")
    myScope.WriteLine("*RST")

# =====
# Capture:
# =====
def capture():

    # Display the input channel.
    myScope.WriteLine(":CHANnel1:DISPlay OFF")
    myScope.WriteLine(f":{input_channel}:DISPlay ON")

    # Use auto-scale to automatically set up oscilloscope.
    print("Autoscale.")
    myScope.WriteLine(":AUToscale:CHANnels DISPlayed")
    myScope.WriteLine(":AUToscale")

    # Set trigger mode.
    myScope.WriteLine(":TRIGger:MODE EDGE")
    myScope.WriteLine(":TRIGger:MODE?")
    qresult = myScope.ReadLine().strip()
    print(f"Trigger mode: {qresult}")

    # Set EDGE trigger parameters.
    myScope.WriteLine(f":TRIGger:EDGE:SOURce {input_channel}")
    myScope.WriteLine(":TRIGger:EDGE:SOURce?")
    qresult = myScope.ReadLine().strip()
    print(f"Trigger edge source: {qresult}")

    myScope.WriteLine(":TRIGger:EDGE:LEVel 1.5")

```

```

myScope.WriteLine(":TRIGger:EDGE:LEVel?")
qresult = myScope.ReadLine().strip()
print(f"Trigger edge level: {qresult}")

myScope.WriteLine(":TRIGger:EDGE:SLOPe POSitive")
myScope.WriteLine(":TRIGger:EDGE:SLOPe?")
qresult = myScope.ReadLine().strip()
print(f"Trigger edge slope: {qresult}")

# Save oscilloscope setup to file.
myScope.WriteLine(":SYSTem:SETup?")
setup_bytes = myScope.ReadLineBinaryBlockOfByte()
File.WriteAllBytes(setup_file_name, setup_bytes)
print(f"Setup bytes saved: {len(setup_bytes)}")

# Change settings with individual commands:

# Set vertical scale and offset.
myScope.WriteLine(f":{input_channel}:SCALE 0.05")
myScope.WriteLine(f":{input_channel}:SCALE?")
qresult = myScope.ReadLine().strip()
print(f"{input_channel} vertical scale: {qresult}")

myScope.WriteLine(f":{input_channel}:OFFSet -1.5")
myScope.WriteLine(f":{input_channel}:OFFSet?")
qresult = myScope.ReadLine().strip()
print(f"{input_channel} offset: {qresult}")

# Set horizontal scale and offset.
myScope.WriteLine(":TIMEbase:SCALE 0.0002")
myScope.WriteLine(":TIMEbase:SCALE?")
qresult = myScope.ReadLine().strip()
print(f"Timebase scale: {qresult}")

myScope.WriteLine(":TIMEbase:POSition 0.0")
myScope.WriteLine(":TIMEbase:POSition?")
qresult = myScope.ReadLine().strip()
print(f"Timebase position: {qresult}")

# Set the acquisition type.
myScope.WriteLine(":ACQuire:TYPE NORMAL")
myScope.WriteLine(":ACQuire:TYPE?")
qresult = myScope.ReadLine().strip()
print(f"Acquire type: {qresult}")

# Or, configure by loading a previously saved setup.

# Read setup string from file.
setup_bytes = File.ReadAllBytes(setup_file_name)

# Restore setup string.
myScope.Write(":SYSTem:SETup ")
write_binary = myScope.WriteBinary.Overloads[Array[Byte]]
write_binary(setup_bytes)
myScope.WriteLine("")
print(f"Setup bytes restored: {len(setup_bytes)}")

```



```

# Capture an acquisition using :DIGitize.
myScope.WriteLine(f":DIGitize {input_channel}")

# =====
# Analyze:
# =====
def analyze():

    # Make measurements.
    # -----
    myScope.WriteLine(f":MEASure:SOURce {input_channel}")
    myScope.WriteLine(":MEASure:SOURce?")
    qresult = myScope.ReadLine().strip()
    print(f"Measure source: {qresult}")

    myScope.WriteLine(":MEASure:FREQuency")
    myScope.WriteLine(":MEASure:FREQuency?")
    qresult = myScope.ReadLineDouble()
    print(f"Measured frequency on {input_channel}: {qresult:f}")

    myScope.WriteLine(":MEASure:VAMPlitude")
    myScope.WriteLine(":MEASure:VAMPlitude?")
    qresult = myScope.ReadLineDouble()
    print(f"Measured vertical amplitude on {input_channel}: {qresult:f}")

    # Download the screen image.
    # -----
    myScope.WriteLine(":HARDcopy:INKSaver OFF")

    # Get the screen data.
    myScope.WriteLine(":DISPlay:DATA? PNG, COLOr")
    image_bytes = myScope.ReadLineBinaryBlockOfByte()
    image_bytes_length = len(image_bytes)
    fStream = File.Open(screen_image_file_name, FileMode.Create)
    fStream.Write(image_bytes, 0, image_bytes_length)
    fStream.Close()
    print(f"Screen image written to {screen_image_file_name}.")

    # Download waveform data.
    # -----

    # Set the waveform source.
    myScope.WriteLine(f":WAVEform:SOURce {input_channel}")
    myScope.WriteLine(":WAVEform:SOURce?")
    qresult = myScope.ReadLine().strip()
    print(f"Waveform source: {qresult}")

    # Set the waveform points mode.
    myScope.WriteLine(":WAVEform:POINts:MODE RAW")
    myScope.WriteLine(":WAVEform:POINts:MODE?")
    qresult = myScope.ReadLine().strip()
    print(f"Waveform points mode: {qresult}")

    # Get the number of waveform points available.
    myScope.WriteLine(":WAVEform:POINts 10240")
    myScope.WriteLine(":WAVEform:POINts?")

```

```

qresult = myScope.ReadLine().strip()
print(f"Waveform points available: {qresult}")

# Choose the format of the data returned (BYTE or WORD):
myScope.WriteLine(f":WAVEform:FORMat {wfm_fmt}")
myScope.WriteLine(":WAVEform:FORMat?")
qresult = myScope.ReadLine().strip()
print(f"Waveform format: {qresult}")

# Display the waveform settings from preamble:
wav_form_dict = {
    0 : "BYTE",
    1 : "WORD",
    4 : "AScii",
}
acq_type_dict = {
    0 : "NORMal",
    1 : "PEAK",
    2 : "AVERage",
    3 : "HRESolution",
}

myScope.WriteLine(":WAVEform:PREamble?")
(
    wav_form, acq_type, wfmpmts, avgcnt, x_increment, x_origin,
    x_reference, y_increment, y_origin, y_reference
) = myScope.ReadLine().split(",")

print(f"Waveform format: {wav_form_dict[int(wav_form)]}")
print(f"Acquire type: {acq_type_dict[int(acq_type)]}")
print(f"Waveform points desired: {wfmpmts}")
print(f"Waveform average count: {avgcnt}")
print(f"Waveform X reference: {x_reference}") # Always 0.

# Get numeric values for later calculations.
myScope.WriteLine(":WAVEform:XINCrement?")
x_increment = myScope.ReadLineDouble()
print(f"Waveform X increment: {x_increment}")
myScope.WriteLine(":WAVEform:XORigin?")
x_origin = myScope.ReadLineDouble()
print(f"Waveform X origin: {x_origin}")
myScope.WriteLine(":WAVEform:YINCrement?")
y_increment = myScope.ReadLineDouble()
print(f"Waveform Y increment: {y_increment}")
myScope.WriteLine(":WAVEform:YORigin?")
y_origin = myScope.ReadLineDouble()
print(f"Waveform Y origin: {y_origin}")
myScope.WriteLine(":WAVEform:YREFerence?")
y_reference = myScope.ReadLineDouble()
print(f"Waveform Y reference: {y_reference}")

# Specify the byte order in WORD data.
if wfm_fmt == "WORD":
    myScope.WriteLine(":WAVEform:BYTeorder MSBF")
    myScope.WriteLine(":WAVEform:BYTeorder?")
    qresult = myScope.ReadLine().strip()
    print(f"Waveform byte order for WORD data: {qresult}")

```

```

# Get the waveform data values.
myScope.WriteLine(":WAVEform:DATA?")
if wfm_fmt == "BYTE":
    values = myScope.ReadLineBinaryBlockOfByte()
elif wfm_fmt == "WORD":
    values = myScope.ReadLineBinaryBlockOfUInt16()
values_length = len(values)
print(f"Number of data values: {values_length}")

# Open file for output.
writer = File.CreateText(waveform_data_file_name)

# Output waveform data in CSV format.
for i in range(0, values_length - 1):
    time_val = x_origin + i * x_increment
    voltage = (values[i] - y_reference) * y_increment + y_origin
    writer.WriteLine(f"{time_val:E}, {voltage:f}")

# Close output file.
writer.Close()
print(f"Waveform format {wfm_fmt} data written to {waveform_data_file_name}.")

# =====
# Main program:
# =====
session = GlobalResourceManager.Open(visa_addr)
session.TimeoutMilliseconds = io_timeout_ms
myScope = MessageBasedFormattedIO(session)

# Initialize the oscilloscope, capture data, and analyze.
initialize()
capture()
analyze()

# Check for oscilloscope instrument errors.
check_instrument_errors("End of program")

# Close the connection to the instrument
session.Dispose()
print("End of program.")

# Wait for a key press before exiting.
print("Press any key to exit...")
Console.ReadKey(True)

```

SICL Examples

- **"SICL Example in C"** on page 1760
- **"SICL Example in Visual Basic"** on page 1769

SICL Example in C

To compile and run this example in Microsoft Visual Studio 2008:

- 1 Open Visual Studio.
- 2 Create a new Visual C++, Win32, Win32 Console Application project.
- 3 In the Win32 Application Wizard, click **Next >**. Then, check **Empty project**, and click **Finish**.
- 4 Cut-and-paste the code that follows into a file named "example.c" in the project directory.
- 5 In Visual Studio 2008, right-click the Source Files folder, choose **Add > Add Existing Item...**, select the example.c file, and click **Add**.
- 6 Edit the program to use the SICL address of your oscilloscope.
- 7 Choose **Project > Properties....** In the Property Pages dialog, update these project settings:
 - a Under Configuration Properties, Linker, Input, add "sicl32.lib" to the Additional Dependencies field.
 - b Under Configuration Properties, C/C++, Code Generation, select Multi-threaded DLL for the Runtime Library field.
 - c Click **OK** to close the Property Pages dialog.
- 8 Add the include files and library files search paths:
 - a Choose **Tools > Options....**
 - b In the Options dialog, select **VC++ Directories** under Projects and Solutions.
 - c Show directories for **Include files**, and add the include directory (for example, Program Files\Agilent\IO Libraries Suite\include).
 - d Show directories for **Library files**, and add the library files directory (for example, Program Files\Agilent\IO Libraries Suite\lib).
 - e Click **OK** to close the Options dialog.
- 9 Build and run the program.

```
/*
 * Keysight SICL Example in C
 * -----
 * This program illustrates a few commonly-used programming
 * features of your Keysight oscilloscope.
 */

#include <stdio.h>                                /* For printf(). */
```

```

#include <string.h>          /* For strcpy(), strcat(). */
#include <time.h>            /* For clock(). */
#include <sicl.h>            /* Keysight SICL routines. */

#define SICL_ADDRESS        "usb0[2391::6054::US50210029::0]"
#define TIMEOUT            5000
#define IEEEBLOCK_SPACE    300000

/* Function prototypes */
void initialize(void);      /* Initialize to known state. */
void capture(void);        /* Capture the waveform. */
void analyze(void);        /* Analyze the captured waveform. */

void do_command(char *command); /* Send command. */
int do_command_ieeeblock(char *command); /* Command w/IEEE block. */
void do_query_string(char *query); /* Query for string. */
void do_query_number(char *query); /* Query for number. */
void do_query_numbers(char *query); /* Query for numbers. */
int do_query_ieeeblock(char *query); /* Query for IEEE block. */
void check_instrument_errors(); /* Check for inst errors. */

/* Global variables */
INST id; /* Device session ID. */
char str_result[256] = {0}; /* Result from do_query_string(). */
double num_result; /* Result from do_query_number(). */
unsigned char ieeeblock_data[IEEBLOCK_SPACE]; /* Result from
do_query_ieeeblock(). */
double dbl_results[10]; /* Result from do_query_numbers(). */

/* Main Program
* ----- */
void main(void)
{
    /* Install a default SICL error handler that logs an error message
    * and exits. On Windows 98SE or Windows Me, view messages with
    * the SICL Message Viewer. For Windows 2000 or XP, use the Event
    * Viewer.
    */
    ionerror(I_ERROR_EXIT);

    /* Open a device session using the SICL_ADDRESS */
    id = iopen(SICL_ADDRESS);

    if (id == 0)
    {
        printf ("Oscilloscope iopen failed!\n");
    }
    else
    {
        printf ("Oscilloscope session opened!\n");
    }

    /* Initialize - start from a known state. */
    initialize();

    /* Capture data. */
    capture();

```

```

/* Analyze the captured waveform. */
analyze();

/* Close the device session to the instrument. */
iclose(id);
printf ("Program execution is complete...\n");

/* For WIN16 programs, call _siclcleanup before exiting to release
 * resources allocated by SICL for this application. This call is
 * a no-op for WIN32 programs.
 */
_siclcleanup();
}

/* Initialize the oscilloscope to a known state.
 * ----- */
void initialize (void)
{
    /* Set the I/O timeout value for this session to 5 seconds. */
    itimeout(id, TIMEOUT);

    /* Clear the interface. */
    iclear(id);

    /* Get and display the device's *IDN? string. */
    do_query_string("*IDN?");
    printf("Oscilloscope *IDN? string: %s\n", str_result);

    /* Clear status and load the default setup. */
    do_command("*CLS");
    do_command("*RST");
}

/* Capture the waveform.
 * ----- */
void capture (void)
{
    int num_bytes;
    FILE *fp;

    /* Use auto-scale to automatically configure oscilloscope.
     * ----- */
    do_command(":AUToscale");

    /* Set trigger mode (EDGE, PULSe, PATtern, etc., and input source. */
    do_command(":TRIGger:MODE EDGE");
    do_query_string(":TRIGger:MODE?");
    printf("Trigger mode: %s\n", str_result);

    /* Set EDGE trigger parameters. */
    do_command(":TRIGger:EDGE:SOURce CHANnel1");
    do_query_string(":TRIGger:EDGE:SOURce?");
    printf("Trigger edge source: %s\n", str_result);

    do_command(":TRIGger:EDGE:LEVel 1.5");
    do_query_string(":TRIGger:EDGE:LEVel?");

```

```

printf("Trigger edge level: %s\n", str_result);

do_command(":TRIGger:EDGE:SLOPe POSitive");
do_query_string(":TRIGger:EDGE:SLOPe?");
printf("Trigger edge slope: %s\n", str_result);

/* Save oscilloscope configuration.
 * ----- */

/* Read system setup. */
num_bytes = do_query_ieeeblock(":SYSTem:SETup?");
printf("Read setup string query (%d bytes).\n", num_bytes);

/* Write setup string to file. */
fp = fopen ("c:\\scope\\config\\setup.stp", "wb");
num_bytes = fwrite(ieeeblock_data, sizeof(unsigned char), num_bytes,
    fp);
fclose (fp);
printf("Wrote setup string (%d bytes) to ", num_bytes);
printf("c:\\scope\\config\\setup.stp.\n");

/* Change settings with individual commands:
 * ----- */

/* Set vertical scale and offset. */
do_command(":CHANnel1:SCALe 0.05");
do_query_string(":CHANnel1:SCALe?");
printf("Channel 1 vertical scale: %s\n", str_result);

do_command(":CHANnel1:OFFSet -1.5");
do_query_string(":CHANnel1:OFFSet?");
printf("Channel 1 offset: %s\n", str_result);

/* Set horizontal scale and position. */
do_command(":TIMEbase:SCALe 0.0002");
do_query_string(":TIMEbase:SCALe?");
printf("Timebase scale: %s\n", str_result);

do_command(":TIMEbase:POSition 0.0");
do_query_string(":TIMEbase:POSition?");
printf("Timebase position: %s\n", str_result);

/* Set the acquisition type (NORMAL, PEAK, AVERage, or HRESolution). */
/
do_command(":ACQuire:TYPE NORMal");
do_query_string(":ACQuire:TYPE?");
printf("Acquire type: %s\n", str_result);

/* Or, configure by loading a previously saved setup.
 * ----- */

/* Read setup string from file. */
fp = fopen ("c:\\scope\\config\\setup.stp", "rb");
num_bytes = fread (ieeeblock_data, sizeof(unsigned char),
    IEEEBLOCK_SPACE, fp);
fclose (fp);
printf("Read setup string (%d bytes) from file ", num_bytes);

```

```

printf("c:\\scope\\config\\setup.stp.\n");

/* Restore setup string. */
num_bytes = do_command_ieeeblock(":SYSTem:SETup", num_bytes);
printf("Restored setup string (%d bytes).\n", num_bytes);

/* Capture an acquisition using :DIGitize.
 * ----- */
do_command(":DIGitize CHANnel1");
}

/* Analyze the captured waveform.
 * ----- */
void analyze (void)
{
    double wav_format;
    double acq_type;
    double wav_points;
    double avg_count;
    double x_increment;
    double x_origin;
    double x_reference;
    double y_increment;
    double y_origin;
    double y_reference;

    FILE *fp;
    int num_bytes; /* Number of bytes returned from instrument. */
    int i;

    /* Make a couple of measurements.
     * ----- */
    do_command(":MEASure:SOURce CHANnel1");
    do_query_string(":MEASure:SOURce?");
    printf("Measure source: %s\n", str_result);

    do_command(":MEASure:FREQuency");
    do_query_number(":MEASure:FREQuency?");
    printf("Frequency: %.4f kHz\n", num_result / 1000);

    do_command(":MEASure:VAMPlitude");
    do_query_number(":MEASure:VAMPlitude?");
    printf("Vertical amplitude: %.2f V\n", num_result);

    /* Download the screen image.
     * ----- */
    do_command(":HARDcopy:INKSaver OFF");

    /* Read screen image. */
    num_bytes = do_query_ieeeblock(":DISPlay:DATA? PNG, COlor");
    printf("Screen image bytes: %d\n", num_bytes);

    /* Write screen image bytes to file. */
    fp = fopen ("c:\\scope\\data\\screen.png", "wb");
    num_bytes = fwrite(ieeeblock_data, sizeof(unsigned char), num_bytes,
        fp);
    fclose (fp);

```



```

printf("Wrote screen image (%d bytes) to ", num_bytes);
printf("c:\\scope\\data\\screen.png.\n");

/* Download waveform data.
 * ----- */

/* Set the waveform points mode. */
do_command(":WAVEform:POINts:MODE RAW");
do_query_string(":WAVEform:POINts:MODE?");
printf("Waveform points mode: %s\n", str_result);

/* Get the number of waveform points available. */
do_command(":WAVEform:POINts 10240");
do_query_string(":WAVEform:POINts?");
printf("Waveform points available: %s\n", str_result);

/* Set the waveform source. */
do_command(":WAVEform:SOURce CHANnel1");
do_query_string(":WAVEform:SOURce?");
printf("Waveform source: %s\n", str_result);

/* Choose the format of the data returned (WORD, BYTE, ASCII): */
do_command(":WAVEform:FORMat BYTE");
do_query_string(":WAVEform:FORMat?");
printf("Waveform format: %s\n", str_result);

/* Display the waveform settings: */
do_query_numbers(":WAVEform:PREamble?");

wav_format = dbl_results[0];
if (wav_format == 0.0)
{
    printf("Waveform format: BYTE\n");
}
else if (wav_format == 1.0)
{
    printf("Waveform format: WORD\n");
}
else if (wav_format == 2.0)
{
    printf("Waveform format: ASCii\n");
}

acq_type = dbl_results[1];
if (acq_type == 0.0)
{
    printf("Acquire type: NORMal\n");
}
else if (acq_type == 1.0)
{
    printf("Acquire type: PEAK\n");
}
else if (acq_type == 2.0)
{
    printf("Acquire type: AVERage\n");
}
else if (acq_type == 3.0)

```

```

    {
        printf("Acquire type: HRESolution\n");
    }

    wav_points = dbl_results[2];
    printf("Waveform points: %e\n", wav_points);

    avg_count = dbl_results[3];
    printf("Waveform average count: %e\n", avg_count);

    x_increment = dbl_results[4];
    printf("Waveform X increment: %e\n", x_increment);

    x_origin = dbl_results[5];
    printf("Waveform X origin: %e\n", x_origin);

    x_reference = dbl_results[6];
    printf("Waveform X reference: %e\n", x_reference);

    y_increment = dbl_results[7];
    printf("Waveform Y increment: %e\n", y_increment);

    y_origin = dbl_results[8];
    printf("Waveform Y origin: %e\n", y_origin);

    y_reference = dbl_results[9];
    printf("Waveform Y reference: %e\n", y_reference);

    /* Read waveform data. */
    num_bytes = do_query_ieeeblock(":WAVEform:DATA?");
    printf("Number of data values: %d\n", num_bytes);

    /* Open file for output. */
    fp = fopen("c:\\scope\\data\\waveform_data.csv", "wb");

    /* Output waveform data in CSV format. */
    for (i = 0; i < num_bytes - 1; i++)
    {
        /* Write time value, voltage value. */
        fprintf(fp, "%9f, %6f\n",
            x_origin + ((float)i * x_increment),
            (((float)ieeeblock_data[i] - y_reference) * y_increment)
            + y_origin);
    }

    /* Close output file. */
    fclose(fp);
    printf("Waveform format BYTE data written to ");
    printf("c:\\scope\\data\\waveform_data.csv.\n");
}

/* Send a command to the instrument.
 * ----- */
void do_command(command)
char *command;
{
    char message[80];

```

```

        strcpy(message, command);
        strcat(message, "\n");
        iprintf(id, message);

        check_instrument_errors();
    }

/* Command with IEEE definite-length block.
 * ----- */
int do_command_ieeeblock(command, num_bytes)
char *command;
int num_bytes;
{
    char message[80];
    int data_length;

    strcpy(message, command);
    strcat(message, " #8%08d");
    iprintf(id, message, num_bytes);
    ifwrite(id, ieeeblock_data, num_bytes, 1, &data_length);

    check_instrument_errors();

    return(data_length);
}

/* Query for a string result.
 * ----- */
void do_query_string(query)
char *query;
{
    char message[80];

    strcpy(message, query);
    strcat(message, "\n");
    iprintf(id, message);

    iscanf(id, "%t\n", str_result);

    check_instrument_errors();
}

/* Query for a number result.
 * ----- */
void do_query_number(query)
char *query;
{
    char message[80];

    strcpy(message, query);
    strcat(message, "\n");
    iprintf(id, message);

    iscanf(id, "%lf", &num_result);

    check_instrument_errors();
}

```

```

    }

    /* Query for numbers result.
     * ----- */
    void do_query_numbers(query)
    char *query;
    {
        char message[80];

        strcpy(message, query);
        strcat(message, "\n");
        iprintf(id, message);

        iscanf(id, "%,10lf\n", dbl_results);

        check_instrument_errors();
    }

    /* Query for an IEEE definite-length block result.
     * ----- */
    int do_query_ieeeblock(query)
    char *query;
    {
        char message[80];
        int data_length;

        strcpy(message, query);
        strcat(message, "\n");
        iprintf(id, message);

        data_length = IEEEBLOCK_SPACE;
        iscanf(id, "%#b", &data_length, ieeeblock_data);

        if (data_length == IEEEBLOCK_SPACE )
        {
            printf("IEEE block buffer full: ");
            printf("May not have received all data.\n");
        }

        check_instrument_errors();

        return(data_length);
    }

    /* Check for instrument errors.
     * ----- */
    void check_instrument_errors()
    {
        char str_err_val[256] = {0};
        char str_out[800] = "";

        ipromptf(id, ":SYSTem:ERRor?\n", "%t", str_err_val);
        while(strncmp(str_err_val, "+0,No error", 3) != 0 )
        {
            strcat(str_out, ", ");
            strcat(str_out, str_err_val);
            ipromptf(id, ":SYSTem:ERRor?\n", "%t", str_err_val);
        }
    }

```

```

    }

    if (strcmp(str_out, "") != 0)
    {
        printf("INST Error%s\n", str_out);
        iflush(id, I_BUF_READ | I_BUF_WRITE);
    }
}

```

SICL Example in Visual Basic

To run this example in Visual Basic for Applications:

- 1 Start the application that provides Visual Basic for Applications (for example, Microsoft Excel).
- 2 Press ALT+F11 to launch the Visual Basic editor.
- 3 Add the sicil32.bas file to your project:
 - a Choose **File > Import File...**
 - b Navigate to the header file, sicil32.bas (installed with Keysight IO Libraries Suite and found in the Program Files\Agilent\IO Libraries Suite\include directory), select it, and click **Open**.
- 4 Choose **Insert > Module**.
- 5 Cut-and-paste the code that follows into the editor.
- 6 Edit the program to use the SICL address of your oscilloscope, and save the changes.
- 7 Run the program.

```

'
' Keysight SICL Example in Visual Basic
' -----
' This program illustrates a few commonly-used programming
' features of your Keysight oscilloscope.
' -----

```

Option Explicit

```

Public id As Integer    ' Session to instrument.

' Declare variables to hold numeric values returned by
' ivscanf/ifread.
Public dblQueryResult As Double
Public Const ByteArraySize = 5000000
Public retCount As Long
Public byteArray(ByteArraySize) As Byte

' Declare fixed length string variable to hold string value returned
' by ivscanf.
Public strQueryResult As String * 200

' For Sleep subroutine.

```

```

Private Declare Sub Sleep Lib "kernel32" (ByVal dwMilliseconds As Long)

'
' Main Program
' -----

Sub Main()

    On Error GoTo ErrorHandler

    ' Open a device session using the SICL_ADDRESS.
    id = iopen("usb0[2391::6054::US50210029::0]")
    Call itimeout(id, 5000)

    ' Initialize - start from a known state.
    Initialize

    ' Capture data.
    Capture

    ' Analyze the captured waveform.
    Analyze

    ' Close the vi session and the resource manager session.
    Call iclose(id)

    Exit Sub

ErrorHandler:

    MsgBox "*** Error : " + Error, vbExclamation
    End

End Sub

'
' Initialize the oscilloscope to a known state.
' -----

Private Sub Initialize()

    On Error GoTo ErrorHandler

    ' Clear the interface.
    Call iclear(id)

    ' Get and display the device's *IDN? string.
    strQueryResult = DoQueryString("*IDN?")
    MsgBox "Result is: " + RTrim(strQueryResult), vbOKOnly, "*IDN? Result"

    ' Clear status and load the default setup.
    DoCommand "*CLS"
    DoCommand "*RST"

    Exit Sub

ErrorHandler:

```

```

MsgBox "*** Error : " + Error, vbExclamation
End

End Sub

'
' Capture the waveform.
' -----

Private Sub Capture()

    On Error GoTo ErrorHandler

    ' Use auto-scale to automatically configure oscilloscope.
    ' -----
    DoCommand ":AUToscale"

    ' Set trigger mode (EDGE, PULSe, PATtern, etc., and input source.
    DoCommand ":TRIGger:MODE EDGE"
    Debug.Print "Trigger mode: " + _
        DoQueryString(":TRIGger:MODE?")

    ' Set EDGE trigger parameters.
    DoCommand ":TRIGger:EDGE:SOURce CHANnel1"
    Debug.Print "Trigger edge source: " + _
        DoQueryString(":TRIGger:EDGE:SOURce?")

    DoCommand ":TRIGger:EDGE:LEVel 1.5"
    Debug.Print "Trigger edge level: " + _
        DoQueryString(":TRIGger:EDGE:LEVel?")

    DoCommand ":TRIGger:EDGE:SLOPe POSitive"
    Debug.Print "Trigger edge slope: " + _
        DoQueryString(":TRIGger:EDGE:SLOPe?")

    ' Save oscilloscope configuration.
    ' -----
    Dim lngSetupStringSize As Long
    lngSetupStringSize = DoQueryIEEEBlock_Bytes(":SYSTem:SETup?")
    Debug.Print "Setup bytes saved: " + CStr(lngSetupStringSize)

    ' Output setup string to a file:
    Dim strPath As String
    strPath = "c:\scope\config\setup.dat"
    If Len(Dir(strPath)) Then
        Kill strPath ' Remove file if it exists.
    End If

    ' Open file for output.
    Dim hFile As Long
    hFile = FreeFile
    Open strPath For Binary Access Write Lock Write As hFile
    Dim lngI As Long
    For lngI = 0 To lngSetupStringSize - 1
        Put hFile, , byteArray(lngI) ' Write data.
    Next lngI

```

```

Close hFile    ' Close file.

' Change settings with individual commands:
' -----

' Set vertical scale and offset.
DoCommand ":CHANnel1:SCALE 0.05"
Debug.Print "Channel 1 vertical scale: " + _
    DoQueryString(":CHANnel1:SCALE?")

DoCommand ":CHANnel1:OFFSet -1.5"
Debug.Print "Channel 1 vertical offset: " + _
    DoQueryString(":CHANnel1:OFFSet?")

' Set horizontal scale and position.
DoCommand ":TIMEbase:SCALE 0.0002"
Debug.Print "Timebase scale: " + _
    DoQueryString(":TIMEbase:SCALE?")

DoCommand ":TIMEbase:POSition 0.0"
Debug.Print "Timebase position: " + _
    DoQueryString(":TIMEbase:POSition?")

' Set the acquisition type (NORMAL, PEAK, AVERage, or HRESolution).
DoCommand ":ACQuire:TYPE NORMAL"
Debug.Print "Acquire type: " + _
    DoQueryString(":ACQuire:TYPE?")

' Or, configure by loading a previously saved setup.
' -----
strPath = "c:\scope\config\setup.dat"
Open strPath For Binary Access Read As hFile    ' Open file for input.
Dim lngSetupFileSize As Long
lngSetupFileSize = LOF(hFile)    ' Length of file.
Get hFile, , byteArray    ' Read data.
Close hFile    ' Close file.
' Write setup string back to oscilloscope using ":SYSTem:SETup"
' command:
Dim lngRestored As Long
lngRestored = DoCommandIEEEBlock(":SYSTem:SETup", lngSetupFileSize)
Debug.Print "Setup bytes restored: " + CStr(lngRestored)

' Capture an acquisition using :DIGitize.
' -----
DoCommand ":DIGitize CHANnel1"

Exit Sub

ErrorHandler:

MsgBox "*** Error : " + Error, vbExclamation
End

End Sub

'
' Analyze the captured waveform.

```



```

' -----

Private Sub Analyze()

    On Error GoTo ErrorHandler

    ' Make a couple of measurements.
    ' -----
    DoCommand ":MEASure:SOURce CHANnel1"
    Debug.Print "Measure source: " + _
        DoQueryString(":MEASure:SOURce?")

    DoCommand ":MEASure:FREQuency"
    dblQueryResult = DoQueryNumber(":MEASure:FREQuency?")
    MsgBox "Frequency:" + vbCrLf + _
        FormatNumber(dblQueryResult / 1000, 4) + " kHz"

    DoCommand ":MEASure:VAMPlitude"
    dblQueryResult = DoQueryNumber(":MEASure:VAMPlitude?")
    MsgBox "Vertical amplitude:" + vbCrLf + _
        FormatNumber(dblQueryResult, 4) + " V"

    ' Download the screen image.
    ' -----
    DoCommand ":HARDcopy:INKSaver OFF"

    ' Get screen image.
    Dim lngBlockSize As Long
    lngBlockSize = DoQueryIEEEBlock_Bytes(":DISPlay:DATA? PNG, COlor")
    Debug.Print "Screen image bytes: " + CStr(lngBlockSize)

    ' Save screen image to a file:
    Dim strPath As String
    strPath = "c:\scope\data\screen.png"
    If Len(Dir(strPath)) Then
        Kill strPath ' Remove file if it exists.
    End If
    Dim hFile As Long
    hFile = FreeFile
    Open strPath For Binary Access Write Lock Write As hFile
    Dim lngI As Long
    ' Skip past header.
    For lngI = CInt(Chr(byteArray(1))) + 2 To lngBlockSize - 1
        Put hFile, , byteArray(lngI) ' Write data.
    Next lngI
    Close hFile ' Close file.
    MsgBox "Screen image written to " + strPath

    ' Download waveform data.
    ' -----

    ' Set the waveform points mode.
    DoCommand ":WAVEform:POINts:MODE RAW"
    Debug.Print "Waveform points mode: " + _
        DoQueryString(":WAVEform:POINts:MODE?")

    ' Get the number of waveform points available.

```

```

DoCommand ":WAVeform:POINts 10240"
Debug.Print "Waveform points available: " + _
    DoQueryString(":WAVeform:POINts?")

' Set the waveform source.
DoCommand ":WAVeform:SOURce CHANnel1"
Debug.Print "Waveform source: " + _
    DoQueryString(":WAVeform:SOURce?")

' Choose the format of the data returned (WORD, BYTE, ASCII):
DoCommand ":WAVeform:FORMat BYTE"
Debug.Print "Waveform format: " + _
    DoQueryString(":WAVeform:FORMat?")

' Display the waveform settings:
Dim Preamble() As Double
Dim intFormat As Integer
Dim intType As Integer
Dim lngPoints As Long
Dim lngCount As Long
Dim dblXIncrement As Double
Dim dblXOrigin As Double
Dim lngXReference As Long
Dim sngYIncrement As Single
Dim sngYOrigin As Single
Dim lngYReference As Long

Preamble() = DoQueryNumbers(":WAVeform:PREamble?")

intFormat = Preamble(0)
intType = Preamble(1)
lngPoints = Preamble(2)
lngCount = Preamble(3)
dblXIncrement = Preamble(4)
dblXOrigin = Preamble(5)
lngXReference = Preamble(6)
sngYIncrement = Preamble(7)
sngYOrigin = Preamble(8)
lngYReference = Preamble(9)

If intFormat = 0 Then
    Debug.Print "Waveform format: BYTE"
ElseIf intFormat = 1 Then
    Debug.Print "Waveform format: WORD"
ElseIf intFormat = 2 Then
    Debug.Print "Waveform format: ASCii"
End If

If intType = 0 Then
    Debug.Print "Acquisition type: NORMal"
ElseIf intType = 1 Then
    Debug.Print "Acquisition type: PEAK"
ElseIf intType = 2 Then
    Debug.Print "Acquisition type: AVERage"
ElseIf intType = 3 Then
    Debug.Print "Acquisition type: HRESolution"
End If

```

```

Debug.Print "Waveform points: " + _
    FormatNumber(lngPoints, 0)

Debug.Print "Waveform average count: " + _
    FormatNumber(lngCount, 0)

Debug.Print "Waveform X increment: " + _
    Format(dblXIncrement, "Scientific")

Debug.Print "Waveform X origin: " + _
    Format(dblXOrigin, "Scientific")

Debug.Print "Waveform X reference: " + _
    FormatNumber(lngXReference, 0)

Debug.Print "Waveform Y increment: " + _
    Format(sngYIncrement, "Scientific")

Debug.Print "Waveform Y origin: " + _
    FormatNumber(sngYOrigin, 0)

Debug.Print "Waveform Y reference: " + _
    FormatNumber(lngYReference, 0)

' Get the waveform data
Dim lngNumBytes As Long
lngNumBytes = DoQueryIEEEBlock_Bytes(":WAVEform:DATA?")
Debug.Print "Number of data values: " + _
    CStr(lngNumBytes - CInt(Chr(byteArray(1))) - 2)

' Set up output file:
strPath = "c:\scope\data\waveform_data.csv"

' Open file for output.
Open strPath For Output Access Write Lock Write As hFile

' Output waveform data in CSV format.
Dim lngDataValue As Long

' Skip past header.
For lngI = CInt(Chr(byteArray(1))) + 2 To lngNumBytes - 2
    lngDataValue = CLng(byteArray(lngI))

    ' Write time value, voltage value.
    Print #hFile, _
        FormatNumber(dblXOrigin + (lngI * dblXIncrement), 9) + _
        ", " + _
        FormatNumber(((lngDataValue - lngYReference) * _
            sngYIncrement) + sngYOrigin)

Next lngI

' Close output file.
Close hFile ' Close file.
MsgBox "Waveform format BYTE data written to " + _
    "c:\scope\data\waveform_data.csv."

```

```

Exit Sub

ErrorHandler:

    MsgBox "*** Error : " + Error, vbExclamation
End

End Sub

Private Sub DoCommand(command As String)

    On Error GoTo ErrorHandler

    Call ivprintf(id, command + vbLf)

    CheckInstrumentErrors

    Exit Sub

ErrorHandler:

    MsgBox "*** Error : " + Error, vbExclamation
End

End Sub

Private Function DoCommandIEEEBlock(command As String, _
    lngBlockSize As Long)

    On Error GoTo ErrorHandler

    ' Send command part.
    Call ivprintf(id, command + " ")

    ' Write definite-length block bytes.
    Call ifwrite(id, byteArray(), lngBlockSize, vbNull, retCount)

    ' retCount is now actual number of bytes written.
    DoCommandIEEEBlock = retCount

    CheckInstrumentErrors

    Exit Function

ErrorHandler:

    MsgBox "*** Error : " + Error, vbExclamation
End

End Function

Private Function DoQueryString(query As String) As String

    Dim actual As Long

    On Error GoTo ErrorHandler

```

```

Dim strResult As String * 200

Call ivprintf(id, query + vbLf)
Call ivscanf(id, "%200t", strResult)
DoQueryString = strResult

CheckInstrumentErrors

Exit Function

ErrorHandler:

MsgBox "*** Error : " + Error, vbExclamation
End

End Function

Private Function DoQueryNumber(query As String) As Double

On Error GoTo ErrorHandler

Dim dblResult As Double

Call ivprintf(id, query + vbLf)
Call ivscanf(id, "%lf" + vbLf, dblResult)
DoQueryNumber = dblResult

CheckInstrumentErrors

Exit Function

ErrorHandler:

MsgBox "*** Error : " + Error, vbExclamation
End

End Function

Private Function DoQueryNumbers(query As String) As Double()

On Error GoTo ErrorHandler

Dim dblResults(10) As Double

Call ivprintf(id, query + vbLf)
Call ivscanf(id, "%,10lf" + vbLf, dblResults)
DoQueryNumbers = dblResults

CheckInstrumentErrors

Exit Function

ErrorHandler:

MsgBox "*** Error : " + Error, vbExclamation
End

```

```

End Function

Private Function DoQueryIEEEBlock_Bytes(query As String) As Long

    On Error GoTo ErrorHandler

    ' Send query.
    Call ivprintf(id, query + vbCrLf)

    ' Read definite-length block bytes.
    Sleep 2000 ' Delay before reading data.
    Call ifread(id, byteArray(), ByteArraySize, vbNull, retCount)

    ' Get number of block length digits.
    Dim intLengthDigits As Integer
    intLengthDigits = CInt(Chr(byteArray(1)))

    ' Get block length from those digits.
    Dim strBlockLength As String
    strBlockLength = ""
    Dim i As Integer
    For i = 2 To intLengthDigits + 1
        strBlockLength = strBlockLength + Chr(byteArray(i))
    Next

    ' Return number of bytes in block plus header.
    DoQueryIEEEBlock_Bytes = CLng(strBlockLength) + intLengthDigits + 2

    CheckInstrumentErrors

    Exit Function

ErrorHandler:

    MsgBox "*** Error : " + Error, vbExclamation
    End

End Function

Private Sub CheckInstrumentErrors()

    On Error GoTo ErrorHandler

    Dim strErrVal As String * 200
    Dim strOut As String

    Call ivprintf(id, ":SYSTem:ERRor?" + vbCrLf) ' Query any errors data.
    Call ivscanf(id, "%200t", strErrVal) ' Read: Errnum,"Error String".
    While Val(strErrVal) <> 0 ' End if find: +0,"No Error".
        strOut = strOut + "INST Error: " + strErrVal
        Call ivprintf(id, ":SYSTem:ERRor?" + vbCrLf) ' Request error message
    .
        Call ivscanf(id, "%200t", strErrVal) ' Read error message.
    Wend

    If Not strOut = "" Then

```

```
        MsgBox strOut, vbExclamation, "INST Error Messages"
        Call iflush(id, I_BUF_READ Or I_BUF_WRITE)

    End If

Exit Sub

ErrorHandler:

    MsgBox "*** Error : " + Error, vbExclamation
    End

End Sub
```

SCPI.NET Examples

You can also program the oscilloscope using the SCPI.NET drivers that come with Keysight's free Command Expert software.

While you can write code manually using the SCPI.NET drivers, you can also use the Command Expert software to:

- Connect to instruments and control them interactively using SCPI command sets.
- Quickly prototype and test command sequences.
- Generate C#, VB.NET, or C/C++ code for command sequences.
- Find, download, and install SCPI command sets.
- Browse command trees, search for commands, and view command descriptions.

The Command Expert suite also comes with Add-ons for easy instrument control and measurement data retrieval in NI LabVIEW, Microsoft Excel, Keysight VEE, and Keysight SystemVue.

To download the Keysight Command Expert software, see:

<http://www.keysight.com/find/commandexpert>

For more on programming with the SCPI.NET drivers, see "Using SCPI.NET Drivers" in the help that comes with Keysight Command Expert.

Index

Symbols

*ESR? (Event Status Register) query, 69
*OPC (Operation Complete) query, 69
+9.9E+37, infinity representation, 1657
+9.9E+37, measurement error, 602

Numerics

0 (zero) values in waveform data, 1443
1 (one) values in waveform data, 1443
10 MHz REF BNC,
 enabling/disabling, 1336
10 MHz reference signal, 292
4000 X-Series oscilloscopes, command
 differences from, 45
82350B GPIB interface, 6

A

A429 SEARCh commands, 1226
A429 serial bus commands, 904
absolute value math function, 488
AC coupling, trigger edge, 1375
AC input coupling for specified
 channel, 331
AC RMS measured on waveform, 621, 681
accumulate activity, 241
ACQuire commands, 281
acquire data, 250, 300
acquire mode on autoscale, 245
acquire reset conditions, 226, 1315
acquire sample rate, 298
acquire sample rate, setting (Digitizer
 mode), 298
ACQuire subsystem, 61
acquired data points, 290
acquired data points, setting (Digitizer
 mode), 290
acquisition anti-alias control, 284
acquisition count, 286
acquisition mode, 282, 289, 1462
acquisition type, 282, 300
acquisition types, 1436
active edges, 241
active printer, 509
activity logic levels, 241
activity on digital channels, 241
add function, 1457
add math function, 487

add math function as g(t) source, 1548
address field size, IIC serial decode, 1010
address of network printer, 514
address value, USB search, 1290
address value, USB trigger, 1167
address, IIC trigger pattern, 1013
Addresses softkey, 50
Adjacent Channel Power Ratio (ACPR), FFT
 analysis measurement, 624
adjacent signal source, USB 2.0 signal
 quality test, 361
AER (Arm Event Register), 242, 262, 265,
 1614
Agilent Flash directory, 863, 883
alignment, I2S trigger, 992
all (snapshot) measurement, 603
ALL segments waveform save option, 896
all segments, :WAVEform:DATA?, 1453
AM demo signal, 376
AM depth, waveform generator
 modulation, 1493
AM modulation type, waveform
 generator, 1503
amplitude profile, Control Loop Response
 (BoDe) power analysis, 776
amplitude profile, frequency response
 analysis, 447
amplitude profile, Power Supply Rejection
 Ratio (PSRR) power analysis, 819
amplitude tracking, dual channel waveform
 generator, 1513, 1515
amplitude, vertical, 617, 674
amplitude, waveform generator, 445, 1517
analog channel coupling, 331
analog channel display, 332
analog channel impedance, 333
analog channel input, 1542
analog channel inversion, 334
analog channel labels, 335, 409
analog channel memory depth,
 automatic, 291
analog channel offset, 336
analog channel protection lock, 1318
analog channel range, 353
analog channel sample rate,
 automatic, 299
analog channel scale, 354
analog channel source for glitch, 1388
analog channel units, 343, 355
analog channels only oscilloscopes, 6
analog probe attenuation, 337
analog probe head type, 344
analog probe sensing, 1543

analog probe skew, 349, 1541
analysis results, save, 884
analyzing captured data, 57
angle brackets, 207
annotate channels, 335
annotation background, display, 395
annotation color, display, 396
annotation text, display, 397
annotation X1 position, 398
annotation Y1 position, 399
annotation, display, 394
anti-alias control, 284
apparent power, 693
apply network printer connection
 settings, 515
arbitrary waveform generator output, 1489
arbitrary waveform, byte order, 1476
arbitrary waveform, capturing from other
 sources, 1484
arbitrary waveform, clear, 1481
arbitrary waveform, download DAC
 values, 1482
arbitrary waveform, download floating-point
 values, 1477
arbitrary waveform, interpolation, 1483
arbitrary waveform, points, 1480
arbitrary waveform, recall, 858
arbitrary waveform, save, 871
area for hardcopy print, 508
area for saved image, 1582
area measurement, 604, 616
ARINC 429 auto setup, 906
ARINC 429 base, 907
ARINC 429 demo signal, 378
ARINC 429 signal speed, 915
ARINC 429 signal type, 913
ARINC 429 source, 914
ARINC 429 trigger data pattern, 917, 1229
ARINC 429 trigger label, 916, 920, 1227
ARINC 429 trigger SDI pattern, 918, 1230
ARINC 429 trigger SSM pattern, 919, 1231
ARINC 429 trigger type, 921, 1228
ARINC 429 user-defined baud rate, 908
ARINC 429 word and error counters,
 reset, 910
ARINC 429 word format, 912
Arm Event Register (AER), 242, 262, 265,
 1614
armed status, checking for, 1621
arming edge slope, Edge Then Edge
 trigger, 1362
arming edge source, Edge Then Edge
 trigger, 1363

arrange waveforms, 1545
 ASCII format, 1445
 ASCII format for data transfer, 1439
 ASCII string, quoted, 207
 ASCiixy waveform data format, 893
 assign channel names, 335
 attenuation factor (external trigger)
 probe, 425
 attenuation for oscilloscope probe, 337
 audio channel, I2S trigger, 1001
 AUT option for probe sense, 1543, 1547
 Auto Range capability for DVM, 418
 auto set up, trigger level, 1355
 auto set up, USB 2.0 signal quality
 test, 358
 auto setup (ARINC 429), 906
 auto setup for M1553 trigger, 1039
 auto setup for power analysis signals, 822
 auto trigger sweep mode, 1345
 automask create, 717
 automask source, 718
 automask units, 719
 automatic measurements constants, 337
 automatic probe type detection, 1543,
 1547
 autoscale, 243
 autoscale acquire mode, 245
 autoscale channels, 246
 AUToscale command, 60
 autosep for FLEXray event trigger, 982
 autosep for FLEXray decode, 972
 average math function, clear, 462
 average value measurement, 618, 675
 Average, power modulation analysis, 799
 averaged value math function, 488
 averaging acquisition type, 282, 1437
 averaging, synchronizing with, 1636
 Ax + B math function, 488

B

backlight, display, 400
 band pass filter math function, 488
 band-pass filter math function, center
 frequency, 478
 band-pass filter math function, frequency
 width, 479
 bandwidth filter limits, 329, 424
 bandwidth filter limits to 20 MHz, 330
 bar chart of current harmonics results, 783
 BARTlett window, 477
 base 10 exponential math function, 488
 base value measurement, 619, 676
 base, ARINC 429, 907
 base, I2S serial decode, 993
 base, MIL-STD-1553 serial decode, 1040
 base, SENT decode, 1088
 base, UART trigger, 1151
 base, USB decode, 1161
 basic instrument functions, 213
 baud rate, 936, 1024, 1140

baud rate, CAN FD, 938
 baud rate, user-defined, ARINC 429, 908
 begin acquisition, 250, 274, 276
 BHARris window for minimal spectral
 leakage, 477
 Bin Size, FFT, 464
 binary block data, 207, 402, 523, 1326,
 1443
 BINary waveform data format, 893
 bind levels for masks, 738
 bit order, 1141
 bit order, SPI decode, 1121
 bit rate measurement, 605
 bit selection command, bus, 305
 bit weights, 218
 bitmap display, 402, 524
 bits in Service Request Enable
 Register, 231
 bits in Standard Event Status Enable
 Register, 216
 bits in Status Byte Register, 233
 bits selection command, bus, 306
 blank, 248
 block data, 207, 221, 1326
 block response data, 64
 blocking commands, 69
 blocking synchronization, 1631
 blocking synchronization example, 1638
 blocking wait, 1630
 BMP format screen image data, 402, 524
 braces, 206
 built-in measurements, 57
 burst data demo signal, 377
 burst width measurement, 606
 burst, minimum time before next, 1370
 bus bit selection command, 305
 bus bits selection commands, 306
 bus clear command, 308
 bus commands, 304
 BUS data format, 1440
 bus display, 309
 bus label command, 310
 bus mask command, 311
 BUS<n> commands, 303
 button disable, 1312
 button, calibration protect, 319
 byte format for data transfer, 1439, 1445
 BYTEorder, 1441

C

C, SICL library example, 1760
 C, VISA library example, 1694
 C#, VISA COM example, 1669
 C#, VISA example, 1713
 C#, VISA.NET example, 1741
 CAL PROTECT button, 319
 CAL PROTECT switch, 314
 calculating preshoot of waveform, 652
 calculating the waveform overshoot, 646
 calibrate, 315, 316, 319, 323

CALibrate commands, 313
 calibrate date, 315
 calibrate introduction, 314
 calibrate label, 316
 calibrate output, 317
 calibrate start, 320
 calibrate status, 321
 calibrate switch, 319
 calibrate temperature, 322
 calibrate time, 323
 CAN acknowledge, 935
 CAN and LIN demo signal, 378
 CAN baud rate, 936
 CAN demo signal, 377
 CAN FD baud rate, 938
 CAN FD data triggers, starting byte
 position, 947
 CAN frame counters, reset, 928
 CAN SEARCh commands, 1232
 CAN serial bus commands, 923
 CAN serial search, data, 1235
 CAN serial search, data length, 1236
 CAN serial search, ID, 1237
 CAN serial search, ID mode, 1238
 CAN serial search, mode, 1233
 CAN signal definition, 937
 CAN source, 939
 CAN symbolic data display, 932
 CAN symbolic data, recall, 859
 CAN trigger, 940, 946
 CAN trigger data pattern, 944
 CAN trigger ID pattern, 948
 CAN trigger pattern id mode, 949
 CAN trigger, ID filter for, 943
 CAN triggering, 900
 capture data, 250
 capturing data, 56
 cardiac waveform generator output, 1488
 center frequency set, 465, 487
 center of screen, 1470
 center screen, vertical value at, 486, 493
 center time base reference, 1337
 CFD demo signal, 379
 channel, 280, 335, 1538, 1540
 channel coupling, 331
 channel display, 332
 channel input impedance, 333
 channel inversion, 334
 channel label, 335, 1539
 channel labels, 408, 409
 channel numbers, 1545
 channel overload, 352
 Channel Power, FFT analysis
 measurement, 625
 channel protection, 352
 channel reset conditions, 226, 1315
 channel selected to produce trigger, 1388,
 1419
 channel signal type, 350
 channel skew for oscilloscope probe, 349,
 1541
 channel status, 277, 1545

- channel threshold, 1540
- channel vernier, 356
- channel, stop displaying, 248
- CHANnel<n> commands, 325, 328
- channels to autoscale, 246
- channels, how autoscale affects, 243
- characters to display, 1310
- chart logic bus state math function, 489
- chart logic bus state, clock edge, 458
- chart logic bus state, clock source, 457
- chart logic bus timing math function, 489
- chart logic bus, units, 461
- chart logic bus, value for data = 0, 460
- chart logic bus, value for data increment, 459
- chart serial signal math function, 490
- Chart Serial Signal math function, serial bus source, 495
- Chart Serial Signal math function, time mode, 496
- Chart Serial Signal math function, time offset, 499
- Chart Serial Signal math function, time range, 498
- classes of input signals, 477
- classifications, command, 1650
- clear, 401
- clear bus command, 308
- clear cumulative edge variables, 1538
- clear markers, 607, 1559
- clear measurement, 607, 1559
- clear message from display, 411
- clear message queue, 215
- Clear method, 59
- clear persistence data from display, 413
- clear reference waveforms, 1523
- clear screen, 1546
- clear status, 215
- clear waveform area, 393
- clipped high waveform data value, 1443
- clipped low waveform data value, 1443
- clock, 1011, 1122, 1126
- Clock Extension Peripheral Interface (CXPI), 953
- clock period, SENT signal, 1086
- clock slope, I2S, 994
- CLOCK source, I2S, 996
- clock source, setup and hold trigger, 1406
- clock timeout, SPI, 1123
- clock with infrequent glitch demo signal, 376
- CLS (Clear Status), 215
- CME (Command Error) status bit, 216, 218
- CMOS threshold voltage for digital channels, 389, 1540
- CMOS trigger threshold voltage, 1587
- code, :ACQuire:COMplete, 285
- code, :ACQuire:SEGmented, 295
- code, :ACQuire:TYPE, 301
- code, :AUToscale, 244
- code, :CHANnel<n>:LABel, 335
- code, :CHANnel<n>:PROBe, 337
- code, :CHANnel<n>:RANGe, 353
- code, :DIGitize, 251
- code, :DISPlay:DATA, 402
- code, :DISPlay:LABel, 408
- code, :DISPlay:ORDer, 1545
- code, :MEASure:PERiod, 662
- code, :MEASure:RESults, 654
- code, :MEASure:TEDGe, 670
- code, :MTESt, 713
- code, :POD<n>:THREshold, 749
- code, :RUN/:STOP, 274
- code, :SYSTem:SETup, 1326
- code, :TIMebase:DElay, 1586
- code, :TIMebase:MODE, 1333
- code, :TIMebase:RANGe, 1335
- code, :TIMebase:REFerence, 1337
- code, :TRIGger:MODE, 1358
- code, :TRIGger:SLOPe, 1378
- code, :TRIGger:SOURce, 1379
- code, :VIEW and :BLANK, 280
- code, :WAVeform, 1458
- code, :WAVeform:DATA, 1443
- code, :WAVeform:POINts, 1447
- code, :WAVeform:PREamble, 1451
- code, :WAVeform:SEGmented, 295
- code, :WGEN:ARbitrary:DATA, 1477
- code, *RST, 228
- code, SICL library example in C, 1760
- code, SICL library example in Visual Basic, 1769
- code, VISA COM library example in C#, 1669
- code, VISA COM library example in Python, 1686
- code, VISA COM library example in Visual Basic, 1660
- code, VISA COM library example in Visual Basic .NET, 1678
- code, VISA library example in C, 1694
- code, VISA library example in C#, 1713
- code, VISA library example in Python, 1734
- code, VISA library example in Visual Basic, 1703
- code, VISA library example in Visual Basic .NET, 1724
- code, VISA.NET library example in C#, 1741
- code, VISA.NET library example in Python, 1753
- code, VISA.NET library example in Visual Basic .NET, 1747
- colon, root commands prefixed by, 240
- color palette for hardcopy, 520
- color palette for image, 878
- Comma Separated Values (CSV) waveform data format, 893
- command classifications, 1650
- command differences from 4000 X-Series oscilloscopes, 45
- command errors detected in Standard Event Status, 218
- Command Expert, 1741, 1780
- command header, 1651
- command headers, common, 1653
- command headers, compound, 1653
- command headers, simple, 1653
- command strings, valid, 1651
- commands quick reference, 71
- commands sent over interface, 213
- commands, more about, 1649
- commands, obsolete and discontinued, 1531
- common (*) commands, 3, 209, 213
- common command headers, 1653
- common logarithm math function, 488
- completion criteria for an acquisition, 285, 286
- COMpliance commands, 357
- compound command headers, 1653
- compound header, 1655
- computer control examples, 1659
- concurrent commands, 69
- conditions for external trigger, 423
- conditions, reset, 226, 1315
- conduction calculation method for switching loss, 844
- Config softkey, 50
- configurations, oscilloscope, 221, 225, 229, 1326
- Configure softkey, 50
- connect oscilloscope, 49
- connect sampled data points, 1544
- Connection Expert, 51
- connection type, hi-speed, USB 2.0 signal quality test, 366
- constants for making automatic measurements, 337
- constants for scaling display factors, 337
- constants for setting trigger levels, 337
- Control Loop Response (Bode) power analysis settings, 759
- control loop response (Bode) power analysis, apply, 760
- control loop response power analysis, single frequency, 767
- control loop response power analysis, sweep start frequency, 768
- control loop response power analysis, sweep stop frequency, 769
- controller initialization, 56
- conversion type of power supply, 796
- copy display, 272
- copy signal, dual channel waveform generator, 1514
- copyright, 2
- core commands, 1650
- count, 1442
- count values, 286
- count, averaged value math function, 456
- count, Edge Then Edge trigger, 1365
- count, Nth edge of burst, 1369
- counter, 369, 608
- counter commands, 369
- counter mode, 372

- counter source, 374
- counter, digits of resolution, 373
- counter, enable/disable, 371
- coupling, 1375
- COUPLing demo signal, 379
- coupling for channels, 331
- CRC format, SENT, 1087
- CRC value, USB search, 1291
- CRC value, USB trigger, 1168
- create automask, 717
- crest factor, 695
- CSV (Comma Separated Values) waveform data format, 893
- CT bits (CXPI), triggering on, 967
- cumulative edge activity, 1538
- current harmonics analysis fail count, 784
- current harmonics analysis results, save, 882
- current harmonics analysis run count, 789
- current harmonics analysis, apply, 781
- current harmonics results data, 782
- current harmonics results display, 783
- current logic levels on digital channels, 241
- current oscilloscope configuration, 221, 225, 229, 1326
- current probe, 343, 355, 427
- CURRent segment waveform save option, 896
- current source, 839
- cursor display setting, X1, 564
- cursor display setting, X2, 567
- cursor display setting, Y1, 573
- cursor display setting, Y2, 575
- cursor mode, 563
- cursor position, 562, 565, 568, 570, 574, 577
- cursor readout, 1560, 1564, 1565
- cursor reset conditions, 226, 1315
- cursor source, 566, 569
- cursor time, 1560, 1564, 1565
- cursor units, X, 571, 572
- cursor units, Y, 578, 579
- cursor values, saving, 885
- cursors track measurements, 659
- cursors, how autoscale affects, 243
- cursors, X1, X2, Y1, Y2, 561
- custom time base reference, 1337
- custom time base reference location, 1338
- CXPI demo signal, 378
- CXPI frame data value, 963
- CXPI parity bit display, 956
- CXPI serial bus commands, 953
- CXPI signal baud rate, 955
- CXPI signal source, 957
- CXPI signal tolerance, 958
- CXPI trigger mode, 959
- CXPI triggering, 900
- cycle count base, FLEXray frame trigger, 985
- cycle count repetition, FLEXray frame trigger, 986

- cycle measured, 622, 642
- cycle time, 649
- cycles analyzed, number of, 823, 824

D

- D- source, USB 2.0 signal quality test, 363
- D- USB signal source, 1162
- D+ source, USB 2.0 signal quality test, 364
- D+ USB signal source, 1163
- data, 1012, 1014, 1443
- data (waveform) maximum length, 895
- data 2, 1015
- data acquisition types, 1436
- data conversion, 1438
- data format for transfer, 1438
- data length, USB search, 1293
- data length, USB trigger, 1170
- data output order, 1441
- data pattern length, 946, 1033
- data pattern, ARINC 429 trigger, 917, 1229
- data pattern, CAN trigger, 944
- data PID, USB search, 1298
- data PID, USB trigger, 1176
- data point index, 1467
- data points, 290
- data record, measurement, 1448
- data record, raw acquisition, 1448
- data required to fill time buckets, 285
- DATA source, I2S, 997
- data source, setup and hold trigger, 1407
- data structures, status reporting, 1599
- data value, USB search, 1292
- data value, USB trigger, 1169
- data, saving and recalling, 393
- date, calibration, 315
- date, system, 1308
- DC coupling for edge trigger, 1375
- DC input coupling for specified channel, 331
- DC offset correction for integrate input, 483
- DC RMS measured on waveform, 621, 681
- DC waveform generator output, 1487
- DCMotor demo signal, 379
- DDE (Device Dependent Error) status bit, 216, 218
- decision chart, status reporting, 1620
- default conditions, 226, 1315
- define channel labels, 335
- define delay measurement parameters, 615
- define glitch trigger, 1386
- define logic thresholds, 1540
- define measurement, 611
- define measurement source, 661
- define trigger, 1387, 1395, 1396, 1398
- defined as, 206
- definite-length block query response, 64
- definite-length block response data, 207
- degauss probe, 339
- delay bits, SPI MISO stream, 1124
- delay measured to calculate phase, 650
- delay measurement, 611, 613
- delay measurement parameters, define, 615
- delay measurements, 670
- delay time, Edge Then Edge trigger, 1364
- DElay trigger commands, 1361
- delay, how autoscale affects, 243
- delayed time base, 1333
- delayed window horizontal scale, 1343
- delete mask, 727
- delta time, 1560
- delta voltage measurement, 1568
- delta X cursor, 561
- delta Y cursor, 561
- demo, 375
- DEMO commands, 375
- demo signal, 377
- demo signal function, 376
- demo signal phase angle, 381
- demo signals output control, 382
- deskew for power measurements, 777
- destination, remote command logging, 1320
- detecting probe types, 1543, 1547
- Device Clear to abort a sequential (blocking) command, 70
- device-defined error queue clear, 215
- DIFF source for function, 1551
- differences from 4000 X-Series oscilloscope commands, 45
- differential D+ and D- USB signal source, 1164
- differential probe heads, 344
- differential probe source, USB 2.0 signal quality test, 362
- differential signal type, 350
- differentiate math function, 287, 487, 1457
- digital channel commands, 384, 385, 386, 387, 389
- digital channel data, 1440
- digital channel labels, 409
- digital channel order, 1545
- digital channel source for glitch trigger, 1388
- digital channels, 6
- digital channels, activity and logic levels on, 241
- digital channels, groups of, 745, 747, 749
- digital pod, stop displaying, 248
- digital reset conditions, 227, 1316
- DIGital<d> commands, 383
- digitize channels, 250
- DIGitize command, 56, 61, 1436
- Digitizer mode, 288
- digits, 207
- disable anti-alias mode, 287
- disable front panel, 1312
- disable function, 1552
- disabling calibration, 319

disabling channel display, 332
 disabling status register bits, 216, 230
 discontinued and obsolete
 commands, 1531
 display annotation, 394
 display annotation background, 395
 display annotation color, 396
 display annotation text, 397
 display channel labels, 408
 display clear, 401
 DISPLAY commands, 391
 display commands introduction, 393
 display connect, 1544
 display date, 1308
 display factors scaling, 337
 display for channels, 332
 display frequency span, 474
 display measurements, 601, 659
 display order, 1545
 display persistence, 412
 display reference, 1334
 display reference waveforms, 1524
 display reset conditions, 227, 1316
 display serial number, 275
 display transparent, 415
 display vectors, 416
 display wave position, 1545
 display, lister, 543
 display, measurement statistics on/off, 664
 display, oscilloscope, 385, 412, 463, 747, 1310
 display, serial decode bus, 902
 displaying a baseline, 1360
 displaying unsynchronized signal, 1360
 divide math function, 487
 DLC data length code (CXPI), triggering
 on, 968
 DLC value in CAN FD trigger, 945
 DNS IP, 50
 domain, 50
 domain, network printer, 516
 driver, printer, 1557
 DSO models, 6
 duplicate mnemonics, 1655
 duration, 1395, 1396, 1398
 duration for glitch trigger, 1382, 1383, 1387
 duration of power analysis, 825, 826, 827, 828, 829, 830
 duration qualifier, trigger, 1395, 1396
 duration triggering, 1346
 duty cycle measurement, 57, 601, 622, 642
 Duty Cycle, power modulation
 analysis, 799
 DVM commands, 417
 DVM displayed value, 419
 DVM enable/disable, 420
 DVM input source, 422
 DVM mode, 421

E

EBUrst trigger commands, 1368
 ECL channel threshold, 1540
 ECL threshold voltage for digital
 channels, 389
 ECL trigger threshold voltage, 1587
 edge activity, 1538
 edge counter, Edge Then Edge
 trigger, 1365
 edge counter, Nth edge of burst, 1369
 edge coupling, 1375
 edge fall time, 623
 edge preshoot measured, 652
 edge rise time, 657
 EDGE SEARCh commands, 1201
 edge search slope, 1202
 edge search source, 1203
 edge slew rate, 660
 edge slope, 1378
 edge source, 1379
 edge string for OR'ed edge trigger, 1390
 EDGE trigger commands, 1373
 edge triggering, 1346
 edges (activity) on digital channels, 241
 edges in measurement, 611
 efficiency, 696
 efficiency power analysis, apply, 778
 elapsed time in mask test, 724
 ellipsis, 207
 enable channel labels, 408
 enabling calibration, 319
 enabling channel display, 332
 enabling status register bits, 216, 230
 end of string (EOS) terminator, 1652
 end of text (EOT) terminator, 1652
 end or identify (EOI), 1652
 endpoint value, USB search, 1294
 endpoint value, USB trigger, 1171
 energy loss, 697
 envelope math function, 489
 EOI (end or identify), 1652
 EOS (end of string) terminator, 1652
 EOT (end of text) terminator, 1652
 equivalent time acquisition mode, 289
 erase data, 401
 erase measurements, 1559
 erase screen, 1546
 error counter (ARINC 429), 909
 error counter (ARINC 429), reset, 910
 error frame count (CAN), 926
 error frame count (UART), 1142
 error messages, 1311, 1589
 error number, 1311
 error queue, 1311, 1610
 error, measurement, 601
 ESB (Event Status Bit), 231, 233
 ESE (Standard Event Status Enable
 Register), 216, 1609
 ESR (Standard Event Status Register), 218, 1608
 ET value, USB search, 1295

ET value, USB trigger, 1172
 ETE demo signal, 377
 event status conditions occurred, 233
 Event Status Enable Register (ESE), 216, 1609
 Event Status Register (ESR), 218, 279, 1608
 example code, :ACQuire:COMplete, 285
 example code, :ACQuire:SEGmented, 295
 example code, :ACQuire:TYPE, 301
 example code, :AUToscale, 244
 example code, :CHANnel<n>:LABel, 335
 example code, :CHANnel<n>:PROBe, 337
 example code, :CHANnel<n>:RANGE, 353
 example code, :DIGitize, 251
 example code, :DISPlay:DATA, 402
 example code, :DISPlay:LABel, 408
 example code, :DISPlay:ORDER, 1545
 example code, :MEASure:PERiod, 662
 example code, :MEASure:RESults, 654
 example code, :MEASure:TEDGe, 670
 example code, :MTEST, 713
 example code, :POD<n>:THReshold, 749
 example code, :RUN/:STOP, 274
 example code, :SYSTem:SETup, 1326
 example code, :TIMebase:DElay, 1586
 example code, :TIMebase:MODE, 1333
 example code, :TIMebase:RANGE, 1335
 example code, :TIMebase:REference, 1337
 example code, :TRIGger:MODE, 1358
 example code, :TRIGger:SLOPe, 1378
 example code, :TRIGger:SOURce, 1379
 example code, :VIEW and :BLANK, 280
 example code, :WAVEform, 1458
 example code, :WAVEform:DATA, 1443
 example code, :WAVEform:POINTS, 1447
 example code,
 :WAVEform:PREamble, 1451
 example code,
 :WAVEform:SEGmented, 295
 example code,
 :WGEN:ARBitrary:DATA, 1477
 example code, *RST, 228
 example programs, 6, 1659
 examples on the website, 1659
 excursion delta for FFT peak search, 1212
 EXE (Execution Error) status bit, 216, 218
 execution error detected in Standard Event
 Status, 218
 exponential fall waveform generator
 output, 1488
 exponential math function, 488
 exponential notation, 206
 exponential rise waveform generator
 output, 1488
 extended video triggering license, 1420
 external glitch trigger source, 1388
 external range, 426
 External Scaling, enable/disable, 340
 External Scaling, gain, 341
 External Scaling, offset, 342
 external trigger, 423, 425, 1379

EXternal trigger commands, 423
 EXternal trigger level, 1376
 external trigger probe attenuation factor, 425
 external trigger probe sensing, 1547
 EXternal trigger source, 1379
 external trigger units, 427

F

fail count, current harmonics analysis, 784
 fail/pass status (overall) for current harmonics analysis, 791
 failed waveforms in mask test, 722
 failure, self test, 235
 fall time measurement, 601, 623
 Fall Time, power modulation analysis, 799
 falling edge count measurement, 643
 falling pulse count measurement, 644
 FAST data, SENT, 1461
 Fast Fourier Transform (FFT)
 functions, 465, 474, 477, 487, 1551
 FF values in waveform data, 1443
 FFT (Fast Fourier Transform) functions, 465, 474, 477, 487, 1551
 FFT (Fast Fourier Transform)
 operation, 1457
 FFT bin size/RBW/sample rate
 readout, 473
 FFT detector decimation type, 467
 FFT detector points, 466
 FFT math function, 287
 FFT vertical units, 476
 FFTPhase (Fast Fourier Transform)
 functions, 487
 fifty ohm impedance, disable setting, 1318
 filename for hardcopy, 1554
 filename for recall, 860, 1485
 filename for save, 873
 filter CXPI trigger by ID, 961
 filter for frequency reject, 1377
 filter for high frequency reject, 1350
 filter for noise reject, 1359
 filter used to limit bandwidth, 330, 424
 filters to Fast Fourier Transforms, 477
 filters, math, 488
 fine horizontal adjustment (vernier), 1340
 fine vertical adjustment (vernier), 356
 finish pending device operations, 222
 first point displayed, 1467
 FLATtop window for amplitude measurements, 477
 FLEXray autoselect for event trigger, 982
 FLEXray autoselect, 972
 FlexRay demo signal, 378
 FlexRay frame counters, reset, 976
 FLEXray SEARCh commands, 1242
 FlexRay serial search, cycle, 1243
 FlexRay serial search, data, 1244
 FlexRay serial search, data length, 1245
 FlexRay serial search, frame, 1246
 FlexRay serial search, mode, 1247
 FLEXray source, 979
 FLEXray trigger, 980
 FLEXray trigger commands, 970
 FM burst demo signal, 377
 FM modulation type, waveform generator, 1503
 force trigger, 1349
 format, 1445, 1450
 format (word), ARINC 429, 912
 format for block data, 221
 format for generic video, 1416, 1420
 format for hardcopy, 1553
 format for image, 876
 format for waveform data, 893
 FormattedIO488 object, 59
 formfeed for hardcopy, 506, 511
 formulas for data conversion, 1438
 frame, 1127
 frame counters (CAN), error, 926
 frame counters (CAN), overload, 927
 frame counters (CAN), reset, 928
 frame counters (CAN), total, 930
 frame counters (FlexRay), null, 975, 977
 frame counters (FlexRay), reset, 976
 frame counters (FlexRay), total, 978
 frame counters (UART), error, 1142
 frame counters (UART), reset, 1143
 frame counters (UART), Rx frames, 1144
 frame counters (UART), Tx frames, 1145
 frame ID, CXPI trigger, 966
 frame ID, FLEXray BSS event trigger, 983
 frame ID, FLEXray frame trigger, 987
 frame type, FLEXray frame trigger, 988
 frame value, USB search, 1296
 frame value, USB trigger, 1173
 framing, 1125
 FRANalysis commands, 429
 frequency deviation, waveform generator FM modulation, 1495
 frequency measurement, 57, 601, 628
 frequency measurements with X cursors, 571
 frequency mode, Control Loop Response (Bode) power analysis, 766
 frequency mode, Power Supply Rejection Ratio (PSRR) power analysis, 811
 frequency resolution, 477
 frequency response analysis, data, 432
 frequency response analysis, enable, 433
 frequency response analysis, run, 439
 frequency response analysis, single frequency, 435
 frequency response analysis, sweep points, 442
 frequency response analysis, sweep start frequency, 436
 frequency response analysis, sweep stop frequency, 437
 frequency span of display, 474
 Frequency, power modulation analysis, 799
 front panel mode, 1360

front panel Single key, 276
 front panel Stop key, 278
 front-panel lock, 1312
 FSK modulation type, waveform generator, 1503
 FSK rate, waveform generator modulation, 1498
 full-scale horizontal time, 1335, 1342
 full-scale vertical axis defined, 492
 function, 463, 465, 474, 477, 486, 487, 492, 493, 494, 1551, 1552
 FUNCTION commands, 449
 function memory, 277
 function turned on or off, 1552
 function, demo signal, 376
 function, first source input, 501
 function, second source input, 503
 function, waveform generator, 1486
 functions, 1457

G

g(t) source, first input channel, 1549
 g(t) source, math operation, 1548
 g(t) source, second input channel, 1550
 gain data, including in control loop response results, 773
 gain data, including in FRA results, 443
 gain data, including in PSRR results, 816
 gain for Ax + B math operation, 484
 gated measurement window, 683
 gateway IP, 50
 gating, FFT math function, 470
 gaussian pulse waveform generator
 output, 1489
 general SBUS<n> commands, 901
 general SEARCh commands, 1196
 general trigger commands, 1347
 GENeric, 1416, 1420
 generic video format, 1416, 1420
 Generic video trigger, edge number, 1421
 Generic video trigger, greater than sync pulse width time, 1424
 Generic video trigger, horizontal sync control, 1422
 Generic video trigger, horizontal sync pulse time, 1423
 glitch demo signal, 376
 glitch duration, 1387
 glitch qualifier, 1386
 GLITCh SEARCh commands, 1204
 glitch search, greater than value, 1205
 glitch search, less than value, 1206
 glitch search, polarity, 1207
 glitch search, qualifier, 1208
 glitch search, range, 1209
 glitch search, source, 1210
 glitch source, 1388
 GLITCh trigger commands, 1380
 glitch trigger duration, 1382
 glitch trigger polarity, 1385

glitch trigger source, 1382
 graticule area for hardcopy print, 508
 graticule axis labels, 404
 graticule colors, invert for hardcopy, 512, 1556
 graticule colors, invert for image, 877
 graticule intensity, 405
 graticule type, 406
 grayscale palette for hardcopy, 520
 grayscale palette for image, 878
 grayscale on hardcopy, 1555
 greater than qualifier, 1386
 greater than time, 1382, 1387, 1395, 1398
 greater than value for glitch search, 1205
 grid axis labels, 404
 grid intensity, 405
 grid type, 406
 groups of digital channels, 745, 747, 749, 1540

H

handshake PID, USB search, 1299
 handshake PID, USB trigger, 1177
 HANNing window for frequency resolution, 477
 hardcopy, 272, 506
 HARDcopy commands, 505
 hardcopy factors, 510, 875
 hardcopy filename, 1554
 hardcopy format, 1553
 hardcopy formfeed, 511
 hardcopy grayscale, 1555
 hardcopy invert graticule colors, 512, 1556
 hardcopy layout, 513
 hardcopy palette, 520
 hardcopy print, area, 508
 hardcopy printer driver, 1557
 Hardware Event Condition Register (:HWERegister:CONDition), 254
 Hardware Event Condition Register (:OPERRegister:CONDition), 1617
 Hardware Event Enable Register (HWEenable), 252
 Hardware Event Event Register (:HWERegister[:EVENT]), 255, 1616
 HARMonics demo signal, 379
 HCOPY commands, 505
 head type, probe, 344
 header, 1651
 high pass filter math function, 488
 high resolution acquisition type, 1438
 high trigger level, 1356
 high-frequency reject filter, 1350, 1377
 high-level voltage, waveform generator, 1518
 high-pass filter cutoff frequency, 480
 high-resolution acquisition type, 282
 histogram axis, 528

histogram commands, 525
 histogram display, 529
 histogram measurement, 530
 histogram mode, 531
 histogram reset, 532
 histogram size, 533
 histogram statistics, saving, 886
 histogram type, 534
 histogram window, bottom limit, 535
 histogram window, left limit, 536
 histogram window, right limit, 537
 histogram window, source, 538
 histogram window, top limit, 539
 histogram, bin width, 629
 histogram, delta value between max and min, 640
 histogram, maximum value, 634
 histogram, mean value, 635
 histogram, median value, 636
 histogram, minimum value, 637
 histogram, mode value, 638
 histogram, number of hits in max bin, 639
 histogram, number of total hits, 630
 histogram, pct of hits within +/- 1 std dev, 631
 histogram, pct of hits within +/- 2 std dev, 632
 histogram, pct of hits within +/- 3 std dev, 633
 histogram, standard deviation, 641
 hold time, setup and hold trigger, 1408
 hold until operation complete, 222
 holdoff time, 1351
 holes in waveform data, 1443
 hop frequency, waveform generator FSK modulation, 1497
 horizontal adjustment, fine (vernier), 1340
 horizontal histogram, 528
 horizontal position, 1341
 horizontal scale, 1339, 1343
 horizontal scaling, 1450
 horizontal time, 1335, 1342, 1560
 Host ID of oscilloscope, 1309
 Host name softkey, 50
 hostname, 50
 hub address value, USB search, 1297
 hub address value, USB trigger, 1174
 hubs, USB 2.0 signal quality, 359
 HWEenable (Hardware Event Enable Register), 252
 HWERegister:CONDition (Hardware Event Condition Register), 254, 1617
 HWERegister[:EVENT] (Hardware Event Event Register), 255, 1616

I

I1080L50HZ, 1416, 1420
 I1080L60HZ, 1416, 1420
 I2C demo signal, 377
 I2S alignment, 992

I2S audio channel, 1001
 I2S clock slope, 994
 I2S CLOCK source, 996
 I2S DATA source, 997
 I2S demo signal, 378
 I2S pattern data, 1002
 I2S pattern format, 1004
 I2S range, 1005
 I2S receiver width, 995
 I2S SEARCh commands, 1248
 I2S serial bus commands, 989
 I2S serial decode base, 993
 I2S serial search, audio channel, 1249
 I2S serial search, data, 1251
 I2S serial search, format, 1252
 I2S serial search, mode, 1250
 I2S serial search, range, 1253
 I2S transmit word size, 1006
 I2S trigger operator, 999
 I2S triggering, 900
 I2S word select (WS) low, 1007
 I2S word select (WS) source, 998
 ID filter for CAN trigger, 943
 id mode, 949
 ID pattern, CAN trigger, 948
 identification number, 220
 identification of options, 223
 identifier, LIN, 1030
 idle, 1370
 idle state, SENT bus, 1092
 idle until operation complete, 222
 IDN (Identification Number), 220
 IEC 61000-3-2 standard for current harmonics analysis, 790
 IEEE 488.2 standard, 213
 IIC address, 1013
 IIC clock, 1011
 IIC data, 1012, 1014
 IIC data 2, 1015
 IIC SEARCh commands, 1254
 IIC serial decode address field size, 1010
 IIC serial search, address, 1257
 IIC serial search, data, 1258
 IIC serial search, data2, 1259
 IIC serial search, mode, 1255
 IIC serial search, qualifier, 1260
 IIC trigger commands, 1008
 IIC trigger qualifier, 1016
 IIC trigger type, 1017
 IIC triggering, 900
 image format, 876
 image invert graticule colors, 877
 image memory, 277
 image palette, 878
 image, save, 874
 image, save with inksaver, 877
 impedance, 333
 infinity representation, 1657
 initial load current, transient response analysis, 851
 initialization, 56, 59
 initialize, 226, 1315

initialize label list, 409
 initiate acquisition, 250
 inksaver, save image with, 877
 input coupling for channels, 331
 input for integrate, DC offset correction, 483
 input impedance for channels, 333, 1542
 input inversion for specified channel, 334
 input power, 699
 input source, Control Loop Response (Bode) power analysis, 771
 input source, Power Supply Rejection Ratio (PSRR) power analysis, 814
 inrush current, 703
 inrush current analysis, 793, 794, 795
 inrush current expected, 831
 insert label, 335
 installed options identified, 223
 instruction header, 1651
 instrument number, 220
 instrument options identified, 223
 instrument requests service, 233
 instrument serial number, 275
 instrument settings, 506
 instrument status, 66
 instrument type, 220
 integrate DC offset correction, 483
 integrate math function, 487, 1457
 Integrate math function, Initial Condition, 482
 INTegrate source for function, 1551
 intensity, waveform, 407
 internal low-pass filter, 329, 330, 424
 introduction to :ACQuire commands, 282
 introduction to :BUS<n> commands, 304
 introduction to :CALibrate commands, 314
 introduction to :CHANnel<n> commands, 328
 introduction to :COUNter commands, 369
 introduction to :DEMO commands, 375
 introduction to :DIGital<d> commands, 384
 introduction to :DISPlay commands, 393
 introduction to :EXTernal commands, 423
 introduction to :FRANalysis commands, 430
 introduction to :FUNCTION commands, 455
 introduction to :HARDcopy commands, 506
 introduction to :LISTer commands, 541
 introduction to :LTESt commands, 546
 introduction to :MARKer commands, 561
 introduction to :MEASure commands, 601
 introduction to :POD<n> commands, 745
 introduction to :RECall commands, 856
 introduction to :SAVE commands, 870
 introduction to :SBUS commands, 899
 introduction to :SYSTem commands, 1307
 introduction to :TIMEbase commands, 1332
 introduction to :TRIGger commands, 1345

introduction to :WAVEform commands, 1436
 introduction to :WGEN<w> commands, 1475
 introduction to :WMEMory<r> commands, 1521
 introduction to common (*) commands, 213
 introduction to root (:) commands, 240
 invert graticule colors for hardcopy, 512, 1556
 invert graticule colors for image, 877
 inverted masks, bind levels, 738
 inverting input for channels, 334
 IO library, referencing, 58
 IO operation complete, waiting for, 1626
 IP address, 50

K

key disable, 1312
 key press detected in Standard Event Status Register, 218
 KEYSight demo signal, 379
 Keysight Interactive IO application, 53
 Keysight IO Control icon, 51
 Keysight IO Libraries Suite, 6, 47, 58, 60
 Keysight IO Libraries Suite, installing, 48
 knob disable, 1312
 known state, 226, 1315

L

label, 1539
 label command, bus, 310
 label list, 335, 409
 label reference waveforms, 1525
 label, ARINC 429 trigger, 916, 920, 1227
 label, digital channel, 386
 labels, 335, 408, 409
 labels to store calibration information, 316
 labels, specifying, 393
 LAN instrument, 52
 LAN interface, 49, 51
 LAN Settings softkey, 50
 landscape layout for hardcopy, 513
 language for program examples, 55
 layout for hardcopy, 513
 leakage into peak spectrum, 477
 learn string, 221, 1326
 least significant byte first, 1441
 left time base reference, 1337
 legal values for channel offset, 336
 legal values for frequency span, 474
 legal values for offset, 486, 493
 length for waveform data, 894
 length of data, CXPI trigger, 964
 less than qualifier, 1386
 less than time, 1383, 1387, 1396, 1398
 less than value for glitch search, 1206

level for trigger voltage, 1376, 1384
 LF coupling, 1375
 license information, 223
 limit bandwidth, 329
 limits for line number, 1416
 LIN acknowledge, 1023
 LIN baud rate, 1024
 LIN demo signal, 377
 LIN identifier, 1030
 LIN pattern data, 1031
 LIN pattern format, 1034
 LIN SEARCh commands, 1261
 LIN serial decode bus parity bits, 1022
 LIN serial search, data, 1264
 LIN serial search, data format, 1266
 LIN serial search, data length, 1265
 LIN serial search, frame ID, 1262
 LIN serial search, mode, 1263
 LIN source, 1025
 LIN standard, 1026
 LIN symbolic data display, 1021
 LIN symbolic data, recall, 861
 LIN sync break, 1027
 LIN trigger, 1028, 1033
 LIN trigger commands, 1019
 LIN trigger definition, 1583
 LIN triggering, 900
 line frequency setting for current harmonics analysis, 785
 line glitch trigger source, 1388
 line number for TV trigger, 1416
 line terminator, 206
 LINE trigger level, 1376
 LINE trigger source, 1379
 Linear units for FFT (Magnitude), 476
 list of channel labels, 409
 LISTer commands, 541
 lister display, 543
 lister time reference, 544
 ln math function, 488
 load utilization (CAN), 931
 local lockout, 1312
 location, custom time base reference location, 1338
 lock, 1312
 lock mask to signal, 729
 lock, analog channel protection, 1318
 lockout message, 1312
 log file name, remote command logging, 1319, 1322
 log math function, 488
 Logarithmic units for FFT (Magnitude), 476
 logic level activity, 1538
 long form, 1652
 low frequency sine with glitch demo signal, 377
 low pass filter math function, 488
 low trigger level, 1357
 lower threshold, 649
 lower threshold voltage for measurement, 1558
 lowercase characters in commands, 1651

low-frequency reject filter, 1377
 low-level voltage, waveform generator, 1519
 low-pass filter cutoff frequency, 481
 low-pass filter used to limit bandwidth, 329, 330, 424
 LRN (Learn Device Setup), 221
 lsbfirst, 1441
 LTEST commands, 545

M

M1553 SEARCh commands, 1270
 M1553 trigger commands, 1038
 M1553 trigger type, 1044
 magnify math function, 489
 magnitude of occurrence, 672
 main sweep range, 1341
 main time base, 1586
 main time base mode, 1333
 making measurements, 601
 MAN option for probe sense, 1543, 1547
 Manchester baud rate, 1048
 Manchester bit order, 1049
 Manchester bit values trigger, data value, 1060
 Manchester bit values trigger, data width, 1061
 Manchester bus display format, 1050
 Manchester decode base, 1047
 Manchester idle time in bits, 1053
 Manchester number of bits in header field, 1052
 Manchester number of bits in trailer field, 1062
 Manchester number of bits per word in data field, 1063
 Manchester number of words in data field, 1051
 Manchester polarity, 1054
 MANChester serial bus commands, 1045
 Manchester signal source, 1055
 Manchester signal tolerance, 1058
 Manchester starting edge, 1057
 Manchester sync size, 1056
 Manchester trigger mode, 1059
 Manchester/NRZ demo signal, 378
 manual cursor mode, 563
 manufacturer string, 1313, 1314
 MARKer commands, 559
 marker mode, 574
 marker position, 576
 marker readout, 1564, 1565
 marker set for voltage measurement, 1569, 1570
 marker sets start time, 1561
 marker time, 1560
 markers for delta voltage measurement, 1568
 markers track measurements, 659
 markers, command overview, 561

markers, mode, 563
 markers, time at start, 1565
 markers, time at stop, 1564
 markers, X delta, 562, 570
 markers, X1 position, 565
 markers, X1Y1 source, 566
 markers, X2 position, 568
 markers, X2Y2 source, 569
 markers, Y delta, 577
 markers, Y1 position, 574
 markers, Y2 position, 576
 mask, 216, 230
 mask command, bus, 311
 mask statistics, reset, 723
 mask statistics, saving, 887
 mask test commands, 711
 Mask Test Event Enable Register (MTEenable), 256
 mask test event event register, 258
 Mask Test Event Event Register (:MTERegister[:EVENT]), 258, 1618
 mask test run mode, 730
 mask test termination conditions, 730
 mask test, all channels, 716
 mask test, enable/disable, 728
 mask, delete, 727
 mask, get as binary block data, 726
 mask, load from binary block data, 726
 mask, lock to signal, 729
 mask, recall, 862
 mask, save, 879, 880
 masks, bind levels, 738
 master summary status bit, 233
 math filters, 488
 math function, stop displaying, 248
 math operators, 487
 math transforms, 487
 math visualizations, 489
 MAV (Message Available), 215, 231, 233
 max hold math function, 489
 maximum duration, 1383, 1395, 1396
 maximum math function, 489
 maximum number of peaks for FFT peak search, 1213
 maximum position, 1334
 maximum random trigger holdoff time, 1352
 maximum range for zoomed window, 1342
 maximum scale for zoomed window, 1343
 maximum vertical value measurement, 677
 maximum vertical value, time of, 684, 1562
 maximum waveform data length, 895
 MEASure commands, 581
 measure mask test failures, 731
 measure overshoot, 646
 measure period, 649
 measure phase between channels, 650
 MEASure power commands, 687
 measure preshoot, 652
 measure start voltage, 1569
 measure stop voltage, 1570

measure value at a specified time, 686, 1571
 measure value at top of waveform, 682
 measurement error, 601
 measurement limit test, copy all limits from results, 548
 measurement limit test, copy limit from results, 547
 measurement limit test, enable, 550
 measurement limit test, enable/disable, 556
 measurement limit test, fail condition, 551
 measurement limit test, lower limit, 552, 557
 measurement limit test, margin when copying from results, 549
 measurement limit test, measurement source, 553
 measurement limit test, results, 554
 measurement limit test, stop on failure, 555
 measurement record, 1448
 measurement results, saving, 888
 measurement setup, 601, 661
 measurement source, 661
 measurement statistics results, 654
 measurement statistics, display on/off, 664
 measurement trend math function, 489
 measurement window, 683
 measurements, AC RMS, 621, 681
 measurements, area, 604, 616
 measurements, average value, 618, 675
 measurements, base value, 619, 676
 measurements, built-in, 57
 measurements, burst width, 606
 measurements, clear, 607, 1559
 measurements, command overview, 601
 measurements, counter, 608
 measurements, DC RMS, 621, 681
 measurements, definition setup, 611
 measurements, fall time, 623
 measurements, falling edge count, 643
 measurements, falling pulse count, 644
 measurements, frequency, 628
 measurements, how autoscale affects, 243
 measurements, lower threshold level, 1558
 measurements, maximum vertical value, 677
 measurements, maximum vertical value, time of, 684, 1562
 measurements, minimum vertical value, 678
 measurements, minimum vertical value, time of, 685, 1563
 measurements, negative duty cycle, 642
 measurements, overshoot, 646
 measurements, period, 649
 measurements, phase, 650
 measurements, positive duty cycle, 622
 measurements, preshoot, 652
 measurements, pulse width, negative, 645
 measurements, pulse width, positive, 653

measurements, ratio of AC RMS values, [680](#)
 measurements, rise time, [657](#)
 measurements, rising edge count, [648](#)
 measurements, rising pulse count, [651](#)
 measurements, show, [659](#)
 measurements, slew rate, [660](#)
 measurements, snapshot all, [603](#)
 measurements, source channel, [661](#)
 measurements, standard deviation, [658](#)
 measurements, start marker time, [1564](#)
 measurements, stop marker time, [1565](#)
 measurements, thresholds, [1561](#)
 measurements, time between start and stop markers, [1560](#)
 measurements, time between trigger and edge, [669](#)
 measurements, time between trigger and vertical value, [672](#)
 measurements, time between trigger and voltage level, [1566](#)
 measurements, upper threshold value, [1567](#)
 measurements, vertical amplitude, [617](#), [674](#)
 measurements, vertical peak-to-peak, [620](#), [679](#)
 measurements, voltage difference, [1568](#)
 memory depth, setting (Digitizer mode), [290](#)
 memory setup, [229](#), [1326](#)
 menu, display, [410](#)
 menu, system, [1585](#)
 message available bit, [233](#)
 message available bit clear, [215](#)
 message decode/triggering format, SENT, [1090](#)
 message displayed, [233](#)
 message error, [1589](#)
 message queue, [1607](#)
 message, CAN symbolic search, [1239](#)
 message, CAN symbolic trigger, [950](#)
 message, clear from display, [411](#)
 message, LIN symbolic search, [1267](#)
 message, LIN symbolic trigger, [1035](#)
 messages ready, [233](#)
 midpoint of thresholds, [649](#)
 MIL-STD-1553 demo signal, [378](#)
 MIL-STD-1553 serial decode base, [1040](#)
 MIL-STD-1553 serial search, data, [1272](#)
 MIL-STD-1553 serial search, mode, [1271](#)
 MIL-STD-1553 serial search, Remote Terminal Address, [1273](#)
 MIL-STD-1553, dual demo signal, [379](#)
 min hold math function, [489](#)
 minimum duration, [1382](#), [1395](#), [1396](#), [1398](#)
 minimum math function, [489](#)
 minimum random trigger holdoff time, [1353](#)
 minimum vertical value measurement, [678](#)
 minimum vertical value, time of, [685](#), [1563](#)

MISO data pattern width, [1131](#)
 MISO data pattern, SPI trigger, [1130](#)
 MISO data source, SPI trigger, [1128](#)
 MISO data, SPI, [1461](#)
 mixed-signal demo signals, [377](#)
 mixed-signal oscilloscopes, [6](#)
 mnemonics, duplicate, [1655](#)
 mode, [563](#), [1333](#), [1417](#)
 mode, serial decode, [903](#)
 model number, [220](#)
 models, oscilloscope, [3](#)
 modes for triggering, [1358](#)
 Modify softkey, [50](#)
 modulating signal frequency, waveform generator, [1494](#), [1496](#)
 modulation (waveform generator), enabling/disabling, [1502](#)
 modulation analysis, [797](#)
 modulation analysis source (voltage or current), [798](#)
 modulation analysis, type of, [799](#)
 modulation type, waveform generator, [1503](#)
 MOSI data pattern width, [1133](#)
 MOSI data pattern, SPI trigger, [1132](#)
 MOSI data source, SPI trigger, [1129](#), [1584](#)
 most significant byte first, [1441](#)
 move cursors, [1564](#), [1565](#)
 msbfirst, [1441](#)
 MSG (Message), [231](#), [233](#)
 MSO models, [6](#)
 MSS (Master Summary Status), [233](#)
 MTEenable (Mask Test Event Enable Register), [256](#)
 MTERegister[:EVENT] (Mask Test Event Event Register), [258](#), [1618](#)
 MTESt commands, [711](#)
 multi-channel waveform data, save, [881](#)
 multiple commands, [1655](#)
 multiple queries, [65](#)
 multiplier value for SENT Fast Channel Signal, [1099](#)
 multiply math function, [487](#), [1457](#)
 multiply math function as g(t) source, [1548](#)

N

N275xA InfiniiMode probe mode, [347](#)
 N275xA InfiniiMode probe quick-action button, [338](#)
 N2820A high sensitivity current probe, [616](#), [617](#), [618](#), [619](#), [620](#), [621](#)
 N8900A InfiniiMode Offline oscilloscope analysis software, [881](#)
 name channels, [335](#)
 name list, [409](#)
 natural logarithm math function, [488](#)
 negative glitch trigger polarity, [1385](#)
 negative pulse width, [645](#)
 negative pulse width measurement, [57](#)
 negative pulse width, power modulation analysis, [799](#)
 negative slope, [1122](#), [1378](#)
 negative slope, Nth edge in burst, [1371](#)
 negative TV trigger polarity, [1418](#)
 network domain password, [517](#)
 network domain user name, [519](#)
 network printer address, [514](#)
 network printer domain, [516](#)
 network printer slot, [518](#)
 network printer, apply connection settings, [515](#)
 new line (NL) terminator, [206](#), [1652](#)
 new load current, transient response analysis, [852](#)
 nibble order for SENT Fast Channel Signal, [1103](#)
 NL (new line) terminator, [206](#), [1652](#)
 NM bits (CXPI), triggering on, [969](#)
 NMOnotonic, dual demo signal, [379](#)
 noise floor, [845](#), [848](#)
 noise reject filter, [1359](#)
 noise waveform generator output, [1487](#)
 noise, adding to waveform generator output, [1501](#)
 noisy sine waveform demo signal, [376](#)
 non-core commands, [1650](#)
 non-interlaced GENeric mode, [1420](#)
 non-volatile memory, label list, [310](#), [386](#), [409](#)
 normal acquisition type, [282](#), [1437](#)
 normal trigger sweep mode, [1345](#)
 notices, [2](#)
 NR1 number format, [206](#)
 NR3 number format, [206](#)
 NRZ baud rate, [1067](#)
 NRZ bit order, [1068](#)
 NRZ bit values trigger, data value, [1079](#)
 NRZ bit values trigger, data width, [1080](#)
 NRZ bus display format, [1069](#)
 NRZ decode base, [1066](#)
 NRZ frame size, [1071](#)
 NRZ idle state level, [1074](#)
 NRZ idle time in bits, [1073](#)
 NRZ number of bits in header field, [1072](#)
 NRZ number of bits in trailer field, [1081](#)
 NRZ number of bits per word in data field, [1082](#)
 NRZ number of start bits, [1077](#)
 NRZ number of words in data field, [1070](#)
 NRZ polarity, [1075](#)
 NRZ serial bus commands, [1064](#)
 NRZ signal source, [1076](#)
 NRZ trigger mode, [1078](#)
 Nth edge burst trigger source, [1372](#)
 Nth edge burst triggering, [1346](#)
 Nth edge in a burst idle, [1370](#)
 Nth edge in burst slope, [1371](#)
 Nth edge of burst counter, [1369](#)
 Nth edge of Edge Then Edge trigger, [1365](#)
 NTSC, [1416](#), [1420](#)
 null frame count (FlexRay), [975](#)

null offset, 845
 NULL string, 1310
 number format, 206
 number of nibbles, SENT message, 1093
 number of points, 290, 1446, 1448
 number of time buckets, 1446, 1448
 numeric variables, 64
 numeric variables, reading query results into multiple, 66
 nwidth, 645

O

obsolete and discontinued commands, 1531
 obsolete commands, 1650
 Occupied Bandwidth, FFT analysis measurement, 626
 occurrence reported by magnitude, 1566
 offset, 455
 offset for Ax + B math operation, 485
 offset value for channel voltage, 336
 offset value for selected function, 486, 493
 offset value for SENT Fast Channel Signal, 1101
 offset, waveform generator, 1520
 one values in waveform data, 1443
 OPC (Operation Complete) command, 222
 OPC (Operation Complete) status bit, 216, 218
 OPEE (Operation Status Enable Register), 260
 Open method, 59
 operating configuration, 221, 1326
 operating state, 229
 operation complete, 222
 operation status condition register, 262
 Operation Status Condition Register (:OPERRegister:CONDition), 262, 1613
 operation status conditions occurred, 233
 Operation Status Enable Register (OPEE), 260
 operation status event register, 265
 Operation Status Event Register (:OPERRegister[:EVENT]), 265, 1611
 operations for function, 487
 operators, math, 487
 OPERRegister:CONDition (Operation Status Condition Register), 262, 1613
 OPERRegister[:EVENT] (Operation Status Event Register), 265, 1611
 OPT (Option Identification), 223
 optional syntax terms, 206
 options, 223
 OR trigger commands, 1389
 order of digital channels on display, 1545
 order of output, 1441
 oscilloscope connection, opening, 59
 oscilloscope connection, verifying, 51
 oscilloscope external trigger, 423
 oscilloscope models, 3

oscilloscope, connecting, 49
 oscilloscope, initialization, 56
 oscilloscope, operation, 6
 oscilloscope, program structure, 56
 oscilloscope, setting up, 49
 oscilloscope, setup, 60
 output control, demo signals, 382
 output control, waveform generator, 1505
 output load impedance, waveform generator, 444, 1506
 output messages ready, 233
 output polarity, waveform generator, 1508
 output power, 702
 output queue, 222, 1606
 output queue clear, 215
 output ripple, 708
 output ripple analysis, 821
 output sequence, 1441
 output source, Control Loop Response (Bode) power analysis, 772
 output source, Power Supply Rejection Ratio (PSRR) power analysis, 815
 overall pass/fail status for current harmonics analysis, 791
 overlapped commands, 69
 overload, 352
 Overload Event Enable Register (OVL), 268
 Overload Event Register (:OVLRegister), 1615
 Overload Event Register (OVLRegister), 270
 overload frame count (CAN), 927
 overload protection, 268, 270
 overshoot of waveform, 646
 overshoot percent for transient response analysis, 832
 overvoltage, 352
 OVL (Overload Event Enable Register), 268
 OVLRegister (Overload Event Register), 270
 OVLRegister bit, 262, 265
 OVLRegister (Overload Event Register), 1615

P

P1080L24HZ, 1416, 1420
 P1080L25HZ, 1416, 1420
 P1080L50HZ, 1416, 1420
 P1080L60HZ, 1416, 1420
 P480L60HZ, 1416, 1420
 P720L60HZ, 1416, 1420
 PAL, 1416, 1420
 palette for hardcopy, 520
 palette for image, 878
 PAL-M, 1416, 1420
 parametric measurements, 601
 parity, 1147
 parity bits, LIN serial decode bus, 1022
 parser, 240, 1655
 pass, self test, 235
 pass/fail status (overall) for current harmonics analysis, 791
 password, network domain, 517
 path information, recall, 863
 path information, save, 883
 pattern, 1013, 1014, 1015
 pattern data, I2S, 1002
 pattern data, LIN, 1031
 pattern duration, 1382, 1383, 1395, 1396
 pattern for pattern trigger, 1392
 pattern format, I2S, 1004
 pattern format, LIN, 1034
 pattern length, 946, 1033
 PATtern trigger commands, 1391
 pattern trigger format, 1394
 pattern trigger qualifier, 1397
 pattern triggering, 1346
 pattern width, 1131, 1133
 pause pulse, SENT messages, 1094
 peak current, 703
 peak data, 1437
 peak detect, 300
 peak detect acquisition type, 283, 1437
 PEAK SEARCH commands, 1211
 peak-peak math function, 489
 peak-to-peak vertical value measurement, 620, 679
 pending operations, 222
 percent of waveform overshoot, 646
 percent thresholds, 611
 period measured to calculate phase, 650
 period measurement, 57, 601, 649
 Period, power modulation analysis, 799
 period, waveform generator, 1510
 persistence data, clear from display, 413
 persistence, waveform, 393, 412
 phase angle, demo signals, 381
 phase data, including in control loop response results, 773
 phase data, including in FRA results, 443
 phase measured between channels, 650
 phase measurements, 670
 phase measurements with X cursors, 571
 phase relationship of frequency-tracked waveform, 1516
 phase shifted demo signals, 376
 PID check value, USB trigger, 1175
 PLL Locked bit, 254
 PNG format screen image data, 402, 524
 pod, 745, 747, 748, 749, 1457, 1540
 POD commands, 745
 POD data format, 1440
 pod, stop displaying, 248
 points, 290, 1446, 1448
 points in waveform data, 1436
 points per decade, Control Loop Response (Bode) power analysis, 770
 points per decade, frequency response analysis, 438
 points per decade, Power Supply Rejection Ratio (PSRR) power analysis, 813
 points, setting (Digitizer mode), 290
 polarity, 1148, 1418

polarity for glitch search, 1207
 polarity for glitch trigger, 1385
 polarity, runt search, 1217
 polarity, runt trigger, 1400
 polling synchronization example, 1638
 polling synchronization with timeout, 1632
 polling wait, 1630
 PON (Power On) status bit, 216, 218
 port value, USB search, 1302
 port value, USB trigger, 1180
 portrait layout for hardcopy, 513
 position, 387, 568, 1334, 1341
 position cursors, 1564, 1565
 position in zoomed view, 1341
 position waveforms, 1545
 positive glitch trigger polarity, 1385
 positive pulse width, 653
 positive pulse width measurement, 57
 positive pulse width, power modulation analysis, 799
 positive slope, 1122, 1378
 positive slope, Nth edge in burst, 1371
 positive TV trigger polarity, 1418
 positive width, 653
 power analysis, enabling, 780
 POWer commands, 751
 power factor, 698
 power factor for IEC 61000-3-2 Standard Class C, 786
 power loss, 704
 power phase angle, 692
 power quality analysis, 820
 power supply rejection ratio (PSRR), 807, 809, 810, 812
 Power Supply Rejection Ratio (PSRR) power analysis settings, 806
 preamble data, 1450
 preamble metadata, 1436
 predefined logic threshold, 1540
 predefined threshold voltages, 1587
 present working directory, recall operations, 863
 present working directory, save operations, 883
 preset conditions, 1315
 preshoot measured on waveform, 652
 previously stored configuration, 225
 print command, 272
 print job, start, 522
 print mask test failures, 732
 print query, 1580
 printer driver for hardcopy, 1557
 printer, active, 509
 printing, 506
 printing in grayscale, 1555
 probe, 1376
 probe attenuation affects channel voltage range, 353
 probe attenuation factor (external trigger), 425
 probe attenuation factor for selected channel, 337

probe head type, 344
 probe ID, 345
 probe sense for oscilloscope, 1543, 1547
 probe skew value, 349, 1541
 process sigma, mask test run, 735
 program data, 1652
 program data syntax rules, 1654
 program initialization, 56
 program message, 59, 213
 program message syntax, 1651
 program message terminator, 1652
 program structure, 56
 programming examples, 6, 1659
 protecting against calibration, 319
 protection, 268, 270, 352
 protection lock, 1318
 PTYPE byte (CXPI), trigger when present, 962
 PTYPE frames, CXPI, 959
 pulse waveform generator output, 1487
 pulse width, 645, 653
 pulse width duration trigger, 1382, 1383, 1387
 pulse width measurement, 57, 601
 pulse width trigger, 1359
 pulse width trigger level, 1384
 pulse width triggering, 1346
 pulse width, waveform generator, 1490
 pwidth, 653
 Python, VISA COM example, 1686
 Python, VISA example, 1734
 Python, VISA.NET example, 1753
 PyVISA package, 1734

Q

qualifier, 1387
 qualifier for glitch search, 1208
 qualifier, runt search, 1218
 qualifier, runt trigger, 1401
 qualifier, transition search, 1222
 qualifier, transition trigger, 1411
 qualifier, trigger duration, 1395, 1396
 qualifier, trigger pattern, 1397
 queries, multiple, 65
 query error detected in Standard Event Status, 218
 query responses, block data, 64
 query responses, reading, 63
 query results, reading into numeric variables, 64
 query results, reading into string variables, 64
 query return values, 1657
 query setup, 506, 561, 601, 1326
 query subsystem, 304, 384
 querying setup, 328
 querying the subsystem, 1346
 queues, clearing, 1619
 quick reference, commands, 71
 quoted ASCII string, 207
 QYE (Query Error) status bit, 216, 218

R

ramp symmetry, waveform generator, 1491
 ramp symmetry, waveform generator modulating signal, 1500
 ramp waveform generator output, 1487
 random trigger holdoff, 1354
 range, 455, 1342
 range for channels, 353
 range for duration trigger, 1398
 range for external trigger, 426
 range for full-scale vertical axis, 492
 range for glitch search, 1209
 range for glitch trigger, 1387
 range for time base, 1335
 range of offset values, 336
 range qualifier, 1386
 range, I2S, 1005
 ranges, value, 207
 ratio measurements with X cursors, 571
 ratio measurements with Y cursors, 578
 ratio of AC RMS values measured between channels, 680
 Ratio, power modulation analysis, 799
 raw acquisition record, 1448
 RCL (Recall), 225
 Rds (dynamic ON resistance) waveform, 844
 Rds(on) power measurement, 705
 Rds(on) value for conduction calculation, 846
 reactive power, 706
 read configuration, 221
 ReadIEEEBlock method, 59, 63, 65
 ReadList method, 59, 63
 ReadNumber method, 59, 63
 readout, 1560
 ReadString method, 59, 63
 real power, 707
 Real Power source in Class D harmonics analysis, 787
 real-time acquisition mode, 289
 recall, 225, 856, 1326
 recall arbitrary waveform, 858
 recall CAN symbolic data, 859
 RECall commands, 855
 recall filename, 860, 1485
 recall LIN symbolic data, 861
 recall mask, 862
 recall path information, 863
 recall reference waveform, 865
 recall setup, 864
 recalling and saving data, 393
 receiver width, I2S, 995
 RECTangular window for transient signals, 477
 reference, 455
 reference clock, 1336
 reference for time base, 1586

reference point, FFT Phase, 471
 reference signal (10 MHz), 292
 reference signal mode, 1336
 reference waveform save source, 897
 reference waveform, recall, 865
 reference waveform, save, 898
 reference waveforms, clear, 1523
 reference waveforms, display, 1524
 reference waveforms, label, 1525
 reference waveforms, save to, 1526
 reference waveforms, skew, 1527
 reference waveforms, Y offset, 1528
 reference waveforms, Y range, 1529
 reference waveforms, Y scale, 1530
 reference, lister, 544
 reference, time base, 1337
 registers, 218, 225, 229, 242, 256, 258, 260, 262, 265, 268, 270
 registers, clearing, 1619
 reject filter, 1377
 reject high frequency, 1350
 reject noise, 1359
 relative standard deviation, 668
 remote command logging, enable/disable, 1319, 1323
 remote control examples, 1659
 Remote Terminal Address (RTA), M1553 trigger, 1043
 remote user interface enabled/disabled status, 263, 266, 1611, 1613
 remove cursor information, 563
 remove labels, 408
 remove message from display, 1310
 reorder channels, 243
 repetitive acquisitions, 274
 report errors, 1311
 report transition, 672
 reporting status, 1597
 reporting the setup, 1346
 request service, 233
 Request-for-OPC flag clear, 215
 reset, 226
 reset conditions, 226
 reset defaults, waveform generator, 1511
 reset mask statistics, 723
 reset measurements, 401
 Resolution Bandwidth, FFT, 472
 resolution of printed copy, 1555
 resource session object, 59
 ResourceManager object, 59
 restore configurations, 221, 225, 229, 1326
 restore labels, 408
 restore setup, 225
 return values, query, 1657
 returning acquisition type, 300
 returning number of data points, 290
 RF burst demo signal, 377
 right time base reference, 1337
 ringing pulse demo signal, 376
 ripple (output) analysis, 821
 ripple, output, 708

rise time measurement, 601
 rise time of positive edge, 657
 Rise Time, power modulation analysis, 799
 rising edge count measurement, 648
 rising pulse count measurement, 651
 RMS - AC, power modulation analysis, 799
 RMS value measurement, 621, 681
 roll time base mode, 1333
 root (:) commands, 237, 240
 root level commands, 3
 RQL (Request Control) status bit, 216, 218
 RQS (Request Service), 233
 RS-232/UART triggering, 900
 R-sense resistor value, N2825A user-defined R-sense head, 348
 RST (Reset), 226
 rules, tree traversal, 1655
 rules, truncation, 1652
 run, 234, 274
 Run bit, 262, 265
 run count, current harmonics analysis, 789
 run mode, mask test, 730
 run state, 273
 run, USB 2.0 signal quality test, 360
 running configuration, 229, 1326
 RUNT SEARCh commands, 1216
 runt search polarity, 1217
 runt search qualifier, 1218
 runt search source, 1219
 runt search, pulse time, 1220
 RUNT trigger commands, 1399
 runt trigger polarity, 1400
 runt trigger qualifier, 1401
 runt trigger source, 1402
 runt trigger time, 1403
 Rx frame count (UART), 1144
 Rx source, 1149

S

S and E or U value, USB search, 1304
 S and E or U value, USB trigger, 1182
 sample rate, 3, 298
 Sample Rate, FFT, 475
 sample rate, setting (Digitizer mode), 298
 sampled data, 1544
 sampled data points, 1443
 SAV (Save), 229
 save, 229, 870
 save arbitrary waveform, 871
 SAVE commands, 867
 save current harmonics analysis results, 882
 save filename, 873
 save image, 874
 save image with inksaver, 877
 save mask, 879, 880
 save mask test failures, 733
 save path information, 883
 save reference waveform, 898
 save setup, 891
 save to reference waveform location, 1526
 save waveform data, 892
 saved image, area, 1582
 saving and recalling data, 393
 SBUS A429 commands, 904
 SBUS CAN commands, 923
 SBUS commands, 899
 SBUS CXPI commands, 953
 SBUS I2S commands, 989
 SBUS MANChester commands, 1045
 SBUS NRZ commands, 1064
 SBUS SENT commands, 1083
 SBUS USBPd commands, 1183
 SBUS<n> commands, general, 901
 SC value, USB search, 1303
 SC value, USB trigger, 1181
 scale, 494, 1339, 1343
 scale factors output on hardcopy, 510, 875
 scale for channels, 354
 scale units for channels, 343, 355
 scale units for external trigger, 427
 scaling display factors, 337
 SCPI commands, 67
 SCPI.NET examples, 1780
 scratch measurements, 1559
 screen area for hardcopy print, 508
 screen area for saved image, 1582
 screen display of logged remote commands, enable/disable, 1321
 screen image data, 402, 523
 SDI pattern, ARINC 429 trigger, 918, 1230
 SEARCh commands, 1195
 SEARCh commands, A429, 1226
 SEARCh commands, CAN, 1232
 SEARCh commands, EDGE, 1201
 SEARCh commands, FLEXray, 1242
 SEARCh commands, general, 1196
 SEARCh commands, GLITCh, 1204
 SEARCh commands, I2S, 1248
 SEARCh commands, IIC, 1254
 SEARCh commands, LIN, 1261
 SEARCh commands, M1553, 1270
 SEARCh commands, PEAK, 1211
 SEARCh commands, RUNT, 1216
 SEARCh commands, SENT, 1274
 SEARCh commands, SPI, 1279
 SEARCh commands, TRANSition, 1221
 SEARCh commands, UART, 1283
 SEARCh commands, USB, 1287
 search event (found) times, saving, 889
 search for found event, 1198
 search mode, 1199
 search state, 1200
 search, edge slope, 1202
 search, edge source, 1203
 search, USB, 1289
 SECAM, 1416, 1420
 seconds per division, 1339
 segmented memory acquisition times, 890
 segmented waveform save option, 896
 segments, analyze, 293
 segments, count of waveform, 1454

- segments, setting number of memory, 294
- segments, setting the index, 295
- segments, time tag, 1455
- select measurement channel, 661
- self-test, 235
- sensing a channel probe, 1543
- sensing an external trigger probe, 1547
- sensitivity of oscilloscope input, 337
- SENT demo signal, 379
- SENT enhanced serial message ID and data lengths, 1117
- SENT fast channel data search value, 1275
- SENT fast channel data trigger setting, 1112
- SENT FAST data, 1461
- SENT input source, 1107
- SENT percent tolerance variation trigger, 1118
- SENT SEARCh commands, 1274
- SENT search mode, 1276
- SENT serial bus commands, 1083
- SENT signal length setting, 1097
- SENT signals display setting, 1096
- SENT slow channel data search value, 1277
- SENT slow channel ID and data trigger data setting, 1113
- SENT slow channel ID search value, 1278
- SENT slow channel ID trigger setting, 1115
- SENT SLOW data, 1461
- SENT trigger mode, 1110
- SENT triggering, 900
- sequential commands, 69
- serial clock, 1011, 1126
- serial data, 1012
- serial decode bus, 899
- serial decode bus display, 902
- serial decode mode, 903
- serial frame, 1127
- serial number, 275
- service request, 233
- Service Request Enable Register (SRE), 231, 1604
- set center frequency, 465
- set cursors, 1564, 1565
- set date, 1308
- set time, 1328
- set up oscilloscope, 49
- setting digital display, 385
- setting digital label, 310, 386
- setting digital position, 387
- setting digital threshold, 389
- setting display, 463
- setting external trigger level, 423
- setting impedance for channels, 333
- setting inversion for channels, 334
- setting pod display, 747
- setting pod size, 748
- setting pod threshold, 749
- settings, 225, 229
- settings conflict errors, 263, 266, 1611, 1613
- settings, instrument, 506
- setup, 283, 304, 328, 384, 393, 506, 1326
- setup and hold trigger clock source, 1406
- setup and hold trigger data source, 1407
- setup and hold trigger hold time, 1408
- setup and hold trigger setup time, 1409
- setup and hold trigger slope, 1405
- setup configuration, 225, 229, 1326
- setup defaults, 226, 1315
- setup memory, 225
- setup reported, 1346
- setup time, setup and hold trigger, 1409
- setup, recall, 864
- setup, save, 891
- shape of modulation signal, waveform generator, 1499
- SHOLd trigger commands, 1404
- short form, 5, 1652
- show channel labels, 408
- show measurements, 601, 659
- SICL example in C, 1760
- SICL example in Visual Basic, 1769
- SICL examples, 1760
- sidebar, display, 414
- sigma, mask test run, 735
- signal speed, ARINC 429, 915
- signal type, 350
- signal type, ARINC 429, 913
- signal value, CAN symbolic search, 1241
- signal value, CAN symbolic trigger, 952
- signal value, LIN symbolic search, 1269
- signal value, LIN symbolic trigger, 1037
- signal, CAN symbolic search, 1240
- signal, CAN symbolic trigger, 951
- signal, LIN symbolic search, 1268
- signal, LIN symbolic trigger, 1036
- signed data, 1439
- simple command headers, 1653
- sine cardinal waveform generator output, 1488
- sine waveform demo signal, 376
- sine waveform generator output, 1486
- single acquisition, 276
- single frequency, frequency response analysis, 434
- single-ended probe heads, 344
- single-ended signal type, 350
- single-shot demo signal, 376
- single-shot DUT, synchronizing with, 1634
- single-shot waveform generator output, 1509
- size, 748
- size, digital channels, 388
- skew, 349, 1541
- skew reference waveform, 1527
- slew rate of an edge, 660
- slew rate power analysis, 841
- slope, 1122, 1378
- slope (direction) of waveform, 1566
- slope not valid in TV trigger mode, 1378
- slope, arming edge, Edge Then Edge trigger, 1362
- slope, Nth edge in burst, 1371
- slope, setup and hold trigger, 1405
- slope, transition search, 1223
- slope, transition trigger, 1412
- slope, trigger edge, Edge Then Edge trigger, 1366
- slot, network printer, 518
- SLOW data, SENT, 1461
- smoothing acquisition type, 1438
- smoothing math function, 488
- smoothing math function, number of points, 500
- snapshot all measurement, 603
- software version, 220
- source, 661, 914, 939, 1025
- source (voltage or current) for slew rate analysis, 842
- source channel, M1553, 1041
- source for function, 1551
- source for glitch search, 1210
- source for Nth edge burst trigger, 1372
- source for trigger, 1379
- source for TV trigger, 1419
- source function for FFT peak search, 1214
- source input for function, first, 501
- source input for function, second, 503
- source, arming edge, Edge Then Edge trigger, 1363
- source, automask, 718
- source, FLEXray, 979
- source, mask test, 743
- source, runt search, 1219
- source, runt trigger, 1402
- source, save reference waveform, 897
- source, transition trigger, 1224, 1413
- source, trigger edge, Edge Then Edge trigger, 1367
- source, waveform, 1457
- span, 487
- span of frequency on display, 474
- Spec error counter (CAN), 929
- special PID, USB search, 1300
- special PID, USB trigger, 1178
- specify measurement, 661
- speed of ARINC 429 signal, 915
- speed, USB decode, 1165
- SPI, 1122
- SPI clock timeout, 1123
- SPI decode bit order, 1121
- SPI decode word width, 1135
- SPI demo signal, 378
- SPI MISO data, 1461
- SPI SEARCh commands, 1279
- SPI serial search, data, 1281
- SPI serial search, data width, 1282
- SPI serial search, mode, 1280
- SPI trigger, 1125, 1131, 1133
- SPI trigger clock, 1126
- SPI trigger commands, 1119
- SPI trigger frame, 1127

- SPI trigger MISO data pattern, 1130
 - SPI trigger MOSI data pattern, 1132
 - SPI trigger type, 1134
 - SPI trigger, MISO data source, 1128
 - SPI trigger, MOSI data source, 1129, 1584
 - SPI triggering, 900
 - square math function, 488
 - square root math function, 488
 - square wave duty cycle, waveform generator, 1492
 - square waveform generator output, 1486
 - SRE (Service Request Enable Register), 231, 1604
 - SRQ (Service Request interrupt), 252, 256, 260
 - SSM pattern, ARINC 429 trigger, 919, 1231
 - standard deviation measured on waveform, 658
 - Standard Event Status Enable Register (ESE), 216, 1609
 - Standard Event Status Register (ESR), 218, 1608
 - standard for CAN FD decode, 934
 - standard for video, 1420
 - standard, LIN, 1026
 - start acquisition, 234, 250, 274, 276
 - start and stop edges, 611
 - start cursor, 1564
 - start frequency, FFT math function, 468
 - start measurement, 601
 - start print job, 522
 - start time, 1387, 1564
 - start time marker, 1561
 - starting bit for SENT Fast Channel Signal, 1105
 - starting byte of data, CXPI trigger, 965
 - state memory, 229
 - state of instrument, 221, 1326
 - state, run, 273
 - statistics increment, 665
 - statistics reset, 667
 - statistics results, 654
 - statistics, max count, 666
 - statistics, relative standard deviation, 668
 - statistics, type of, 663
 - status, 232, 277, 279
 - Status Byte Register (STB), 230, 232, 233, 1602
 - status data structure clear, 215
 - status registers, 66
 - status reporting, 1597
 - STB (Status Byte Register), 230, 232, 233, 1602
 - steady state output voltage expected, 836, 837, 838
 - step size for frequency span, 474
 - stop, 250, 278
 - stop acquisition, 278
 - stop cursor, 1565
 - stop displaying channel, 248
 - stop displaying math function, 248
 - stop displaying pod, 248
 - stop frequency, FFT math function, 469
 - stop on mask test failure, 734
 - stop time, 1387, 1565
 - storage, 229
 - store instrument setup, 221, 229
 - store setup, 229
 - storing calibration information, 316
 - string variables, 64
 - string variables, reading multiple query results into, 65
 - string variables, reading query results into multiple, 65
 - string, quoted ASCII, 207
 - subnet mask, 50
 - subsource, waveform source, 1461
 - subsystem commands, 3, 1655
 - subtract math function, 487, 1457
 - subtract math function as g(t) source, 1548
 - sweep mode, trigger, 1345, 1360
 - sweep speed set to fast to measure fall time, 623
 - sweep speed set to fast to measure rise time, 657
 - switch disable, 1312
 - switching level, current, 845
 - switching level, voltage, 848
 - switching loss per cycle, 694
 - switching loss power analysis, 843
 - sync break, LIN, 1027
 - sync frame count (FlexRay), 977
 - syntax elements, 206
 - syntax rules, program data, 1654
 - syntax, optional terms, 206
 - syntax, program message, 1651
 - SYSTem commands, 1305
 - system commands, 1308, 1310, 1311, 1312, 1326, 1328
 - system commands introduction, 1307
- T**
- table of current harmonics results, 783
 - tdelta, 1560
 - Tektronix probe model number, 346
 - telnet ports 5024 and 5025, 1443
 - Telnet sockets, 67
 - temporary message, 1310
 - TER (Trigger Event Register), 279, 1605
 - termination conditions, mask test, 730
 - test sigma, mask test run, 735
 - test, self, 235
 - test, USB 2.0 signal quality, 365
 - text, writing to display, 1310
 - THD (total harmonics distortion), 792
 - threshold, 389, 749, 1540, 1587
 - threshold for FFT peak search, 1215
 - threshold voltage (lower) for measurement, 1558
 - threshold voltage (upper) for measurement, 1567
 - thresholds, 611, 1561
 - thresholds used to measure period, 649
 - thresholds, how autoscale affects, 243
 - time base, 1333, 1334, 1335, 1339, 1586
 - time base commands introduction, 1332
 - time base reference, 1337
 - time base reset conditions, 227, 1316
 - time base window, 1341, 1342, 1343
 - time between points, 1560
 - time buckets, 285, 286
 - time delay, 1586
 - time delay, Edge Then Edge trigger, 1364
 - time delta, 1560
 - time difference between data points, 1465
 - time duration, 1387, 1395, 1396, 1398
 - time holdoff for trigger, 1351
 - time interval, 672, 1560
 - time interval between trigger and occurrence, 1566
 - time marker sets start time, 1561
 - time measurements with X cursors, 571
 - time per division, 1335
 - time record, 477
 - time reference, lister, 544
 - time specified, 686, 1571
 - time, calibration, 323
 - time, mask test run, 736
 - time, runt pulse search, 1220
 - time, runt trigger, 1403
 - time, start marker, 1564
 - time, stop marker, 1565
 - time, system, 1328
 - time, transition search, 1225
 - time, transition trigger, 1414
 - time/div, how autoscale affects, 243
 - time-at-max measurement, 1562
 - time-at-min measurement, 1563
 - TIMEbase commands, 1331
 - timebase vernier, 1340
 - TIMEbase:MODE, 62
 - time-ordered label list, 409
 - timeout, SPI clock, 1123
 - timing measurement, 601
 - title channels, 335
 - title, mask test, 744
 - token PID, USB search, 1301
 - token PID, USB trigger, 1179
 - tolerance for determining valid SENT sync pulses, 1109
 - tolerance, automask, 720, 721
 - top of waveform value measured, 682
 - total frame count (CAN), 930
 - total frame count (FlexRay), 978
 - Total Harmonic Distortion, FFT analysis measurement, 627
 - total harmonics distortion (THD), 792
 - total waveforms in mask test, 725
 - touchscreen on/off, 1329
 - trace memory, 277
 - track measurements, 659

- tracking, dual channel waveform generator, 1512
- transfer instrument state, 221, 1326
- transforms, math, 487
- transient response, 709
- transient response analysis, 849, 850, 853
- TRANSition SEARCh commands, 1221
- transition search qualifier, 1222
- transition search slope, 1223
- transition search time, 1225
- transition trigger qualifier, 1411
- transition trigger slope, 1412
- transition trigger source, 1224, 1413
- transition trigger time, 1414
- transmit word size, I2S, 1006
- transparent screen background, remote command logging, 1324
- transparent, display, 415
- tree traversal rules, 1655
- trend measurement, 504
- TRG (Trigger), 231, 233, 234
- TRIG OUT BNC, 317
- trigger armed event register, 262, 265
- trigger armed status, checking for, 1621
- trigger burst, UART, 1152
- trigger channel source, 1388, 1419
- TRIGger commands, 1345
- TRIGger commands, general, 1347
- trigger data, UART, 1153
- TRIGger DELay commands, 1361
- trigger duration, 1395, 1396
- TRIGger EBUrSt commands, 1368
- TRIGger EDGE commands, 1373
- trigger edge coupling, 1375
- trigger edge slope, 1378
- trigger edge slope, Edge Then Edge trigger, 1366
- trigger edge source, Edge Then Edge trigger, 1367
- trigger event bit, 279
- Trigger Event Register (TER), 1605
- TRIGger FLEXray commands, 970
- TRIGger GLITCh commands, 1380
- trigger holdoff, 1351
- trigger idle, UART, 1154
- TRIGger IIC commands, 1008
- trigger level auto set up, 1355
- trigger level constants, 337
- trigger level voltage, 1376
- trigger level, high, 1356
- trigger level, low, 1357
- TRIGger LIN commands, 1019
- TRIGger M1553 commands, 1038
- trigger occurred, 233
- TRIGger OR commands, 1389
- TRIGger PATtern commands, 1391
- trigger pattern qualifier, 1397
- trigger qualifier, UART, 1155
- trigger reset conditions, 227, 1316
- TRIGger RUNT commands, 1399
- TRIGger SHOLd commands, 1404
- trigger SPI clock slope, 1122
- TRIGger SPI commands, 1119
- trigger status bit, 279
- trigger sweep mode, 1345
- TRIGger TV commands, 1410, 1415
- trigger type, ARINC 429, 921, 1228
- trigger type, SPI, 1134
- trigger type, UART, 1156
- TRIGger UART commands, 1136
- TRIGger USB commands, 1158
- TRIGger ZONE commands, 1425
- trigger, ARINC 429 source, 914
- trigger, CAN, 940
- trigger, CAN FD sample point, 933
- trigger, CAN pattern data length, 946
- trigger, CAN pattern ID mode, 949
- trigger, CAN sample point, 935
- trigger, CAN signal baudrate, 936
- trigger, CAN signal definition, 937
- trigger, CAN source, 939
- trigger, duration greater than, 1395
- trigger, duration less than, 1396
- trigger, duration range, 1398
- trigger, edge coupling, 1375
- trigger, edge level, 1376
- trigger, edge reject, 1377
- trigger, edge slope, 1378
- trigger, edge source, 1379
- trigger, FLEXray, 980
- trigger, FLEXray error, 981
- trigger, FLEXray event, 984
- trigger, force a, 1349
- trigger, glitch greater than, 1382
- trigger, glitch less than, 1383
- trigger, glitch level, 1384
- trigger, glitch polarity, 1385
- trigger, glitch qualifier, 1386
- trigger, glitch range, 1387
- trigger, glitch source, 1388
- trigger, high frequency reject filter, 1350
- trigger, holdoff, 1351
- trigger, I2S, 999
- trigger, I2S alignment, 992
- trigger, I2S audio channel, 1001
- trigger, I2S clock slope, 994
- trigger, I2S CLOCKsource, 996
- trigger, I2S DATA source, 997
- trigger, I2S pattern data, 1002
- trigger, I2S pattern format, 1004
- trigger, I2S range, 1005
- trigger, I2S receiver width, 995
- trigger, I2S transmit word size, 1006
- trigger, I2S word select (WS) low, 1007
- trigger, I2S word select (WS) source, 998
- trigger, IIC clock source, 1011
- trigger, IIC data source, 1012
- trigger, IIC pattern address, 1013
- trigger, IIC pattern data, 1014
- trigger, IIC pattern data 2, 1015
- trigger, IIC qualifier, 1016
- trigger, IIC signal baudrate, 1024
- trigger, IIC type, 1017
- trigger, LIN, 1028
- trigger, LIN pattern data, 1031
- trigger, LIN pattern data length, 1033
- trigger, LIN pattern format, 1034
- trigger, LIN sample point, 1023
- trigger, LIN signal definition, 1583
- trigger, LIN source, 1025
- trigger, mode, 1358
- trigger, noise reject filter, 1359
- trigger, Nth edge burst source, 1372
- trigger, Nth edge in burst slope, 1371
- trigger, Nth edge of burst count, 1369
- trigger, Nth edge of Edge Then Edge trigger, 1365
- trigger, SPI clock slope, 1122
- trigger, SPI clock source, 1126
- trigger, SPI clock timeout, 1123
- trigger, SPI frame source, 1127
- trigger, SPI framing, 1125
- trigger, SPI pattern MISO width, 1131
- trigger, SPI pattern MOSI width, 1133
- trigger, sweep, 1360
- trigger, threshold, 1587
- trigger, TV line, 1416
- trigger, TV mode, 1417, 1588
- trigger, TV polarity, 1418
- trigger, TV source, 1419
- trigger, TV standard, 1420
- trigger, UART base, 1151
- trigger, UART baudrate, 1140
- trigger, UART bit order, 1141
- trigger, UART parity, 1147
- trigger, UART polarity, 1148
- trigger, UART Rx source, 1149
- trigger, UART Tx source, 1150
- trigger, UART width, 1157
- trigger, USB, 1166
- truncation rules, 1652
- TST (Self Test), 235
- tstart, 1564
- tstop, 1565
- TTL threshold voltage for digital channels, 389, 1540
- TTL trigger threshold voltage, 1587
- turn function on or off, 1552
- turn off channel, 248
- turn off channel labels, 408
- turn off digital pod, 248
- turn off math function, 248
- turn off time, 700, 803, 804
- turn on channel labels, 408
- turn on channel number display, 1545
- turn on time, 701, 803, 804
- turn on/turn off analysis thresholds, 804
- turn on/turn off time analysis, 800, 801, 802, 803
- turning channel display on and off, 332
- turning off/on function calculation, 463
- turning vectors on or off, 1544
- TV mode, 1417, 1588
- TV trigger commands, 1410, 1415
- TV trigger line number setting, 1416
- TV trigger mode, 1419

TV trigger polarity, 1418
 TV trigger standard setting, 1420
 TV triggering, 1346
 tvmode, 1588
 Tx data, UART, 1461
 Tx frame count (UART), 1145
 Tx source, 1150
 type of power being converted, efficiency measurement, 779
 type, USB 2.0 signal quality test, 367

U

UART base, 1151
 UART baud rate, 1140
 UART bit order, 1141
 UART frame counters, reset, 1143
 UART parity, 1147
 UART polarity, 1148
 UART Rx source, 1149
 UART SEARCh commands, 1283
 UART serial search, data, 1284
 UART serial search, data qualifier, 1286
 UART serial search, mode, 1285
 UART trigger burst, 1152
 UART trigger commands, 1136
 UART trigger data, 1153
 UART trigger idle, 1154
 UART trigger qualifier, 1155
 UART trigger type, 1156
 UART Tx data, 1461
 UART Tx source, 1150
 UART width, 1157
 UART/RS232 demo signal, 377
 UART/RS-232 triggering, 900
 units (vertical) for FFT, 476
 units per division, 343, 354, 355, 427, 1339
 units per division (vertical) for function, 354, 494
 units, automask, 719
 units, X cursor, 571, 572
 units, Y cursor, 578, 579
 unsigned data, 1439
 unsigned mode, 1463
 upper threshold, 649
 upper threshold voltage for measurement, 1567
 uppercase characters in commands, 1651
 URQ (User Request) status bit, 216, 218
 USB (Device) interface, 49
 USB 2.0 signal quality test results, save, 872
 USB 2.0 triggering, 900
 USB demo signal, 379
 USB PD demo signal, 379
 USB PD trigger mode, 1185
 USB PD trigger, control message type, 1188
 USB PD trigger, data message type, 1190, 1191

USB PD trigger, header content trigger qualifier, 1194
 USB PD trigger, header type, 1186
 USB PD trigger, user-defined header value, 1193
 USB PD triggering, 900
 USB PD waveform source, 1184
 USB SEARCh commands, 1287
 USB storage device, recalling files from, 856
 USB storage device, saving files to, 870
 USB trigger commands, 1158
 USB triggering, 1346
 USBPd serial bus commands, 1183
 user defined channel labels, 335
 user defined threshold, 1540
 user event conditions occurred, 233
 User Files directory, 863, 883
 user name, network domain, 519
 User's Guide, 6
 user-defined baud rate, ARINC 429, 908
 user-defined Real Power in Class D harmonics analysis, 788
 user-defined threshold voltage for digital channels, 389
 user-defined trigger threshold, 1587
 USR (User Event bit), 231, 233
 utilization, CAN bus, 931

V

valid command strings, 1651
 valid pattern time, 1395, 1396
 value, 672
 value measured at base of waveform, 619, 676
 value measured at specified time, 686, 1571
 value measured at top of waveform, 682
 value ranges, 207
 values required to fill time buckets, 286
 VBA, 58, 1660
 Vce(sat) power measurement, 710
 Vce(sat) value for conduction calculation, 847
 vectors turned on or off, 1544
 vectors, display, 416
 vectors, turning on or off, 393
 vernier, channel, 356
 vernier, horizontal, 1340
 vertical adjustment, fine (vernier), 356
 vertical amplitude measurement, 617, 674
 vertical axis defined by RANGE, 492
 vertical axis range for channels, 353
 vertical histogram, 528
 vertical offset for channels, 336
 vertical peak-to-peak measured on waveform, 620, 679
 vertical scale, 354, 494
 vertical scaling, 1450
 vertical threshold, 1540
 vertical units for FFT, 476
 vertical value at center screen, 486, 493
 vertical value maximum measured on waveform, 677
 vertical value measurements to calculate overshoot, 646
 vertical value minimum measured on waveform, 678
 video line to trigger on, 1416
 video standard selection, 1420
 view, 280, 1464, 1545
 view turns function on or off, 1552
 VISA COM example in C#, 1669
 VISA COM example in Python, 1686
 VISA COM example in Visual Basic, 1660
 VISA COM example in Visual Basic .NET, 1678
 VISA example in C, 1694
 VISA example in C#, 1713
 VISA example in Python, 1734
 VISA example in Visual Basic, 1703
 VISA example in Visual Basic .NET, 1724
 VISA examples, 1660, 1694
 VISA.NET example in C#, 1741
 VISA.NET example in Python, 1753
 VISA.NET example in Visual Basic .NET, 1747
 VISA.NET examples, 1741
 Visual Basic .NET, VISA COM example, 1678
 Visual Basic .NET, VISA example, 1724
 Visual Basic .NET, VISA.NET example, 1747
 Visual Basic 6.0, 59
 Visual Basic for Applications, 58, 1660
 Visual Basic, SICL library example, 1769
 Visual Basic, VISA COM example, 1660
 Visual Basic, VISA example, 1703
 visualizations, math, 489
 voltage (waveform generator), frequency response analysis, 445
 voltage crossing reported or not found, 1566
 voltage difference between data points, 1468
 voltage difference measured, 1568
 voltage in, frequency response analysis, 440, 441
 voltage level for active trigger, 1376
 voltage marker used to measure waveform, 1569, 1570
 voltage offset value for channels, 336
 voltage probe, 343, 355, 427
 voltage profile, frequency response analysis, 447
 voltage ranges for channels, 353
 voltage ranges for external trigger, 426
 voltage source, 840
 voltage threshold, 611
 voltage, maximum expected input, 833, 834, 835

W

WAI (Wait To Continue), 236
 wait, 236
 wait for operation complete, 222
 Wait Trig bit, 262, 265
 warranty, 2
 waveform base value measured, 619, 676
 WAVEform command, 57
 WAVEform commands, 1433
 waveform data, 1436
 waveform data format, 893
 waveform data length, 894
 waveform data length, maximum, 895
 waveform data, save, 892
 waveform generator, 1475
 waveform generator amplitude, 445, 1517
 waveform generator function, 1486
 waveform generator high-level voltage, 1518
 waveform generator load impedance, Control Loop Response (Bode) power analysis, 774
 waveform generator load impedance, Power Supply Rejection Ratio (PSRR) power analysis, 817
 waveform generator low-level voltage, 1519
 waveform generator offset, 1520
 waveform generator output control, 1505
 waveform generator output load impedance, 444, 1506
 waveform generator output mode, 1507
 waveform generator output polarity, 1508
 waveform generator period, 1510
 waveform generator pulse width, 1490
 waveform generator ramp symmetry, 1491
 waveform generator reset defaults, 1511
 waveform generator square wave duty cycle, 1492
 waveform introduction, 1436
 waveform maximum vertical value measured, 677
 waveform minimum vertical value measured, 678
 waveform must cross voltage level to be an occurrence, 1566
 WAVEform parameters, 62
 waveform peak-to-peak vertical value measured, 620, 679
 waveform period, 649
 waveform persistence, 393
 waveform RMS value measured, 621, 681
 waveform save option for segments, 896
 waveform source, 1457
 waveform source subsource, 1461
 waveform standard deviation value measured, 658
 waveform vertical amplitude, 617, 674
 waveform voltage measured at marker, 1569, 1570
 waveform, byte order, 1441
 waveform, count, 1442
 waveform, data, 1443
 waveform, format, 1445
 waveform, points, 1446, 1448
 waveform, preamble, 1450
 waveform, type, 1462
 waveform, unsigned, 1463
 waveform, view, 1464
 waveform, X increment, 1465
 waveform, X origin, 1466
 waveform, X reference, 1467
 waveform, Y increment, 1468
 waveform, Y origin, 1469
 waveform, Y reference, 1470
 WAVEform:FORMat, 62
 waveforms, mask test run, 737
 waven output amplitude(s), Control Loop Response (Bode) power analysis, 775
 waven output amplitude(s), Power Supply Rejection Ratio (PSRR) power analysis, 818
 Web control, 67
 website, examples on, 1659
 WGEN commands, 1471
 WGEN trigger source, 1379
 what's new, 41
 width, 1157, 1387
 window, 1341, 1342, 1343
 window time, 1335
 window time base mode, 1333
 window, measurement, 683
 windows, 477
 windows as filters to Fast Fourier Transforms, 477
 windows for Fast Fourier Transform functions, 477
 WMEMory commands, 1521
 word counter (ARINC 429), 911
 word counter (ARINC 429), reset, 910
 word format, 1445
 word format for data transfer, 1439
 word format, ARINC 429, 912
 word select (WS) low, I2S trigger, 1007
 word select (WS) source, I2S, 998
 word width, SPI decode, 1135
 write mode, remote command logging, 1319, 1325
 write text to display, 1310
 WriteIEEEBlock method, 59, 65
 WriteList method, 59
 WriteNumber method, 59
 WriteString method, 59

X

X axis markers, 561
 X cursor units, 571, 572
 X delta, 562, 570
 X delta, mask scaling, 740
 X1 and X2 cursor value difference, 562, 570

X1 cursor, 561, 565, 566
 X1, mask scaling, 739
 X2 cursor, 561, 568, 569
 X-axis functions, 1332
 X-increment, 1465
 X-of-max measurement, 684
 X-of-min measurement, 685
 X-origin, 1466
 X-reference, 1467
 X-Y mode, 1332, 1333

Y

Y axis markers, 561
 Y cursor units, 578, 579
 Y offset, reference waveform, 1528
 Y range, reference waveform, 1529
 Y scale, reference waveform, 1530
 Y1 and Y2 cursor value difference, 577
 Y1 cursor, 561, 566, 574, 577
 Y1, mask scaling, 741
 Y2 cursor, 561, 569, 576, 577
 Y2, mask scaling, 742
 Y-axis value, 1469
 Y-increment, 1468
 Y-origin, 1469, 1470
 Y-reference, 1470

Z

zero values in waveform data, 1443
 zone 1 or zone 2 mode, 1428
 zone 1 or zone 2 placement, 1429
 zone 1 or zone 2 state, 1431
 zone 1 or zone 2 validity, 1430
 zone qualified trigger source, 1426
 zone qualified trigger state, 1427
 ZONE trigger commands, 1425
 Zoom In/Out setting for N2820A channel, 351
 zoomed time base, 1333
 zoomed time base measurement window, 683
 zoomed time base mode, how autoscale affects, 243
 zoomed window horizontal scale, 1343